

[Skip to main content](#)

Feedzai Pulse

Feedzai Pulse is a cutting-edge enterprise data science platform that integrates big data analytics and machine learning techniques with a native runtime production environment to fight financial crime in real time. Feedzai combines in a single enterprise platform all the critical tools needed to manage data science projects across their entire life cycle. Pulse is designed with an open and flexible data architecture. This makes it a powerful platform to solve any number of different business use cases across financial institutions and merchants.

[Get started](#)

[Skip to main content](#)

Search the documentation

-



- About Pulse

On this page

About Pulse

Was this page helpful? 

The Feedzai Platform processes events to detect and remediate financial crime. In the life cycle of the events, Pulse performs two main roles:

- At its core, Pulse comprises a **Real-Time Risk Engine** that processes and scores events for the different use cases, makes an automated decision about them and triggers alerts if necessary.
- Pulse also functions as a **Data Science Studio** for the Feedzai Platform, providing a user interface for data scientists to create rules and models that will be used by the Risk Engine to detect financial crime and suspicious activities.

Pulse in the Feedzai Platform context

Pulse in the Feedzai Platform context

Main functionalities

The following sections describe Pulse's main components and their functionalities.

Source Data Modeling

With **Source Data Modeling** capabilities, Pulse supports a schema-agnostic architecture, making it easy to customize the schemas of master data streams for each business use case. Data scientists and engineers can use the built-in data analysis tools to validate data schemas, clean data and analyze datasets quickly.

Pulse automatically analyzes imported datasets, creating field mappings and histograms that identify patterns and provide insights into the data. These insights can be used to identify parameters which can be used to enhance the projects. Data scientists and engineers can create stratified samples and natively design new features that are semantically equivalent both at model training and at runtime.

Segment-of-One Profile Engine

One of the most powerful concepts within the platform, **Segment-of-One Profiles** allow Pulse to create hundreds of granular profiles, for different attributes in an event, rather than just estimating behavioral correlations.

This means that events can be scored using each Segment-of-One attribute within the data stream, making the results become significantly more precise.

For example, instead of evaluating a single debit transaction against a cohort of other similar debit cards (e.g. Visa Debit cards from ABC Bank), Segment-of-One will allow Pulse to, in just a few minutes, build a profile for every single debit card. Finally, Pulse evaluates the incoming event against these profiles, which can number in hundreds or even thousands.

Data Science Studio

Pulse's **Data Science Studio** is the ultimate unified toolset for any data science team. It simplifies many aspects of traditional data science work, effectively making it accessible to other user profiles including fraud and business analysts.

Data scientists now have a powerful, yet easy to use, design studio to build data science projects and train [machine learning models](#). Teams can collaborate together on projects, share insights and improve outcomes, focusing on the real data science and not on the tooling and infrastructure.

Data transformations can be developed to create new [data sources](#) and [metrics](#) that can be used in models and runtime workflows.

Plans allow data science teams to quickly and easily transform data, combine data sources, and natively deploy these transformations to the runtime environment.

Machine Learning Models

Pulse is designed to support all stages and iterations of the data science loop. The main end goals of this data science work are to efficiently iterate and train **machine learning models**, visually evaluating and easily comparing them.

As the data scientist uncover patterns, understand and transform data, Pulse lets them go even further by easily enriching events with simple features, metrics and any other advanced derived features.

Models are continuously retrained with new data as events are processed and alerts are decisioned. This native retraining of models allow financial institutions and merchants to evolve models and adapt to changing consumer behavior rapidly.

Rule Engine

A simplified **Rule Engine** and its respective management UI allow data science teams to develop any detection and decision rules needed to optimize the performance of their projects. This adds an extra layer of depth to machine learning models.

Similarly to models in the data science loop, Pulse extends the same iterative approach to the development and evaluation of rules.

The performance of rules can be evaluated in real time, making it possible to iterate over them and achieve optimal outcomes. The engine is able to execute any rule in real time, regardless of its complexity, including access to Segment-of-One Profiles, or sequence in the execution workflow.

Workflow Orchestrator

Pulse **Workflows** define the step-by-step process that the Real-Time Risk Engine follows when processing events. Workflows deploy the data science work seamlessly by combining data, plans, models and rules to drive real-time outcomes.

In Pulse, deploying a model into production takes hours instead of months. Users are able to leverage Segment-of-One Profiles in models and rules, and execute all data science work on a single platform without the need for creating new custom code. Pulse's integrated runtime environment ensures that the result of the data science loop is ready to go into production and yield the same decisions as observed at design time.

Pulse's execution engine consists of a series of high performance enterprise services that operate symbiotically in real time to process and score events with sub-millisecond latency.

Feedzai IQ

In addition to the advanced data science capabilities, Pulse also offers the option to leverage **Feedzai IQ**, a suite of ready-to-adopt risk assessment solutions providing AI-powered insights based on aggregated intelligence from across the Feedzai global network.

Feedzai IQ can be used out of the box without requiring any [data preparation](#), [model training](#), or [historical data collection](#), thus enabling rapid deployment and immediate value realization without extensive data science work, and saving potentially months of lead time.

Specifically, Feedzai IQ provides **TrustScore**, a federated risk score that encompasses the insights and patterns from Feedzai's global footprint to enable real-time scoring of fraud and financial crime risk.

TrustScore

Feedzai **TrustScore** is a federated machine learning model that provides a real-time risk score for every event processed by Pulse, with high accuracy and lower false positive rates than traditional rules, based on global risk patterns aggregated through the Feedzai community.

TrustScore models are continuously updated and fine-tuned, reacting quickly to changes in the fraud landscape.

Federated learning

The training of the TrustScore models is done by Feedzai within its machine learning infrastructure, gathering insights from the diverse range of financial risk patterns across its entire client base, by means of federated learning techniques. These techniques allow financial intelligence sharing while keeping the data segregated and confidential within each customer's environment.

The federated learning approach works by using [aggregations](#) from across the Feedzai network to train models which are kept isolated from each other. These models are later deployed to process real-time data streams in production environments. When a new transaction or event occurs in a client's production environment, it is simultaneously analyzed by all these specialized expert models. The resulting federated scores are aggregated via quality-aware weighting algorithms, producing a single, accurate and privacy-preserving TrustScore for each event.

Risk Strategy integration

TrustScore is available for deployment as a ready-to-use risk score, either standalone or as part of a broader risk strategy. It can be seamlessly plugged into [rules](#) or [combined](#) with other signals for a more comprehensive risk assessment.

It can also be used as a starting point for developing custom [models](#), or as an additional input to influence an existing risk scoring model.

Administration📖

Pulse includes an **Administration** module for system administrators to easily configure and maintain the platform. System administrators are able to set up environments quickly, configure clusters, and deploy Pulse in high availability configurations.

Pulse's security framework supports granular security configuration and access control. Access management and permissions can be configured within Feedzai.

Getting started with Feedzai Pulse📖

Feedzai Pulse is a state-of-the-art and very complex platform. It is designed to fully support tasks for different user profiles while abstracting away most of this complexity and making sure that users do not become overwhelmed with the amount of possibilities.

Now that you have a glimpse of what Feedzai Pulse is, you can read more about the [core concepts](#) to further understand how Pulse works and what you can accomplish with this platform.

-



- About the Security Guide

About the Security Guide

Was this page helpful? 

This guide helps system architects and administrators understand the following security concepts in Pulse, and get an overview of the related tasks:

- [Security Entities](#)
- [Component Encryption](#)
- [Security Configurations](#):
 - [SSL Certificate](#)
 - [Kerberos](#)
- Single Sign-On options:
 - [Configuring AD FS](#)

- [Configuring SSO using Generic IdP](#)
- [Configuring Keycloak](#)

-



- About the User Guide

On this page

Was this page helpful? 

About the User Guide

The **User Guide** leads you on the recommended tasks you must perform in Pulse throughout the complete lifecycle of your project. It provides instructions on what you should do to perform each task and how you should keep moving forward, often complemented with examples and always providing more context and [conceptual details](#).

Pulse focuses on a democratization of Data Science allowing data scientists and fraud experts to use the platform without losing any of the performance that Feedzai always delivers. All of the tasks in the user journey are initiated, heavily influenced, or built around Data Science and Machine Learning. This guide focuses on providing you a clear path on how you should progress from beginning to a complete project in Pulse. The **User Guide** includes the following sections.

How Pulse projects workâ

Provides an overview of how Pulse is able to seamlessly integrate a Data Science stage with a Production stage, with a unified modeling and runtime environment.

The main goal is to provide you with important context so that you fully understand the big picture, and how these two main stages are connected.

Learn [how Pulse projects work](#).

Creating a data science projectâ

Helps you map typical data science stages to Pulse, so that your learning curve is minimized, and you quickly get up to speed with Pulse.

The main goal is to allow you to become more productive by allowing you to focus on real data science instead of engineering.

Learn how to [create a data science project](#).

Deploying a project into Productionâ

Teaches you to easily configure a runtime workflow to process real-time events, based on all the work accomplished during the Data Science stage.

The main goal is to make it extremely simple to transport all Data Science work to Production, minimizing the effort spent on engineering issues.

See [how to deploy a project into Production](#).

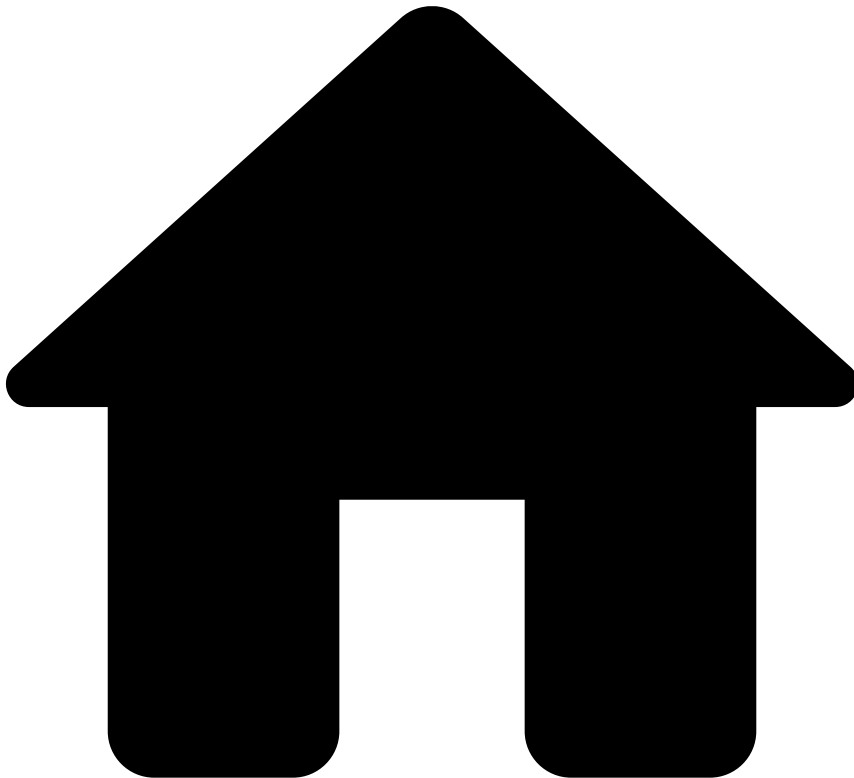
Working with applicationsâ

Explains how you can interact with your applications through Pulse by performing actions such as importing and exporting them, publishing changes or versioning their resources.

The main goal is give you an overview on Pulse applications lifecycle as well as how you can manage them throughout these stages.

Read about [working with applications](#).

-



- Securing an Application
- Security Entities

On this page

Was this page helpful? 

Security Entities

This section gives you a high-level overview of the main security components or entities that exist in Pulse.

Security in Pulse has four core entities, each with its purpose:

- Permissions
- Roles
- Groups
- Users

Together they provide a flexible and comprehensive security model.

Permissions

In Pulse, permissions are used for controlling access to resources. For example, the permission `application:apps:create` allows the creation of a new application.

Pulse already comes with a list of predefined internal permissions. They can be used when creating roles.

A full list with all the permissions in Pulse can be found on the [List of Permissions](#) page.

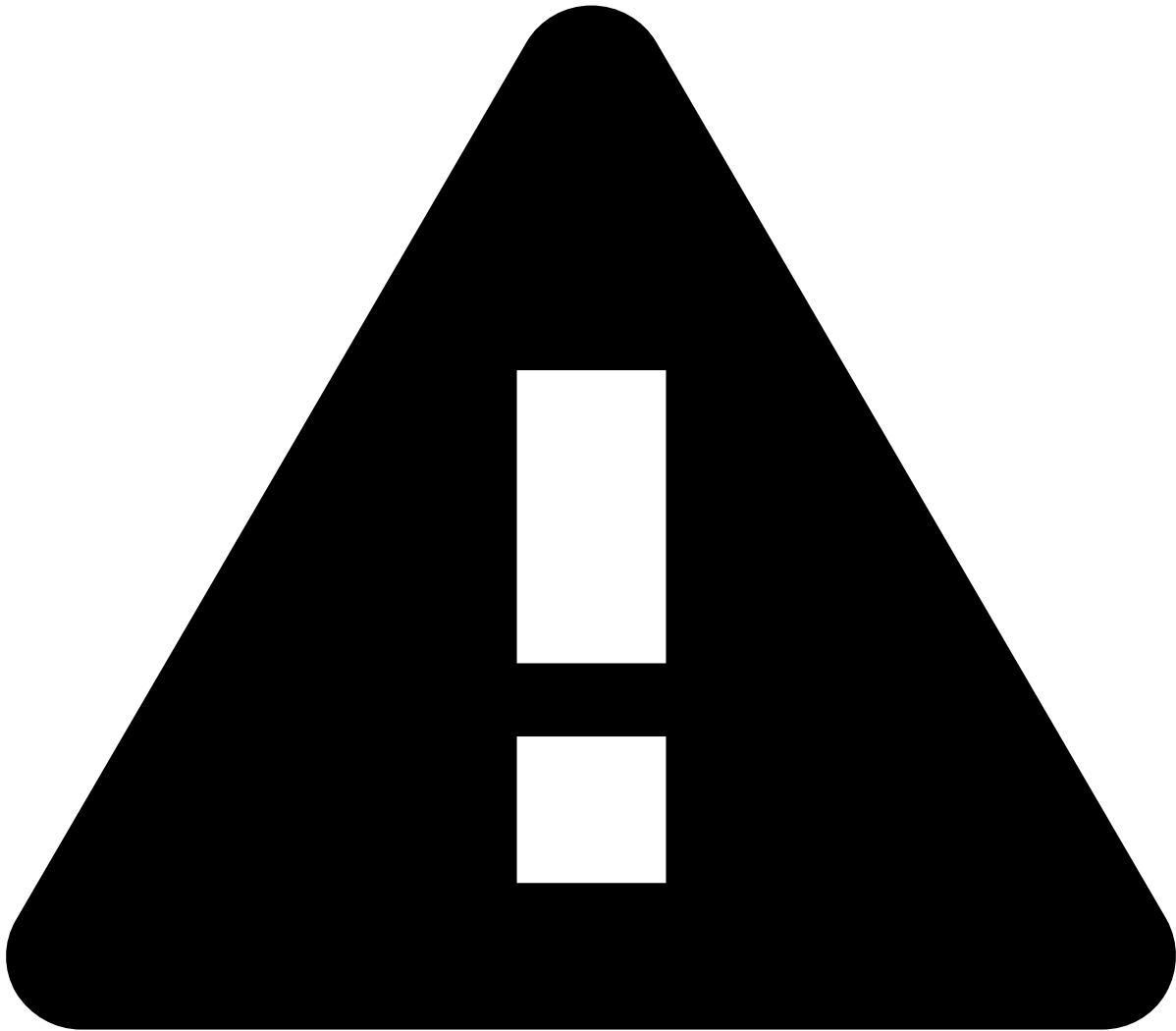
Permission contexts

Permissions have the following structure: `<context>:[<resource>]:[<right>]`.

For example, to allow a user to create a sandbox evaluations within the application context, the user must have the `application:sandbox_evaluations:create` permission.

The permission contexts are the following:

- **admin** - All things related to administration (e.g. configuration, deployment, etc). This context is intended only for system administrators.
- **application** - This is the context of all resources (e.g. data science models, rules, schemas, etc).
- **pulseviews** - This is the context for UI-related actions (e.g. login, tab management, etc).
- **security** - Security-related actions belong to this context (e.g. creating users, groups, etc).



Warning

To log in, users must have the `pulseviews:login` permission.

Roles🔗🔗

A role is a set of permissions. Each user or group can have many roles. If a user belongs to a group, it has all the roles of that group.

Roles are typically created to ease the management of permissions. For instance, the update permissions of different resources can belong to a single role called *updater-role*. From now on, we can use the *updater-role* to refer to all update permissions, instead of using individual permissions.

Default roles🔗🔗

Pulse already has some roles created by default. These roles can be selected when creating a new user from the Administration view:

- **admin** - The administration role. It has permissions to do everything.
- **pulseviews** - This role can view, update and delete all resources, but it only has view rights over security.
- **pulseviews-reader** - This role can only read resources, nothing more.
- **developer** - The user with this role has permissions to do everything except system administration.

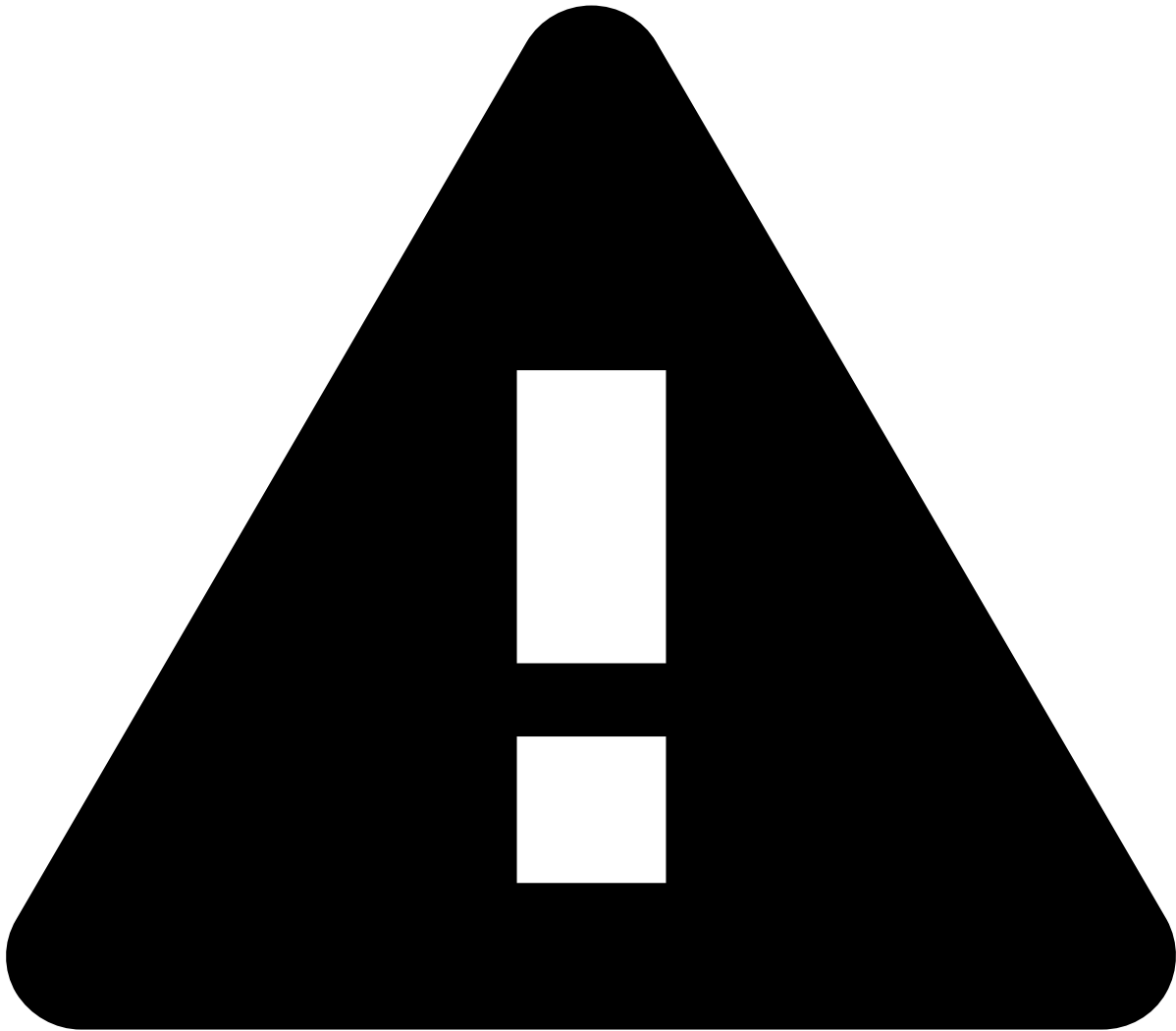
The following table shows the permissions for each default role.

Permission name	Permission	pulseviews	pulseviews-reader	developer	admin
Group hierarchy read	security:ownership:read	(tick)	(tick)		
Role read	security:roles:read	(tick)	(tick)		
Permissions read	security:permissions:read	(tick)	(tick)		
Tenant read	security:tenants:read	(tick)	(tick)		
Tenant domain read	security:tenantdomains:read	(tick)	(tick)		
Pulseviews all	pulseviews:*	(tick)			
Pulseviews login	pulseviews:login		(tick)		
Application all CRUD	application:*	(tick)			
Application all read	application:*:read		(tick)		
Management tabs all	pulseviews:management:tabs:*		(tick)		
History read (single resource)	application:history:resource:read		(tick)		
History read (application-wide)	application:history:global:read		(tick)		
Everything	*			(tick)	(tick)

Groups

A group is a security component that can have multiple roles. Not only that, they also provide a hierarchical way to control access to resources.

The hierarchical structure of groups is what makes them a powerful authorization abstraction, because **a group can do, at least, everything that its descendants do.**



Roles are not propagated

If User A belongs to a child group that can update a given hierarchy, User B, belonging to a parent group, will also be able to do it. The only exception to this rule is if User B does not have the static permission to update hierarchies. Groups entitle static permissions via roles but these are not propagated in any way through the group hierarchy.

Theoretical use case

An easy way to understand how groups behave is to map them to an hypothetical company organizational positions. Each group is a position occupied by some employees. An employee in a management position can see the work items of its subordinates, but the reverse is not true.

Consider the following positions of an hypothetical company.

And consider some of that company employees connected to their respective positions. The connections between users and groups can be read as: Lisa is a Managing Director.

Now imagining Lisa has some reports she wants to share with specific positions, lets see how she can accomplish that using groups.

- **Human Resources Report** - Peter, Kate and any future HR worker should be able to see it, so this report should be visible to the group '*HR Worker*'. Notice that both Carl and Lisa will be able to see this report. Also notice that besides being respectively '*HR Salaries Worker*' and '*Hiring Worker*', both Peter and Kate must also be '*HR Worker*'.

- **Salaries Report** - This report should be visible for all workers responsible by salaries, so it should be visible to the groups '*Finance Salaries Worker*' and '*HR Salaries Worker*'. This report will be visible to Arthur, Mary, Peter and their superiors, John, Carl and Lisa.
- **Arthur Performance Report** - This report should only be visible to Arthur and his bosses. This is not a good use-case for groups because in order to solve it, a new child group of '*Finance Manager*' called '*Arthur (Finance)*' has to be created and linked to Arthur. If the report is then made visible to this group, both Arthur and his superiors will be able to see it.

Example use case

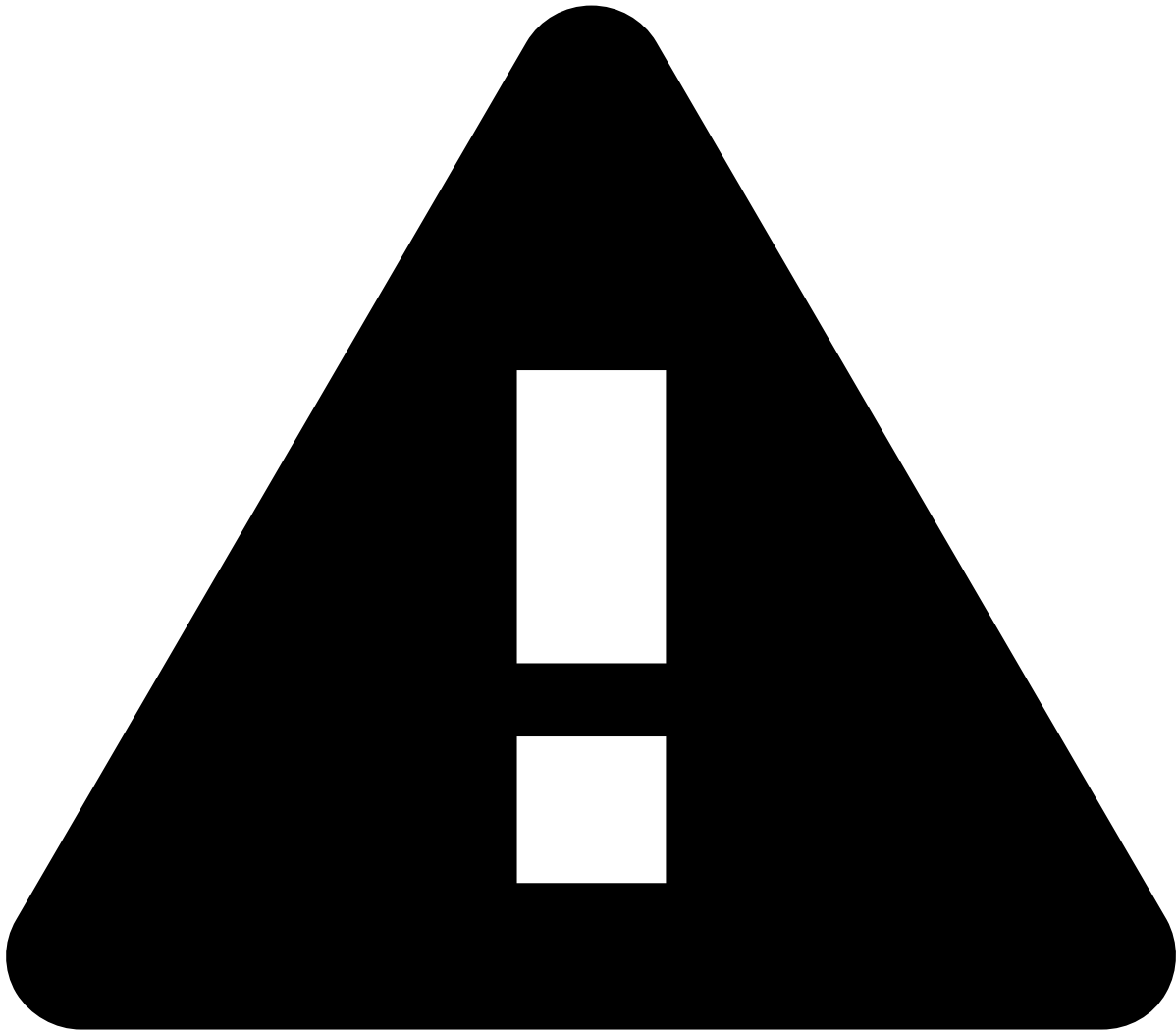
Consider the following scenario, a card brand company provides different data to issuers and acquirers, in their turn, each acquirer will only have access to a subset of that data, and so on. A possible groups hierarchy to map this scenario could be as follows.

Considering the structure above, deploying Pulse in this company to **provide services to other companies** is just a matter of assigning users to each position and sharing them with other companies. Each application resource must then be given the correct visibility so that only users of those particular groups can see it.

Users

A user is usually associated with "someone" using Pulse. It is used to prove the entity is who it says it is, through **authentication**, and to make sure it has the rights to do what it is trying to do, through **authorization**.

Users can have both **roles** and **groups**. Roles are usually associated with a user to define its **static permissions**. That is, permissions that **always** apply to a user, and that cannot be controlled for each application or resource instance. On the other hand, groups can be used to specify **per-instance permissions**, and are used to control the visibility a user will have over each resource instance, see [Securing Metadata](#).



warning

A user can be authenticated through external authentication. In this case, the password field is not required.

Managing users and credentials

Pulse has several features designed to help manage users and their credentials. These features need to be explicitly configured if you want to use them.

There are several features designed to prevent access by unauthorized users. These features block a user thus preventing him from signing in and use his account. The user is informed that his account is blocked when he tries to log in. Users can be blocked by:

- Explicitly blocking them in the user page in the administration section of the user interface.
- Users that do not login for a given period of time can be blocked automatically.
- Block the user after multiple failed login attempts.

After being blocked, users need to be enabled by an administrator in the web user interface administration section.

There are also several features designed to enforce that users have secure credentials:

- Password strength verification. Passwords are checked to contain three or more of the following types of characters:
 - 1 upper case letter

- 1 lower case letter
- a number (0-9)
- a special character (!@#\$\$%^&*)
- Password uniqueness verification. It is not possible for a user to re-use one of their previous passwords.
- Administrators can mark the password of a user as expired. This will prompt the user to insert a new password before he can log in. Alternatively, the user password can also be changed in the administration user page.
- Password age verification. If the user did not change the password in a given period of time, the password is automatically marked as expired.

Other features include:

- Invalidate user session after a period of inactivity.
- Log authentication related events to the audit log:
 - login
 - logout
 - session expiration
 - account disable
 - password expire
 - password change
 - user creation

Default users

Users in Pulse can be disabled due to a variety of reasons, including too many failed login attempts, password expiration or inactivity. However, Pulse relies on making internal calls for communicating between the Pulse Servers and its App Engines to function properly, and those calls require authentication.

In order to handle internal and external authentication and ensure PCI-DSS compliance, Pulse creates the following two users on startup:

- **pu1se** : User with full permissions for external authentication, e.g. in Pulse APIs.
- **system** : User with full permissions that cannot be used for external authentication, i.e. it is impossible to log in to Pulse. This user is exclusive to internal Pulse usage, so you should not try to use it.

•



- Project Organization
- App Collaboration Loop

On this page

App Collaboration Loop

Was this page helpful? 

To build a Fraud Prevention solution, [Rules](#) and Data Science resources such as [Plans](#) and [Models](#) have to be developed, tested, and then deployed into Production to score transactions in real-time. These stages are often performed in different environments to ensure that the work in Development does not affect the work in Production. The stages are also performed by different teams at Feedzai and from Customers, including Data Scientists, Engineering Teams, and Fraud Analysts. The concept of collaborating on a Fraud Prevention solution using Pulse Applications (**Pulse apps**) is known as [App Collaboration](#).

Similarly to the [Data Science Loop](#), **App Collaboration** is not a one-time process. It requires a constant integration between the work developed in offline environments with the work in Production. This concept is described in this guide as **App Collaboration Loop**.

This guide covers the following two key ideas:

- How to structure an application in different environments.
- Workflow between environments.

Introduction to App Collaboration Loop

App Collaboration Loop sets a working methodology so that different teams can collaborate together by developing various resources simultaneously and then bring the latest improvements faster and reliably to Production. **App Collaboration Loop** is recommended whenever you want to update Pulse resources (e.g. **Models**, **Plans**, [Rules](#), [Lists](#)).

This working methodology is performed by different teams and requires different environments as follows:

- Offline environments:
 - **Data Science environment (DS)**: Used by Data Science teams to develop tasks included in the **Data Science Loop** (e.g. improve **Models**).
 - **Development environment (DEV)**: Used by Engineering teams to test more complex **Rules**, add schema fields, as well as to integrate Pulse resources developed in DS and in Production with the resources evolved in DEV.
- Runtime environment:
 - **Production environment (PROD)**: Used by Analysts to process transactions in real-time, and to write [Self-service Rules](#). PROD is set up and then continuously updated by the Engineering team (e.g. update **Pulse app** or software versions).

Block ownership

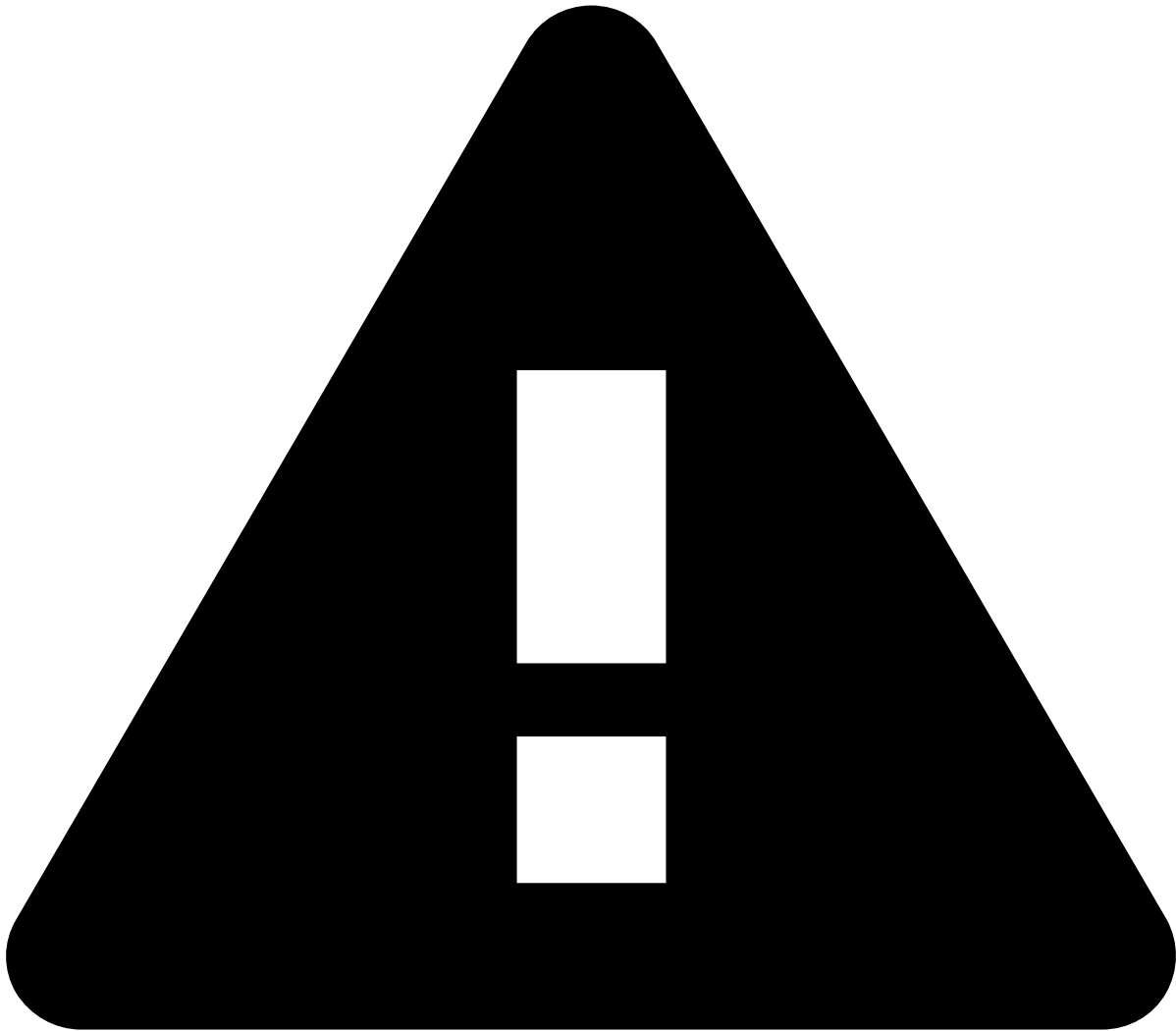
If you are getting started with the App Collaboration approach, this section will help you understand how to set up your environment using **Block ownership** to develop Pulse resources.



App Collaboration approach

If you are already familiarized the **Block ownership** concept, you can skip this section and visit the section [Workflow between environments](#) to learn how to evolve a **Pulse app** from DEV/DS to PROD.

To evolve an application in a collaborative way, App Collaboration uses the concept of Block ownership. With this concept, each Pulse resource is evolved as a block by a single team within a single environment. As an example, a machine learning Model is evolved only by Data Scientists within the DS environment. The image below illustrates a typical example of three environments (DEV, DS and PROD) using Block ownership:



caution

The block ownership illustrated above assumes the following:

1. Between each deployment, each block evolves in a single environment (i.e. in DEV, DS, or PROD).
2. For each environment, all the resources are evolved using a single Pulse cluster.

Some resources are often developed by various teams (i.e. owners), for example, [Rules Projects](#). Using the concept of resource ownership, each Rules Project should be owned by one team and should be only modified (e.g. adding/removing/editing Rules) by the team that owns each Rules Project. One possible solution is to divide the Rules Projects as follows:

- **Eng. Rules:** Represents Rules Projects with higher complexity, which are only evolved by Engineering teams in the DEV.
- **DS Rules:** Represents Rules Projects that are closely related to Data Science, which are only evolved by Data Scientists in DS.
- **Self-service Rules:** represents Rules Projects containing simple Rules which are only written by Analysts in PROD, using Case Manager.

See the page [How to organize your work with blocks](#) to learn to set a **Pulse app** with this structure.

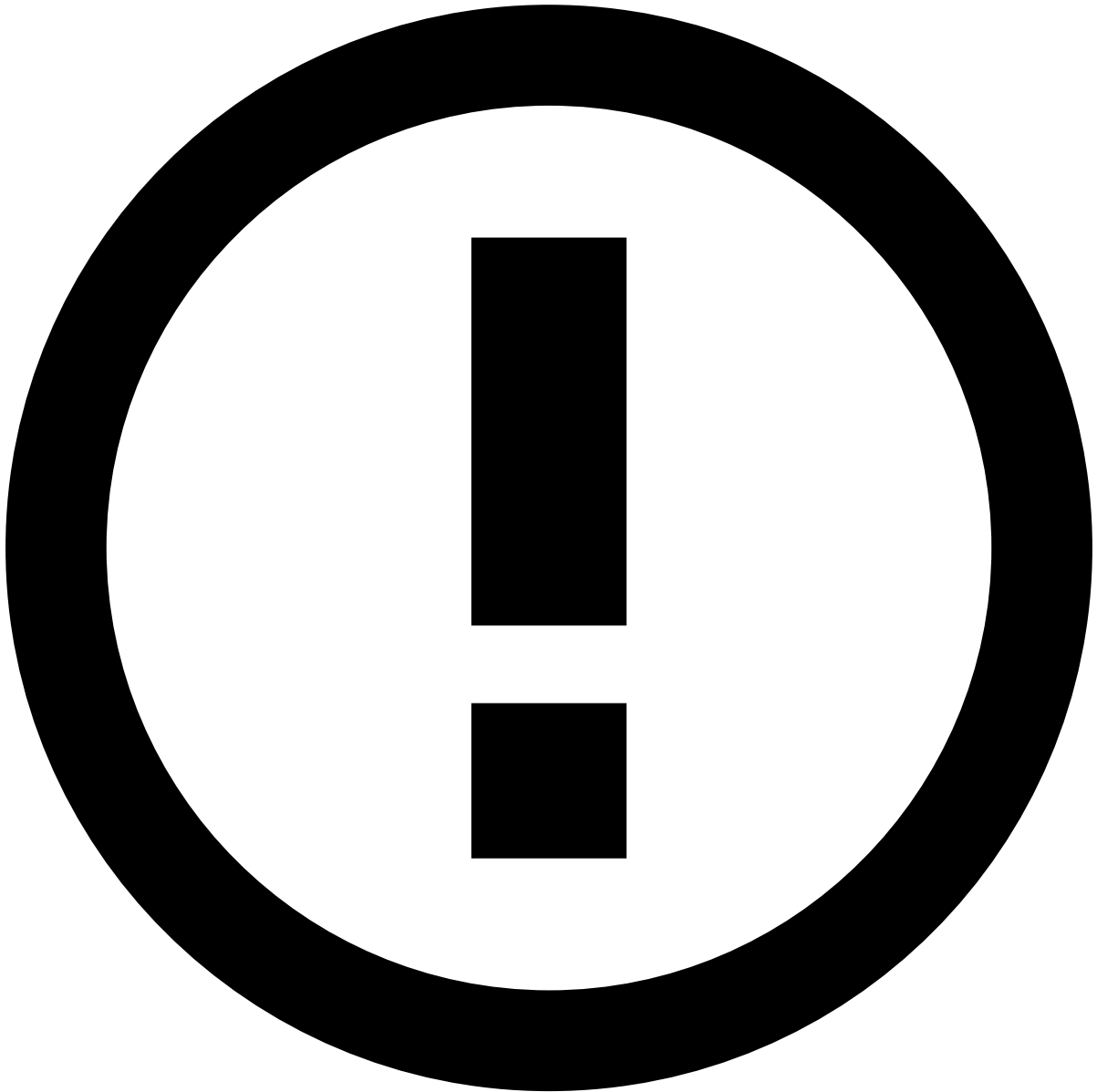
Workflow between environmentsâ

The figure below illustrates the procedure to bring new improvements to PROD after the initial deployment:



When to use this approach

You should follow the procedure below to deploy any resource type (e.g. Models, Plans, **Rules**, **Lists**). This procedure allows Fraud Analysts to use PROD to score transactions and evolve SSR, while Data Scientists and Engineers evolve resources in DS and DEV.



App Collaboration approach

The workflow explained in this section is specially designed to be adopted after the initial deployment, i.e. when you have already a live solution, to continuously improve the project with subsequent deployments.

The numbers shown in the diagram represent the following steps:

- **Step 1:** Export the resources that are being developed in DS and import them in DEV. See the Section [Migrating Data Science work to DEV](#) for more information about this step.
- **Step 2:** Export SSRs from PROD to DEV. This is required because SSRs might have changed while you evolved your applications in DEV and DS. Therefore, you need to ensure that the SSRs being written in PROD are migrated to DEV to test the full solution before the new deployment. See the Section [Migrating SSRs from PROD to DEV](#) for more information about this step.

After having your work fully developed and tested, you can bring the latest version of your application to PROD. One option is to perform a blue-green deployment.

The steps above are broken down into key topics in the next subsections.

Step 1: Migrating Data Science work to DEV

Migrating the resources to DEV requires loading them in the Data Science tab and applying them in the runtime Workflow. Check the page [How to organize your work with blocks](#) to understand how to map resources between Data Science tab and runtime Workflow.

The following pages will help you to accomplish this stage:

- [Exporting](#) and [Importing Lists](#)

Allows you to migrate Lists to DEV. By default, when you run the export and import wizards, all the Lists are migrated automatically (i.e. this only include List metadata). All the other resources (i.e. Plans, Models and Rules Projects) can be optionally migrated using the selection options provided on the wizards.

- [Exporting](#) and [Importing Plans](#)

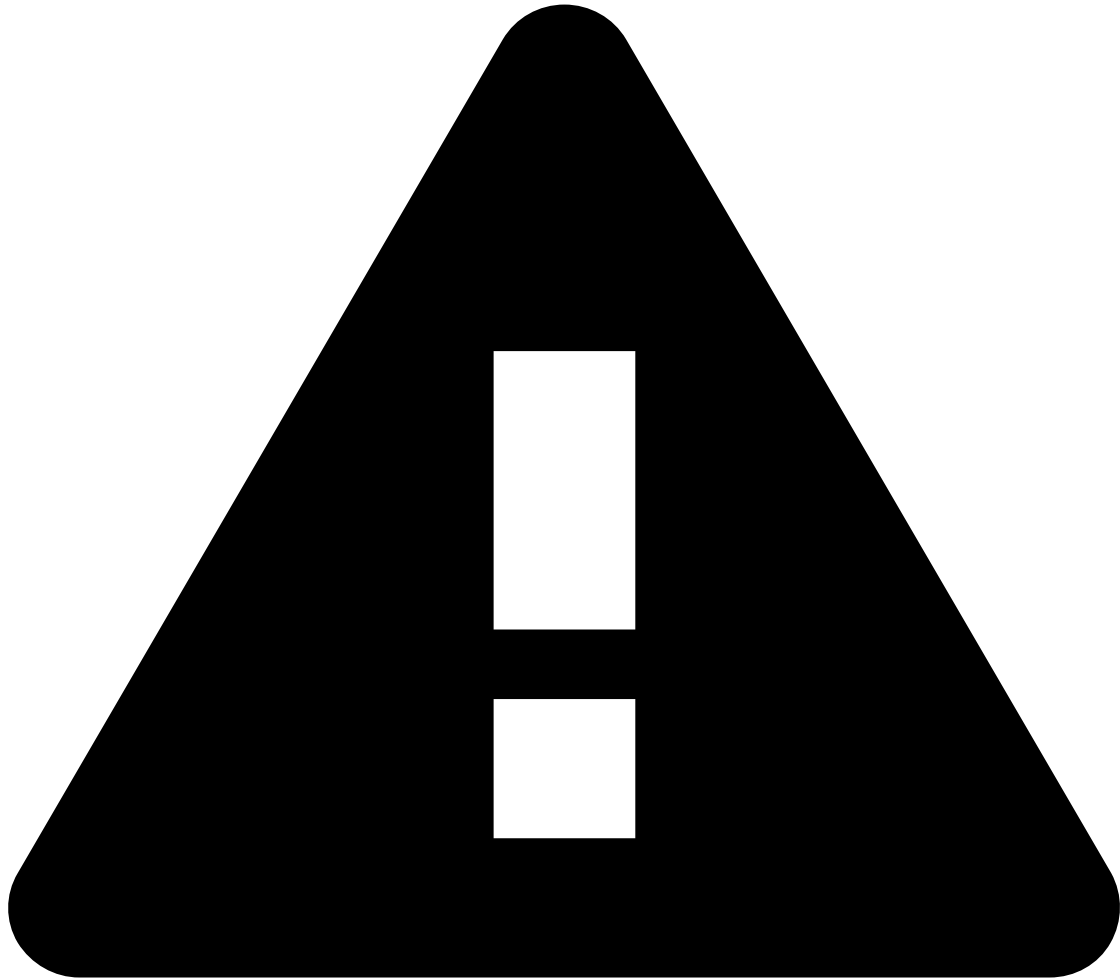
Allows you to migrate the work related to the [Feature Engineering](#) stage of the Data Science loop. This includes the configurations that add fields to your dataset schema once the Plan is executed in the new environment (e.g. to add derived features, historical profiles). Optionally, you can choose to migrate Plan executions. Note that, every time that Data Scientists works on a Plan that has impact on the Data Source, the Model has to be re-trained. In this case, you also need to export and import the Model.

- [Exporting](#) and [Importing Models](#)

Allows you to migrate the Model configurations, and optionally, you can include the binary of the Model.

- [Exporting](#) and [Importing Rules Projects](#)

Allows you to migrate the block of Rules developed by the Data Scientists in DS. By exporting and importing Rules Projects you are migrating the entire Rules Project with its Rules, and optionally, you can include a list of Rules Snapshots.

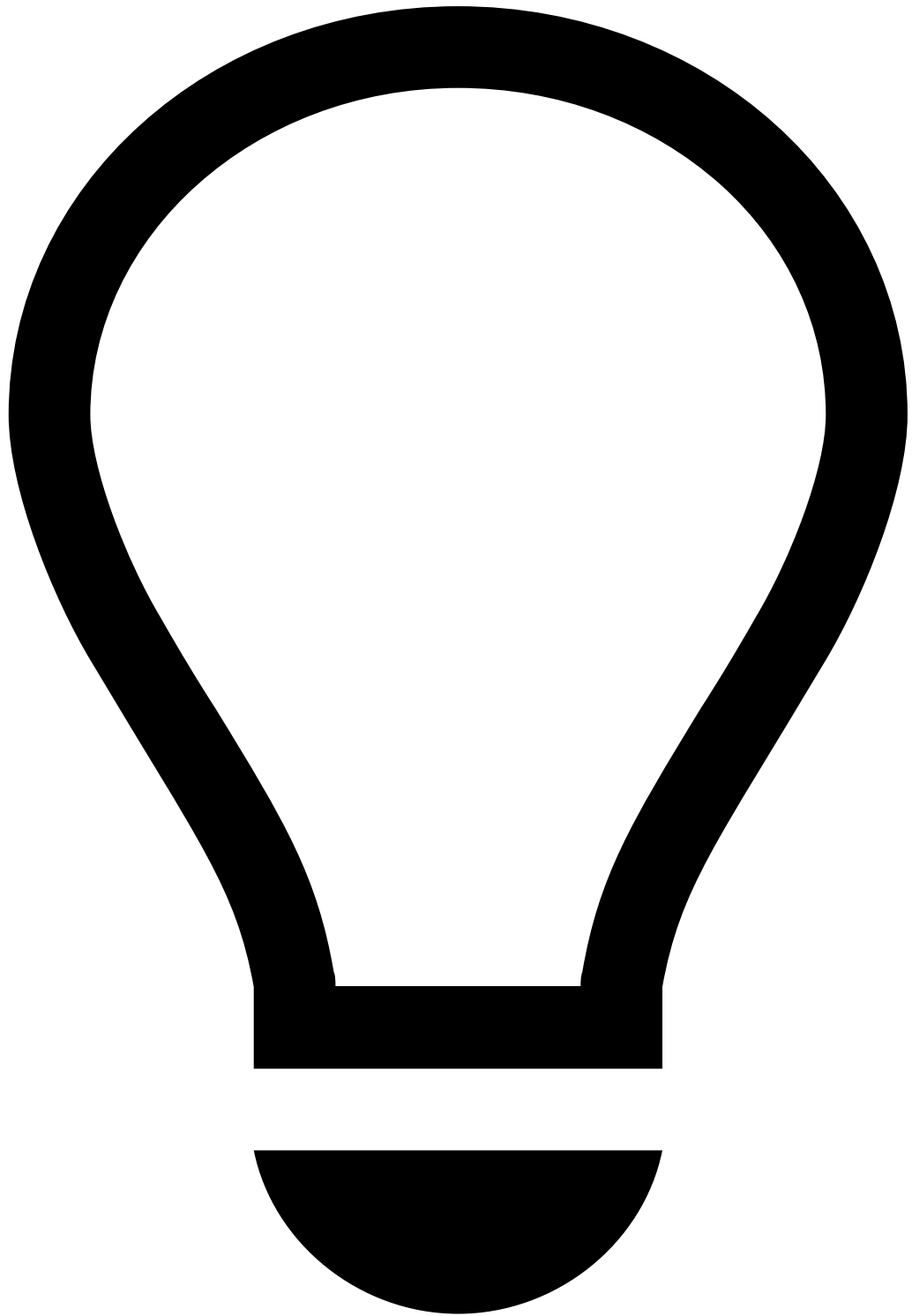


Block ownership - Rules Projects

Feedzai strongly recommends to evolve Rules Project using the concept of [Block ownership](#) to get the most out of the App Collaboration Loop with multiple teams working on the project. If multiple teams make changes simultaneously in the same Rules Projects using different environments (i.e. not adopting the resource ownership), you will have to merge changes manually by opening and editing the JSON file obtained by the Export Resource tool.

In addition to the links above, consider the following typical scenario:

- An engineer is working on a Plan to add derived features to a data schema in the DEV environment. At the same time, a Data Scientist is working on a Rules Project that is using the old schema. The Import resource option allows you to choose the new schema when importing the Rules Project from DS to DEV, assuming that the new schema is already configured in that Rules Project in the DEV environment.



tip

Check the section [Schema Conflicts](#) in the page [Importing resources](#) to learn how to perform this operation.

Step 2: Migrating SSRs from PROD to DEV

During the DS and DEV work, Fraud Analysts might evolve the blocks of SSRs in PROD. Completing this stage ensures that you integrate the most recent changes in PROD with the new improvements in DEV. This stage should be done before bringing the new app version to PROD to test the full solution before the new deployment. The easiest way to accomplish this stage is by exporting and importing SSRs from PROD to DEV, respectively.

The following pages will help you to accomplish this stage:

- [Exporting](#) and [Importing SSR](#)

In PROD, you can use the Export Resources wizard to select the Rules Projects related to SSRs and export as a zip file. This file can be then imported in DEV using the Import Resources wizard.

Learn More

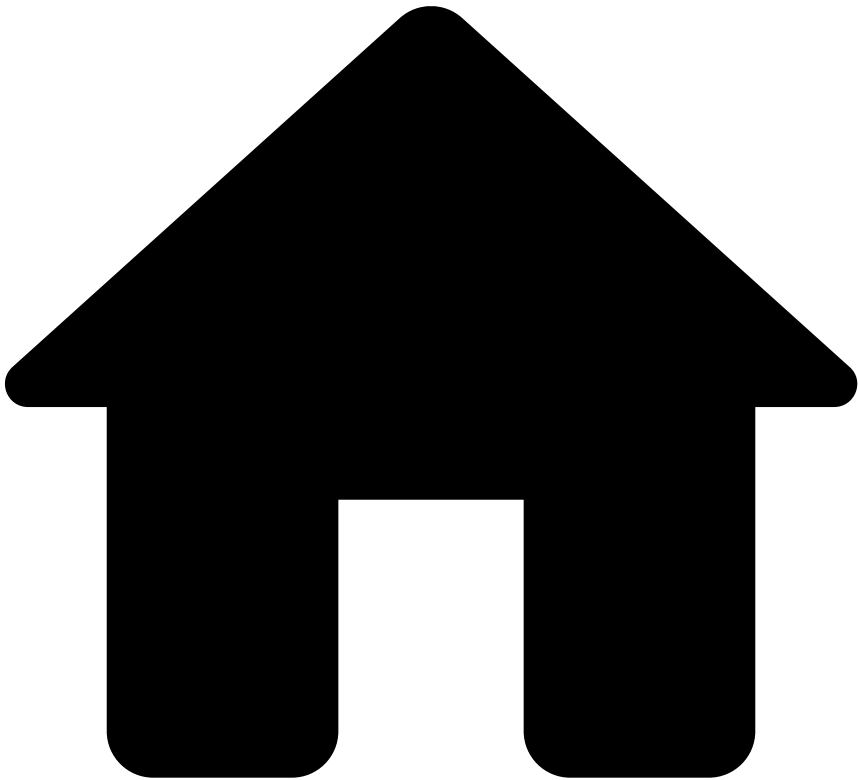
The following pages allow you to complete different tasks related to the App Collaboration Loop:

- [How to organize your work with blocks](#)
- [How to version and store](#)
- [How to merge](#)
- [How to code review](#)
- [Errors in App Collaboration](#)

Additionally, learn more about the concept of [App Collaboration](#) by checking the User Guide, including the following key pages:

- [Exporting Resources](#)
- [Importing Resources](#)

-



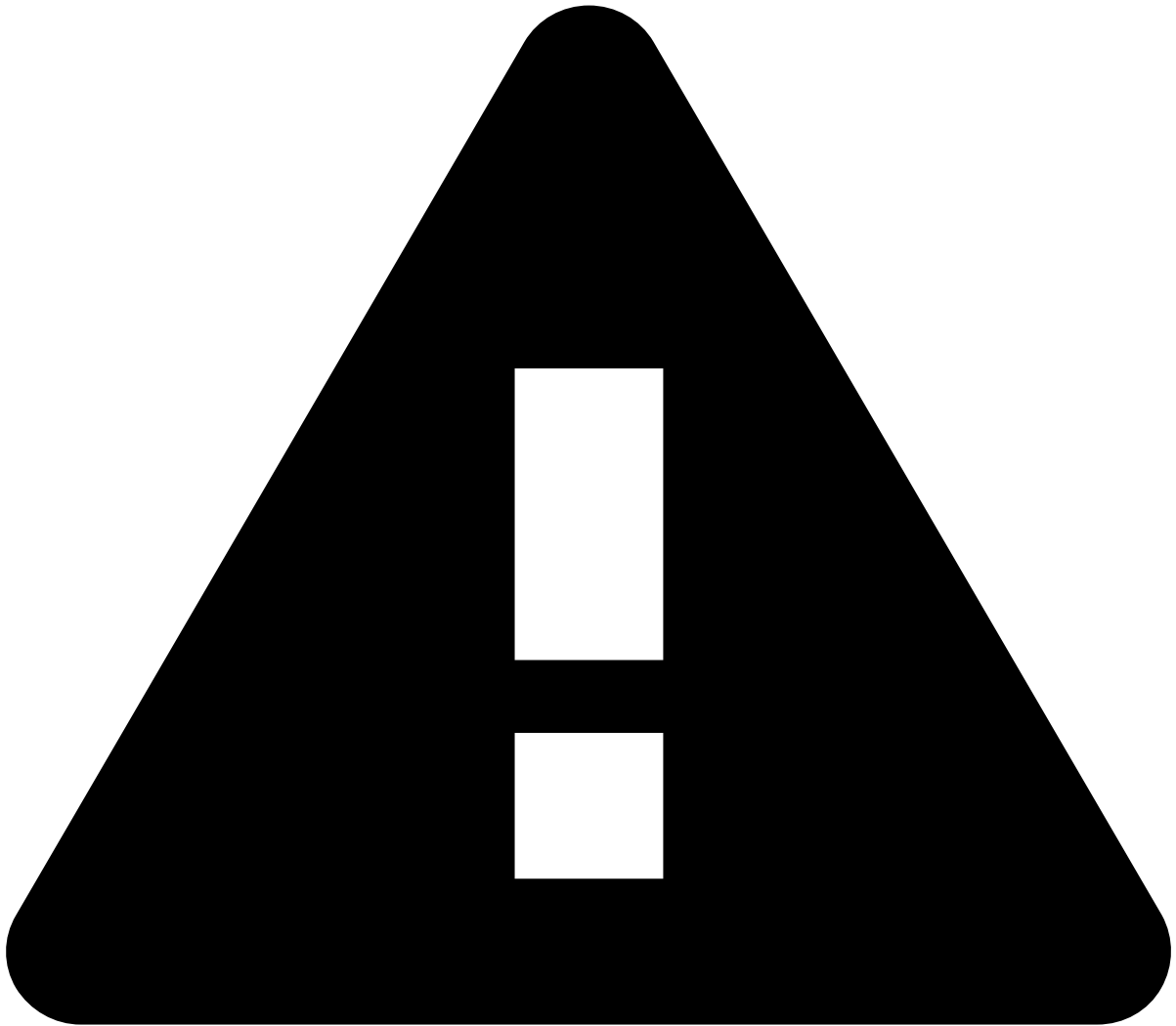
- Project Organization
- Managing Users

On this page

Managing Users

Was this page helpful? 

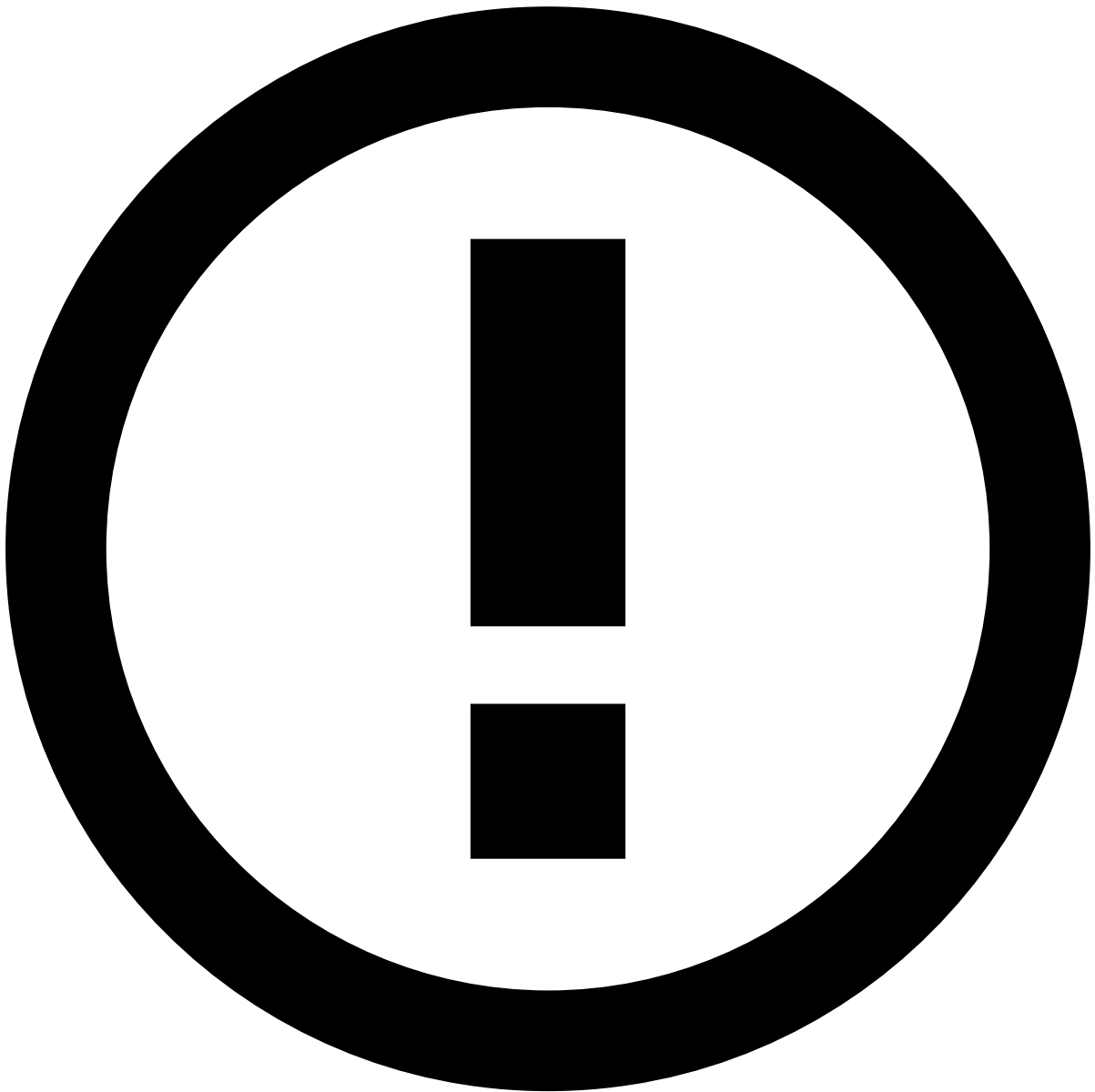
The following sections describe how to create and manage users in Pulse.



caution

Only users with the `usermanager` role can manage other users. Users tagged with the `admin` or `developer` roles **can not** be edited or deleted.

The **User Management** dashboard can be accessed by selecting the User Management icon in the navigation bar.



info

Users tagged with the `admin` role will have access to additional administration features, and must therefore access the **User Management** tab by first selecting the Administration icon in the navigation bar, and then selecting the User Management icon in the sidebar.

Create a user📄

1. In the navigation bar, select the User Management icon to access the **User Management** page.
2. Select the New User button to open the prompt to create a new user.

Screenshot of the Create User dialog

3. Enter all necessary details.
 - **Full Name** (mandatory)
 - **Username** (mandatory)

- **Email** (mandatory)



info

The email input will be used to set and recover the user's password.

- **Roles** (mandatory)
- **Tenants** (optional)

4. Select the **Create** button.

The newly created user will receive an email in the assigned email address to set a password. They may also need to activate 2FA, if Pulse is configured to make it mandatory. After the password and 2FA are set, the new user will be able sign in to their account and use it normally.

Edit a user

To edit a user, do the following steps:

1. In the navigation bar, select the User Management icon to access the **User Management** page.
2. In the list of users, select the Edit icon next to the username of the user you want to update.
3. Edit the fields in the dialog to make the intended changes. **Mandatory** fields cannot be left empty.
4. Select the **Save Changes** button.

Unlock a user

A user can be locked out of the system if one of the following scenarios occur:

- If the user tries to sign into the account unsuccessfully too many times.
- For inactivity, i.e. if the user doesn't sign in for a prolonged period of time.

If a user gets blocked as described above, a **User Manager** can change their status. To do this, do the following steps:

1. In the navigation bar, select the User Management icon to access the **User Management** page.
2. Select the **Unblock User** button in the **Status** section of the blocked user.

The user is now unblocked and can sign into the account.

Disable a user

Users can be disabled permanently, for example when a person no longer is associated with the company through which they had access to Pulse.

A **User Manager** can disable another user by performing the following steps:

1. In the navigation bar, select the User Management icon to access the **User Management** page.
2. Select the Change status (green checkmark) icon in the **Status** column of the intended user.
3. Select the **Confirm** button in the confirmation prompt.

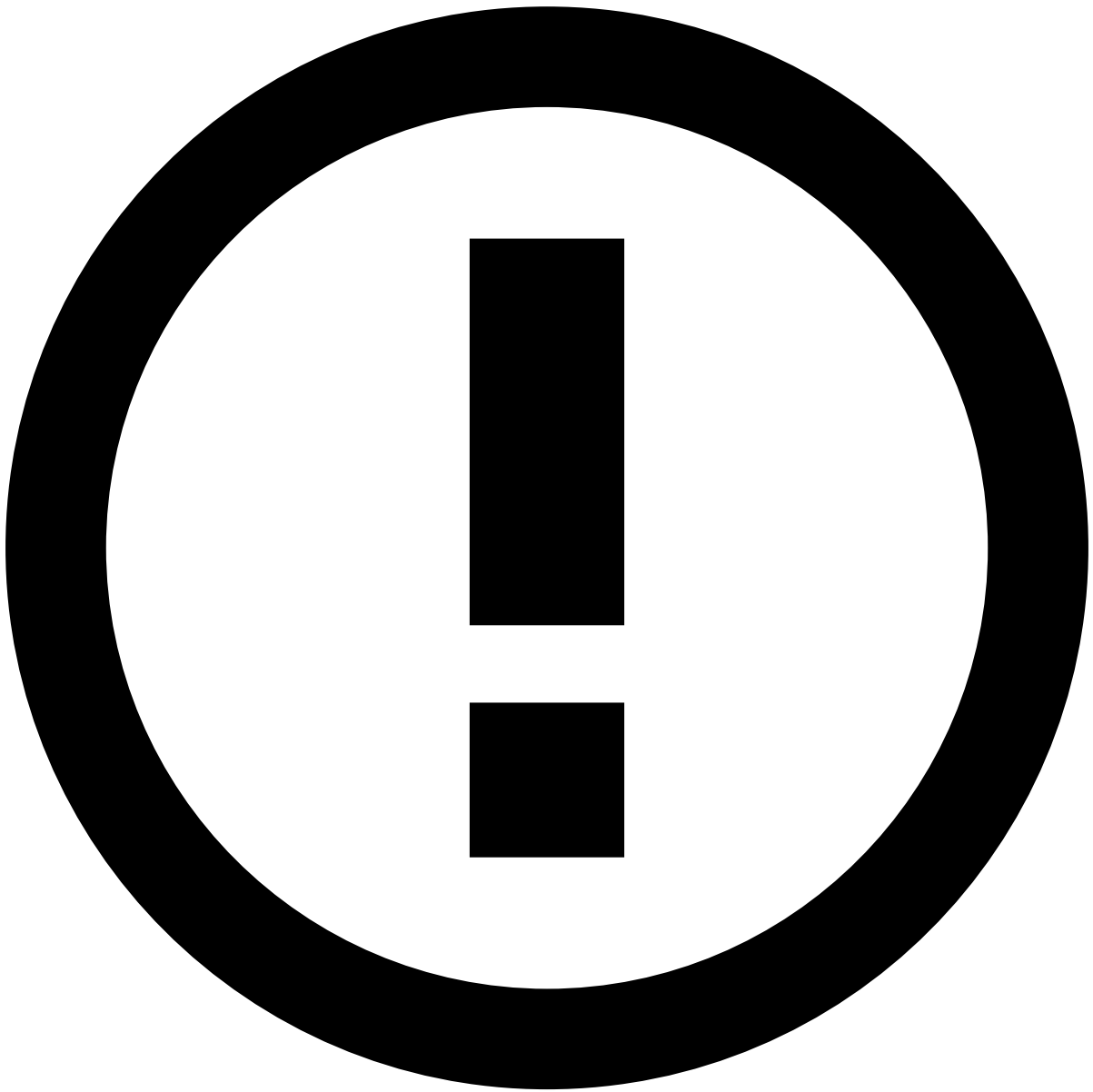
The user will no longer be able to login into the platform with their credentials.

Export users

A User Manager can export a list of users, alongside some of their attributes, by performing the following steps:

1. In the navigation bar, select the User Management icon to access the **User Management** page.
2. Select the [Export] button in the table header.

This will trigger the download of a CSV (comma-separated value) file with the following attributes: name, email, full name, roles, groups, tenant groups, enabled/blocked state, and External Auth.



info

Users stored in an external Identity Provider (such as Okta, Microsoft Active Directory, etc.) will only be exported after the user logs in for the first time in the Feedzai system. These users can be identified by having the External Auth attribute set to true.

Next steps📖

For user management in a multi-tenancy context, see [Tenant User Management](#).

-



- Project Organization
- Monitoring

Monitoring

Was this page helpful? 

This section introduces how to monitor Pulse and its applications. We'll cover the multiple channels that can be used to gather information about what's happening inside Pulse: Log files, the UI and programmatically via JMX.

- [JMX and Notifications](#) â Learn how to access metrics from the Pulse Server and applications via JMX MBeans.
- [Logging](#) â Learn how to configure and read Pulse logs
 - [Event Storage Dump Logging](#)
 - [Logging Confidential Data](#)
 - [Logging Partitioning Information on Plan Operator](#)
 - [Logging Reference for Security Audits](#)
 - [Microsoft LightGBM logging](#)
 - [Configuring Jetty to Log HTTP Requests](#)

- [Pulse Probes](#) â Learn how to use Pulse metric probes, that provide instrumentation data on pulse processing steps in real time.
 - [PKernel Probes](#)
- [Monitoring Metrics](#) â Learn about monitoring metrics exposed in the /metrics endpoint.
 - [Monitoring Processing of Profiles in PKernel](#) â Learn about PKernel monitoring metrics.
 - [Monitoring Replication of Events in the Workflow](#) â Learn about metrics to monitor the replication of events in the workflow.
 - [Monitoring the Persistence of Events in the Workflow](#) â Learn about metrics to monitor the persistence of event data.
 - [Monitoring Cassandra Driver Sessions](#) â Learn about Cassandra driver sessions monitoring metrics.
 - [Monitoring Processing of Feedback Records in Pulse](#)
 - [Monitoring Event Storage Dumps in Pulse](#)
 - [Monitoring DynamoDB Profile Store](#) â Learn about metrics to monitor the DynamoDB profile store.

-



- Project Organization
- Reviewing List Changes

On this page

Reviewing List Changes - Four Eyes Check

Was this page helpful? 

[Lists](#) provide a way to amplify the power of [rules](#) by allowing repetitive patterns to be defined as a single rule that then gets applied to a large number of values. Given the high level of automation that lists enable, ensuring every change is reviewed by a second pair of eyes helps prevent unwanted or unplanned changes from being published.

If the **Four-Eyes Check for Lists** functionality is enabled in your Pulse environment, changes to lists will not be published right away; instead, they will be queued for review. Such pending list changes only go live after a reviewer approves them.

Reviews of changes made to lists can be requested and performed directly in the Pulse interface. This page describes both sides of the review process: how to request a review for changes to lists, and how to perform such a review.

Prerequisites^â

To allow reviews of list changes, an administrator must set the `pulse.global.managedListsRevision.enabled` property to `true` in the Administration > Configuration view. The default value of this property is `false`.

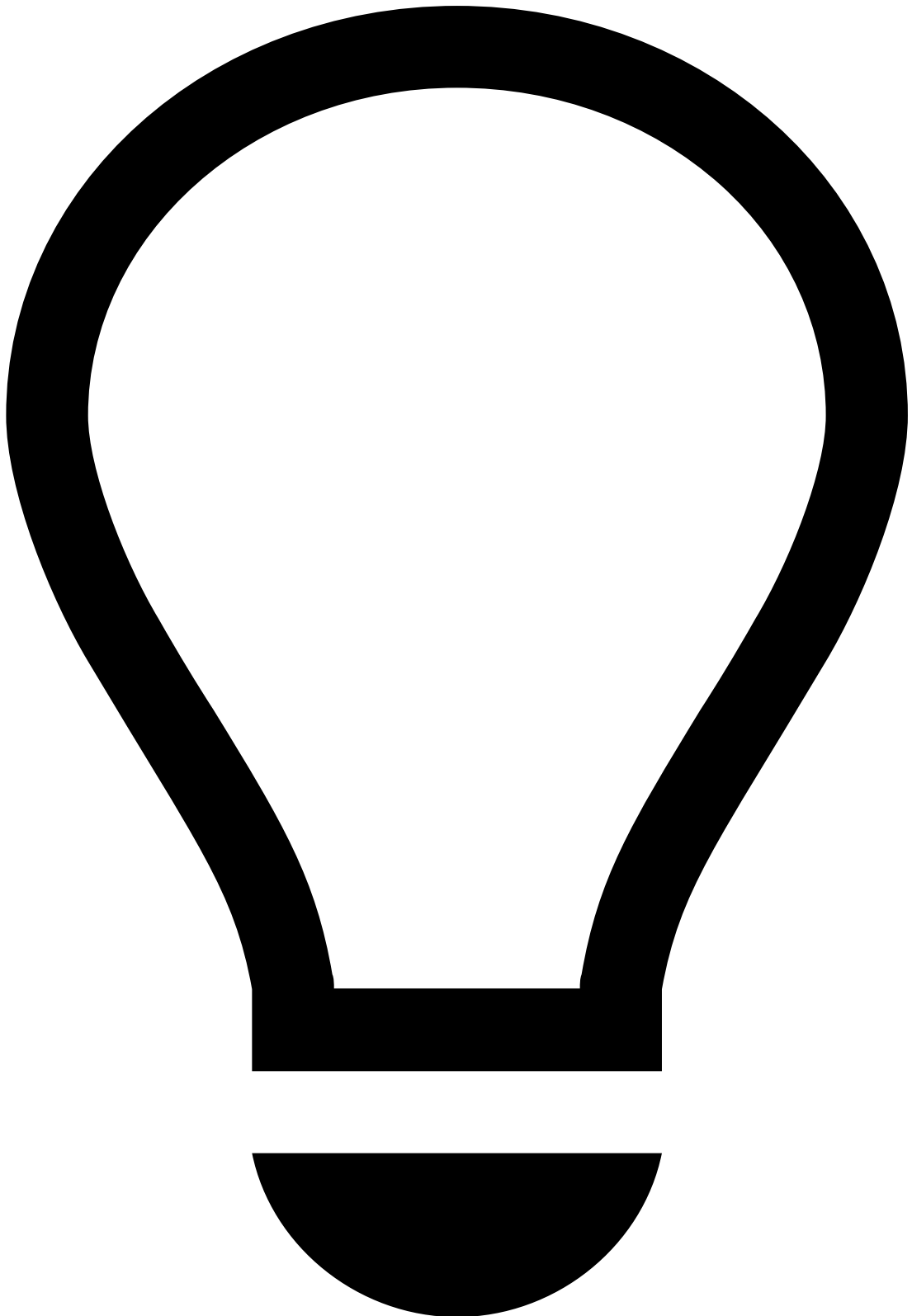
Optionally, for the notification emails to work as designed, the following settings can be configured as well. However, if you don't intend to use the notification emails, these can be skipped:

- The SMTP properties must be configured, so that the notification emails are properly sent.
- The `pulse.global.managedListsRevision.notifications.email.from` property: sets the email address from which Pulse sends the notification email to the reviewers. By default, it's `[email protected]`.
- The `pulse.global.ui.host` property is used to configure the URL where Pulse runs, and by default it's set to `http://localhost:8080/`. The notification email contains a link to the list changes that need to be reviewed, and the link starts with the URL set in this property. If the default value is not set to the actual value, the link in the email won't work.

Get in touch with Feedzai to set up four eyes check for lists in your Pulse environment.

Set up a static e-mail to receive all Four Eyes Check notifications^â

In addition to direct notifications sent to the author and reviewer in four eyes checks, Pulse allows setting up a static e-mail address which will receive all four eyes check notifications. This allows Pulse users to review the open revisions more thoroughly, and thus catch errors that might otherwise be missed. Note that the recipient of these emails will not be addressed directly, as they will be merely copies of the notifications sent to the author and reviewer of the corresponding four eyes check.



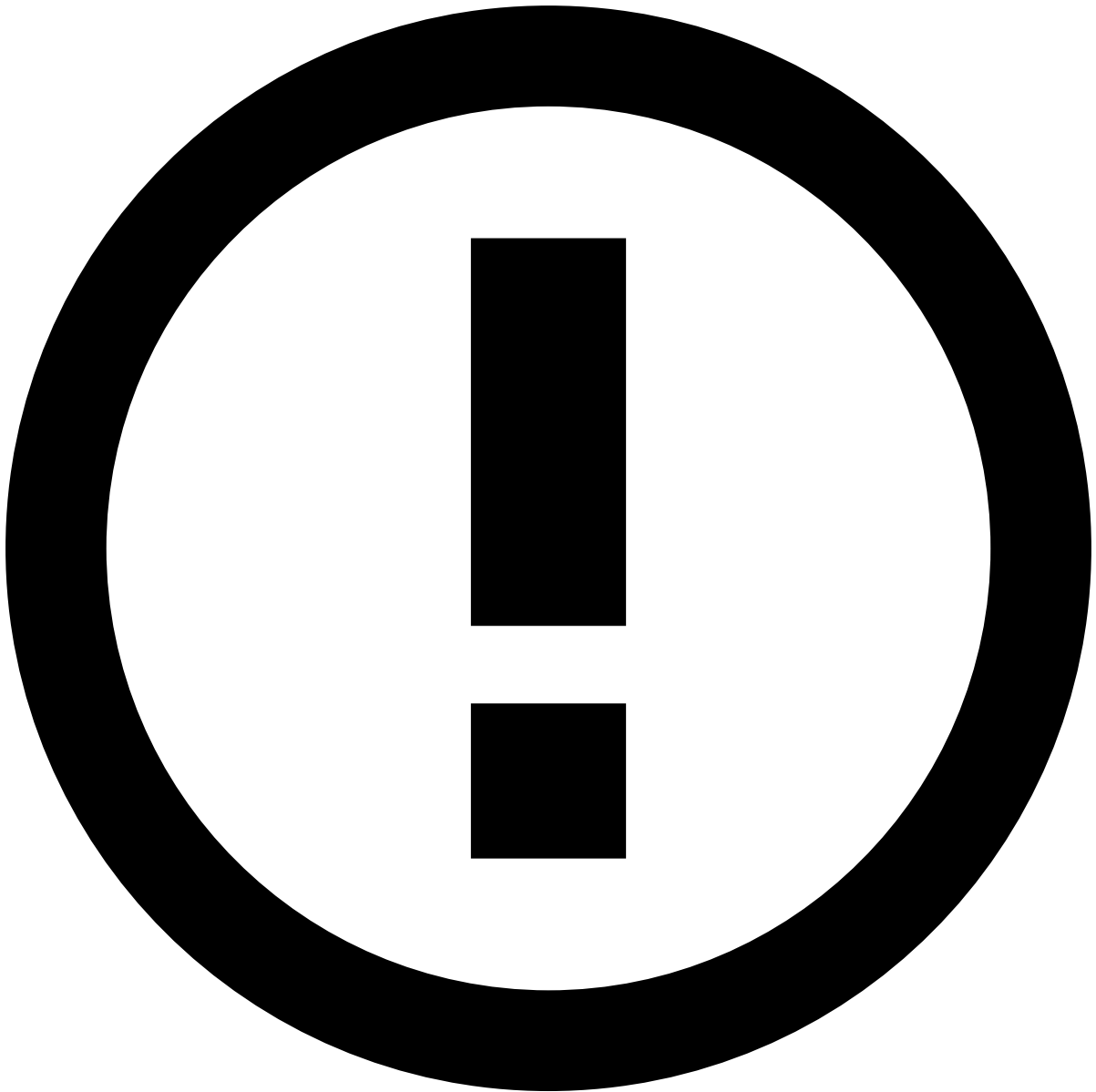
Notifying multiple users

This e-mail address can be set to that of a mailing list, which allows multiple users to receive every four eyes notifications.

To enable notifications for Pulse managed lists, provide the e-mail address via the following Pulse property:

```
pulse.global.managedListsRevision.notifications.email.staticreviewer .
```

Request a review for list changes

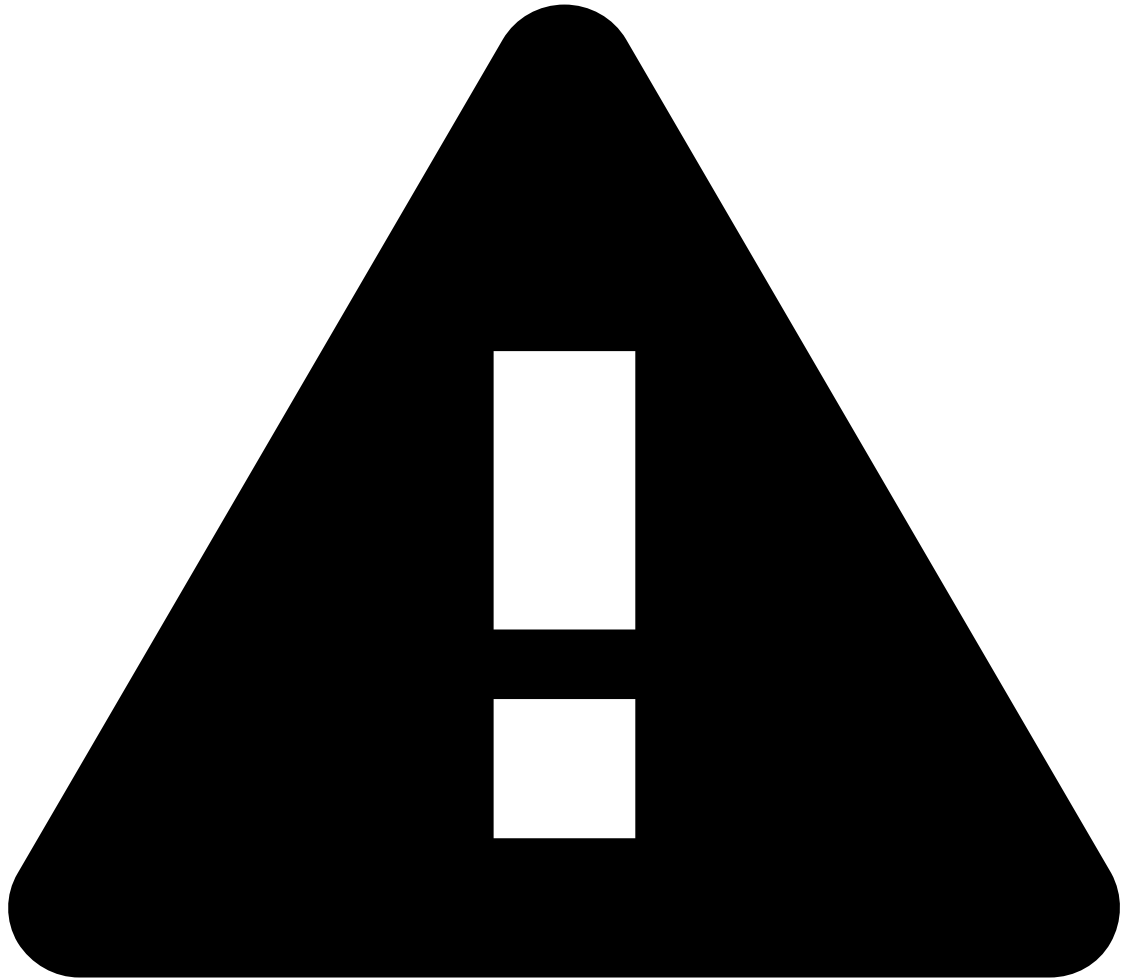


info

Users need to have the `application:managedlistitems:revision:create` [permission](#) to request reviews of list changes from others.

To request a review of a change made to a list, perform the following steps:

1. In the sidebar, select the Lists icon to open the **Lists** view.
2. Select the list you want to make changes to.
3. Make the desired changes to the list. For example, adding a new item, editing or deleting an existing item, or importing a batch of items from a CSV file.

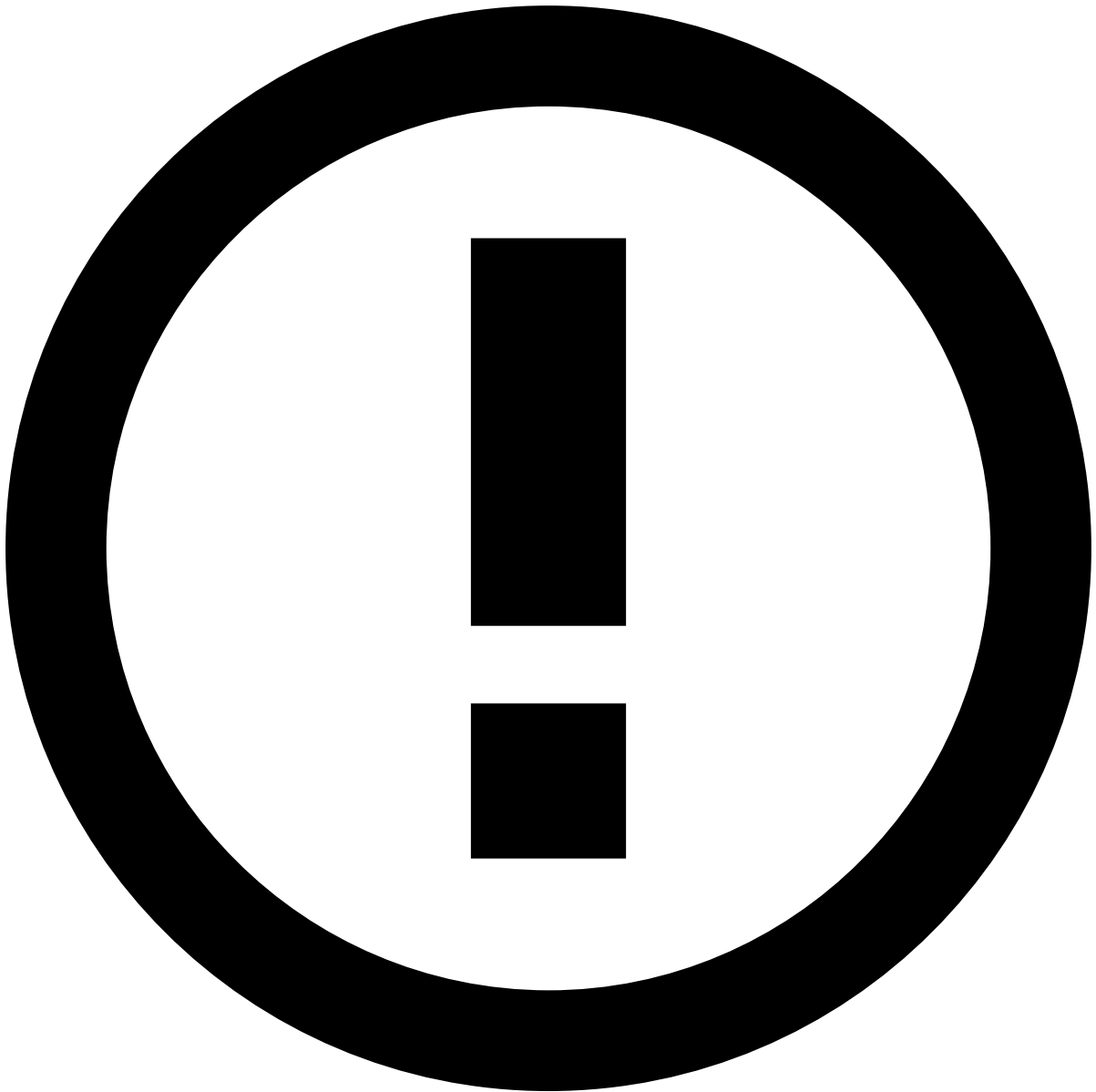


caution

You won't be able to make changes to list items for which there are already pending changes. Attempting to do so will show an error message indicating the ID of the pending review.

4. Optionally fill the **Notify users** field to indicate specific users you wish to inform of your review request. If the notification settings are [properly configured](#), and the users have an email address associated with their Pulse account, they will receive an email message notifying them of the pending changes.
5. Select the Submit for review button to add the changes to the review queue.
A floating success message will be displayed, confirming that the changes have been submitted for review.

Review pending changes to lists



info

Users need to have the `application:managedlistitems:revision:read` and `application:managedlistitems:revision:update` [permissions](#) to view pending changes to lists and review them.

1. In the sidebar, select the Lists icon to open the **Lists** view.
Lists with pending changes will be marked with an **Unreviewed** label.
2. Open the list with pending changes that you wish to review. Each item with changes will be marked with a label indicating the type of change (e.g. "Edited", "Added", "Deleted", etc.)

List with unreviewed changes

3. Select the Review Changes button at the top right of the table of lists.
The number in parenthesis is the count of unreviewed changes.

4. Review the overall changes to the list. Change details will at first be collapsed, as indicated by the  symbol to their left.

List changes for review, collapsed

5. Expand the details of each change using the  button. This will display a side-by-side comparison of the old and new data associated with the change. Created items will have no content on the left side; deleted items will have no content on the right side. The image below shows an example with two changes one adding a new item, and the other editing an existing item:

List changes for review, expanded



caution

Changes that consist of batch imports of list items will have no preview (as those can typically include thousands of items). In such cases, the only details that are shown are the number of imported items (in the "Name" column), the import mode (append without overwriting existing items, append with overwriting existing items, and complete rewrite), the change author, and the change timestamp.

6. Select "accept" (, ) or "decline" (). For either action, a confirmation dialog is shown. It's possible to disable the display of this dialog in the future, but it's strongly recommended to keep it. A floating success message will be displayed, confirming that the changes have been reviewed and (if approved) will be applied to the list.

•



• Concepts Glossary

On this page

Concepts Glossary

Was this page helpful? 

Acquirer

An acquirer represents the bank or the financial institution that acquires money from the client on behalf of the merchant as a payment for products or services.

In an every day scenario the acquirer is responsible for handling the merchant's transactions and collect payments from the clients [issuers](#).

Action

Actions are asynchronous flags that are raised on events when some predefined condition is met.

For example the **Blocked** action is raised when a fraudulent transaction is found.

Administrator🔗🔗

Pulse user that has admin role.

This user can:

- **Monitor and Control** Pulse services.
- **Manage** security configuration.
- **Manage** Pulse's configuration attributes,
- **View** Pulse's log and general information.

Aggregation🔗🔗

An aggregation is the calculation of a [metric](#) for a combination of dimensions 🔗 i.e. one or more fields that are used to [group](#) the data into sets, each of which gets its own aggregation value.

For example, if the dimensions are State and Bank, the aggregation will be calculated for all the possible combinations of the available states and banks. If there are 50 states and 10 banks in the dataset, the aggregation needs to be calculated 500 times.

AppEngine🔗🔗

An AppEngine is a conceptual group of services that run on their own process.

Services that run on an AppEngine are said to run on an **AppEngine Context**.

Batch🔗🔗

The amount of [events](#) to be processed at the same time.

By default the events are processed one by one. However, most use-cases don't need a sub-second real-time resolution (where resolution is the max timeframe in which to report notifications). It's very common that a resolution of one second, or even more, is sufficient. Given that the current latency to process an event is low (sub-millisecond), multiple events can be processed at the same time, in **batches**. Batch option for stream definition, for example, serves for this purpose.

Chargeback🔗🔗

A chargeback happens when a transaction made previously between an customer and a merchant is returned to the customer.

While this can happen in many different use cases, such a case would be making a return after purchasing an item in a store. Here the merchant will make a chargeback of the transaction made previously. It's basically a reversal mechanism for transactions.

Clock🔗🔗

An important Pulse property that defines how a [timestamp](#) is generated for each event injected to [stream](#).

If you are already using Feedzai Pulse interface you might already be familiar with choosing option for clock.

In this context three types of clock mean the following:

1. **Event-based**-means that each event itself contains the timestamp
2. **System clock**- means that when the event arrives, the server marks it with the current time of the machine
3. **Stream clock**- means that when an event arrives its time is set according to the clock of the corresponding stream

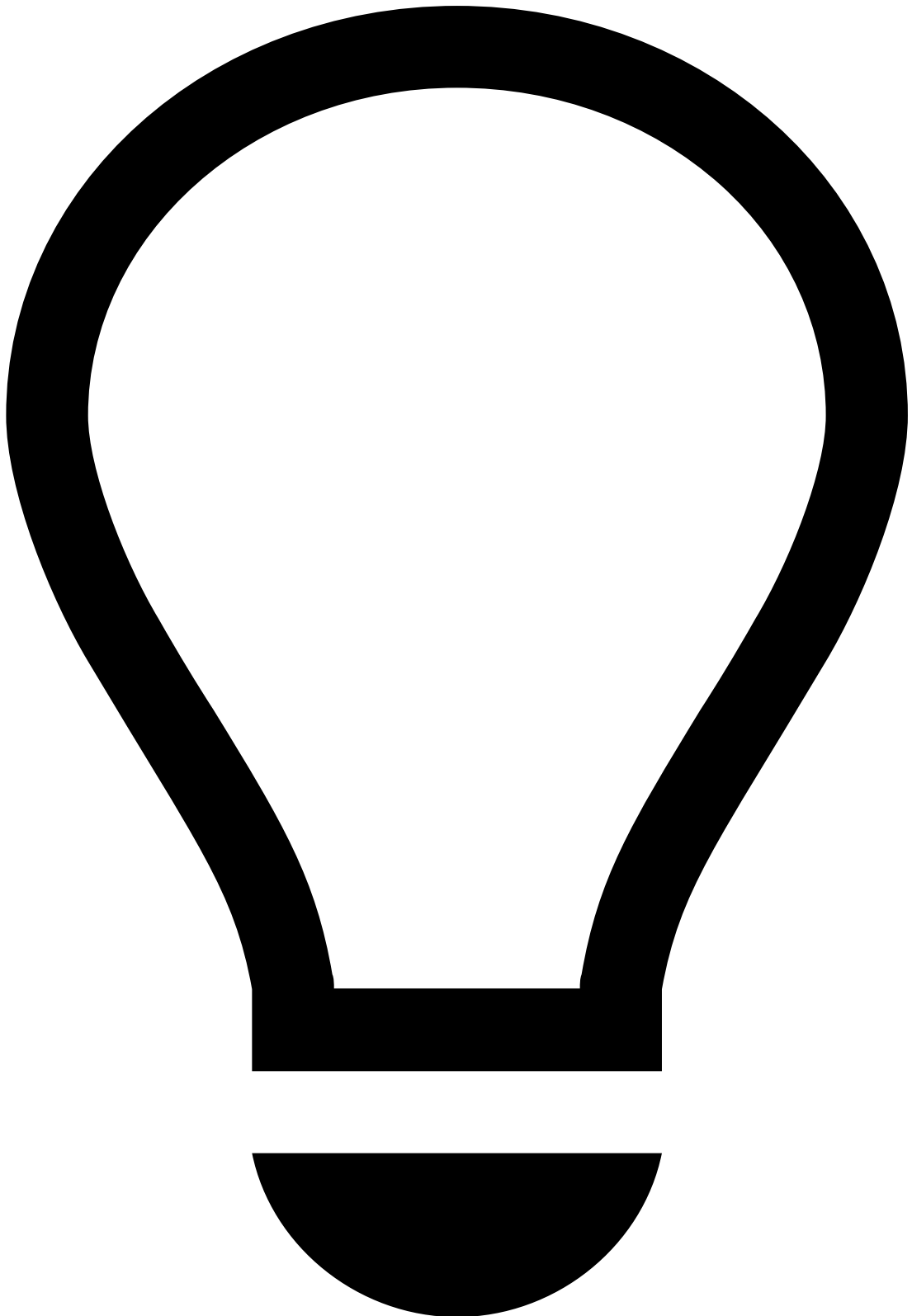
Cluster

All the Pulse software components that are connected to Zookeeper, these include Pulse Servers, Application Engines and external Batch Jobs. Each cluster lives in a single Data Center, and when referring to a Pulse deployment spanned across multiple Data Centers the used term is usually Global Cluster, but it will nonetheless be made clear.

CNP

Card Not Present transaction represents a payment card transaction that was made without the customer being physically present with the card.

Payments made over Internet are one example of CNP transactions since both the customer and the card wasn't physically present in the POS when the transaction was made. These types of transactions have a much higher rate of credit card fraud than their CP counterpart, mainly for the easiness of using counterfeit or stolen credit card data.



CP

Inversely, [Card Present](#) transactions represent payment card transactions where the card was physically available when the purchase was made.

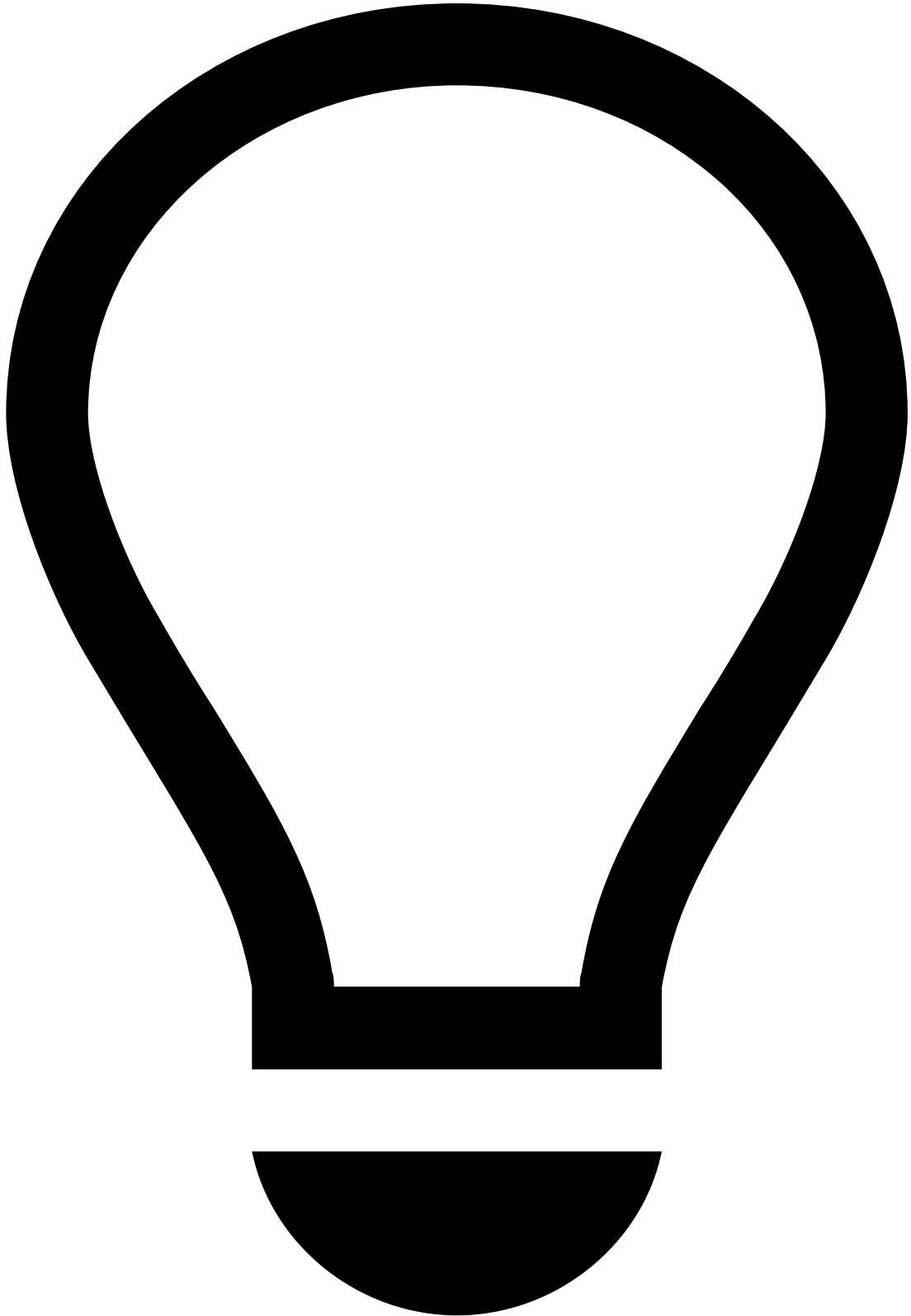
Control Flow

Control flow is a mechanism by which it is possible for a computer program to make choices.

In the context of a [Workflow](#), Control Flow are the mechanisms that allow to control how an event progresses through the Workflow Elements, namely if it should skip none, some or all of the Workflow Elements that follow.



Card Present transactions represent payment card transactions where the card was physically available when the purchase was made.



Inversely, [Card Not Present](#) transaction represents a payment card transaction that was made without the customer being physically present with the card.

Data Center🔗📄

A logical and physical relation of nodes. Each Data Center has its own cluster and must have all the components of the system to work by itself. This means that it follows a shared-nothing approach in order to support scenarios of disaster recovery where a single Data Center must be able to fully operate without need of a system that belongs to another.

Dead Letter🔗📄

Built-in [Action](#) that is raised when an event reaches the end of its path in the [Workflow](#).

This [Action](#) is used by the Event Storage Service to store the event in persistent storage.

Dictionary🔗📄

Is a data-type which is similar to a map or a hash table in a typical programming language. It also maps keys to values. In PulseQL the keys would be the groups and the values are the aggregator results.

So, for example, for the transactions stream example, if we create a dictionary grouped by bank, the keys are names of banks and the values can be the average amounts. A snapshot of this dictionary could look like this;

bank	avg_amount
Bank A	\$ 50
Bank B	\$ 30
Bank C	\$ 65.5

Dictionaries support lots of operations. You can, for example, filter them, project their entries, aggregate them or lookup the value of a single entry.

Dimension🔗📄

A dimension is an analysis axis that contains a limited set of values from which some can be selected, thus drilling in into that sub set. Along with Dimension Mappings they can transform machine codes into human readable information: "0" s and "1" s into "United States of America" and "Portugal" .

For example the total USA transactions could be at a given time 150K, drilling in in this case could be choosing to view the transactions only for the state of New York (which could be 10K). We could drill even further, if we had, for example, a bank dimension. In this case we would only see the transactions for a given bank in New York (which could be 2K).

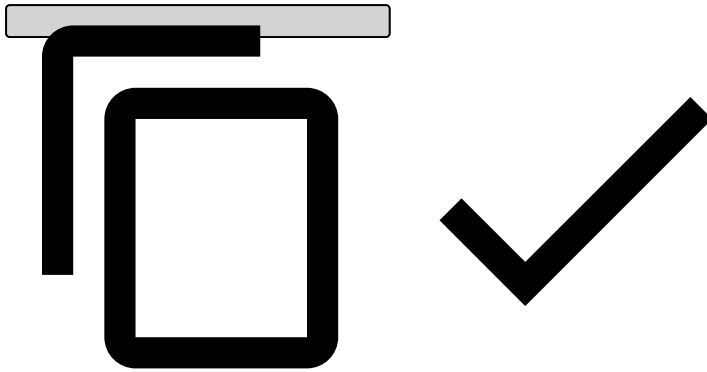
Event🔗📄

A single *packet* of data within a [stream](#); also called a "tuple". An event consists of [fields](#) with associated values, conforming to the stream's data [schema](#).

The following is an example of a tuple for a stream of [transactions](#):

```
{
  timestamp:    1,
  state:        'Alaska',
  bank:         'Bank of America',
  rejected:     1000,
  duration:     15.5,
  amount:       1000,
  operation_type: 'Withdrawals',
  card_type:    'Visa',
}
```

```
client_type: 'ATM'
}
```



Event Processing¹

A set of technologies designed to process high-volumes of continuously arriving data in real-time.

In this context, continuously means that data may arrive at any moment over the course of arbitrarily long periods of time. For example, an application designed to monitor traffic in a highway system may be required to execute without interruptions for months or even years. During all this time, the application is continuously receiving notifications such as "car entered the highway", analyzing them and producing meaningful results like "there are currently 1000 vehicles inside the highway" or "the vehicle with license plate 11-22-AA drove through the highway at an average speed of 150 km/h". This information could then be displayed in a dashboard where a human operator could look at it and use it to make informed decisions. The important point to retain is that, in this kind of applications, one can't wait for all data to arrive before it is processed. Moreover, in most scenarios, it is desirable to process incoming data as soon as possible so that the resulting analysis is still up-to-date and relevant when it is acted upon.

Explanation¹

An Explanation details why a given transaction reached a certain outcome and got a given [score](#). It contains a list of reasons that help explain the outcome and gives the user further hindsight about the final decision.

Please refer to [this section](#) to learn more about Explanations.

Feature¹

A feature is an attribute of an [instance](#). It represents information necessary for a Machine Learning algorithm to identify a pattern.

Please refer to [this section](#) in order to obtain more details.

Feedzai IQ¹

Feedzai IQ is a suite of ready-to-adopt risk assessment solutions providing AI-powered insights based on aggregated intelligence from across the Feedzai global network.

For more information about Feedzai IQ, please refer to [this section](#).

Field¹

A Field is a component of a data source's [schema](#). In CSV file or other tabular data formats, a field corresponds to a column.

To learn more about fields, refer to the [Data fields](#) section in the documentation about Data Sources.

Group

A virtual subset of members of the cluster that have similar functionality, i.e. all the Pulse Servers belong to a group of pulse servers but the Application Engines belong to another group.

Note that this is a distinct concept from the [Group By](#) operator used in [plans](#).

Instance

Instance is an event that is enriched with the [Features](#) from the [Fraud Model Definition](#).

The instance does not travel the [Workflow](#), it is computed using the event data and then sent to FOS either for training or scoring.

Instance may also be used to refer to an incarnation of a Service like the [Fraud Rules Service](#).

Issuer

An issuer represents the bank or the financial institution that legally emitted the payment card to the cardholder.

In an every day financial transaction between a client and a merchant, the issuer is responsible for issuing the money to the [acquirer](#) bank. He does this on behalf of the client, usually assuming the liability for the client's capacity to pay off the debts.

MCC

Merchant Category Code is a four-digits number assigned to a merchant that represents the type of goods or services it performs.

This code is used to categorize the merchants by business area and activity in order to improve the transaction context. It's easy to understand why, the average transaction amount between a Bus Line merchant (MCC 4131) and a Car Dealership (MCC 5511) will be vastly different.

There are some merchants that have special MCC numbers assigned, the most known case being Airlines.

Member

A software component that is part of a cluster. Core members include Pulse Server and Application Engines.

Merchant

A merchant represents a business person who trades in commodities produced by others with the public, also known as clients. It is usually represented by an [acquirer](#) in a financial transaction.

Metric

A metric (also known as **profile** or **historical profile**) adds new fields to a data source, based on an aggregation of events over a period of time, to track entity behavior patterns.

At its core, a metric performs calculations, called [aggregations](#) —such as sums, counts, maximums, averages, etc.— on a specific data field (e.g. transaction amount), grouped by one or more fields (e.g. a credit card), and over a defined [time window](#) (e.g. the last 24 hours). This allows quantifying and monitoring behavioral patterns, making it easier to detect changes that may signal suspicious activity.

For more information about metrics, refer to the [Metrics](#) section in the documentation about Plans in Workflows.

Node^â

An instance of an operating system. It doesn't relate to physical machines *per se* since several operating system instances can reside in the same physical machine for example by the means of virtualization.

Operation^â

Term used in the context of [Resource Versioning](#) to represent a modification made to a Resource.

POS^â

Point of Sale is the equipment used by a merchant to handle a customer payment inside a store.

Can also be called checkout and it calculates the amount owed by the customer in exchange for goods or services acquired. Typically issues a receipt after the payment was made.

Plan^â

A plan is a recipe of how to augment a data source via data transformations, essentially transforming one data source into another. Plans are typically used to prepare the data to be used to train machine learning models, or to transform the data in the runtime environment for processing via rules or models.

To learn more about plans, refer to the [Plans](#) page.

Profile Update Job (PUJ)^â

A batch process that reads transactions from the database, computes the [metrics](#) using the event processing engine and stores them to a data store.

Reason^â

A Reason represents a fact that was considered by Pulse when scoring a transaction that went through an workflow. It can detail both fraudulent and non-fraudulent behavior and is usually part of an [Explanation](#).

Replication Factor^â

Replication factor is the number of nodes which are running the same component in distinct physical machines. For example, a replication factor of two means that the Pulse Server is replicated across two physical machines as well as the Application Engines.

Resource^â

Represents any entity inside Pulse. Examples of resources include KPIs, Baselines, Lists, Rules, etc.

Schema^â

A data schema is a set of metadata that describes the attributes available in an [event](#), and in particular their [data types](#).

Refer to [Data schemas](#) for a detailed explanation.

Score^â

Score is the value that represents the probability of an event being fraudulent. The higher the score the more likely is that the event is fraudulent.

The score is an integer value between 0 and 1000.

The process of calculating the score is named **scoring**. Please refer to [this section](#) to learn about models and training mechanisms.

Serviceâ

Service is an isolated piece of software that provides some kind of functionality as a service to other services.

Settlementâ

Usually the third and final step in a transaction, the settlement occurs between the issuers and the acquirer about two days after the original transactions were made.

It is in this step that the money is really transferred from the client (issuer) into the merchant (acquirer).

The reasoning behind this is that if a business receives 100 daily payments from bank A, it does not makes sense to make each individual transaction right in that moment. The transaction fees would be too high. What happens instead is that these 100 payments will summed in a single transaction that occurs X time later.

Slackâ

The amount of time the events can "wait" for each other when they are coming from different sources.

Sometimes you can be interested in receiving events from multiple sources where clocks are almost (but not quite) synchronized. In this case you can defined a time slack where the events are re-ordered and processed in-order.

Snapshotâ

A picture of the current state of data contained in the object requested at a pre-defined interval in time. It contains all the data of that object at that exact moment. It applies to all objects except streams.

Streamâ

A stream represents a source of real-time data. It's like a connection port between the world and Pulse.

Similar to a table of a database, it is the data structure inside Pulse that defines the schema of the data that will be injected into Pulse.

Tenantâ

Tenants are the individual entities to which Multi-Tenancy is applied to. They represent separate entities to whom data and certain resources (Rules, Environments and Templates) should be isolated. In order to allow this separation they are associated to users which inherit their isolation.

Tenant Domainâ

Tenant domains are used to group multiple tenants based on a common domain, they work much like a normal group. It is through them that we specify which Tenants can have resources on a given [workflow](#) element.

Timestampâ

The time the event occurs.

Transactionâ

A transaction is an exchange of goods between two parties.

TrustScore^â

TrustScore is an aggregated risk score that combines the outputs from multiple models trained on relevant patterns from Feedzai's global client base. It uses federated learning techniques to allow financial intelligence sharing without relying on consortium-based data sharing or aggregation, thus ensuring data segregation and confidentiality.

For more information about TrustScore, please refer to [this section](#).

Tuple^â

A synonym for [event](#).

Window^â

Time interval which defines which real-time events from the data stream should be used for calculating aggregations in a [metric](#). For details on the syntax used to define windows, please refer to the [Time window](#) section of the documentation on Plans in Workflows.

There are two major types of windows:

Sliding Window^â

A [sliding window](#) contains the events received within a fixed-size interval. This interval could be temporal or tuple-based. For example, a sliding window with interval equal to 5 minutes will contain only the events received during the past 5 minutes. As soon as the event becomes older than this threshold, it is expired from the window. The window in the example above is a time-based sliding window.

Tumbling Windows^â

Unlike sliding windows, tumbling windows expire all events at once, once they reach their predetermined size. Afterwards, they refill again and the cycle repeats itself.

Landmark Windows^â

[Landmark windows](#) are similar to time-based tumbling windows except you may control the exact time the window expires.

Non-Windowed Metrics^â

Non-windowed metrics are a type of [metric](#) that is not bound by a time window. They are limited only by the available historical data at the time of the metric's creation or update, which is determined by the data retention policy. For more information on this type of metrics, refer to the [Non-windowed Metrics](#) section.

Workflow^â

A workflow defines how a transaction event is routed between processing steps in a fraud detection pipeline.

Each processing step of a workflow is performed by a service (e.g. rules service, scoring service). Thus, a workflow is defined by a list of services that process the transactions and the connections between them.

Please refer to [this section](#) to learn what are workflows and how they work.

-



- Core Concepts

On this page

Core Concepts of Pulse

Was this page helpful? 

All tasks you perform with Pulse rely on a set of concepts that support the core of Pulse's functionality.

This page gives you the basics about the most important concepts of Pulse. For more details or for other concepts read the pages later in this chapter.

Data science loop

The [data science loop](#) is the process of creating and testing rules and models.

Data sources

[Data sources](#) are historical datasets imported to a data science environment. You can use them for data exploration, feature engineering, model training, rule testing, etc.

Plans

[Plans](#) define a transformation flow of one data source into another data source. They are useful to systematize data cleaning, data processing, and feature engineering, both in a data science pipeline and in runtime.

Models

Pulse enables you to quickly train [machine learning models](#) and evaluate the results over several iterations. It provides tools for you to extract relevant performance metrics and track your progress along each iteration of the data science loop.

Rules

[Rules](#) define a pattern based on the fields of a data source that produces a binary result for each record that matches a condition. Pulse lets you define and evaluate the performance of blocks of rules using a data science approach.

Workflows

[Workflows](#) represent the life cycle of a real-time event through the system. A workflow maps other concepts, such as data science concepts, into runtime components and ensures they are properly translated and orchestrated.

Lists

A [list](#) represents a set of values and aims at improving the efficiency of rules when the same pattern needs to be checked against a large number of entries. For example, a list of credit card numbers, a list of merchants, etc.

User-Defined functions

[User-defined functions](#) (UDFs) enable you to extend Pulse beyond its off-the-shelf capabilities. They are especially useful when you need custom logic to process data, such as transforming data source fields with Plans or configuring conditions for rules.

-



- Getting Started

On this page

Getting Started

Was this page helpful? 

The following pages are a step-by-step tutorial on how to develop a simple application using Pulse. They will guide you through Pulse concepts and tasks described in the User Guide by presenting concrete examples.

These examples focus on detecting fraud in payments, in a transaction monitoring scenario. However, Pulse can tackle other use cases such as anti-money laundering.

This tutorial is just an overview of Pulse's fraud prevention applications to help you get started. You should explore each feature with greater attention to get a better understanding of all the concepts, resorting to the more detailed documentation whenever necessary.

Objectivesâ

With this tutorial, you will learn how to execute a typical iteration of the [data science loop](#), which includes the following stages:

1. Loading data.
2. Processing data.
3. Building and evaluating models.
4. Building and evaluating rules.
5. Creating a workflow that can be deployed in the production environment to process events.

Materialsâ

During the course of the tutorial, you will need to use two sample data files:

- [transactions.csv](#)
- [input.csv](#)

Download them to the `PULSE_SERVER_HOME/var/datasources/` path on the Pulse server where you will be working.

In addition, for the last steps you will need to have an input adapter set up on your server where you will be working.

To get help with the data files or with the input adapter, get in touch with your contact person at Feedzai.

Next stepsâ

Now that you have everything set up, go ahead and read the following pages to create your first data science resources and compose an event-processing workflow with them.

-



- Creating a Data Science project

On this page

Creating a Data Science project

Was this page helpful? 

Whether you are an experienced Data Scientist or still new to the job, learning how to use a new tool can seem a daunting task. This section of the User Guide will lead you through the several stages involved in your Data Science project and how to make the most out of Pulse.

It will help you how to map typical Data Science stages to Pulse, so that your learning curve is minimized, and you quickly get up to speed with Pulse. The main goal is to allow you to become more productive with Pulse than you already are with your current toolset. Pulse fully supports a complete [Data Science Loop](#).

Pulse, despite having fraud detection as a major use case, is completely problem-agnostic. This means that you can use Pulse to solve other types of problems, and apply the same Data Science approach to other domains and use cases.

To create a Data Science project in Pulse:

1. In the Applications tab, create a
2. Check the following pages to start your work in your new application:
 - [Cleaning and loading your data](#)
 - [Tuning Job executions with Job Profiles](#)
 - [Processing your data](#)
 - [Creating features and models automatically](#)
 - [Building and evaluating your model](#)
 - [Working with rules](#)
 - [Testing your work in a Sandbox environment](#)
 - [Interacting with Pulse via JupyterLab](#)

Additionally, it is also recommended that you fully acquire all the necessary context and understand what concepts you are manipulating in your project. To learn more about how Data Science works in Pulse, and Pulse's core concepts, check the following pages:

- [Data Science Loop](#)
- [Channels](#)
- [Data Sources](#)
- [Plans](#)
- [Models](#)
- [Rules](#)
- [Sandbox Evaluations](#)
- [Workflows](#)
- [Outcome Combinator](#)
- [Lists](#)
- [Jobs and Job Profiles](#)
- [User-Defined Functions](#)
- [Audit](#)
- [Concepts Glossary](#)

Using the Data Science tab

The **Data Science** tab is the entry point for Data Science tasks in Pulse. It enables you to manipulate core Pulse concepts, such as [Data Sources](#) and [Plans](#), and assists you throughout all stages of your Data Science project.

The following image displays the Data Science tab :

The Data Science tab contains the following sections:

1. Data Sources

List with all configured [Data Sources](#) and respective analysis status. Select a Data Source to configure, examine and run analyses on your data. Use the Duplicate option to avoid creating/recreating a Data Source from scratch.

2. Plans

List with all configured [Plans](#) and respective type (Deployable or Non-Deployable). Select a Plan to design, edit and execute your data transformations. Use the Duplicate option to avoid creating/recreating a Plan from scratch.

3. Model Projects

List with all configured [Model Projects](#). Select a Model Project to configure models and evaluations.

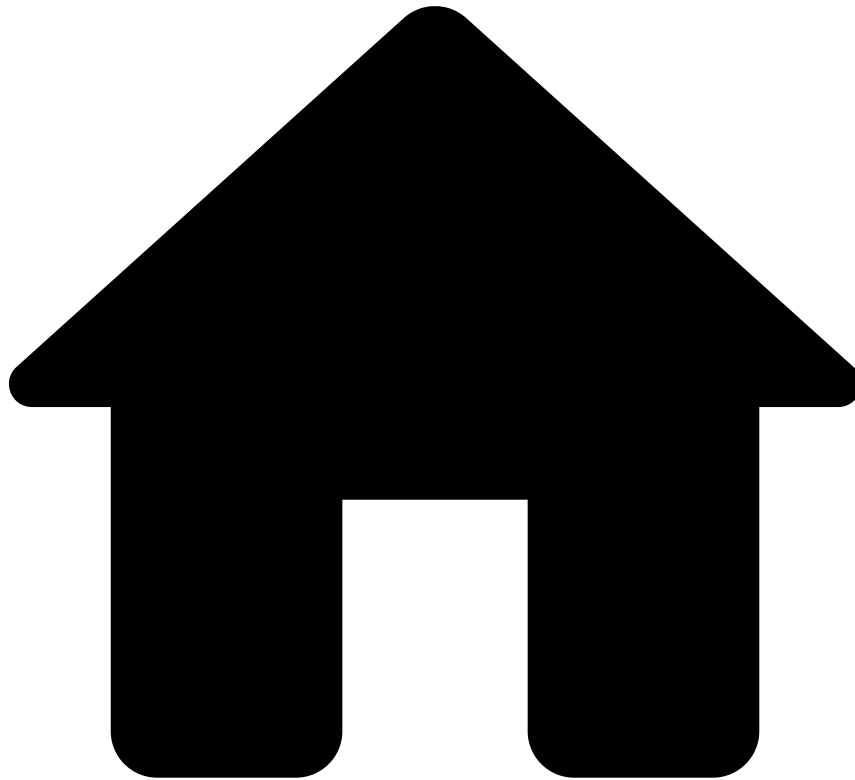
4. Rules Projects

List with all configured [Rules Projects](#). Select a Rules Project to configure rules, templates, snapshots and evaluations.

Getting Started

Start working on your Data Science project by learning how to [clean and load your data into Pulse](#).

-



- Deploying a project into Production

On this page

Deploying a project into Production

Was this page helpful? 

Pulse delivers a full Data Science platform aimed at helping data scientists do their jobs, with increased performance. Additionally, Pulse makes Data Science much more accessible to other user profiles, such as risk analysts, without requiring them to acquire the same set of skills a data scientist must have.

Similarly, Pulse is also a platform that makes life easier for engineers whose task is to take the work of a Data Science team to Production. In reality, since Pulse is focused on removing barriers for every user profile, deploying a project to Production and managing its lifecycle can even be performed by data scientists themselves with minimal assistance from engineers.

It is quite common for engineers to struggle when translating offline Data Science and Machine Learning work into executable software that runs properly in a real-time Production environment. It may take a lot of iterations to finally deploy

the Data Science work into Production and keep the same semantics and results that Data Scientists have designed and achieved previously.

Therefore, Pulse projects are designed to work in a way that abstracts away a lot of the complexity that engineers usually have to deal with. With a fully integrated Data Science modeling and run-time environment, Pulse enables a direct and seamless deployment, without any major effort.

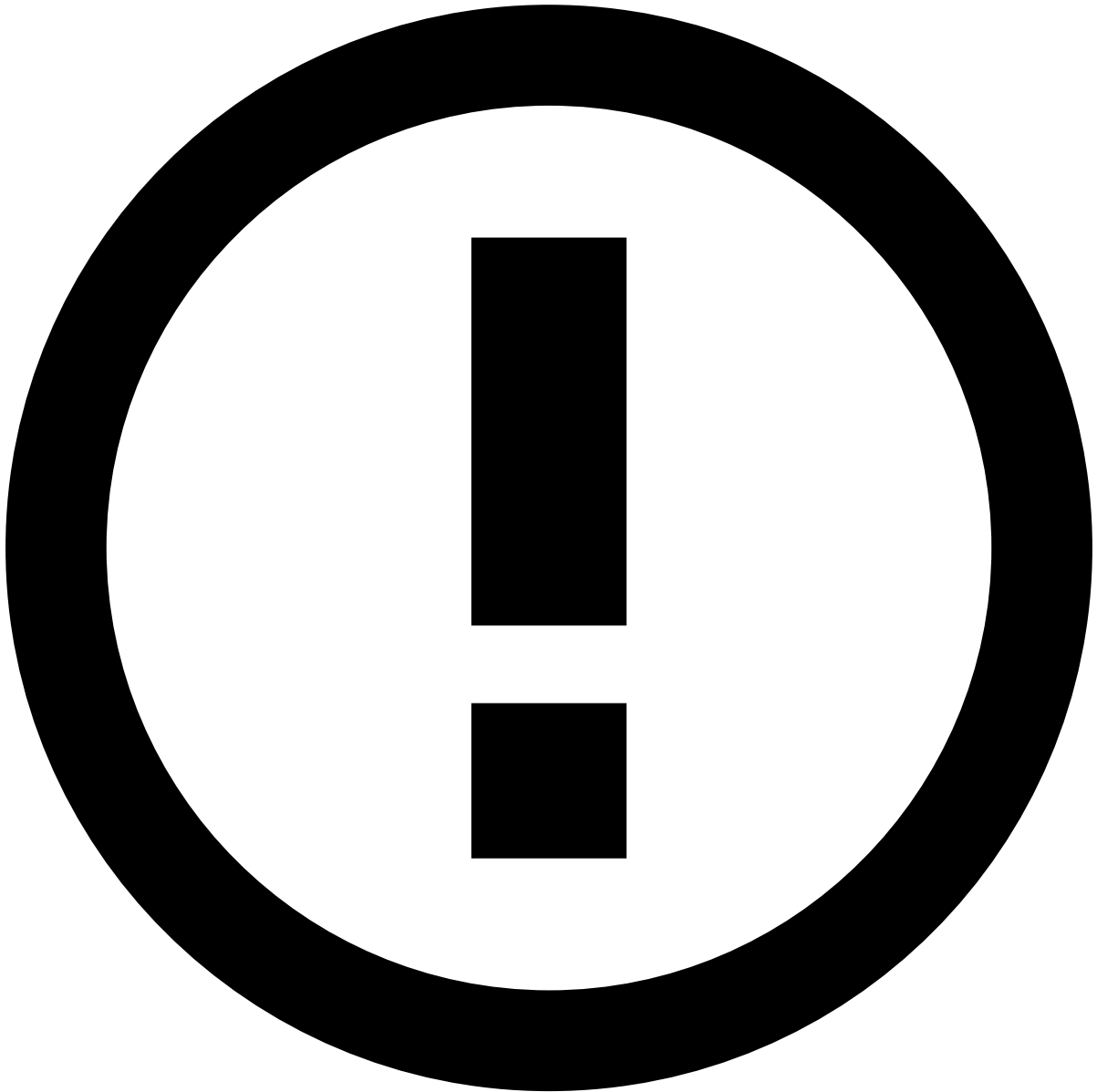


How Pulse projects work

To learn more about how Pulse fully integrates Data Science with deployment and run-time, make sure to learn more about [how Pulse projects work](#).

Once you have sign-off to deploy a Data Science project into Production, you must execute the following tasks:

- [Creating a schema](#)
- [Designing a workflow](#)
- [Managing the project life cycle](#)



Configuring components directly in Production

Since in Production, data comes into Pulse in real-time, there are some activities in Production that are not mandatory to be started in the Data Science stage. For example:

- It is very simple to generate a **Schema** from a [Data Source](#). However, in Production you only need the data structure, not data itself, so you can [define a data Schema](#) manually,
- Pulse provides a way to treat and evaluate the performance of **Rules** much alike a Data Science approach to evaluate, compare and iterate over **Models**. However, you can [deploy a Rules Snapshot](#) with only the definition of a block of **Rules** into Production, without evaluating it against test data.

Additionally, it is recommended that you fully acquire all the necessary context and understand what concepts you are manipulating in your project. To learn more about how Data Science works in Pulse, and Pulse's core concepts, check the following pages:

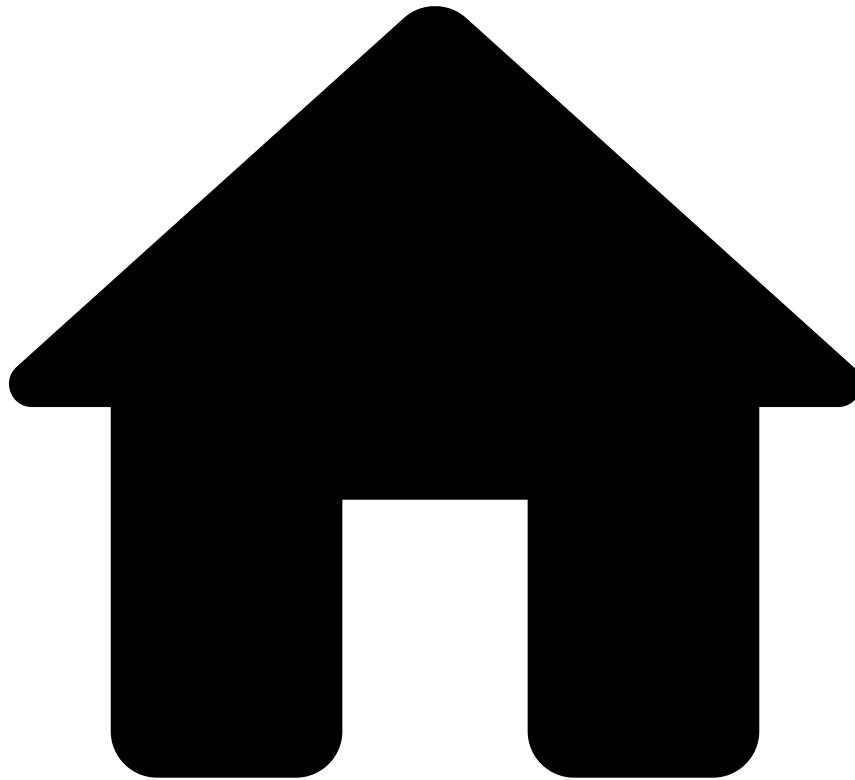
- [Data Science Loop](#)
- [Channels](#)
- [Data Sources](#)

- [Plans](#)
- [Models](#)
- [Rules](#)
- [Sandbox Evaluations](#)
- [Workflows](#)
- [Outcome Combinator](#)
- [Lists](#)
- [Jobs and Job Profiles](#)
- [User-Defined Functions](#)
- [Audit](#)
- [Concepts Glossary](#)

Getting Started[^]

Start working on your deployment by learning how to [configure your input data schema](#) and [how to consume external data](#).

-



- Feature Engineering

Feature Engineering

Was this page helpful? 

This section shows how to transform your raw and cleaned data into information that correlates with fraud or any other target variable, and select only information that is useful as input to your models.

Pulse supports the design of features and the selection of the relevant fields for your model with the following tasks.

1. [Selecting important fields directly from your dataset](#)

If you want to create features to use them in models, it's useful to have initial statistics about which input fields are correlated with the target variable. In Pulse, you can generate statistics over your dataset to uncover correlations between data fields and the target variable you want to predict.

2. [Creating a plan for the features](#)

In Pulse, a plan entity is a recipe of how to transform one data source into another. A plan contains map and metric operators that represent the new features. If you already have a plan for features, skip this step.

3. [Adding lists to the plan](#)

If you want to access list items of complex lists, you need to add them to the plan. For example, if you have a list with the information about the risk level each country has, you can add that list to the plan, and use it in your features and rules.

4. [Using string fields in models](#)

If your string fields have indexers, you can use the indexer in the model to score events.

5. [Creating derived features](#)

Apply complex expressions over your dataset's existing fields to extract new features.

6. [Creating metrics \(historical profiles\)](#)

Generate a large number of profiles and include time-based features.

7. [Using PQL in features](#)

Learn how to use the Pulse Query Language (PQL) in map and metrics operators to define the "formula" of the new features.

8. [Following best practices for plans](#)

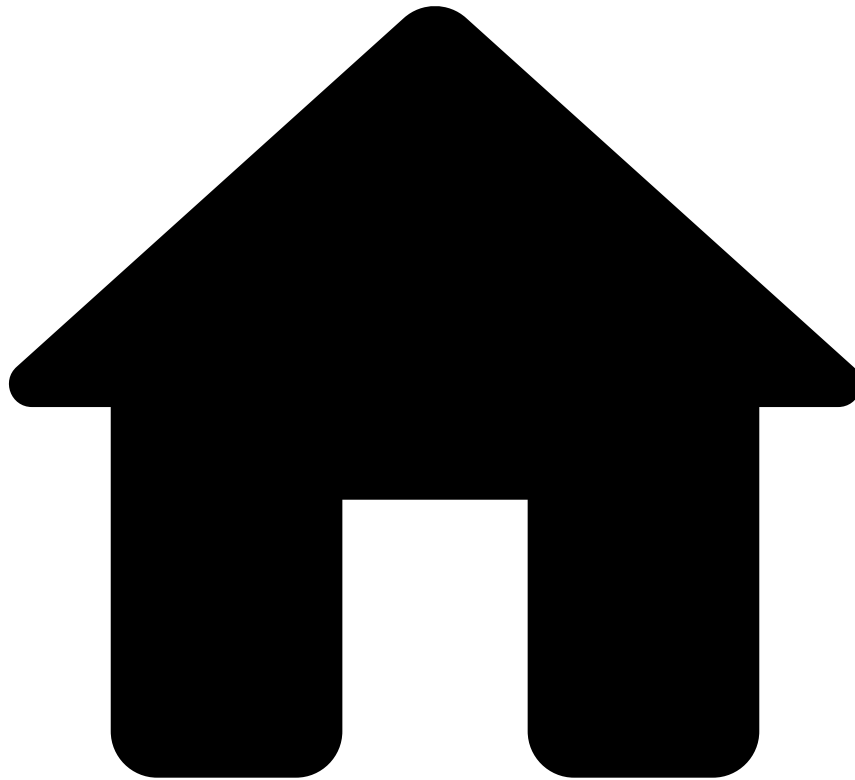
Use best practices to organize map and metrics operators within a plan in a meaningful way.

9. [Creating a data source with the features](#)

Run the plan to create the data source that contains the new fields.

The pages in this chapter walk you through these tasks.

-



- How Pulse projects work

On this page

How Pulse projects work

Was this page helpful? 

Pulse projects integrate the work performed during the Data Science loop with runtime to accomplish a defined objective, e.g. detect fraud.

Data Scientists, Software Engineers and Fraud Analysts play a key role in the field of fraud prevention to transform raw data into valuable insights. However, this work has a number of complex challenges to address:

- With traditional tool sets, Data Scientists often need to dig into different tools and techniques to execute their tasks. For each problem, Data Scientists need to adapt their approach and write customized code, e.g. scripts to import data, database queries, data visualization techniques.
- Developing and iterating Machine Learning [Models](#) require Data Scientists with special skills who are difficult to find in the market.

- Organizations struggle when they need to deploy efficiently and accurately Data Science work in runtime. Usually, it takes several months for engineering teams to deploy a trained Model and ensure that is working as designed and tested during the Data Science stage.

Pulse projects are designed to address these challenges. The ultimate goal of Data Science work is to configure Pulse to run as a real time [Workflow](#) to accomplish a target objective, e.g. detect fraud. Pulse projects provide the following:

- Pulse delivers a full Data Science suite to help Data Scientists while handling most of the heavy work by running all the technological complexity behind the scenes.
- Pulse makes it easy to perform Data Science work, allowing personas with different profiles to use the platform.
- Pulse focuses on bridging Data Science and runtime together under the same platform. When you are developing your activities along the [Data Science Loop](#), you are also performing some configurations that allow easy and quick deployments.
- Pulse also allows access to [Feedzai IQ](#), a suite of risk assessment solutions that provide out-of-the-box AI-powered risk insights based on aggregated financial risk intelligence from Feedzai's global network.

Environments

A key idea in Pulse is the possibility of working in different environments to develop and test your work before deploying it into production. In fact, regardless of the project, developing and testing Pulse resources is recommended to be performed in offline environments, i.e. it should never be done in the same environment that contains the runtime stage evaluating real-time transactions (i.e. production environment).

Applications

Each project in Pulse is built as a Pulse application (**Pulse App**).

A Pulse App consists of a user defined session in which resources such as **Plans**, **Models** and **Rules** are developed.

Each application typically handles a specific use case (for example, Transaction Fraud or AML), and a set of channels (see [Channels](#) for more details).

With this architecture, users can develop work in different applications using offline environments which can be then moved to production environment. This concept is referred to as **App Collaboration**.

Check the page [Working with Applications](#) to learn more about App Collaboration in Pulse.

Project stages: Data Science and Runtime

Pulse projects are based on two main stages:

- **Data Science:** Fully supports all the steps in the **Data Science Loop**.
- **Runtime:** Executes the work that was developed in the Data Science phase in a runtime **Workflow**.

Similarly to configuring settings on a device, for example a smartphone, in which the general settings are used by various apps that might be installed; the Data Science phase can be considered as the initial configuration stages of your Workflow.

While running the **Data Science** phase in Pulse, you are:

- Configuring and validating all the [concepts](#) that your project needs to make good decisions in real-time. For example, configuring [Data Sources](#) and use them to train Models, transform incoming data into new features for your Model input with Plans.
- Use of Pulse as a unified set of Data Science tools that fully supports your Data Science Loop, while keeping your focus on the ultimate goal of using this work to perform live decisions in runtime. Manipulate huge volumes of data, explore it efficiently and evaluate all your work using different tools (e.g. [Feature Importance](#)). This allows you to be confident that when you bring your Data Science work into a live Workflow.

While running the **Runtime** phase in Pulse, you are:

- Applying the resources developed in the Data Science phase to a real-time scenario.

- Using a native resource in Pulse referred to as **Workflow** with a final goal of performing decisions in real-time.
- Defining a sequence of steps in the Workflow that will process events as the input and produce a final decision as the output.



info

There are some scenarios where you don't necessarily need to go through the Data Science phase. For example, deploying [Rules](#). In this case, you would only need to define the Rules and the input schema for your Workflow. In such scenarios, Pulse lets you jump straight into configuring these concepts in a runtime Workflow so that it doesn't cause limitations to your work.

Mapping between Data Science and Runtime

To combine the Data Science and Runtime phases, Pulse maps the resources that are developed in the Data Science stage to an element in the Workflow referred to as **Service**.

The following diagram illustrates how [concepts in Pulse](#) translate from Data Science to [Workflow](#) elements:

In general, to deploy your work in a Workflow, you will be required to:

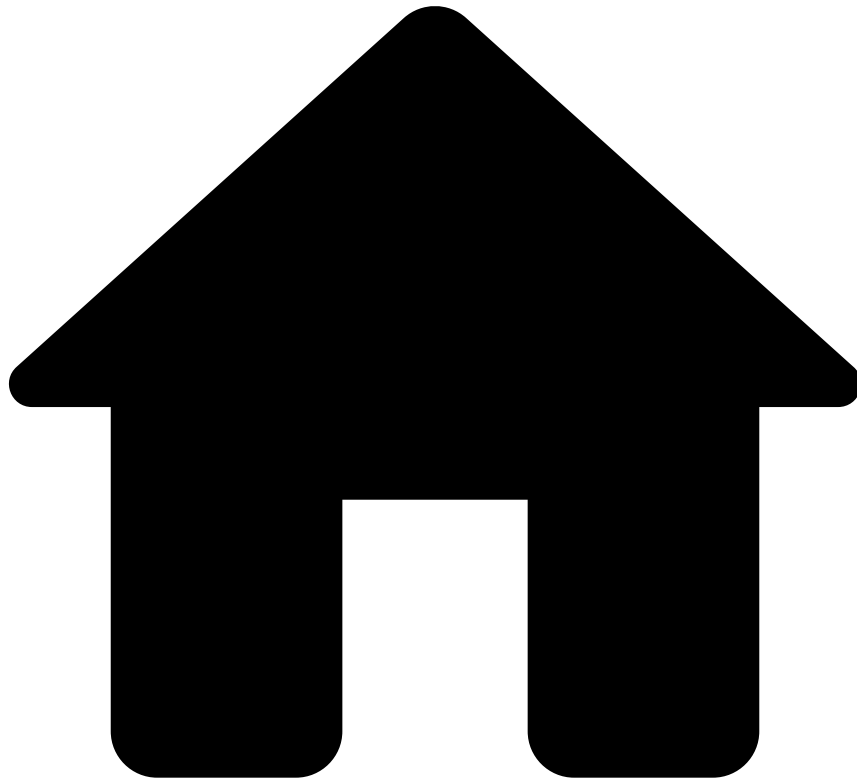
- Generate the Workflow input Schema by exporting a Data Source and copying its structure and format.
- Add runtime elements (i.e. services) to the Workflow:
 - **Plan Services:** Configure your deployable Plans and directly deploy them.
 - **Scoring Services:** Configure your Machine Learning Models and directly deploy them.
 - **Rules Services:** Configure your Rules in a Rules Project and directly deploy them.
 - **Custom Workflow Services:** To consume data, according to the input Schema from an external system, or to implement custom logic.
- Configure your Scoring Services and Rules Services to contribute to the Workflow's decisions, i.e. [Outcomes](#).

Learn More

Now that you have an overall picture of how Pulse brings **Data Science** and **Runtime** together on the same platform, you can start working on your project as explained in the following sections:

- [Creating a Data Science project](#)
- [Deploying a project into Production](#)
- [Working with Applications](#)
- [Channels](#)

•



- Project Organization
- Investigating Changes and User Actions

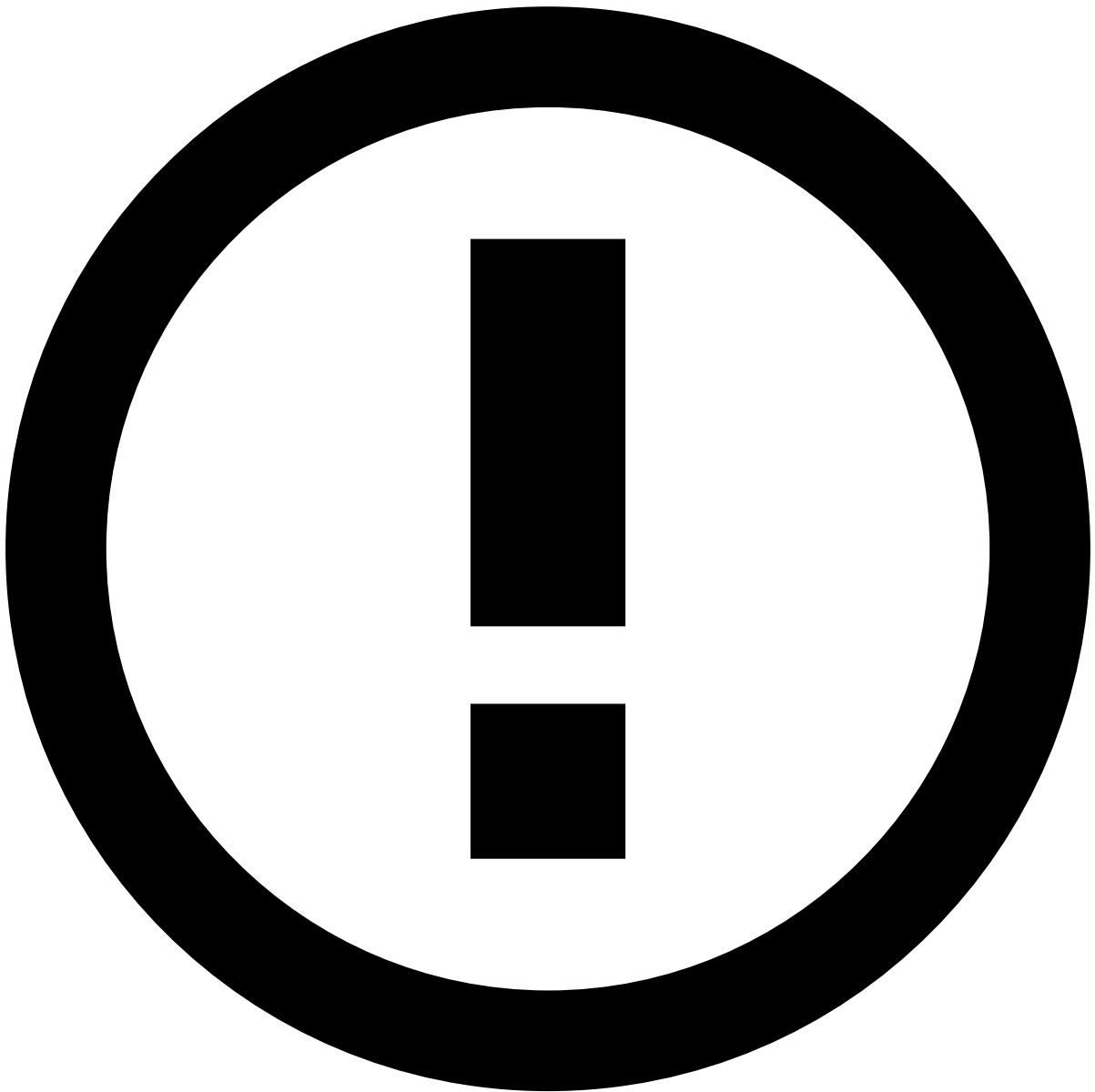
On this page

Investigating Changes and User Actions

Was this page helpful? 

Pulse provides audit logs of changes that were made via user interface or via REST API calls. This can be useful for example when investigating a change in the behavior of the system, or during a security audit.

This topic describes how to view the audit logs.



Audit capabilities

Pulse audit can track what a certain user did or edited at a certain time.

Note that more complex investigations may require the assistance of Feedzai experts. In those cases, turn to your Feedzai contact person.

Introduction

Audit entries are records of user actions and resource changes. Each audit entry contains different data, depending on the type of action or the change that was made.

Since the data contained in the audit entries covers a wide range of event types, Pulse adds **labels** to some specific data fields to make it easier to search the entries. For example, Pulse puts a label on the application name, so that users can filter entries by application name.

You can read more about how audit works and stores data in the [Audit](#) topic.

How to

Select the **Audit** option in the top right corner of the menu bar to view the list of changes made in the Pulse environment.

Each line in the displayed list is an audit entry, representing a specific user action or change.

Viewing an audit entry's details

Select the audit entry line to expand its details. The details contain two sections: **Context** and **Labels**.

The Context section shows the data that was included in the request of the Pulse UI or REST API that initiated the change.

The Labels section shows the data fields of the audit entry to which Pulse added a label.

Viewing differences for update actions

For update actions, the Context section shows a comparison between the previous and updated version:

Searching by label value

Select any label value (highlighted in blue), to filter for all audit entries containing that label.

Upon selecting a label value, the label value is shown in the search box, and the list is filtered to only show audit entries that have the same label value.

Filtering the listed audit entries

Use the **built-in filters** to filter the audit entries by time **Period** , by the type of **Action** , or by **User** .



info

Remember that changes made to resources may not always be automatically taken into consideration by other parts of the application.

For example, if you change a rule, it is not automatically updated in a workflow. Workflows use a snapshot of the rules project, which include a certain version of the rule. Also, the application needs to be published after the changes, for the changes to be active.

Examples

Identifying changes made to an application on a specific day

To find all the changes made on a specific day of the last two months, perform the following steps:

1. In Pulse, navigate to the **Audit** page.
2. In the **Period** filter, select *Last 2 months*.
3. Type the name of the application in the search box, and select **Search**.

4. Use the web browser's search to find the entries of the day you are looking for. Use the format YYYY-MM-DD.

Identifying actions taken by a specific user

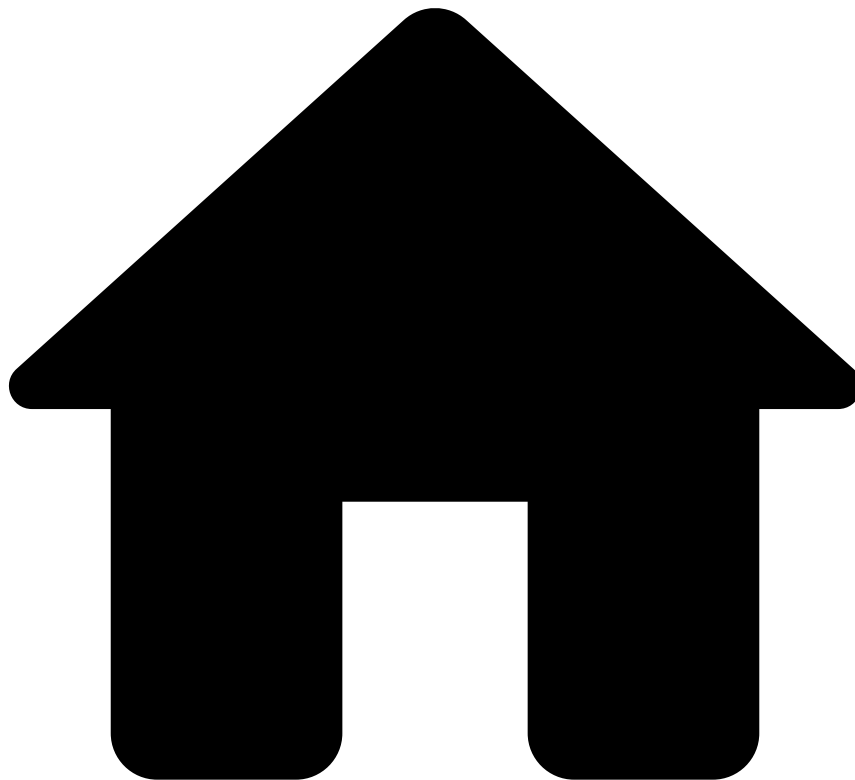
To find all the changes made by a specific user in the last two months, perform the following steps:

1. In Pulse, navigate to the **Audit** page.
2. In the **Period** filter, select *Last 2 months*.
3. Use the **User** filter to find changes and actions performed by the specific user in the last two months.

Learn more

Read [how to compare rules in two different snapshots](#).

-



- Model Guide

On this page

Building and evaluating your model

Was this page helpful? 

Pulse supports your Data Science loop by enabling you to quickly train your models and evaluate the results over several iterations.

A model project aggregates several iterations of a model and provides tools for you to assess whether the features you designed are useful, well constructed, etc. At each iteration, you are able to understand how close you are to reaching your projects goals and what actions you took to reach those results.

Building and evaluating your models with Pulse involves:

- Training a model
- Evaluating a model
- Tuning model parameters
- Comparing models

Edit model evaluation page

The **Edit Model Evaluation** page allows you to configure how you look at the results of your model and how you can evaluate them against models in your project.

The following image displays the **Edit Model Evaluation** page:

The **Edit Model Evaluation** page contains the following areas:

1. Evaluation target and comparison

Defines the target value and model comparisons for your model evaluation. Select the **Target Value** you want to analyze in your model evaluation. This allows for the evaluation of any value of your model's target variable. For example, both **Fraud** and **Not Fraud**. Select the models you want to **Compare With** to have their evaluations plotted on the ROC Curve area and displayed in the Metrics table.

1. ROC curve

Graphically displays the typical evaluation metrics of Machine Learning models, such as Recall and FPR.

Adjust the **X-Axis Range** and **Y-Axis Range** sliders to zoom in and out of your model's results. Select the **X-Axis** and **Y-Axis** metrics to generate a custom chart in the **X/Y Plot**. Any count and cost-based metrics are available.

1. Metrics table

Displays the results plotted in the **ROC Curve** area in a tabular manner. Includes all metrics for all possible combinations of the configured breakdowns for the evaluation.

Adjust the **Threshold** slider to define the confidence you have in the model. This means that for example, all Data Source instances above the threshold are evaluated as Fraud. Select the **View ROC** option in a row to display the graphical evaluation in the **ROC Curve** area for the selected breakdown.

Evaluation Metrics

Select Evaluation Metrics from the options menu to define custom metrics. For example, calculating your profit in using your model to automatically detect fraud vs. what you would have spent doing a manual revision. You can then display these custom metrics in the **ROC Curve** and **Metrics** table.

Comparison Details

It is common that several models are trained and configured mostly identically. Select the Comparison Details option to quickly examine the difference between models you are comparing. Changes are highlighted in a different color.

Exploration page

The **Exploration** page is designed to assist you in fine tuning your [model](#). It lets you:

- Check the performance of your models at the defined thresholds.
- Assess how close is the model to your success/acceptance criteria, without breaking client SLAs.
- Detect possible cost-efficiencies when you make very small adjustments.

Select the option from the **Model Project** page.

The following image displays the **Exploration** page:

The **Exploration** page contains the following areas:

1. Targets

Displays comparative analyses of your evaluations under different criteria. For example, determines, for each evaluation, which one performs the best, i.e. captures the most fraud money, when you set the FPR at approximately 1%.

Displays the threshold and selected metrics for which the configured criteria reach their best performance.

Use the option to configure the criteria you will use to explore a new target.

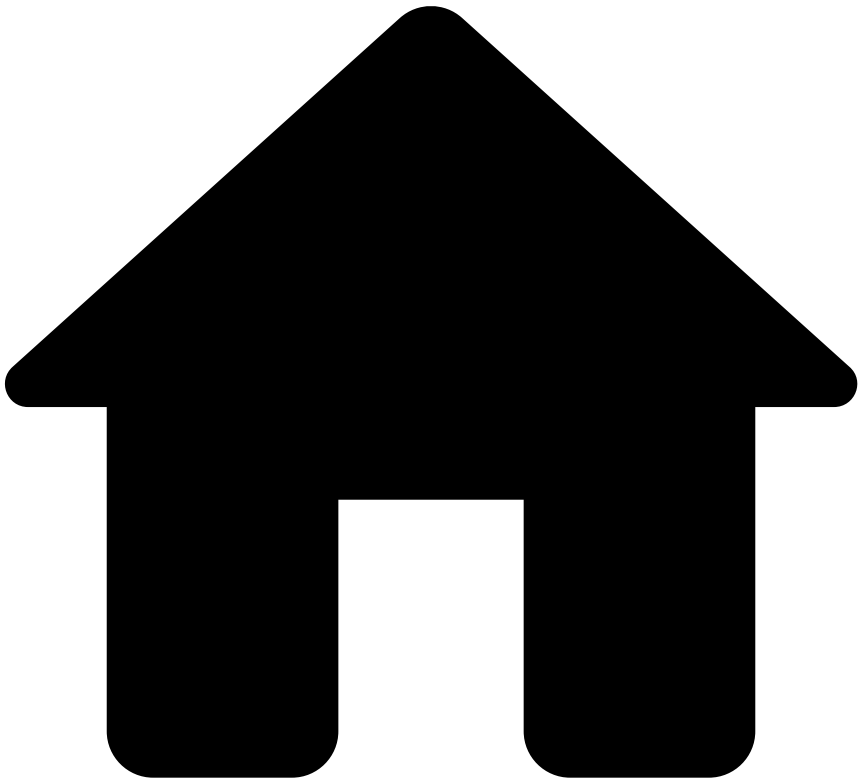
1. Evaluations

Lists all existing evaluations in your model project, both resulting from single model evaluations and hyperparameter tuning. Use the search filter to navigate through these two evaluation types and use checkboxes to select which **Evaluations** should be displayed for each item in the **Targets** area.

Getting Started

To start working on your models, learn how you can train a Model and/or perform a Hyperparameter Tuning.

-



- Navigating Pulse

On this page

Navigating Pulse

Was this page helpful? 

This reference will help you understand how your Pulse application works and all the sections, pages and information it offers. It serves as a tool to understand how Pulse works quickly, but also as a source for consulting whenever you are not sure about the functionalities and options that Pulse offers.

Toolbar

On the top of the screen, the toolbar displays the following information and offers the following options:

Icon	Functionality	Observation
	Open JupyterLab IDE	Opens JupyterLab, if the integration is enabled and the current computer is set up for it.

Icon	Functionality	Observation
	Settings	
	Audit	
	Help	Opens the lateral help panel .
	Administration	Opens the Administration page of Pulse, where you can perform administrative tasks like configuring clusters, security settings and multi-tenancy, viewing server logs, and so on.
	Log out	

Applications

When you log in to Pulse, the first screen you see is the list of applications you have in your environment.

Application-level options

Once you select an application, you can find the following application-level options using the **Options** cogwheel menu located in the top-right corner, below the logout button:

Let's see each of these options in more details.

Editing the application

You can **Edit** the name and the description of the application.

In addition to the name and description, the following options are also available:

- **Auto-Healing**: When toggled, it monitors the state of the cluster and decides whether it needs to be repaired. If the application is in an invalid state, Pulse automatically triggers repair actions on the respective components.
- **Lock Simple Rules Projects**: When toggled, Pulse stops all modifications to the Simple Rules Projects, depriving fraud analysts in Case Manager from adding any new rules to Simple Rules Projects.

Evaluation metrics

Under **Evaluation Metrics**, you can add custom metrics to the application and use them when evaluating a model or rules. This allows you to inspect their performance based on their business value.

Export and import options

With the **Export Application** option, you can download the application in a .papp file. You can choose to export the application using the legacy version. However, it is recommended to stick with the newer implementation since it's more efficient.

With the **Export Resources** and **Import Resources** options, you can download/upload resources (features, rules, models, etc.) to transfer them from one Pulse environment to another.

Application sidebar

Once you open an application, you can navigate through its areas using the available tabs:

Icon	Tab
	Data Configuration
	Data Science
	Workflows
	Lists
	Jobs
	Backfills

Icon	Tab
	History

Data Configuration tab

In the **Data Configuration** tab, you can manage [schemas](#), [custom services](#) and [outcome combinators](#).

You can search the list of resources or add a new resource. Also, you can duplicate , delete or configure these resources.

Schemas

In the **Schemas** section, you can see the list of schemas created. You can also enter the schemas creation screen:

In here, you can see:

1. **Schema definition:** The necessary fields to create a new schema.
2. **Advanced options:** The advanced options will show when you click on the dropdown button.
3. **Actions:** The buttons to save (and create one after it, if needed) and cancel the creation of the schema.

Data Science tab

In the **Data Science** tab, you can create an AutoML run, or manage data sources, plans, model projects, rules projects and sandbox evaluations.

You can search the list of resources or add a new resource. Also, you can duplicate , delete or configure these resources.

AutoML

When entering the AutoML run creation, there are two screens in which you can configure it.

1. **Data Source and Basic Settings:** Define the data source for the AutoML run, add a semantic file for tagging, and define some basic settings for the models.
2. **Training Configuration:** Define how the data will be split, configure which metric you want to optimize, and set advanced options for the model training.

Data Sources

When entering the **Data Source** creation, there are three screens in which you can configure it.

1. **Data Source Selection:** Define the source of the Data used.
2. **Data Source Preview:** Preview the first lines of the data.
3. **Data Source Characterization:** Configure the fields of the Data Source.

Plans

In the **Plans** section, you can see the list of plans created. By adding a new plan, you can enter the plan creation screen:

1. **Plan definition:** The general fields about that Plans's information: its name and whether it is deployable.
2. **Advanced options:** The advanced options on the dropdown button.
3. **Actions:** The buttons to save (and create one after it in the dropdown arrow) and cancel the creation of the plan.

You can also copy [plan operators](#) to the clipboard in order to paste them in other plans.

Model Projects

In the **Model Projects** section, you can see the list of models created. You can also add a new model project:

1. **Model definition:** The name field of that model.
2. **Advanced options:** The advanced options on the dropdown button.
3. **Actions:** The buttons to save (and create one after it in the dropdown arrow) and cancel the creation of the plan.

Rules Projects

In the **Rules** section, you can see the list of rules created. You can add a new rules project and enter the rules creation screen:

1. **Rules Options:** The basic fields of the plan: the name, origin schema, actions and multi-tenancy configuration.
2. **Advanced options:** The advanced options on the dropdown button.
3. **Actions:** The buttons to save (and create one after it in the dropdown arrow) and cancel the creation of the rule.

You can also copy rules and templates to the clipboard in order to paste them in other rules projects.

Sandbox Evaluations

When entering the screen to create a **Sandbox Evaluation**, there are three screens in which you can configure it.

1. **Data source:** Define the Data Source and outcome variable.
2. **Resources:** Specify the resources and the Outcome Combinator.
3. **Reporting:** Define the configuration for reporting.

Workflows tab

In the **Workflows** tab, you can manage workflows.

You can search the list of workflows or add a new workflow. Also, you can delete workflows.

Workflows

When adding a workflow, the screen shows two tabs: the configuration of the **Workflow** and the **Advanced** configuration.

The **Workflows** tab allows you to configure:

1. **General fields:** Name and Origin schema.
2. **Workflow design:** The services and diagram.
3. **Actions:** The buttons to save (and create one after it in the dropdown arrow) and cancel the creation of the plan.

The **Advanced** tab allows you to configure:

1. **Actions configuration:** The actions and listeners.
2. **Event storage and recover.**
3. **Partition Keys definition.**
4. **Properties definitions.**
5. **Advanced options:** The advanced options on the dropdown button.
6. **Actions:** The buttons to save (and create one after it in the dropdown arrow) and cancel the creation of the rule.

Lists tab

In the **Lists** tab, you can see all of the lists created.

You can search for a list or add a new one. Also, you can delete or configure these resources. In addition, you can see if a list is simple or complex .

Lists

When adding a new list, the following screen appears, in which you will be able to specify:

1. **List definition:** The general information of that list.
2. **Key Type:** whether string or number.
3. **Advanced Options:** The advanced options through a dropdown button.
4. **Actions:** The buttons to save (and create one after it in the dropdown arrow) and cancel the creation of the rule.

Jobs tab

In the **Jobs** tab, you can see job executions, registered jobs and job configuration profiles:

1. **Jobs executions:** List of jobs that have been executed.

2. **Registered Jobs:** List of jobs scheduled to run as configured in the scheduled metrics jobs.
3. **Job Configuration Profiles:** List of profiles including the Pulse Properties Profile which has the configurations that are loaded from a source file (e.g. pulse.properties), and Job Profiles with custom configurations.

You can also the list of jobs and you can use the button configure defaults to associate a profile to a resource type.

Backfills tab

In the **Backfills** tab, you can see the list of backfill jobs and manually manage the backfill jobs queue:

Backfills Management Area

In the **Trigger Backfill** section, you can configure and manually trigger a new backfill job.

In the **Triggered Jobs History** table you can review the list of backfill jobs, including their status, start and end time, trigger method, and the affected metrics. You can also cancel a backfill job that is in progress.

History tab

In the **History** tab, you can see the history of your resources and interact with it.

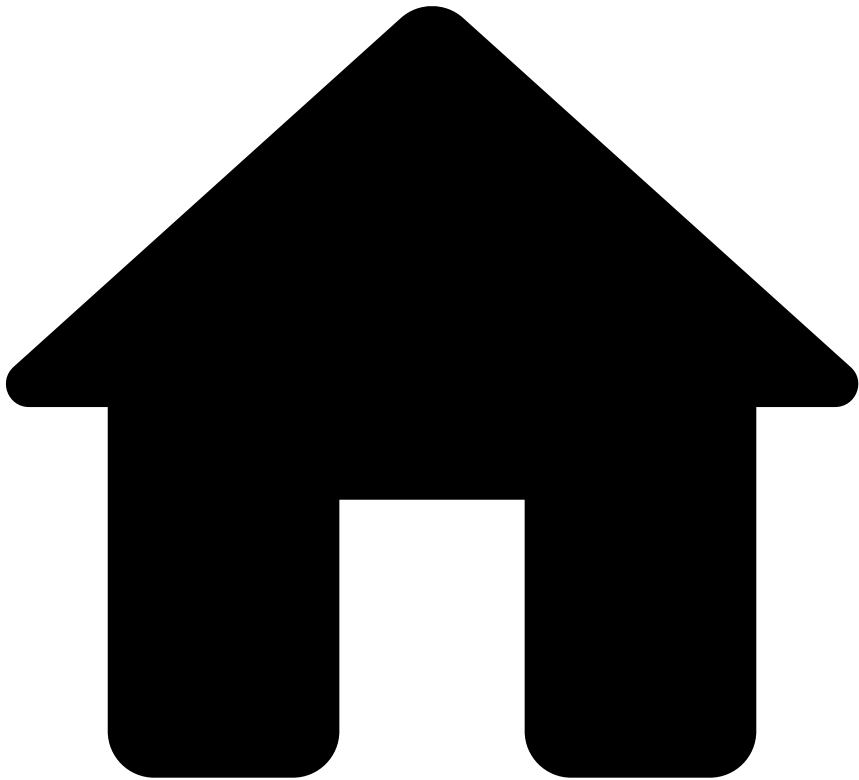
You can see the associated operations or revert that change.

Help panel

On the right side of the screen in Pulse's interface, you can open a lateral panel giving further information on the actions you can perform on the screen that is shown. This panel helps you understand the concepts in each of the interface's tabs and links to a complete online documentation regarding that matter.

To open this panel, select the **Help** button in the toolbar.

-



- Rule Guide

On this page
Was this page helpful? 

Rule Guide

This guide helps you write and maintain rules in Pulse. It also gives a short overview of how rules are deployed from the data science environment to production.

The following topics are not in the scope of this guide:

- Self-service rules
- Stateful rules

Prerequisites

This guide presumes the following:

- The rules will be created in a data science environment by a data scientist or analyst who is familiarized with the [basic concepts of Pulse](#).
- The data science environment has [cleaned historical data](#) that represents well the real-time data.
- The data science environment has job profiles set up for resource-intensive plan executions. Contact Feedzai if you need help with this.
- In the case of writing rules for an existing production environment, the data science environment is in the same state as the production environment.

This means that rules, models, plans and lists are identical in the two environments.

The objective of rules

Pulse rules can be used to control what happens with the transactions that have been previously scored by machine learning models. They can also be used to directly score and act upon events, a common case for starting projects with no available data yet to train models. Here business and compliance rules can be put in place to fight fraud.

In summary, with Pulse rules you can:

- control what happens with a transaction that falls into a score threshold;
- reflect business restrictions;
- implement compliance rules;
- react on newly found fraud patterns by implementing and testing a rule, without training a new model right ahead. This can be advantageous when a new fraud pattern is found and there is not enough historical data with that pattern to train a model.

Rule life cycle

When setting up Pulse to process events, rules are put in place to reflect business rules, compliance rules, and other preventive measurements.

Besides creating the rules when setting up Pulse for the first time, you may need to add new rules to your environment later on as well.

This happens when you detect new financial crime trends, or when there is a new compliance or business rule in your organization.

The following image shows the life cycle of rules that are created in Pulse. Note that self-service rules created in Case Manager are not included in this life cycle, as they follow a completely different creation process.

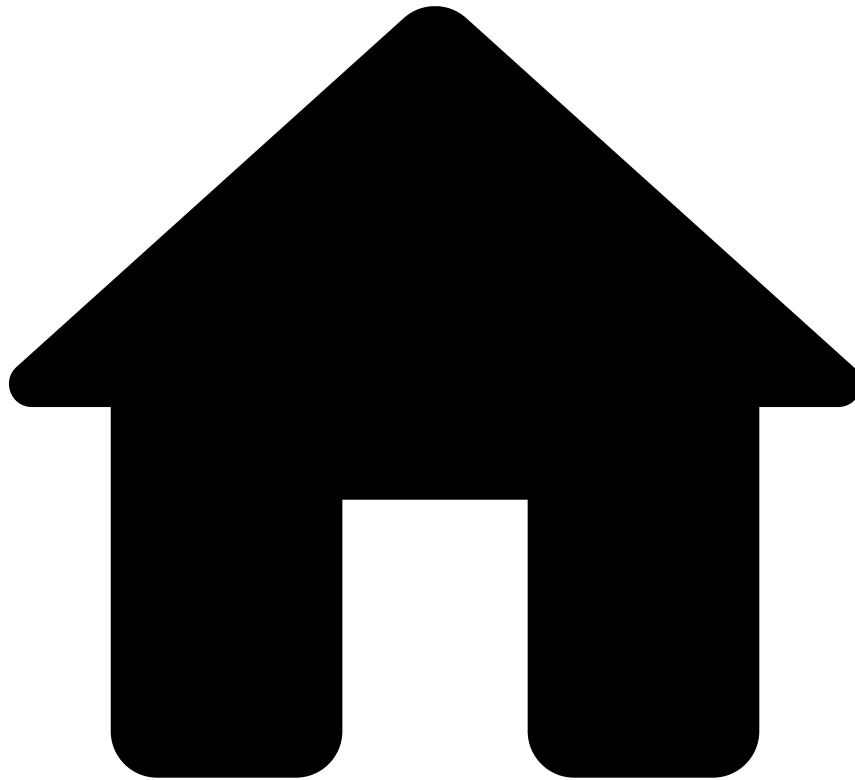
The rule life cycle has the following stages:

1. [Creating necessary lists](#): if the rules you want to create use lists, those need to be created before writing the rules.
2. [Creating necessary features](#): analyze whether the fields that the rule needs are available in the data schema. If not, create the necessary features in a plan of Pulse, and run the plan to create the new fields in a new data source.
3. [Creating or updating the rules project](#): when creating the first rules of a new Pulse setup, you first create a rules project that will group them. Rules projects are bound to the data schema. Hence, when adding rules to an existing set of rules, you have to update the data schema of the rules project if you have added new features as well. This will ensure that the new fields are available in the rules.
4. [Creating or editing the rules](#).
5. [Creating a rule snapshot](#), and [evaluating rule performance using the snapshot](#). A rule snapshot is a "photo" of the current rule configurations. It can be deployed and used later on in production. In addition, it can contain an evaluation of the included rules, which lets you assess the rule performance. The evaluation shows metrics like FPR and Recall for the rules. You can also export individual rule contribution during the evaluation, for further analysis.

6. [Creating a sandbox evaluation](#), and [evaluating rule performance in sandbox](#). To evaluate decisions on transaction level, create a sandbox evaluation. Sandbox evaluations allow you to see the outcome of the rules project for each transaction.
7. [Rule review](#). To ensure that you made the right changes and created the right rules, you can request a review from a peer.
8. [Deploying the rules to the production environment](#): when the new set of rules matches your expectations, you export the created entities from the data science environment. Engineers of the production environment then import the exported entities to the production environment.
9. [Monitoring rules](#): Feedzai Insights provides customizable dashboards to monitor the performance of rules.

To understand the basic concepts (rules, rule snapshots, plans, workflows, outcomes, etc.), check the [glossary](#).

•



- Signing in to Pulse
- About Signing in to Pulse

On this page

About Signing in to Pulse

Was this page helpful? 

Pulse provides a default sign in form in which you can enter your Username and Password to start a Pulse session. This Username and Password are associated with a Pulse User account that can be configured with a number of settings such as Pulse Groups, Pulse Roles, Tenants, among others.

There are two key functionalities in Pulse that impact the sign in process namely:

- **Two-Factor Authentication (2FA):** Users are requested to sign in with two pieces of information to verify user identity. In this case, the user is prompted with a sign in form to enter its credentials (i.e. user account and password), followed by a second separate step in which the user has to provide a security code.
- **Single Sign-On (SSO):** Users are created in Pulse by the concept of user provisioning and the sign in process include two different scenarios: Sign in initiated via an Identity Provider and Sign in initiated via a Service Provider.

Learn More

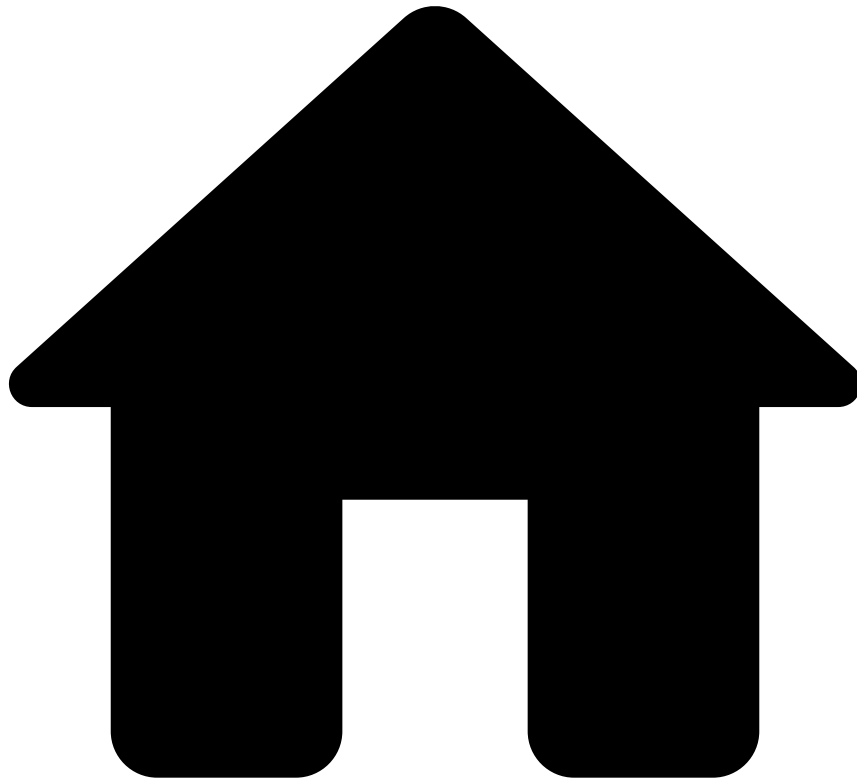
Learn more about User sign in with **2FA**:

- [Signing in with Two-Factor Authentication](#)

Learn more about User sign in with **SSO**:

- [Signing in with Single Sign-On](#)

-



- Working with Applications

On this page

Working with Applications

Was this page helpful? 

A Pulse application (**Pulse app**) is an interface that hosts resources developed by various end-users. With **Pulse apps**, Data Scientists, Software Engineers, and Fraud Analysts are able to collaborate on developing a Fraud Prevention solution. This concept is referred to as [App Collaboration](#).

Pulse supports easy-to-use tools that allow users to develop resources in separate offline environments, and then combine the work in a production environment (see [App Collaboration tools](#)).

Pulse Applications

Similarly to software apps which are interactive programs that allow users to perform tasks, a **Pulse app** is an interface that hosts resources developed by various end-users to perform certain tasks.

The ultimate goal of a **Pulse app** is to support the configuration of resources that work together to solve a well defined objective, e.g. detect fraud. To achieve this, **Pulse apps** are used to configure [Data Sources](#), and develop [Plans](#), [Models](#), [Rules](#), among other functionalities. These resources are then configured on [Workflows](#), also hosted by **Pulse apps**, to process transactions in real-time.

Pulse resources are not bound to the host application. This allows multiple teams to develop resources in parallel using different environments. This concept is referred to as **App Collaboration** and is explained in more detail in the next subsection.

App Collaboration

Developing a fraud-prevention application requires multiple teams to work together:

- **Data Scientists:** Develop [Metrics](#) and **Models** using Data Science environments.
- **Delivery teams:** Configure **Schema** and **Workflows** using Development environments.
- **Engineering teams:** Write advanced **Rules** and populate [Lists](#) using Development environments.
- **Fraud Analysts:** Write Self-service Rules (**SSR**) and populate **Lists** directly in Production.

Using the **App Collaboration** concept, each team evolves resources in parallel using **Pulse apps** configured on various offline environments. The changes are then transferred and combined in a trustful way with the resources that are in Production. The image below illustrates the idea of **App Collaboration**:

The flow illustrated in the diagram above is not a one-time process. As fraud patterns evolve, and you gain new knowledge on your data, there is a need to re-engineer new features, retrain **Models** and write new **Rules**. These needs require a constant integration between work developed in offline environments with work that is in Production. Therefore, Feedzai recommends a working methodology that is explained in the page [App Collaboration Loop](#) under the Operations Guide. This working methodology covers the following key points:

- How to structure an application in different environments.
- Workflow between environments.

App Collaboration tools

To make **App Collaboration** easy-to-use, Pulse provides a set of tools that allows you to migrate selected resources or the full **Pulse app** (i.e. PAPP file) between environments. The export/import tools foster the following:

- **Merge resources:** Multiple teams working simultaneously on different resources can merge resources using Pulse UI.
- **Standard working method:** You can easily manage a standard procedure to transfer resources to Production using **App Collaboration Loop**.
- **Frequent updates:** Pulse services configured in runtime can be frequently updated in production because deployments are fast and easy to track.

Migrating Resources and Pulse apps

Pulse allows migrating selected resources or entire **Pulse apps**. The decision to use one or the other depends on the type of changes that you want to migrate or the procedure that is in place for each project. Check the page [App Collaboration Loop](#) for more information about the recommended procedure.

Export and import Resources

Pulse allows exporting and importing the following resources:

- **Rules Projects** with or without snapshots.
- **Lists** metadata.
- **Models** with or without the binary.
- **Plans** with or without executions.

Check the pages [Exporting Resources](#) and [Importing Resources](#) to learn how to perform these operations.

Export and Import Pulse apps

Pulse allows exporting and importing the entire **Pulse app** through the UI. When updating a **Pulse app** in a live production environment, you should adopt the blue-green method to perform the update with zero downtime.

Check the page [Exporting and Importing Applications](#) to learn how to perform these operations.

Application changes in runtime

In Pulse, you can make changes on your **Workflow**, even when is processing transactions. These changes, however, are not immediately applied to the running application. Instead, they are only applied when you explicitly **Publish** the application.

Check the page [Publishing Application changes](#) to learn how to make your changes effective in the runtime **workflow**.

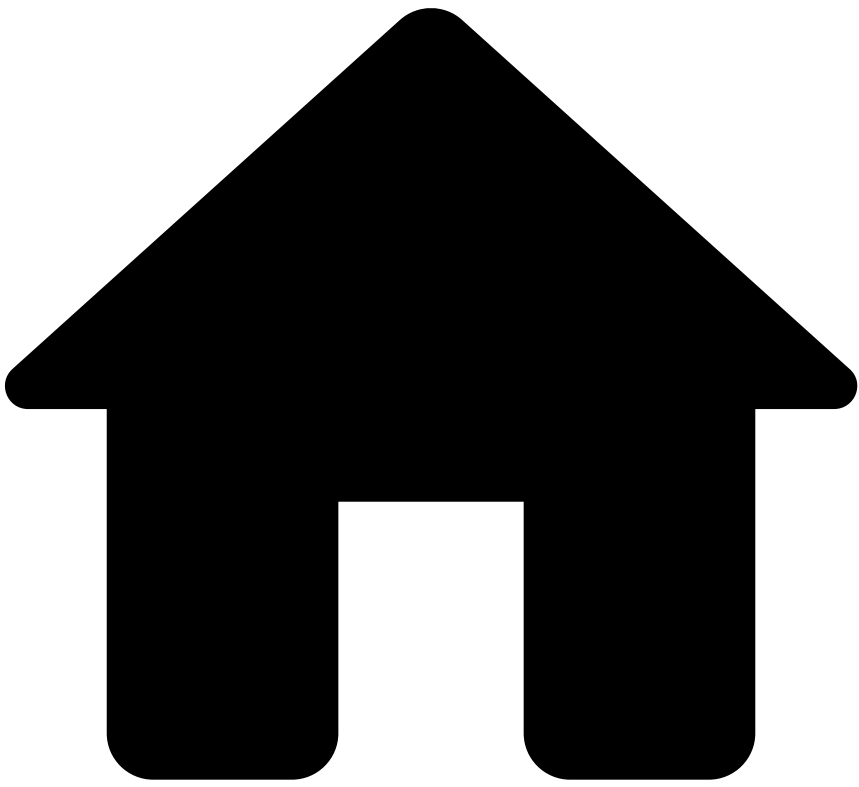
Learn more

The following pages allow a better understanding of how a **Pulse App** works:

- [The Application Life Cycle](#)
- [Publishing Application Changes](#)
- [Exporting Resources](#)
- [Importing Resources](#)
- [Exporting and Importing Applications](#)
- [Resource Versioning](#)
- [Automatic Application Repair](#)

Learn how to evolve a **Pulse app** from end-to-end by checking [App Collaboration Loop](#).

-



- Securing an Application
- Creating and Assigning Roles

On this page

Was this page helpful? 

Creating and Assigning Roles

A [role](#) is a set of [permissions](#) on Pulse resources (such as applications, models, rules, etc.). You can reuse roles across multiple users or groups for a centralized control and consistency.

The following sections describe how to create and use a role in Pulse.

Creating a role

To create a new role, complete the following steps:

1. Select the button in the menu bar to navigate to the **Administration** page.
2. Open the **Security** tab.

3. In the **Roles** section, add a new role.
4. Enter a **Name** for the role.
5. You can add **Users** to the role now, or return to this form later to add users.
6. In the **Permissions** field, select the [permissions](#) that this role will grant to users.
7. Use the **Save** button to create the role. If you want to create another role, you can use the button next to the **Save** button and select **Save and New**.

You can find examples of typical roles in the [Typical permission setups](#).

Assigning a role to a user

To assign a role to a user directly, edit the role (see [below](#)) and add the user in the **Users** field.

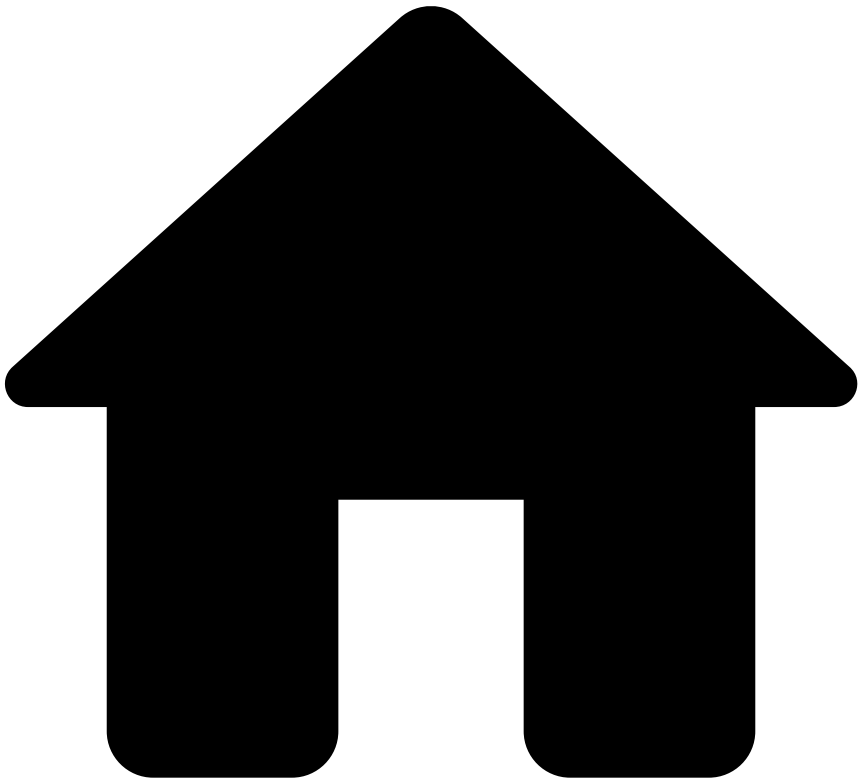
Alternatively, [edit the user](#) and add the new role in the **Roles** field.

Editing a role

To edit a role, complete the following steps:

1. Select the button in the menu bar to navigate to the **Administration** page.
2. Open the **Security** tab.
3. In the **Roles** section, select the role that you want to edit.
4. You can edit the list of **Users** that have this role, or the [Permissions](#) that the role grants to users.
5. Use the **Save** button to save the role.

-



- Securing an Application
- Creating and Using Groups

On this page

Was this page helpful? 

Creating and Using Groups

When Pulse users create a resource (rules project, plan, model project, data source, etc.) they can decide to make it visible only to certain groups.

For example, if a rules project is restricted to be visible only to a group, users of that group can view and edit that rules project if they have a role that allows them to view and edit rules projects.

The following sections explain how to create a group, how to add members to it, and how to set the visibility of a resource.

Creating groups¹

To create a new group, complete the following steps:

1. Select the button in the menu bar to navigate to the **Administration** page.
2. Open the **Security** tab.
3. In the **Groups** section, select *Hierarchical Groups*.
4. In the group tree, select the add child button to add a child group to an existing group.
5. Enter a **Name** for the group.
6. In the **Group Properties** field, you can add properties to the group in the form of key-value pairs.

Adding users to a group²

To add a user to a group, complete the following steps:

1. Select the button in the menu bar to navigate to the **Administration** page.
2. Open the **Security** tab.
3. In the **Users** section, select the user.
4. In the **Groups** field, add the group.
5. **Save** the user.

Using groups to limit visibility³

To limit the visibility of a resource, complete the following steps:

1. Open the resource and use the button to edit the visibility options.
2. In the Visibility area, select the **Specific Groups** option.
3. Select the groups to which the resource should be visible.
4. For each group, select the actions (*Read*, *Update*, *Delete*) that can be performed on the resource.

Note that users will be able to perform the selected actions only if an administrator assigned them a role that allows performing that action.

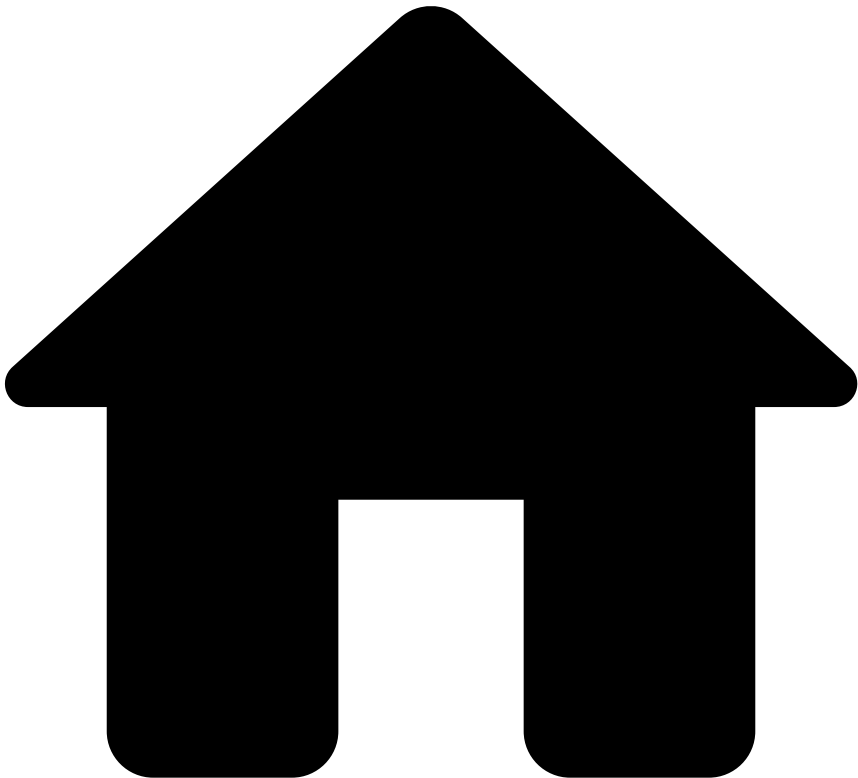
Example⁴

Consider the following scenario.

- Kate and John belong to the *data-scientist* group.
- John has a role that grants him Read permissions on model projects, but doesn't have any role with the *Update* permission.
- Kate creates a model project and sets it visible to the *data-scientists* group, allowing group members to use their *Read* and *Update* permissions.

In this case, John can see the model project, but he cannot update or delete it.

•



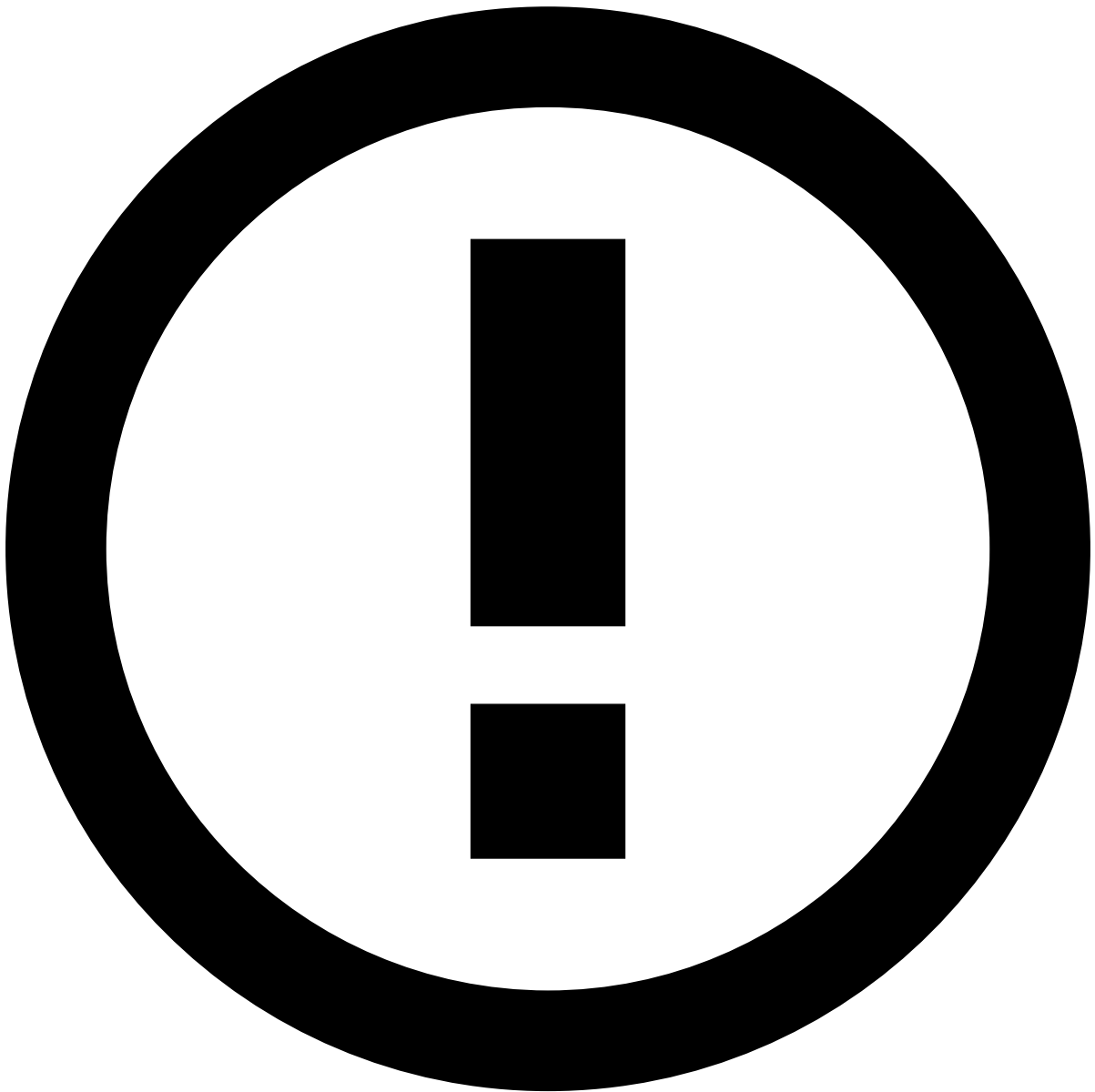
- Securing an Application
- List of Permissions

On this page

Was this page helpful? 

List of Permissions

The following sections list the permissions that are available to control access to resources of Pulse.



Resource dependencies

Resources sometimes have dependencies between them, hence to access one, you might need to have access to another. These dependencies are not always obvious. If you are having difficulties with resource permissions, contact Feedzai's technical support.

Global🔑🔑

Name	Description
*	Permission that gives complete access to Pulse.

Administration🔑🔑

Name	Description
admin:logs:read	Permission that grants access to the logs.

Name	Description
admin:cluster:manage	Permission that grants management access to the cluster.
admin:cluster:read	Permission that grants read access to the cluster.
admin:configuration:delete	Permission to delete configurations on the administration screens.
admin:configuration:read	Permission to read the configurations on the administration screens.
admin:configuration:update	Permission to update configurations on the administration screens.
admin:deploy	Permission to deploy services through the REST API.
admin:security:reload	Permission to reload security through REST API.
admin:service:control	Permission to start and stop services via the Administration API.
admin:view	Permission to access the administration screens.
admin:usermanagement	Permission to manage users.

Application

Management

Name	Description
application:*	Permission to perform all CRUD actions in application models.
application:*.read	Permission to read applications.
application:apps:create	Permission to create new applications.
application:apps:delete	Permission to delete applications.
application:apps:manage	Permission to start/stop/publish/export applications.
application:apps:partialExport	Permission to partially export applications.
application:apps:partialImport	Permission to partially import applications.
application:apps:read	Permission to read applications.
application:apps:update	Permission to update applications.

Data science

Data sources

Name	Description
application:datascience_datasources:create	Permission to create data science data sources.
application:datascience_datasources:delete	Permission to delete data science data sources.
application:datascience_datasources:read	Permission to read data science data sources.
application:datascience_datasources:update	Permission to update data science data sources.

Evaluation metrics

Name	Description
application:datascience_evaluationmetrics:create	Permission to create data science evaluation metrics.
application:datascience_evaluationmetrics:delete	Permission to delete data science evaluation metrics.
application:datascience_evaluationmetrics:read	Permission to read data science evaluation metrics.
application:datascience_evaluationmetrics:update	Permission to update data science evaluation metrics.

Model projects

Name	Description
application:datascience_modelprojects:create	Permission to create data science model projects.
application:datascience_modelprojects:delete	Permission to delete data science model projects.
application:datascience_modelprojects:read	Permission to read data science model projects.
application:datascience_modelprojects:update	Permission to update data science model projects.
application:datascience_modelprojects_evaluations:create	Permission to create data science model evaluations.
application:datascience_modelprojects_evaluations:delete	Permission to delete data science model evaluations.
application:datascience_modelprojects_evaluations:read	Permission to read data science model evaluations.
application:datascience_modelprojects_evaluations:update	Permission to update data science model evaluations.
application:datascience_modelprojects_featureimportance:create	Permission to create feature importance.
application:datascience_modelprojects_featureimportance:delete	Permission to delete feature importance.
application:datascience_modelprojects_featureimportance:read	Permission to read feature importance.
application:datascience_modelprojects_featureimportance:update	Permission to update feature importance.
application:datascience_modelprojects_hyperparam_tuning:create	Permission to create hyperparameter tuning.
application:datascience_modelprojects_hyperparam_tuning:delete	Permission to delete hyperparameter tuning.
application:datascience_modelprojects_hyperparam_tuning:read	Permission to read hyperparameter tuning.
application:datascience_modelprojects_hyperparam_tuning:update	Permission to update hyperparameter tuning.
application:datascience_modelprojects_hyperparam_tuning_eval:create	Permission to create hyperparameter tuning evals.
application:datascience_modelprojects_hyperparam_tuning_eval:delete	Permission to delete hyperparameter tuning evals.
application:datascience_modelprojects_hyperparam_tuning_eval:read	Permission to read hyperparameter tuning evals.
application:datascience_modelprojects_hyperparam_tuning_eval:update	Permission to update hyperparameter tuning evals.
application:datascience_modelprojects_models:create	Permission to create Data Science models.
application:datascience_modelprojects_models:delete	Permission to delete Data Science models.
application:datascience_modelprojects_models:read	Permission to read Data Science models.
application:datascience_modelprojects_models:update	Permission to update Data Science models.
application:datascience_modelprojects_targets:create	Permission to create Data Science targets.
application:datascience_modelprojects_targets:delete	Permission to delete Data Science targets.

Name	Description
application:datascience_modelprojects_targets:read	Permission to read Data Science targets.
application:datascience_modelprojects_targets:update	Permission to update Data Science targets.

Plans

Name	Description
application:datascience_plans:create	Permission to create Data Science Plans.
application:datascience_plans:delete	Permission to delete Data Science Plans.
application:datascience_plans:read	Permission to read Data Science Plans.
application:datascience_plans:run	Permission to run Data Science Plans.
application:datascience_plans:update	Permission to update Data Science Plans.
application:datascience_plans_executions:create	Permission to create Data Science Plan Executions.
application:datascience_plans_executions:delete	Permission to delete Data Science Plans Executions.
application:datascience_plans_executions:read	Permission to read Data Science Plans Executions.
application:datascience_plans_executions:update	Permission to update Data Science Plans Executions.

Rules projects

Name	Description
application:datascience_rulesprojects:create	Permission to create data science rules projects.
application:datascience_rulesprojects:delete	Permission to delete data science rules projects.
application:datascience_rulesprojects:read	Permission to read data science rules projects.
application:datascience_rulesprojects:self-service-publish	Permission to publish self-service rules.
application:datascience_rulesprojects:update	Permission to update data science rules projects.
application:datascience_rulesprojects_environments:create	Permission to create data science rules project evaluations.
application:datascience_rulesprojects_environments:delete	Permission to delete data science rules project evaluations.
application:datascience_rulesprojects_environments:read	Permission to read data science rules project evaluations.
application:datascience_rulesprojects_environments:update	Permission to update data science rules project evaluations.
application:datascience_rulesprojects_evaluations:create	Permission to create data science rules project evaluations.
application:datascience_rulesprojects_evaluations:delete	Permission to delete data science rules project evaluations.
application:datascience_rulesprojects_evaluations:read	Permission to read data science rules project evaluations.
application:datascience_rulesprojects_evaluations:update	Permission to update data science rules project evaluations.
application:datascience_rulesprojects_rules:create	Permission to create Data Science Rules that are not based from templates.

Name	Description
<code>application:datascience_rulesprojects_rules:create:fromAst</code>	Permission to create self-service rules.
<code>application:datascience_rulesprojects_rules:create:fromTemplate</code>	Permission to create data science rules from templates.
<code>application:datascience_rulesprojects_rules:delete</code>	Permission to delete data science rules.
<code>application:datascience_rulesprojects_rules:read</code>	Permission to read data science rules.
<code>application:datascience_rulesprojects_rules:update</code>	Permission to update data science rules that are not based on templates.
<code>application:datascience_rulesprojects_rules:update:fromAst</code>	Permission to update self-service rules.
<code>application:datascience_rulesprojects_rules:update:fromTemplate</code>	Permission to update data science rules from templates.
<code>application:datascience_rulesprojects_snapshots:create</code>	Permission to create data science rules project snapshots.
<code>application:datascience_rulesprojects_snapshots:delete</code>	Permission to delete data science rules project snapshots.
<code>application:datascience_rulesprojects_snapshots:read</code>	Permission to read data science rules project snapshots.
<code>application:datascience_rulesprojects_snapshots:update</code>	Permission to update data science rules project snapshots.
<code>application:datascience_rulesprojects_templates:create</code>	Permission to create data science rules project templates.
<code>application:datascience_rulesprojects_templates:delete</code>	Permission to delete data science rules project templates.
<code>application:datascience_rulesprojects_templates:read</code>	Permission to read data science rules project templates.
<code>application:datascience_rulesprojects_templates:update</code>	Permission to update data science rules project templates.
<code>application:ssr_audit:privilegedView</code>	Permission to read Self-Service Rules Audit entries in privileged views – for instance, retrieving the audit respective to all Rules present in a given application.
<code>application:ssr_audit:read</code>	Permission to read self-service rule audit entries.
<code>application:ssr_test:read</code>	Permission to read self-service rule test results.
<code>application:ssr_test:run</code>	Permission to run self-service rule tests.
<code>application:ssr_test:update</code>	Permission to update self-service rule test results.

Schemas

Name	Description
<code>application:datascience_schemas:create</code>	Permission to create Data Science Schemas.
<code>application:datascience_schemas:delete</code>	Permission to delete Data Science Schemas.
<code>application:datascience_schemas:read</code>	Permission to read Data Science Schemas.
<code>application:datascience_schemas:update</code>	Permission to update Data Science Schemas.

Tags

Name	Description
application:datascience_tag_groups:create	Permission to create Data Science Tag Groups.
application:datascience_tag_groups:delete	Permission to delete Data Science Tag Groups.
application:datascience_tag_groups:read	Permission to read Data Science Tag Groups.
application:datascience_tag_groups:update	Permission to update Data Science Tag Groups.
application:datascience_tag_types:create	Permission to create Data Science Tag Types.
application:datascience_tag_types:delete	Permission to delete Data Science Tag Types.
application:datascience_tag_types:read	Permission to read Data Science Tag Types.
application:datascience_tag_types:update	Permission to update Data Science Tag Types.

White-box explanations

Name	Description
application:datascience_white_box_explanations:create	Permission to create White Box Explanation configuration.
application:datascience_white_box_explanations:delete	Permission to delete White Box Explanation configuration.
application:datascience_white_box_explanations:read	Permission to read White Box Explanation configuration.
application:datascience_white_box_explanations:update	Permission to update White Box Explanation configuration.

JupyterLab

Name	Description
userid:management	Permission to access and restart the JupyterLab process.

Sandbox evaluations

Name	Description
application:sandbox_evaluations:create	Permission to create sandbox evaluations.
application:sandbox_evaluations:delete	Permission to delete sandbox evaluations.
application:sandbox_evaluations:read	Permission to read sandbox evaluations.
application:sandbox_evaluations:update	Permission to update sandbox evaluations.

Custom services

Name	Description
application:dependencies:create	Permission to create new custom services.
application:dependencies:delete	Permission to delete custom services.
application:dependencies:read	Permission to read custom services.
application:dependencies:update	Permission to update custom services.

History

Name	Description
<code>application:history:global:read</code>	Permission to read history log application-wide.
<code>application:history:global:revert</code>	Permission to revert changes application-wide.
<code>application:history:resource:read</code>	Permission to read history log of a single resource.
<code>application:history:resource:revert</code>	Permission to revert changes to a single resource.

Data-processing jobs

Name	Description
<code>application:jobexecutions:abort</code>	Permission to abort job executions.
<code>application:jobexecutions:create</code>	Permission to create new job executions.
<code>application:jobexecutions:delete</code>	Permission to delete job executions.
<code>application:jobexecutions:read</code>	Permission to read job executions.
<code>application:jobexecutions:start</code>	Permission to start job executions.
<code>application:jobexecutions:update</code>	Permission to update job executions.
<code>application:job_profiles:create</code>	Permission to create job profiles.
<code>application:job_profiles:delete</code>	Permission to delete job profiles.
<code>application:job_profiles:read</code>	Permission to read job profiles.
<code>application:job_profiles:update</code>	Permission to update job profiles.

Managed lists

Name	Description
<code>application:managedlists:create</code>	Permission to create managed lists.
<code>application:managedlists:delete</code>	Permission to delete managed lists.
<code>application:managedlists:read</code>	Permission to read managed lists.
<code>application:managedlists:update</code>	Permission to update managed lists.
<code>application:managedlistitems:create</code>	Permission to create managed list items.
<code>application:managedlistitems:delete</code>	Permission to delete managed list items.
<code>application:managedlistitems:read</code>	Permission to read managed list items.
<code>application:managedlistitems:update</code>	Permission to update managed list items.
<code>application:managedlistitems:decrypt</code>	Permission to download encrypted values of managed list items in plain text format. The plain text download needs to be enabled using the <i>pulse.global.lists.allowListDownloadInPlainText</i> property.
<code>application:managedlistitems:revision:create</code>	Permission to create pending changes to managed list items. These are edits that another user must review (and approve) before they are applied to the list.

Name	Description
application:manage dlistitems:revision :update	Permission to mark pending changes to list items as accepted or rejected. Accepted revisions are applied as definitive edits to the list.
application:manage dlistitems:revision :read	Permission to read pending changes to managed list items.

Outcome combinators

Name	Description
application:outcome_combinators:create	Permission to create new outcome combinators.
application:outcome_combinators:delete	Permission to delete outcome combinators.
application:outcome_combinators:read	Permission to read outcome combinators.
application:outcome_combinators:update	Permission to update outcome combinators.

Workflows

Name	Description
application:rte_workflows:create	Permission to create new workflows.
application:rte_workflows:delete	Permission to delete workflows.
application:rte_workflows:read	Permission to read workflows.
application:rte_workflows:update	Permission to update every parameter in a workflow.
application:rte_workflows:update:schema	Permission to update a workflow's schema.
application:rte_workflows:update:name	Permission to update a workflow's name and description.
application:rte_workflows:update:outcomes	Permission to update workflow outcomes.
application:rte_workflows:update:outcomecombinator	Permission to update a workflow's outcome combinator.
application:rte_workflows:update:services	Permission to update workflow services.
application:rte_workflows:update:advancedconfigs	Permission to update a workflow's advanced configurations.

User interface

Name	Description
pulseviews:login	Permission to log in in pulseviews.
pulseviews:management:tabs:*	Permission to allow users to access all pulseviews tabs.
pulseviews:management:tabs:dataconfiguration	Permission to allow users to access pulseviews' data configuration tab.
pulseviews:management:tabs:workflows	Permission to allow users to access pulseviews' workflows tab.
pulseviews:management:tabs:lists	Permission to allow users to access pulseviews' fraud managed lists tab.
pulseviews:management:tabs:jobs	Permission to allow users to access pulseviews' jobs tab.
pulseviews:management:tabs:history	Permission to allow users to access pulseviews' history tab.

User managementâ

Auditâ

Name	Description
security:audit:read	Permission to read audit logs.

Ownershipâ

Name	Description
security:ownership:read	Permission to read group hierarchy.
security:ownership:update	Permission to update group hierarchy.

Permissionsâ

Name	Description
security:permissions:create	Permission to create new permissions.
security:permissions:delete	Permission to delete permissions.
security:permissions:read	Permission to read permissions.
security:permissions:update	Permission to update permissions.

Rolesâ

Name	Description
security:roles:create	Permission to create new roles.
security:roles:delete	Permission to delete roles.
security:roles:read	Permission to read roles.
security:roles:update	Permission to update roles.

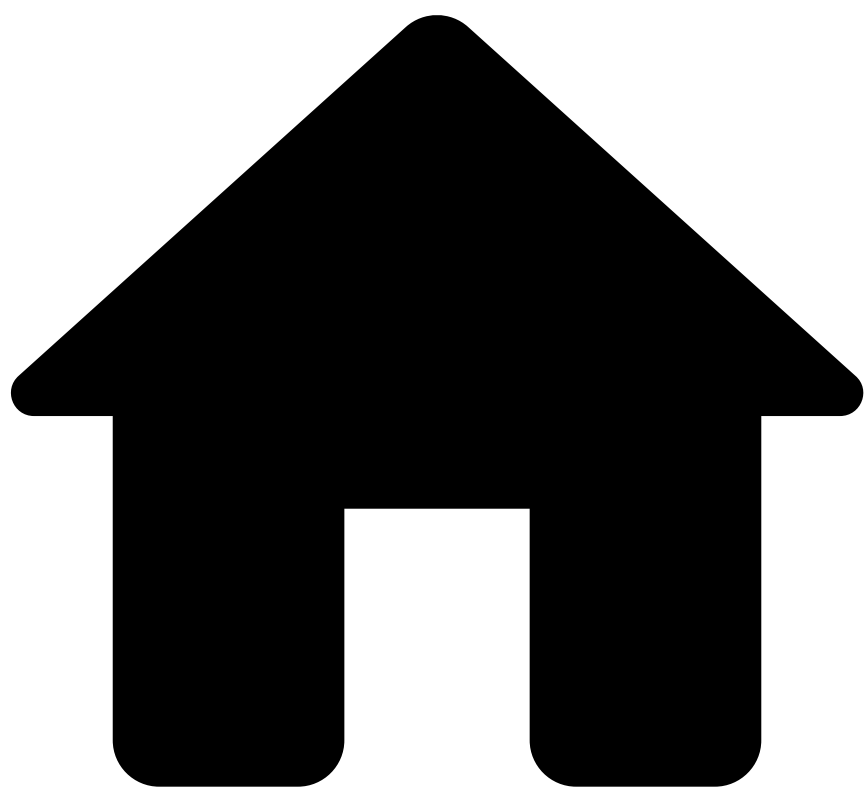
Tenantsâ

Name	Description
security:tenants:create	Permission to create tenants.
security:tenants:delete	Permission to delete tenants.
security:tenants:read	Permission to read tenants.
security:tenants:update	Permission to update tenants.

Usersâ

Name	Description
security:users:create	Permission to create new users.
security:users:delete	Permission to delete users.
security:users:expirepassword	Permission to expire users' passwords.
security:users:read	Permission to read users.
security:users:update	Permission to update users.

-



- Securing an Application
- Securing Metadata

On this page

Was this page helpful? 

Securing Metadata

Pulse is designed to be used by multiple users. However, not all users need to have full rights over all metadata. As such, the metadata in Pulse can be secured to prevent undesired accesses.

Since controlling the permissions for each individual user in an application would not be manageable, Pulse allows resource instances to be shared with other [groups](#), instead of users. For instance, you can have a group *BankA* where all users that are from Bank A will be on. Instead of allowing each user from this bank to have a certain right, you give these rights to the *BankA* group. Because groups are hierarchical, you can have a sub-group of *BankA* that has even less permissions (e.g. *BankA-Cashiers*).

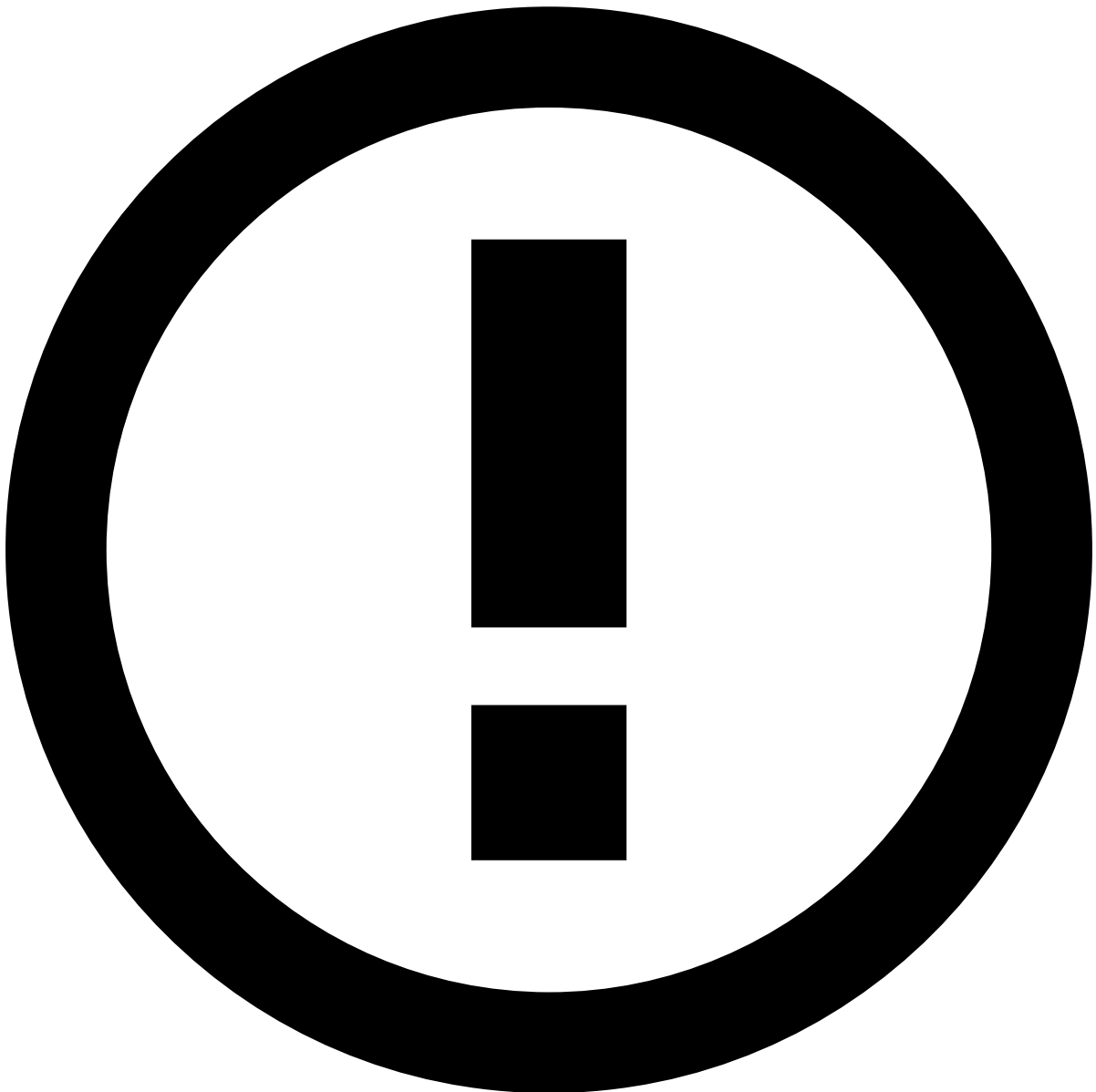
Setting the visibility of resources

Resources in Pulse are all configurable elements in an application – schemas, models, rules, data sources and so on. Given the multi-user nature of Pulse, it is common some resources not to be available for some users.

To secure resources, Pulse provides a way of setting the **visibility** of each resource instance.

That is, it is possible to allow a group to access KPI *A* but not KPI *B*. It is also flexible to the point where you can define which of three different rights a group has over the instance – *read*, *update* and *delete*.

There is a fourth right, *info*, that is activated whenever none of the previous rights are selected. When resources have *info* right they will appear on the listing views but without any link to edit them. This serves the purpose of making the user aware of their existence without giving any further details about its properties.



Other rights

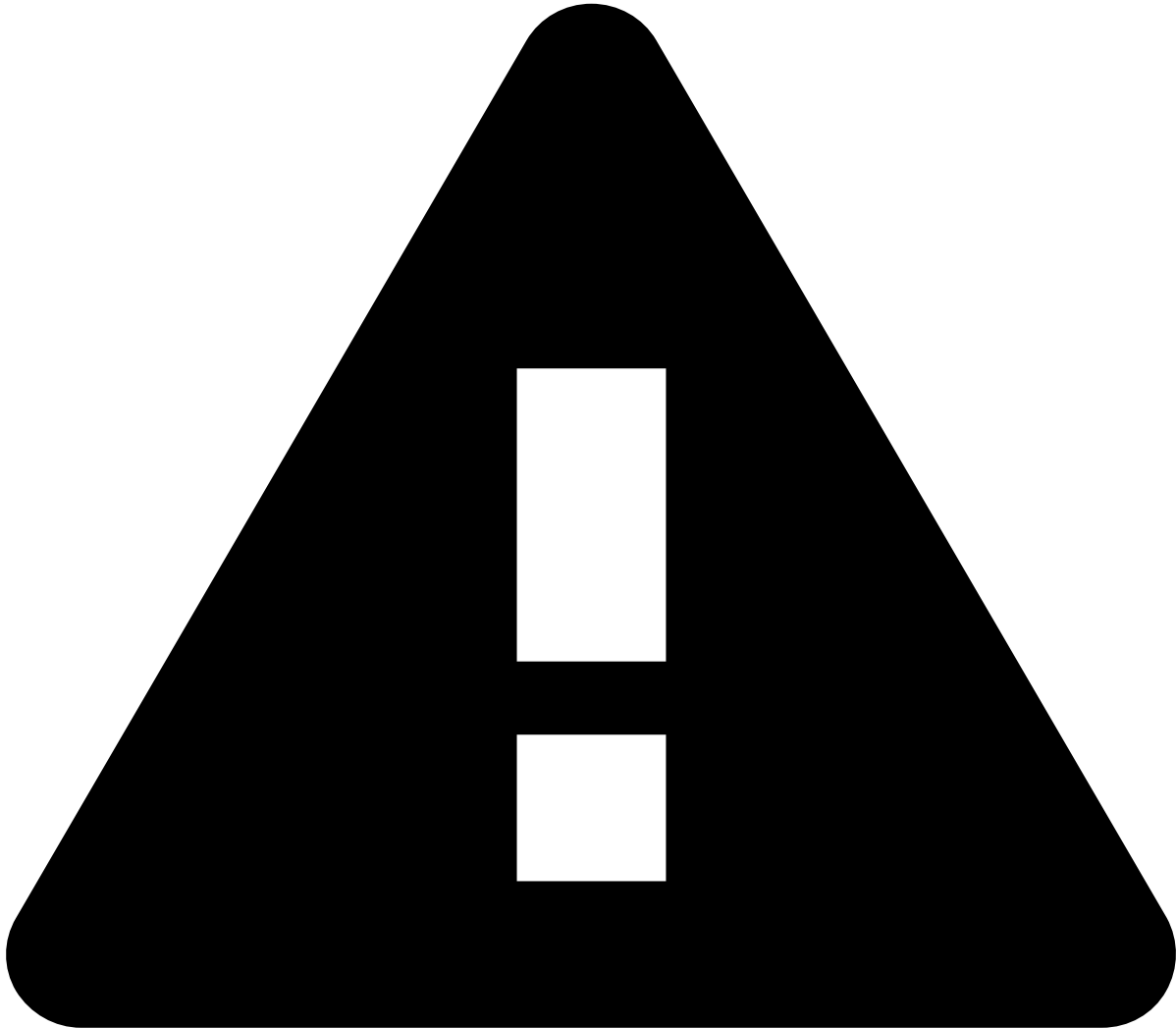
Read, update and delete rights are orthogonal to all types of resources. But some resources might have other specific rights. Applications, for instance, have a specific *manage* right that controls the access to Publish/Stop the application.

The visibility of a resource can be configured on its view, under Advanced Options.

Making a resource public🔒

The **default** security for a resource instance is basically no security. We call this **public visibility**. Essentially it means that everyone is able to view, update and delete it (as long as that user has the static permissions to do so).

Therefore, public resources should be used with care, given their non-existing security.



warning

Since resources are public by default, it is recommended that you configure their visibility upon creation, to avoid non-desired accesses.

Making a resource private🔒

The higher security level for a resource instance is to make it private. If configured this way, an instance will only be visible to the user who created it. No other user will be able to view it or modify it. Not even users that belong to the same group as the creator.

Sharing a resource with specific groups🔒

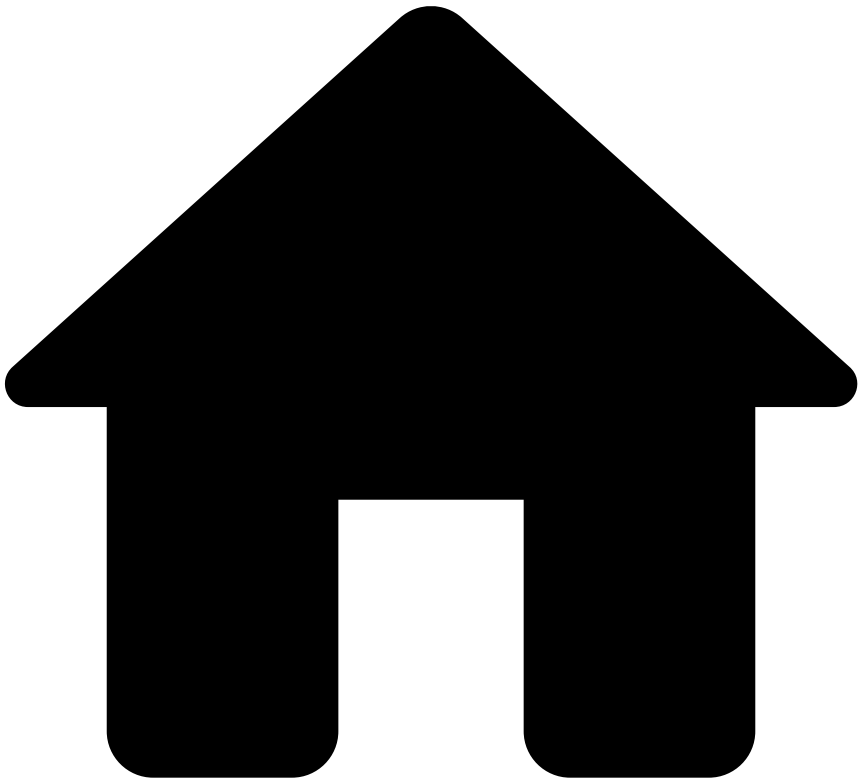
Besides making a resource public or hiding it from all users, it is also possible to share it with some groups. Not only that, you can also decide what actions a group can perform on that instance.

Groups are hierarchical. This is very useful when sharing a resource with a group. If you give *delete* permissions to *Group A*, **all ancestors** of *Group A* will be able to delete the instance. The same applies to other rights.

Configuring default visibility settings

It is possible to define the default ownership settings with which new resources are created with. The common scenario of default ownership usage is the following: the need for some user's resources to have the same visibility settings (for example, being private resources or being resources that belong to some group). Such behavior would require applying the same visibility settings to each resource created by user. To facilitate this procedure, Pulse provides the mechanism that allow assigning the default settings for visibility per user. These settings will then be applied automatically to each created resource.

•



- Securing an Application
- Security Best Practices

On this page

Was this page helpful? 

Security Best Practices

This section covers some of the best practices for securing Pulse applications. It shows a sample security scenario and explains how each security entity helps keeping the application secure and maintainable. The page covers some generic security best practices you should pay attention to, even if your project organization differs from the sample scenario presented here.

Local data source permissions

By default, a Pulse user who has permissions to create data sources can access any file in the local filesystem of the Pulse server.

This means that by specifying `file://<path-to-file>` in the URL path, the user might be able to have the contents of sensitive files displayed in the data source preview. Examples include:

- `/etc/passwd`
- `/etc/shadow`
- Credentials to other machines such as databases that are present in config files under Pulse directory: `conf/`

For this reason, Pulse has a property that limits the allowed paths. Note that this property only applies locally, and thus, remote locations such as HDFS should restrict file accesses using the remote system's permissions, using the property `allowedLocalDatasourcePaths`.

Below is an example on how to restrict the allowed local paths of source file locations for data sources, assuming that Pulse is installed in the path `/opt/pulse`:

```
pulse.global.allowedLocalDatasourcePaths=/opt/pulse/var/datasources
```



info

The restriction on allowed local paths can also be specified per **Pulse app**.

Sample security scenarioâ

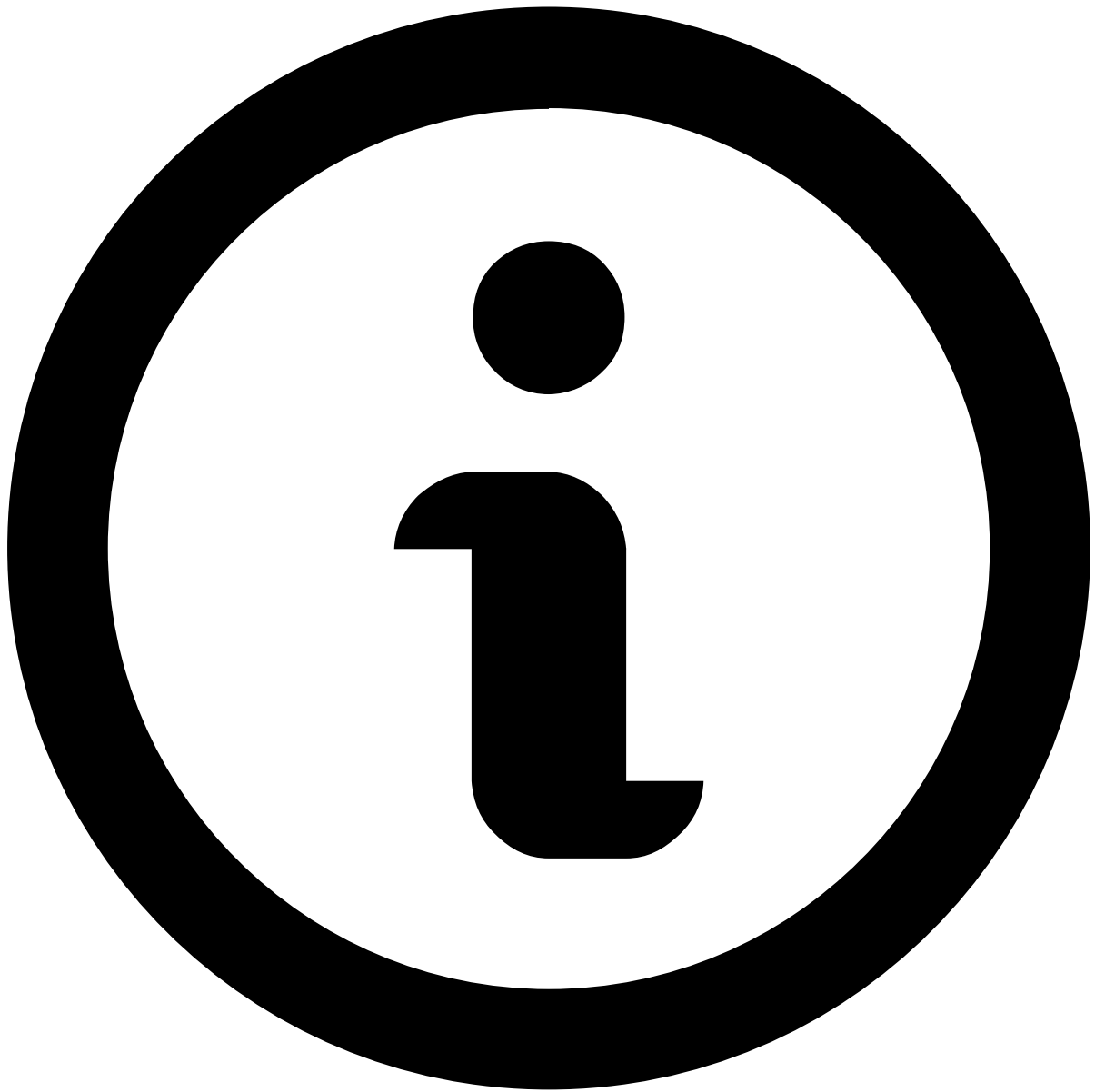
The following diagram depicts a hypothetical scenario of an acquiring company deploying Pulse to provide its merchants with fraud detection and prevention capabilities. All merchants will benefit from the acquirer defined fraud detection rules and each merchant's own fraud analysts can create customized rules tailored to their specific cases.

Security entities and their relationshipsâ

We will now look at each security entity, explain why it is needed and how it interacts with the other security entities. We will also analyze the role of each security entity on building our sample security scenario.

Usersâ

Users are the most well known security entity. They are present in almost every system with authentication. Every subject that authenticates in Pulse must have a unique user. The authentication of the user can be managed by Pulse itself or by an external authentication system such as Active Directory. Users are only responsible for the authentication part. Authorization is handled by permissions and resource rights. Users belong to groups which are the easiest and more maintainable way of managing authorization.



note

When creating new users, it is a good security practice to always set the default ownership. In practice, this will ensure that resources created by that user will, by default, be created with those chosen groups and rights for each group.

Groups

Groups represent organization positions that users of the system will belong to. In our sample scenario, **user Merchant B Employee 1 belongs to the groups Merchant B Employees and Merchant Employees**. We can say this user is an employee of Merchant B and also a generic Merchant employee.



info

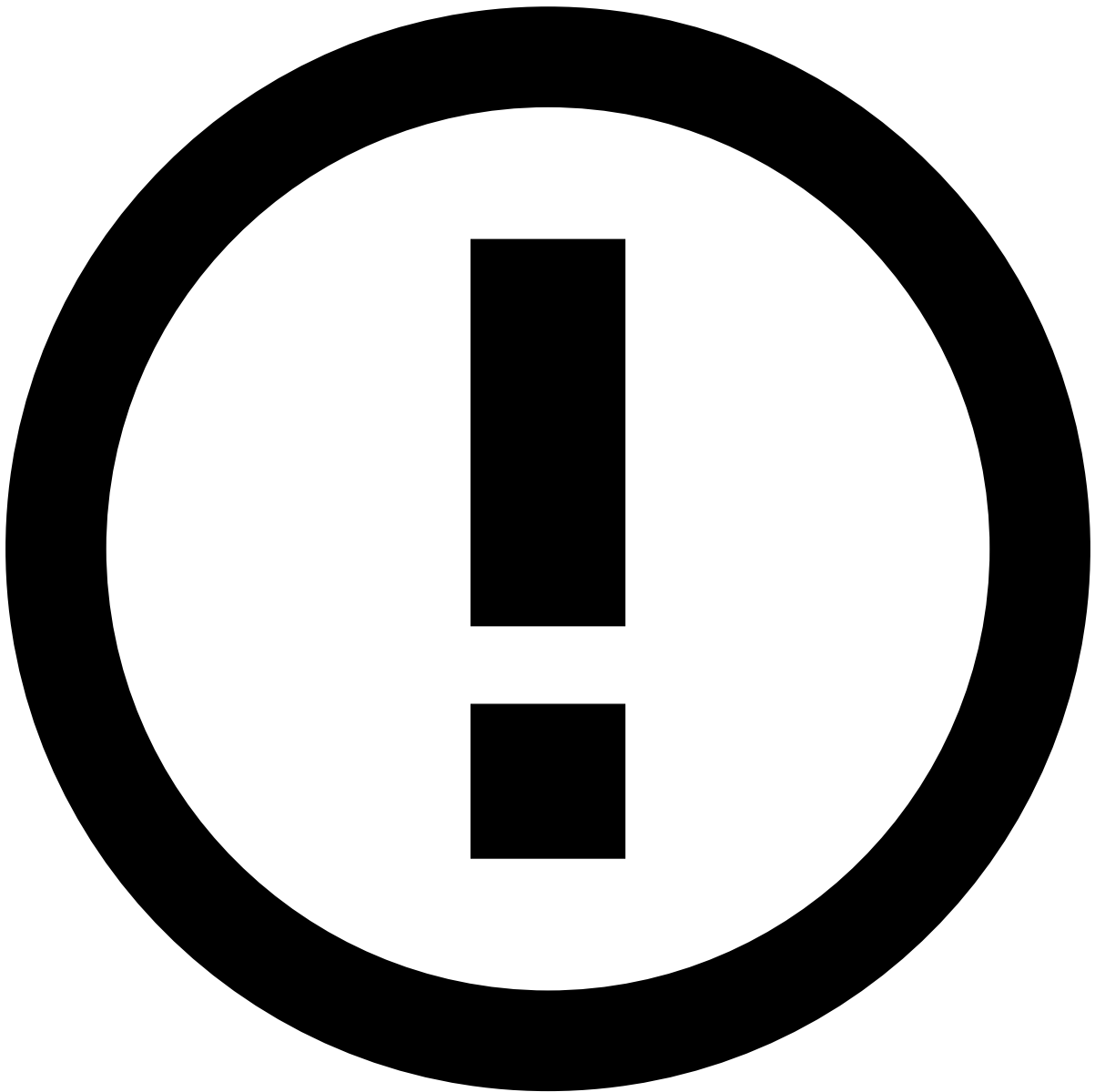
It is useful to have the user belong to a generic group of merchant employees because some of the system resources might have to be shared to all merchant employees regardless of the merchant. This greatly simplifies the maintenance of the system.

In our sample scenario, an example of a resource that should be shared to the *Merchants Employees* group is the *Generic Merchant Rules* resource. In this case, the sharing would be **read-only**, we will see how to do that later.

Each group might have zero or more associated roles. Having roles associated to groups is useful in several ways. It helps managing what each user can do by just assigning them to their groups, instead of having to give roles to each user. It also helps when you need to revoke some role from all users in a group, in this case you just have to edit the group, instead of editing all the users. Learn everything about groups in the [Security Entities](#) page.

Roles and permissions🔒

A [permission](#) is a right to do a very specific action. Pulse has many built-in permissions for managing the resources; for instance, the permission to *read fraud rules*, or the permission to *create new fraud rules*.



info

A permission is just a right to do some action on a resource (e.g. *read fraud rules*). It says nothing about the right to do that action on a particular resource instance (e.g. *read fraud rule X*). Resource rights are the topic of the next section.

A [role](#) is just a set of permissions.

Permissions are the most fine-grained security entity. After creating suitable roles, you will not have to use permissions directly. They will be enforced through the association of users to specific roles and groups. As we said earlier when discussing groups, roles can be associated to groups, making the group indirectly associated with the permissions in those roles. Groups are the highest level security entity and you should use them to create secure and maintainable deployments.

In our example scenario, both *Merchants Managers* and *Merchants Employees* groups are associated with roles. The roles depicted here do not exist in Pulse and have to be created with the relevant permissions. The *rules-management* role might give users of the *Merchants Managers* group permissions to create, read, update and delete fraud rules. While a user of the *Merchants Employees* group might only have permissions to manage templated fraud rules and read fraud rules.

Learn everything about roles and permissions in the [Security Entities](#) page.

Resource rights ⓘ

Resource rights dictate what users of a group can do to a resource instance.

Resource rights are granted when the resource instance is created or updated. To understand this section you should be familiar with the concepts covered in the [Securing Metadata](#) page.



info

Resource rights never override the user's permissions. If a user does not have the permission to read fraud rules, a fraud rule shared with that user's group will still be invisible to the user.

Considering our sample scenario, *Merchant A Rules* and *Merchant B Rules* must be visible only to users of *Merchant A Employees* and *Merchant B Employees* groups respectively. To achieve this, the user creating the rule must share it with the group and grant that group the desired rights. In this case it could be the rights to do everything with the rule, since this would apply only to users of that group. The *Generic Merchant Rules* should be shared with the *Merchants Employees*

group but only having *Read* right. Remember that users belonging to groups higher on the groups hierarchy will have the same rights, if not more.

Read the [Typical permission setups](#) for guidance on specific cases related to rights management.

Use cases

This section covers some of the best practices related to common security use cases. Some are related to the sample scenario presented before while others are more generally applicable.

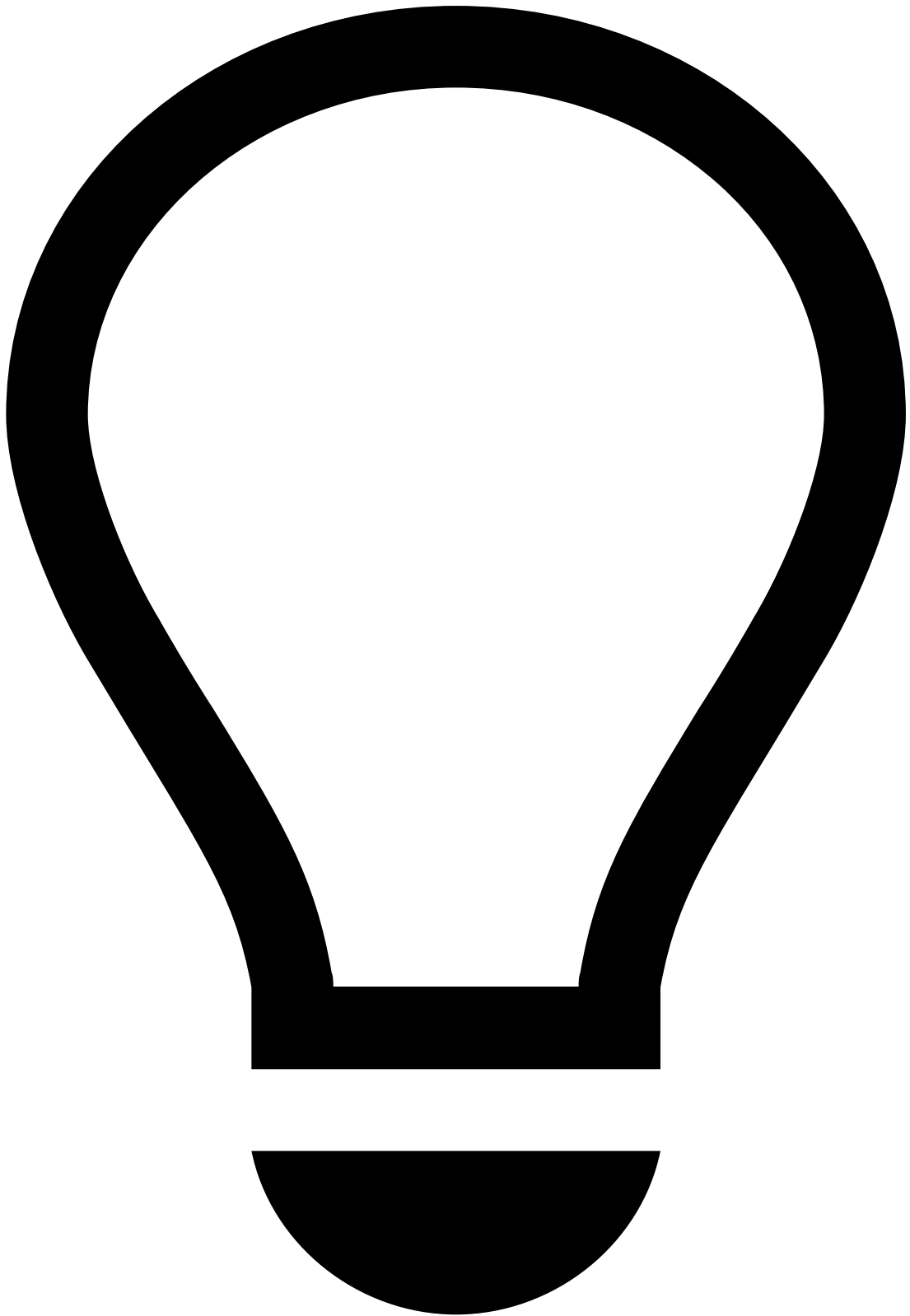
Restricting the manage right on applications

When creating an application remember to configure the application ownership to Groups and grant the *manage* right only to a restricted group of application administrators. This will prevent any user with access to the application from inadvertently or maliciously publishing or stopping the application.

Creating a new user

When creating new users, there are no default assignments regarding groups, roles, etc. The administrator must know beforehand how to configure the new user for it to reach its goals. Usually adding users is not time consuming, since it's quite common to already have some roles completely configured, hence it's only a matter of connecting the new user to the designated groups and it's done.

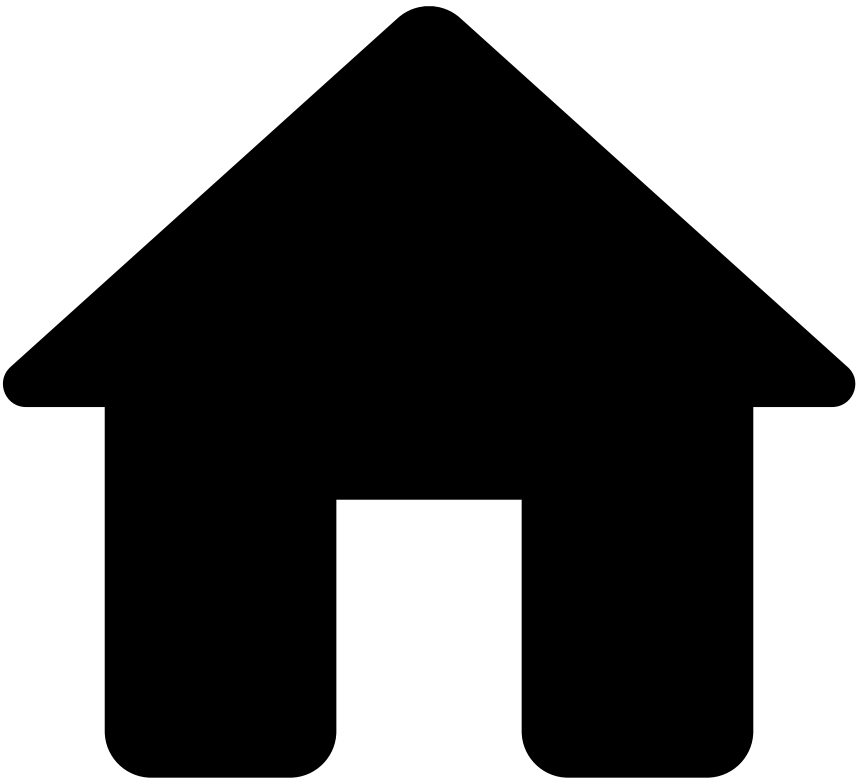
So let's say you need to create a user that manages all the merchants - let's call it *Merchant Manager 1*. If you assign this user to the *Merchants Managers* group, it will have access to all the groups that are below in the group hierarchy - that is *Merchant A**,** *Merchant B** and *Merchants Employees* - besides all resources and roles assigned to the original assignment group (*Merchants Manager*). All the resources and roles bound to these groups will be available as well to this new user.



tip

The higher a user is in the groups hierarchy, the more rights will accumulate and therefore more power over the application resources.

•



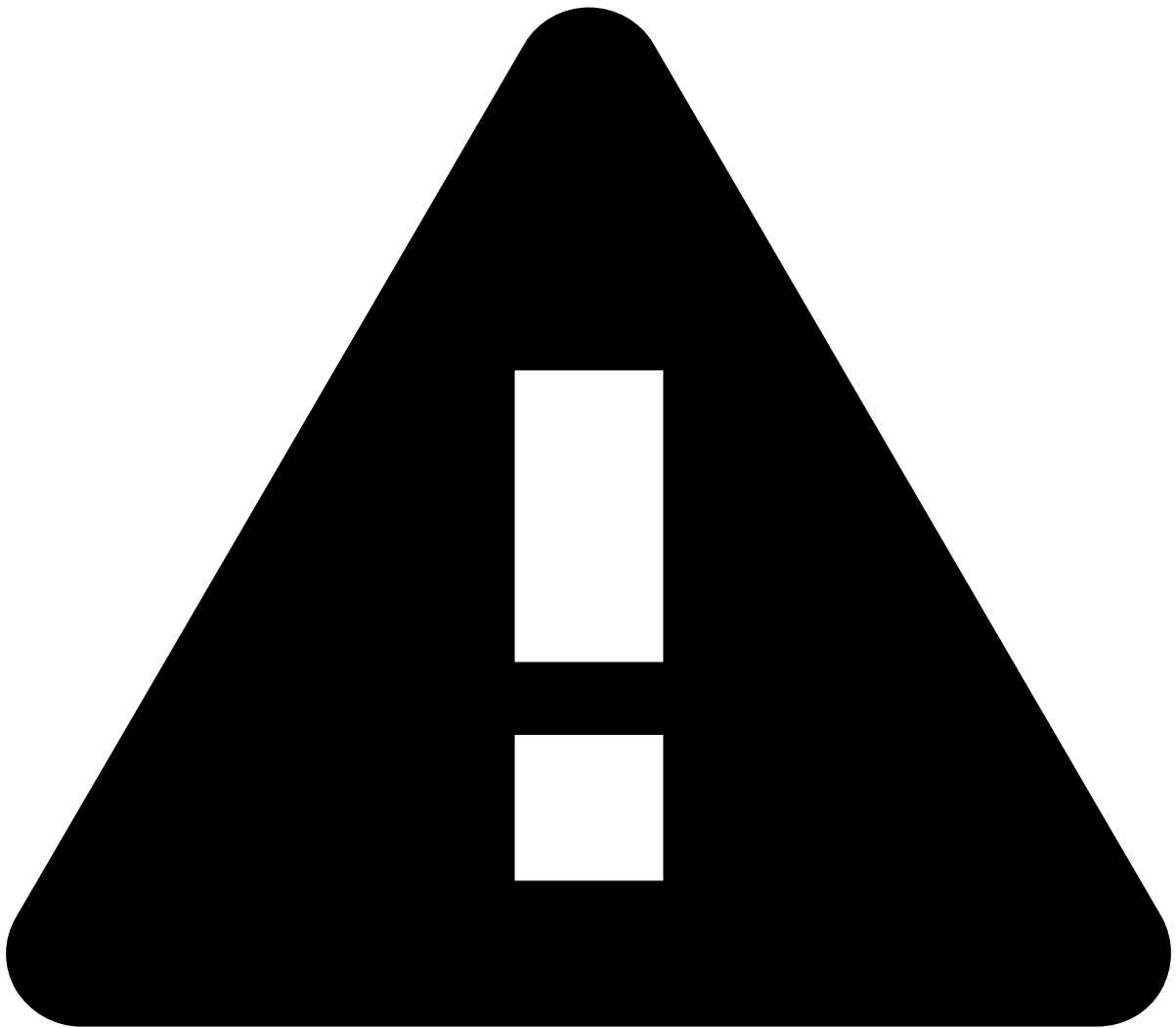
- Securing an Application
- Typical Permission Setups

On this page

Was this page helpful? 

Typical Permission Setups

This page describes several scenarios that administrators often need to configure in Pulse.



Resource dependencies

Resources sometimes have dependencies between them, hence to access one, you might need to have access to another. These dependencies are not always obvious. If you are having difficulties with resource permissions, contact Feedzai's technical support.

Basic read-only access

A good reference of the permissions necessary for a user to be able to sign in to Pulse is the `pulseviews-reader` role. With the permissions of this role, a user can log in and view all the resources.

The permissions assigned to `pulseviews-reader` are as follows:

Name	Description
<code>application*:read</code>	Allows the user to get the list and read metadata resource instances (e.g., applications, KPIs).
<code>pulseviews:login</code>	Allows the user to log in.
<code>pulseviews:management:tabs:*</code>	Allows the user to see all the tabs available in Pulse.
<code>security:ownership:read</code>	Allows the user to read ownership information (necessary to check if he can see specific resource instances).

Name	Description
<code>security:users:read</code>	Allows the user to read its metadata.

Restrict tenant permissions with roles

In the context of multi-tenancy, Pulse allows large companies to manage multiple clients by configuring a tenant for each client. Multiple users can be associated with a tenant to allow access and management of each account in Pulse. The level of access and permissions given to each tenant is controlled by adding roles to the user with defined permissions. The following is the list of permissions for tenant users:

- `pulseviews:login`
- `security:ownership:read`
- `security:permissions:read`
- `security:roles:read`
- `security:users:update`
- `security:users:read`

These are the minimum permissions for a user to be able to log in. A user with this role will be able to log in to Pulse, but will not be able to do anything else (e.g. to open Pulse applications). The same applies to Case Manager; the user will need to have other roles with Case Manager permissions (`cm: . . .`), according to the work that user needs to do (e.g. CM admin, CM analyst, etc.).

Pulse permissions

A full list with all the permissions in Pulse can be found in the [List of Permissions](#) page.

-



- Component Encryption

On this page

Component Encryption

Was this page helpful? 

Component encryption provides secure communication between Pulse components.

Protecting sensitive data including credit card numbers, names and addresses is a big concern in the field of risk management. To address this concern, component encryption creates a trustful environment by securing in-transit data that is communicated across Pulse components. The goal of component encryption is to ensure data privacy and integrity.

In Pulse, component encryption is implemented using SSL/TLS.

Introduction to component encryption

Component encryption aims to protect the communication between components in Pulse. This includes securing the communication between the following components:

- Internal Pulse components (e.g. between Pulse server and application engine).
- Internal and external components (e.g. between Pulse server and storage).
- External components (e.g. between various nodes of external components such as Cassandra nodes).

How does encryption work?

Pulse implements component encryption using SSL/TLS. With these protocols, the data communicated between components is encrypted using a shared secret referred to as encryption key.

The following steps summarize the process of generating and using an encryption key:

1. Server sends its public key to the client. This key is included in a digital certificate chain that contains additional information about the server.
2. Client generates the encryption key.
3. Client sends the encryption key to the server. In this step, the encryption key is encrypted with the server public key.
4. Server decrypts the encrypted key sent by the client using the server private key.
5. At this point the server and the client share the encryption key. All the communications between the server and the client during an active session can be encrypted with this key.

Authentication

In addition to encryption, SSL/TLS typically requires an authentication step to establish a connection between two components. In a typical case, the authentication is performed by the client (i.e. client authenticates the server) which is referred to as server authentication. However, authentication can be also extended to enable mutual authentication, i.e. client authenticates the server and the server authenticates the client. This latter option is commonly referred to as client authentication.

To perform server authentication, the client is required to trust a certificate presented by the server. More specifically, the component acting as a client checks:

- If the server certificate is valid.
- If the server certificate is signed by a trusted Certificate Authority (CA). There are different approaches to have signed certificates which are explained in the section [SSL/TLS certificates](#).

Component encryption with server authentication

Component encryption with server authentication requires information that must be configured on both the server and the client side to establish a secure connection:

- **Server side:** a private key and a signed certificate which certifies the information contained in the certificate.
- **Client side:** one or more certificates that the client trusts.

Typically, the keys and the certificates can be stored in various formats. In Pulse, this information is stored in Java KeyStore (JKS) format. Pulse follows a conventional nomenclature:

- **keystore:** A file that contains private keys and certificates.

A keystore file is protected with two passwords: one for the whole store and one for accessing the private key. Note that a keystore can contain more than one private key as long as each private key has a different password.

- **trustore:** A file that contains only certificates.

A certificate contains a public key and a digital signature generated with the private key of the certificate authority that issued it.

The access to a truststore file can be optionally secured with a password named truststore password. Technically, a truststore file is just keystore with no private keys; we make this distinction only to make it clear the purpose of the store.

Component encryption with server authentication only requires configuration of a keystore on the server side and a truststore on the client side.

The next section presents a diagram of the Pulse architecture for a better visualization of the components that require configuration of component encryption.

Pulse architecture with component encryption

The following diagram shows a basic Pulse architecture comprising internal Pulse components (e.g. Pulse server, application engine) and external components (e.g. Cassandra, Spark):

The numbers represent communication encryption between components. As shown in figure, the protocol SSL/TLS for component encryption is combined with additional communication protocols depending on the component.

Configuring component encryption in Pulse

To configure component encryption, Pulse requires a keystore when acting as a server and a truststore when acting as a client. The truststore has to contain a valid certificate signed by a trusted CA.

The default configuration of component encryption is done using `pulse.global` properties as described in the page [Configuration Overview](#) page.

In addition to setting default Pulse properties, there are components that cannot be configured with the default approach (e.g. relational databases) or that need custom configurations. This is described in the [Configuring Encryption for External Components](#) page.

SSL/TLS Certificates

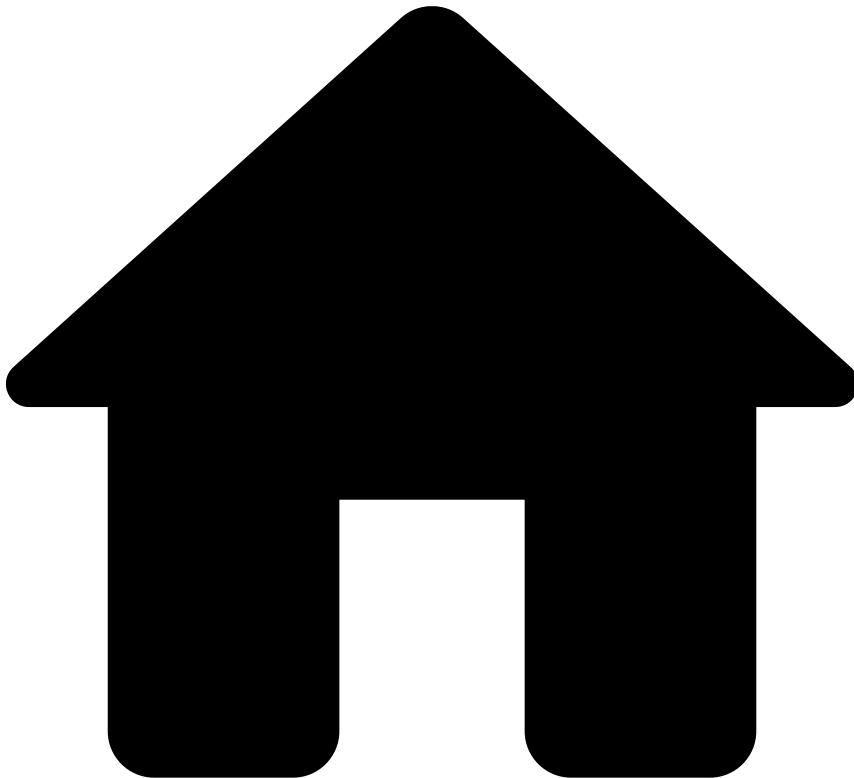
SSL/TLS certificates are stored inside keystores and truststores which are used in the authentication process.

There are different options to configure certificates for authentication including:

- Using self-signed certificates and configuring all components to trust them.
- Using certificates signed by an internal CA and configuring all components to trust this CA.
- Using certificates signed by a commercial external CA. This is technically identical to the previous one but has an higher administrative overhead; however, well known CAs are typically trusted without requiring a truststore to be setup.

The second approach is the recommended in a production environment in Pulse, whereas the first approach is typically used in a testing environment. To help the configuration process, Pulse includes scripts to help creating keystores and truststores files with all the required certificates. Visit the [Creating Certificates](#) page for more information on how to configure these scripts.

•



- Security Configuration
- Security Configuration

On this page

Was this page helpful? 

Security Configuration

Similar to other services the configuration for this module is performed in the main configuration file `PULSE_HOME/conf/pulse.properties`. Properties can be configured with name space `pulse.manager` or `pulse.global`.

Complete example configuration

The following is a complete example configuration.

```
<target_namespace>.services.core.security.createDefaults=true
<target_namespace>.services.core.security.hash=SHA-256
<target_namespace>.services.core.security.database.schemapolicy=none

# LDAP example:
```

```

<target_namespace>.services.core.security.realm.ldap.sample1.enable=false
<target_namespace>.services.core.security.realm.ldap.sample1.systemusername=cn=admin,ou=people,dc=example,dc=com
<target_namespace>.services.core.security.realm.ldap.sample1.systempassword=superduperpassword
<target_namespace>.services.core.security.realm.ldap.sample1.connection=ldap://example.com
<target_namespace>.services.core.security.realm.ldap.sample1.template=cn={0},ou=people,dc=example,dc=com

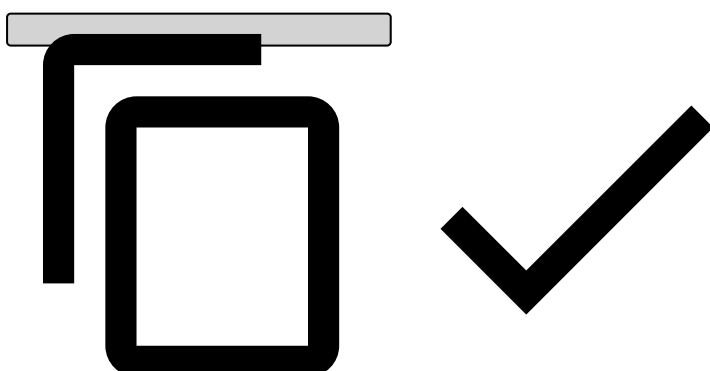
# LDAPS example:
<target_namespace>.services.core.security.realm.ldap.sample2.enable=false
<target_namespace>.services.core.security.realm.ldap.sample2.systemusername=cn=admin,ou=people,dc=example,dc=com
<target_namespace>.services.core.security.realm.ldap.sample2.systempassword=superduperpassword
<target_namespace>.services.core.security.realm.ldap.sample2.connection=ldaps://example.com
<target_namespace>.services.core.security.realm.ldap.sample2.template=cn={0},ou=people,dc=example,dc=com

# AD example:
<target_namespace>.services.core.security.realm.activedirectory.sample1.enable=true
<target_namespace>.services.core.security.realm.activedirectory.sample1.url=ldap://test-ad.zai
<target_namespace>.services.core.security.realm.activedirectory.sample1.searchfilter=(&(objectClass=user)(userPrincipalName=${login}))
<target_namespace>.services.core.security.realm.activedirectory.sample1.searchbase=DC=feedzai,DC=com
<target_namespace>.services.core.security.realm.activedirectory.sample1.searchscope=2

# Another AD example with Default Domain and system's account:
<target_namespace>.services.core.security.realm.activedirectory.sample2.enable=true
<target_namespace>.services.core.security.realm.activedirectory.sample2.url=ldap://test-ad.zai
<target_namespace>.services.core.security.realm.activedirectory.sample2.searchfilter=(&(objectClass=user)(userPrincipalName=${login}))
<target_namespace>.services.core.security.realm.activedirectory.sample2.searchbase=DC=feedzai,DC=com
<target_namespace>.services.core.security.realm.activedirectory.sample2.searchscope=2
<target_namespace>.services.core.security.realm.activedirectory.sample2.defaultdomain=feedzai.com
<target_namespace>.services.core.security.realm.activedirectory.sample2.systemaccountusername=pu
lse-ad-username
<target_namespace>.services.core.security.realm.activedirectory.sample2.systemaccountpassword=7a724f8fe49eb0dc241d27e6b71e000a

# Another Example for configuration with PAM:
<target_namespace>.services.core.security.realm.pam.sample3.enable=true
<target_namespace>.services.core.security.realm.pam.sample3.service=common-auth

```



Available properties

The next table summarizes all available properties for this service.

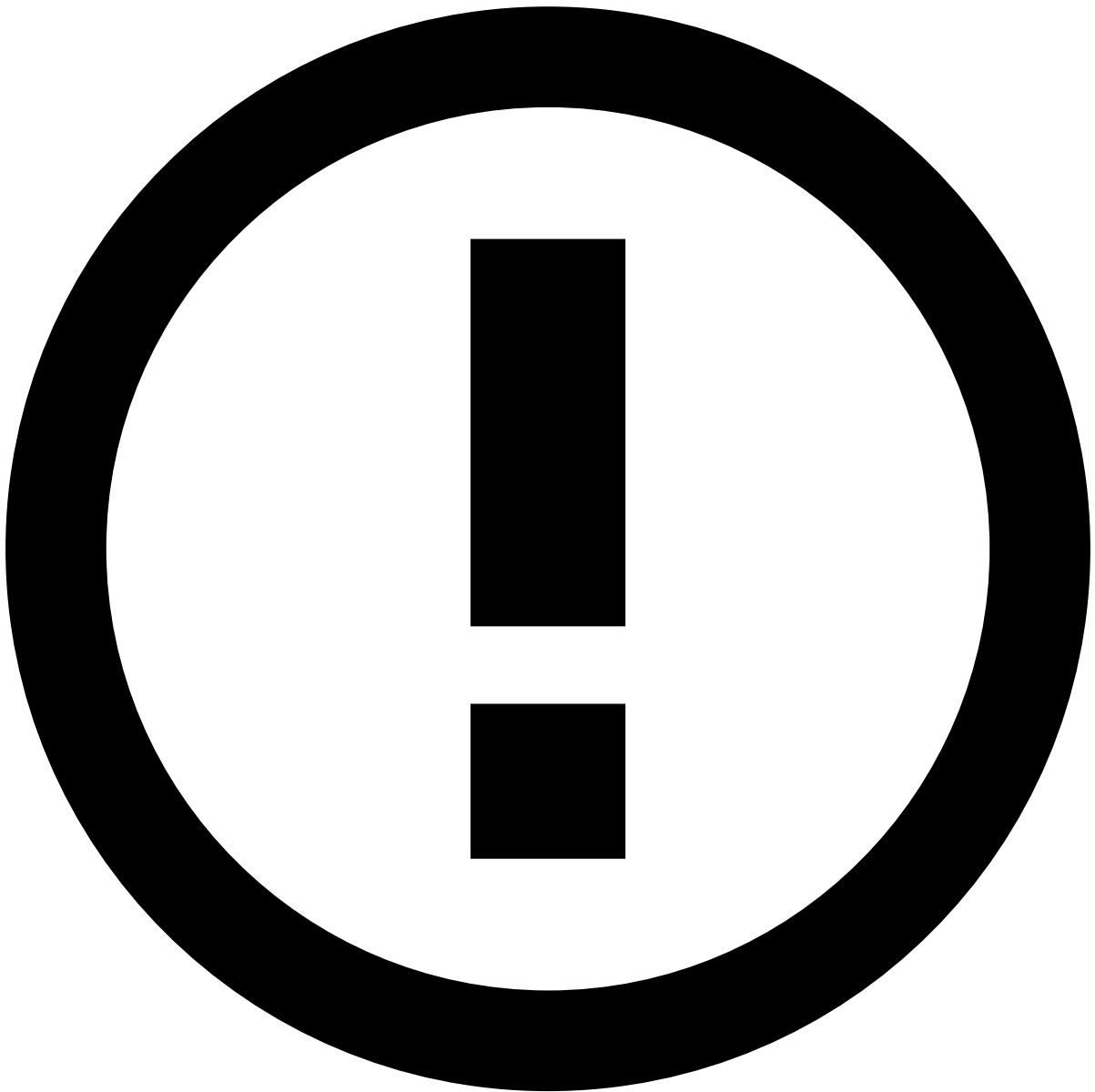
Parameter name	Type	Description	Mandatory	Default value
database	Database	The database configuration (overrides the defaults).	No	Inherited from <code>pulse.global.database</code> configuration
hash	String	The hash function for passwords. If you don't want encryption set to "none".	No	SHA-256
realm	List<AbstractRealm>	Possible realms are <code>LdapRealm</code> (used for LDAP authentication only, authorization is always retrieved from the database) and <code>ActiveDirectoryRealm</code> (used for active directory authentication)	No	N/A
xRequestedWithHeaderRequired	Boolean	Whether the REST API is to require the 'X-Requested-With' header to be present on all HTTP requests to protect against CSRF attacks.	No	<code>true</code>
transactions	TransactionManagerConfiguration	The configuration for the transaction manager used in the security metadata.	No	See table below
poolSize	Integer	The database pool size used for security metadata management.	No	<code>5</code>
poolLazy	Boolean	Whether the database pool used for security metadata management is lazy creating connections or eager.	No	<code>true</code>

Transaction Manager configuration

Parameter name	Type	Description	Mandatory	Default value
afterCommitActionsThreads	Integer	The number of threads used to run the after commit actions.	No	2
afterCommitActionsTimeoutSeconds	Integer	The timeout in seconds to wait for the execution of the after commit actions.	No	60

Configuring the security realm

You can also configure the security realm that you are going to use in this service's properties. `<name>` can be any user-defined string in case there are multiple realms configured. One name per realm.



info

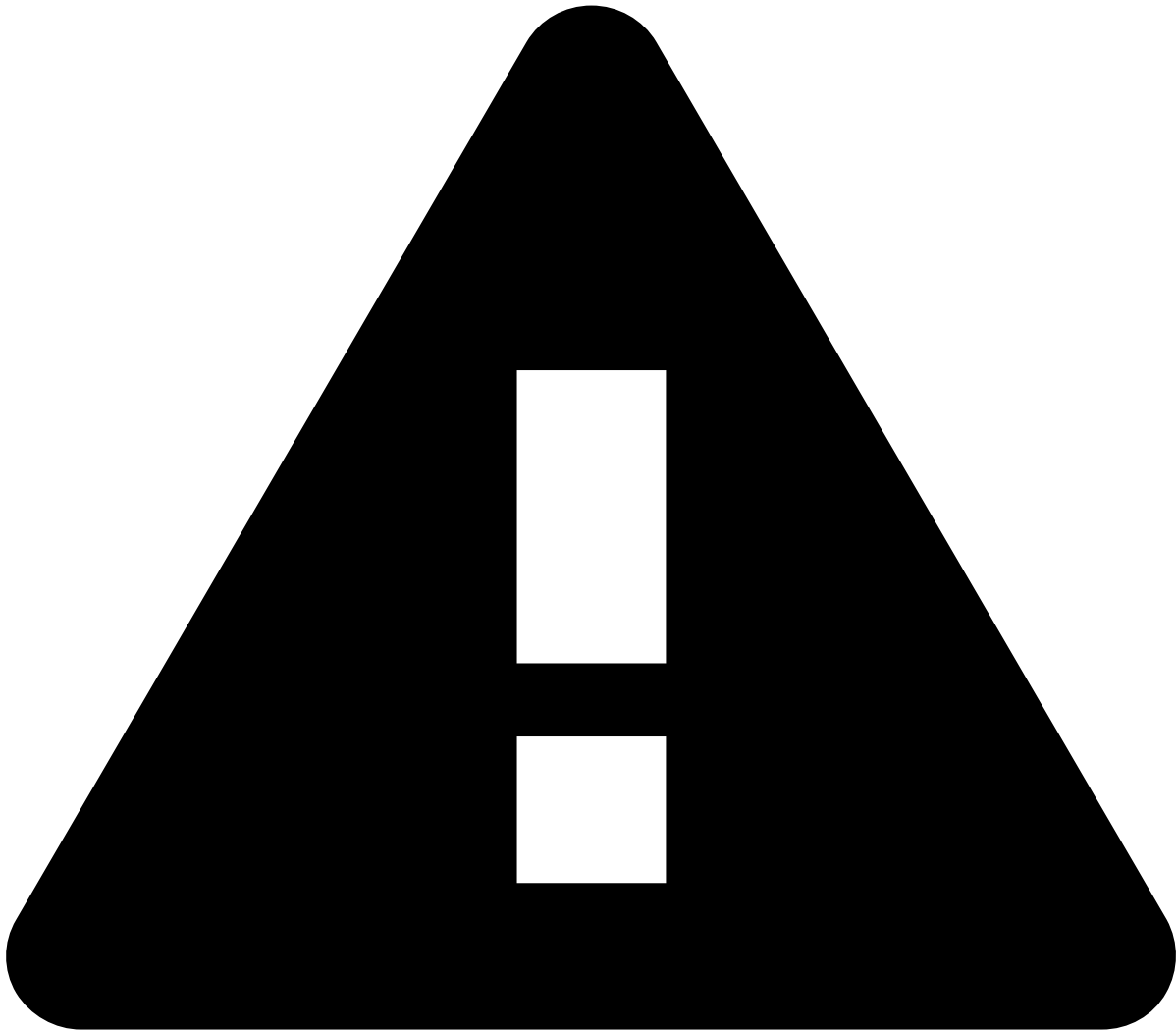
If you are using LDAP or PAM, note that you will need to first create the users in the Pulse database, and mark them as external. Only afterwards you can authenticate using these services.

Parameter name	Type	Description	Mandatory	Default value
<code>realm.ldap.<name>.systemusername</code>	String	The system username (use only if the LDAP server needs system authentication).	No	N/A
<code>realm.ldap.<name>.systempassword</code>	String	The system password (use only if the LDAP server needs system authentication). This field has configuration encryption support.	No	N/A
<code>realm.ldap.<name>.connection</code>	String	The LDAP / LDAPS connection. When using LDAPS, add the certificate to the truststore of Pulse, using the keytool .	Yes	N/A
<code>realm.ldap.<name>.template</code>	String	The template to use for authentication. When performing authentication, {0} will be replaced by the username.	Yes	N/A

Configuring the security realm's active directory

An active directory can be also be configured for the security realm.

Parameter name	Type	Description	Mandatory	Default value
<code>realm.activeDirectory.<name>.url</code>	String	The Active Directory LDAP endpoint (e.g. <code>ldap://test-ad.zai</code>). To use a SSL connection (e.g. <code>ldaps://test-ad.zai</code>), the Active Directory server certificate must be installed into the JVM key store.	Yes	N/A
<code>realm.activeDirectory.<name>.searchBase</code>	String	The base tree node to start a search (e.g. <code>DC=feedzai,DC=com</code>).	Yes	N/A
<code>realm.activeDirectory.<name>.searchScope</code>	Integer	The type of search scope to use: 0 (Base), 1 (One level) and 2 (Subtree)	No	0 (Base)
<code>realm.activeDirectory.<name>.searchFilter</code>	String	The filter to use for authentication. When performing authentication <code>\${login}</code> will be replaced by the username (e.g. <code>(&(objectClass=user)(userPrincipalName=\${login}))</code>).	Yes	N/A
<code>realm.activeDirectory.<name>.defaultDomain</code>	String	A default domain that will be appended to the username if no domain is already provided with the username (e.g. <code>feedzai.com</code>).	No	N/A
<code>realm.activeDirectory.<name>.systemAccountUsername</code>	String	System's account username to be used on searching across the directory for the authorized users, instead of the end-users account. When provided, this should be specified in conjunction with the <code>systemAccountPassword</code> . If not provided, the end-user's account will be used.	No	N/A
<code>realm.activeDirectory.<name>.systemAccountPassword</code>	String	System's account password to be used on searching across the directory for the authorized users. Check further information on <code>systemAccountUsername</code> .	No	N/A



caution

LDAP protocol searches can have results with continuation references (<https://tools.ietf.org/html/rfc4511#section-4.5.3>).

In Pulse's context, this can impact the system at 2 situations:

1. The allowed user set is spread across multiple servers (forest setup);
2. A valid user credentials are provided, outside of the search parameters scope (user not authorized to access Pulse).

When either of these situations occurs, connectivity is required towards all possible endpoints which includes IP connectivity and SSL configurations (e.g. server's certificate validation).

Not doing so could lead to:

1. Prevent login for authorized users;
2. Long lasting requests waiting for connect timeouts.

Configuring the Security Realm's PAM

PAM can also be configured for the security realm.

Parameter name	Type	Description	Mandatory	Default value
<code>realm.pam.<name>.service</code>	String	The PAM service name. The name of the PAM service Pulse should use for authentication (must be in the <code>/etc/pam.d</code> directory).	Yes	N/A

Default security credentials

By default, the module is configured to clean all security tables and create a default user with access to everything when you start the Pulse Server. If you don't want this to happen, you **must** set the `policy` to create (or simply remove the tag) and set `createDefaults` to `false`.

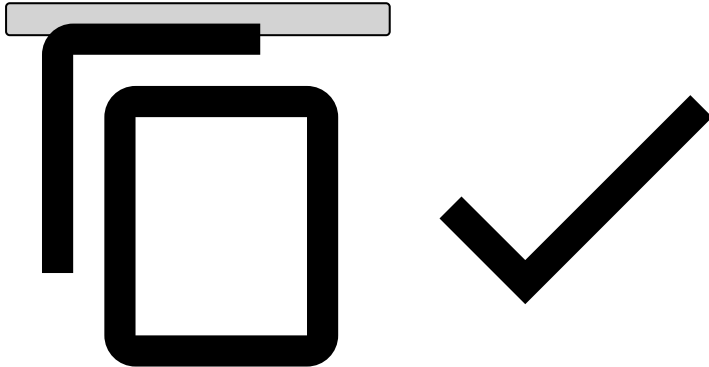
The default credentials use the username `pulse` and password `pulse`.

HTTP adapter security configuration

The security configuration for the HTTP adapter is in the file `PULSE_HOME/conf/engine/web/http-adapter.ini`. There you can find the filters and the URL patterns being filtered. By default it looks like the following.

```
[main]
authc.loginUrl = /login.jsp
ssl.port = 8443

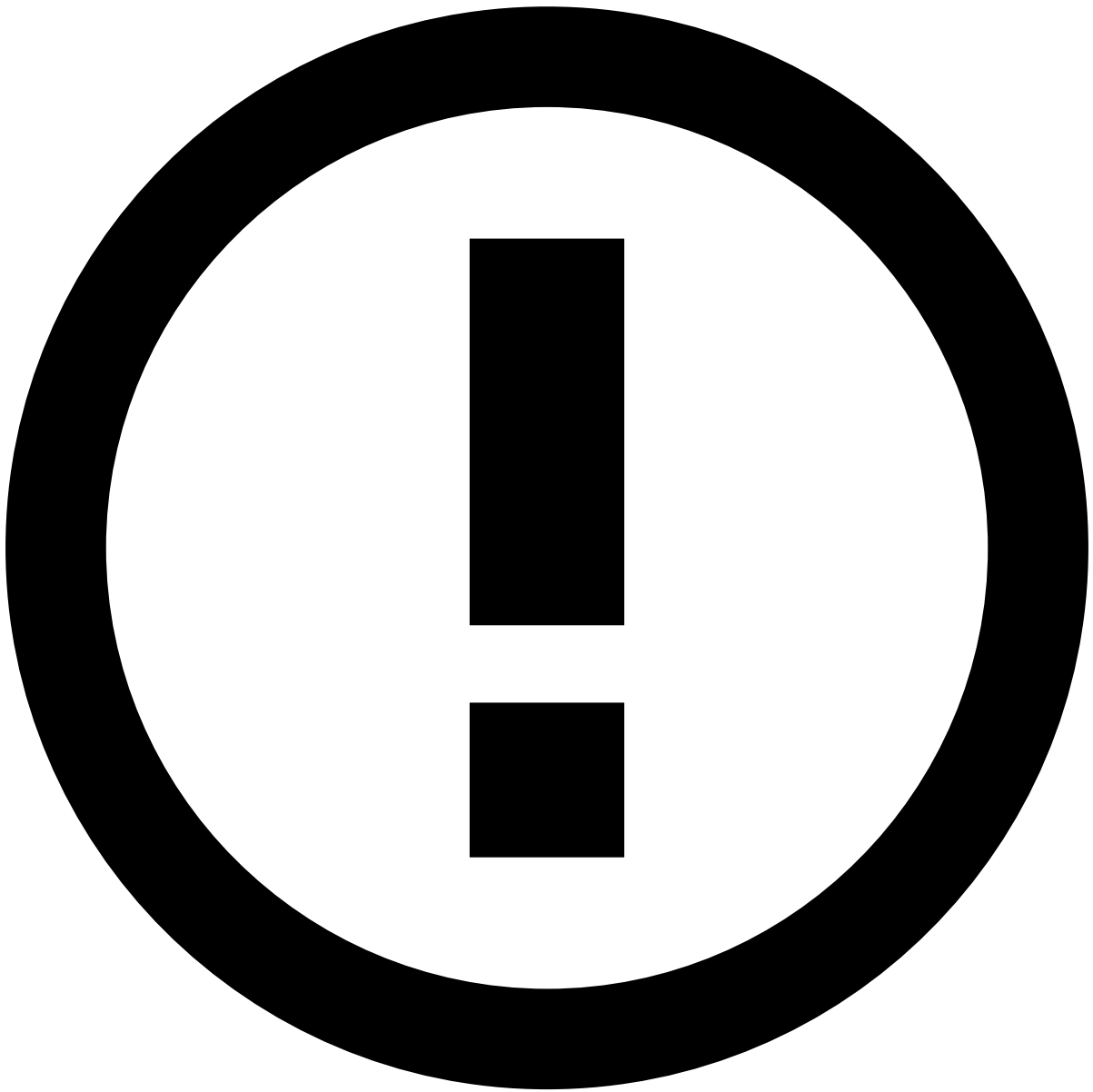
[urls]
/login.jsp = ssl, authc
/streams/** = ssl, authc
/baselines/** = ssl, authc
/kpis/** = ssl, authc
```



The **main** section is used to configure the filters. In this case, we are stating that the login page is "login.jsp" and that the SSL port is "8443".

The **urls** section is used to define the paths that will be filtered. In this case, everything beyond "streams", "baselines" and "kpis" will be secured using both defined filters.

If you want to learn more about Shiro filters, please refer to [Shiro Web Configuration](#).



info

Pulse overrides the default Shiro behavior. For example, if you declare a new Shiro instance, rules or permissions will not apply.

Using SSL in Pulse



Must change the default certificate

Pulse bundles a self-signed certificate which allows users to connect to it through HTTPS. This can be found in the `conf/engine/jetty/` directory, comprising a keystore and a `jetty.xml` file. This certificate **must not be used in production** environments, as it is insecure. Learn how to set up a certificate in [Configuring an SSL Certificate for Pulse](#).

-



- Single Sign-On

On this page

Single Sign-On

Was this page helpful? 

Single Sign-On (**SSO**) is a functionality that allows users to access various applications (*i.e.* to start a session) by signing in to only one of the applications. This eliminates the need for users to set different passwords as well as to sign in to each application individually.

Another component associated with this functionality is allowing users to sign out from all the applications at once (*i.e.* without having to sign out individually from all the applications). This component is commonly referred to as Single Logout (**SLO**).

SSO in Pulse is done using Security Assertion Markup Language (SAML) as the protocol to authenticate and optionally authorize users that are provisioned by an external Identity Provider.

Introduction to SAML

SAML is an XML-based open standard for exchanging user authentication and authorization information between security domains. The primary use case of SAML is to enable **SSO**, which increases security, allows practical access to multiple applications and decreases administrative time and costs relative to the traditional option of signing in to each application individually.

SAML uses security tokens that contain assertions to communicate user information between a SAML authority, referred to as Identity Provider (IdP), and a SAML consumer, referred to as Service Provider (SP). At a high level, the IdP is the asserting party which is responsible for managing the identity information of users and to provide the information in form of tokens to the SPs. The SP corresponds to an application that requires user authentication to start an active session. The SP constitutes the relying party because it allows the user to sign in based on the information provided by the IdP. Note that the SP is not responsible for managing user credentials and does not even have access to the user password.

SSO in Pulse using SAML 2.0

SAML 2.0 is the current version supported by Pulse. In the context of **SSO** integration through SAML, Pulse acts as the SP, and the IdP is the server in which the user has an account and the user credentials are stored. There is a number of ways in which the IdP can be configured to manage user information, one of which is by a self-hosted Active Directory Federation Services (AD FS) server.

AD FS is a service provided by Microsoft as a standard role for Windows Server that provides web sign in using existing Active Directory (AD) credentials. In the case of **SSO** implemented in Pulse, the IdP of choice is AD FS; however, any other standard-compliant IdP service may be used instead (Pulse is agnostic to the implementation). Since the implementation of SAML is not dedicated to Pulse, Feedzai's Clients may have already a SAML architecture implemented before acquiring any of the Feedzai's products.

Regarding SSO, Pulse has two different types of users:

- **Non-SSO Users:** These users are created and their information is managed inside Pulse. They also [sign in using a Pulse form](#).
- **SSO Users:** External users that are able to [sign in into Pulse by using an external page](#) which is managed by the IdP. Depending on how the user **authorization** is configured in Pulse, **SSO Users** can be handled in two distinct ways:
 - If authorization is configured to be managed by the IdP (default behaviour), users can be created only by the **automatic user provisioning**. When a new user signs in for the first time, it does not exist as a **Pulse User** (i.e. is not yet manually configured in Pulse). This Pulse user is created on-the-fly after being authorized by the IdP. The user credentials and additional user information are managed by the IdP which communicates with Pulse to provide user authentication and authorization.
 - If authorization is explicitly configured to be managed by Pulse, SSO users can login only after they are created in Pulse. In this case, user authorization (roles, groups, etc) is entirely managed in Pulse, and the IdP manages only the authentication.

Learn more about [user provisioning](#) in Pulse.

Scenarios of SSO in Pulse

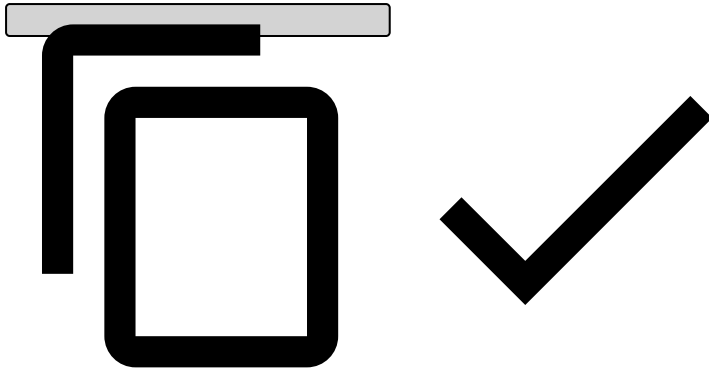
There is currently one supported scenario of **SSO** using SAML, namely the **SP-initiated** configuration.

In this case, when a user opens a Pulse URL, and is not already signed in, they are redirected to a URL that is managed by the IdP. In this IdP URL, the user is prompted with a sign-in form, and after being authenticated, they are automatically redirected back to the Feedzai URL. The user is then signed in to the application that they were accessing in the first place (i.e. Pulse URL) as well as to all the other applications that are configured as SPs.

More on SP-initiated configuration

Consider the scenario in which the user browses to a Pulse URL that they have saved as a bookmark, or was sent by email. E.g.:

```
http://pulseviews/management/#apps/datasciencedemo/datascience
```

The following steps explain sign in via SP-initiated:

1. User opens the Pulse URL.
2. The user is redirected to the URL of the IdP, which is configured in Pulse, by sending a SAML authentication request.
3. If the user does not have an active session, the user is prompted with a sign in form that is configured by the IdP and requires authentication (e.g. username and password).
4. The IdP builds the SAML authentication response in the form of XML containing the user's username and/or email address as well as other optional [provisioning data](#), signs it using an X.509 certificate, and sends the information to Pulse. Pulse retrieves the authentication response and validates it using the certificate fingerprint. The user is now signed in to Pulse.

Note that, if the user is already signed in when opening the Pulse URL (i.e. Step 1), Steps 2 and 4 still apply; however, the user is not prompted with the sign in form mentioned in Step 3. The user might notice the address bar flicking as a result of Steps 2 and 4, but no further interaction is required.

Learn more about [sign in via SP-initiated in Pulse](#).

User Provisioning

Depending on how Pulse is configured, user authorization settings can be managed by the IdP (default behavior) or by Pulse.

External authorization (using IdP)

Automatic user provisioning is a mandatory component of Pulse SSO for external authentication. It is the concept of creating users on-the-fly at the first time that they sign in without being previously created as Pulse Users. In the case of SSO Users, provisioning is the only option to create users, since they cannot be manually created using the Pulse UI first, and after configured as SSO Users. When an SSO User signs, Pulse updates its information regarding the referred user. User provisioning is done through a SAML assertion that is sent by the IdP to the SP. This assertion contains attributes that can be configured in IdP service of choice. The assertion can be used to send additional information beyond the user identity (i.e. username) which Pulse will map to the correspondent configurations in the User Settings. This includes **Full Name**, **Language**, **Groups**, **Roles**, and **Tenants**.

Internal authorization

If Pulse is set up to use internal authorization, then SSO users need to be created both in the IdP and in Pulse. That means that an IdP user is authorized to access Pulse only if a user with the same identifier is created in Pulse. The authorization properties of that user (roles, groups) are managed in Pulse, as with normal users. In this case, there is no need to configure authorization settings in IdP since it will be ignored by Pulse.

Sign out

Sign out is performed by the **SLO** component. Similarly to the sign in requests, the sign out comprises a LogoutRequest and a LogoutResponse.

Pulse supports sign out via **SP-initiated** - a user signs out from one of the SPs.

There are different ways to implement the SP-initiated sign-out; for example, the application can initiate the LogoutRequest to the IdP. The IdP communicates with all the SPs to sign out all the active sessions without requiring further user interaction. The SPs reply back with a LogoutResponse.

Session Management

According to the SAML 2.0 specification, there is an attribute that may be included in the assertion named `SessionNotOnOrAfter`. This attribute corresponds to the time instant, encoded in UTC, at which the IdP and the SP sessions must be considered ended.

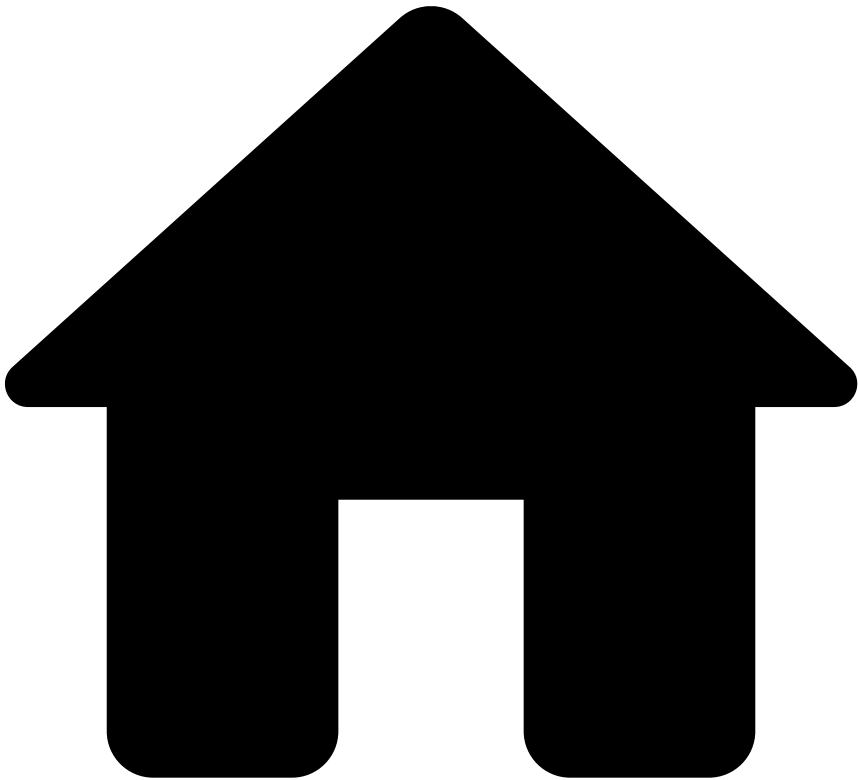
This is an optional attribute that can be added to the SAML assertion. If the attribute is not included, the time for the session to end is set by the same logic Pulse uses for Non-SSO user sessions. When the session is expired, the user can sign in via SP-initiated and continue the work at the point that was left when the session was expired.

Learn More

To expand your knowledge on the SSO functionality, visit the following pages:

- [Sign in via SP-initiated in Pulse](#)
- [Signing in as Pulse Administrator](#)
- [Configuring AD FS](#)

.



- Project Organization
- [App Collaboration Loop](#)
- Errors in App Collaboration

On this page

Errors in App Collaboration

Was this page helpful?

This page provides a list of errors that might occur when performing tasks associated to App Collaboration.

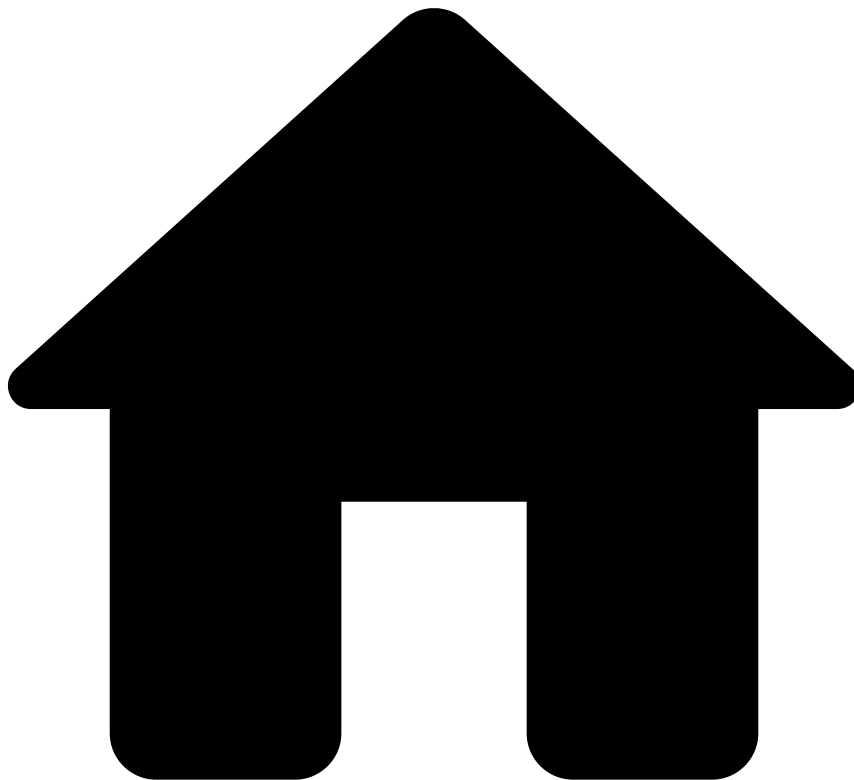
List of errors

See a list of error codes, descriptions and additional information in the following table:

Error Code	Error description	Additional information
APP_COLLAB_001	There is an existing List with the same name but it is not compatible	With App collaboration , all the Lists are exported when using the partial export option and imported in the target environment using the

Error Code	Error description	Additional information
	with the List contained in the zip file generated by the partial export (i.e. resource bundle).	partial import option. A List that does not exist in the target environment is created during the import. This error may occur when a List that is in the resource bundle already exists in the target with the same name and both Lists are not compatible to each other.
APP_COLLAB_002	A partial request configuration was not found with the given id.	This error should not occur under normal operation. The browser issues the export in separate requests, one to prepare the export and another to actually perform the export. This error means that the actual export request is not able to find the previous export preparation data.
APP_COLLAB_003	Partial import file not found or not readable.	This error should not occur under normal operation. It means that the uploaded zip file that was stored in disk can no longer be used in subsequent operations.
APP_COLLAB_004	Non-deployable Plans cannot be partial exported.	This error occurs because only deployable Plans can be exported.
APP_COLLAB_005	The Simple Rules Project must be exported with at least one snapshot.	
APP_COLLAB_006	The Simple Rules Project must be imported with at least one snapshot.	
APP_COLLAB_007	Binary Model requested but not found in the training folder.	This error occurs when the user selects to export the binary of the Model ; however, this binary cannot be found.
APP_COLLAB_008	Configuration for importing a Model must specify the destination Model Project id.	Model Projects are not created automatically when importing a Model , and thus a target Model Project must be specified in the import configuration.
APP_COLLAB_009	Workflow validation error.	When trying to change the Workflow element to reference an imported resource (Rules snapshot, Plan execution or Model) the Workflow became invalid. The work-around may be to import without specifying new mappings in the Workflow.
APP_COLLAB_010	The imported Plan becomes invalid using the local schema.	The Plan was valid with its source schema, but the user decides to import the Plan using the schema in the target environment, and the Plan becomes no longer valid. This might occur because some field referenced by the Plan does not exist in the target schema or the field type is not compatible.
APP_COLLAB_011	Cannot override a Plan of a different type.	This error occurs because only deployable Plans can be imported.
APP_COLLAB_012	Some of the resources selected for importing are missing in the zip file.	The import configuration is referencing resources that are not included in the zip file.

•



- Project Organization
- [App Collaboration Loop](#)
- How to Code Review

On this page

How to Code Review

Was this page helpful? 

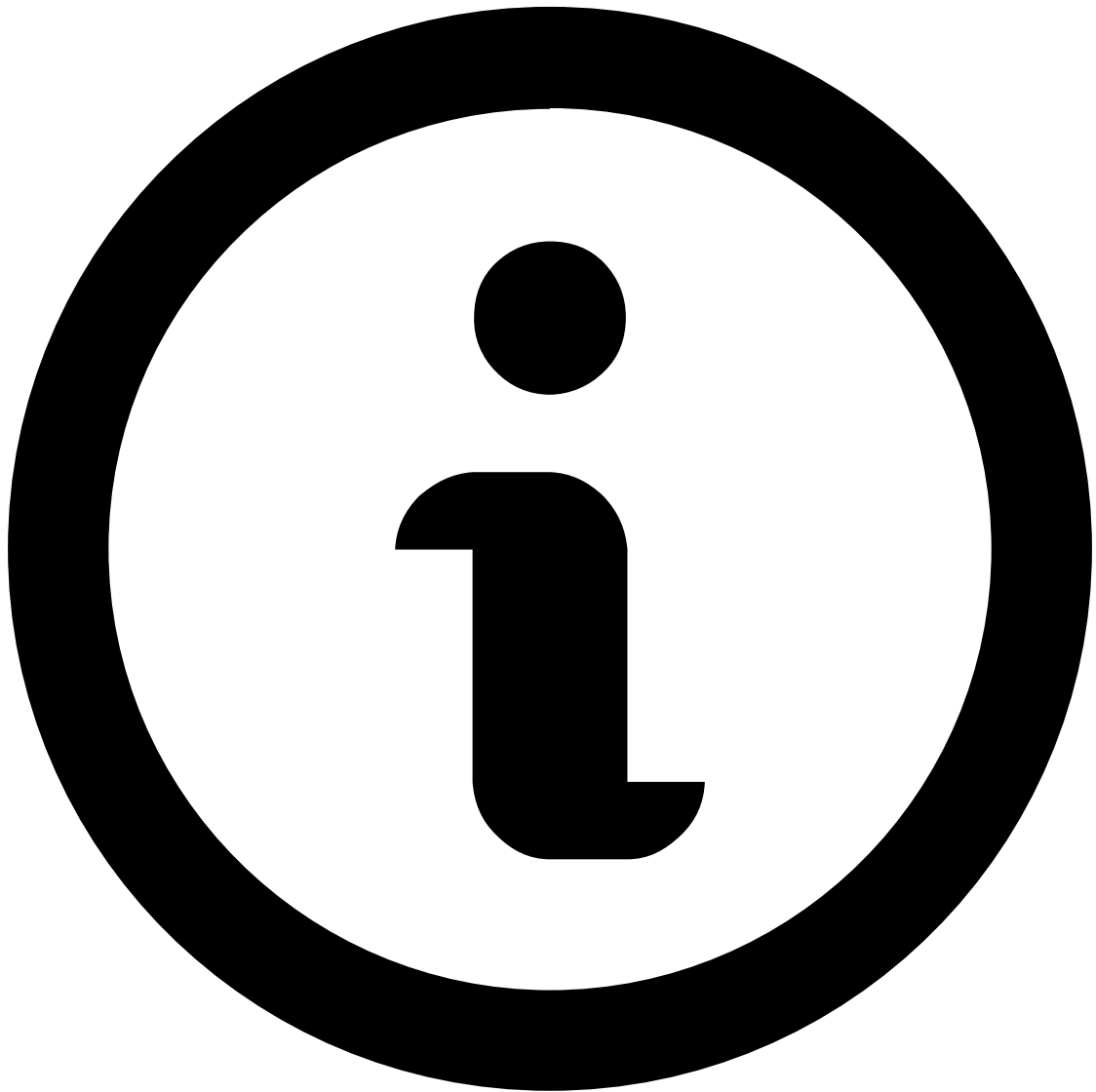
Why

Code review is an essential step to ensure that you deploy resources that meet the quality standards required in the field of Fraud Prevention. This guide provides you recommendations on how to accomplish this action.

How to

As an example, consider that you want to migrate a resource from **DS environment** to **DEV environment**. The recommended approach for sharing changes and perform code review comprises the following steps:

1. Perform a [partial export](#) of the relevant resources, for example, a deployable [Plan](#) in the **DS environment** (i.e. imagine that this step is executed on day 1).
2. Import the resources in the **DEV environment** (e.g. the deployable **Plan**).



note

The resources will keep evolving in the **DS environment**.

3. Perform a **partial export** of the same resource (i.e. the deployable **Plan**) in the **DS environment** (i.e. imagine that this step is executed on day 2).
4. Compare the partial exports of the resource between day 1 and day 2. To do so, there are two options:
 - Version the day 1 export in git and replace it with the day 2 export, and then send the changes to review using the code review tools supported by your project (e.g. [Differential](#)).

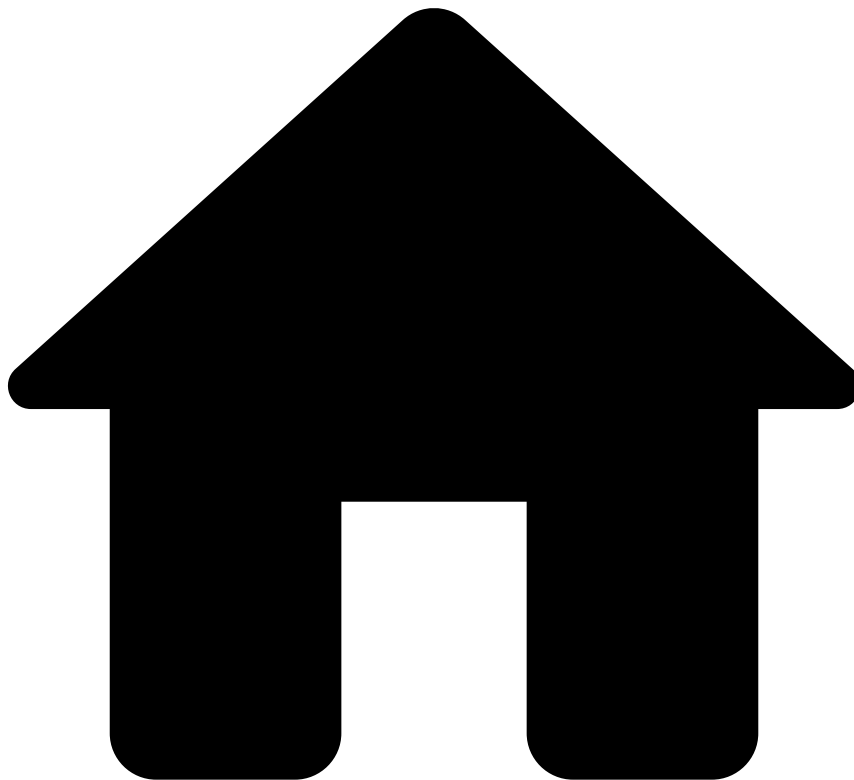
- Export the resource from the DEV environment and perform a diff against the resource exported in day 2. See the page [Comparing resource versions](#) for examples on how to perform this step.

5. If you agree with the changes, import the resource obtained in Step 3 in the **DEV environment**.

Next Steps

Now that you know the code review process, check the page [How to merge](#) to learn how to migrate your work to the target environment.

•



- Project Organization
- [App Collaboration Loop](#)
- How to Merge

On this page

How to Merge

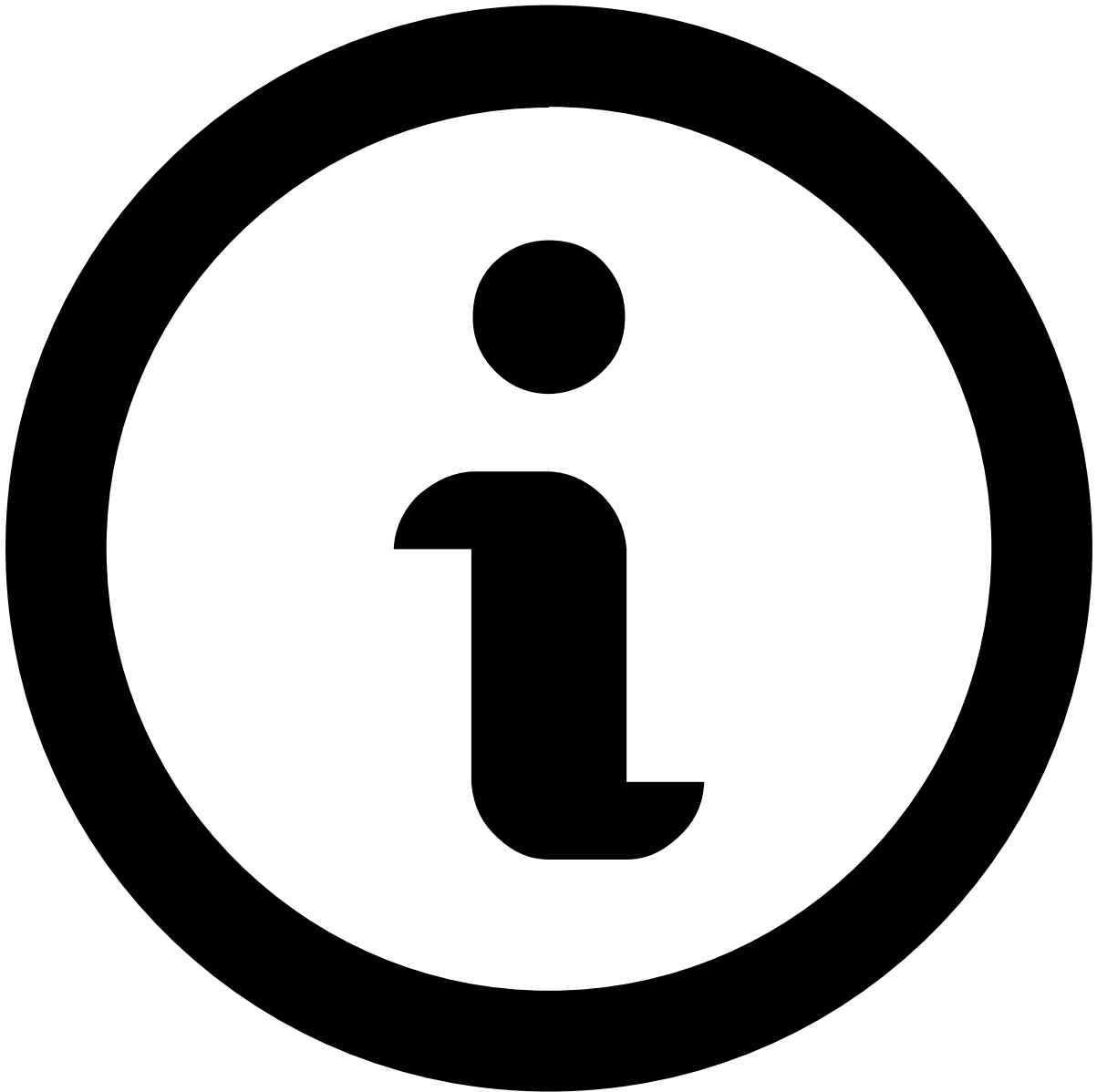
Was this page helpful? 

Why

The options [Export Resources](#) and [Import Resources](#) allow you to migrate Pulse resources from one environment to another. Sometimes you might need to merge the work developed by various teams in a single environment.

This guide applies when you want to merge more than one bundle containing changes that are in different folders among all the bundles (see page [Bundle structure](#) to learn how files are organized in a bundle). More specifically, this applies to the following situations:

- Merge changes of different bundles containing different Pulse resources. For example, one bundle file contains **Rules Projects** and another bundle file contains **Models**.
- Merge changes in different bundles containing the same type of resource but with different resource projects. For example, two bundles with **Rules Projects**: one of them contains a **Rules Project** named *Card Transactions* and the other contains a **Rules Project** named *Country transactions*.



Merge the same resource

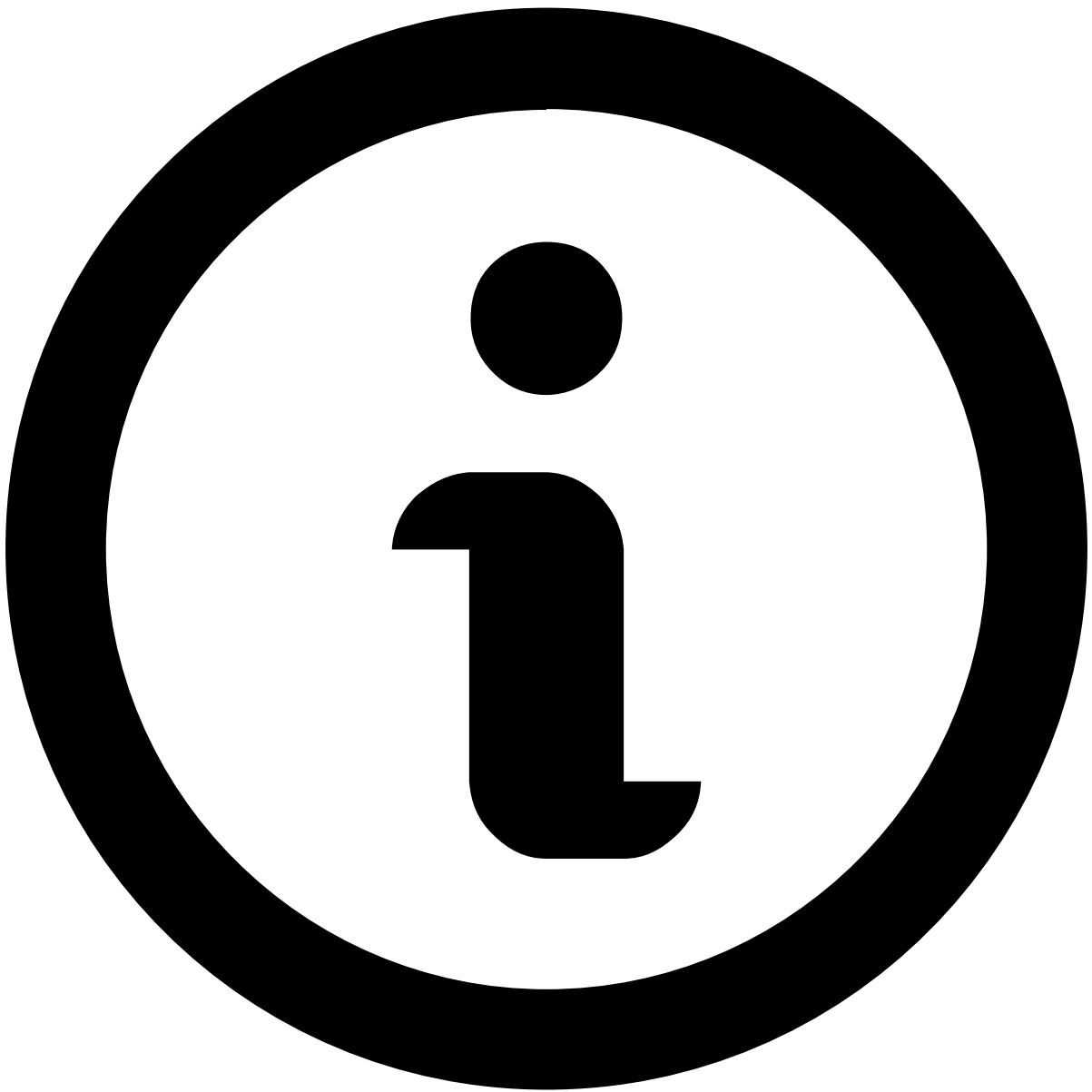
If you have more than one bundle containing the same JSON file that has evolved simultaneously in different environments (e.g. two files contain different **Rules** inside a **Rules Project** named *Card Transactions*), you might have to open the two JSON files to combine the changes. This operation is **not recommended** because it is extremely error prone.

How to

You have two methods to merge your work in a new environment:

- **Recommended approach:** Merging using Pulse UI.
- **Alternative approach:** Merging manually by copying and paste the folders in the bundle file.

Merging using Pulse UI



Recommended approach

Merging using Pulse UI is the recommended approach to minimize risks

If you are importing multiple bundles using the Import option, execute the following actions for each file you want to import:

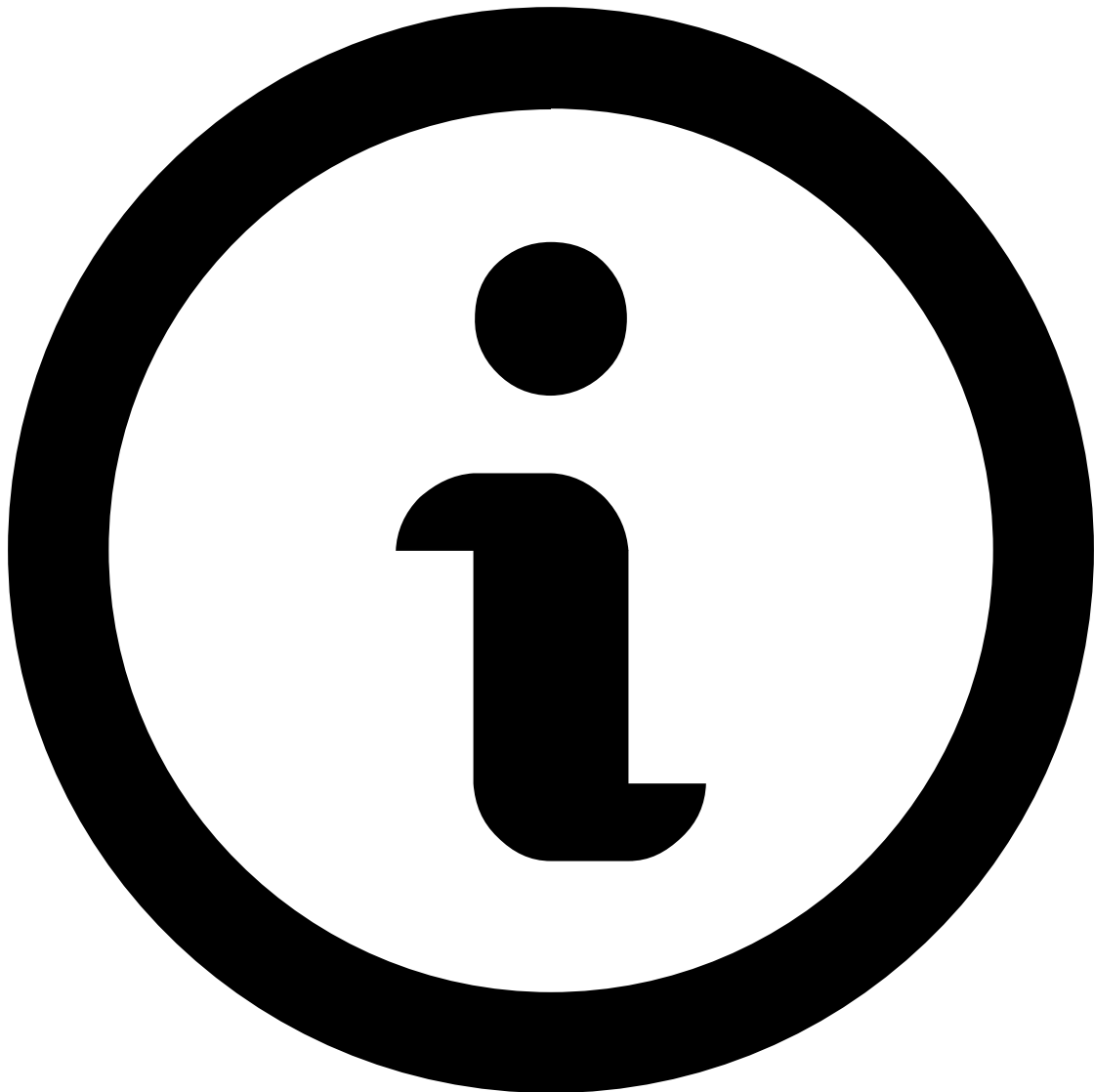
1. Select the **Pulse app** that will host the new resources
2. Select the button to open the application options:

3. Select the option Import Resources to open the import wizard.
4. Follow the instructions in the page [Importing Resources](#) to complete the steps required by the import wizard.
5. After importing, repeat Steps 2-4 to import the other bundle files.

Merging manually

If you choose to manually merge the file before importing the bundle, execute the following actions:

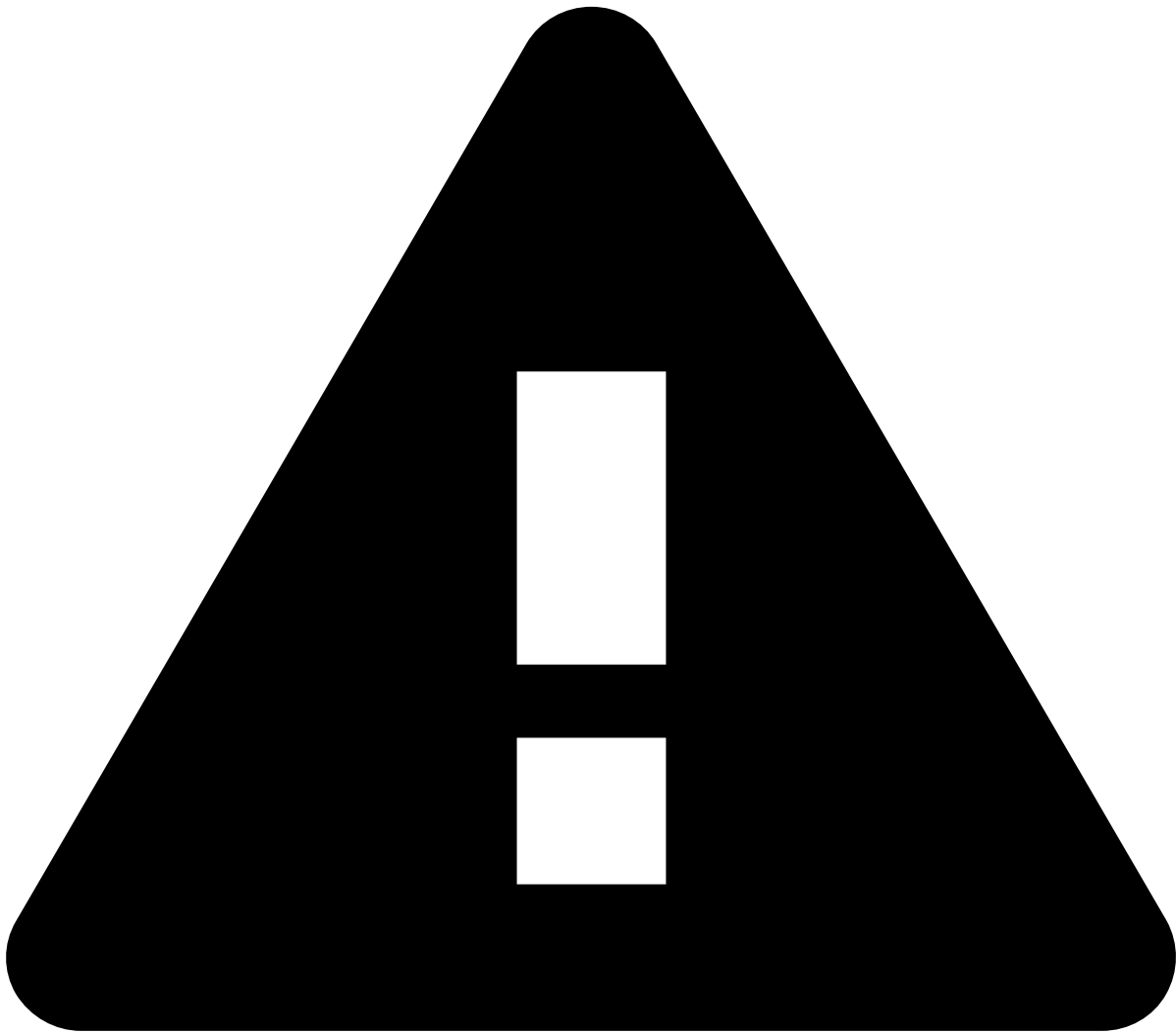
1. Copy and paste the JSON files of each bundle folder to a single folder on your operating system, following the [bundle structure](#).
2. Zip the folder of the resources to a final folder to be imported by the UI.



Zip the folders of the resources

You need to zip directly the folders of the resources rather than their parent folder. This is shown in the figure below:

3. Select the option Import Resources to open the import wizard.
4. Follow the instructions in the page [Importing Resources](#) to complete the steps required by the import wizard.



Approach not supported

Merging between JSON files has to be done manually and is not supported by the product.

When you have JSON files with different versions of the same resource resulting from evolving the same block in different environments, you cannot use the options described above to merge the changes. In this case, you need to open the JSON files and combine the changes manually.

The page [Comparing resource versions](#) provides typical examples of comparisons between changes within **Rules Projects**, **Models** and **Plans**. This examples give you an idea of what needs to be combine when merging JSON files.

However, you should keep in mind that merging changes in JSON files might require extra effort and planning. For example, if you merge a **Rule** into a **Rules Project**, there are a number of implications including:

- Define the **Rule** order.
- Create a new snapshot to include the new **Rule**.
- Add the snapshot to the runtime **Workflow**.

Next Stepsâ

Now that you understand how to merge files, take a look at the page [Errors in App Collaboration](#).

•



- Project Organization
- [App Collaboration Loop](#)
- How to Organize Your Work with Blocks

On this page

How to Organize Your Work with Blocks

Was this page helpful? 

Why

While developing new profiles with Plans, re-training Models with new data, and testing new Rules, using offline environments, your Pulse Application (Pulse app) will keep evolving in Production. By structuring your work with the concept of [Block ownership](#) you are allowing:

- Multiple teams to work in parallel using different environments.
- Easy and quick migration of resources between environments.

This page provides recommendations on how to set Pulse resources in the Data Science tab, and map them into the runtime Workflow using the concept of **Block ownership**.

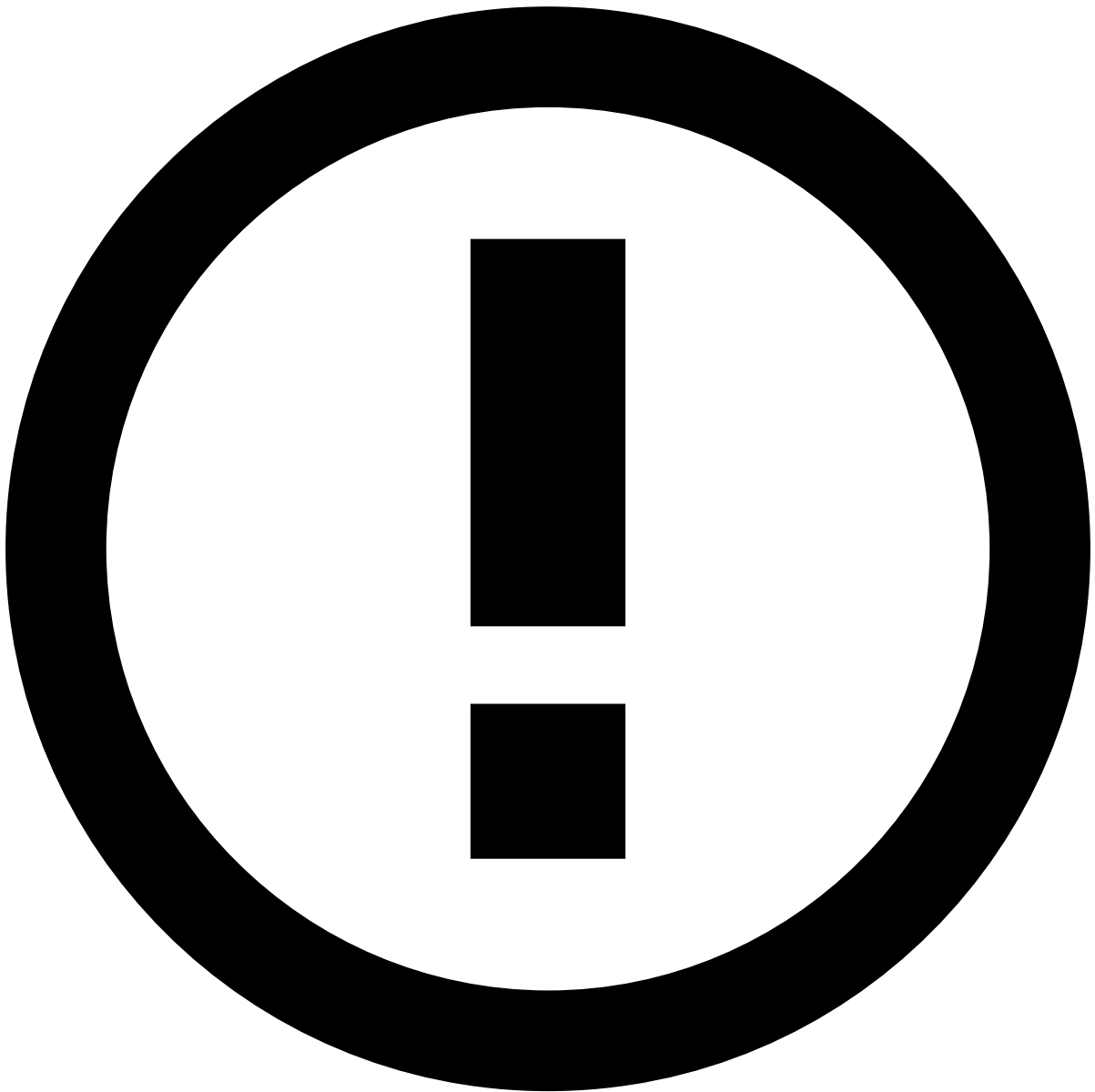
How to

This section guides you on how to combine resources developed in different environments into a single environment using a DEV environment as an example. Typically, a DEV environment contains the resources that are evolved by engineers, as well as the resources that are being evolved in PROD and in DS, which are recurrently [imported](#) to DEV.

Use-case example

Imagine that you have three environments with the following resources:

	DS Environment	DEV Environment	PROD Environment
Model	Evolving: Model Project named <i>Card Transactions Detection</i> .	Contains the Model evolved in DS	Contains the Model evolved in DS
Plan	Evolving: Plan named <i>Features Plan</i> .	Contains the Plan evolved in DS	Contains the Plan evolved in DS
Rules	Evolving: Two Rules Projects named <i>DS Rules Project 1</i> , <i>DS Rules Project 2</i> .	Evolving: One Rules Project named <i>Eng Rules Project 1</i> .	Evolving: One Rules Project named <i>Self-service Rules 1</i> .



Nomenclature

The names used in this scenario are given as examples to illustrate ownership. For example, in a real project, you will not have all the engineer Rules in the same Rules Project. In fact, you will most likely have the Rules grouped by logic in multiple Rules Projects. Therefore, the key point is to define which team will be the owner of each Rules Project (i.e. each block).

Mapping resources in DEV

To start working on resources and use them in runtime, you have to create the resources in Data Science tab and configure them in the Workflows tab.

Data Science tab

The figure below shows the example of a DEV environment with the following resources highlighted:

- The **Plan** *Features Plan* imported from DS (see Block 1).
- The **Model Project** *Card Transactions Detection* imported from DS (see Block 2).

- Four **Rules Projects**:
 - *DS Rules Project 1* imported from DS (see Block 3).
 - *DS Rules Project 2* imported from DS (see Block 4).
 - **Eng. Rules Project 1 evolved in DEV (see Block 5).**
 - *Self-service Rules 1* imported from PROD (see Block 6).

Workflows tab

The figure below shows the configuration of the runtime Workflow using the resources configured on the [Data Science](#) tab.

The key idea is to show you that each Project imported from a different environment and the Project that is being evolved in the current environment need to be mapped to a Workflow service (e.g. in this case four Rules Projects were mapped to four Rules services).

You can see the following Workflow services highlighted:

- One Plan service: Corresponds to the Plan *Features* (see Block 1).
- One Scoring service: Corresponds to the Model Project *Card Transaction Detection* (see Block 2).
- Four Rules services: Correspond to the Rules Projects:
 - *DS Rules Project 1* (see Block 3).
 - *DS Rules Project 2* (see Block 4).
 - **Eng. Rules Project 1 (see Block 5).**
 - *Self-service Rules 1* (see Block 6).

Check the page [Designing a Workflow](#) under the User Guide to learn how to configure a workflow.

Next Steps

Now that you know how to organize your Pulse App, learn [how to version and store](#) your exported files.

•



- Project Organization
- [App Collaboration Loop](#)
- How to Version and Store

On this page

How to Version and Store

Was this page helpful? 

Why

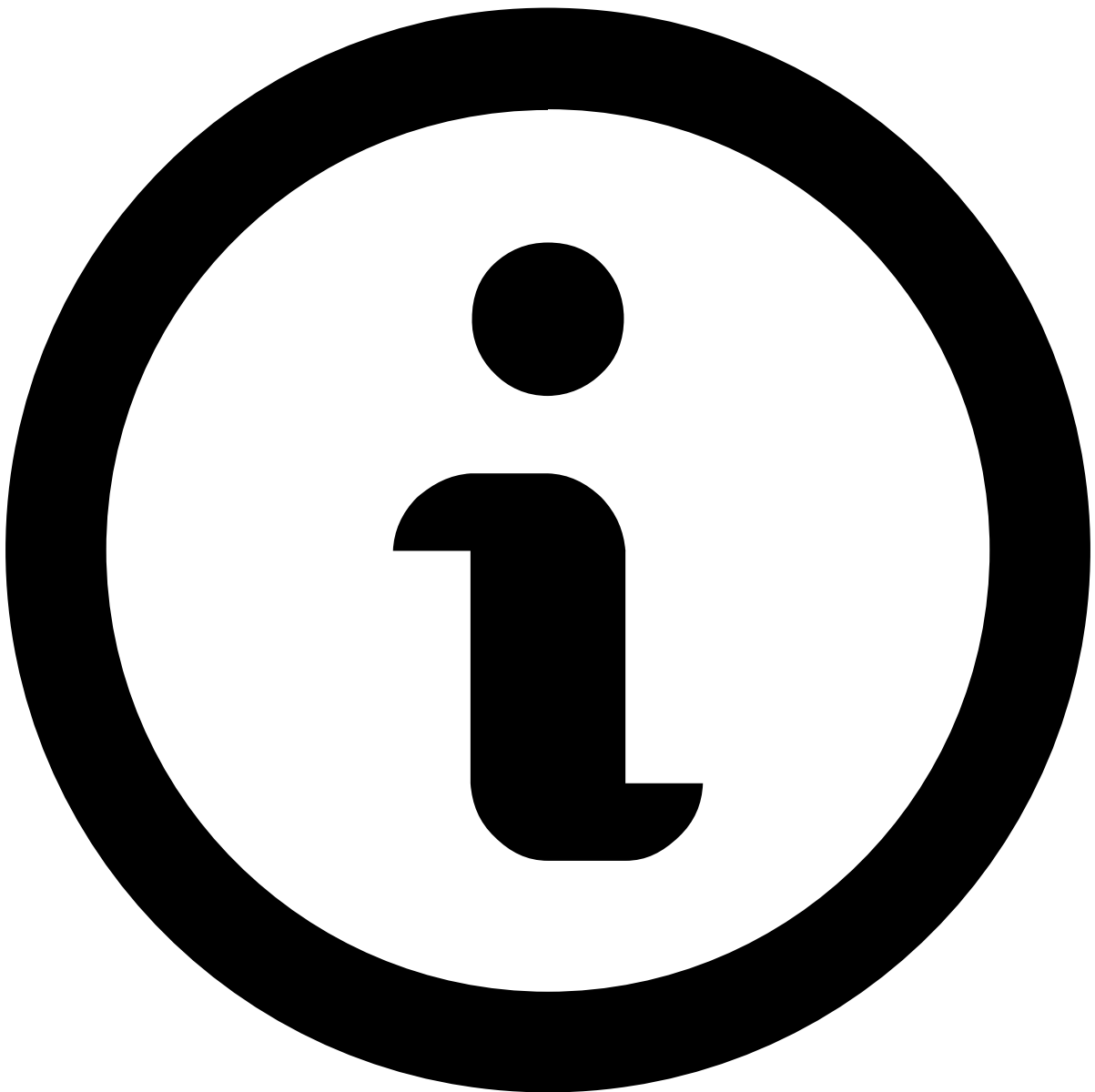
The main goals for versioning your work in a distributed version control system such as git are:

- To have a bundle of the application ready for deployment (stable versions that are promoted to Production environments may be tagged).
- To track the evolution of a Pulse application (**Pulse app**).

How to

The method of versioning you work depends on the project specifications; however, the recommended approach is to version the **Pulse app** as a whole. This is done by:

1. Generating the .papp file, for example by:
 - a. Using the option [Export Application](#).
 - b. Using the REST API;
2. Storing the file in git.



Limitations of this recommendation

There are a few aspects to have in mind associated to this recommendation as described below:

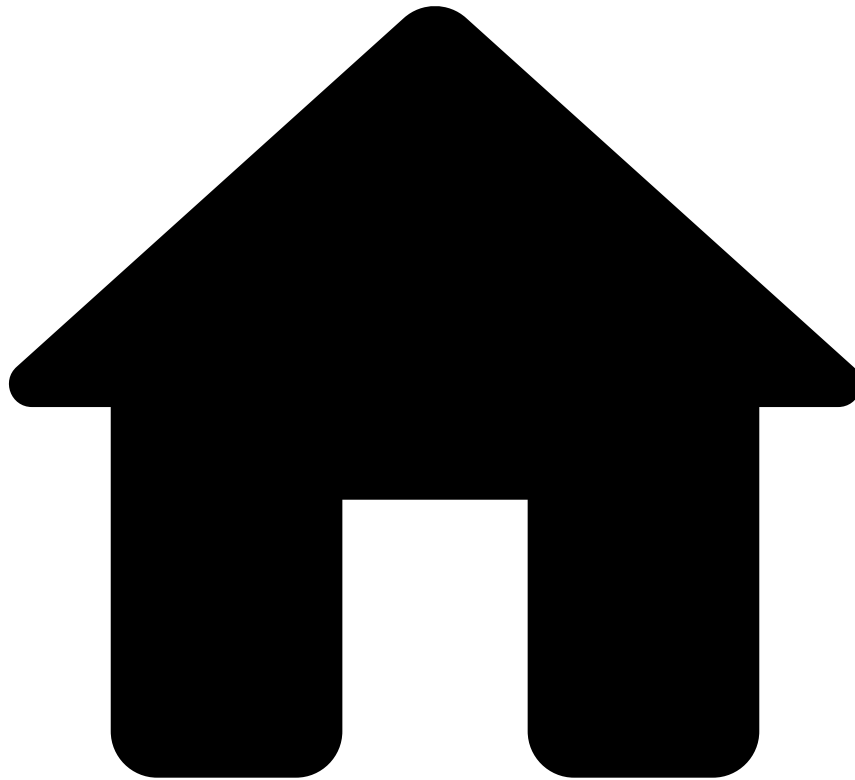
- The binaries of the data science **Models** are not exported with the .papp file, and thus, they binaries have to be versioned separately.

- The .papp file containing managed **Lists** with many items might become too large, and in this case, it might be preferable to store the .papp without **List** items.

Next Steps

Now that you know how to version and store your work, check the page [How to code review](#) to learn this process in the context of **App Collaboration**.

•



- Project Organization
- [Managing Users](#)
- Tenant User Management

Tenant User Management

Was this page helpful? 

For use cases that support multi-tenancy, it is possible for system administrators to delegate [user management](#) capabilities to specific users from their tenants.

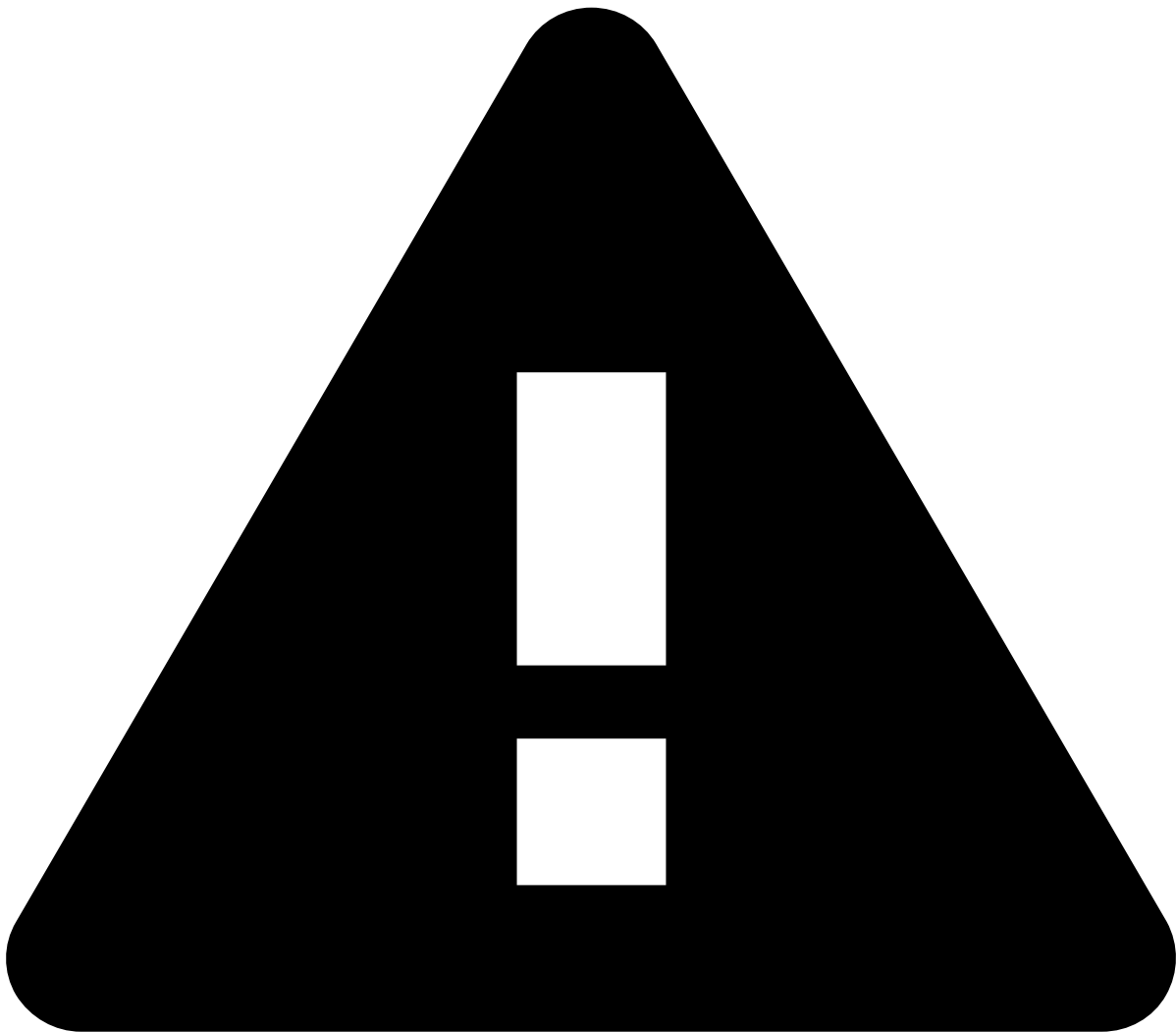
Such users need to have the `usermanager` role associated to them, and are referred to as **Tenant User Managers**.

Tenant User Managers have the following capabilities:

- They are **only able to see and manage users that are associated with the same tenant** or a subset of their tenants. This ensures that the Tenant User Manager can't create, see or change users from a tenant they don't belong to.
 - If a user is associated to a tenant that the Tenant User Manager is not associated with, then this user won't be visible to the latter. For example, if John is the User Manager for the tenants A, B and C, and Philip is a Fraud Analyst associated with tenants C and D; then, John can't see or manage Philip in the

User Management feature. Only a User Manager not associated with any specific tenant (typically a Landord user) can manage Philip.

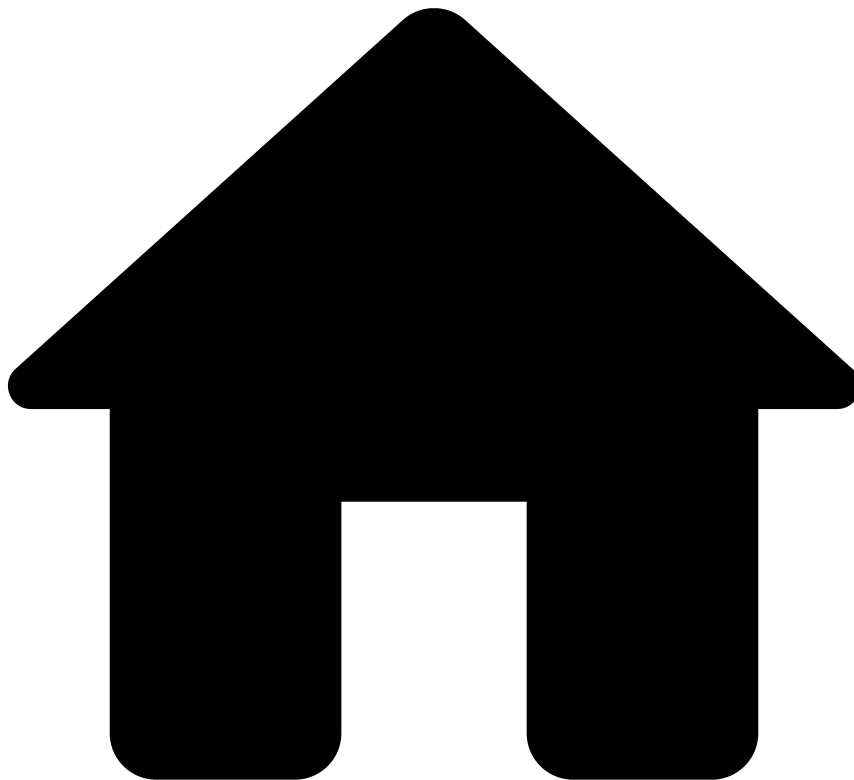
- They are able to update any field from the users listed on the previous point, with the following limitations:
 - **Tenants:** The values for this field must be among the tenants that the Tenant User Manager is associated with.
 - **Roles:** These must be selected from a list of tenant roles configured globally by the Feedzai Customer Success (CS) team for all tenants.
 - **Groups:** These must be selected from among the groups that the Tenant User Manager is associated with, or any of their sub-groups.
- They can create users. Such users must have a role and tenant, but assigning them to groups is optional.



caution

Prior to the introduction of the Tenant User Management feature, it was possible for a User Manager in a multi-tenant setup to be assigned to a tenant while remaining able to manage users from all tenants. This is no longer the case. For User Managers to be able to manage users from all tenants, they must not be assigned to any tenant themselves.

•



- Project Organization
- [Monitoring](#)
- Cluster Monitoring

On this page

Cluster Monitoring

Was this page helpful? 

Pulse allows monitoring the cluster through its UI. This topic explains how to set up notifications reporting status changes, how to analyse the cluster status, and how to change the leadership.

Examining the Cluster

The Pulse cluster is organized in data centers. The image below shows the **Cluster** tab . At the top of this panel there is a list of data centers (two data centers in this example).

Each line of this list includes the data center name and the the data center overall status, which in this case is *Healthy*, meaning that all nodes are running as expected.

However, the data center status may also be *Issue*, *Critical*, *Unreachable* or *Disabled*. *Issue* means that although there are some faults the data center is able to work properly. The *Critical* status is reported whenever there are faults and the data center cannot work properly. A data center is *Unreachable* when it cannot be reached usually due to communication problems. The *Disabled* status means that the data center was disabled manually.

The following table shows all data center statuses.

Status	Description
	All nodes are running as expected
	Although, not all nodes are running, the data center will work as expected
	There are faults and the data center cannot work as expected
	Cannot communicate with the data center
	Manually disabled for failover or maintenance

Select the data center name to show its details.

The image above shows the data center panel. At the top, we can see the data center information including the total number of nodes, the number of nodes down and the overall status. At the bottom it shows a list of cluster components. These components may be the pulse server or applications engines. Each line includes the component type, the number of nodes running, the number of nodes expected to be running (replication factor), a link to view the nodes (View Hosts), and the component status.

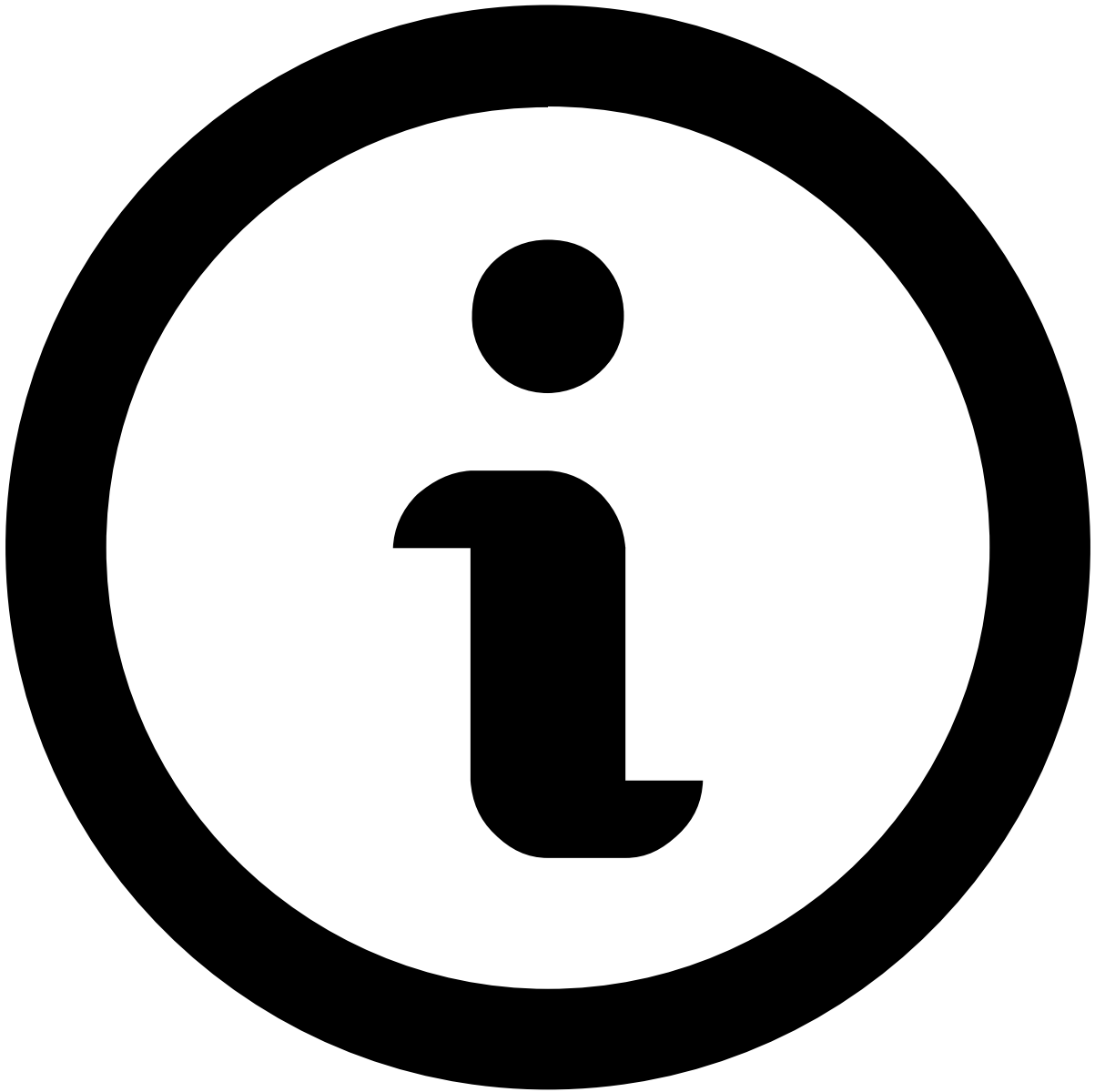
After clicking in a View Hosts link, it presents the list of nodes and its status as the image below illustrates.

In the example illustrated by the image above, there are three pulse server nodes and all of them are up (running). If by any chance a node fails. The cluster monitor will report it by changing the node icon. The image below shows the data center panel when a pulse server node is down.

Moreover, the Pulse Server component also changes from healthy to Issue, the data center nodes down counter is increased and the data center status icon changes to a warning sign meaning that, although there is a node down, the system will work as expected. Nonetheless, we should investigate the reason of this fault and then fixing and restarting the node if possible.

The node details may be consulted by clicking on the node IP/Port link. As an example, the image below shows the down node details.

The node details include: the node type, a pulse server in this case; the uptime/downtime, here we can see, as the node status is down, that the *downtime* is 13 minutes; the node status; the replication factor (the number of nodes expected to be running); the node leadership (whether if is the leader or not) and its location given by its IP address and port.



Status

Although, nodes only have two possible status (up and down), data center and cluster components may be:

- **Healthy**, meaning that all nodes are up;
- **Warning** (Issue), meaning that at least a node is up;
- **Critical**, meaning that at least one of the cluster component has all its nodes down.

The image below illustrates a critical status. In this case, application *rta1* is not running as all its nodes are down.

Setting Up Status Changes Notifications

Pulse allows sending email notifications reporting cluster status changes. These messages include reporting node status changes from UP to DOWN or DOWN to UP in both pulse server and application engine cluster nodes. To use this feature is important to have the SMTP settings properly configured. For example, for a Google SMTP account we would need to add the following configuration in **PULSE_HOME/conf/pulse.properties**:

SMTP Configuration Example

```
pulse.global.smtp.host=smtp.gmail.com

pulse.global.smtp.port=465

pulse.global.smtp.encryptiontype=SMTPS

pulse.global.smtp.user=<USERNAME>

pulse.global.smtp.password=<PASSWORD>

pulse.global.smtp.from=<SENDER_EMAIL>
```

In this example, `<USERNAME>` and `<PASSWORD>` stand for the user credentials and `<SENDER_EMAIL>` stands for the email one should see in the from field when a notification is received.

The cluster status changes notification recipients are configured in the UI by selecting the cluster tab in the Administration view illustrated by the first image in the previous section.

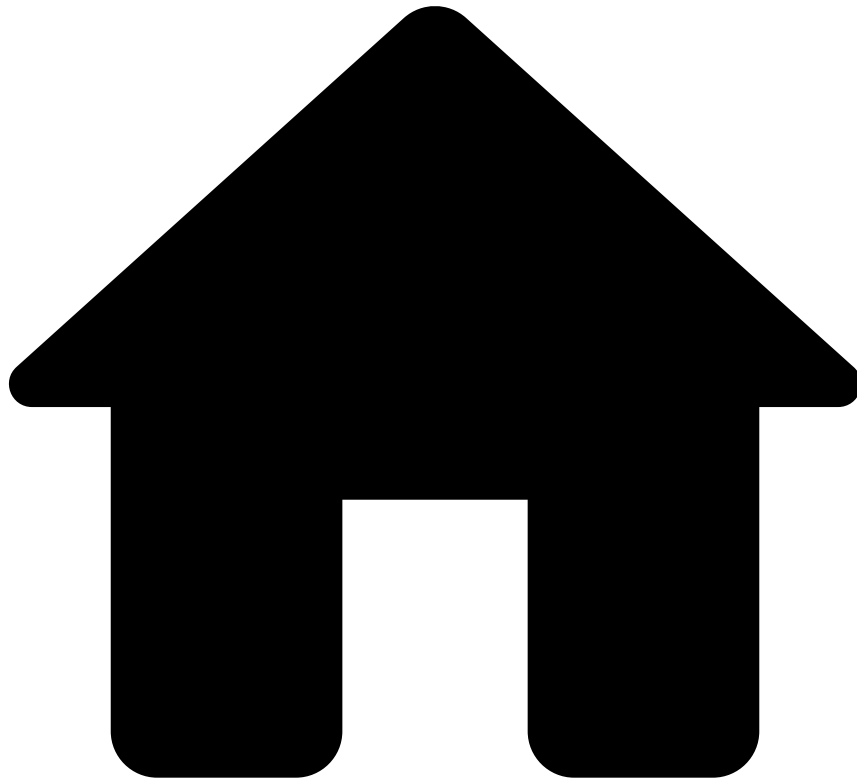
By clicking in the add button (represented by a plus sign), a new recipient may be included to receive status changes notifications.

Leadership change

You can request to change the leadership to another node in the node group (of Pulse, or of the application) by selecting the button next to the node that is the current leader.

Optionally, you can change the leadership status in the command line tool or via the Application Management REST API.

•



- Project Organization
- [Monitoring](#)
- JMX and Notifications

On this page

JMX and Notifications

Was this page helpful? 

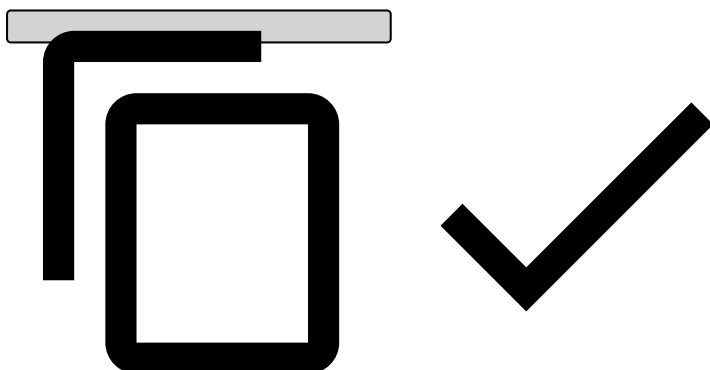
Feedzai Pulse provides a way of notifying monitoring teams when some indicators reach a specified threshold. Moreover, it exposes JMX Beans to extract JVM-related metrics meaning that it can be integrated with third-party monitoring tools such as Nagios.

Notifications

Monitoring teams can configure the system to send notifications when certain thresholds are surpassed. The configuration can be global to all applications running in the server or specific per application and both must be done in **PULSE_HOME/conf/pulse.properties**.

Global configuration

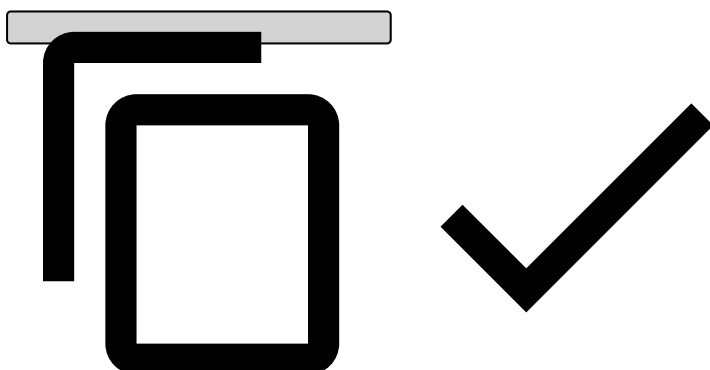
```
pulse.global.appengine.sendemailonfailure=true  
pulse.global.appengine.monitoringemailaddress=[email_protected]  
pulse.global.appengine.heapthreshold=<MEMORY-IN-MB>  
pulse.global.appengine.permgenthreshold=<MEMORY-IN-MB>
```



In order to accomplish the same per application one must change `global` to `app.<APP-NAME>` .

Per-application configuration

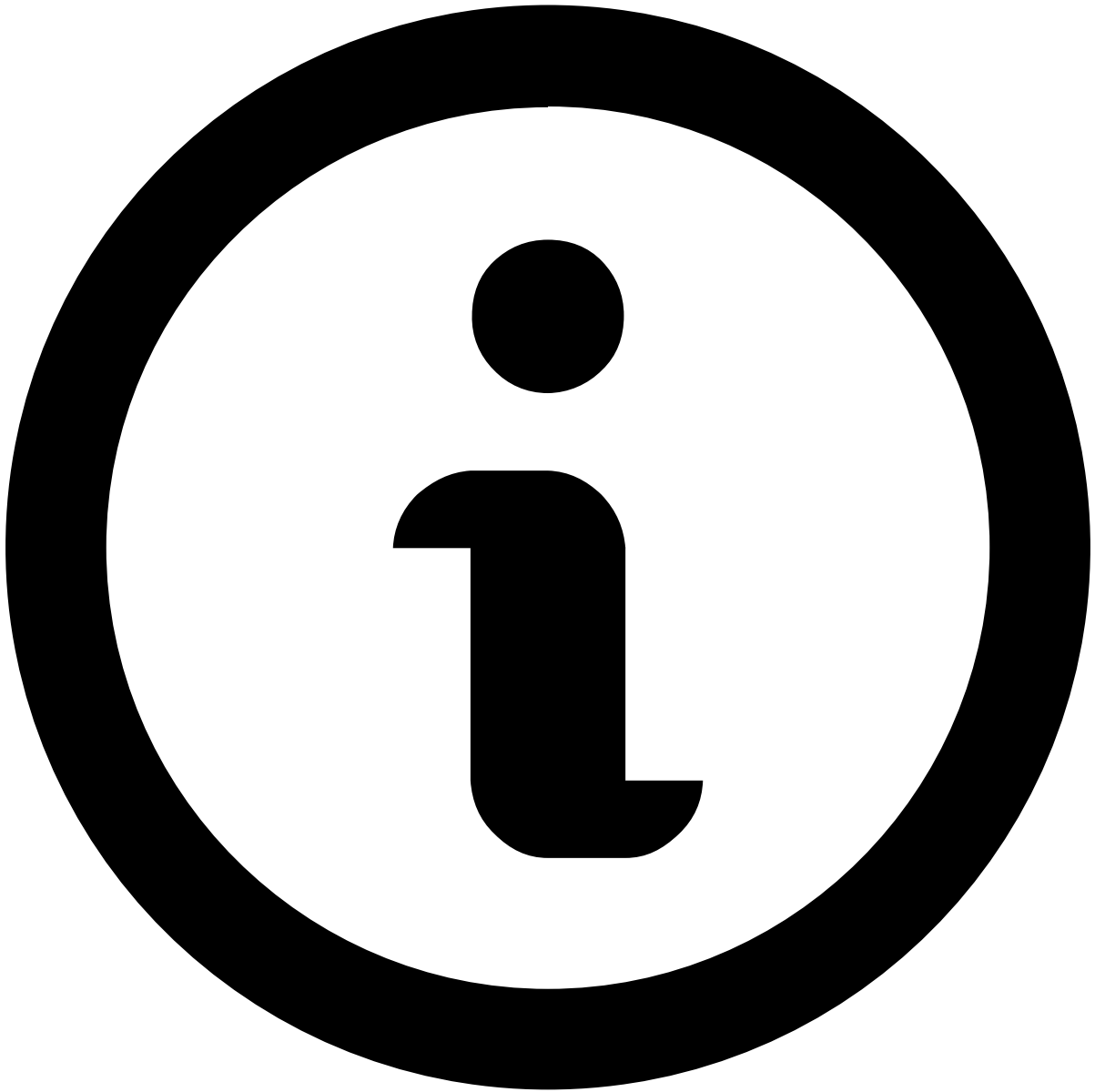
```
pulse.app.<APP-NAME>.appengine.sendemailonfailure=true  
pulse.app.<APP-NAME>.appengine.monitoringemailaddress=[email_protected]  
pulse.app.<APP-NAME>.appengine.heapthreshold=<MEMORY-IN-MB>  
pulse.app.<APP-NAME>.appengine.permgenthreshold=<MEMORY-IN-MB>`
```



The property names are quite self-explanatory so we won't get into much detail with them.

When the amount of memory used reaches the specified threshold, an e-mail is sent to the specified e-mail address informing about the occurrence. If you don't want a particular metric to be monitored, you can delete the entry or assign to it a negative value.

This configuration also notifies when applications crash. That kind of notification cannot be turned off once "sendemailonfailure" is set to "true".



note

Note that e-mail notifications will only be sent if the application crashes while running. If a crash occurs during a publish, or while the application is stopping, a crash notification e-mail is not guaranteed to be sent.

JMX

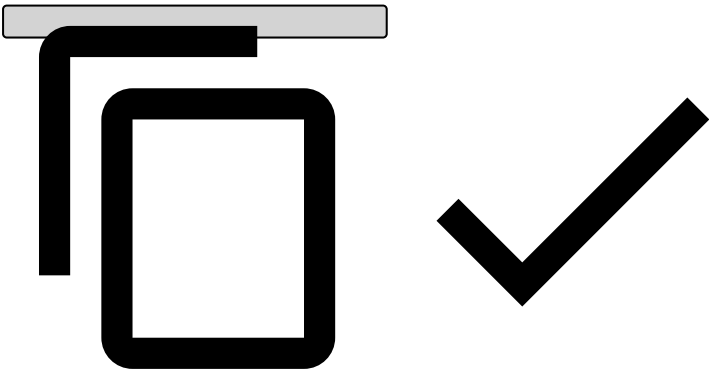
Feedzai Pulse exposes JMX interfaces so that monitoring information can be retrieved and some operations can be performed. The JMX interface is only available when in the same host (localhost). It can be accessed both programmatically or using tools like [IBM Tivoli](#), [Nagios](#).

If a quick and dirty option is needed, even [JConsole](#) can be used to access the JMX endpoints available on Pulse. The following figure shows some of the information available when connected to the Pulse server running a Fraud Prevention application:

Programmatic access

It is possible to access Pulse's JMX endpoints using Java by getting a reference to the `MBean` . In order to accomplish that one must do the following:

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi://localhost:9010/jndi/rmi://localhost:9010/jmxrmi");
JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();
ObjectName mbeanName = new ObjectName(StatusConsole.MBEAN_OBJECT_NAME);
StatusConsoleImplMBean console = JMX.newMBeanProxy(mbsc, mbeanName, StatusConsoleImplMBean.class, true);
```



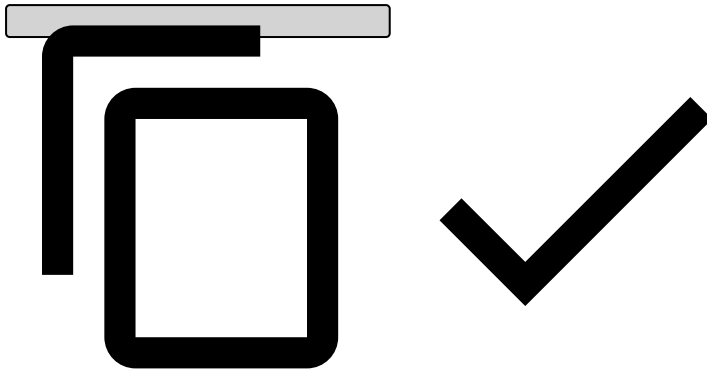
Having the reference, calls to the MBean can be made just by calling the desired methods. The following methods are available:

Method	Return	Description
<code>getName()</code>	String	The console name: StatusConsole
<code>getUptime()</code>	String	A string representation of the uptime
<code>getStartDate()</code>	Date	The start date
<code>getJavaVMVendor()</code>	String	The VM vendor
<code>getJavaVMVersion()</code>	String	The VM version
<code>getOSName()</code>	String	The name of the Operating System
<code>getOSArch()</code>	String	The architecture of the host
<code>getOSVersion()</code>	String	The version of the operating system
<code>getClusterNodes()</code>	List<String>	The list of cluster nodes that compose that Pulse instance. Contains information such as the host, port and whether the instance is hot (the running instance) or not
<code>startShutdown()</code>	Void	Orders Pulse Server to gracefully shutdown itself

The JMX endpoint described above is related to Pulse Server. It's also possible to access information about the running applications. For that, a lookup to the specific AppEngine must be made. This can be done by specifying the data center ID, the application slug (name), the instance IP address and RMI Port as in the following example:

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi:///jndi/rmi://localhost:6990/jmxrmi");
JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();
ObjectName mbeanName = new ObjectName(AppStatusConsoleImplMBean.BASE_MBEAN_OBJECT_NAME + "<DATA_CENTER_ID>,nodeType=Apps,app=<APP-SLUG>,node=<HOST>-<PORT>");
```

```
AppStatusConsoleImplMBean console = JMX.newMBeanProxy(mbsc, mbeanName,
AppStatusConsoleImplMBean.class, true);
```



After getting the reference, the following methods will become available:

Method	Return	Description
<code>getName()</code>	String	The name of the console: AppStatusConsole
<code>getUptime()</code>	String	The uptime of the application
<code>getStartDate()</code>	Date	The start date of the application
<code>getAllocatedHeap()</code>	double	The total amount of memory in the JVM (Mb)
<code>getUsedHeap()</code>	double	The amount of used memory in the JVM (Mb)
<code>getInitialHeap()</code>	double	The amount of memory that the JVM initially requests from the OS for memory management (Mb)
<code>getMaxHeap()</code>	double	The maximum amount of memory that can be used for memory management (Mb)
<code>getInitialPermGen()</code>	double	The amount of PermGen that the JVM initially requests from the OS for memory management (Mb)
<code>getMaxPermGen()</code>	double	The maximum amount of PermGen that can be used for memory management (Mb)
<code>getUsedPermGen()</code>	double	The amount of PermGen used (Mb)
<code>getTotalCPU()</code>	double	The CPU usage of all running processes, as a percentage of all processors.
<code>getProcessLoad()</code>	double	The CPU usage of the process running this App Engine, as a percentage of all processors.
<code>getActiveServices()</code>	List<String>	The list of service names running in the application

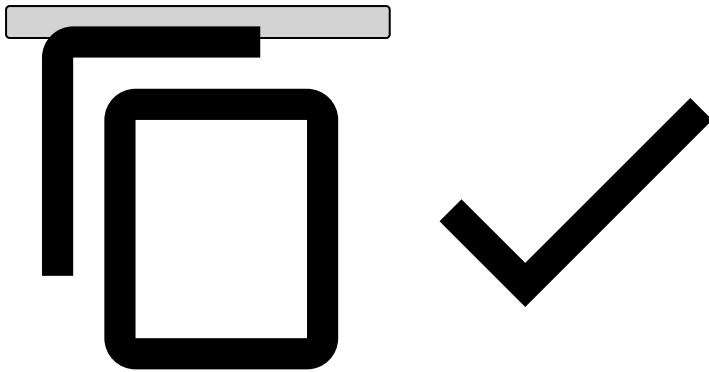
For a reference with the difference between "allocated", "used", "initial" and "max" Heap and PermGen memory spaces, refer to the following [StackOverflow answer](#).

Configuring the JMX Server

It's sometimes required to configure Pulse's JMX server due to firewall restrictions.

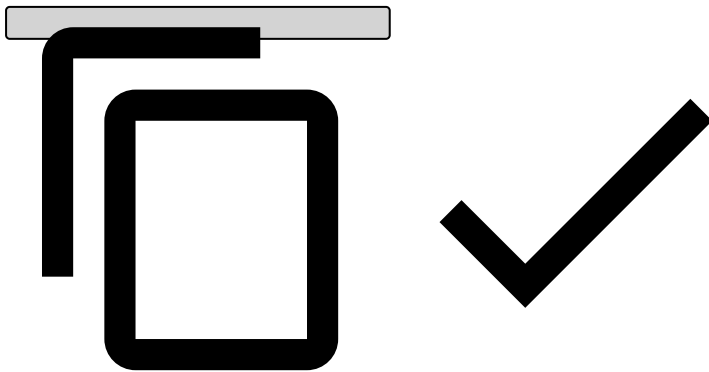
By default Pulse uses port 9010 for JMX access, this can be configured in Pulse's properties:

```
pulse.manager.jmxport=9010
```



After configuring the port you can use the following JMX URL to connect to:

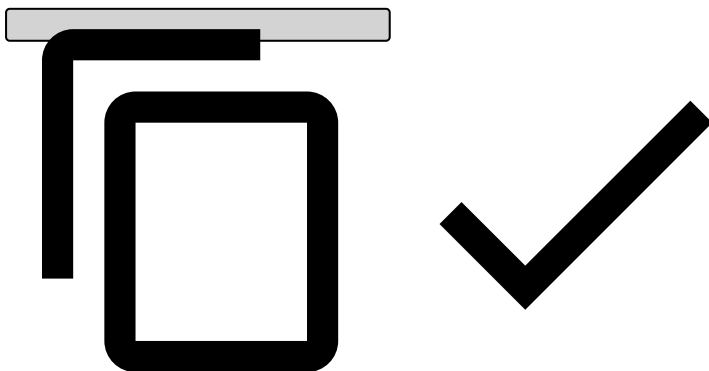
```
service:jmx:rmi:///<listenAddress>:<jmxport>/jndi/rmi:///<listenAddress>:<jmxport>/jmxrmi
```



As for Application Engines, the default port used to establish the connection is the same as the RMI port. However, a random port will be automatically generated for the actual JMX connection. If you want to use a fixed port you must configure it:

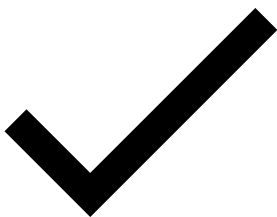
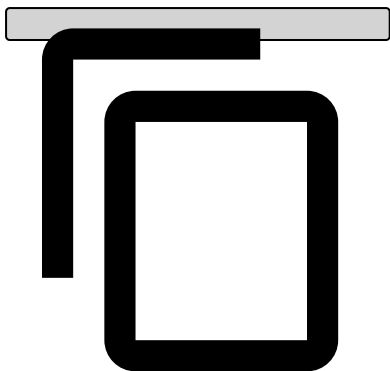
```
#Global setting for all applications
pulse.global.appengine.jmxport=9011

#Configuration for a specific application
pulse.app.<app_slug>.appengine.jmxport=9013
```



In this case the JMX URL will be:

```
service:jmx:rmi:///<listenAddress>:<jmxport>/jndi/rmi:///<listenAddress>:<jmxport>/jmxrmi
```



•



- Project Organization
- [Monitoring](#)
- Logging

On this page

Logging

Was this page helpful? 

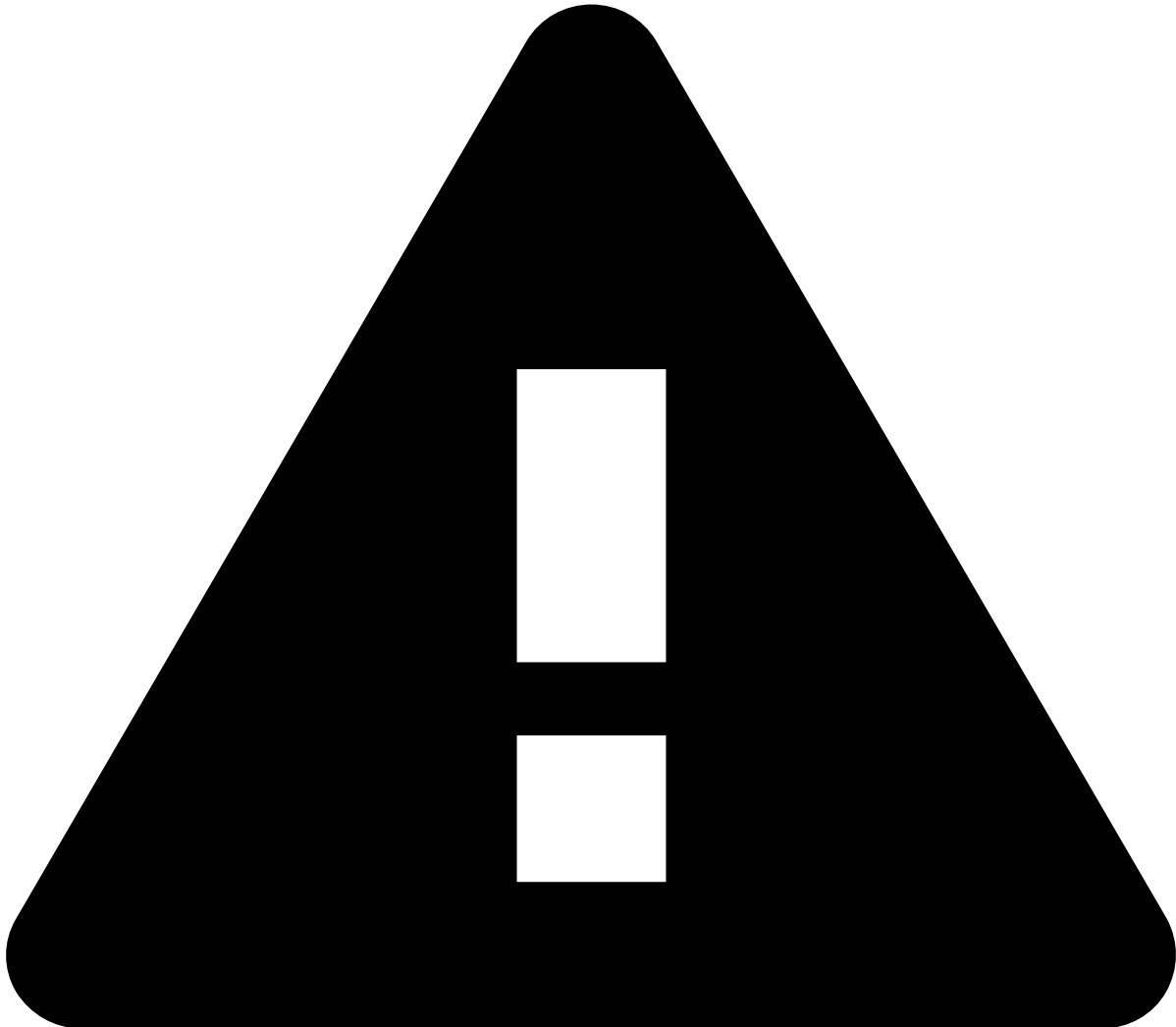
All operations executed internally by Pulse are logged according to several levels of detail in log files located in the `PULSE_HOME/log` directory.

Pulse uses [SLF4J](#) logging facade making all the log calls agnostic to the actual logging framework in place. This allows changing the logging framework at deployment time if necessary. Nevertheless, Pulse is distributing [Logback](#) logging framework by default and this page discusses how to configure it.

Configuring⚠️

The logging configuration file is located in `PULSE_HOME/conf/logger.xml`. This file includes different configuration parts from files present in `PULSE_HOME/conf/logger/*`.

For a more advanced configuration not covered in this article please consult the [Logback official documentation](#).



warning

Changing the logging configuration is on one's own responsibility. Changes might result in the loss of valuable information for investigating problems. If the default configuration is changed, support teams might not be able to help so one should be extremely careful when editing this file.

Logging configuration files⚠️

As mentioned above, the main logging configuration file is located in **`PULSE_HOME/conf/logger.xml`**. This file shouldn't be changed by clients as it does the inclusion of other logging configuration files present in **`PULSE_HOME/conf/logger/*`**:

- **`jmxconfigurator.xml`**: Enables exporting logger configurations via JMX.
- **`filters.xml`**: Defines filters to be applied to log messages.
- **`conversion-rules.xml`**: Formatting rules for log messages.
- **`default-appenders.xml`**: Defines the default appenders enabled in Pulse. **Clients in production should change this file** to disable BUFFER and STDOUT appenders.

- **logger-jupyterlab.xml**: Logger and appender configuration to send all JupyterLab logs to its own file - *log/jupyterlab.log*
- **logger-confidential.xml**: Logger and appender configuration to send all confidential logs to its own file - *log/CONFIDENTIAL-*.log*
- **logger-metrics.xml**: Logger and appender configuration to log probes to disk.
- **logger-legacy-audit.xml**: Logger and appender configuration to log legacy audit messages to disk.
- **logger-audit.xml**: Logger and appender configuration to log new audit entries to disk.
- **logger-events.xml**: Logger and appender configuration to log events to disk.
- **logger-rules.xml**: Logger and appender configuration to log rules to disk.
- **logger-default.xml**: Default logger configurations that are shipped with Pulse.
- **logger-project.xml**: File that needs to be created by clients if they need to override logger definitions that are shipped with Pulse or if they need to add new logging configurations such as appenders or loggers.

Log Files

In order to easily pin point possible issues when running Pulse, Pulse Server and Applications write their messages to different log files.

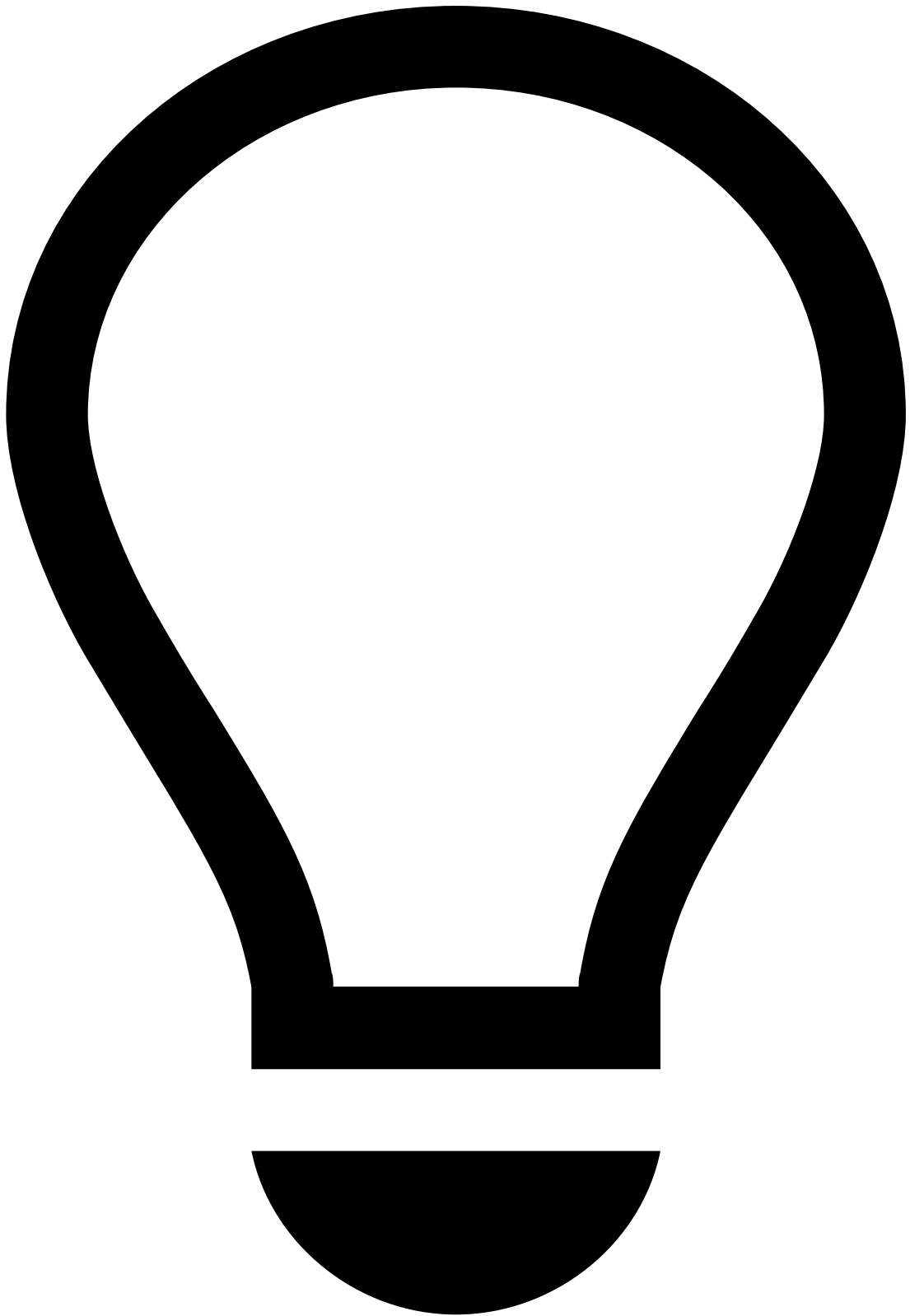
Whatever the configuration in place might be, Pulse will always generate the following:

- **pulse.log**: Logs all operations performed inside Pulse server
- **app-<APP NAME>.log**: Logs all operations performed in an application engine

For additional GC (Garbage Collection) logs, the following flags were added by default to the JVM_OPTS of the Pulse and the applications:

- **-Xloggc:log/pulse-%pid.gc -XX:+PrintGCDateStamps -XX:+PrintGCApplicationStoppedTime**

They will appear in a format similar to **<pulse|APP_SLUG>_GC_pid<PID>_<DATE>_log.0.current** inside the logging folder.



tip

If the JVM is already running and it did not happen to be started with the necessary JVM_OPTS, it is still possible to enable some of them dynamically using the jinfo tool, e.g.:

- **jinfo -flag +PrintGCDateStamps <pid>**
- **jinfo -flag +PrintGC <pid>**

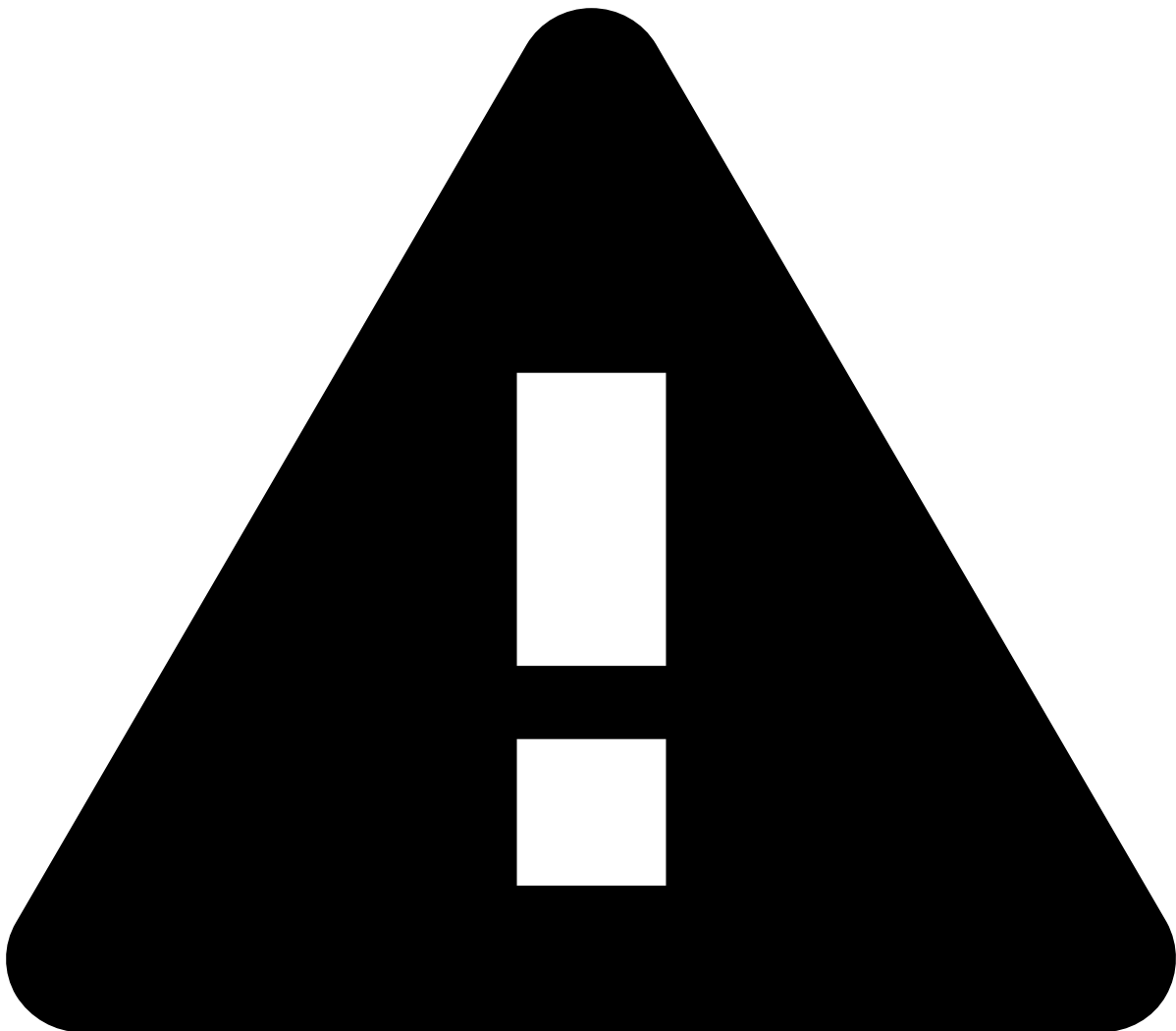
DEV Marker

The development marker provides a much refined logging useful when developing Pulse Services and web applications. To turn it **on** or **off** just set the **OnMatch** tag to **ACCEPT** or **DENY** accordingly.

Appenders Section

The appenders section define where the logging messages will end up. By default, Pulse is configured with three appenders: **STDOUT**, **FILE** and **BUFFER**.

- **STDOUT**: Defines how messages are presented in the standard output (console);
- **FILE**: Defines how messages are presented in the log file (for both server and applications). By default these files are rolled every week (each week will be in a zipped file) and a maximum of 8 week worth of history is kept;
- **BUFFER**: Defines how messages are presented in the UI.



Do not use stdout append in PROD

It is *not* recommended to have the STDOUT appended in production scenarios as it may cause the app-engines to block in some extreme error scenarios (for example [REQ-2666](#)).

An appender is defined in the configuration file with the `<appender>` XML tag. Inside each appender tag, there are several options mainly the **filter** and the **encoder**.

- **FILTER**: Defines the minimum logging necessary (inclusively) in order for the encoder to be used;

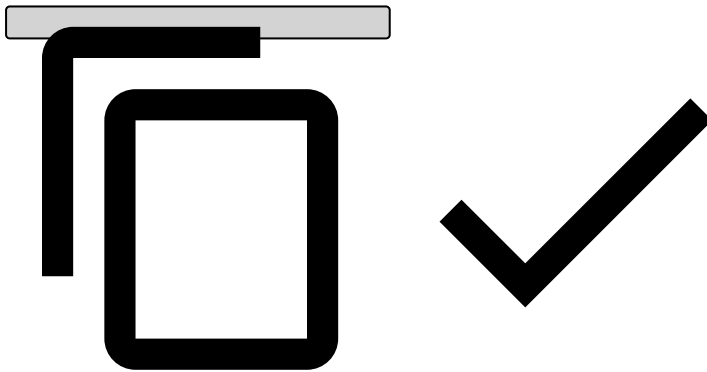
- **ENCODER**: Defines tags and colors for the logging messages given the package of the logger in use.

Under **filter** all available logging levels can be used (**ERROR**, **WARN**, **INFO**, **DEBUG** and **TRACE**). When you define a logging level, that level and the above ones (more critical than the one defined) will always satisfy the filter. For instance if you define **INFO**, then the **WARN** and **ERROR** messages are also logged.

With the encoder it's possible to define the timestamp format of the logging events and tag them with colors and labels. The following is an example of tagging the all the messages that have to do with the rules engine.

```
<encoder class="ch.qos.logback.core.encoder.LayoutWrappingEncoder">
  <layout class="com.feedzai.pulse.server.manager.logging.LoggerLayout">
    <colored>true</colored>
    <!-- Abbreviate the package -->
    <abbrevLevel>true</abbrevLevel>
    <!-- Show stacktrace -->
    <showTraces>false</showTraces>
    <!-- Color of the levels -->
    <debugColor>1;30</debugColor>
    <infoColor>0</infoColor>
    <studioColor>0;35</studioColor>
    <tagColor>1;30</tagColor>
    <!-- Timestamp format -->
    <timestampFormat>dd-MMM HH:mm:ss</timestampFormat>

    <!-- Tag the events the match the package with text -->
    <tagEvents>
      <from>com.feedzai.pulse.service.rules</from>
      <with>RULES</with>
      <color>1|1;33</color>
    </tagEvents>
  </layout>
</encoder>
```



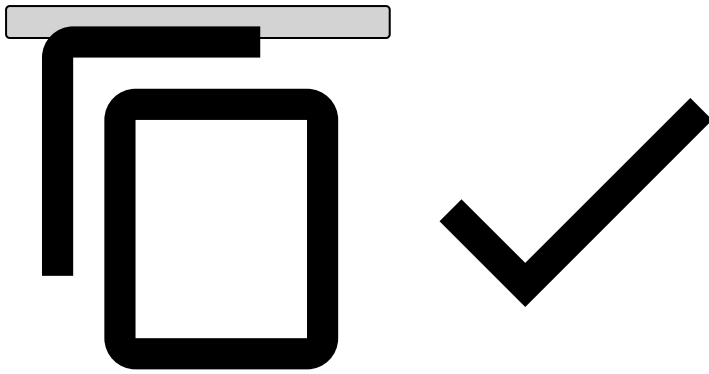
Logger Section [\[1\]](#)

The logger section is where the main level filtering is made. Here levels are assigned to packages or even to their classes using the **name** property. This configuration can be overridden ultimately in the final appenders. For instance if you define here that a logger is enabled at the **DEBUG** level, but an appender is specified to only allow above **INFO**, the messages won't appear in that appender.

It's also possible to assign a logger some configuration. For instance you might want for all the logging messages of your application to go inside a specific file. For that you could create an appender and forward all the messages to go there.

```
<appender name="MY_SERVICE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>log/my_service.log</file>
</appender>

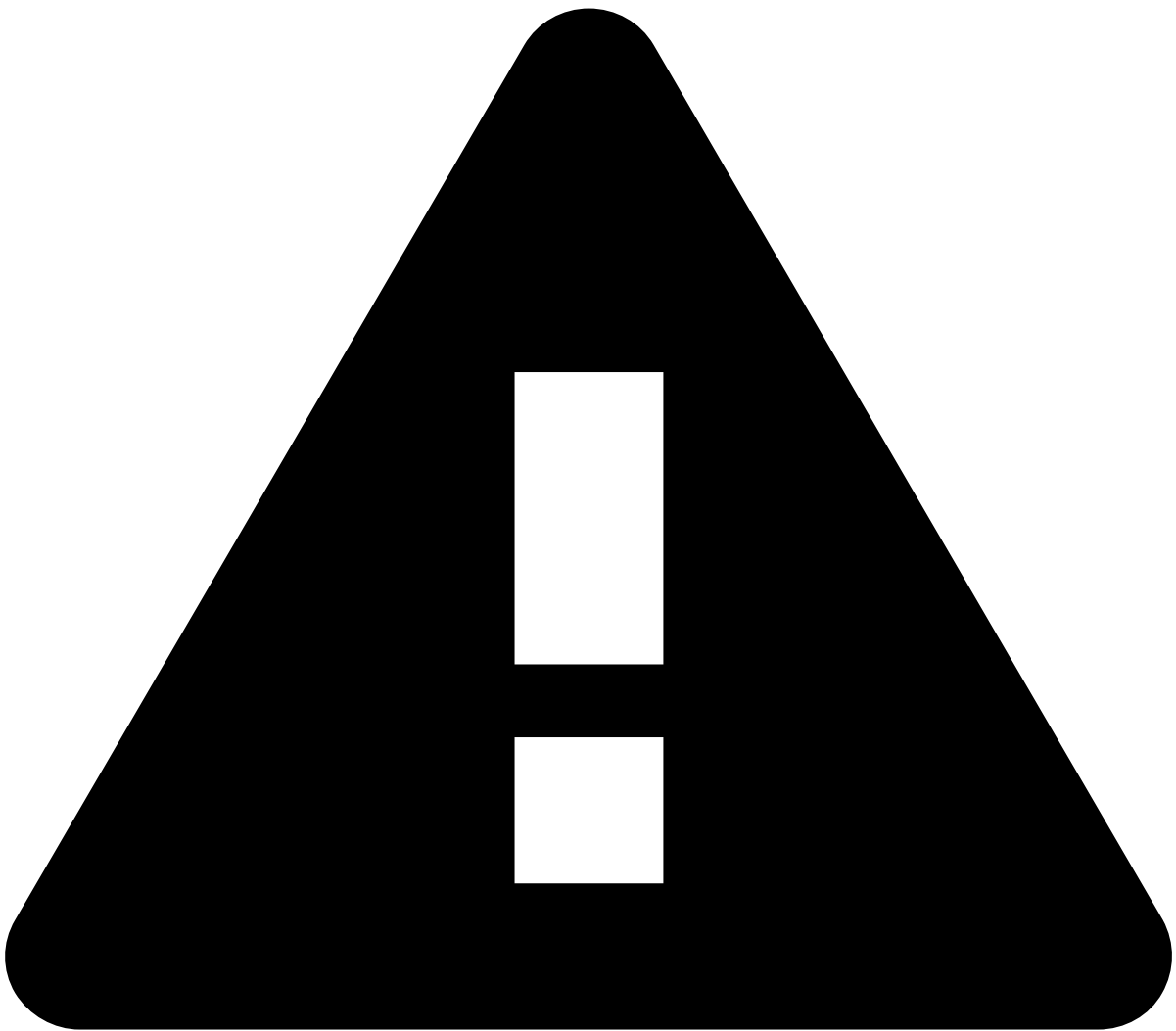
<logger name="com.company.my.service" additivity="false">
  <appender-ref ref="MY_SERVICE" />
</logger>
```



The **additivity** parameter tells the logger that it mustn't be used in other loggers and therefore appenders. In this case, logging calls to that logger will not reach the **CONSOLE**, **BUFFER** or **FILE** appenders but only the **MY_SERVICE** one.

Root Logger⚠️

The root logger is the default logger where all the messages will wind up by default. The only case where that doesn't happen is when a logger is defined with the **additivity** parameter as **false**. Here are usually defined the destination appenders of the messages.



caution

We strongly advise not to change the appenders already under the root logger section.

Lnav Support

[Lnav](#) is a log viewer that can help read and analyze logs. Among other features, it offers highlighting and a unified view over multiple log files.

For Lnav to support Pulse's log format you need to install the [Pulse log Lnav configuration](#):

```
lnav -i feedzai-pulse-log-format.json
```

This log format is valid for Pulse 15.1.5 and above. For previous 15.1 versions you should use [this configuration file](#).

To launch lnav, do:

```
lnav <pulse-path>/log/
```


•



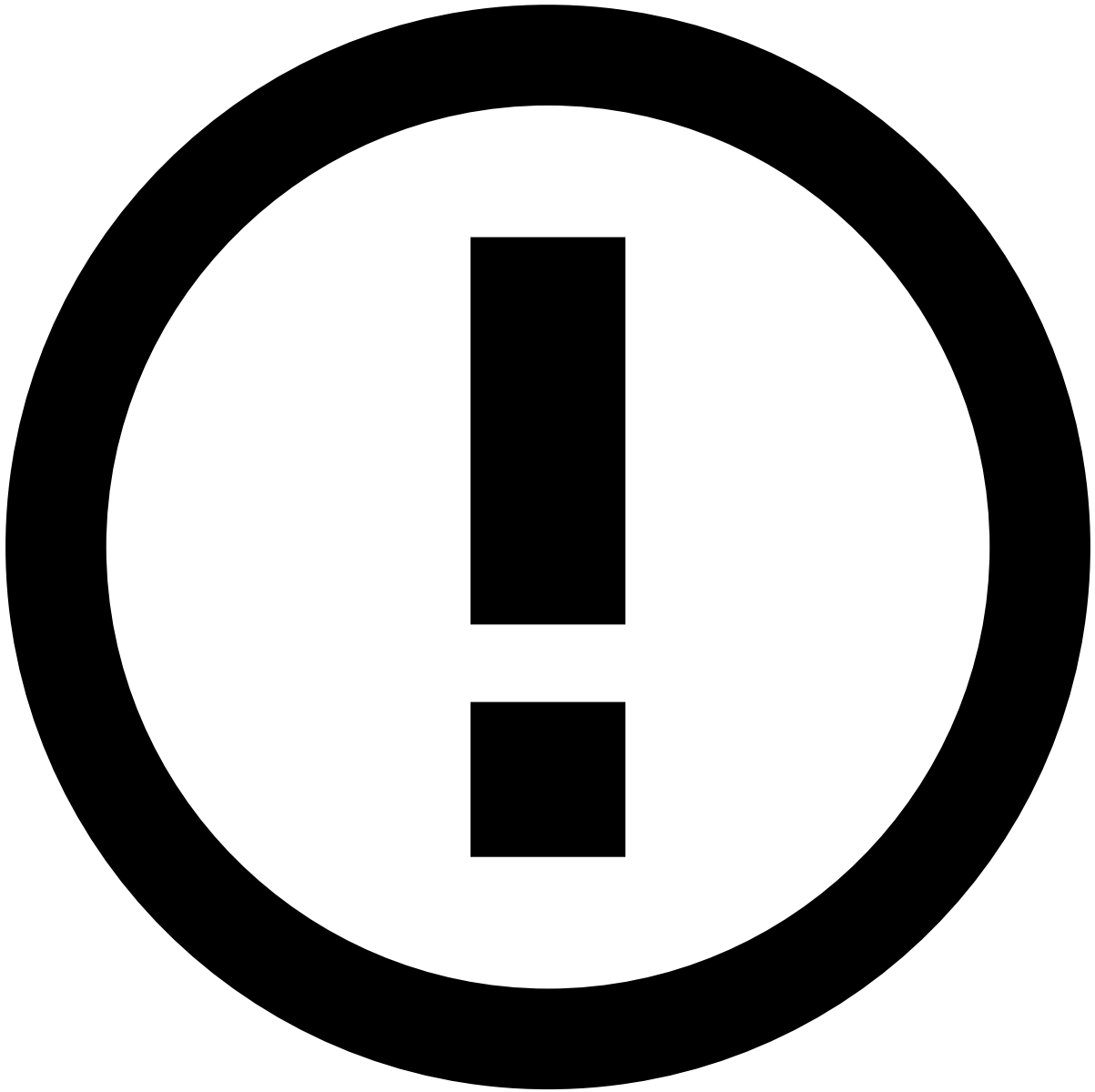
- Project Organization
- [Monitoring](#)
- Monitoring Metrics

Monitoring Metrics

Was this page helpful? 

Pulse and App Engine instances collect monitoring metrics in different components that are then exposed through the `/metrics` endpoint. This endpoint has a default timeout of 1 second when retrieving the metrics from the App Engines. However, it's possible to use a timeout parameter to override this default. For example, calling `/metrics?timeout=3` will wait 3 seconds at most for metrics from the App Engines.

To give more visibility about these metrics to Pulse's operators, this section will hold pages describing these metrics.



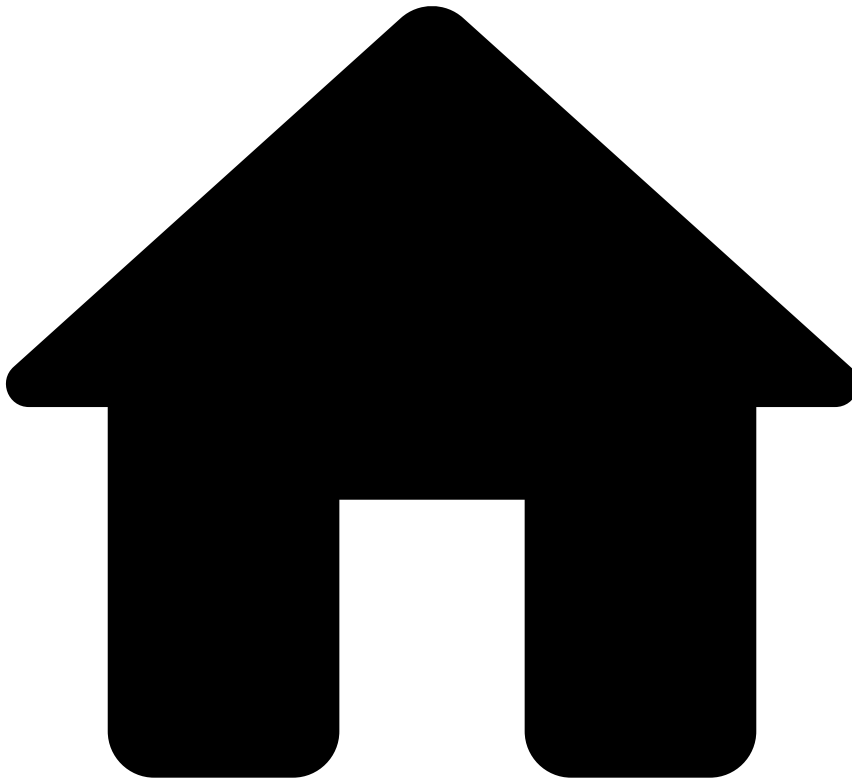
Metrics Self Documentation

Even when not described in this section, all metrics have a short description and other information which can be obtained at the `/metrics/rich` endpoint.

This section will start presenting information about monitoring metrics related to PKernel but will be iteratively incremented with new information related to other components of the product.

- [Monitoring Processing of Profiles in PKernel](#) â Learn about PKernel monitoring metrics.
- [Monitoring Replication of Events in the Workflow](#) â Learn about metrics to monitor the replication of events in the workflow.
- [Monitoring the Persistence of Events in the Workflow](#) â Learn about metrics to monitor the persistence of event data.
- [Monitoring Cassandra Driver Sessions](#) â Learn about Cassandra driver sessions monitoring metrics.
- [Monitoring Processing of Feedback Records in Pulse](#)
- [Monitoring Event Storage Dumps in Pulse](#)
- [Monitoring DynamoDB Profile Store](#) â Learn about metrics to monitor the DynamoDB profile store.

•



- Project Organization
- [Monitoring](#)
- Pulse Probes

On this page

Pulse Probes

Was this page helpful? 

Pulse probes provide a means to expose Pulse internal instrumentation data, allowing it to be stored in a central location for subsequent analysis.

Probing Components

The Pulse probing architecture is composed of two components:

- **Probes** that periodically measure some relevant metrics (for example, processing latency) and broadcast their values to a **metric server** that may be listening to them. Probing is performed either as a response to a pulse internal event (e.g. a fraud event passing by a fraud rules service workflow element) or on a periodic basis (e.g.

get sample of X every Y seconds). Each probe has an associated **metric schema**, that contains the fields that correspond to the metric. Probes are normally tightly coupled to Pulse components.

- **Metric Server**, that are subscribers of the metric information published by the probes and that store received metric information in CSV files. Metric servers may register/unregister themselves on the existing probes.

Note that the term "metric" in this context refers only to whatever is measured by probes, and should not be confused with PQL metrics.

Metric servers communicate with probes for registering/unregistering themselves and for probe discovery using a topic named `metrics_control_topic` ; metric events are broadcast to metric servers through direct TCP connections.

This architecture is extensible, allowing you to create your own probes for any pulse component you develop (e.g., custom workflow services).

Running a Metrics Server🔗

A metrics server deployment is composed of a single standalone jar file (**metrics-server-<version>.jar**) and a properties file with its configuration, named **metrics.properties** by default. A sample configuration file is included. In order to launch a metric server just run:

```
java -jar metrics-server-<version>.jar [properties_file]
```

Metrics servers can run in two modes:

- **Interactive (REPL enabled)**, where a shell is provided for registering/unregistering the server on probes in an interactive way.
- **Non-interactive (REPL disabled)**, where the server will only handle auto-started probes (see *metrics.autostartprobes* [below](#)). When used this way, the metric server must be started before the Pulse server (or, more exactly, before the probes that are to be listened for start).

The server run mode is specified in the *metrics.replenabled* property of the configuration file (see [below](#)).

The *properties_file* argument can be used in case you need multiple configurations on the same host. If not specified, the default *metrics.replenabled* properties will be used.

Interactive Commands🔗

The commands available in interactive mode are the following:

- `l[ist] [pattern1[,pattern2]]` - shows the available probes and their status. Absence of a pattern means all probes.
- `r[eport] pattern1[,pattern2]` - start listening on the probes specified in pattern.
- `s[ilence] pattern1[,pattern2]` - stop listening on the probes specified in pattern.

Probe pattern names support the wildcard characters `*` and `?`. The `*` wildcard matches a sequence of zero or more characters and the `?` matches exactly one character.

The list of existing commands can be displayed on the shell just by entering an empty line.



< 15.1.10

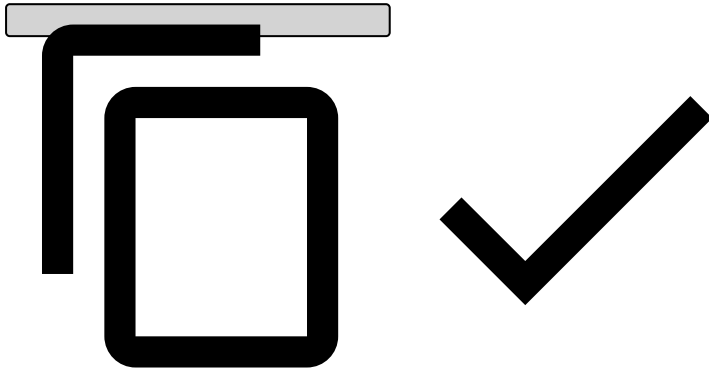
Make sure that you silence your patterns before exiting the server! If you report for a given pattern, shutting down the server and leaving interactive mode does not stop the pattern from being reported.

Since 15.1.11 metrics server will silence all probes when successfully shutdown.

Example

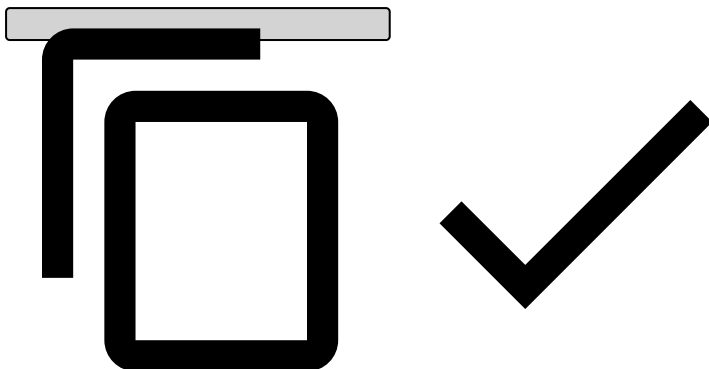
Start by running the interactive mode and list all the available patterns.

```
$ l
(...)
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,msg_type:STRING,rule_process_latency:LONG,rule_total_latency:LONG)
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_latency:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,msg_type:STRING,rule_process_latency:LONG,rule_total_latency:LONG)
(...)
```



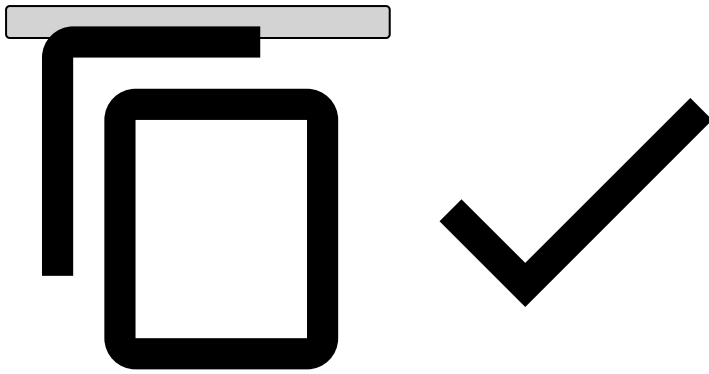
Select one of the available patterns, say `[email protected]` , to check its current status and schema.

```
$ r [email protected]
$ l [email protected]
[email protected] reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_latency:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
```



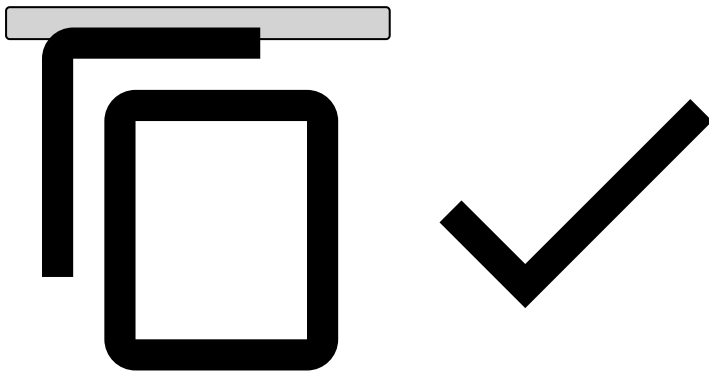
Enable reporting for the above pattern.

```
$ r [email protected]
$ l [email protected]
[email protected] reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_latency:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
```



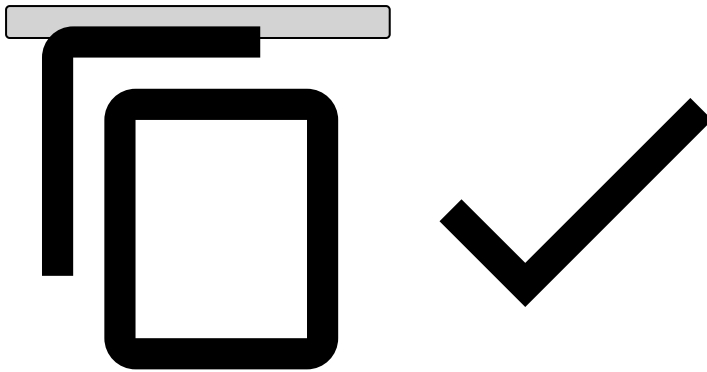
Once instances pass through the defined fraud scoring service, some lines will be generated in the output file, define in the configuration file. Note that the time is represented in nanoseconds.

```
mylocalhost,3538b328-134f-4d63-84cf-bd4c50db93e6,1465421548284,NORMAL,false,
1967209,377659,2426728
mylocalhost,65b02454-11b1-42e9-b2ba-f0438c5933e0,1465421549518,NORMAL,false,795394,272169,1116582
mylocalhost,8bbbed484-177e-4985-b835-7a4cb85c3950,1465421549676,NORMAL,false,844627,267752,1160902
mylocalhost,f62d3877-ed3b-4489-85fb-472c465a7c14,1465421549844,NORMAL,false,805862,239146,1107133
mylocalhost,2b892352-1c93-485b-9646-1ac18455a067,1465421550011,NORMAL,false,631836,219795,910742
mylocalhost,047e3b92-21b3-4703-8c97-5aa74765d9c5,1465421550158,NORMAL,false,903389,252477,1223005
mylocalhost,d5e2c84d-f1a8-46bb-a7ae-0157ce6433bf,1465421550481,NORMAL,false,289246,227748,558441
mylocalhost,edb06d6c-86fd-42bd-a2ed-1afe7bd742fa,1465421550683,NORMAL,false,796873,242708,1086698
mylocalhost,c5d1743f-6bd7-4e38-80c8-8f0220653d07,1465421550885,NORMAL,false,595244,233126,873616
mylocalhost,d3c5e203-b021-4100-af89-ece74af997b1,1465421551068,NORMAL,false,840138,238641,1124537
mylocalhost,811ad876-fdbd-410a-b15e-1dfe5ed363dc,1465421551277,NORMAL,false,747988,255783,1053942
mylocalhost,0f2de8ea-000e-4933-8307-c127e2bac71a,1465421551471,NORMAL,false,799524,253927,1102078
mylocalhost,40b65b62-c5ba-4e92-9124-a2f8143dda1a,1465421551640,NORMAL,false,678110,235491,988029
mylocalhost,7d1d57f7-0553-4434-afa3-ab29a726e6de,1465421552376,NORMAL,false,817818,258881,1125458
mylocalhost,69e3fa38-ae32-4840-a706-6a6afb031fc,1465421552705,NORMAL,false,814170,225829,1086237
mylocalhost,5caff4a5-2773-42f1-a1e2-d6b2df1d3637,1465421553119,NORMAL,false,783721,258148,1089784
mylocalhost,d22a1a12-fcc1-4187-a650-cb3f68d7bc64,1465421553350,NORMAL,false,864084,248861,1161982
mylocalhost,3eabdb18-8ff4-44a0-a20f-119464632d87,1465421553652,NORMAL,false,807886,240421,1100518
mylocalhost,7a14f56b-aa60-4900-86af-c4cb3519de65,1465421553863,NORMAL,false,663703,236307,946143
mylocalhost,30358947-0550-480a-aa57-8ab390465378,1465421554032,NORMAL,false,811868,238845,1096526
mylocalhost,f16f61e2-01c6-47d7-9668-503ea9742ac1,1465421554223,NORMAL,false,830213,242055,1133387
```



Before leaving the interactive mode, silence the reported pattern.

```
$ s [email protected]
$ l [email protected]
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_late
ncy:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
```



Metrics Server Properties

The `metrics.properties` has the following properties.

Property	Description	Default Value
<code>metrics.autostartprobes</code>	The server will register on the probes that match these patterns and that start after the metrics server starts. Patterns support the wildcard characters <code>*</code> and <code>?</code> . The <code>*</code> wildcard matches a sequence of zero or more characters and the <code>?</code> matches exactly one character.	None
<code>metrics.host</code>	The host of the metrics server. Probes will report connecting to this host.	hostname reported by <code>InetAddress.getLocalHost().getHostName()</code> or <code>localhost</code> if that fails.
<code>metrics.port</code>	The port the metrics server is listening on.	16161
<code>metrics.replenabled</code>	Whether the metrics server should run in interactive mode or not.	true
<code>metrics.store.outputdir</code>	Metrics will be stored to files in this folder. This folder will be created if does not exist.	Mandatory property.
<code>metrics.messaging.*</code>	Messaging can be configured with the usual messaging configuration properties prefixed by <code>metrics.messaging</code> . Must match the pulse-side messaging configuration.	Embedded ActiveMQ. Only works when probes and metrics server are on the same machine.

Existing Probes

The following probes are built-in in the Pulse Server/App Engines (where `<workflow_element>` stands for the service name sanitized):

Probe Target	Description	Probe Name	Schema	Notes
All Workflow Input Services	Latency associated to input event publishing. A metric record is generated for each event published into the workflow.	service_runner_input<workflow_element>	timestamp: Long	
			uuid: String	
			message_type: String	
			runner_total_latency: Long	- Services treating the message synchronously - Latency between the message entering the

				service's internal queue. The message enters the queue when <code>post()</code> is called on the message object. The message is then being ready to be processed by the next workflow element. • Services treating the message asynchronously Latency between the message entering the service's internal queue and the message being ready to enter the next workflow element. Note that in this case, the queuing of the message is ready to be consumed by the next element and not accounted for. The time the service spends treating the message and the time the message spends traversing to other workflow elements are included.
			<code>event_total_latency</code>: Long	The time it takes to process the full workflow and for a call-back to be sent to this input service runner informing that the event finished. This probe is the same as <code>runner_total_latency</code> with the additional time that it takes to completely treat the request throughout the workflow.
All Workflow Input Services	Latency associated with the time spent by a message in a workflow queue, waiting to be processed by the next workflow internal service.	<code>service_runner_input_queue_<workflow_element></code>	<code>timestamp</code>: Long <code>uuid</code>: String <code>message_type</code>: String <code>input_queue_latency</code>: Long	Â
	Latency associated with	<code>service_runner_<workflow_element></code>	<code>timestamp</code>: Long	Â

All Workflow Internal Services	the processing in an internal (i.e., non-input) service. This is measured at the "service runn level, i.e., the holder that encapsulates the service and allows it to run in the workflow (including custom services). A metric record is generated for each event that goes through such a runner.		uuid: String	Â
			message_type: String	Â
			runner_total_latency: Long	• Normal message handling: - Latency between the message entering the service's internal <code>post()</code> and the message being ready to enter the next workflow element. • Error treating the message - The latency between the message entering the internal service's <code>post()</code> and an error being caught. This includes the time to process the task on the service's <code>onMessage</code> and any setup before enqueueing the message to the workflow. The time taken to enqueue to the workflow for further processing is not included.
			failure: Boolean	Whether there was an error condition or some exception thrown while processing a <code>post()</code> message.
Workflow Rules Service	Latency for processing all the rules which were evaluated for an event. A metric record is generated for each event processed in each rules service.	rules_service_<workflow_element>	timestamp: Long	Â
			uuid: String postLatency: Long	
Workflow Scoring Service	Latency for scoring an event.	scoring_service_<workflow_element>	timestamp: Long uuid: String	Â

	A metric record is generated for each event processed in each scoring service.		scoreLatency: Long	
Metric Fetcher	Latency for fetching metrics for a MetricListOperator of a plan used in a Plan Workflow Service. A metric record is generated for each event that traverses such an operator in each plan. Note that each Metric Fetcher has its own ID (shown in the 3rd column). This is because there may exist multiple plans and/or multiple metric operators with scheduled metrics.	scheduled_metrics_fetcher	timestamp: Long message_id: String fetcher_id: Long fetch_success: Boolean fetch_size: Integer fetch_selector_build_latency: Long fetch_profile_store_latency: Long fetch_resolve_latency: Long fetch_total_latency: Long	The <i>fetch_profile_store_latency</i> measures the time between triggering the fetch of the metric and either its successful retrieval or its cancellation due to timeout. These things are done in separate threads. Due to the way that the scheduling happens in the Java Virtual Machine, there is no guarantee that threads execute immediately when scheduled. As such, the times shown in <i>fetch_profile_store_latency</i> and <i>fetch_total_latency</i> entries might be greater than the actual time it took to either fetch or timeout.
Real-time PKernel	Latency for processing real-time events in each PKernel. This is analogous to the Metric Fetcher, but for the real-time part of the metrics calculation (whereas the Metric Fetcher is for the fetch of the scheduled metrics). Note that each PKernel has its own ID, also analogously to the Metric Fetcher. Furthermore, it	pkernel_realtime_metrics	timestamp: Long pkernel_id: Long partition: Long event_id: String latency: Long	Â

	has a partition identifier since the same (conceptual) PKernel (in a given Metric operator of a given plan) may be partitioned (according to the various existing workflow partitions).			
Real-time checkpointable PKernel (this only available when PKernel with persistent state is enabled)	Latency for processing real-time events in each PKernel. This is analogous to the Metric Fetcher, but for the real-time part of the metrics calculation (whereas the Metric Fetcher is for the fetch of the scheduled metrics). Note that each PKernel has its own ID, also analogously to the Metric Fetcher. Furthermore, it has a partition identifier since the same (conceptual) PKernel (in a given Metric operator of a given plan) may be partitioned (according to the various existing workflow partitions).	checkpointable_pkernel_post	timestamp: Long operator_name: String partition: Long pkernel_id: String event_id: String latency: Long	This probe is similar to pkernel_realtime_metric but it provides a scheduler that reports in addition the name of the operator in the Plan and the PKernel id. This only available when PKernel with persistent state is enabled.
GC Pauses	Probe used in the metrics agent to monitor garbage collection pauses from the jvm gc log file.	gcPausesMetric property configured in the metrics agent.	timestamp: Long gc_pause_in_nanos: Long	Â
End of the workflow	The latency spent at the end of the workflow.	post_workflow	timestamp: Long uuid: String	Current timestamp in milliseconds. Id of the message.

			message_type: String	The type of the message.
			deadletters_notify_before_event_storage_latency: Long	The latency (in nanoseconds) for the time spent at the end of the workflow notifying deadletters listeners before storing the message in the Event Storage.
			deadletters_notify_after_event_storage_latency: Long	The latency (in nanoseconds) for the time spent notifying deadletters listeners after storing the message in the Event Storage.
			eventstorage_queue_latency: Long	The latency (in nanoseconds) for the time spent at the end of the workflow sending the message to the pending queue which will then asynchronously save it to the Event Storage.
			processing_total_latency: Long	The latency (in nanoseconds) for the total time spent at the end of the workflow.
workflow_type: Long	The value is 1 if the message was from the past, or 0 if the message was a real-time one.			
DynamoDB BatchGetItem	Latency spent by metric fetcher on each BatchGetItem operation.	batch_get_item	timestamp: Long	Current timestamp in milliseconds.
			message_id: String	Id of the message.
			fetcher_id: Long	The fetcher id.
			batch_num: Long	The BatchGetItem being performed.
			batch_total: Long	The total number of BatchGetItem requests to be performed.
			unprocessed: Boolean	If the BatchGetItem has unprocessed items that require a retry.
			latency: Long	The latency of the BatchGetItem.

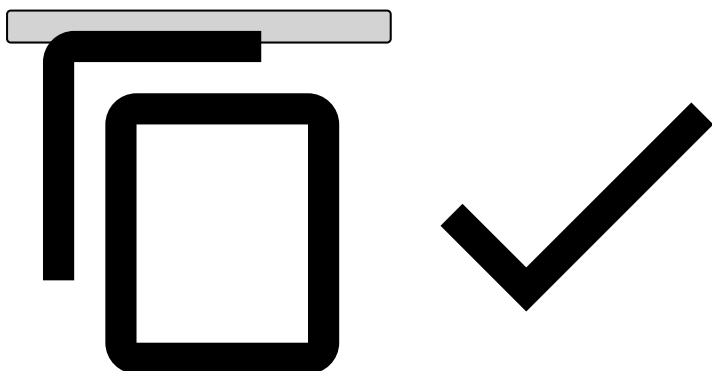
Probe names include the pulse node ip address appended to what is specified in the "Probe Name" column.

Storing Metric Data Locally

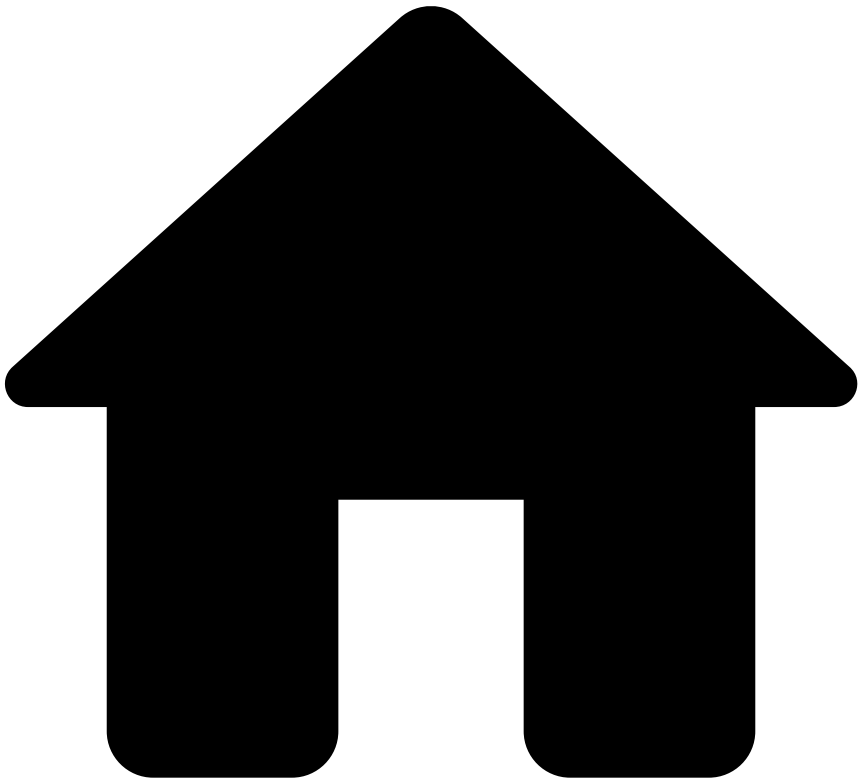
Probes can also be configured to write metric data directly to a local text file, using the logging framework. This setup doesn't require a metric server but allows only for metric data to be stored locally on (each) Pulse Server and doesn't allow individual probes to be "disabled", as when using the metric server.

The Pulse standard log configuration file (`PULSE_HOME/conf/logger/logger-metrics.xml`) includes a commented out section with an appender (`METRICS-ASYNC` , that in turn uses the `METRICS` appender) and a logger for that appender. In order to have the probes writing directly to files it is enough to uncomment this section.

```
<!--  
    Start of metrics server local logging.  
    By default this logger is disabled. This means that probes will not be logged to disk,  
    only reported to the metrics server if enabled.  
  
    With this logger metrics server doesn't need to be enabled.  
  
    Use an AsyncAppender to avoid impacting the critical path of execution degrading processing  
    latencies.  
-->  
<!-- METRICS SERVER LOGGING TAG  
<appender name="METRICS" class="ch.qos.logback.core.rolling.RollingFileAppender">  
    <file>log/${pulse.logcontext}-metrics.log</file>  
    <encoder>  
        <pattern>%msg</pattern>  
    </encoder>  
    <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">  
        <fileNamePattern>log/${pulse.logcontext}-metrics.%d{yyyy-MM}-week-%d{WW}.log.gz</  
fileNamePattern>  
        <maxHistory>8</maxHistory>  
    </rollingPolicy>  
</appender>  
  
<appender name="METRICS-ASYNC" class="ch.qos.logback.classic.AsyncAppender">  
    <appender-ref ref="METRICS" />  
    <queueSize>2048</queueSize>  
    <discardingThreshold>0</discardingThreshold>  
    <includeCallerData>false</includeCallerData>  
</appender>  
  
<logger name="com.feedzai.metrics.probes.TextAbstractProbe" level="TRACE" additivity="false">  
    <appender-ref ref="METRICS-ASYNC" />  
</logger>  
METRICS SERVER LOGGING TAG -->  
<!-- End of metrics server local logging. -->
```



•



- Project Organization
- [Monitoring](#)
- Web Monitoring

On this page

Web Monitoring

Was this page helpful? 

Pulse provides ways of managing and accessing information through the UI. This section describes what can be done and how it can be done regarding fine tuning your Pulse server via UI.

What Can Be Controlled?[a](#)

Using the Pulse web interface it's possible to perform the following tasks:

- **Monitor and control** Pulse services
- **Manage** Pulse's configuration attributes
- **View** Pulse's log and general information

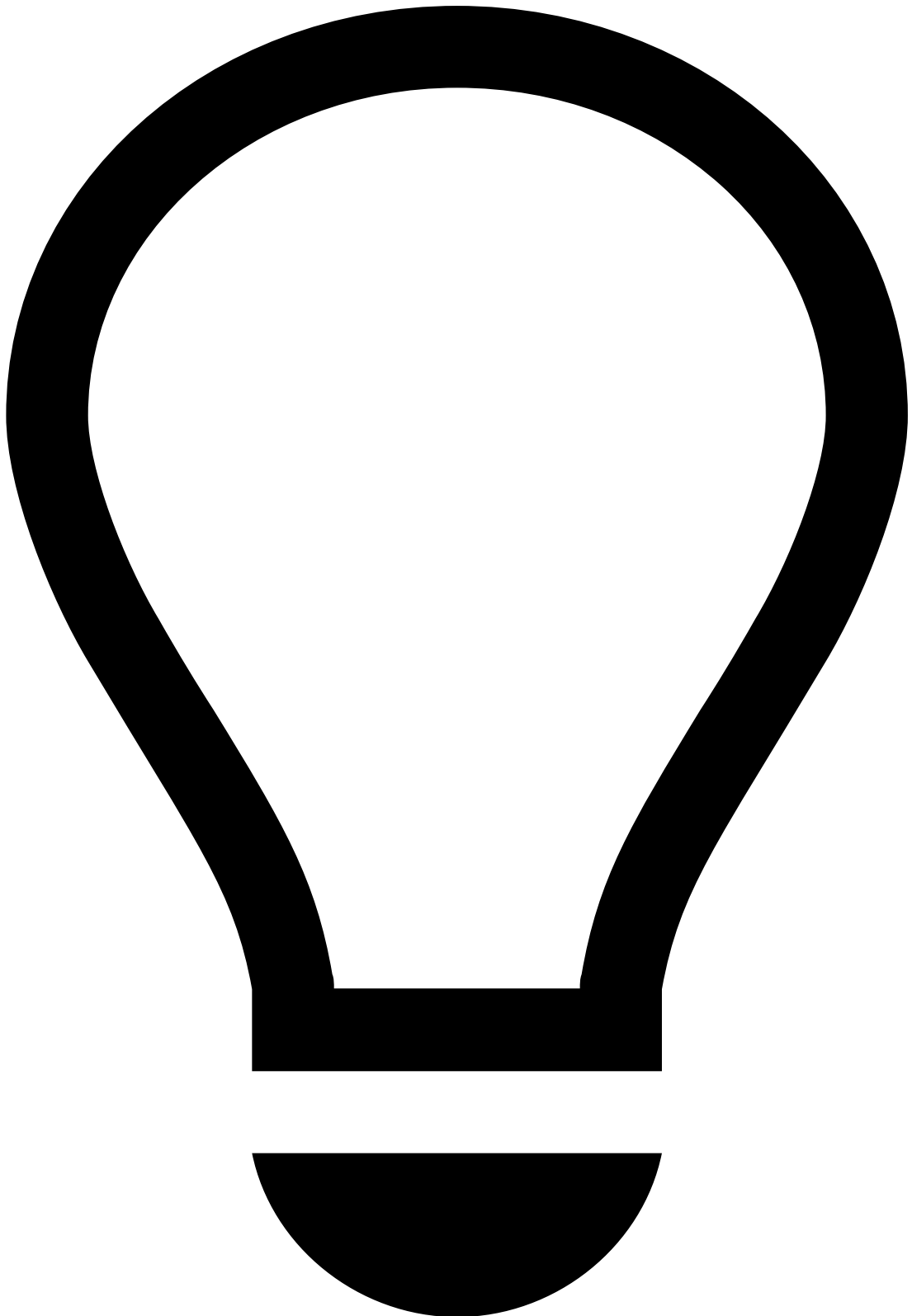
Manage Pulse Services

Pulse Server is organized in services that handle each task comprised in it separately. In this section you will learn how to check what services are started and how they can be controlled. Below is an image of the UI to monitor Pulse services as well as a small description about each component:



Beware

Do not stop or restart services if you don't understand the service dependencies of Pulse Server.



Switch between server node and application view

By default this tab shows the services information for the whole Pulse Server node. You can choose an application to manage its resources in the top right corner of the tab.

Core

The core services are the base components of the Pulse Server. You cannot restart them because they are required for the correct operation of other Pulse services. You can, however, view their state.

Modules

Modules are components that provide advanced features such as KPIs and Baselines. These services can be fully administered using the UI, just make sure you know the implications of restarting them.

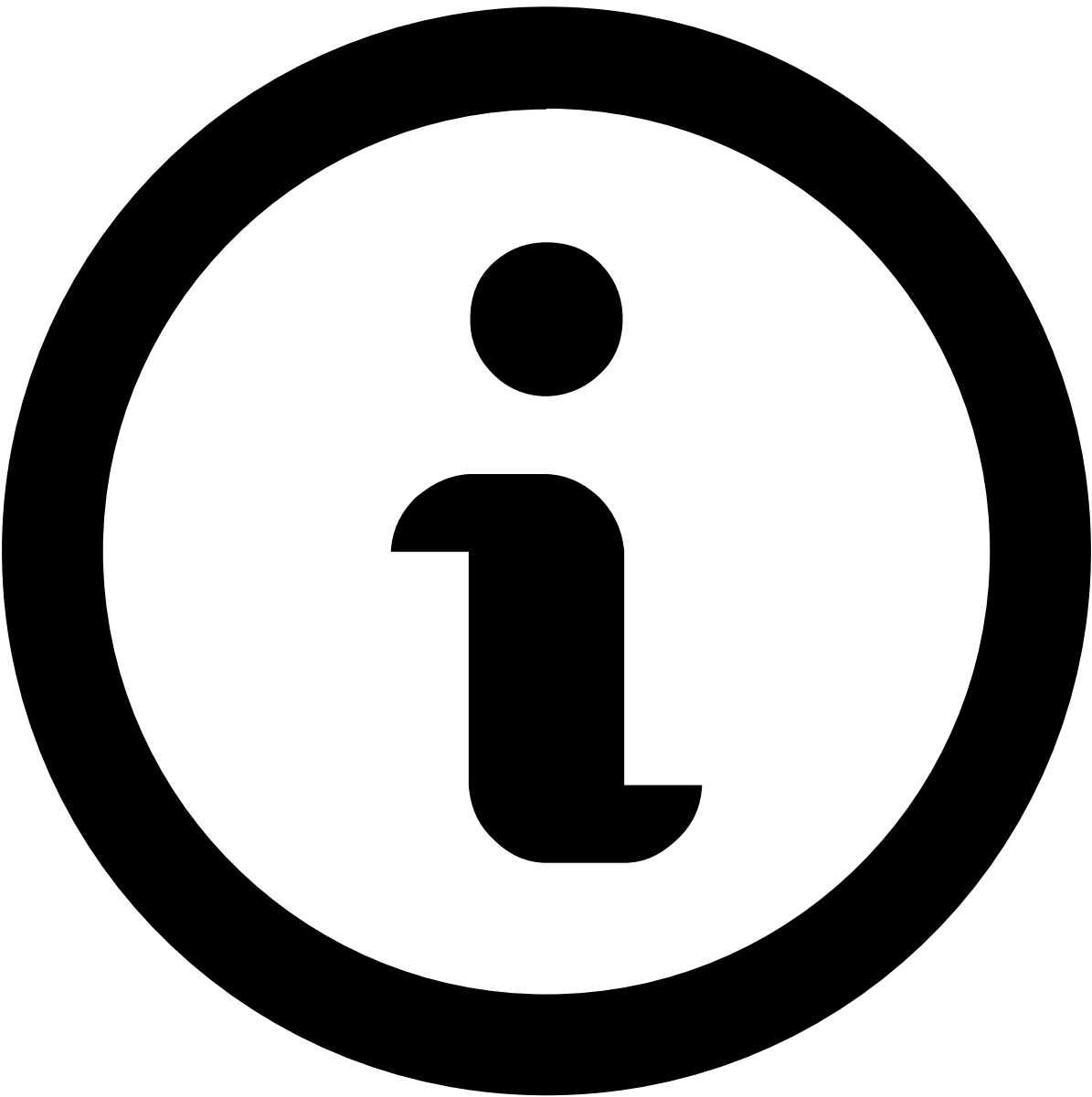
Adapters

In the adapters section you can check the input and output adapters available in the system. This table will list all the adapters in action currently as well as their state.

Standalone

This section contains the services that were uploaded using the custom services interface.

Web



Web services listing is only available for the whole server node view.

The web section shows all the applications running in Pulse Web Server. There are two types of applications:

- **Core Service** - these are the applications deployed with Pulse.
- **Web App** - applications deployed by the users.

Info

This tab provides us with an overview of the system running the Pulse Server along with a few other interesting details. It also shows how many services are running, applications deployed and statistics about Kpis, Baselines and PKernel objects. It is also possible to check the Cluster information where all the nodes are listed along with their types, status and IP addresses.

If your application is already running, you can choose it in the list of applications (Application Statistics sub-section) and pull charts that plot the CPU Usage and JVM Heap.

Log

Feedzai Pulse logs all the operations being executed in the server in files. For ease of use the UI also allows you to access the last operations logged in your Pulse Server as well as download its log files.

To change the level of the logger being used in the web console go to your **PULSE_HOME/conf/logger.xml** file and search for the **BUFFER** appender section.

Configuration

Internally Feedzai Pulse uses a *properties file* to save its configuration, but connecting to the server and editing a file isn't always a feasible option.

This tab allows you to configure your Pulse Server using solely the UI.



Restart Pulse Server

All changes in configuration will need a restart of the service or even a full restart of Pulse Server.

What's Next? 

Head over to [this section](#) and learn how to secure your application resources.

•



- [Core Concepts](#)
- Audit

On this page

Audit

Was this page helpful? 

Pulse keeps an audit log of changes made to the Pulse applications and resources, as well as user actions (e. g. login). Pulse logs these actions when they are initiated via user interface or via Pulse REST API endpoints.

What is logged

The audit log consists of the following:

- **Audit entries:** the logs that capture a user action or a change. It contains data that was included in the request that initiated the action. Example: if you create a list item, Pulse includes in the audit entry the list name, the user session identifier, the values added, whether the list item is enabled, etc.

- **Audit entry labels:** labels that are added to the data fields of the audit entries, to make them easy to browse in the Pulse UI. If a data field of the audit logs has a label, you can filter the audit entries by the value of that data field simply by clicking on the value.
- **Audit entry search terms:** terms that you can search for in the audit entries, using the Pulse UI. Search terms don't appear on the UI, and they are not configurable. They can be used to search resources by identifier on advanced investigations.

Pulse creates an audit entry for the following:

- Deletion, creation and update of data science entities (rules, models, lists, etc.)
- Deletion, creation and update of security entities (users, tenants, permissions, groups etc.)
- User actions (login, logout, change password, viewing or exporting list items, etc.)
- Application import, export, publish, start, stop, etc.
- Certain configuration changes (job profiles, autohealing, self service, etc.)

For a full list of logged actions, check the Action drop-down on the Audit audit screen in the Pulse UI. See the [Investigating changes and user actions](#) topic.

Pulse keeps all audit logs, but the Pulse UI shows only the last two months' audit data.

Who can see the audit logs?

Users with the `security:audit:read` permission have access to all audit entries.

Audit is enabled by default.

Where are the logs stored?

The audit log is stored in the Pulse database, in the following tables:

- AuditEntry
- AuditEntrySearchTerm

It's not recommended to access these tables directly. The recommended way to access audit logs is via the Pulse UI.

Learn more

Read how to [investigate changes and user actions](#) in the Pulse application.

•



- [Core Concepts](#)
- Channels

On this page

Channels

Was this page helpful? 

This page gives you a short overview of what channels are and how they can be used in an omnichannel detection scenario.

A **channel** is the means of communication through which data is received by the financial institution. Examples for different channels can be: ATM, bank branch, card, Internet banking, mobile banking, etc.

Typically, each channel has a different schema and different [data patterns](#). Because of this, each channel may require a different strategy to prevent financial crime, as well as a different approach to alert management and reporting. For example, rules or machine learning model features based on payment terminal identifiers will be relevant for card transactions, but not relevant for internet banking transactions. Likewise, terminal identifiers are relevant to analysts looking at card alerts, but not to analysts looking at internet banking.

Channels in Pulse and Case Manager

Runtime channels

In Pulse, each channel corresponds to the data of a specific runtime [workflow](#). Workflows can be further organized into applications.

This setup helps teams working on different channels to manage the system effectively:

- Users can configure each workflow with flexible logic that applies to a specific channel.
- Updates to the detection strategy can be deployed independently for each channel. This is especially relevant when independent teams work on different channels.

Further, in Case Manager, users can connect each Pulse workflow to a different alert management configuration. For more details, see the Case Manager documentation.

Channels in the data science environment

In the data science environment, historical data from each channel is represented as a [data source](#). These data sources can be used, for example, to build and train machine learning models.

Cross-channel attacks

Treating each channel independently makes it easier to configure the system with the specifics of each channel in mind. However, it's also important to look at the data from multiple channels holistically for detection, alert management and reporting. This is because financial crime has evolved, and there are attacks that exploit gaps in systems where channels are disconnected. For example, if we just look at ATM data, all withdrawals look the same. But if we couple this with Internet banking data, we may notice that some withdrawals happen after a user logs into their account using a new device and raises the withdrawal limit. Such withdrawals are much more suspicious, because there is a risk that what preceded them was an account takeover.

Cross-channel attacks require an omnichannel approach to detection.

Using data from multiple channels

The main principle behind **omnichannel detection** in Pulse is to enable data scientists and analysts to build rules and models that use data from multiple channels, while still allowing teams to work on channels independently. This is achieved as follows:

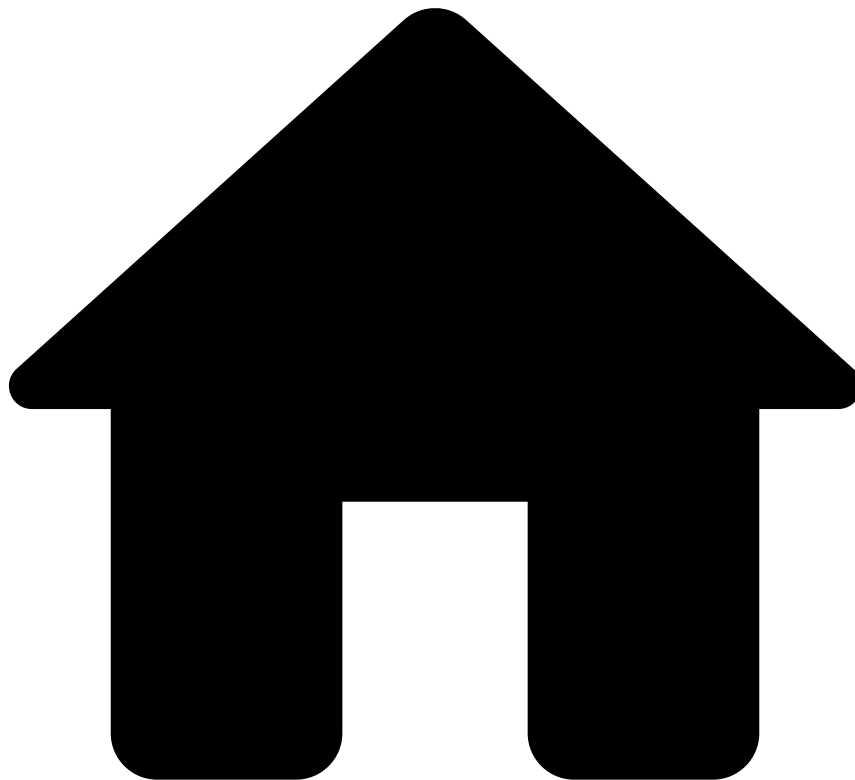
- Each channel has its own detection strategy, implemented in a workflow. Within that workflow, the channel is referred to as the **main channel**.
- Data scientists and analysts working on the main channel can incorporate data from other channels, which are referred to as **secondary channels**.

In Pulse, data is analyzed with [plans](#). A typical workflow will include one or more plans that compute features based on the data.

Pulse supports [omnichannel plans](#). Using these plans, data scientists and analysts can build profiles based on data from multiple channels.

- In the data science environment, users can create and run omnichannel plans that consume data from multiple data sources—one data source for the main channel, and one for each secondary channel.
- In the [workflow](#), omnichannel plans are connected to other workflows—one workflow for each secondary channel.

-



- [Core Concepts](#)
- Data Science Loop

On this page

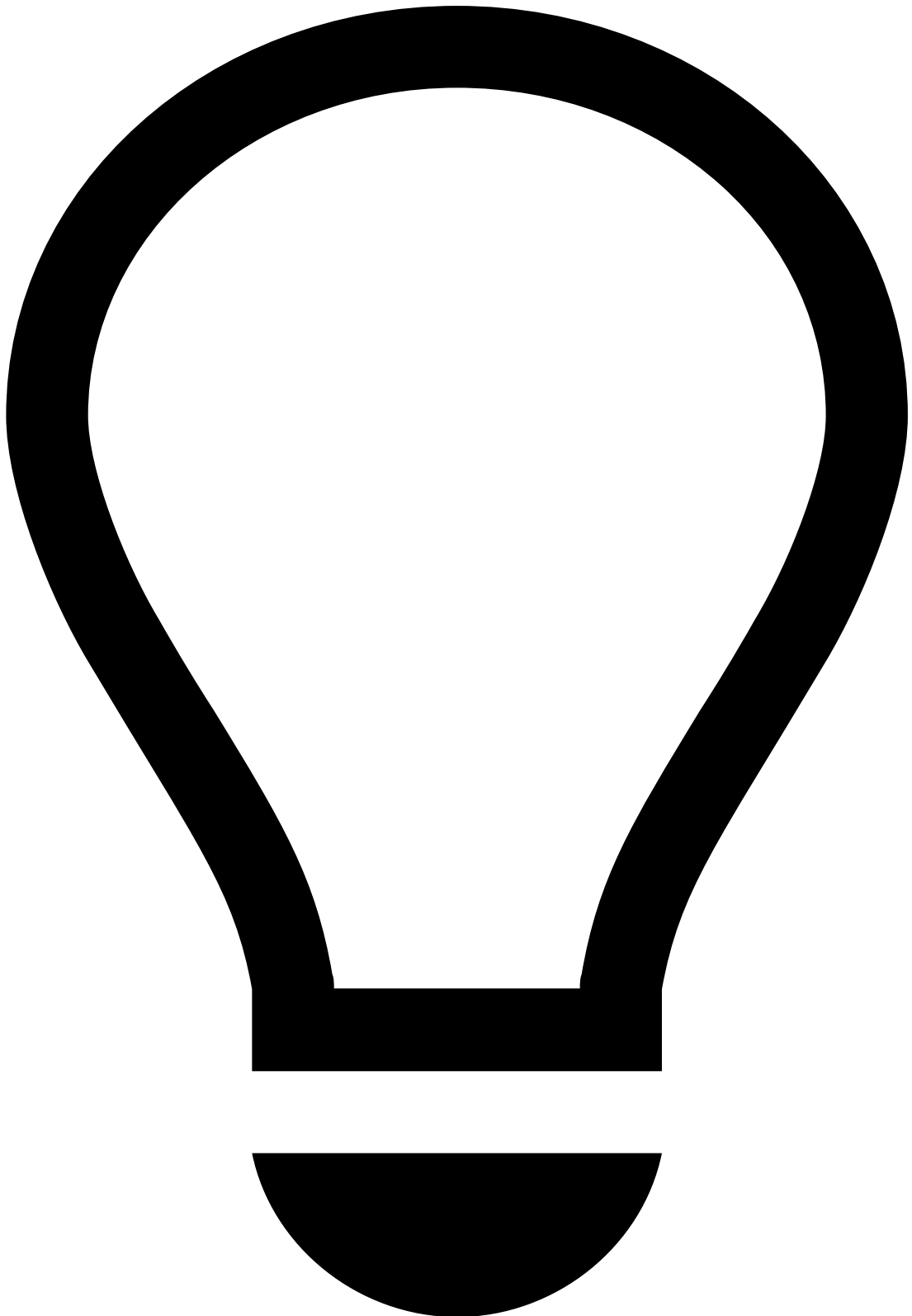
Data Science Loop

Was this page helpful? 

Pulse delivers a full data science platform that is focused on making your data science work much more efficiently than with your traditional toolset. Pulse is designed so that you can easily map your current paradigm to Pulse's concepts, while fully supporting the same data science loop.

Pulse supports the activities in the data science loop by enabling you to:

- Extract deep knowledge over your data to obtain the most value out of it.
- Iterate your work by performing continuous experiments and tuning to extract the best performance out of your model.
- Focus on the real data science by iterating quickly to accelerate your experiments and ultimately obtain better results.



Feedzai IQ

With [Feedzai IQ](#), you can also use ready-to-adopt risk assessment solutions without extensive data science work, thanks to federated financial risk intelligence aggregated from the Feedzai global network.

The following diagram illustrates the sequence of stages in the data science loop.

Data science loop

Data science stages

The data science loop is composed of a sequence of stages with very specific goals and iteration cycles. The following sections explain each of these stages and cycles in more detail.

Additionally, you can learn more about the dedicated environment used in risk assessment strategies according to the data science loop in a dedicated page: [The Data Science Framework \(DSF\)](#).

Clean and load

Clean and load

The main goal of this initial stage is to improve the quality of your raw data so that you can extract meaningful insight from it and ultimately train your model with relevant data. The datasets you receive from your client often have quality issues and a different structure from the one you need to work with.

At this stage you will apply some preprocessing transformations on your data to:

- [Import data](#)

Load your raw historical data, up to Terabyte size, to a system so that you can have access to it and start manipulating it with proper tools. For example, load a CSV file to a database or to a Hadoop cluster.

- [Validate the data schema](#)

Map the raw data to your project's specifications. The raw data can be structured in an outdated or not representative way, so, for example, you may need to change the names of fields or convert the original classification labels into a binary set (e.g. several types of fraud to just *fraud/not fraud*).

- [Clean up the data](#)

Fix inconsistencies in your raw data, remove irrelevant data/fields for data science (usually strings) or join datasets. For example, you may need to join a raw historical transactions dataset with the corresponding labels (e.g. *fraud* or *not fraud*) and set all negative values in transaction amount fields to zero.

This stage is usually performed only once when you receive the historical data from your client.

Analyze and sample

Analyze and sample

During this stage, the goal is to understand your data and prepare it to be intensively transformed and experimented on during the Feature Engineering stage.

At this stage you start to uncover relations in your input data and transform it to be more representative of the problem you are solving to:

- [Identify patterns](#)

Use data visualization tools and apply data visualization techniques over the data to understand it and to discover immediate patterns. For example, correlating the occurrence of fraud with its prevalence in a *Card Not Present* situation.

- [Create dataset sample](#)

Reduce the size of your dataset by generating another one that is better manageable and still representative of the specific problem you are solving. Extremely big datasets are not practical to iterate on because of their size. In addition, there isn't a clear advantage of training machine learning models over those big datasets. In fact, it may even be worse due to the curse of big data, where even noise can be considered a pattern when you have millions of data points. A solution is to balance the dataset by sampling 1%-10% of non-fraud transactions, making it better suited to a machine learning scenario. Reducing its size also reduces the requirements for distributed processing of large volumes of data while making it much easier/quicker to iterate on.

Feature engineering

Feature engineering

Your model decisions are as good as the data that you feed to it during its training. Feature engineering represents the process of transforming the raw data (already cleaned and sampled) into selected and enriched data. The goal is to obtain the dataset that defines what features you will use as input to your machine learning model.

This is the most intelligence-intensive task in the entire cycle and the one where you will probably spend most of your time. Feature engineering is the most important contribution of the data scientist to the model. It includes the following tasks:

- [Select important data for your model](#)

Choose which of the fields in your intermediate dataset are significantly related to the problem you are solving and can be used directly as features. For example, a *card_present* field is much more relevant for a fraud detection scenario than the first name of the customer.

- [Design derived features](#)

Enrich your dataset by highlighting specific characteristics observable in the data. For example, add a boolean *isNational* field based on the transaction country and add enrichment features such as *isDebit*, *isCredit* and *isTorNode* to improve the data with insight from additional external sources.

- [Design metrics \(historical profiles\)](#)

Characterize your data with time-based information that creates a unique profile for each individual data entity. For example, characterize spending habits of customer transactions by merchant type, country, or other field and aggregated over multiple time ranges such as 24 hours, 1 week, 1 month and 6 months.

- [Generate instance/features dataset](#)

Bring all your feature engineering work together by transforming your intermediate data into a single dataset containing all the features you identified/designed. This final dataset is the input for your machine learning model and contains all the relevant data for the model to learn with the appropriate input to solve your target problem, for example, fraud detection.

Build a model

Build a model

The goal of this stage is to take the transformed dataset, including all the computed features, and feed it to a machine learning algorithm in order to produce a model. The model is trained on a subset of the data to learn how to predict future behaviour based on features over historical data and evaluated on a different test dataset.

To create a machine learning model in Pulse, you can:

- [Train a model](#)

Feed your features dataset to teach a machine learning algorithm how to predict future behaviour based on the knowledge you have generated in your dataset. The quality of your model's decisions is a direct consequence of the quality and relevance of the data in the training dataset. Additionally, when you test your model, you must not use any data contained in the training dataset. You should always have a separate dataset, for example, a small subset you extracted from your data before training, as you must always test the model over unknown data.

- [Import a model that you created and trained externally](#)

Use the tools that you are familiar with to build and train your model, and then import it into Pulse.

Optimize model parameters

Within the process of building your model, one recommended step is to tune the model parameters so that you can optimize the performance of your model. There are no guidelines on which parameters to select for a given use case. As such, in order to select an optimum set of parameters, different combinations should be tested to find the one that makes your model operate at its maximum performance.

To adjust your model, you must:

- [Tune the parameters of your model](#)

Iterate your models by configuring different sets of parameter values for each of your algorithm's parameters before training your final model. Try more than one algorithm simultaneously as well to get to the best set of parameters that produces the optimal model for your use case.

Evaluate and compare

Evaluate and compare

At the end of your data science process, you must assess the performance of your trained model. Evaluating a model is not about just generating a single "OK/not OK" metric. It must rather focus on analyzing the results for different metrics, under different perspectives, and for different data subsets. For example, you may want to understand if your model is performing better when catching *Card Present* or *Card Not Present* fraud.

Since this is an iterative process, you can compare the performance of your latest model against the performance of previous iterations and earlier models.

You can also evaluate the impact of each feature on your model with [Feature Importance](#).

To track how close you are to reaching your objectives you can:

- [Evaluate your model](#)

Generate metrics and data visualization objects to let you analyze the performance of your model over an entire test dataset and over different segments and data subsets. Understand the impact of each feature on your model with feature importance. You can use this functionality to identify the subset of variables that contribute the most and the least to the model predictions. As a result, you can reduce model complexity and computation time. Understanding which features have higher impact on the model might also help you to identify new relevant features.

- [Compare your models](#)

Understand how the current iteration of your model performs against previous versions and how close your current performance is to reaching your project's goals. You can compare metrics such as Precision, Recall and False Positive Rate (FPR), cost-based metrics such as fraud money Recall, ROC curves, etc. You can also compare the results of feature importance between two model evaluations.

In fact, while you are trying to reach your business objectives, evaluating, comparing and applying feature importance to your models does not end the data science process. It just closes the data science loop and starts a new iteration. Usually, lots of iterations are required to reach your final goal. You will often uncover new patterns, gain new insight, re-engineer your features and test different parameter combinations to maximize the performance of your model.

Iteration occurs at the following stages:

Model building and parameter optimization

Model building and parameter optimization

The most common iteration is to tune the parameters of your model with different parameter configurations and retrain it on the same dataset. A model has lots of parameters to configure and much more combinations that you can try out before you feel that the model is delivering top performance.

The approach that Feedzai uses to optimize parameters is to perform a grid search (parameter sweep) where Pulse creates a model for every combination of values.

Feature engineering

Feature engineering

You can reiterate your data science work from the Feature engineering stage. You can try to improve your model by extracting extra/different features from the original data that better correlate with the problem you are solving, for example, fraud detection, define more profiles and further enriching the data.

Getting average results is usually fairly straightforward when you already have some domain knowledge and compute a set of generic features. However, pushing a model to its optimal performance, and addressing the long tail of the problem is very hard and requires many Feature engineering iterations. You'll most likely end up designing much more features, that are very specific to your use case and data than you did for the initial iterations.

Data analysis and sampling

Data analysis and sampling

Reiterating your (almost) entire project is more common in the earlier stages when you are still unfamiliar with the available data. As you know the dataset better, your need to re-analyze the data will gradually decrease.

It is also common that you only uncover problems in your sampled data after you complete a full iteration and evaluate the results against external data, causing you to have to resample it and better balance it for your specific use case.

Automate data science tasks

To accelerate the data science loop, you can automate some of the above tasks.

In Pulse, there are two possibilities for that:

1. Automate steps 2-5 with AutoML

The [AutoML](#) feature of Pulse automates almost the whole data science loop, from step 2 (Analyze & Sample) to step 5 (Evaluate & Compare). It helps at the kickoff of a new data science project, or when you want to augment the feature set of an existing model. [Start an AutoML run](#) by defining the starting data source, the metric to optimize, and some other parameters. AutoML does the following for you:

- Creates a feature [Plan](#) and executes it, resulting in a new dataset with the engineered features.
- Creates a Hyperparameter Tuning to train models, and to evaluate them. AutoML prepares the best model for usage.

2. Automate steps 4-5 with Hyperparameter Tuning

Use [Hyperparameter Tuning](#) to automate steps 4-5 of the data science loop: model building, parameter tuning, and model evaluation. To [start a Hyperparameter Tuning job](#), select a set of ML algorithms and the parameter values you want to test them with. As a result, Pulse does the following for you in an automatized way:

- Trains all the selected models with all the defined parameter values (parameter sweep).
- Evaluates the trained models.

Runtime activities

Runtime activities

When your data science work is finished and the data science loop is closed, it's time to deploy it to Production. These deployment activities are usually reserved for engineers as the runtime environment is greatly different from the offline data science stage.

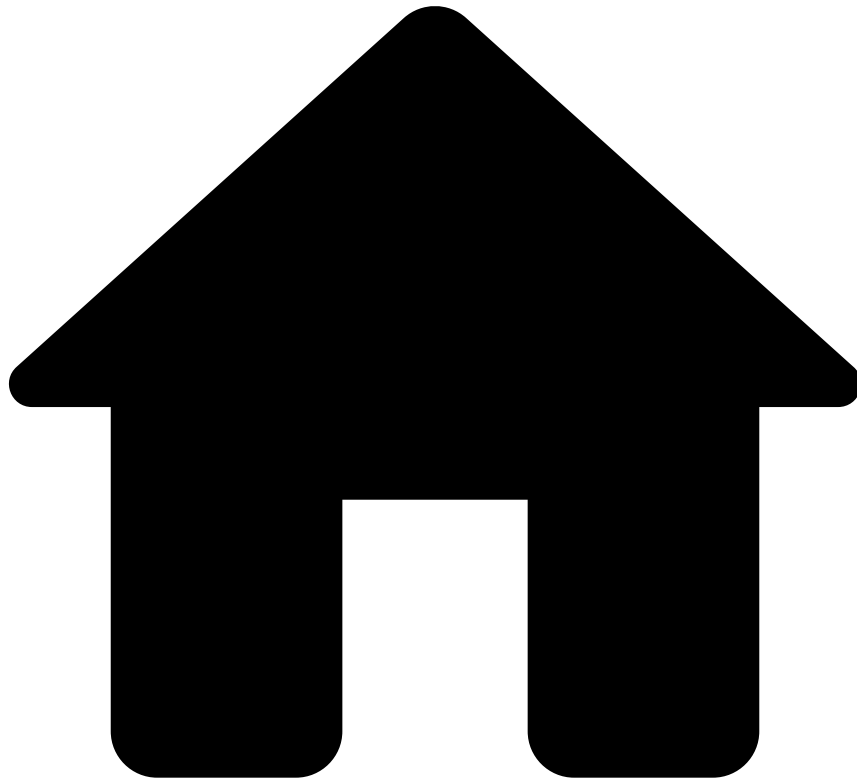
Integrating the results of the data science loop with the runtime environment is usually very challenging, time-consuming and requires advanced technical skills. It's also an iterative process of deploying, evaluating and benchmarking the results against the expected performance as designed during the data science stage. For every model that needs to be deployed, new engineering work is required.

It may take up to several months for engineers to be able to ensure that your data science work is correctly translated to runtime, maintaining the same semantics for data transformation, time-based profiles, enrichment and any feature, and that it yields the same results.

However, Pulse already integrates the data science and runtime environments making it much easier and quicker to deploy a project to Production. This makes runtime iterations much faster and safer as Pulse ensures that data science semantics are correctly translated to Production without requiring you to have a set of very sophisticated technical skills.

Learn more about [how Pulse projects work](#) and how they integrate the data science loop and runtime tasks.

-



- [Core Concepts](#)
- Data Sources

On this page

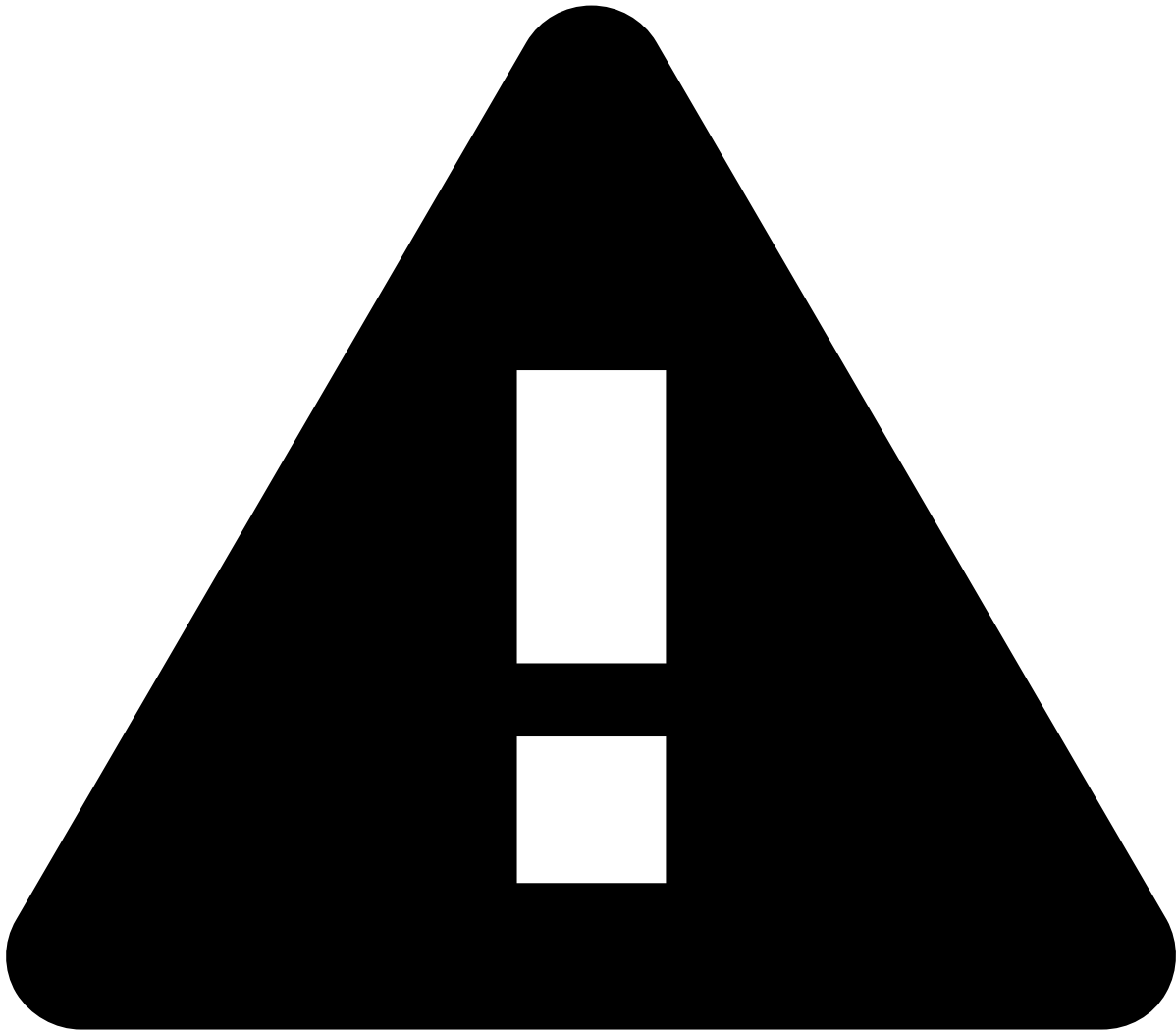
Data Sources

Was this page helpful? 

Data sources represent datasets that will be used for data science purposes within Pulse. An example of a source of data is a CSV file.

All data science concepts of the [data science loop](#) depend on data or its schema:

- [Models](#) and [rules](#) use data sources to perform decisions.
- [Plans](#) receive a data source as input, and generate a transformed data source as output.



warning

When you create a new data source, you never load the actual data into the system. Pulse only maintains a pointer to the physical location of the data file, as loading the whole data set would be too memory-consuming. Pulse fetches the data from the file(s) from the physical location whenever a data science task requires it.

Do not delete the data file from your system, otherwise any subsequent data science tasks that depend on it will not work.

Data fields📄📄

Data sources have **fields**. For example, in a CSV file, these correspond to the columns. Each field is characterized by a [data type](#), which defines the possible values of the field.

A data source is comprised of records. For example, in a CSV file, each record is represented as a row. Typically, each record contains a value for each field. However, it is possible for a field in a record to have no value, i.e. *missing value*.

The following image shows how Pulse displays fields and values of a data source.

Data schemas📄📄

When you create a data source and import information into Pulse, it also creates a **schema**. A schema describes the available data fields of a data source, as well as their [types](#).

Data might come from different sources and in different ways, and be manipulated at almost every stage of your project. Schemas, therefore, provide formatting and structure for data source manipulation/exploration during a data science stage and for real-time data coming into Pulse in Production.



info

All intermediate data manipulations in Pulse, whether during the Data Science stage or in Production, generate new schema instances. However, these schemas are internal and not accessible to characterize runtime data.

To translate the underlying (and implicitly created) data schema of a data source into a schema that you can configure for a runtime [workflow](#), you must explicitly export the data source to a schema.

As an accelerator, the workflow allows you to directly copy the schema from an existing data source.

Schema immutability

When you successfully create a data source and import data into Pulse, it automatically also defines the corresponding schema. Once you have a data source, you cannot edit or update its data schema.

Data schemas, and consequently, data sources are immutable due to the fact that most data science concepts and tasks fully rely on data sources. Therefore, to keep your project consistent, whenever you change the underlying structure of a data source, Pulse would have to propagate those changes further down your data science pipeline to [plans](#), [models](#), and [rules](#), where the data source has been used. Additionally, this would have potentially disastrous and hard-to-solve implications if you were already using plans, models, and rules in a runtime [workflow](#).

However, changing a data schema after it has been involved, for example, in training a model, wouldn't even make sense, since model configurations and results are, by definition, also immutable.

On the other hand, changing a data source schema, with different field names and data types, and not propagating changes would, for example, break plans that were already configured to consume data in a specific way.



info

If you need to change the data schema of your data sources, you can do it by [duplicating the data source and then editing the data configuration](#).

In reality, since the schema is immutable, this effectively generates a new data source with an updated schema.

In a data science process, whenever your data moves along a plan and suffers transformations, the data schema of the input data source remains unchanged. To reflect the result of data manipulation and transformations, since schemas are immutable, Pulse generates a new data source with a new schema. The subsequent stages in your data science pipeline should be configured accordingly to use the output data source.



info

Since everything relies on data, every time you use data and its schema definition during the data science stage, Pulse must ensure that everything is correctly translated into the runtime environment.

Every time you create/configure an entity that requires data and can be deployed directly into Production, Pulse uses a copy of the original schema to maintain the integrity of the original. For example, when specifying the input data schema for a [deployable plan](#) or when configuring a [rules snapshot](#).

Compatible data sources ⓘ

In data science mode, operations are always done over the data source files that you import, such as CSV or Parquet files. After deployment, the data can come in different formats. For example, you can receive data over a socket reading

ISO8583 messages or via a REST API. Production data must be compatible with your data sources to ensure the data processing operations will continue to work.

Moreover, if you want to, for example, run an evaluation on a trained model using different data, you also need to make sure it is compatible with the dataset originally used to train the model. Otherwise, the experiment will not be useful for your data science project.

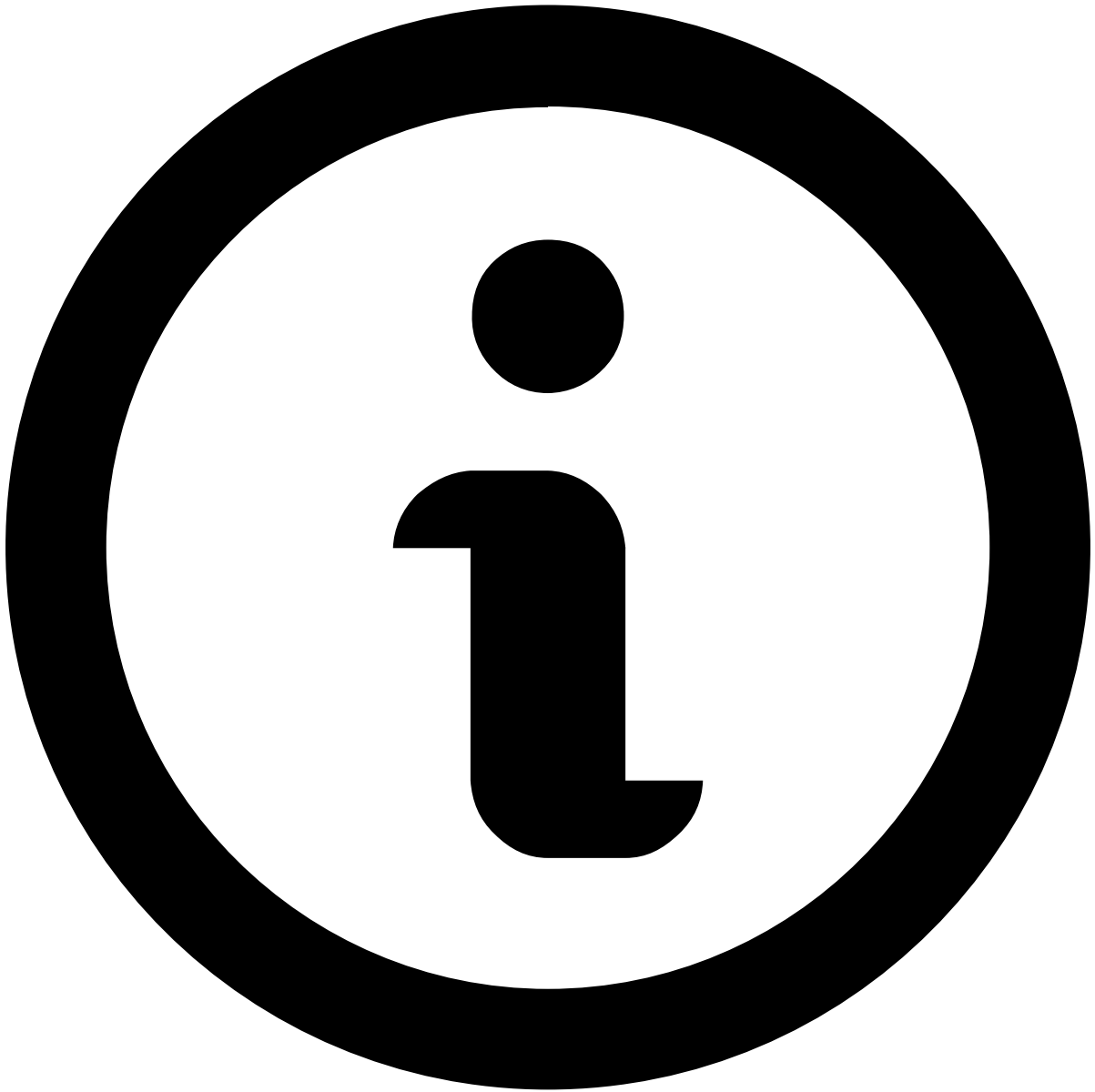
In Pulse, two data sources are considered compatible according to the following table:

Concept	Compatibility strategy	Description
Plans	Equivalent	New schema must be equal to the one of the original data. Data must have the exact same format (type and possible values), order and names.
Models	Superset	New schema can have any fields as long as it contains all fields of the original schema, with the exact same format (type and possible values) and names. The order is irrelevant in this case.
Rules	Superset	New schema can have any fields as long as it contains all fields of the original schema, with the exact same format (type and possible values) and names. The order is irrelevant in this case.

Invalid instances🔗📄

Invalid instances are data rows (aka. instances, records, or events) that could not be read from the dataset for one of the following reasons:

- The data row contains more or less data fields than the schema.
- The data in one or more fields of the row is not in the format that the schema expects. This has three cases:
 - The type of the input data does not match the data type defined in the schema. Example: "10.0", when the field in the schema is an int, instead of a double.
 - The format of the input data does not match the format configurations of the data type defined in the schema. Example: "01-01-1990", when the field in the schema is a Date Time configured with the YYYY-MM-DD format.
 - The data field is a categorical, and the value of the input data is not in the list of the possible values defined in the schema. Example: "CAN", when the categorical type of the schema is configured to accept "CAD", "CNY", "EUR" and "USD".



note

Invalid instances are ignored in data science activities, for example when calculating metrics in a [data source analysis](#), executing a [Plan, training a model](#), etc.

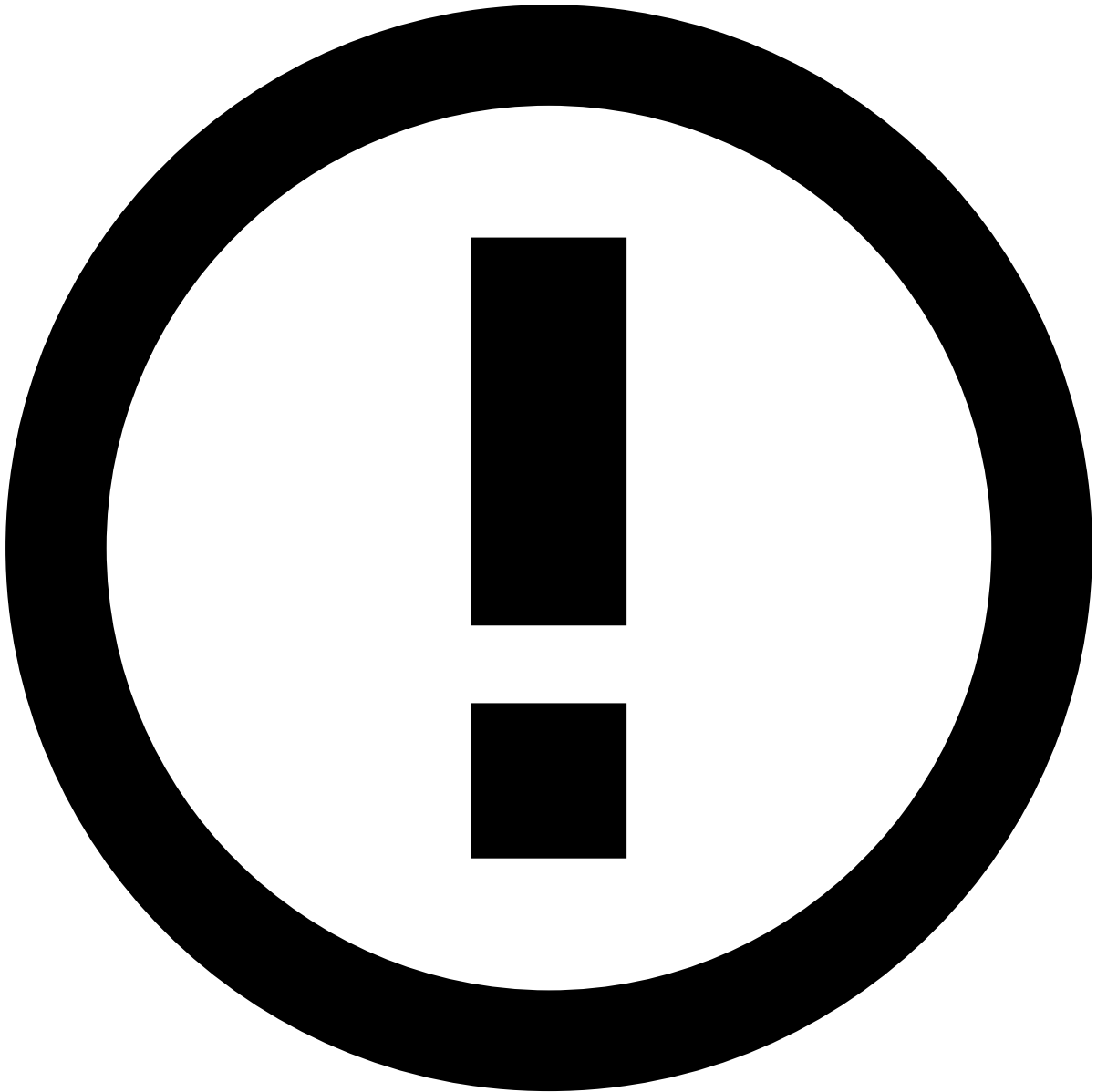
The Pulse UI shows information about invalid instances after you run a [data source analysis](#).

To fix invalid instances, after investigating their cause in the UI, you can combine the following two approaches:

- Change the data.
Preprocess the dataset externally (e. g. using Python), to transform invalid values into valid ones.
This approach helps when in the same data field, in different records there are different types or formats.
- Change the schema.
Duplicate the data source, and in the new data import, change the data type or the configuration of the data type.
This can help if there is an issue with the field's data type in the schema, affecting all values of that field. For instance, in the case of a missing categorical value, or if you picked the wrong setting when defining a data type.

Missing values

Some records in your dataset will most likely have empty fields (perhaps caused by faulty equipment). These, if untreated, would also lead to misleading results or failure during the training. Hence Pulse allows you to define how to represent missing values.



info

When you create a data source from a CSV file, you can define any of the following representations to be treated as missing values:

- NULL
- null
- N/A
- Empty string

When you run an [analysis on your data source](#), it might indicate that the count of missing values is very high for an important field. In these cases, you may need to re-engineer your features or build a [data cleaning plan](#) that detects and removes records that match the missing value representation you configured.

In case there are missing values for a target variable, those records are ignored in [model evaluations](#) and [rule evaluations](#).

•



- [Core Concepts](#)
- Jobs and Job Profiles

On this page

Jobs and Job Profiles

Was this page helpful? 

Jobs allow Pulse to execute computational tasks in a distributed way in a Spark cluster. Job profiles are useful to fine-tune how Jobs use available resources in a Spark node. The following sections describe in more details what they are and how they work.

Jobs, Job Executions

A job is a set of computational tasks for a certain goal. For example, to run a Model Evaluation the Pulse server needs to execute a set of calculations. Those calculations are the **tasks** of that model evaluation job.

To get the result of a job faster, Pulse distributes the tasks between the nodes of a Spark cluster. Hence jobs are often referred as **Spark jobs** or as **distributed jobs**.

A job execution is an instance of a job, which is created when the job is executed. Imagine a job as a list of instructions on how to execute a complex process or computation (e. g. a Hyperparameter Tuning or a Plan Execution). The job execution, on the other hand, is an instance of a job, created when a user or a program triggers Pulse to execute those instructions on a specific set of data.

Pulse creates a job execution for the following:

- [Data Source Analysis](#)
- [Feature Importance](#)
- [Model training](#)
- [Model Evaluation](#)
- [Plan Execution](#)
- [Rules Evaluation](#)
- [Sandbox Evaluation](#)
- [Hyperparameter Tuning](#)
- [AutoML run](#)
- [Scheduled Metrics Jobs](#) (SMJs)

You can check job executions and their progress in the **Jobs** tab , under **Job Executions**.

Job Profiles

A job profile is a set of Spark configurations that control how Spark jobs behave. For example, you can use them to ensure that Spark assigns [enough memory](#) to the node where the job is executed.

You can check Job Profiles in the **Jobs** tab , under **Job Profiles**.

The scope of Job Profiles can be the following:

- **Default for the App:** applies to all jobs of the Pulse application. For each Pulse application, only one job profile can be [configured as default](#).
- **Default for a specific resource type:** each of these job profiles apply to one resource type of the following list:
 - Data Source Analysis
 - Feature Importance
 - Model Training
 - Model Evaluation
 - Plan Execution
 - Rule Snapshot
 - Sandbox Evaluation
 - Hyperparameter Tuning
 - SMJs

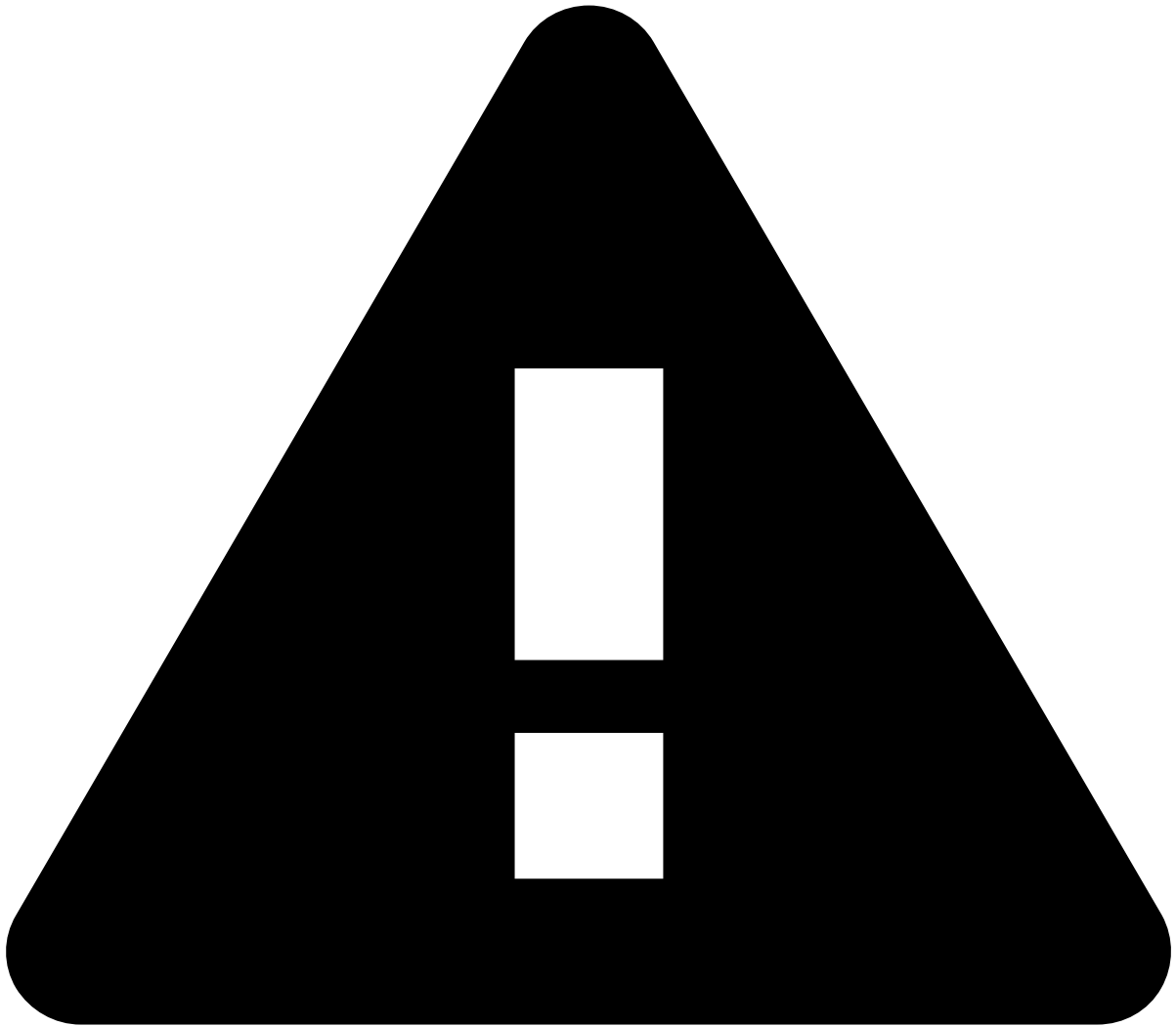


info

It's optional to have a default job profile for a resource type. If there is [a job profile configured as default for a resource type](#), for that resource it overrides the default job profile of the application.

- **Applied to a single job:** when creating a Data Science resource that can use job profiles, you can [select a job profile other than the default](#), or you can [edit the properties on demand](#), for that single job execution that you are creating.

To understand which properties were used in a job execution, open the Job Execution and select the button.

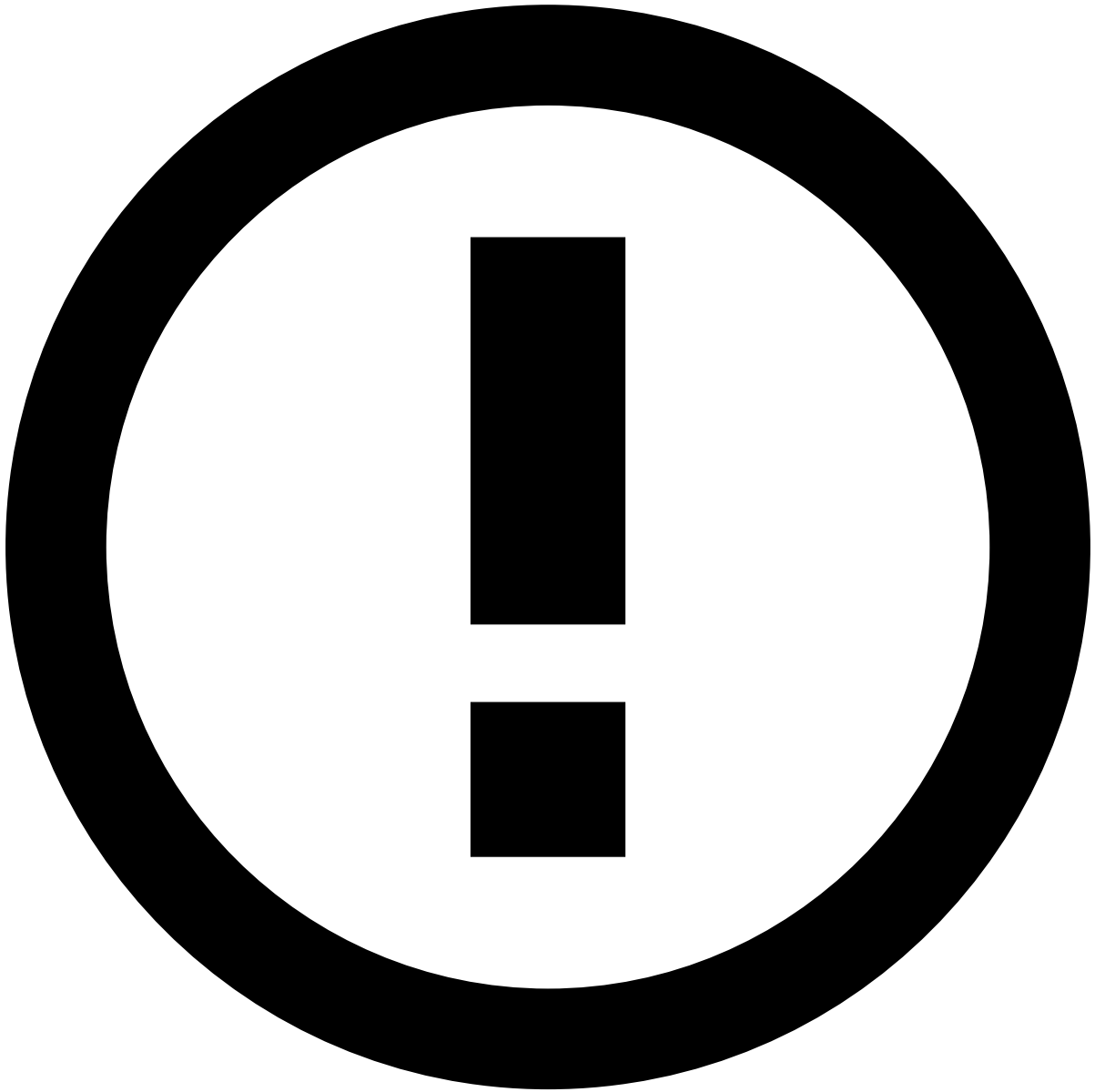


Resources without default Job Profile

- It's not possible to use job profiles with Registered Jobs and [Scheduled Metrics Jobs](#).
- Although an AutoML run creates a job, you cannot assign a default Job Profile to AutoML jobs. This is because the AutoML job will start other jobs (Plan execution, Hyperparameter tuning, etc.) which can have their default job profile.

Pulse Properties Profile

The Pulse Properties Profile (PPP) contains a read-only list of the Spark properties that are loaded from a preconfigured source.



info

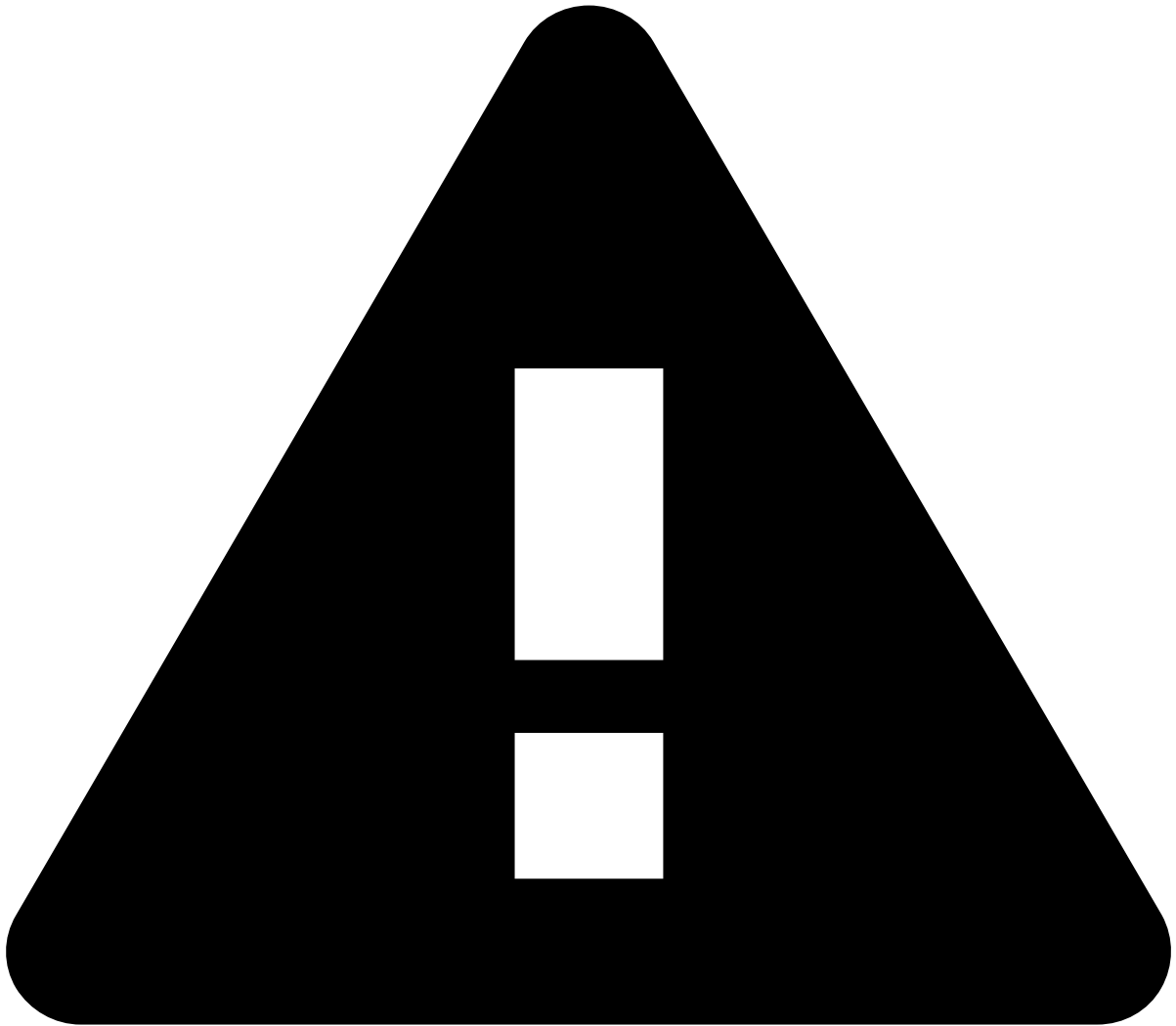
By default that source is the **pulse.properties** file located in the **conf** folder. However, the PPP can be configured to read from other types of sources, such as RDBMS, NoSQL (Cassandra), etc.

You can check the PPP by accessing the **Jobs** tab and opening the PPP in the **Job Configuration Profiles** table as shown in the following image:

When you [create a new job profile](#), the properties in the PPP are automatically inherited by this new profile. You can edit them, or add new Spark properties to the new job profile.

Updates in the PPP

When properties in the PPP are updated, those changes are also automatically propagated to the other job profiles. For example, if the PPP is updated with a new Spark property, it is also automatically included in all the existing job profiles.

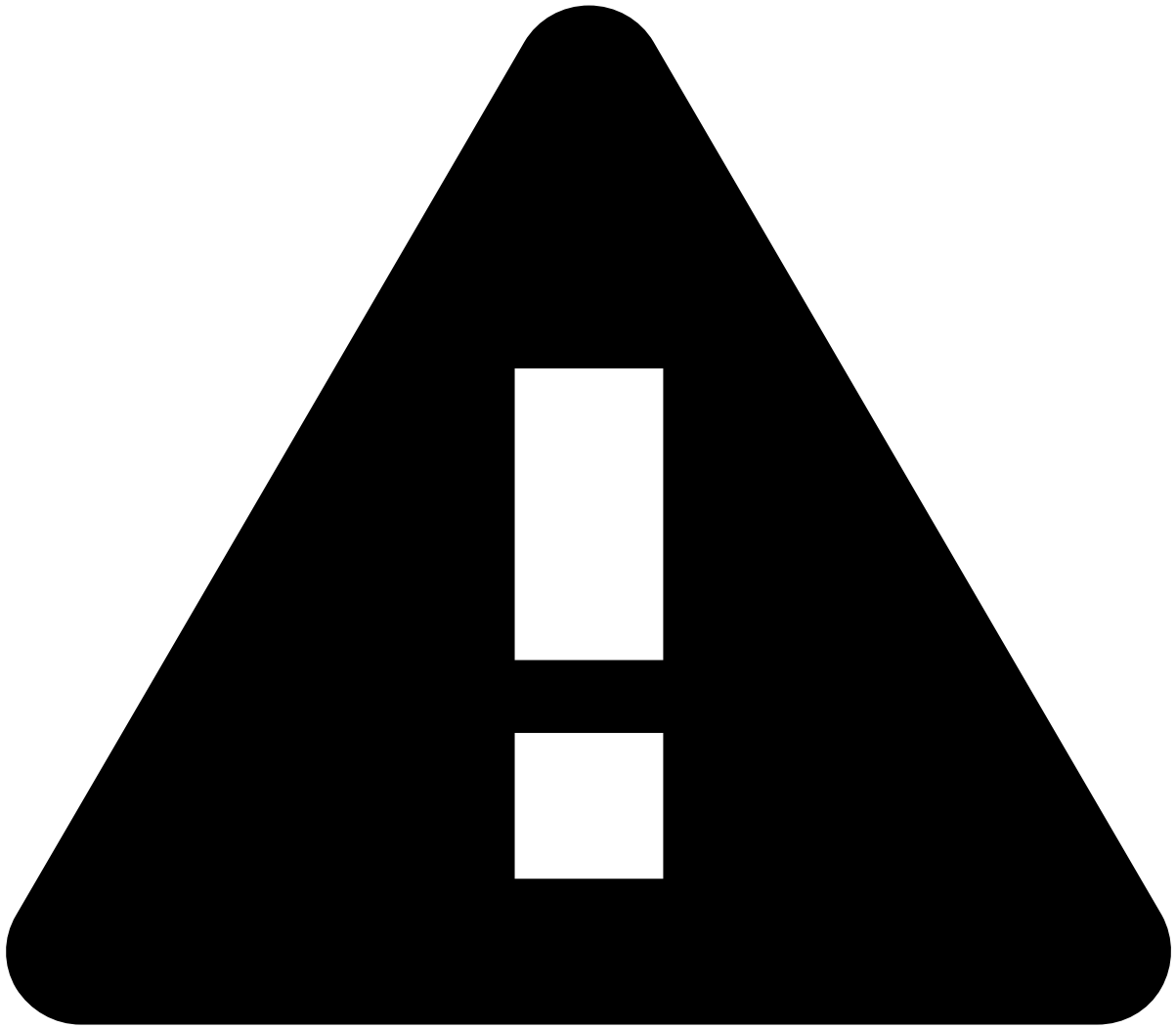


Exception: updates that are not propagated

When in the PPP you update a property which already exists in a Job Profile with a different value, the change is not populated from the PPP to the Job Profile.

Group Properties Profile [🔗](#)

The Group Properties Profile allows specifying Spark properties at the level of Hierarchical Groups.



caution

Spark properties defined in the Group properties override those defined in the Job Profile, which in turn override those from the PPP. The order of precedence is therefore as follows: Group properties > Job Profile > PPP.



danger

This is an experimental feature that currently only works for Plan Execution jobs that employ a persistent PKernel via RocksDB.

In addition to static job profiles, we can also generate automatic configurations: this is an experimental feature that currently only works for Plan Execution jobs that employ a persistent PKernel via RocksDB.

The Spark configuration properties that are generated by the Job Profile Tuner are:

- spark.memory.fraction
- spark.memory.storageFraction
- spark.task.cores
- spark.executor.cores
- Spark.executor.memory
- spark.executor.memoryOverhead

For more information about these properties you may refer to <https://archive.apache.org/dist/spark/docs/3.3.2/configuration.html>. These properties override the ones in the default static profile.

When running a plan execution be aware that not all plans are eligible for auto-profiling. The conditions for eligibility are:

- The plan must not be omnichannel
- The plan must be a directed path graph (or linear graph); i.e., only a data source and a sink and each other operator in the plan must have at maximum one upstream operator as input and one downstream operator as output.

Under the hood, the job profile tuner (JPT), which is the component in Pulse that generates automatic profiles, uses heuristics that based on the job's input data and workload, as well as cluster availability state, attempt to generate a configuration that not only make a job viable (by executing it successfully) but also performant (minimizing latency while using resources efficiently). In a nutshell, the JPT attempts to predict the maximum memory that a Spark executor will require by assessing the artifacts that need to be maintained in memory during the job execution, such as the state of stateful operators (e.g., metrics), Pulse managed lists, and enrichers.

Since using a persistent PKernel all the times can incur a penalty in the job performance, we have a mechanism to automatically select the most appropriate state backend of the PKernel based on job characteristics, namely data distribution and window durations. To enable it, check the property `automaticStateBackend` in DataScience PKernel configurations. The state backend selection will happen before the tuning of job profiles and, therefore, a profile can only be generated if the selector picks a persistent state backend for PKernel.

To help on debugging eventual problems with the JPT, which is still an experimental feature, users may also enable the automatic collection of job performance metrics: see the property `collectPerformanceMetrics` in Spark Job Service Configuration.

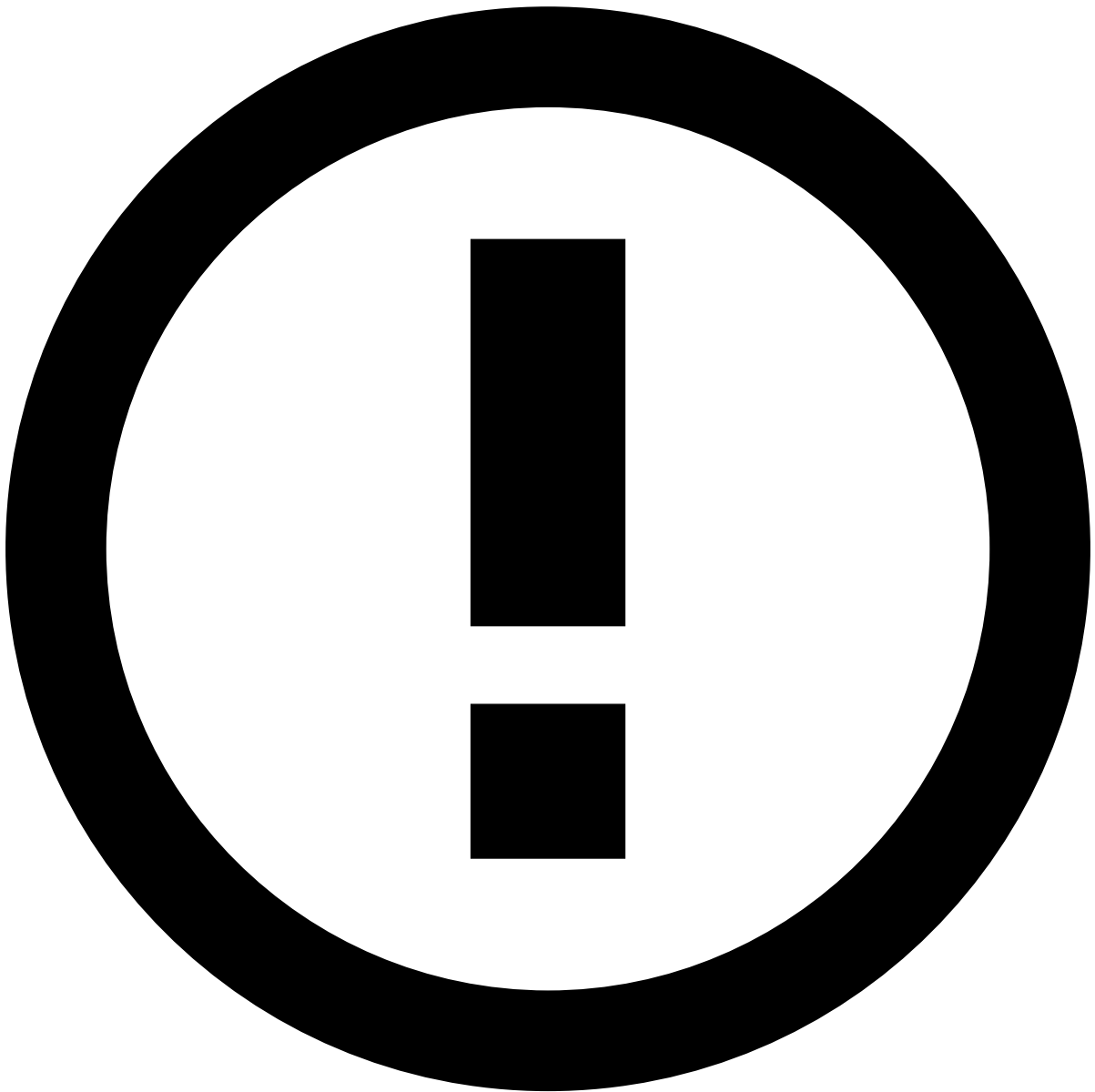
Memory considerations

When running Spark jobs, it's recommended to have a default profile for all jobs, and define job profiles for executing resource-intensive jobs.

Calculation tasks may fail if they have insufficient memory available. This might happen because each Spark node has fixed resources available, for instance a fixed number of cores and a fixed amount of memory. Some nodes may have larger resources than others, but by default it's not sure that the tasks that need large resources get assigned to those nodes. Typical examples of tasks needing larger memory are Model Trainings, Hyperparameter Tunings and Plan Executions.

If smaller jobs occupy the resources that have larger capacity, Pulse doesn't leverage the full potential of the Spark cluster. To ensure that the necessary resources are available for jobs that need them, you can use Spark properties to influence the number of cores and the amount of memory used by a job. Job profiles can regulate what kind of resources should be assigned to a job which is running with a specific job profile. For instance, you can have a default profile for all jobs, but define a job profile for executing Plans, and another one for Hyperparameter Tunings to make sure that these get the proper resources they require.

If these job profiles are created, you can use them when creating a job execution of a related Data Science resource.



Restrictive default settings

The default Spark settings are restrictive, for example, they allow 1GB of memory per node. It is recommended to create a Spark configuration strategy early on in a project, and keep adjusting the settings as you monitor the Job Executions.

Learn more📖

Learn how to do the following tasks related to Jobs and Job Profiles:

- [Creating a job profile](#)
- [Selecting a job profile](#)
- [Edit properties for a specific job execution on demand](#)

-



- [Core Concepts](#)
- Limits

On this page

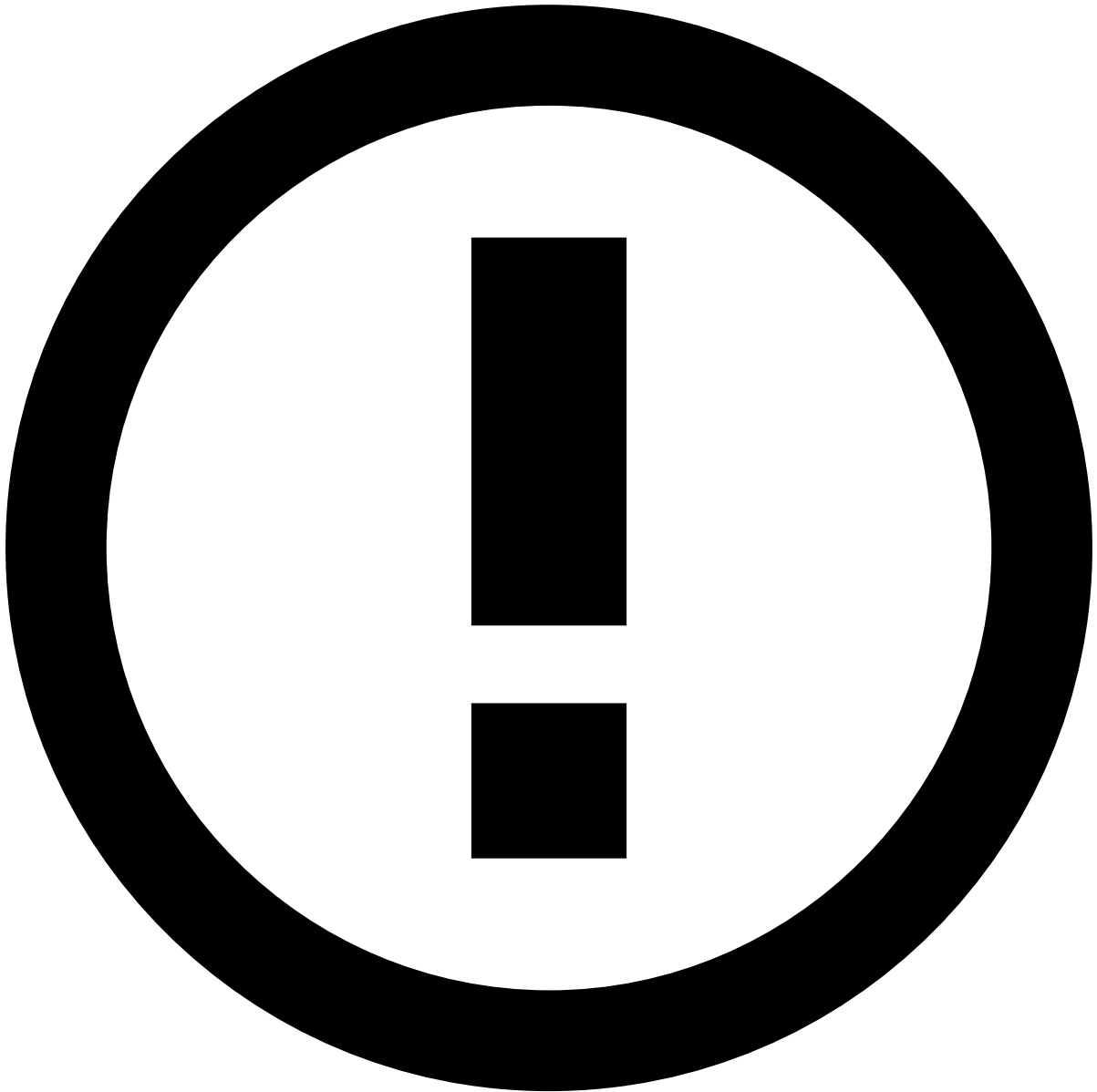
Was this page helpful? 

Limits

Within Pulse, **limits** (also referred to as **quotas**) are upper bounds on the number or size of resources that can be configured in an [application](#).

Limits act as guardrails to guide users towards recommended usage patterns and best practices, with the goal of helping ensure optimal runtime performance, and preventing issues such as degraded stability in production environments.

Pulse limits are defined individually for each environment, allowing flexible setups while maintaining control over resource usage. Specifically, they are primarily targeted at runtime environments, where performance and stability are critical.



Limit adjustments

Pulse limits are performance safeguards, not rigid constraints.

If your usage requires higher limits, reach out to the Feedzai Customer Service to request the necessary adjustments.

Limits on Metrics operators in Plans

Pulse limits the number of [Metrics operators](#) that can be included in a [Plan](#). By default, the limit is set to two Metrics operators per Plan, which allows common use cases such as cascading metrics.

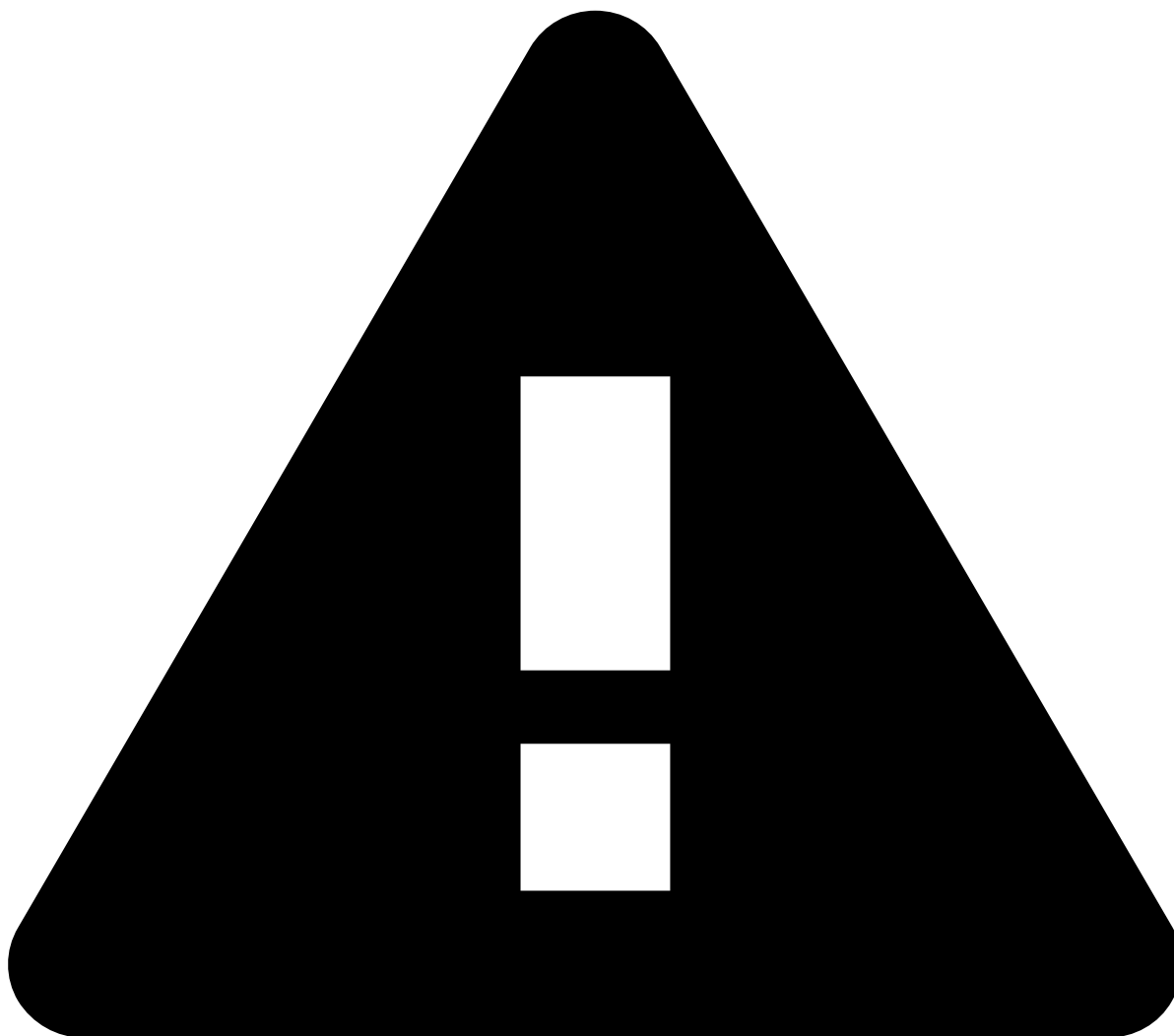
The [Create/Edit Plan page](#) displays the usage of the Metrics operators quota above the plan data flow diagram. The indicator shows how many Metrics operators are currently used out of the allowed limit, e.g., "Metrics operators in use: 1 of 2":

When the limit is reached, the **Metrics** operator becomes disabled, so that no more operators can be dragged onto the plan diagram:

Additionally, dragging Metrics operators from the clipboard pop-up into the plan diagram is blocked if it would exceed the limit:

Existing Plans exceeding the limit

If an existing Plan already had more than the allowed number of Metrics operators before this limit was introduced, it can still be opened and edited as usual. However, no new Metrics operators can be added until the total number is reduced below the limit.



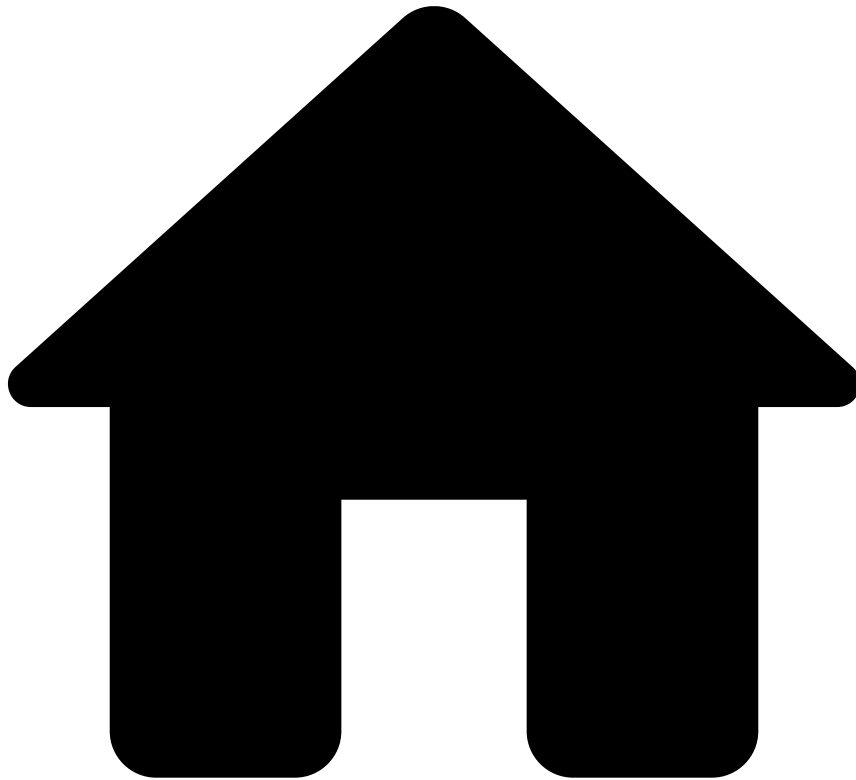
Importing Plans exceeding the limit

Support for existing Plans exceeding the Metrics operators limit only applies to Plans currently in the application. [Importing resources](#) that include Plans with more than the allowed number of Metrics operators will fail with an error message indicating that the Metrics operators limit has been exceeded.

Learn More

For more detailed information about organizing metrics across multiple operators, see [Best Practices for Plans](#).

-



- [Core Concepts](#)
- Lists

On this page

Lists

Was this page helpful? 

A list stores a set of items. Lists are available for [rules](#) to check the same pattern against a large number of values. E.g., a list of credit card numbers, a list of merchants, etc.

In the process of fraud detection is very common to define blacklists for blocking transactions from certain cards, or even define transactions with common parameters that should be up for review (triggering an alarm every time a transaction occurs). Lists allow you to group and manage these values so that you can use them efficiently with Rules.

When you define a list, the first thing you need to consider is which list type is best suited to your use case. Lists are essentially made of items, typically hundreds of them, where information will be stored. Depending on the items you must choose between one of the following:

- **Simple list:** When only one piece of information is needed. Represents a list containing only numbers or strings (commonly used for blacklists/whitelists).
- **Complex list:** When you need to store complex values, for example [tuples](#). Each item is identified by a key and that key has a predefined set of fields. For example, the key of an item may be *country* and its fields can be a set of *safe_merchants* for that country.

Complex lists allow you to define rules that first check for the existence of the key and then verify conditions over the other fields. For example, when a transaction comes into the system, lookup if the country of the transaction is in the list of risky countries. If so, check if the transactions were done on "safe merchants" for that country.

List Configuration

Imagine you want to create a list to store patterns used by potentially dangerous transactions. A [rule](#) can then verify if a transaction matches any dangerous pattern stored in the list and help reduce fraudulent transactions.

As an example, we will define the dangerous payments pattern as a country, a list of [MCCs](#) and a list of terminal authentication methods. The next steps describe how such a list can be created.

When configuring a list, pay special attention to the following parameters:

- **Name:** Choose a representative name as it will be used as the internal list name in RDL. This internal name can't be changed and has some differences to the list name. For example, "DangerousPayments" will generate an internal name of "dangerous_payments".
- **Tags:** Describe briefly the purpose of the list. Can be used in [API calls](#) to filter existing lists. For example, add the `suspicious` keyword to all the lists that contain dangerous patterns.
- **Key Label:** Lists are implemented as a key-value store. The key can store multiple types of information ranging from card numbers, to MCCs or country names.
- **Key Type:** Pulse supports two key types: **String** and **Number**. Consider the internal implementation of the data structure and access capabilities of lists to understand why limits the available key types.
- **Fields:** Defines the schema, or the internal structure, of items in a complex list.



High cardinality fields

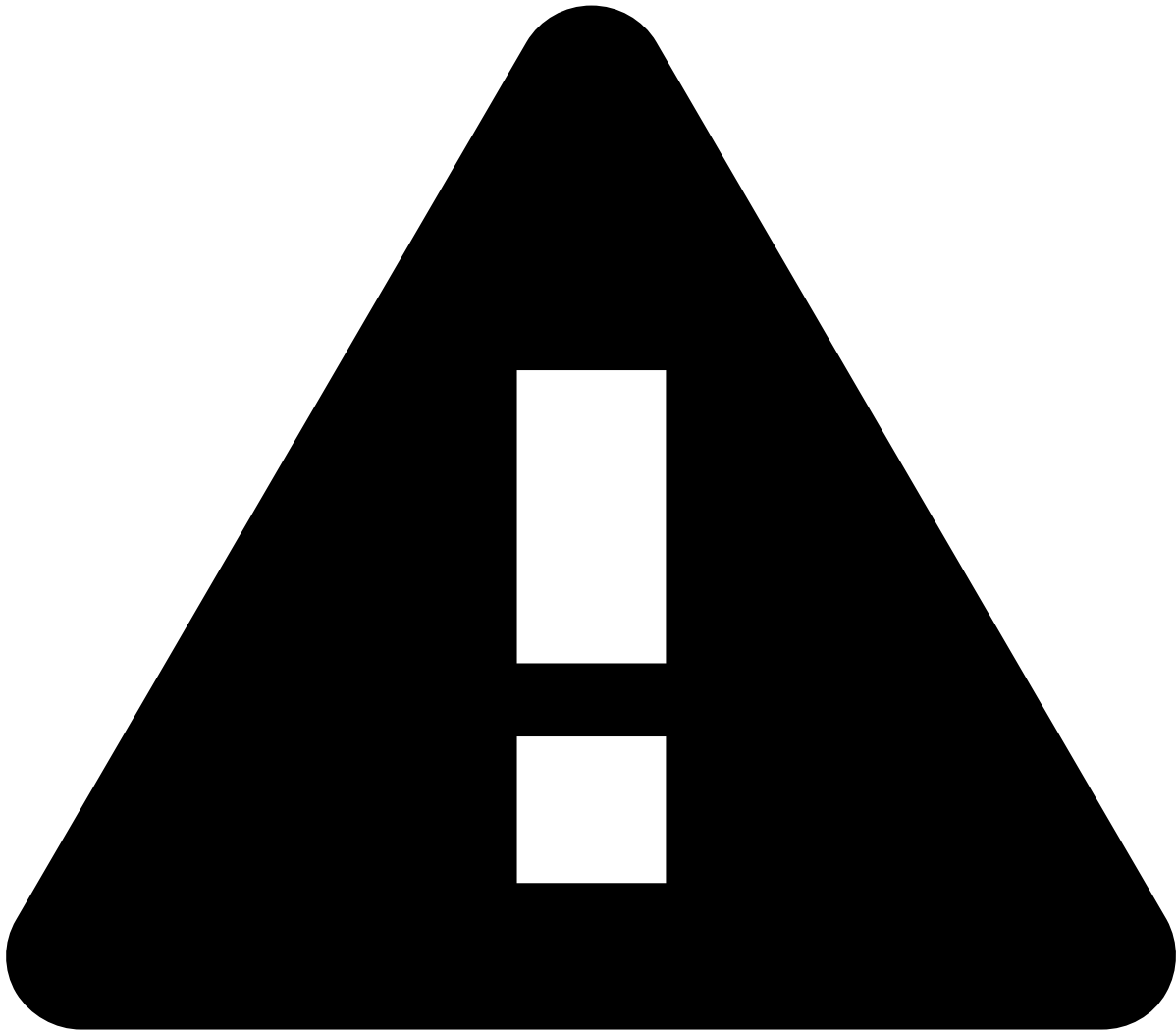
Each list item will be stored in memory from the moment Feedzai Pulse starts.

This has a direct impact on the consumed memory, thus having the potential to cause problems with fields that have high value cardinality.

List Items

After you have created a List you need to add items to it. In the case of a `DangerousPayments` list, the items are fraudulent patterns that represent suspicious transactions.

You can load entire lists, with a large number of entries, in bulk. Feedzai Pulse accepts a CSV file with the items' information and handles the addition process for each item transparently.

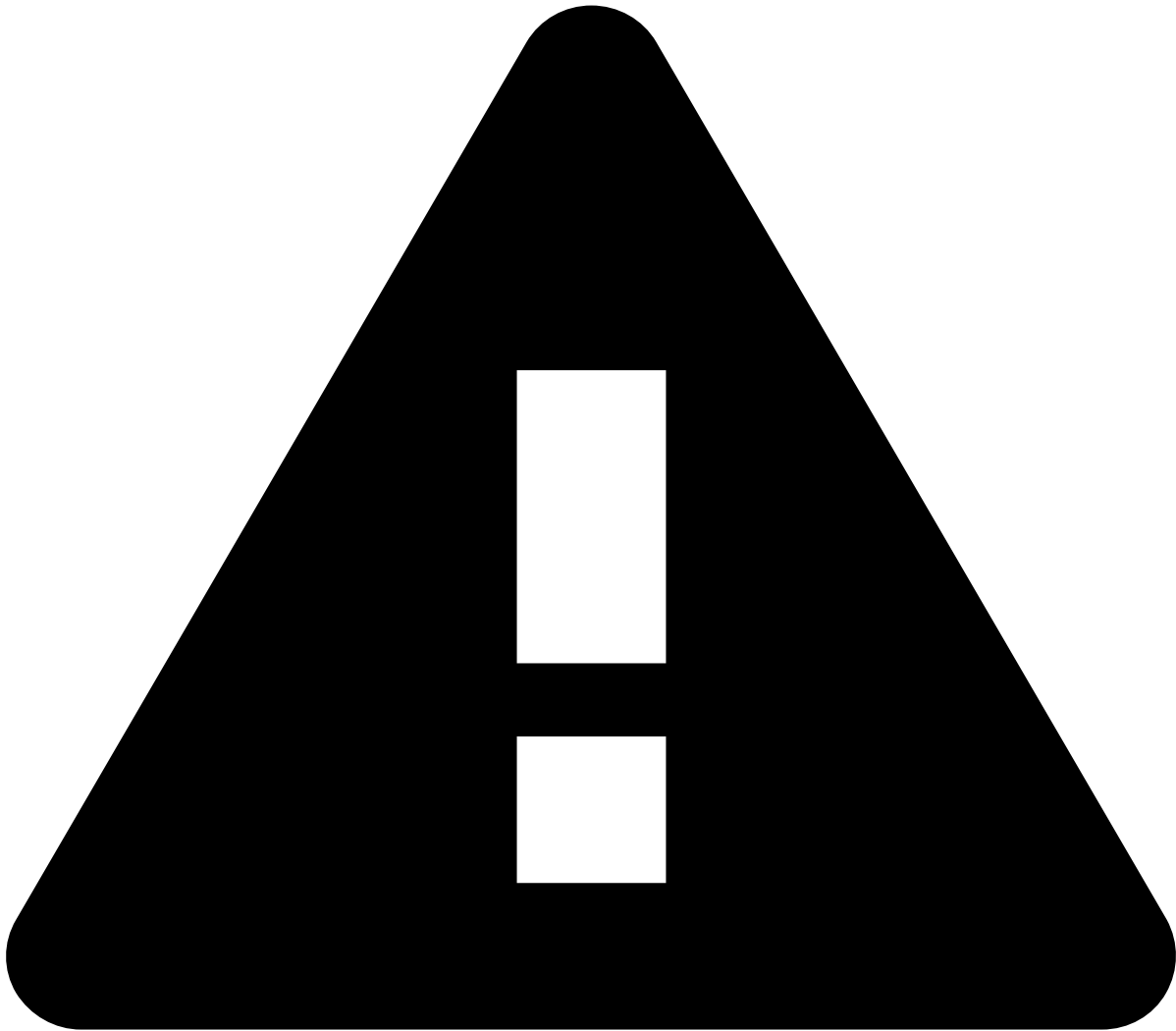


Upload Limit Recommendation

To ensure reliable processing and avoid straining system resources, it is recommended that each uploaded list file contains at most 40,000 items.

Note that this figure is a general guideline; the actual threshold for optimal performance can vary depending on the specific contracted resource sizing. If your list is larger than that, split it into multiple files before uploading.

Timespan support in items is an essential feature when creating watchlists. These lists allow you to, for example, add a card number that should be under watch during a given period of time after having suspicious activity. Lists support the following three timespan types:



Key uniqueness

The conjunction of the item `key + timespan` must be unique. In other words, Pulse will not add the card `0000-0000-0000-0000` to a List if it is already there with an overlapping timespan. For multi-tenant lists, the tenant unique id is also part of the unique key: `key + timespan + tenant`.

Note that the timespans are interpreted according to the workflow clock. See [Time, schedules and timezones](#) for more information.

Multi-Tenancy📄

Lists in Pulse also support the multi-tenancy concept. A list can be of the type **Global** if the list is aimed to be used in a context that is not Multi-Tenanted or of the types **By Tenant** and **By Tenant Group** if the list is aimed to be used in a multi-tenancy project.

In a list of the type **By Tenant**, each Tenant will be able to see and manage a subset of all the items without impacting other Tenants.

To further detail how to create lists for multi-tenancy, visit [Creating Lists and Adding Items](#).

Tokenized Lists

Once you have set up the Tokenizer, you will also be able to tokenize lists. For example, black lists can be sometimes composed of Credit Card numbers, which in order to fulfil PCI DSS compliance, must be encrypted. Therefore, lists can be tokenized by specifying it during its [creation](#). After defining the list as tokenized, you can see how the sensitive information has been encrypted.

For instance, when downloading the list of items, the CSV file no longer shows clear text PAN numbers, but tokens instead. All CSVs also show a column that indicates whether the key of the item is tokenized. In downloads, that column displays a **True** value if the list is encrypted, or **False** if it is not. When uploading, if set as **True**, it works as selecting the **Encrypted** checkbox in the item creation screen.

Reviewing List Changes

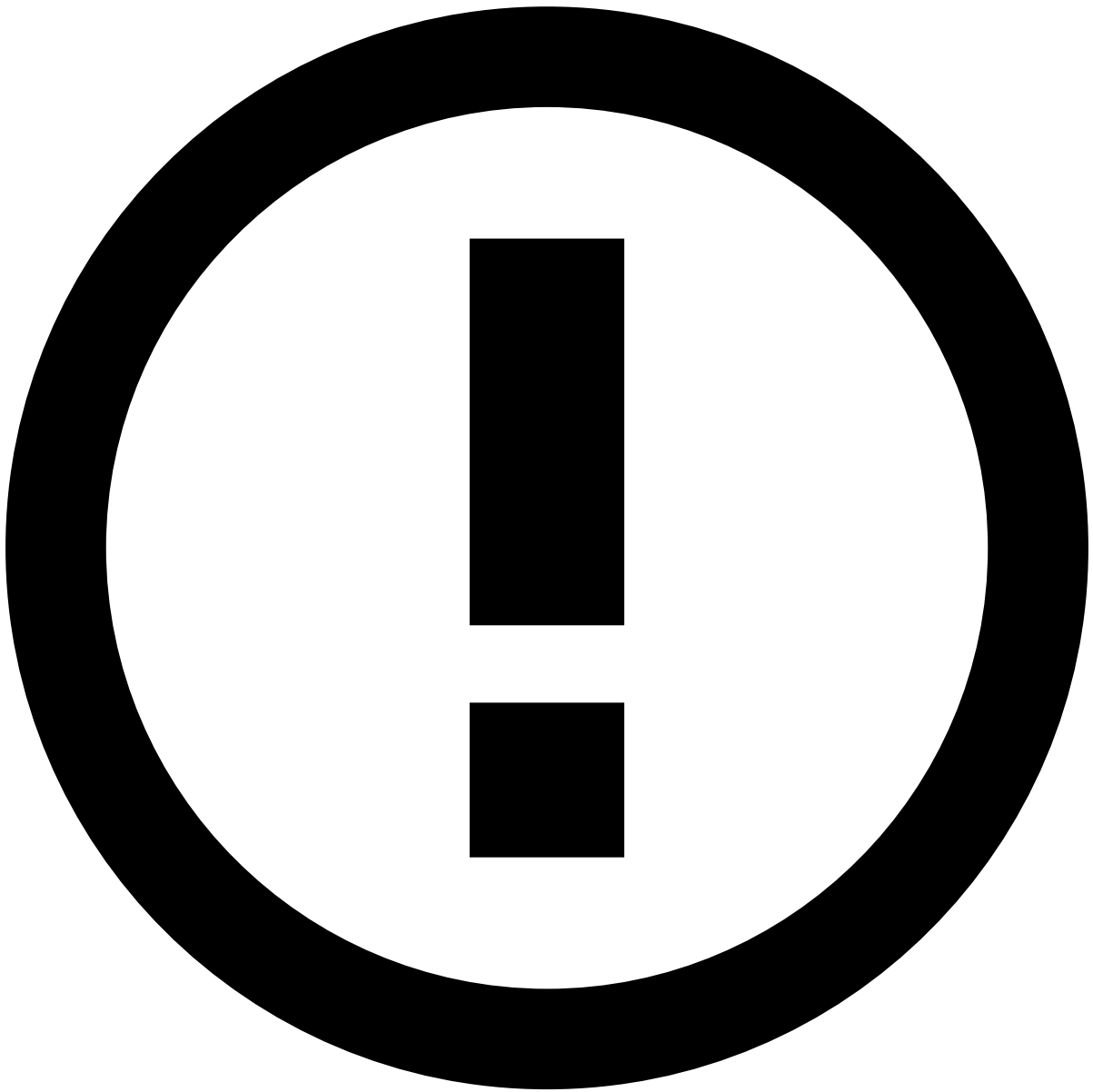
Pulse supports a "4-eye check" review functionality for list changes, which places edits made to lists into a review queue, and only applies the changes if they are approved by a second person.

To learn more about this functionality, see [Reviewing List Changes - Four Eyes Check](#).

Auditing

The purpose of auditing is to store the logs of changes (creation, edition and deletion) made to items in the lists. These audit logs display, among other data, when and who made changes to the items, as well as the corresponding new field values.

The audit entries are only accessible via Pulse REST API through a query by item key.



Pulse Database

Auditing is supported only by the relational Pulse Database (PDB).

Note that the audit entries are not visible in Pulse User Interface (UI); they are stored and available for other applications to fetch them.

-



- [Core Concepts](#)
- Models

On this page

Models

Was this page helpful? 

A **model** represents knowledge collected about the behavior of a **target variable** in a data source. You collect this knowledge by running a machine learning algorithm against a historical dataset, to investigate the data patterns in it. This process is called [training a model](#). You create a model in order to use it later in a [workflow](#) to predict the value of the target variable in live data.

After training a model, you can evaluate its accuracy and compare it to other models. For this, Pulse generates statistics in a so-called [model evaluation](#).

Models and model evaluations are organized into **model projects**.

Model life cycle

Models give your application an intelligent edge by allowing you to configure, train, fine tune and deploy a binary classifier which will score each instance. The model lifecycle is fully aligned with Pulse's [data science loop](#).

Before using a model to score a system's incoming transactions for fraud detection, it must be properly trained using representative and verified data labeled for your target variable, for example, both fraud and non-fraud cases.

Model usage is separated into two distinct lifecycle phases:

- **Training and evaluation** (in the data science environment)

Is an offline process that can be repeated with different settings and data until you are satisfied with the results.

The performance of trained models is [evaluated](#) using common performance metrics, such as F1-Measure, False Positive Rate, among others and by visualizing and comparing its ROC curve.

- **Scoring** (in the production environment)

When model performance is good, meaning that it is achieving high detection and low false-positive rates, it is ready to be used in the fraud detection workflow of your application.

Learn more

To fully understand models, make sure you also go through the following concepts:

- [Model training](#)
- [Model evaluation](#)
- [Feature importance](#)
- [White-box explanations](#)
- [Hyperparameter tuning](#)
- [AutoML](#)

-



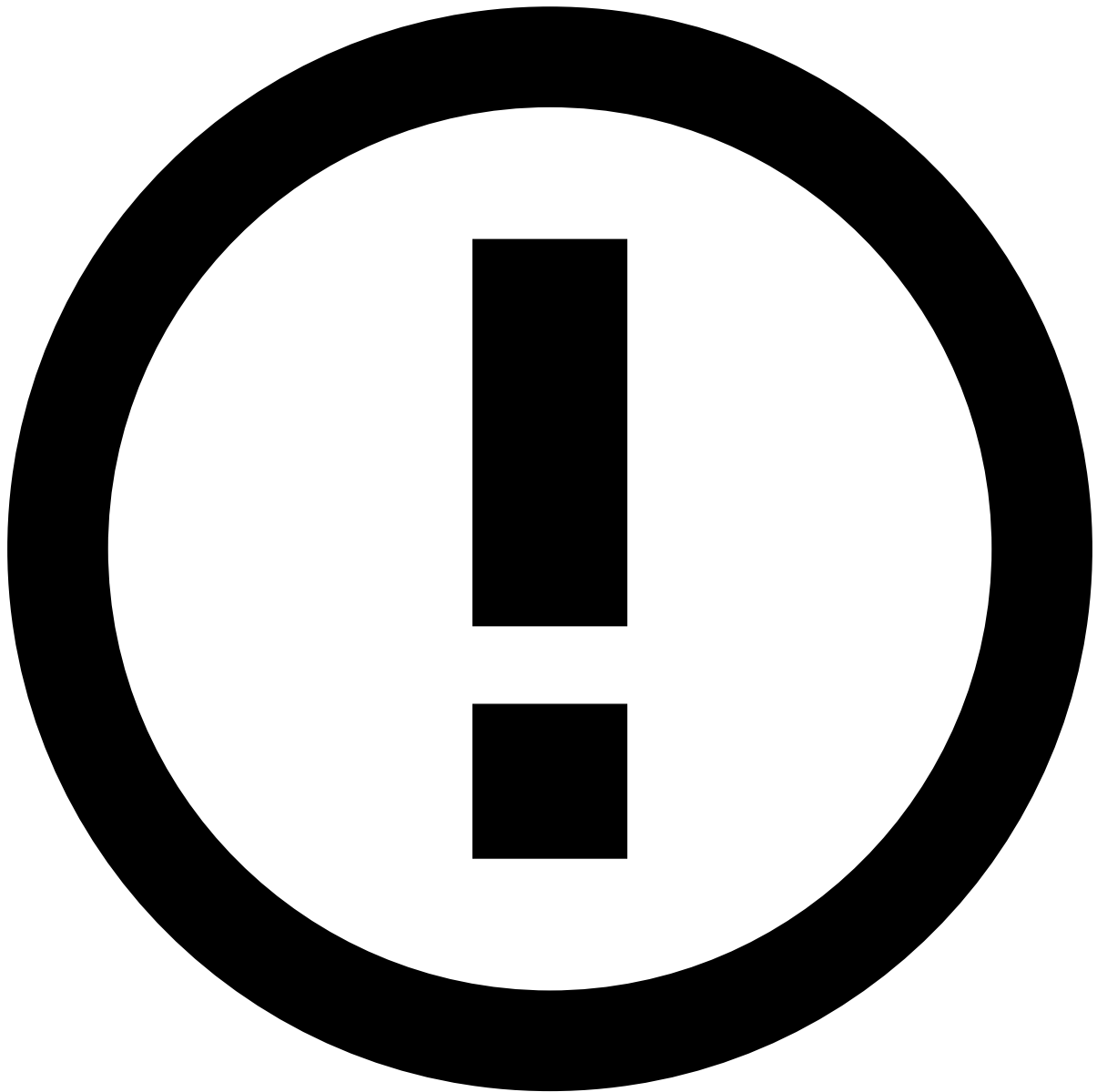
- [Core Concepts](#)
- Outcome Combinator

On this page

Outcome Combinator

Was this page helpful? 

The outcome combinator is a key concept in [workflows](#) and [sandbox evaluations](#) that helps to create more stable Data Science projects and allows you to define the strategy on how to combine results into outcomes in workflows.



Outcomes

Outcomes represent the decisions produced as output based on the way a workflow has been modelled, e.g. Fraud/Not fraud. Pulse adds each outcome decision field (for instance, *fraud*) and outcome score field (for instance, *fraud_score*) to the schema of events moving along the workflow.

How Outcome Combinators Work

Scoring and rules services contribute to outcomes and respective scores with partial values. The message associated with every event that enters a workflow is complemented with further information on outcomes whenever the event is scored or processed by [rules](#).

The outcome combinator therefore takes all those partials results, retrieves any other relevant data from each of the steps (rules, [models](#), etc) and combines all information. This expands the message and updates the decision information related to that given event.

Outcome combinators act every time that an event is processed by a workflow service, or resources in a sandbox evaluation, that contributes to a decision and adds results to the event. It is then when the combinator:

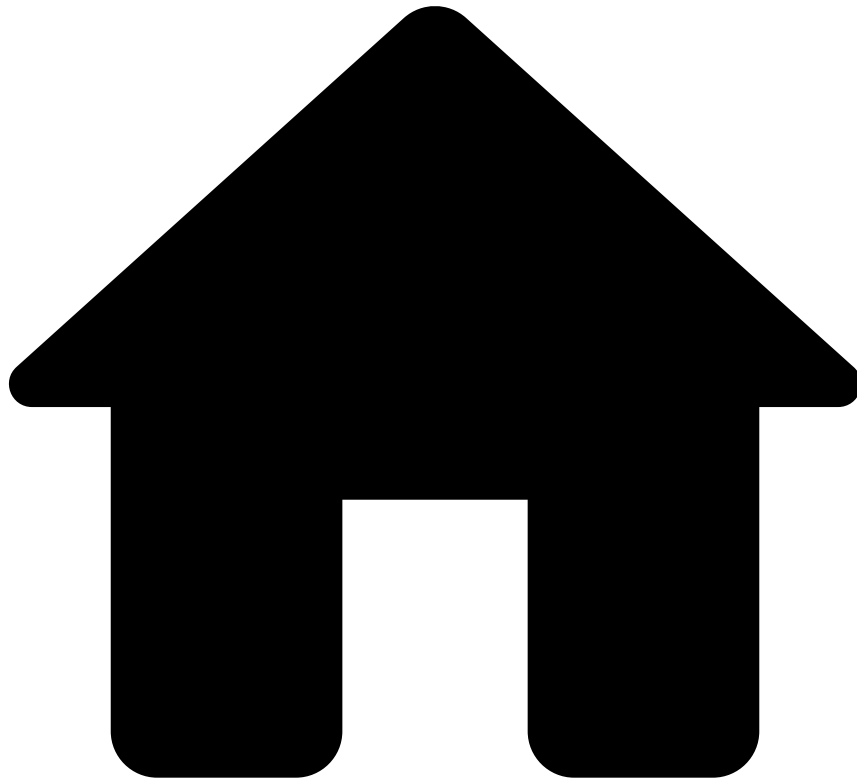
1. Is executed in order to combine all the partial results generated up to that point, according to a combination strategy – typically a weighed average.
2. Updates the fields associated to the outcomes. E.g. *fraud* and *fraud_score*.

Outcome combinators guarantee that those fields always reflect the combined scores and decisions taken in the workflow up to that point, which can be used as input for the next service being processed. Outcome combinators define how to calculate the final score for an outcome, i.e. how to combine all partial scores produced by any resource.

Learn More

Refer to the [Testing your work in a Sandbox environment](#) page for more details on how Sandboxing works.

-



- [Core Concepts](#)
- Plans

On this page

Plans

Was this page helpful? 

Data science plans are functions that receive data and produce a transformation of that data (not necessarily with equal size).

Plans do much of the heavy processing required before you train your machine learning [models](#) in the [data science loop](#). Plans are executed as distributed jobs in Spark which allows handling large datasets (e.g. hundreds or even thousands of TBs). In addition, they allow you to natively deploy a subset of these data transformations to a runtime [workflow](#) while keeping the same semantics as in the data science stage.

Check the [operators reference](#) to learn more about the possible elements you can include in your data science plans.

What are plans used for?

Plans define a transformation of one or more [data sources](#) into another data source. They are a vital concept in Pulse and they are a powerful contribution to the data science set of tools. Plans represent a way to boost the performance of:

- **Data scientists:** Delivering a set of tools that make it easier to process data for the training of machine learning models.
- **Engineers:** Abstracting a lot of the usual complexity involved in deploying data science work into production. With a fully integrated modeling and run-time environment, Pulse enables a seamless deployment of data transformations, without any major effort.

In a plan, it is possible to add, update and remove fields, calculate [metrics](#) on the data source, and even join data sources.

Plans are useful to systematize data cleaning, data processing, and feature engineering, both in a data science pipeline and in production. Typically, a data science project will have several plans that are executed in sequence and generate new data sources each as the [output](#). These intermediate data sources are subsequently used as input for other plans.

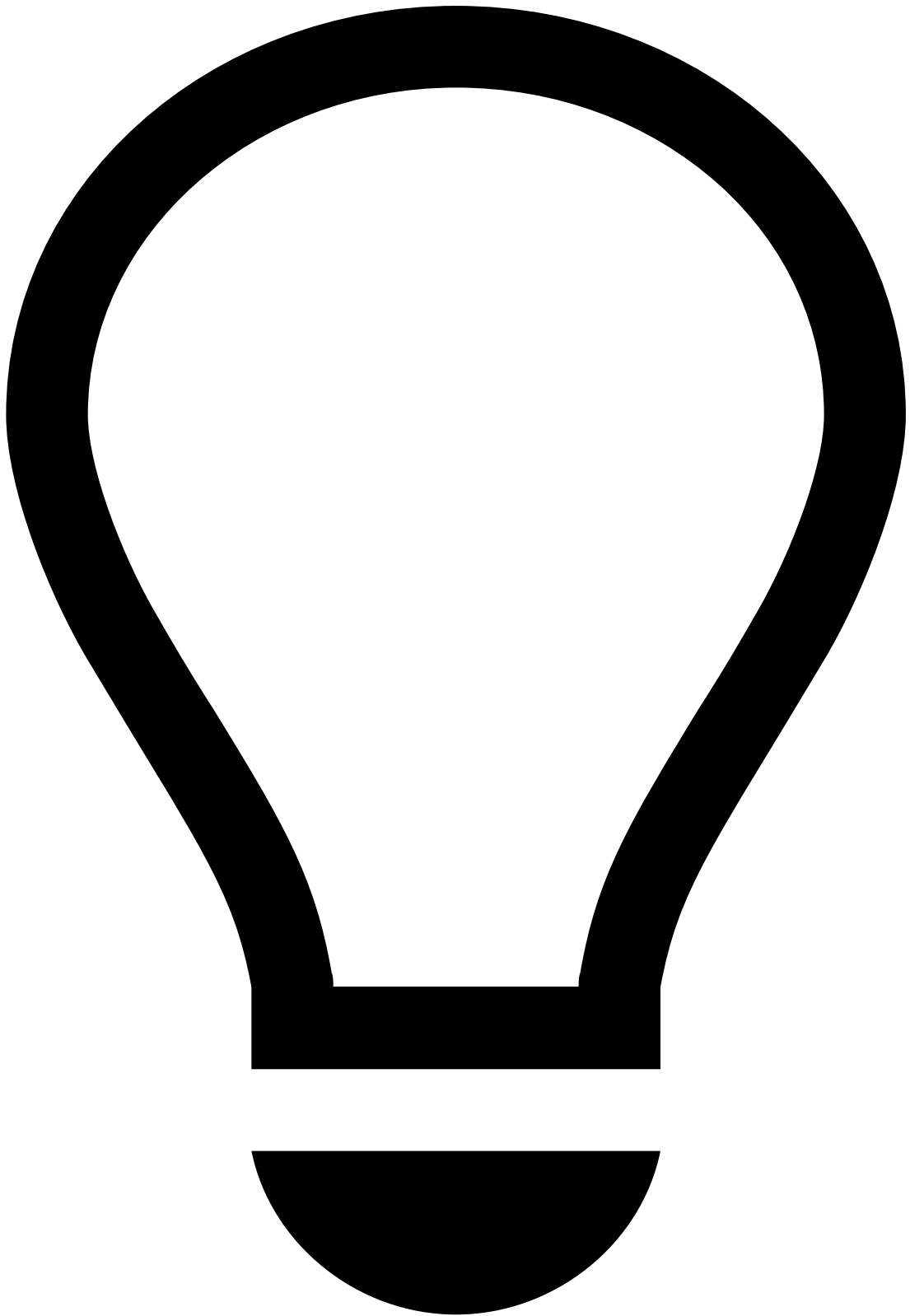
Types of plans

Pulse lets you design three main types of plans, each one aimed at different stages in your project:

- **Deployable plans**
It allows you to design and test, during the data science stage, the data transformations that you will use in a runtime workflow to process real-time events. For example, one or more plans that generate all features that your deployed machine learning [model](#) needs to output a decision in runtime. Pulse ensures that all transformations in a deployable plan will work in runtime exactly as you designed them during the data science loop.
- **Deployable omnichannel plans**
In addition to the functionality offered by regular deployable plans, this type of plan allows to process data from multiple sources, to enable [channels](#). Each input data source corresponds to a separate channel. At runtime, when omnichannel plans are deployed to the workflow, multiple workflows feed into the plan.
- **Non-deployable plans**
The purpose of these plans is to perform data transformations that are purely for offline data exploration purposes, during the data science loop, and that will not be directly deployed in a runtime workflow. For example, a plan that addresses quality issues in your raw data, a plan that samples your data into a more representative dataset, or a plan that joins the raw transaction data with the respective labels, e.g. *fraud/not fraud*.

Deployable plans

A subset of the [transformations available](#) in plans can be deployed into production - typically the plan, or plans, that define the features to be used as input for your machine learning models - so that events can then be turned into instances and scored in real-time.



Using Plans in Production

This is extremely important since Pulse guarantees that such subset of the plans functionality will work properly in production for you to be able to score your instances in real-time.

That is very powerful, since in a deployable plan you devise a single piece of logic, and Pulse will apply the same semantics:

- Offline during your data science iterations with great scalability.
- In real-time, in production, with very low latency.



Workflow branching

Due to the fact that Pulse does not support parallel branches in runtime workflows, deployable plans only allow for a single data source to be configured as the input, and only one sink as the only output.

Integrated exploration and runtime stages

A unique feature to Pulse is enabling you to deploy feature plans into runtime without effort. This tight integration of data science and engineering represents a huge boost in performance and productivity.

The way deployable plans work in Pulse removes the obstacles that commonly affect your project when it is ready to go into production. Pulse ensures the following:

- Your data science work is seamlessly deployed into Production without any need for additional development.
- Semantics do not change whether you are exploring your data and modeling your project, or your work is being executed in a production scenario. For example:
 - How Pulse partitions your data in a distributed environment.
 - How Pulse performs real-time and scheduled aggregations.

- Off-the-shelf integration with Cassandra and batch processes to replicate aggregations with long time windows.
- Execution in a high-performance streaming environment delivering sub-millisecond latencies.

The following image depicts a deployable plan — in this case, a plan that filters out internal transactions, unnecessary for the model, and generates metrics and features. Note that the plan's internal definition already includes a single [Source operator](#) and a single [Sink operator](#) (not editable) to ensure that the plan executes a linear path when deployed in runtime.

Deployable omnichannel plans

In addition to the functionality offered by regular deployable plans, deployable omnichannel plans allow to process data from multiple sources, to enable [channels](#).

These plans support working with two or more channels:

- As explained in the [Channels](#) page, there is always a single main channel. When you place the plan in the Workflow, this is the channel processed and scored by that Workflow.
- The remaining channels are secondary channels — they are only fed into the plan for additional information and to improve the detection on the main channel.

The main channel is equivalent to the input source in non-omnichannel plans. You can perform all the same transformations on the main channel that you would do in non-omnichannel plans.

The secondary channels can only be used in aggregations (in the [Metrics operator](#)).

For example, let's say that you are processing ATM withdrawals (debit channel) and would like to check if the customer has recently logged into internet banking with a new device and increased the withdrawal limit (internet banking channel). In this case, *debit* is the main channel (because you are scoring debit card events) and *internet banking* is the secondary channel (because you are consulting it to improve the detection). See the [Creating metrics](#) page for more information on how to implement the relevant aggregation in Pulse.

At runtime, omnichannel plans are fed by workflows:

- The workflow where the plan is deployed corresponds to the main channel, i.e. the data passing through this workflow will be sent to the input of the plan.
- For each secondary channel, you will need to select a different workflow which is processing data for that channel. That data will be automatically fed into the plan.

Non-deployable plans

A non-deployable plan is used only for data exploration purposes and any ETL task, for example in traditional data manipulation when joining and grouping data. When processing a data stream in real-time, in a Production environment, for example, to classify transactions as fraud or not, these activities are not repeatable nor applicable, such as joining data from separate sources.

These plans have no restrictions as to the [available operators](#) they can execute.

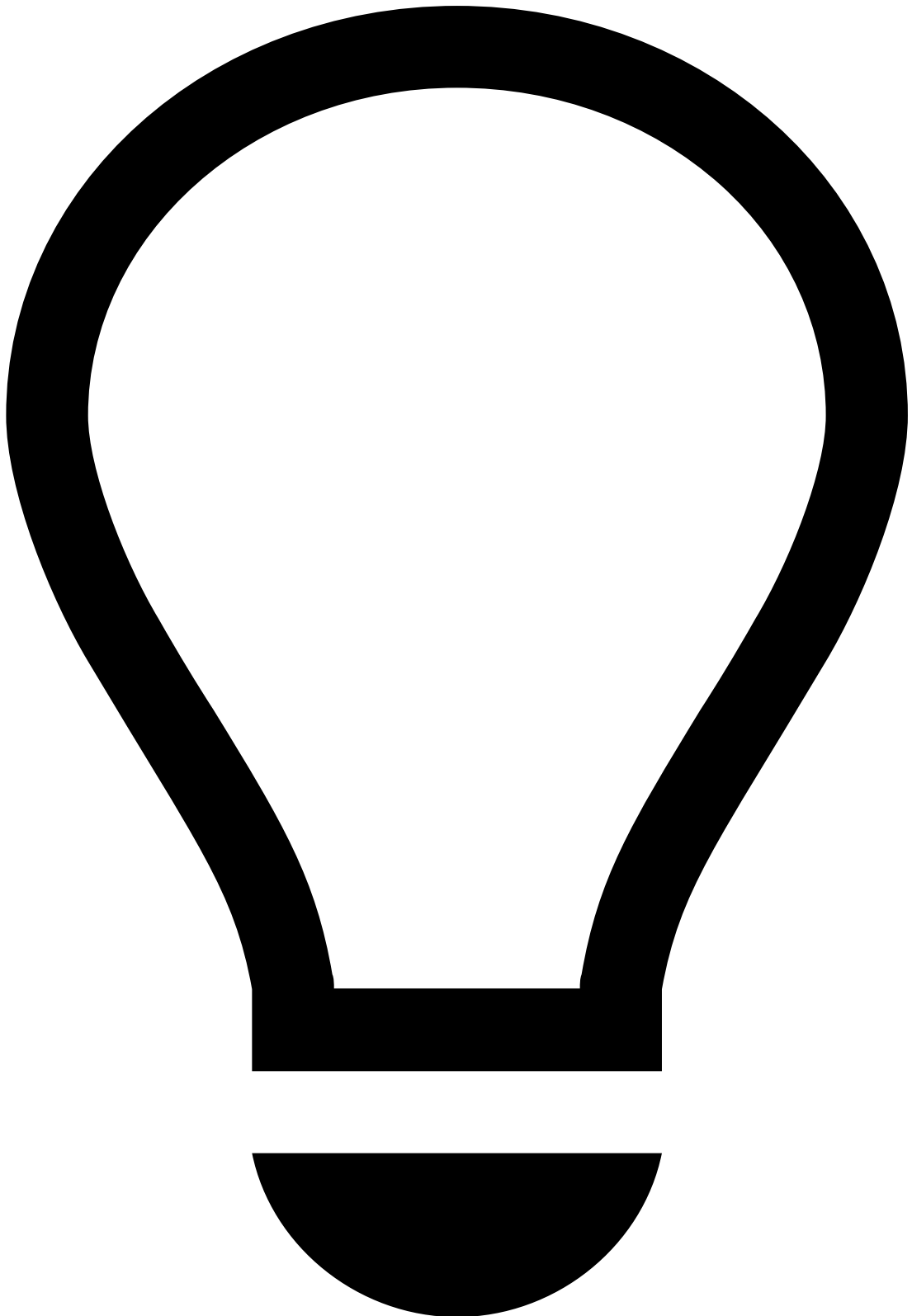
The following image depicts a non-deployable plan that joins data from different sources, in this case, historical transactions with the respective labels, and performs data cleaning operations such as removing unnecessary fields.

Plan execution

A plan is just a definition, which means that there isn't actually any data bound to it, only an input [data schema](#). To actually compute the output data source it is necessary to run it.

Running the plan creates a plan execution. A plan execution represents a snapshot of the plan at the moment it ran, the inputs used and the outputs created, during the data science stage. It allows you to deploy a (deployable) plan to runtime with the same exact configuration and semantics it had when you ran it initially.

For all [operators](#) that require you to define expressions (for example, adding and updating fields with the [Map operator](#)), you must do so with the [Pulse Query Language \(PQL\)](#). Thus, whenever Pulse runs data through a plan, whether it is during the data science stage or in real-time, Pulse is actually executing [PQL commands](#).



Distributed execution

The execution of plans runs distributed in Spark, so Pulse is able to transform data in parallel within a set of machines of a cluster.

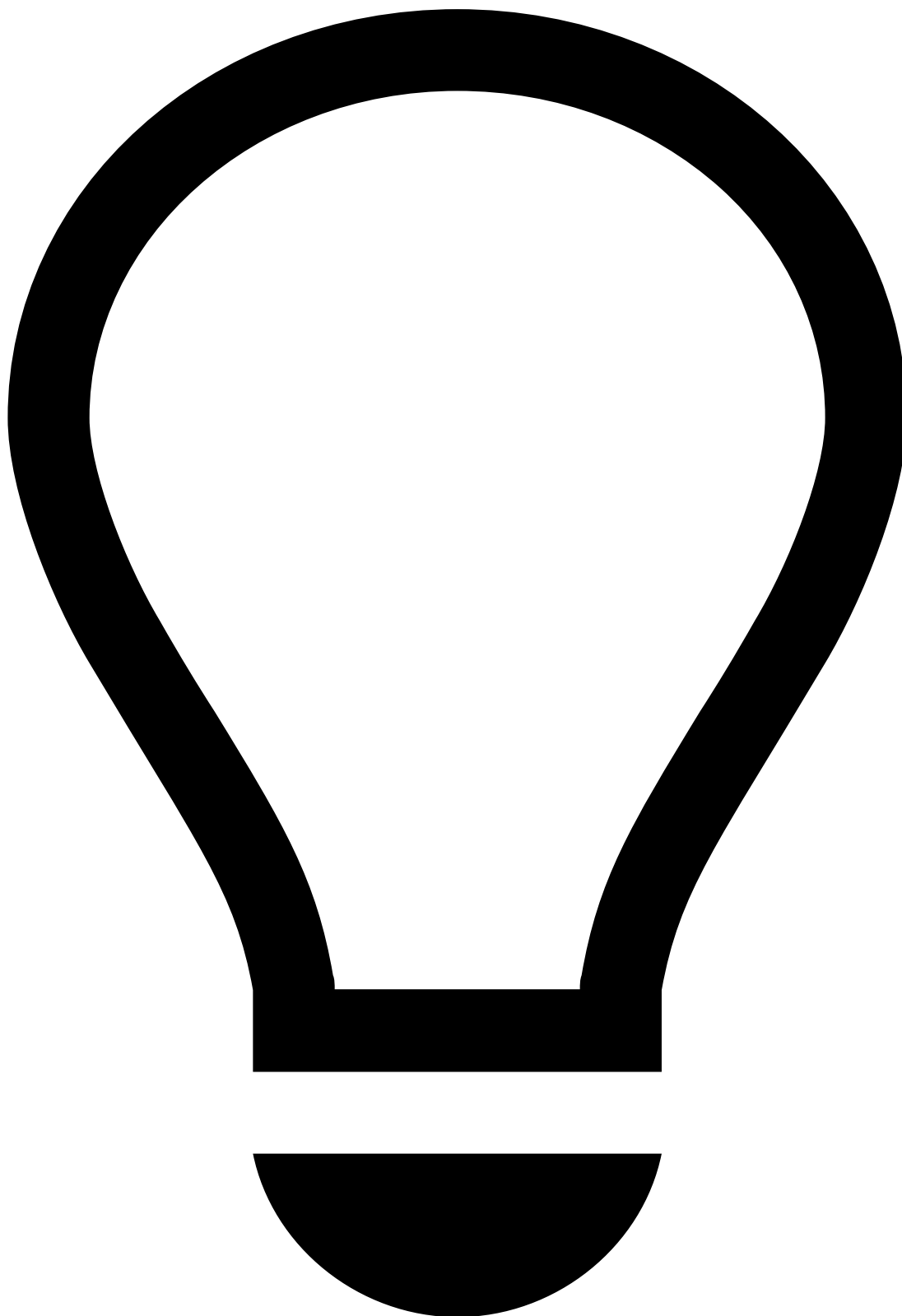
This means that even for very large datasets, with several TB in size, Pulse is very fast and efficient when computing data transformations on your [data sources](#).

Immutability

Plan executions ensure that plans, like data sources and schemas, are immutable.

When you add a deployable plan to a runtime workflow, you must specify a plan execution. The plan execution contains the plan definition at the moment it ran during the data science loop, the inputs used and the outputs created.

This isolates the plan during data science from the same plan when deployed to production. This way you can safely change the plan definition itself without affecting your production workflow.



Correcting a plan

If you need to deploy corrections to your plan, you can simply run it to create a new plan execution and generate a new snapshot of the plan. Then configure the plan service in your workflow with the new plan execution.

Data partitioning¹

A plan in Pulse is executed as a distributed job in Spark. This allows you to set up various plan operators to perform different data transformations and process large volumes of data.

Spark handles data that is organized into smaller logical chunks referred to as partitions to achieve high performance.

The way that data is re-organized into partitions is specific to the operators that you add to your plan. For example, to calculate metrics defined in the Metrics operator, Spark needs to re-organize the data by the *Group By* fields configured in this operator.

Pulse has different options available to configure data partitioning so that the data re-organization is performed as homogeneous and well distributed as possible.

Check the [Data partitioning in Metrics operator](#) page to learn more about this functionality.

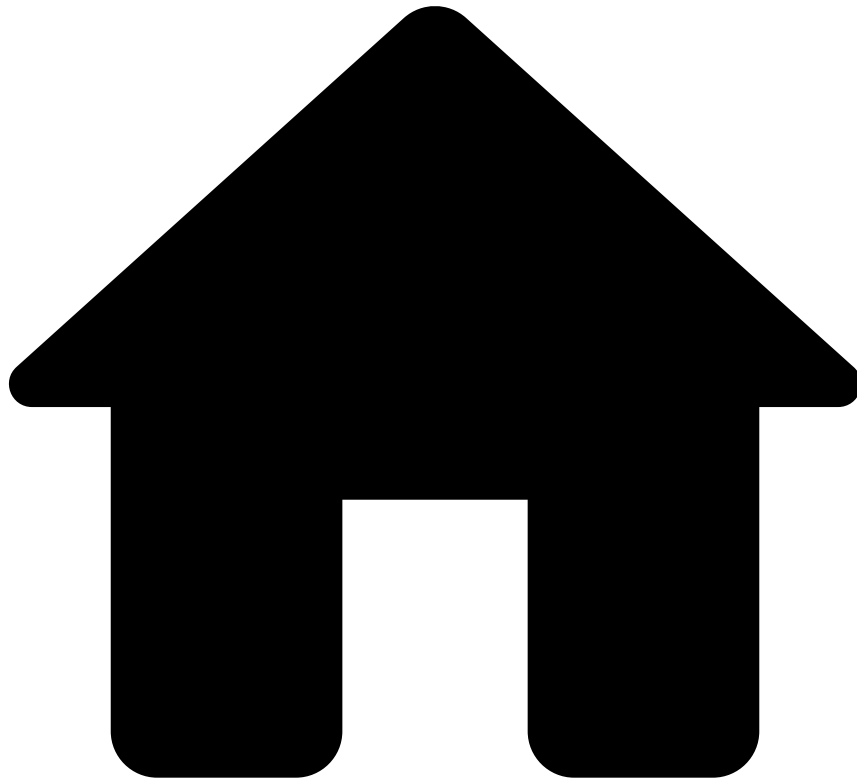
Compressed file output¹

Pulse implements a configuration that enables the output of plans to be directly compressed in storage.

This feature causes the underlying files to be stored as BZip2-compressed, and thus, taking up to 8 times less space on disk. As the trade-off, plans with compressed file output are slightly slower to read when used as input for other jobs such as model training, model evaluations, among others.

Check with your Feedzai contact person how to configure plan output file compression on the server.

-



- [Core Concepts](#)
- Rules

On this page

Rules

Was this page helpful? 

A rule defines a condition based on the fields of a [data source](#) that produces a binary result for each record that matches the condition. The condition to evaluate is defined by coding in [RDL \(Rule Definition Language\)](#).

Defining rules

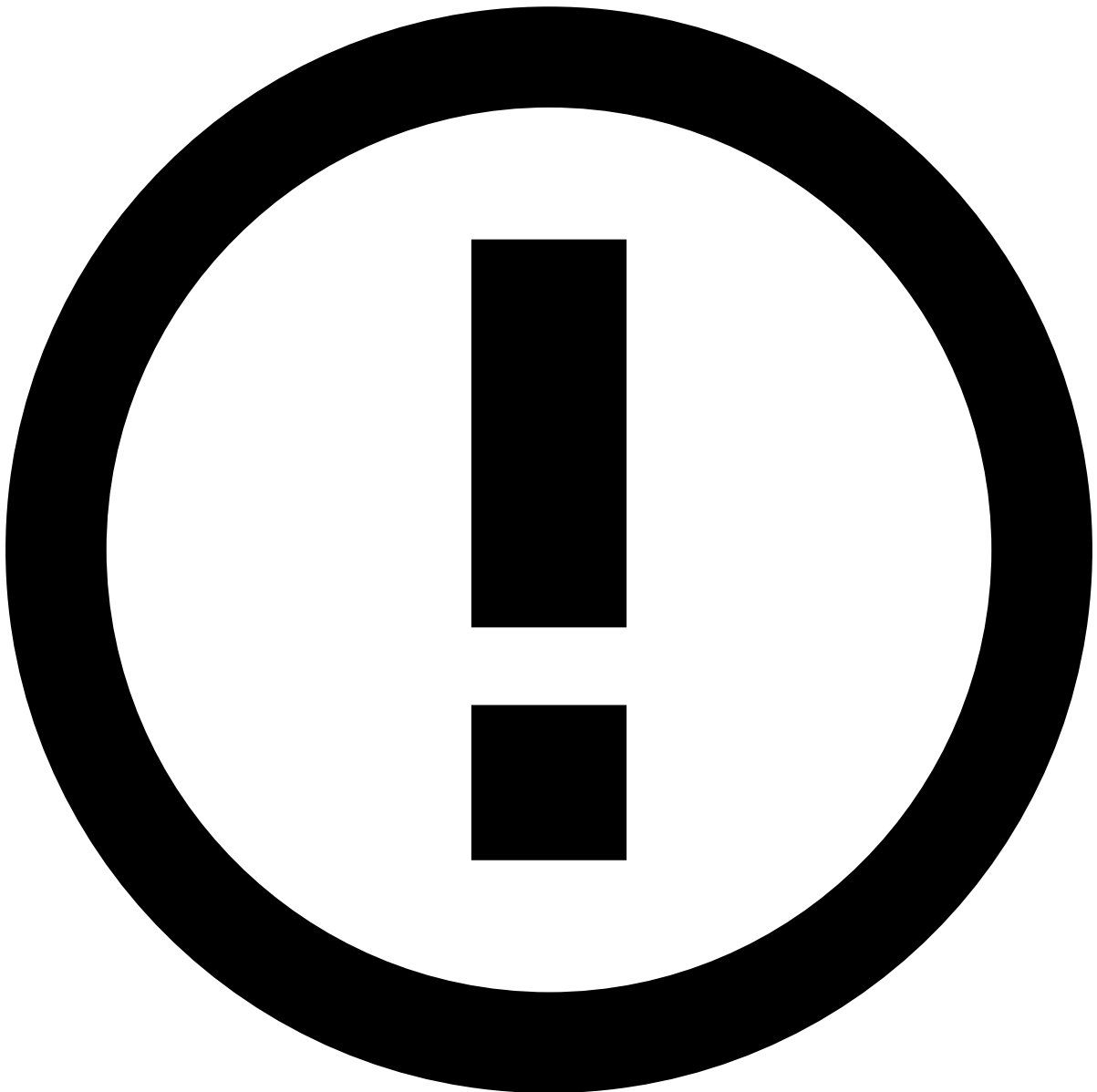
The following image shows how a rule is defined. This example depicts a rule for an account opening scenario to automatically accept the customers' application if their income is significantly above the average for the area where they live.

To fully understand rules in Pulse you must check all details regarding [how a rule is composed](#) and [how it works](#).

Rules projects

Similarly to [models](#), rules are organized into rules projects. Rules projects define which data source field should be available for rules to target, for example, to set all transactions from a specific card automatically as fraud. In addition, rules projects act as logical containers that group rules related with the same context.

A rules project is associated with a data source. Since the rule concept is not directly related to machine learning, it does not involve a training stage. Thus, it is possible to deploy rules directly into Production without having them run over test data. So in fact, a rules project is bound to the schema of a data source and does not depend on the data itself.



info

You can evaluate your rules over any data source as long as it is [compatible](#) with the data schema specified for the rules project.

Target variable

When creating a rules project, you must define the data source on which the rules will operate. In addition, you must choose a **target variable**, which is the field whose value you are trying to detect with your rules. For example, *fraud/not fraud*.

Pulse lets you target any categorical field in the data source schema. In practice, this means that rules can be useful both for binary and multi-class classification.

For each rule in the rules project, Pulse allows you to select which of the possible values should be set to the target variable if the rule is triggered. Thus, in the same rules project, you can configure some rules to set transactions as *fraud*, and other rules that set transactions as *not fraud*.

When you evaluate rules in a rule snapshot, Pulse calculates the rule performance for each possible value of the target variable, letting you adjust the resulting visualization accordingly.

State

Rules are usually stateless, but when you cannot use [plans](#) to calculate certain types of windows and add metrics as fields of a schema, Pulse lets you define these windows directly for a rules project.

Therefore, when you need to develop very robust rules that perform aggregations over time windows, you must configure how Pulse should manage their internal state.



info

It is recommended that whenever possible, you use [plans](#) to handle your time-aware operations in data science. The plan will generate data fields with these metrics, making them accessible to rules.

Activating state in your rules project is an advanced operation that must be carefully done and requires that the user understands the full implications of his actions:

- By enabling state in a rules project, Pulse manages an isolated environment where all rules contained in the project will have access to [Pulse Query Language metrics](#) (i.e., aggregations) and [Rule Definition Language variables](#).
- To activate state, the schema in your rules project must include a timestamp field. Otherwise, you cannot perform time-based aggregations.
- When configuring partitioning fields, you must be extremely careful. They define how to distribute processing when generating a rule evaluation. Make sure data is correctly partitioned, otherwise the evaluation results may not make sense.

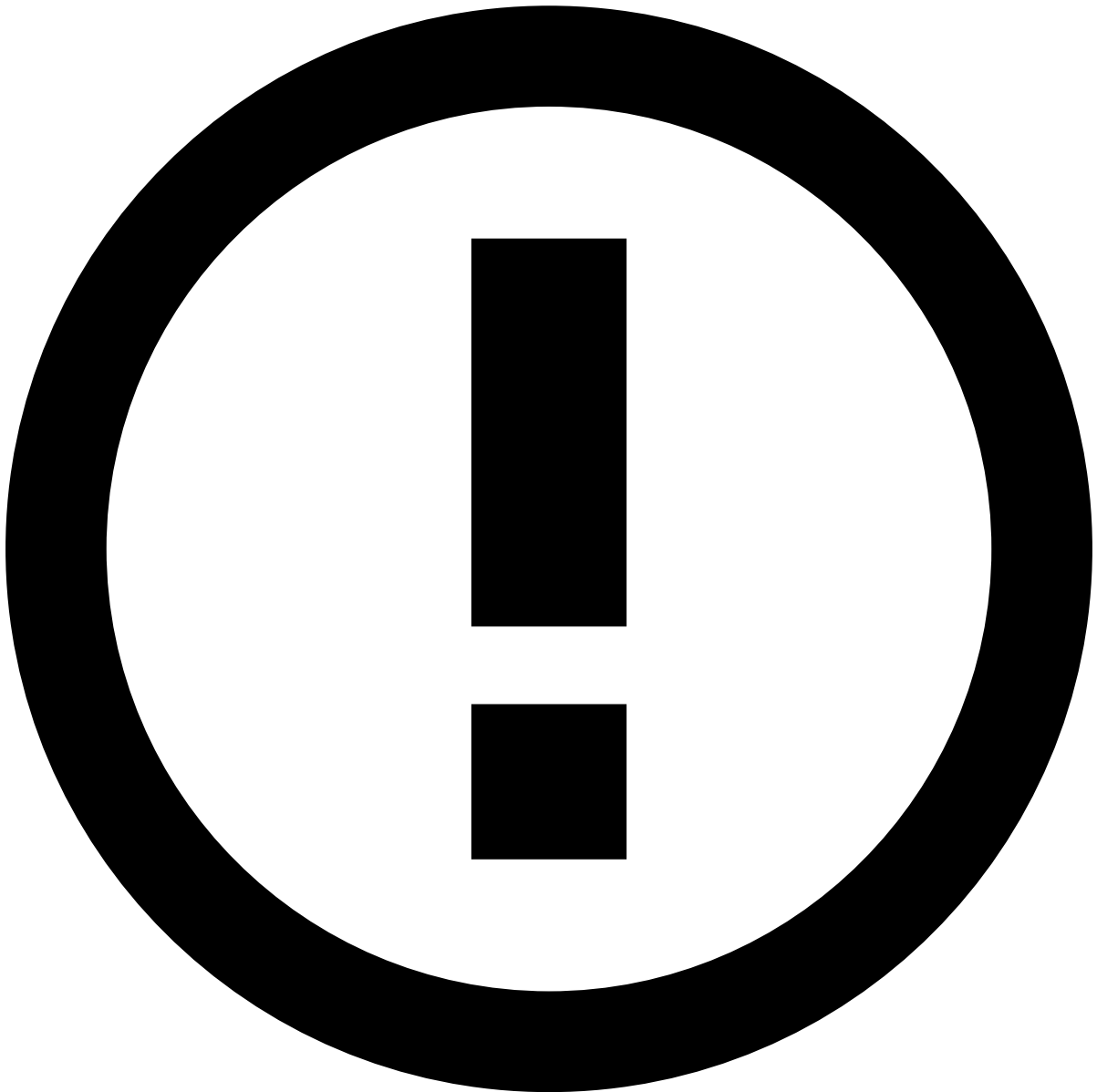
Learn more about state in rules by checking [how a rule is composed](#).

Multi-tenancy📖

Multi-tenancy is the ability to support multiple tenants in the same environment offering a practical solution to apply rules to several tenants at the same time, as well as to customize rules for each tenant.

When creating a rules project, the user can select from the following options:

- **Global** is used for rules that are not multi-tenanted.
- With **Duplicated for Tenant**, the same rule applies to all tenants, but rule conditions can be computed using tenant-specific data.
- With **By Tenant**, custom rules apply to individual tenants.



info

A **landlord** is a client who has a fraud prevention installation that he can offer to other clients, also known as tenants, acting as a reseller. A **tenant** is a client that uses a fraud prevention application provided by a landlord. Tenants have the possibility to create, manage and deploy their own rules and lists depending on the permissions the landlord has given to them.

Duplicated for Tenant

When a rules project is set as **Duplicated for Tenant** it means that the set of rules applies to all the tenants in the same way. In terms of deployment, a rule snapshot is created and applied in runtime, the events are then grouped by each tenant and all rules defined in the rule snapshot are applied for each tenant individually, as illustrated in the following image.

By Tenant

When a rules project is set as **By Tenant**, it means that a set of rules applies to specific tenants. In terms of deployment, a rule snapshot is created and applied in runtime, the events are then grouped by tenant and the corresponding set of rules are applied for each tenant, as illustrated in the following image.

Advanced vs. simple rules

Users can set the type to advanced or simple, the former being the default type.

This setting has the following meaning:

- **Advanced rules projects** contain rules that can be written using a template and which may have an environment and state.
- **Simple rules projects** contain rules that are not editable inside Pulse and may be used for self-service rules created in Case Manager.

When a rules project is set as *Simple*, the *State* input is hidden, the label for the *Actions* becomes *Secondary Actions*, and the multi-tenancy options are limited to *Global* and *By Tenant*.

Self-service rules

With **self-service rules** (SSR), fraud analysts can write and deploy rules themselves very quickly.

The characteristics of self-service rules in a nutshell are as follows:

- They are deployed directly to the Production environment, in real time.
- They are created in Case Manager by analysts and/or their managers.
- Governance can audit and support self-service rules.

This lets you implement completely new processes and achieve higher operational efficiencies while dynamically following changes in fraud patterns.

Testing self-service rules

Self-service rules can be tested in Case Manager, allowing production environments to execute Spark jobs processing the most recent events that have gone through a self-service rule block of a given workflow. These events will be tested against an updated version of that rule block and measure *Hit Rate*, *Money Captured*, etc.

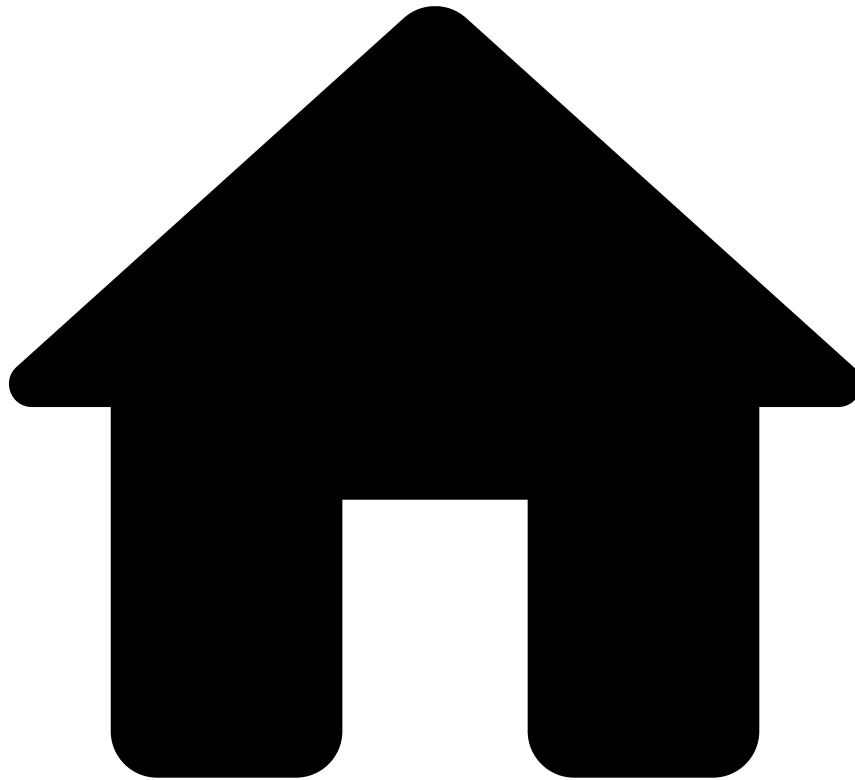
In order for Pulse to process the data in the same way as when going through the live workflow, those events are stored in a custom data store used only for self-service rule testing.

This data store thus receives every event that goes through a self-service rule block in the workflow. Depending on the testing configuration in Case Manager, this data store keeps the data of the time interval that is necessary for rule testing.

Auditing self-service rules

Pulse can be configured to keep audit logs of the creation, edition, deletion, ordering, testing, and publishing operations made to rules. The raw logs are accessible via a REST API through a query that specifies a rule, a workflow element or an application. Get in touch with your contact person at Feedzai to get more information on the audit logs and the Self-Service Rules REST API.

•



- [Core Concepts](#)
- Sandbox Evaluations

On this page

Was this page helpful? 

Sandbox Evaluations

Sandbox Evaluations allow you to test a sequence of [Model](#) and [Rule projects](#) you have created in your Data Science application as if they were in production. By [creating](#) and [analyzing the results](#) of **Sandbox Evaluations**, you will be able to have a clear picture of how your Data Science project is performing before actually deploying it.

Sandboxing allows you to simulate and check the accuracy of your system before it is in production, so you can improve the performance of the metrics of your system in advance. For example, the fraud detection rate of the system is understood more precisely than by analyzing **Models** and **Rules** separately. This helps you have a broader understanding of your Data Science project and spot flaws that would be harder to detect just by analyzing resources in isolation.

Although Models can be trained in order to understand their performance in a specific field, this does not reflect the reality of a deployed system. Sandboxing enables the analysis of the performance of all these resources in a simulated [Workflow](#), being able to recognize those weaknesses that do not offer any added value to your system. For example, by

testing **Models** and **Rules** together, you can see whether the configuration of your **Rules** does offer a better tool for detecting fraud.

Additionally, it allows you to see not only the performance by score, but also by decision, whilst testing rules only regard the decision itself.

Improving the visibility over your project

Sandboxing is also a way to validate [Lists](#) and [Outcome Combinators](#). The following image depicts a Sandbox that analyzes a **Model** followed by a set of **Rules**, combining the results with a weighed average.

You can also get a clearer picture of how your system is doing by selecting partitioning fields: The Sandbox thus analyzes the different components of the workflow separately, the node only having visibility of that partition. Sandbox Evaluations also simulate the order in which events are processed.

You can also select **Models** and **Rules** to be segmented by particular values of breakdown fields, allowing you to analyze the performance of Models and Rules at once.

Evaluating the Results

Once that the parameters of your **Sandbox Evaluation** have been specified, you can evaluate its results in order to find points to improve, creating a more robust Data Science project. You can visualize the results by Score and by Outcome separately in order to see how the system behaves independently of the outcome decision that was specified on the outcome decision field. You will be able to analyze various metrics that will allow you to grasp the functioning of your set.

The measured performance is therefore as accurate as when the system is deployed in production. With this, you can:

- Identify problems with recall/catch rate, false positive rate or risk workflow optimization.
- Use a system that runs past production traffic faster than a Production environment.
- Reduce the time needed to validate system results previous to the production environment in order to test the performance of **Models** and **Rules** as a single system.
- Simplify the testing of the entire system (**Model**, **Rules** and **Lists**).

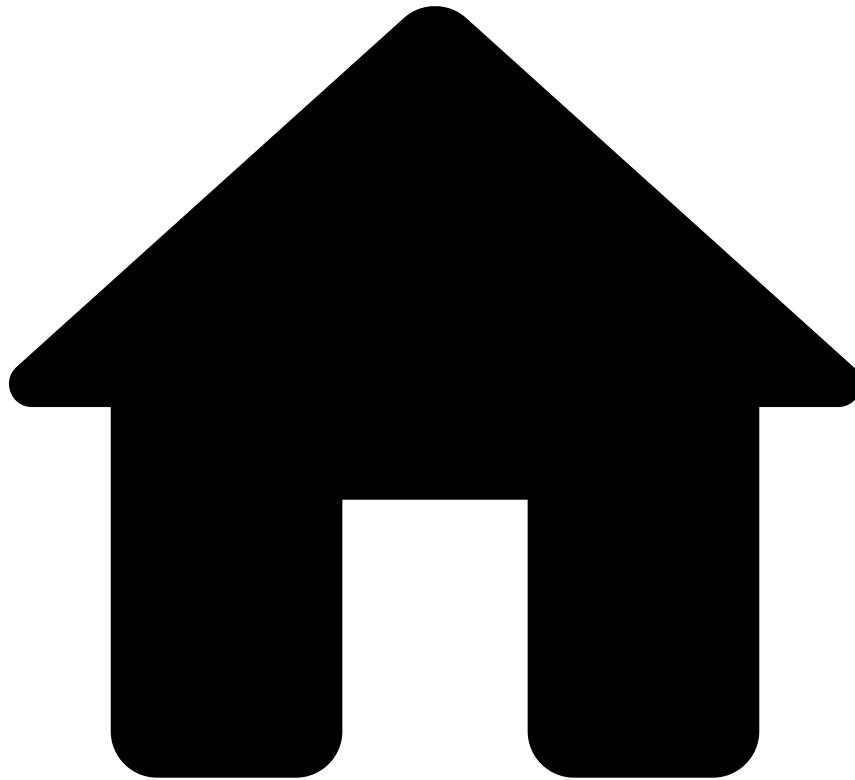
Evaluating your Sandbox results against a test dataset is crucial in understanding how it is performing, and how close you are to reaching the objectives of your Data Science project. With a complete set of metrics, you will be able to have a complete understanding of how all the components of your Data science project work together and identify the points to improve for your project to be as precise as possible.

Learn More

Discover more related to Outcome Combinators in the following pages:

- [Outcome Combinator](#): Learn more about what it is and how it works.
- [Testing your work in a Sandbox environment](#): Have a better grasp on how Sandboxing works.
- [Evaluating your Sandbox Results](#): Understand all the information that is shown after creating a Sandbox Evaluation.

•



- [Core Concepts](#)
- [Data Science Loop](#)
- The Data Science Framework (DSF)

On this page

The Data Science Framework (DSF)

Was this page helpful? 

The Data Science Framework (DSF), also known as the Data Science Studio, is a dedicated environment used in risk assessment strategies for building, testing, and evaluating rules and models according to the [Data Science Loop](#).

Its main capabilities include:

- **Data preparation:** Read production data or ingest historical data, clean it, and analyze its quality and statistics to ensure it fits your schema. This is crucial to work with Rules, Models and/or ad-hoc analysis.
- **Feature engineering and plan execution:** Create and test new features using Plans, which can then be deployed to production. Plans allow the application of generic data computations, typically to transform it or extend it to a new file output.

- **Model and rule development:** Train Machine Learning (ML) models and create rule sets, and evaluate their performance against historical data to identify if further improvements are needed.
- **Ad-hoc analysis:** Study and request ad-hoc reports or analysis via the DSF.
- **Sandbox Evaluations:** Test a sequence of a Model and Rule project in the Data Science application.

What are Data Science jobs?

Data Science (DS) jobs are jobs initiated in the DSF environment, either via Pulse or JupyterLab, and processed through the Pulse application. While Pulse manages the job lifecycle, the heavy computational workload is shifted to the cluster, which allocates **executors** (i.e. processing units) throughout its **task nodes** (i.e. the actual machines that compose the cluster) to perform the computations as a Spark job.

Refer to [Spark's documentation](#) to further understand its core functionality.

The resources allocated to each job are guided by the job configuration and workload, and bounded by the cluster configuration (i.e. size and number of task nodes, as well as the executors per node).

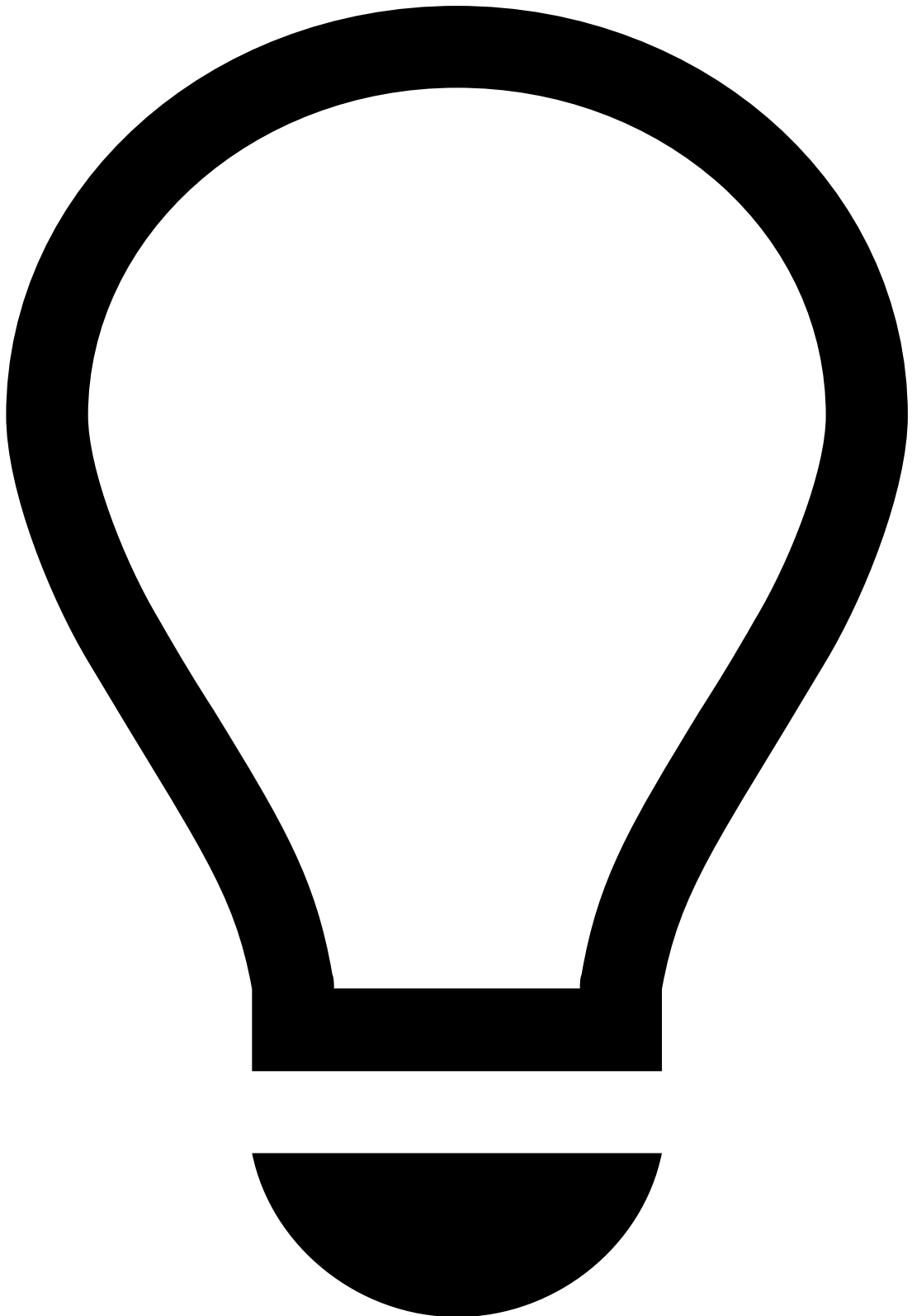
Pulse provides one default job profile that applies to all DS jobs, but it is recommended that you adapt this default profile to better fit the jobs you'll be executing. The default job profile is, essentially, a starting point to create other profiles.

What are executors?

Executors represent the **main processing unit** that do the actual work in your job (i.e. reading data, transforming it, caching it, and writing results).

When a job is launched/scheduled, the cluster allocates executors on the task nodes (i.e. machines). These executors borrow resources (i.e. cores and memory) from the task nodes to perform the computations and execute DS jobs.

When sizing the executors, be mindful of this to avoid setting unrealistic bounds on the amount and/or size of your executors.



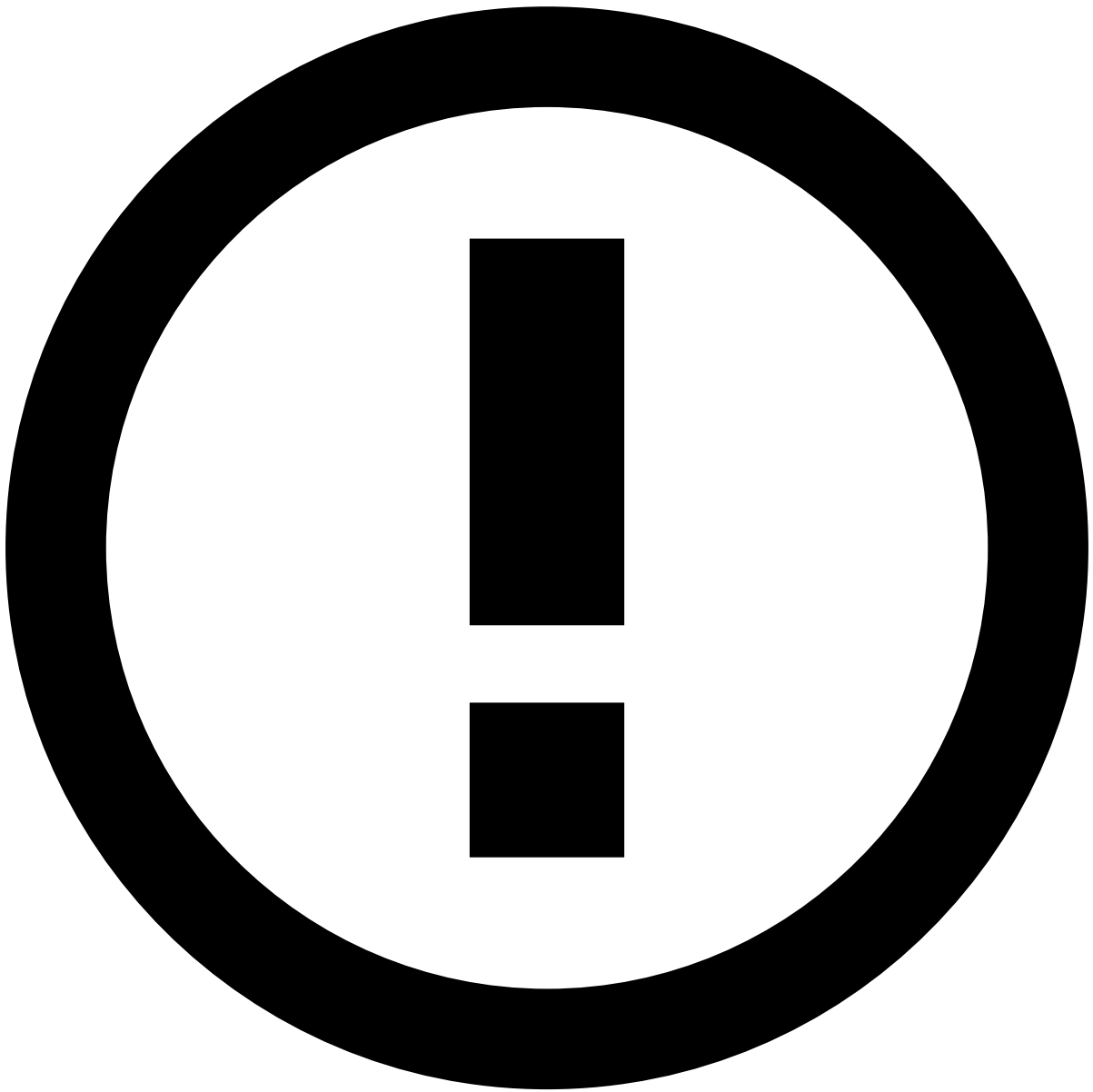
Practical example

If you consider a task node with 16 cores and 128GB of memory, configuring executors to have nine cores and 50GB of memory would represent poor cluster resource usage, which would, ultimately, lead to poor performance. Here's why:

After allocating one executor (with nine cores), the cluster is left with seven available. As each executor borrows nine cores, the cluster is not able to allocate any more executors for that job.

This way, it would only be possible to fit one executor, leaving seven cores unutilized.

Note that the remaining seven cores could still be used by other jobs, as long as the executors fit in the task nodes.

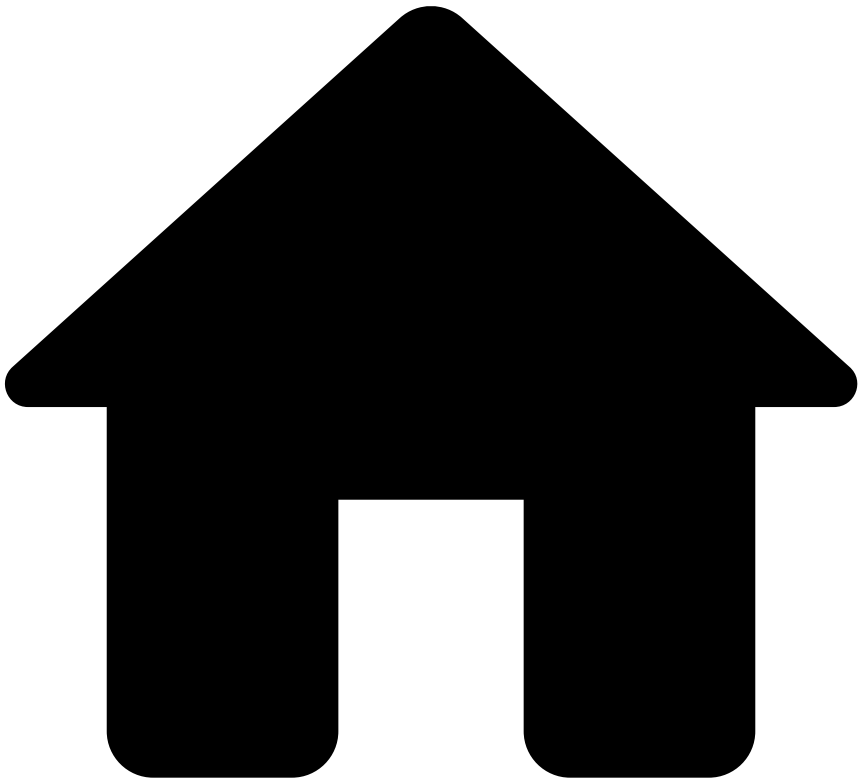


Dynamic executor allocation

Note that while you can explicitly define how many executors to run, the **general rule-of-thumb favors dynamic allocation of executors**. This lets the cluster decide how many executors to run depending on the workload.

Learn all about the importance and impact of tuning DS jobs in the [Using the Data Science Framework](#) page.

.



- [Core Concepts](#)
- User-Defined Functions

On this page

User-Defined Functions

Was this page helpful? 

Pulse comes with a set of User-Defined Functions (UDFs) that can be used in PQL expressions to implement richer logic than what's available from the basic built-in functions. This page provides an overview of the available UDFs.

To learn about how to use UDFs, see [Function Calls in PQL](#).

Working with strings

`DataConversionTools` class 

Package: `com.feedzai.common.udfs`

Modifier and Type	Function	Description
static Integer	stringToInt(String s)	Converts a String into an Integer. If the given s is null or the value exceeds Integer.MAX_VALUE or Integer.MIN_VALUE , then null is returned.
static Long	stringToLong(String s)	Converts a String into a Long. If the given s is null or the value exceeds Long.MAX_VALUE or Long.MIN_VALUE , the null is returned.
static Double	stringToDouble(String s)	Converts a String into a Double. If the given s is null or the value exceeds Double.MAX_VALUE or Double.MIN_VALUE , then null is returned.
static Float	stringToFloat(String s)	Converts a String into a Float. If the given s is null or the value exceeds Float.MAX_VALUE or Float.MIN_VALUE , then null is returned.
static String	integerToString(Integer i)	Converts an Integer into a String. If the given Integer is null, null is returned.
static String	longToString(Long l)	Converts a Long into a String. If the given Long is null, null is returned.
static String	doubleToString(Double d)	Converts a Double into a String. If the given Double is null, null is returned.
static String	floatToString(Float f)	Converts a Float into a String. If the given Float is null, null is returned.

Strings class (fraud)🔗📄

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Integer	levenshteinDistance(String str1, String str2)	Computes the Levenshtein distance (edit distance) of two strings.
static Boolean	matches(String input, String like)	Returns true if the input matches the 'like' pattern, using case-sensitive matching.
static Boolean	matchesIgnoreCase(String input, String like)	Returns true if the input matches the 'like' pattern, using case-insensitive matching.
static String	strcat(Object obj1, Object obj2)	Concatenates two strings.

Strings class (cloud)🔗📄

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Integer	tokenCount(str: String, delimiter: String)	Returns the number of tokens in a given string, separated by the defined delimiter. For example, <code>tokenCount("Number of substrings", " ") = 3</code> .
Double	twoGramSimilarity(str1: String, str2: String)	Computes the 2-gram similarity between two strings. The returned value is a double in a range from 0.0 to 1.0, where 0.0 represents the minimum similarity and 1.0 represents maximum similarity. The 2-gram similarity between strings is computed after white space removal. If the given strings are null they will be considered empty strings. For more details see the documentation of NGramDistance .
String	normalize(str: String)	Removes accents and converts the string to lower case. If the input string is null, an empty string is returned.
String	normalizeDelWhitespace(str: String)	Removes the accents, deletes white spaces and converts the string to lower case. If the input string is null, an empty string is returned.
String	normalizeToAlphanumeric(str: String)	Normalizes the string to contain only ascii alphanumeric symbols. Note that this will strip accents, so "Ãƒ" will become "a".

Modifier and Type	Function	Description
Boolean	isSubstring(str: String, sub: String)	Returns true if a string contains a given substring in a case insensitive manner.
Boolean	isAllCaps(str: String)	Returns true if the input string is all upper case.
Boolean	isAllNonWhitespaceCaps(str: String)	Returns true if the input string is all upper case. Ignores white spaces.
Integer	strDistance(str1: String, str2: String)	Returns the Levenshtein-distance between two strings.
String	extractDigits(str: String)	Returns the digits from a string.
String	dropDigits(str: String)	Returns the string after dropping the digits from it.
Boolean	isDigits(str: String)	Returns true if the string consists only of digits.
Integer	length(str: String)	Returns the length of the string, or 0 if the string is null.
Integer	countChars(str: String, chars: String)	Counts the number of characters of "chars" in "str", or 0 if any of the input strings is null.
Integer	longestSameLetterSubstringLength(str: String)	Returns the length of the longest substring that is a repetition of the same letter.
Integer	longestIncreasingSubstringLength(str: String)	The length of the longest substring that is strictly increasing by one unit, e.g. "defgh".
Boolean	containsCyrillic(str: String)	Returns true if a string contains Cyrillic characters. If the string is empty, returns false.
Boolean	regexFind(regex: String, str: String)	Returns true if a string matches a given regex pattern. If any of the parameters is empty, the function returns false.
Double	keyboardDistance(str: String, step: Int)	Returns the average keyboard distance between every character in a string and the character that precedes it by a given step.
Double	numRatio(str: String)	Returns the ratio of digits in the input string.
Double	vowelRatio(str: String)	Returns the ratio of vowels (a, e, i, o, u, Ð°, Ð¾, Ñ, Ñ, Ñ) in the input string.
Double	distinctCharsRatio(str: String)	Returns the ratio of distinct characters in the input string.

StringTools class

Package: `com.feedzai.commons.udfs`

Modifier and Type	Function	Description
static Integer	compareTo(String f, String s)	Lexicographically compares string f with string s.
static String	concat(String...s)	Joins the elements of the provided array s into a single string object.
static Boolean	contains(String f, String s)	Checks if string f contains string s.
static String	deleteWhitespace(String s)	Deletes all whitespaces from a string as defined by Character.isWhitespace(char).
static Double	getDiceIndex(String f, String s)	Returns the Š,ensenšDice index for a given pair of strings.
static Integer	getLevenshteinDistance(String s, String t)	Returns the Levenshtein distance between two strings.
static Double	getNormalizedLevenshteinDistance(String s, String t)	Returns the Levenshtein distance between two strings, divided by the length of the longest string.
static Integer	length(String text)	Returns the length of a string.
static String	lowerCase(String s)	Converts string s to lower case as per String.toLowerCase() .

Modifier and Type	Function	Description
static Boolean	regexMatches(String regex, String text)	Checks if a string matches a given regular expression.
static String	replaceAll(String regex, String text, String replacement)	In the text, this function replaces every subsequence that matches the regular expression with the defined replacement.
static String	replaceFirst(String regex, String text, String replacement)	In the text, this function replaces the first subsequence that matches the regular expression with the defined replacement.
static java.util.List<String>	regexSplit(String regex, String text)	Splits a given text according to a given regex.
static String	replace(String text, String find, String replace)	Replaces all occurrences of a string (find) within a text.
static java.util.List<String>	split(String s)	Splits the provided text into an array, using whitespace as the separator.
static java.util.List<String>	split(String s, String sep)	Splits the provided text into an array, using whitespace as the separator.
static String	stripAccents(String s)	Removes accents from a given string.
static String	substring(String text, Integer begin)	Gets a substring from the specified string, avoiding exceptions.
static String	substring(String text, Integer begin, Integer end)	Gets a substring from the specified string, avoiding exceptions.
static String	trim(String s)	Deletes all start and end whitespaces from a string as String.trim().
static String	trimOrNull(String s)	Deletes all start and end whitespaces from a string as String.trim().
static String	upperCase(String s)	Converts a string to upper case as per String.toUpperCase().

Parse class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Integer	asInt(str: String)	Convert the given string to an integer.
Long	asLong(str: String)	Convert the given string to a Long.
Double	asDouble(str: String)	Convert the given string to a Double.
Boolean	asBoolean(str: String)	Convert the given string to a Boolean.

MoreStrings class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Integer	countDigits(String s)	Counts the number of digits in a string.
static Integer	countLetters(String s)	Counts the number of letters in a string.
static Integer	countLowerCase(String s)	Counts the number of lower case letters in a string.
static Integer	countUpperCase(String s)	Counts the number of upper case letters in a string.
static String	replaceAll(String text, String regex, String replacement)	Replaces the regex occurrence in the text with the replacement.

Working with location and addresses

Address class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
String	joinAddressFields(name: String, line1: String, line2: String, zip: String, city: String, region: String, country: String)	Joins all the address fields into a single string, including the person's name. Null fields are treated as empty strings.
String	joinAddressFields(line1: String, line2: String, zip: String, city: String, region: String, country: String)	Joins all the address fields into a single string, excluding the person's name. Null fields are treated as empty strings.
String	extractNumericFields(str: String)	Returns the space separated numeric fields from the address, or an empty string if no numeric fields exist.
String	extractNumericFields(line1: String, line2: String, zip: String)	Returns the numeric fields from an address broken down into two lines and zip code. Returns an empty string if no numeric fields exist.
Boolean	isMismatch(fields1: String, fields2: String)	Returns true if the fields are not empty and different.
String	basicNormalization(address: String)	Normalizes the address to alphanumeric characters.
String	basicNormalization(name: String, line1: String, line2: String, zip: String, city: String, region: String, country: String)	Concatenates the parameters and normalizes the string to alphanumeric characters.
String	addressNormalizationReplacement(address: String, AddressNormalizationReplacements: Seq[(String, String)])	Applies replacement rules to a basic-normalized address, replacing each source token by its configured abbreviation/value.
String	advancedNormalization(address: String)	Replaces address suffixes with their abbreviation (e. g.: street to str), and normalizes the string to alphanumeric characters.
String	aggressiveNormalization(address: String)	Executes the advancedNormalization() function, then moves the numbers in the address to the front, and removes arithmetic progression substrings.
String	concatNormalizedFields(fields: String*)	Normalizes and concatenates the input strings into a single string. Removes all whitespaces and non-word characters, including applying Strings.normalizeToAlphanumeric().
String	concatNormalizedFieldsList(fields: JList[String])	Normalizes and concatenates the input strings into a single string. Removes all whitespaces and non-word characters, including applying Strings.normalizeToAlphanumeric().
String	normalizeUsaAddress(zip: String, line1: String)	Normalizes a US address to the most essential information of the address: the first digits of the zip code up to 4 digits, and the first characters of the normalized address line (10-14 characters, depending on the length of the zip), concatenated in <zip>_<normalizedaddress> format, and cut to 15 characters.
String	aggressiveNormalizations(input: String*)	Applies aggressive normalization after concatenating the nullable sequence of strings given as input.

AddressUtils class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Integer	countAddressSuffixes(String input)	Counts the suffixes in an address. Examples for suffixes: <i>av</i> , <i>ave</i> , <i>avenue</i> , <i>st</i> , <i>str</i> , <i>street</i> .

Modifier and Type	Function	Description
static Boolean	isEqualNormalized(String a, String b)	Returns true if the two input strings are equal after making both lower case, removing the address suffixes, ordinals, special characters and spaces.
static String	normalizeZipUK(String input)	Normalizes a UK zip code by removing non-alphanumeric characters and, when possible, keeping only the outward-code portion.

Country class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
staticString	alpha2ToAlpha3(String country)	Converts an alpha2 country code to alpha3 .
staticInteger	alpha2ToIso(String country)	Converts an alpha2 country code to numeric.
staticString	alpha3ToAlpha2(String country)	Converts an alpha3 country code to alpha 2.
staticInteger	alpha3ToIso(String country)	Converts an alpha3 country code to numeric.
staticString	getContinentFromAlpha2(String country)	Gets the continent from a country by its alpha2 code.
staticString	getContinentFromAlpha3(String country)	Gets the continent from a country by its alpha3 code.
staticString	getContinentFromIso(Integer country)	Gets the continent from a country by its iso numeric code.
staticString	isoToAlpha2(Integer country)	Converts a numeric iso country code to alpha 2.
staticString	isoToAlpha3(Integer country)	Converts a numeric iso country code to alpha 3.

DistanceUtils class

Package: `com.feedzai.common.udfs.distance`

Modifier and Type	Function	Description
static Double	distance(String curZipCode, String prevZipCode, String curCityName, String prevCityName, String curStateCode, String prevStateCode)	Calculates the approximate distance between two transactions.
static Double	gaussianProbability(Double value, Double avg, Double std)	Calculates the gaussian probability (from avg, std) of the given value.
static Double	getMostAccurateLatitude(String zipCode, String cityName, String stateCode)	Calculates the most accurate latitude by zip code, city name and state code.
static Double	getMostAccurateLongitude(String zipCode, String cityName, String stateCode)	Calculates the most accurate longitude by zip code, city name and state code.
static Double	haversine(Double lat1, Double lon1, Double lat2, Double lon2)	Calculates the approximate distance between two locations.

Geo class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Double	easting(lat: Double, lon: Double, granularity: Double = 1.0)	Returns the X ordinate (easting) in kilometers from a latitude and longitude coordinate pair, apart from a granularity scaling factor. The granularity scale factor defaults to 1.0. Scales the result to 1:granularity km.
Double	northing(lat: Double, granularity: Double = 1.0)	Returns the Y ordinate (northing) in kilometers from a latitude coordinate pair, apart from a granularity scaling

Modifier and Type	Function	Description
		factor. The granularity scale factor defaults to 1.0. Scales the result to 1:granularity km.
String	extractContinentFromCountry(country: String)	Returns the continent code correspondent to the input country code.
Double	countryDistInKm(country1: String, country2: String)	Returns the distance of two countries in kilometers.
String	extractCountryCodeFromTopLevelDomain(topLevelDomain: String)	Returns an ISO 3166-1 alpha-2 country code from a top-level domain. Returns "AA" if there is no associated country, and null for invalid top-level domains. Example: <code>Geo.extractCountryCodeFromTopLevelDomain(".pt")</code> returns <code>PT</code> .

Location class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
staticDouble	calculateTravelVelocity(Double distance, Long timestampStart, Long timestampEnd)	Returns travel velocity in kilometers per hour.
staticDouble	distFrom(Double latitude1, Double longitude1, Double latitude2, Double longitude2)	Distance in meters between two points on Earth's surface.
staticDouble	distFromInKm(Double atitude1, Double longitude1, Double latitude2, Double longitude2)	Distance in kilometers between two points on Earth's surface.
static Double	getCartesianX(Double lat, Double lon)	Converts polar coordinates into absolute coordinates.
static Double	getCartesianY(Double lat, Double lon)	Converts polar coordinates into absolute coordinates.
static Double	getCartesianZ(Double lat)	Converts polar coordinates into absolute coordinates.
staticDouble	getHaversineDistance(Double latitude1, Double longitude1, Double latitude2, Double longitude2)	The Haversine distance (distance on a spherical surface) between two points on Earth's surface.
static Double	getLatitudeFromCartesian(Double x, Double y, Double z)	Converts absolute coordinates into polar coordinates.
static Double	getLongitudeFromCartesian(Double x, Double y, Double z)	Converts absolute coordinates into polar coordinates.

LocationBean class

Package: `com.feedzai.common.udfs.distance`

Functions using a JavaBean that stores the location information from the location CSV file in the ProfileUtils class. See the source of the location data in <http://download.geonames.org/export/zip/>.

Modifier and Type	Function	Description
String	getCityName()	Gets the city name of the location record.
Double	getLatitude()	Gets the latitude of the location record.
Double	getLongitude()	Gets the longitude of the location record.
String	getStateCode()	Gets the state code of the location record
String	getZipCode()	Gets the zip code of the location record.
void	setCityName(String cityName)	Sets the city name of the location record.

Modifier and Type	Function	Description
void	setLatitude(Double latitude)	Sets the latitude of the location record.
void	setLongitude(Double longitude)	Sets the longitude of the location record.
void	setStateCode(String stateCode)	Sets the state code of the location record.
void	setZipCode(String zipCode)	Sets the zip code of the location record.
String	toString()	Gets the string representation of the location record.

TaxHeaven class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Boolean	isTaxHaven(countryCode: String)	Returns true if the country is on the taxHavensISO2CountryCodes list of tax haven country codes. Based on Hines' list from 2010. The country code must be an ISO 3166-1 alpha-2 country code.

Zip class

Package: `com.feedzai.pulse.fraud.udf`

Functions for working with postal codes. Country strings must be in [ISO 3166-1 alpha-2](#) format, e.g. US, RU, PT.

Modifier and Type	Function	Description
static Double	distFrom(String country1, String zipCode1, String country2, String zipCode2)	Gets the distance in meters between two zip codes
static Double	getLatitude(String country, String zipCode)	Returns the latitude from a country and zip code pair
static Double	getLongitude(String country, String zipCode)	Returns the longitude from a country and zip code pair
static <code>com.feedzai.pulse.fraud.udf.Zip.ZipData</code>	getZipData(String country, String zipCode)	Returns the <code>{ZipData}</code> object with the specified country and zipCode

Working with business-related data

Card class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Boolean	isValidCardExp(cardExp: String)	Checks if a string is a valid card expiration date using <code>cardExp.matches(regex = "(?:0[1-9] 1[0-2])/[0-9]{2}")</code>
Boolean	isDebit(cardCategory: String)	Check whether the card category is "debit". Case insensitive.
Boolean	isCredit(cardCategory: String)	Check whether the card category is "credit". Case insensitive.
Boolean	isPremium(cardCategory: String)	Check whether the card category is "premium". Case insensitive.
Long	daysUntilExpiration(cardExp: String, timestamp: Long)	Returns the number of days until the card expires, counting from the value received in the timestamp parameter.
Integer	compareNameToCardname(uname: String, cname: String)	Normalizes the two received parameters, and returns the distance between the two strings using <code>Strings.strDistance()</code> . Case insensitive.

Business class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Integer	<code>extractMCGFromMCC(mcc: String)</code>	Returns the merchant category group (MCG) code of a given merchant category code (MCC).

Email class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Boolean	<code>isValid(email: String)</code>	Checks if an email is valid.
String	<code>normalizeEmail(email: String)</code>	Normalizes the given email string by converting to lower case and removing whitespace characters.
String	<code>extractEmailDomain(email: String)</code>	Extracts the domain of a nullable email address.
String	<code>extractEmailTopLevelDomain(email: String)</code>	Extracts the top-level domain of an email address (for example, <code>.pt</code>). Returns null if the email is null or invalid. Example: <code>Email.extractEmailTopLevelDomain("")</code> returns <code>.pt</code> .

EmailUtils class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static String	<code>normalizeEmailAddress(String str)</code>	Normalizes an email address: • Removes dots, whitespaces, plus signs, and any tag added after the plus sign. • Converts the email address to lower case. • Converts the <code>googlemail.com</code> domain to <code>gmail.com</code>.

Items class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Boolean	<code>keyContains(Object obj, String key, String value)</code>	Returns true if, for the given key, there is at least one value equal to the provided string value.
static Boolean	<code>keyContains(Object obj, String key, Boolean value)</code>	Returns true if, for the given key, there is at least one value equal to the provided boolean value.
static Boolean	<code>customFieldsContains(Object obj, String key, String value)</code>	Returns true if, for the given key, there is at least one value in <code>custom_fields</code> equal to the provided value.
static Boolean	<code>containsProduct(Object obj, String productName, String productStyle, String productColor)</code>	Returns true if there is at least one product matching the given name, style, and color combination.
static Integer	<code>getDistinctSizes(Object obj)</code>	Returns the number of distinct product sizes in the provided items set, or <code>-1</code> if the input is invalid.
static Integer	<code>getDistinctPrices(Object obj)</code>	Returns the number of distinct prices in the provided items set, or <code>-1</code> if the input is invalid.

Modifier and Type	Function	Description
static Integer	sumQuantity(Object obj)	Returns the sum of item <code>quantity</code> values, or <code>-1</code> if the input is invalid.
static Long	sumPrice(Object obj)	Returns the sum of item <code>price</code> values, or <code>-1</code> if the input is invalid.
static Long	sumPriceUSD(Object obj)	Returns the sum of item <code>price_usd</code> values, or <code>-1</code> if the input is invalid.

Phone class

Package: `com.feedzai.cloud.udf`

Country codes use ISO 3166-1 two-letter country-code format.

Modifier and Type	Function	Description
String	countryCode(number: String)	Returns the country code in ISO 3166-1 two-letter country-code format.
Boolean	isValidForCountry(number: String, country: String)	Returns true if the phone number is valid for a given country. If no country is provided, the function returns false.
String	numberType(number: String, country: String)	Gets the phone number type: Fixed-line, Mobile, Toll-free, Premium Rate, Shared Cost, VoIP and Personal Numbers.
Boolean	isMobile(number: String, country: String)	Returns true if the phone number type is mobile.
Boolean	isTollFree(number: String, country: String)	Returns true if the phone number type is toll-free.
Boolean	isVoIP(number: String, country: String)	Returns true if the phone number type is "VoIP".
Boolean	isPremiumRate(number: String, country: String)	Returns true if the phone number type is Premium Rate ("uan").
Boolean	isSharedCost(number: String, country: String)	Returns true if the phone number type is Shared Cost.
Boolean	isFixedLine(number: String, country: String)	Returns true if the phone number type is Fixed Line.
Boolean	isPersonalNumber(number: String, country: String)	Returns true if the phone number type is Personal Number.
String	normalizePhoneNumber(number: String)	Normalizes a phone number by removing spaces, replacing '+' by '00', and removing all other characters that are not numbers.

Working with timestamps

Time class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Long	getDays2Expire(Long timestamp, String expiredate)	Gets the time delta (in days) to expiration.
static Long	getDays2Expire(Long timestamp, String expiredate, String format)	Gets the time delta (in days) to expiration.
static Integer	getHourOfDay(Long timestamp, String timezone)	Gets the hour of the day for a specific timestamp.
	getLocalHourOfDay(Long timestamp)	

Modifier and Type	Function	Description
static Integer		Gets the hour of the day for a specific timestamp assuming the local timezone.
static Integer	getLocalMonth(Long timestamp)	Gets the month for a specific timestamp assuming the local timezone.
static Integer	getLocalMonthDay(Long timestamp)	Gets the day of the month for a specific timestamp assuming the local timezone.
static Integer	getLocalWeekDay(Long timestamp)	Gets the day of the week for a specific timestamp assuming the local timezone. 1 - Monday; 7 - Sunday.
static Integer	getLocalYear(Long timestamp)	Gets the year for a specific timestamp assuming the local timezone.
static Integer	getLocalYearDay(Long timestamp)	Gets the day of the year for a specific timestamp assuming the local timezone.
static Long	getMillis2Expire(Long timestamp, String expiredDate)	Gets the time delta (in milliseconds) to expiration.
static Long	getMillisAtStartOfDay(Integer year, Integer month, Integer dayOfMonth)	Obtains a UTC timestamp (in milliseconds) from the provided calendar year/month/dayOfMonth.
static Integer	getMonth(Long timestamp, String timezone)	Gets the month for a specific timestamp.
static Integer	getMonthDay(Long timestamp, String timezone)	Gets the day of the month for a specific timestamp.
static Integer	getUTCHourOfDay(Long timestamp)	Gets the hour of the day for a specific timestamp assuming the UTC timezone.
static Integer	getUTCMonth(Long timestamp)	Gets the month for a specific timestamp assuming the UTC timezone.
static Integer	getUTCMonthDay(Long timestamp)	Gets the day of the month for a specific timestamp assuming the UTC timezone.
static Integer	getUTCWeekDay(Long timestamp)	Gets the day of the week for a specific timestamp assuming the UTC timezone. 1 - Monday; 7 - Sunday.
static Integer	getUTCYear(Long timestamp)	Gets the year for a specific timestamp assuming the UTC timezone.
static Integer	getUTCYearDay(Long timestamp)	Gets the day of the year for a specific timestamp assuming the UTC timezone.
static Integer	getWeekDay(Long timestamp, String timezone)	Gets the day of the week for a specific timestamp. 1 - Monday; 7 - Sunday.
static Integer	getYear(Long timestamp, String timezone)	Gets the year for a specific timestamp.
static Integer	getYearDay(Long timestamp, String timezone)	Gets the day of the year for a specific timestamp.
static Long	parseDate(String strDate, String format)	Obtains a UTC timestamp (in milliseconds) from a string representation of a date.
static Long	parseExpirationDate(String strDate, String format)	Obtains a UTC timestamp (in milliseconds) from an expiration date string. If the format has no day component, returns the first timestamp of the subsequent month. Example: Time.parseExpirationDate("202003", "yyyyMM") returns the timestamp for 2020-04-01 00:00:00.000 (UTC).
static Long	parseDateOrNullOnError(String strDate, String format)	Checks if the given parameters can be parsed into a Date in milliseconds.

TimeTools class

Package: `com.feedzai.commons.udfs`

Modifier and Type	Function	Description
static Integer	dayOfMonth(Long timestamp)	Returns the "day of month" portion of the given timestamp at the default system timezone.
static Integer	dayOfMonth(Long timestamp, String timezone)	Returns the "day of month" portion of the given timestamp at the given timezone.
static Integer	dayOfWeek(Long timestamp)	Returns the "day of week" portion of the given timestamp at the default system timezone.
static Integer	dayOfWeek(Long timestamp, String timezone)	Returns the "day of week" portion of the given timestamp at the given timezone.
static Integer	dayOfYear(Long timestamp)	Returns the "day of year" portion of the given timestamp at the default system timezone.
static Integer	dayOfYear(Long timestamp, String timezone)	Returns the "day of year" portion of the given timestamp at the given timezone.
static Integer	hour(Long timestamp)	Returns the "hour" portion of the given timestamp at the default system timezone.
static Integer	hour(Long timestamp, String timezone)	Returns the "hour" portion of the given timestamp at the given timezone.
static Integer	minute(Long timestamp)	Returns the "minute" portion of the given timestamp at the default system timezone.
static Integer	minute(Long timestamp, String timezone)	Returns the "minute" portion of the given timestamp at the given timezone.
static Integer	month(Long timestamp)	Returns the "month" portion of the given timestamp at the default system
static Integer	month(Long timestamp, String timezone)	Returns the "month" portion of the given timestamp at the given timezone.
static Integer	second(Long timestamp)	Returns the "second" portion of the given timestamp at the default system timezone.
static Integer	second(Long timestamp, String timezone)	Returns the "second" portion of the given timestamp at the given timezone.
static Integer	weekOfYear(Long timestamp)	Returns the "weekOfYear" portion of the given timestamp at the default system timezone.
static Integer	weekOfYear(Long timestamp, String timezone)	Returns the "weekOfYear" portion of the given timestamp at the given timezone.
static Integer	year(Long timestamp)	Returns the "year" portion of the given timestamp at the default system timezone.
static Integer	year(Long timestamp, String timezone)	Returns the "year" portion of the given timestamp at the given timezone.

MoreTime class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Integer	getMonthLength(Long timestamp, String timezone)	Calculates the number of days of the month for a given timestamp and timezone.

Modifier and Type	Function	Description
static Integer	getUTCMonthLength(Long timestamp)	Calculates the number of days of the month for a given timestamp in UTC timezone.

Working with JSON

Json

Package: `com.feedzai.cloud.udf`

For big JSON documents, consider using the functions described in the `AbstractCachedJson` class.

Modifier and Type	Function	Description
String	getString(id: String, json: String, path: String, forceCoersion: Boolean)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, it can be coerced to one, if the corresponding parameter is set. Otherwise the function logs an error and returns null.
String	getString(id: String, json: String, path: String)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, the function logs an error and returns null.
String	getAsString(id: String, json: String, path: String)	Gets a field from a JSON document, as a String, according to a given JSON path expression. If the field is not a string, it will be coerced to one.
Boolean	getBoolean(id: String, json: String, path: String)	Gets a Boolean field from a JSON document, according to a given JSON path expression.
Integer	getInteger(id: String, json: String, path: String)	Gets an Integer field from a JSON document, according to a given JSON path expression.
Long	getLong(id: String, json: String, path: String)	Gets a Long field from a JSON document, according to a given JSON path expression.
Float	getFloat(id: String, json: String, path: String)	Gets a Float field from a JSON document, according to a given JSON path expression.
Double	getDouble(id: String, json: String, path: String)	Gets a Double field from a JSON document, according to a given JSON path expression.
String	getAndConcatStrings(id: String, json: String, paths: JList[String], delim: String)	Gets multiple fields, converted to String, from a JSON document as a concatenated string with their values, according to a given JSON path expression.
Boolean	contains(id: String, json: String, path: String)	Checks if a field exists in a JSON document, according to a given JSON path expression.

AbstractCachedJson

Package: `com.feedzai.cloud.udf`

This class includes functions for handling big JSON strings. The functions in this class cache the result of the parsing to avoid reparsing on every call. For smaller files, consider using the functions described under the `Json` class.

Modifier and Type	Function	Description
String	getString(id: String, json: String, path: String, forceCoersion: Boolean)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, it can be coerced to one, if the corresponding parameter is set. Otherwise the function logs an error and returns null.
String		

Modifier and Type	Function	Description
	getString(id: String, json: String, path: String)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, the function logs an error and returns null.
String	getAsString(id: String, json: String, path: String)	Gets a field from a JSON document, as a String, according to a given JSON path expression. If the field is not a string, it will be coerced to one.
Boolean	getBoolean(id: String, json: String, path: String)	Gets a Boolean field from a JSON document, according to a given JSON path expression.
Integer	getInteger(id: String, json: String, path: String)	Gets an Integer field from a JSON document, according to a given JSON path expression.
Long	getLong(id: String, json: String, path: String)	Gets a Long field from a JSON document, according to a given JSON path expression.
Float	getFloat(id: String, json: String, path: String)	Gets a Float field from a JSON document, according to a given JSON path expression.
Double	getDouble(id: String, json: String, path: String)	Gets a Double field from a JSON document, according to a given JSON path expression.
String	getAndConcatStrings(id: String, json: String, paths: JList[String], delim: String)	Gets multiple fields, converted to String, from a JSON document as a concatenated string with their values, according to a given JSON path expression.
Boolean	contains(id: String, json: String, path: String)	Checks if a field exists in a JSON document, according to a given JSON path expression.

Working with collections and complex fields

Collections class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static String	getFieldByKey(Object complexField, String key, String keyValue, String fieldToRetrieve)	Retrieves a field from a complex field by matching tuples with the provided key and key value. Returns an empty string on any validation or lookup error. Example: given <code>sender_account_limits</code> with <code>{"event_type": "ACC_LIMIT", "daily_amount": 500000}</code> , calling <code>com.feedzai.pulse.fraud.udf.Collections.getFieldByKey(sender_account_limits, "event_type", "ACC_LIMIT", "daily_amount")</code> returns <code>500000</code> .
static Boolean	contains(java.util.Collection<java.lang.Object> collection, java.lang.Object valueToFind)	Checks if a collection contains a specific value.
static Boolean	containsCaseInsensitive(java.util.Collection<java.lang.Object> collection, java.lang.String valueToFind)	Checks if a collection contains a specific value. Ignores casing when testing for equality.

Collections class

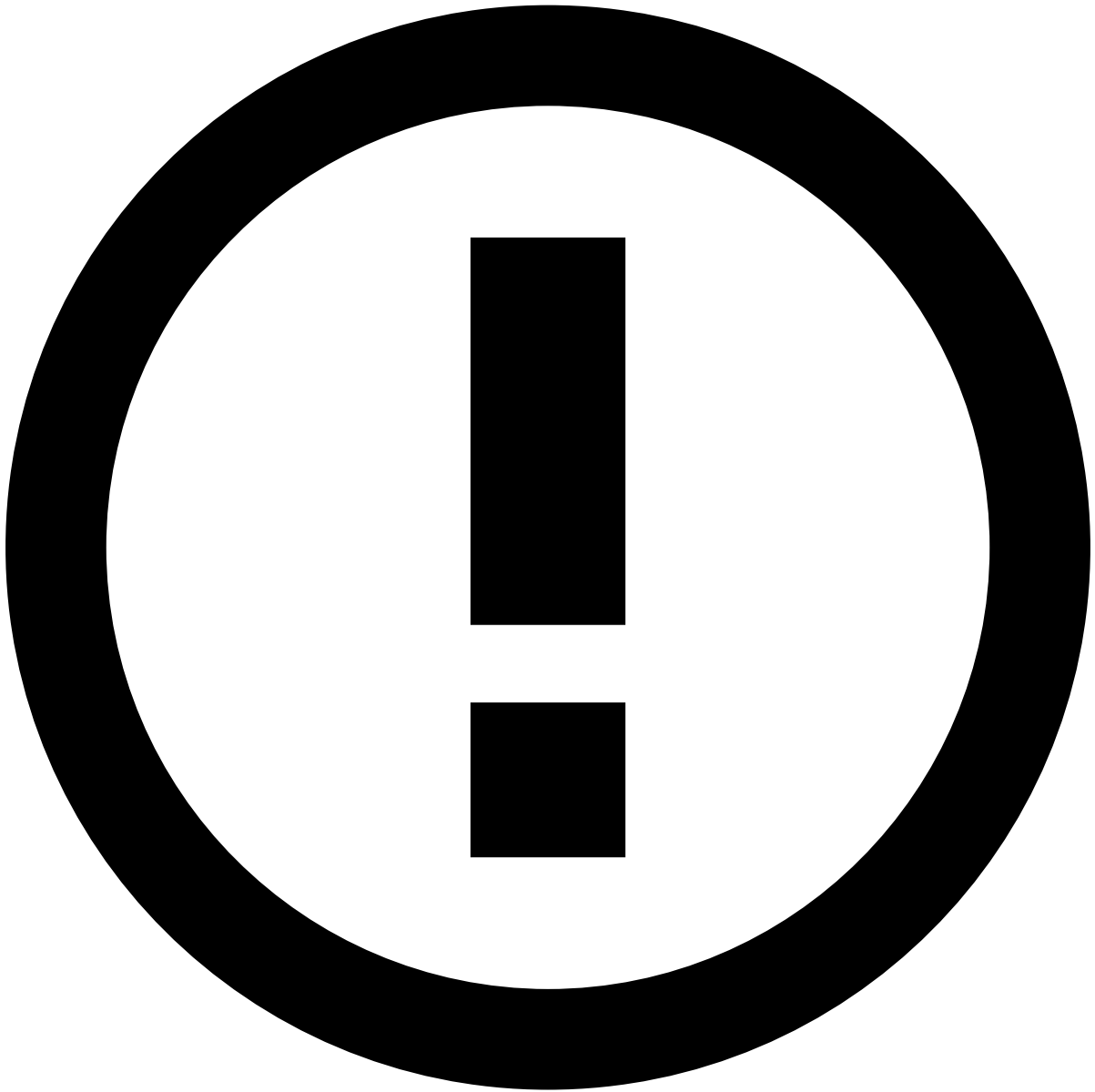
Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Tuple	getSampleTuple(set: JSet[Tuple])	Retrieves the next tuple from a JSet of Tuples.

CollectionUDFs class

Package: `com.feedzai.pulse.pkernel.runtime`

As the CollectionUDFs class is included in the runtime package of Pulse, you don't need to define an alias to use the functions of this class.



Lists vs Maps

The collection called Map of Pulse Query Language (PQL) and Rule Definition Language (RDL) is the equivalent of a Pulse list created under **Lists** . The following table refers to them as **Maps**.

Modifier and Type	Function	Description
Integer	count(Object collection)	Returns the number of entries in a collection Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a Collection.

Modifier and Type	Function	Description
		<pre>// Returns 2 CollectionUDF.count(map("a": 1, "b": 2))</pre>
Boolean	contains(Object collection, Object element)	Checks if <i>element</i> is present in <i>collection</i>. Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a Collection. <pre>// Returns true CollectionUDF.contains(map("a": 1), 1)</pre>
Integer	getInt(Object collection, Object? key)	Returns the integer value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns 5 CollectionUDF.getInt(list(1, 3, 5), -1)</pre>
Long	getLong(Object collection, Object key)	Returns the long value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns 3 CollectionUDF.getLong(map("a": 3), "a")</pre>
Float	getFloat(Object collection, Object key)	Returns the float value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer.

Modifier and Type	Function	Description
		<pre>// Returns null CollectionUDF.getFloat(list(null), 0)</pre>
Double	getDouble(Object collection, Object key)	Returns the double value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns 2.5 CollectionUDF.getDouble(list(3.14, 2.5), 1)</pre>
string	getString(Object collection, Object key)	Returns the string value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a string. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns "value" CollectionUDF.getString(map("key": "value"), "key")</pre>

Tuples class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
AnyRef	getField(tuple: Tuple, field: String)	Returns the field value, or null if the tuple doesn't contain the field or if the tuple is null.

ComplexFields class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
String	getString(map: JMap[String, String], key: String)	Get a string field from a string JMap, according to a given key. If the field is not a string or doesn't exist, the function returns null. This function can't be used in Plans, because PQL does not support calls to UDFs with non-primitive parameters. As an alternative use the getStringObj UDF instead.

Modifier and Type	Function	Description
		This function can't be used in Plans , because PQL does not support calls to UDFs with non-primitive parameters. As an alternative use the getStringObj UDF instead.
String	getStringObj(map: AnyRef, key: String)	Get a string field from a string JMap, according to a given key. If the field is not a string or doesn't exist, the function returns null. This function is analogous to getString() but receives the map as an AnyRef. This function is useful when using complex fields in PQL, because PQL does not support calls to UDFs with non-primitive parameters.

Mathematical calculations

Maths class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Integer	binNumber(Double value, Double binSize)	Gets the bin number of a value
static Double	coalescedSafeDivision(Double numerator, Double denominator)	Divides numerator by denominator with safeguards for bad divisions
static Double	diffNormalizeAndCap(Double value, Double expected, Double cap)	Computes the difference between a value and the expected, normalized by the expected value, capped to a maximum
static Double	logOfNormalPDF(Double mean, Double std, Double x)	Natural logarithm of normalPDF(Double, Double, Double)
static Double	normalCDF(Double mean, Double std, Double x)	Cumulative distribution function (CDF) for a normal distribution with parameters <code>mean</code> and <code>std</code> at value <code>x</code> .
static Double	minimum(Double a, Double b, Double c)	Three-way minimum
static Integer	minimum(Integer a, Integer b, Integer c)	Three-way minimum
static Double	maximum(Double a, Double b, Double c)	Three-way maximum
static Integer	maximum(Integer a, Integer b, Integer c)	Three-way maximum
static Double	normalPDF(Double mean, Double std, Double x)	Density of the normal distribution with parameters <code>mean</code> and <code>std</code> at the value <code>x</code>
static Double	safeDivision(Double numerator, Double denominator)	Executes a safe division that returns the division value or zero if the operation is not a number or division by zero
static Double	safeDivision(Long numerator, Long denominator)	Executes a safe division that returns the division value or zero if the operation is not a number or division by zero
static Double	toDeg(Double angle)	Converts radians to degrees
static Double	toRad(Double angle)	Converts degrees to radians



PDF

The `normalPDF` function has a corner case defined for when the standard deviation (`std`) is zero. In that case, it will return either 1 if the value `x` equals the mean parameter or 0 otherwise. This might affect your feature distribution because the outcome of `normalPDF` is the likelihood, which is not bound.

Working with model scores

ScoreTransformationUdfs class

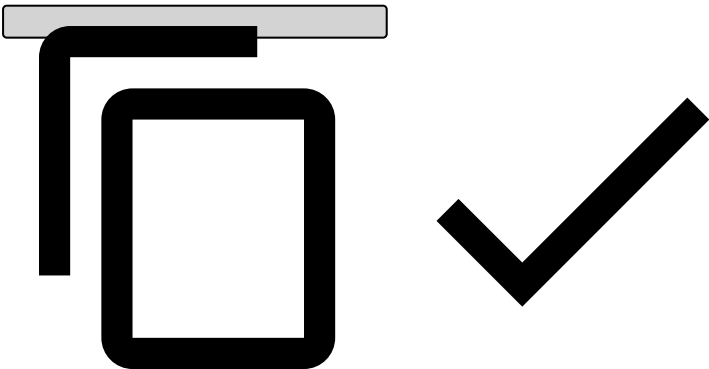
Package: com.feedzai.ds.udf

Modifier and Type	Function	Description
static Double	amountDependentScoreTransformation(Double rawScore, Double amount, Double intensity, Double cap)	Transforms the model score using the transaction amount parameterized by an intensity factor and an amount cap.
static Double	thresholdAndAmountDependentScoreTransformation(double rawScore, long amount, double anchor, Object amountBinAndThresholdObject, double defaultThreshold)	Transforms the model using amount-dependent thresholds and an anchor value.

Usage examples:

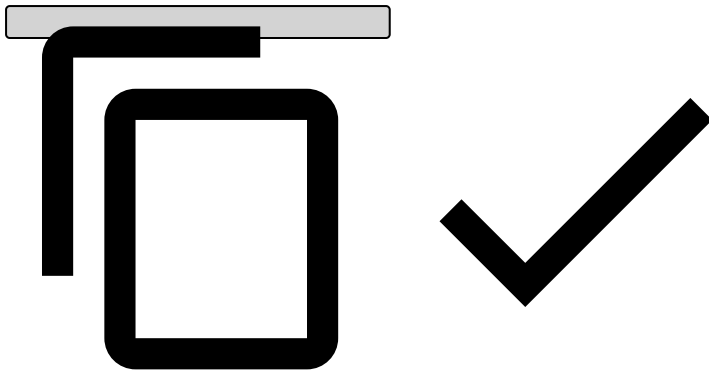
Transforming scores with the amount-dependent score transformation:

```
com.feedzai.ds.udf.ScoreTransformationUdfs.amountDependentScoreTransformation(  
    fraud_true_scored // original score to be transformed  
    amount           // amount field  
    0.3              // lower intensity means lower amounts are greatly affected  
  
    5000000 // when 'amount > cap', the score remains unchanged. In this case scores for amounts  
            over $50k are unchanged  
)
```



Transforming scores with piecewise linear transformations dependent on the amount:

```
com.feedzai.ds.udf.ScoreTransformationUdfs.thresholdAndAmountDependentScoreTransformation(  
    fraud_true_scored, // original score to be transformed  
    amount,           // amount field  
    0.850,             // lower anchor means the raw score is more affected by the transformations  
    {1000: 0.800, 6750: 0.750, 20420: 0.654}, // mapping amounts to thresholds {amount1:  
    threshold1, amount2: threshold2, ...}  
    0.500 // default threshold, ie for amounts greater than the max amount above  
)
```



Cryptography🔒📄

Crypto class🔒📄

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static String	<code>cryptoHash(String algo, Object obj)</code>	Hashes an object using a hashing algorithm.
static String	<code>cryptoHash(String algo, Object obj, String salt)</code>	Hashes an object using a hashing algorithm.
static String	<code>md5(Object obj)</code>	Computes the md5 hash of the object.
static String	<code>sha512(Object obj)</code>	Computes the sha-512 hash of the object.

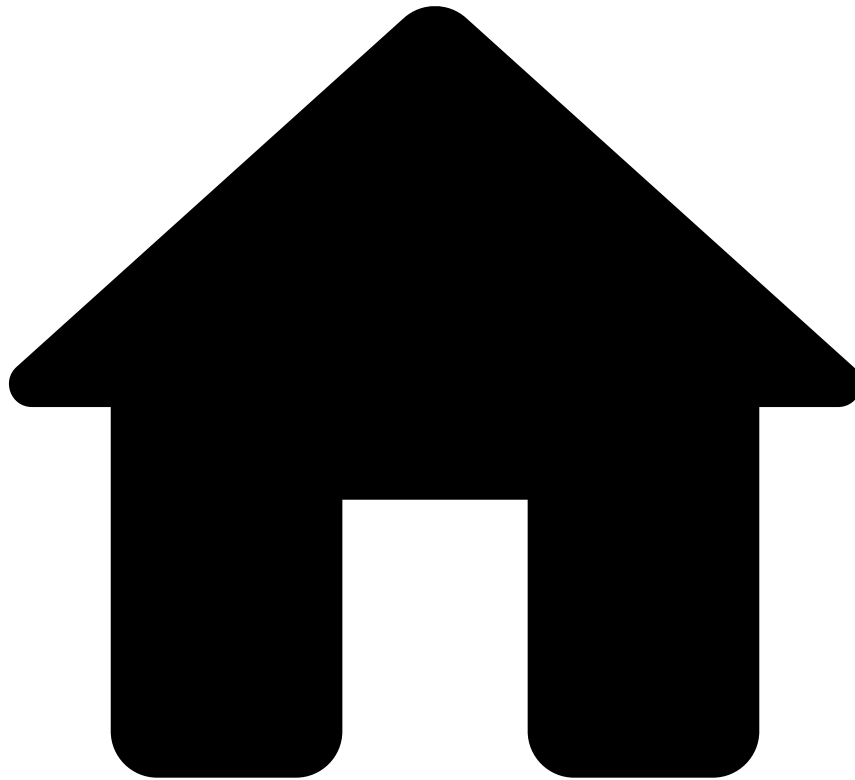
Debugging🔍📄

Debug class🔍📄

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Double	<code>debug(Double value, String comment)</code>	Debug function to log a message and a value and return the original object.
static Float	<code>debug(Float value, String comment)</code>	Debug function to log a message and a value and return the original object.
static Integer	<code>debug(Integer value, String comment)</code>	Debug function to log a message and a value and return the original object.
static Long	<code>debug(Long value, String comment)</code>	Debug function to log a message and a value and return the original object.
static Short	<code>debug(Short value, String comment)</code>	Debug function to log a message and a value and return the original object.
static Integer	<code>debugObject(Object value, String comment)</code>	Debug function to log a message and a value.

•



- [Core Concepts](#)
- Workflows

On this page

Workflows

Was this page helpful? 

"A workflow consists of a sequence of connected steps where each step follows without delay or gap and ends just before the subsequent step may begin." (in [Wikipedia](#))

Workflows represent what Pulse executes in runtime. As Pulse integrates the [data science concepts](#) with their runtime counterparts, a workflow is the result of deploying those concepts directly into a Production environment. A workflow is where you can compose these concepts into a sequence of processing steps that produce a decision in real time.

As an example scenario, fraud detection is a task comprised of several steps with the aim of blocking fraudulent transactions. Usually, transactions must go through one or more sets of [rules](#), one or more [models](#) for scoring and [plans](#) that perform transformations on data, in order to determine if they are fraudulent.

Thus, we group these processing steps into a workflow. Workflows represent the life cycle of an event through the system. They define the path that information traverses between components that may filter and/or transform the event.

If there is one thing essential to workflows, it is data. Without it, there is no sense in having workflows and all the remaining application resources. So, workflows always operate over an expected input [data schema](#).

The following image displays a runtime workflow in Pulse:

Channels

Each workflow typically corresponds to a particular channel through which the system receives information. For example, one workflow could be processing debit card transactions (debit channel), while another workflow would be dedicated to internet banking transfers (internet banking channel). Because different Channels typically have different Schemas (see below) and require a different set of rules and models, this approach ensures that each Workflow is optimized for the channel it's processing. For more information on channels, see [Channels](#).

Outcomes

The ultimate goal of a runtime workflow is to produce a generic decision and insight on each event. Outcomes are flexible and use case-agnostic representations of those decisions. For example, *fraud/not fraud*, or *suspicious/not suspicious*.

A workflow can be configured with several outcomes for finer-grained decisions. For example, adding the type of fraud to events, instead of just classifying them as *fraud/not fraud*.

Pulse adds each outcome decision field (e.g *fraud*) and outcome score field (e.g. *fraud_score*) to the schema of events moving along the workflow.

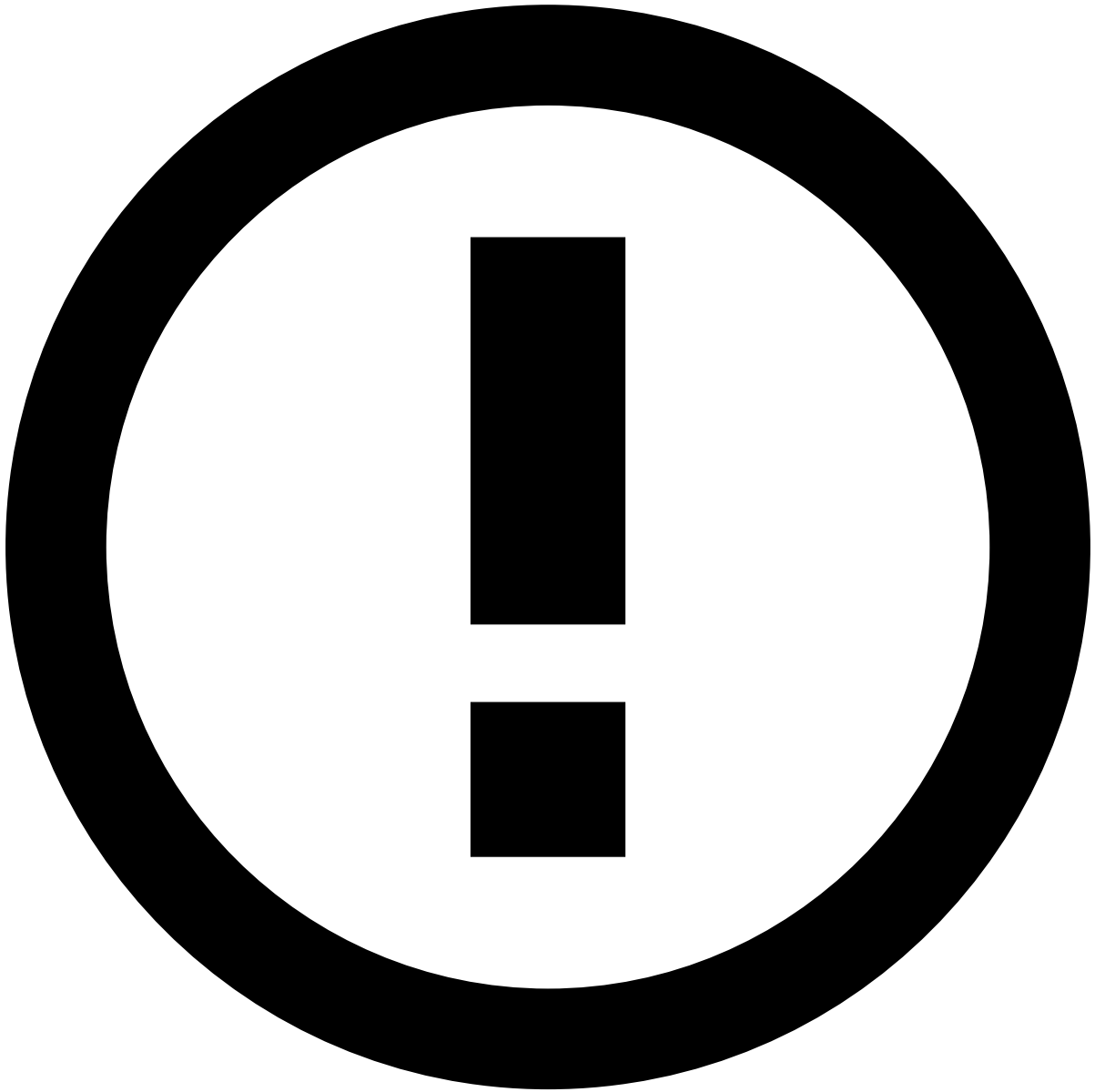
As you add elements such as scoring services and rules services, you must also configure how they affect these fields, i.e. how they contribute to each of the final decisions and scores of the workflow.

Another very important capability of outcomes is enabling you to manage feedback loops better, and to integrate with human validation. Whenever an event traverses a workflow, Pulse saves it in the [event storage](#). Thus, it is possible to recover the Outcome of events at a later time to:

- Generate reports on model accuracy and outcome distribution for a consistent evaluation of runtime performance.
- Retrain models using the event storage as a source of data after receiving human input. This is especially important to close the loop on human feedback, for example, when Pulse classifies a transaction as fraud but a human analyst overrides the decision based on real-world evidence.

Workflow schemas

Runtime workflows in Pulse must always be configured with an origin schema. This schema, configured at the workflow level, defines the structure and formatting of the events that Pulse consumes from an external source of data.

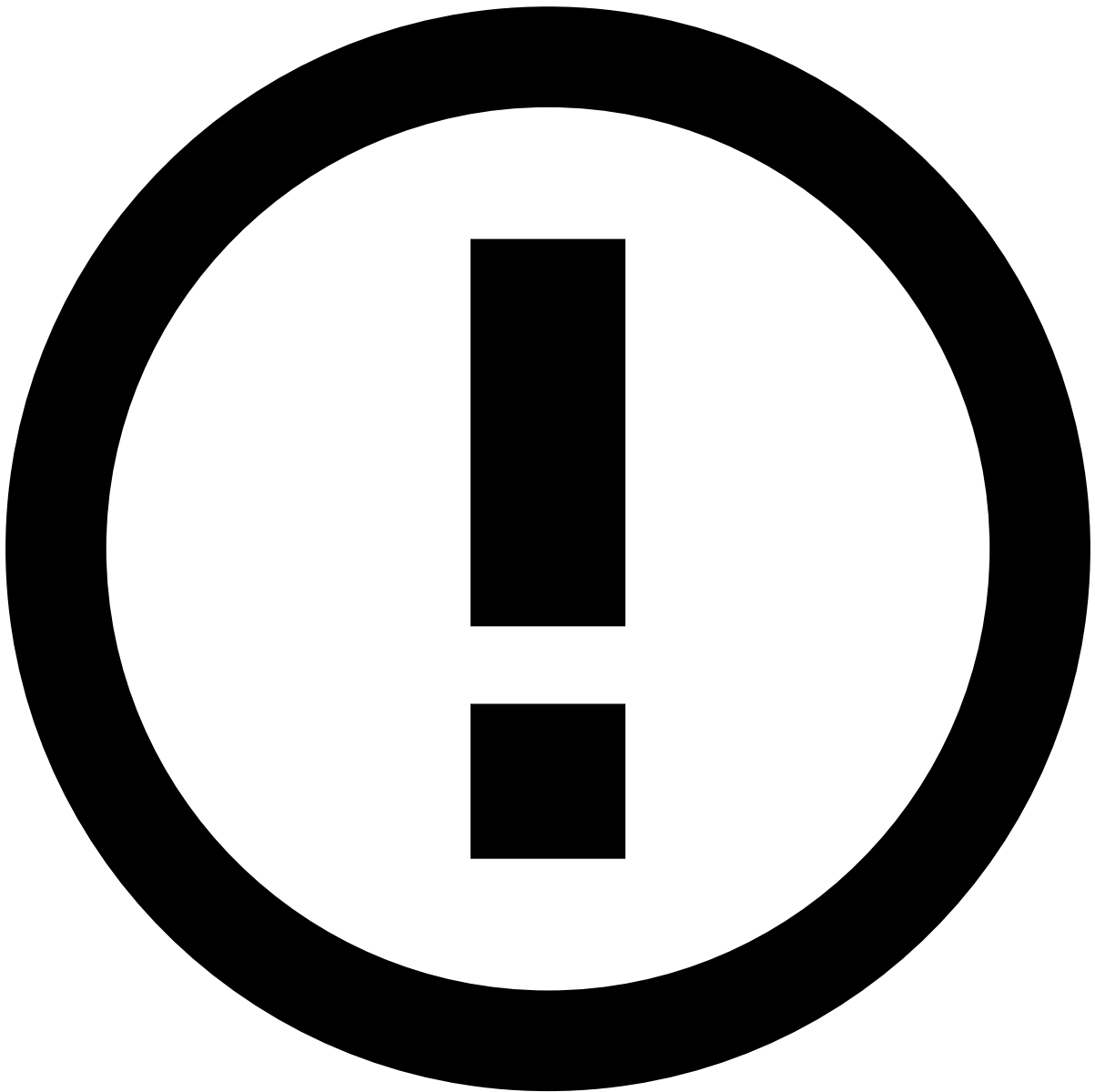


Workflow schema immutability

To ensure that the workflow is always capable of correctly consuming data, the [origin schema is immutable](#).

However, as events flow through the different stages of the workflow, their schema can change. For example, new fields are added to the event when it is processed in a [plan](#) and enriched with features and profiles.

Additionally, if a workflow element only requires a subset of fields in the schema as input, only those are affected in the event once it receives the output of the element. The rest of schema fields remain unchanged and still available for subsequent workflow elements.



Mutable element schemas

Despite the fact that the schema you configure for input data in the workflow is immutable, the input and output schemas for each workflow element can be different.

This depends on the logic you implement for each workflow element, for example in a plan, by removing irrelevant fields, changing the names of fields to match project specifications, adding profiles, generating features and adding enrichment fields.

Conflicts

Since workflow schemas are mutable, when an event reaches a Workflow Service its schema may have been transformed previously by another service.

When configuring a workflow service, if Pulse is unable to match fields from the workflow schema to the input or output schemas of the service, it will generate conflicts and invalidate the workflow configuration. Pulse generates a conflict for

each field required by the service that is not present in the workflow schema with the same name and type. This can be due to:

- Input fields with different names, for example *merchant_name* and *mcn*.
- When you remove fields with a [plan](#).
- When the service outputs a new field that already exists in the workflow schema.

In these cases, to fix your workflow, you must solve conflicts by manually mapping fields from the workflow schema to the input and output schemas expected in the service.



Removed fields

If a plan you configured for a plan service removes fields from the workflow schema, then you must ensure that subsequent services in the workflow do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your workflow, and you will have no available fields to map from.

In these cases, you should redesign your plan, articulate it with the remaining data science work to be used in runtime, and redeploy it to your workflow.

Extra fields

In runtime tasks, for example, when you [design a workflow](#), Pulse automatically adds the following fields to the event schema:

- **timestamp**: The timestamp field is used to signal the date of a transaction in the UNIX Timestamp Format. See [Time, schedules and timezones](#) for more information.



Timestamp

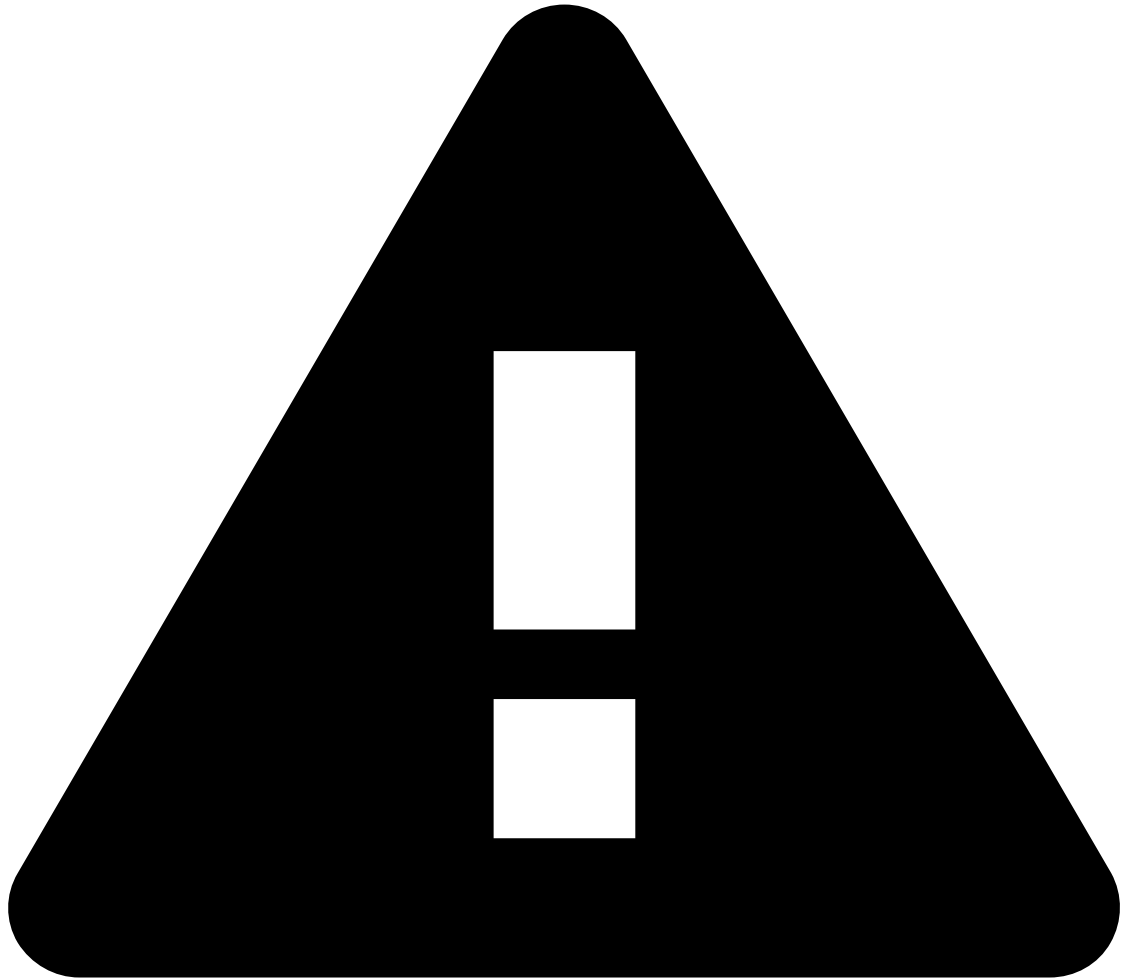
The value for this field must always be greater than or equal to the previous value processed since it's not possible to go back in time.



Reserved words

The word *timestamp* is reserved. Pulse does not allow you to use a [metrics plan](#) to calculate metrics over a field named *timestamp*.

- **UUID:** The unique id of the event. This field is used by several internal services and can also be used by external ones to uniquely identify an event, correlate results, access stored events, among others.



UUID

The system does not fill in this field automatically. The user building the application is responsible for setting it correctly when creating the events.



Reserved words

The word *uuid* is reserved. Pulse does not allow you to name any field name as *uuid*.

- **Outcome:** [Outcomes](#) represent decisions of your workflow, for example, *fraud/not fraud*. It is a categorical field used to mark an event as, for example, either fraudulent or not.
- **Outcome score:** The transaction score that corresponds to the workflow decision represented by the outcome. Scoring services and rules services contribute to outcome scores.

Workflow elements

A workflow consists of processing steps (elements) and links between them. Each workflow element can process or transform the data as soon as the data arrives at it. Workflow elements can be seen as components that process data with the ability to apply operations such as filter and store.

Feedzai Pulse provides three main services:

- **Rules service:** Is a Pulse service used to declare multiple [rules](#) to detect and mark patterns, optionally contributing to an outcome.
- **Scoring service:** Is a module that feeds events into a [model](#) and produces a score, optionally contributing to an outcome.
- **Plan service:** Enables you to deploy data transformations you have defined and tested in [plans](#) at the data science stage, for example, to generate features before passing events onto the scoring service.

In addition to these core components provided off-the-shelf, Pulse also lets you include [custom services](#) in your workflows.

Rules service

The **rules service** allows defining the [rules](#) to be applied to a transaction event and the actions to be taken when the rule is matched. Additionally, a rules service can output a score and contribute to an outcome.

There are different mechanisms that can be applied when a given rule is matched:

- The corresponding event contributes to an outcome, similarly to how the record is immediately marked as a target value during the data science stage.
- Jump to the next workflow step, or even skip the remaining steps if the control flow of the rule is configured as *Stop Workflow*.
- A specific action can be triggered and processed by [custom services](#). They can then apply specific modifications to the corresponding transaction event when handling the action.



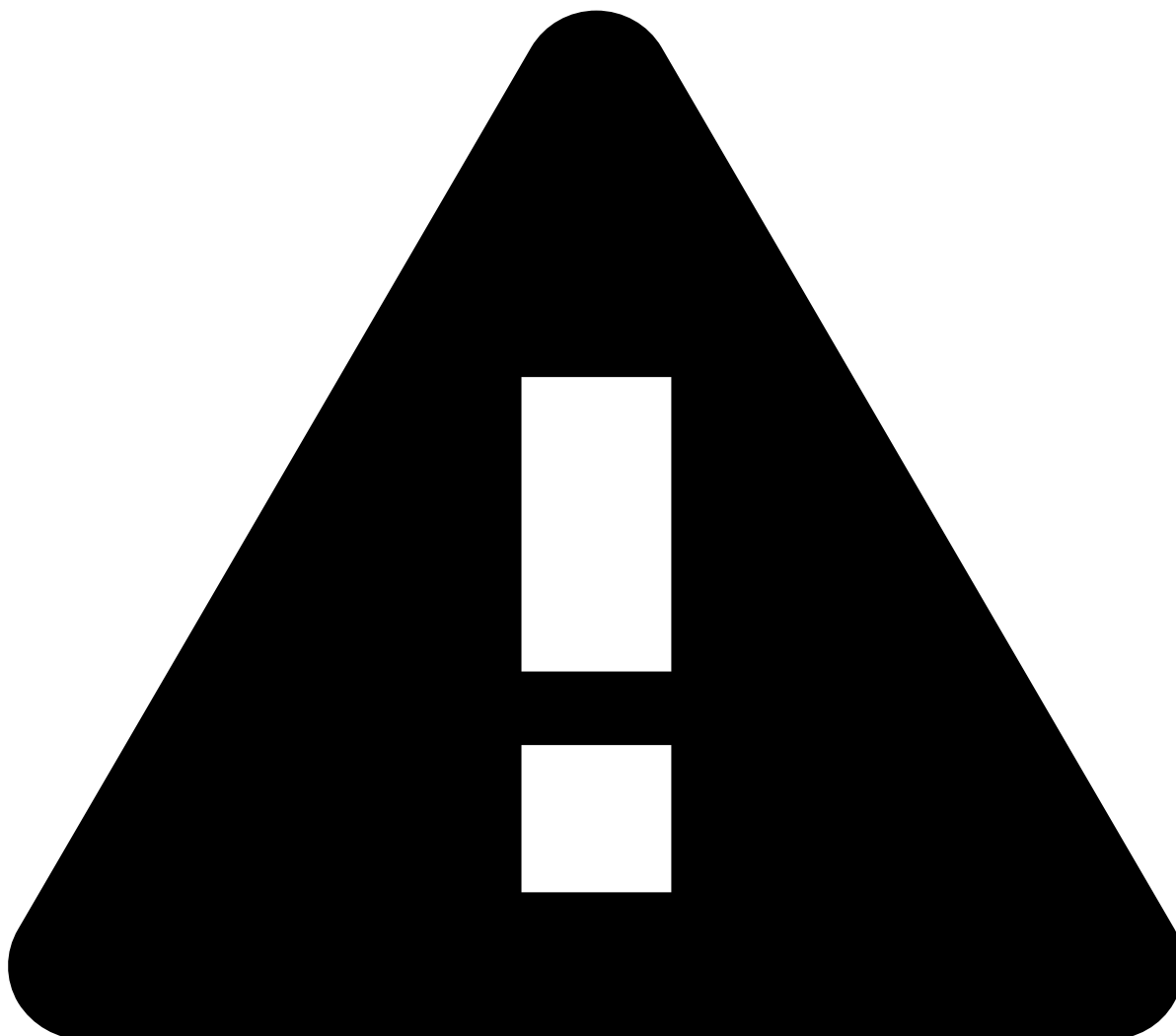
danger

The **rules service does not evaluate the rules** for every record that passes on Pulse. If the message respective to the record appeared due to a replica or recovery process and the rules project configuration is set to be [stateless](#), then the rules will not be evaluated. This change is motivated by performance, since the calls to rules engine are expensive, avoiding some of the calls will reduce the execution time.

Scoring service🔗📄

A scoring service executes a [model](#) that was previously trained during the [data science loop](#). The job of a scoring service consists in **producing a score for each event** that goes through the service. For example, a score that denotes the likelihood of the event representing a fraudulent transaction.

Additionally, and depending on how you configure the scoring service, it will also **contribute to one of the outcomes** in the workflow. In some cases, it may make sense for you to have a deployed model without influencing the final decisions of the workflow. For example, you may want to temporarily deploy a model just to test its live performance for a few days/weeks before phasing it in and start contributing to outcomes. In these cases, you can retrieve the scores, for example with a [custom service](#), from the [event storage](#) or using Pulse's API.



White-box explanations in the scoring service

You may need to edit the scoring services that are in production since a previous Pulse version and update explanations with the latest version of white-box explanations (WBE).

Note that you can still use the older version of WBE, but you will not be able to change the critical path.

Due to the complexity of this type of workflow element, you must first understand [models](#) in Pulse in order to learn how scoring works.

Plan service📄

The plan service reflects, in production, all the work done at a data science stage.

A unique feature to Pulse is precisely enabling you to deploy feature [plans](#) into runtime without effort. This tight integration of [data science](#) and [engineering](#) represents a huge boost in performance and productivity.

Thus, the plan service takes a set of data transformations, for example, to generate features out of raw data, that have been thoroughly tested as part of the data science stage in your project and marked as fit to be used on live data.

Custom services🔗🔗

Custom workflow services are tailor-made programs that allow you to extend or modify Pulse's behavior to fit specific needs.

Their tasks can vary greatly in intent, but one common usage is to configure them as an [input adapter](#). These services are an essential tool to feed data to a workflow. They can be used to build tailor-made connectors that integrate your custom data sources (for example: CSV files, databases, sockets, etc.) with the workflow inside Feedzai Pulse.

Another usage is, generically, when you might also want to modify the events as they make their way through the workflow to match some specific condition.

Designing a workflow🔗🔗

Workflows in Pulse offer a lot of flexibility in how you can combine elements and design a complex event processing pipeline in runtime.

However, most of the concepts you deploy to runtime in a workflow are originated during an offline data science stage. Thus, Pulse has to ensure the [consistency of these concepts in a Pulse project](#), for example, [deployable plans](#), and guarantee that they produce the same results both during the [data science loop](#) and during runtime.



Parallel forks in workflows

Pulse does not support parallel forks in your workflows. Since Pulse is unable to enforce the consistency at design-time, it will always force a real-time event to follow a single linear path from input until the workflow terminates.

If you configure branches as mutually exclusive, you are explicitly guaranteeing that an event can only go through one path. If the branches are parallel, then Pulse will automatically route the event through the first branch that evaluates as *true* and log a warning.

When joining branches into a single path at a workflow element, you must make sure that all incoming paths carry events with the exact same schema.

For more details on workflow design patterns and recommendations on how you can process events in runtime, check the [User Guide](#).

Example

Let's look at a simple use case and see what can happen to events while they're flowing through a workflow. The following diagram shows how different transactions are processed by the workflow:

Design

Events have only two fields:

- amount
- country

The workflow is composed of the following elements:

- One [input adapter](#). It connects to both rules services by forking the events flow through a filter: local and non-local transactions.
- Two [rules services](#), for both local and foreign transactions.
- One [scoring service](#).

Consider that for each rules service you are using rules based on the amount field.

- For the local transactions, the rules service executes a rule where the amount needs to be greater than 200
- For foreign transactions, the second rules service executes a rule where the amount is greater than 1000.

For both cases, the rules are configured to stop the workflow.

Execution

In this scenario, the events flow as follows:

1. The input adapter consumes the events.
2. The workflow forks the events whether they are local or not.
3. The corresponding rules service runs the rule block for each type of events.
4. Due to the **Stop Workflow** configuration, events that satisfy a rule condition will not be passed to other workflow elements.
5. Only two events reach the end of the workflow.

Workflow lifecycle

A workflow runs as a long-lived loop of reading and submitting events while managing the lifecycle of each workflow service as well. The workflow lifecycle is composed of three main moments as depicted in the following diagram:

- **onCreate**: While you add elements to the workflow, configure and connect them, Pulse instantiates all services and performs internal actions such as validating the input and output schemas. This stage is executed at workflow design time. While the workflow execution is not started, all services should be ready to receive recovery events.
- **onStart**: When you publish your Pulse application, the actual runtime begins, launching the workflow and starting to consume events through the input adapter. Additionally, Pulse assumes that all workflow and services configuration is valid. From this stage on, the workflow can only receive normal or replica events. For each service, depending on its type, Pulse executes specific logic to process events:
 - **Input adapter**: Run a single specific processing thread for the input adapter that consumes and submits events continuously while the workflow is running.
 - **Internal services**: Deliver messages to each service as a normal event traverses the workflow. After a service finishes processing the event it will reply indicating what the workflow should do with the event. For example, moving the event forward in the workflow and delivering a new message to the next workflow service, or dropping the event from the workflow and ending it.
- **onStop**: When the Pulse application stops, the workflow also stops receiving and processing events.



Destroying a workflow

The full lifecycle of a runtime workflow does not work in a cycle. This means that when you stop an application and its runtime processing, Pulse will not reset the state of the workflow to the previous *onCreate* stage.

In reality, when you stop an application, Pulse tears down the current internal representation of the workflow and creates a completely new one.

The workflow teardown includes the following tasks, in this order:

1. Stop the partitioning router from receiving messages

2. Stop the workflow synchronizer from receiving messages
3. Stop the workflow services and deregister jobs
4. Destroy workflow services
5. Close the partitioning router
6. Close the workflow connectors
7. Close the workflow synchronizer
8. Stop the workflow job manager
9. Close the executors
10. Close the connection to the event storage
11. Stop collecting performance metrics about the workflow

Workflow time🔗🔗

Many workflow elements are tied to the notion of time, for example:

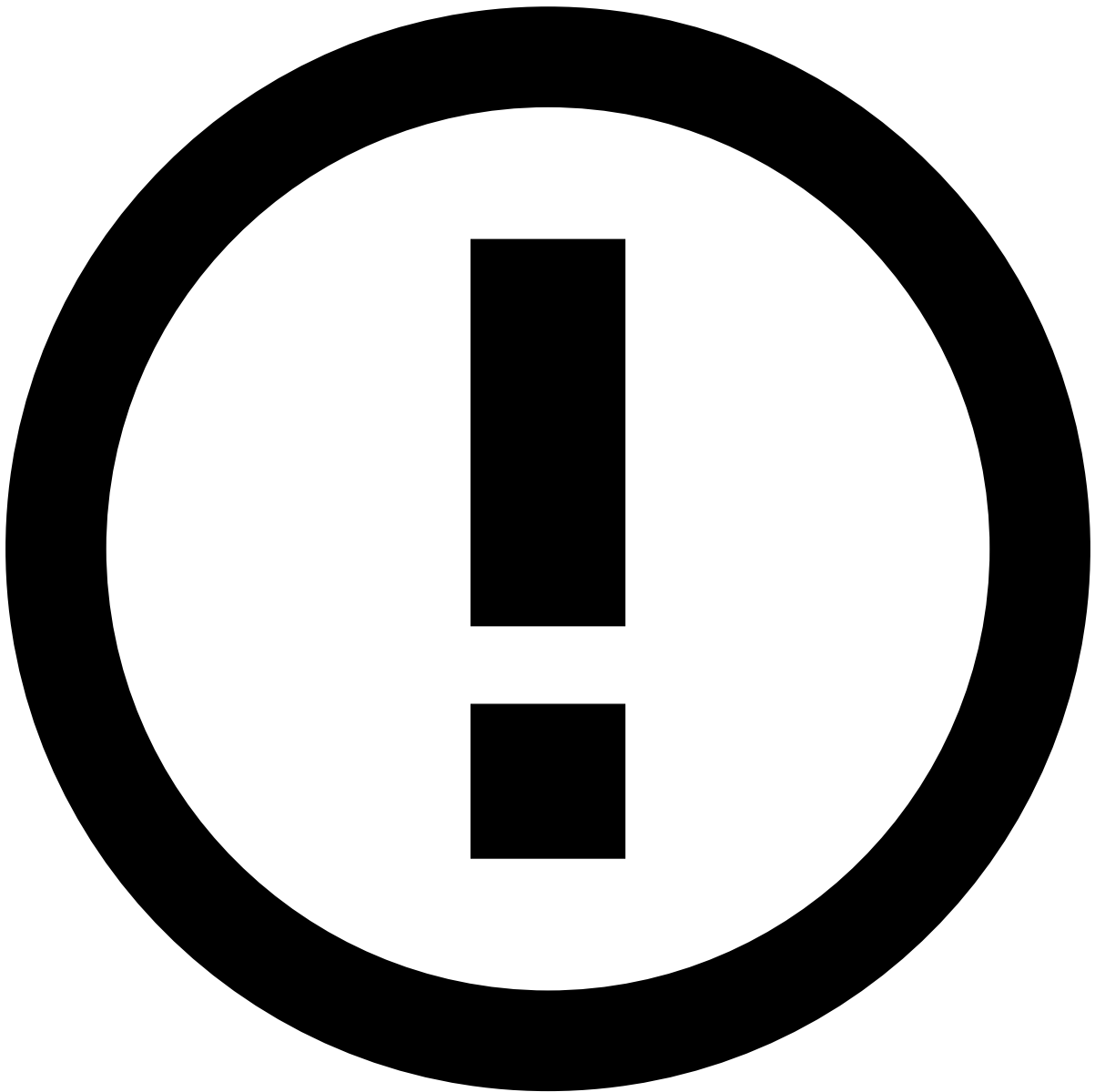
- Enabling a list item at a given time.
- Triggering a [Scheduled Metrics Job](#) at a given time.

The way Pulse workflows deal with time might not be intuitive at first, so it's crucial to understand the concepts of workflow clock and internal timezone. See [Time, schedules and timezones](#) for more information.

Message processing🔗🔗

Workflow recovery🔗🔗

The recovery procedures for a workflow consists of replaying events which have already been processed and are present in the [event storage](#). This is of the utmost importance since the underlying application may crash or fail for some reason, and upon its restart, all of its state is gone. For instance, you may have rules depending on windows, which will be empty and, thus, may be incorrectly evaluated.



Event Storage

Note that these events are obtained from the [event storage](#), so the event storage must be enabled in order for the recovery to work.

Furthermore, in a high availability environment you may have more than one replica of a given application, and if one fails Pulse must restore its state in order to have a consistent event evaluation.

An application may have several workflows, and when starting/restarting the application all the workflows that have the recovery option enabled will be recovered synchronously. This means that Pulse will publish the events to the respective workflows on a timestamp basis so that scheduled jobs will run correctly.



Specific considerations

There are some caveats to the recovery procedures:

- The event storage option must be enabled, otherwise, there is no event source;
- If a workflow event storage database table is empty or simply does not exist, that workflow will be ignored;
- In order for the recovery procedures to publish to the workflow, the workflow must contain an instance of a `WorkflowInputService`;
- The Fraud Models are not updated in recovery;
- The recovered events will not be stored in the event storage again.

Check how you can configure recovery for your workflows in the **Workflow recovery configuration** section of [this topic](#).

Checkpoint-based workflow recovery

In a production environment, Pulse can be configured to store the necessary events for real-time metrics calculation on disk. In this case, Pulse creates **checkpoints**, i. e. regular snapshots about the events and the state of the metrics. After a server failure, Pulse can reload the last checkpoint and recover the metrics status starting from there. This saves time

when the server needs to be restarted. However, it takes toll on the metrics calculation speed, as Pulse puts metrics calculation on hold while creating a checkpoint. Hence, this setting needs to be carefully tested.



Limitations

Note that this feature is in continuous development, and as of now it does not provide full feature parity to the previous, RAM-based version.

Additionally, if this feature is enabled, do not edit or remove existing metrics. Edited metrics are not recoverable after a server failure, meaning that those metrics will only take new events into consideration after any server restart.

Workflow synchronization

When in a distributed environment with more than one replica of an application, Pulse uses the underlying messaging platform to synchronize every message that reaches the workflow's [dead letter](#) to the other replicas.

Since it only sends the message in the dead letter, it has already been processed by the necessary workflow services, thus diminishing the work needed to be done in the other replicas, for instance not having to perform any scoring at a scoring service.

This behavior is similar to the workflow recovery procedures.

Message processing summary

	Normal	Workflow Recovery	Workflow Synchronization
Actions	Yes		
Dead Letter	Yes	Yes	Yes
Explanations	Yes		
Event Storage	Yes		
Profile Update Jobs	Yes (only if leader)		Yes (only if leader)
Rule Evaluation	Yes	Yes	Yes
Scoring	Yes		

Interaction between workflows

In order to enable [Channels](#), you might want to share certain information between workflows. For example, in the workflow processing ATM withdrawals, you might want to consult the number of recent devices used by the same customer in internet banking (assuming ATM and internet banking events are processed in different workflows). This can be achieved with [Omnichannel Plans](#). See [Plans in workflows - Omnichannel metrics](#) for more information on how these Plans work in the Workflow.

Learn more

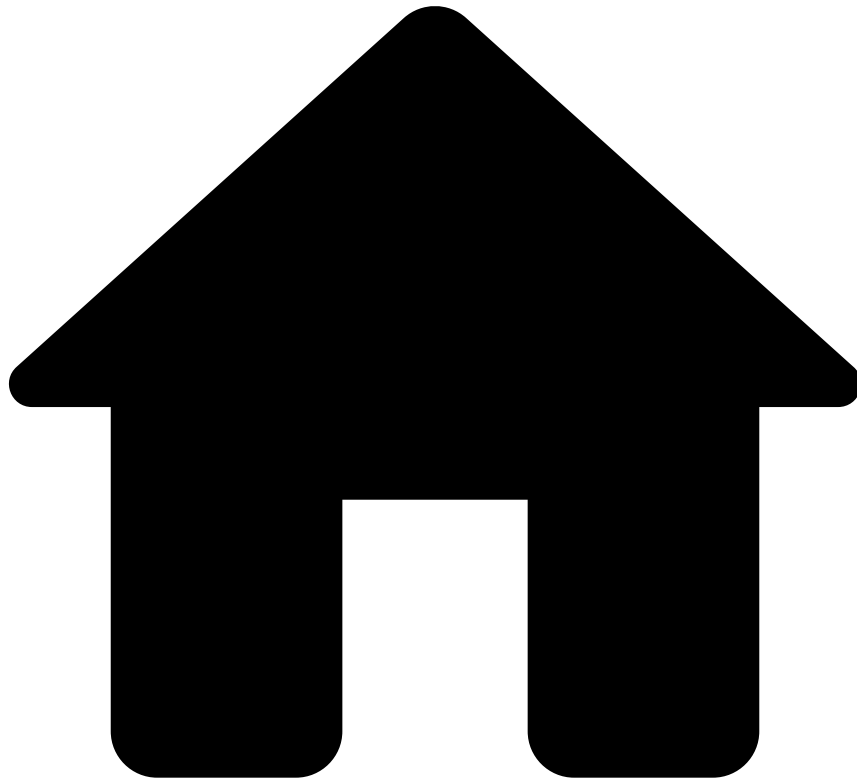
Check the following pages in the [User's Guide](#) to learn how to put workflows into practice:

- [Configuring outcomes](#)
- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding rules](#)
- [Adding custom services](#)
- [Advanced workflow configurations](#)
- [Workflow services properties](#)

Additionally, check:

- The **User's Guide** for more information on [how Pulse projects work](#) and how Data Science and runtime are integrated.

-



- Building a Model

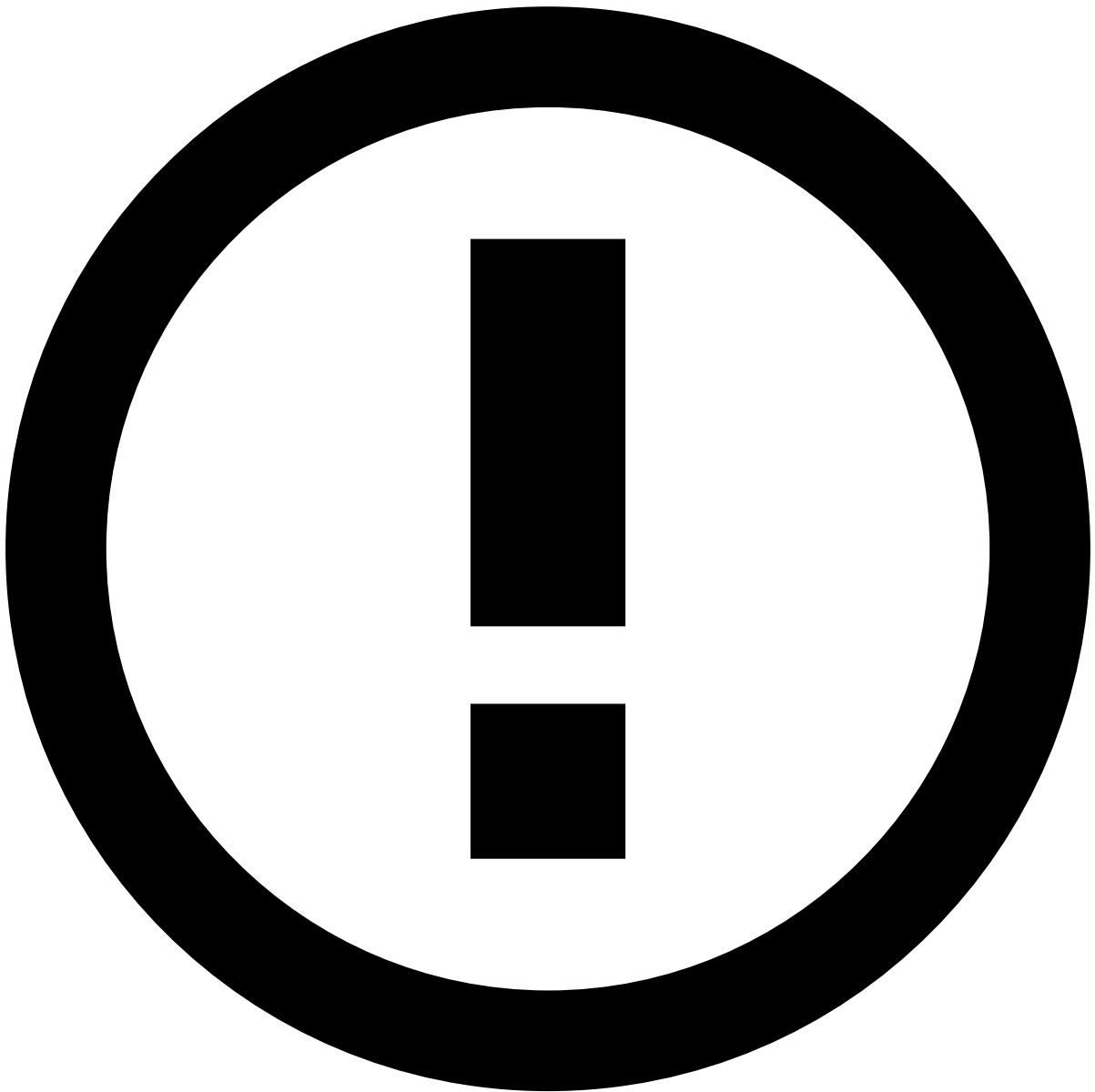
On this page

Building a Model

Was this page helpful? 

In this tutorial step, you will train and evaluate two models. They will be trained with the same parameters, and each will use a different training set:

- The original dataset
- The features dataset



info

In a real application, you would have to iterate further and try a greater number of models, varying not only the training data but also the model's parameters.

Training a model📖

To train a model in Pulse, execute the following actions:

1. In the **Data Science** tab, add a **Model Project** named *Models*.
2. Add a new model. On the New Model screen select **Train new model**. Name the model *RawData* and set the data source and the features as follows*:
 1. Use *RawTransactions* as the training **Data Source**.



info

At a later stage, you will be using the enhanced data set, with the features you generated with the Features plan, to train a second model and compare it with this one.

2. Use *Feedzai - Random Forest* as Algorithm.
3. Set *Fraud* as the **Target Variable**. This is the field you are trying to predict on this project.
4. Look for the *FraudStatusDate* field in the **Fields to Select** table and remove it. This field indicates when a transaction has been marked as fraudulent. Like *Fraud*, it would not be present in an input dataset. If you are predicting if fraud occurred, then you surely don't know when fraud has been detected. So, if you consider this field when training your model, results are completely useless.

You should remove any fields that you have found that don't contribute to a decision. In this case, any fields that don't help discriminate between fraud and not fraud. Also, note that the target variable "Fraud" was automatically removed. It wouldn't make sense to try to predict a field that is served as input to the model.

5. Keep all other fields in the data source as input to train your model.
6. Change the **Sampling Type** to *Under Sampling* with 10%. You should do this since the proportion of interesting transactions, in this case, fraudulent ones, is very reduced. You can check the documentation to learn more about [sampling strategies at model training](#).

7. Use the default configurations for the **Provider** and Machine Learning **Algorithm**. Pulse uses the Feedzai Random Forest (FZRF) algorithm by default.
 8. Set the **Validation** to *Split Percentage* with 80%. This means that only 80% of the dataset will be used for training the model. The remaining 20% are used to then test the performance of the model on unknown transactions and avoid [overfitting](#).
 9. Add an **Additional Cost Field** for *TransactionAmount*. This will allow you to measure the performance of the model in terms of cost, in this case, generating metrics such as *money FPR* and *money Recall*.
 10. Configure your model to calculate **breakdowns** over all possible values for the *CardholderPresentCode* field.
 11. Finally, **train** your model.
3. Repeat the previous steps and create a second model with the same parameters as the previous one. Use *TransactionsFeatures* as the input **Data Source** and name the model *FeaturesData*.

Note that Pulse automatically generated a model evaluation for each of the trained models. This happens when you specify a **Validation Method** when you train your model.

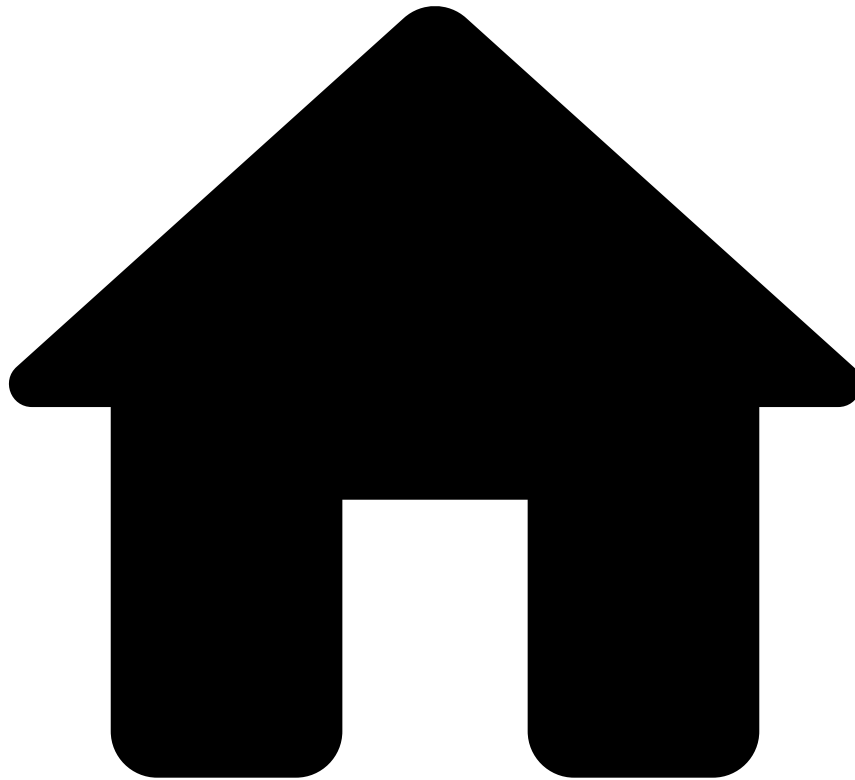
Evaluating and comparing your models

To evaluate and compare your two models in Pulse, execute the following actions:

1. Select the *rawdata-trainEvaluation* model evaluation and examine the information on the page.
 - **ROC and X/Y curves:** Plot the relationship between two metrics on a bidimensional chart. The ROC curve plots the Recall against the False Positive Rate.
 - **Recall:** Represents the percentage of *fraud* instances that the model correctly identified, meaning that a recall of 50% tells you that if there are 10 fraudulent transactions, then the model identified 5 of them.
 - **False Positive Rate (FPR):** Represents the percentage of cases that are *not fraud* that the model wrongly identified as *fraud*.
 - **Threshold:** Feedzai score that represents fraudulent transactions. Try moving it to see how it affects the Recall, the FPR, and any other metrics. Basically, what you are doing by using the slider is traveling along the [ROC Curve](#): the greater FPR you are willing to tolerate, the greater Recall you can achieve.
2. Assess which of the two models is more suitable given your project's requirements. Set the **Compare With** field to *featuresdata-trainEvaluation* so you can analyze metrics and performance for both models side by side.

Note that the *featuresdata-trainEvaluation* is closer to the top-left corner of the chart (the ideal point), which is a strong indication that the Model usually performs better. For the same FPR (ideally low) it has higher Recall (as high as possible, since it identified a greater portion of *fraud* cases). There are other metrics besides **Recall** and FPR and you can even [define your own metrics](#).

-



- Building Rules

On this page

Building Rules

Was this page helpful? 

[Models](#) are one way to contribute towards the score of a given decision, *fraud/not fraud* in this case, while the other are rules.

Rules add an extra layer of security on top of machine learning models by letting you define conditions that, when met, help you classify your events.

This tutorial stage will help you define and evaluate simple rules. However, rules in Pulse are a very robust and complex feature that allows you to do much more. Don't forget to check the [Rules](#) page for further details.

Defining rules

To create rules in Pulse, execute the following actions:

1. Create a [list](#) for suspicious merchants as follows:
 1. Name the list as *MerchantsAlert*. Leave all other fields with the default values.
 2. **Save and add items** with the following values: *Royal Fitness*, *WALGREENS* and *value_5*.
2. In the **Data Science** tab, under the Rules Projects section, add a new rules project, and configure it as follows:
 1. Name it *Rules*.
 2. Copy the schema from *TransactionsFeatures*.
 3. Set the **Target** to the *Fraud* field.
 4. Add an **Action** named *MerchantsAlert*.
3. Add a new rule named *MerchantsAlert* to the rules project and define its behavior as follows:
 1. In the **Variables** field, set an *isSuspicious* variable that reflects if the *MerchantName* of the transaction is contained in the *MerchantsAlert* list.
 2. In the **Condition** field, set the rule to trigger if the merchant is suspicious and the *TransactionAmount* is greater than 50.
 3. In the **Score Contribution** field, set the contribution to the *Fraud* score as 200.
 4. In the **Actions** field, select the *MerchantsAlert* action. This means that the rule will trigger this action if the Condition is met.

This rule is not very smart and serves as only as a demonstration of how you can configure a rule. This rule's behaviour defines that whenever a transaction with an amount above 50 was performed at a suspicious merchant, then Pulse should trigger an action and add 200 to the event's score. However, the rule does not mark the transaction as *Fraud* since it this only a strong indication of fraud and not a certainty.
4. Add a new rule to the rules project. This rule is simpler than the first one, but it actually marks transactions as *Fraud* based on the observation that fraud is more prevalent in transactions with a higher amount. Configure it as follows:
 1. Name it *TransactionAmount*.
 2. In the **Condition** field, trigger the rule when the *TransactionAmount* is greater than 600.
 3. Trigger the **Action** named *Block*.
 4. Mark the transaction as *Fraud*.

Exporting and evaluating rules

Now that you have defined a block of rules, you must create a [snapshot](#) so that you can use them in a workflow. While you are configuring the snapshot, you can also evaluate the rules performance over a specific data source as follows:

1. Publish your application.
2. In the *Rules* rules project, add a new snapshot called *Snapshot*.
 1. Include all your rules in the snapshot.
 2. **Evaluate** it using the *TransactionsFeatures* data source.
3. Once the evaluation finishes, open the snapshot and notice that there are 3 kinds of evaluations:
 1. **Block Evaluation**: shows the evaluation regarding the target field (*Fraud*) after all rules are executed.

2. **Block Rule Evaluation:** similar to block evaluation, but instead of seeing all rules as a black box, you can drill down into the performance of each rule.
3. **Individual Rule Evaluation:** similar to block rule evaluation, however the rules are not evaluated in sequence, but rather in isolation. This means that the result of a rule does have an impact on the evaluation of the next one.

In this scenario, you won't see any difference between individual rule evaluation and block rule evaluation because the first rule being applied does not mark events as *fraud* or *not fraud*.

•



- Designing a Runtime Workflow

Designing a Runtime Workflow

Was this page helpful? 

Now that all is set up, combine the previously created resources in a [workflow](#). This workflow will be ready to be applied in the Production environment, to process events.

Before starting the following steps, get in touch with your contact person at Feedzai to make sure that you have an Input Adapter set up in the test environment where you are working. In the following steps, we will use an Input Adapter that is named *file-injector*. In addition, make sure that the sample data files mentioned at the beginning of this tutorial are downloaded to the Pulse server.

Then, go ahead and design your workflow as follows:

1. In the **Workflows** tab, add a new workflow named *Workflow*.
2. Set the **Origin Schema** to be the same as *RawTransactions*. In a real application, the schema would likely not contain the field you are trying to predict. In this case, it would have neither *Fraud* nor *FraudStatusDate*.

3. Represent the final decision of the workflow by adding an **Outcome** named *isFraud* with a **Default Value** of *false*.

4. Add an **Input Adapter** as follows:

1. Name it *Input Adapter*.
2. Under **Custom Service**, set it to use the *file-injector* custom service.
3. Select **Properties** to confirm that the *filepath* is properly set. You can use a CSV with the same format as the workflow input schema. If you downloaded the sample data as suggested at the beginning of the tutorial, you can use the input.csv file under the `PULSE_SERVER_HOME/var/datasources/` path.

5. Add a **Plan** service as follows:

1. Name it *Features*.
2. Configure it to run the *Features* plan with the plan execution you generated when you first ran it.
3. Connect it to the *Input Adapter*.

6. Add a **Rules** service as follows:

1. Name it *Rules*.
2. Set it to contribute to the *isFraud* outcome.
3. Configure it to use the *Rules* rules project and the *Rules Snapshot* snapshot that you created of the rules.
4. Connect it to the *Features* plan service.

7. Add a **Scoring** service as follows:

1. Name it *FZRF Scoring*.
2. Set it to contribute to the *isFraud* outcome.
3. Configure it to use the model project named *Models*, and the *FeaturesData* model.
4. Connect it to the *Rules*.

8. Finally, add a **Custom Workflow Element** to the end of the workflow that uses the *sink-logger* custom service.

Connect it to the *FZRF Scoring* service.

-



- Loading Data

Loading Data

Was this page helpful? 

The first step in your data science project is to load data into Pulse and begin your [data science loop](#).

Before starting the steps, ensure that the sample data file is downloaded to the `PULSE_SERVER_HOME/var/datasources/` path on the Pulse server where you are working. To get help with this, get in touch with your contact person at Feedzai.

To load a dataset into Pulse, execute the following actions:

1. In a browser, navigate to your data science environment. Log in by using the credentials provided, which by default are **pulse/pulse**.
2. Create a new application named *Fraud Training*.
3. In the **Data Science** tab, add a new [data source](#) named *RawTransactions*.

The URL depends on the operating system of your Pulse server and the path to the Pulse server home. For example: `file:///PULSE_SERVER_HOME/var/datasources/transactions.csv`.

4. Examine the dataset by scrolling through the different columns and understanding what the dataset contains.

1. Note that there are different ways this dataset represents missing values, such as using `-` in *AVSAddressNumber* or `\N` in *AuthSequenceNumber*.
2. Add both `-` and `\N` to the **Missing Values** list.
3. Proceed to the **Next** step.

5. After Pulse has inferred the data type for each field, in the **Data Source Characterization** step, check if all the fields have suitable [data types](#) and correct those that don't.

1. Change the type of some of the amount fields from *Categorical* to *Numeric*, such as *AdditionalAmount1* and several others, whenever you think it may make sense.

Be careful not to change the data type of true categoricals such as *CashAdvanceAmount* (does not contain numbers) or *AVSZipCode* (even though it has numbers, it can be argued there is not an order in zip codes).

2. Change the type of *ID* fields (such as *AccountID* or *RouterID*) to have a *String* type rather than *Numeric* or *Categorical*, since it was not correctly inferred.
3. Change the type of *RecurringBillingKey* and *InternetAuthorizationIndicator* to *Date Time*. Set the format of these fields to *Date Time*, with *dd/MM/yyyy* format.
4. Use the button to change the data type configuration of the *AdditionalAmount1* field that you set up earlier. It is currently an *Integer* field. Set it to be *Float*. This makes sense because *AdditionalAmount1* has decimal places and you don't want to lose information.

6. Scroll down to the bottom of the page and create your improved data source.

7. Select your data source from the **Data Sources** list.

8. Use the **Analyse** button to analyze the data source. Select *Fraud* as a breakdown field. This way you can analyze correlations between fields of the data set, in particular, the *Fraud* field, which is the domain goal in the scope of this application.

1. Use the **View Statistics** option to better understand fields in your dataset, including arithmetic statistics of the field and its value distribution. It is also possible to click over the histogram to access the statistics view.
2. View the statistics of the *TransactionAmount* field by using the button. Break the analysis down by *Fraud* to observe that the mean value of *TransactionAmount* is greater for fraudulent transactions.

-



- Processing Data

On this page

Processing Data

Was this page helpful? 

Now that you have loaded your data into Pulse, the next step is to transform your data and extract value from it. At this stage you will learn how to design [plans](#) so that you can design derived features or extract subsets of your original data set, before training a model.

Designing derived features

It is often useful to enrich your dataset with derived features that can be useful to distinguish between the classes you wish your model to predict (in this case, *fraud* and *not fraud*). Design a plan that calculates a feature as follows:

1. In the **Data Science** tab, add a new plan named *Features*.

2. Set the plan as **Deployable**. Generating derived features is useful both in data science and in production [workflows](#). Enabling this option allows you to take this work directly into the runtime environment without any extra effort.
3. In the **Schema** field, set the schema to be copied from the *RawTransactions* data source (confirm the orange warning sign below) and **Save** it.
4. Use a **Map** operator to calculate derived features. You can design a feature that represents the distance between the merchant and the customer's home location (since a greater distance has a higher correlation with fraud). Create a **Map** named *AddDistance* to add a new *distance* field to your data and configure it as follows:

Designing historical profiles

Another kind of features that is also often useful are the ones based on [historical behaviour](#). To calculate historical profiles (i.e. metrics), such as the daily count and sum of transactions, execute the following actions:

1. In the *Features* plan, use the button to select other options, and select the **Plan Settings** option.

Under **Timestamp Field**, select the field called *Timestamp*. This is required so that Pulse knows which field from the dataset represents when events occurred so that it can build time-based features.

2. Add a **Metrics** operator and connect it after the *AddDistance* operator.
3. Edit the metrics operator to name it *Metrics* and check the **Output Incomplete Instances** option.

Initial events will not have accurate metrics. For example, a metric with a window of 24 hours will only be correct after the first 24 hours of the data. Hence these events are called incomplete instances. By default, Pulse discards the initial events with incomplete metrics, to improve model training and evaluation. Select the **Output Incomplete Instances** option if you want to include them anyways.

4. Add a new metric, and define it as follows:

1. In the **Group By** section, select the *AccountID* field. This means that the metric will be calculated for each account.
2. In the **Window** section, leave the default *1 hour*. This means that you want to calculate a metric based on the events of the last hour.
3. In the **Aggregation** section, select the function *Count*. This means that the metric calculates the number of events in the respective window (last hour).
4. Use the **Save** button to add the metric to the operator.

5. Add another metric following the same steps, but this time the window is *1 day*. This metric calculates the number of events in the last day.

6. Add a new metric, and define it as follows:

1. In the **Group By** section, select the *AccountID* field.
2. In the **Window** section, leave the default *1 hour*.
3. In the **Aggregation** section, select the function *Sum*, and the field *TransactionAmount*. This means that the metric calculates the total amount of the events in the respective window (last hour).
4. Use the **Save** button to save the metrics.

7. Add another metric following the same steps, but this time the window is *1 day*. This metric calculates the total amount of the events in the last day.

8. Use the button next to the metrics' names to change them to *HourlyTrCount*, *DailyTrCount*, *HourlyTrSum*, and *DailyTrSum*, respectively.

9. Use the **Save** button to save the metrics operator. Your feature plan will look like the following image:

10. Use the button to run the plan and create a new and enhanced data source including your features. Configure the execution as follows:

1. In the **Data Source** section, select *RawTransactions* as the input data source.

2. In the **Data Sinks** tab, name the data sink as `TransactionsFeatures`, and leave the file type as the default (Parquet). In the **Directory Path** field, specify the output file URL. Set the path to the Pulse server home directory, ending with `/var/datasources/<filename>`.

Example: `file:///PULSE_SERVER_HOME/var/datasources/transactions-features.parquet`

The file path may vary according to the operating system of the Pulse server. You can not have a file with the same name in the `datasources` folder.

3. Use the **Run** button to execute the plan.

11. **Save** the plan and then press the **refresh** button on browser.

You can see in the **Data Science** tab that you now have a new data source. You can analyze it as you have done before with *RawTransactions*, and assess the usefulness of the features you have designed.

Sampling your data

It is often useful, in the context of Data Science, to reduce the size of the dataset while generally maintaining the distributions of the random variables representing each feature. This is done via [stratified sampling](#).

Sampling improves the performance of your Data Science Loop since you will be able to iterate much faster on feature calculation and model training.

Create a stratified sample to reduce the number of *Accounts* where there was no *Fraud* to be 10% of its original size, as follows:

1. Add a new plan named *Stratify*.

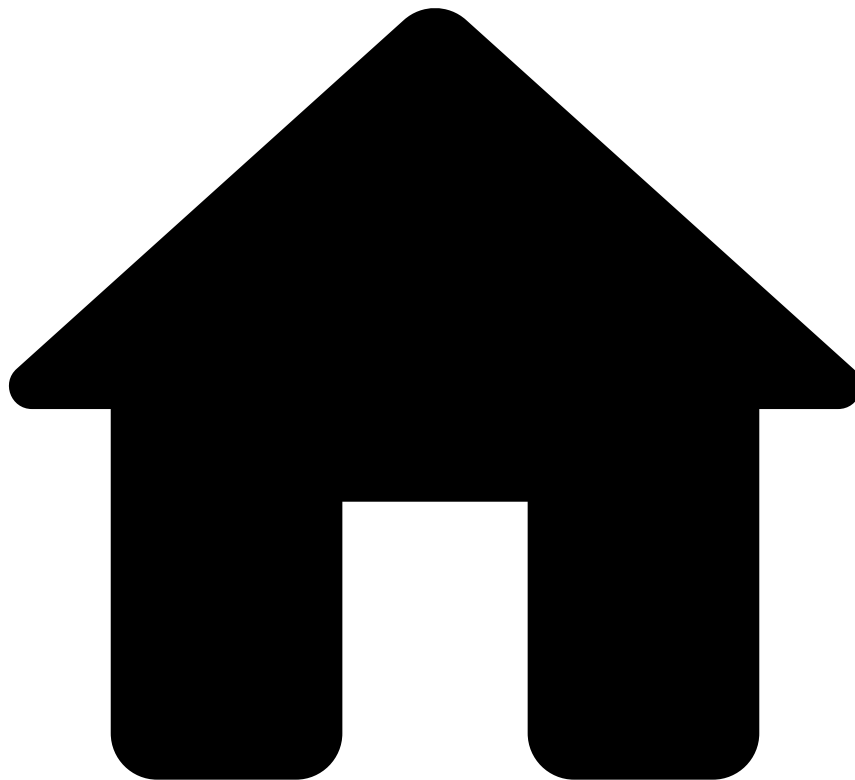
Make sure that the **Deployable** checkbox is **not** selected. This means that this plan will be non-deployable. Non-deployable plans are useful for sampling data, which typically happens only in the data science environment, hence the plan won't need to be deployed to production.

2. Add a new **Source** operator named *Transactions* that copies the *RawTransactions* schema.
3. Calculate the number of *Fraud* transactions per *Account*. First, add a **Map** operator, and connect it to the `source**, **` with the following configuration:
4. Sum up the number of *Fraud Transactions* by adding a **Group By** operator like the one below. This way, you will have the total of *Fraud* transactions per account stored in a field named *nrFrauds*.
5. Split the data set into two: accounts where there was fraud and accounts where there wasn't. Add a **Filter** operator as follows:
6. Repeat the previous step, but this time with a *Fraudulent* filter (use *nrFrauds > 0* as the expression).
7. Apply a **Sample** operator to the *Legitimate* transactions to get only 10% of them.
8. Join both sets from *Fraudulent* and the reduced *Legitimate* with a **Union** operator.
9. Use a **Join** operator to join the result with the original *Transactions*, so that you end up with the original data set minus 90% of the *Transactions* belonging to an *Account* where there was no *Fraud*.
10. Finally, add a **Sink** operator.

You can run this plan as you have done with the *Features* plan to generate a new data source (do not forget to select the *RawTransactions* as your base data source). You can then verify, by evaluating it and performing an exploratory data analysis, if this sample really is representative of the original data set. For most data sets, this is true. You will not be using this sample (or any other) during this tutorial since the original data set is fairly small already. This allows you to operate on it without resorting to sampling.

However, it is encouraged that you to use this technique on real projects with substantial data sets.

-



- Best Practices
- AutoML

AutoML Best Practices

Was this page helpful? 

Throughout the pages of this chapter, you will find some good practices for each stage of the AutoML process:

1. Feature engineering
 1. Data source split
2. Feature selection
3. Training and evaluation
 1. Hyper-parameter tuning
 2. Model selection

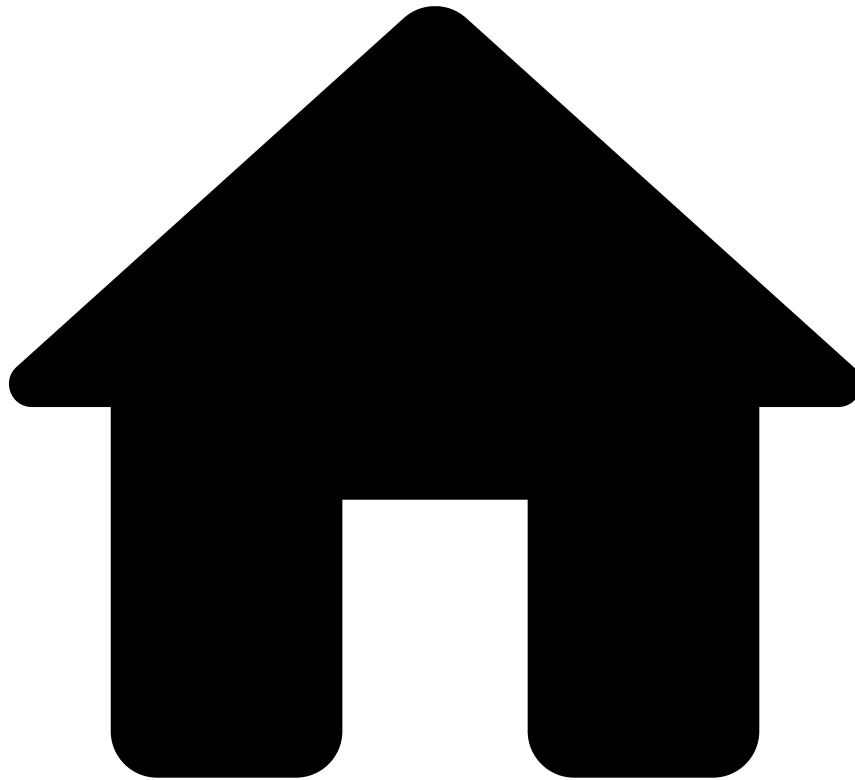
As a general rule, a full AutoML usage (going through all three stages in one go) is preferable when you want to build your first model. In this case, AutoML serves as a kickstart to your data science loop.

When looking at AutoML as a whole, always try to run it first with a very aggressive sample of your data. The model results won't be useful for any deployment, but you will be able to "benchmark" how much time the tasks take. More

importantly, you will be able to understand if your job profiles are configured correctly for this task: if using a random sample of 1% of your data only works with 1 GB of memory per executor to run, then the full data is expected to take at most 100 GB to run safely.

From the user perspective, most of the effort should be put into creating a good semantic file. Analyze your data, understand data skewness and relationships between variables. With that in mind you can avoid downstream issues in AutoML.

-



- Best Practices
- Metrics

On this page

Metrics Best Practices

Was this page helpful? 

[Metrics](#) can be seen as the "basic unit" of Data Science work. Use cases like fraud prevention and anti-money laundering revolve around the concept of **change in behavior**. This is where metrics shine, by providing a straightforward way of aggregating data points through time efficiently. Metrics are also commonly referred to as **profiles** or **historical profiles**.

This chapter will cover topics such as the differences between Data Science Platform and Runtime, and a few optimizations that are helpful to bear in mind. In addition to the [general best practices](#) section below, the chapter includes subpages covering different aspects of best practices for working with metrics.

When thinking of a typology for [features](#), we can conceive the following categories:

- **Atomic**: features that rely only on information present in their respective instance. Example: "Keyboard distance of the email address".

- **Composite:** Features that combine two or more previously computed values. Example: "Ratio between transaction amount and account balance".
- **Lookup:** Features that depend on a lookup table. This table can have the following behaviors:
 - **[Near-]static:** Never or rarely updated. Example: mapping country codes to coordinates, MCC to MCG.
 - **Dynamic:** Updated in batches at regular but sporadic intervals. Example: risky MCC, risky countries, etc.
- **Metrics:** Temporal aggregations. These features require the definition of a time window, an aggregation field, and how the metric should update. Metrics can optionally include filters. Example: average money transferred by this customer in the last 24 hours.

Generally, you should only create features through the metrics operator if they fall into the "Metrics" category explained above.



info

Metrics are the most complex computations that the system will do both offline and in runtime. Expressions in metrics are written in Pulse Query Language (PQL). For details on semantics, symbols, and operators see the [official documentation](#).

As a rule of thumb, when defining the **Window update period**, it is recommended that short windows (1 day or less) be set to **real time**, i.e. events enter the window as they arrive; whereas longer windows (1 week, 1 month, ...) should be **scheduled**, i.e. events enter the window in batches. This aspect of how a window is updated is one of the major constraints when designing metrics. It will be explained more in depth in the [Constraints](#) page.

General Good Practices

The following are some general good practices to consider when designing your metrics.

Do

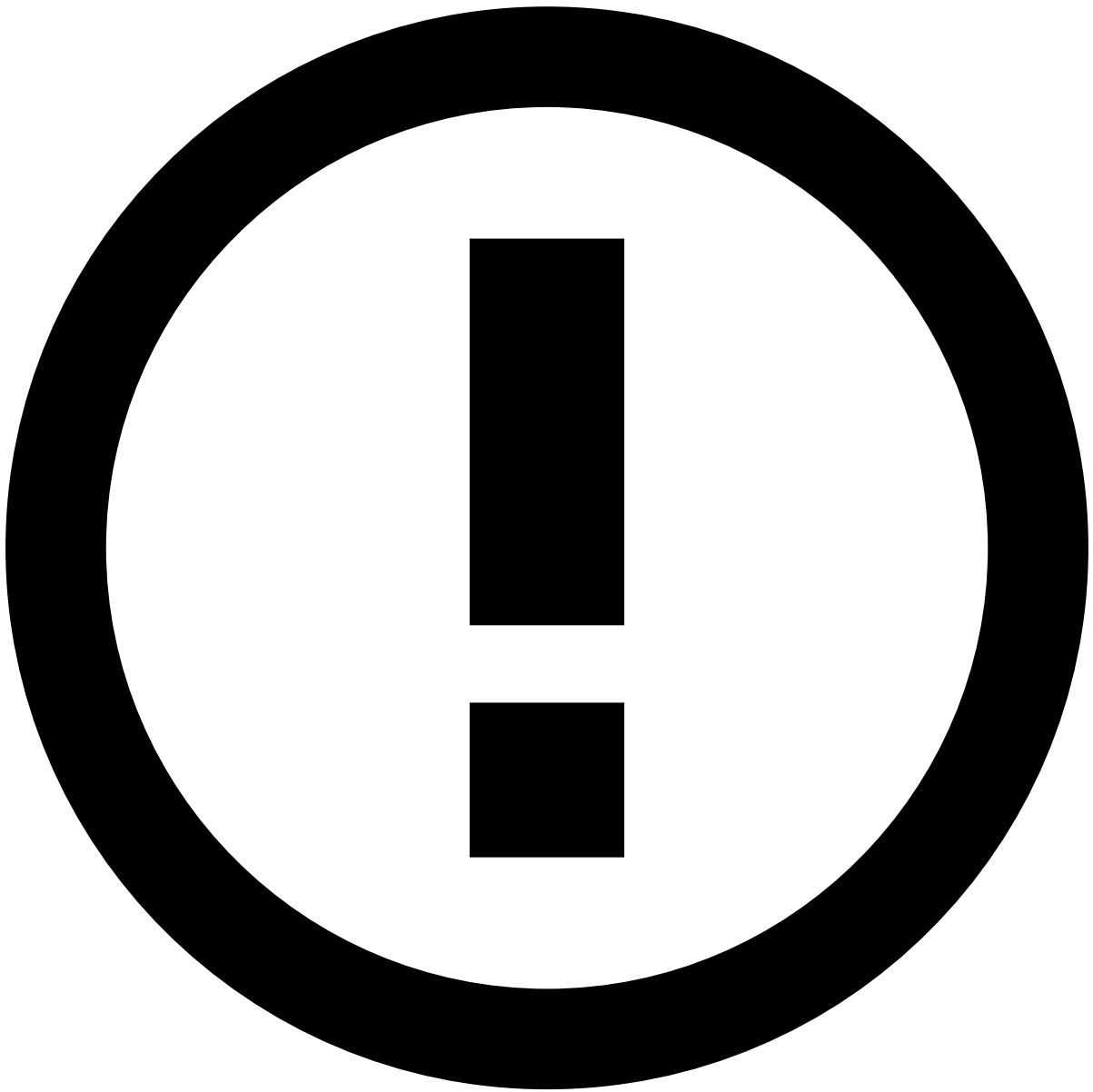
- Metrics in the same Metrics operator cannot depend on each other.
- Best: 1 big metrics box with all the metrics in it (SMJs or not).
- Second best: 1 box with SMJs and another with real-time metrics.
- Make sure to **enable *Output incomplete instances*** in the metrics operators when running jobs in the Data Science Platform. Otherwise, the plan output will be missing data corresponding to the largest profile window (for example, if the dataset goes from Jan to Dec and the largest window is 3 months, the output dataset will start in April). This is especially important when there are metrics with very large windows (which could result in an empty output dataset) or multiple metrics operators (the effect will compound, and overall the sum of the largest window of each operator will be cut from the output).

Don'ts

- Avoid dependencies between metrics.
- Avoid cascading [Scheduled Metrics Jobs \(SMJs\)](#).
- **Avoid redundant metrics.** While you may be tempted to compute the count of transactions for each customer over lots of different time windows (5 minutes, 15 minutes, 30 minutes, 1 hour, 3 hours, ...), it might cause a degradation in runtime performance (latency and memory) and sometimes even in DS metrics. It's a good idea to check for overly correlated features at least once during the modeling process, either directly by computing the pairwise correlations between features, or indirectly by trying different combinations of features and comparing the results of Feature Importance. For very large data sets you can compute the cross correlations in a sample of your data. Another source of excessive correlation is profiles with correlated group by fields: for example, card number and customer ID likely overlap a lot, while card number and merchant ID less so.

Advanced techniques

- **Salting grouping entities with lots of missing values.** Suppose you want to compute profiles grouped by the field `ip_address`. However, in your dataset about half of the transactions don't have an IP address. You start by adding a filter to the metric window, something like `ip_address != null`; however this is not enough: when computing the profile, all the transactions will still be loaded and distributed to the executors before the filter is applied. Thus, there will be a very large partition with all the missing IPs, which will likely crash the Spark job. One way to circumvent that is to substitute that missing value by something else, such as the transaction ID. For that, create a new field `ip_address_salted` in a map operator before the metrics operator. Assign it a value similar to `ip_address ?? transaction_id`. Then, update the metric window filter to `ip_address_salted != transaction_id`. This will still yield the same end result, but without generating a massive partition in the Spark job.
- It's seldom necessary to use cascading metrics (a metric that depends on a different metric from a previous metrics operator). However, if it is a deal-breaker for the project (such as in compliance cases) it is important to notify the team responsible for runtime deployments, as there is an important property that needs to be set to ensure correct computation of cascading metrics.



Advanced notes

Metrics in plans within the Data Science Platform will run the Feedzai stream processing engine on top of Apache Spark. This combination of technologies allows the processing of metrics in Data Science to be horizontally scalable, while maintaining the same feature definition that goes into runtime.

Despite Spark helping on the parallelization of metrics computation, you should bear in mind "the 4 Ss": **Spill**, **Skew**, **Shuffle**, **Small Files** — which are the most common sources of issues when running Spark. These topics will be covered throughout this documentation, when pertinent.

•



- [Creating a Data Science project](#)
- Cleaning and loading your data

On this page

Cleaning and loading your data

Was this page helpful? 

This topic gives you an overview on how to load and clean the input data that you want to use in Pulse.

Overview

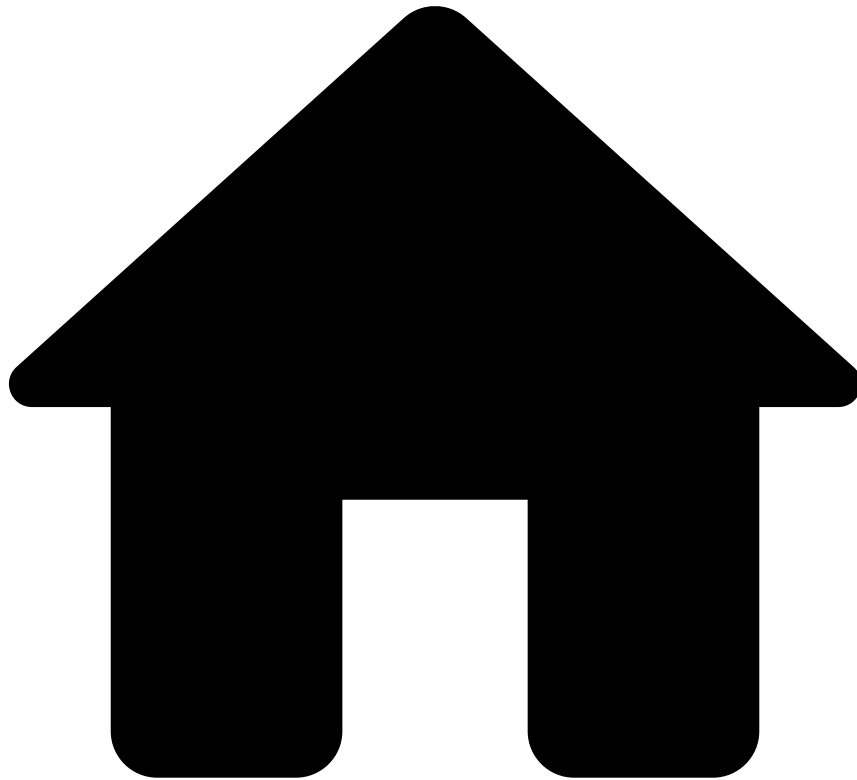
To load and clean your data, perform the following steps:

1. Import the data into a [Data Source](#) of Pulse. Learn [how to import data](#) into Pulse.
2. Use a [Plan](#) to validate if the data schema of the input data is in the format that your Pulse project uses. If not, use the Plan to transform the data to your project's needs. Learn how to [validate and transform the input data schema](#).
3. Clean the data. Transform values that make no sense in your application into meaningful values using a Plan. Learn [how to clean up the data](#).

Learn more 

- [Data Sources](#)
- [Data types](#)
- [Plans](#)

-



- [Creating a Data Science project](#)
- Creating Features and Models Automatically

On this page
Was this page helpful? 

Creating Features and Models Automatically

You can use [AutoML](#) to create features and train models in an automated way. In addition, AutoML evaluates the models and recommends the most performant one. This saves you time to focus on more complex tasks.

The following sections explain how to create an AutoML run to reach those goals.

How to create an AutoML run

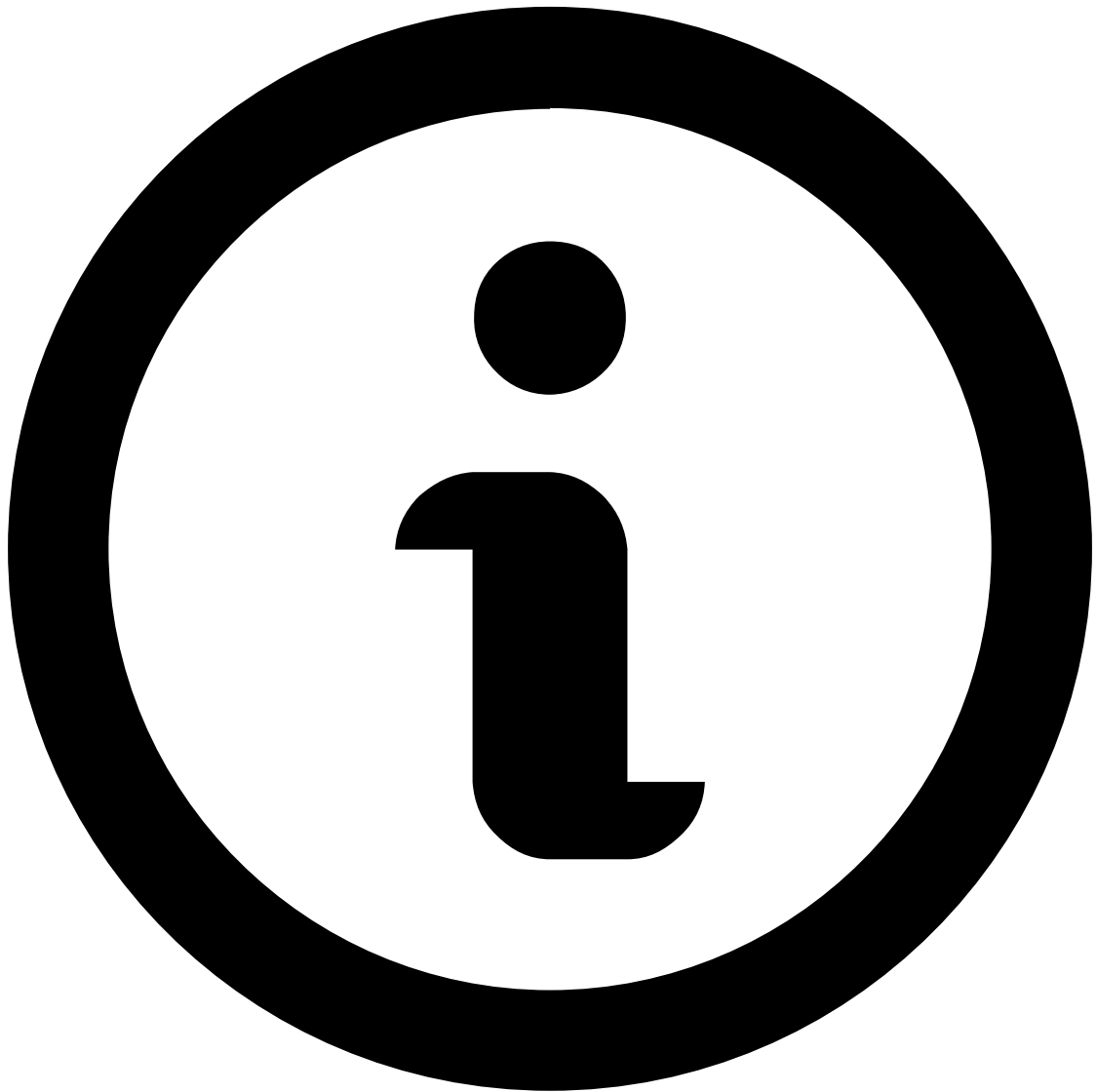
To create an AutoML run that finds features and train models, execute the following actions:

1. In the **Data Science** data science tab, expand the AutoML section if it is collapsed, using the AutoML button on the top of the page.

2. Select the add  or the run AutoML button to start a new AutoML run.
3. Configure the training properties for your new models:
 1. Select the **Data Source** you will use to create features and train models.
 2. Upload the **Semantic File**, which contains the AutoML tags for the data fields. The tags link the fields to their meaning. To create the semantic file for your data source, you can download a template by using the **Download template file** option in the Semantic File field. Then simply add the tags to the fields. Learn more about [how to create a semantic file for a data source](#).
 3. Select the **Timestamp Field** from the list of available fields. Pulse uses the timestamp field to aggregate the data by time windows for the calculation of metrics.
 4. Select a categorical data field for the **Target Variable**. This is the variable that the models will predict.
 5. Select the Target Value, which is the value of the Target Variable that you want to detect.

For example, in the fraud detection scenario the Target Variable would be the *Fraud* field, and the possible Target Values would be *True* and *False*. Given that you want to detect fraud, select *True* for the Target Value.
 6. Enable an **Additional Cost Field** to be available for the model evaluations. Select a field that represents the transaction amount, to have metrics such as the *Money Recall* and *Money FPR*.
 7. Optionally, you can filter which events are taken into consideration when AutoML processes them to train the model.
 1. If you define a **profile filter**, AutoML will create metrics that consider events filtered by the defined expression. Example: you might want to filter for only transaction submissions and payment updates `event_type = "transaction_submission" OR event_type = "transaction_payment_status_update"`
 2. If you define a **scorable filter**, AutoML will train the models based on events that satisfy the expression of this filter. Example: you might want to filter for only transaction submissions `event_type = "transaction_submission"`
4. Split the data source into smaller data sets for training, validation and evaluation. To do so, define the following:
 1. **Time range for training** - The training data set will be used for training models during the hyperparameter tuning phase of the AutoML run.
 2. **Time range for validation** - The validation data set will be used for evaluating the models during the hyperparameter tuning phase, to find the best model.
 3. Use the **Enable test dataset** switch to define the time range for the dataset that will be used for the final evaluation of the best model found. If you don't define a test data set, Pulse will use the validation data set for this purpose.

In the time ranges the beginning timestamp is inclusive, while the ending timestamp is exclusive.



Planning the training, validation and test data sets

To avoid under- or overfitting, ensure that the selected time ranges don't result in empty data sets, or in data sets with too few data. A typical split ratio is 50% for training, 30% for validation and 20% for evaluation, as long as the data source size is sufficient for that.

5. Select the **Metric to optimize**, and a fixed metric value that you want to achieve. For example, you want to optimize *Recall*, while ensuring that the *FPR* stays around 1%.
6. You can configure **Advanced options** as follows:
 1. Set the **Number of Models** you want to train in this AutoML run. Note that this value heavily influences the run time of the AutoML job. We recommend that you test the run time with a few models first.
 2. Select the **Path to generated data source**, if you want to save the data in a specific path.
 3. Select the **Window sizes** for the [metrics](#). If the window size of a metric is 1 hour, it means that the metric calculated for a certain point in time is based on the data of the last hour before that given time. Each defined window size will generate another metric.
 - Metrics with **Real-time Updates** will be updated at every event.

- Metrics with **Scheduled Updates** will be updated at the start of each day.

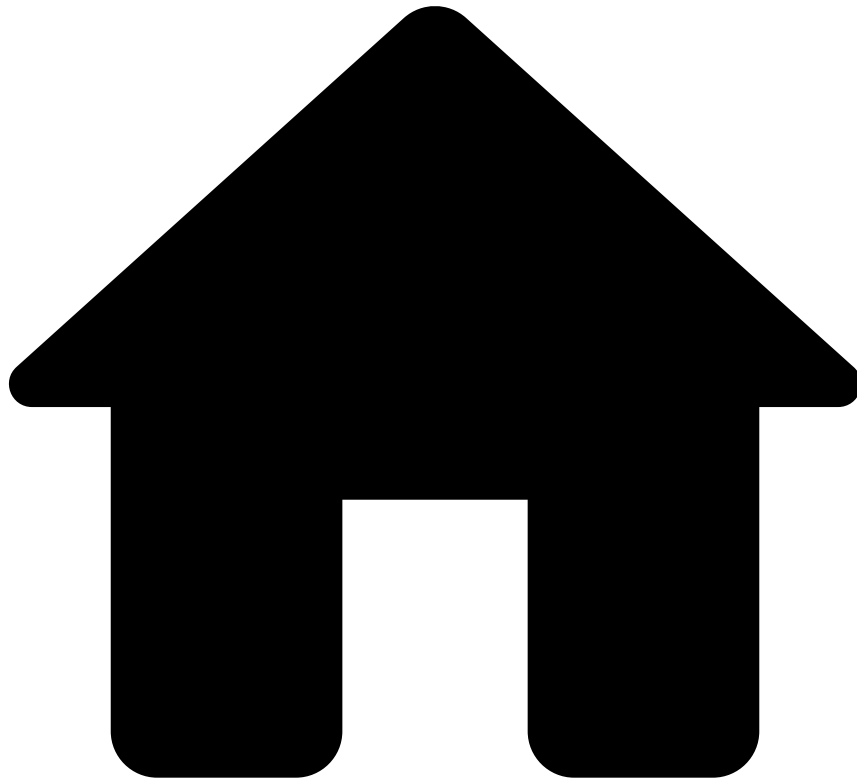
After setting up the AutoML process, you can follow up on its status on the top of the **Data Science** tab. When the AutoML job is finished, you can view the results in the same place:

- The **Best Model** trained in the last AutoML run.
- The **Features Plan** that created a data source with features based on Feedzai's experience in fraud detection.
- **Details** about the job that executed the last AutoML run. In case of a failed AutoML run, you can start investigating [here](#).

Next steps

- Dive into the created model project, and use the [Exploration Mode view](#) to understand the best model.
- Open the job details and go through the textual reporting that has details on the trained models.

-



- [Creating a Data Science project](#)
- Interacting with Pulse via JupyterLab

On this page

Was this page helpful? 

Interacting with Pulse via JupyterLab

JupyterLab is one of the most widespread tools used by data scientists to work with data. Pulse provides integration with JupyterLab, in the form of a service and an API that allows JupyterLab users to interact with Pulse data.

This topic contains information about how you can interact with Pulse using JupyterLab.

Prerequisites

The following are the prerequisites to interact with Pulse using JupyterLab:

- The JupyterLab Service of the Pulse server has to be enabled.
- The client computer has to be set up for the interaction.

Using JupyterLab to interact with Pulse

When in Pulse, select the button in the [toolbar](#) at the top of the page to use JupyterLab with Pulse. It opens a new tab in your browser with Jupyterlab, and shows a set of notebooks that demonstrate how to interact with Pulse.

The commands shown in the notebooks use the DS-API, an API provided by Pulse that allows you to explore the data and train models via JupyterLab, among other actions.

The possible actions are documented in the JupyterLab notebooks that are included with the JupyterLab integration on the client computer.

JupyterLab notebooks

The following list gives you an overview of the notebooks that are available with the JupyterLab integration:

- **Getting Started** notebook
First steps of using the DS-API: loading data from/to Pulse and converting data to a format that you can use in JupyterLab.
- **Datasources, Plans, and Models** notebooks
How to interact with Pulse's data sources, plans and models using the DS-API.
- **Data Loading** notebook
How to use the DS-API to programmatically load data into Pulse.
- **Data Exploration** notebook
How to perform efficient data exploration, for instance, during the pre-modelling stage of a data science project. It shows how to compute descriptive statistics, perform semantic validation of features, and generate a report of the dataset. It also introduces the interaction with the exploration API to get more details of each feature, including distributions and plots.
- **Data Sampling** notebook
How to make use of Feedzai's sampling library to apply appropriate sampling strategies on large datasets.
- **Join Data Labels Plan** notebook
How to create and run a data science plan that joins two data sources.
- **Bias Audit** notebook
This notebook allows you to run a bias audit on your ML model for a given dataset.
- **Rule Recommendation** notebook
This notebook suggests which rules can be removed or reprioritized without affecting the system performance negatively, or sometimes even improving it. It's useful for reducing complexity in large rule sets.
- **Indexers** notebook
This notebook describes how to train and fit an indexer for a string field, how to customize indexer clipping, and how to fit multiple indexers in bulk.
- **Feature Investigation** notebook
This notebook allows you to uncover changes in data distribution in your feature space. This can be useful to understand and debug unexpected losses of performance in your models and rule systems.

-



- [Creating a Data Science project](#)
- Processing your data

On this page

Processing your data

Was this page helpful? 

[Plans](#) are Pulse's way of helping you become much more efficient with some of the most time consuming and technically demanding Data Science tasks.

Plans are a vital concept in Pulse, and one that makes it a very powerful Data Science tool. They represent a way to boost the performance of:

- **Data scientists**
Delivering a set of tools that make it easier to process data for the training of Machine Learning models. Data Scientists can now focus on real data science instead of solving engineering and tooling problems.

- **Engineers**

Abstracting a lot of the usual complexity involved in deploying Data Science work into Production. With a fully integrated modeling and run-time environment, Pulse enables a seamless deployment, without any major effort.

You can design **Plans** to assist you in the following activities:

- Data cleaning, transformation, formatting, joining
- Schema validation
- Feature engineering

Typically, a data science project will have several plans that are executed in sequence and generate new data sources each as the output. These intermediate data sources are subsequently used as input for other plans.

Create/Edit plan page

The **Create/Edit plan page** lets you design a plan to transform your data from data sources. It enables you to visually compose a flow that takes your data input and outputs a new and transformed data source.

The following image displays the **Create/Edit plan page**:

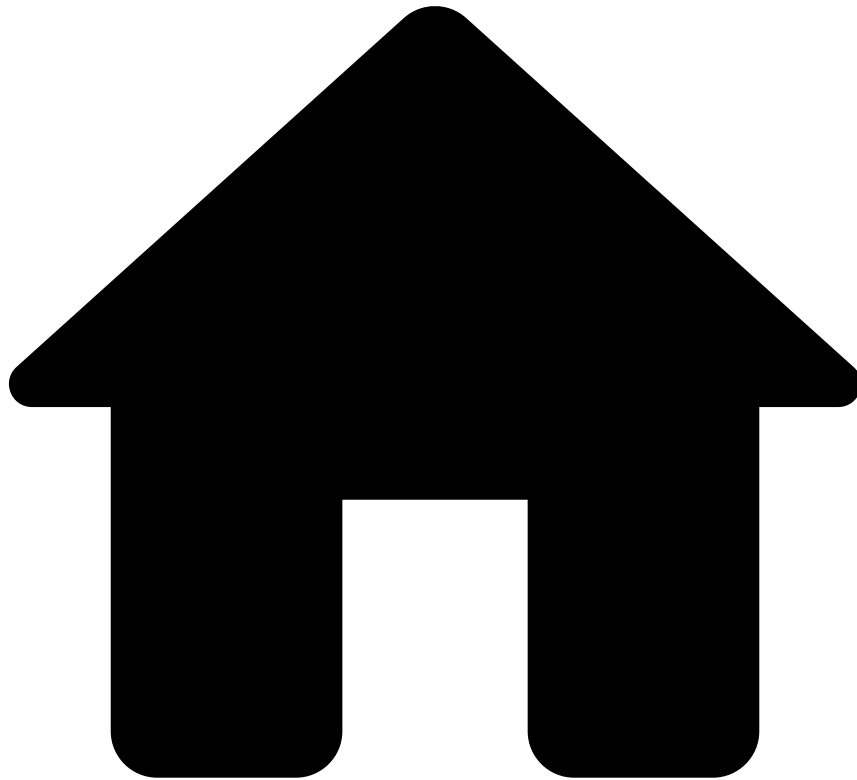
The **Create/Edit plan page** contains the following areas:

1. **Title and status**
Name and type of the plan. Indicates whether the plan is [Deployable or Non-Deployable](#).
2. **Actions**
Available operations over the plan. Use the option to transform your data using the plan, and generate a new data source.
3. **Operators**
Available elements to compose the data flow of your plan. If the plan is deployable, you cannot use some of the operators. Check the [Operators Reference](#) for detailed information on all available elements.
4. **Plan**
Area where you design the data flow of your plan. Drag operators and drop them into the canvas to create the different stages in your plan. Drag the handles from an operator to another operator to connect and define the data path between them. Use the **Edit Operator** option to configure the behavior of each stage of your plan.

Getting Started

To start processing and transforming your data to determine which fields you should use to train your models, learn how to perform [data analysis and sampling](#) tasks.

-



- [Creating a Data Science project](#)
- Testing your work in a Sandbox environment

On this page

Testing your work in a Sandbox environment

Was this page helpful? 

Pulse supports your Data Science loop by enabling you to quickly train your [Models](#) and [Rules](#) as a whole and evaluate their results.

With Sandbox Evaluation, you can simulate the work of your system and check its accuracy before it is deployed. This way, you are able to improve the performance of the metrics of your system, understanding these metrics more precisely than by analyzing Models and Rules separately.

Testing your work in a Sandbox Environment involves:

- [Creating a Sandbox Evaluation](#)
- [Evaluating your Sandbox Results](#)

Viewing Sandbox Evaluations

The View Sandbox Evaluation page allows you to analyze the results of your Sandbox Evaluation. This page contains the following areas:

Results by Score

1. Outcome value

Defines the target value and model comparisons for your evaluation. Select the Target Value you want to analyze in your Sandbox Evaluation. This allows for the evaluation of any value of your variables. For example, both Fraud and Not Fraud. Select the Sandbox Evaluation you want to compare it with to have their evaluations plotted on the ROC Curve area and displayed in the Metrics table.

2. ROC curve

Graphically displays the typical evaluation metrics of Machine Learning models, such as *Recall* and False-Positive Rate (*FPR*) . Adjust the X-Axis Range and Y-Axis Range sliders to zoom in and out of your model's results. Select the X-Axis and Y-Axis metrics to generate a custom chart in the X/Y Plot. Any count and cost-based metrics are available.

3. Metrics per breakdown table and per services tables

Displays the results plotted in the ROC Curve area in a tabular manner. Includes all metrics for all possible combinations of the configured breakdowns and services for the evaluation.

Adjust the Threshold slider to define the confidence you have in the model. Select the View ROC option in a row to display the graphical evaluation in the ROC Curve area for the selected breakdown. You can Choose Metrics to decide what data to analyze and Export Results to CSV.

Results by Outcome

1. Metrics per breakdown and per services tables

Displays the results plotted in the ROC Curve area in a tabular manner. Includes all metrics for all possible combinations of the configured breakdowns for the evaluation. Adjust the Threshold slider to define the confidence you have in the model. You can Choose Metrics to decide what data to analyze and Export Results to CSV.

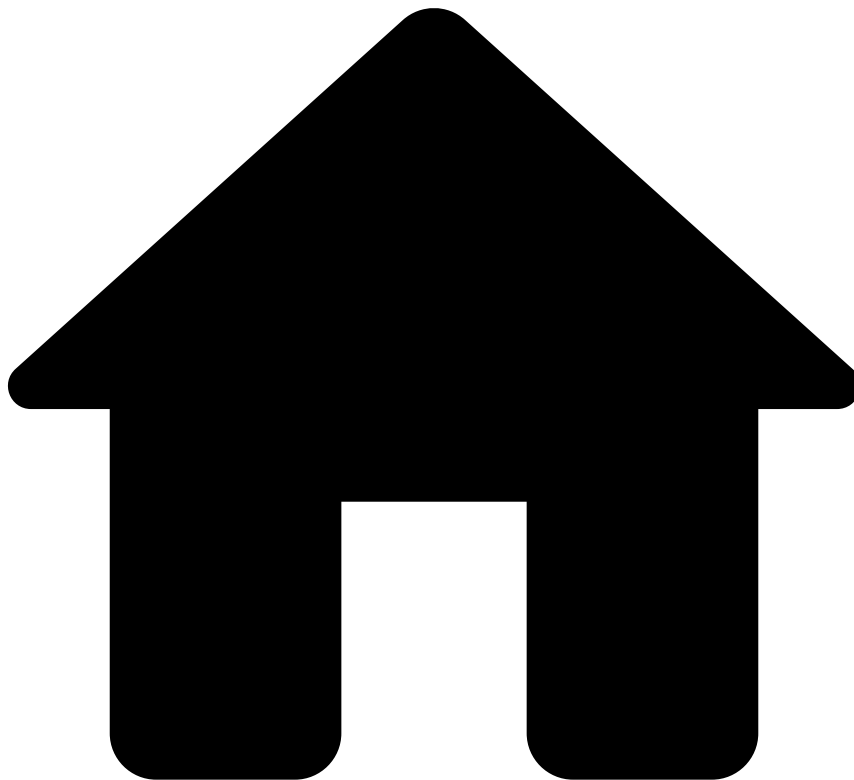
Configuration

In the Configuration tab, you can see all of the configuration information of the Sandbox Evaluation you have chosen to analyze.

Getting Started

To start working on your Models, learn how you can [create a Sandbox Evaluation](#).

•



- [Creating a Data Science project](#)
- Tuning Job executions with Job Profiles

On this page

Tuning Job executions with Job Profiles

Was this page helpful? 

Pulse enables you to optimize configurations that are used when handling large amounts of data.

Data scientists can use this functionality to tweak the properties used in various tasks within the [Data Science Loop](#) that are performed in a cluster-computing framework. For example, most of the Data Science tasks including running [Plans](#), [Model Evaluations](#), [Rules Evaluations](#), are executed by Spark jobs.

Each type of Job may require different configuration settings to be performed efficiently. For example, depending on the size of your dataset you might have to adjust the Spark memory allocated to train a Model Evaluation.

Pulse allows you to configure Job Profiles with different sets of Spark properties. You can then select the Job Profile that best fulfils the needs of the target task that you want to execute.

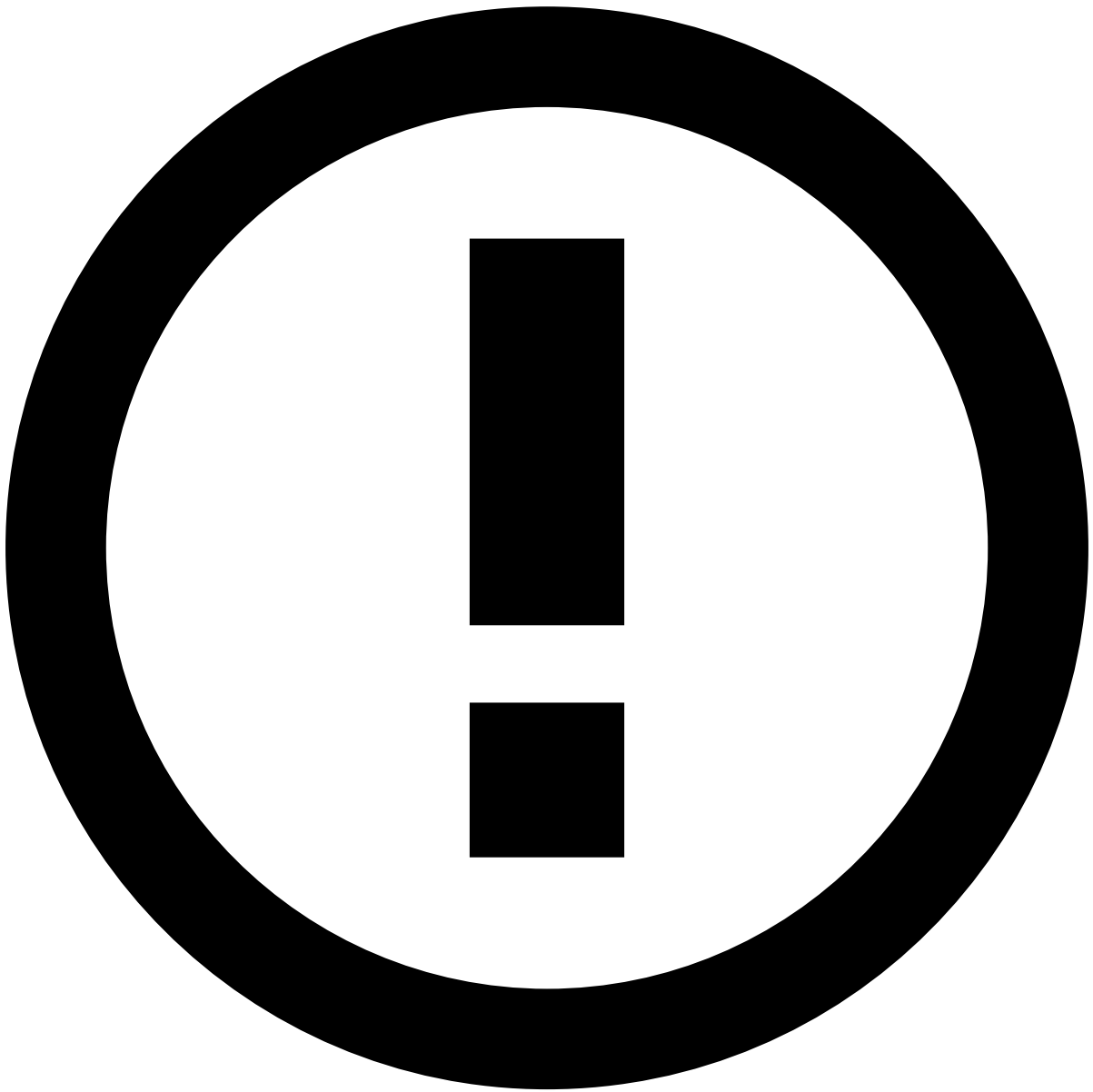
Memory considerations

When running Spark jobs, it's recommended to have a default profile for all jobs, and define Job Profiles for executing resource-intensive jobs.

Calculation tasks may fail if they have insufficient memory available. This might happen because each Spark node has fixed resources available, for instance a fixed number of cores and a fixed amount of memory. Some nodes may have larger resources than others, but by default it's not sure that the tasks that need large resources get assigned to those nodes. Typical examples of tasks needing larger memory are Model Trainings, Hyperparameter Tunings and Plan Executions.

If smaller jobs occupy the resources that have larger capacity, Pulse doesn't leverage the full potential of the Spark cluster. To ensure that the necessary resources are available for jobs that need them, you can use Spark properties to influence the number of cores and the amount of memory used by a job. Job Profiles can regulate what kind of resources should be assigned to a job which is running with a specific Job Profile. For instance, you can have a default profile for all jobs, but define a Job Profile for executing Plans, and another one for Hyperparameter Tunings to make sure that these get the proper resources they require.

If these Job Profiles are created, you can use them when creating a job execution of a related Data Science resource.



Restrictive default settings

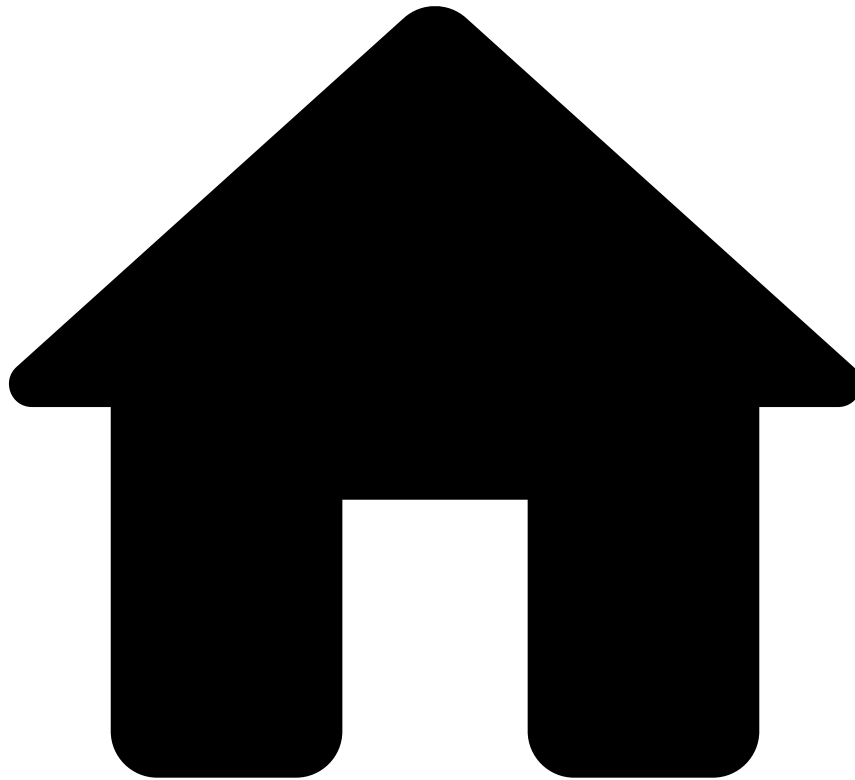
The default Spark settings are restrictive. For example, they allow 1GB of memory per node. It is recommended to create a Spark configuration strategy early on in a project, and keep adjusting the settings as you monitor the Job Executions.

Learn More

Check the following pages to learn more about [Job Profiles](#) in Pulse:

- [Creating a Job Profile](#)
- [Selecting profiles](#)
- [Editing properties on demand](#)

•



- [Creating a Data Science project](#)
- Using the Data Science Framework (DSF)

On this page

Using the Data Science Framework (DSF)

Was this page helpful? 

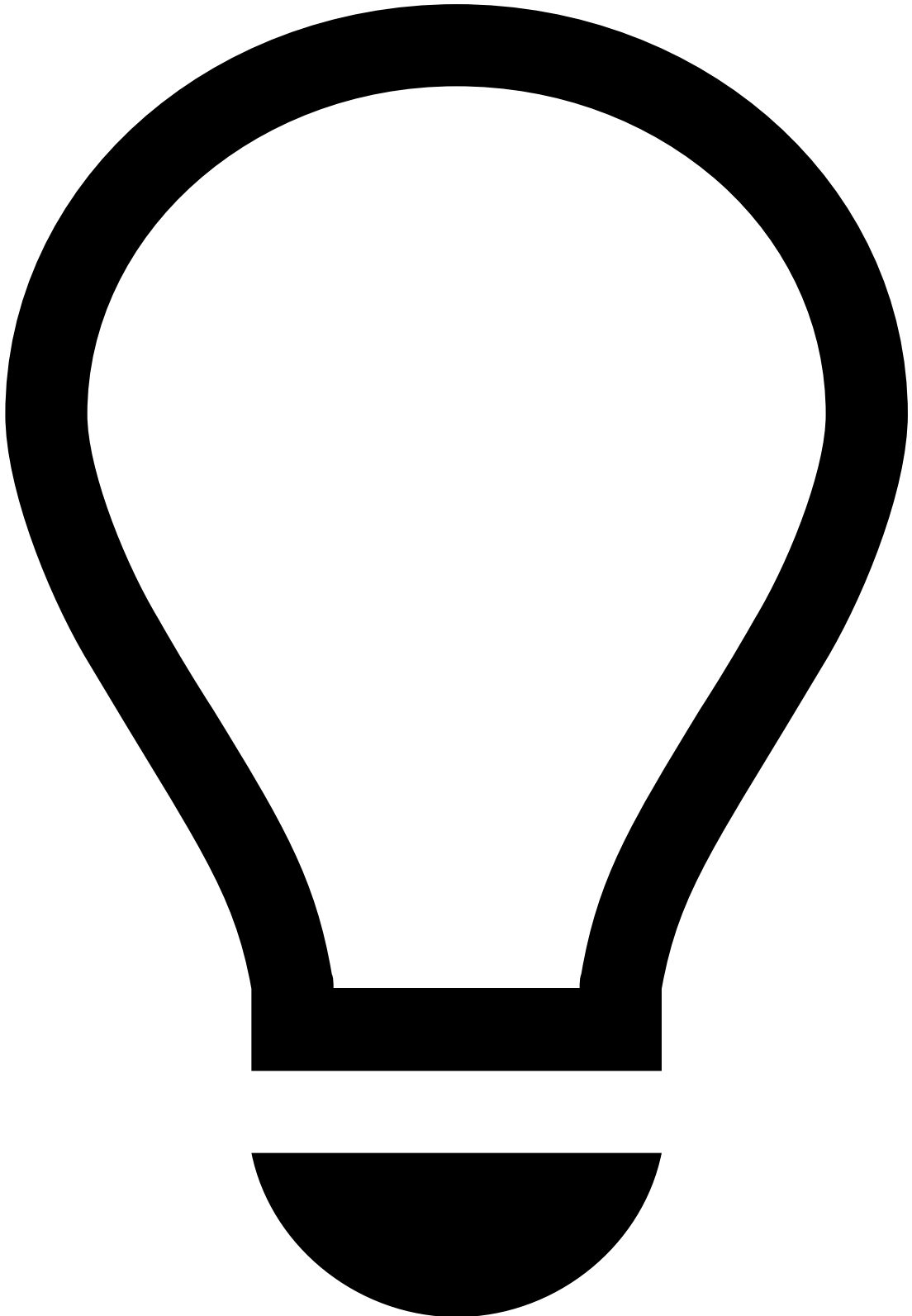
The Data Science Framework (DSF) is a dedicated environment used in risk assessment strategies for building, testing, and evaluating rules and models according to the Data Science Loop.

It allows for initiating Data Science (DS) jobs, either via Pulse or JupyterLab, that are then processed through the Pulse application. Before diving into the importance and impact of tuning jobs, learn more about the DSF and DS jobs [here](#).

Why tune DS jobs?

Tuning DS jobs is the process of optimizing Spark configurations used by the cluster to handle specific workloads efficiently. This helps prevent:

- Executions from hanging due to the lack of requested resources.
- Executions crashing due to cheap job configurations.
- Resource starvation caused by greedy job configurations.



tip

Choosing the appropriate configuration is **trade-off between what's available and what's needed**.

Note that there isn't a "one-size-fits-all" configuration. Heavier jobs will take advantage of more generous configurations, while lighter jobs will perform well with more sensible ones.

Additionally, clusters are shared by all team members, so guaranteeing that everyone is mindful of their job configurations is key to avoid concurrency issues and leverage the resources appropriately.

Jobs can be tuned either within Pulse (using Job Profiles) or in JupyterLab (by providing the Spark configuration when opening a session).

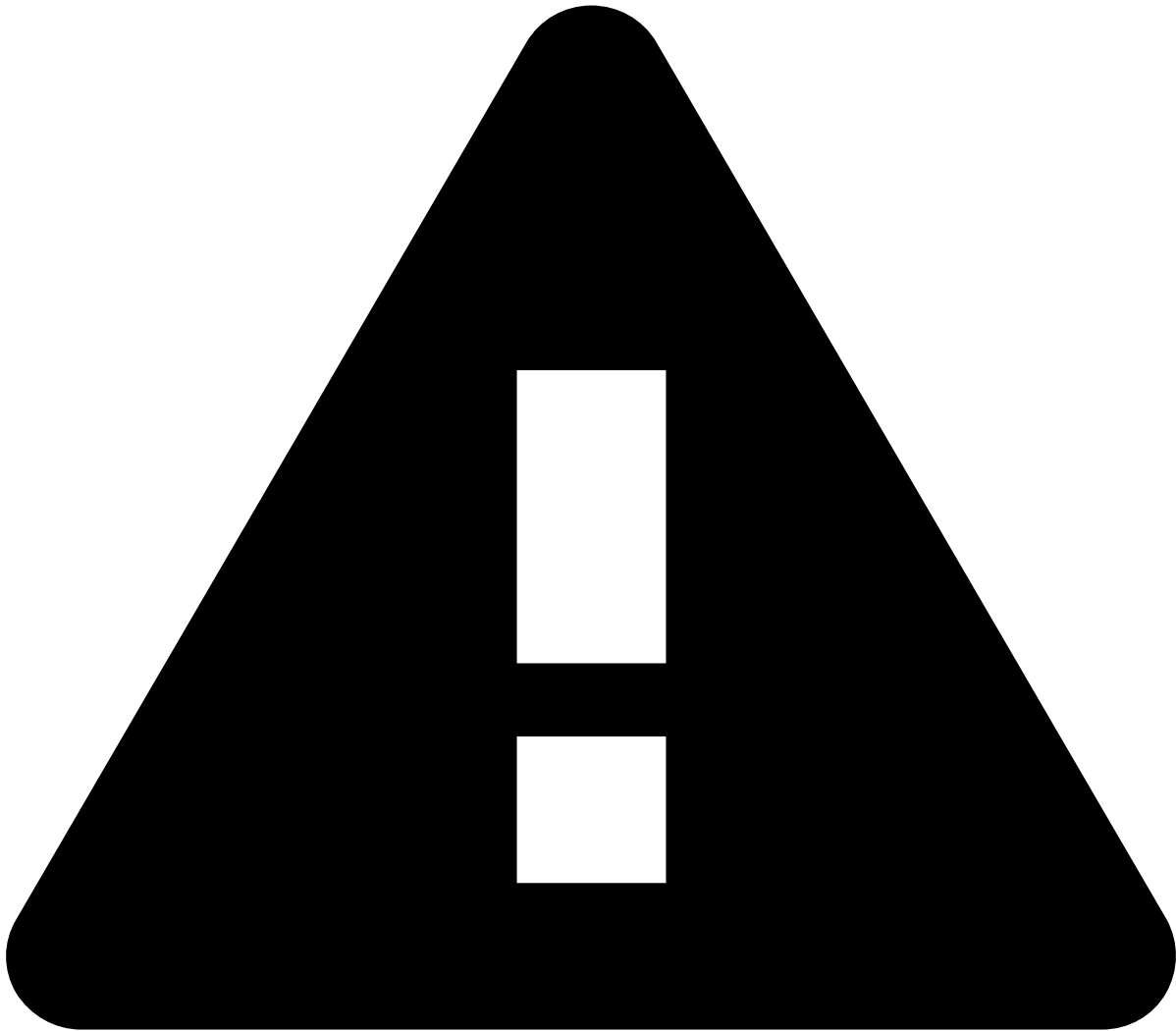
Impact of tuning in the DS job life cycle

To better understand how job configurations affect the life cycle of a DS job, it is important to consider the [scaling capacity](#) and how [resources are allocated and shared](#).

Scaling capacity

Scaling refers to the cluster's capacity to add and remove task nodes from the pool of resources at any time depending on workload. It occurs automatically based on demand given by the number of executors that are waiting to be allocated.

The scale-up strategy follows a tiered approach: the higher the demand (i.e. jobs pending/waiting for allocation), the more capacity is allocated. These tiers can be classified based on how fast more capacity is added (e.g., light, moderate, high, very high).



Capacity limits

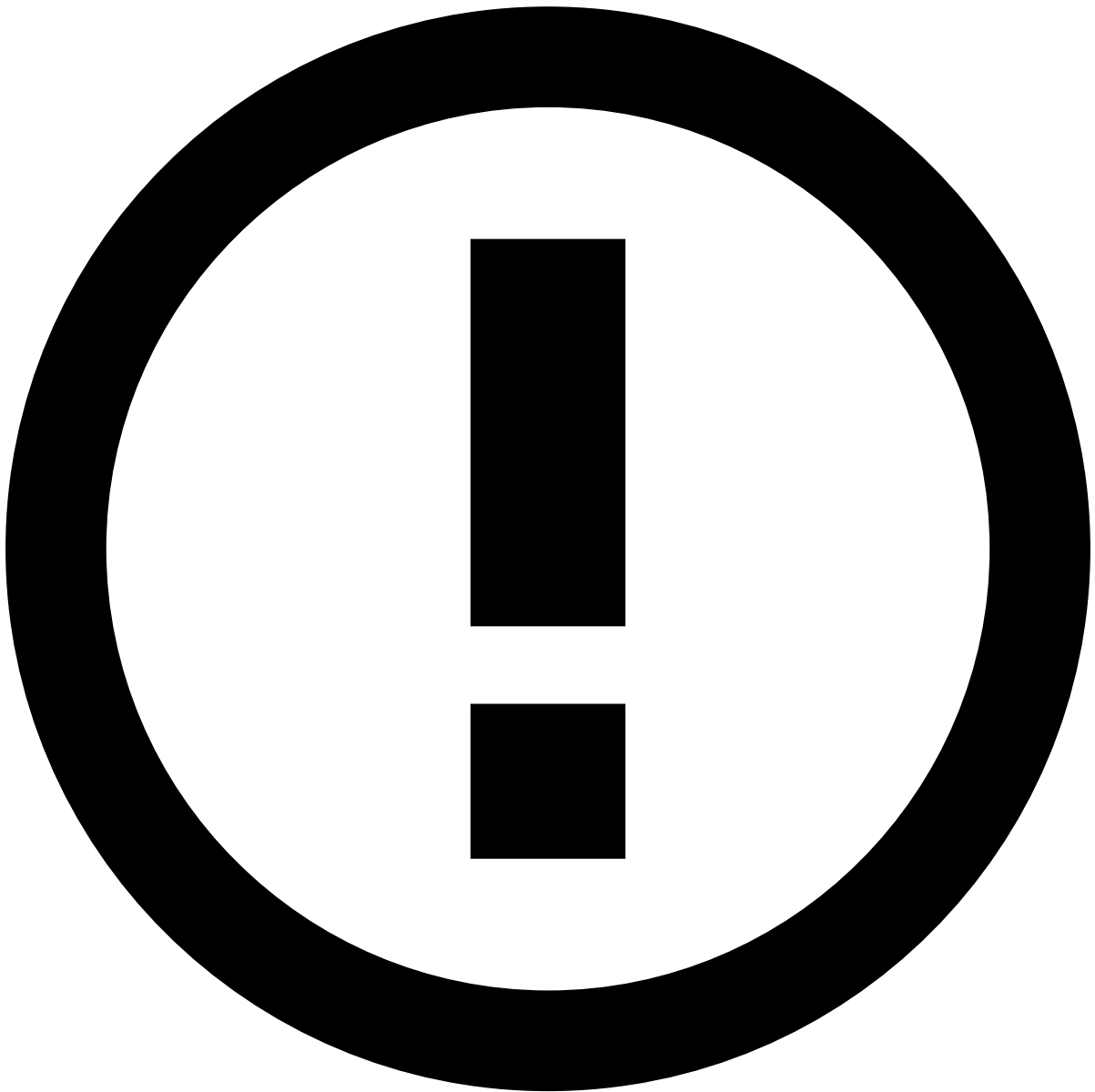
There is an upper bound to the maximum capacity attributed to you based on your contract with Feedzai.

Resource allocation and sharing^â

Choosing adequate capacity for a job is important to allow for optimal execution times and sharing that capacity with other people in the team. To better understand how resources are allocated and shared within the cluster, it's important to know [how jobs allocate capacity](#) and [what it means to have running Spark sessions](#).

How do jobs allocate capacity?^â

When you submit a Pulse job or open a Spark session, the EMR (Elastic MapReduce) cluster registers the request and allocates resources dynamically depending on the workload. This means that if you request a job with a minimum of 1 and a maximum of 8 executors, the job begins executing as soon as the cluster is able to allocate 1 executor. If during the job execution more cluster resources become available (e.g., because other jobs have finished or the cluster has scaled up), the cluster allocates additional executors, up to a maximum of 8.



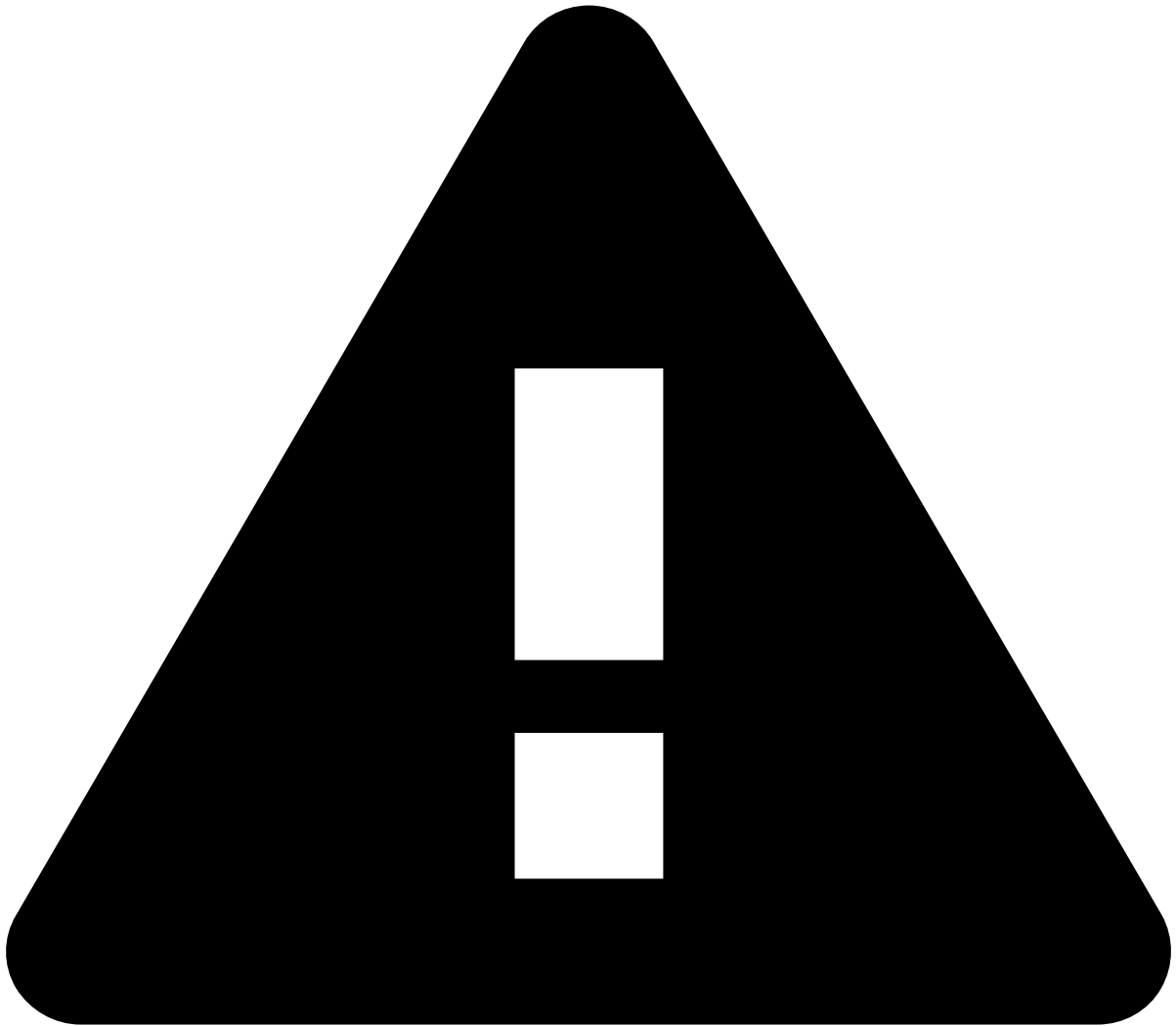
info

Note that executors are main processing units for jobs and are indivisible. This means that while executors for the same job can potentially “live” in a different cluster node, each executor alone must fit entirely in a given cluster node.

Refer to the [What are executors?](#) section for more details on these processing units.

On the other hand, if dynamic allocation is disabled, the job only begins executing when the cluster is able to allocate all the executors requested in the job configurations. For example, if you request a job with 8 executors (each with 12GB and 4 cores), the job will start once the cluster is able to fit those 8 executors within the pool of cluster resources available at that moment.

Refer to [Applying configurations in JupyterLab](#) and [Applying configurations in Pulse](#) for details on how to configure jobs/sessions, respectively.

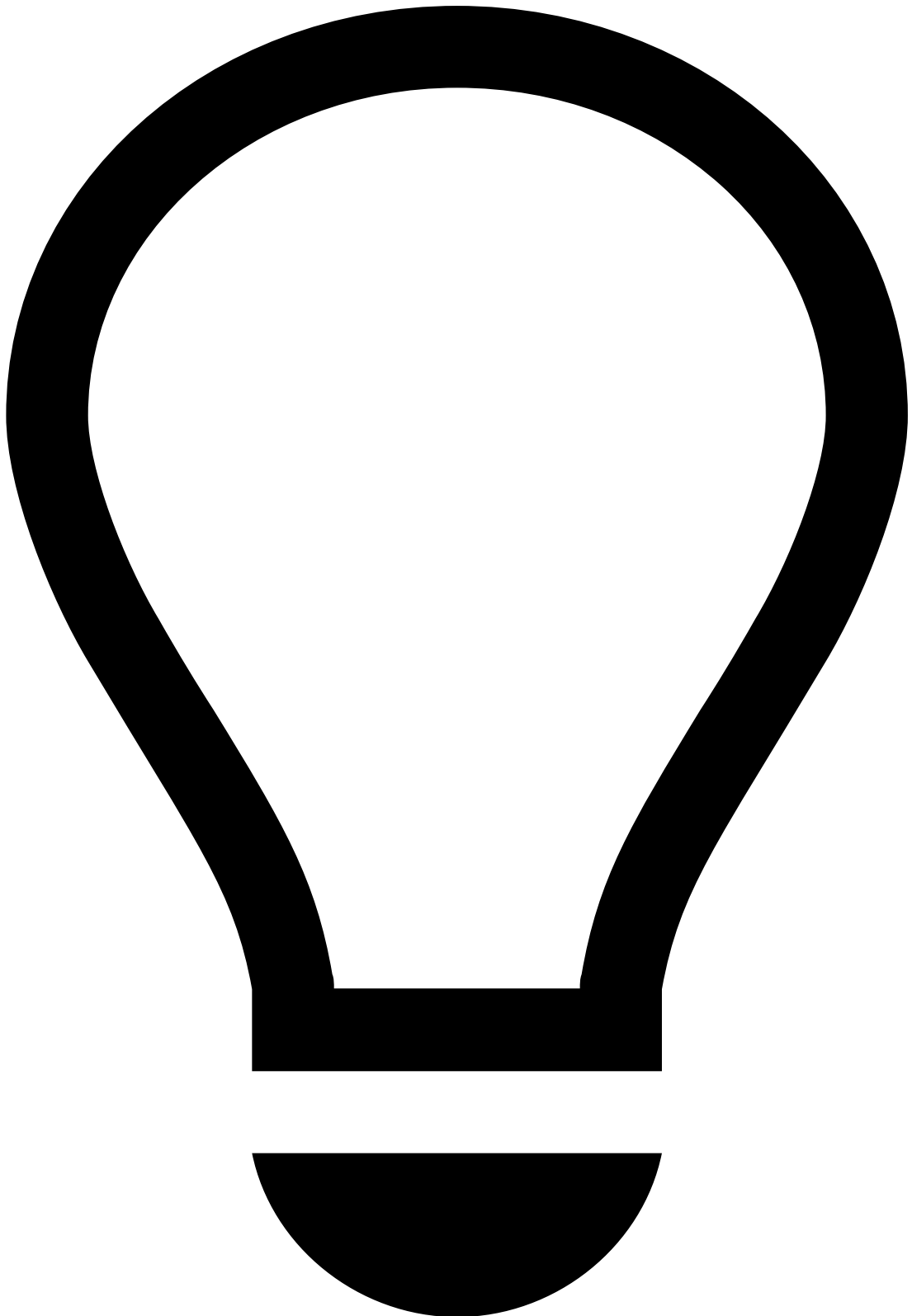


warning

Disabling dynamic allocation is only recommended when debugging issues with the cluster.

Depending on the number of executors being requested by all the jobs registered in the cluster at a given moment (including jobs waiting for resources to begin execution, and dynamic allocation jobs already executing and requesting more executors), the cluster can scale up and add more nodes to its pool of resources.

In contrast, if the cluster resources are idle for a certain amount of time (e.g., because there are few jobs running) it can scale down and remove cluster nodes from the pool of resources.



What does this mean?

Both the cluster resource usage (i.e. all jobs from all users using the cluster at the same time) and the cluster total pool of resources (i.e. if the cluster is scaled up or scaled down) have an impact on the duration of your job. Therefore, the exact same job might take a different amount of time to complete on separate occasions.

When you submit your job, you can check its status (i.e. if it's running or waiting for resources) in the **YARN Web UI**. There, you can also find the total and available pool of cluster resources. These metrics can help you understand why your job hasn't started yet (e.g., if there isn't enough memory or cores to satisfy your job configurations).

You can also use the **YARN Web UI** to check the resource usage of your running jobs to make sure they aren't exhausting most of the cluster resources and adhere to fair cluster usage.

What does it mean to have running Spark sessions? 📖

Spark sessions are created in JupyterLab and represent the connection between your notebook and the cluster. Refer to the [Interacting with Pulse via JupyterLab](#) page for additional information, including a list of available notebooks.

Note that once created, Spark sessions remain active until you either:

- Close them using `spark_session.stop()` .
- OR
- Kill the notebook kernel.



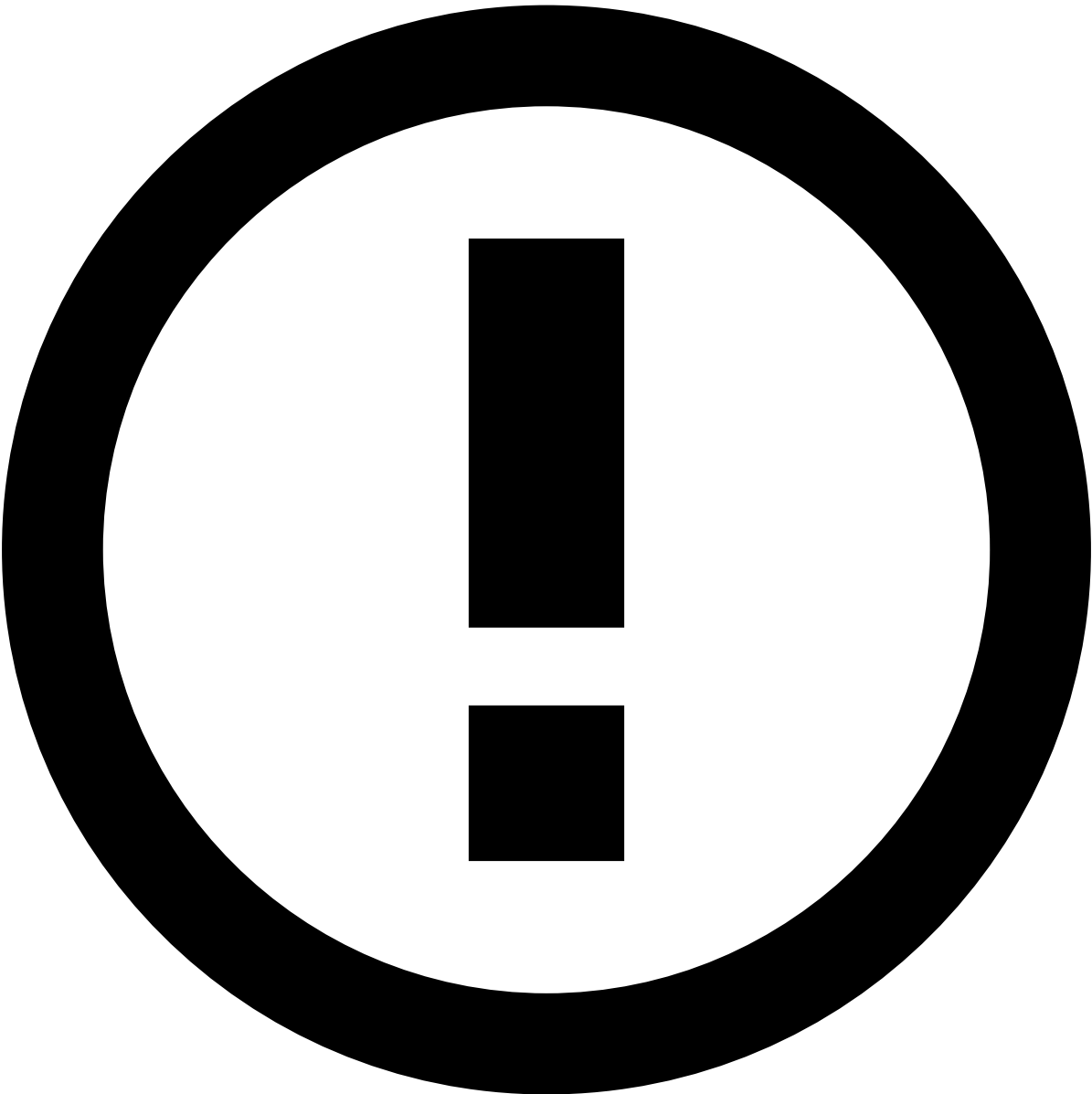
warning

Be mindful of your active Spark sessions, and make sure you manually stop them if they're no longer needed. Hanging (idle) sessions can occupy cluster resources and impact the other cluster users' experience.

In some cases, Spark sessions that are idle for a long period are automatically closed, but this depends on the cluster's configuration.

Common scenarios

Here are some scenarios where tuning Spark configurations is most commonly needed and what changes they require:

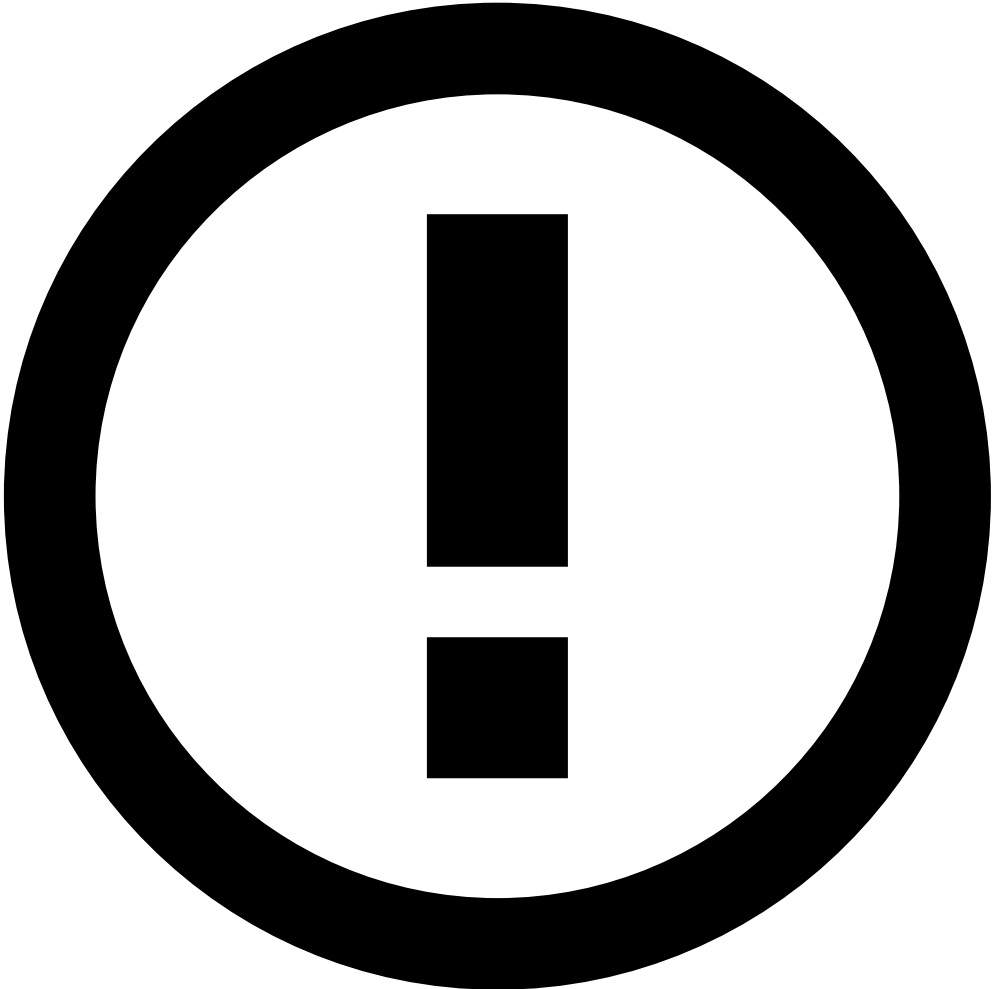


info

The following table provides guidance in terms of how different parameters could be configured, but choosing the appropriate configuration is **trade-off between what's available and what's needed**. Be mindful of job configurations to avoid concurrency issues and leverage the resources appropriately.

There isn't a "one-size-fits-all" configuration. Heavier jobs will take advantage of more generous configurations, while lighter jobs can be executed in an acceptable timeframe with less demanding configurations, and without compromising other user's jobs.

Scenario	Considerations
Dataset analysis and rule evaluations	This scenario doesn't require you to limit the number of executors.

Scenario	Considerations
Model training	This is a <u>single-task/executor</u> job, so: • The number of cores per executor equals the number of cores per task. • The rule-of-thumb for job memory and cores per executor is 1/4 of the task node size for both. This allows for running four models in parallel in just one task node, which complies with the default HPT notebooks (that launch four model training jobs simultaneously).  note Even though model training jobs are single-task/executor jobs, you must allocate more than one CPU per task using <code>spark.task.cpus</code> . This is set by default to <code>1</code> . However, if you don't override this default, you're essentially running a job with X executors but only <code>1</code> will be used (which is a waste of resources). Since model training is a single-task/executor job, make sure the task gets all the executor cores by matching the <code>spark.task.cpus</code> to the <code>spark.executors.cores</code> . Essentially, this allows you to increase the number of cores per executor in order to leverage model training multi-threading (as long as that property remains aligned with the number of CPUs per task).
Plan executions	In this scenario: • Memory-heavy jobs benefit from parallelization. • Both the cores and memory per executor should be maximized.
Data exploration/manipulation	In this scenario: • The necessary configurations vary depending on the data and type of exploration/manipulation that will be performed.

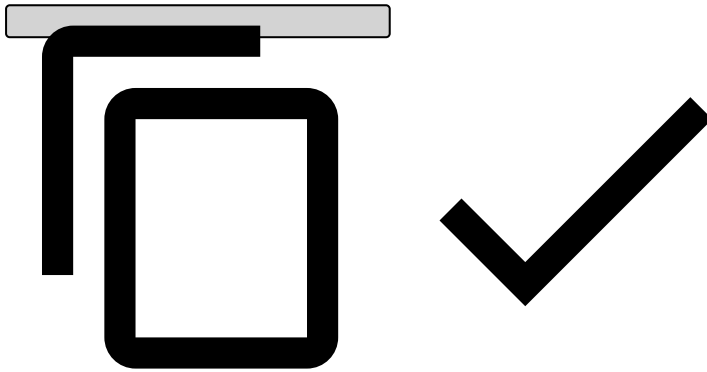
Scenario	Considerations
	• A mixture of approaches is recommended.

Applying configurations in JupyterLab

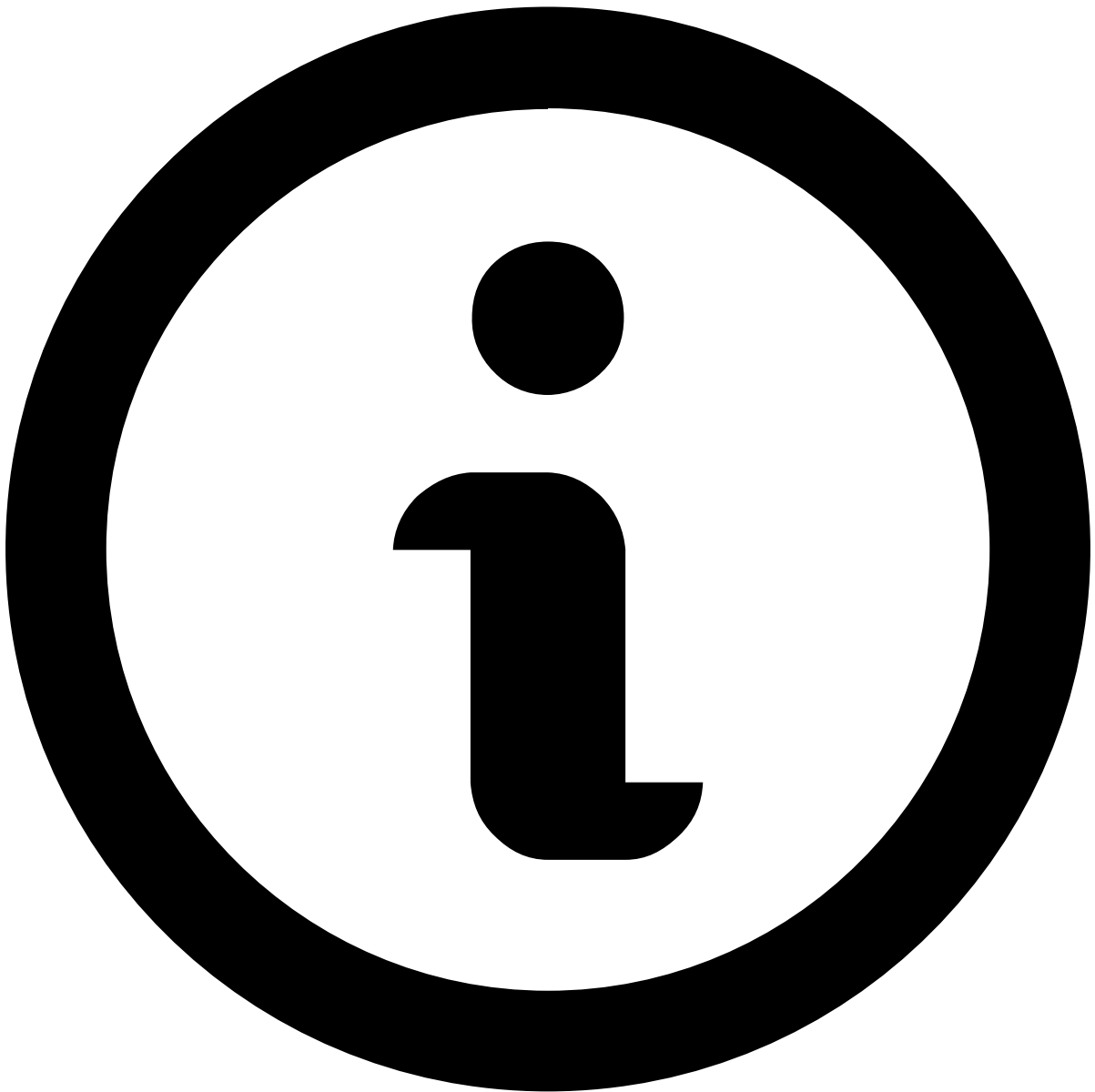
When creating Spark sessions via JupyterLab (e.g., when using notebooks), you need to provide the Spark configuration at the beginning:

```
# Spark Session Parameters
SPARK_PARAMETERS = {
    "spark.executor.cores": "7",
    "spark.executor.memory": "8g",
    "spark.executor.memoryOverhead": "5g",
    "spark.dynamicAllocation.enabled": True,
    "spark.dynamicAllocation.initialExecutors": "2",
    "spark.dynamicAllocation.minExecutors": "1",
    "spark.dynamicAllocation.maxExecutors": "6",
    "spark.dynamicAllocation.executorIdleTimeout": "300s",
}

# Get SparkSession with default Pulse's configuration
spark = feedzai.get_spark_session(SPARK_PARAMETERS)
```



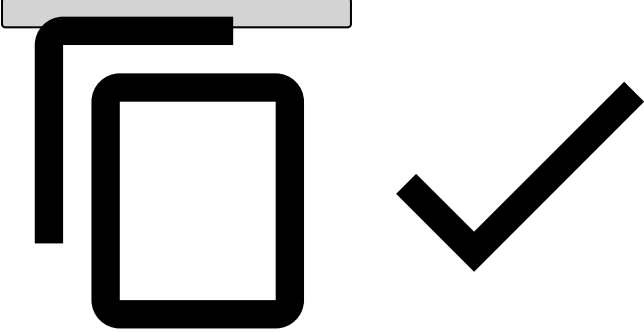
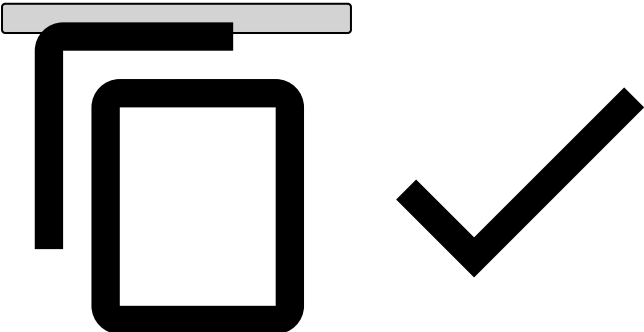
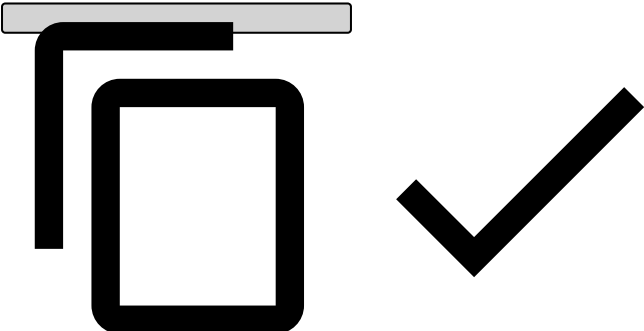
The configuration examples below can be used to configure your Spark jobs:



note

Ensure you replace the values for each as needed.

Configuration aspect	Example
Cores per executor	<pre>SPARK_PARAMETERS = { "spark.executor.cores": "7", }</pre>

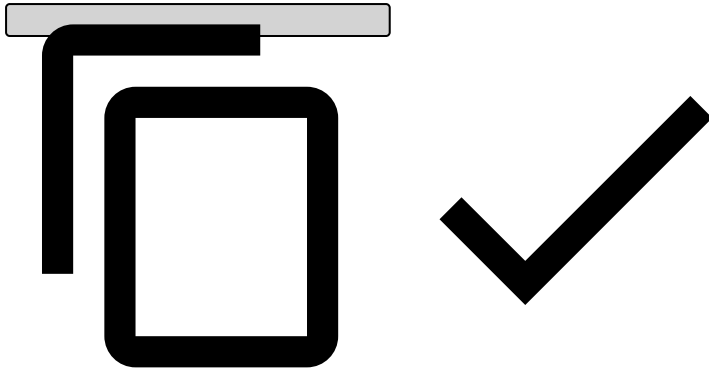
Configuration aspect	Example
	
Memory per executor	<pre>SPARK_PARAMETERS = { "spark.executor.memory": "8g", "spark.executor.memoryOverhead": "5g", }</pre> 
Dynamic allocation	<pre>SPARK_PARAMETERS = { "spark.dynamicAllocation.enabled": True, "spark.dynamicAllocation.initialExecutors": "2", "spark.dynamicAllocation.minExecutors": "1", "spark.dynamicAllocation.maxExecutors": "6", "spark.dynamicAllocation.executorIdleTimeout": "300s", }</pre> 

Validating Spark session configurations [🔗](#)

Once you've completed the configuration and before running a full job, verify jobs are behaving as expected. This helps spot issues early on, before committing to executing real jobs.

To do this, run the following test job:

```
spark.range(100_000_000).count()
```



Then, via the **YARN Web UI**:

- Confirm the Spark session is in the list of **Applications**.
- Monitor the Spark session **State** (running sessions should have `State = RUNNING` , while sessions that are waiting for resources should have `State = ACCEPTED`).

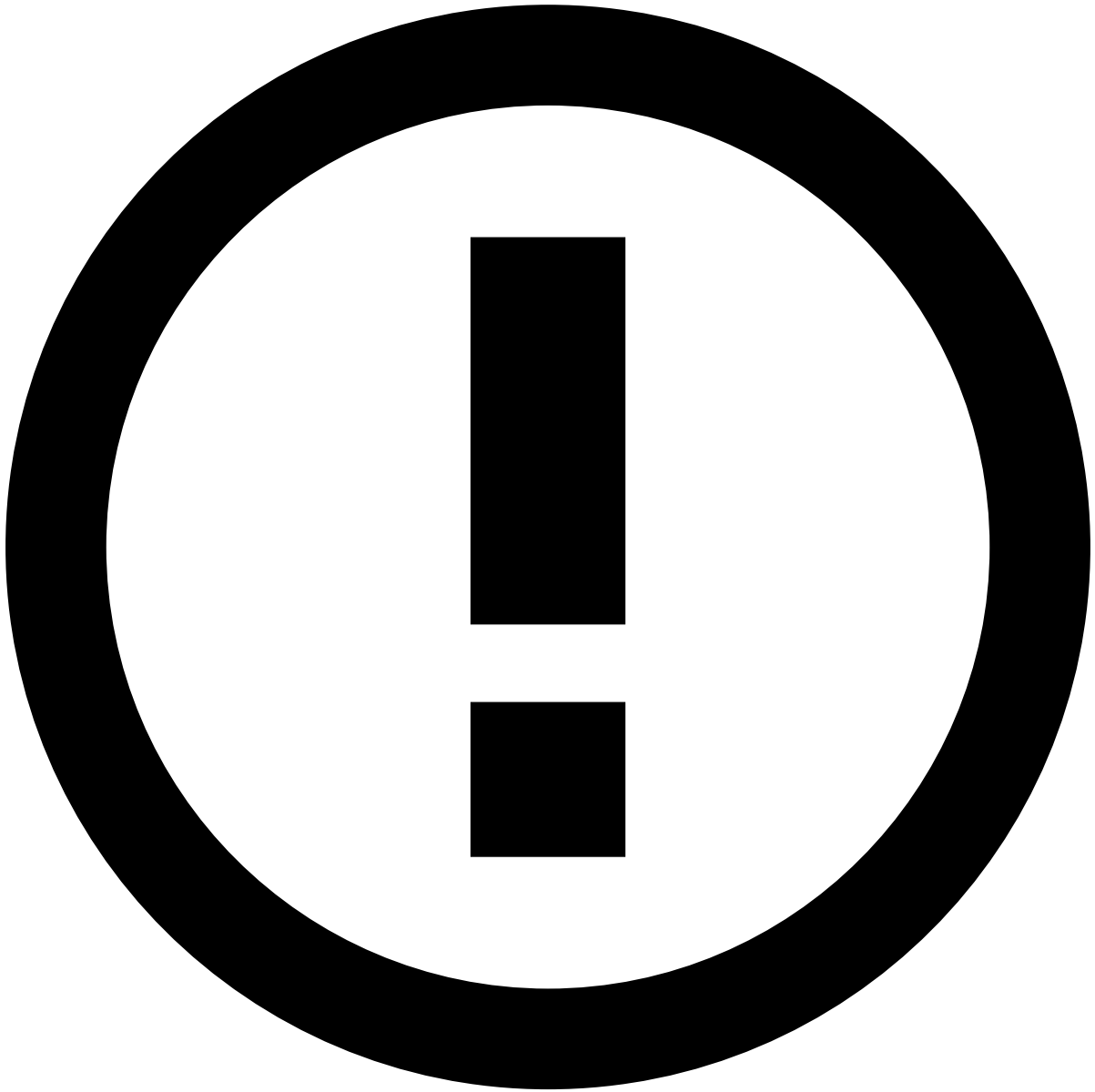
Additionally, check the **Spark History Server** to confirm that your Spark session configurations were correctly set. This can be accessed in the full list of configurations via the **Environment** tab.

Applying configurations in Pulse

For DS jobs triggered in Pulse (e.g., plan executions, model training, rule evaluations, etc.), you can use [Job Profiles](#) to provide a Spark configuration.

Read all about [Tuning Job executions with Job Profiles](#) and learn how to:

- [Create a Job Profile](#) to apply a set of property configurations when running jobs.
- [Select the profile](#) that is more suitable for the Spark job that you want to run.
- [Edit Spark properties on demand](#) to control the configuration of the Spark job that performs the calculations for a specific Data Science resource.



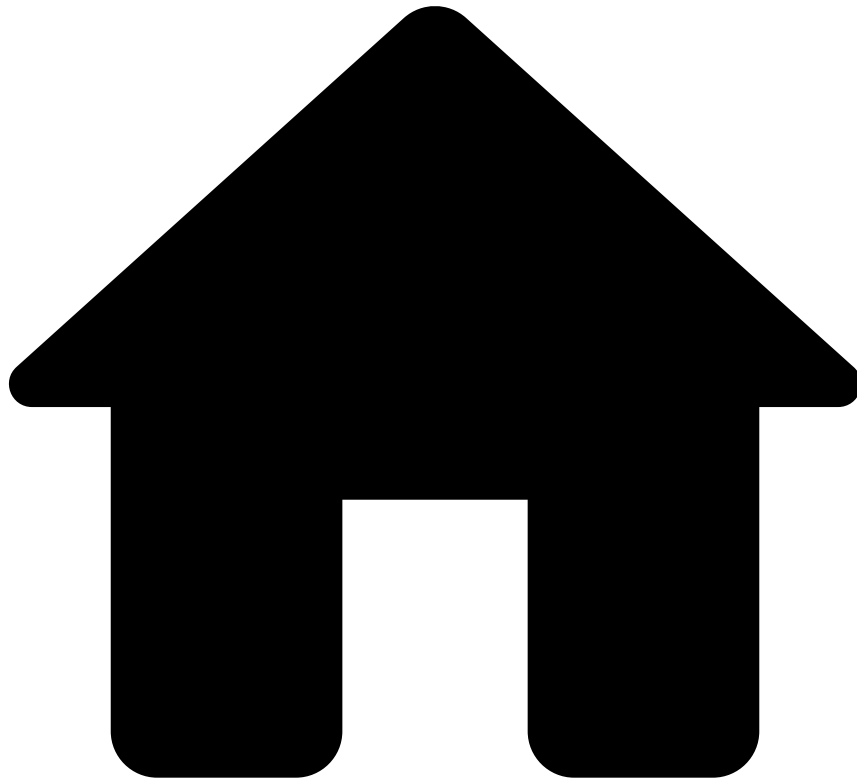
Jobs triggered via the DS-API

You can also trigger Pulse jobs from JupyterLab using the **DS-API**. In these cases, the job spark configurations are managed through a combination of:

- Parameters sent through the DS-API method calls.
- Pulse Job Profiles (**not** through `SPARK_PARAMETERS`).

As an example, refer to the [plan execution DS-API method documentation](#) (see the `job_profile` and `job_properties` arguments).

-



- [Deploying a project into Production](#)
- Creating a schema

On this page

Creating a schema

Was this page helpful? 

Why

The first stage involved in taking your Data Science project into Production, and designing a runtime [Workflow](#), is configuring how Pulse should expect to consume real data from an external source.

A **Schema** defines exactly how the real input data looks like. It defines the types of each field for transactions being processed in a runtime workflow.

How to

To start configuring your data and add a schema to Pulse, execute the following steps:

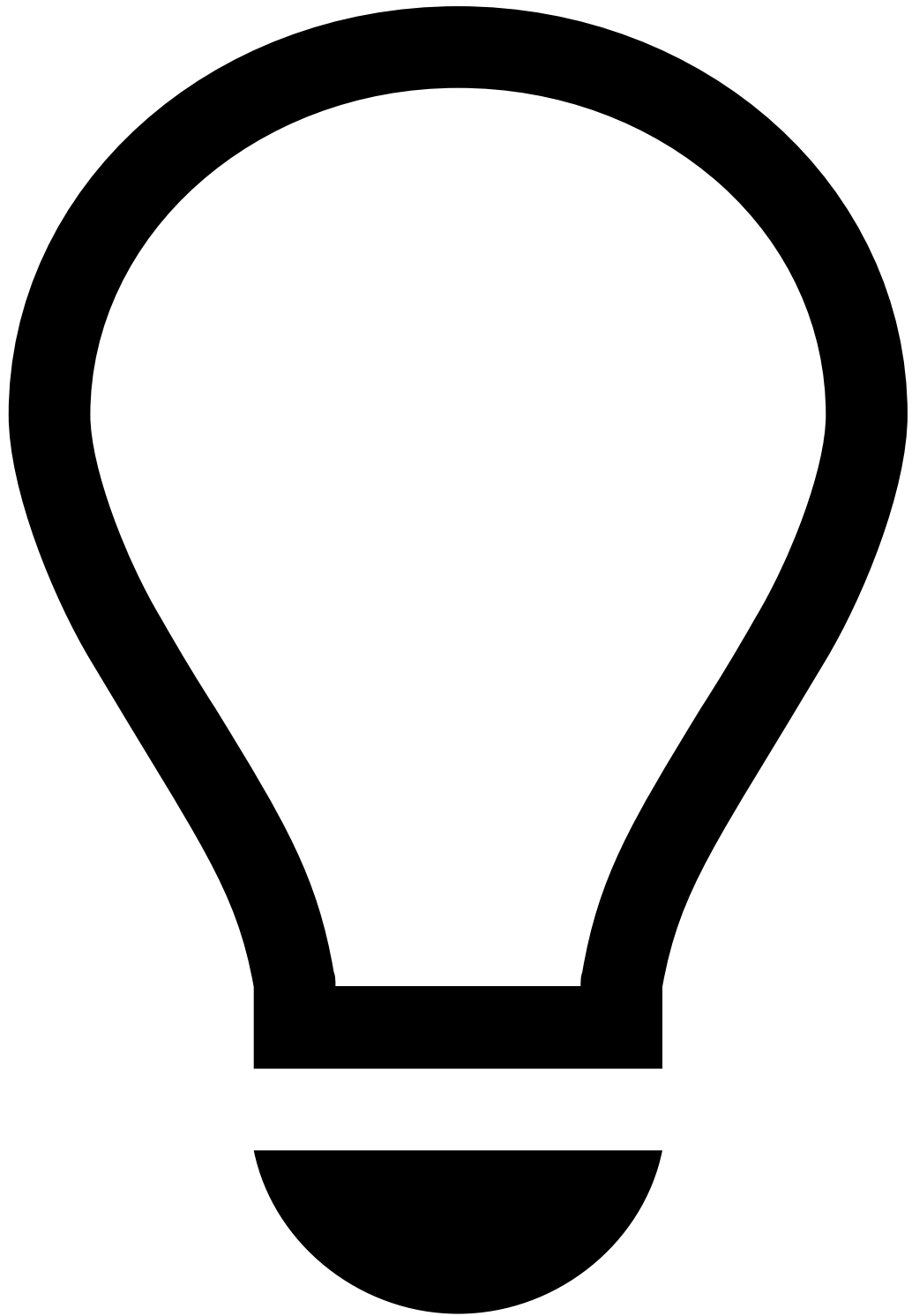
1. When you have completed your Data Science tasks and the project is ready to move to Production, in the **Data Science** tab, select a Data Source.
2. Export the Data Source to a Schema. Pulse automatically generates a Schema with the same name as the Data Source. For example, Card Transactions.
3. Navigate to the **Data Configuration** tab and select the newly created Schema.



Changing a Schema exported from a Data Source

You must take into account that adding or reordering fields in a **Schema** is not allowed since it is [immutable](#).

4. Examine the Schema fields and types to make sure it matches the incoming data to your fraud detection workflow.



Configuring the Workflow Schema

In alternative, Pulse allows you to automatically generate a Schema from a [Data Source](#) when you configure the **Origin Schema** field of a [Workflow](#).

Manually creating a schema

In alternative cases when you don't have a dataset available, and thus go straight to deployment, you can manually specify a Schema. To do so, execute the following steps:

1. In the **Data Configuration** tab, **Add** a new Schema.
2. For each Field that your Schema should have, you must:
 1. Name the Field.
 2. Select the Field type.

3. Optionally, select **Advanced options** to set the field as a [Dimension](#).
4. Add another field if your Schema is still not complete. Repeat step 2.
5. When you have added all Fields, save your work as a new Schema.

Configuring Dimension Mappings

Dimension Mappings allow you to define how dimension codes can be translated to human-readable descriptions.

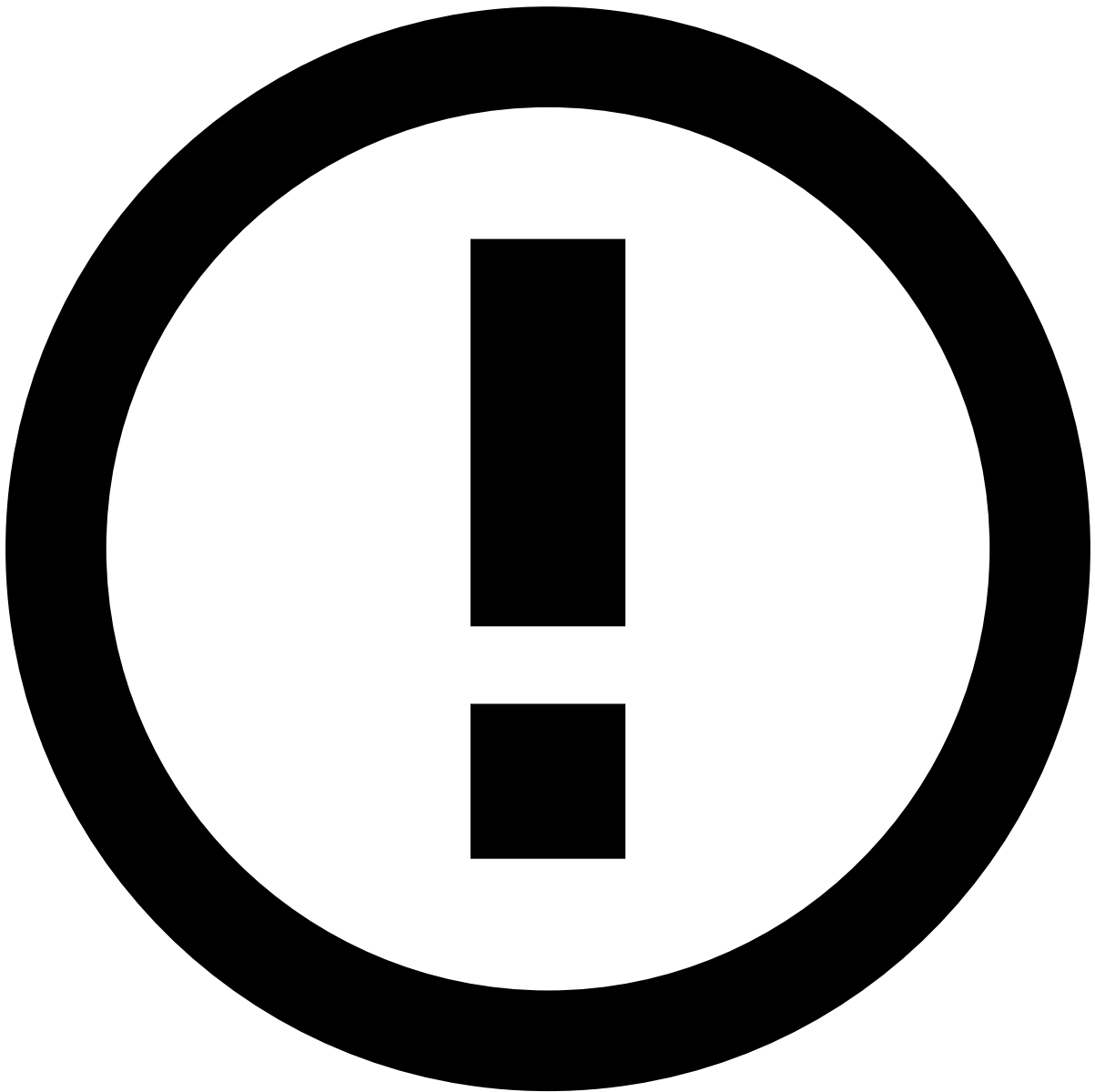
After defining dimension mappings, their human-readable descriptions will be available throughout the whole product: dashboards, tooltips, labels, live-drill and so on.

Learn more about how dimension mappings help you [manage Dimensions](#) in real-time applications.

When you create a [Data Source](#) and export it, Pulse also accelerates your work by automatically generating a dimension mapping for each observed values in categorical fields.

To complete a configuration for a dimension mapping:

1. In the **Data Configuration** tab, select a **Dimension Mapping**, or **Add** a new one.
2. Finalize your configuration with any of the following operations.
 1. Adding a new Item that maps a specific code to a description.
 2. Editing an existing Item to update the code-description mapping.
 3. Deleting an existing Item.



Dimension Mappings applicability

Dimension Mappings are only available for fields that have been defined as [Dimensions](#).

Next Steps

Now Pulse knows how input data is formatted and what your fraud prevention workflow will consume. Proceed by configuring how you must [configure custom elements](#) your workflow will use; for example, input adapters to consume data.

•



- [Deploying a project into Production](#)
- Designing a workflow

On this page

Designing a workflow

Was this page helpful? 

Classifying real-time events comprises several stages. For example, fraud detection is a task comprised of several steps with the aim of deciding whether or not an event represents a fraudulent transaction. Thus, Pulse lets you group these processing steps into a workflow.

Workflows represent the life cycle of an event through the system. They define how information is passed on between components that may filter and/or transform the event.

A **Workflow** consists of processing steps (elements) and links between them. Each workflow element processes or transforms the data as soon it receives data. Workflow elements can be seen as components that process data with the ability to apply operations such as feature extraction, model scoring, decision/detection rules, etc.

The following image depicts a **Workflow** where input data is processed with a [Plan](#) to generate features, scored with a [Model](#), and evaluated by a block of [Rules](#).

When designing a **Workflow** to process events in real time, the following activities are involved when adding and configuring elements:

- [Configuring outcomes](#)
- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding rules](#)
- [Adding custom services](#)

Workflow patterns

Workflows give you a lot of flexibility on how you can compose elements and define the path that events should follow. Check to following sections on some patterns you can use when designing a workflow.

Branching

Pulse allows you to configure your workflow path in a way that there are multiple branches that can process each event. A common scenario is to apply different operations based on the transaction's country.

This can be achieved by adding a filter to each of the branching service's links, as depicted in the following image. Note that the expression for the filter can make use of any **Workflow** schema field, and must be a stateless [PQL](#) expression. Additionally, [UDEs](#) are not supported.



Parallel branching

Pulse does not execute separate workflow branches in parallel. Thus, although branching is possible, a real-time event will always follow a single linear path from input until the workflow terminates.

You must always make sure that when you branch the workflow, all possible paths are mutually exclusive. This means that **an event must only be routed to exactly one of the possible paths.**

The following examples show how to branch a workflow into two paths without leading to unsupported behavior:

- Link A filters by "amount > 2000" and Link B filters by "amount <= 2000"

- Link A filters by "location == USA" and Link B filters by "location != USA"

If do not take these precautions there is more than one branch where the filter condition is *true*, than Pulse will route the event to the first one that evaluates as *true*, and write an exception to the log.

When joining branches into a single path you must make sure that all incoming paths carry events with the same **Schema**, as shown in the following diagram.

Since Pulse never executes both branches in parallel, once an event arrives at the [Plan Service](#), it must have the same schema regardless of the original path it took.



Deployable Plans

[Deployable Plans](#) only allow for a single [Data Source](#) to be configured as input.

Therefore, you must ensure that if you perform data transformations in any of the branches when an event arrives at the join point it still has a data schema that is compatible with the expected input for the **Plan Service**.

Branching with Tenants^â

Multi-Tenancy allows managing multiple Tenants in the same installation. The resources that support **Multi-Tenancy** are **Rules** and **Lists**. You can, for example, apply the same **Rule** to all the Tenants, or different **Rules** to different Tenants with just a few [configuration steps](#).

However, imagine that you would like to have different **Plans** or different **Models** applied to different Tenants. This is still possible using **Branching with Tenants** as an alternative approach to the native **Multi-Tenancy** solution, as long as the different configured **Plans** and **Models** expect the same input and produce the same output [Schema](#), so that subsequent Workflow steps, such as **Rules**, can be then applied to these non Multi-Tenanted resources. This configuration can only be performed with special permissions, and thus, is at the level of the Landlord (i.e. the Tenants cannot use **Branching with Tenants**).



Branching with Tenants

Note that this alternative approach to the **Multi-Tenancy** solution is viable because normally the same **Plan** and **Model** is used with all the Tenants and only in exceptional cases, a few Models or Plans are segregated by tenant.

However, this is not the case for **Rules**. The normal scenario is to have many Rules applied to all the Tenants or many Rules segregated by Tenant. Thus, **Branching with Tenants** is not a practical solution to be used with **Rules Projects**.

Considering the scenario of two different **Plans**, this can be achieved with **Workflow** configuration by adding a filter to the branches that connect the **Input Adapter** to both **Plans**, as displayed in the following image.



Adapt Schema

Note that the Schema must have a field identifying the Tenant so that it can be used to configure the filter and allow a real-time event to follow the right path.

Considering the scenario of two different **Models**, this can be achieved with **Workflow** configuration by adding a filter to the branches that connect the **Plan** to both **Models**, as displayed in the following image.



Adapt Schema

Note that the Schema has to have a field identifying the Tenant so that it can be used to configure the filter and allow a real-time event to follow the right path.

Applying different models to the same event

Depending on the scenario you might need to get the contribution of several models before reaching a final decision, or just running events through different models to select the best result.

Contributing to an outcome

To take into account the contribution of several models to the same [outcome](#) (for example *fraud* and *fraud_score*), you must run events through different **Model Services** in sequence.

This pattern is very similar to a regular **Workflow** with a **Scoring Service** followed by a **Rules Service**. The contributions of the several models are simply reconciled using the configured weights for each **Scoring Service**.

The following image displays an example where events are scored by two models in sequence, contributing to the same decision. The final score takes the weights of each model into account.

Selecting the best model

In scenarios where you want to run an event through several models and make your final decision based on the best absolute result, your **Workflow** should:

- Include a **Scoring Service** for each model.
- Configure separate [Outcomes](#) that represent the partial decisions of each model.
- Include a [Custom Workflow Service](#) that takes the outputs and outcomes of all **Scoring Services** and produces the final decision.

The following image displays a workflow where events are scored by two models in sequence, each one contributing to a different outcome, and a **Custom Service** to perform the final decision.

Using detection and decision rules

It is common to route transactions through different sets of [Rules](#) at different points in the **Workflow**. Typically, you may need to add the following types of **Rules**:

- **Detection Rules** Used in conjunction with [Lists](#), let you immediately identify events that match an entry in a list. For example, blacklisting and whitelisting.
- **Decision Rules** Used in conjunction with [Models](#), weigh in on the final [Outcomes](#) and scores for an event after it has been transformed and scored by a model.

The following image shows how such a workflow will look like.

Edit Workflow page

The **Edit Workflow page** provides a design area where you can model your [Workflow](#) through which your application will process real-time events.

The following image displays the **Edit Workflow page**:

The Edit Workflow page contains the following areas:

1. Configurations

Depending on the active tab, **Workflow** or **Advanced**, contains the fields that you need to configure for your workflow. The most important fields are:

- **Origin Schema**
The [Schema](#) definition of the input data expected by the workflow. Select an existing Data Source to automatically [generate the respective Schema](#).
- **Outcomes**
[outcomes](#) to add decision and score fields to events, and effectively define to which decisions will workflow **Services** be contributing to. For example, fraud/not-fraud and the fraud score.
- **Actions**
The actions that workflow **Services** can generate. For example, a **Rules Service** may generate a **Block** action for transactions identified as fraud.
- **Action Listeners**
[Custom Services](#) that execute custom logic to handle specific **Actions** whenever a **Workflow** element raises them.

2. Services

The available elements you can use to compose your workflow. The following elements are available:

- **Input Adapter**
Applies a [Custom Service](#) to [consume external data](#) and inject it into the workflow.
- **Plan**
Deploys [data transformations](#), exactly as modeled during the Data Science stage. Typically, to generate features out of raw input data before feeding it to a Machine Learning Model.

- **Scoring**

Runs the events through a Machine Learning [Model](#), exactly as parameterized and trained during the Data Science stage. Contributes to an outcome and the corresponding score.

- **Rules**

Runs the events through a [Rules](#) block, and similarly to the **Scoring Service**, also contributes to an outcome and the corresponding score.

- **Custom Workflow Service**

Deploys a [Custom Service](#) to [perform specific operations](#) on events, that Pulse does not implement out-of-the-box.

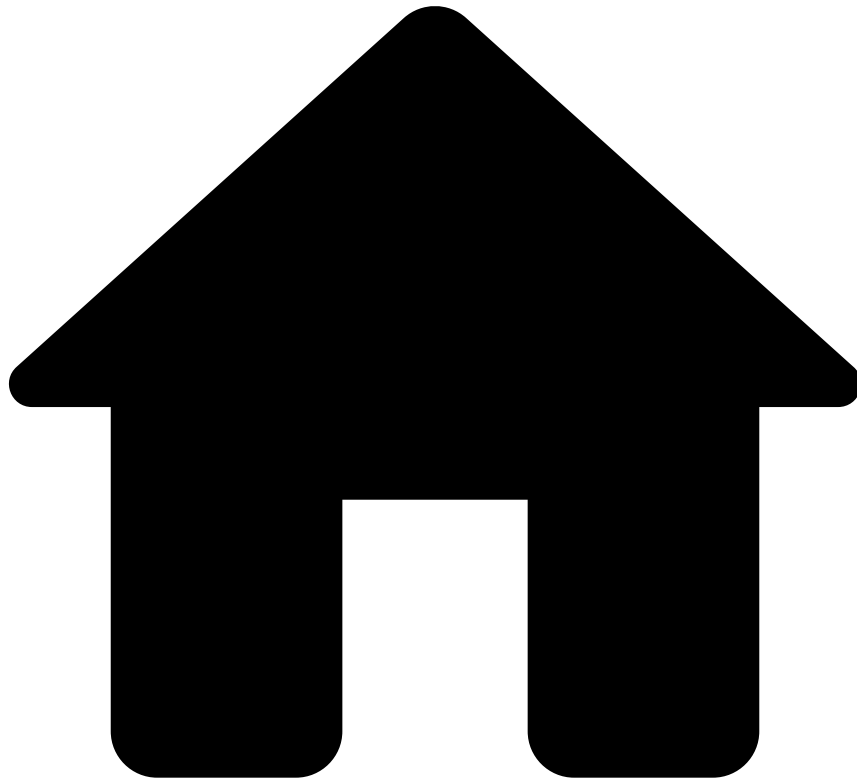
3. Diagram

Canvas to where you must drag the **Services** and visually design your workflow.

Edit services to define their main configurations such as Data Science snapshots, verify their input and output schemas and their mapping with the event schema, and properties.

Drag and drop the handles to connect two services in the workflow. **Double-click** an edge connecting two services to edit a filter condition, for example, only allowing events from a specific country to advance through this edge. This can be used to route these events to a different model for scoring.

•



- [Deploying a project into Production](#)
- Managing the project life cycle

On this page

Managing the project life cycle

Was this page helpful? 

Why

When you finally deploy your project into Production, it marks the end of a cycle in your application. However, projects are not static and, most likely, require periodic maintenance and upgrading.

It is common that you receive recent and better data from your client, for example including human feedback with the final classification for transactions that Pulse original sent for review. Therefore, quickly closing the feedback loop is what allows your project to be up-to-date with how your business evolves, without falling behind due to cumbersome modeling, training, deployment and upgrading procedures.

You must be aware of the actions you need to undertake to upgrade your Pulse application, from Data Science stages to redeploying that work in your [Workflow](#), without breaking it or having to start from scratch.



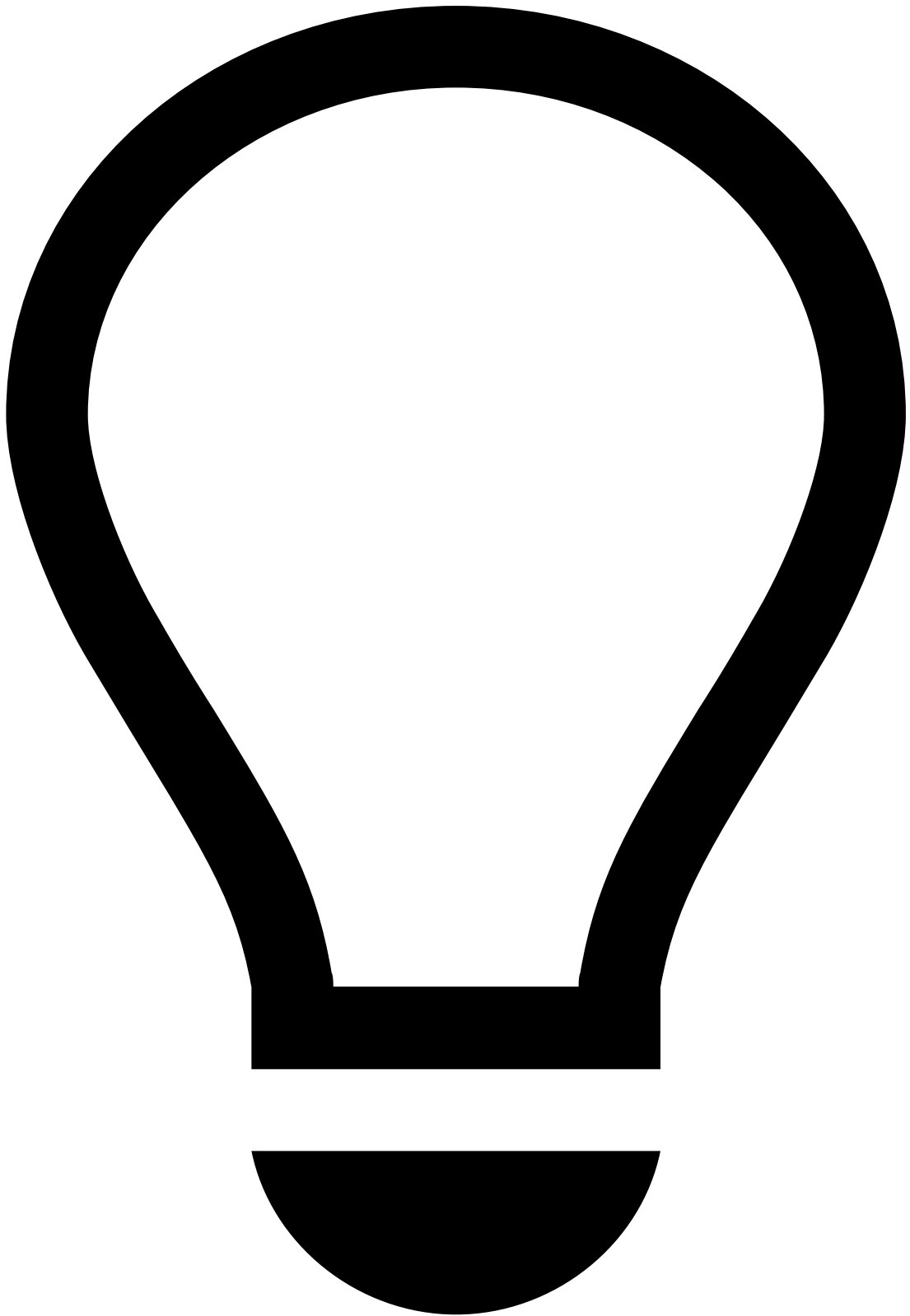
How Pulse projects work

Make sure you fully understand [how Pulse projects work](#) when you maintain and update your applications.

How to [📄](#)

Consuming new data and deploying new Models [📄](#)

When you need to take new data into account, you must repeat some of the Data Science tasks you have already completed before training your **Model** for the first time. Re-deploying the **Model** is also accomplished via the same procedure.



Consuming new data

The core of this task is to go through a new iteration of your [Data Science loop](#) and retrain your Machine Learning **Model** before you configure it in a **Scoring Service** in your **Workflow**.



Retraining Models

Pulse lets you push your new data through the exact same analysis and transformation pipeline you used when first training the current **Model**.

So if you only need to retrain a **Model**, with the exact same configuration as the current one, you can directly reuse the Data Science **Plans** you used before.

Nevertheless, when analyzing the new data you may come to the conclusion that you need to reengineer your features or configure the **Model** with different parameters.

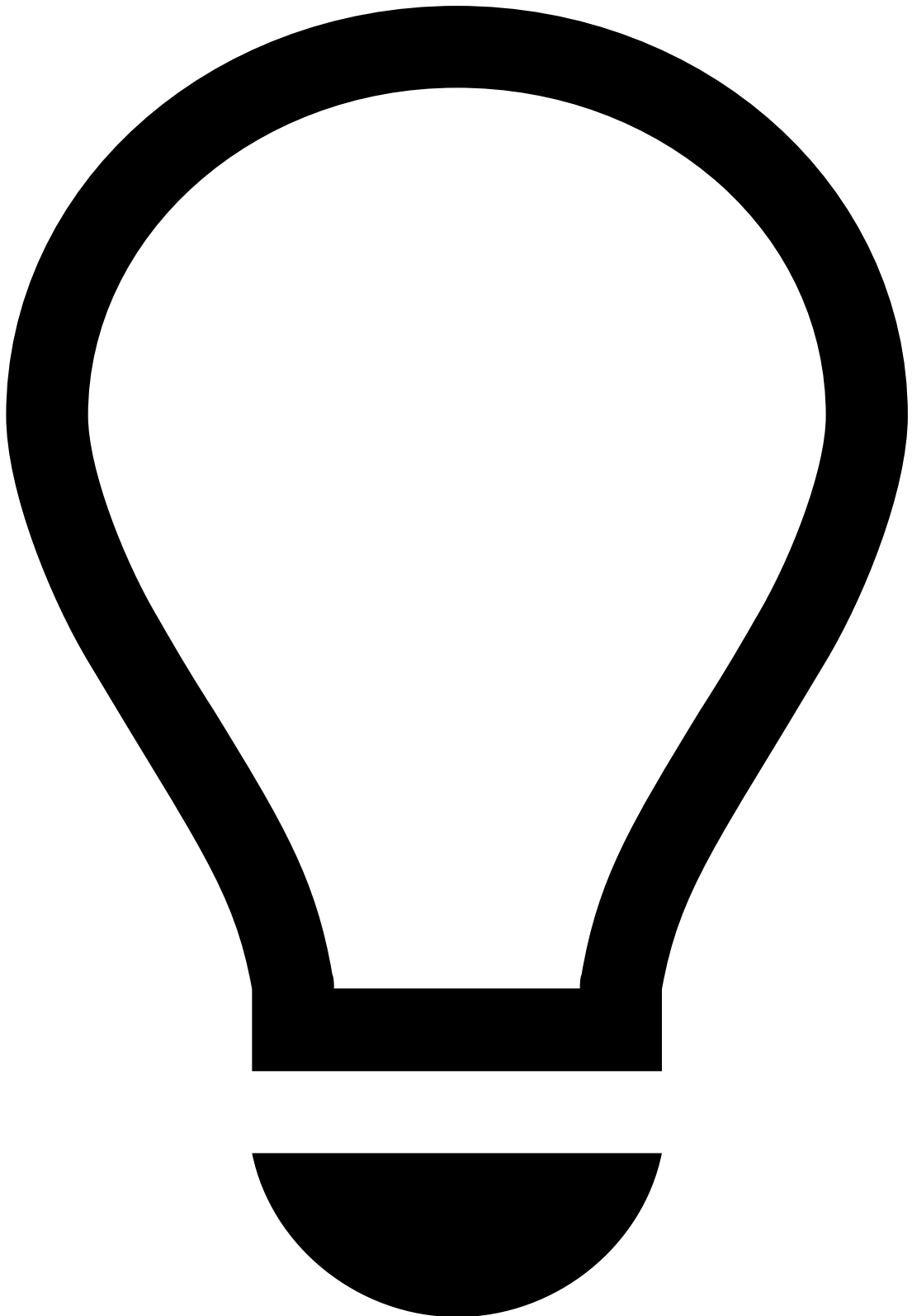
In this case, although it may be more complex as you need to create new **Plans** and evaluate new **Model** configurations, the process includes no additional/different tasks to the ones you first executed for the current **Model**.

To retrain your Machine Learning [Model](#) with improved/more recent data and deploy the new **Model** to Production, execute the following actions:

1. Run your new data through the same Data Science pipeline you have used before. This includes configuring [Data Sources](#) and transformations with [Plans](#).
 1. [Clean and load your new data](#) to create a new **Data Source** and obtain a high-quality dataset that adheres to your project's schema and requirements.
 2. [Understand and process your new data](#) to [analyze and sample it](#), and [engineer your features](#) to generate a dataset that you can use to retrain your **Model**.
 3. Optionally, compute extra information and new features for your [features/instances dataset](#).
2. [Build and evaluate a new Model](#).
 1. [Train a new Model](#) within your Model Project. In alternative, you can start a new **Model Project**.
 2. [Evaluate the new Model](#).
 3. [Compare your new Model](#) with the current one to assess your performance with the new data.
 4. [Adjust the Model parameters](#) and repeat the process until you reach your goal.
3. Deploy the new **Model** in your **Workflow** . [Edit an existing or configure a new Scoring Service](#) . Specify the new **Model** , and if applicable, specify a new **Model Project** .

Deploying new Rules

Adding new **Rules** or editing existing ones and taking them to Production is accomplished by going through some of the same tasks you have already completed when building and deploying your **Rules** for the first time.



Rules Snapshot

The core of this task is to create a new **Rules Snapshot** and configure it in a **Rules Service** in your **Workflow**.

To add a new [Rule](#) to your project, or adjust an existing one, and redeploy it to Production, execute the following actions:

1. [Build and evaluate your new Rule](#).
 1. [Define/Edit a Rule](#). In alternative, you can start a new **Rules Project**.
 2. Create a new **Rule Snapshot** that includes the new **Rule** and new configurations you may have added, such as **Templates** and **Environment** code.
 3. Optionally, [evaluate the Rule](#).

4. Repeat the procedure if you need to further adjust and re-evaluate the **Rule** within the rule block.
2. Deploy the new **Rule** in your **Workflow**.
 1. In your [runtime Workflow](#), [configure a new Outcome](#) if the new **Rule** contributes to a different decision than the ones already configured.
 2. [Edit an existing or configure a new Rules Service](#). Specify the new **Rule Snapshot** that includes your new **Rule**, and if applicable, specify a new **Rules Project**.

Next Steps

Your deployment is now complete. However, when you need to perform maintenance and update tasks to your Pulse applications revisit the relevant sections of this [User Guide](#).

•



- [Feature Engineering](#)
- Best Practices for Plans

On this page

Best Practices for Plans

Was this page helpful? 

Within a map or metric operator, you cannot refer to a field that has been added inside that same operator. In case of such dependencies, you have to use more than one operator to calculate the final field. Also, performance considerations need to be taken into consideration when organizing your maps and metrics into plans.

The following sections describe best practices regarding maps, metrics and plans.

A simple plan

Due to performance reasons, it is recommended to organize map and metrics operators in a single plan, containing all feature engineering logic.

In a simplified example, the plan could have the following operators:

1. A map operator to calculate simple features.
2. A metrics operator to calculate time-based aggregations.
3. Typically another map operator to calculate features based on the metrics.

Large number of metrics

It can happen that the plan has to calculate hundreds of metrics.

Due to limitations of Java, when a metrics operator contains around 200 metrics, the plan might crash. This number depends on the length of the metrics' PQL code. The solution for this is to separate the metrics into two or more operators.

When separating the metrics into two or more different metrics operators, it's a good practice to keep metrics in the same operator if they have the same field in the first place in the *Group By* field. For example, if you have some metrics that group events by *Card*, and some other metrics that group events by *Card* and *ShippingAddress*, then you can keep these metrics in the same operator to speed up the computation.

This way, Spark has to do only one shuffle for those metrics, which is usually the most time-consuming part of Spark computations. However, this technique has the cost of higher memory usage.

Scheduled metrics

When you have scheduled metrics in a plan, Pulse has to recompute everything up to that point in the plan. If you have a lot of computations before the scheduled metrics operator, it might create a big overhead in computation time. This is usually a non-issue since scheduled metrics are normally computed during the night and are not subject to the real-time constraints. Nevertheless be aware of this in case you think your computations will be taking longer than a night in production.

As an exception to the one plan rule, having two plans in this situation can be a trade-off between storage space and computational times.

Addressing errors in plan definition

When defining the components of a **Plan**, you may encounter errors that would result in failures during the [plan execution](#).

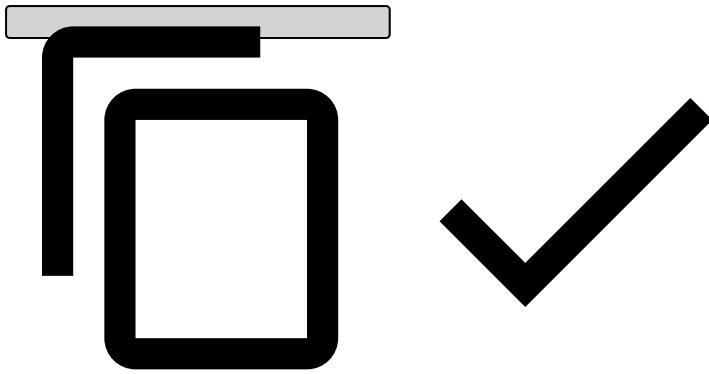
Some of those errors are detectable when defining the Plan, and Pulse will show warnings to allow them to be fixed before deploying the plan to production.

One such error is defining [fields](#) based on a [PQL expression](#) (e.g. within a [Metrics or Map operator](#)). Operators that require null coalescing (i.e. [default fallback values](#)) will produce a warning in the UI if the coalescing operator (`??`) and value are missing:

To fix this situation, provide a fallback value in the PQL expression input field, e.g. `?? 0` or `?? ""`.

A similar warning will be shown if built-in functions are used in collections (like `List` s, `Set` s or `Map` s). For example, an expression like

```
["test", "test1", "test2"].contains("testing")
```

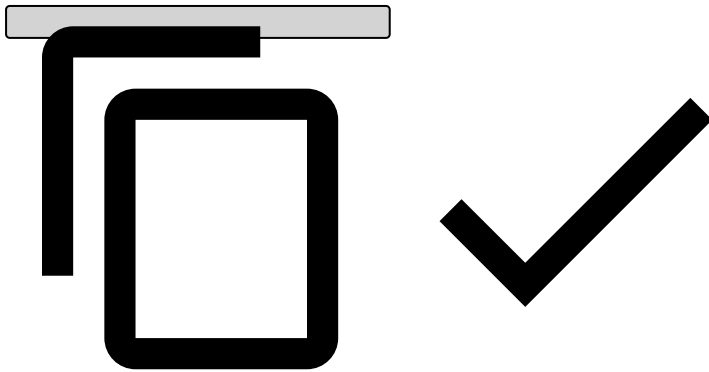



will fail because the `contains` function is not supported in collections used in the PQL fields of Map Operators or Metric Operators.

Attempting to use such functions in that context will result in a warning in the UI:

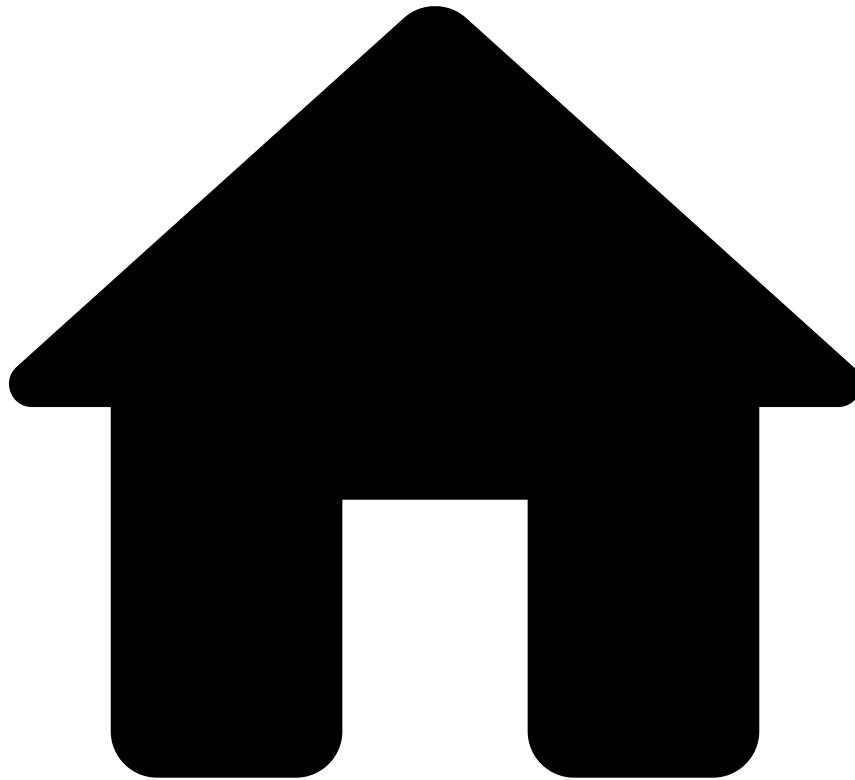
The alternative is to use [UDFs \(User Defined Functions\)](#), e.g.:

```
com.feedzai.pulse.pkernel.runtime.CollectionUDF.contains(["test", "test1", "test2"], "testing")
```



Besides these preventive warnings, Pulse will also allow recovering from execution errors that are not detectable at design time. Refer to [Automatic Application Repair](#) and [Workflow recovery](#) for more information on how to recover from execution errors.

•



- [Feature Engineering](#)
- Creating a Plan for Features

Creating a Plan for Features

Was this page helpful? 

In Pulse, a plan entity defines a transformation flow of one data source into another data source. In a plan it is possible to add, update and remove fields, or calculate metrics on the data.

To create a plan, execute the following steps.

1. In the **Data Science** tab, add a plan.
2. Add a **Name** for the plan.
3. Select the **Deployable** option. This option ensures that you can deploy your plan to another environment (e.g. QA or Production) later on.
4. In the **Schema** field, specify the data schema that the plan is built on. You can select an existing schema, or an existing data source.

For example, if you select the* Data with Features for Models* data source, then the plan will be based on a copy of the current schema of that data source.

5. Use the **Confirm** button to acknowledge that future changes in the origin schema won't change the schema of the plan.
6. In the **Timestamp Field**, select the schema field that represents the timestamp of the events. When Pulse aggregates the data over a time period, it will consider this field as the timestamp of the event.

Only fields of [timestamp](#) type can be selected.



info

If you don't select a timestamp field for the plan, you won't be able to create metrics in the plan later on.

7. If you want to use data from more than one channel, select the **Omnichannel** option, and configure the channels as follows:
 1. Add a **Name** for the **Main Channel**. This channel contains the events that you want to process, and uses the schema selected in step 4.

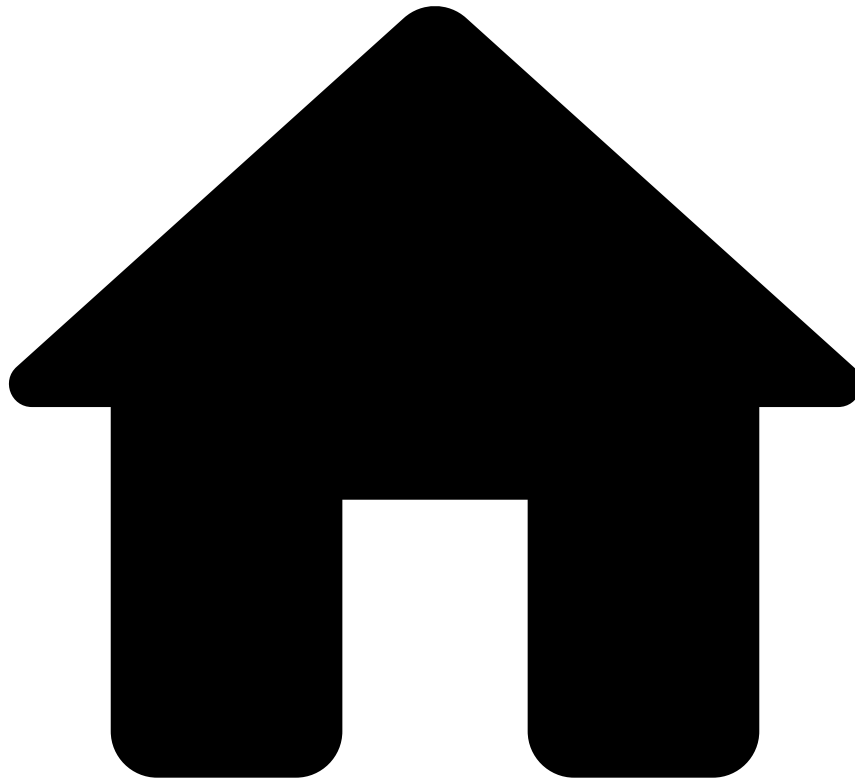
2. **Secondary channels** provide additional information that you can use to process the events of the main channel. You can use the **Add Channel** button to add secondary channels.

For each secondary channel, add a **Name**, a **Schema** and the **Timestamp Field** that that channel will use.

There should be at least some fields that can be cross-referenced between the channels. For example, both schemas might include a customer identifier, even though it might be actually called differently in each schema.

8. **Save** the plan settings.

•



- [Feature Engineering](#)
- Creating Derived Features

On this page

Creating Derived Features

Was this page helpful? 

This page describes how to edit and run an existing plan that creates a new data source with new features.

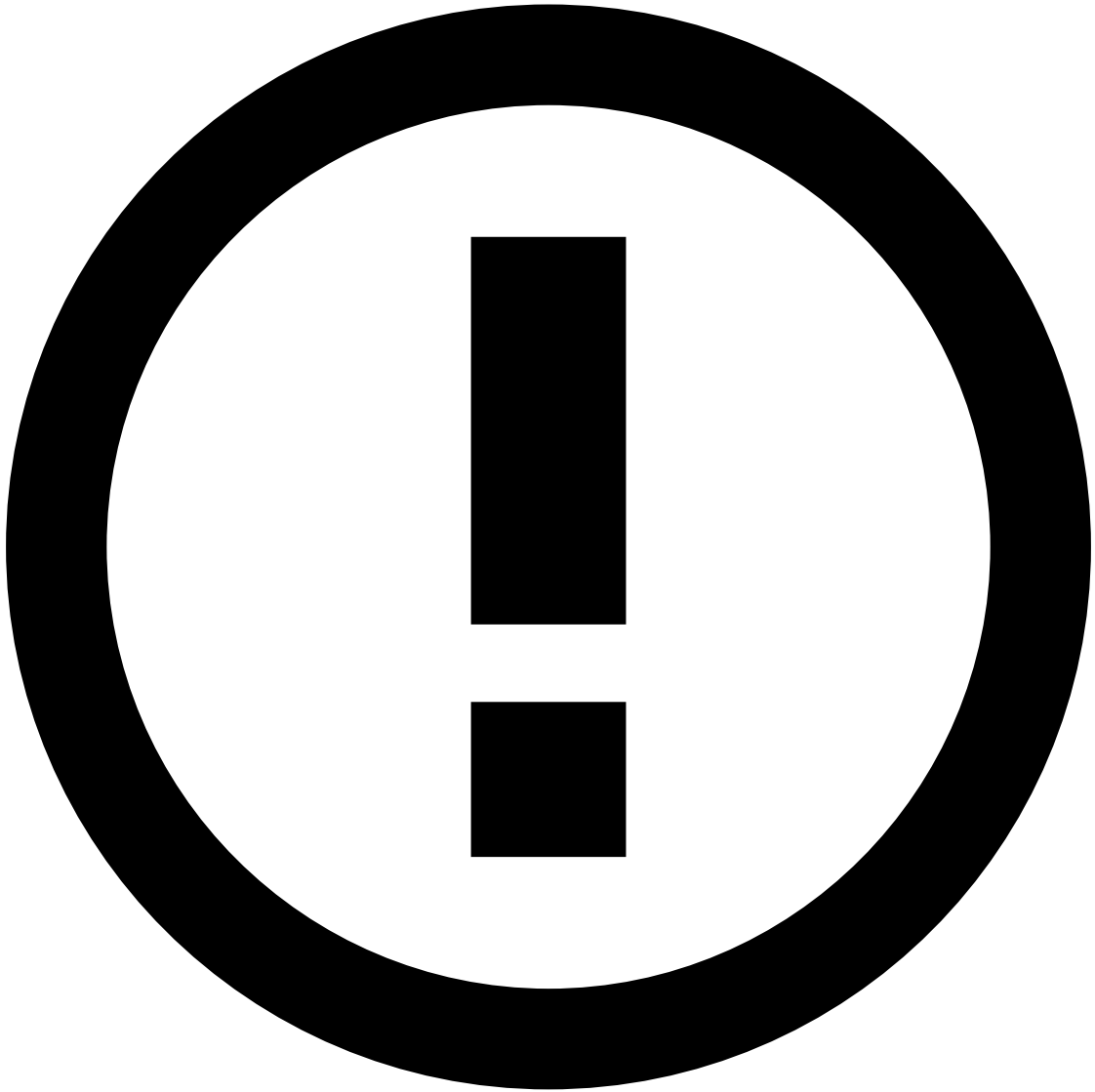
In the example shown, you can see how to add a new field that is calculated based on a single transaction. If you want to create a feature that is based on many aggregated transactions, you must use [metrics](#), i.e. historical profiles – refer to the page on [creating metrics](#).

Adding the features

To add new features, execute the following actions:

1. In the **Data Science** tab, open the features plan.

2. Drag a **Map** operator to the plan if there is none yet.



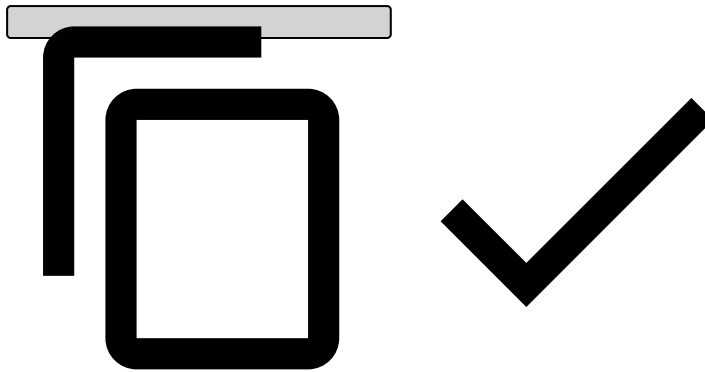
Clipboard

To save time, you can use the clipboard to copy-paste operators in plans. This allows you to add operators without the need of specifying all their fields each time you create a new plan.

3. Edit the map as follows:

1. Select the **Fields to Add** tab.
2. In the **Output Field Name** column, specify the name of the new feature.
3. In the **Expression** column, write the [Pulse Query Language expression](#) that calculates the output value.
For example, the following expression calculates whether the credit card was present at the transaction:

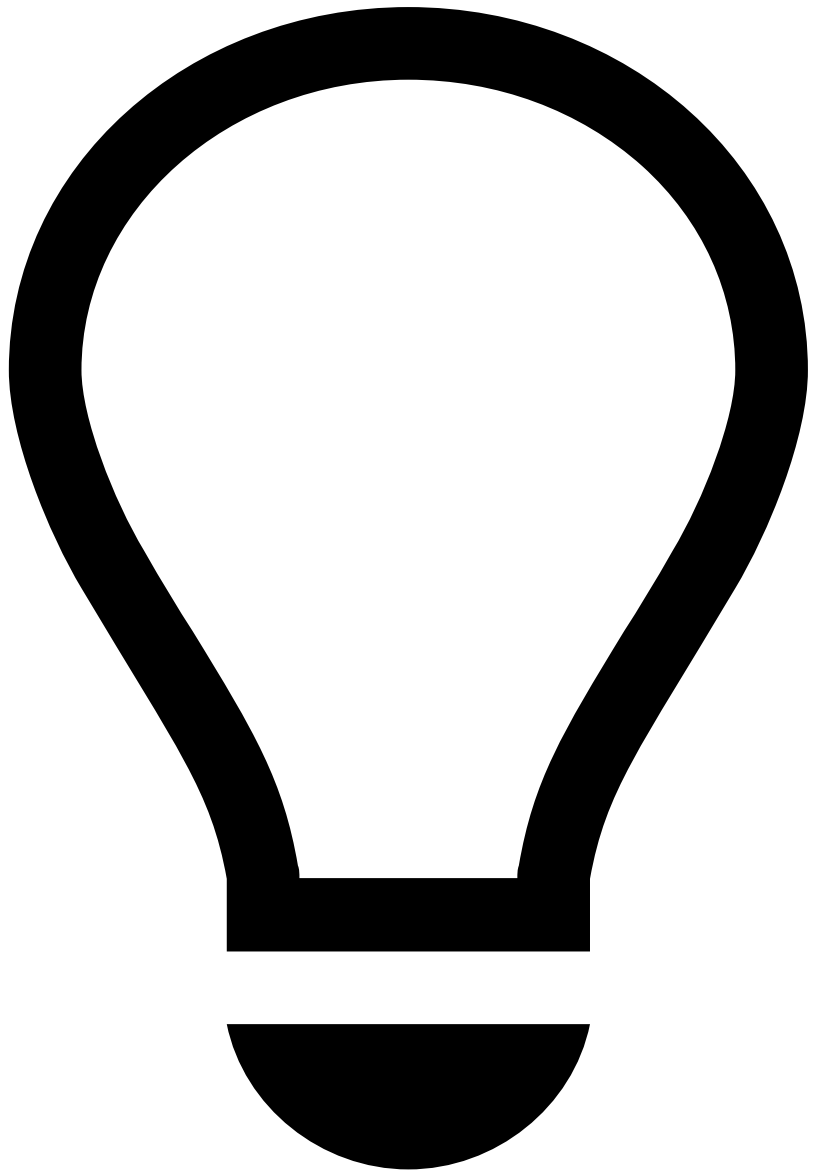
```
if (POSEntryMode == 'Swiped' or POSEntryMode == 'Keyed' or POSEntryMode == 'Chip')  
then 'true' else 'false'
```



4. Define the **Type** of the output field. In the example of calculating whether the credit card was present at the transaction, the type of the output field is *Categorical*, with two possible values: *true* and *false*.
5. Confirm that you want to add the field to the output schema, using the add button.
6. Add all the features to the map operator by following the previous steps.
7. **Save** the map operator.

Creating the data source with the features

1. To create a new data source which includes the new field(s), execute the following actions:
 1. With the plan still open, use the button to execute the plan.
 1. In the **Data Source** tab, select the data source that will be the input for the plan. For example *Cleaned Transactions*, or *Model Features v1*. If the plan has more than one channel, select the input data source for each channel.
 2. In the **Data Sinks** tab, configure the output of the plan:
 - **Name** for the output data source. For example, *Rule Features v1*. **Type**: *Parquet* (default) or *CSV*, depending on what type of output file you want to generate.



Parquet format is recommended

Using the Parquet file format for data files is highly recommended. Besides allowing a direct encoding of data types (which automates the mapping of field types when importing datasets), it also improves performance and disk usage: compared to CSV, Parquet files occupy less space and take less time to load.

- **Directory Path:** the path and file name where Pulse will store the output file, containing the data of the new data source. Example: `file:///datasets/rule-features-v1.parquet` . Note that the format varies according to the operating system.

3. Use the run button to execute the plan. It generates a new data source in Pulse, named *Rule Features v1*. Pulse also saves an output file with the transformed data to your filesystem.

2. Check the new data source to confirm that its schema includes new fields.

-



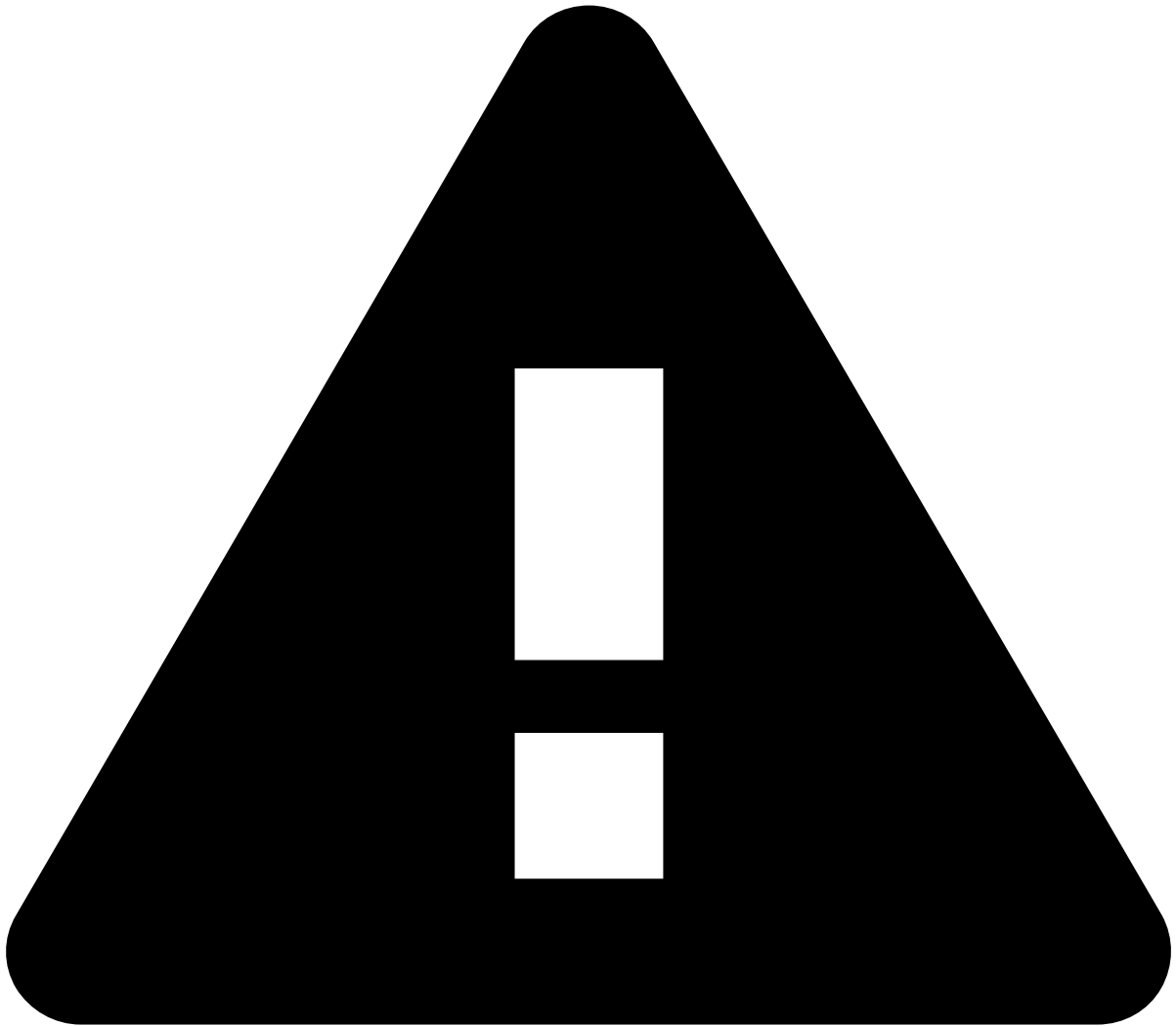
- [Feature Engineering](#)
- Creating Metrics

Creating Metrics

Was this page helpful? 

You can create new fields to augment a [data source](#) based on an aggregation of events over a period of time. These kind of features are called [metrics](#), or **historical profiles**.

Metrics describe typical behavior. For example: this customer typically uses two devices, and makes one transfer per month to pay the rent. Certain behavior patterns, or changes in the behavior, may help you detect financial crime.

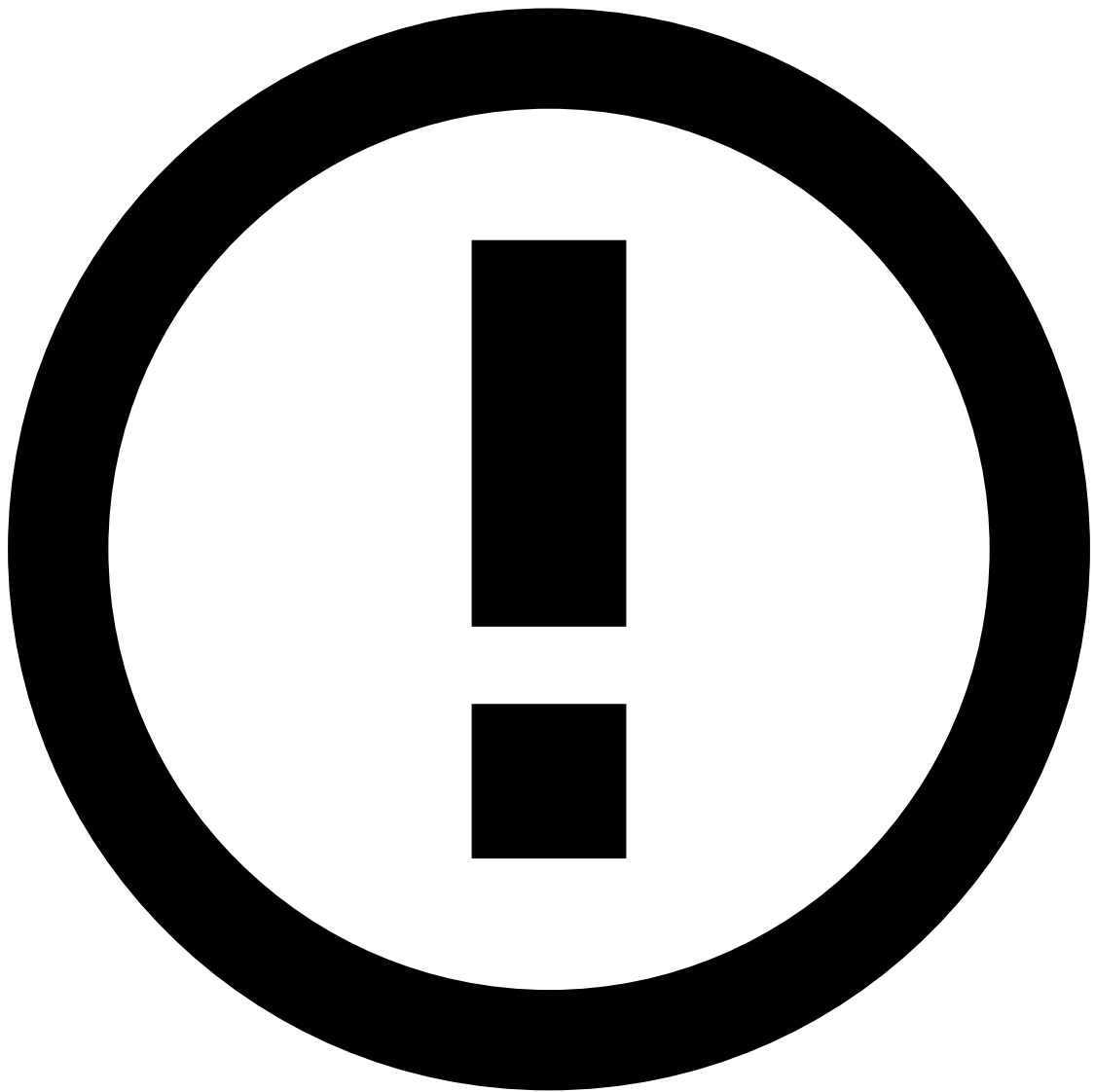


Metrics operator limits

For performance reasons, there are limits to the number of Metrics operators that can be added to a Plan. For more information about this, see [Limits](#).

To add new metrics, execute the following actions:

1. In the **Data Science** tab, open the features plan.
2. Drag a **Metrics** operator to the plan if there is none yet.

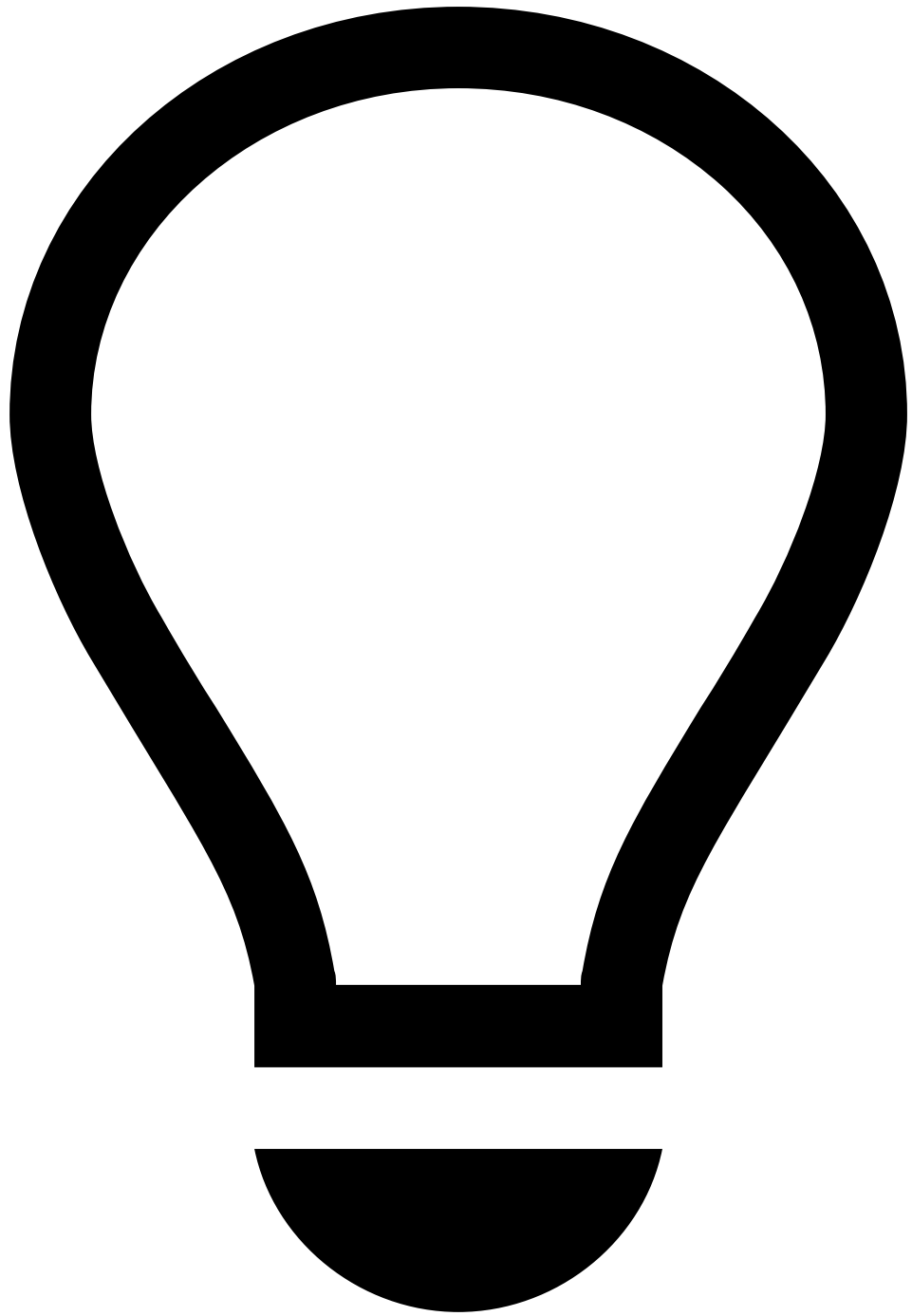


Adding metrics operators

To save time, you can use the clipboard to copy-paste operators in plans. This allows you to add operators without specifying all their fields each time you create a new plan.

3. Edit the metrics operator as follows:

1. Add an **Operator Name** for the metrics operator.
2. Add a new metric to the operator. Later, when you run the plan, Pulse will add this metric to the output data source as a new field.
3. Set the **Group By** fields over which to calculate aggregated metrics. For example:
 1. *AccountID*, to aggregate metrics for each card number.
 2. *MerchantID*, to aggregate metrics for each merchant.
 3. *AccountID*, *MerchantID*, to aggregate metrics for the profile of each card number in each merchant.
4. If relevant, set the **Lookup By** fields (matching the count and types of the **Group By** fields) to define which group to retrieve.



tip

For more detailed information on setting up Lookup By metrics, refer to the [Using Lookup Metrics](#) guide.

5. In the **Window** section, configure the time window over which to calculate the metric.

1. In the **Expression** field, add the time interval that you want to include. For example, *1 hour* or *7 days*.

This means that you want to calculate a metric based on the events of the last hour, or of the last seven days.

For details on the syntax supported in window expressions, please refer to the [Time window](#) section of the documentation on Plans in Workflows.

2. If you use the **Delay** field, recent events will not be immediately accounted for. For example, a 3-hour window with 1-hour delay means that out of the last $3+1=4$ hours of events, the last 1 hour

will not be considered. This is useful if you are computing a historical baseline profile and don't want it to be influenced by recent behavior.

3. **Is tumbling:** tumbling windows expire all events at once, once they reach their predetermined size. Afterwards, they start filling again and the cycle repeats itself.
4. You can define a **Filter** for the time window. Only events that fulfill the filter condition will be taken into account when calculating the metric. Use [Pulse Query Language \(PQL\) expressions](#) returning a boolean value to define the filter.
6. In the **Edit Update Period** section, choose if the metric should be updated in real time, or periodically on a schedule.

1. **Real Time:** ideal for metrics with a small window (e.g. less than one day), to ensure that rapid attacks are detected.

Under Real Time, select the *Update on every event* option. The *Update every* option is available for legacy reasons and should not be selected. This option is currently not accepted for omnichannel plans. It will be supported in future versions of Pulse.

2. **Scheduled:** ideal for metrics with a large window (e.g. 1 month). Since there is less fluctuation in the long-term behavior, the metric can be updated less frequently. To specify when the metric should be updated, you can either set *Start of* or *Scheduled*.

In the latter, you must type a [Cron expression](#), e.g. `0 0 12 * * ?` (i.e. *At noon every day*).

7. In the **Aggregation** section, select the **Function** you want to use to calculate the metric. If the function needs a parameter, select the **Field** that provides that parameter.



Note that storing the state of metric on disk is in continuous development, and it does not provide full feature parity to the previous, RAM-based version.

Examples of functions:

- To calculate the sum of the *TransactionAmount* field over each time window, for each *group by* field: the function is *Sum*, and the parameter field is *TransactionAmount*.
- To calculate the number of the transactions over each time window, for each *group by* field: the function is *Count*, and it has no parameters.

8. Optionally, in the **Advanced** tab of the Aggregation section, you can define a [Pulse Query Language \(PQL\) expression](#). This tab allows you to compute more complex values. When calculating metrics in a plan that is based on more than one channel, the Advanced tab is not available.

9. If your plan has more than one channel, do the following:

1. Toggle off all secondary channels that are not relevant for the metric using the toggle next to the channel's name in the Group By section.
2. You can decide not to include main channel data in the aggregation. To do so, select the **Do not include main channel data in the aggregation** option. Select this option when the main channel has no data relevant for the aggregation. For example, when processing debit card transactions, it might be useful to know how many devices the customer has used. This information can be aggregated from the online banking channel. But unlike online banking, the main channel (debit cards) will not contain any information about devices.
3. After selecting the *group by* field(s) for the main channel, select the matching field(s) for each active secondary channel. If there is more than one *group by* field, the order of the fields **must** match across the different channels.
4. If you selected an aggregation function that has parameters, select the parameter fields for each channel.
5. Note that only scheduled metrics can use data from secondary channels. If you create a real-time metric with data from secondary channels, you will receive an error message when saving the metric.

10. In the **Details** section, add a description for the metric.

4. Use the **Save** button to add the metric to the operator.

5. The metrics get a default name. You can hover over a metric and use the pencil button to rename it.

-



- [Feature Engineering](#)
- Generating the Features Dataset

On this page

Generating the Features Dataset

Was this page helpful? 

Typically, you want to train your machine learning model on a data source that contains derived fields and metrics. You can achieve this by building up your feature plan(s) and generating a feature data source.

This page gives you instructions on how to create such a data source.

Preparing the plan

To produce your feature dataset, compose a plan as described in the following pages:

- [Designing derived features](#)

- [Designing metrics \(historical profiles\)](#)

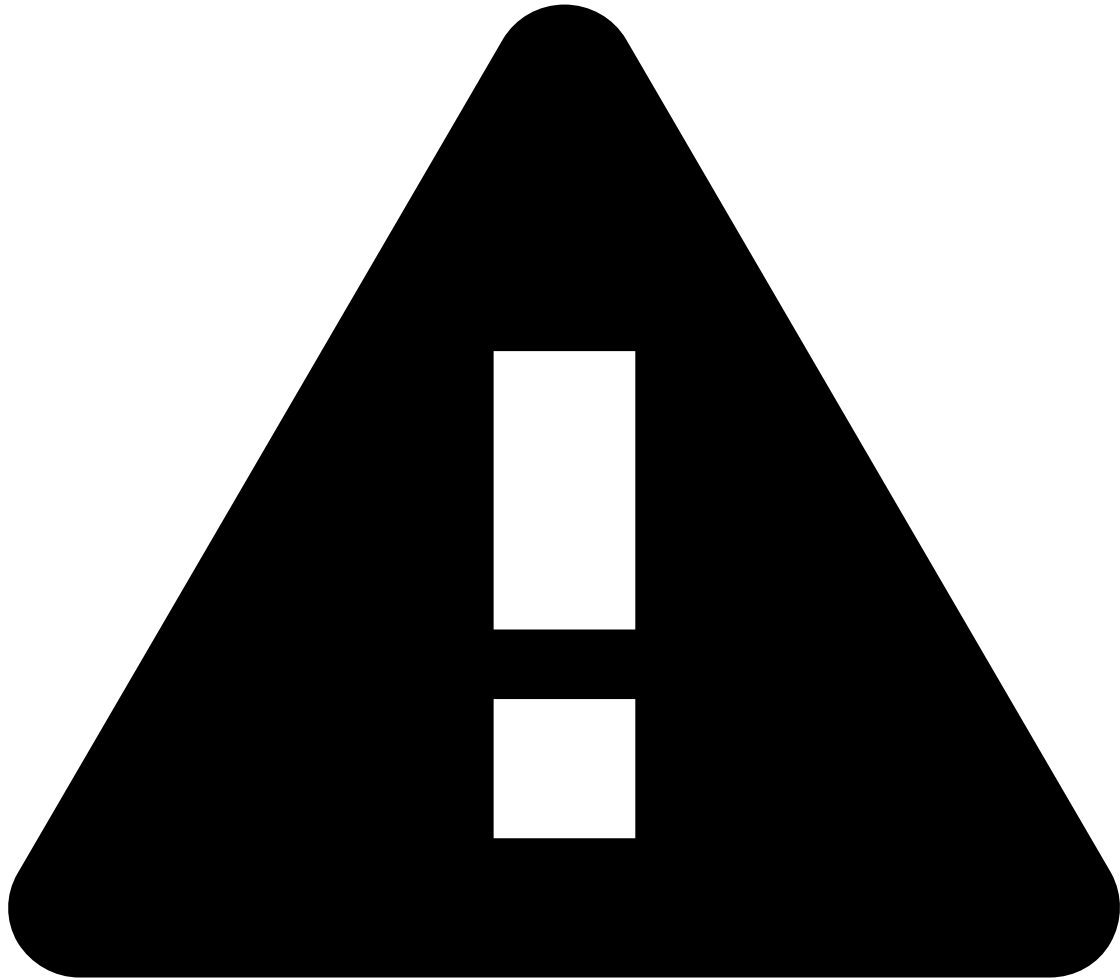


Target field

Real-time data, coming into Pulse in Production, will not include the target field you aim at predicting during the data science stage. In fact, in a workflow, you configure that target as an outcome that represents Pulse's decision for each instance.

Before deploying your feature plan to runtime, you may want to:

1. Edit the map operator to remove the target field from the plan's output data source.
2. Export the output data source to generate a schema without the target field.

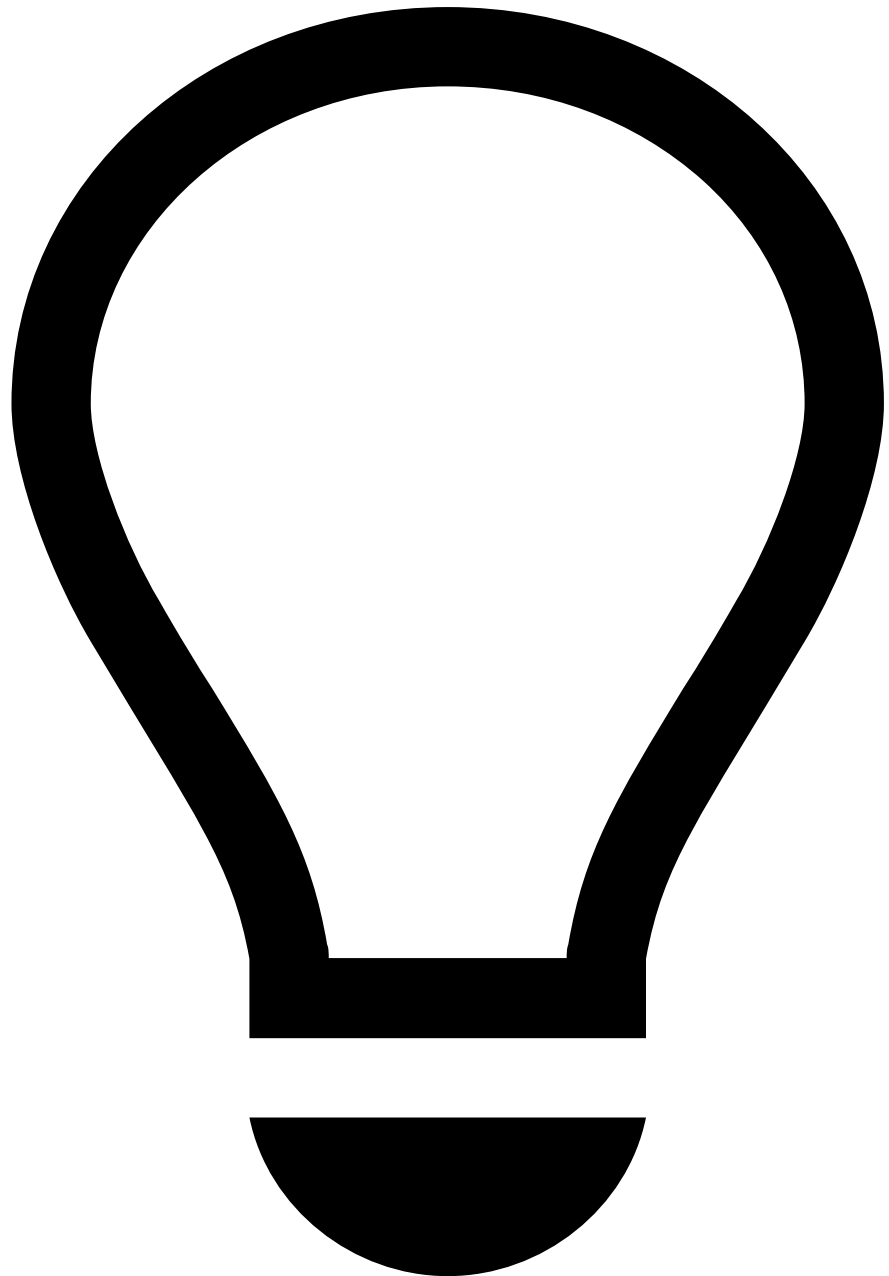


Field dependencies

An [operator in a plan](#) that can add fields to the schema cannot have intra-operator dependencies. In map and metrics operators, you cannot refer to a field that has been added inside that same operator.

Running the plan to generate the enhanced data source

1. Use the run button to execute the plan.
 1. In the **Data Source** tab, select the data source that will be the input for the plan. For example *Cleaned Transactions*. If the plan has more than one channel, select the input data source for each channel.
 2. In the **Data Sinks** tab, configure the output of the plan:
 - **Name** for the output data source. For example, *Features v1*.
 - **Type**: the file format of the output data  *Parquet* (default) or CSV.



Parquet format is recommended

Using the Parquet file format for data files is highly recommended. Besides allowing a direct encoding of data types (which automates the mapping of field types when importing datasets), it also improves performance and disk usage: compared to CSV, Parquet files occupy less space and take less time to load.

- **Directory Path:** the path and file name where Pulse will store the output CSV file, containing the data of the new data source. Example: `<file:///datasets/rule-features-v1.parquet>`. Note that the format varies according to the operating system.

3. Use the run button to execute the plan. It generates a new data source in Pulse, named *Features v1*. Pulse also saves an output file with the transformed data to your filesystem.

2. Check the new data source to confirm that its schema includes new fields.

Designing the final feature dataset

There are multiple approaches when using plans to obtain your final feature dataset. The one you choose comes down to how it best fits your requirements and work methodology.

Single plan with one map

The most straightforward approach is encapsulating all feature engineering logic into one map operator. This is the simplest approach and the one proposed in the current topic.

The following image depicts a simple features plan where all features are designed in the metrics operator and a single map operator.

Advantages

- Straightforward

Disadvantages

- Hard to maintain and reuse. With the number of features easily rising to hundreds, it is very hard to keep track of all features, group them by type, and generally keep them clean and organized in the same map.

Sequential plans

A more segmented approach consists in using isolated plans to sequentially manipulate data, add features and metrics to datasets. In this case, you have to manually run each plan and manage how all intermediate data sources are consumed and produced.

For example, having separate plans to [design metrics \(historical profiles\)](#) and to [design derived features](#), that you manually run one after the other.

The following image depicts a composition of four separate plans. Each plan generates different features/historical profiles, taking as input the data source that the previous plan produced.

Advantages

- Feature engineering logic is isolated by plan, improving maintainability and reuse.

Disadvantages

- Hard and time-consuming to execute and reproduce the entire sequence. You have to manually execute each intermediate step, such as running each plan. You also must be extremely careful to ensure that you chain them together with the correct data sources to be consumed and produced.

Single plan with several maps

Another possibility, which Feedzai **recommends**, is using a single plan that contains all Feature Engineering logic split among several map operators. Each map adds different types of features to the dataset, effectively isolating logic while keeping complexity under control.

The following image displays an example of a features Plan where all features are grouped by type. Note that auxiliary fields are removed in the end and that all features that do not depend on historical profiles are designed before the metrics operator.

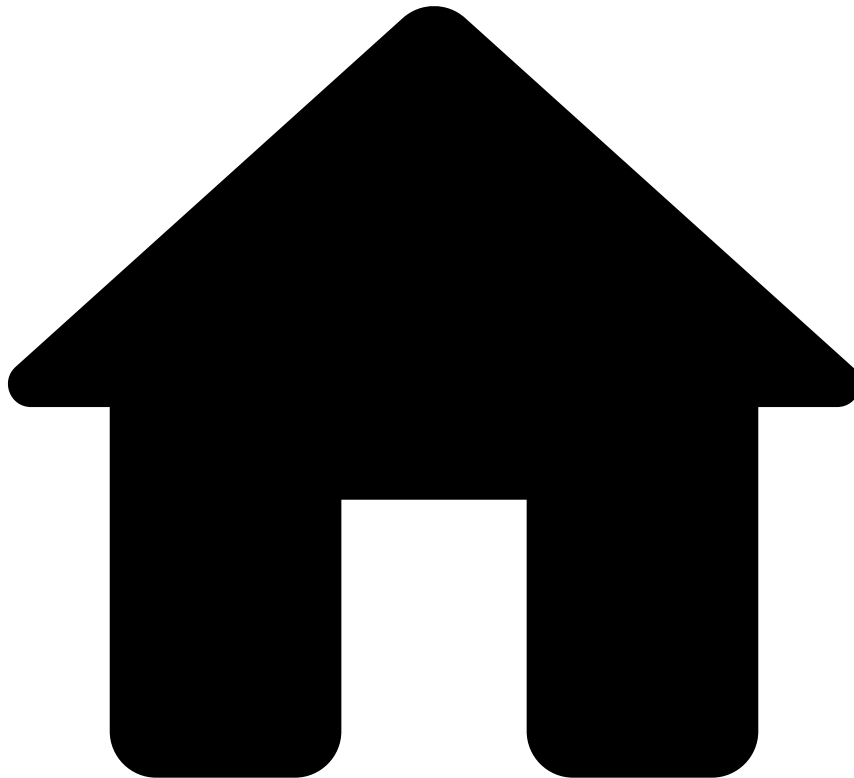
Advantages

- All feature engineering logic is contained in the same plan, simplifying the process. There is no need for manual, intermediate, actions and datasets.
- Feature engineering logic is isolated by map, improving maintainability and reuse for each type of features. Even when the number of features becomes too large, they are still grouped by type. This allows you to:
 - Easily keep track of related features.
 - Know where each type of features is being generated.
 - Work on each type of features at a time and iteratively test and build up the plan.
 - Easily detect possible dependencies among features. For example, time-based features are derived from historical profiles.

Disadvantages

- The plan, although linear, becomes more complex with a lot more steps.

-



- [Feature Engineering](#)
- Managing Metrics Backfills

On this page

Was this page helpful? 

Managing Metrics Backfills

The **Backfills** management area allows you to monitor, manually trigger, and manage [backfill jobs](#) for your [metrics](#). While backfills are automatically triggered when you publish changes to metrics, this area gives you more control over the process.



Backfills Management Area

Here, you can:

- Manually start a backfill for specific metrics in a workflow.
- Monitor the status and progress of all backfill jobs (both manual and automatic).
- Cancel backfill jobs that are no longer needed.

Triggering a new backfill job

To manually trigger a backfill job, follow these steps:

1. Navigate to the **Backfills** tab in your application.
2. In the "Trigger Backfill" section, use the dropdown menus to select the metrics you want to backfill:

Trigger Backfill form

- **Workflow:** Choose the [workflow](#) containing the metrics. You can only select one workflow at a time.
- **Plan Services:** Select one or more [plan services](#) within the chosen workflow.
- **Metrics Operators:** Select one or more [metrics operators](#) from the selected plan services.
- **Metrics:** Select the specific [metrics](#) to be backfilled.

For Plan Services, Metrics Operators, and Metrics, you can use the "Select All..." option to include all items in the list. All fields are mandatory.

3. Select the **Trigger Backfill** button.

A new backfill job will be created and added to the queue, visible in the "Triggered Jobs History" table underneath. Once it is picked for execution, the associated Pulse job(s) will be displayed in the **Jobs** tab:



Backfill jobs in the Job Executions table

The jobs listed in this view can be matched to the corresponding backfill jobs in the **Backfills** tab via the **Name** column in both tables, which displays the full hash of the backfill job in the former, and a 6-character shortened version in the latter.

Monitoring backfill jobs

The "Triggered Jobs History" table provides a detailed overview of all backfill jobs. For each backfill job, this table displays the following information:

Column	Description
Name	The name of the job, composed of the name of the associated workflow and a 6-character shortened hash in parentheses e.g., workflow123 (az5LYD) .
Type	The type of job, which is always BACKFILL .
Status	The current status of the job. See the list of possible statuses below .
Description	A brief description regarding the nature of the job (e.g., "Backfill triggered manually via Pulse UI" or "Backfill created automatically from publish").
Triggered by	The user who initiated the job. For automatic backfills, this will be "system".
Triggered at	The timestamp when the job was created.
Started at	The timestamp when the job execution began. This may differ from "Triggered at" if the job waited in the queue, and will show "No info available" if the job is still queued.
Finished at	The timestamp when the job was completed. It shows "No info available" for running jobs.
Updated at	The timestamp of the last status update for the job.

This table is updated in real time, so it can be used to monitor the backfill jobs' details and progress. The most relevant columns to track the progress of a backfill job are:

- **Status:** Indicates the current state of the job (e.g., SCHEDULED , EXECUTING , FINISHED). Refer to the full list of possible statuses [below](#).
- **Started at** and **Finished at:** Show when the job began and completed, helping to estimate duration.
- **Updated at:** Reflects the last time the job's status changed, useful for tracking recent activity.

Backfill job statuses

The possible statuses for a backfill job, and the corresponding meanings, are as follows:

- **SCHEDULED** : The job is in the queue, waiting to be executed.
- **SKIPPED** : The job was skipped due to [automatic optimization](#).
- **STARTING** : The job has been picked up from the queue and is being prepared for execution.
- **EXECUTING** : The job is currently running.
- **FINISHED** : The job has completed successfully.
- **STOPPING** : The job has been [manually canceled](#) and is in the process of a graceful shutdown.
- **CANCELLED** : The job was successfully stopped after a manual cancellation.
- **FAILED** : The job terminated due to an error.

Viewing backfill job details

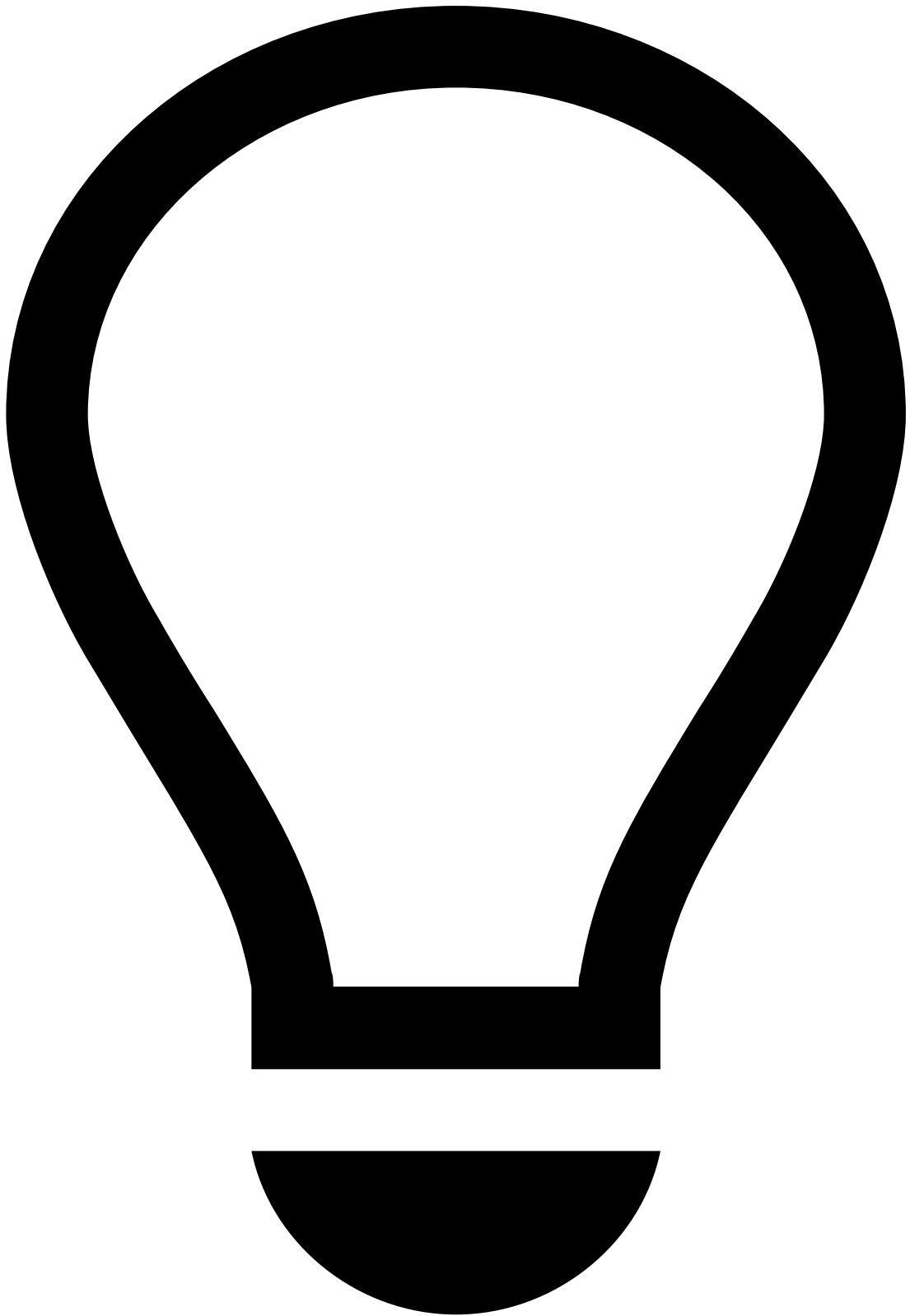
Besides monitoring the job's status, you can click on a backfill job's name in the "Triggered Jobs History" table (or the **Expand row** icon at the right edge of the row) to view more detailed information about that specific job, including its full ID (i.e. its unabbreviated hash), which metrics are being backfilled, which plan services and operators are involved, and what workflow they belong to.



Expanded row of the Triggered Jobs History table, showing details of a backfill job

In this expanded view, the Workflow and Plan execution boxes contain links that you can use to navigate to them.

There is also a **Copy to clipboard** button next to each box, that allows copying the corresponding element name.



tip

In the metrics boxes, using the **Copy to clipboard** button will add a comma-separated list of metric names to the clipboard. This list can then be pasted in the metrics dropdown to schedule a new backfill, as a more convenient way to select multiple metrics.

Canceling a backfill job

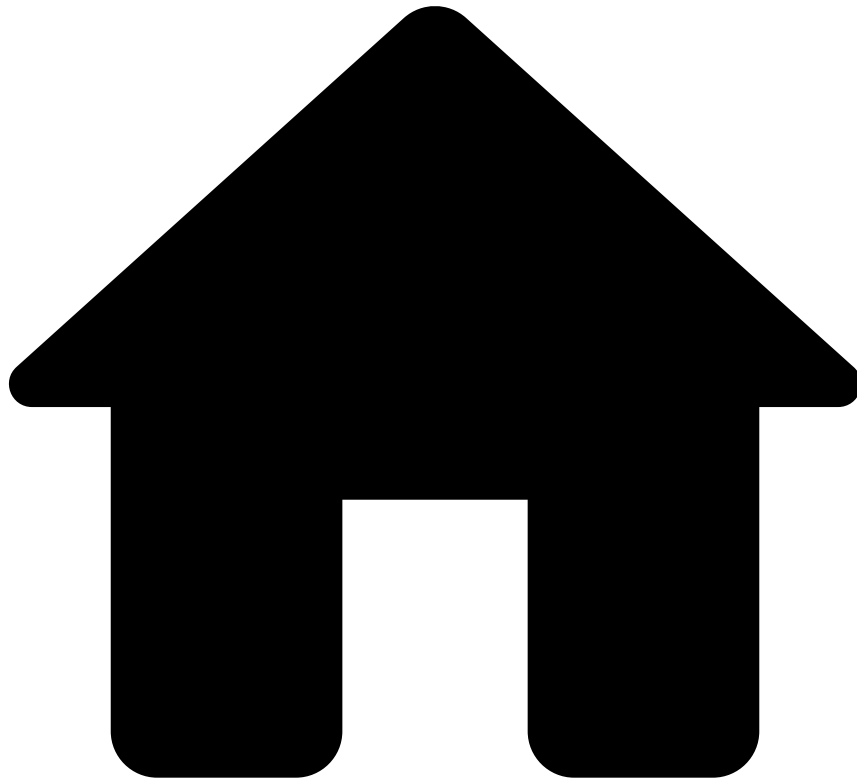
You can manually cancel a backfill job if it is in the `SCHEDULED` , `STARTING` , or `EXECUTING` status. This is useful if you need to prioritize other jobs or if a backfill is no longer necessary. Refer to the [best practices for managing the queue of backfill jobs](#) for more information on when to manually cancel jobs.

To cancel a backfill job:

1. Locate the job in the "Triggered Jobs History" table.
2. Click the **Cancel Backfill**  button in the **Actions** column for that job.

A confirmation message will appear, and the job's status will change to `STOPPING` while it cleans up. Once the process is complete, the status will update to `CANCELLED` .

•



- [Feature Engineering](#)
- Selecting Important Data for Your Model

On this page

Was this page helpful? 

Selecting Important Data for Your Model

If you want to use models to score events, as part of your feature engineering work you need to find out which fields in your clean data source relate directly with fraud (or any other target variable). This will help you decide which fields to use in your models.

To examine your data source and discover which fields you can directly use as model features, execute the following actions:

1. In the **Data Science** tab, select your data source from the **Data Sources** list.
2. Use the option to generate statistics for the data source. Select which are the breakdown fields and values to slice the results into. For example, the *Fraud* field.

3. Use the **View Statistics** option for the field you want to inspect, to go in depth into the generated insight. For example, you can select the *TransactionAmount* field. When looking into the analysis details to understand the data, you can do the following:
 1. Use the **Breakdown Fields** dropdown to break down the statistics by *Fraud*.
 2. Examine results for both **Breakdown Values**. Usually, fraud is correlated with higher transaction amounts. Thus, if when you look at statistics for fraudulent transactions you find that the *TransactionAmount* is generally higher than for non-fraudulent ones, then it is likely that the *TransactionAmount* field will make sense as a feature.

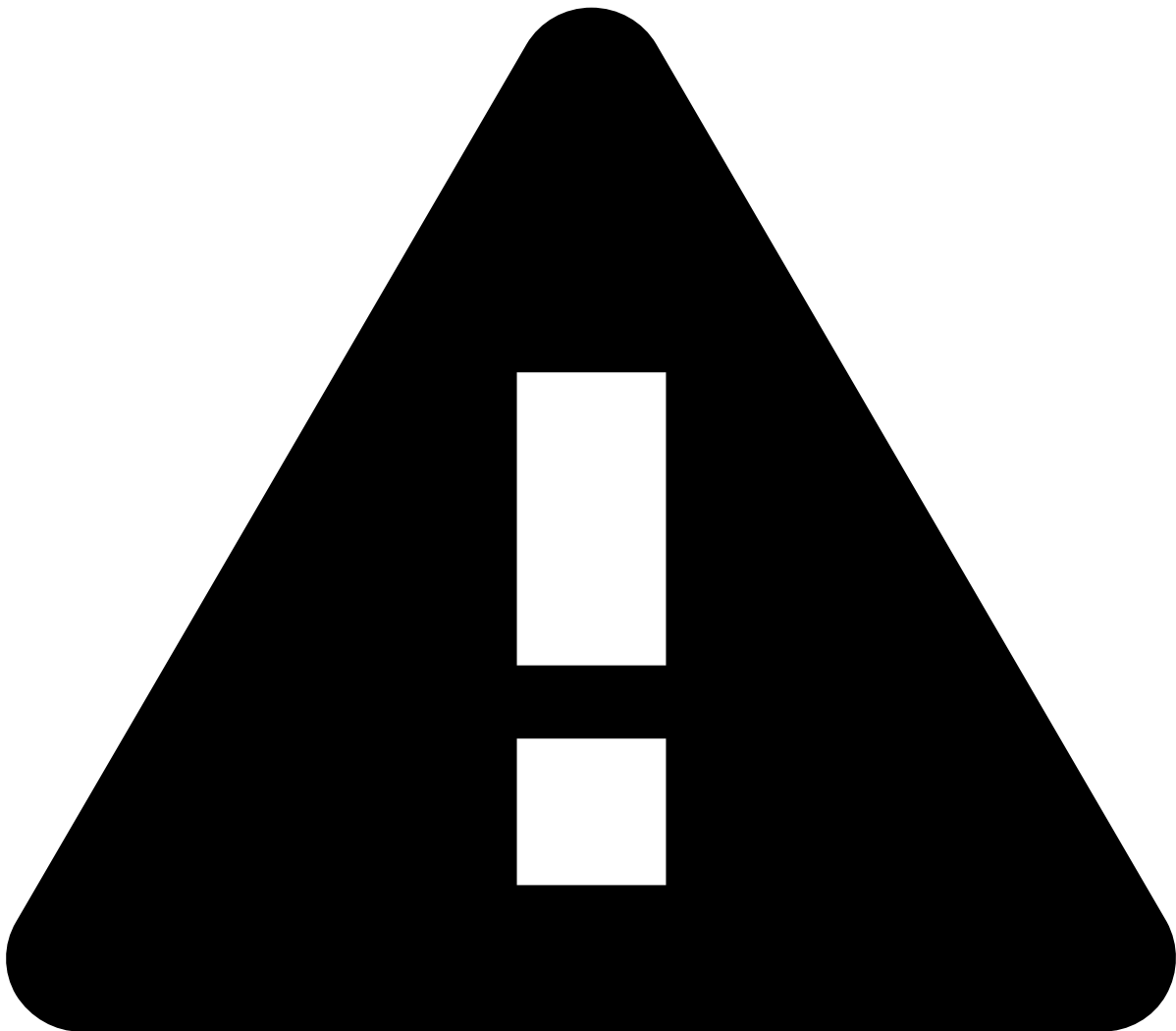
The following screenshot shows an analysis for events where *Fraud*=true:

The following screenshot shows an analysis for events where *Fraud*=false:

Generated statistics

Depending on the data types you defined for each field when [importing your data](#), Pulse generates different metrics.

For numerical fields Pulse generates arithmetic metrics such as mean, minimum and maximum, while for categorical data Pulse only generates the total count and distinct values.



Breakdowns

When you run an analysis on a data source, you must always select for which breakdown fields and values you want Pulse to calculate statistics.

Skipping this action and simply calculating broken-down metrics for all possible combination of fields and values could rapidly cause Pulse to crash due to lack of computational resources.

•



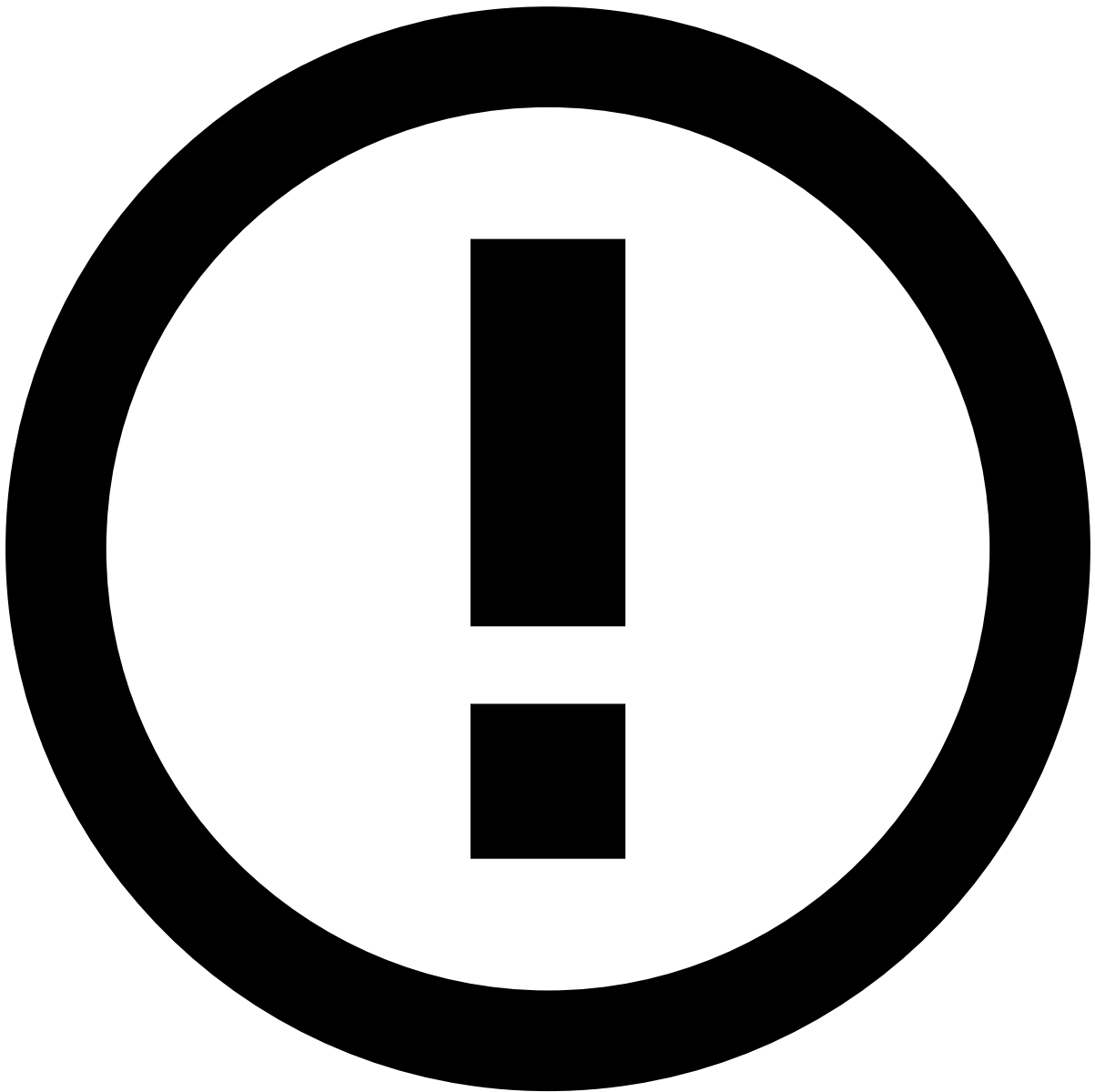
- [Feature Engineering](#)
- Using Lists in Features

On this page

Using Lists in Features

Was this page helpful? 

To access a field of a list item in a plan, you need to add the field to the plan. This can be useful when you have reference data stored in lists, like risk levels, or country-related information. This topic describes how to add fields of lists to a plan, and how to access them in a feature.



Features cannot use multi-tenant lists

Multi-tenant lists are not supported in plans. More specifically, the user cannot select multi-tenant lists in a list operator.

If in a plan you want to use list values to compute features, you need to add the list field to the plan. This is the objective of the list operator. For each event that traverses the list operator, this field will be populated with the corresponding list item value from the list.

List items can have an **expiration date**. This expiration date applies to the list operator as well, meaning that if the corresponding list item is not valid at the time of the timestamp of the event, the field in the event will have a null value. You can set the timestamp field in the following places:

- for a deployable plan under the Configure Plan Schema option.
- in a non-deployable plan: for each operator, you can select the timestamp field individually, in the Timestamp Field option.

If there is no timestamp field defined, the list items that have expiration date are ignored.

How to

To add a field of a list to a plan, perform the following steps:

1. In the **Data Science** tab, open the plan you want to edit.
2. Drag a **List** operator to the plan.
3. Type a **Name** for the list operator.
4. For each list field that you want to add to the plan, perform the following steps:
 1. In the **Output Field Name** column, type a name for the new field. This is the name by which you will be able to find the list value when using it in the plan.
 2. In the **List** column, select the list that you want to use in the plan.
 3. In the **Operation** column, select what you want to do with the list. The available operations are:
 1. **Get**: returns the value of the field specified in the List Field column, corresponding to the list item identified by the Item Value Expression. Applies only to complex lists.
 2. **Contains**: returns true if the list contains an item identified by the Item Value Expression.
 4. In the **Item Value Expression** column, type an expression that defines the key for which the list operator will retrieve the value. This expression is the connection between the event and the list. It is a [PQL expression](#), meaning that you can use anything you can normally use in a map operator, for example:
 - event fields
 - constants
 - boolean and arithmetic operators
 - UDFs
 - coalesce operator
 5. In the **List Field** column, select the field of the list that you want to use in the plan. This field is available only if you selected a complex list in the List column.
 6. Use the add button to add the field to the list operator.
5. **Save** the list operator.

As a result, in the features (map and metric operators) of the plan you can access the list value associated to the current event, by typing the name you gave in the *Output Field Name* column. The value that this field contains is the value that belongs to the key you defined in the *Item Value Expression*.

Example

You may have a positive/negative list about known merchants. A merchant may be risky, hence transactions with that certain merchant might need to be declined. Transactions with other merchants might need to be approved, or might need a review. A positive/negative list accumulates all this knowledge in one place.

In this case, the list is defined in the following way:

Example with a posneg list

The list items have the *Merchant Id* as key, and either 'decline', 'approve' or 'review' in the *Action* field. Example:

Merchant Id	Action
898776	'approve'
556949	'decline'
132214	'review'

Now, you may want to use the knowledge accumulated in this list in the plan. For instance, when processing a transaction, you might want to calculate the number of transactions initiated by the same card with risky merchants.

To allow that, add the *Action* field of the *Merchant PosNeg* list to the plan. The key that will connect the event with the list is the *MerchantId*. The new field of the event will be called *MerchantAction*.

Example list operator

Now you can use the *MerchantAction* field in the metric, as any other event field. It contains the action ('decline', 'approve' or 'review') according to the merchant of the event.

Example metric using the MerchantAction field

-



- [Feature Engineering](#)
- Using PQL in Features

On this page

Using PQL in Features

Was this page helpful? 

Pulse Query Language (PQL) is Pulse's query language that allows users to compute complex values based on input data fields.

This topic introduces the basic concepts of PQL.

To learn how to use PQL in some specific scenarios, read the following user guides:

- [Handling NULL values with PQL](#)
- [PQL in derived features](#)
- [Calling UDFs](#)

Usage

The following image shows an example of how PQL is used in a map operator to create a derived field in a plan.

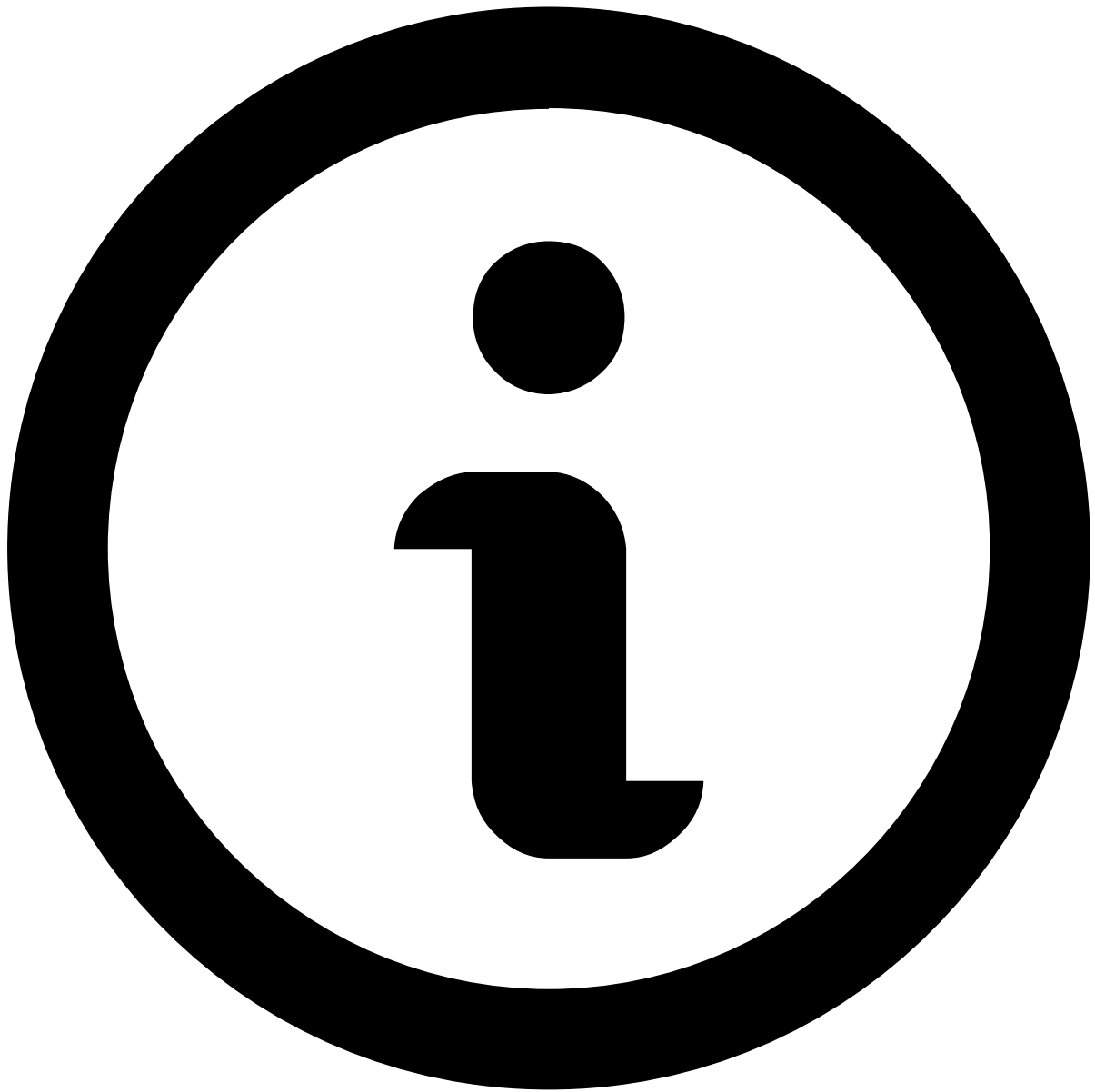
This map operator adds a categorical field called *Card Present Flag*, representing whether there was a bank card present in the transaction. The PQL expression is in the *Expression* column.

PQL expressions

In the Pulse UI, users can combine the following building blocks (values, operators, functions, subclauses) into PQL expressions:

Building block / subclause	Purpose, observation	Example(s)
Constants of primitive data types	Use a fixed value	1.999 'true' 'Swiped'
Input data fields	Use values of the input data. After typing the name of the data field, resolve the typed name to an existing data field name from the list that the UI offers.	Amount
: (i.e. the cast operator)	Convert data into another data type	Amount:int
Arithmetic operators	Calculate a new value from two values of int, long, float or double type	Balance - Amount
Boolean operators	Calculate a new true/false value	and or ! ==
Function calls	You can call two types of functions: built-in functions, and custom functions (aka. UDFs)	The following example calls the <i>between()</i> built-in function: between(Amount, 0, 200)
Conditional expressions	Return a value based on a condition	if Amount > 1000 then 'HighAmount' else 'LowAmount'
Variable declarations	Declare auxiliary variables that help you calculating other values	The following example shows how you define two variables (first_timestamp and last_timestamp) in order to calculate a third value (last_timestamp - first_timestamp) / 60*60*1000.0) (let first_timestamp = first().Timestamp ?? 0 in let last_timestamp = last().Timestamp ?? 0 in (last_timestamp - first_timestamp) / 60601000.0)
Collections, tuples	Work on tuples, lists, sets, and maps of data instead of a single data field	

Building block / subclause	Purpose, observation	Example(s)
		In the following example, OrderItems is a list of orders. Each item of the list is a tuple with the following fields: ItemId, Quantity, Price <pre>OrderItems.get(0).Quantity</pre>
Aggregations, and functions over aggregated datasets	Aggregate data to calculate minimum, maximum, average, or alternatively to access specific records of the aggregated dataset	<pre>avg(Amount) - prev().Amount</pre>
?? (i.e. the coalesce operator)	Handle NULL values	<pre>Amount ?? 0</pre>



Reserved keywords

Some keywords and symbols cannot be used outside of their original purpose. See the list of [reserved keywords](#) for more information.

•



- [Feature Engineering](#)
- Using String Indexing

On this page

Was this page helpful? 

Using String Indexing

When scoring an event, models can take into consideration information stored in strings. For that, first you must generate a new numerical field (an index) from the string field, so that you can use the index as a feature in the model.

To index a string field, use the Data Science API (DS-API) of Pulse to create an indexer. An indexer allows to obtain the index value of the string field.

This page describes how to create new features based on a string field for which an indexer was already created.

Prerequisite

The steps described in this topic presume that a data scientist created an indexer for the string field that you want to use. The indexer is a list in Pulse, created with the frequency indexing algorithm. You can find the indexers in Pulse at the bottom of the list of lists, prefixed with `{StringIndexer}`.

Learn how to use the DS-API to create an indexer in the JupyterLab notebook provided with the Feedzai package.

Using an indexed string field in a Pulse plan

To use the indexed value of a string field, you must add it to the schema of the data source. For that, use a map operator that retrieves the index value from the indexer and adds it to the event. You can access the index in the map operator by using the user-defined function `FrequencyIndexerUDF(indexer_name, input_field_name)`. The following table shows the obligatory parameters you must provide when calling this UDF.

Parameter	Type	Description
indexer_name	String	The indexer_name to be used here is the one defined when the indexer is created. Request this information from the data scientist that created the indexer.
input_field_name	Categorical	The data source field that you want to use.

Example

The following snippet returns the index value for the `merchant_id` field of the event:

```
FrequencyIndexerUDF.transform("v1_merchant_indexer", merchant_id)
```

Advanced usage: clipping

Pulse's string indexer uses frequency indexing. The frequency index assigns a ranking to each value of the indexed field. The different values are called categories. The more times a string or category shows up in the training data set, the higher its ranking. Example:

If there is too much information stored in the indexer (i.e. too many categories), the new feature might cause overfit in the machine learning model. To limit that, you can use fewer categories through clipping.

You can use clipping in two different stages:

- **When you create the indexer**, i.e. you don't include less frequent categories in the indexer at all.

The default clipping of DS-API uses 10000 categories as default for the maximum of categories stored in the indexer, and minimum of 100 samples per category. If there are more than 10000 categories, the less frequent ones are dropped. If any category has less than 100 observations, it is also dropped.

In the documentation of the DS-API included in the JupyterLab notebook, you can read how to clip the indexer when you create it.

- **When you use the indexer in Pulse to retrieve the ranking for a string**, i.e. you ignore less frequent categories at lookup.

See the next section of this topic to learn how to influence the index clipping when you are retrieving the index values.

Clipping indexers in Pulse

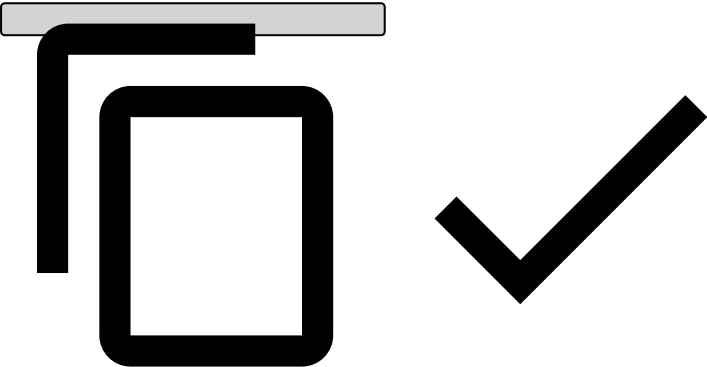
When you use the indexer to retrieve the ranking for a string, Pulse's default clipping ignores the less frequent categories of the indexer so that maximum 90% of the indexer's categories are taken into consideration, but minimum 10 categories.

The following table shows the two optional parameters that you can use to control index clipping for string fields in Pulse:

Parameter	Type	Description	Default value	Value range
clipping_max_categories_percentage	Float	Uses only the given percentage of categories from the indexer. For ignored categories and for other unknown input values, the indexer returns a default value, which is the number of categories in the clipped indexer.	90.0	[0.0, 100.0]
clipping_categories_limit	Int	Clips the least frequent categories until leaving only the defined number of categories, or until the clipping_max_categories_percentage criterion is met.	10	>= 1

With the clipping parameters, the full signature of the UDF is:

```
FrequencyIndexerUDF.transform(indexer_name, input_field_name, clipping_max_categories_percentage, clipping_categories_limit)
```

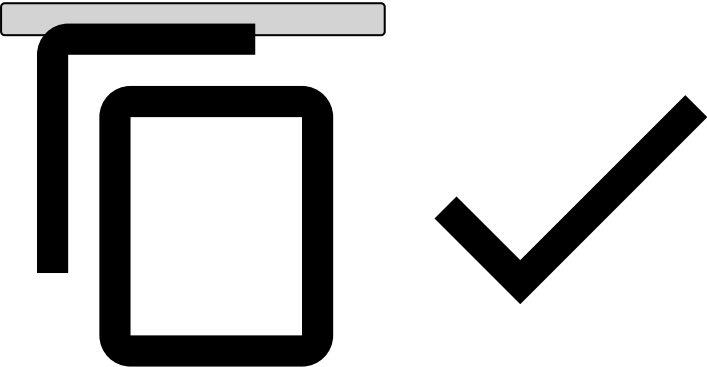


Examples for clipping

The following examples show how to use the parameters of the FrequencyIndexerUDF:

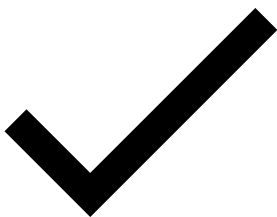
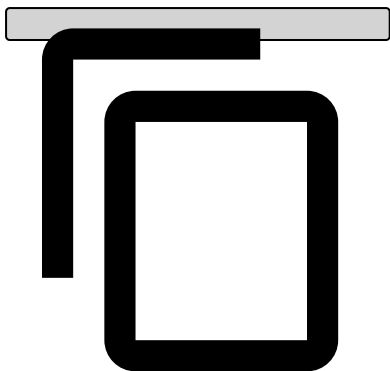
- 1. To use all the indexer categories, use the following expression in the map operator:

```
FrequencyIndexerUDF.transform("v1_merchant_indexer", merchant_id, 100)
```



- 1. To eliminate the smaller categories in the indexer until only 16.5% of the indexer's categories remain, or only 20 categories remain:

```
FrequencyIndexerUDF.transform("v1_merchant_indexer", merchant_id, 16.5, 20)
```



-



- [Model Guide](#)
- Comparing models

On this page

Comparing models

Was this page helpful? 

Why

At each iteration of your model, you will need to evaluate it. In reality, not only evaluate it but also assess how it performs in comparison with the previous iteration.

Pulse helps you in taking your conclusions on how your [feature engineering](#) work, [model training](#), and [parameter tuning](#) is evolving and progressing towards the goals of your Data Science project.

It may make sense to, for example, try out your model not only with adjusted parameters but also for different test datasets, different transaction time periods, etc.

How to

While you evaluate your [models](#), Pulse enables you to compare their performances:

1. [Directly by overlaying model evaluations.](#)
2. [Using the exploration mode for business targets.](#)

Model Evaluations

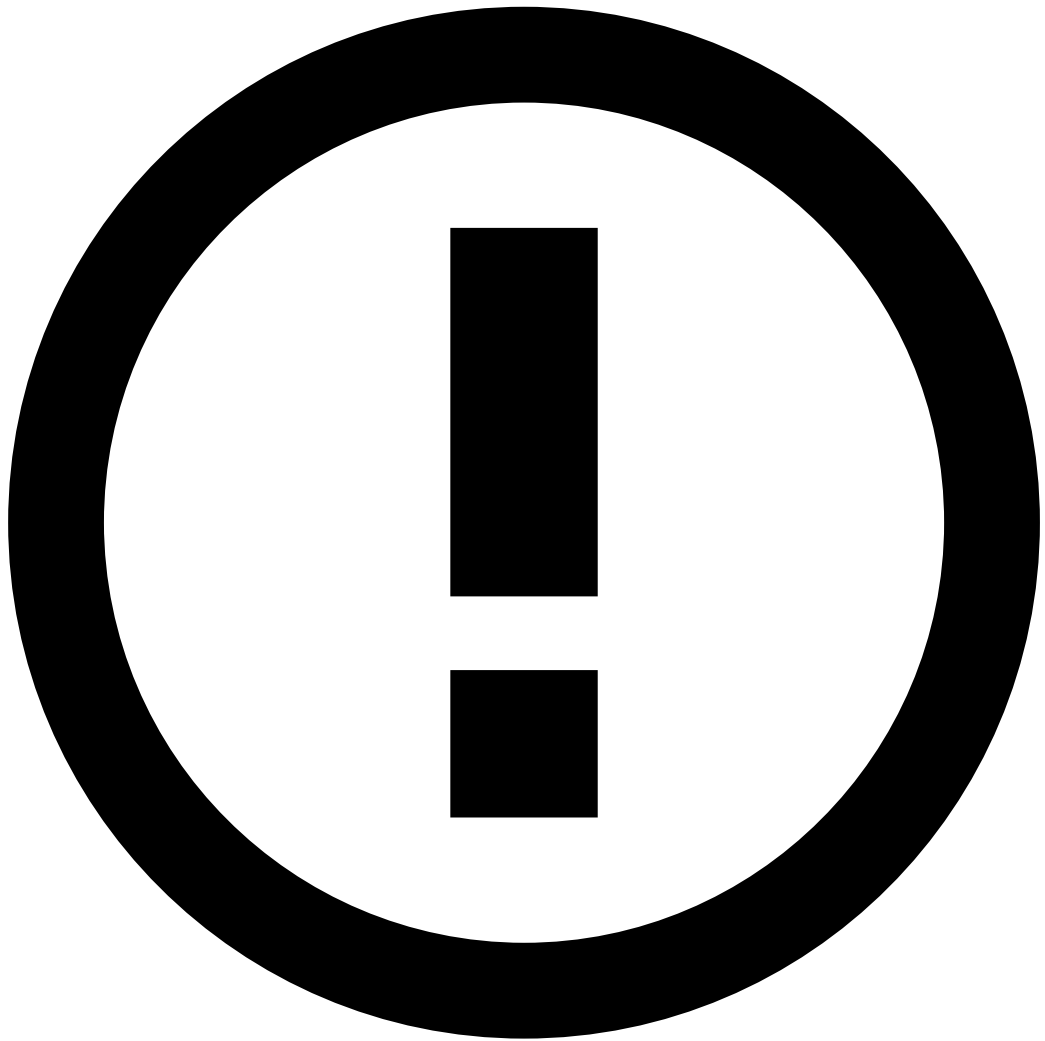
To compare two **Model Evaluations**, execute the following actions:

1. In the **Data Science** tab, select a **Model Project** from the **Model Projects** list.
2. Select an existing **Model Evaluation**.
3. Select which is the evaluation you want the current one to be **Compared With**.

The information in **Model Evaluation** is divided into different panels that can be accessed for: Comparing **Performance Metrics** or Comparing **Feature importance**.

Comparing Performance Metrics

1. To visualize comparison results:
 1. Analyze the **ROC Curve** and **X/Y Plot**, now with both the **Model Evaluation** and the **Comparison** evaluation visible.
 2. Analyze the Comparison results displayed at the top row of the **Metrics** table.



Comparison results

When you select a **Model Evaluation** for comparison, the **ROC Curve**, **X/Y Plot** and **Metrics** table display the evaluation results only for the main combination of breakdowns (All x All). This occurs even if the model evaluation has configured breakdowns.

2. Select the Comparison Details option to display evaluation parameters for both the selected and the comparison models. Differences are highlighted so that you can easily spot all [parameter adjustments](#) you are testing.

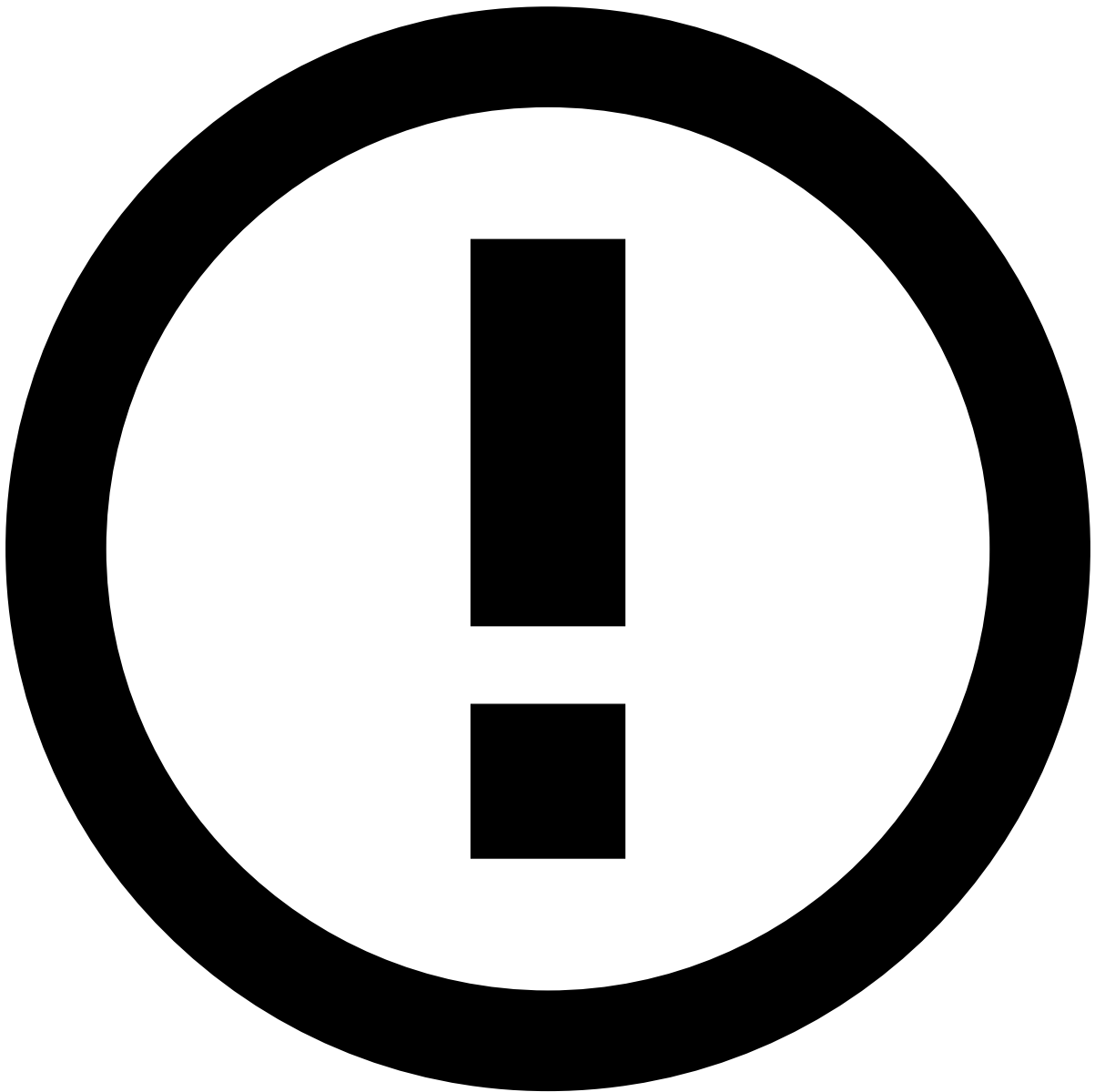
Comparing Feature Importance

This option is only available if the current Model Evaluation has Feature Importance computed (i.e. regardless of whether or not the model evaluation under comparison has it computed).

1. Select to visualize comparison results:

The following metrics are presented by default:

- **#FPR**: Values of the count FPR for each feature for the current **Model Evaluation** (A) and for the evaluation under comparison (B).
- **Absolute importance**: Bar chart comparing the **Absolute Importance** of both model evaluations.
- **Rank**: The rank of the importance of each feature for each model evaluation.
- **Delta**: The difference between Rank A and Rank B.



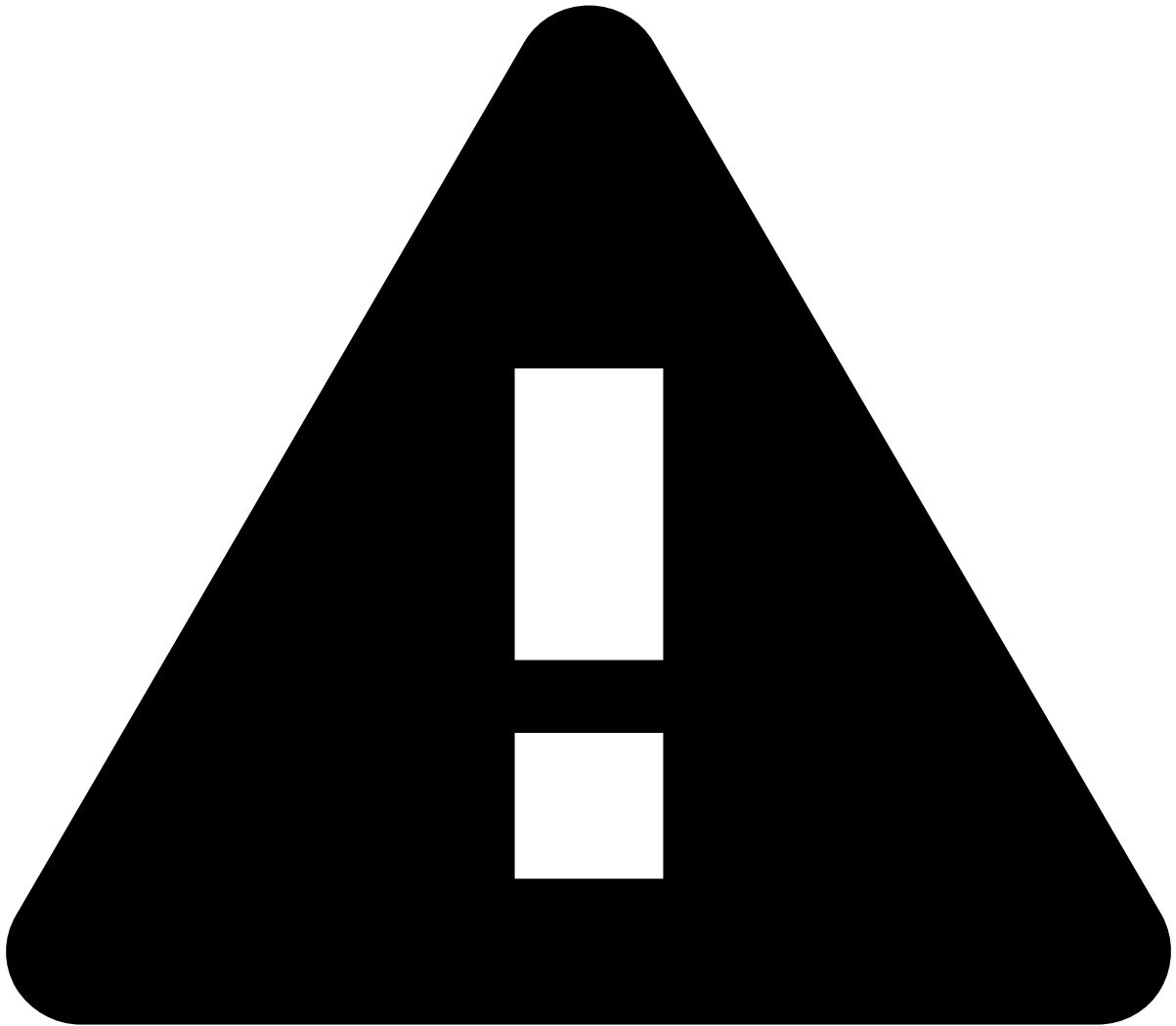
Feature Importance view

If you compare the current model evaluation with an evaluation that does not have feature importance computed, the **Feature Importance** view will only display data for the current model evaluation.

Exploration Mode

To compare how different **Model Evaluations** perform for a Target, execute the following actions:

1. In the **Data Science** tab, select a **Model Project** from the **Model Projects** list.
2. Select the Exploration Mode option to explore evaluation results in your **Model Project**.
3. [Configure Targets for evaluation](#).
4. Analyze the results for each **Target** and compare them among all selected **Evaluations**.



Comparing models

You can compare models using the [exploration mode within a hyperparameter tuning](#) or using the [global exploration mode](#) in your model project, where you can combine results from model evaluations and hyperparameter tuning.

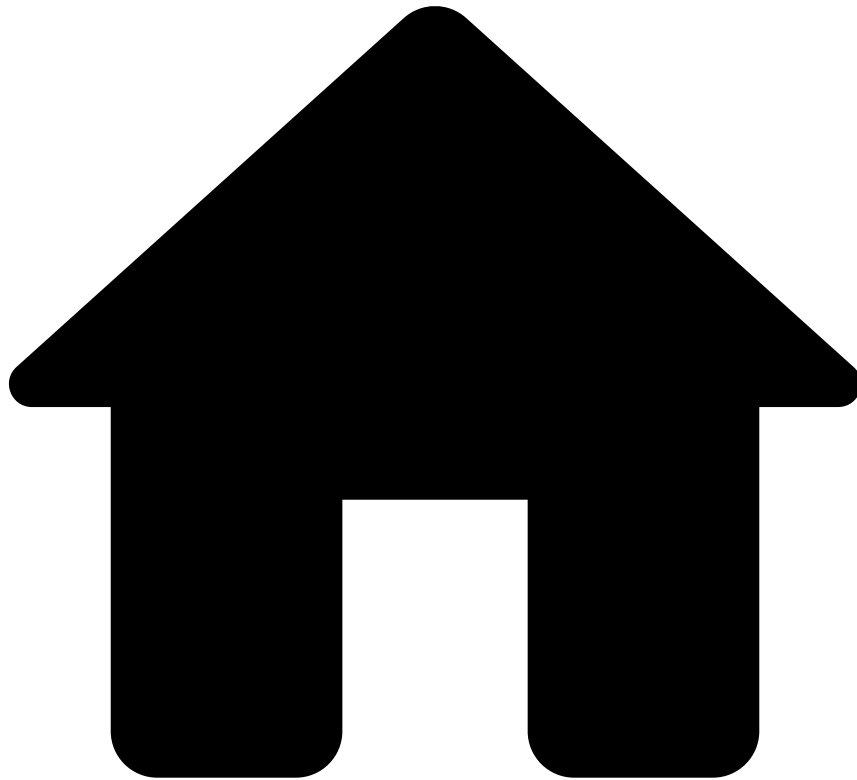
Next Steps

If your model produces the desired results, then your project is ready for [deployment into Production](#). Check also [how Pulse projects work](#) for some background to understand what is involved at this stage.

If your model needs further tuning, then start a new iteration in your Data Science project. For example, by [improving your model's features](#) and [adjusting parameters](#).

To add an extra layer of security on top of your Machine Learning model, you can also [build and evaluate rules](#).

•



- [Model Guide](#)
- Evaluating a Model

On this page

Evaluating a model

Was this page helpful? 

Evaluating your [model](#) against a test dataset is crucial in understanding its performance. This topic explains how to generate a [model evaluation](#) over a trained model, and how to analyze the evaluation results.

Generating a model evaluation

To generate an evaluation, execute the following actions:

1. In the Data Science tab, select a model project from the **Model Projects** list.
2. Add a new model evaluation.

3. Add a Name for the model evaluation, select the **Model**, and the **Data Source**.



Training and testing datasets

Make sure that when explicitly evaluating a model you **do not** select a testing data source that intersects with the dataset you used for training.

4. Select the **Target Variable**. It must match the format (data type and possible values) of the target variable of the model, if it has any.
5. In the **Fields to Select** area, choose which fields in your test data source Pulse will feed to the model and press . Only *Numeric* and *Categorical* fields are available.

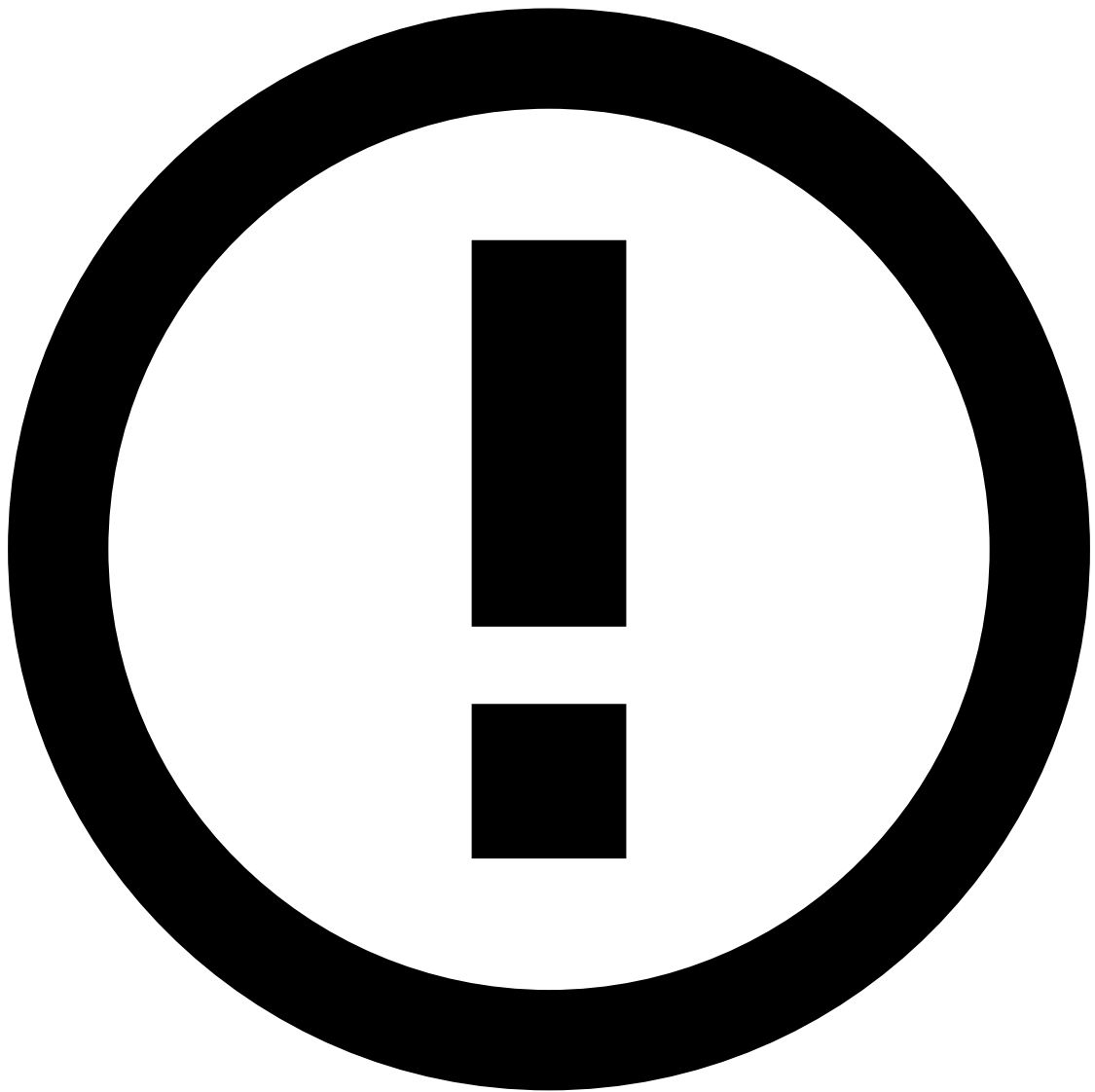


Testing data fields

Pulse feeds the test data to the model according to the data schema you provided at training time. This means that although different test field names are allowed, you must always specify your fields in the same order and with the same data types.

If the model training used a supervised ML algorithm, this rule applies to that field as well. If the model used an unsupervised ML algorithm, then the target variable can be in any place within the schema of the evaluation data source.

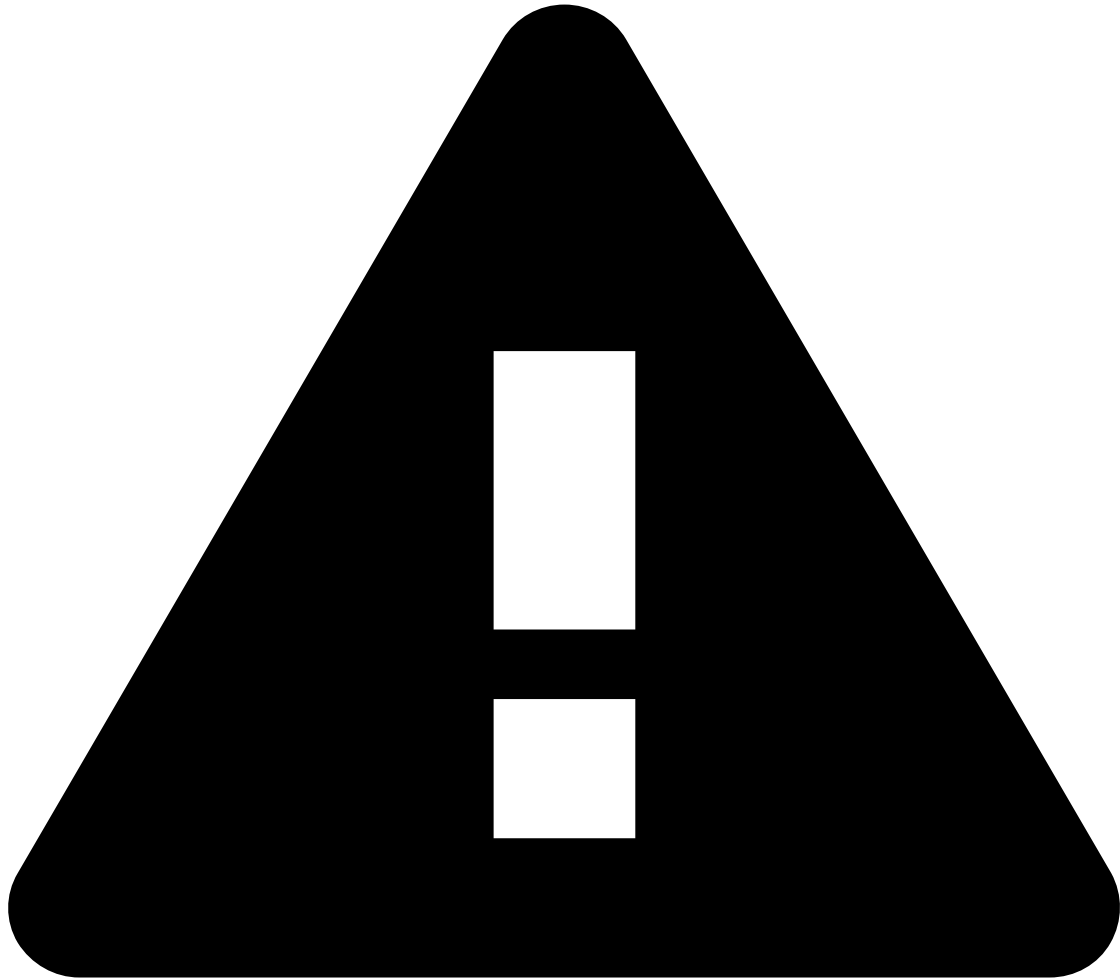
6. Enable an **Additional Cost Field** to be available for the model evaluation. Typically, you can select a field that represents the transaction amount to have metrics such as the *fraud money recall* and *fraud money FPR*.
7. Enable **Feature Importance** and configure the following input parameters. Check the page [Feature Importance](#) for more information about these parameters:
 1. **False Positive Rate**. The default value is 1.00%.
 2. **Sample Percentage**. The default value is 30%.
 3. **Random Seed**
8. Select which are the **Breakdown** fields and values to slice the results into. For example, *Card Present* vs *Card Not-Present*. Pulse will automatically display the number of possible breakdown combinations.



Breakdowns

The higher the number of possible breakdown combinations, the more computational resources will be consumed by training, and the longer will take to generate an evaluation for your model.

9. In the **Output Result** area, select the **Score Instances** checkbox to generate a new data source and a CSV file in your filesystem, adding a Pulse risk assessment to each instance. The new data source includes extra fields with a Feedzai score, each one representing the likelihood of the transactions being classified as each of the possible values of the target variable. For example, in a fraud scenario, Pulse adds two fields, one with the score for fraud and the other representing the score for not fraud. To configure the output, add a **Name** for the data source and a **Directory Path** where it will be stored.
10. **Evaluate** the Model using the button, to create your model evaluation and compute feature importance.



Incompatible schema

If the data source selected in step 3 has a different schema than the data source used to train the model, an error message is displayed. In this case, you can select the option to change the schema of the model evaluation.

Analyzing the model evaluation¹

When you open an existing model evaluation, you have access to metrics to analyze the performance of your model. Additionally, if the model evaluation has feature importance computed, you have access to a tab to analyze the results of feature importance. However, if the model evaluation does not have feature importance computed yet, you can still perform this computation.

The following subsections will guide you through the following topics:

- [Analyzing performance metrics](#)
- [Computing feature importance from an existing model evaluation](#). This is only needed if feature importance has not been yet calculated while generating the model evaluation.
- [Analyzing feature importance](#)

Analyzing performance metrics²

To analyze evaluation results, execute the following actions:

1. In the **Data Science** tab, select a model project from the **Model Projects** list.
2. Select an existing model evaluation.

3. Perform any of the following operations:

1. Change the **Target Value** of your target variable for which you want to evaluate the results. For example, 0 representing *Not-Fraud* and 1 representing *Fraud*.



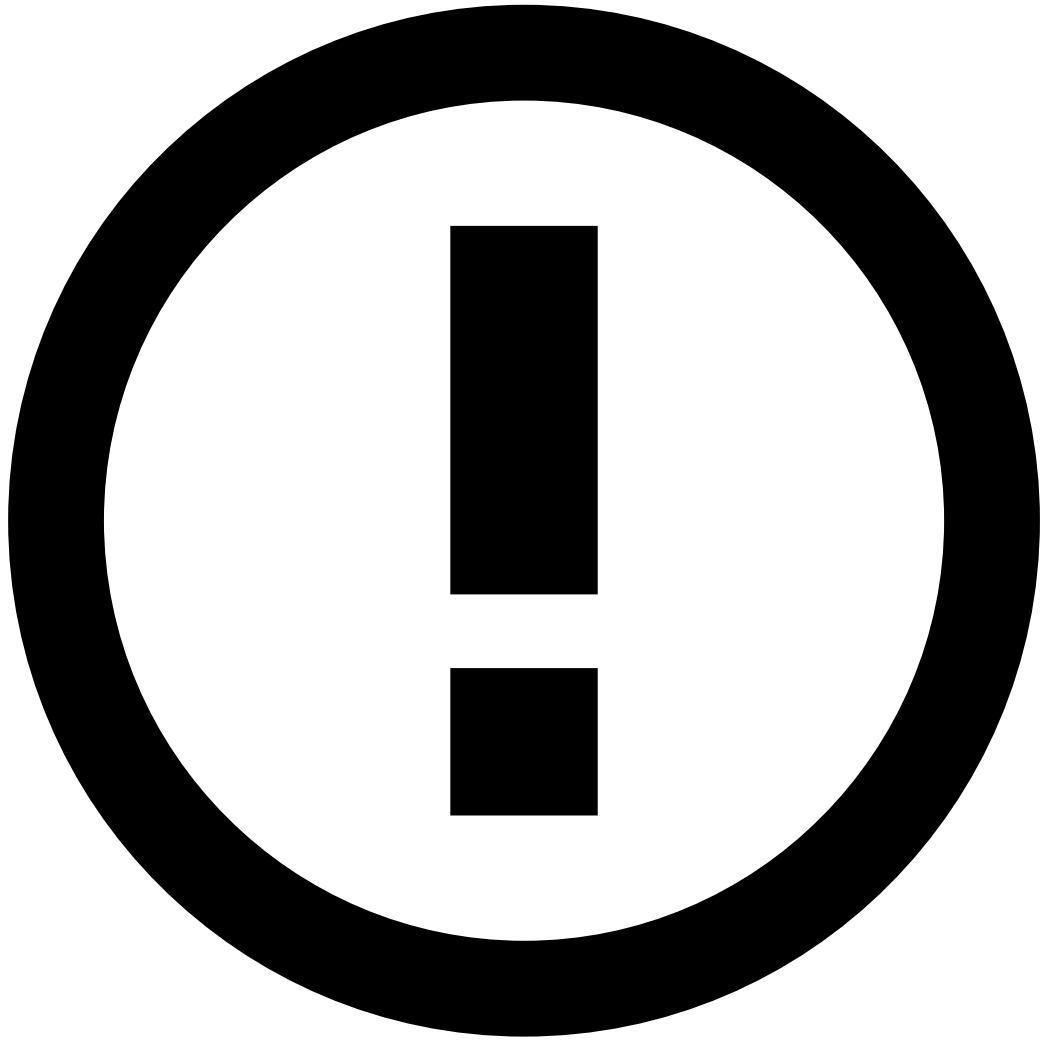
When changing the Target Value

The following happens when you change the target value:

- The specified target value will be interpreted as positive, while other values will be interpreted as negative. The ROC curve will reflect this reinterpretation.
- The Score Distribution graph gets updated as well. The graph shows the distribution associated to the likelihood of the instance to be classified as the specified target value. In the case of two classes, the score distributions associated to them are mirror images with respect to the X axis.

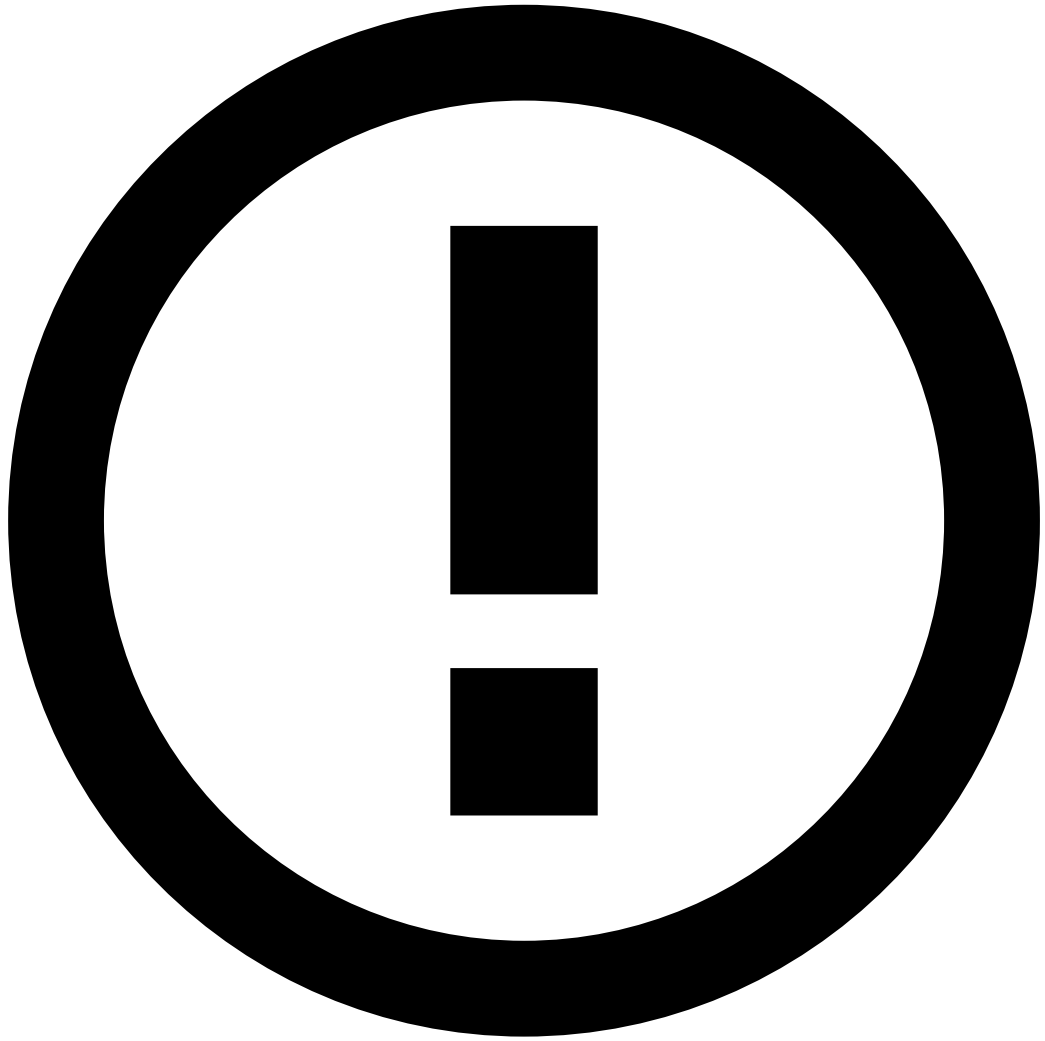
If you make the non-usual choice of GENUINE as target value, then the score distribution will have high values for transactions that look legitimate, and low values for high risk of fraud. The FPR would be reinterpreted as the rate of fraudulent transactions mistakenly predicted as legitimate. Recall would be the "detection rate" of legitimate transactions, instead of the usual fraudulent transactions.

2. Analyze the **ROC Curve** for the selected evaluation.



Exact values

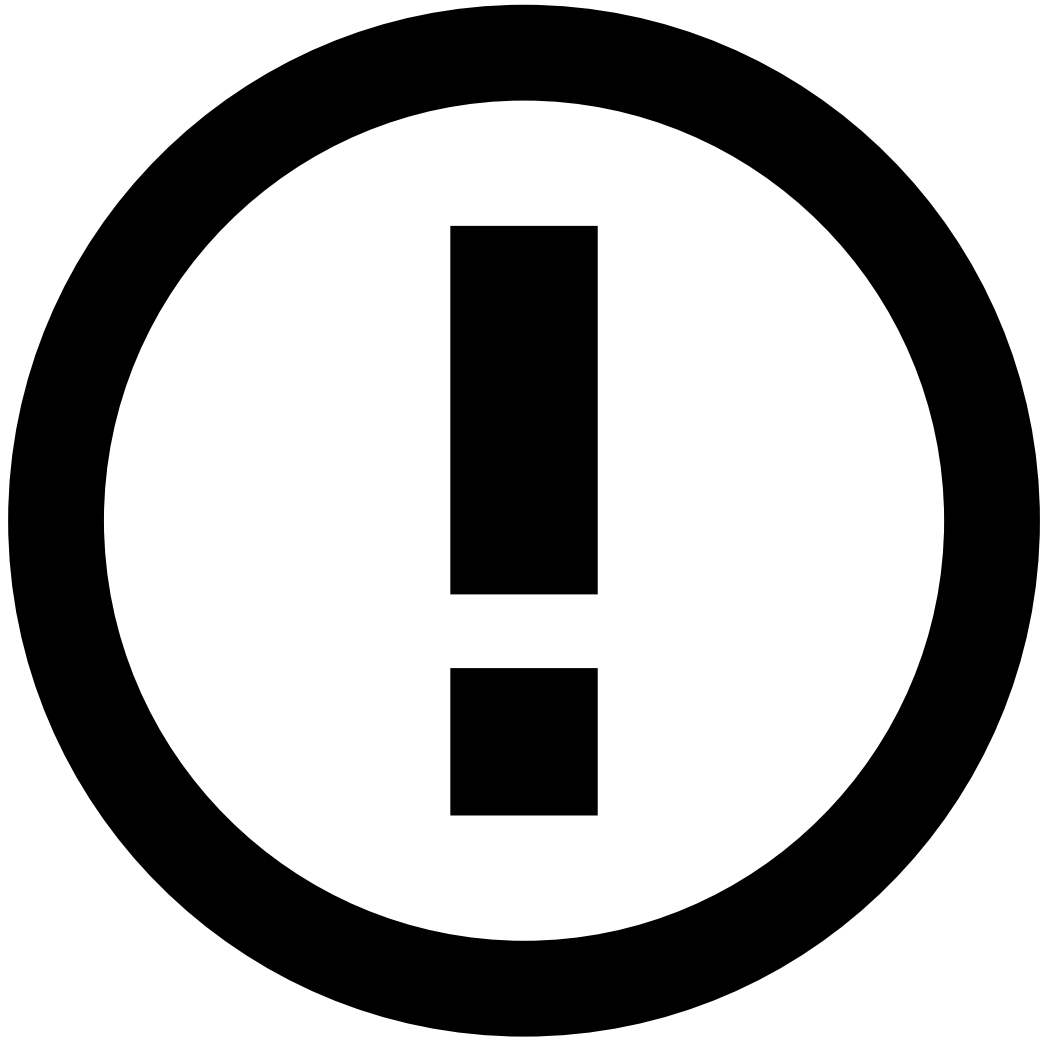
Hover the mouse pointer over the plot line to inspect the exact values for the **Recall** and **FPR** metrics and the respective **Threshold**.



Cost-based Metrics

If you define a **Cost Field**, then the **ROC Curve** will also plot the values for **Cost Recall** vs **Cost FPR**.

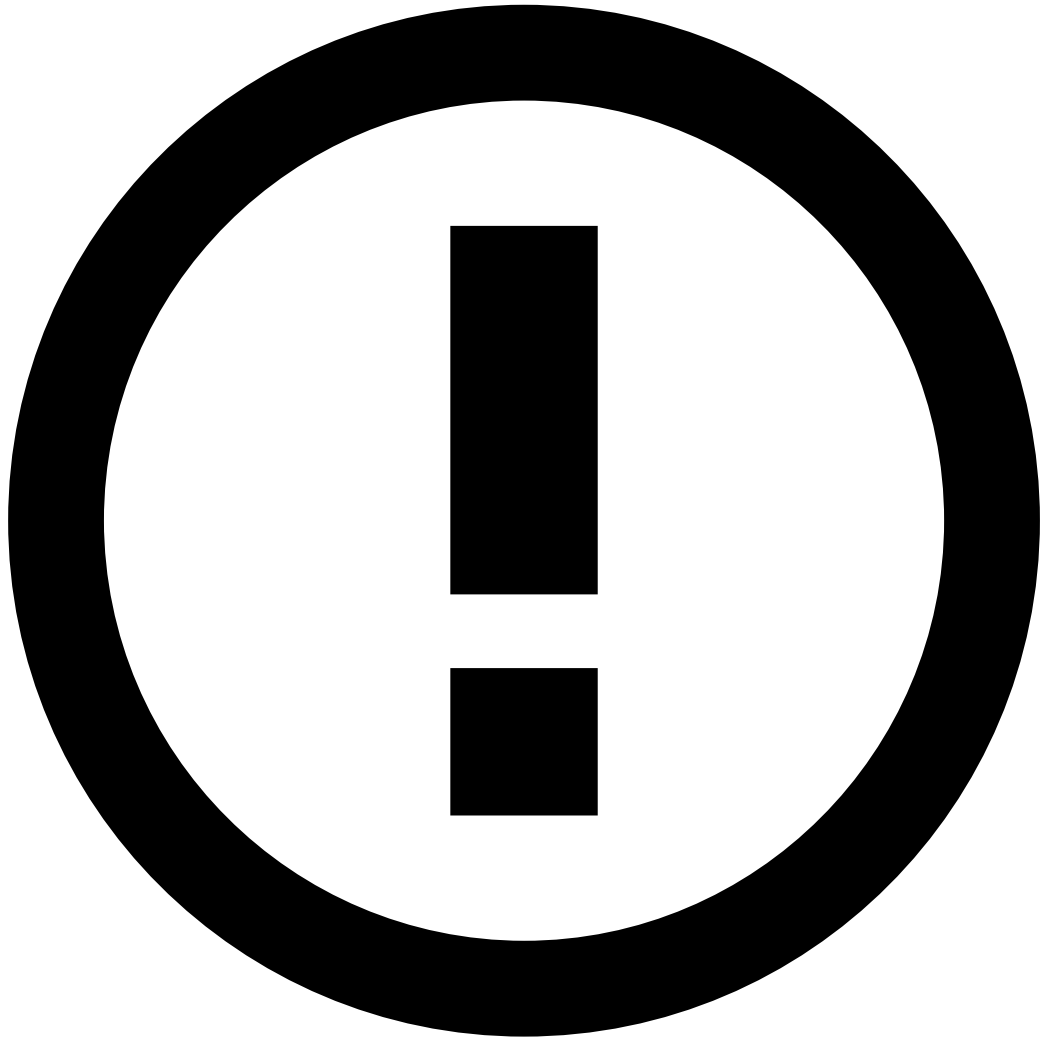
3. Configure the **X/Y Plot** to display a curve for any **X-Axis** and **Y-Axis**.



Exact values

Hover the mouse pointer over the plot line to inspect the exact values for the **X-Axis** and **Y-Axis** metrics and the respective **Threshold**.

4. Analyze the **Metrics** table, displaying the selected metrics for all breakdown combinations you configured. Adjust the Feedzai score **Threshold** to visualize the respective results in the **Metrics** table.



Custom Evaluation Metrics

If you do not define a [Cost Field](#) for an evaluation, then Pulse will not be able to compute any custom evaluation metrics you may have configured and you will not have access to them.

5. Use the button to display a specific breakdown in the ROC Curve and X/Y Plot areas.

6. Analyze the **Score Distribution** for the selected evaluation.

In a good model, the values of the target variable can be visually separated on the graph. In the following example, setting the threshold between 50 and 100 will separate *False* from *True*.

4. If you want to take your evaluation further at any point, you can enrich it by [comparing with other model evaluations](#).



Compute Feature Importance in existing Model Evaluations

The option to compute feature importance from an existing model evaluation is available only if feature importance has not been computed yet for the selected model evaluation.

To compute feature importance from an existing model evaluation, execute the following actions:

1. In the **Data Science** tab , select a model project from the **Model Projects** list.
2. In the **Model Project** window, select a model evaluation from the **Model Evaluations** list.
3. Select the button to configure the input parameters for Feature Importance:
4. Set the following input parameters. Check the page [Feature Importance](#) for more information about these parameters:
 1. **False Positive Rate**. The default value is 1.00%.
 2. **Sample Percentage**. The default value is 30%.
 3. **Random Seed**
5. Use the button to compute feature importance for this model evaluation.

Analyzing feature importance^â

To analyze the results of feature importance, execute the following actions:

1. In the **Data Science** tab, select a model project from the **Model Projects** list.
2. Select an existing model evaluation.
3. Select the button to access the feature importance results:



Feature importance

If an error occurs during the computation of the feature importance, the tab displays a warning. Check [Handling Feature Importance errors](#) to learn how to re-compute feature importance.

By default, the list presents three columns: **Feature**, **Absolute Importance** and **Relative Importance**. Features are sorted by absolute importance. Absolute and relative importance are calculated based on the algorithm described in the [Feature Importance](#) topic.

You can perform any of the following operations:

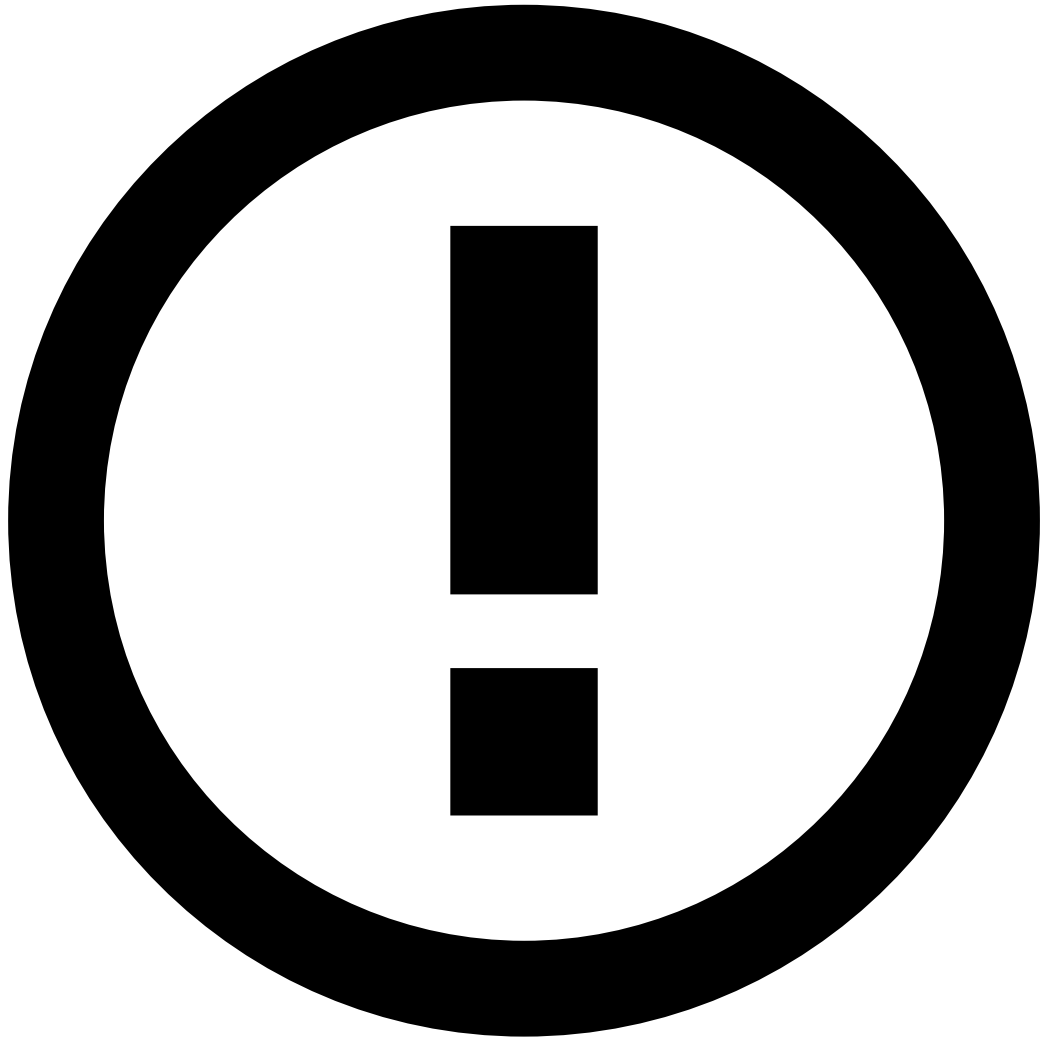
- **Sort and reverse order:** You can sort the list by any of the columns using the arrows on the right hand side of the column name (). You can reverse the sort by order by selecting the same column twice.
- **Choose Metrics:** Use the button to customize the columns that are listed in the feature importance results.
- **Export Results to CSV:** Use the button to export the results of feature importance to a CSV file with the following information: Feature, Target FPR, #Recall, #FPR, Absolute Importance, Relative Importance.

Exploring Evaluation results

The **Exploration Mode** in your model project allows you to explore results both from single Model Evaluations and Hyperparameter Tuning.

To explore evaluation results and examine Model performance for business targets, for example, client SLAs, execute the following actions:

1. In the Data Science tab, select a model project from the **Model Projects** list.
2. Select the option to explore evaluation results in your model project.
3. Select the to view model evaluation metrics according to the target you configure. As an example, consider that your client SLA for project acceptance includes detecting fraud with 1% recall. Configure a target as follows:
 1. Define which is the metric that the target will reflect, in this case, *#Recall*.
 2. Define the **Configuration** for the new target by selecting which is the target metric and which is the preferred strategy to reach the target. In this case, *Under 1%*.
 3. Choose which are the **Metrics to Show**. This means that Pulse displays the values for each selected metric when the model detects fraud with recall the closest (and inferior) to 1%.



Custom Evaluation Metrics

If you do not define a [Cost Field](#) for an evaluation, then Pulse will not be able to compute your [custom evaluation metrics](#) and you will not have access to them.

4. Use the search filters under **Evaluations** to select model evaluations and/or hyperparameter tuning results. Note that you can combine all results from one type of evaluations with all results from the other type.
5. Analyze the results of the evaluations for each target.

Using custom evaluation metrics

To configure a [custom evaluation metric](#) and make it available for model evaluations execute the following steps:

1. In your Pulse application, use the **Options menu** and select .
2. Add a new evaluation metric.
3. Name and add a textual **Description** to the evaluation metric.
4. Configure an arithmetic **Expression** to calculate the value of the metric.



Available values

The available values you can use for your metric's expression are the ones of the confusion matrix metrics both for count and cost field sums. For example, `count_FalsePositives` and `cost_field_FalsePositives`.

Next Steps📖

More than generating model evaluations you must be able to track how the performance of your model is improving. As model training and evaluation is an iterative process, once you evaluate your current model you should [compare it with previous iterations](#).

•



- [Model Guide](#)
- Training a Model

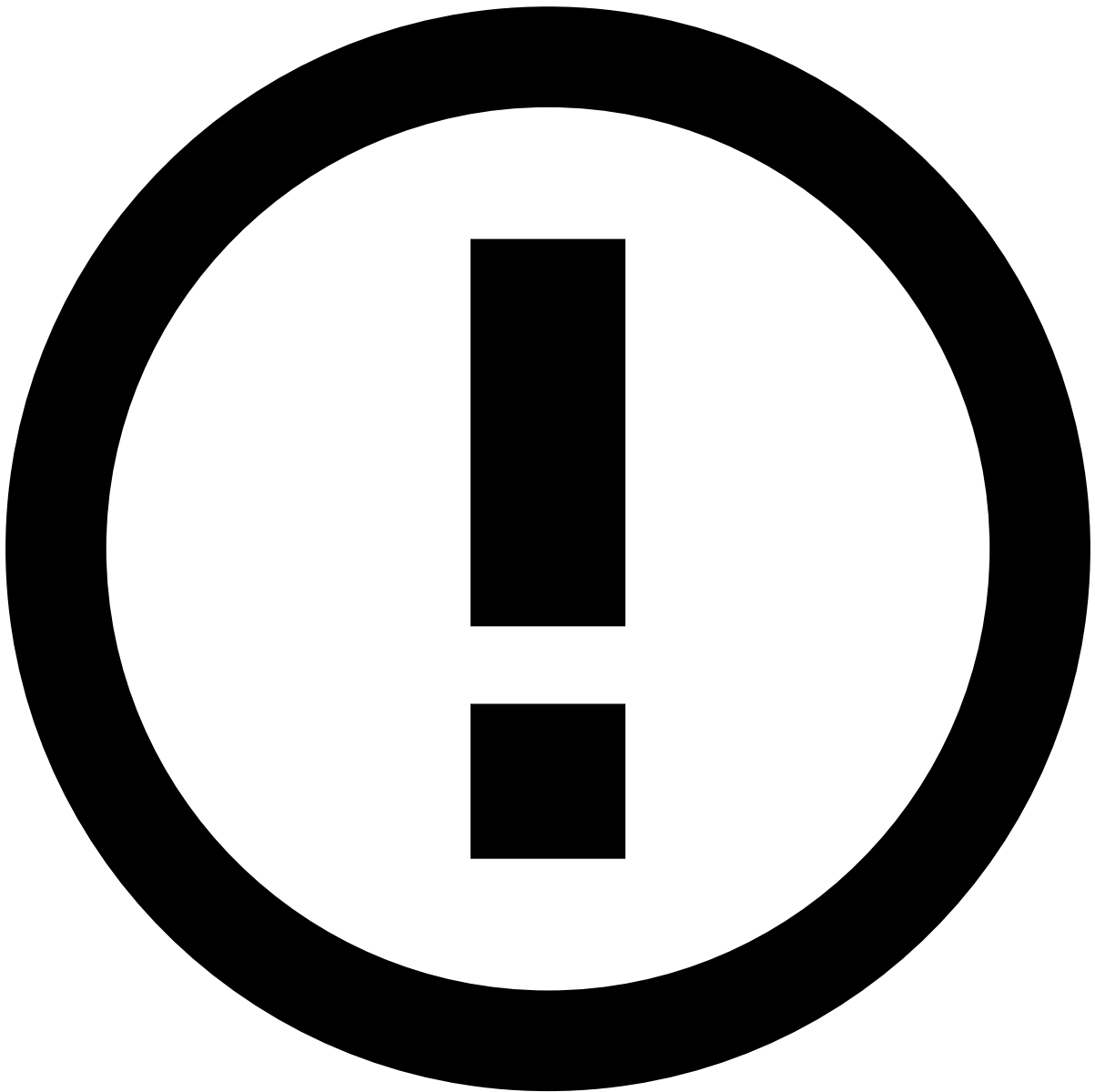
On this page

Training a Model

Was this page helpful? 

Once you have [designed features for your data](#), the next stage is to assess whether they are useful for fraud detection, or for predicting any other variable.

At this point you need the [data source](#) that contains the features you have generated, to train a [model](#) and to validate it.



Importing a Model

Instead of training a model in Pulse, it is also possible to import a model that you created and trained externally. See [Importing an external model](#).

How to

To train a model, execute the following actions:

1. In the **Data Science** tab , select a model project from the model projects list.
2. Add a new model.



Duplicating an existing model

Instead of creating a model from scratch, you can duplicate an existing model that is similar to the one you want to create. For that, use the **Duplicate and Change** option. This option loads a copy of the model, with all the properties and values that were selected when the model was originally created.

This way you can fine-tune a model that needs only some adjustment.

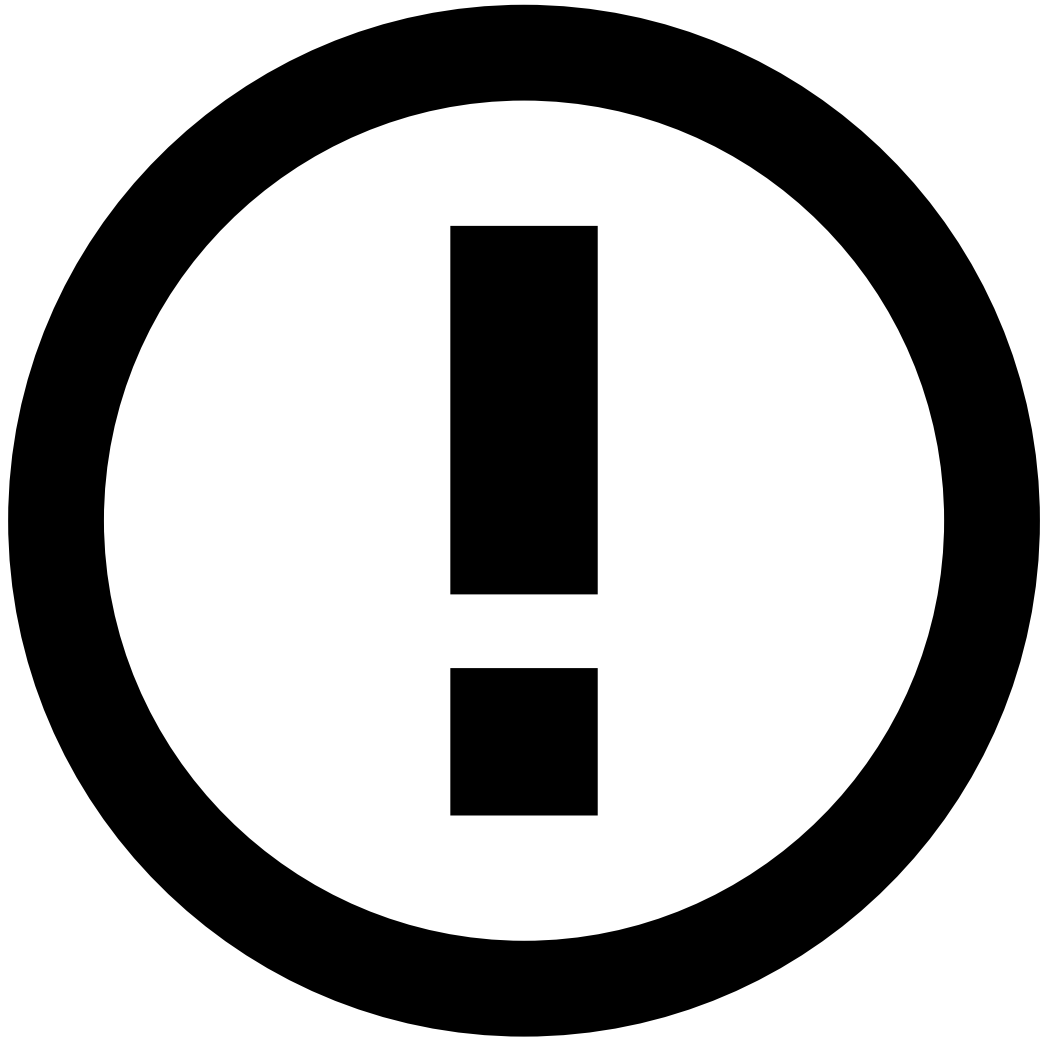
3. Select **Train model**.

4. Configure the training properties for your new model as follows:

1. Select the **Data Source** you will use to train the model. Typically one that has been the output of [running a features plan](#), or the result of splitting your dataset into training and testing subsets.
2. Select the Algorithm. See the [limitations](#) that you need to consider.

At the end of the line in the Algorithm drop-down box, you can select the button to get information about the algorithm.

3. Set the **Target Variable** that the model will predict. For example, *Fraud*. This step is not required when using an unsupervised machine learning algorithm.



Target variables

Although the main goal of Pulse is to detect fraud, models can target any [categorical](#) variable.

4. Choose which **Fields** in your Data Source the model will use in the prediction. Only *Numeric* and *Categorical* fields are available.



Removing fields from the model

If you suspect that some of the fields add only noise to the model, you can just remove them. Run an analysis on your Data Source, as you would do to [detect fraud patterns](#), and examine the statistics for each suspected field. For example, categorical fields with only one category, e.g. *empty value*, should not be considered when training your model.

You can also compute the [Feature Importance](#) and remove the features that had a negative impact on the previous model iteration.

In the case of an **unsupervised machine learning algorithm** (e. g. Isolation Forest), remove the target variable. This avoids label leakage, since a realistic anomaly detection model does not have access to the label.

Training parameters

When training a model, there are some typical configurations that control how Pulse will execute the training.

To configure the training, execute the following actions while you are adding a new model:

1. Set a **Random Seed**. If you evaluate models with the same parameter configurations and the same seed, you should be able to replicate results for different models. If you execute several validations for the same model parameters configuration with different seeds, you should obtain statistically similar results.
2. Select the **Sampling Type** and the Sample Percentage to define how to undersample or oversample your data.

3. Configure the **Parameters** of the selected algorithm. For information on the parameters, select the Info button at the end of each line.

4. Configure the [White-box Explanations](#) (WBE).

If WBE is set up for a trained model, Pulse calculates the features that led to a transaction's score in run time when the model is used. The output is available in Case Manager. You can select the following:

- **Algorithm:** the algorithm that Pulse uses to calculate the contribution of the features.
- **Number of Explanations:** the number of explanations that users want to obtain in Case Manager.

Only the most important features (i.e. the features that influence the transaction's score the most) will be shown in Case Manager.

- **Target Value:** the value that the user wants to predict (e.g. false vs. true). This component is crucial, given that WBE are agnostic to any value.

5. After selecting the above parameters, you have to edit the explanations. To do so, expand the White-box Explanations area using the button.

There are 2 ways to edit the explanations:

- Direct editing in the Pulse UI:
 1. Use the checkboxes under **Features** to select which features' explanations you want to take into consideration when analyzing a score in Case Manager.
 2. Optionally, add the features to an existing or new **Group**. If you group features, they will count as one in the list, leaving room for other important features to be on the list as well. For example, you can create a group called "amount" and use it to group several features that have the same property (e.g. amount).
 3. Add an **Explanation Message** to the features you selected. This message will be visible in the features that contributed most to the score.

The explanation can include the value of any feature or data field of the current event. To do so, start typing `/` followed by the name of the field, and select the desired field from the suggestions. You can use the `/current_value` shorthand to insert the value of the feature whose explanation you're currently editing.

- Editing in an external editor:
 1. Select **Export Explanations** to save the current explanations as a CSV file, containing 4 columns: "Selected", "Feature", "Group" and "Explanation Message", and listing all the features with the current information.
 2. Edit the file in your preferred CSV editor (e.g. Excel).

The "Selected" column should have an 'X' for features you want to select, and be empty otherwise.

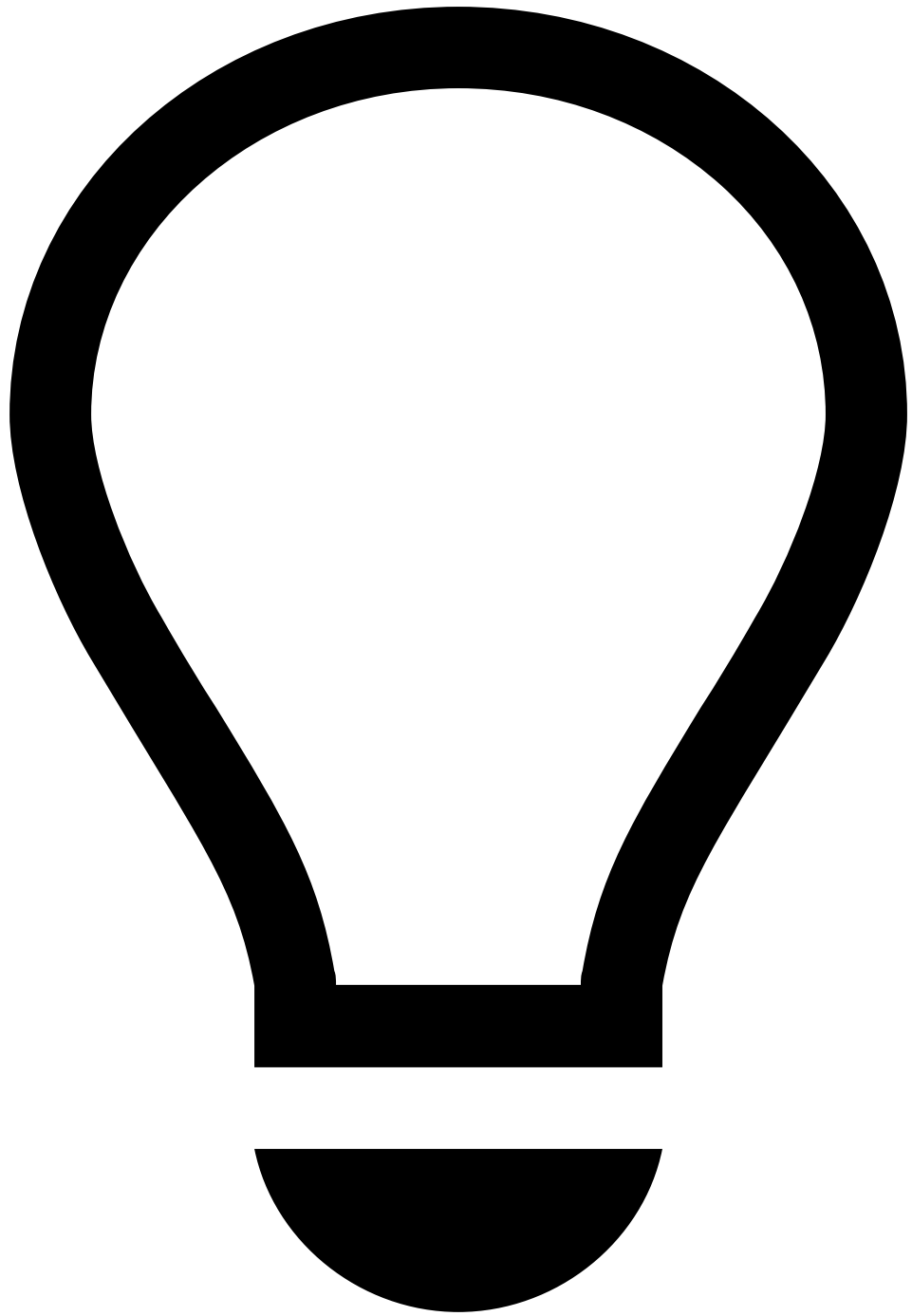
The "Explanation Message" column can contain `/` -prefixed references to a feature name that will be replaced by the actual value, as described above for editing via the Pulse UI.
 3. Select **Import Explanations** to upload the edited file back to Pulse and replace the explanations with the file's content.



caution

All the information in Pulse will be replaced by the file's content, which means you can lose explanations that may have been added in Pulse and are not present in the file.

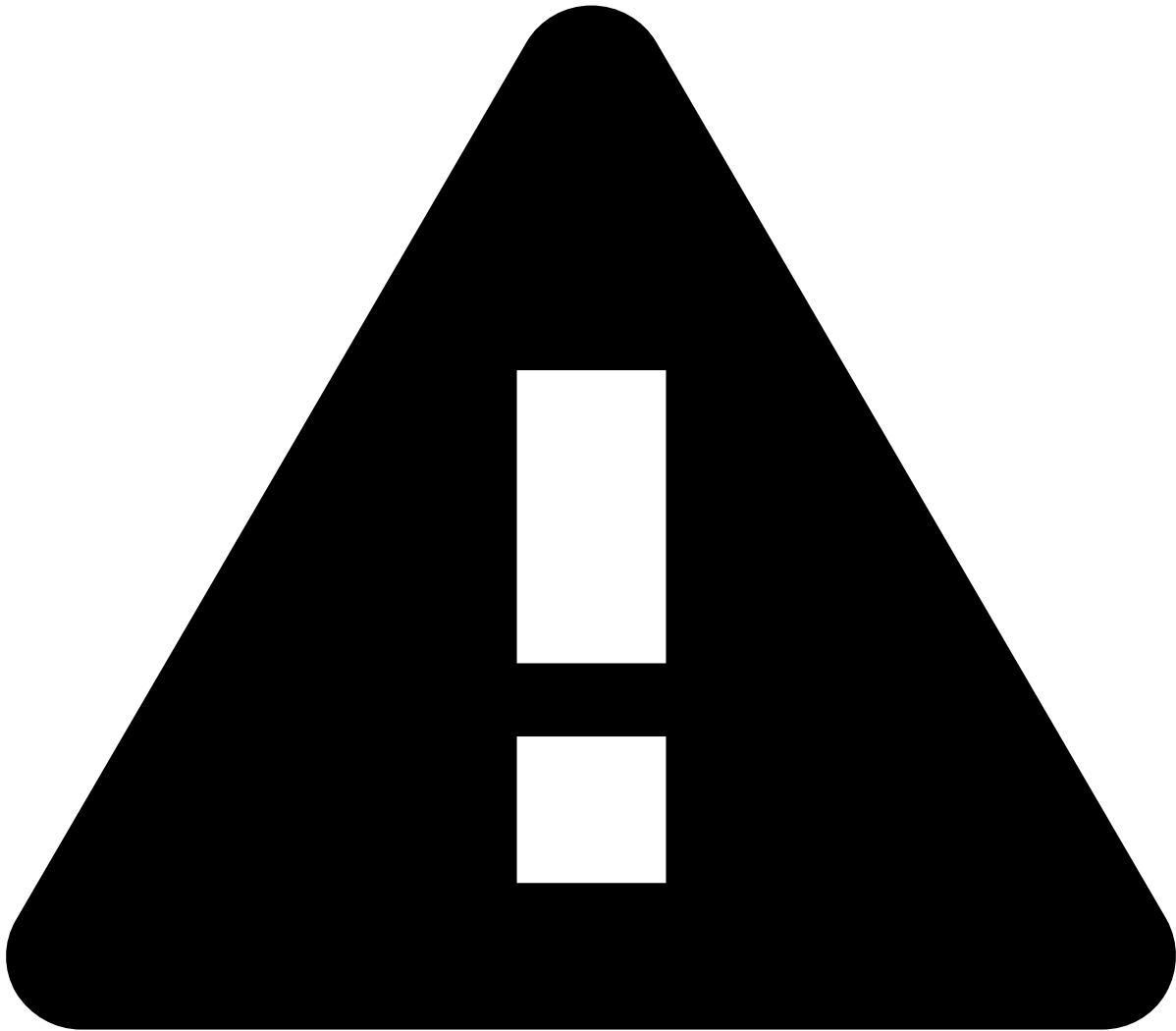
If there are fewer features in the file than in the model, or if some features are not fully filled, a warning will be displayed to alert you to of the missing information.



tip

Export the files after filling all the fields, so that when editing the explanations on a similar model you just need to import the file you saved before. This can also be done by going to the edit page of a model and exporting the explanations it currently has.

While editing, you can search for features with a specific text, sort the order of the features by their name or their group, and hide the features that you have filled by clicking the "Show only incomplete features" button.



caution

After creating a model with WBE, it is possible to edit explanations again. In that case, do not forget to update the [Scoring Service](#) afterwards.

Testing and validating your model

When training a model, Pulse can automatically generate an initial Model Evaluation.

To configure how the initial Model Evaluation is performed, execute the following actions while you are adding a new model:

1. Define the Validation Type to generate the initial evaluation. The following validation types are possible:
 - **Cross Validation:** Configure the number of folds in the dataset to generate the evaluation.
 - **Split Percentage:** Configure which percentage of the dataset you will use to train and which percentage you will use to test the model.
 - **No validation:** Pulse will not create an initial evaluation.
2. Select the **Validation Data Source** that will provide the data for the initial model evaluation. It must have the same schema as the training data source.

3. In the case of an unsupervised machine learning algorithm, select the **Target Variable** to be used for the evaluation. This is the field that the model aims to predict.
4. Enable an **Additional Cost Field** to be available for the model evaluation. Typically, you can select a field that represents the transaction amount to have metrics such as the *money recall* and *money FPR*.
5. Select which are the **Breakdown** fields and values to slice the results into. For example, *Card Present* vs *Card Not-Present*. Pulse will automatically display the number of possible breakdown combinations.



Breakdowns

The higher the number of possible combinations, the more computational resources will be consumed during training time, and the longer will take to train your model.

6. Configure the **Output Result** to generate a new Data Source and a CSV file in your filesystem, adding a Pulse risk assessment to each instance. The new Data Source includes extra fields containing the score distribution of the model among all possible target values of the Target Variable. For example, in a fraud scenario, Pulse adds two fields, one with the score for fraud, and the other representing the score for not fraud.

After Pulse finishes training your model, it automatically generates a Model Evaluation in your Model Project. For a model named *MyModelV1*, Pulse generates a Model Evaluation named *mymodelv1-trainEvaluation*.

Limitations📄

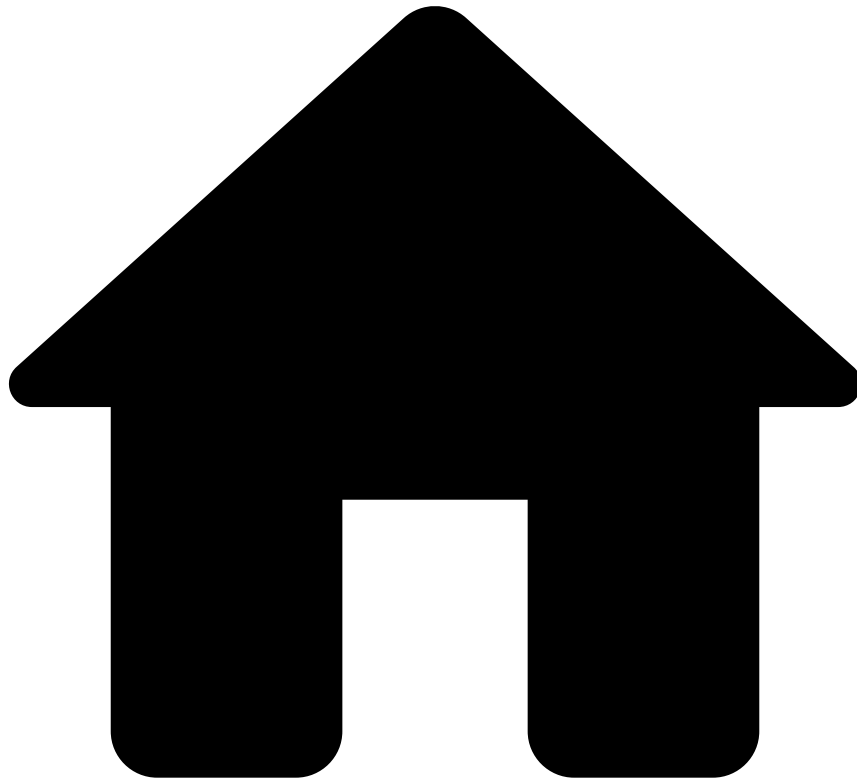
When training models with DataRobot algorithms, consider the following limitations:

Provider	Limitation	Workaround
DataRobot	DataRobot converts binary categorical field values with the following content to "True/False": • 1/0 • true/false • TRUE/FALSE However, when using the model to score transactions in Pulse, the input data may contain values in the above listed format. In those cases, the scoring of the transaction fails with an error.	n/a
DataRobot	Although string values can be used for features in DataRobot, they cannot be used for that purpose in Pulse.	Use a Categorical field instead.

Next Steps📄

Now that your model is trained, you can [evaluate its performance](#), [tune parameters](#) and [compare it with other models](#).

•



- [Model Guide](#)
- Tuning Model Parameters

On this page

Tuning model parameters

Was this page helpful? 

Why

To have the model with the best possible performance, you want to fine-tune parameters by doing multiple experimentations and combinations on them. The goal is to have a model that is better adjusted to a specific data pattern. To perform a hyperparameter Tuning is the fastest and easiest way to get to the model that yields the best results.

Pulse uses the **Parameter Sweep**, or in Machine Learning terms the **Grid Search**, to perform a hyperparameter tuning. This method is one of the standard methods used to retrieve meaningful results from a given search universe.

How to

To perform an hyperparameter tuning, proceed as follows:

1. In the **Data Science** tab, open a **Model Project** from the **Model Projects** list and start by adding a **New Hyperparameter Tuning**.
1. Select the **Data Source** you will use to execute your parameter tuning. Typically one that has been the output of [running a features Plan](#) or the result of splitting your dataset into training and testing subsets.
2. Set the **Target Variable** that you want to predict, like *Fraud*, for instance.
3. Choose the **Fields** the model will use to predict the **Target Variable**. Note that only *Numeric* and *Categorical* fields are available.
4. Use the default **Random Seed** or set a new one in case you want to replicate past results.
5. Set the **Sampling Type**. Note that if you select other than the **None** option, the **Sample Percentage** field is displayed for you to define how to undersample or oversample your data.
6. Select the **Provider** for the machine learning algorithm and the **Algorithm** itself. Hyperparameter Tuning allows you to add more than one algorithm, which in practice means that various algorithms are trained and evaluated, each containing multiple parameters simultaneously.



Adding algorithms

You can add the same algorithm unlimited times. This can prove useful to evaluate different combinations of parameters.

7. Configure the values you want to train both for **Common** and **Advanced** parameters. Each parameter field allows you to add multiple values. For information on the parameters, select the Info button .
8. Configure how the evaluations will be performed by selecting the Validation Type from the available options:
 - **Validation Data Source:** To select the data source that will be used for testing the model performance.
 - **Cross Validation :** To configure the number of folds the dataset is split into.
 - **Split Percentage:** To configure which percentage of the dataset will be used for training and which percentage will be used for testing the models.
9. Enable an [Additional Cost Field](#) to be available for the evaluations. Usually, you select a field that represents the transaction amount to have metrics such as the percentage of fraudulent amount caught (*money recall*).
10. Select which are the **Breakdown** fields and values to slice the results into. For example, *Card Present vs Card Not-Present* Pulse will automatically display the number of possible breakdown combinations.

When the hyperparameter tuning is complete, Pulse will display the Finished status, meaning that you can open and start analyzing your hyperparameter tuning.

Evaluating the results of hyperparameter tuning

After creating a hyperparameter tuning, you want to evaluate and explore its results to learn which are the evaluations that performed better in comparative terms. To evaluate the results:

1. Open the hyperparameter tuning.
2. Switch to the **Exploration mode** tab, which displays the results already filtered for that specific hyperparameter tuning.
3. Create **Targets** and configure them. Add a set of metrics that you find more accurate for your purpose, such as finding fraud, for instance.
4. Use the search box in Evaluations to filter the results, thus making it easier for you to visualize information.
5. Analyze the results.

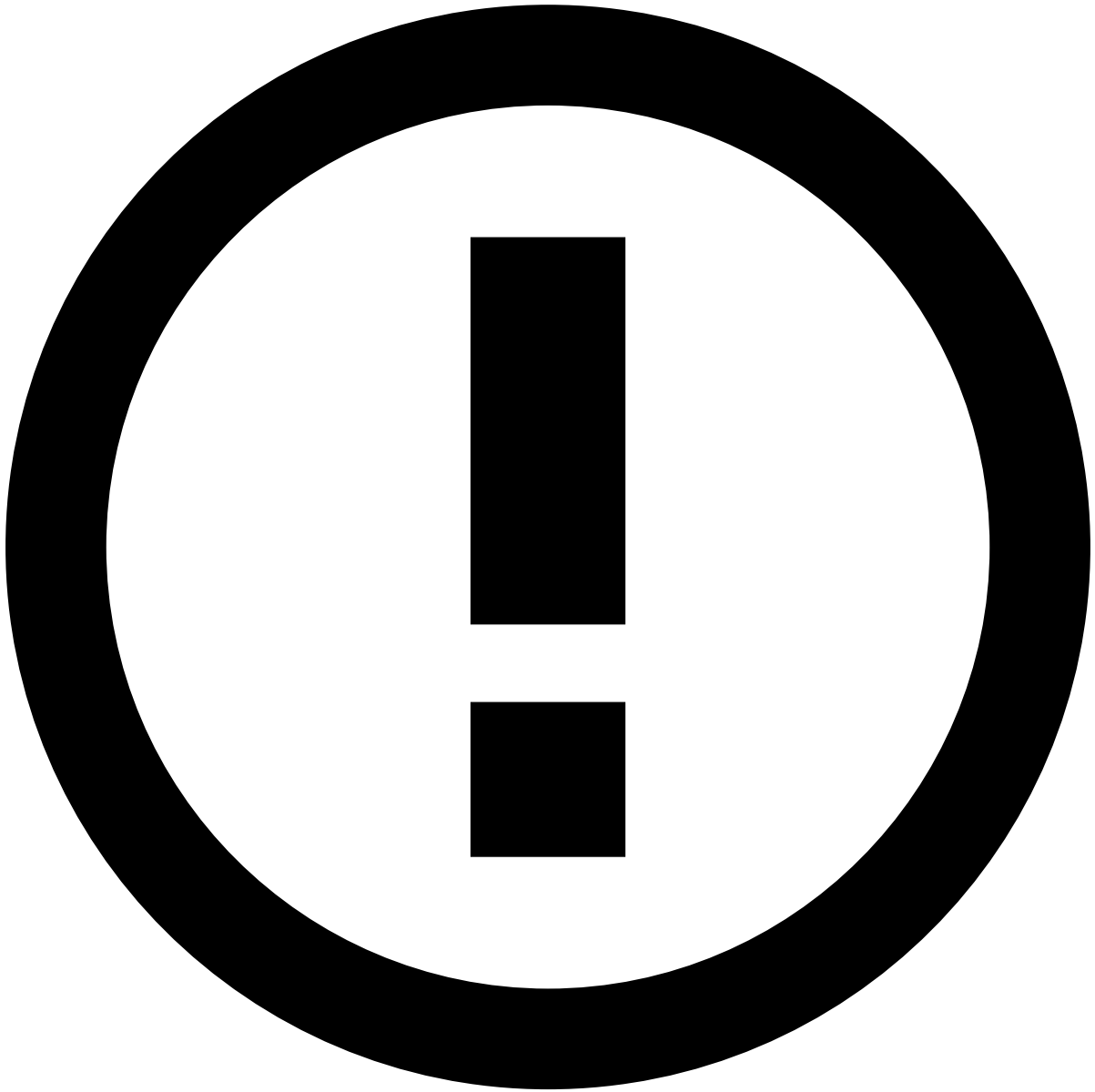
Learn more on the exploration mode and how to use it by reading the [Exploring hyperparameter tuning results](#) page.

Another way to evaluate and explore data is by opening the global exploration mode of models and hyperparameter tunings in the **Model Project**. Learn more about the global Exploration Mode by accessing the [Evaluating a model](#) page. This exploration view enables you to compare single model evaluations with hyperparameter tuning evaluations.

Exporting to a model

Once you decide on the best set of parameters, the next step is to export the corresponding evaluation to a model. This is a necessary step to later be able to apply your model to a workflow.

Since the main goal of hyperparameter tuning is to allow you to iterate on a significant amount of different parameter sets, Pulse doesn't generate model binaries for each combination of parameters. For this reason, it is necessary to train the model with the best parameter combination during the export phase. However, Pulse keeps the metadata with the results of your hyperparameter tuning. So, when you export to a model, all fields will be already filled in with the optimized set of parameters you have previously chosen and you only need to go through the steps by selecting **Next**.



Exporting a hyperparameter tuning to a model

Even though the model is pre-filled with the parameters, you can still make changes to any field of the model.

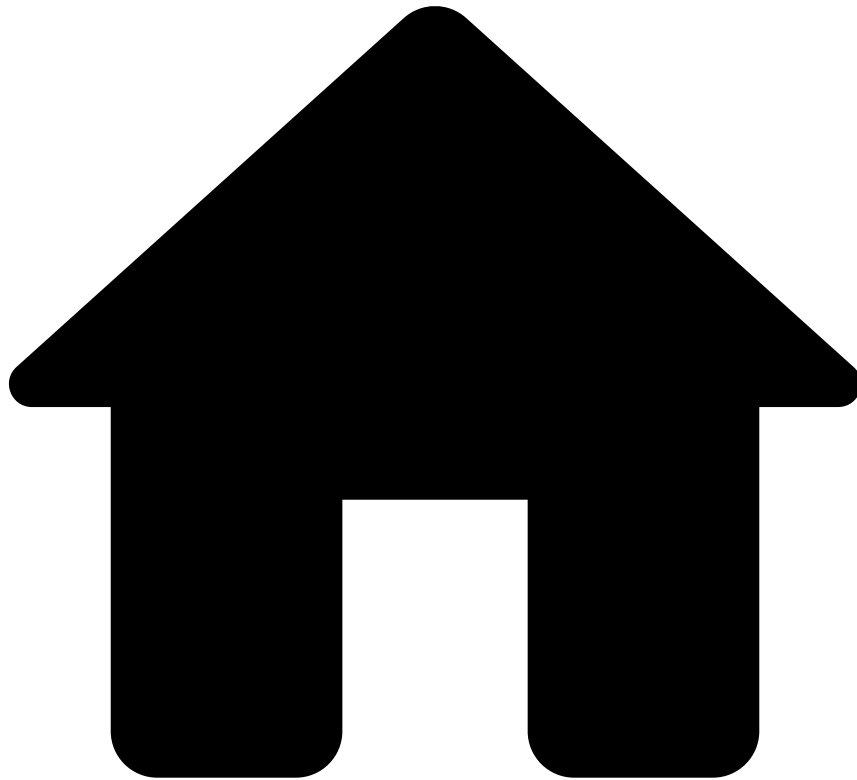
Proceed as follows in order to train your model:

1. Open the hyperparameter tuning from the corresponding list box in your **Model Project**.
2. Open the **Evaluations** tab.
3. Select the **Export to model** button of the corresponding hyperparameter tuning.
4. Go through the different steps by selecting **Next**.
5. Perform any other adjustment to a parameter, if needed. You may want to detail more on the evaluation of the model by defining breakdowns, for example.
6. Finalize the creation of the **Model**.

Next steps📖

You may want to compare this model against other existing model evaluations, for that consult the [Comparing models](#) page.

-



- [Rule Guide](#)
- Creating a Rule Snapshot

On this page

Was this page helpful? 

Creating a Rule Snapshot

Snapshots capture the current state of the components in a rules project, namely the rules and the execution order. Snapshots are deployable to other environments. You can use snapshots to test rule performance, as they can include an evaluation of the rules.

When working in a team, many colleagues may contribute to rules in Pulse, making various changes to rules at different times. When it's time to create a snapshot of the changes, it can be challenging to keep track of these changes. To ease this task, you can compare the current state of rules with an earlier snapshot's rules, and pick which versions you want to include in your new snapshot.

Also, in multi-tenant environments, the snapshot creation is slightly different. The following sections describe each of these cases.

Creating a simple snapshot

To include the newest version of rules in your snapshot, perform the following actions:

1. In the **Data Science** tab, select an existing rules project.
2. In the **Snapshots** area, add a snapshot.
3. **Name** the rule snapshot.
4. You can enable or disable rules in the snapshot using the checkbox next to the rule's name.

You can also choose the execution order by dragging rules into the right order.

5. Select the **Create Snapshot** button to create the rule snapshot with the selected name and configuration.

Combining rule versions

When working in teams, you may want to compare the current state of rules with an earlier snapshot's rule version, and combine the versions you want to include in your new snapshot. This option is available only for:

- Rules projects that are not stateful.
- Snapshots that were created in Pulse version 20.0 or later.

To combine rule versions that you want to include in a new snapshot, perform the following actions:

1. In the **Data Science** tab, select an existing rules project.
2. In the **Snapshots** area, add a snapshot.
3. **Name** the rule snapshot.
4. You can enable or disable rules in the snapshot by using the checkbox next to the rule's name. The rules you disable here won't be included in the snapshot. You can also choose the execution order by dragging rules into the right order. After selecting the rules and their order, select the **Compare** button to move forward.
5. In the **With** field, select a snapshot with which you want to compare the current state of rules.
6. In the **Snapshot Comparison** area, you can see the list of differences between the selected snapshot and the current status of the rules. The following statuses are possible:
 1. **Deleted**: a rule that existed in the selected snapshot was either deleted, or you deactivated it at the beginning of the new snapshot creation.
 2. **New**: a new rule that didn't exist in the selected snapshot.
 3. **Edited**: a rule that is present in the selected snapshot, but has been edited since then.
7. Use the radio buttons in the **Rule Version** column to proceed with the new version or to roll back. Select **Accept** or **Reject** accordingly.
8. For a side-by-side comparison of both versions of an edited rule, select the rule.

The pane below the Snapshot Comparison area displays both versions.

9. After selecting the versions you want to include in the new snapshot, you can go ahead and **create** the snapshot. Optionally, you can **preview** the new snapshot and create an evaluation for it. [Read how to configure a snapshot evaluation](#).

Creating a multi-tenant snapshot

When rules of a certain rules project apply only to a certain tenant, the steps to create a snapshot are slightly different. For each tenant, you need to select which rules you want to set enabled in the snapshot. To do so, execute the following steps:

1. In the **Data Science** tab, select the rules project.

2. On the **Tenants** tab, choose a tenant.
3. In the **Rules** area, use the button to select which rules of the tenant will be active in the snapshot.
4. Repeat the previous steps for each tenant.
5. Go to the **Overall** menu and add a snapshot.
6. **Name** your snapshot. Verify the information about the number of tenants with and without active rules.
7. **Create** the rule snapshot with the selected name and configuration.

-



- [Rule Guide](#)
- Creating a Sandbox Evaluation for Rules

On this page

Was this page helpful? 

Creating a Sandbox Evaluation for Rules

By creating a sandbox evaluation, you will be able to test and evaluate your rules on a transaction level, i.e. the overall decision of the rules project for a particular transaction.

How to

To create a sandbox evaluation, execute the following actions:

1. In the **Data Science** tab, add a sandbox evaluation.
2. **Name** the sandbox evaluation.
3. Specify the **Data Source**, i.e. the data on which rules will be tested.

4. If you choose to use **Partitioning Fields**, transactions will be partitioned by the selected fields. That means that if you select *Card* as a partition field, all transactions from that card will be processed together. Therefore, the node in charge of processing it will not have visibility of the transactions not associated to that card. This is typically what happens in production environments.

5. Select the **Outcome Variable**, i.e. the value against which the evaluation will process its results.

6. Choose the **Resources to Analyze**. You can add many rules, from different rules projects. For each rule you want to include, select the following:

- **Resource Type**: Rules Project.
- **Resource Parent**: the name of the rules project that includes rules you want to test.
- **Resource to Evaluate**: the rule itself.

You can reorder the resources so that your sandbox evaluations simulate a production workflow as close as possible. The sandbox evaluation will apply rules to the events in the defined order.

7. Choose your **Reporting** configuration. In this step, you will choose how the results of the evaluation will be presented.

- By selecting no **Additional Cost** and no **Breakdowns**, you will see all metrics in general.

If you add an additional cost field, the system will collect all those metrics that are also weighed by a data source field (e.g. making events with higher values of the field impact the metrics more).

- You can also choose **Breakdowns** to analyze metrics according to the possible values of a categorical field in the data source.

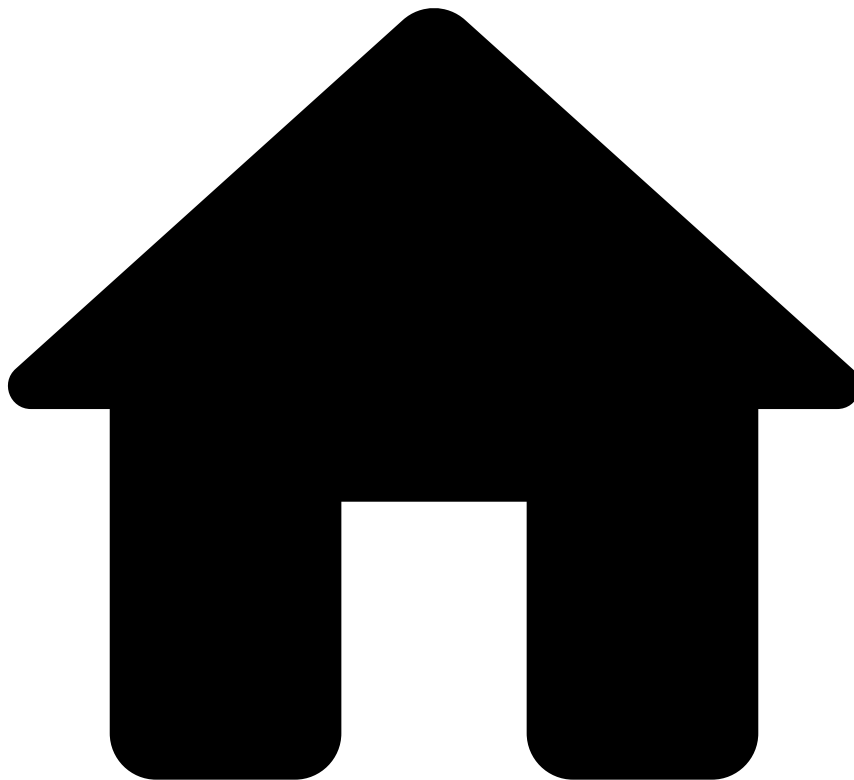
Breakdowns allow users to understand the impact of rules and models as segmented by the particular values of the breakdown fields that are selected, similarly to rule and model evaluations.

- In the **Output Result** area, configure the new data source that will be created with the evaluation results.

8. After running the sandbox evaluation, a new data source is created with the name you configured for the output result. Open the data source, and observe the new columns:

1. In the first new column, you can find the outcome per transaction.
2. In the second new column, you can find the score distribution per transaction. This will only be relevant if your rules contribute with a score for the final outcome.

•



- [Rule Guide](#)
- Creating a self-service rules project

On this page
Was this page helpful? 

Creating a self-service rules project

Why

You must set up Self-Service **Rules** in Pulse so that users in Case Manager can write **Rules** themselves. These simple Rules are written in a language that is a subset of the RDL language used in Pulse. They differ from the regular **Rules** by not having windows (i.e. metrics), although they can use them if they are computed by a **Plan** and not by the **Rules** themselves.

The first step of Self-Service **Rules** configuration is to create a simple **Rules Project** that will store the custom **Rules** that analysts write in Case Manager.

How To📖

Creating a Simple Rules Project📖

In order to create a Simple **Rules Project** in Pulse, you have to follow the usual workflow. To create a Rules Project, execute the following actions:

1. In the **Data Science** tab, add a new a Rules Project.
2. Set the Rules Project as **Simple**.



Self-Service Rules Projects

This kind of Rules Project is read-only in the UI after its creation and has an empty snapshot by default.

Managing Multi-Tenancy in simple rules projects📖

Simple rules projects are associated to a workflow element that belongs to a workflow inside an application.

There are two kinds of Rules Projects that can be created: Global and ByTenant.

- For non-tenanted solutions, the Rule Project must be configured as Global.

- For tenanted solutions, landlords view Global rules separately from the Tenant ones so the project can be of two kinds:



Multi-Tenanted Scenarios

In a Multi-Tenant scenario you might need to have two projects:

- One project set as **Global**: rules shared across tenants
- One project set as **By-tenant**: with tenant-specific rules
- Global: This kind of Rules Project contains the information (schema, decisions, etc) for the global rules.
- ByTenant: This kind of Rules Project contains the information for the tenants rules. Tenant Groups are not supported.

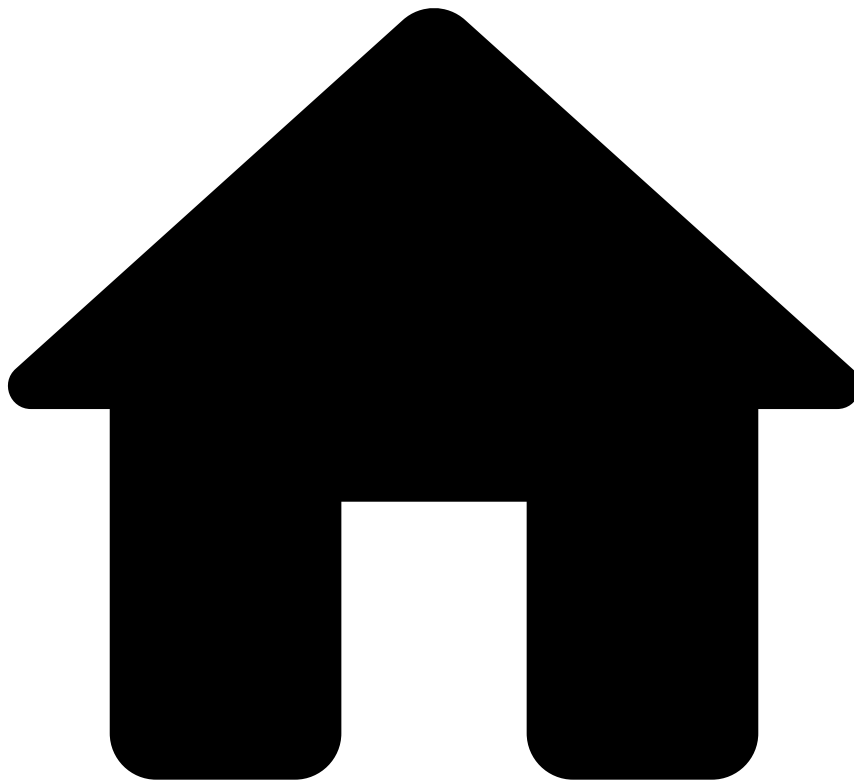
Next Steps

Learn more about Self-Service Rules and how to enable them in the following pages:

- [Adding rules](#): Learn how to add the Rules projects you have created to the workflow.

- [App Collaboration Loop](#): Understand the constant integration between the work developed in offline environments and the work in Production.

-



- [Rule Guide](#)
- Creating Features

Was this page helpful? 

Creating Features

This section shows how to create new features that can be used in rules:

1. [Creating a plan for the features](#)

In Pulse, a plan entity is a recipe of how to transform one data source into another. A plan contains map and metric operators that represent the new features. If you already have a plan for features, skip this step.

2. [Adding lists to the plan](#)

If you want to access list items of complex lists, you need to add them to the plan. For example, if you have a list with the information about the risk level each country has, you can add that list to the plan, and use it in your features and rules.

3. [Creating derived features](#)

Apply complex expressions over your dataset's existing fields to extract new features.

4. [Creating metrics \(historical profiles\)](#)

Generate a large number of profiles and include time-based features.

5. [Using PQL in features](#)

Learn how to use the Pulse Query Language (PQL) in map and metrics operators to define the "formula" of the new features.

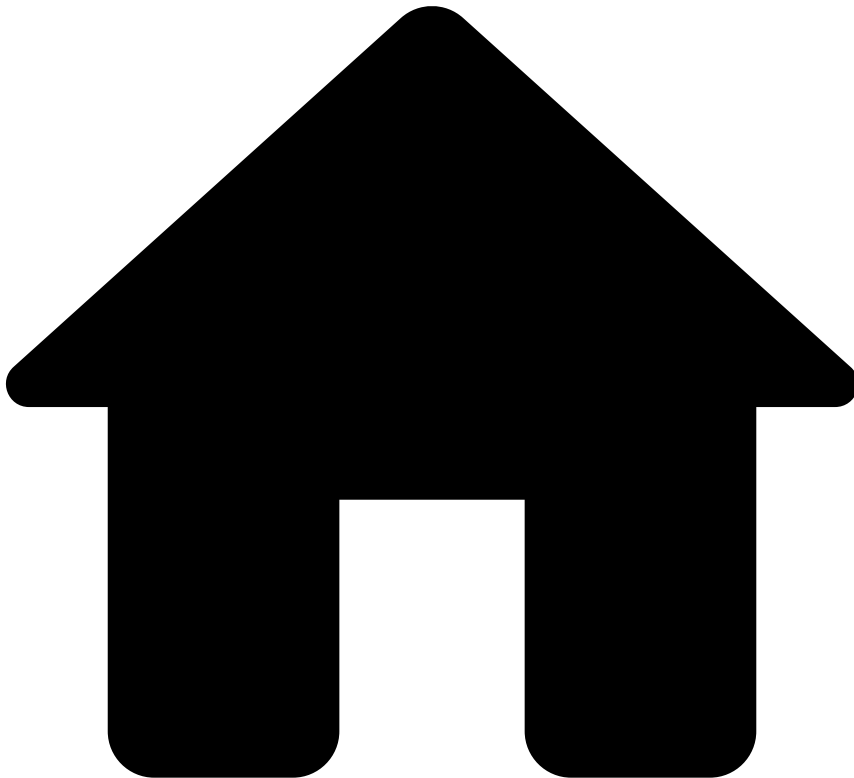
6. [Following best practices for plans](#)

Use best practices to organize map and metrics operators within a plan in a meaningful way.

7. [Creating the data source with the features](#)

Run the plan to create the data source that contains the new fields.

•



- [Rule Guide](#)
- Creating Lists and Adding List Items

On this page
Was this page helpful? 

Creating Lists and Adding List Items

A list stores items, for example blocked credit card numbers. A list can be used in rules to check if at your organization a field of a transaction is known for a specific characteristic. For example, the blocked credit card numbers are known for being correlated with fraud.

This page describes how to create a new list, and how to add list items to a list.

Creating a list

To create a new list, complete the following steps:

1. Go to the **Lists** tab and add a new list.

The **New List** form is prompted, as shown in the following image:

2. **Name** your list.
3. Select the **Value Type**, which determines what kind of list items the list will store.
 1. **Simple lists** consist of list items that contain data only in one field: the key. For example, a suspicious country list is a simple list if the only information that the list stores is the country code of suspicious countries.
 2. **Complex lists** consist of list items that have more than one field. The key field is still the identifier of the list item, but additionally there are other value fields. For example, a suspicious country list is a complex list if besides the country code (key), it stores a riskiness rate as well.

If you want to create a complex list, you need to specify the **Fields** that the list will store together with the key.
4. Select the **Key Type**, which determines if the unique key for the list items is a *string* or a *number*.
5. Select a **Case Matching** option: *Sensitive* or *Insensitive*. These options are only available for lists with the key type selected as string. After saving the form, the case matching cannot be edited.



Case-insensitive lists

In case-insensitive lists, the match between the list items and the value of the field of events processed by rules accessing this list is done by comparing case-insensitive strings. For example, consider a case-insensitive list of countries with the item *Portugal* (i.e. upper case). Imagine that you are processing transactions with a country field and a given transaction has an entry with a string *portugal* (i.e. lower case). In this case, a rule comparing these two values will be triggered.

6. Enter a **Key Label**. This is the name of the field that is the list's key.
7. Select a **Multi-Tenancy** option: *Global*, *By Tenant*, or *By Tenant Group*. This attribute controls who has access to the list items. In the case of options *By Tenant* or *By Tenant Group*, items added to a list of a tenant or tenant group are available only in that tenant or tenant group.

This attribute cannot be changed after saving the form.

8. Use the Save and add items button to save your list preferences and start adding list items.

Adding list items

To add items to a list, perform the following steps:

1. Go to the **Lists** tab and select the list you want to edit.
2. If you are adding items to a multi-tenant list, then use the **View as Tenant** or **View as Tenant Group** dropdown box to select the tenant or tenant group to which you want to add items.
3. Add items to the list.

By default, new list items are enabled and active, if no dates are set in the **Active from - To** fields.

Importing list items

Instead of adding each list item individually, you can import list items in bulk from a CSV file.

To do so, complete the following steps:

1. Go to the **Lists** tab and select the list you want to edit.
2. If you are adding items to a multi-tenant list, then use the **View as Tenant** or **View as Tenant Group** dropdown box to select the tenant or tenant group to which you want to add items.
3. Select Import Items.
4. Upload or drag-and-drop the CSV file that contains the list items you want to import.

The fields and data type in the file must match the fields and data types of the list in Pulse. To see the CSV file format Pulse expects, add a couple of items manually to the list, and export them. Then examine the export file for the necessary info.

5. Select if you want to **Add items** or **Replace all items**.

1. The **Add items** option adds every item in the file, overriding any existing key.

If you select the **Do not overwrite duplicated items** checkbox, Pulse tries to add all items that are in the file. If any of the keys already exists, the import is cancelled with an error. Select this option if you want to avoid updating existing items.

2. The **Replace all items** option deletes the whole list, and creates only the list items defined in the import file.

6. **Save** the import settings to start the import and return to the list. Pulse will show a notification when the import is finished.

Reviewing list changes

By default, changes to lists can only be reviewed after the fact. However, it is possible to set up 4-eye check functionality for lists, which places edits made to lists into a review queue, and only applies the changes if they are approved by a second person. To learn more, see [Reviewing List Changes - Four Eyes Check](#).

-



- [Rule Guide](#)
- Creating or Updating the Rules Project

On this page

Was this page helpful? 

Creating or Updating the Rules Project

Rules projects group rules that target the same variable of a specific data source, and can be executed in sequence.

This topic describes the following options for ensuring that your Pulse has a rules project for the rules:

- Creating a new rules project
- Updating an existing rules project in case you already had one before you added new features to the data source

Creating a rules project

If you don't have a rules project for the rules yet, create one by executing the following actions:

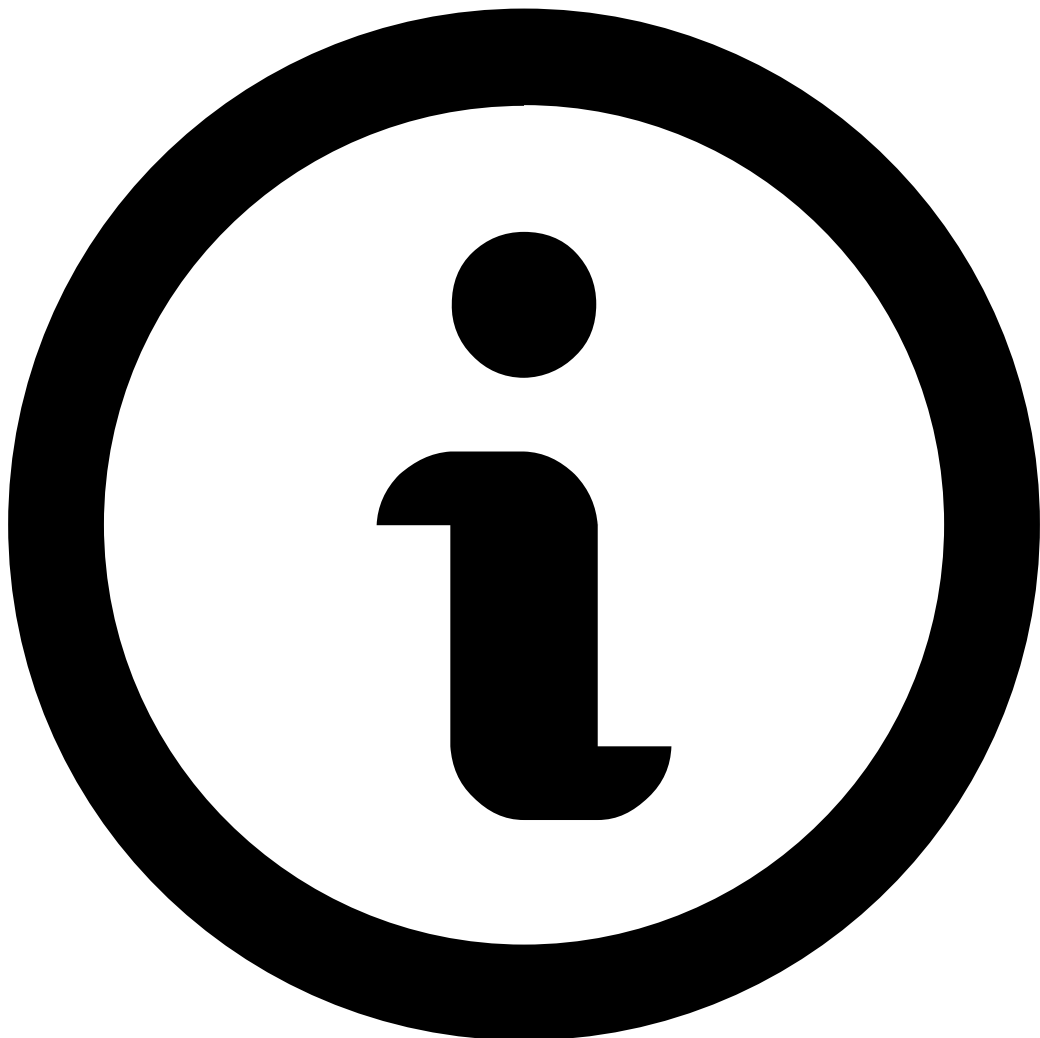
1. In the Data Science tab, add a new rules project.

2. Give a **Name** to your rules project.
3. Select *Advanced* for the **Type** of your rules project.
4. Choose the **Origin Schema** on which the rules will be based.
5. Choose a **Target Variable** which defines which field from the data source will be the target variable (e.g. Fraud/ Not Fraud). You can target any categorical field of the data source.
6. Create the list of **Actions** that will be possible to execute when the rules are triggered. For instance 'Block' and 'Accept'.

You can add parameters to pass information to the action you added. Use the settings button next to it to do so.

7. Select the **Multi-Tenancy** option that will be applied to the rules project:

1. *Global*: disables multi-tenancy for this rules project.
2. *Duplicated for Tenant*: applies all the rules to all the tenants included in the application.
3. *By Tenant*: allows selection of the tenants to which the rules will be applied.



Tenant alert

Tenant accounts should be created in the system, otherwise you can not manage multi-tenant rules projects.

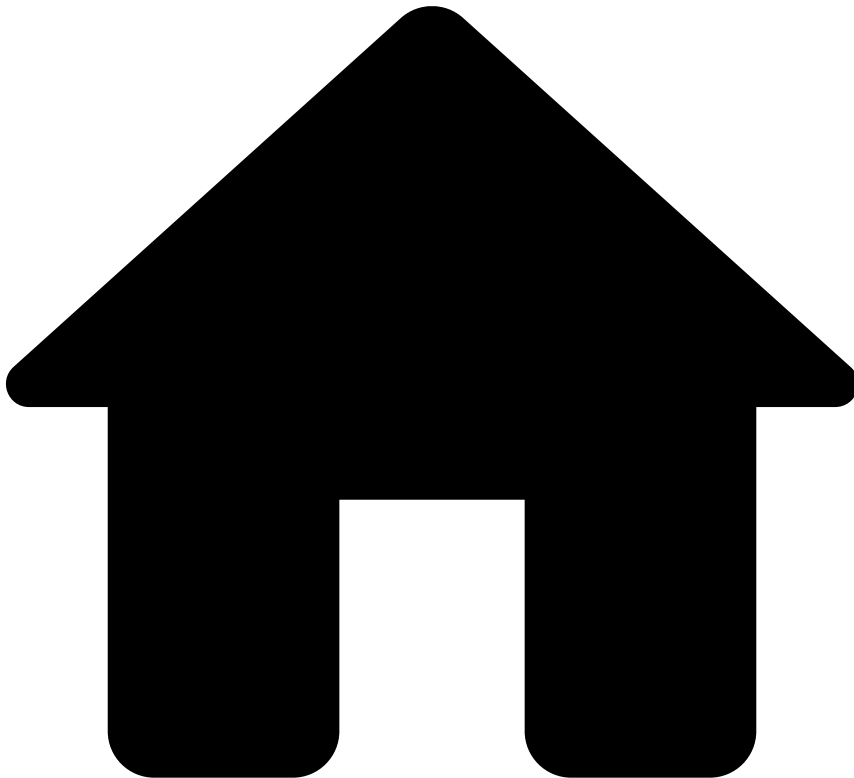
Updating an existing rules project

If you already have a rules project for your rules, and you added new features in the previous step, the rules project needs to be updated to reflect the schema changes. Namely, the schema associated to the rules project has to be changed to the schema of the data source which contains the new feature.

You can do this as follows:

1. Go to the **Data Science** tab.
2. In the **Rules Project** section, select the configure option next to the rules project.
3. For the **Origin Schema**, select the Data Source that contains the features you want to include in the rules.
4. Save the changes.

•



- [Rule Guide](#)
- Creating Rules

On this page

Was this page helpful? 

Creating a Rule

A rule, when used in a workflow, evaluates if a certain condition is true for the current transaction. According to the result, the rule can trigger actions in the workflow like rejecting or accepting a transaction, and contribute to the score of the transaction.

The following sections describe how to create a new rule, or edit an existing one after adding new features.

How to

To create a new rule from scratch, execute the following actions:

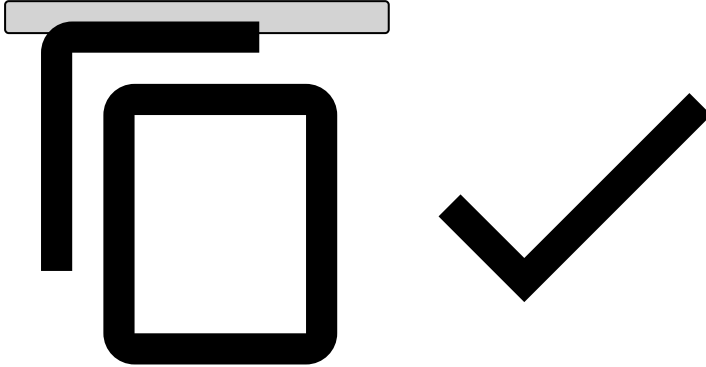
1. In the **Data Science** tab, select an existing rules project, and add a new rule.

2. Give your rule a **Name**, and describe it in a **Comment**.

3. Write the [Rule Definition Language \(RDL\) expression](#) for the condition of the rule. If a transaction matches the condition, it triggers the rule.

For example, to check if the current transaction's card number is on the watch list, the condition is:

```
CardWatchList.contains(TokenizedCardPAN)
```

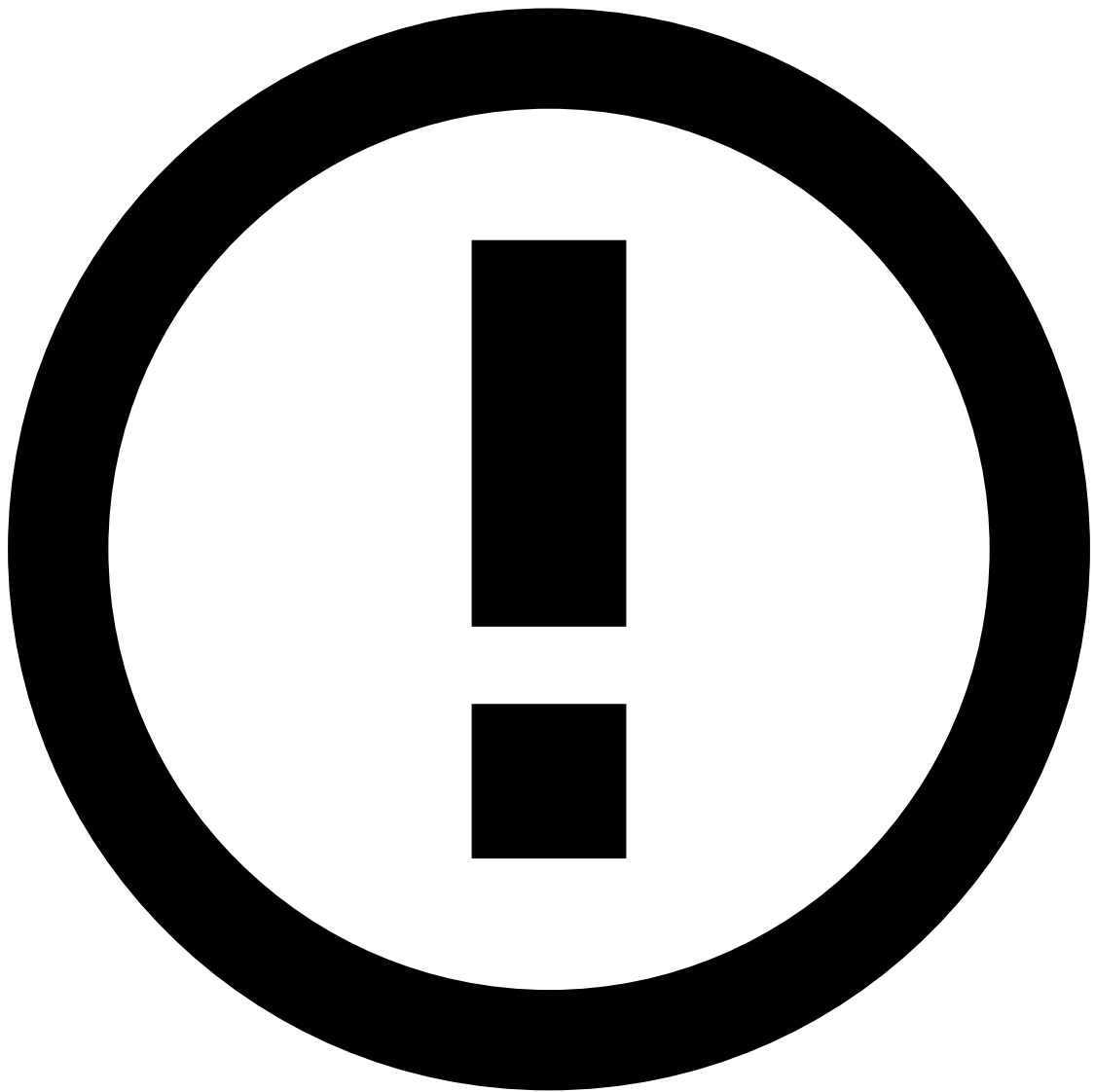




Best practice

Although it is possible to write complex expressions in the condition of a rule, it is recommended to [create a feature](#) instead, using a map or metrics operator in a plan.

You have access to **Lists** in the drop-down menu as shown in the figure below. If the type of your rules project is *Global*, you only have access to global lists. If the type of rules project is *Duplicated for Tenant* or *By Tenant*, you have access to all lists.



Rule Definition Language

Check the following pages to learn how to write expressions for rules:

- [Operators in rules](#)
- [Data types in rules](#)
- [Built-in UDEs](#)

4. Optionally, write an **Explanation**. The explanation can be shown to users who review transactions when the rule is triggered. You can type the explanation text and use data field values, list items, and variable values to give details to the reader.
5. The following four configuration options are available for you to control what consequences it has when the rule is triggered. They can be combined according to your business needs:
 1. Optionally, you can select which **Actions** of the rules project will the rule generate when triggered in a workflow.

Actions work like events, to which a custom workflow service can listen. For example, you can trigger the *Suspicious-transaction* action in the current rule. If you have a custom workflow service listening to the *Suspicious-transaction* action, the custom workflow service can send that transaction to an external

review system for analysis. A typical example of this is the Alert Storage Service, which sends the suspicious transactions to Case Manager.

2. Optionally, in the **Mark As** field you can select a value for the target variable field. This way you can mark an event as potentially fraudulent, or as a high risk transaction - depending on your use case and the target variable of your rules project.

If you don't select a value in the *Mark As* field, the rule won't give any value to the target variable.

Note that the possible values of the *Mark As* field come from the field that you set as target variable when [creating the rules project](#).

3. Optionally, you can set how the triggering of a rule generates a **Score Contribution** to a workflow outcome.

The **Score** of the current rule will be combined with other scoring elements of the same workflow: models and other blocks of rules. The **Weight** you assign to the current rule can be taken into consideration when calculating the final score.

4. In **Control Flow**, set how the workflow should proceed after triggering and processing the current rule. For example, you can choose to immediately stop the workflow when the current rule is triggered and processed.

This is especially useful when you want to ignore the evaluation of further rules, or even abort the rest of the workflow when a rule is triggered. Available options are as follows:

- **Continue**

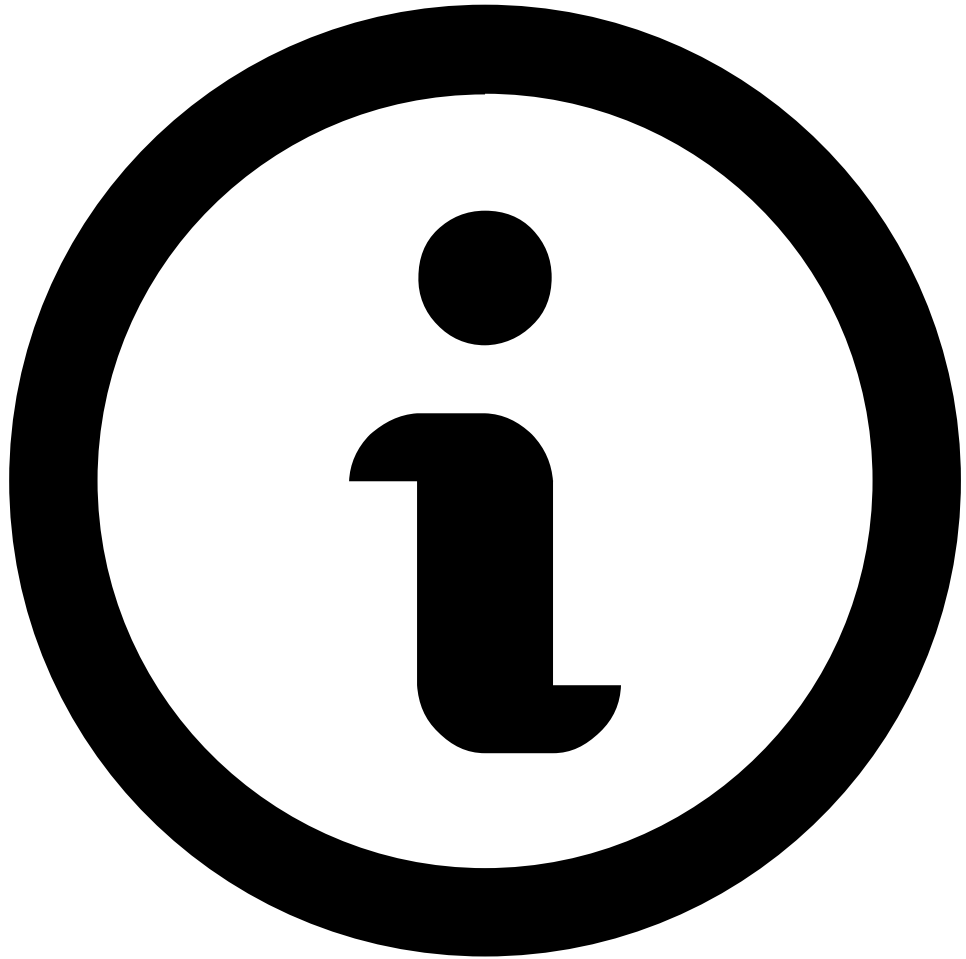
The execution continues with the next rule in the rule block. If there is no other rule in the rule block, then the workflow continues with the next workflow element.

- **Stop Workflow**

The workflow execution stops. Actions resulting from the current rule will be triggered for the current event.

- **Next Workflow Element**

No more rules are executed in this rule block. The execution continues with the next workflow element.



What happens to events after a workflow terminates?

The event is not lost. If the event storage is enabled in the workflow, once the event is out of the workflow it is stored in the database.

Editing a rule after adding new features🔗📄

If you added new features and want to include them in an existing rule, you just need to ensure that the origin schema of the rules project was updated. After that, the features will be available in the rules. You can edit the rule using the steps described in the previous section.

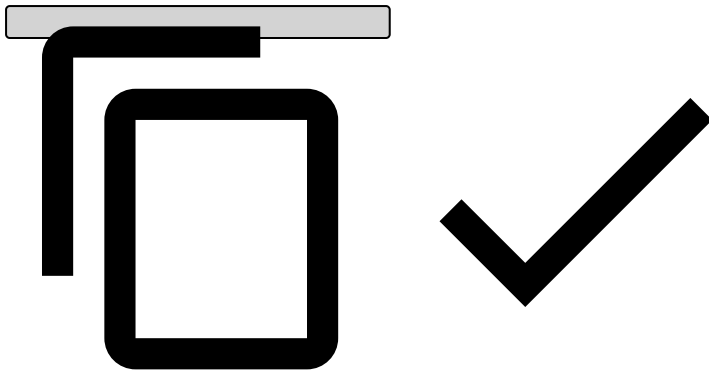
Examples of rules🔗📄

This section gives generic examples of conditions used in rules.

Count🔗📄

The following rule uses a [feature](#) called *Weekly_Count_per_Card_Pan*, which is the number of transactions in the last week with the same PAN card as the current transaction. The rule is triggered if the same card has been used more than twenty times in the last week.

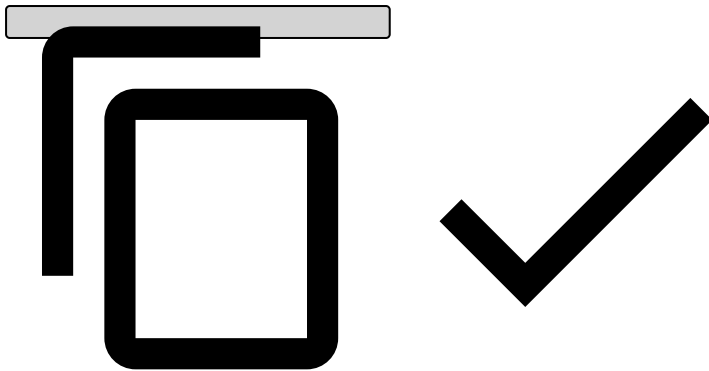
```
Weekly_Count_per_Card_Pan >= 20
```



Mismatch

The following rule uses two [features](#) that calculate the city of the customer's address, and of the device of the transaction. The rule triggers if the two cities don't match.

```
Customer_Addr_City != Device_IP_City
```

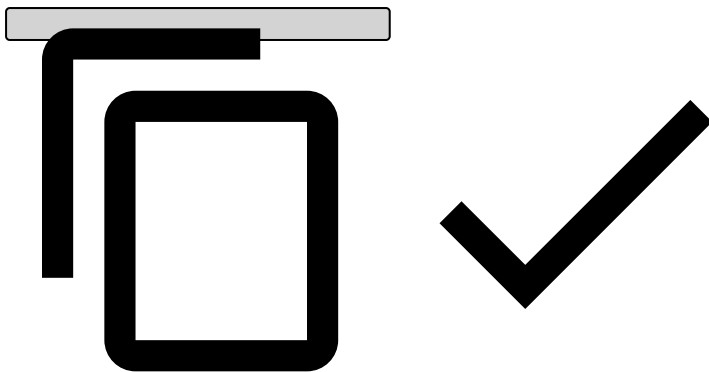


UDF call

To call a [User-Defined Function](#) (UDF) from RDL, the administrator has to define an alias for the UDF class in the Pulse properties.

The following rule calls a UDF that returns the keyboard distance of a string. The rule fires if the user's email address seems suspicious because the characters in it are very close to one another on a standard QWERTY keyboard.

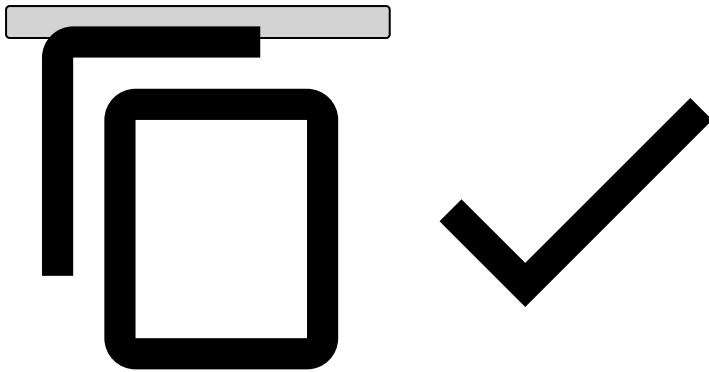
```
Strings.keyboardDistance(user_email_addr, 2) <= 5
```



List

The following rule condition uses a list. The rule fires if the IP address of the device associated to the current transaction is on a watch list.

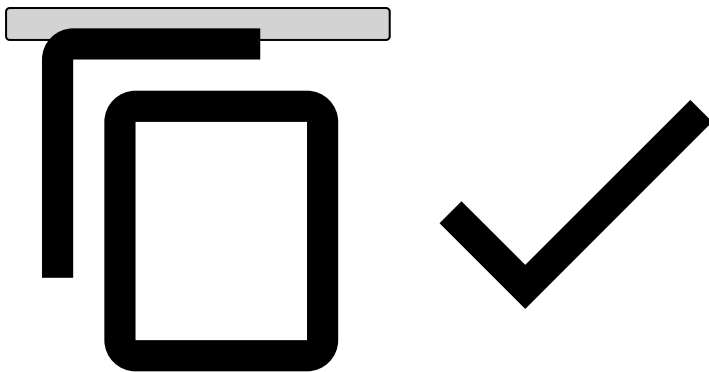
```
IP_WatchList.contains(Device_IP ?? "") // Tip: coalesce the argument to ensure it's not null.
```



Complex list

The following example shows the usage of a PosNeg list:

```
PosNeg_List.contains(Email_addr) and PosNeg_List[Email_addr].status == "block"
```



Using rule templates

Rule templates allow to create several rules with the same pattern, but with different values in custom variables. For example, you can define a rule template that checks if the transaction occurred in a certain currency, and the transaction amount is over a certain limit value. In this case, the currency and the limit value can be a parameter of the rule template. Then, you can instantiate this template with a different value for each country. This way you can have several instances of the same logic but with different parameters. Using a template in these cases avoids errors and allows you to maintain all the rule instances centrally in the template.

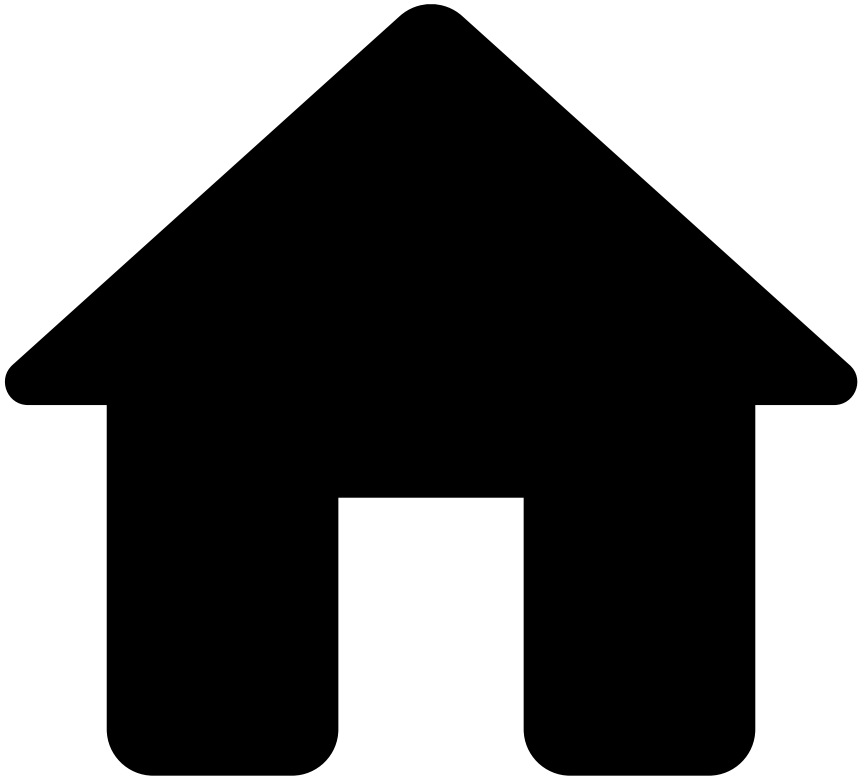
Example for rule templates

Let's see an example for checking if the transaction occurred in a certain currency, and the transaction amount is over a certain limit value.

The rule template, with two input parameters:

A rule that creates an instance of this template with a specific currency and a specific transaction amount:

•



- [Rule Guide](#)
- Deploying Rules to the Production Environment

On this page

Was this page helpful? 

Deploying Rules to the Production Environment

This topic gives a summary of how the rules are deployed to the development environment. Since the deployment is not performed by data scientists, the steps are not covered in detail in this document. The description gives only an overview of what happens next, as the planning process needs input from data scientists as well.

The deployment process of rules is different depending on whether this is the first deployment, or an iteration over an existing deployment. The following sections give a summary of both processes.

First-time deployment

The following sections describe the steps of deploying a project to production for the first time.

Planning the workflow and the outcome combinators

A workflow defines the journey of the transaction through Pulse.

A typical workflow contains a combination of the following steps:

1. Data input from the main data stream.
2. Data input from enrichers (e. g. card-related information).
3. Calculating new features based on the existing data fields.
4. Applying blocks of rules.
5. Running the scoring service with a model, i.e. applying a model on the data, thus contributing to the final score.
6. Running custom services. A custom workflow service is custom Java code that performs a specific task.
7. At the end of the workflow Pulse runs an outcome combinator that is configured for calculating the final outcome and score.

Outcome combinators

Outcome combinators define how to calculate the partial and final decisions and scores for an outcome in a workflow or sandbox evaluation. Outcome combinators take into account the triggered rules and applied models to provide a combination of all partial scores.

Pulse has a built-in outcome combinator, which calculates a weighted average of all the scores that the transaction got while traveling through the workflow. However, you can develop your own Java code to use it as a custom outcome combinator. This way, when a transaction travels through the workflow, the final outcome will be calculated by the custom outcome combinator according to your definition. Each of the triggered rules and applied models can be taken into consideration in a different way, and with a different weight.

Transferring resources and creating the workflow

In the previous step, engineers planned the journey of an event through Pulse. In this step, this journey is implemented in the form of a workflow of Pulse.

To create the workflow, engineers first export the resources (schemas, plans, rules, models) from the data science environment, and import them to the production environment. Then they create the necessary services (input adapters, enrichment services, outcome combinators, etc.) and use them in the workflow, implementing the journey planned in the previous step.

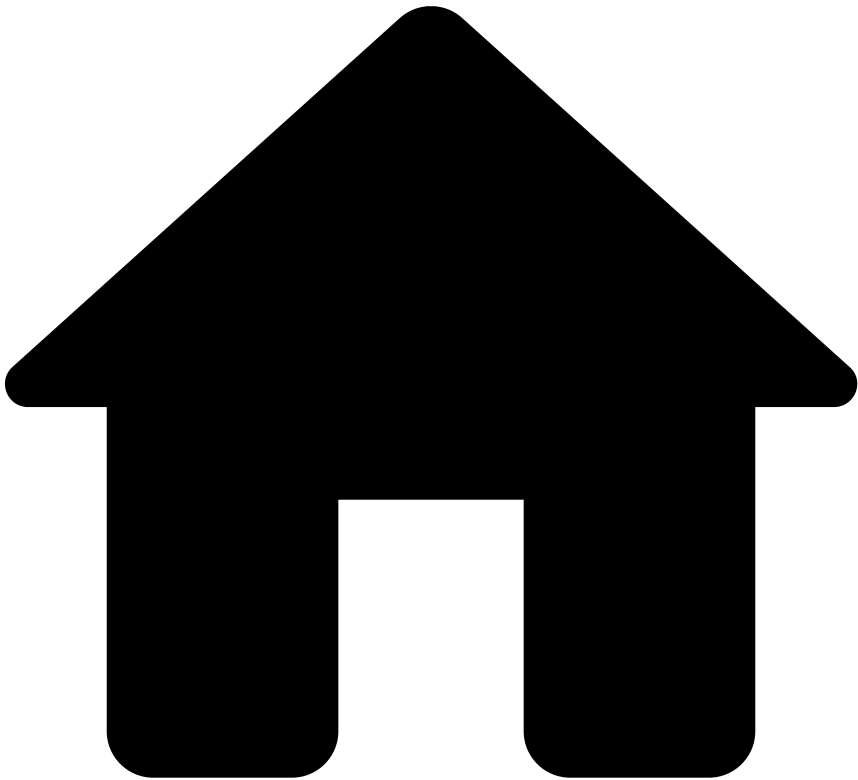
Updating an existing deployment

To deploy the rule changes from the data science environment to production, engineers first have to export the resources that changed (schemas, plans, rules projects, etc.). Then they import these resources in the production environment.

In some cases, while data scientists are working in the data science environment on the rules, in the production environment changes may have occurred. For instance, adding a new self-service rule, or adding a new feature to a plan in production. In this case, before transferring the changes from the data science environment to production, the data science environment needs to be brought to the same state as the production environment. The best approach to perform this is to have a freeze period on the production environment, while no new changes are introduced until the resource transfer is done.

After the resource transfer, engineers update the production workflow to use the new version of the plans and rules you created, and they publish the new version of the application.

-

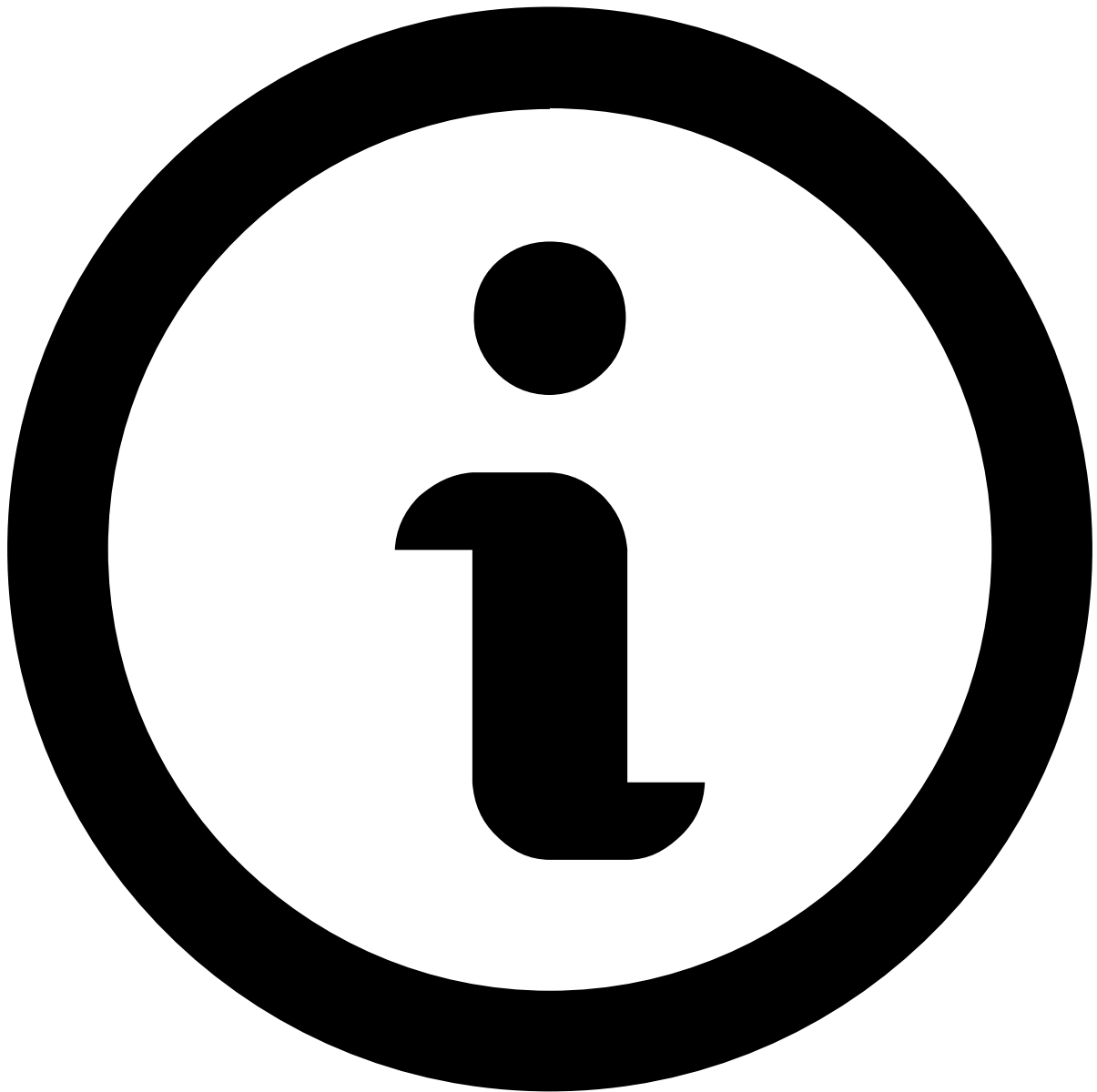


- [Rule Guide](#)
- Evaluating Rules Using a Snapshot

On this page
Was this page helpful? 

Evaluating Rules Using a Snapshot

You can evaluate your rules to assess their effectiveness, by analyzing a rule snapshot evaluation. This topic describes how to create an evaluation for a snapshot.



Multi-tenant rules projects

The snapshot evaluation is not available in multi-tenant rules projects.

Prerequisite

The following process presumes that you have a historical dataset where the feedback about the target variable's final value is saved in a separate field. This field is called the feedback field.

How to

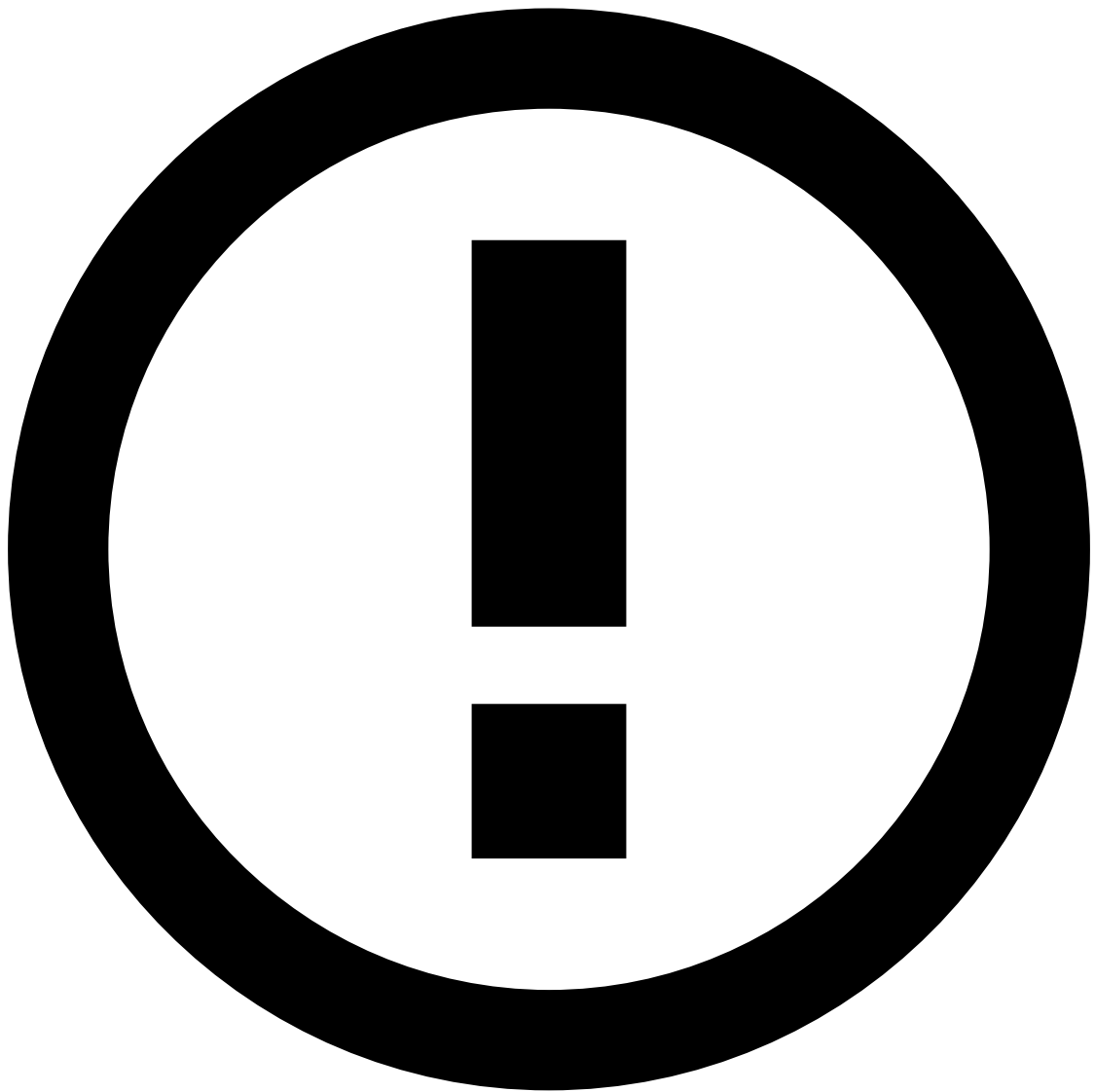
To run an evaluation of your rules over a historical dataset, execute the following actions:

1. In the **Data Science** tab, select the rules project that you want to evaluate.
2. The following step is to clone your rules project.

The objective of this step is to ensure that changes that you do merely for the performance assessment do not affect the rules project that you want to deploy in production later on.

To clone the rules project, execute the following steps:

1. If in your rules project you did not create a rule snapshot yet, [add a rule snapshot](#) now.
 2. Create a new rules project from the rule snapshot using the To Rules Project button in the list of snapshots. This new rules project is where you create and test your rules.
 3. In the **Data Science** tab, in the Rules Projects area find the newly created rules project and use the configure button to configure it.
 4. Change the **Target Variable** from the outcome field to the feedback field. For example, the outcome field can be the *Decision* field, with the possible values *Approve*, *Block*, *Review*. Change it to the feedback field, which is the one that gives the information on whether the transaction is Fraud (e.g. received a fraud chargeback or was marked as Fraud by analysts), or Not Fraud. **Save** the changes.
 5. Open the rules project. Go to every enabled rule and edit the rule by updating the **Mark as** field. Change the field to mimic production. For example, if it is a block rule, select *Mark As*, and select the value that the rule is supposed to act on, e. g. *Fraud*, *Not Fraud*, *Suspicious* or *Legitimate*. This information is what allows the system to measure the performance of the rules.
 6. **Save** the rules project.
 7. [Create a new snapshot](#) for the cloned rules project. Enable and sort the rules for the snapshot according to the project preferences.
3. Open the rule snapshot of the cloned rules project, and use the evaluate button to start creating an evaluation for the snapshot.



Evaluation performance

Evaluations run distributed in a Spark cluster, thus being able to efficiently process very large volumes of data in consistently short amounts of time.

4. Define the following configurations for the evaluation:

- Select the **Data Source** on which you want to test your rules.
- Define a **Filter**. During the evaluation, rules will be applied only to those instances that match the defined filter.
- Define an **Additional Cost Field**, such as *Amount*, if it makes sense in your scenario.
- Define the **Breakdowns** of the charts that will show the evaluation: select **Fields** and **Values** for which you want Pulse to break down and explore the data. Pulse will calculate metrics for every combination of all selected values.
- In the **Output Results** field you can use the **Export Evaluation** checkbox to export the output into a file. The output file will contain the individual rule contribution.

5. Use the evaluate button to start the job that calculates the metrics for the evaluation. The progress is shown next to the rule snapshot's name.

6. When Pulse finished to calculate the evaluation metrics (Recall, FPR, Precision, and so on), analyze the results in the snapshot, on the following tabs:
- **Block Evaluation:** shows the metrics calculated for the entire set of rules.
 - **Block Rule Evaluation:** shows the metrics per rule, but taking into account the Control Flow field set for each rule (Stop Workflow, Continue Workflow, Next Workflow Element). For example, if the first rule applied to the data is set to Stop Workflow, none of the remaining rules will be evaluated against transactions that triggered the first rule.
 - **Individual Rule Evaluation:** shows the metrics calculated for each rule individually, for each breakdown field combination (aka. series) of the evaluation. This tab presents evaluation results as if each rule runs individually, without taking into consideration the *Control Flow* option and always assuming the *Continue Workflow* option. By default, if more than one rule having the option *Mark As* are triggered by the same transaction, then the first triggered rule's value prevails.
7. You can show different evaluation metrics. To do so, use the drop-downs next to X-Axis and Y-Axis below the graph. To show different evaluation metrics in the Results table, use the **Choose Metrics** button.

What to look for in the analysis

Each business is different, and each scenario requires a different type of analysis. Still, there are some simple things that you can always check when looking at the metrics included in the snapshot evaluation:

- Does the current **block of rules** meet the expectations regarding metrics that are important to my business? The list of important metrics depends on the business scenario, but Recall, Precision, and FPR are good candidates to look at.
- Check the **Alert Rate** metric for the rule block and for the individual rules. Does it look healthy? Does a rule trigger too many times, compared to your experience on the field?
- If you exported the individual rule contribution into an output file (see Output Result field), you can check which rules triggered for each event.

This can help you to understand why a rule is performing differently from what you expected. It's also useful for validating whether or not the rules are catching the attacks they are designed to catch, by checking if they triggered for the corresponding events.

The output file is a .csv file stored in the path you defined in the Output Result option. For each event, the file contains a field called *Overall*, which shows if the event triggered any rule.

In addition it contains one field for each rule, which shows if the event triggered that particular rule. A value of 1 indicates that the rule triggered, and a value of 0 indicates the contrary.

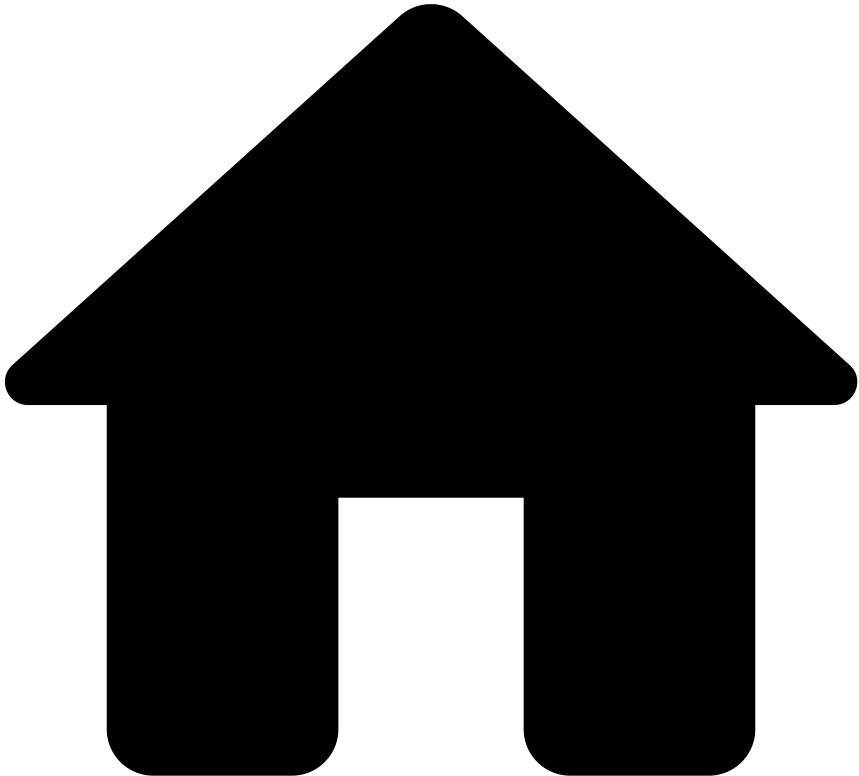
- The **breakdown field** values can help you to identify when a threshold value needs to be fine-tuned. Does a certain value trigger too many alerts?

Adding custom evaluation metrics

In addition to the metrics shown by default when assessing the rules performance, you can add your custom metrics. To do so, perform the following steps:

1. Select the options menu in your Pulse application.
2. Select the **Evaluation Metrics** option.
3. Add an evaluation metric to your project by defining a function that receives the following data:
 1. The number of True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN).
 2. The cost-TP, -TN, -FP and -FN. The cost field variants are based on a data field in the event that represents an amount, typically the transaction amount in a fraud detection scenario, rather than using a count-based approach.

•



- [Rule Guide](#)
- Evaluating Rules with Sandbox Evaluation

On this page

Was this page helpful?

Evaluating Rules with Sandbox Evaluation

This topic describes how to analyze rule performance using sandbox evaluation results.

How to

To analyze the sandbox evaluation results, go to the **Data Science** tab. From the **Sandbox Evaluations** list select the sandbox evaluation that you want to analyze.

Result by Score

The **Result by Score** tab focuses on the field of the **Outcome Score** and the real final label. Based on that particular field, this allows you to understand the performance of your sandbox evaluation. Therefore, you can understand how the system

behaves independently of the outcome decision that was specified on the outcome decision field. For example, it will help you decide whether to change the score threshold for the final decision.

In the **Result by Score** tab, you can perform the following actions:

1. Analyze the **ROC Curve** for the selected evaluation.



Exact values

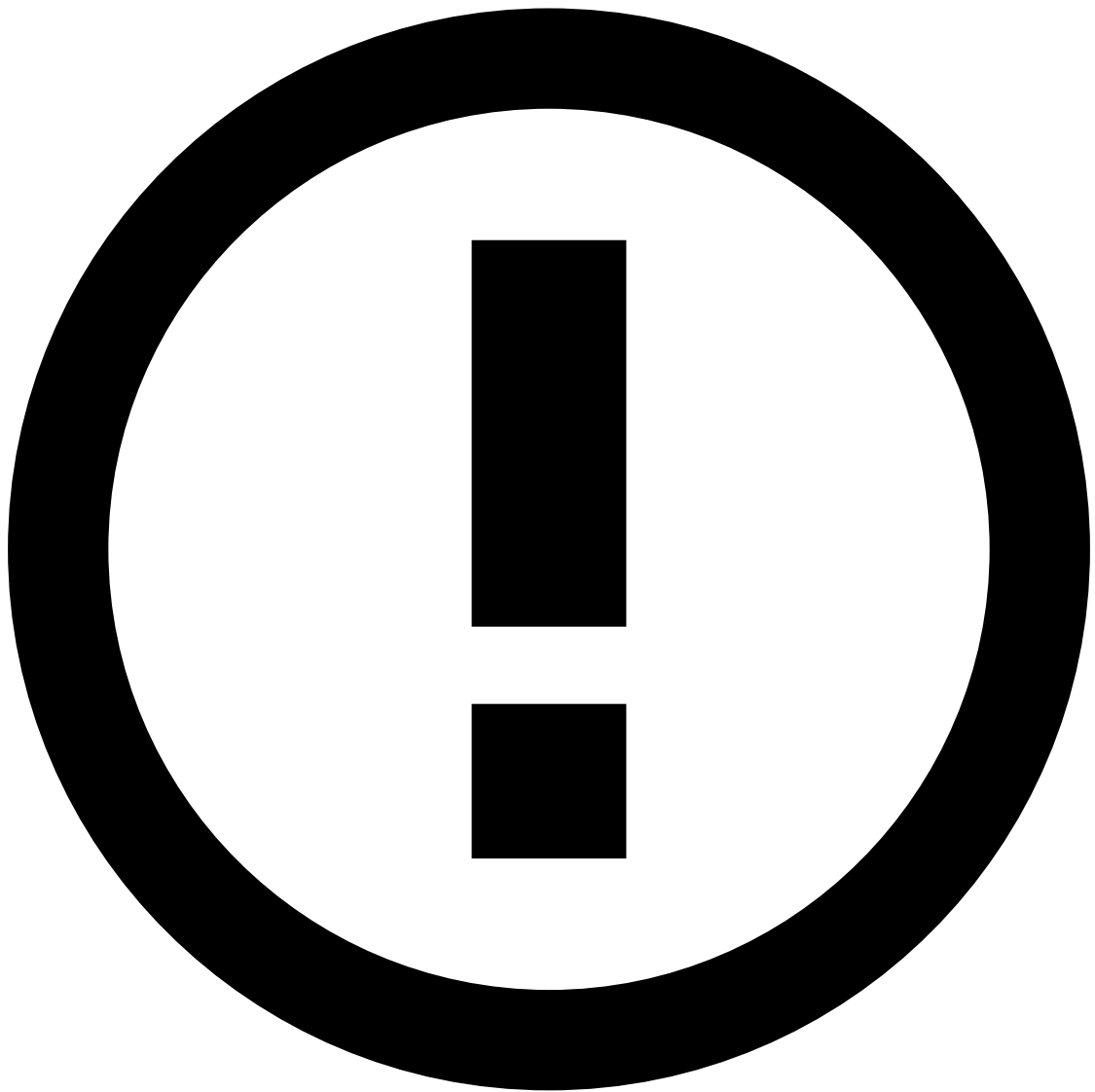
Hover the mouse pointer over the plot line to inspect the exact values for the **Recall** and **FPR** metrics and the respective **Threshold**.



Cost-based Metrics

If you define a **Cost Field**, then the **ROC Curve** will also plot the values for *Cost-Recall* vs *Cost-FPR*.

2. Configure the **X/Y Plot** to display a curve for any **X-Axis** and **Y-Axis**.



Exact values

Hover the mouse pointer over the plot line to inspect the exact values for the **X-Axis** and **Y-Axis** metrics and the respective **Threshold**.

3. During the creation of a sandbox evaluation, you can select the breakdown for the metrics. Therefore, here you can see all the values for the metrics of the segmentation and its combinations. You can also change the threshold.
4. Use the **View ROC curve** option to see which of the breakdown metrics can be seen in the ROC curve.
5. You can also see the metrics per service for all the resources the sandbox evaluation has. This is useful for understanding the performance computation times of the resources analyzed in the sandbox. It helps you to grasp the performance gains or losses you will experience in your live workflow and evaluate the trade-offs.

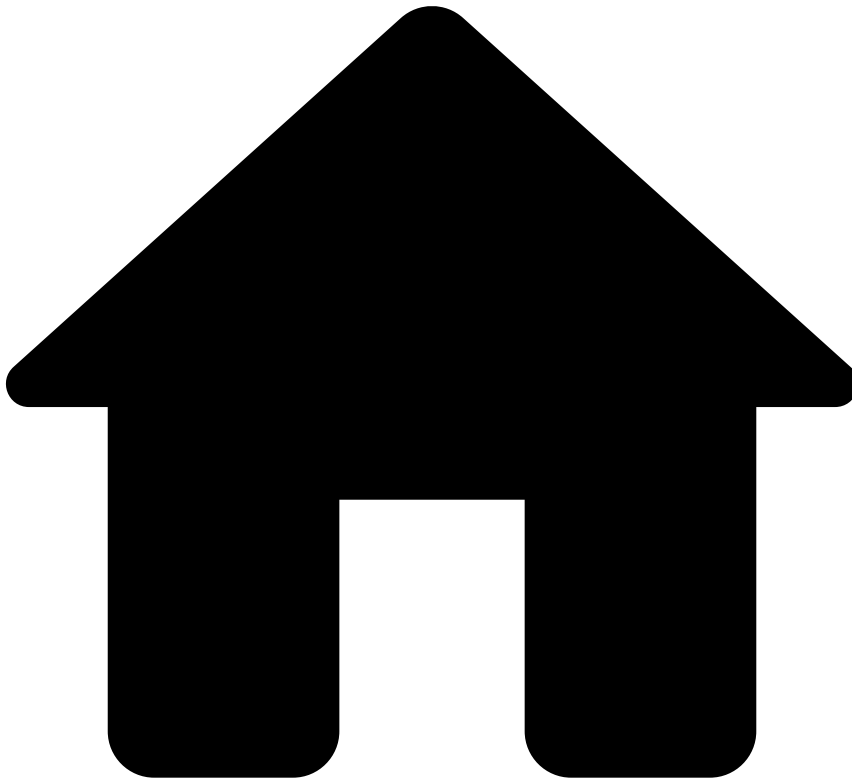
For instance, you can identify whether a rules project takes too long (in the **Ind. Avg.**) whilst not adding much to the performance (by analyzing **Recall** and **FPR** metrics), therefore allowing you to consider whether to keep it.
6. Export the results to a CSV file, if desired.

Result by Outcome

By analyzing the **Result by Outcome**, you can understand how the system behaves regarding the outcome decision that was specified on the outcome decision field. It works similarly to the **Result by Score**, but it does not provide a movable threshold. You can learn more about the performance of your Sandbox evaluation by analyzing these results:

1. Select the outcome values to be shown either if *True* or *False*.
2. You can export the results to a CSV, both **Per Breakdown** and **Per Services**.

•



- [Rule Guide](#)
- Reviewing Rule Changes

On this page

Was this page helpful? 

Reviewing Rule Changes - Four Eyes Check

When creating or editing rules, you may want to ask other colleagues to review your work to ensure rule safety and high quality. If four eyes check is enabled in your Pulse environment, you can request a review directly in Pulse.

This page describes both sides of the review: how to request a rule review from a peer, and how to give feedback if you are requested to review rules.

Prerequisites

To allow reviews of rule changes, an administrator must configure the following:

- The `fourEyeCheck.enabled` property must be set to `true`. After this property is set to `true`, you need to refresh the Pulse page in your browser to be able to request reviews.

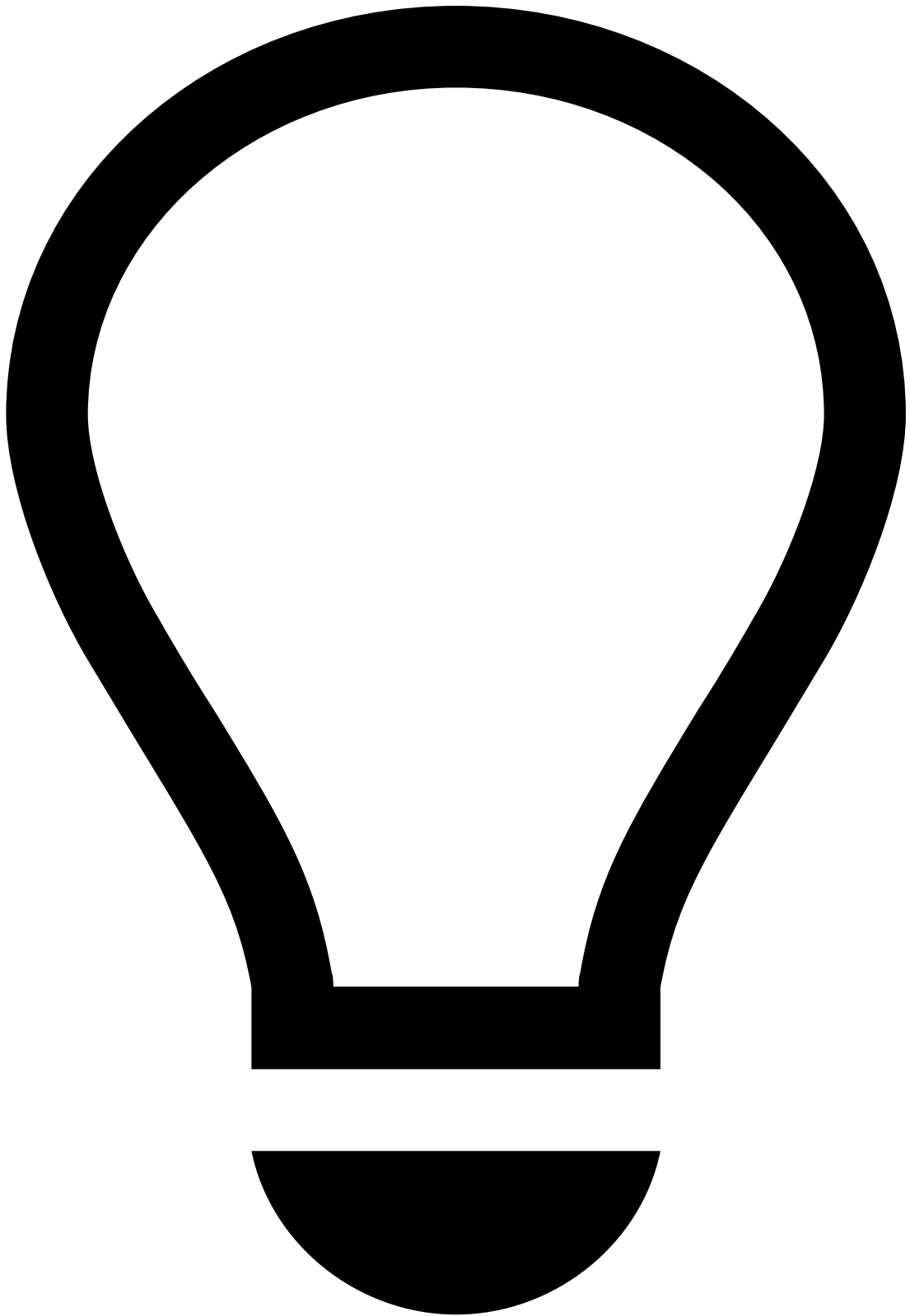
Optionally, for the notification emails to work as designed, the following settings can be configured as well. However, if you don't intend to use the notification emails, these can be skipped:

- The SMTP properties must be configured, so that the notification emails are properly sent.
- The `pulse.global.fourEyeCheck.notifications.email.from` property: sets the email address from which Pulse sends the notification email to the reviewers. By default, it's [\[email protected\]](#).
- The `pulse.global.ui.host` property is used to configure the URL where Pulse runs, and by default it's set to `http://localhost:8080/`. The notification email contains a link to the rule snapshot that needs to be reviewed, and the link starts with the URL set in this property. If the default value is not set to the actual value, the link in the email won't work.

Get in touch with Feedzai to set up four eye check in your Pulse environment.

Set up a static e-mail to receive all Four Eyes Check notifications

In addition to direct notifications sent to the author and reviewer in four eyes checks, Pulse allows setting up a static e-mail address which will receive all four eyes check notifications. This allows Pulse users to review the open revisions more thoroughly, and thus catch errors that might otherwise be missed. Note that the recipient of these emails will not be addressed directly, as they will be merely copies of the notifications sent to the author and reviewer of the corresponding four eyes check.



Notifying multiple users

This e-mail address can be set to that of a mailing list, which allows multiple users to receive every four eyes notifications.

To enable notifications for Pulse rules, provide the e-mail address via the following Pulse property: `pulse.global.fourEyesCheck.notifications.email.staticreviewer`.

Purpose of peer reviews

Ensuring every change is reviewed by a second pair of eyes helps prevent unwanted or unplanned changes from being published.

Snapshots that were not approved by a peer may still be published live, but the user importing the snapshot must explicitly confirm such a control bypass.



info

If the Four Eyes Check feature is enabled, changes by all users (even those with the approval role) are subject to a review from a peer. In other words, users cannot self-approve their own changes.

Request a review

To request a review of a snapshot, execute the following steps:

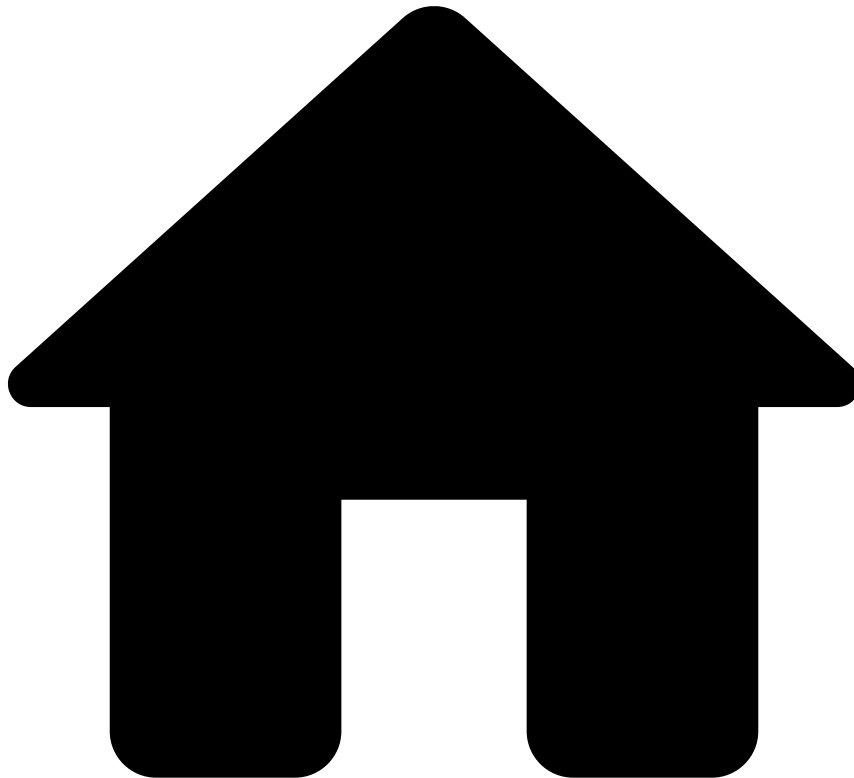
1. In the **Data Science** tab, go to the rules project that contains the rule snapshot. Under **Snapshots**, select the snapshot that you created.
2. Use the **Request Review** button to submit the snapshot for review.
3. In the **Submit Snapshot for Review** window, fill in the following fields:
 1. **Change Description:** Describe the changes you made to rules, data fields and lists. Include the name of the snapshot that contains the rule's previous state for reference, if there is such a snapshot. This will let the reviewer know the name of the snapshot they should compare yours to. Usually it is the snapshot that was last published live.
 2. **Notify User(s):** Type the name of the Pulse users that you want to invite to review the snapshot. In this field, you can select only users that have an email address configured in Pulse. If you don't find a user, contact your Pulse administrator.
4. Use the **Submit** button to send the snapshot for review and request the selected users to review the changes included in the snapshot.
5. As a result, the users you selected will receive an email with the following information:
 1. Review submitted on your behalf.
 2. The name of the rules project.
 3. The name of the snapshot.
 4. A link to the snapshot.
6. You can follow up on the status of the review next to the snapshot's name:
7. You will receive an email notification when the snapshot is accepted or rejected.

Review rules

If you are asked to review a rule snapshot, perform the following steps:

1. Open the link from your email, or go to Pulse in the **Data Science** tab and look for the snapshot submitted for review.
2. If there is a previous snapshot that you want to compare it with, use the **Compare with another Snapshot** button to see the differences.
3. When you have decided whether the snapshot should be published or not, select the **Review Snapshot** button.
4. Add your feedback in the **Explanation** field, and select the **Accept** or **Reject** button according to your decision about the snapshot.
5. The user who requested this review will be notified of your decision via email.

•



- [Rule Guide](#)
- Rules Glossary

On this page

Was this page helpful? 

Rules Glossary

Here you can find a short definition for the most important concepts that this guide refers to.

List

A list stores items, for example blacklisted credit card numbers. It can be used in rules, to check if at your organization a field of a transaction is known for a specific characteristic. For example, in your organization the blacklisted credit card numbers are known for being correlated with fraud.

Lists can apply to the entire Pulse environment, or a single tenant, or a group of tenants.

You can set the expiration date of a list item, meaning that after a certain date the list item will be inactive. It doesn't get deleted from the list, but it gets disabled. Rules don't consider disabled list items.

Outcome combinatorâ

Outcome combinators define how workflows and sandbox evaluations calculate the partial and final decisions and scores for an outcome. The default outcome combinator calculates a weighted average. However, you can develop your own outcome combinator service.

Planâ

Plans define a transformation flow of one data source into another data source. In a plan it is possible to add, update and remove fields, calculate metrics on the data, and even join two data sources.

They are useful to systematize data cleaning, data processing and feature engineering, both in a data science pipeline and in production.

A plan doesn't have any data bound to it, only the data schema. To actually compute the output data source it is necessary to run the plan; see [plan execution](#).

Plan executionâ

When your plan is ready, you can test it by running it. Running the plan generates a plan execution and an output data source. A plan execution represents a snapshot of the plan at the moment it was executed. You can analyze the performance of a plan by looking at the statistics in the plan execution.

To deploy a plan to production, for example a features plan, you must run it in advance and create a plan execution. This means that you must always test and sign off on a plan for production before it can be deployed.

Ruleâ

A rule is an elementary part of the real-time scoring system. A rule defines a set of consequences that happen if the current event of the workflow satisfies a certain condition. The consequences can be a combination of the following:

- Making an outcome decision about the event, such as marking a transaction as fraud.
- Contributing to the score of the event.
- Calling actions, which are asynchronous notifications. Custom services catch the action calls, and as a result they perform a specific task. You have to implement custom services using Java.
- Controlling the workflow, by determining what happens with it after the action is executed. You can continue with the next rule, or stop the workflow, or jump to the next workflow element.

Rule snapshotâ

A rule snapshot represents a static picture of a set of rules. It contains an evaluation of the included rules, which lets you assess the rule performance.

To use a block of rules in a workflow, you must create a snapshot about the rules first. Running a rule evaluation for the snapshot ensures that you always test and sign off rule blocks for production before they can be used.

Rule templateâ

Rule templates allow to reuse the rule definition in several rules. Sometimes it is useful to define the same rule with different parameters several times. For example, you may want to generate alerts if the total transaction amount for a card number in the last 4 hours is above a tenant-specific threshold. In this case, you don't have to copy and paste the rule's whole code for each tenant. You can create a rule template with the rule's code using a parameter for the threshold. Then you can instantiate that template in a rule, using the parameter to inject the right value.

Rules projectâ

A rules project contains a group of rules that target the same variable of a specific data source, and can be executed in sequence.

Additionally, rules projects can contain the following components:

- Rule templates, which make it easier to create many similar rules.

- Rule snapshots, which serve to evaluate the performance of rules, and to sign them off for usage.

Schema

A schema describes the available data fields of a data source, as well as their types.

Tenant

Tenants are useful for partners of Feedzai, to separate their customers' data. Each tenant can represent a customer, having access to that tenant's data exclusively.

Tenant group

Tenants are organized into tenant groups. They help partners of Feedzai to manage similar tenants in a centralized way. For example, they can apply the same configurations to all tenants in a group.

Tokenizer

The Tokenizer is a component of Pulse that transforms sensitive information (e. g. customer name and credit card number) into tokens that can be shared without security risks. Data tokenization creates a more secure system that complies with PCI-DSS security standards.

Workflow

Workflows represent the life cycle of an event through the system. They define the path that information traverses between components that may filter and/or transform the event, so that Pulse can calculate decisions and scores.

-



- [Rule Guide](#)
- Monitoring Rules

Was this page helpful? 

Monitoring Rules

Once the new rules are deployed, it is important to monitor their performance closely, so that they can be adjusted or disabled in case of unintended behavior.

By using the Rule Monitoring dashboard in Feedzai Insights, you can select a rule or a group of rules and view the number of times they were triggered over time. Learn more about the Rule Monitoring dashboard in the Insights documentation.

-



- [Rule Guide](#)
- Reviewing Rule Changes in a Snapshot

On this page

Was this page helpful? 

Reviewing Rule Changes in a Snapshot

Rule snapshots capture the current state of the rules in a rules project. In addition, snapshots represent a unit of rules that is deployable to other environments.

If you edited existing rules, you may want to validate if you made the right changes, before using the new rule version. This topic describes how you can compare rules in two different snapshots of Pulse, to validate if you have made the changes you planned to.

The same procedure can be used for audit purposes, if you want to understand what changed in the rules compared to an earlier state.

Prerequisites

A rule snapshot is available for comparison if the following conditions are true:

- The snapshot was created in Pulse version 20.0 or later.
- The snapshot was created for a rule project that is not stateful.

How to

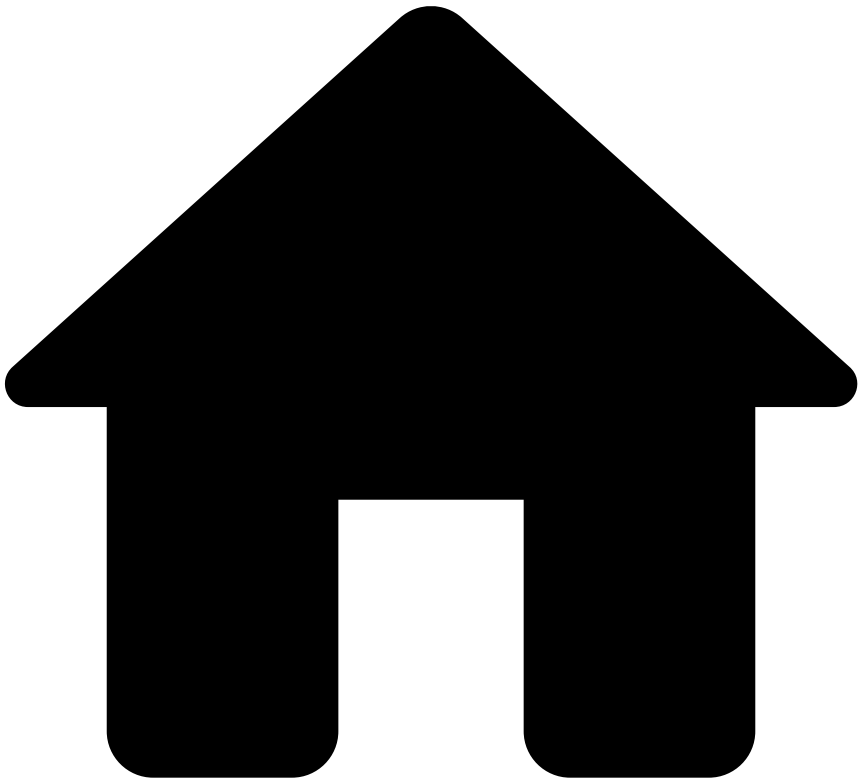
To compare the rules in two snapshots, execute the following steps.

1. In the **Data Science** tab, select the rules project that the snapshots were created from.
2. In the **Snapshots** area, select the snapshot that you want to review.
3. Select the **Compare with Snapshot** button, which takes you to the Compare Snapshots page.
4. In the **With** field, select the other snapshot you want the first to be compared with. Typically, this second snapshot is an older version.
5. In the **Snapshot Comparison** area, you can see the list of the rules included in the snapshots, and what changed from one to the other. The following statuses are possible:
 - **No change**: the rule is present in both snapshots, in an identical form.
 - **Deleted**: a rule that existed in the second snapshot isn't present anymore.
 - **New**: a new rule that didn't exist in the second snapshot.
 - **Edited**: a rule that is present in both snapshots, but with different content.
6. To see a side-by-side comparison of the two versions of an edited rule in the two snapshots, select the rule. The pane below the Snapshot Comparison area displays the two versions.
7. In addition, if the rule order is different in the two snapshots, you see this stated at the top of the Snapshot Comparison list.

Learn more

Learn more about [auditing in Pulse](#), and how to [investigate changes and user actions](#) using the audit logs.

-



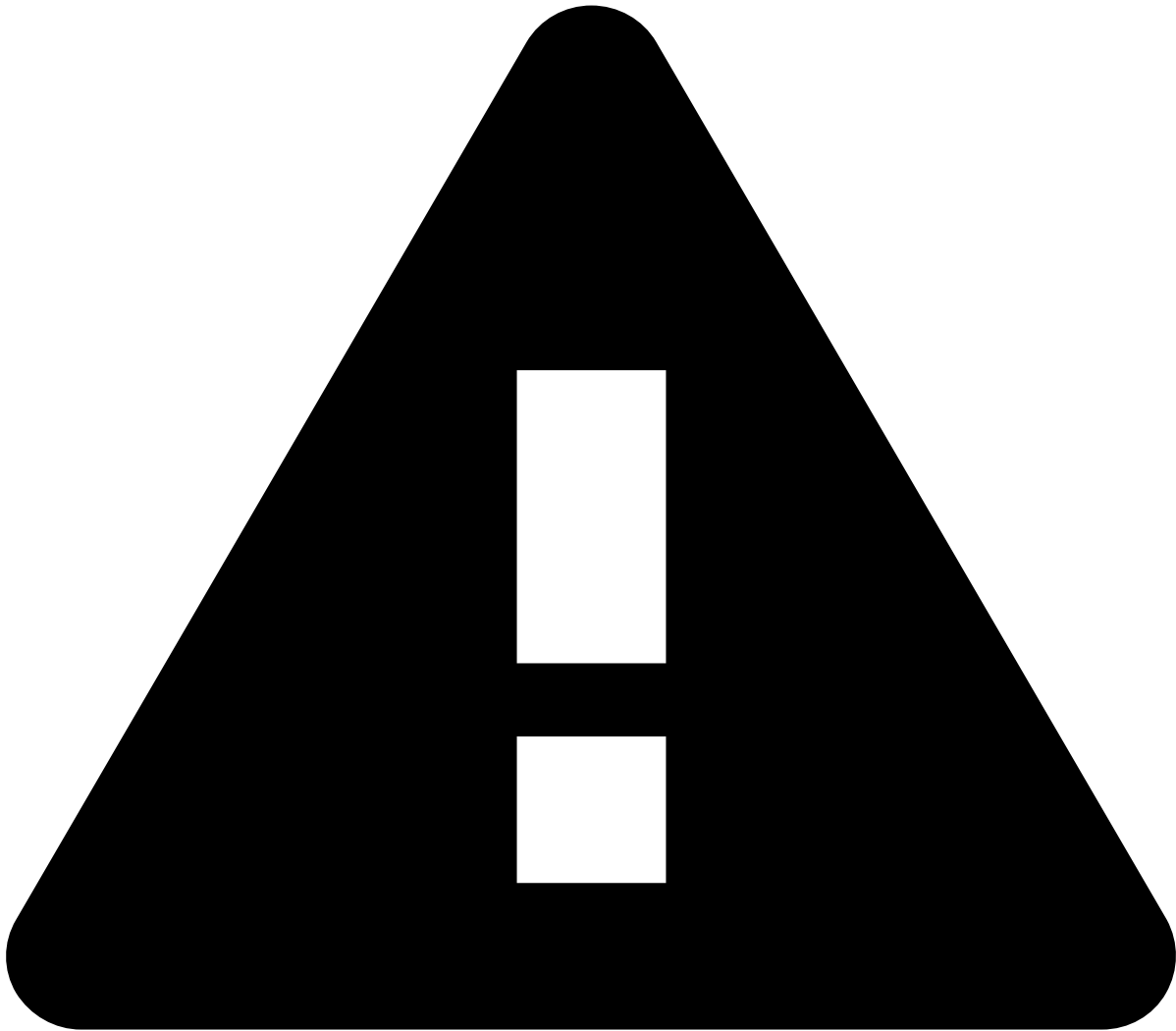
- [Rule Guide](#)
- Rules for Advanced Users

On this page

Was this page helpful? 

Rules for Advanced Users

This page provides advanced examples of using Pulse to write [Rules](#), via a selection of use cases.



The Examples are not Production Code

Note that the examples were simplified to better showcase the concepts and constructs of RDL. The code available in this page may not cover all the cases of each scenario and should not be used without review in Production environments.

Access to Historical Data🔗

In this example we will see a use-case where we need to access historical data. Let us say we want to detect three consecutive transactions with an amount greater than \$1000 that occurred in the last two hours.

The first thing to do is to write some auxiliary variables. These will contain the main logic that will be used in the condition block.

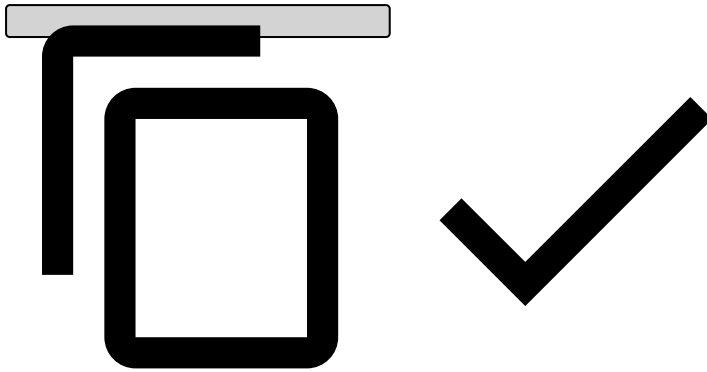
Variables

```
// Collect all transactions in a two hours window
window2hours = event[every 3 tuples for 2 hours where amount >= 1000];

maybeFraud = match
  if (event.amount < 1000) then // This breaks the consecutive >= $1000 transactions.
    window2hours.clear();
    return false;
  else if (window2hours.size () == 3) then // We got 3 transactions >= $1000 within 2 hours.
    return true; // We got 3 transactions in our 2 hours window. Maybe that is fraud.
```



```
else
    return false; // Still no fraud.
```



We start by declaring a window where events will stay for 2 hours, since that is our time-span. Also, that will contain at most 3 events that specify the `amount >= 1000` predicate. Every other event will not be part of the window.

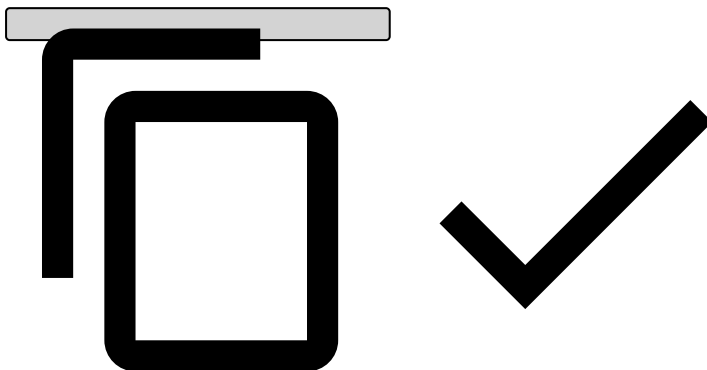
So, every time a new event arrives, it will either enter the window or not. Either way, we still have to decide what to do with this event. This leaves us with three possible scenarios:

1. **The event has an amount of less than \$1000** - This breaks our requirement of three consecutive transactions of more than \$1000. In this case, we must explicitly clear the window, because previous transactions of less than \$1000 can no longer be taken into consideration for our fraud pattern.
2. **The event has an amount greater or equal than \$1000 and the window already has three events** - This represents the fraud pattern we are trying to detect: three consecutive transactions over \$1000 over a two hours window.
3. **The event has an amount greater or equal then \$1000 and the window still has less than three events** - The current event contributes to our pattern, but it is still not the third consecutive transaction over \$1000.

On each incoming event, after we have gone through all possible scenarios, the `maybeFraud` variable will be **true** if the pattern has been found or **false** otherwise. The condition will be a simple check on this variable:

Condition

```
maybeFraud
```



Using Simple Lists to Group Data

The most common case for using a simple [List](#) is to maintain lists of items and then check for the existence of a specific item. For example, to maintain a list of blacklisted card numbers:

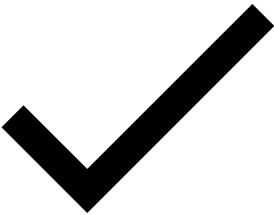
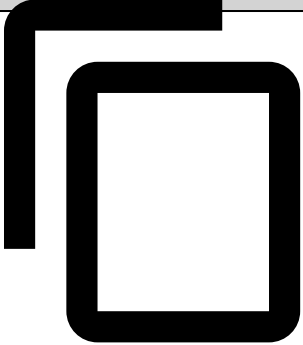
Figure 1. Creating a New Simple List for Blacklisted Card Numbers

Figure 2. Adding Items to the Card Blacklist

To check if a card is in a list write the condition as follows:

How to check if a card is in a List

```
card_blacklist.contains(event.card_number)
```





Explicitly iterating over Lists is not recommended

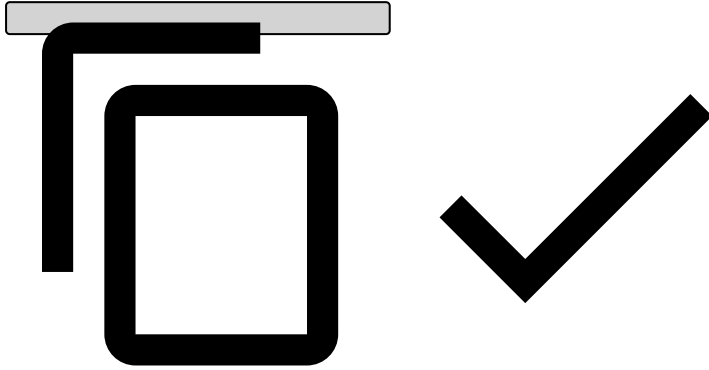
If you find yourself wanting to explicitly iterate over a List (e.g., using `filter`, `all`, etc) consider rewriting the rule, as this is highly inefficient as Lists are optimized for `contains`/lookup operations.

Using Complex Lists to Efficiently Store Common Patterns [🔗](#)

Imagine that it is necessary to block certain card numbers for certain countries. Suppose for example that we want to block 379867957393437 in USA and Canada and 340735948409065 in Spain and Italy. One possible approach would be to code this directly in RDL:

```
// Variables
matchCard3437 = event.cardnumber == "379867957393437" and
(event.country == "USA" or event.country == "CANADA");
matchCard9065 = event.cardnumber == "340735948409065" and
(event.country == "SPAIN" or event.country == "ITALY");

// Condition
matchCard3437 or matchCard9065
```



There are several problems with this approach:

- It does not scale well as you need to add more card numbers and countries
- It makes the process of maintaining the rule error prone
- It does not allow to an Administrator to create a permission for a specific user to add or remove credit card numbers (and not change the rule logic)

The way to address these issues is to use the [Lists](#), which allow us to create a list with the card number to blocked country list mapping.

First we want a complex list that has the card number as the key and has a blocked countries list as a field:

Figure 3. Creating a Complex List to Store Card Number to Blocked Countries List Pairs

Note that we choose the type of the key as string. This is a good idea for credit card numbers because it allows you to have hashed card numbers as keys.



Choosing the Correct List Type (Simple or Complex)

Complex lists are a good choice when it is necessary to perform lookups because they are internally implemented as a Map. Simple lists on the other hand are more suited for contains and exists operations because they are internally implemented as a Set.

Next we need to create some items:

Figure 4. Adding an Item to a Complex List

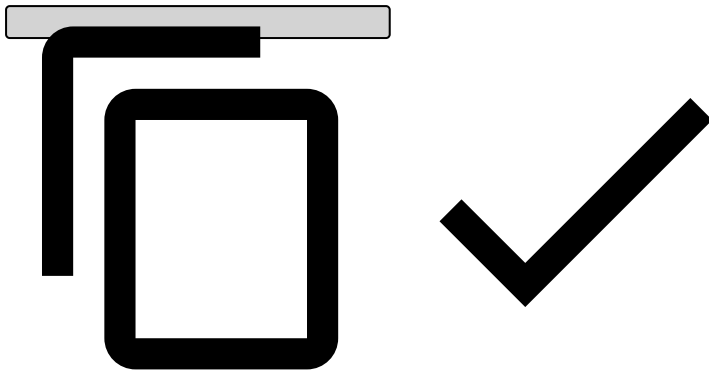
Figure 5. Adding Another Item to a Complex List

Once we got the list setup, we can use it to rewrite our rule, resulting in a much more simple and maintainable code:

```
// Variables
cardnum_entry = cardnumber__country[event.cardnumber];

// Condition
// We first need to check if there is any entry for the given card number
```

```
// and then we check if the blocked countries list contains the country
// where the current transaction originated from
cardnum_entry != null and cardnum_entry.blocked_countries.contains(event.country);
```



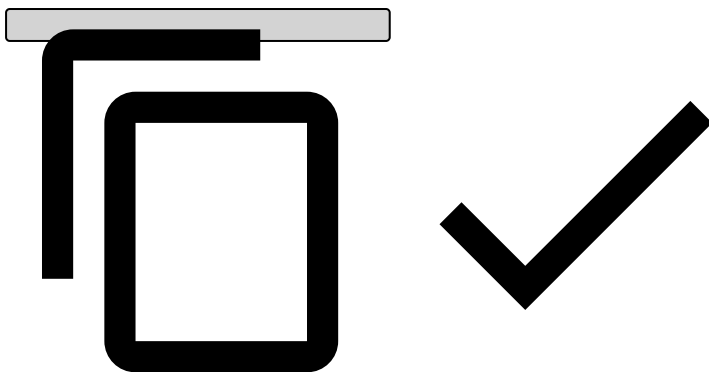
Correlating Multiple Transactions to Detect Known Fraud Patterns

To access past transactions a window can be used. In RDL, a window is a time ordered set of transactions that has a range, which is either numerical or temporal.

Example 3: match transactions of 1\$ followed by a transaction in a different location

```
// Variables
// Get the last 2 transactions for the current card number
last2Events = event[2 tuples by card_number];
previous = last2Events.at(1); // Collections in RDL start in index 1

// Condition
// The count check is necessary because the window may have 0, 1 or 2 events
(last2Events.count() == 2) and (previous.amount <= 1) and (previous.locations != event.location)
```



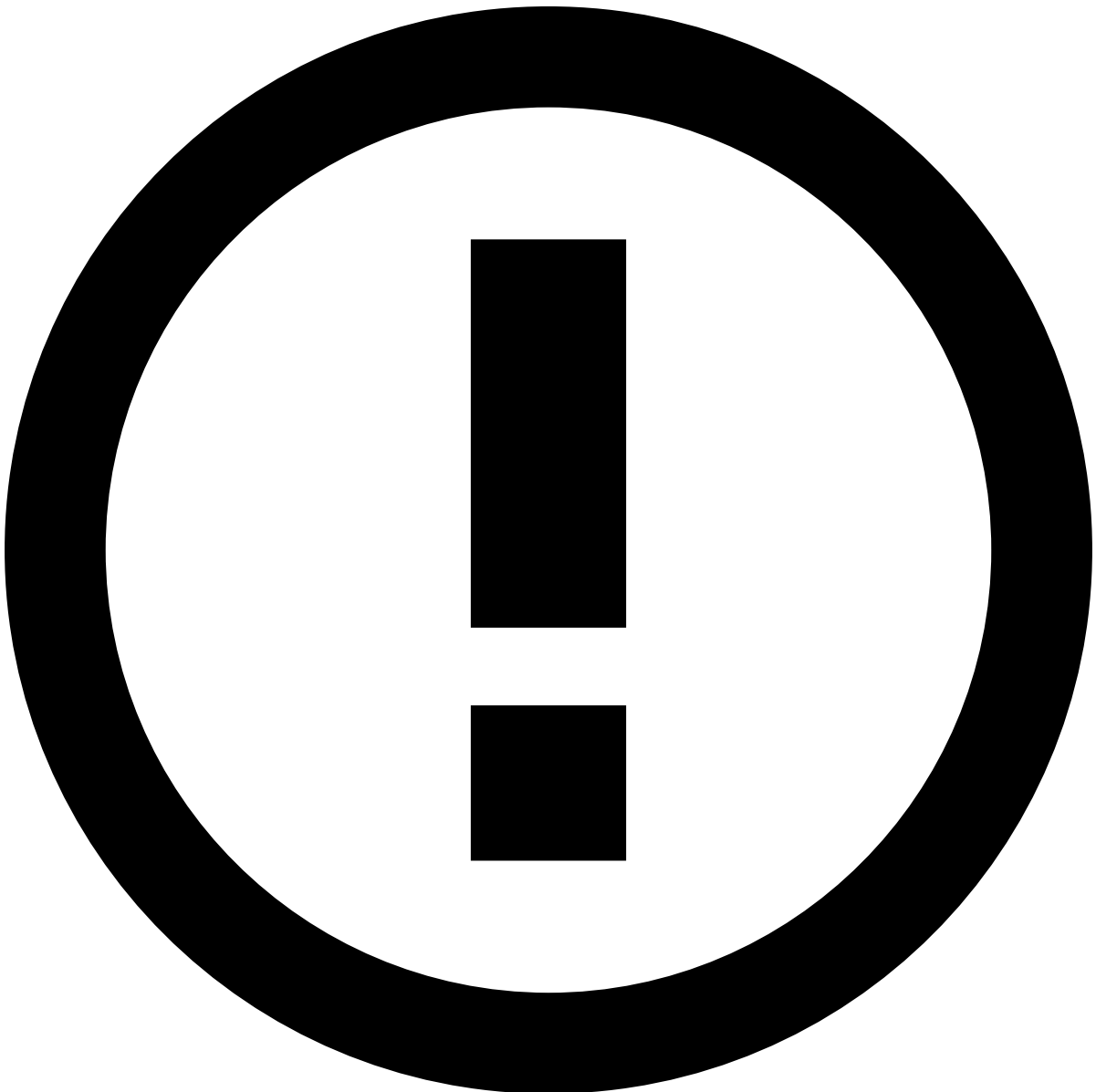
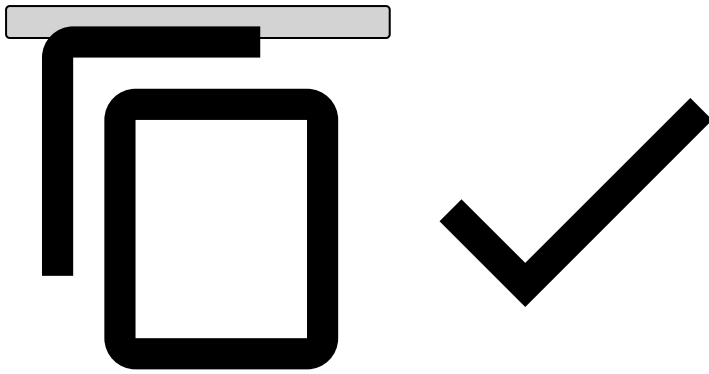
When it is necessary to correlate multiple transactions, try to write the rule by looking at past transactions instead of future ones. Instead of "The next transaction will be in the same location", think "The previous transaction was on the same location".

Since windows are just lists, it is possible to use the collection methods:

Example 4: match transactions that surpass a \$1000/hour limit

```
// Variables
// Get the last 1 hour of transactions for the current card number
last1hEvents = event[1 hour by card_number];

// Condition
last1hEvents.sum(amount) > 1000
```



Methods of Collections in RDL

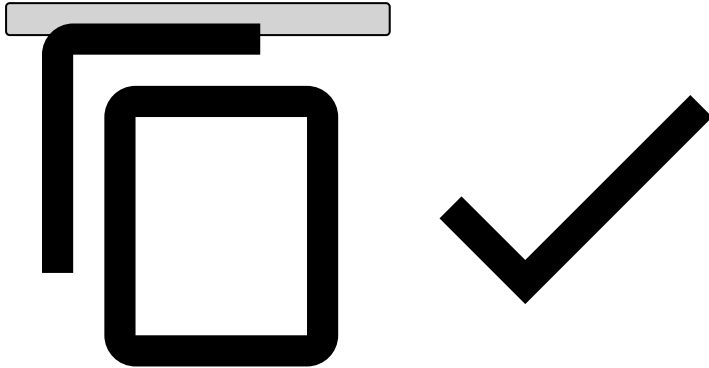
Collections in RDL have several methods, including `all` , `any` , `contains` , `filter` , `count` , etc.

It is also possible to use lambdas:

Example 5: match 5 sequential transactions of the same card that have a low amount (< \$5)

```
// Variables
last5Events = event[5 tuples by card_number];

// Condition
last5Events.count() == 5 and lastEvents.all(e => e.amount < 5)
```





Lambdas Can Impact Performance Severely

Try to only use lambdas for collections whose size is known to be small (<100 elements) and prefer filtering in the window (using the **where** operator) to the filter method.

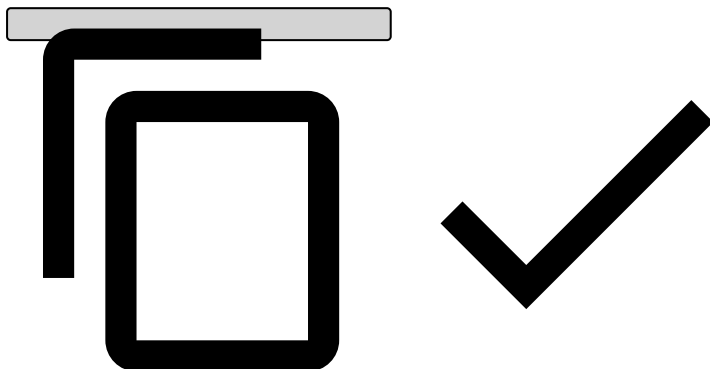
Using Templates to Reuse Code📖

Rule Templates allow to parameterize rules and as such are useful when it is necessary to have several instances of the same logic but with different parameters.

One example would be writing a rule to trigger an alert if the sum of amount of the last N minutes of a specific card and terminal type is more than Y. If you needed to write only one or two rules with this logic it would be perfectly acceptable to write it like:

```
// Variables
lastEvents = event[10 minutes where terminaltype == &quot;ATM&quot; by cardnumber];

// Condition
lastEvents.sum(amount) &gt; 1000
```

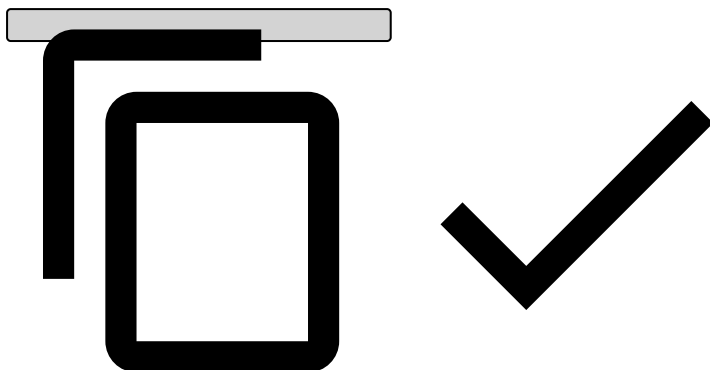


However repeating this several times is error prone and cumbersome.

Using templates, we replace the literal values by template parameters:

```
// Variables
lastEvents = event[$nMin minutes where terminaltype == $terminaltype by cardnumber];

// Condition
lastEvents.sum(amount) &gt; $yDolars
```



Then to create rules we select the template and fill in the specific values:

Figure 6. Reusing RDL Code with Templates

Detect Repeated Transaction Attempts With the Same Location and Amount

A common pattern for fraudulent transactions is to make several transaction attempts in the same location or change the location after several attempts. For example:

- Transaction 1, Location = Lisbon, amount = 50: accept
- Transaction 2, Location = Lisbon, amount = 50: accept
- Transaction 3, Location = Lisbon, amount = 50: accept
- Transaction 4, Location = Lisbon, amount = 50: accept
- Transaction 5, Location = Lisbon, amount = 50: block

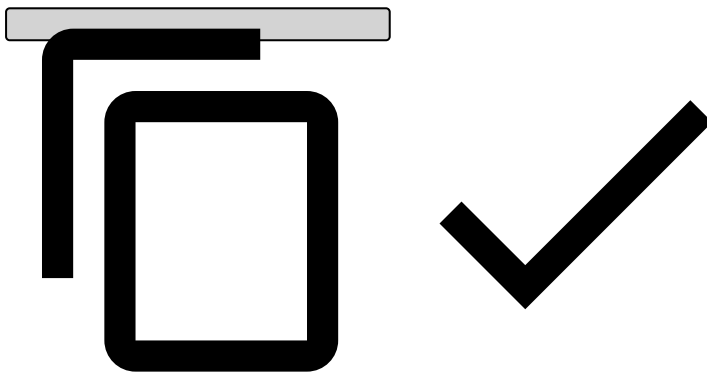
Or:

- Transaction 1, Location = Lisbon, amount = 50: accept
- Transaction 2, Location = Lisbon, amount = 50: accept
- Transaction 3, Location = Lisbon, amount = 50: accept
- Transaction 4, Location = Lisbon, amount = 50: accept
- Transaction 5, Location = Coimbra, amount = 50: block

The following RDL code detects such patterns by first checking if either all or just the first 4 events have the same location (`ev.location == first.location`) and secondly by making sure that that *all* lambda always evaluates to true for the last event (`ev.uuid == event.uuid`):

```
// Variables
lastEvents = event[5 tuples by card];
first = lastEvents.at(1);

// Condition
lastEvents.count() == 5 and
  lastEvents.all(ev.location == first.location or ev.uuid == event.uuid)
```



Detect Transactions With a Considerable Amount After a 1\$ Transaction in the Same Location

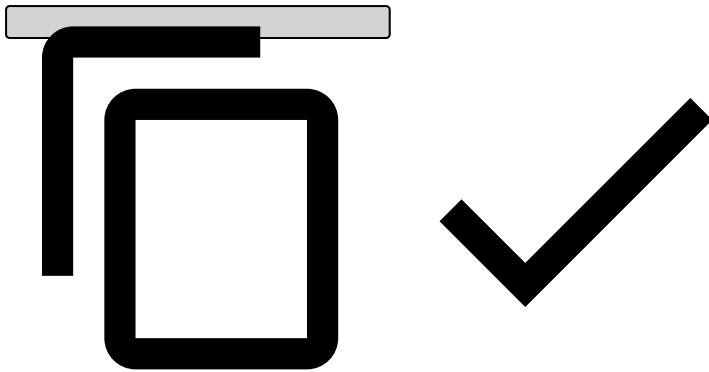
A common pattern for fraudulent transactions is to make an initial test transaction with a low amount and then make transaction with a large amount. For example:

- Transaction 1, Location = Lisbon, Amount = 1: accept
- Transaction 2, Location = Lisbon, Amount = 200: block

The following RDL detects such a pattern by picking the last two events (for the current event card number) and checking if the location is the same in the two and if the respective amounts are in the ranges described before:

```
// Variables
lastEvents = event[2 tuples by card];
first = lastEvents.at(1);

// Condition
event.amount > 100 and lastEvents.count() == 2 and
  first.amount <= 1 and
  first.location == event.location
```



Detect Repeated Transaction Attempts With Decreasing Amounts [🔗](#)

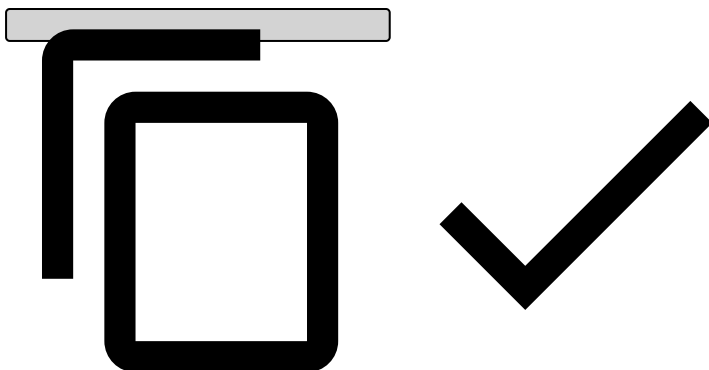
A common pattern for fraudulent transactions is to try to make a transaction with a large amount, decrease it if it fails (usually by dividing the amount by 2 or 3) and try again. For example:

- Transaction 1, amount = 1000: accept
- Transaction 2, amount = 500: accept
- Transaction 3, amount = 250: block

The following RDL detects such a pattern for a multiple of 2 by asserting if the first 2 events of the last 3 events (if there are 3 events in the window) are multiples of the current amount.

```
// Variables
lastEvents = event[3 tuples by card];

// Condition
lastEvents.count() == 3 and
lastEvents.at(1) == event.amount * 4 and
lastEvents.at(2) == event.amount * 2
```



Detect Bursts of Transaction Attempts and Alert When the Transaction Comes Through [🔗](#)

A common pattern for fraudulent transactions is to make attempts at performing a given operation (e.g., buying some goods) until the transaction is accepted (trying a different CCV at each attempt).

Suppose that code = 5 means that the transaction failed and code = 10 means that the transaction was accepted.

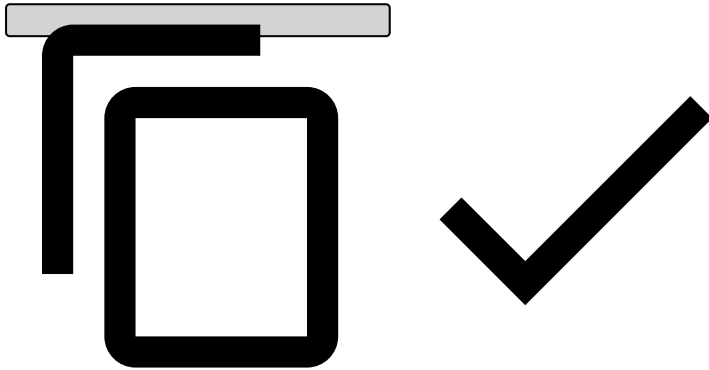
One example of this pattern could be:

- Transaction 1, Location = Lisbon, code = 5: no alert
- Transaction 2, Location = Lisbon, code = 5: no alert
- Transaction 3, Location = Lisbon, code = 10: alert

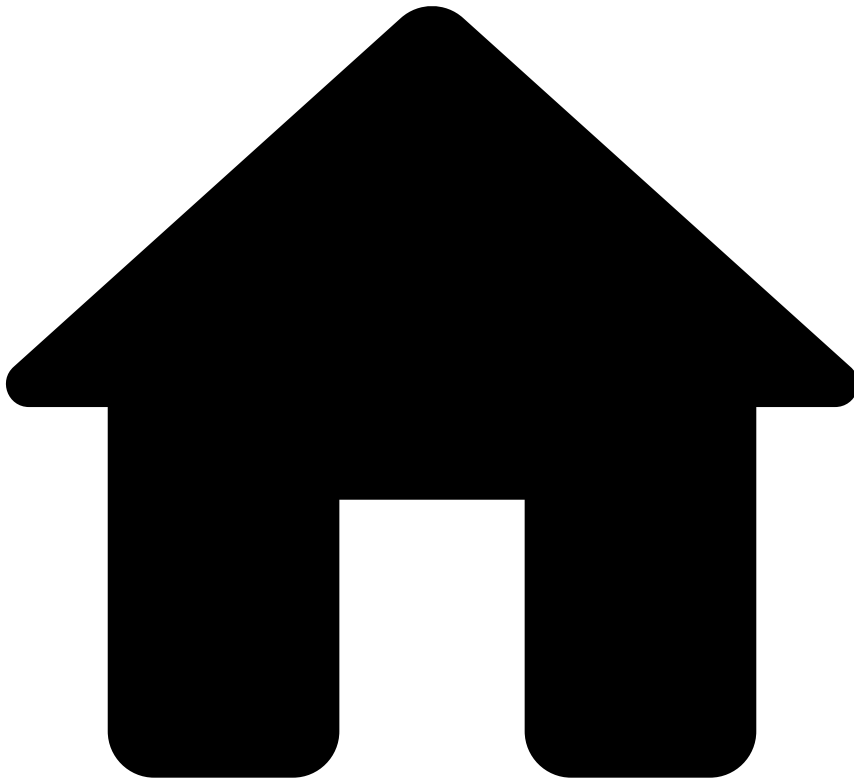
The following RDL detects such a pattern by first checking if the location is the same in all the transactions (to determine that the origin is the same) and then by checking that all the transactions have code 5 (failed) but the last (the current) which was accepted:

```
// Variables
lastEvents = event[3 tuples by card];
firstEvent = lastEvents.at(1);

// Condition
lastEvents.count() == 3 and
lastEvents.all(
  ev => ev.location == firstEvent.location and
  (ev.code == 5 or (ev.uuid == event.uuid and ev.code == 10))
)
```



•



- [Signing in to Pulse](#)
- [Password recovery](#)

On this page

Password recovery

Was this page helpful? 

If a user needs a new password, there are two options to reset it:

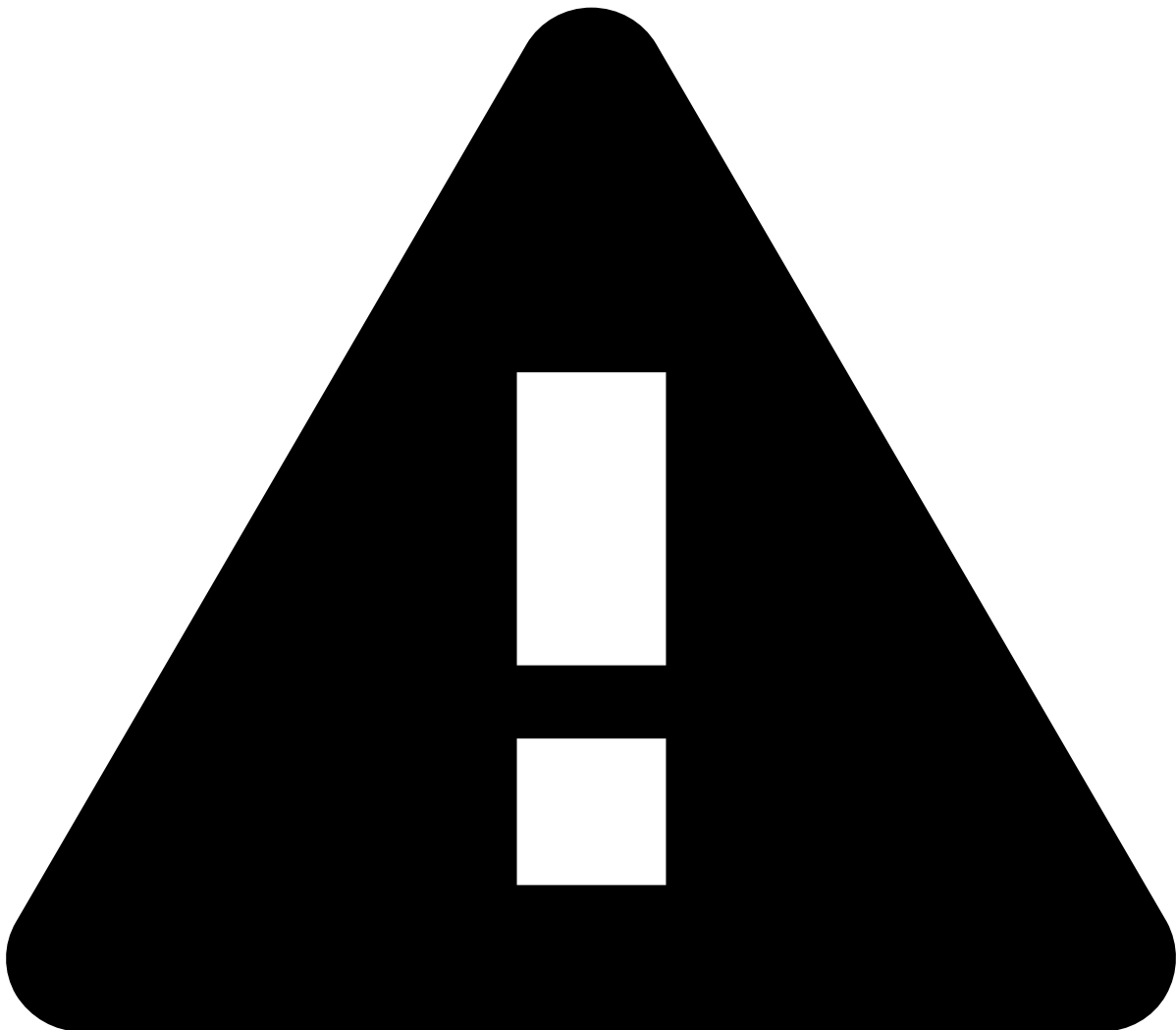
- By the administrator, on the user profile, under **Administration - Security - Users**.
- By the user.

The following chapter describes how users reset their password without involving an administrator.

Password reset by the user

To reset your own password, perform the following steps:

1. Open Pulse.
2. On the login window, select **Forgot password?**
3. Provide your username. If the user profile has an email address configured, Pulse sends an email to that address with a token, which is a number. It is possible to request a new token, 30 seconds after the last one. Only the last token is valid, and it has a configurable validity time. By default, tokens are valid during 1 hour after they are generated.
4. In the **Token** field, enter the token you received via email.
5. Type the new password in the **New password** field, and repeat the same password in the **Confirm password** field.

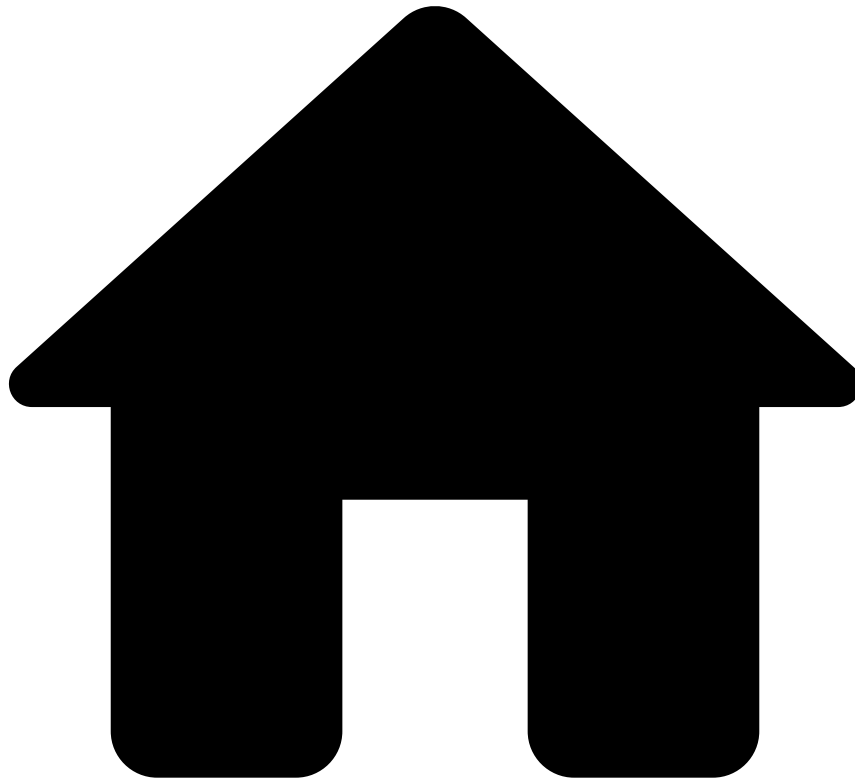


Limited number of tries

The number of tries to save a new password with a certain token is limited. Consider the password policy before continuing to change the password. If you exceed the configured limit to change the password with a certain token, you can request a new token and try again.

1. Save the change using the **Change password** button. Pulse saves the change and logs you in.

•



- Signing in to Pulse
- Signing in with Single Sign-On

On this page

Signing in with Single Sign-On

Was this page helpful? 

Pulse supports Single Sign-On (SSO) which allows you to access various applications (i.e. to start a session) by signing in to only one of the applications. You can learn more about Pulse SSO in the [Single Sign-On](#) page under the Administrator's Guide.

Sign in Scenarios

Pulse supports **SP-initiated** sign-in.

This scenario is initiated by the Service Provider (SP) and occurs when you access a Pulse URL, and you are not signed in. You are redirected to a URL that is managed by the Identity Provider (IdP). In this IdP URL, you are prompted with a

sign in form and after authentication you are automatically redirected back to the Feedzai URL. Check [Sign in via SP-initiated](#).

User Provisioning^â

With Pulse SSO, users are created by the concept of user provisioning. If you are signing in for the first time and if your account does not exist as a Pulse User, your account is created on-the-fly after being authorized by the IdP.

Sign in as Administrator^â

The recommended method to create users in Pulse is by User Provisioning. However, Administrators can still sign in to Pulse using a static URL and then add new user accounts. Note that this option should only be used under special situation as the recommended method to add user accounts is by User Provisioning.

Check [Signing in as Pulse Administrator](#).

Sign out^â

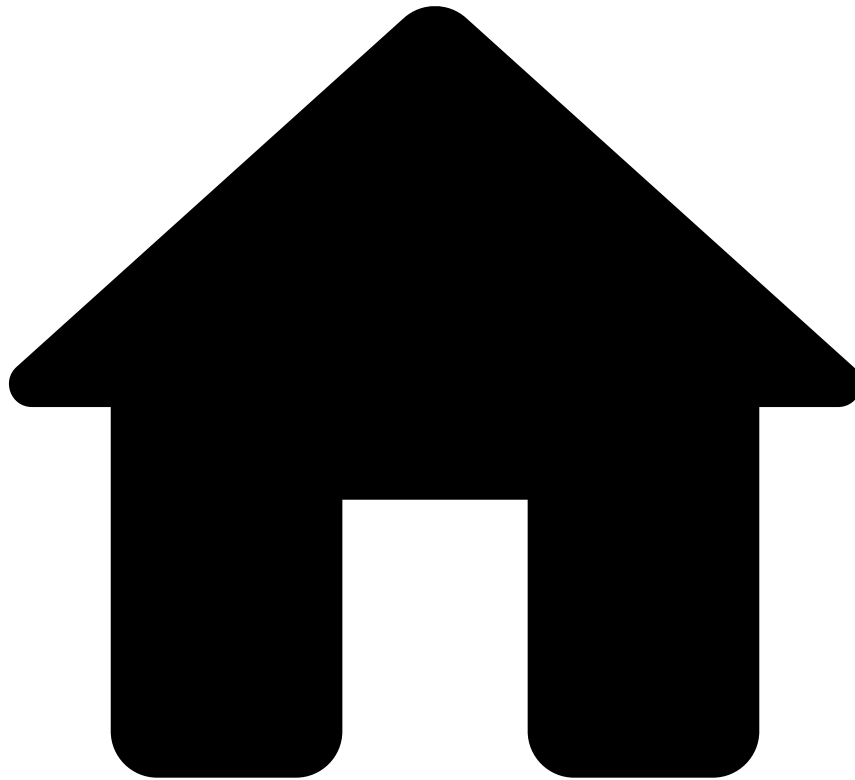
One component associated with SSO is allowing users to sign out from all the applications at once (i.e. without having to sign out individually from all the applications). This component is commonly referred to as Single Logout.

Learn more^â

Now that you have an overall picture of Pulse SSO you can check user guide details presented in the following sections:

- [Signing in via SP-initiated](#)
- [Signing in as Pulse Administrator](#)

•



- [Signing in to Pulse](#)
- [Signing in with Two-Factor Authentication](#)
- [About Signing in with Two-Factor Authentication](#)

On this page

About Signing in with Two-Factor Authentication

Was this page helpful? 

Two-Factor Authentication (**2FA**) is a functionality that requires you to sign in with two pieces of information to verify your identity. **2FA** creates a more secure authentication system while complying with PCI-DSS security standards.

Sign in steps

When you access any Pulse URL, and you are not signed in yet, you are redirected to the main Pulse sign in form.

1. In this form, insert your Username and Password and use the Sign In button to continue to the authentication step.

2. You are prompted with another form that requests a second step validation to access Pulse. Enter the PIN Code generated by the code generator that you are using on your mobile device, such as Google Authenticator. Use the Verify Code button to access Pulse.



Authentication failure

If you either introduce wrong credentials (username or password) or a wrong PIN Code, you are redirected to the first Sign in form (i.e. step 1) to start the sign in process again from the beginning. This occurs because 2FA provides authentication only if username/password/code are correct without giving any clue to which step the user failed. This is a 2FA requirement to comply with PCI-DSS security standards.

Configuring 2FA on your device

To sign in with 2FA, a code generator application has to be configured on your mobile device(s). Pulse is agnostic to the application used to generate the code. As an example, the instructions to configure 2FA on your device using Google Authenticator are provided in page [Configuring Two-Factor Authentication](#).

Resetting 2FA on your device[â](#)

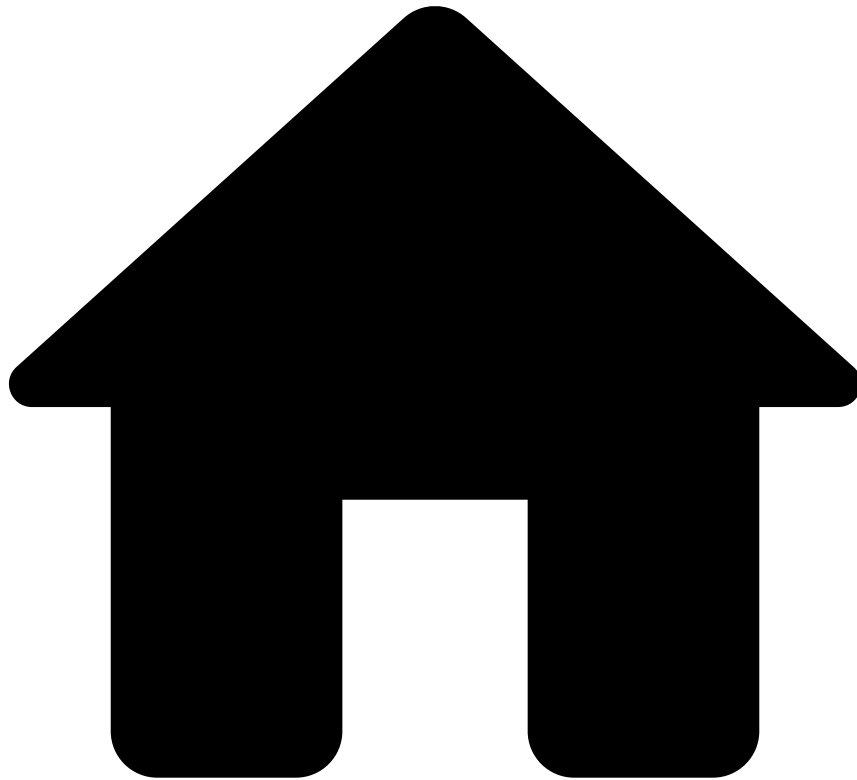
There are a number of scenarios in which you might want to reset the **2FA** configuration on your mobile device. Check the page [Resetting Two-Factor Authentication](#).

Learn More[â](#)

Learn more about **2FA** configurations:

- [Configuring Two-Factor Authentication](#)
- [Resetting Two-Factor Authentication](#)

•



- [Working with Applications](#)
- Automatic Application Repair

On this page

Automatic Application Repair

Was this page helpful? 

Pulse has an **Auto-Repair** functionality that monitors the state of the cluster and decides whether it needs to be repaired or not. If the application is in an invalid state, Pulse will automatically trigger repair actions on the respective components. Fully repairing the cluster might involve executing several fix actions in sequence.

This functionality is not designed to fix the entire application all at once. It rather focuses on maintaining a small number of state validation rules, which when triggered and composed, allow the overall system to converge to a stable state.

It is possible that the **Auto-Repair** triggers a fix to a problem that will not return the system to a stable state. However, it guarantees that the system will be one step closer to a stable state. When each step is finished, it will allow for more fixes to be issued until the stable state is reached.

Fixing issues🔗📄

Pulse **Auto-Repair** consists in actions that aim at fixing the following issues:

- **Under replication:** Pulse will ensure that there are at least as many App Engines as the configured replication factor. If an AppEngine crashes, Pulse will automatically launch a new AppEngine node until reaching the replication factor.
- **Over replication:** Pulse will ensure that the number of AppEngines for a provided application does not exceed the configured replication factor. If Pulse detects that there are more nodes running the application than the number defined in the replication factor, then Pulse will shut down the application in as many nodes as required to adhere to the replication factor. Selecting which nodes to shut down is not deterministic.
- **Inconsistent versions:** When different nodes are running different versions of the application, or are not running the expected version, Pulse will automatically try to publish the expected version. In most cases this is the latest stable published version. If the application is in a *bootstrapped* state, then Pulse ensures that all AppEngines are in that state.

Configuring Auto-Repair🔗📄

To enable or disable the **Auto-Repair**, execute the following actions:

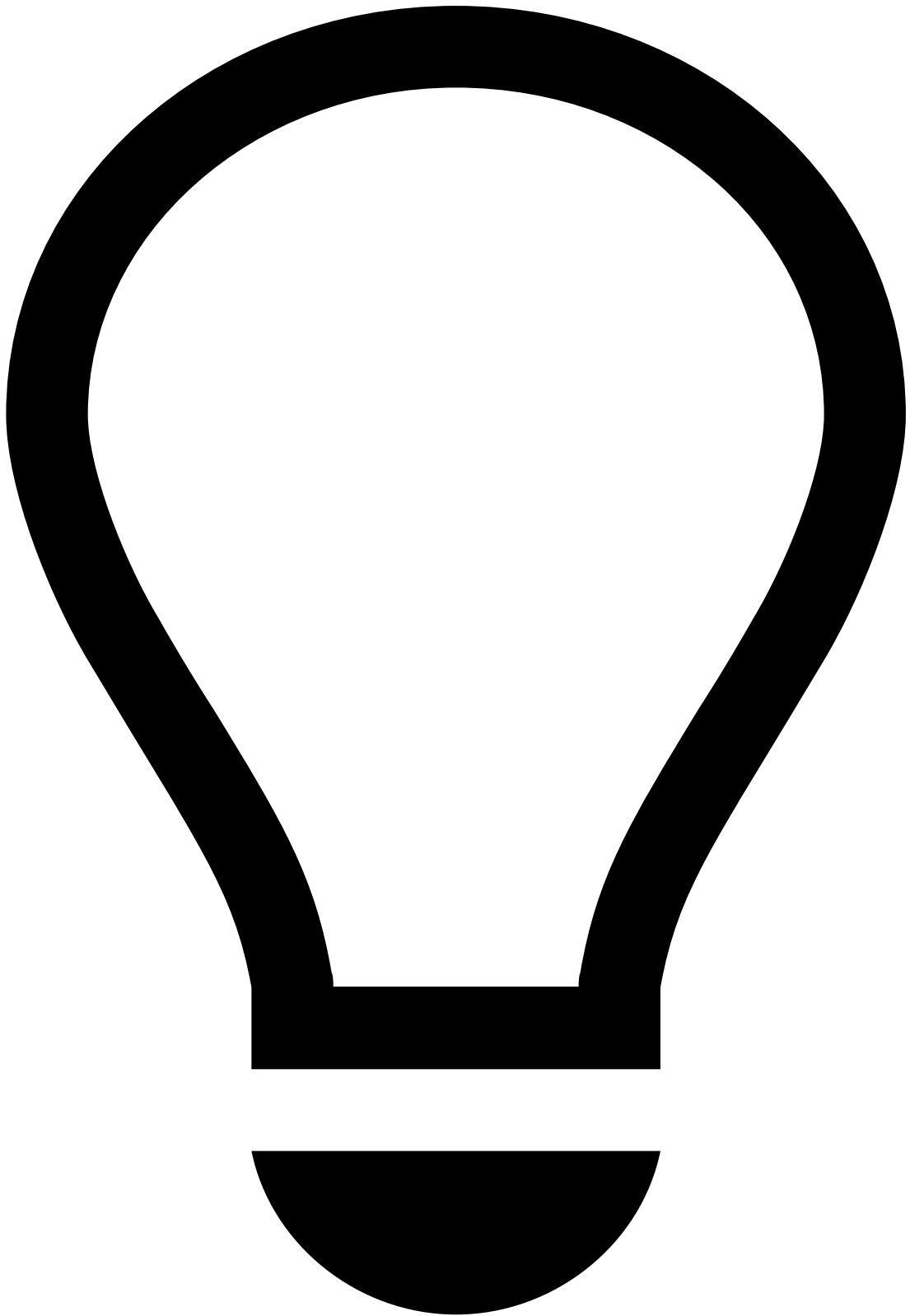
1. In your Pulse application, use the **Options** menu and select the **Edit** option.
2. Toggle the **Auto-Repair** configuration accordingly:

When the **Auto-Repair** feature is taking action, Pulse informs you that it is fixing your application.

In addition, Pulse also includes configuration properties that allow you to define the behavior of the **Auto-Repair** functionality individually by each faulty detection rule.

To configure the **Auto-Repair** functionality, execute the following actions in the Pulse advanced configuration:

1. Activate or deactivate each type of issues to be detected and fixed. For example, if you set the `fixInconsistentVersions` property to `false`, whenever Pulse detects you are trying to publish an application and there are different versions running, you must synchronize them manually.
2. Set the `checkPeriodMillis` property to, for example, 2 minutes.
3. Set the `latencyActuationCycles` property to, for example, 3 checks. Pulse only resets the cycle count when the entire cluster is stable, and not when a fix action is triggered.



Recommended configuration

The recommended configuration is that you set:

- `checkPeriodMillis` to a small interval.
- `latencyActuationCycles` to approximately 3 cycles.

This ensures some balance as fixes are quickly issued, and Pulse still allows for some space when waiting for momentary problems (for example, network partition) to fix on their own. In this example, it would always take minimum 6 minutes to issue a fix.

•



- [Working with Applications](#)
- Exporting and Importing Applications

On this page

Exporting and Importing Applications

Was this page helpful? 

Pulse supports application import and export not only to augment interoperability between different installations but also to give the ability to save an application in an exact state.

Although that can be partially achieved using [versioning](#), exporting an application gives one the possibility to archive it.

Exporting an Application

Applications are exported in a single `PAPP` file that contains all the application resources: KPIs, Baselines, Workflows and so on. In some situations there are security resources that are also exported with the applications; more on that later.

Exporting an application is actually quite simple: the user can do it through the `Export` option present in the application configuration menu. This is illustrated in Figure 1 and will prompt the download of the application `PAPP` file.

Importing an Application🔗

Importing an application is usually as simple as exporting it. The user only has to click on the *Import Application* button present in the **Applications** tab and an informative wizard will guide the entire process.

The *Import Application* wizard can have one to three steps depending on the complexity of the application being imported. If the application has neither security [Groups](#) nor Multi-Tenant resources, only one step will be needed which allows the user to upload the application `PAPP` file.

Applications With Groups🔗

Whenever an application has resources that have security Groups configured the Import Application wizard will require one additional step. In this step the user will match the Groups used by the application being imported and the groups that exist in the Pulse installation.

This matching operation is necessary to guarantee that security permissions remain unchanged.

The UI will automatically do a best-effort match of the groups by name and will also allow the user to remove the groups permissions from the resources being imported.

Applications With Multi-Tenancy Enabled🔗

Importing and Exporting Applications that contain Multi-Tenancy resources do not have any additional configuration step. However, there are a few considerations worth mentioning regarding the Application import with Multi-Tenancy. Tenants stored in the `PAPP` file are merged automatically by Pulse using the following algorithm:

1. If the Tenant being imported has the same Unique Id (Key) of a Tenant in the current Pulse Installation, all resources of the Tenant being imported will be added/mapped to the Tenant in the current Pulse Installation.
2. If there is no Tenant with the the same Unique Id in the current Pulse Installation, a new Tenant will be created verifying before hand that there is no naming collision.
 1. If there is a name collision, the Tenant Unique Id will be appended at the end of the name, *e.g.* "Tenant Name (Tenant Unique Id)".
3. All the tenants stored in the `PAPP` file will be imported to the current Pulse Installation. Even if they are not being explicitly used in the Application being imported. Additional details:
 1. Even the Tenants without any explicit resource are used in runtime in case of a Rules of the type "Shared by All Tenants".
 2. When Exporting an Application, all the Tenants in the Pulse installation are stored in the `PAPP` file.

•



- [Working with Applications](#)
- Exporting resources

On this page

Exporting Resources

Was this page helpful? 

Why

Exporting resources allows you to build a bundle with resources developed in a Pulse application (**Pulse App**). You can use the option [Importing Resources](#) to easily import the resources bundle in another **Pulse App** in the same environment or between different environments. You can export the following resources:

- [Lists](#)' metadata without List items.
- [Models](#) with or without the binary.
- [Plans](#) with or without executions.
- [Rules Projects](#) with or without snapshots.

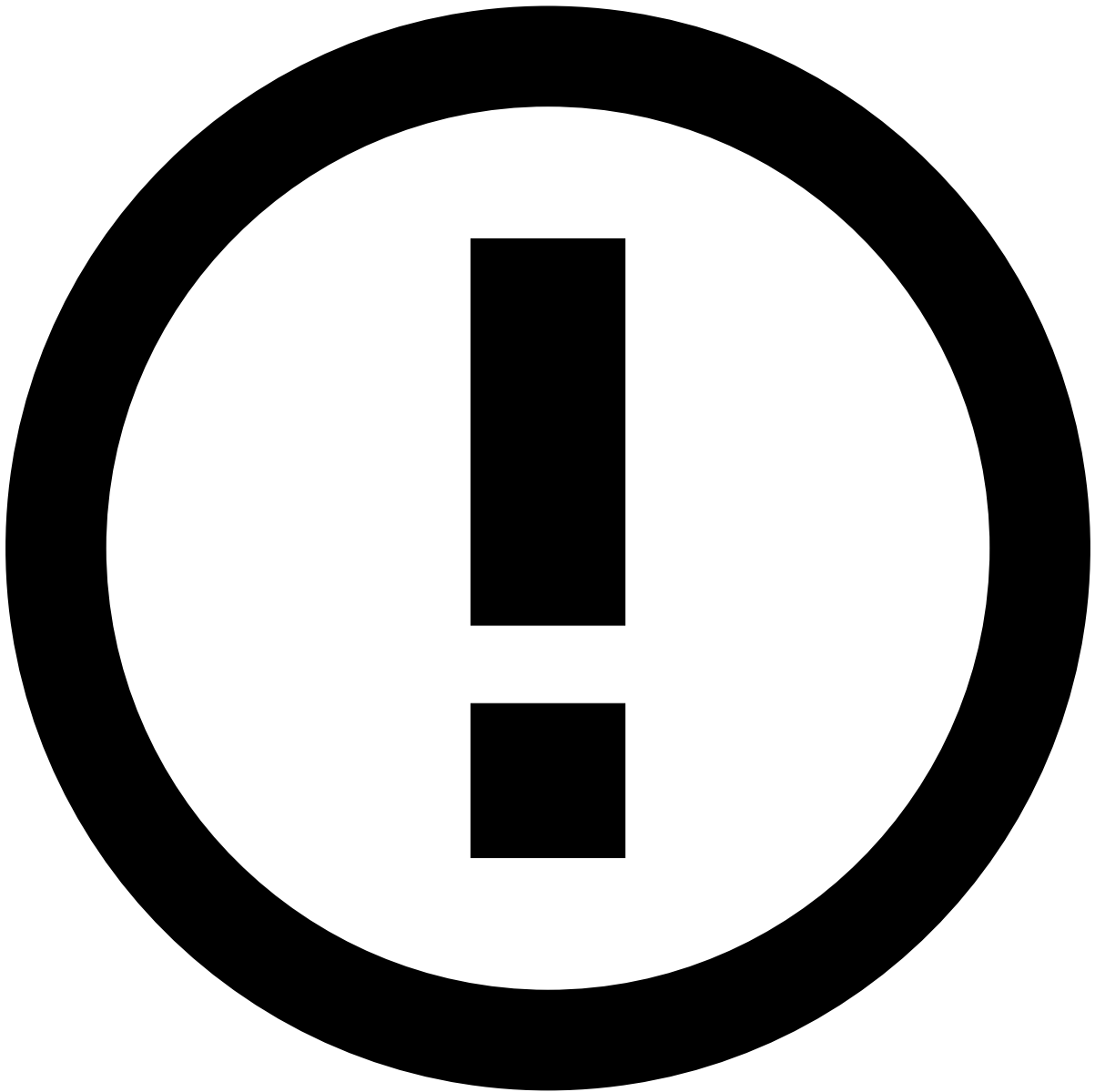
By default, the **Export Resource** option also adds automatically to the resource bundle the metadata related to all the **Tenants** and **Tenant Groups** of the Pulse instance in which your **Pulse App** is running. See the page [Bundle structure](#) to learn how the files are organized.

How to

Exporting Pulse Resources

To **Export Resources** execute the following steps:

1. Select the Options button to open the application options menu:
2. Select the option Export Resources to open the export wizard. The image below shows the **Export Resources** wizard:
3. Check the subsections below for more details about each resource type that you want to export.
4. Once you have chosen all the resources that you want to export, select the button Export Bundle



List export

List metadata is automatically exported because **Rules** can have a dependency on any **List**. When importing the bundle, all the **Lists** are imported to the **Pulse app**, either by replacing or creating new ones.

To exclude the **List** metadata you have to edit the export bundle manually.

Models Projects

While you are in the export wizard, to add **Model Projects** to your bundle execute the following steps:

1. Select the tab  **Model Projects** to access the exporting options of this resource.
2. To add a **Model** to the list of exported resources:
 1. Search by the Name of your **Model Project** using the drop-down menu.
 2. Search by the Name of your **Model** using the drop-down menu.
 3. (Optional) Choose to  Include the binary of the **Model**.

4. Select the **Add Resource** button to add this resource to the exporting list.
3. Repeat Step 2 to complete the list of **Model Projects** that you want to export.
4. Check how to add another resource or return to the subsection [Exporting Pulse Resources](#) to save your bundle.

Plans

While you are in the export wizard, to add **Plans** to your bundle execute the following steps:

1. Select the tab **Plans** to access the exporting options of this resource.
2. To add a **Plan** to the list of exported resources:
 1. Search by the Name of your **Plan** using the drop-down menu.
 2. (Optional) Choose the Plan Execution that you want to add to this Plan using the Drop-down menu under Execution List.
 3. Select the **Add Resource** button to add this resource to the exporting list.
3. Repeat Step 2 to complete the list of **Plans** that you want to export.
4. Check how to add another resource or return to the subsection [Exporting Pulse Resources](#) to save your bundle.

Rules Projects

While you are in the export wizard, to add **Rules Projects** to your bundle execute the following steps:

1. Select the tab **Rules Projects** to access the exporting options of this resource.
2. To add a **Rules Project** to the list of exported resources:
 1. Search by the **Name** of your **Rules Project** using the drop-down menu.
 2. (Optional) Choose the Snapshots that you want to add to this **Rules Project** using the Drop-down menu under **Snapshot List**.
 3. Select the **Add Resource** button to add this resource to the exporting list.
3. Repeat Step 2 to complete the list of **Rules Projects** that you want to export.
4. Check how to add another resource or return to the subsection [Exporting Pulse Resources](#) to save your bundle.

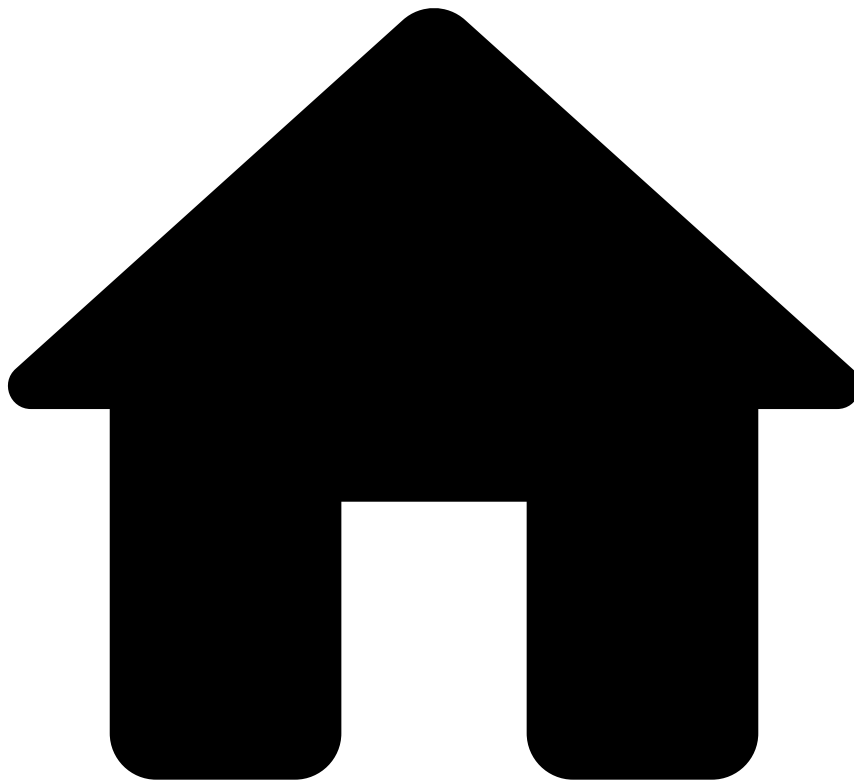
Learn more

To learn more about the resource bundle check the following pages:

- [Bundle structure](#)
- [Comparing resource versions](#)

Check the page [Importing Resources](#) to learn how to import your resources.

•



- [Working with Applications](#)
- Importing Resources

On this page

Importing Resources

Was this page helpful? 

Introduction

After [exporting](#) the resource bundle, you can import the zip file in another Pulse application (**Pulse App**). The following data and resources can be imported:

- Resources imported by user selection: **Models, Plans, and Rules Projects.**
- Metadata of resources automatically imported if the resources are included in the export bundle: **Lists, Tenant Groups, and Tenants.**

This guide explains you how to import resources in Pulse using the partial import option.

How to import Pulse resources

To import resources execute the following steps:

First, select the Options button to open the application options menu:

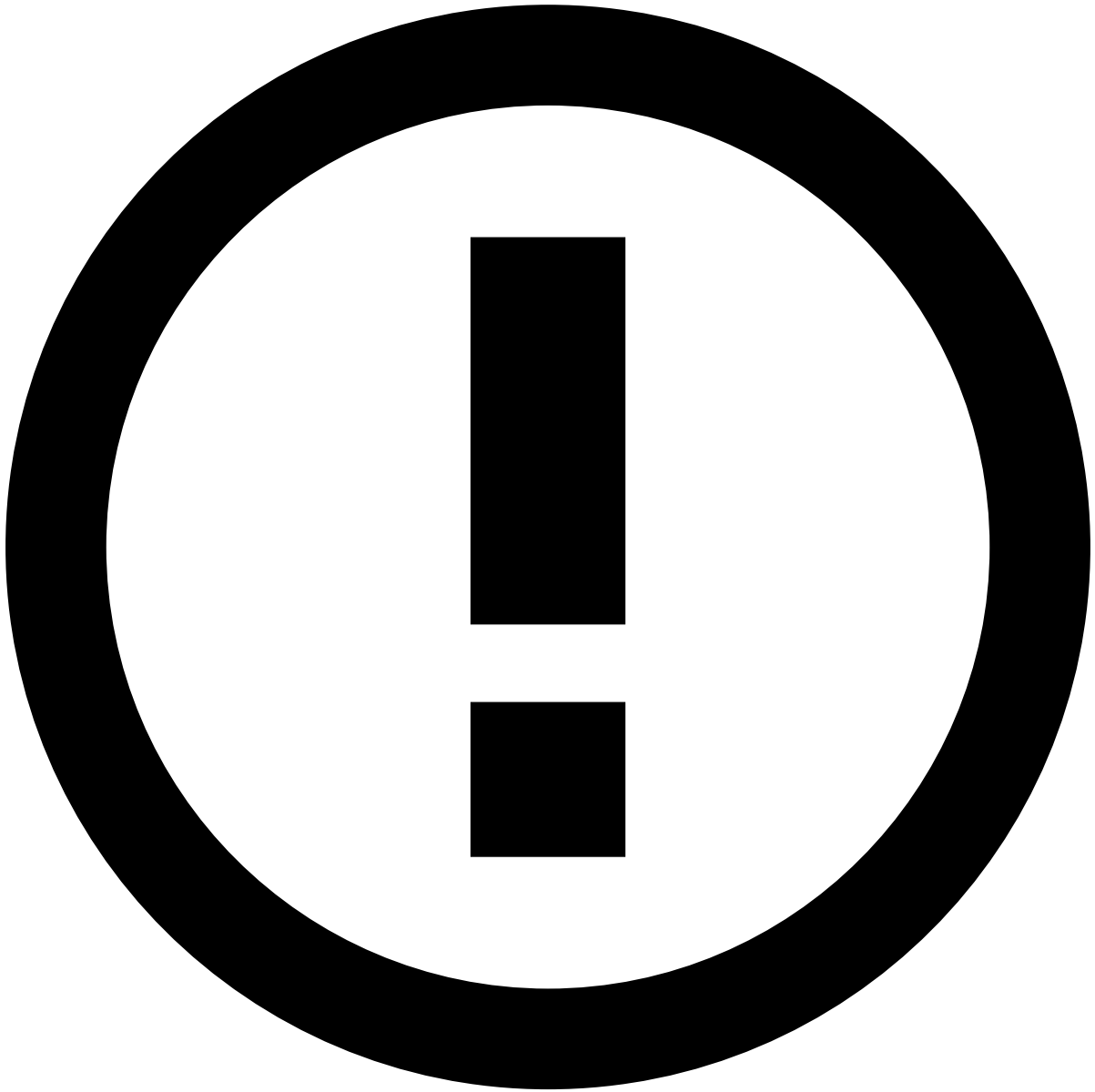
Then, select the option  Import Resources to open the import wizard. The wizard guides you through several stages that are displayed in different tabs as described in the sections below.

Stage 1: Upload File

1. "Drag & Drop" the zip file containing the export bundle to the dashed box, or alternatively, use the option "browse for a file".
2. Use the button  to proceed to Stage 2.

Stage 2: Choose Resources

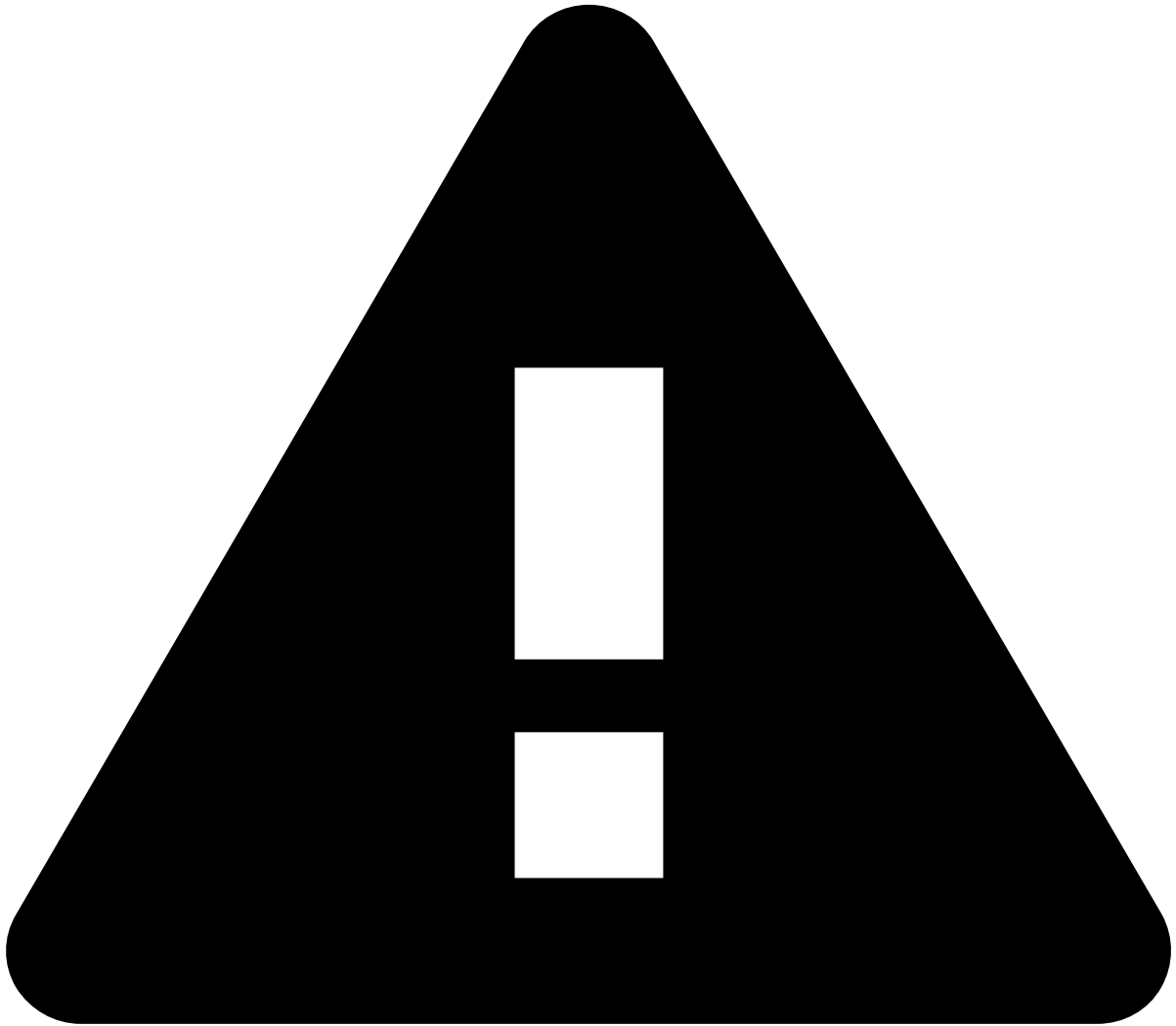
In the **Stage 2: Choose Resources**, you can select the resources that you want to import as described in the subsections below.



Lists

The resource **List** does not have a tab menu because the metadata of **Lists** is automatically imported as follows:

- **Lists** that do not exist in the target environment are created with no items.
- **Lists** that already exist in the target environment are updated with new metadata, however, their items are not affected by the import.



warning

If models are imported via App Collaboration and imported to a clustered environment, the model binary is not propagated to all nodes and remains only in the leader node. It is recommended to set up the binary location for the Pulse folders in the same shared drive across all nodes.

To import a **Model** execute the following actions:

1. Choose the tab Models. The following image shows the import wizard:
2. Use the checkbox to choose the **Models** that you want to import. In Pulse, **Models** are imported individually by user selection rather than importing the full **Model Project** with several **Models** at once.
3. For each **Model**, choose the local **Model Project** to which you want to store the **Model** in the target **Pulse App**.



Model Projects in the local Pulse app

You must have a **Model Project** in the target application to be able to import a new **Model**. If you don't select a *Local Model Project Name* you will receive a warning message to add one.

4. Optionally, choose the binary of each **Model** by selecting the ☐checkbox.
5. Once you have chosen all the resources that you want to import (i.e. **Models**, **Plans** and **Rules Projects**), select the button ☐Next to proceed to the next stage.



Dummy Data Source

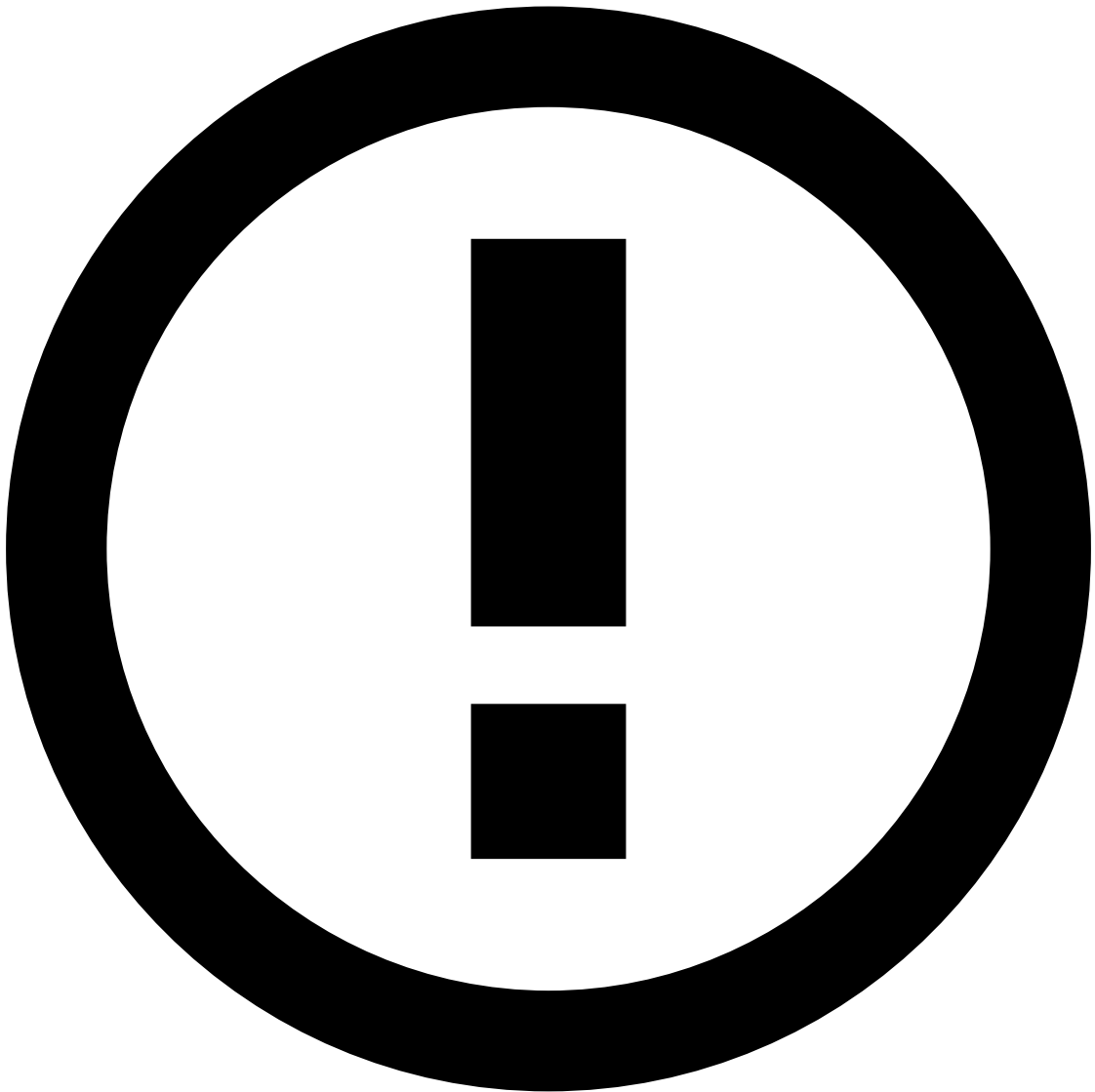
When you import a **Model** that does not have a compatible data source available in the target environment, Pulse automatically creates a new data source. This new data source contains only the schema, without data associated to it.

Plans

To import a **Plan** execute the following actions:

1. Choose the tab Plans.
2. Use the checkbox to choose the **Plans** that you want to import.
3. Choose the *Local Plan Name* for the **Plan** in the target **Pulse app**:
 1. You can replace an existing **Plan** by choosing a target name.
 2. You can choose to save it as a new **Plan** by choosing the option *project_name* New.
4. Optionally, choose the **Plan Executions** that you want to import by expanding the menu under the *Plan Executions* column. By default, the **Plan Executions** are selected.

5. Once you have chosen all the resources that you want to import (i.e. **Models**, **Plans** and **Rules Projects**), select the button **Next** to proceed to the next stage.



Dummy Data Source

When importing a **Plan** with **Executions** that do not have a compatible data source available in the target environment, Pulse automatically creates a new data source for each **Execution** that you choose to import. This new data sources contain only the schema, without data associated to them.

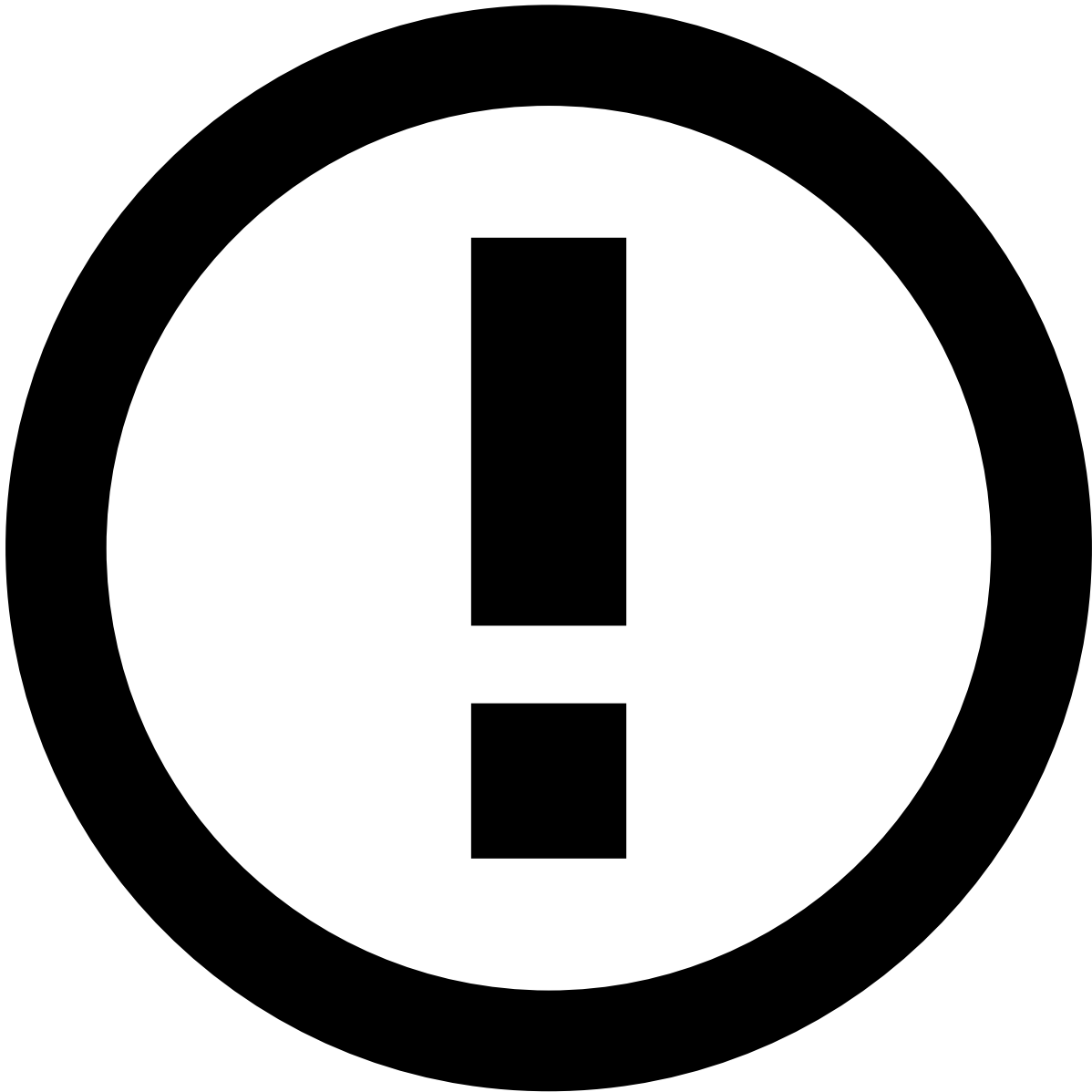
Rules Projects [↗](#)

To import a **Rules Project** execute the following actions:

1. Choose the tab **Rules Projects**.
2. Use the checkbox to choose the **Rules Projects** that you want to import.
3. Choose the *Local Rules Project Name* for the **Rules Project** in the target Application:
 1. You can replace an existing **Rules Project** by choosing a target name.
 2. You can choose to save it as a new **Rules Project** by choosing the option *project_name* **New**.
4. Optionally, choose the **Rules Snapshots** that you want to import by **expanding** the menu under the *Snapshots* column. By default, the **Snapshots** are selected.

5. Once you have chosen all the resources that you want to import (i.e. **Models**, **Plans** and **Rules Projects**) select the button Next to proceed to the next stage.

Stage 3: Schema Conflicts



Stage display

This stage is only displayed if you import a **Rules Project** or a **Plan**, and when both of these conditions are met:

- You choose to override an existing resource in the target application.
- The resource in the export bundle has a different schema compared to the same resource in the target application.

To complete the **Stage 3: Schema Conflicts** execute the following actions:

1. For each resource, choose the schema that you want to keep:
 - **Keep Imported**: This allows you to keep the schema that is specified in the resource that is being imported.
 - **Keep Local**: This allows you to keep the schema that is currently in your target application.

2. Select the button  Next to proceed to the next stage.

Stage 4: Workflow

This stage allows you to associate the resources that you chose to import in Stage 2 to **Workflow** services. See the page [How to organize your work with blocks](#) to understand how to map resources in the Data Science tab to corresponding services in the Workflows tab in the context of **App Collaboration**.



Workflow configuration

To apply the new resources to the **Workflow** you need to have a **Workflow** already created with the service operators placed in the diagram. However, the services don't need to be fully preconfigured (e.g. the service Rules doesn't need to have either a **Rules Project** or a **Snapshot**) because the import wizard allows you to add the configuration to the **Workflow**.

To add or update new resources directly into the **Workflow** execute the following actions:

1. Use the checkbox to choose the *Resource Type* that you want to map to a **Workflow** service.

2. Select the name of the **Workflow** that you want to update with the new resource using the combobox under the column *Workflow*.
3. Select the **Workflow** service to which you want to map the resource using the combobox under the column *Workflow service*.
4. Repeat Steps 1-3 to map the desired resources to **Workflow** services.
5. Complete the Wizard stages by selecting the button  Import.

Learn more

When you export and import resources it might be useful to have the following pages at hand:

- [Bundle structure](#): to learn how the folders are organized in the zip file
- [Errors in App Collaboration](#): To learn possible errors that might occur during export and import operations.

Now that you know how to export and import resources, check the page [Exporting and Importing Applications](#) to learn how to migrate the full **Pulse App** file (`.papp`).

-



- [Working with Applications](#)
- Publishing Application Changes

On this page

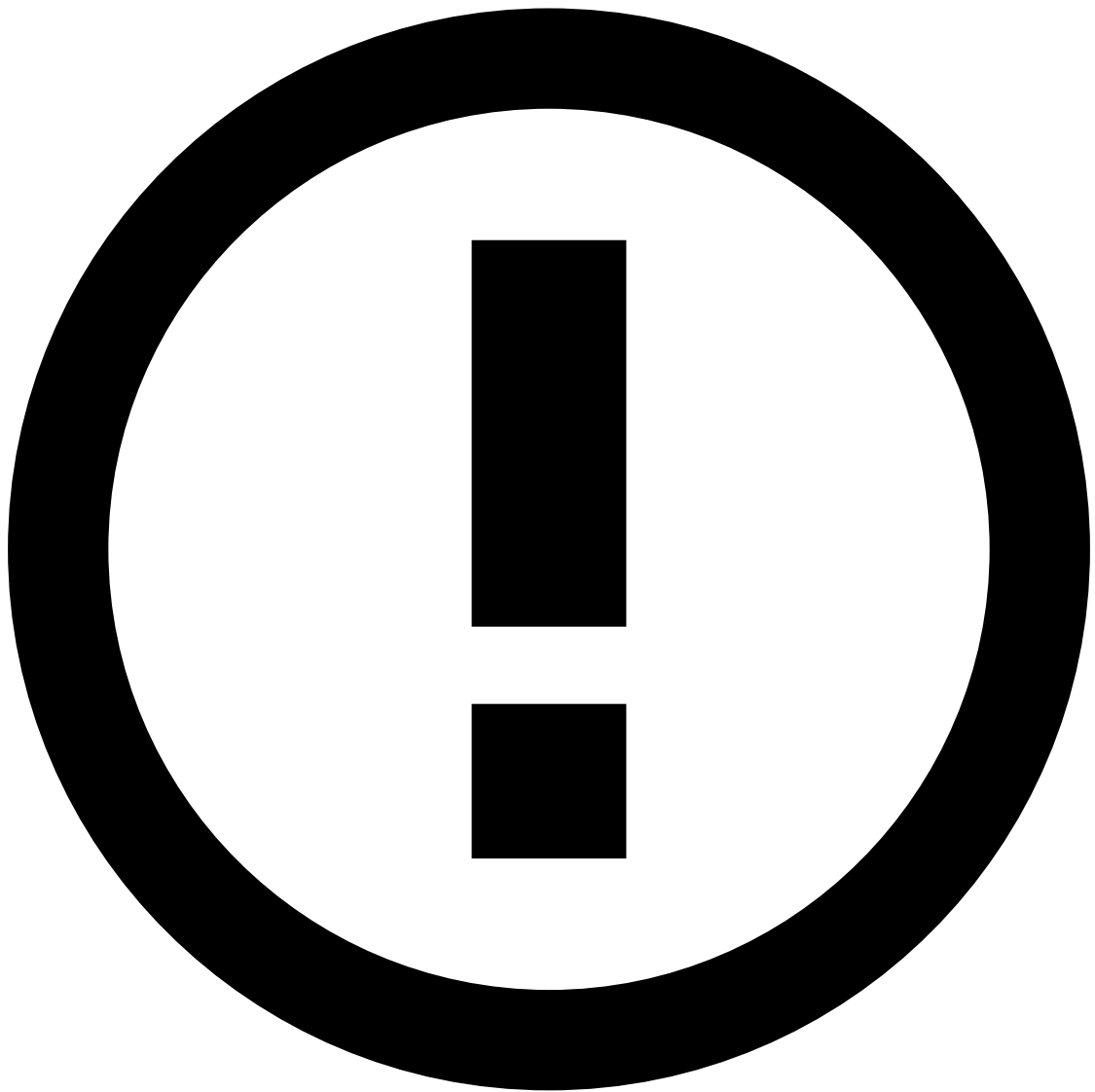
Publishing Application Changes

Was this page helpful? 

In Pulse, you can make changes to the application even when it is running. However, these changes are not immediately applied to the running application. Instead, they are applied only when you explicitly re-publish the application.

To publish changes made to a Pulse application, complete the following steps:

1. Select the  button in the top right corner.
2. Review the changes that were made since the last publish.
3. Select the **Rolling Publish** button to start publishing.



Workflow validations

The **Rolling Publish** button will be disabled while Pulse validates all workflows in the application, or if those validations fail. For more details on this integrity protection mechanism, refer to the [Pre-publish workflow validations](#) section.

Pulse publishes application changes on only one node/application replica at a time to avoid going offline entirely. Only minor delays in some events may occur.

In very rare cases, if you want to publish the changes on all nodes at the same time, select the arrow next to **Rolling Publish**, and select the **Publish** option.



Publish option causes downtime

If you select the **Publish** option instead of **Rolling Publish**, Pulse will have a downtime. In production environments, the **Publish** option should be used only in exceptional cases by users that are very knowledgeable about the system.

Note that if you made changes to the schema that is in use in the workflow (and select the new version of the schema for the workflow), Rolling Publish cannot be used.

4. Pulse shows the progress during the publish process:
5. If you select the **Close** button during the publish process, Pulse still finishes publishing in the background. You can see that it was successful if the **Publish** button disappears from Pulse. This means that there are no changed waiting to be published.

6. If you select the **Abort** button during the publish process, Pulse stops the process at that point. The state of the application after the abort is not predictable. You can roll back the application to the state at which it was before the publish. Read more about rollbacks in the [Error handling](#) section.
7. If you keep the **Publish application** window open, it shows the result of the publish process. See the [Error handling](#) section if you encounter an issue.

Advanced usage

Live publish

If the update to the application can be done without the necessity to restart Pulse nodes, Pulse performs a **live publish**. Only minor delays in the processing of some events may occur.

You can see on the **Publish** button when a change is eligible for a live publish:

Live publish is restricted to changes in rules and lists.

More specifically, if any of the following have changed since the last publish, the application is not eligible for a live publish:

- input schema
- name and description
- outcome configuration
- outcome combinator
- the connections between workflow elements
- partition keys
- advanced properties
- action handlers
- event storage enabled state
- the set of workflow elements comprising the workflow
- for workflow elements:
 - id
 - name
 - output and input mapping
 - outcome configuration

Even when everything in the previous list remains unchanged, if the workflow element configuration changes (e.g. referencing a different plan snapshot), the workflow will require a full publish. The only exception to this is that rule snapshots referenced in a rules workflow element may be updated while still allowing a live publish.

Synchronizing inconsistent versions

When Pulse is running different versions of an application in different cluster nodes, Pulse warns you to synchronize all nodes to use the same version before publishing.

To manually fix inconsistent versions, complete the following steps:

1. Select the Synchronize button.
2. Optionally, view unstable nodes to see which application versions are running on different nodes.
3. Choose the application version to publish in all nodes. By default, Pulse synchronizes all nodes with the most recent stable version.

Order of changes on each app replica

When Pulse completes a **Rolling Publish** on a cluster with many app replicas, it proceeds in the following order:

1. App replicas that are not published.
2. App replicas that are published but not on the target version.
3. App replicas that are published on the target version.
4. Inside each category of the previous points, the leader app replica of each data center is the last one to be rolled out.

If you change the publish type to **Publish** instead of **Rolling Publish**, the rules are the following:

- Pulse takes offline all nodes.
- The "Stop", "Bootstrap", and "Publish" operations are executed on all data centers in parallel.
- Other than the operations mentioned in the previous point, Pulse completes every operation on one data center at a time. The primary data center is always updated first. For all others, no order is guaranteed. If anything fails on one data center, it does not proceed to the next one.

Error handling🔗

When an error occurs, you have the following options:

- **Retry**: The application will be published again in all the replicas. Replicas that already applied the application changes will not do any work since they are already in the expected state.
- **Rollback**: Every replica will recover its last stable version of the application, i.e., the last version successful published in all the replicas. The rollback preserves the operation configurations (e.g., restart, rolling vs parallel, etc...)
- **Abort**: The application publish will be stopped. Notice that nothing else will be done. This means the state after the abort is not defined.

When publishing fails on a replica, the whole operation is considered as failed. Pulse will not publish the application in the remaining replicas. It shows the error and waits for the user to decide how to proceed.

Schema changes🔗

Rolling Publish does not apply when you change a schema that is used in a workflow, and select the new version of the schema for the workflow. Changing the schema makes messaging inconsistent and error prone since there could be replicas using an older version of the application. For example:

- Trying to process rules that match removed schema fields.
- Adding new fields to the schema. In this case, when persisting the events in the [Event Storage](#), they will have more columns than expected.

Multiple rollbacks🔗

When there is an error during a publishing operation and the rollback also fails, you can use the **Rollback** button again.

Instead of restoring another previous version of the application, this will work as a retry to restore the last stable version of the application in all replicas.

Multiple online versions🔗

When an update operation fails, there may be two or more application replicas with different versions of the same application.

If this occurs, then the application state will be set to *DEGRADED*. Whenever there are different versions of the same application in the data center, [workflow synchronization](#) messages from replicas running a different version of an application will be discarded to keep consistency.

In addition, if the [Auto-Repair](#) capabilities of Pulse are enabled, then Pulse will correct the inconsistent versions automatically, otherwise, it is the responsibility of the user to manually synchronize the applications as shown in this page.

Pre-publish workflow validations🔗

Workflow validations occur when users open or modify a workflow. It is possible to save changes to a workflow before the validation phase is complete, or even if validation fails, to allow users to do incremental work on workflows without being blocked by validation errors. However, publishing an app with invalid workflows would result in an [error](#) and set the App into a degraded state (which in turn would create pressure on the remaining healthy apps). To prevent this, Pulse only allows publishing an application when all its workflows are successfully validated.

When an app's Publish button is selected, Pulse displays the Publish window, allowing the user to review the changes made since the last publish. At this point, Pulse automatically validates all the workflows in that app, and only enables the **Rolling Publish** button if all workflow validations succeed.

-



- [Working with Applications](#)
- Resource Versioning

On this page

Resource Versioning

Was this page helpful? 

Versioning is an orthogonal concept to Pulse. Whenever a user creates, updates or deletes any of the application resources, Streams, Dimensions, etc. or the applications themselves, these operations are logged to persistence storage. There are three types of revert operations possible:

- Revert a specific resource to how it was before a given change occurred
- Revert all latest changes made to an application up to some point in time
- Revert the deletion of an application

We will look at how each of these revert types work, their restrictions and implications.

Structure of the Operations Log

Each visible row in the log depicts a change. A change maps to one or more operations. Most of the changes will have only one operation but others will be a group of related operations. An example of a change comprising several operations is creating a Schema. Each schema field marked as dimension will also trigger the creation of a Dimension Mapping. So a new schema with two dimension fields will insert three entries in the operations log. One create entry for the schema and one create entry for each new dimension mapping. When a change is comprised of several operations it is possible to see the individual operations.

Change entries in operations log are always shown in descending time order, i.e. the newest change on top.

Querying the Operations Log

The operations log is visible in several Pulseviews screens.

Depending on where in Pulseviews you are viewing the operations log, the information presented therein will vary. All application resources edit screens show the operations log for that particular resource. To view the application log for a particular resource the user must have the `applications:history:resource:read` permission.

A filterable in-application log is available in the History of the application. To view the global application log the user must have the `applications:history:global:read` permission.

A log of deleted applications is also available to administrators and allows the administrator to recreate the application as it was before being deleted.

Reverting the Operations

This section details how the reverting works and the requirements for each case.

Reverting a specific resource operation

The main use-case for this revert strategy is to enable a user to revert a specific resource to the way it was somewhere in the past without changing the other resources in the application. If two users are editing different rules in the same application each user can revert his rule without impacting the other user's rule.

To revert a specific resource operation the user must have the `applications:history:resource:revert` permission.

Note that operations are grouped in changes, so when a revert operation is requested all the operations in that particular change will be reverted. This is necessary to ensure consistency when handling dependent resources.

This particular revert strategy is not always possible. If any resource that depends on this one or that this resource depends on has changed after the operation we want to revert, the revert will not be available. This limitation exists because it is not possible to guarantee that the resource would still have valid relationships with both the resources it depends on and the resources depending on them after the revert operation. In these cases, the only options left are reverting the changes by manually editing the resource or revert the application. See the next revert strategy.

The revert operation itself can also be reverted leaving the resource as it was if no revert operation has occurred in the first place.

Example of reverting a specific resource

Assuming the user actions are:

1. Create schema 1
2. Create workflow 1
3. Update workflow 1
4. Create rule 1
5. Update rule 1
6. Update rule 1 (again)
7. Create rule 2

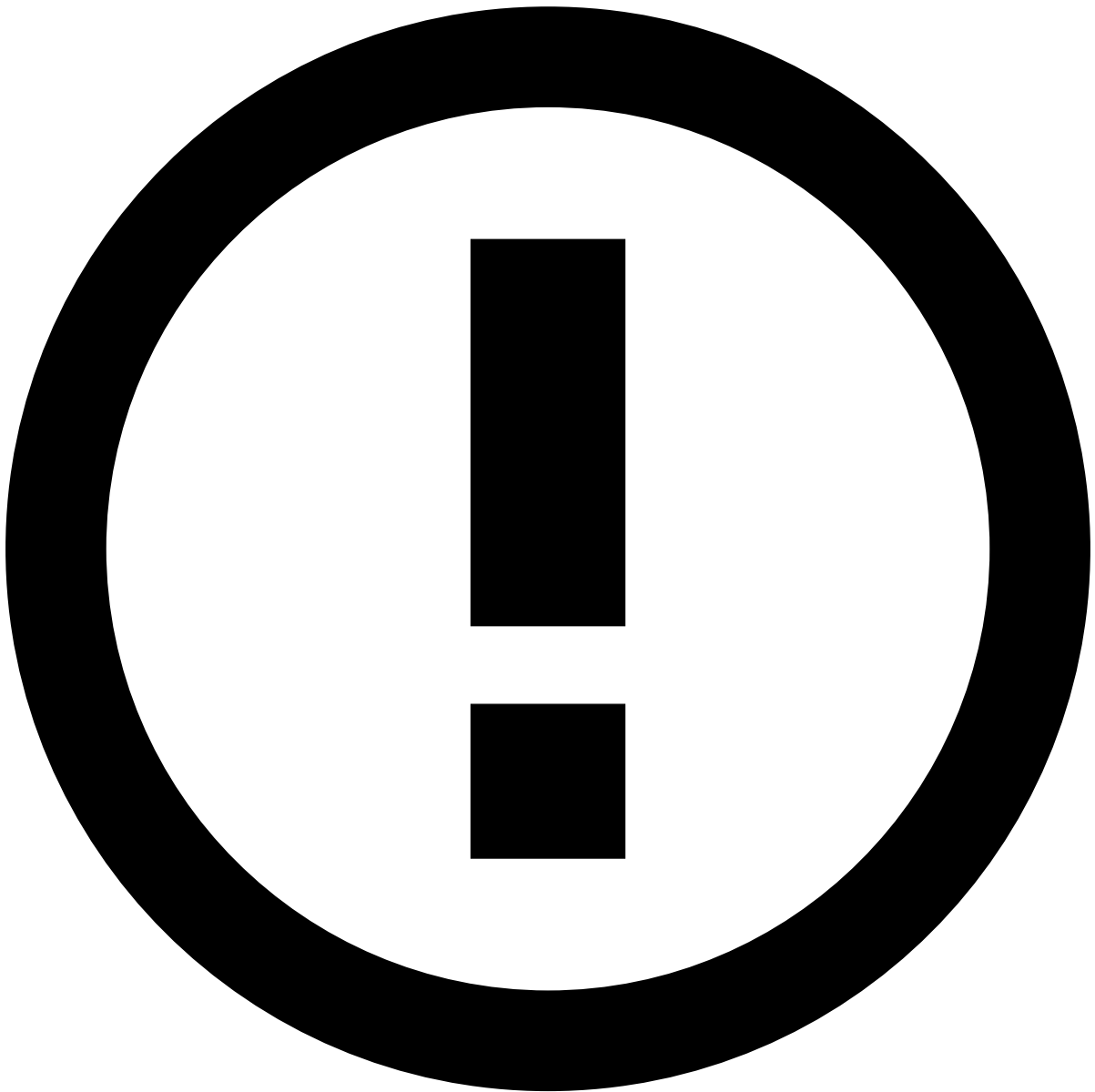
For these operations, the operations log is as follows (oldest operations at the bottom):

Change	Resource	Operation
7	RULE 2	CREATE
6	RULE 1	UPDATE
5	RULE 1	UPDATE
4	RULE 1	CREATE
3	WORKFLOW 1	UPDATE
2	WORKFLOW 1	CREATE
1	SCHEMA 1	CREATE

By reverting a single resource, it is possible to get the rule 1 as it was before change 5 (highlighted in red) without undoing the create of rule 2. This is achieved by reverting to the rule 1 as it was before change 5.

After this revert, the operations log would have a new entry, the opposite operation of the one we are reverting:

Change	Resource	Operation
8	RULE 1	UPDATE
7	RULE 2	CREATE
6	RULE 1	UPDATE
5	RULE 1	UPDATE
4	RULE 1	CREATE
3	WORKFLOW 1	UPDATE
2	WORKFLOW 1	CREATE
1	SCHEMA 1	CREATE



info

It is not possible to revert change 3 using this strategy because dependencies of that workflow (rules 1 and 2) have changed.

Reverting an Application to a specific point in time🔖📅

This revert strategy is very simple and works by applying the contrary operation to each operation down to the chosen point. A

s long as the user has permissions to do so, it should always be possible to apply this strategy. One possible downside of this revert strategy is that all changes made from that point on will be reverted.

To revert an application the user must have the `applications:history:global:revert` permission.

Note that reverting multiple changes will be grouped together as one single revert changed, composed of multiple operations. In this revert strategy, the revert operation can also be reverted itself in one step.

Example of reverting an Application

Assuming the user actions are:

1. Create schema 1
2. Create workflow 1
3. Update workflow 1
4. Create rule 1

The operations log is:

Change	Resource	Operation
4	RULE 1	CREATE
3	WORKFLOW 1	UPDATE
2	WORKFLOW 1	CREATE
1	SCHEMA 1	CREATE

The user might want to revert the application state back to as it was before change 3 (highlighted). This strategy would achieve this by reverting change by change down to change 3 (inclusive) so that in the end the operations log would be:

Change	Resource	Operation
5	WORKFLOW 1	UPDATE
5	RULE 1	DELETE
4	RULE 1	CREATE
3	WORKFLOW 1	UPDATE
2	WORKFLOW 1	CREATE
1	SCHEMA 1	CREATE

Notice that the change id is the same (5) in all the operations pertaining to this revert, so that this change can be undone in a single step, as the user expects.

Reverting a deleted Application

The log of deleted applications shows all deleted applications that can be recovered to the state it was before being deleted. Applications that were deleted and then imported are excluded from the log since they currently exist. Only applications that don't currently exist are shown. To revert a deleted application the user must be an **administrator**. If the application was deleted multiple times, only the latest delete is shown. After reverting the delete, the application looks exactly as it was before being deleted.

On this page

Was this page helpful? 

TCP ports used by Pulse

This page provides a list of TCP ports opened by the components of Pulse, when running it locally with default setup (embedded Zookeeper, ActiveMQ and H2 database).

Pulse Server

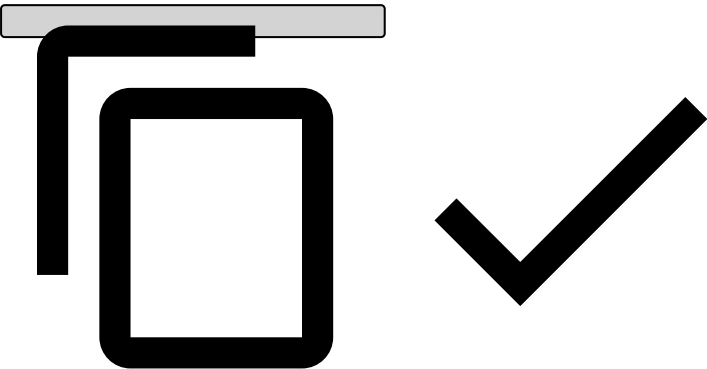
Fixed ports

- 2181: Zookeeper
- 61615: ActiveMQ
- 1099: Rmi Registry
- 1100: Remote Pulse Server object
- 9010: Jmx
- 8080: http
- 8443: https

Random ports

- H2 Database (Not used in production environment)
- ActiveMQ Jmx (Not used in production environment)
- JMX: (only when started with *startup-debug.sh* because *-Dcom.sun.management.jmxremote* is being used to launch the JVM)
- EhCache RMI port: can be configured on *conf/engine/shiro/ehcache.xml* by editing the *cacheManagerPeerListenerFactory* element and adding *properties="port=40001"*, for using port 40001.

```
<cacheManagerPeerListenerFactory>  
  class="net.sf.ehcache.distribution.RMICacheManagerPeerListenerFactory"  
  properties="port=40001"/>
```



App Engine

Fixed ports

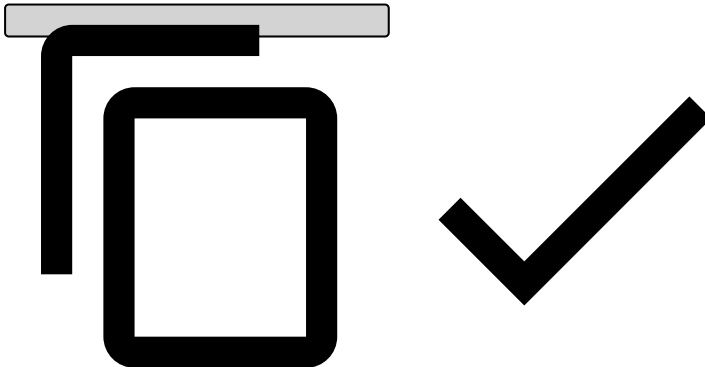
- 6990: App remote object and RMI registry

Random ports

- Jmx: configurable with:

```
# Global setting for all applications
pulse.global.appengine.jmxport=9011

# Configuration for a specific application
pulse.app.<app_slug>.appengine.jmxport=9013
```



- Other Jmx (because app engine is being started with* `-Dcom.sun.management.jmxremote`* a second random port is being open. This can probably be removed)
- On AppEngineProcess start, when getting the Pulse Server cluster leader remote reference, a RemoteExporter is being created and this opens a random RMI port as well. (This can potentially be fixed to use the configured RMI Object port. Not sure about the amount of effort because this code path is being used by other components)

Running processes on Spark (e.g. Data Source Analysis, Rules Snapshot Evaluation)

- A random port is opened (didn't have the chance to check where it comes from)

Conclusions

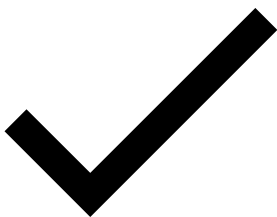
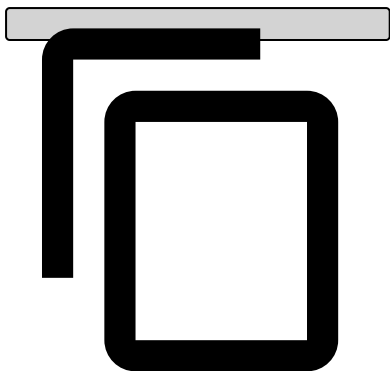
The table below summarizes the possible causes for TCP connections to random ports that are showing up on customer's logs and what could be a possible fix for each one:

Random port origin	Possible fix
EhCache RMI port	Configure as described above
App Engine Jmx	Configure as described above
App Engine Other Jmx	Remove <code>-Dcom.sun.management.jmxremote</code> from JVM args when launching application engine (if really not needed).

Typical customization

Typically, the following configurations are added/edited in `pulse.properties` to match the customer's environment.

```
pulse.manager.rmi.localregistryport=1099
pulse.global.services.modules.datascience.spark.forward.spark.driver.port=36560
pulse.global.services.modules.datascience.spark.forward.spark.blockManager.port=20800
pulse.global.services.modules.datascience.spark.forward.spark.port.maxRetries=20
pulse.manager.rmi.objectport=1100
```



-



- [Working with Applications](#)
- The Application Life Cycle

The Application Life Cycle

Was this page helpful? 

An application can be in one of the following states:

- **STOPPED**

In the **STOPPED** state, the application is completely shut down, without any alive App Engine.

- **BOOTSTRAPPED**

The **BOOTSTRAPPED** state means that the application is in *standby* state. All its services are started but they only validate the changes being done.

- **STARTED**

An application that is **STARTED** is assumed to have all of its application replicas alive.

- **DEGRADED**

The **DEGRADED** state indicates that an application may have an issue and may not fully provide all the processing guarantees. For example, there might not be any replica handling the events of a given partition. The application transitions to this state when one of the following conditions are met:

- There are application replicas running distinct versions.
- Not all of the replicas are started.
- Some replicas are down.

- **CRASHED**

An application becomes **CRASHED** if there are no application replicas alive that can be considered healthy.

- **SELF-HEALING**

An application starts a self-healing process if it has reached an unhealthy state and Pulse began performing recovery actions in an attempt to restore the application to a healthy state. To learn more about how and when these recovery actions are performed, see [Automatic Application Repair](#).

Furthermore, there are also four *maintenance* states, meaning that some state transition is taking place:

- **APPLYING CHANGES**

When updating the application.

- **BOOTSTRAPPING**

When the application crashed and is being restarted, or after stopping an application.

- **STARTING**

When the application changes from **BOOTSTRAPPED** to **STARTED**.

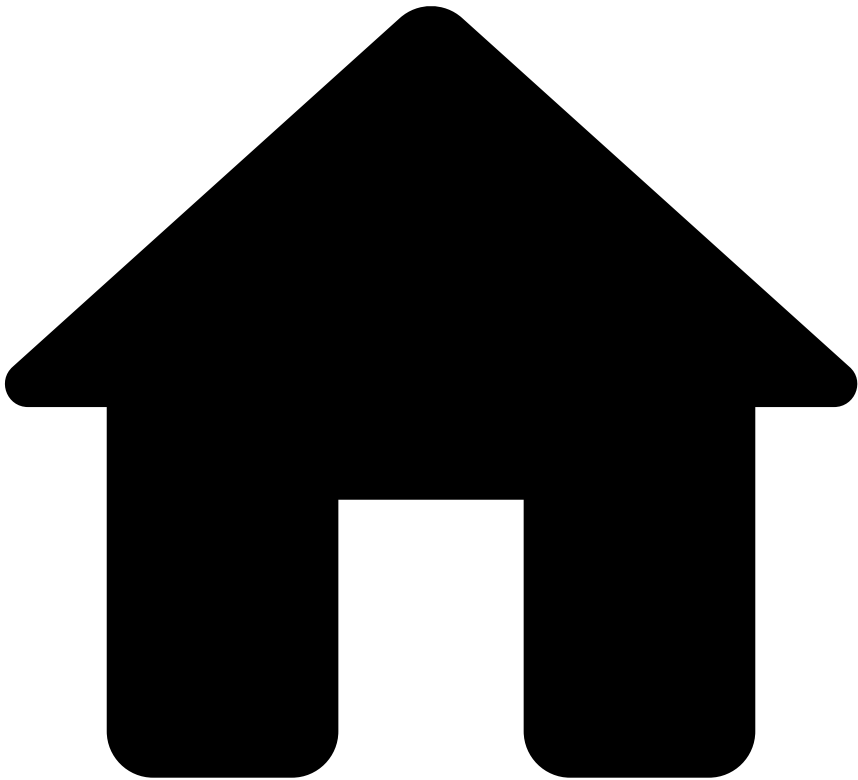
- **STOPPING**

When the application is being stopped.

An application in a maintenance state should be displayed as follows. Note that the wheel on the top right corner of the image should be spinning.

When the application is in a maintenance state, crashed application replicas will not be recovered. Moreover, no other Application Lifecycle operations (publish, stop, etc.) can be performed in parallel. They will, however, be queued if Pulse is configured to do so.

•



- [Component Encryption](#)
- Configuration Overview

On this page

Was this page helpful? 

Configuration Overview

Setting up component encryption requires configuring encryption between internal Pulse components as well as between external components (e.g. between RabbitMQ brokers, or between Cassandra Nodes). On the Pulse side, this is done by setting a default encryption configuration in the `pulse.global` properties, which can be overridden for each component that require different configurations.

Configuring encryption between Pulse and an external component requires configurations both in Pulse and in the external component. The configuration of the properties in Pulse are generally the same, regardless of the external component whereas the configuration in external components is specific to the component. The table in the section [Configuring external components](#) summarizes the main configuration requirements for each external component. The page [Configuring External Components](#) describes the configuration requirements for external components in more detail.

The following sections give you an overview of the security concepts and settings for component encryption.

Understanding keystores and truststores

Component encryption requires configuration of keystores on the server side and truststores on the client side. This applies regardless of the component; however, the format of the keystore and the truststore may vary between components. The following terminology is used to identify the different formats:

- **Java Truststore:** contains the public certificate of the CA that signs the server certificates in JKS format (.jks file). Is referred in this article as truststore.jks.
- **Java Keystore:** contains the server certificate signed by the CA and the server's private key (.jks file). Is referred in this article as keystore.jks.
- **Certificate Authority (CA):** contains the same as a truststore, but in a PEM format (.pem file). This format is required by Rabbit MQ. Is referred in this article as ca.pem.
- **Server certificate:** contains the same as a Keystore, but in PEM format (.pem file). Is referred in this article as certificate.pem.
- **Server certificate private key:** (.pem file). Referred in this article as key.pem.

The files mentioned above are used in the case when all certificates are signed by an internal CA and all components are configured to trust this CA (i.e. approach implemented in Pulse by default). Other scenarios are out of the scope of this page. In Pulse, the configuration files depend whether Pulse acts as a server or as a client. Pulse acts as a client in most cases in which Pulse server/application engine communicate with an external component (e.g. between Cassandra or RabbitMQ). It acts as a server in communications between Pulse internal components (e.g. between the application engine and the Pulse server with RMI).

To help the configuration process, Pulse includes scripts to help creating keystores and truststores. You can visit the page [Creating Certificates](#) for more information on how to configure these scripts.

Configuring Pulse properties

Component encryption is implemented in Pulse using a default configuration specifying the required files in the client and server side for all components. This default configuration can be overridden for any component. For example, if Cassandra needs a different file configuration, the properties can be overridden to create an exception for Casandra. With this approach, the Pulse properties of the components that share the same encryption configuration only need to be defined once.



Configuring custom properties

Some components require separate configuration not fully covered in the global encryption properties (e.g. relational databases). Visit the page [Configuring External Components](#) to learn more about this.

In the following example, all the components have default configuration for server authentication except for Cassandra, in which the encryption is disabled:

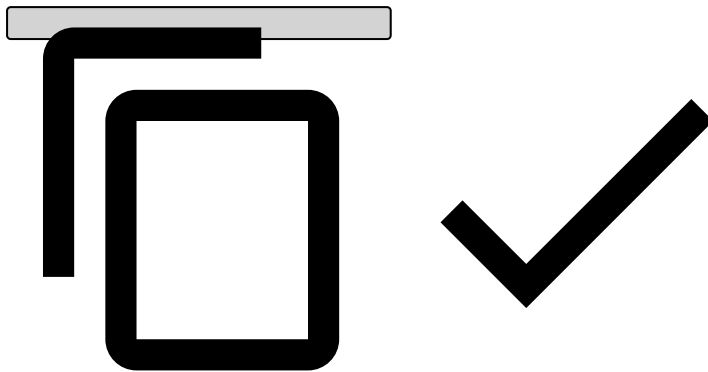
```
# Default encryption properties
pulse.global.encryption.trustLevel=SERVER_AUTH
pulse.global.encryption.protocol=TLS

# Default encryption configuration for client-side components
pulse.global.encryption.client.truststore.file=cert/trusted-ca.jks
pulse.global.encryption.client.truststore.storepwd=trusted-ca-store-pwd

# Default encryption configuration for server-side components
pulse.global.encryption.server.keystore.file=cert/my-identity.jks
pulse.global.encryption.server.keystore.storepwd=my-identity-store-pwd
```

```
pulse.global.encryption.server.keystore.keypwd=my-identity-key-pwd

# No encryption on Cassandra
pulse.global.cassandra.encryption.trustLevel=ENCRYPTION_NONE
```



The configuration above specifies the passwords for the truststore on a client side (trusted-ca.jks) and the keystore on the server side (my-identity.jks). Similarly to database connection passwords, store passwords may be encrypted.

Authentication

The authentication is specified in the trustLevel property as described in the above code block.

Pulse supports the following authentication:

- **Encryption Only:** Pulse does not validate the certificate presented by the server, and thus, truststores are not required on the Pulse side. A keystore is still required if RMI is to be encrypted since this component acts as a server. The value of the property is ENCRYPTION_ONLY.
- **Server Authentication:** Pulse validates the server certificate by checking if the certificate is valid and signed by a trusted entity. This requires a truststore on the client configuration and a keystore on the server configuration. The value of the property is SERVER_AUTH.
- **Client Authentication:** In addition to server authentication, clients present a certificate to the server and the server must to trust it. This requires an additional keystore on the client configuration and a truststore on the server configuration. The value of the property is CLIENT_AUTH.

Configuring external components

The following table provides a summary of the files that are required for each external component that communicates with Pulse.

The page [Configuring External Components](#) describes the configuration requirements for external components in more detail.

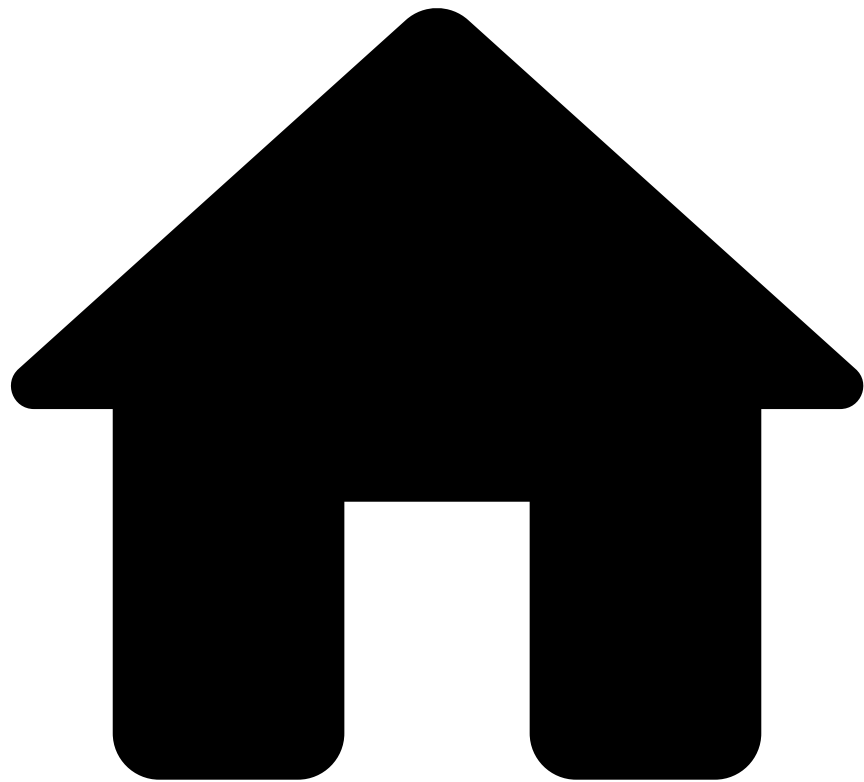
Origin (Client)	Destination (Server)	Client Configuration	Server Configuration
Pulse server, App Engine	Rabbit MQ	truststore.jks, truststore password	ca.pem, certificate.pem, key.pem
Rabbit MQ	Rabbit MQ	ca.pem	ca.pem, certificate.pem, key.pem
Pulse server, App Engine, Spark, any other java client app	Pulse server, App Engine	truststore.jks, truststore password	keystore.jks, server certificate key password, keystore password
Web Browser	Pulse server	should contain the certificate authority installed	keystore.jks, server certificate key password, keystore password
Pulse server, App Engine	Cassandra	truststore.jks, truststore password	keystore.jks, keystore password (needs to be the same as the server private key password)
Cassandra (inter-node communication)	Cassandra	truststore.jks, truststore password	keystore.jks, keystore password (needs to be the same as the server private key password)

Origin (Client)	Destination (Server)	Client Configuration	Server Configuration
Pulse server, App Engine	Spark	truststore.jks, truststore password	keystore.jks, keystore password
Pulse server, App Engine	Relational database	truststore.jks, truststore password	On Posgres: keystore.jks, keystore password. Other servers may have distinct requirements.

Next steps📄

Now that you have basic knowledge about the default Pulse properties, check the page [Configuring External Components](#) for more information about configuring component encryption.

•



- [Component Encryption](#)
- Configuring Encryption for External Components

On this page

Was this page helpful? 

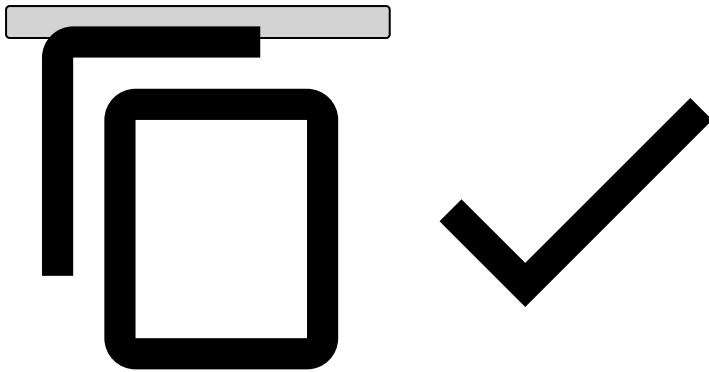
Configuring Encryption for External Components

This page walks you through the configuration of component encryption, with focus on external components.

RMI configuration

The RMI encryption configuration is inherited from the default one (`pulse.global.encryption.*`) unless it is overridden in `pulse.manager.rmi.encryption.*` . For example, to disable encryption on RMI communications, set this property as follows:

```
pulse.manager.rmi.encryption.trustLevel=ENCRYPTION_NONE
```



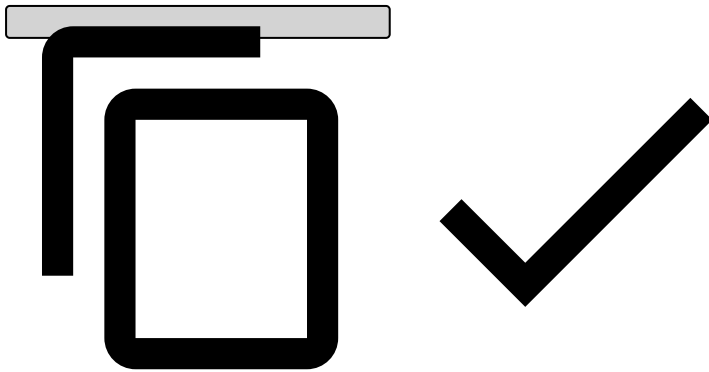
External RMI clients (e.g., custom services) need to be updated if they communicate with a Pulse server with encrypted RMI – see the API documentation for further information.

Pulse Management Tool configuration [↗](#)

The Pulse Management Tool (`pulsemgmt.sh`) communicates with a running Pulse server using JMX through an RMI connector. If RMI is configured to use encryption, this tool must be invoked with a correspondent client-side encryption configuration.

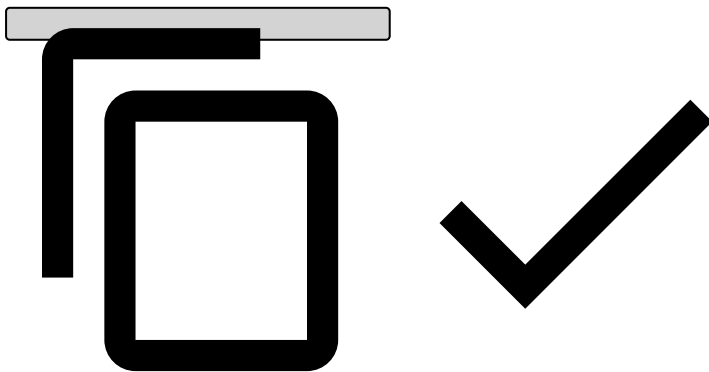
The Pulse Management Tool is completely standalone (i.e. it only manipulates MBeans), and so it doesn't access the Pulse configuration. To use it with encryption, you must set up (and system protect) a file with the encryption configuration and invoke the tool with the **-encryption** command line option, e.g.:

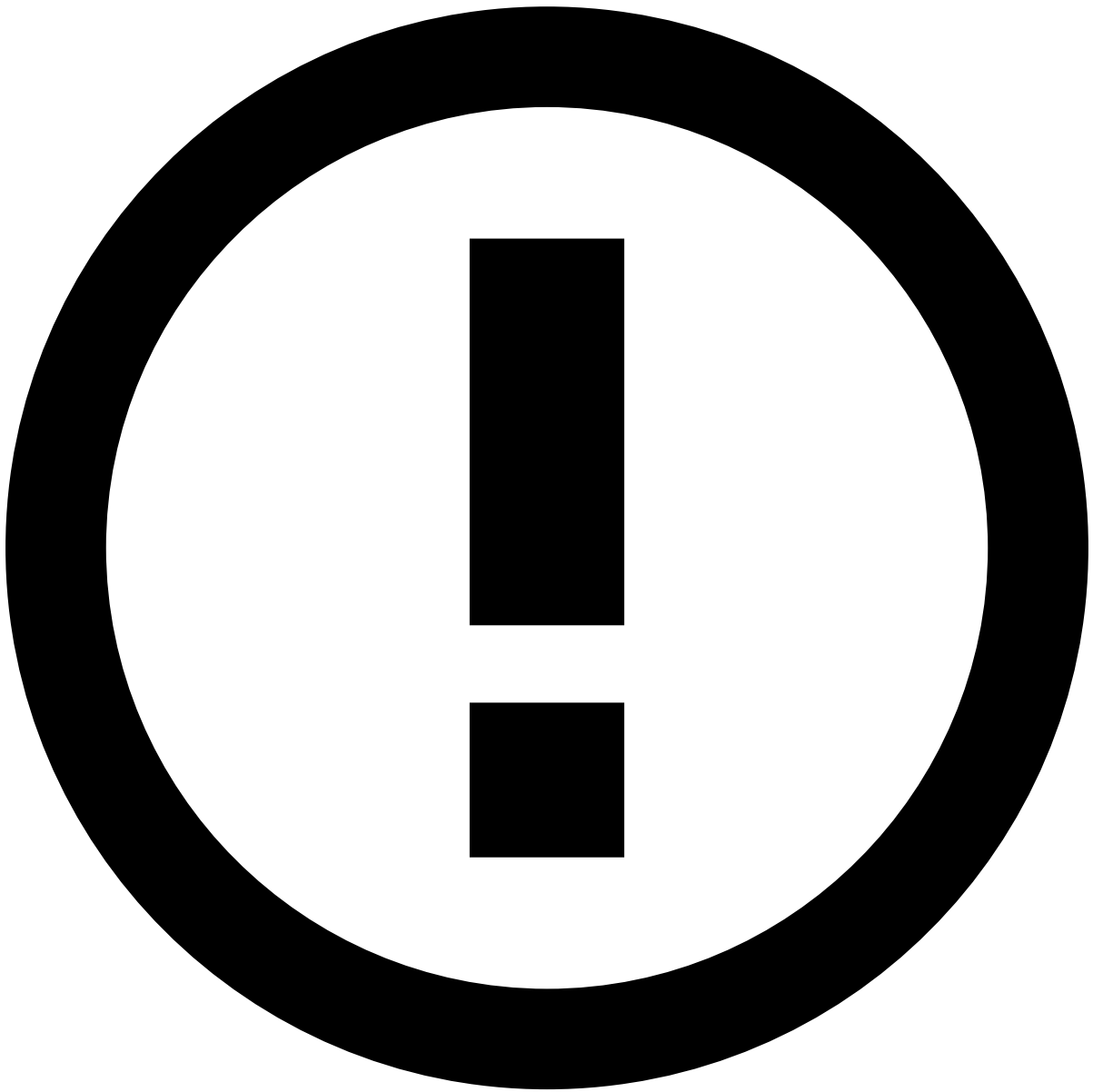
```
$ pulsemgmt.sh -shutdowncluster -encryption encrypt.properties
```



The following is an example of an `encrypt.properties` file:

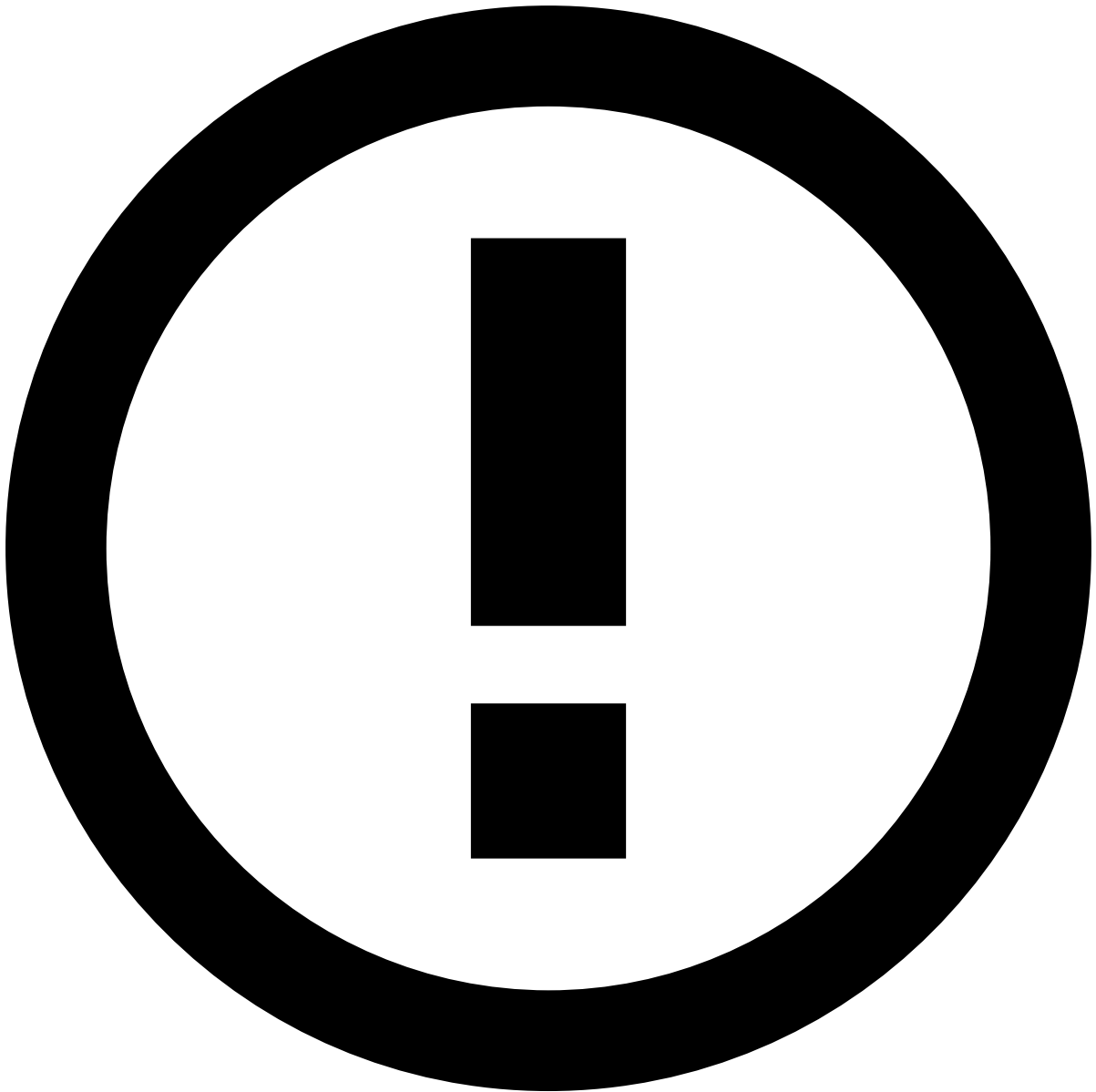
```
encryption.trustLevel=SERVER_AUTH
encryption.client.truststore.file=/opt/pulse/conf/engine/secret/trusted-ca.jks
encryption.client.truststore.storepwd=trusted-ca-store-pwd
```





info

The `encrypt.properties` file must be protected with restricted read permissions. Note that the passwords in this file are not encrypted.



Rabbit Version

Use of SSL in Rabbit was verified in version 3.6.14 / Erlang 20.1. Previous versions have known issues with SSL, see <https://www.rabbitmq.com/which-erlang.md>.

Rabbit requires separate configurations for each of the following channels:

- Connections between the RabbitMQ broker (server) and Pulse (client).
- Intra-datacenter broker connections, for mirroring.
- Inter-datacenter broker connections, for federation.

These are documented in the following sections.

Between broker (RabbitMQ) and Pulse

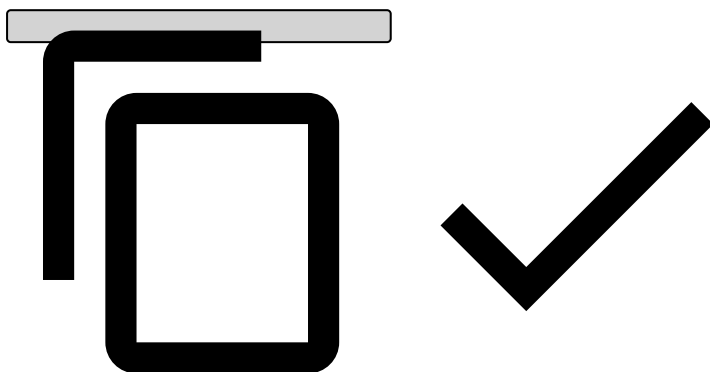
Pulse side

The Rabbit encryption configuration is inherited from the default client one (`pulse.global.encryption.client.*`) unless it is overridden in `pulse.manager.cluster.dc.messaging.encryption.*` . Having SSL requires the broker IPs to have the port explicitly configured in `pulse.manager.cluster.dc.messaging.brokers` . This port is defined on the Rabbit-side configuration defined below.

Rabbit side

Rabbit requires the CA certificate, server certificate and server private key to be in separate files, in .PEM format. The certificate generation documentation shows how to generate these in a PKCS12 file (keystore.pkcs12). Having this file, the above PEM files can be extracted as follows:

```
$ openssl pkcs12 -in keystore.pkcs12 -passin file:secret/keystore.pwd -nodes -nokeys -cacerts -out cacert.pem
$ openssl pkcs12 -in keystore.pkcs12 -passin file:secret/keystore.pwd -nodes -nokeys -clcerts -out cert.pem
$ openssl pkcs12 -in keystore.pkcs12 -passin file:secret/keystore.pwd -nocerts -out key.pem
```





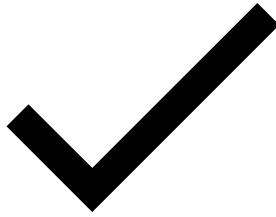
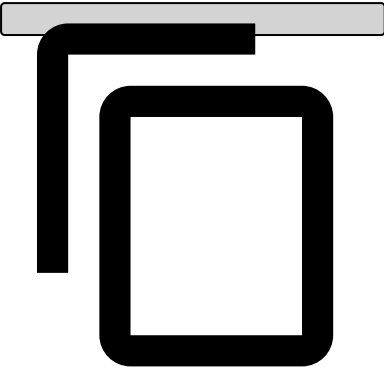
info

The key.pem password is requested interactively by openssl when introducing the third line of the above code block.

These PEM files are then configured on the rabbitmq.config file (see <https://www.rabbitmq.com/ssl.html> and <http://erlang.org/doc/man/ssl.html> for detail the `ssl_options` section):

```
[
  {rabbit, [
    {loopback_users, []},
    {ssl_listeners, [5671]},
    {ssl_options, [
      {cacertfile, "/path/to/cacert.pem"},
      {certfile, "/path/to/cert.pem"},
      {keyfile, "/path/to/key.pem"},
      {password, "<key.pem password>"},
      {verify, verify_none},
      {fail_if_no_peer_cert, false}
    ]}
  ]}
```

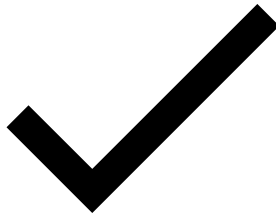
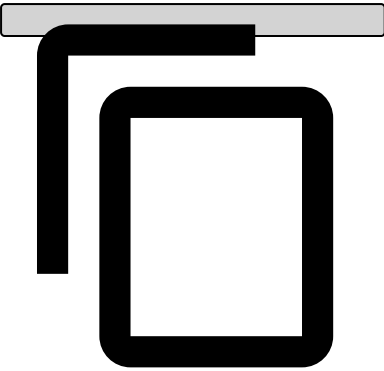
```
}  
1.
```



Intra-datacenter broker communication[🔗](#)

Intra-datacenter communication in RabbitMQ requires the certfile and unencrypted keyfile to be included in a single file:

```
$ cat /path/to/cert.pem > /path/to/chain.pem  
$ openssl rsa -in /path/to/key.pem -passin pass:<key.pem password> >> /path/to/chain.pem
```





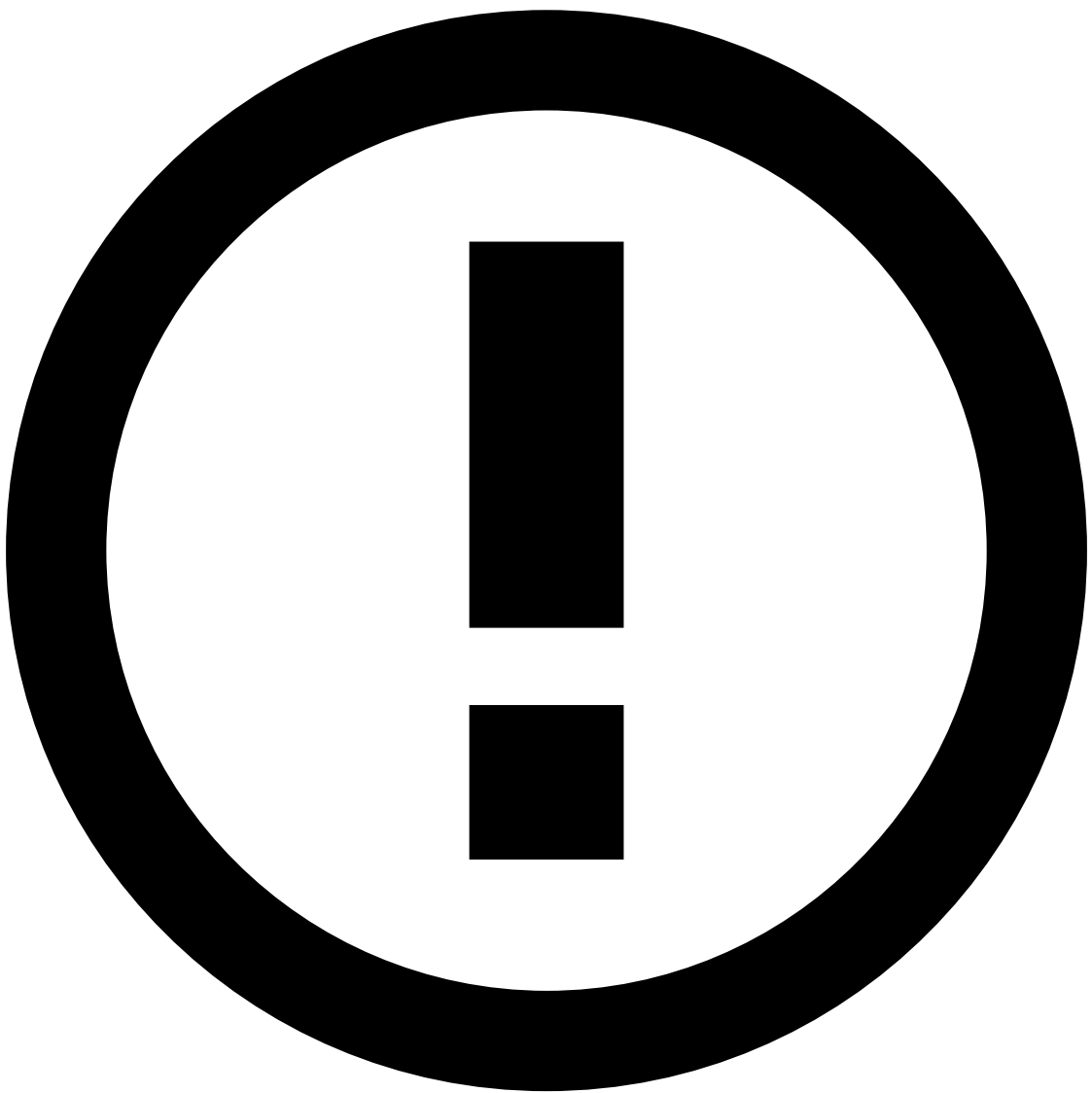
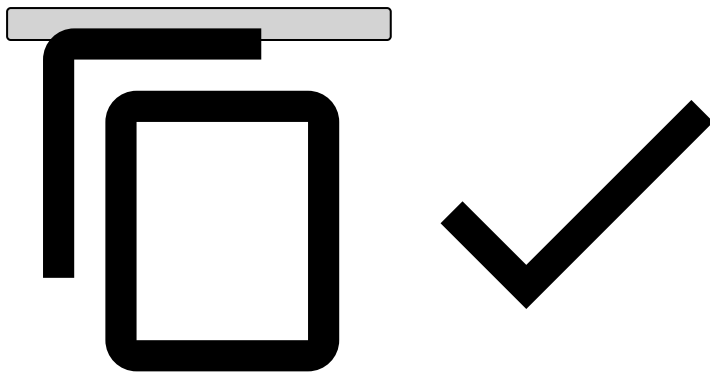
info

The chain.pem contains the unencrypted key, and the reading access should be restricted.

Since RabbitMQ relies on an underlying implementation of Erlang to establish its connections, some Erlang-level configurations must be provided. These are done via environment variables which are set through the rabbitmq-env.conf, which is automatically loaded when Rabbit MQ server is started.

1. The following is an implementation of using Erlang:

```
$ echo `erl -eval 'io:format("~p", [code:lib_dir(ssl, ebin)]),halt().' -noshell`  
"/path/to/erlang/<ssl-version>/ebin"
```



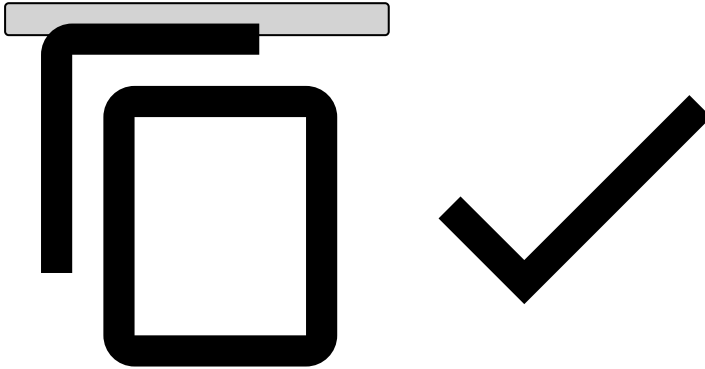
info

The output of the command above includes quotes, so it cannot be directly used via a `$()` subshell, or they would be escaped and considered part of the string.

2. Create the file `rabbitmq-env.conf` in the directory `/etc/rabbitmq`.

3. Add environment variables to the file created in step 2. Remember to replace the SSL path that you obtained in step 1. A recommended approach is to take value of the output of step 2 and set the environment variable for the Erlang path without the quotes. See an example in the next code block:

```
ERL_SSL_PATH="/path/to/erlang/<ssl-version>/ebin"
RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS="-pa ${ERL_SSL_PATH} -proto_dist inet_tls -
ssl_dist_opt server_certfile /path/to/chain.pem -ssl_dist_opt server_secure_renegotiate
true client_secure_renegotiate true"
RABBITMQ_CTL_ERL_ARGS="-pa ${ERL_SSL_PATH} -proto_dist inet_tls -ssl_dist_opt
server_certfile /path/to/chain.pem -ssl_dist_opt server_secure_renegotiate true
client_secure_renegotiate true"
```



See <https://www.rabbitmq.com/clustering-ssl.html> for RabbitMQ's own information.

Inter-datacenter broker communication (federation)

Configuring federation with encryption requires the following changes relative to a non-encrypted setup:

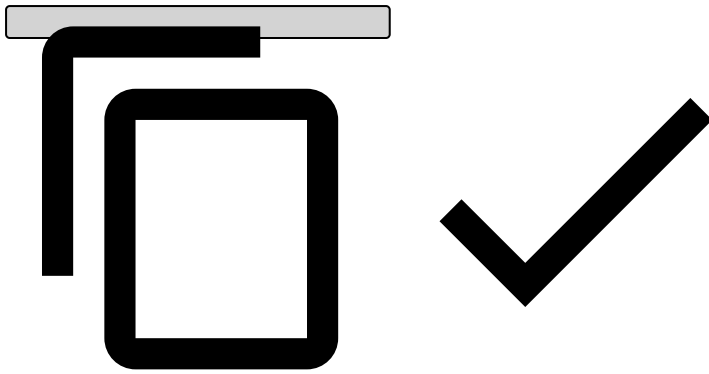
1. The protocol changes from "amqp" to "amqps"
2. The address now receives the explicit port (:5671)

An example of SSL federation URI with the changes already applied is provided below:

```
# RabbitMQ Cluster 1
rabbitmqctl set_parameter federation-upstream my-upstream \
'{"uri":["amqps://<user>:<pass>@<server in cluster 2>:5671", "amqps://<user>:<pass>@<server
in cluster 2>:5671", ...], "ack-mode":"no-ack"}'
rabbitmqctl set_policy --apply-to exchanges federate-me "^global-" \
'{"federation-upstream":"my-upstream"}'

# RabbitMQ Cluster 2
rabbitmqctl set_parameter federation-upstream my-upstream \
'{"uri":["amqps://<user>:<pass>@<server in cluster 1>:5671", "amqps://<user>:<pass>@<server
in cluster 1>:5671", ...], "ack-mode":"no-ack"}'
rabbitmqctl set_policy --apply-to exchanges federate-me "^global-" \
'{"federation-upstream":"my-upstream"}'

# Make federation queues replicated so they are also highly available
rabbitmqctl set_policy replicate-queues-federation "^federation" '{"ha-mode":"all"}' --apply-to
queues
```



Kafka¹

Kafka supports all encryption trust levels defined in Pulse. As a Java-based system, it supports the standard Java certificate formats used in Pulse and other Java-based components.

Pulse side¹

The Kafka encryption configuration is inherited from the default client configuration defined by properties with the `pulse.global.encryption.client.*` prefix. However, it can be overridden by configurations defined with the `pulse.manager.cluster.dc.<dc>.messaging.encryption.*` prefix. In addition to the standard Pulse encryption, Kafka also supports server authentication based on the IP address of the certificate. This is not enabled by default. To enable it, set the `ssl.endpoint.identification.algorithm` raw property to `https`.

Broker side¹

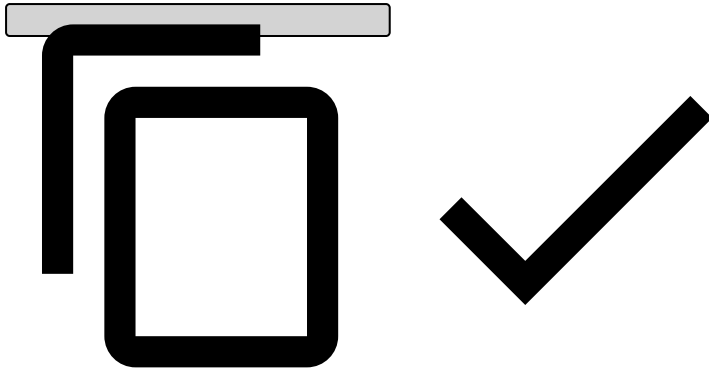
The Kafka broker configuration is very similar to the Pulse/Server configuration. Server authentication requires a keystore in jks format, whereas client authentication requires a truststore for the trusted clients. These are configured in `server.properties` as shown below (for server authentication only):

```
# Listener protocol must be either SSL or SASL_SSL, depending on SASL authentication being used
or not
listeners=SASL_SSL://172.17.0.3:9092

# Key and truststore properties
ssl.keystore.location = /path/to/keystore.jks
ssl.keystore.password = <the.keystore.password>
ssl.key.password = <the.keystore.private.key.password>
ssl.truststore.location = /path/to/trustedca.jks
ssl.truststore.password = <the.truststore.password>

# Disable hostname verification
ssl.endpoint.identification.algorithm=

# Needed because otherwise kerberos is assumed
sasl.enabled.mechanisms=PLAIN
sasl.mechanism.inter.broker.protocol=PLAIN
```

This configuration includes SSL on inter-broker communications.

Mirror-maker^â

Mirror-maker is used on multi-datacenter setups to achieve topic federation between data centers. Should SSL be used, then the *server.properties* SSL and SASL properties described earlier in this page must also be specified in the mirror-maker *consumer.properties* and *producer.properties* files.

Relational DB configuration^â

Postgres^â

Server configuration^â

Enabling SSL on a postgres server requires a server certificate and private key with unprotected PEM format. This can be extracted using the same procedure as the one described above for Rabbit.

These files can be installed on the server as described in <https://www.postgresql.org/docs/9.5/static/ssl-tcp.html>.

Client (Pulse) configuration^â

The client side is configured at the JDBC connection string level with two available options (see <https://jdbc.postgresql.org/documentation/91/ssl-client.html>):

- **Without server certificate validations:** Connection URL parameter `ssl=true&sslfactory=org.postgresql.ssl.NonValidatingFactory` needs to be defined. This skips server certificate validations, and thus, does not require a truststore. However, this option is less secure.
- **With server certificate validations:** Connection URL parameter `ssl=true` needs to be defined as well as the two system properties `javax.net.ssl.trustStore` and `javax.net.ssl.trustStorePassword`. The last two system properties only need to be defined if the global pulse configuration is not set.

Oracle^â

Client (Pulse) configuration^â

The Oracle JDBC thin driver supports JSSE encryption through its standard `javax.net.ssl.*` properties, which are automatically set on all Pulse components as long as a global encryption configuration is defined. The Pulse side configuration requires the following:

- JKS keystore and truststore configured in the `pulse.global.encryption.*` properties.
- *tnsnames* definition in the connection URL specifying the tcps protocol, as described in https://docs.oracle.com/cd/B19306_01/java.102/b14355/sslthin.htm#BABHFBJD.

Both server and client authentication are supported.

Cassandra configuration^â

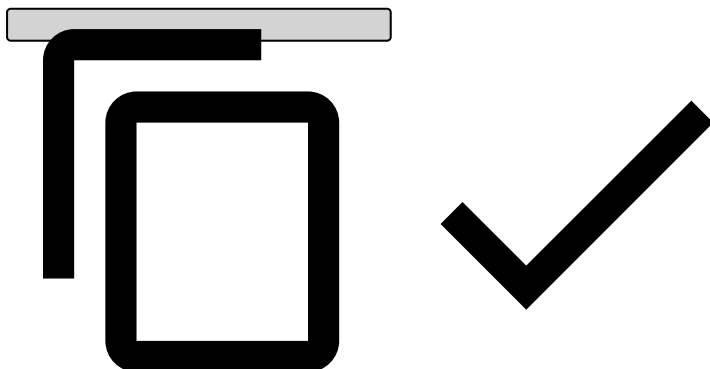
The full Datastax documentation can be found [here](#).

Client (Pulse) to node

Cassandra configuration in `cassandra.yaml` is very similar to the Pulse/Server configuration. Server authentication requires a keystore in jks format whereas client authentication requires a truststore for the trusted clients. These are configured in the `cassandra.yaml` `client_encryption_options` as shown below (for server authentication only):

```
# enable or disable client/server encryption.
client_encryption_options:
  enabled: true
  # If enabled and optional is set to true encrypted and unencrypted connections are handled.
  optional: false
  keystore: path/to/keystore.jks
  keystore_password: <the.keystore.password>
  # require_client_auth: false
  # Set trustore and truststore_password if require_client_auth is true
  # truststore: conf/.truststore
  # truststore_password: cassandra

# More advanced defaults below:
# protocol: TLS
# algorithm: SunX509
# store_type: JKS
# cipher_suites:
[TLS_RSA_WITH_AES_128_CBC_SHA,TLS_RSA_WITH_AES_256_CBC_SHA,TLS_DHE_RSA_WITH_AES_128_CBC_SHA,TLS_
DHE_RSA_WITH_AES_256_CBC_SHA,TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA,TLS_ECDHE_RSA_WITH_AES_256_CBC_
SHA]
```



The keystore file can be the same as in Pulse as long as it is signed with the CA that Pulse trusts. Additional certificates are not required on the Pulse side.

The encryption configuration on the Pulse side can be specified either under `pulse.global.encryption.client.*` (or under `pulse.global.cassandra.encryption.*` if a specific configuration is required for Cassandra).

Node-to-node

Node-to-node encryption is configured similarly to the client-to-node encryption. Client-to-node is defined in the `client_encryption_option`, whereas the node-to-node is defined in the `server_encryption_options` section.

As with the node-to-node configuration, JKS key and trust stores may be required.

See <https://docs.datastax.com/en/cassandra/3.0/cassandra/configuration/secureSSLNodeToNode.html> for details.

Jetty server / REST API configuration

REST API encryption can be configured with self-signed certificates (without CA) as described [here](#). This method can be used in a scenario in which only Jetty SSL is configured. However, the certificates generated by the scripts described in the page [Creating Certificates](#) could be used instead. The scripts are configured in `$PULSE_HOME/conf/engine/jetty/jetty.xml` as defined [here](#).

Spark configuration

Configuring communications security on Spark requires addressing:

- Communications between Pulse code running on Spark Executors and Pulse.
- Communications between Pulse code running on Spark Executors and the Spark Cluster (e.g., HDFS access on data source analysis).
- Communications between Pulse and the spark cluster (e.g., launching jobs, HDFS access on data source previews).
- Communications between cluster components. These depend on the containers where spark is deployed.

These are documented in the following sections.

Securing communications between Pulse executors and Pulse

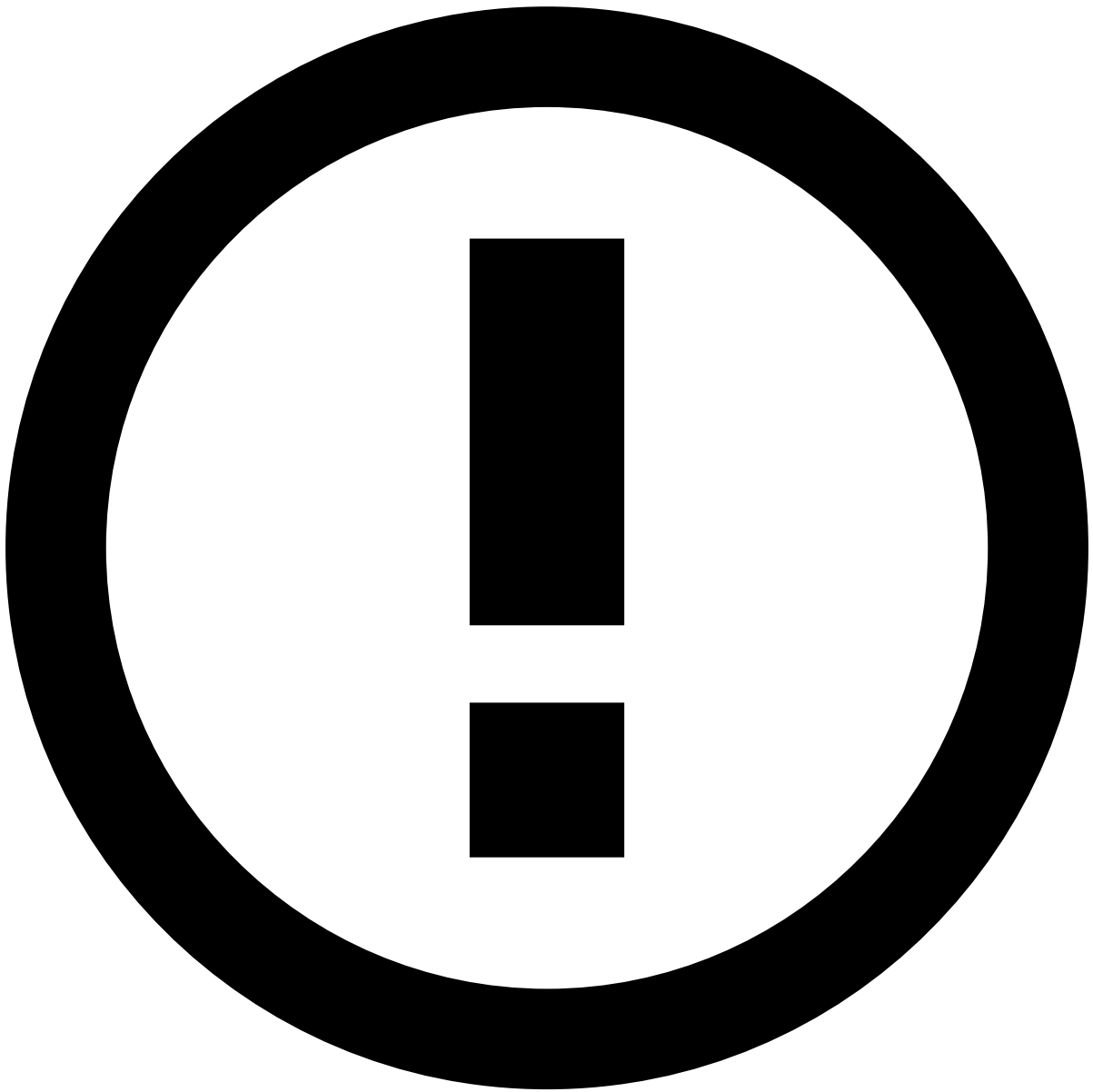
Pulse jobs submitted to Spark need to communicate with Pulse through RMI or access event/profile storage, and so they require truststores (and possibly keystores if client authentication is used). By default, executors inherit the encryption configuration defined in `pulse.global` and the event/profile storage config, and thus a truststore must be present on the configured location and with the same passwords. This truststore can be the same as the one specified for Spark internal protocols.

Spark standalone

Protocols over SSL

Pulse side

SSL security configuration passed to Spark when submitting jobs is specified, in Pulse, under the `pulse.global.services.modules.datascience.spark.forward.spark.ssl.*` space.



info

The `spark.*` properties mentioned in this section are set under `pulse.global.services.modules.datascience.spark.forward.*`.

The properties below inherit the settings set in the Pulse global encryption configuration if not set explicitly:

- `spark.ssl.protocol`
- `spark.ssl.keyPassword`
- `spark.ssl.keyStore`
- `spark.ssl.keyStorePassword`
- `spark.ssl.trustStore`
- `spark.ssl.trustStorePassword`

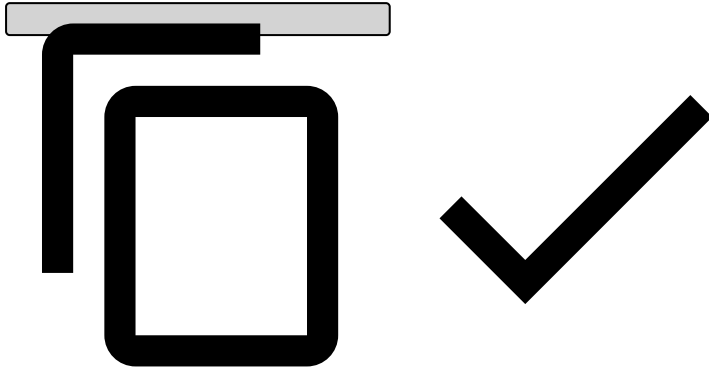
The SSL properties above specify default properties for the following protocols (Spark 1.6.3):

- fs (for file distribution).
- akka (essentially job control).

See <https://spark.apache.org/docs/1.6.3/security.html#ssl-configuration> for additional encryption properties and instructions on how to configure each protocol separately.

Since the properties above are automatically set by Pulse, to enable SSL the only setting to be configured is the following:

```
spark.ssl.enabled=true
```



Note that Spark 1.6.3 does not support at-rest encryption of temporary files such as shuffle files and cached data.

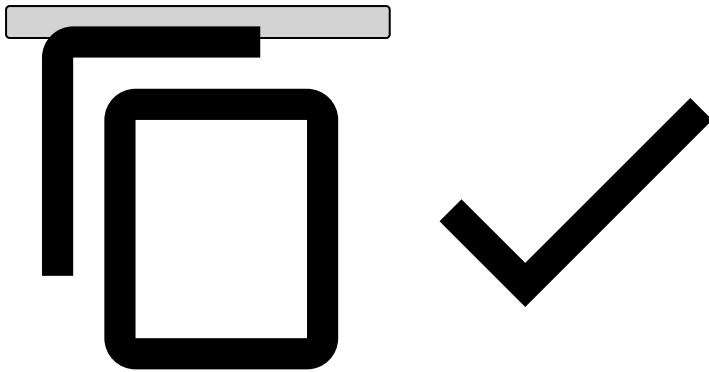


info

The Spark nodes require `openssl` to be installed in order to be able to create the connection between them, even before Pulse is started.

The following needs to be configured on each Spark node (`conf/spark-defaults.conf` file) for SSL to be used between nodes:

```
spark.ssl.enabled true
spark.ssl.keyPassword <Password>
spark.ssl.keyStore /path/to/keystore.jks
spark.ssl.keyStorePassword <KeyStorePassword>
spark.ssl.protocol TLS
spark.ssl.trustStore /path/to/trustedca.jks
spark.ssl.trustStorePassword <TrustStorePassword>
```



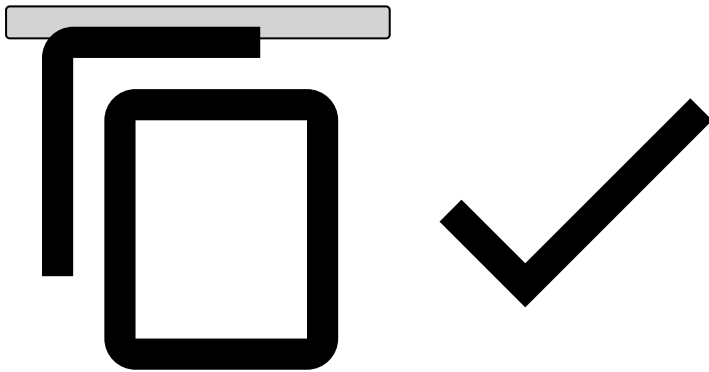
Protocols over TCP with SASL

The RCP protocol is implemented over SASL and must be encrypted because store sensitive properties are passed over it. On a setup with no user authentication (i.e., kerberos), a shared secret string must be configured in Pulse and Spark nodes.

Pulse side

SASL is enabled with the following Pulse properties:

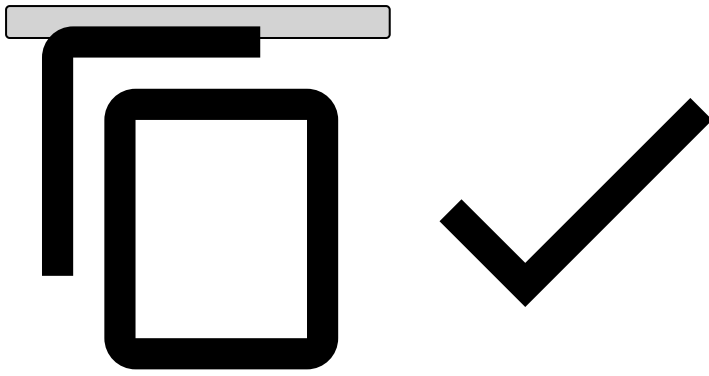
```
pulse.global.services.modules.datascience.spark.forward.spark.authenticate=true
pulse.global.services.modules.datascience.spark.forward.spark.authenticate.secret=<SECRET>
pulse.global.services.modules.datascience.spark.forward.spark.authenticate.enableSaslEncryption=true
pulse.global.services.modules.datascience.spark.forward.spark.network.sasl.serverAlwaysEncrypt=true
```



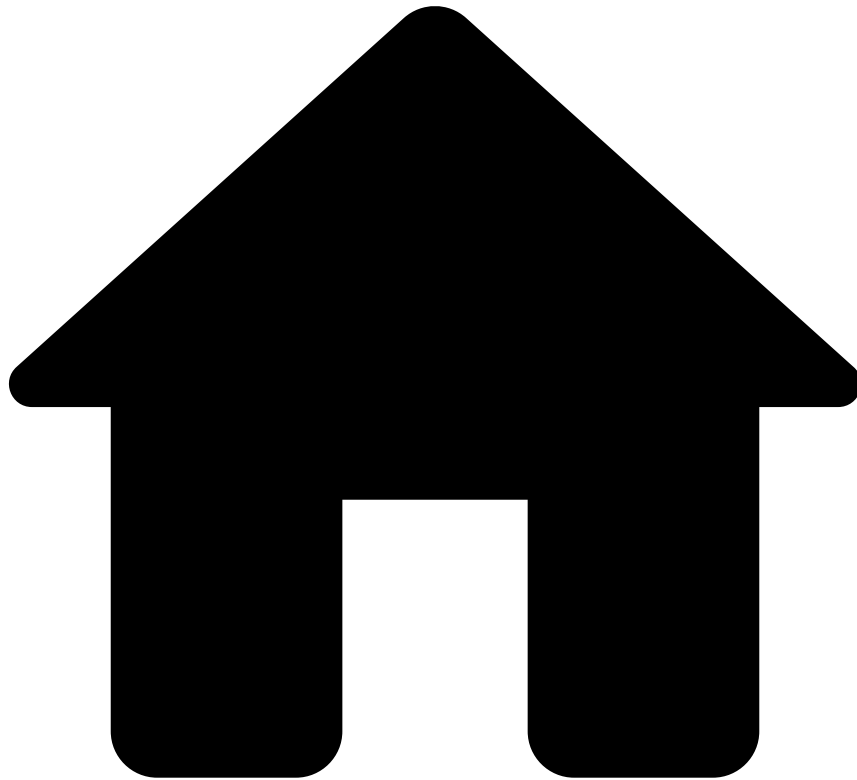
Spark side

The following must also be configured in each Spark container machine, by creating/modifying the conf/spark-defaults.conf file in the Spark nodes:

```
spark.authenticate true
spark.authenticate.secret <SECRET>
spark.authenticate.enableSaslEncryption true
spark.network.sasl.serverAlwaysEncrypt true
```



•



- [Component Encryption](#)
- Creating Certificates

On this page
Was this page helpful? 

Creating Certificates

Pulse includes scripts to help creating keystores and truststores for component encryption.

Both of these files contain certificates that are used for authentication. This guide explains how to use the scripts to configure [Component Encryption with Server Authentication](#).

The scripts generate:

- Server keystores with a private key and a certificate chain with two entries:
 - Server certificate signed by a Certificate Authority (CA).
 - Public certificate issued by the CA (i.e CA certificate).
- Client truststore with the CA certificate.

The certificates are generated both in JKS which is typically required by Java-based components (e.g. application engine, Pulse server) and PEM formats required by some external components (e.g. Rabbit).

Selecting server authentication approach

This section explains component encryption configured with server authentication. For each Feedzai's customers, the authentication approach is selected on a case-by-case basis, and normally fall under one of these options:

- **Self-signed certificates**

Requires generating self-signed certificates and configuring all Pulse components to trust them. This approach assumes that the person generating the certificates has also the role of internal CA. As an example, this could be configured by a Feedzai employee from the delivery team (i.e. referred to as Feedzai in this guide). A typical scenario for this approach is to use it in a test environment.



info

Pulse distributions include a `bin/create-keystore.sh` script to generate self-signed certificates. This could also be used on a configuration where self-signed certificates are used instead of certificate chains. This option is simpler when only one component is secured (i.e. typically jetty).

- **Certificates signed by an internal CA**

Requires receiving certificates signed by an internal CA and configuring all the Pulse components to trust this CA. In this approach, the internal CA is generally handled by a separate department belonging to a Feedzai's customer.

A typical scenario to use this approach is a production environment. Typically, Feedzai's customers have an internal CA that might be handled by a department of security (i.e. referred to as client security team in this guide).

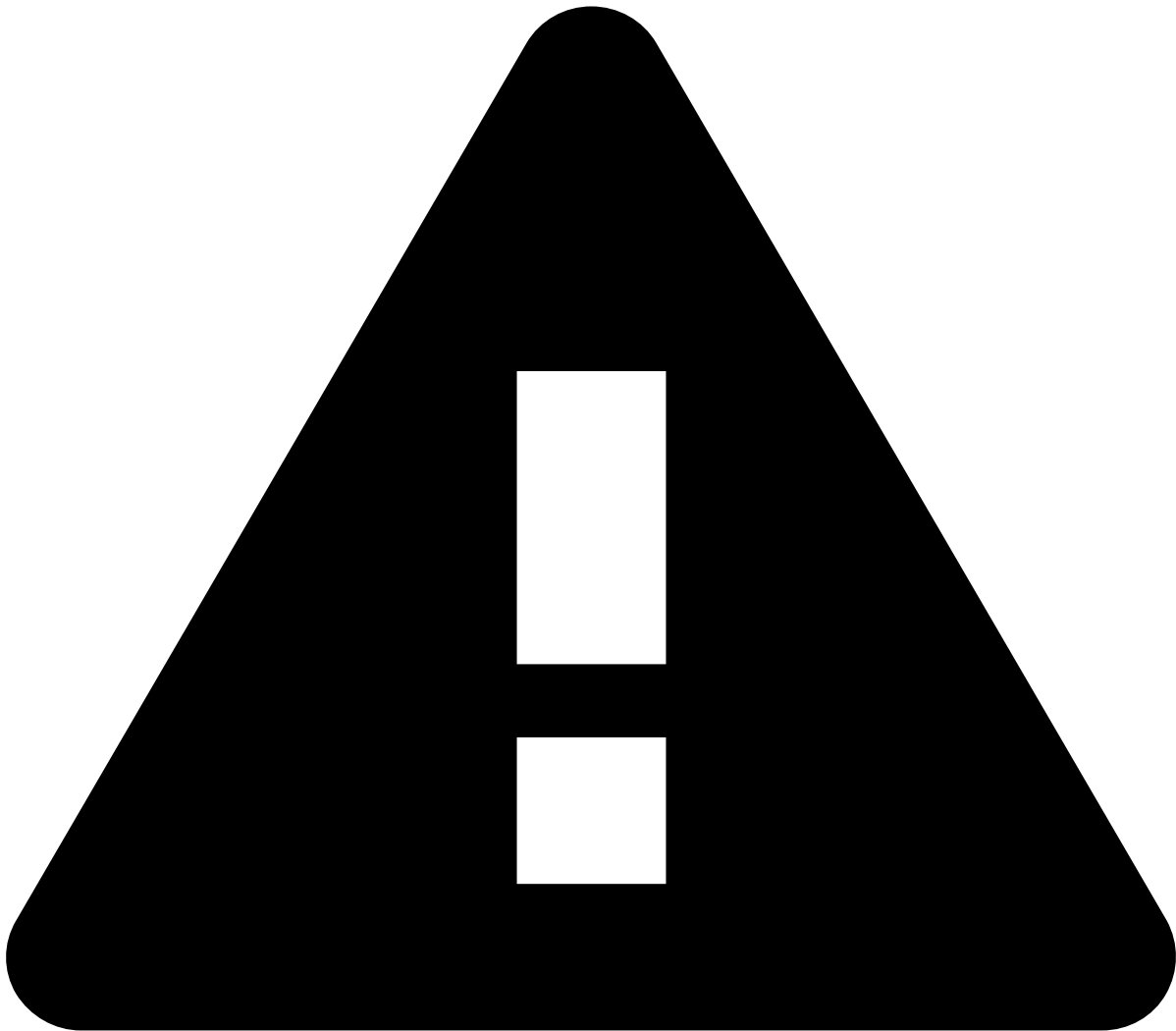
Handling the configuration procedure📖

The configuration procedure can be divided in 3 main steps:

1. Configure a CA that is used to issue a CA certificate as well as to sign the server certificate. Generally, this is only required for the self-signed certificate approach.
2. Configure a truststore with a CA certificate that can be then placed in Pulse components acting as clients.
3. Configure a keystore that can then be placed in Pulse components acting as servers. The keystore contains a certificate chain (server certificate signed by the CA with a digital signature, and the CA certificate) and a private key.

The following diagram illustrate the 3 main steps, including the main differences between a testing and a production environment.

Configuring encryption with server authentication^â



warning

The scripts below generate working files with sensitive information such as private keys and passwords. The files are stored in a directory named "secret" that is created by the scripts, but are not required by Pulse or any component.

The scripts should not be executed in a running pulse machine. The scripts only require the openssl and keytool tools to be available on the machine that is used to execute them.

The scripts are located in the folder *certificates* inside the pulse folder.

The scripts can be open using a text editor and they contain configuration instructions. The recommended order to configure the scripts is the following:

1. Open and execute commands in **create-ca.sh**.

This script must be executed first because it contains the CA Certificate that will be used in commands by the other scripts.

2. Open and execute commands in **create-truststore.sh**.

Generates a truststore in a JKS file with the CA Certificate (file truststore.jks). The password to access this file is stored in the secret/truststore.pwd file.

3. Open and execute commands in **create-keystore.sh**.

Generates a keystore with a private key, a certificate signed by the CA, and the CA certificate (file keystore.jks). The keystore password and the private key password are stored in the secret/keystore.pwd and secret/privatekey.pwd files, respectively.

Alternatively, the execution order of 2 and 3 can be switched.

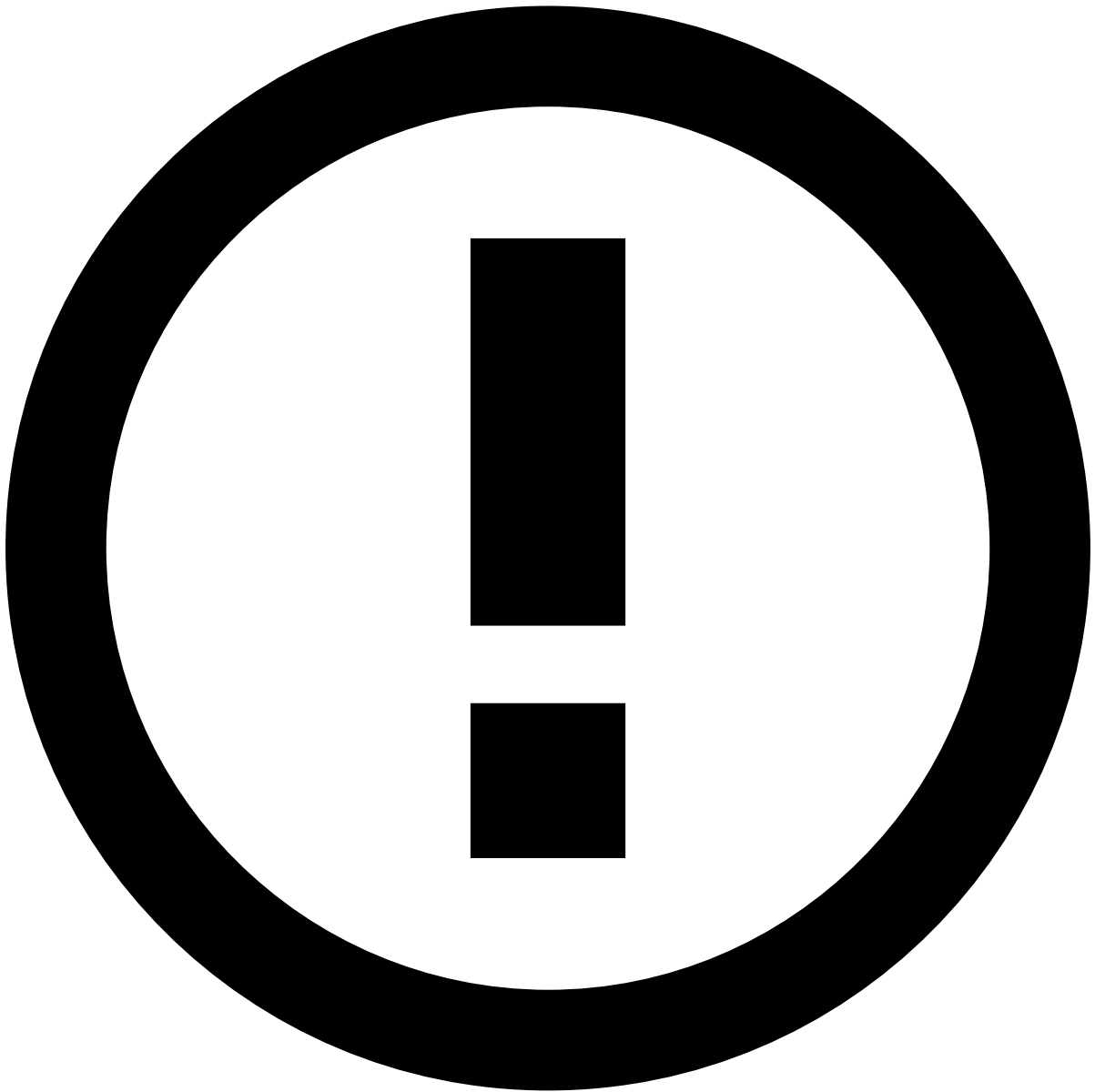
The size and validation of the private keys are set when the scripts are configured. The recommended values are 2048 byte and 10 years, respectively.

Testing environment

This section guides you through the configuration of the scripts in a typical testing environment scenario. In addition to configure truststores and keystores in Pulse components, the Feedzai team plays the role of CA.

To configure component encryption with server authentication execute the following actions:

Run `create-ca.sh` to generate the CAâ

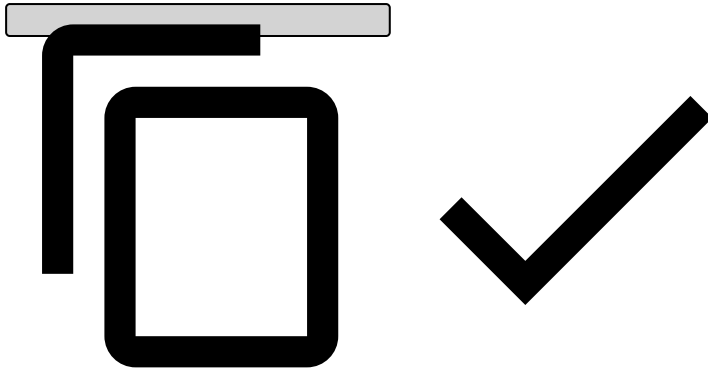


Do you have CA configured?

If you have already a CA configured, skip this step and jump to the subsection [Run create-truststore.sh to generate the truststore](#).

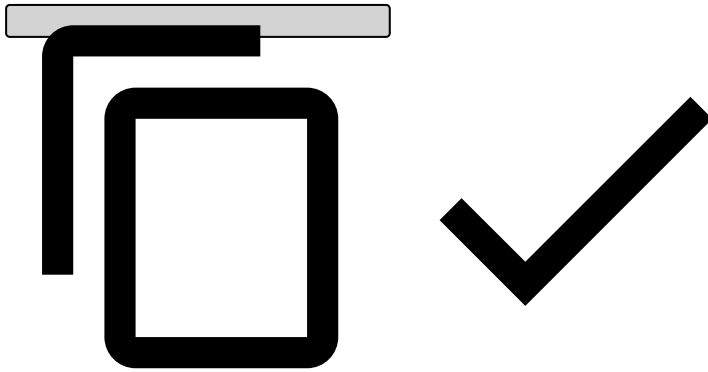
1. Navigate to the `pulse/certificates` folder and open the script `create-ca.sh` with a text editor.
2. Follow the instructions described in the script. The main actions are the following:
 1. Create a directory with the name `secret` (i.e. `certificates/secret`) to store your CA password. If you have already this directory, you can skip this step and continue to the next step.
 2. Generate a file with a CA password using the following command:

```
$ apg -n 1 -m 32 > secret/ca.pwd  
$ CERT_PWD=`cat secret/ca.pwd`
```



3. Generate the CA Certificate file and the CA private key by executing the following (file output `rootCA.crt`):

```
$ openssl req -new -x509 -nodes -newkey rsa:$KEY_SIZE -subj /CN=rootCA/OU=Pulse/O=F  
eedzai/C=PT/ -keyout $CA_KEY_FILE -out $CA_CERT_FILE -days $CERT_DAYS -passin pass:$  
CERT_PWD
```

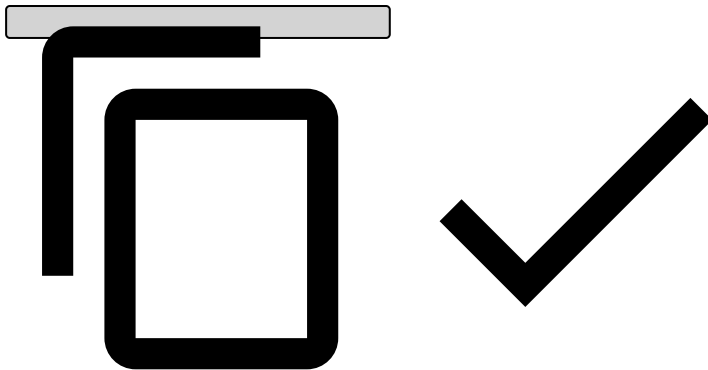


where the variables are replaced by the following (recommended naming):

Variables	Values
<code>\$CA_CERT_FILE</code>	<code>rootCA.crt</code>
<code>\$CA_KEY_FILE</code>	<code>secret/rootCA.key</code>
<code>\$CERT_NAME</code>	<code>rootCA</code>
<code>\$CERT_DAYS</code>	<code>3650</code>
<code>\$KEY_SIZE</code>	<code>2048</code>
<code>\$CERT_PWD</code>	Defined in step 2(b)

As a verification step you can check the content of the CA Certificate with the following command:

```
$ openssl x509 -in $CA_CERT_FILE -text -noout
```

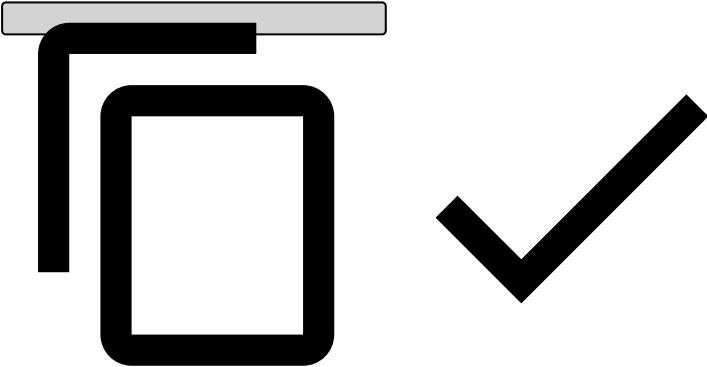


Run create-truststore.sh to generate the truststore

- 1. Navigate to the pulse/certificates folder and open the script create-truststore.sh with a text editor.
- 2. Follow the instructions described in the script. The main actions are the following:

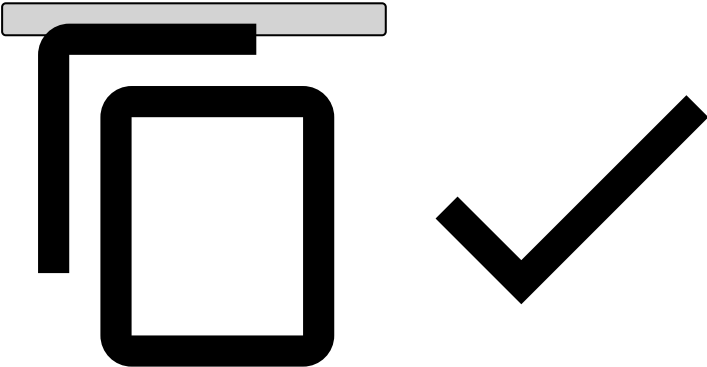
- 1. Generate a file with the truststore password by executing the following:

```
$ apg -n 1 -m 32 > secret/truststore.pwd
$ TRUST_STORE_PWD=$(cat secret/truststore.pwd)
```



- 2. Generate the truststore file in a JKS format using the CA Certificate that is trusted by all components acting as clients (i.e. certificate rootCA.crt generated in the subsection [Run create-ca.sh to generate the CA](#)). To do so, execute the following command (file output trustedca.jks):

```
$ keytool -importcert -keystore $TRUST_STORE_FILE -alias rootCa -file $CA_CERT_FILE
-noprompt -storepass $TRUST_STORE_PWD
```

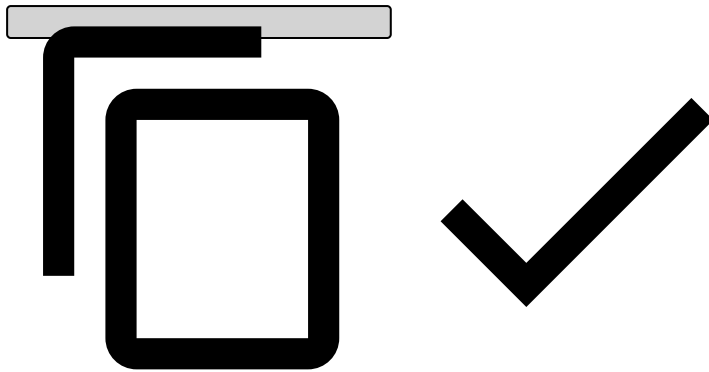


where the variables are replaced by the following (recommended naming):

Variables	Values
\$TRUST_STORE_FILE	trustedca.jks
\$CA_CERT_FILE	rootCA.crt
\$TRUST_STORE_PWD	Defined in step 2(a)

The password defined for the variable \$TRUST_STORE_PWD is requested when opening the file. This password avoid unapproved file editing, for example, changing the certificates within the file. As a verification step, check the file trustedca.jks by executing the following command:

```
$ keytool -list -keystore $TRUST_STORE_FILE -storepass $TRUST_STORE_PWD
```

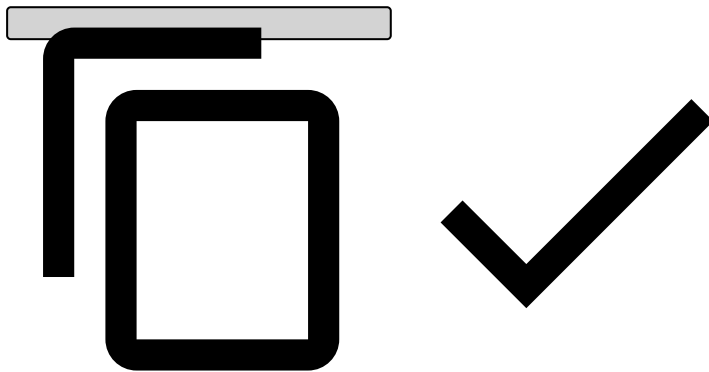



Run `create-keystore.sh` to generate the keystore[🔗](#)

1. Navigate to the `pulse/certificates/` folder and open the script `create-keystore.sh` with a text editor.
2. Follow the instructions described in the script. The main actions are the following:

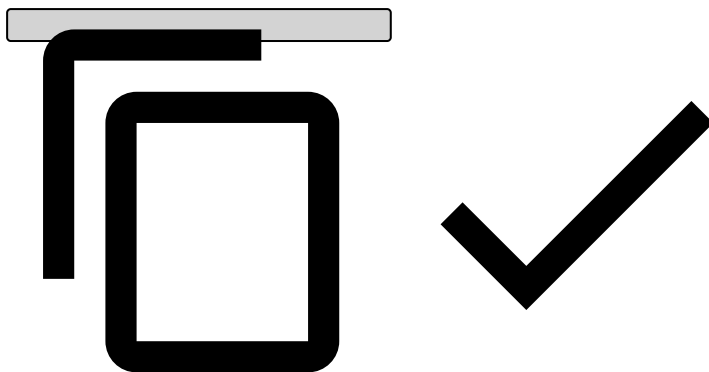
1. Generate a file with the keystore password by executing the following:

```
$ apg -n 1 -m 32 > secret/keystore.pwd  
$ SERVER_STORE_PWD=`cat secret/keystore.pwd`
```



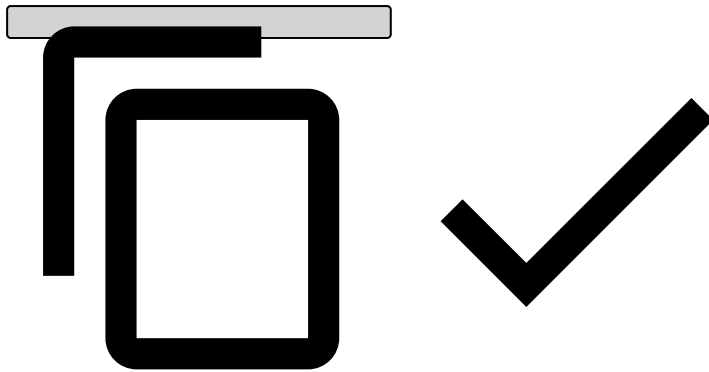
2. Generate a file with the private key password by executing the following:

```
$ apg -n 1 -m 32 > secret/privatekey.pwd  
$ SERVER_KEY_PWD=`cat secret/privatekey.pwd`
```



3. Generate a private key with the following command (file output `key.pem`):

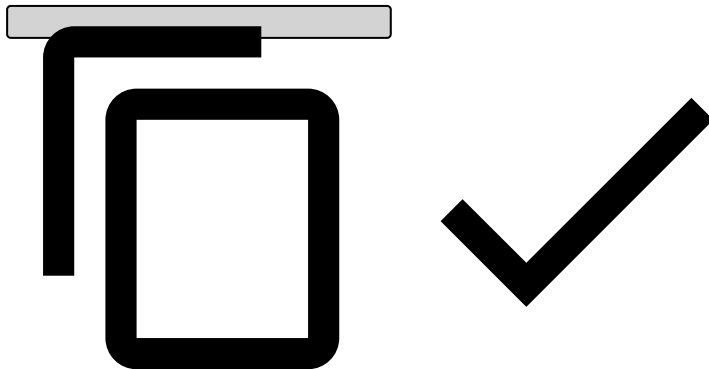
```
$ openssl genrsa -out key.pem $KEY_SIZE
```



where the variable `$KEY_SIZE` can be replaced by 2048.

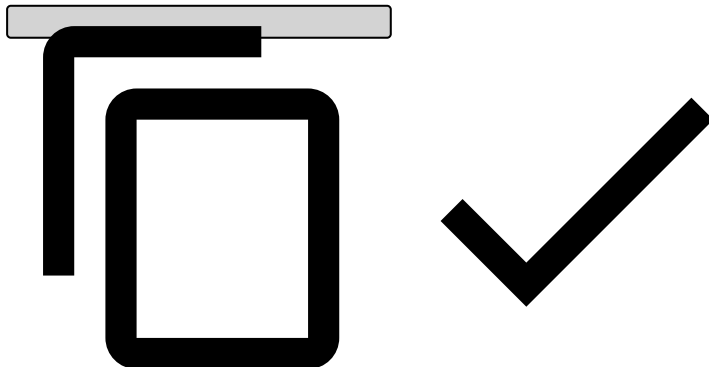
4. Generate the certificate signing request (csr) with the following command (file output `req.pem`):

```
$ openssl req -new -key key.pem -out req.pem -outform PEM -subj /CN=pulse/0=server/ -nodes
```



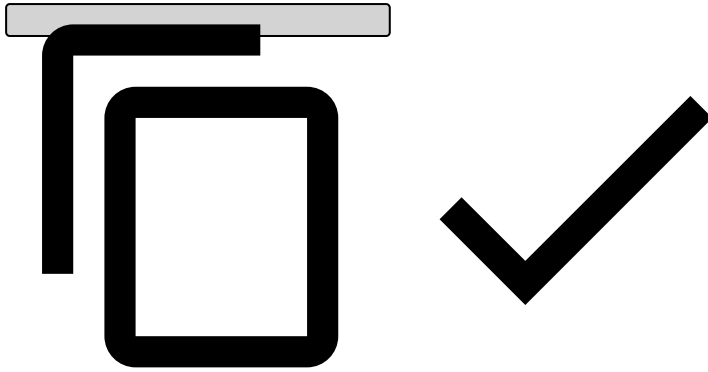
Then run the following command to visualize the certificate signing request and verify that it has no issuer yet:

```
$ openssl req -text -in req.pem
```



5. Performing the CA role, sign the certificate signing request using the certificate `rootCA.crt` and the root private key with the following command (file output `req.signed.pem`):

```
$ openssl x509 -req -CA $CA_CERT_FILE -CAkey $CA_KEY_FILE -in req.pem -out req.signed.pem -days $CERT_DAYS -CAcreateserial -passin pass:$CERT_PWD
```



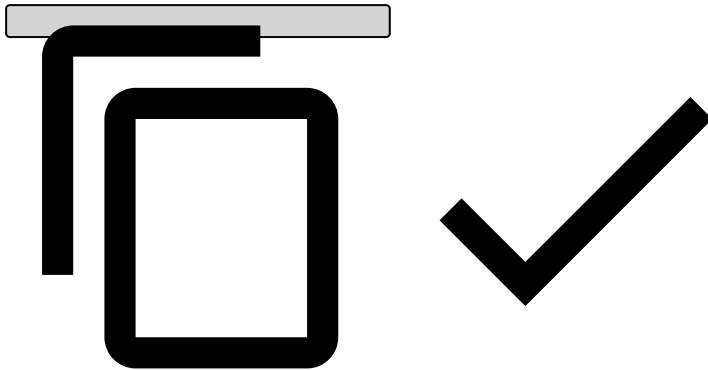
where the variables are replaced by the following (recommended naming):

Variables	Values
<code>\$CA_CERT_FILE</code>	rootCA.crt
<code>\$CA_KEY_FILE</code>	secret/rootCA.key
<code>\$CERT_DAYS</code>	3650
<code>\$CERT_PWD</code>	Choose a password

The password chosen for the variable `$CERT_PWD` is used to generate the digital signature that is added to the certificate.

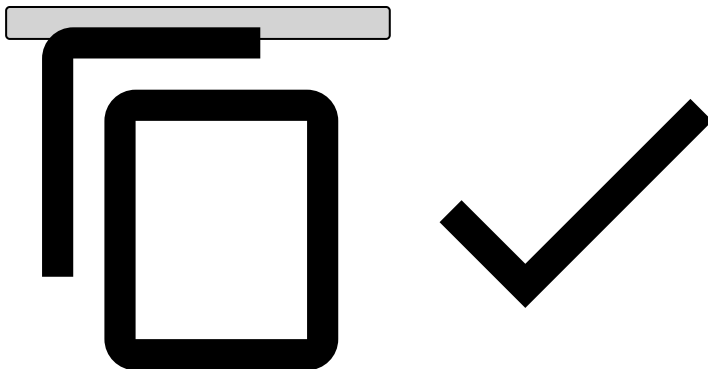
As a verification step, run the following command and verify that the certificate has already an issuer (i.e. rootCA), in opposition to the verification that was done in Step 2(d).

```
$ openssl x509 -in req.signed.pem -text -noout
```



6. Create a package in a pkcs12 format containing the certificate chain (self-signed certificate and the CA certificate) and the server private key with the following command (file output `server.pkcs12`):

```
$ openssl pkcs12 -export -out keystore.pkcs12 -in req.signed.pem -inkey key.pem -certfile $CA_CERT_FILE -passout pass:$SERVER_STORE_PWD
```



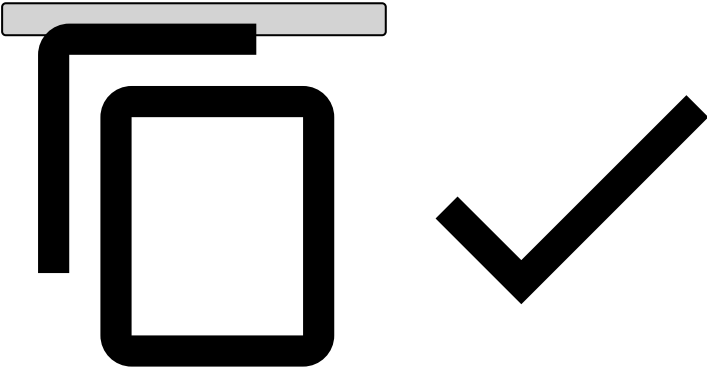
where the variables are replaced by the following (recommended naming):

Variables	Values
\$CA_CERT_FILE	rootCA.crt
\$SERVER_STORE_PWD	Defined in Step 2(a)

The password defined for the variable `$SERVER_STORE_PWD` is used in the next step to convert the file to a JKS format.

7. Convert the pkcs12 file to JKS format using the following command (file output `keystore.jks`):

```
$ keytool -importkeystore -srckeystore keystore.pkcs12 -srcstoretype PKCS12 -srcstorepass $SERVER_STORE_PWD -destkeystore $SERVER_KEY_STORE_FILE -deststoretype JKS -noprompt --deststorepass $SERVER_STORE_PWD -destkeypass $SERVER_KEY_PWD
```

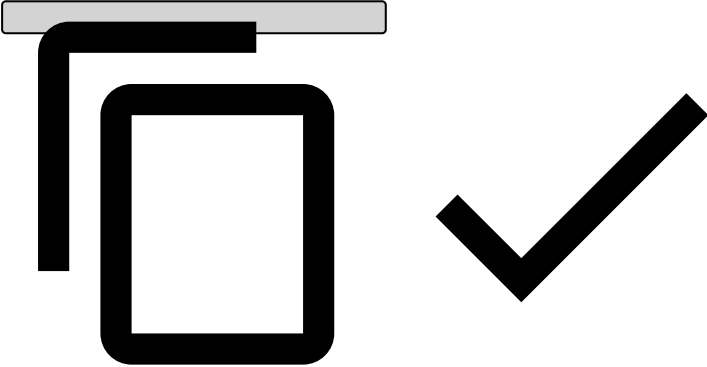


where the variables are replaced by the following (recommended naming):

Variables	Values
\$SERVER_STORE_PWD	The password defined in Step 2(d), i.e. ServerStorePassValue
\$SERVER_KEY_STORE_FILE	keystore.jks
\$SERVER_STORE_PWD	Defined in Step 2(a)
\$SERVER_KEY_PWD	Defined in Step 2(b)

As a verification step run the following command:

```
$ keytool -list -v -keystore $SERVER_KEY_STORE_FILE -storepass $SERVER_STORE_PWD -keypass $SERVER_KEY_PWD
```

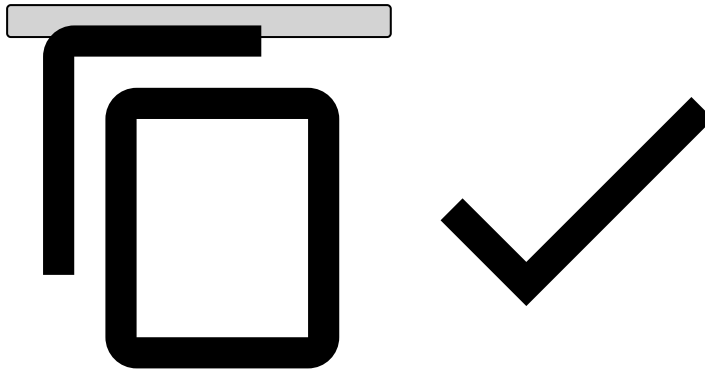


The output gives you information about:

- Number of keystores: "Your keystore contains N entry"
- Self-signed certificate: The first certificate fingerprints
- CA certificate: The second certificate fingerprints

8. Clean up temp files with the following command:

```
$ rm -f key.pem req.pem req.signed.pem
```



Production environment

In a typical production environment, Feedzai has the responsibility to configure truststore and keystore files in Pulse components whereas Feedzai's customer manages the CA.

The procedure to configure the scripts in a production environment is similar to the procedure used in testing environments. Therefore, this section only highlights the differences between both environments and you are referred to section [Testing environment](#) for more details about the full procedure.

Generate a CA

In a scenario of production environment, the script `create-ca.sh` is not needed because the CA is managed by Feedzai's customers.

Run `create-truststore.sh` to generate the truststore

In a testing environment, Feedzai creates a truststore using the CA certificate generated by this team using the script `create-ca.sh`. In a Production environment, Feedzai has to:

1. Contact the client security department that is responsible for managing CA certificates to request a CA certificate file.
2. Generate a truststore using the CA certificate sent by the CA (see image below).

For more information about creating a truststore, check the subsection [Run `create-truststore.sh` to generate the truststore](#) under the section [Testing environment](#).

Run `create-keystore.sh` to generate the keystore

In a testing environment, Feedzai team signs the certificate signing request by performing the role of internal CA.

In a Production environment, Feedzai has to:

1. Generate the certificate signing request.
2. Contact the client security department that is responsible for managing the CA certificates and send the certificate signing request file to be signed by the CA.
3. Create a keystore containing the certificate chain (certificate signed by the CA and the CA certificate) and the server private key (see the following image).

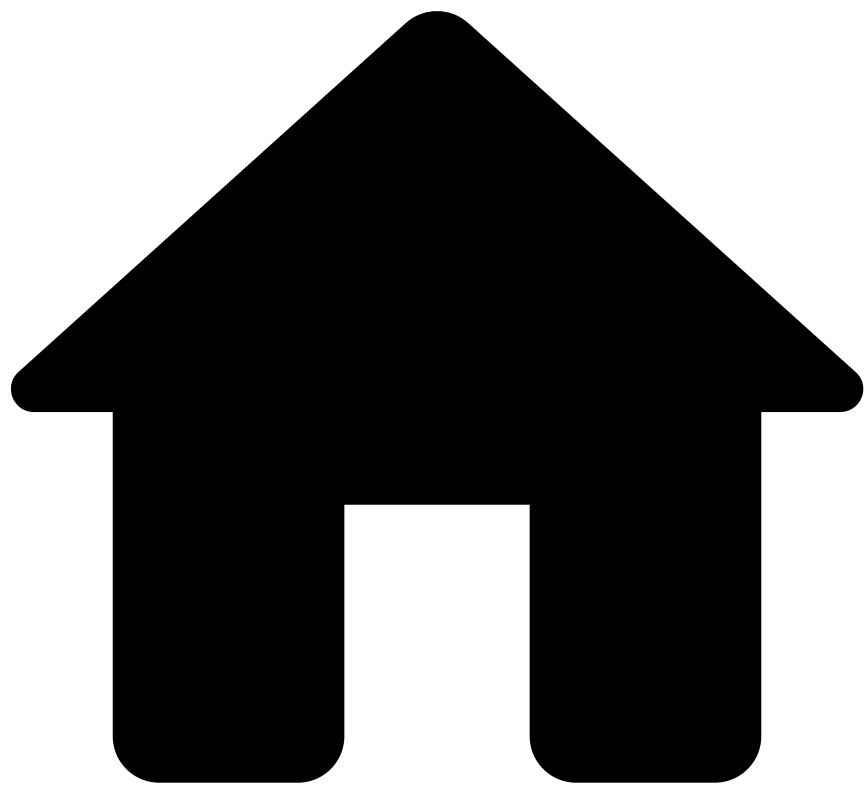
For more information about creating a keystore, check the subsection [Run `create-keystore.sh` to generate the keystore](#) under the section [Testing environment](#).

Learn more

Now that you have basic knowledge about the scripts available to generate keystores and truststores, check the page [Configuration Overview](#) to learn more details about configuration for each component in a Fraud Prevention solution.

You can also learn more about the concept of [Component Encryption](#) in Pulse.

-

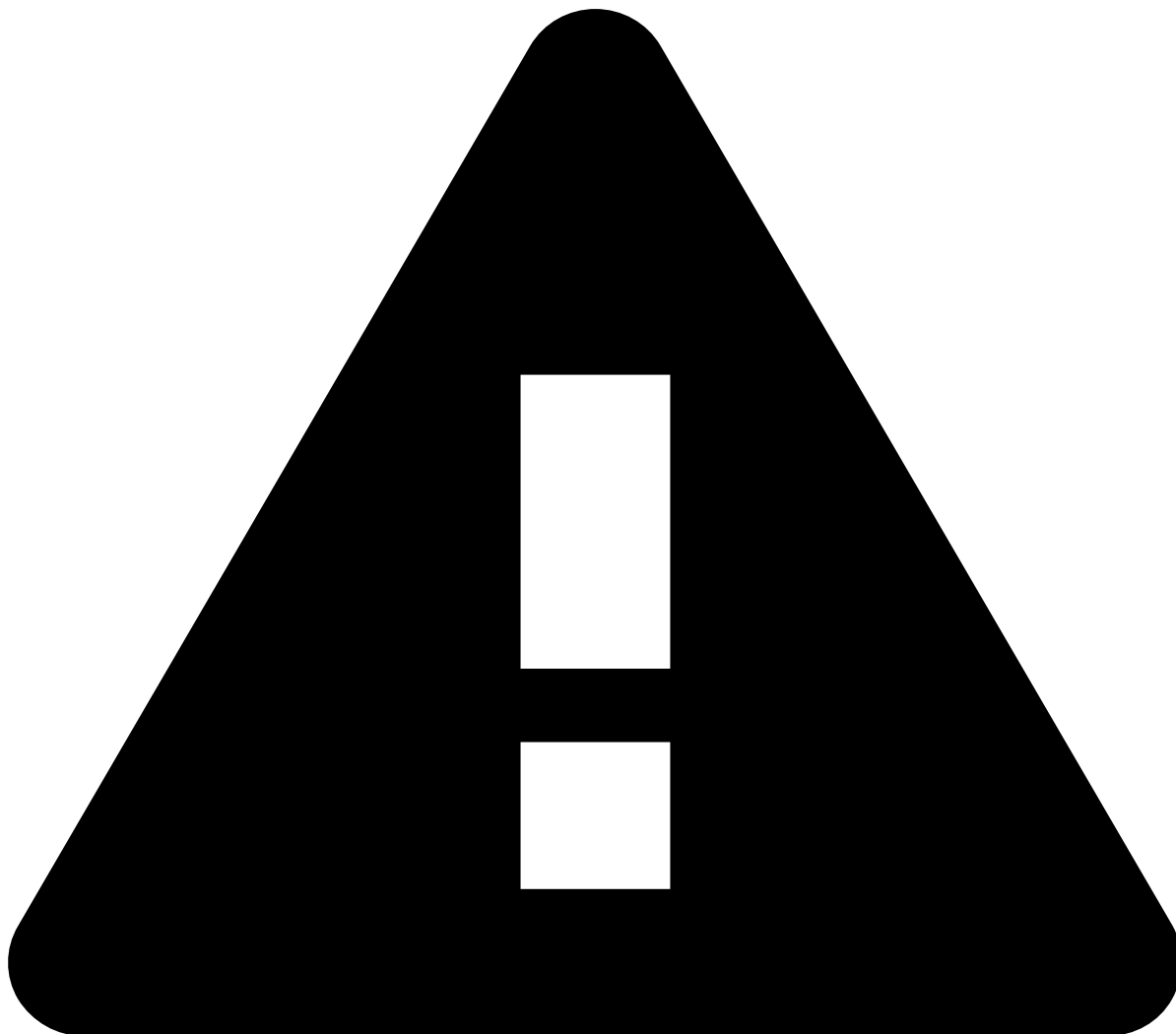


- Security Configuration
- Configuring an SSL Certificate for Pulse

On this page

Was this page helpful? 

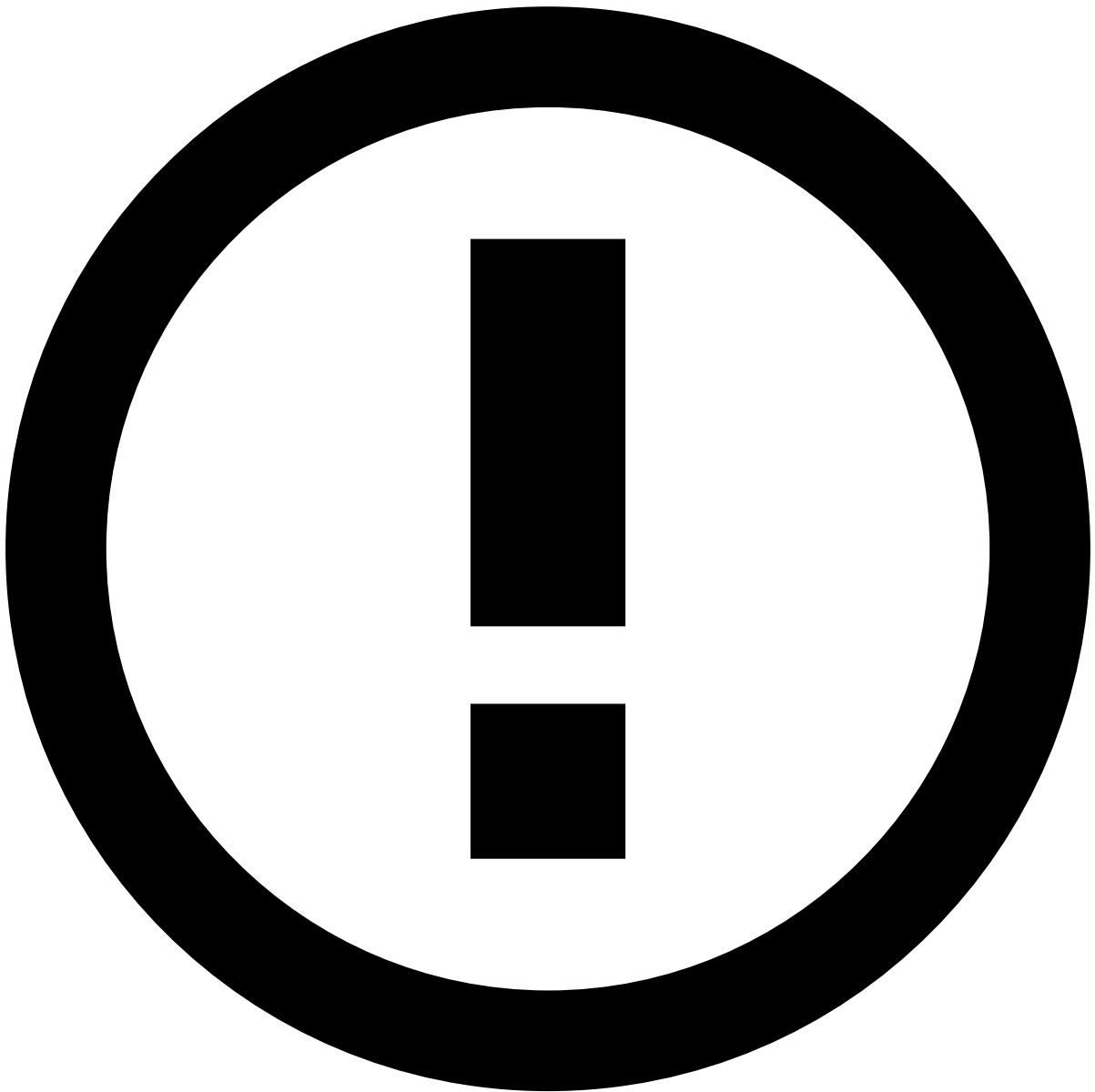
Configuring an SSL Certificate for Pulse



Must change the default certificate

Pulse bundles a self-signed certificate which allows users to connect to it through HTTPS. This can be found in the `conf/engine/jetty/` directory, comprising a keystore and a `jetty.xml` file. This certificate **must not be used in production** environments, as it is insecure.

In this page you can learn how to set up a certificate for SSL in Pulse.



info

This section describes how to create a self-signed certificate and install it in Pulse for use in the Jetty server. See [Component Encryption](#) for instructions on securing all Pulse communications.

Creating your self-signed certificate🔒📄

The **openssl** toolkit is used to generate an **RSA Private Key** and **CSR (Certificate Signing Request)**. It can also be used to generate self-signed certificates which can be used for testing purposes or internal usage.

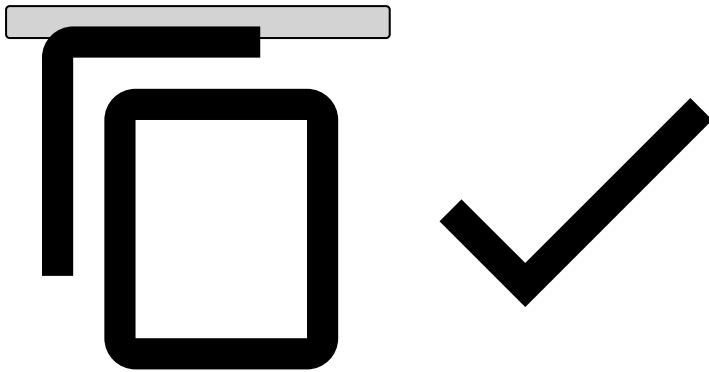
The first step is to create your RSA Private Key. This key is a 1024 bit RSA key which is encrypted using Triple-DES and stored in a PEM format so that it is readable as ASCII text.

```
$ openssl genrsa -des3 -out server.key 1024
```

```
Generating RSA private key, 1024 bit long modulus
```

```
.....++++++  
.....++++++
```

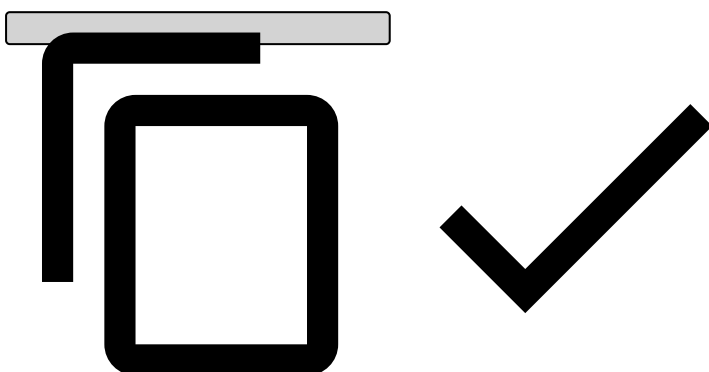
```
e is 65537 (0x10001)
Enter PEM pass phrase:
Verifying password - Enter PEM pass phrase:
```



Once the private key is generated a Certificate Signing Request can be generated. The CSR is then used in one of two ways. Ideally, the CSR will be sent to a Certificate Authority, such as Thawte or Verisign who will verify the identity of the requestor and issue a signed certificate. **The second option is to self-sign the CSR, which will be demonstrated in the next section.**

During the generation of the CSR, you will be prompted for several pieces of information. These are the X.509 attributes of the certificate. One of the prompts will be for "Common Name (e.g., YOUR name)". It is important that this field be filled in with the fully qualified domain name of the server to be protected by SSL. If the website to be protected will be <https://acme.org>, then enter acme.org at this prompt. The command to generate the CSR is as follows:

```
$ openssl req -new -key server.key -out server.csr
Country Name (2 letter code) [XX]:US
State or Province Name (full name) []:California
Locality Name (eg, city) [Default City]:San Francisco
Organization Name (eg, company) [Default Company Ltd]:Acme
Organizational Unit Name (eg, section) []:IT
Common Name (eg, your name or your server's hostname) []:acme.org
Email Address []:[email protected]
Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

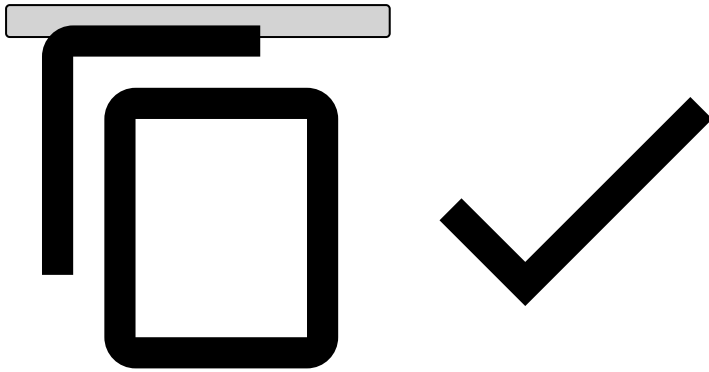


At this point you will need to generate a self-signed certificate because you either don't plan on having your certificate signed by a CA, or you wish to test your new SSL implementation while the CA is signing your certificate. This temporary certificate will generate an error in the client browser to the effect that the signing certificate authority is unknown and not trusted.

To generate a temporary certificate which is good for 365 days, issue the following command:

```
$ openssl x509 -req -days 365 -in server.csr -signkey server.key -out server.crt
Signature ok
subject=/C=US/ST=California/L=San Francisco/O=Acme/OU=IT/CN=acme.org/[email protected]
```

```
Getting Private key
Enter pass phrase for server.key:
```

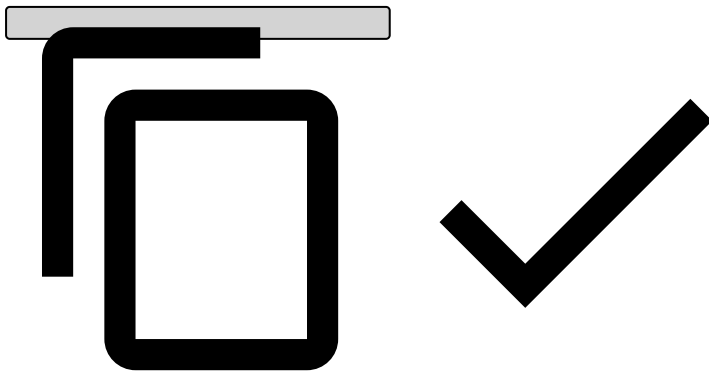


Add your certificate to Pulse [🔗](#)

Once you have your certificate generated (self signed, or signed by an authorized CA), you can import your certificate into the embedded Jetty web server of Pulse.

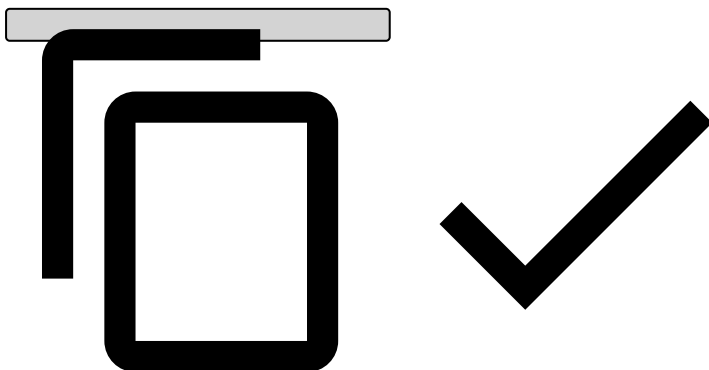
First you need to generate a password to encrypt your keystore:

```
$ apg -n 1 -m 32
PhrirrAyfluwawkofNaljamBytbiaken
```



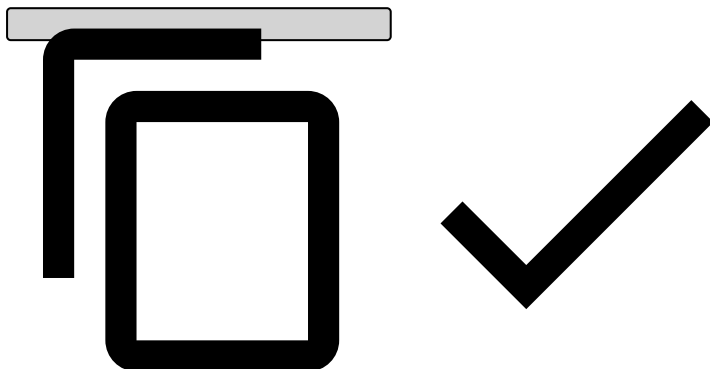
If you have a key and certificate in separate files, you need to combine them into a PKCS12 format file to load into a new keystore. The certificate can be one you generated yourself or one returned from a CA in response to your CSR.

```
$ openssl pkcs12 -inkey server.key -in server.crt -export -out server.pkcs12
Enter pass phrase for server.key: (the password of your certificate)
Enter Export Password: (the new password generated to protect the pkcs12 file, i.e.
PhrirrAyfluwawkofNaljamBytbiaken)
Verifying - Enter Export Password: (the new password generated to protect the pkcs12 file, i.e.
PhrirrAyfluwawkofNaljamBytbiaken)
```



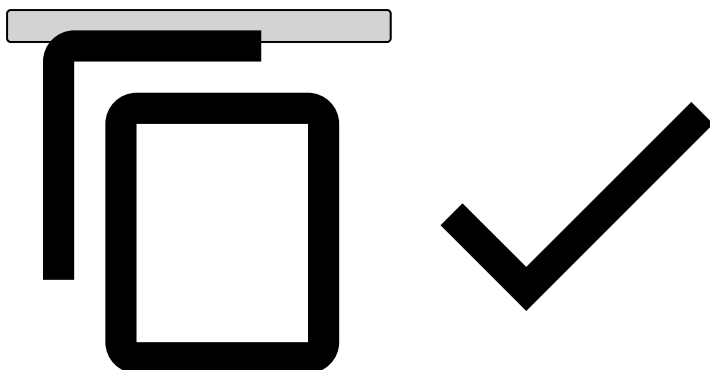
Now lets create the keystore where Pulse will read the certificate:

```
$ $JAVA_HOME/bin/keytool -importkeystore -srckeystore server.pkcs12 -srcstoretype PKCS12 -destkeystore keystore
Enter destination keystore password: (the new password generated to protect the pkcs12 file, i.e. PhrirrAyfluwawkofNaljamBytbiaken)
Re-enter new password: (the new password generated to protect the pkcs12 file, i.e. PhrirrAyfluwawkofNaljamBytbiaken)
Enter source keystore password: (the password of your certificate)
Entry for alias 1 successfully imported.
Import command completed: 1 entries successfully imported, 0 entries failed or cancelled
```



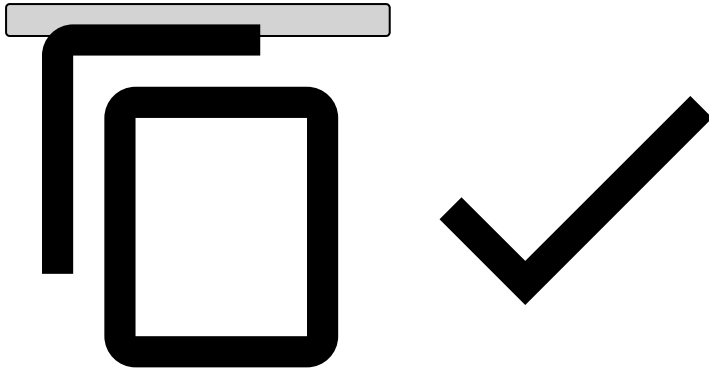
Now copy the resulting file **keystore** to `$PULSE_HOME/conf/engine/jetty/keystore` :

```
$ cp keystore $PULSE_HOME/conf/engine/jetty/keystore
```



Finally edit your `$PULSE_HOME/conf/engine/jetty/jetty.xml` :

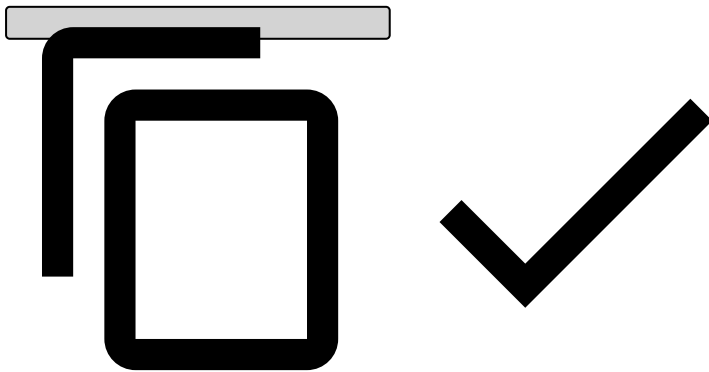
```
<New id="sslContextFactory" class="org.eclipse.jetty.http.ssl.SslContextFactory">
  <Set name="KeyStore">conf/engine/jetty/keystore</Set>
  <Set name="KeyStorePassword">PhrirrAyfluwawkofNaljamBytbiaken</Set>
  <Set name="KeyManagerPassword">PhrirrAyfluwawkofNaljamBytbiaken</Set>
  <Set name="TrustStore">conf/engine/jetty/keystore</Set>
  <Set name="TrustStorePassword">PhrirrAyfluwawkofNaljamBytbiaken</Set>
</New>
```



Enforce SSL Configuration

Define the following line in the pulse.properties file:

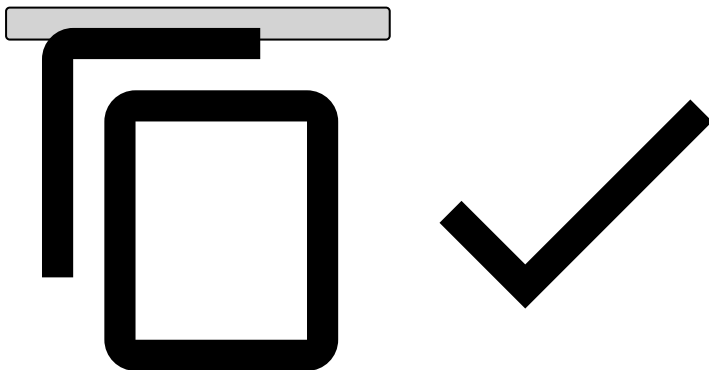
```
pulse.manager.enforcessl=true
```



Configure SSL in FIPS mode

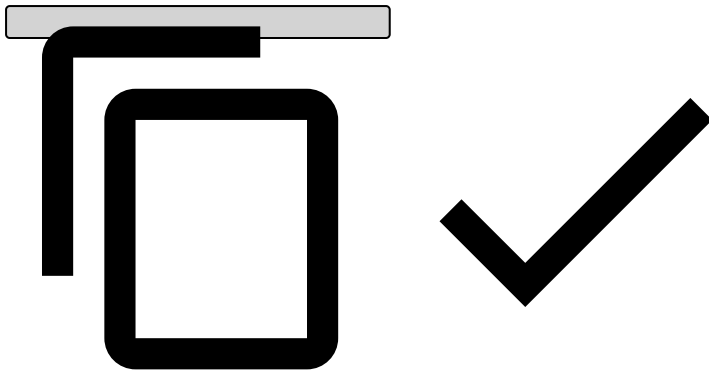
Running Pulse in a FIPS (Federal Information Processing Standards) compliant system, and with a keystore certificate that contains aliases, wildcards, or multiple hosts, may result in the following error when starting the server:

```
java.security.KeyManagementException: FIPS mode: only SunJSSE KeyManagers may be used
```



To configure Pulse to run in FIPS mode, edit the Jetty configuration at `$PULSE_HOME/conf/engine/jetty/jetty.xml`, and change the `sslContextFactory` definition as follows:

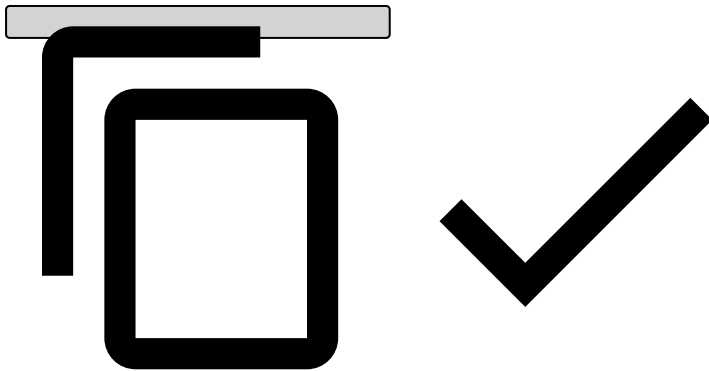
```
- <New id="sslContextFactory" class="org.eclipse.jetty.http.ssl.SslContextFactory">  
+ <New id="sslContextFactory"  
class="com.feedzai.pulse.service.webapps.jetty.FIPSSslContextFactory">
```



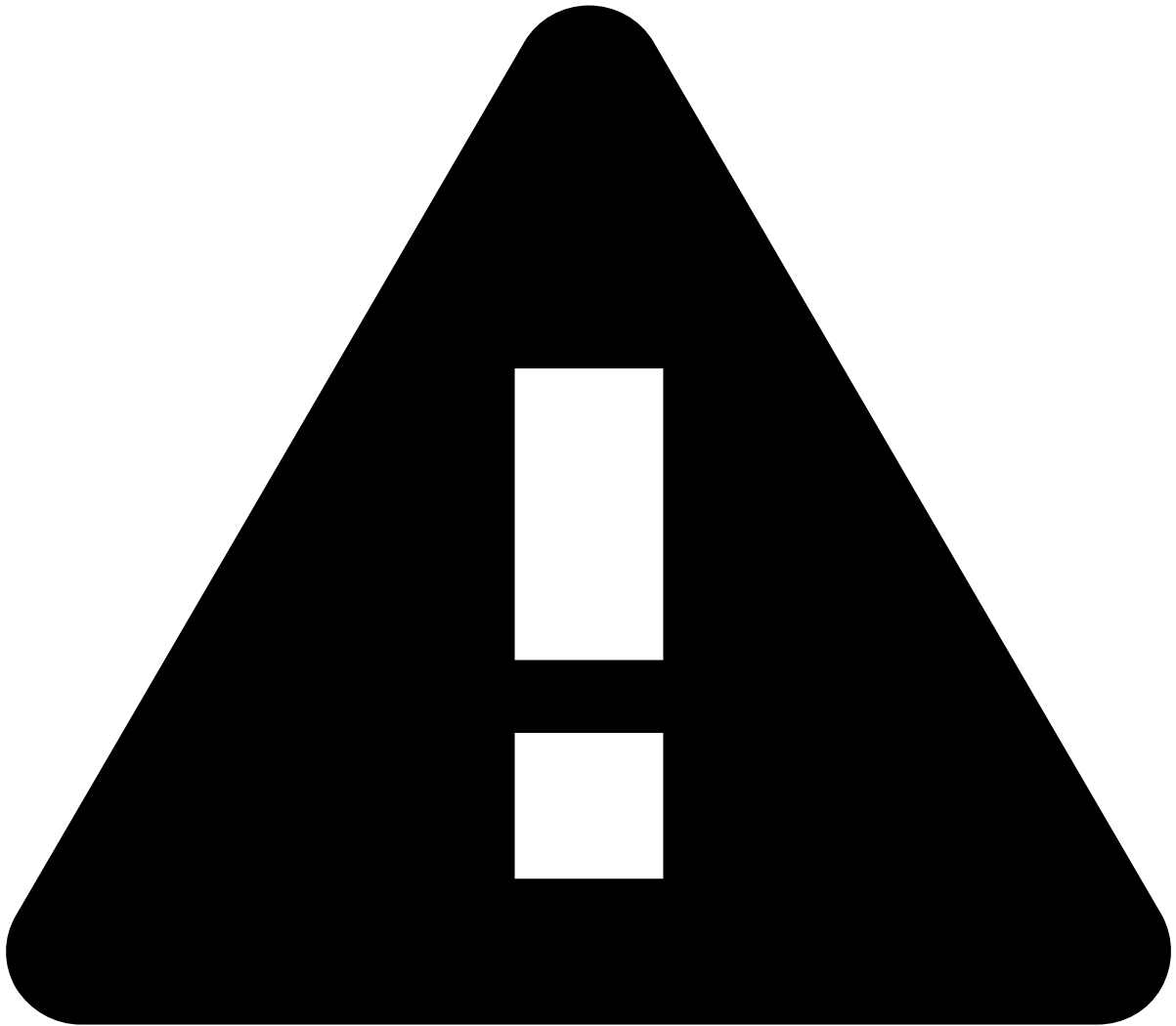
Depending on the used database, a similar error may prevent the connection from Pulse to the database. This has been observed when trying to connect to PostgreSQL using the PostgreSQL JDBC Driver. In such cases, it may be required to explicitly configure the driver to use the appropriate SSL factory.

For PostgreSQL, this can be achieved by configuring the JDBC connection URL to explicitly specify the SSL factory to be used:

```
pulse.global.database.jdbc=jdbc:postgresql://<HOST:PORT>/<DATABASE>?  
ssl=true&sslfactory=org.postgresql.ssl.DefaultJavaSSLFactory
```



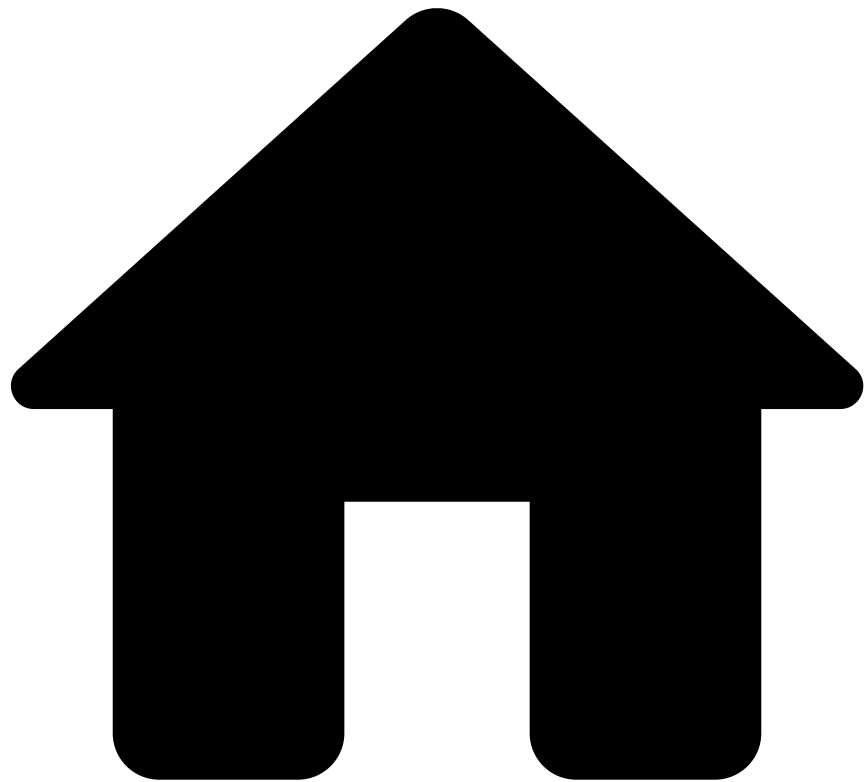
The above configuration will use the default Java SSL factory provided by the JVM, rather than the PostgreSQL JDBC driver's custom SSL implementation. This ensures that the database connection uses the FIPS-compliant security providers from the Java runtime environment.



warning

This configuration disables the certificate aliases and SNI (Server Name Indication). As of now, there are no alternatives to run Pulse with FIPS and certificate aliases, or SNI.

•



- Security Configuration
- Configuring Kerberos for Data Science Resources

On this page

Was this page helpful? 

Configuring Kerberos for Data Science Resources

Pulse stores data science resources on the filesystem. To work with those resources in Pulse, users need to have access to the respective files. You can use group properties to grant access.

This page describes how to use groups and group properties to grant a group access to the following data science resources:

- Data sources (created/used by plan executions, model evaluations, rule evaluations, sandbox evaluations)
- Statistic file of data source analysis
- Model binaries from model training and hyperparameter tuning
- Spark archive file (initialization of a leader app jvm process)
- Spark-libs (initialization of a leader app jvm process)

Hierarchical groups and access rights

When Pulse users try to access a data science resource, Pulse needs to access the respective files using Kerberos credentials. These credentials can be defined either in the Group Properties field of a group, or in the *pulse.properties* file.

Pulse doesn't only consider the groups that are explicitly assigned to the user, but also their parent groups in the group hierarchy.

If none of the user's groups or their parent groups has any Kerberos credentials defined for the given resource, Pulse tries to fall back on credentials defined in the *pulse.properties* file, if there are any.

Group properties

To configure a group to use specific Kerberos credentials, add the following key-value pairs in the **Group Properties** field of the group:

Key	Value
defaultPath	The path to the folder of the tenant on the HDFS.
kerberos.keytabPath	The path to the keytab file of the tenant.
kerberos.principal	The principal authentication to be used for Kerberos. Normally, it has the following format: <code>username/resource@REALM</code> .

Using groups to control visibility

The members of the groups that are set up this way can control who can see resources stored in the group's directory.

To set the resource visible only to that specific group, complete the following steps. Note that the steps presume that the user has the membership of the group(s) that they select in step 3.

1. Open the resource.
2. Select the Advanced Options button, and select **Specific Groups**. Select the group from the list of groups.
3. In the list of permissions, select **Read / Update / Delete** according to your choice. A check mark appears next to them when selected.
4. **Save** the resource.

If the visibility of a data science resource is public (i.e. not limited to a specific group), then Pulse will use the Kerberos credentials defined in the *pulse.properties* file, if there are any.

Example

Let's consider an example where you want to configure the following Pulse groups to access HDFS directories:

- *TeamA* and *TeamB* are two groups that should have access only to their own respective directories, plus a directory with common resources (containing Spark archives and Spark libs).
- The members of the *landlords* group should have access to all directories. They also have a directory for their own resources that should not be accessed by anyone else.

To configure this, the following needs to be set up:

1. HDFS directories:

Directory	Description	Permissions	Owner	Group
/user/ pulse_common/	The common directory that stores files common to all teams, for example spark-libs, spark-archive, etc. The user of each team must belong to its group to be able to access this directory.	770	pulse_common	pulse_common

Directory	Description	Permissions	Owner	Group
/user/ pulse_landlord/	This directory is specific to the landlord. It is used to store files with sensitive data of all teams.	770	pulse_landlord	pulse_landlord
/user/teamA/	This directory is specific to TeamA. It stores the data science resources of this team.	770	pulse_teamA	pulse_teamA
/user/teamB/	This directory is specific to TeamB. It stores the data science resources of this team.	770	pulse_teamB	pulse_teamB

2. HDFS users:

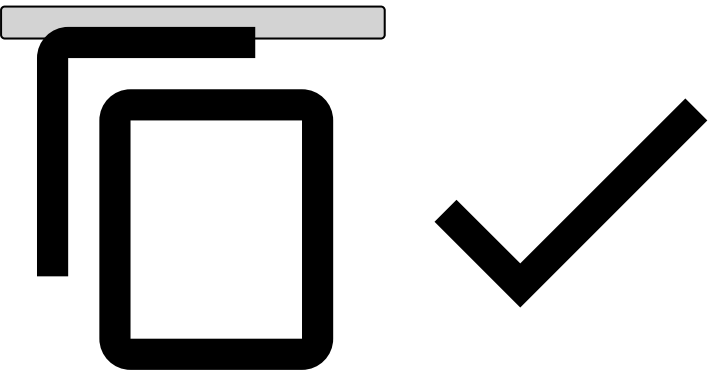
Each of these users must be configured with Kerberos authentication using a keytab file.

User name	Description	Member of the group(s)	Maps to Pulse group
pulse_common	The HDFS user through which Pulse users will access common files.	pulse_common	all groups in Pulse
pulse_landlord	The HDFS user through which the landlords will access landlord-specific files.	pulse_landlord, pulse_common, pulse_teamA, pulse_teamB	landlords
pulse_teamA	The HDFS user through which members of TeamA will access their own files.	pulse_teamA, pulse_common	TeamA
pulse_teamB	The HDFS user through which members of TeamB will access their own files.	pulse_teamB, pulse_common	TeamB

3. Pulse properties in the *pulse.properties* file:

Replace the placeholders (marked with `<...>`) with the respective values. These properties allow Pulse users to access common files.

```
pulse.global.services.modules.distributed.jobs.kerberos.enabled=true
pulse.global.services.modules.distributed.jobs.kerberos.principal=<principal_of_pulse_common>
pulse.global.services.modules.distributed.jobs.kerberos.keytabPath=<keytab_path_of_pulse_common>
pulse.global.services.modules.distributed.jobs.spark.master=yarn-client
pulse.global.services.modules.distributed.clusterDirToReceive=hdfs:///user/pulse_common/spark-libs
```



4. Pulse groups and users:

In Pulse, create three groups:

- landlords
- TeamA
- TeamB

For each of these groups, add the following key-value pairs in the **Group Properties** field:

Key	Value
defaultPath	The path to the respective HDFS directory of the group.
kerberos.keytabPath	The path to the group's keytab file.
kerberos.principal	The principal authentication to be used for Kerberos. Normally, it has the following format: username/resource@REALM.

Add the users to their respective group. Landlord users must be added to the teams' groups as well.

After configuring the above, users must set the resources visible only to the group. If the visibility is not restricted to the respective group, Pulse will use the credentials that are defined in the *pulse.properties* file to access the resource.

•



- [Single Sign-On](#)
- Configuring AD FS

On this page

Was this page helpful? 

Configuring AD FS

AD FS is one of the available services that can act as an IdP enabling SPs to communicate with it using the SAML 2.0 standard. This service is responsible to provide Single Sign-On (SSO) of existing users as well as to enable user provisioning. For certain actions, the AD FS can be configured in different ways. Full description of the AD FS configuration is beyond the scope of this guide, however, basic configurations that enable SSO in Pulse are described below as examples. These examples include:

- Obtaining Pulse IdP properties when using AD FS.
- Configuring Pulse in AD FS as a Relying Party Trust (i.e. SP).
- Configuring users in AD FS that are mapped to Pulse users.
- Configuring claim rules in AD FS that will be part of the SAML assertion to communicate user information to Pulse such as language, groups, roles, or tenants.

Obtaining Pulse IdP properties^â

When the IdP used is the AD FS, all the Pulse's IdP related properties (prefix: `pulse.manager.services.core.security.authplugin.filter.samlso.idp`) can be obtained from the AD FS federation metadata file. This federation metadata is available using the following URL:

```
https://<ADFS_DOMAIN>/FederationMetadata/2007-06/FederationMetadata.xml
```

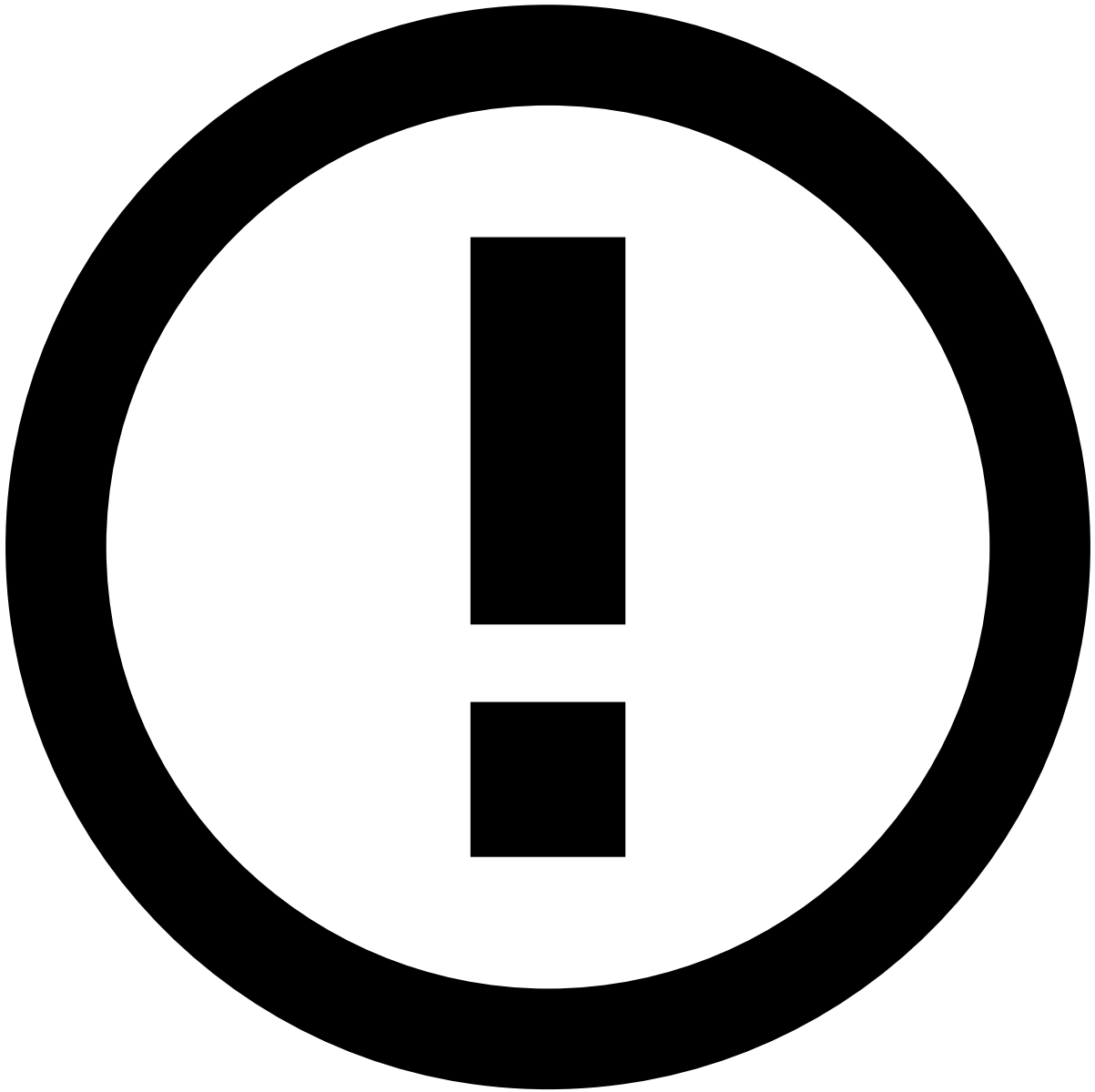
The federation metadata file contains various information about the AD FS instance. To know how to map the information contained on this XML to the Pulse properties, please check: [Configuring SSO using a generic IdP](#).

Add a relying party trust^â

This section describes how to configure a Pulse instance as a relying party trust (i.e. SP).

The SP can be configured manually or automatically by importing configuration data generated by metadata. The second method is described in this section as an example.

At this point you should have AD FS installed, and the properties that enable SSO configured on Pulse.



AD FS configurations

Note that the SP configuration based on importing configuration data from metadata described in this section is not necessary the preferred method for all the production systems. This method is illustrated in this section as an example. Refer to the [AD FS Documentation](#) for full configuration details.

1. Open the Pulse URL that provides metadata information for the specific instance, e.g. `https://<BASE_PULSE_ADDRESS>/pulseviews/api/sessions/samlssso/metadata` , and export the file as XML.
2. Open the **Server Manager AD FS**.
3. In the Server Manager AD FS, open the **Tools** bar and select the option **AD FS Management**.
4. In the AD FS window, expand the folder **Trust Relationships**.
5. Open the folder **Relying Party Trusts**.
6. In the **Actions** panel, choose the option **Add Relying Party Trust...**
7. Click start, and choose the option **Import data about the relying party from a file**.

8. Open the XML file generated in Step 1 and click **Next** to create the SP. Note that at the end of this step you can open the IdP sign in webpage and check that the new SP is already available.

Add new users

This section describes how to configure a user in the active directory that will be mapped to a Pulse user with two roles, i.e. *Pulse Admin* and *Pulse Reader*. These two roles are used in this section as examples and the configuration to map user roles is optional. However, if users are configured to map user roles, the roles have to exist in Pulse.

1. Open the **Server Manager AD FS**.
2. Open the **Tools** bar and select the option **Active Directory Users and Computers**, which will open the following window:
3. In the window **Active Directory Users and Computers**, select the option **Create a new user in the current container**.
4. Add the **First name** *SsoUserTest* and the **User logon name** [\[email protected\]](#) and click **Next >**.
5. Set a password for this user.
6. Open the **SsoUserTest** user and select the option **Member Of**.
7. Select the option **Add** to add new groups: *Pulse Admin*, *Pulse Reader*.

Create claim rules

This section describes how to configure claim rules to map Pulse user name and two user roles. The AD FS claims rules are sent as attributes within SAML assertions. They are essential for Pulse to perform user provisioning correctly.

The configuration of the AD FS to map user roles is optional along with languages, groups, and tenants. Therefore, the configuration described in this section is only an example of a configuration option.

1. Open the **Server Manager AD FS**.
2. Open the **Tools** bar and select the option **AD FS Management**.
3. In the AD FS window, expand the folder **Trust Relationships**.
4. Open the folder **Relying Party Trusts**.
5. Choose an item from the list of relying party trusts that you wish to establish the claims, right click on the item and select **Edit Claim Rules...**
6. Select **Add Rule**, and select the claim rule template *Send LDAP Attributes as Claims*.
7. Provide the following information:
 1. Claim rule name: *Name ID*.
 2. Attribute store: *Active Directory*.
 3. LDAP Attribute: *User-Principal-Name*.
 4. Outgoing Claim Type: *E-Mail Address*.
8. Add another rule and select the claim rule template *Transform an Incoming Claim*.
9. Provide the following information:
 1. Claim rule name: *Email claim*.
 2. Incoming claim type: *E-Mail Address*.
 3. Outgoing claim type: *Name ID*.
 4. Outgoing name ID format: *Email*.
 5. Select the option **Pass through all claim values**.
10. (Optionally) Add another rule and select the claim rule template *Send Group Membership as a Claim*.



AD FS configurations

Note that the choice of the claim rule template described in this section is not necessary the preferred option for all the production systems. This option is illustrated in this section as an example. Another option is to extract the claims from the LDAP attributes. Refer to the [AD FS Documentation](#) for full details about configuring claim rules.

11. Provide the following optional information:

1. Claim rule name: e.g. *RoleAdminCheck*.
2. Outgoing claim type: e.g. *User.roles*. Note that this property is used by Pulse by default to map to a user role. For example, the properties *User.language*, *User.groups* and *User.tenants* would map to languages, groups and tenants.
3. Outgoing claim value: e.g. *admin*. Note that this value maps to a user role in Pulse, and thus, it has to match the name of the role configured in Pulse. For example, to map to a tenant, the claim value has to match the tenant name configured in Pulse.

12. (Optionally) Repeat Steps 10 and 11 to add a claim rule to map to the Pulse role *pulseviews*.

13. Repeat Steps 5 to 12 to create claim rules for other SPs.

•



- [Single Sign-On](#)
- Configuring Keycloak

On this page

Was this page helpful? 

Configuring Keycloak

This page describes how to integrate Keycloak as the Identity Provider (IdP) into the Pulse environments. The integration is tested for Keycloak version 16.1.1. Take this into consideration when following this guide and the official Keycloak documentation.

Feedzai uses Keycloak to provide single sign-on (SSO) across its products: Pulse, Case-Manager (CM) and SAR Manager (SARM).

In Feedzai deployments, Keycloak is not used for user management, only for session management and the authentication protocol layer. Instead, users are authenticated and fetched from a federated Identity Provider.

The authentication request/response calls follow the standard [SAML](#) (for Pulse and CM) and [OIDC Authorization Code Flow](#) (for Pulse and SARM).

The following diagram shows how Feedzai products connect to Keycloak and where relevant information for single sign-on is stored.

Installing Keycloak¹

Refer to the official Keycloak guide for installation instructions. In Feedzai Cloud, Keycloak is configured in Standalone Clustered Mode for high-availability and with frequent database backups for an added layer of data loss protection. Consider these options for on-premise deployments.

After successfully starting Keycloak, create a new realm as described [here](#), from now on referred to as REALM_NAME.

Federating an Identity Provider²

For customers that have an IdP available, Keycloak should be configured to federate user management and authentication to that IdP. Keycloak has out-of-the-box support for configuring federation using Security Assertion Markup Language (SAML) and OpenID Connect (OIDC). Multiple Social login providers can also be configured. Refer to [chapter 12](#) of the Keycloak Server Administration Guide for specific configurations.

Keycloak can be configured to transparently redirect to the federated IdP by setting an Action in the Browser Flow for the Identity Provider Redirector as explained in [this section](#) of the Keycloak documentation.

Connecting Feedzai products to Keycloak³

In Feedzai Cloud, products are configured out-of-the-box for a seamless single sign-on experience. For on-premise deployments each product must be individually configured to connect to Keycloak.



info

Note that all URLs mentioned in this document are the ones visible externally to the cluster, not internal ones. These authentication protocols work at the browser level.

The settings below include `{VARIABLES}` and `<manual instructions>` that need to be **replaced with the actual values**.

Pulse real-time scoring[ⓘ]

Keycloak Configuration[ⓘ]

To configure Keycloak to accept Pulse connections, complete the following steps:

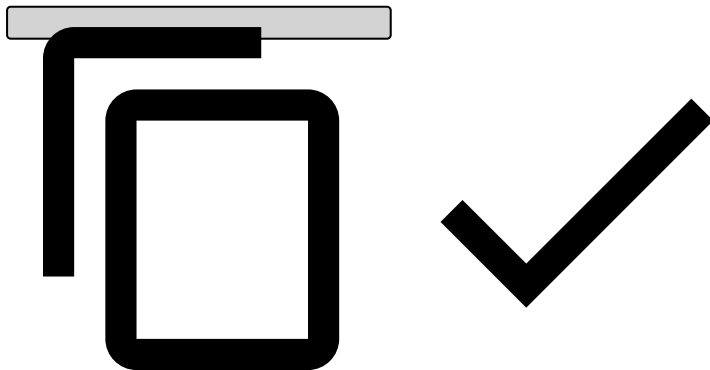
1. Create a new SAML client where the Client ID is the HTTPS URL of the real-time scoring Pulse environment (from now on referred as `PULSE_SCORING_URL`). Before saving, do the following changes to the default configuration:
 1. Turn on Sign Assertions

2. Set SAML Signature Key Name to `NONE`
 3. Set Valid Redirect URIs to `{PULSE_SCORING_URL}/*`
 4. Set Base URL to `{PULSE_SCORING_URL}`
 5. Set Logout Service Redirect Binding URL to `{PULSE_SCORING_URL}/pulseviews/api/sessions/samlso/logout`
2. Save the client and go to the Mappers tab to configure how the user roles are propagated.
 3. Create a new mapper of type User Attribute:
 1. Set SAML Attribute Name to `User.roles`.
 2. Set SAML Attribute NameFormat to `Basic`.
 3. Set User Attribute to `roles`.
 4. Save the mapper.

Pulse properties

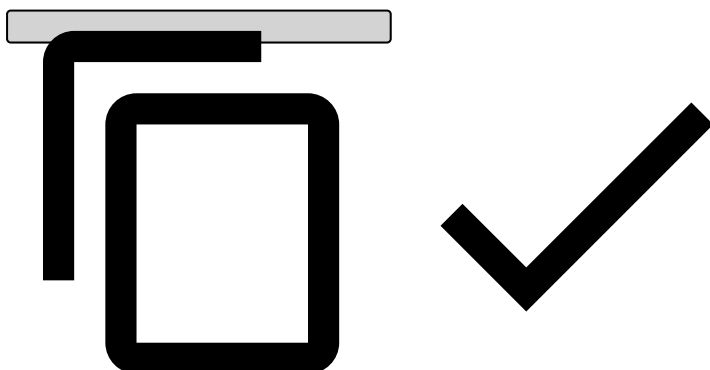
Create the following properties to configure Pulse to connect to Keycloak for SSO.

```
pulse.manager.services.core.security.authplugin.filter.samlso.enabled=true
pulse.manager.services.core.security.authplugin.filter.samlso.idp.entityId=https://
{KEYCLOAK_HOSTNAME}/auth/realms/{REALM_NAME}
pulse.manager.services.core.security.authplugin.filter.samlso.idp.sso=https://
{KEYCLOAK_HOSTNAME}/auth/realms/{REALM_NAME}/protocol/saml
pulse.manager.services.core.security.authplugin.filter.samlso.idp.slo=https://
{KEYCLOAK_HOSTNAME}/auth/realms/{REALM_NAME}/protocol/saml
pulse.manager.services.core.security.authplugin.filter.samlso.idp.x509cert=<Copy the Keycloak
realm certificate from the Realm Settings Keys tab>
pulse.manager.services.core.security.authplugin.filter.samlso.security.authnRequestsSigned=true
pulse.manager.services.core.security.authplugin.filter.samlso.security.wantAssertionsSigned=true
pulse.manager.services.core.security.authplugin.filter.samlso.sp.privateKey=<Follow Appendix A
of this page to use the private key from Keycloak client SAML Keys>
pulse.manager.services.core.security.authplugin.filter.samlso.sp.x509cert=<Copy the certificate
from Keycloak client SAML Keys>
```



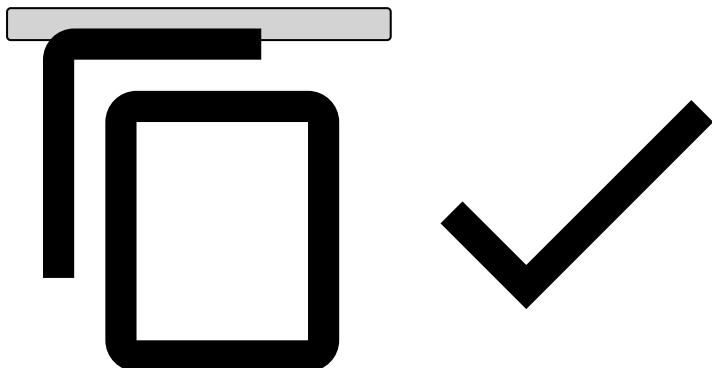
If Pulse is behind an SSL termination proxy, also add the following property:

```
pulse.manager.services.core.security.authplugin.filter.samlso.sp.forceHttps=true
```



The real-time scoring Pulse environment works as a SAML proxy for CM. Create the following properties to configure this proxy behavior. These properties do not apply to the Data Science environment of Pulse.

```
pulse.manager.services.core.security.authplugin.proxy.samlso.cm.privateKey=<Follow Appendix A to use the private key from CM Keycloak client SAML Keys>
pulse.manager.services.core.security.authplugin.proxy.samlso.cm.x509cert=<Copy the certificate from CM Keycloak client SAML Keys>
```



Data science environment¹

Keycloak configuration¹

Follow the same instructions to create a SAML client as described in the [Pulse Real-time Scoring](#) section of this page, adjusting the URL for the Pulse Data Science environment.

Pulse properties¹

Configure the same properties as described in the [Pulse Real-time Scoring](#) section of this page, adjusting the SAML Keys for the Pulse Data Science client created in Keycloak and excluding the CM proxying properties.

Case Manager¹

Keycloak Configuration¹

Follow the same instructions to create a SAML client as described in the [Pulse Real-time Scoring](#) section of this page, adjusting the URL for Case Manager.

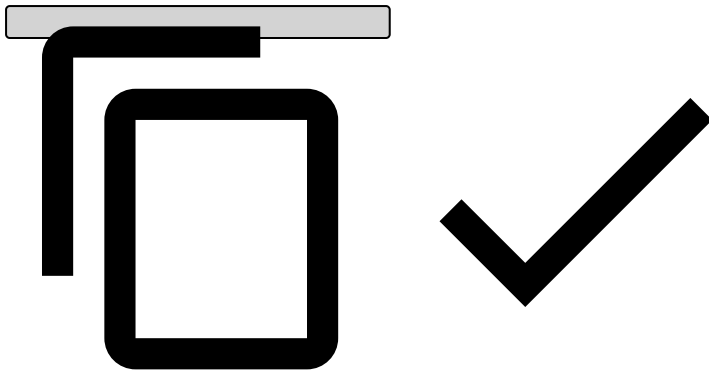
The property for the Logout Service Redirect Binding URL in CM is different from Pulse, the correct URL to use is `{CASE_MANAGER_URL}/api/session/saml-logout`.

Case Manager Configuration¹

The CM configuration is simpler because it uses the real-time scoring Pulse environment as its SAML proxy.

Merge the following JSON settings in `{CASE_MANAGER_INSTALL}/conf/config.json` replacing `CASE_MANAGER_URL` with the correct value:

```
{
  "cm": {
    "url": "{CASE_MANAGER_URL}"
  },
  "pulse": {
    "sso_saml_auth": true
  }
}
```



SAR Manager^â

Keycloak Configuration^â

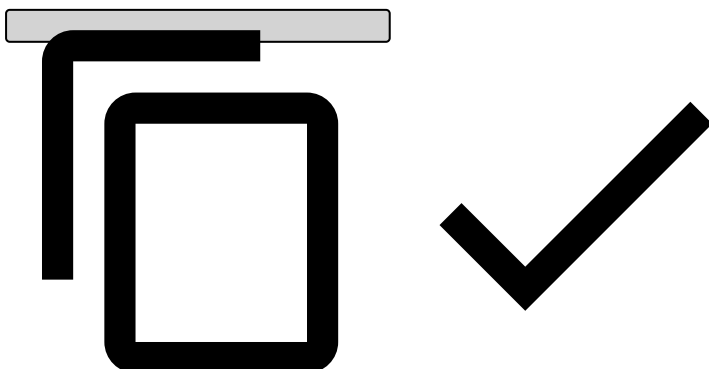
To configure Keycloak to accept connections from SAR Manager, complete the following steps:

1. Create a new openid-connect client with Client ID SARM (in this case the name doesn't need to match the SARM URL). Before saving, do the following changes to the default configuration:
 1. Set Access Type to confidential.
 2. Turn on Authorization Enabled.
 3. Set Valid Redirect URIs to `{SARM_URL}/*`
 4. Set Web Origins to `{SARM_URL}`
2. Save the client and go to the Mappers tab to configure how the user roles are propagated.
3. Create a new mapper of type User Attribute:
 1. Set User Attribute to roles.
 2. Set Token Claim Name to roles
 3. Turn on Multivalued
4. Save the mapper.

SARM Configuration^â

Merge the following top-level YAML settings in the `{SARM_INSTALL}/config/sarm.yaml` :

```
auth:
  oidcJwkUrl: {KEYCLOAK_URL}/auth/realms/{REALM_NAME}/protocol/openid-connect/certs
  oidcDiscoveryUri: {KEYCLOAK_URL}/auth/realms/{REALM_NAME}/.well-known/openid-configuration
  oidcClientId: sarm
  oidcClientSecret: {SECRET} // Copy the secret from SARM Keycloak client Credentials tab
  oidcCallbackUrl: {SARM_URL}/auth/callback
```



Testing the integration^â

Single sign-on should now be working. When a user logs in into one of the Feedzai products and then navigates to another one, no further login should be required.

If it is not working as expected, check the logs for both Keycloak and the affected Feedzai product for troubleshooting error messages.

For single logout to work properly, the Federated Identity Provider must support this feature and must be specifically configured to do so.

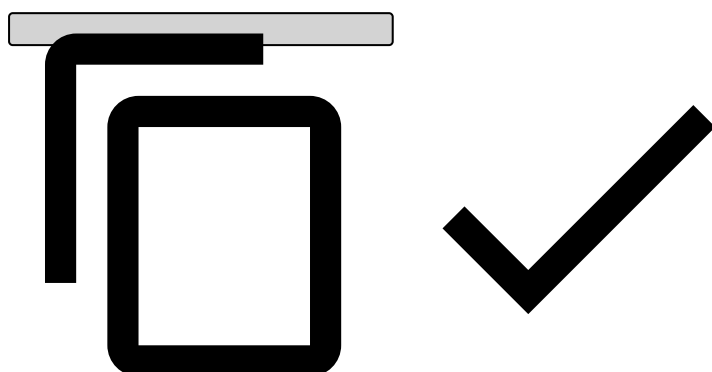
Security hardening🔒🔒

Secrets (such as the OIDC client secret) are shown in clear text in the Keycloak's Admin UI and are also stored in clear text in Keycloak's database. Because of this, it is recommended to restrict access to Keycloak Admin UI as [described here](#) and make sure that the database is only accessible from Keycloak nodes and access is granted only to authorized accounts. Note that the OIDC client secret is also stored in clear text in SARM configuration so this configuration file should also be protected with appropriate permissions.

Appendix A - Convert Private Key Format🔒🔒

To use the private key from Keycloak SAML client in Pulse, it needs to be converted to PKCS#8. Copy the private key and save in file `pk.rsa` and run the following Linux command:

```
paste -d'\n' <(echo "-----BEGIN RSA PRIVATE KEY-----") \
pk.rsa <(echo "-----END RSA PRIVATE KEY-----") | \
fold -w64 | openssl pkcs8 -topk8 -nocrypt | \
sed '1d;$d' | tr -d '\n'
```



If Pulse is configured to use a secret to encrypt sensitive configuration properties, the private key must be encrypted before setting the property.

Copy the resulting output and use it to set the private key Pulse property. Delete the `pk.rsa` file because it's no longer necessary.

On this page

Configuring Keycloak in Pulse with OpenID Connect

Was this page helpful?

This page describes how to integrate Keycloak as the Identity Provider (IdP) into the Pulse environments using OpenID Connect. The integration has been tested for Keycloak version 20.0.1. Take this into consideration when following this guide and the official Keycloak documentation.

Connecting Feedzai products to Keycloak

Pulse real-time scoring

Keycloak Configuration

To configure Keycloak to accept Pulse connections, complete the following steps:

1. Create a new OpenID Connect client, filling a descriptive Client ID (from now on referred as `CLIENT_ID`), e.g. `pulse-oidc`, and optionally define Name and Description. Before saving, and referring to the HTTPS URL of the real-time scoring Pulse environment as `PULSE_SCORING_URL`, do the following changes to the default configuration:
 1. Enable Client authentication (sets this client as *confidential*) and Service accounts roles.
 2. Set Root URL to `{PULSE_SCORING_URL}`.
 3. Set Valid Redirect URIs to `{PULSE_SCORING_URL}/*`.
2. Save the client and go to the Client scopes tab to configure the claims to include in JWTs besides the standard ones.
3. Create a new OpenID Connect Client scope (or change an existing one) such that the following claims are included:

JWT Claim	Mapper	Required?	Where to add
preferred_username	username (predefined/built-in)	Yes	Access token
aud	audience resolve (predefined/built-in)	Required if the server validating the tokens requires audience claim	
name	full name (predefined/built-in)	No, but recommended, so that the user's Full Name is available to Pulse	
email	email (predefined/built-in)	Required when using features that send emails to users (e.g. 4eye check); otherwise, recommended	
locale	locale (predefined/built-in)	No, but highly recommended if internationalization is enabled at the IdP and the locale is used in Pulse	
<code>roles.\$CLIENT_ID</code> is the default in Pulse, but can be configured	client roles (predefined/built-in)	Yes, because the users need roles to be able to access the Pulse UI	Userinfo
		Required only when users are associated with groups and	

JWT Claim	Mapper	Required?	Where to add
<code>groups</code> is the default in Pulse, but can be configured	Custom mapper created by configuration with Mapper Type "Group Membership"; "Full group path" must be disabled	that information is to be propagated to Pulse	
<code>tenants</code> is the default in Pulse, but can be configured	Having tenants associated with users is not a concept of Keycloak; the workaround is to have that information as a user attribute, and then use a custom mapper created by configuration with Mapper Type "User Property", to map the value of that attribute to the "tenants" claim	Required only when users are associated with tenants and that information is to be propagated to Pulse	

4. Save the Client scope and add it to the client for Pulse, making sure it is set as "Default".
5. Configure other settings, such as token/session durations, flows, etc.
6. Save the client.

Pulse properties

Create the following properties to configure Pulse to connect to Keycloak for SSO using OpenID Connect.

The client secret (referred as `CLIENT_SECRET` in the configuration example below) can be retrieved at the Credentials tab of the client in Keycloak. Since this is sensitive information, the corresponding property in Pulse has encryption enabled. Therefore, if Pulse has an encryption key for properties set up, then the value of this property needs to be encrypted using that key.

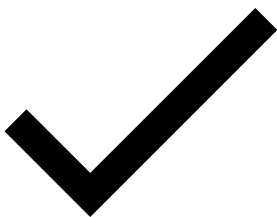
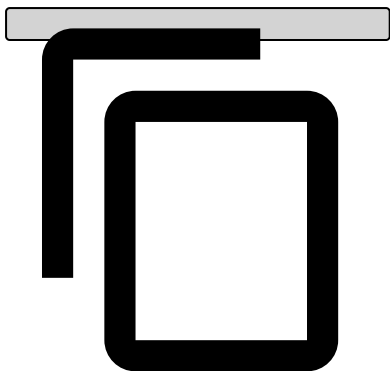
The `discoveryURI` (referred as `DISCOVERY_URI` in the configuration example below) is the URL of the "well known" endpoint of the IdP, and appears at the Realm Settings page of Keycloak as a link to "OpenID Endpoint Configuration".

```
#####
#                               OIDC
#####
pulse.global.oidc.clientId=<CLIENT_ID>
pulse.global.oidc.secret=<CLIENT_SECRET>
pulse.global.oidc.discoveryURI=<DISCOVERY_URI>

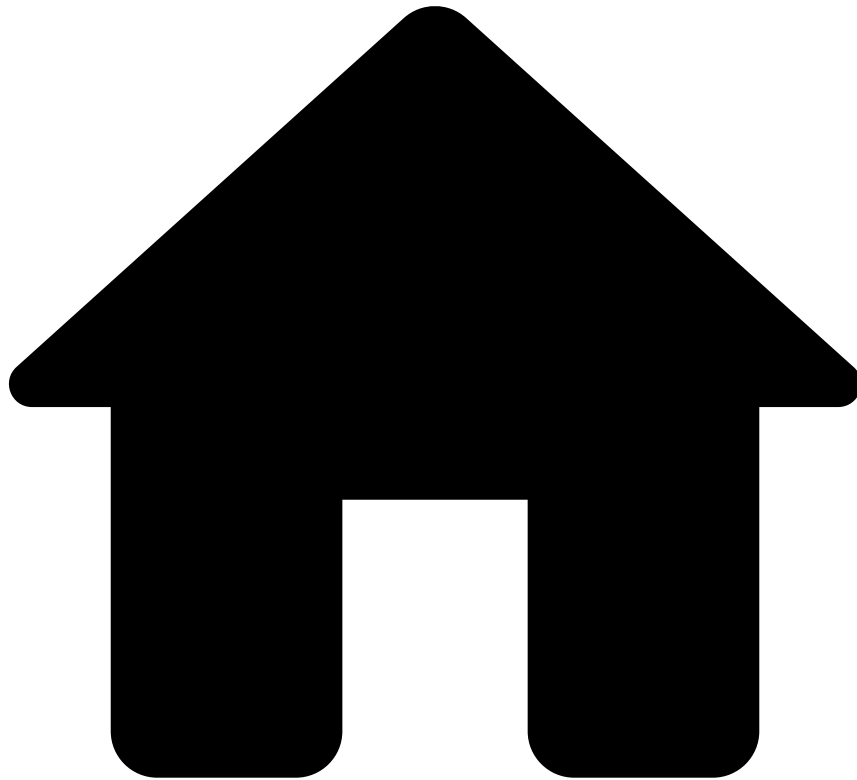
#####
#                               JWT Auth Provider
#####
# The following property is the id of the configuration, only needed to enable the plugin
# if no other property is set
pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.id=feedzai-keycloak

# Defaults for connection timeouts to the IdP and JWT clock skew
#pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.connectTimeoutMs=500;
#pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.readTimeoutMs=500;
#pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.maxClockSkewSeconds=60;

# Default values for JWT claim names/paths, for clientId = `CLIENT_ID`
#pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.tenantsPath=/tenants
#pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.groupsPath=/groups
#pulse.manager.services.core.security.authplugin.provider.jwt.idpConfig.rolesPath=/roles/
<CLIENT_ID>
```



-



- [Single Sign-On](#)
- Configuring SSO Using a Generic IdP

On this page
Was this page helpful? 

Configuring SSO Using a Generic IdP

Single Sign-On (SSO) in Pulse uses SAML 2.0 standard as the protocol to authenticate and authorize users that are provisioned by an external Identity Provider (IdP).

This guide describes the configuration of SSO using a generic IdP. If you are using Microsoft AD FS as the IdP of choice, please visit the page [Configuring AD FS](#).

Pulse SSO properties

Configuring **SSO** in Pulse requires setting a number of properties. Pulse needs to:

- Know what is the IdP that it will connect.

- Have a private key for signing Pulse requests and a corresponding X509 certificate.
- Have a private key for signing On-Behalf Proxy requests and a corresponding X509 certificate.

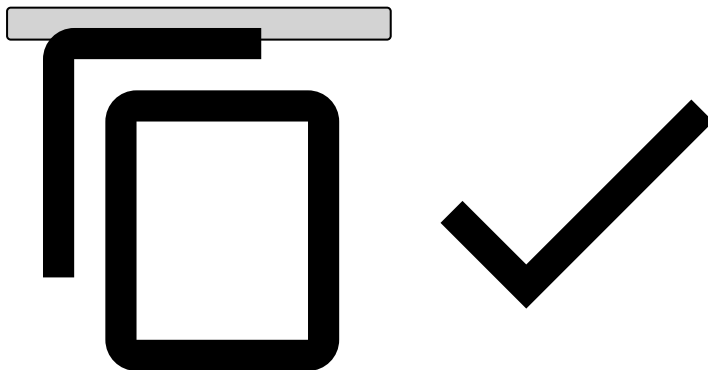
Contact Feedzai to set up these properties for your environment.

Identity provider properties

The IdP properties have the `pulse.manager.services.core.security.authplugin.filter.samlssoidp` prefix and the easiest way to know what to include in them is to obtain the **IdP XML metadata**. This XML contains configuration and integration details for SAML 2.0 and is usually made available on a URL. Alternatively, it can be exported as a file on the IdP configuration interface.

The following code block shows the relevant portion of this XML that contains the information that Pulse needs to communicate with the IdP and to which properties the information should be copied.

```
<EntityDescriptor xmlns="urn:oasis:names:tc:SAML:2.0:metadata" ID="..." entityID="{idp.entityId}">
  ...
  <IDPSSODescriptor protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol">
    ...
    <KeyDescriptor use="signing">
      <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#">
        <X509Data>
          <X509Certificate>
            {{idp.x509cert}}
          </X509Certificate>
        </X509Data>
      </KeyInfo>
    </KeyDescriptor>
    ...
    <SingleLogoutService Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect" Location="{idp.slo}" />
    ...
    <SingleSignOnService Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect" Location="{idp.sso}" />
    ...
  </IDPSSODescriptor>
</EntityDescriptor>
```



Service Provider Properties

The SAML 2.0 standard requires the sign out requests to be signed by the two parties (i.e. Pulse and the IdP). This requires Pulse to have a private key and a X509 public certificate. Optionally, Pulse and IdP can also be configured to sign the Sign in requests.

The following code block exemplifies how to generate the private key and the respective self-signed x509 certificate.

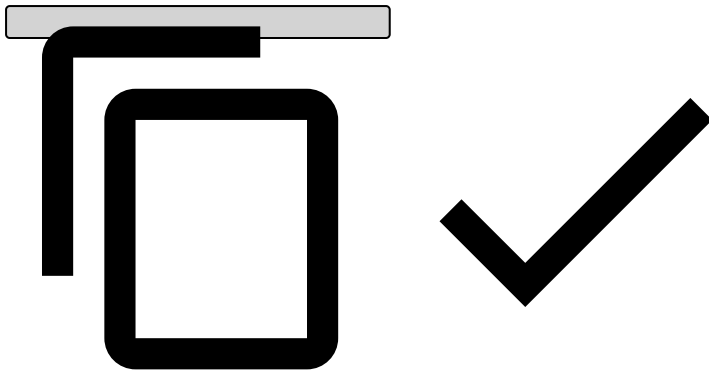


info

Please check if you can use self-signed certificates for **SSO** in your deployment.

```
# Generate a new Private Key and respective self-signed Certificate, valid for $CERT_DAYS days
openssl req -x509 -nodes -newkey rsa:$KEY_SIZE -keyout $PK_TEMP_FILE -out $CERT_FILE -days $CERT_
DAYS -subj "/C=PT/O=Feedzai/OU=Pulse/CN=feedzai.com"

# Convert the Private Key to PKCS#8 format
openssl pkcs8 -topk8 -inform pem -nocrypt -in $PK_TEMP_FILE -outform pem -out $PK_FILE
```



where the variables are replaced by the following:

Variable	Value	Notes
<code>\$KEY_SIZE</code>	1024	1024 is the minimum key size required by some IdPs
<code>\$CERT_DAYS</code>	3650	
<code>\$PK_TEMP_FILE</code>	pkTempFile.pem	
<code>\$PK_FILE</code>	pkey.pem	The PKCS#8 private key file.
<code>\$CERT_FILE</code>	cert.pem	The X509 certificate file.

The contents of `$PK_FILE` and `$CERT_FILE` must be copied to the corresponding properties in Pulse, respectively:

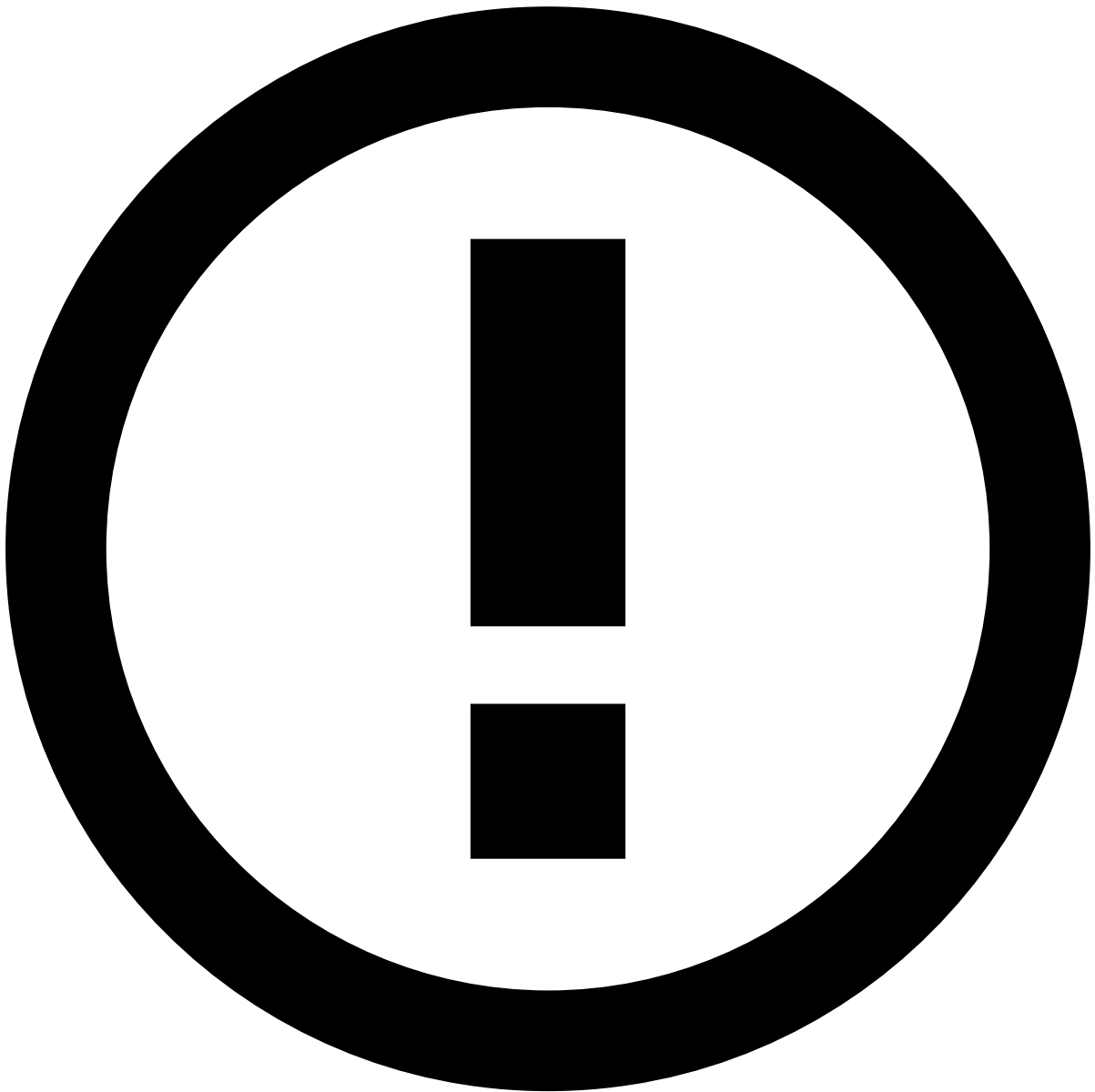
- `pulse.manager.services.core.security.authplugin.filter.samlso.sp.privateKey`
- `pulse.manager.services.core.security.authplugin.filter.samlso.sp.x509cert`

On-behalf service provider properties

Pulse is also able to act as a non-standard on-behalf service provider proxy (e.g. Case Manager uses this option).

For each proxy service provider that will use Pulse to handle SAML messages on its behalf, you have to repeat the process of obtaining a private key and respective certificate, but copy the contents of `$PK_FILE` and `$CERT_FILE` to the following Pulse properties instead:

- `pulse.manager.services.core.security.authplugin.proxy.samlso.<spKey>.privateKey`
- `pulse.manager.services.core.security.authplugin.proxy.samlso.<spKey>.x509cert`



info

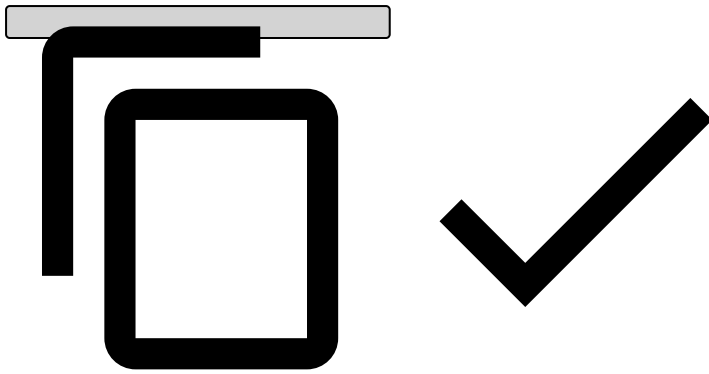
For Case Manager, replace `<spKey>` with `cm` .

Configure IdP to communicate with Pulse

After configuring Pulse to communicate with the IdP and trust its information, the IdP also needs to be configured to work with Pulse. The SAML 2.0 standard also defines a standard metadata XML for service providers to make their configuration and make their integration details available (e.g. identification, login and logout endpoints, public certificate).

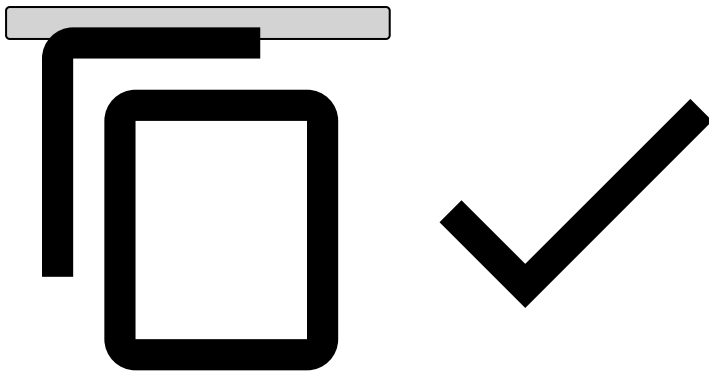
After configuring Pulse for SSO, the XML metadata can be obtained by accessing the following address:

```
https://<BASE_PULSE_URL>/pulseviews/api/sessions/samlssso/metadata
```

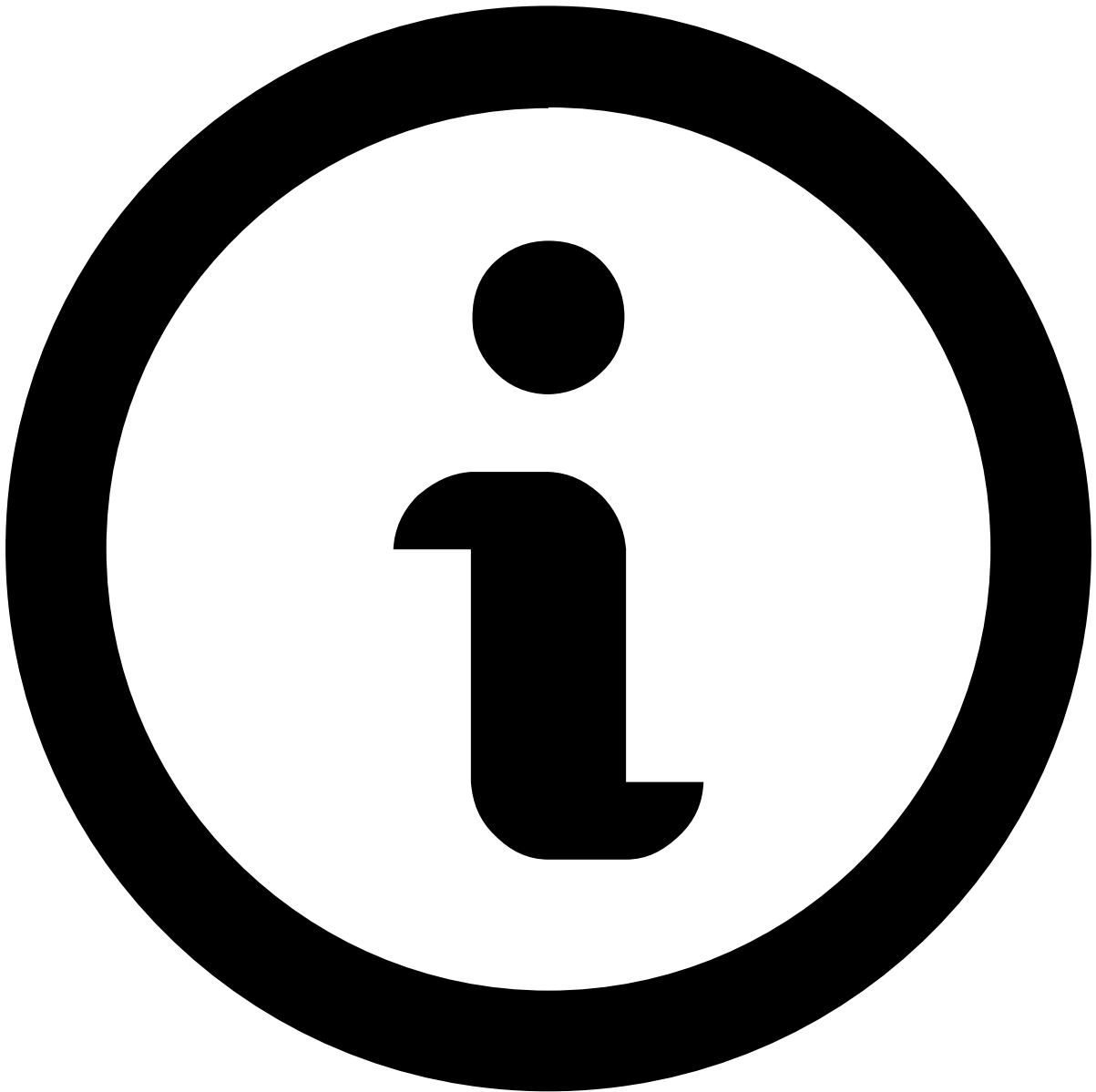


If Pulse is configured to act as an on-behalf service provider (e.g. Case Manager), you can obtain the metadata for the referred service provider by using the following curl request:

```
curl -X POST \
  https://<BASE_PULSE_URL>/pulseviews/api/samlproxy/metadata \
  -H 'Content-Type: application/json' \
  -H 'Cookie: SS0cookie=<A_VALID_PULSE_COOKIE>' \
  -d '{
    "spKey": "<spKey>",
    "baseUrl": "<BASE_HTTPS_URL>",
    "loginEndpoint": "<SAML_LOGIN_ENDPOINT>",
    "logoutEndpoint": "<SAML_LOGOUT_ENDPOINT>"
  }'
```



Most IdPs have a method to import these XML metadata files. Even if they don't, this metadata contains all the information that you might need to add Pulse as a trusted service provider on the IdP.



note

This step is not required if authorization is configured to be performed in Pulse.

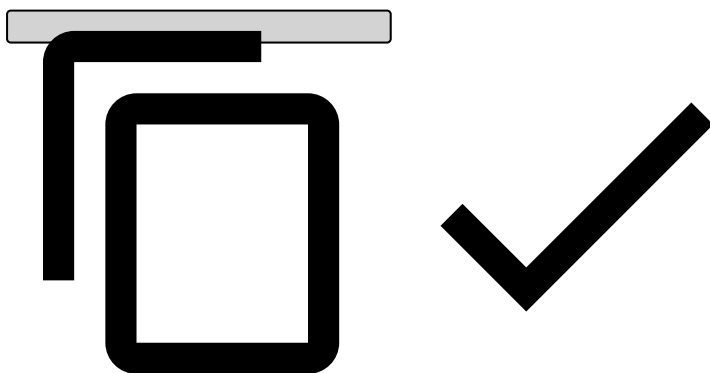
Now that Pulse and the IdP trust and know how to communicate with each other, you can configure user provisioning. Besides the authentication info, Pulse and Case Manager also need to receive authorization information along with users authentication. This information is essential to perform user provisioning (i.e. creating a Pulse user on the fly).

The user provisioning configuration must be set on the IdP configuration interface. Since this configuration is different according to the used IdP, this section focuses on explaining what needs to be configured, as opposed to how to configure it. If Microsoft AD FS is the IdP of choice in your deployment, please visit the page [Create claim rules on AD FS](#).

Pulse performs user provisioning based on the values of SAML attributes within an authentication response. SAML attributes are sent inside the AttributeStatement XML element. Each of these attributes has a name and one or more values.

The following code shows a sample SAML authentication response sent by the IdP to Pulse:

```
<samlp:Response ID="..." Version="2.0" IssueInstant="..." Destination="https://pulse-local-
signed:8443/pulseviews/api/sessions/samlso" Consent="..."
  xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol">
  <Issuer xmlns="urn:oasis:names:tc:SAML:2.0:assertion">{{idp.entityId}}</Issuer>
  <samlp:Status>...</samlp:Status>
  <Assertion ID="..." IssueInstant="..." Version="2.0" xmlns="urn:oasis:names:tc:SAML:
2.0:assertion">
    <Issuer>{{idp.entityId}}</Issuer>
    <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">...</Signature>
    <Subject>
      <NameID Format="...">{{user.username}}</NameID>
      <SubjectConfirmation Method="urn:oasis:names:tc:SAML:2.0:cm:bearer">...</SubjectConf
irmation>
    </Subject>
    <Conditions NotBefore="..." NotOnOrAfter="...">...</Conditions>
    <AttributeStatement>
      ...
      <Attribute Name="User.roles">
        <AttributeValue>UserRole1</AttributeValue>
        <AttributeValue>UserRole2</AttributeValue>
      </Attribute>
      <Attribute Name="User.groups">
        <AttributeValue>UserGroup1</AttributeValue>
        <AttributeValue>UserGroup2</AttributeValue>
      </Attribute>
      <Attribute Name="User.tenants">
        <AttributeValue>UserTenant</AttributeValue>
      </Attribute>
      <Attribute Name="User.language">
        <AttributeValue>en-US</AttributeValue>
      </Attribute>
      <Attribute Name="User.fullname">
        <AttributeValue>User Full Name</AttributeValue>
      </Attribute>
      ...
    </AttributeStatement>
    <AuthnStatement AuthnInstant="..." SessionIndex="...">...</AuthnStatement>
  </Assertion>
</samlp:Response>
```



After receiving the message above, besides the user name Pulse will extract the following information for the authenticated user:

- **Roles:** UserRole1, UserRole2
- **Groups:** UserGroup1, UserGroup2
- **Tenant:** UserTenant
- **Language:** en-US
- **Full Name:** User Full Name

When working with these settings, consider the following:

- Roles, groups and tenants are matched by name. They have to previously exist in Pulse, otherwise the authorization will fail. Exception messages to search for:
 - Group <name> does not exist
 - Role <name> does not exist
- All SSO-Users must have at least one group or role associated with them. Exception message to search for:
 - User <username> has no group/role
- The name of SAML attributes presented above may be changed.
- Besides being created when a user signs into Pulse for the first time, the user information is updated every time the user signs in.
- The content of exchanged SAML messages may be logged. Package: com.onelogin.saml2, Level: debug.

-



- Project Organization
- [Monitoring](#)
- [Logging](#)
- Configure Jetty to Log HTTP Requests

Configure Jetty to Log HTTP Requests

Was this page helpful? 

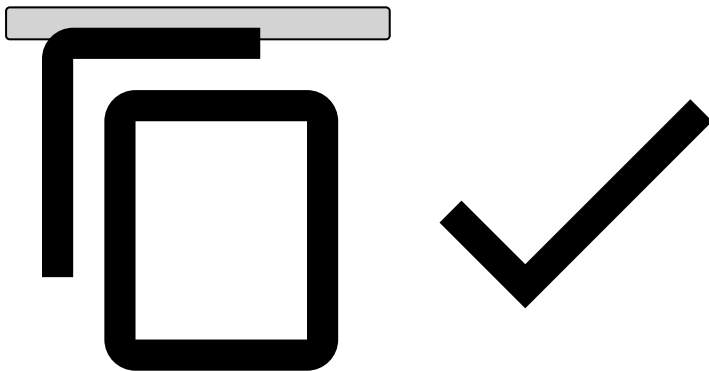
Pulse uses Jetty **9.4.X** as the back-end engine for the web server. It is possible to configure the engine to automatically log every HTTP request into a log file (which will appear in the `log/` folder). To do so, it's necessary to add a new logger entry to the file `conf/engine/jetty/jetty.xml` in Pulse:

```
<Configure id="Server" class="org.eclipse.jetty.server.Server">
  <Arg name="threadpool"><New id="threadpool" class="org.eclipse.jetty.util.thread.QueuedThreadPool"/></Arg>
  ...
  <Set name="RequestLog">
    <New id="RequestLog" class="org.eclipse.jetty.server.CustomRequestLog">
      <!-- Writer -->
```

```

<Arg>
  <New class="org.eclipse.jetty.server.AsyncRequestLogWriter">
    <!-- Make http request log rotate by day. -->
    <Arg><Property name="solr.log.dir" default="log"/>/>yyyy_mm_dd.request.log</Arg>
    <Set name="filenameDateFormat">yyyy_MM_dd</Set>
    <!-- Only delete rotated files after a week. -->
    <Set name="retainDays">
      <Property name="solr.log.requestlog.retaindays" default="7"/>
    </Set>
    <Set name="append">true</Set>
    <Set name="timeZone">UTC</Set>
  </New>
</Arg>
<!-- Default format String: -->
<!-- Arg><Get class="org.eclipse.jetty.server.CustomRequestLog"
name="EXTENDED_NCSA_FORMAT"/></Arg -->
<!-- Custom format String: https://javadoc.io/doc/org.eclipse.jetty/jetty-server/
9.4.51.v20230217/org/eclipse/jetty/server/CustomRequestLog.html -->
<Arg>{{date: %{YYYY-MM-dd:hh:mm:ss:SSSZ}t},{response status:%s},{hostname: %{client}a},
{response time: %{ms}Tms}, {hostname: %{Host}i}, {user: %u}, {Connection status: %X}, {response
status: %s}, {first line: %r}, {address client: %{client}a}, {address server: %{server}a}, {MAC:
%{local}a}}</Arg>
</New>
</Set>
...

```



The API of CustomRequestLog for **Jetty v9.4.51.v20230217** (used by **Pulse 25.0.15+**) is described at [the jetty-server documentation](https://javadoc.io/doc/org.eclipse.jetty/jetty-server/9.4.51.v20230217/org/eclipse/jetty/server/CustomRequestLog.html).

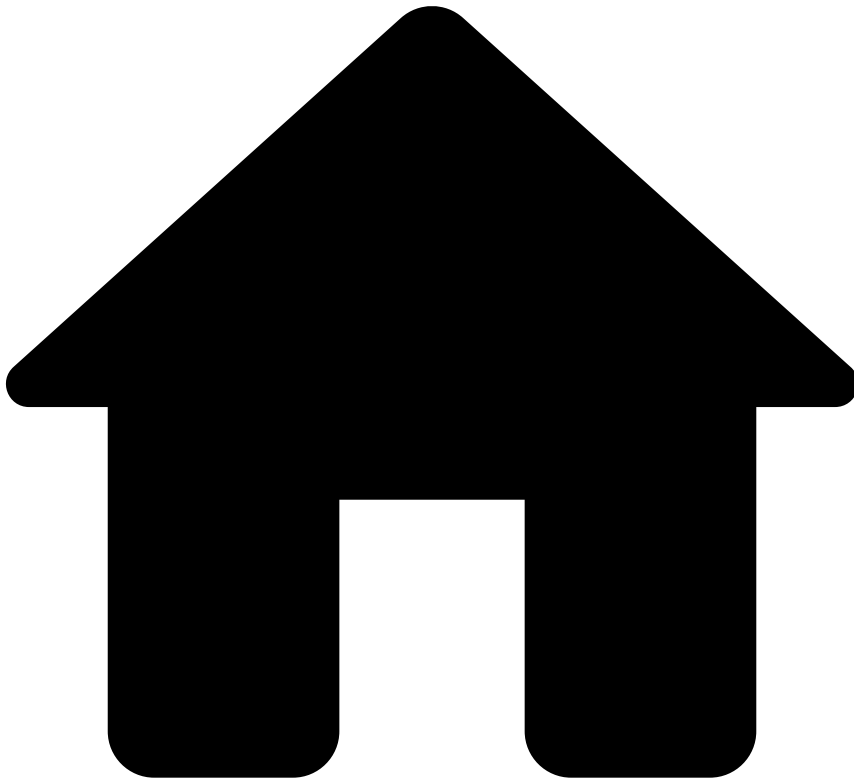
By using this CustomRequestLog it's possible to satisfy the following requirements from [CR707 PCR4293 Fraud Event Logging Components_Requirements](#):

Requirement	Loggable	RequestLog Argument (string format)
Properly formatted and accurate timestamp	(tick) Must be in a format supported by Jetty DateCache.	{date: %{YYYY-MM-dd:hh:mm:ss:SSSZ}t}
Status code for the event type request was successful or not	(tick) It's possible to know via the HTTP response code.	{response status:%s}
Device identifier (MAC address or other unique identifier)	(tick) Likely not possible since printing physical address is the same as printing the IP of the client or server in our tests.	{MAC: %{local}a} It prints the IP as physical address.
Session / Transaction ID		

	(tick) Likely not possible since API doesn't provide a way to get an unique identifier for the request.	
Autonomous System Number (hostname)	(tick) Possible. Could be necessary to print the host header for it.	<code>{hostname: %{Host}i}</code> Might be possible for client to send wrong information.
Source IP (IPv4)	(tick)	<code>{address client: %{client}a}</code>
Source IP (IPv6)	(tick) Likely possible since IPv4 works, but untested.	<code>{address client: %{client}a}</code> It should also print IPv6 if used. Not tested.
Status Code	(tick)	<code>{response status: %s}</code>
Response Time	(tick)	<code>{response time: %{ms}Tms}</code>
Additional headers (i.e., HTTP headers)	(tick)	<code>{Header_Name: %{Header_Name}i}</code>
Where appropriate, the username and/or userID shall be included	(tick) Likely not possible since API doesn't provide a way to get an unique identifier for the request.	
Where appropriate, the command executed shall be included	(tick)	<code>{first HTTP line: %r}</code>
Where possible, all data shall be formatted as key-value-pairs allowing for easy extraction	(tick)	E.g.: <code>{{date: %{YYYY-MM-dd:hh:mm:ss:SSSZ}t},{response status:%s},{hostname: %{client}a}, {response time: %{ms}Tms}, {hostname: %{Host}i}, {user: %u}, {Connection status: %X}, {response status: %s}, {first line: %r}, {address client: %{client}a}, {address server: %{server}a}, {MAC: %{local}a}}</code>
Where possible, a unique event identifier shall be included for event correlation; a unique event identifier shall be defined per event type6	(tick) Likely not possible since API doesn't provide a way to get an unique identifier for the request.	

The third column of the table above shows examples of possible values for the argument `formatString` that needs to be passed to the constructor of `org.eclipse.jetty.server.CustomRequestLog#CustomRequestLog( org.eclipse.jetty.server.RequestLog.Writer, java.lang.String )` in order to print the requirements of the first column.

•



- Project Organization
- [Monitoring](#)
- [Logging](#)
- Event Storage Dump Logging

On this page

Event Storage Dump Logging

Was this page helpful? 

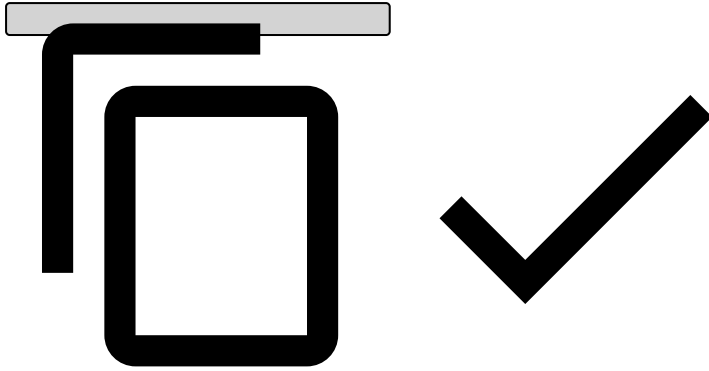
Event Storage Dump Logging

In case of a major failure, the Smart Event Storage Dump will log the event that failed to be persisted in the file. This event can contain confidential information, therefore, these logs should only be logged to the [Confidential Logger](#). In order to do that the following configurations must be added to the `logger.xml` :

To the "common" logger:

Main Log Appender

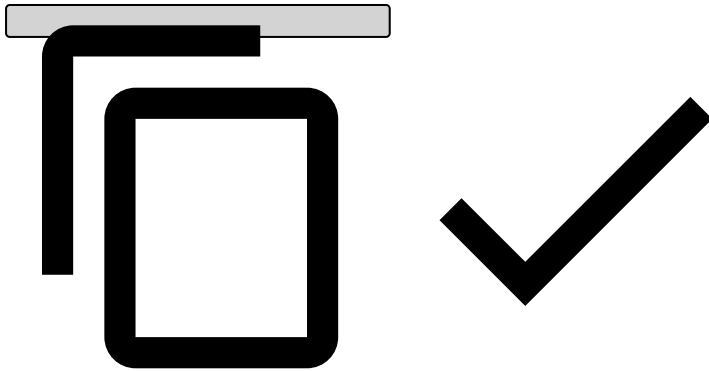
```
<logger name="com.feedzai.dump.storage.impl.files.FileDumpWriter" level="off"/>
```



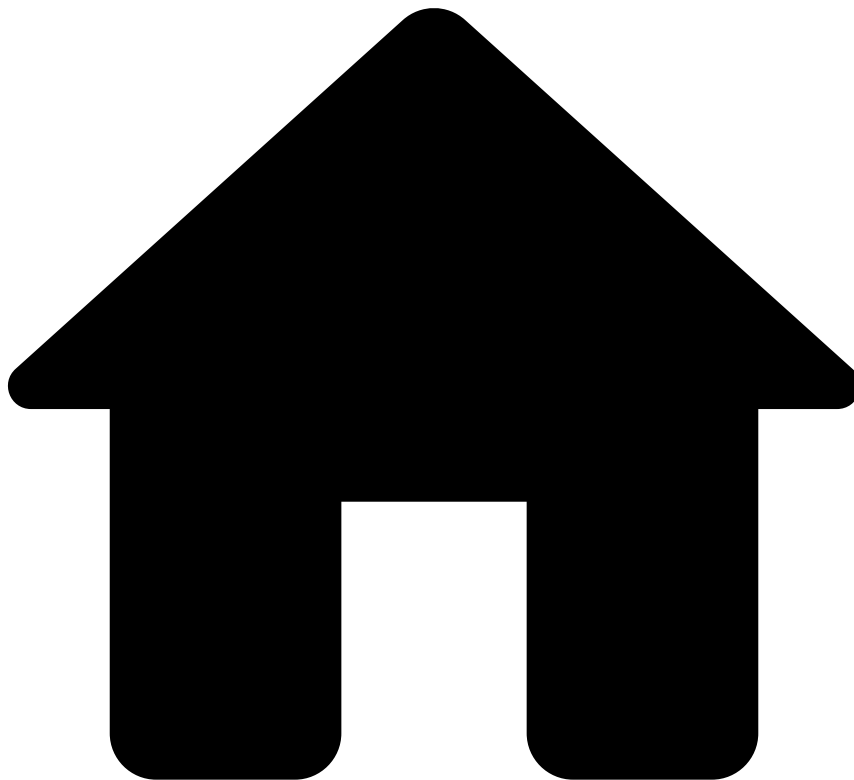
To the confidential logger:

Confidential Log Appender

```
<!-- In trace level it will log the events that failed to be persisted into disk -->  
<logger name="com.feedzai.dump.storage.impl.files.FileDumpWriter" level="off" additivity="false">  
  <appender-ref ref="CONFIDENTIAL-ASYNC"/>  
</logger>
```



•



- Project Organization
- [Monitoring](#)
- [Logging](#)
- Log Confidential Data

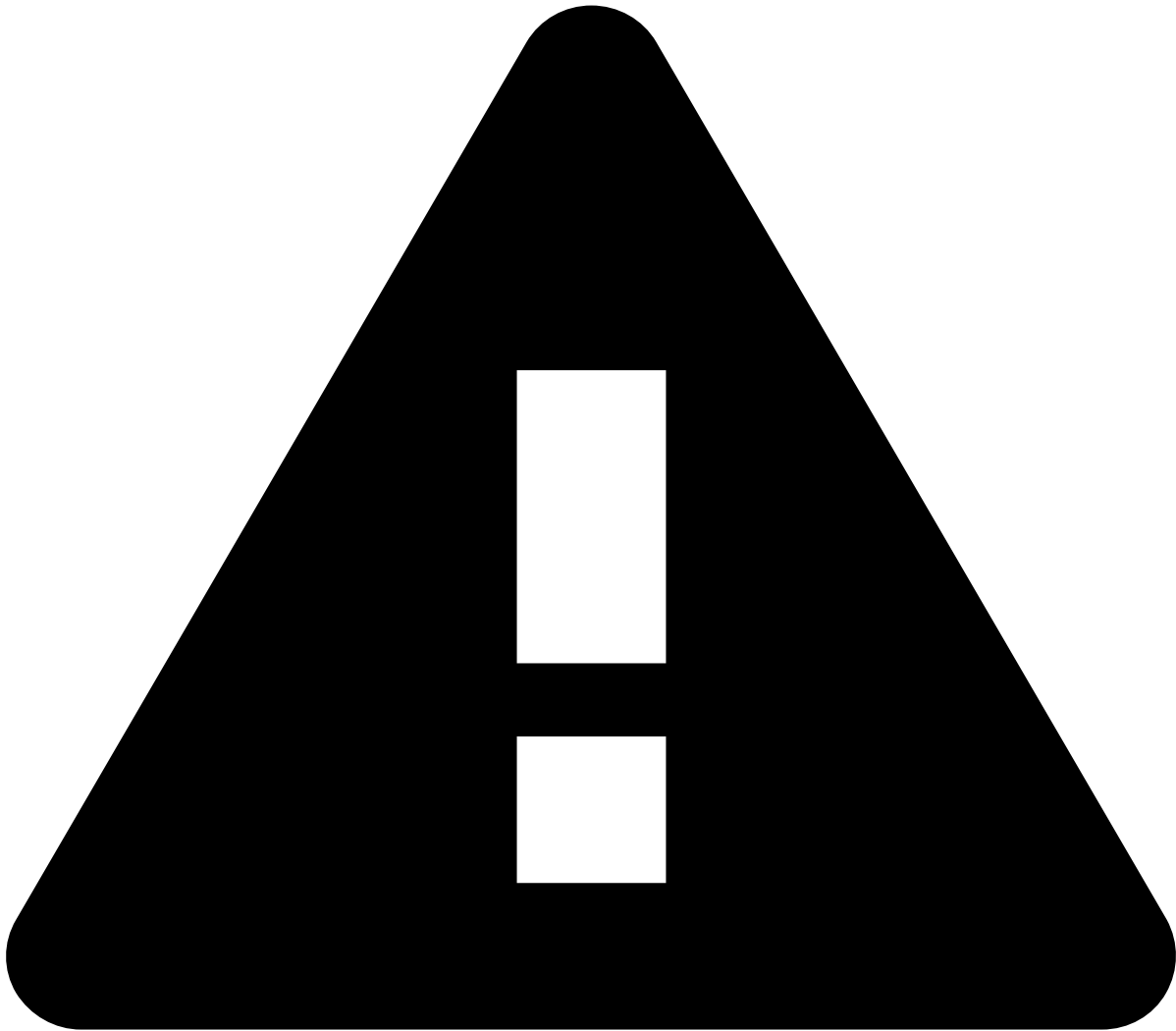
On this page

Log Confidential Data

Was this page helpful? 

Confidential or Personal Identifiable Information (PII) must **NEVER** be logged. However, in some cases, it might be necessary to access confidential data in logs to investigate production issues. In such cases, Pulse can expose logs with confidential data to a set of files that is separate from the normal application logs. Confidential logging should be activated only when it is absolutely necessary, for the time of an investigation.

This page explains how to configure the confidential logger.



warning

The confidential log files **MUST NEVER** be sent to uncontrolled environments (e.g., cloud services such as Amazon CloudWatch) and they **MUST** be deleted immediately after the investigations.

Introduction🔒

Pulse uses the Logback framework to create confidential logs. Confidential logging uses two main concepts of Logback:

- **loggers**: they receive the log messages from Pulse, and pass them to the right appender.
- **appenders**: they store the log messages in their final destination.

In Pulse, every process (i.e. Pulse server, or app engine) can send messages to more than one logger. Each logger can be connected to one or more appenders. Each appender sends the messages to a different file or to the network/terminal. This way, you can log each process' data to a separate file, and you can turn confidential logging on or off separately for each process.

For example, confidential logs of the Pulse server process will be written to `CONFIDENTIAL-pulse.log`, while a confidential log of an app engine named *my-app* will be written to `CONFIDENTIAL-app-my-app.log`. This helps you minimize the amount of PII that is logged.

You can configure confidential logging by editing the file `PULSE_HOME/conf/loggers/logger-confidential.xml` of the server(s) that you want to log data from. This file uses Logback's log configuration syntax.

To ensure that confidential logs aren't appended to the normal log files, such as `pulse.log` and `app-<APP-NAME>.log`, you must have well-configured confidential loggers and confidential appenders in the `logger.xml` file.



Avoid exposing confidential information

You must ensure that your `logger.xml` file has the correct confidential logging setup. Otherwise, you may be exposing confidential logs in the normal log files.

Enabling confidential logging🔒

To enable confidential logging, perform the following steps in the file `PULSE_HOME/conf/loggers/logger-confidential.xml` using [Logback's syntax](#) in the `logger.xml` file:

1. Create a confidential log appender. See an example in the following sections.
2. Create a confidential logger. Configure that logger so that it uses the appender created in step 1. See an example in the following sections.
3. To kick off logging, set the `level` property of the logger to the desired level (e.g., "info", "trace", ...). Any confidential log written by Pulse with a logging level above or equal to the configured level will appear in the respective `CONFIDENTIAL-*.log` files.

Example: creating confidential log appenders

The following example creates two confidential log appenders, the "CONFIDENTIAL-ASYNC" appender is the one that receives the log messages, and passes them to the "CONFIDENTIAL" appender, which sends them to the defined output. Since it uses the `pulse.logcontext` variable, it creates a separate file for each process.

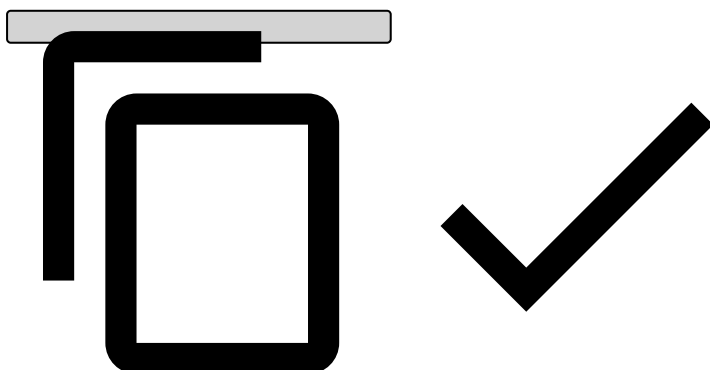
Confidential log appenders

```
<appender name="CONFIDENTIAL" class="ch.qos.logback.core.FileAppender">

  <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
    <level>TRACE</level>
  </filter>

  <file>log/CONFIDENTIAL-`${pulse.logcontext}.log</file>

  <encoder>
    <pattern>%-5level %d{ISO8601} }%-65(%P{slug} %P{appId} %P{member}) %P{isLeader}] %-40({%thread}) %-65(%logger{0}).%M:%line) %msg%n%xEx{full}</pattern>
  </encoder>
</appender>
<appender name="CONFIDENTIAL-ASYNC" class="ch.qos.logback.classic.AsyncAppender">
  <appender-ref ref="CONFIDENTIAL"/>
  <queueSize>2048</queueSize>
  <discardingThreshold>0</discardingThreshold>
  <includeCallerData>true</includeCallerData>
  <neverBlock>true</neverBlock>
</appender>
```



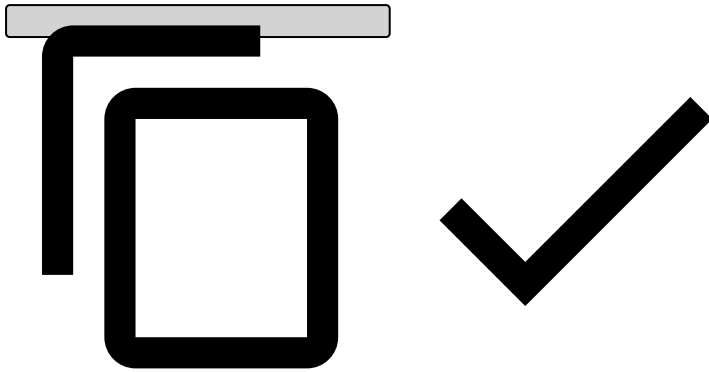
Example: configuring the confidential logger

To ensure that confidential logs written by Pulse are never exposed in the normal log files and are always appended to the `CONFIDENTIAL-*.log` files, you MUST ensure that the following logger is configured in the `PULSE_HOME/conf/loggers/logger-confidential.xml` file.

Confidential Logger

```
<!--
  WARNING:
    - This logger MUST NEVER be commented. Otherwise, confidential logs will end up in the
      normal log files.
    - This logger MUST NEVER be set to a level different than 'off', UNLESS it is strictly
      necessary to perform investigations that requires the visualization of confidential logs.
    - This logger MUST NEVER be set to a level different than 'off' if the logs written by
      the configured appender will end up on insecure environments (e.g., Amazon CloudWatch).
    - If this logger is set to a level different than 'off' to perform strictly necessary
      investigations, it MUST be set to 'off' again immediately after ending the investigations and all
      the produced log files must be immediately deleted without being sent to any untrusted/insecure
      medium.
-->
<logger name="confidential" level="off" additivity="false">
```

```
<appender-ref ref="CONFIDENTIAL-ASYNC"/>
</logger>
```

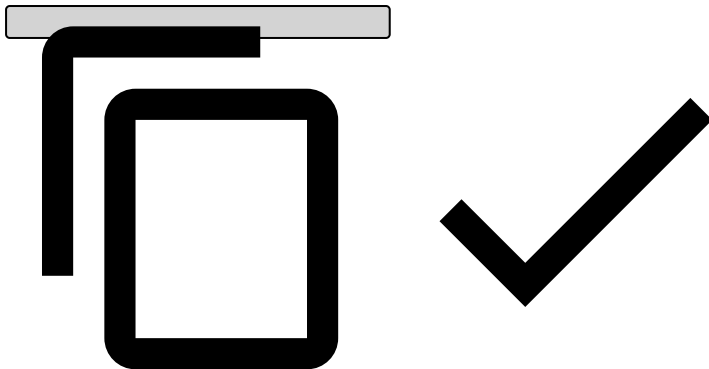


This logger MUST NEVER be commented. Otherwise, confidential logs written in Pulse will end up in the normal log files.

It is possible to configure more granular confidential log levels, by setting the level at class/package scope. To do this, configure loggers with classes/package names prefixed with `confidential.`, as shown in the following example:

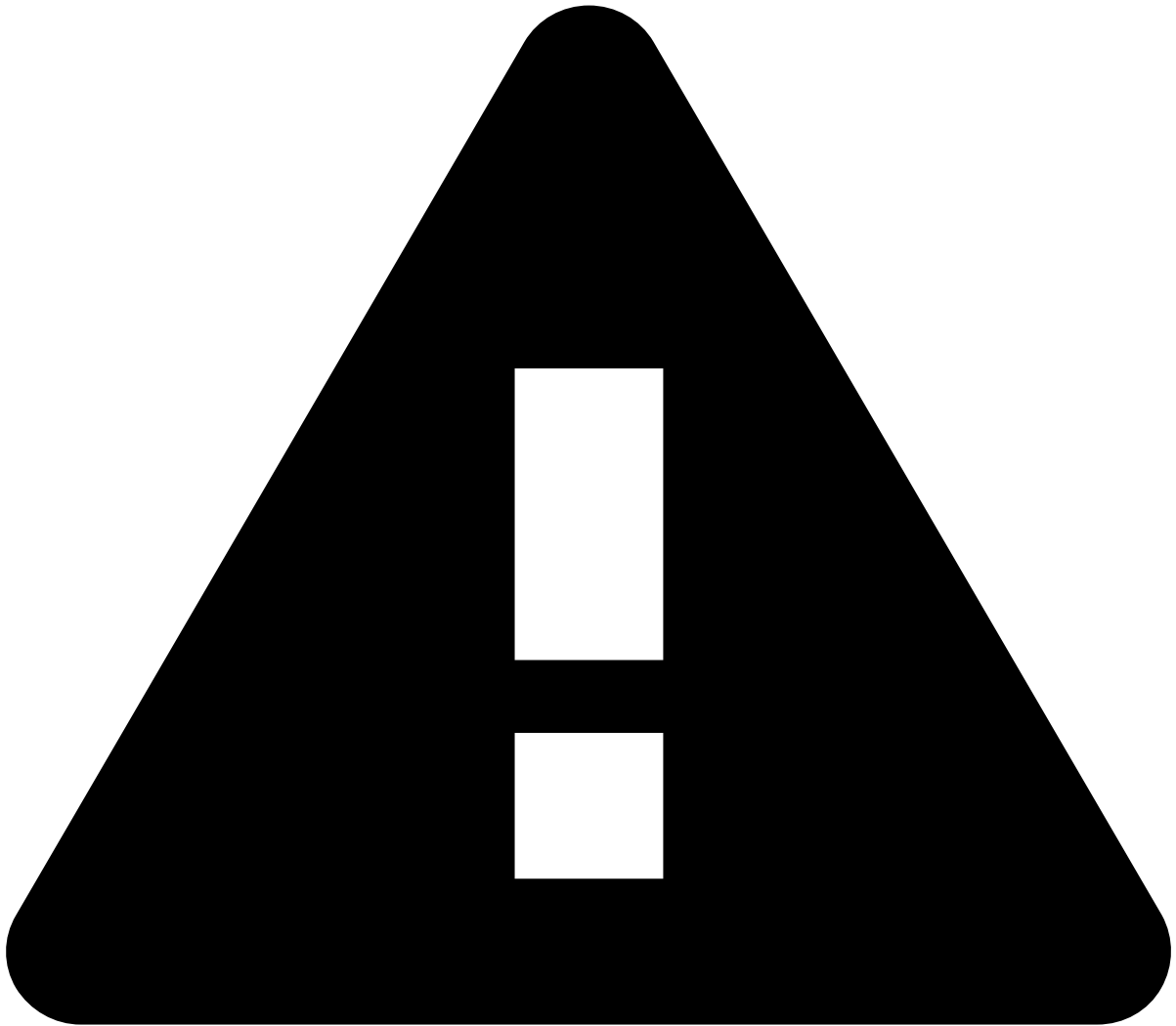
Examples of more granular confidential logging

```
<logger name="confidential.com.feedzai.pulse.libs.lists.UpdatableInMemoryList" level="error"/>
<logger name="confidential.com.feedzai.pulse.libs.lists.ImmutableListsRuntimeLoader"
level="info"/>
```



Turning off confidential logging

To turn off confidential logging completely, set the logger's level to "off" and ensure that the logger has the following property set: `additivity="false"`.

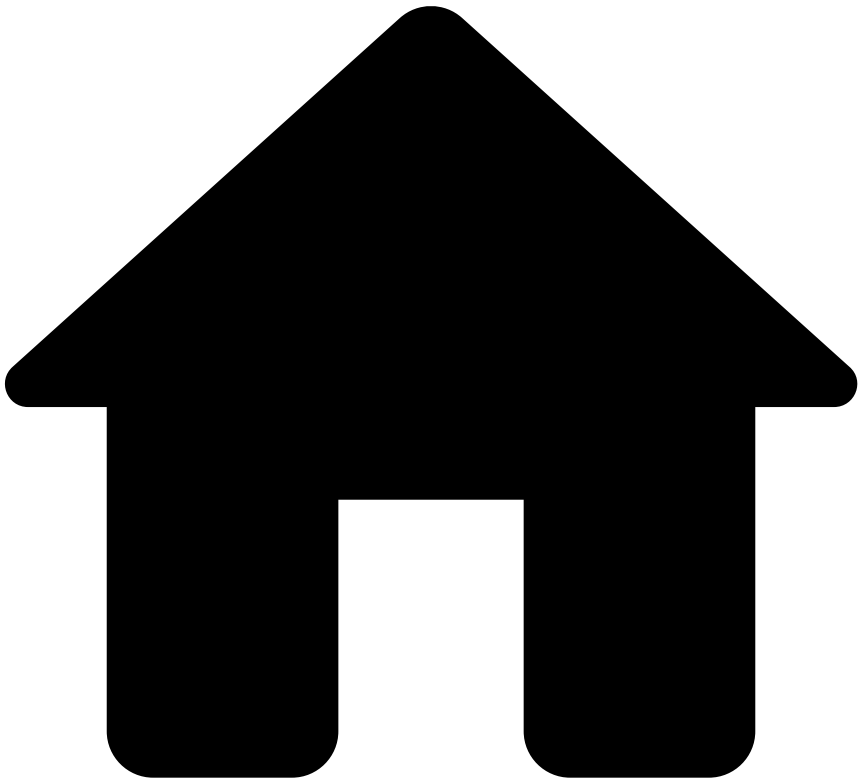


Reset confidential loggers level to OFF

Make sure that the level of the confidential logger is set to "off" after ending any investigation. Otherwise, confidential logs continue to be written to the `CONFIDENTIAL-*.log` files.

Never comment or change any property besides the level in the main confidential logger.

•



- Project Organization
- [Monitoring](#)
- [Logging](#)
- Log Partitioning Information on Plan Operator

Log Partitioning Information on Plan Operator

Was this page helpful? 

Pulse can be configured to log information about the way Pulse partitions work when calculating the values in the plan operator.

The logger property `com.feedzai.pulse.datascience.plan.operators.impl` allows to configure which details should be logged. The following logging levels are available:

Level	What is logged
info	A set of partitioning fields with corresponding metrics. Computed metrics partitioning schemas of multiple <i>Group By</i> fields.
debug	

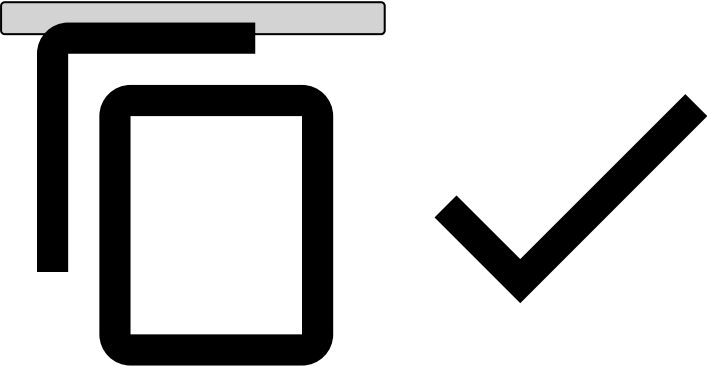
Level	What is logged
	The name of the algorithm that was used to calculate the partitioning schema for shuffles, i.e., Optimal algorithm or Fast Suboptimal algorithm.

Example

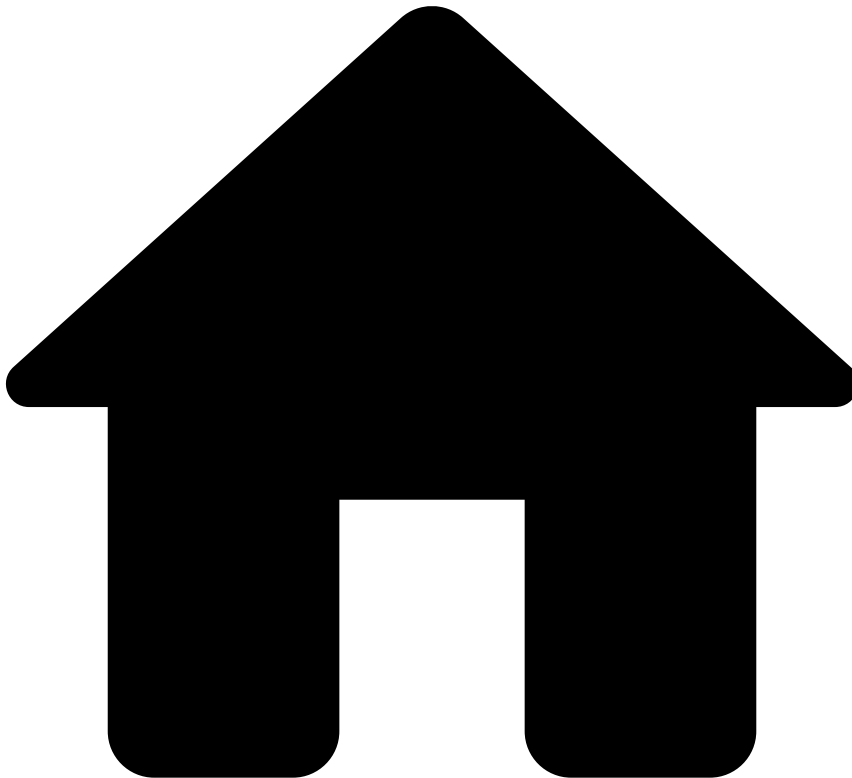
The following example sets the logging about partitioning information to info level. It needs to be added to the `logger.xml`.

Info Log Appender

```
<logger name="com.feedzai.pulse.datascience.plan.operators.impl" level="info"/>
```



•



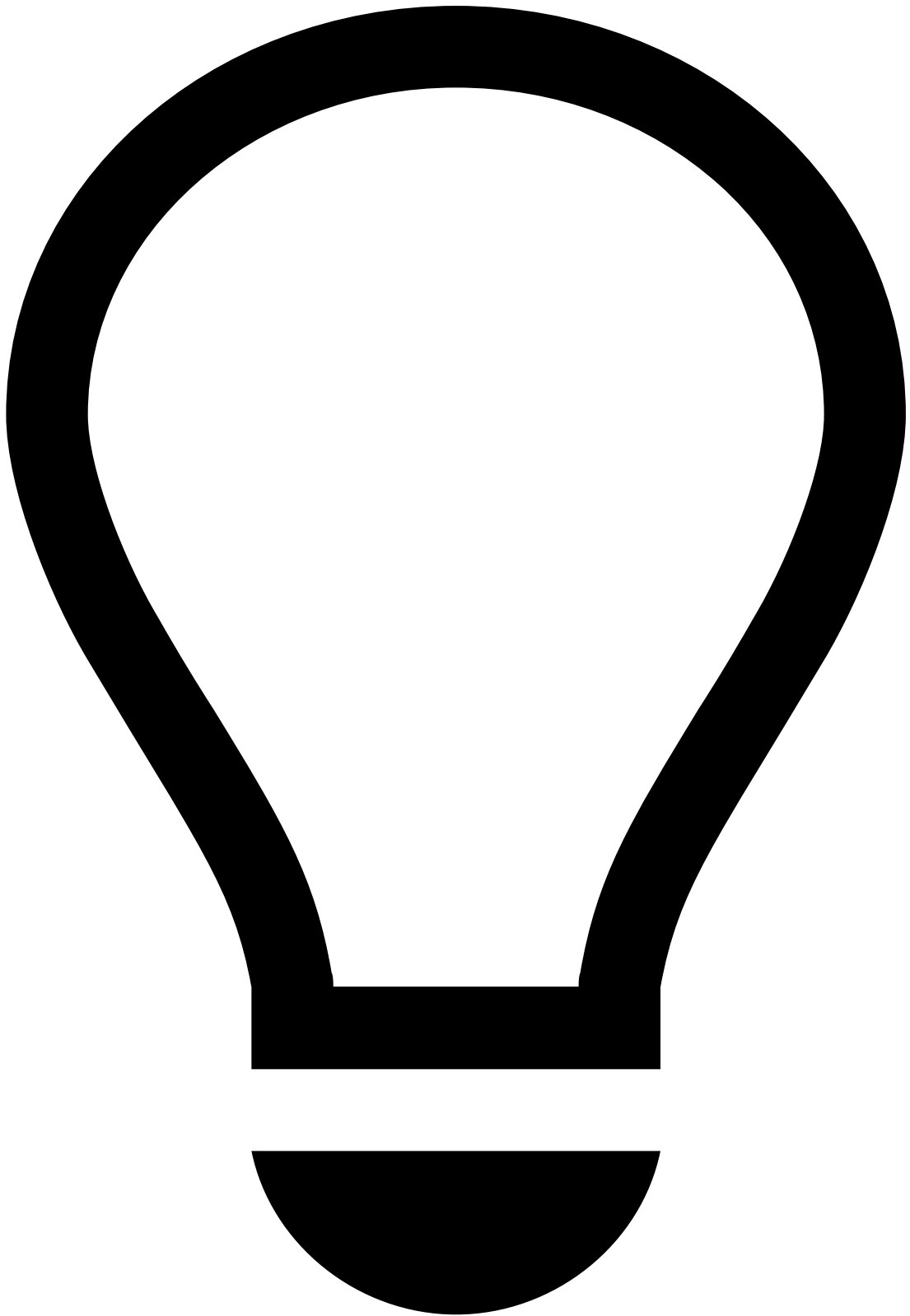
- Project Organization
- [Monitoring](#)
- [Logging](#)
- Logging Reference for Security Audits

On this page

Logging Reference for Security Audits

Was this page helpful? 

Feedzai Pulse contains three distinct logs that can be tailored to obtain the exact needs of a security audit, e.g. a PCI-DSS security audit. This section documents the relevant logging configurations in Feedzai Pulse and how to perform them, including examples and recommendations. The main goal is to allow engineers to perform specific actions more easily, such as editing the logging configuration during audits.



tip

There are three main logging approaches that may be helpful in audits:

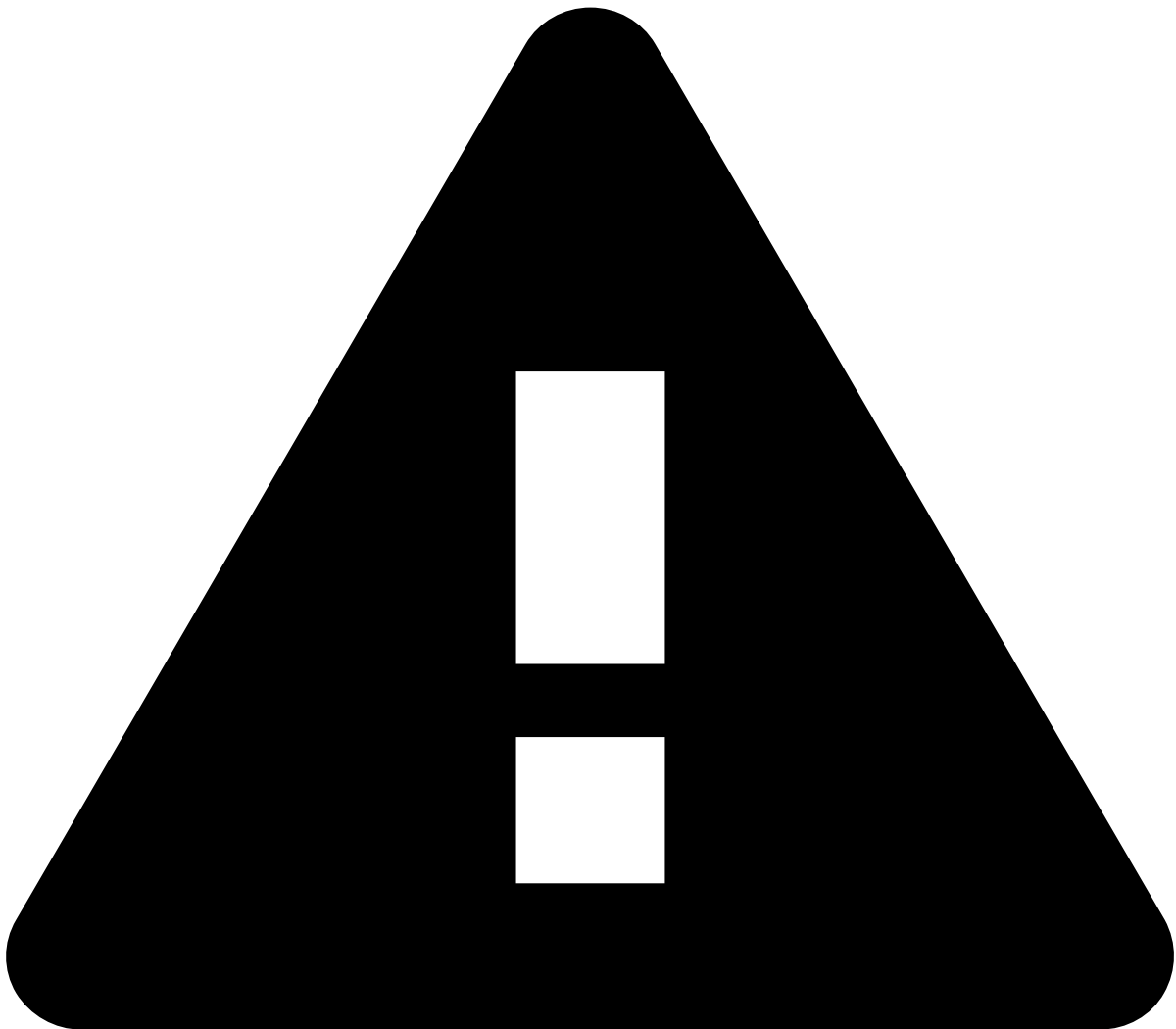
- **Security audit:** contains information usually very relevant for security audit, i.e. for PCI-DSS compliance;
- **Tracing interceptor:** includes logging at the HTTP request level;
- **Auditing:** log calls made to resource managers.

Logging🔒🔒

All operations executed internally by Pulse are accordingly logged to several levels of detail in log files located in the **PULSE_HOME/log** directory. In some specific cases, logs can also be logged into databases.

Find more information about logging and the respective configuration in the [Logging](#) page.

The logging configuration file located in **PULSE_HOME/conf/logger/logger-legacy-audit.xml** allows the configuration of **security audit** and **tracing interceptor** logs. For **auditing**, it is also necessary to edit the Pulse properties file in **PULSE_HOME/conf/pulse.properties**.



warning

It is the user's own responsibility to change the logging configuration. Changes may result in the loss of valuable information for investigating problems. If the default configuration is changed, support teams may not be able to help, therefore, the user should be extremely careful when editing this file.

Security audit logging🔒🔒

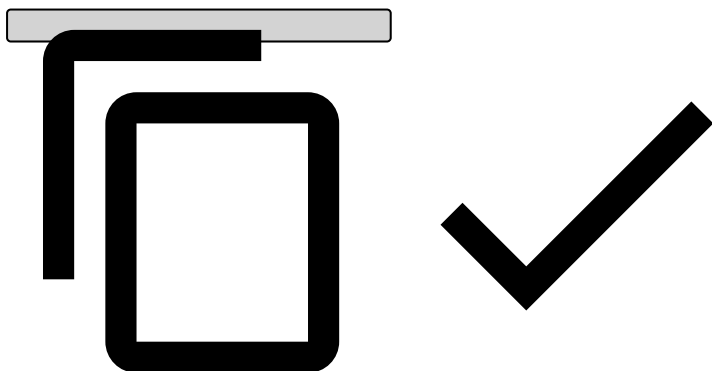
To enable the security audit file, the user must change the configuration file that is located in **PULSE_HOME/conf/logger/logger-legacy-audit.xml**.

The appenders section define where the logging messages will end up. Find more information about the different available appenders in the **Appenders** section in the [Logging](#) page.

By default, this is the code block where the security logging is disabled:

```
<!--
    Start of security audit logging.
-->
<!-- SECURITY-AUDIT LOGGING TAG
<appender name="SECURITY-AUDIT" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>log/${pulse.logcontext}-security-audit.log</file>
  <filter class="ch.qos.logback.core.filter.EvaluatorFilter">
    <evaluator class="ch.qos.logback.classic.boolex.OnMarkerEvaluator">
      <marker>SECURITY_AUDIT</marker>
    </evaluator>
    <onMismatch>DENY</onMismatch>
    <onMatch>NEUTRAL</onMatch>
  </filter>
  <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
    <level>TRACE</level>
  </filter>
  <encoder>
    <pattern>[%-65(%P{slug} %P{appId} %P{member}) %P{isLeader}] %msg%n</pattern>
  </encoder>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>log/${pulse.logcontext}-security-audit.%d{yyyy-MM}-week-%d{WW}.log.gz</
fileNamePattern>
    <maxHistory>16</maxHistory>
  </rollingPolicy>
</appender>

<logger name="com.feedzai.pulse.service.security.SecurityLogging" level="trace"
additivity="false">
  <appender-ref ref="SECURITY-AUDIT"/>
</logger>
SECURITY-AUDIT LOGGING TAG -->
<!-- End of security audit logging. -->
```



To enable it, comment in both "Security-audit logging tag" strings as shown below:

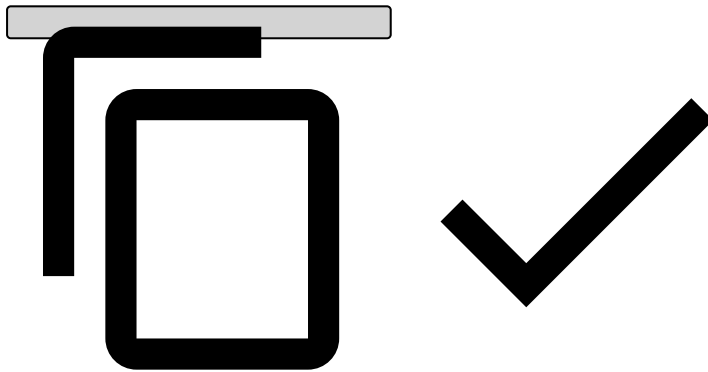
```
<!--
    Start of security audit logging.
-->
<!-- SECURITY-AUDIT LOGGING TAG -->
<appender name="SECURITY-AUDIT" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>log/${pulse.logcontext}-security-audit.log</file>
  <filter class="ch.qos.logback.core.filter.EvaluatorFilter">
    <evaluator class="ch.qos.logback.classic.boolex.OnMarkerEvaluator">
      <marker>SECURITY_AUDIT</marker>
    </evaluator>
    <onMismatch>DENY</onMismatch>
```

```

    <onMatch>NEUTRAL</onMatch>
  </filter>
  <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
    <level>TRACE</level>
  </filter>
  <encoder>
    <pattern>[%-65(%P{slug} %P{appId} %P{member}) %P{isLeader}] %msg%n</pattern>
  </encoder>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>log/${pulse.logcontext}-security-audit.%d{yyyy-MM}-week-%d{WW}.log.gz</file
NamePattern>
    <maxHistory>16</maxHistory>
  </rollingPolicy>
</appender>

<logger name="com.feedzai.pulse.service.security.SecurityLogging" level="trace"
additivity="false">
  <appender-ref ref="SECURITY-AUDIT"/>
</logger>
<!-- SECURITY-AUDIT LOGGING TAG -->
<!-- End of security audit logging. -->

```



All output will be generated in the file located in **PULSE_HOME/log/your-pulse-log-context.security-audit.log.xml**. In the above configuration, it is possible to change this file's name and location as well as other parameters, such as the logging level and compressed name file.

Events logged in the security audit [🔗](#)

User:

- USER_CREATED,
- USER_UPDATED,
- USER_DELETED,
- USER_LOGIN,
- USER_LOGIN_FAILED,
- USER_PASSWORD_CHANGE,
- USER_PASSWORD_CHANGE_FAILED,
- USER_PASSWORD_EXPIRED,
- USER_PASSWORD_MANUALLY_EXPIRED,
- USER_DISABLE_BY_INACTIVITY,
- USER_MANUALLY_DISABLED,
- USER_MANUALLY_ENABLED,
- USER_DISABLED_BY_TOO_MANY_WRONG_LOGIN_ATTEMPTS,
- USER_TWO_FACTOR_CONFIG_DELETED,

Roles:

- ROLE_CREATED,
- ROLE_UPDATED,

- ROLE_DELETED,

Tenants:

- TENANT_CREATED,
- TENANT_UPDATED,
- TENANT_DELETED

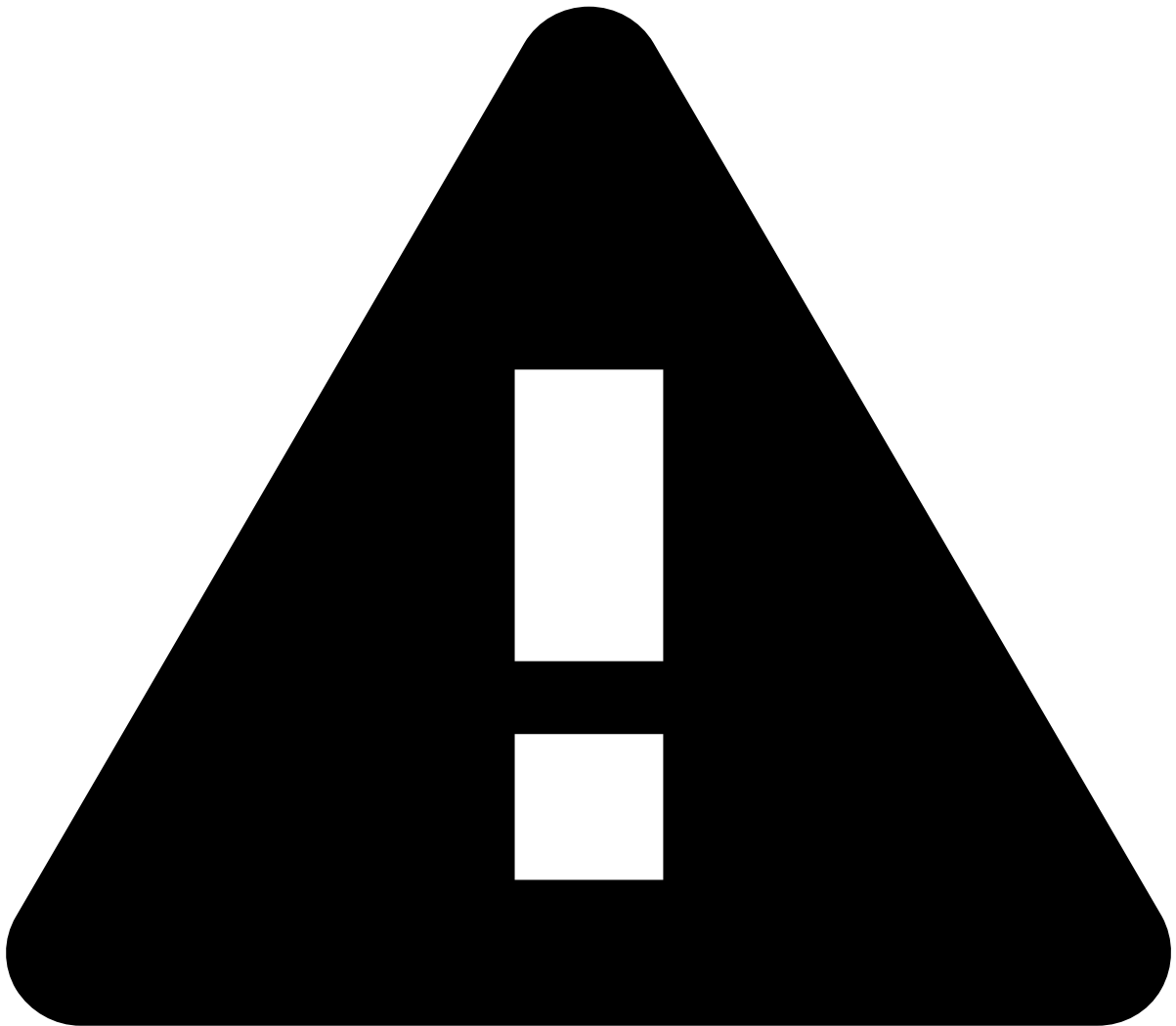
Tenant Groups:

- TENANT_GROUP_CREATED,
- TENANT_GROUP_UPDATED,
- TENANT_GROUP_DELETED

Groups:

- GROUP_CREATED,
- GROUP_UPDATED,
- GROUP_DELETED

Tracing interceptor logging



caution

Note that the tracing interceptor logging is very wordy and the log files will grow at a very fast rate. Enable this logging only if strictly necessary.

By using the tracing interceptor, you enable the logging of events at the level of HTTP requests.

It is possible to enable the logging of events by adding the following line to the **PULSE_HOME/conf/logger/logger-project.xml** configuration file:

```
<logger name="com.feedzai.pulse.web.api.common.exceptions.TracingInterceptor" level="trace"/>
```

Auditing logging🔒



caution

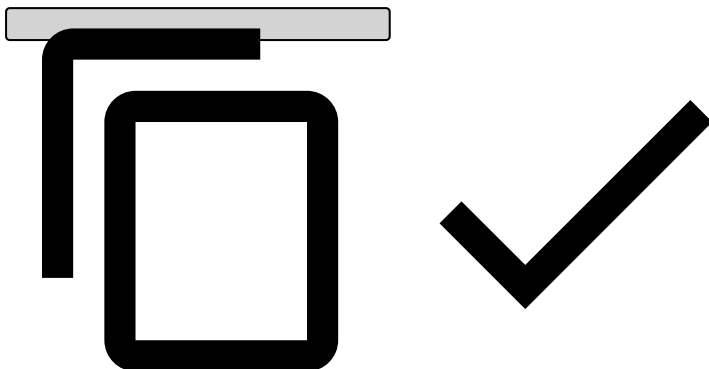
Note that the **auditing logging** is very wordy and the log files will grow at a very fast rate. Enable this logging only if strictly necessary.

The purpose of auditing is to keep a log of calls that are made to resource managers. To enable it, follow the instructions in the Auditing page.

The configuration below represents a specific security audit whose purpose is to enable logging to a file. To do so, perform the following changes displayed in **pulse.properties**, as well as in the **PULSE_HOME/conf/logger/logger-legacy-audit.xml** file.

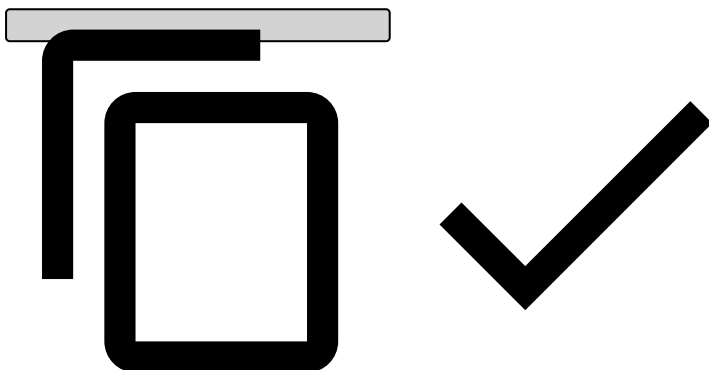
pulse.properties

```
pulse.manager.auditing.logger.name=auditing
pulse.manager.auditing.logger.level=trace
```

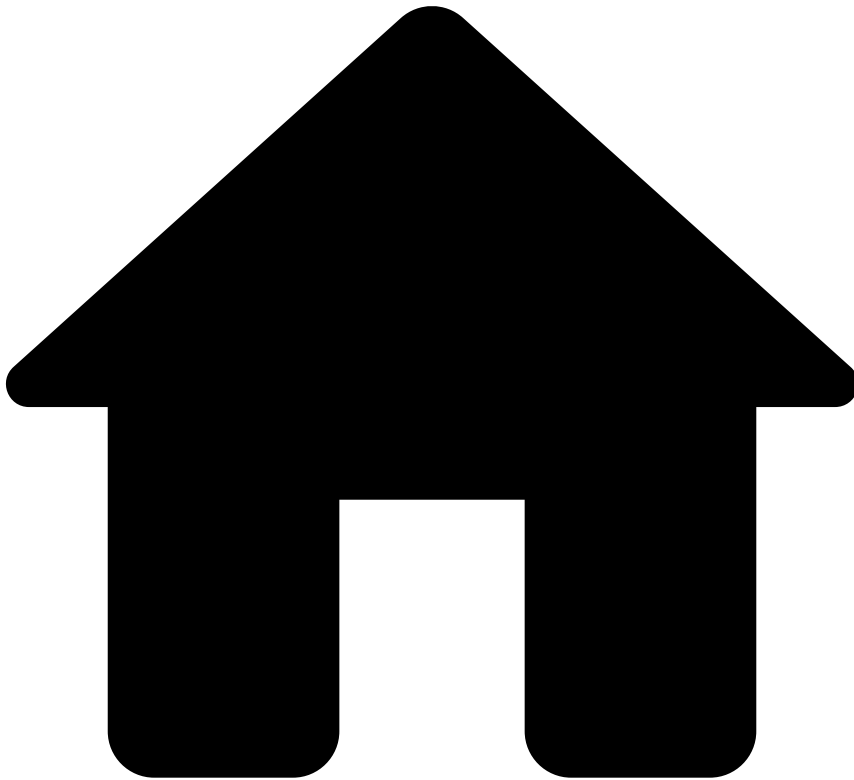


logger.xml

```
<!-- Start of auditing logging. -->
<appender name="AUDITING" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>log/${pulse.logcontext}-auditing.log</file>
  <encoder>
    <pattern>%msg</pattern>
  </encoder>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <!-- Daily rollover with gzip compression for the archives, keeping up to 30 -->
    <fileNamePattern>log/${pulse.logcontext}-auditing.%d.log.gz</fileNamePattern>
    <maxHistory>30</maxHistory>
  </rollingPolicy>
  <encoder>
    <pattern>%-5level %d{ISO8601} [%-65(%P{slug} %P{appId} %P{member}) %P{isLeader}]
    %-40([%thread]) %-65(%logger{0}).%M:%line) %msg%n%Ex{full}</pattern>
  </encoder>
</appender>
<appender name="AUDITING-ASYNC" class="ch.qos.logback.classic.AsyncAppender">
  <appender-ref ref="AUDITING" />
  <queueSize>8192</queueSize>
  <discardingThreshold>0</discardingThreshold>
  <includeCallerData>false</includeCallerData>
</appender>
<logger name="auditing" level="trace" additivity="false">
  <appender-ref ref="AUDITING-ASYNC"/>
</logger>
<!-- End of auditing logging. -->
```



•



- Project Organization
- [Monitoring](#)
- [Logging](#)
- Set Up Microsoft LightGBM Logging

On this page

Set Up Microsoft LightGBM Logging

Was this page helpful? 

To log information about what happens when training a model with the LightGBM provider, you can configure the logging levels from the most basic "INFO", up to the most extreme level "TRACE".

Logging levels

The LightGBM provider has several modules, each with different responsibilities. You can control the logging level per module.

Module name	Description	Logging level description
LightGBMBinaryClassificationModel	Implements the OpenML class responsible for loading a model and scoring instances. It is also called when saving the model to disk.	ERROR: Report that saving model to disk failed.
LightGBMBinaryClassificationModelTrainer	Responsible for training a LightGBM model. At the end of the train it saves an intermediate representation to disk that is then passed to LightGBMBinaryClassificationModel for the final model save to disk.	• INFO: Information messages reporting the stage at which the training is. • ERROR: For errors during train. • DEBUG: Report #train-instances, training parameters and features, more details on the sub-stages of training, progress in number of train iterations.
LightGBMModelCreator	Starts all load/fit operations and validations prior to starting the aforementioned model load/fit process.	• INFO: Message with filepath from which the model is being loaded. • ERROR: Errors when training the model or the validations for a model fit/load fail and where it is it trying to read from.
LightGBMSWIG	Low-level implementation for model scoring.	• INFO: Saving model to disk calls. • ERROR: Errors saving model to disk. • DEBUG: Saving model to disk calls with file paths. • TRACE: Show prediction score per instance. Note that this level is very verbose!
LightGBMUtils	Loads the LightGBM libraries.	• INFO: When the libraries are loaded. • DEBUG: Names of the libraries being loaded.
SWIGResources	Lowest-level implementation level for resources handling during model scoring.	• DEBUG: In-memory instantiation of model. • TRACE: Resource release calls.
SWIGTrainResources	Lowest-level implementation level for resources handling during model fitting.	DEBUG: Details for train memory allocation.

How to set up logging🔗

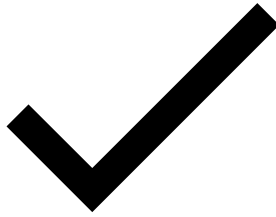
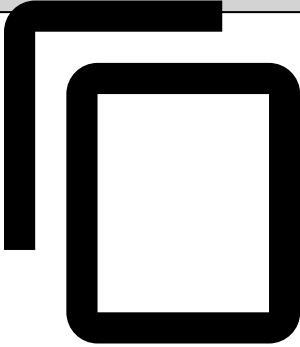
To configure such log levels, edit the file `conf/logger.xml`. Also, for Spark Standalone `spark-lib/logback-executors.xml` should also have such log levels.

Example configuration🔗

An example of configuring for maximum log level follows. Feel free to copy-paste, but beware of having too much logging enabled (see table above):

`conf/logger.xml` & `spark-lib/logback-executors.xml`

```
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMBinaryClassificationModel" level="trace"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMBinaryClassificationModelTrainer" level="trace"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMModelCreator" level="trace"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMSWIG" level="trace"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMUtils" level="trace"/>
<logger name="com.feedzai.openml.provider.lightgbm.SWIGResources" level="trace"/>
<logger name="com.feedzai.openml.provider.lightgbm.SWIGTrainResources" level="trace"/>
```



•



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring Cassandra Driver Sessions

On this page

Monitoring Cassandra Driver Sessions

Was this page helpful? 

Multiple Pulse components can be configured to work with a Cassandra database, such as the Profile Store and the Event Storage. The `/metrics` API exposes several metrics that allow Pulse operators to monitor the Cassandra drivers that these different components use to communicate with the Cassandra cluster(s).

This page describes how to configure the monitoring of these drivers, as well as the available metrics.

Cassandra drivers and their configuration properties^â

Within a Pulse App Engine, several Cassandra drivers can be created depending on the defined configurations. If a Cassandra connection configuration is provided at the global level, a Cassandra driver may be created at the App Engine level, which can then be used by the multiple components within the App Engine, as long as the Cassandra connection configuration of these components is either not provided (defaults to global) or equal to the one defined at the global scope. Otherwise, the component itself will create its own Cassandra driver.

The prefix of the Cassandra driver metric names, as well as the metric tags, changes depending on whether the driver is created by the App Engine or by a specific component within the App Engine:

Metric Name Prefix	Description	Tags
<code>pulse.cassandra.driver.stats</code>	For the Cassandra driver that can be created by the App Engine.	<i>appSlug</i> <i>metricType</i> <i>appType</i>
<code>pulse.workflow.cassandra.driver.stats</code>	For the Cassandra driver that can be created within the scope of a given workflow component.	<i>appSlug</i> <i>metricType</i> <i>appType</i> <i>workflowId</i> <i>workflowDesc</i> <i>elementId</i> <i>elementDesc</i> <i>context</i>

Available metrics^â

The following table lists all the metrics that are exposed for each Cassandra driver:

Name	Description	Unit
Counters		
<code><prefix>.blocking-executor-queue-depth</code>	The total number of queued-up tasks in the Cassandra driver's blocking executor.	#
<code><prefix>.connect-to</code>	The total number of Cassandra hosts the driver is currently connected to.	#
<code><prefix>.executor-queue-depth</code>	The total number of queued-up tasks in the Cassandra driver's main internal executor.	#
<code><prefix>.inflight-requests</code>	The total number of inflight requests to Cassandra hosts.	#
<code><prefix>.known-hosts</code>	The total number of Cassandra hosts currently known by the driver, regardless of whether the hosts are up or down.	#
<code><prefix>.open-connections</code>	The total number of currently open connections to Cassandra hosts.	#
<code><prefix>.reconnection-scheduler-task-count</code>	The total number of queued up tasks in the Cassandra driver's reconnection executor.	#
<code><prefix>.request-queue-depth</code>	The total number of enqueued requests on all Cassandra hosts.	#
	The total number of queued-up tasks in the Cassandra driver's scheduled tasks executor.	#

<prefix>.task-scheduler-task-count		
<prefix>.trashed-connections	The total number of currently trashed connections to Cassandra hosts.	#
Gauges		
<prefix>.authentication-errors	The number of authentication errors while connecting to Cassandra nodes.	#
<prefix>.client-timeouts	The number of requests that timed out before the driver received a response.	#
<prefix>.connection-errors	The number of errors while connecting to Cassandra nodes.	#
<prefix>.ignores	The number of times a request was ignored due to the retry policy, for instance, due to timeouts or unavailability.	#
<prefix>.ignores-on-client-timeout	The number of times a request was ignored due to the retry policy after a client timeout.	#
<prefix>.ignores-on-connection-error	The number of times a request was ignored due to the retry policy after a connection error.	#
<prefix>.ignores-on-other-errors	The number of times a request was ignored due to the retry policy after an unexpected error.	#
<prefix>.ignores-on-read-timeout	The number of times a request was ignored due to the retry policy after a read timed out.	#
<prefix>.ignores-on-unavailable	The number of times a request was ignored due to the retry policy after an unavailable exception.	#
<prefix>.ignores-on-write-timeout	The number of times a request was ignored due to the retry policy after a write timed out.	#
<prefix>.other-errors	The number of requests that returned errors not accounted for by another metric.	#
<prefix>.read-timeouts	The number of read requests that returned a timeout (independently of the final decision taken by the retry policy).	#
<prefix>.retries	The number of times a request was retried due to the retry policy.	#
<prefix>.retries-on-client-timeout	The number of times a request was retried due to the retry policy after a client timeout.	#
<prefix>.retries-on-connection-error	The number of times a request was retried due to the retry policy after a connection error.	#
<prefix>.retries-on-other-errors	The number of times a request was retried due to the retry policy after an unexpected error.	#

<code><prefix>.retries-on-read-timeout</code>	The number of times a request was retried due to the retry policy after a read timed out.	#
<code><prefix>.retries-on-unavailable</code>	The number of times a request was retried due to the retry policy after an unavailable exception.	#
<code><prefix>.retries-on-write-timeout</code>	The number of times a request was retried due to the retry policy after a write timed out.	#
<code><prefix>.speculative-executions</code>	The number of times a speculative execution was started because a previous execution did not complete within the delay specified by speculative execution policy.	#
<code><prefix>.unavailables</code>	The number of requests that returned an unavailable exception (independently of the final decision taken by the retry policy).	#
<code><prefix>.write-timeouts</code>	Returns the number of write requests that returned a timeout (independently of the final decision taken by the retry policy).	#
Meters		
<code><prefix>.bytes-received</code>	The number of bytes received by the Cassandra driver so far.	#
<code><prefix>.bytes-sent</code>	The number of bytes sent by the Cassandra driver so far.	#
Timers		
Metrics regarding the number of user requests performed on the Cassandra cluster: • The total number of requests. • The requests rate (in requests per second), including 1, 5 and 15-minute rates. • The mean, min and max latencies, as well as latency at a given percentile.		

API response example

The following is an excerpt of a response from a `/metrics` call focusing on the Cassandra driver monitoring metrics:

[Click here to expand...](#)

```
{
  "version": "3.1.3",
  "gauges": {
    "pulse.cassandra.driver.stats.blocking-executor-queue-
depth,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "value": 0
    },
    "pulse.cassandra.driver.stats.connected-
to,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "value": 1
    },
    "pulse.cassandra.driver.stats.executor-queue-
depth,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "value": 0
    },
    "pulse.cassandra.driver.stats.inflight-
requests,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
```

```

        "value": 0
    },
    "pulse.cassandra.driver.stats.known-
hosts,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 1
    },
    "pulse.cassandra.driver.stats.open-
connections,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 2
    },
    "pulse.cassandra.driver.stats.reconnection-scheduler-task-
count,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 0
    },
    "pulse.cassandra.driver.stats.request-queue-
depth,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 0
    },
    "pulse.cassandra.driver.stats.task-scheduler-task-
count,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 1
    },
    "pulse.cassandra.driver.stats.trashed-
connections,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 0
    }
},
"counters": {
    "pulse.cassandra.driver.stats.authentication-
errors,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.client-
timeouts,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.connection-
errors,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.ignores,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APP
PLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.ignores-on-client-
timeout,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.ignores-on-connection-
error,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.ignores-on-other-
errors,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.ignores-on-read-
timeout,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    },
    "pulse.cassandra.driver.stats.ignores-on-
unavailable,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "count": 0
    }
},

```



```

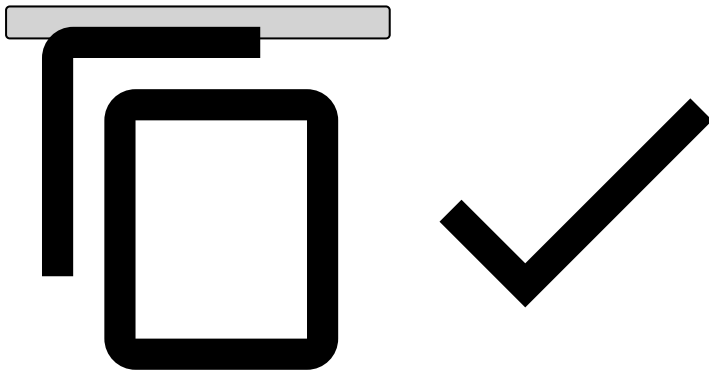
    "pulse.cassandra.driver.stats.ignores-on-write-
timeout,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.other-
errors,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.read-
timeouts,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APP
PLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries-on-client-
timeout,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries-on-connection-
error,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries-on-other-
errors,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries-on-read-
timeout,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries-on-
unavailable,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.retries-on-write-
timeout,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.speculative-
executions,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.unavailables,appSlug=myapp,metricType=APPLICATION,appType=PULS
E_APPLICATION": {
      "count": 0
    },
    "pulse.cassandra.driver.stats.write-
timeouts,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 0
    }
  },
  "meters": {
    "pulse.cassandra.driver.stats.bytes-
received,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "count": 70085,
      "m15_rate": 13655.832257436685,
      "m1_rate": 10008.181863718019,
      "m5_rate": 13062.761137000047,
      "mean_rate": 2685.961431360291,
      "units": "events/second"
    },
    "pulse.cassandra.driver.stats.bytes-

```

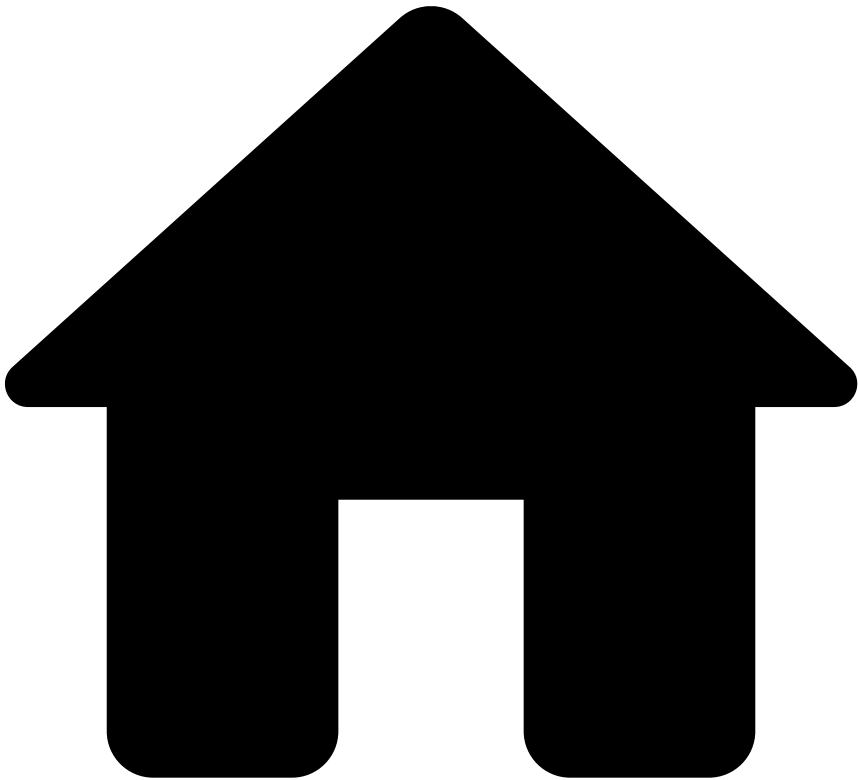
```

sent,appSlug=myapp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
  "count": 26621,
  "m15_rate": 2357.49710743481,
  "m1_rate": 1913.6517203981277,
  "m5_rate": 2285.3326489363553,
  "mean_rate": 1020.2309350480018,
  "units": "events/second"
},
"timers": {
  "pulse.cassandra.driver.stats.requests,appSlug=myapp,metricType=APPLICATION,appType=PULSE_AP
PLICATION": {
    "count": 52,
    "max": 1.1036750240000002,
    "mean": 0.08373713328355051,
    "min": 0.0025661990000000003,
    "p50": 0.007133643,
    "p75": 0.0085934430000000001,
    "p95": 0.80259582300000001,
    "p98": 0.97049927,
    "p99": 1.1036750240000002,
    "p999": 1.1036750240000002,
    "stddev": 0.23199159630369318,
    "m15_rate": 6.085718936919922,
    "m1_rate": 4.7259628149837045,
    "m5_rate": 5.864636322164412,
    "mean_rate": 1.9928190509216706,
    "duration_units": "seconds",
    "rate_units": "calls/second"
  }
}

```



•



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring DynamoDB Profile Store

Monitoring DynamoDB Profile Store

Was this page helpful?

When your application has workflows with scheduled metrics, they fetch the metric values from the profile store.

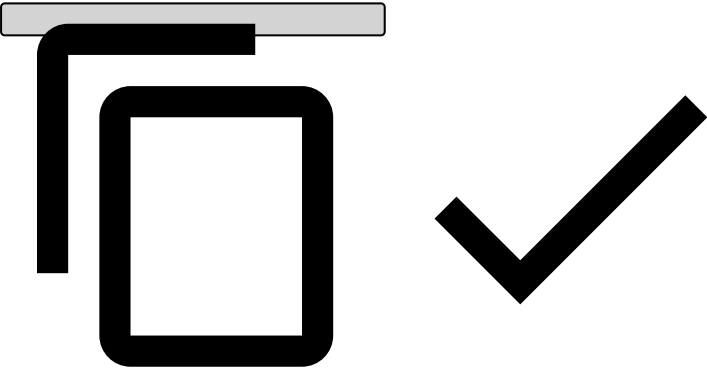
If your profile store uses DynamoDB, it's possible to access the following monitoring metrics.

Name	Description	Unit	Tags
Timers			
<code>pulse.workflow.profileStore.waitWorker</code>	Time waiting for a thread to handle the requests to DynamoDB. The number of threads is configurable via <code>profileStore.selectorThreads</code> .	seconds	<code>workflowID</code> <code>workflowDesc</code> <code>appSlug</code> <code>metricType</code>

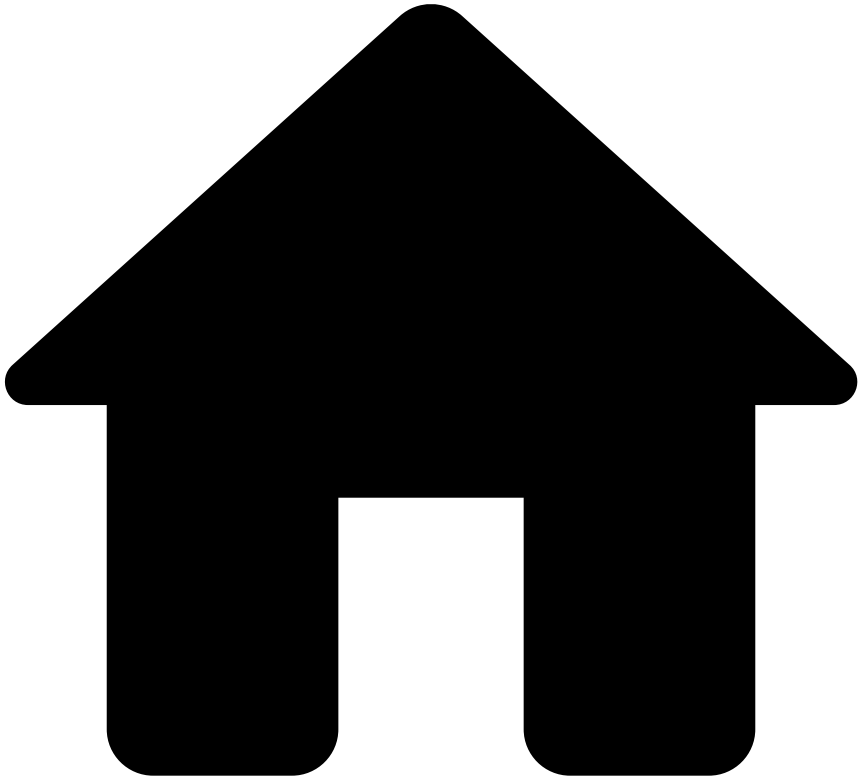
		<i>appType</i> <i>elementId</i> <i>elementDesc</i>
--	--	--

An example of the metrics:

```
"pulse.workflow.profileStore.waitWorker,elementId=4c030bd0-b184-47aa-9b5d-c0c451e16116,appSlug=MyApp,metricType=APPLICATION,appType=PULSE_APPLICATION,workflowDesc=MyWorkflow,workflowId=rya73kj3TXuu5IeathNIXg,elementDesc=MyWorkflowPlan": {  
  "count": 30,  
  "max": 0.060647473,  
  "mean": 0.015117776753910768,  
  "min": 0.006571861,  
  "p50": 0.010779865000000001,  
  "p75": 0.012081169000000001,  
  "p95": 0.056311958,  
  "p98": 0.060647473,  
  "p99": 0.060647473,  
  "p999": 0.060647473,  
  "stddev": 0.0131817571588833,  
  "m15_rate": 0.4164373137301911,  
  "m1_rate": 0.6044556218797338,  
  "m5_rate": 0.44797335122240606,  
  "mean_rate": 0.9025331257980914,  
  "duration_units": "seconds",  
  "rate_units": "calls/second"  
}
```



•



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring Event Storage Dumps in Pulse

On this page

Monitoring Event Storage Dumps in Pulse

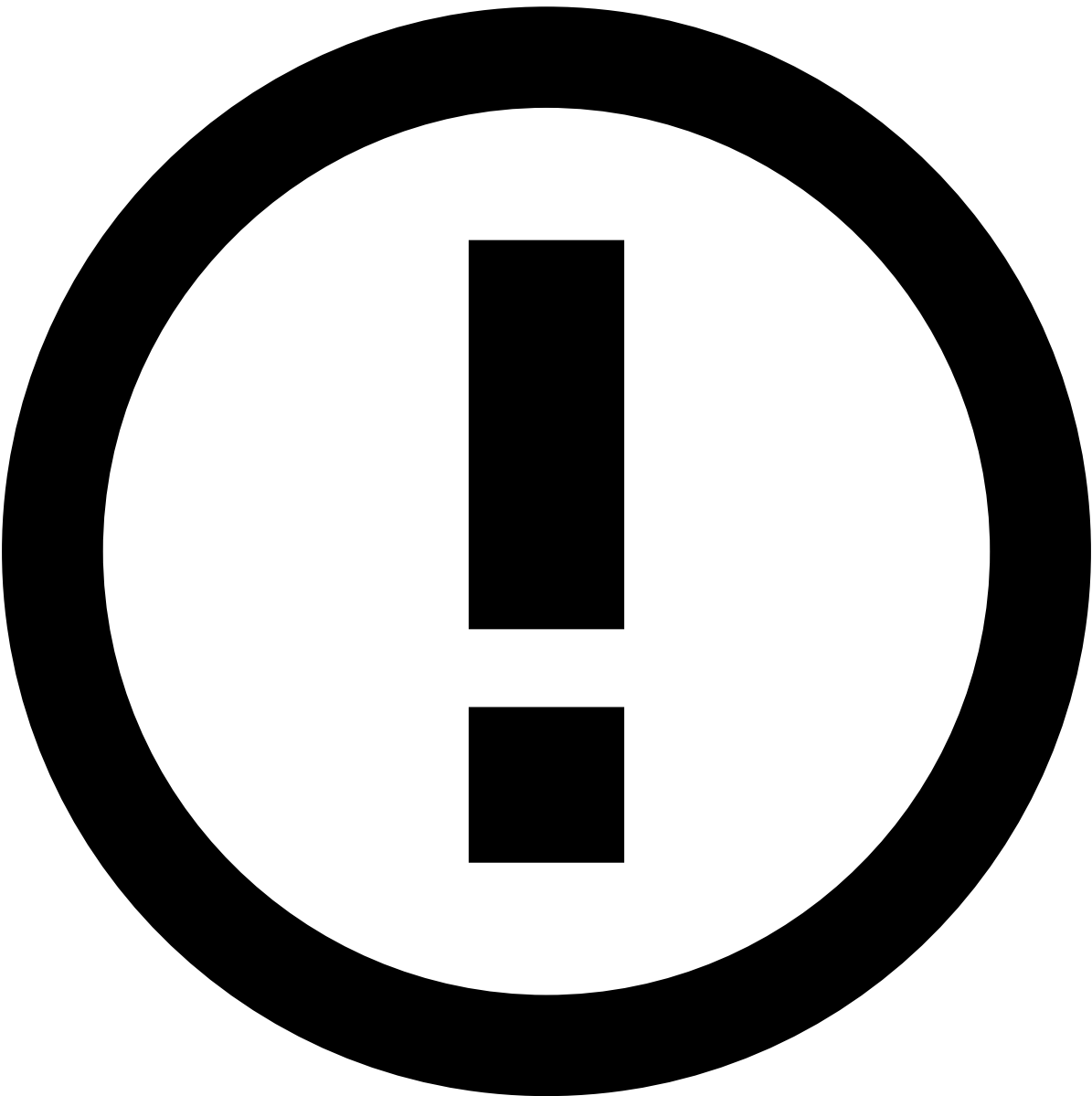
Was this page helpful? 

This page describes how to monitor the dumping of events when they fail to be written to the Event Storage in Pulse using the `/metrics` endpoint.

Introduction

In Pulse, each Workflow has it own Event Storage. When events reach the end of the workflow they are persisted in the Event Storage. If the Event Storage isn't avaiable (e.g.: Database is offline) the events are dumped to disk and are reprocessed later on by the Event Storage Dump component.

It is essential to know if this component is operating within reasonable limits. By taking monitoring metrics from the Event Storage Dump, Pulse operators can have access to different information related to event dumps operations via the `/metrics` endpoint.



info

To enable the component that reports this metrics, reach out to the Feedzai Customer Success (CS) team.

Metrics📄

The metrics are associated with an application, identifiable via an `appSlug` tag, where the service is running.

The `appType` and `metricType` tags, that are also associated with these monitoring metrics, are common to other metrics exposed in Pulse and App Engines. They inform where the metrics were collected (Pulse Server or App Engine) and what type of metric it is being reported (e.g. APPLICATION), respectively.

Name	Description	Unit	Possible Values	Tags
------	-------------	------	-----------------	------

Gauges				
pulse.dump.storage.retry.queue.size	The total number of events that are present in the in-memory queue to be written to the dump file in the retry folder.	#	>= 0	appSlug metricType appType workflowID workflowDesc
pulse.dump.storage.failed.queue.size	The total number of events that are present in the in-memory queue to be written to the dump file in the failed folder.	#	>= 0	appSlug metricType appType workflowID workflowDesc
pulse.dump.storage.recovery.job	Whether the auto recovery job that reprocesses events dumped into the retry folder, is running or not.	Boolean	true - Job is running false - Job is not running	appSlug metricType appType workflowID workflowDesc
Counters				
pulse.dump.storage.dump.entries.retry.counter	The total number of events that are currently stored in the disk to be retried by the auto recovery job.	#	>= 0	appSlug metricType appType workflowID workflowDesc
pulse.dump.storage.dump.entries.failed.counter	The total number of events that are currently stored in the disk but that already reached the maximum amounts of retry, therefore are no longer reprocessed and need to be recovered manually.	#	>= 0	appSlug metricType appType workflowID workflowDesc

Response

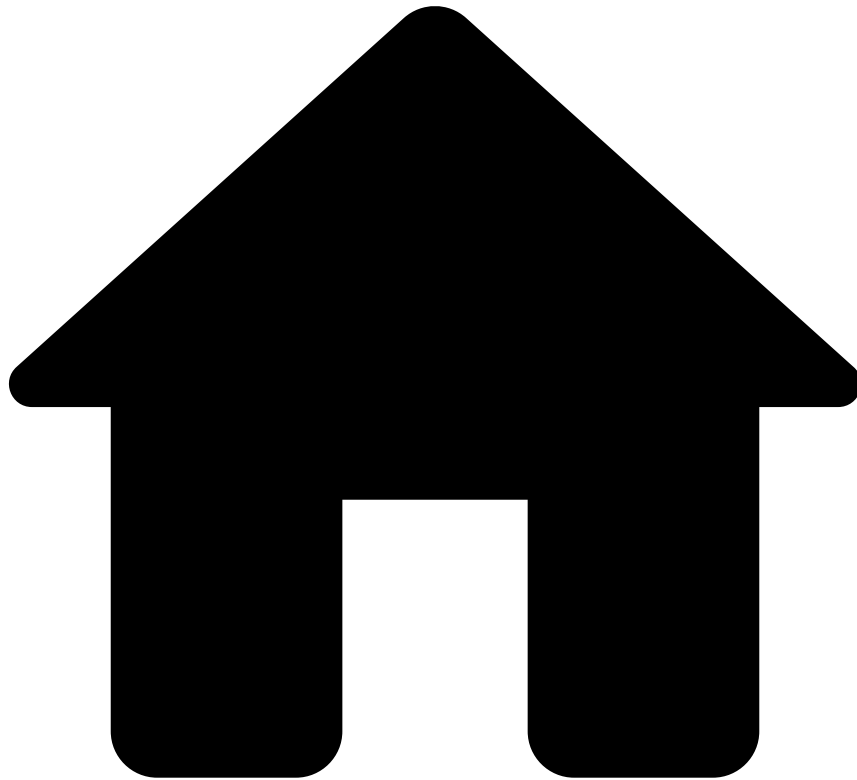
Here you can see an excerpt of a response from a `/metrics` call focusing on the Event Storage Dump monitoring metrics provided by Pulse:

[Click here to expand...](#)

```
{
  "version": "3.1.3",
  "gauges": {
    "pulse.dump.storage.retry.queue.size,appSlug=,metricType=APPLICATION,workflowDesc=,appType=PULSE_APPLICATION,workflowID=": {
      "value": 0
    },
    "pulse.dump.storage.failed.queue.size,appSlug=,metricType=APPLICATION,workflowDesc=,appType=PULSE_APPLICATION,workflowID=": {
      "value": 0
    },
    "pulse.dump.storage.recovery.job,appSlug=,metricType=APPLICATION,workflowDesc=,appType=PULSE_APPLICATION,workflowID=": {
      "value": false
    }
  },
  "counters": {
    "pulse.dump.storage.dump.entries.retry.counter,appSlug=,metricType=APPLICATION,workflowDesc=,appType=PULSE_APPLICATION,workflowID=": {
      "count": 0
    },
    "pulse.dump.storage.dump.entries.failed.counter,appSlug=,metricType=APPLICATION,workflowID=": {
      "count": 0
    }
  }
}
```

```
Desc=,appType=PULSE_APPLICATION,workflowID=": {  
    "count": 0  
  }  
},  
"histograms": {},  
"meters": {},  
"timers": {}  
}  
}
```


•



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring the Persistence of Events in the Workflow

On this page

Monitoring the Persistence of Events in the Workflow

Was this page helpful? 

When your Application is published with one or more workflows executing, there are various storages to which events' data can be persisted.

The following sections detail the metrics available for each one of them.

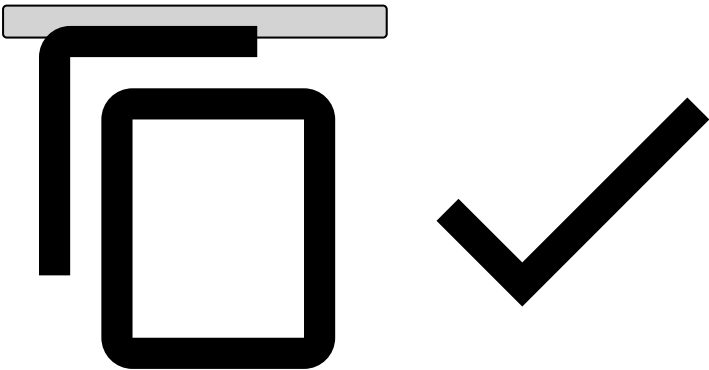
Workflow Event Storage

Each Workflow has 1 connector to the event storage (either to a relational or Cassandra database), used for storing events near the end of processing and to recover via replay.

Name	Description	Unit	Tags
Counters			
pulse.workflow.eventstorage.eventsQueuedSize	The number of events pending in-memory to be persisted to the storage.	#	workflowID appSlug metricType appType

Example of live metrics

```
"pulse.workflow.eventstorage.eventsQueuedSize,appSlug=nank,metricType=APPLICATION,appType=PULSE_APPLICATION,workflowID=pBqsDBqu114nuQhbTKLPjw":{
  "value":92
}
```



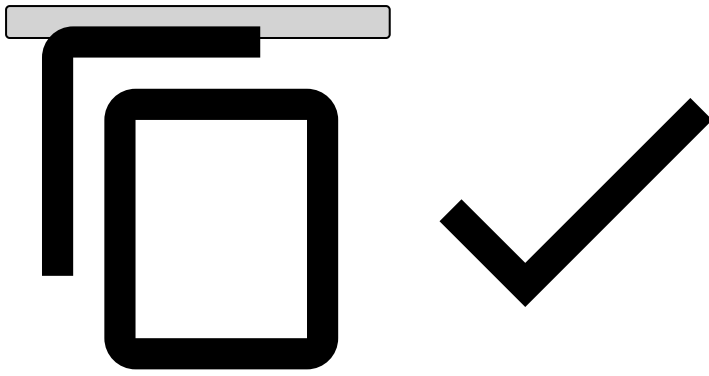
Self Service Rules Testing Event Storage

In this case, there is 1 connector to an SSR Event Storage for each Rules Element in the workflow that is running a snapshot for simple rules.

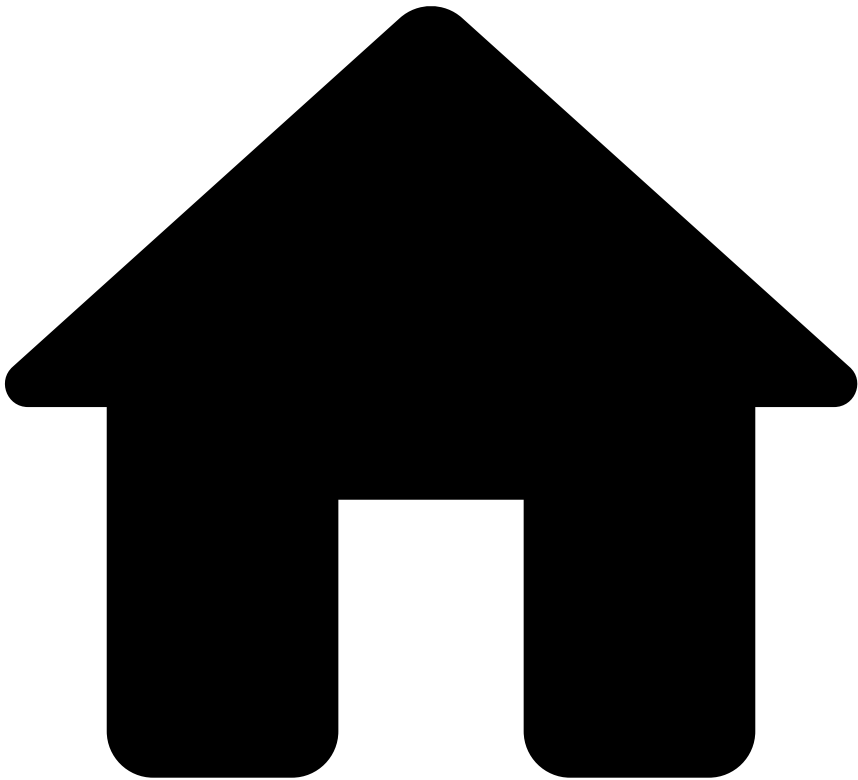
Name	Description	Unit	Tags
Counters			
pulse.workflow.rules.selfServiceRulesTestingStorage.eventsQueuedSize	The number of events pending in-memory to be persisted to the storage.	#	workflowID workflowElementID appSlug metricType appType

Example of live metrics

```
"pulse.workflow.rules.selfServiceRulesTestingStorage.eventsQueuedSize,elementId=1f8204a3-43d0-4031-9783-fa0200939522,appSlug=bank,metricType=APPLICATION,appType=PULSE_APPLICATION,workflowDesc=payments,workflowId=pBqsDBqu114nuQhbTKLPjw,elementDesc=BlackRules":{
  "value":32
}
```



•



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring Processing of Feedback Records in Pulse

On this page

Was this page helpful? 

Monitoring Processing of Feedback Records in Pulse

This page describes how to monitor the processing of feedback records in Pulse using the `/metrics` endpoint.

Introduction

Feedback records are a core concept of Pulse that allow users to provide feedback about past decisions from the system, e.g. when a transaction was previously marked as not fraudulent, but was actually fraudulent. This mechanism allows:

- Data Scientists to re-iterate over the metrics, rules and Machine Learning models used by the workflow, and use the feedback to iteratively improve the performance of the system.

- Users to evaluate the performance of the system by comparing the decisions of Pulse with the received feedback.

The Feedback Storage Service is the service responsible to manage all the CRUD operations related with feedback records.

It is essential to know if this service is operating within reasonable limits. By taking monitoring metrics from the Feedback Storage Service, Pulse operators can have access to different information related to feedback operations via the `/metrics` endpoint.

Metrics

The metrics are associated with an application, identifiable via an `appSlug` tag, where the service is running.

The `appType` and `metricType` tags, that are also associated with these monitoring metrics, are common to other metrics exposed in Pulse and App Engines. They inform where the metrics were collected (Pulse Server or App Engine) and what type of metric it is being reported (e.g. APPLICATION), respectively.

Name	Description	Unit	Possible Values	Tags
Gauges				
<code>pulse.feedbackstorage.status</code>	The current status of the Feedback Storage Service. This state is computed by looking at the current state of the internal components of the service. This means that: • If all components are UNAVAILABLE, then the service is UNAVAILABLE. • If all components are AVAILABLE, then the service is AVAILABLE. • If the components have different states then the service is DEGRADED. The DEGRADED state can happen, when for example feedback messages are dropped to the deadletter because of an issue in either the Event Storage or the Messaging Broker.	#	0 - UNAVAILABLE 1 - AVAILABLE 2 - DEGRADED	<i>appSlug</i> <i>metricType</i> <i>appType</i>
<code>pulse.feedbackstorage.feedbacks.queue.size</code>	The total number of entries that are present in the feedback messaging queue to be processed by the service.	#	≥ 0	<i>appSlug</i> <i>metricType</i> <i>appType</i>
<code>pulse.feedbackstorage.event-storage-worker-app-{appId}-wf-{workflowId}.queue.size</code>	The total number of operations pending in the executor of a given event storage. There is 1 executor per AppID-WorkflowID, the name of the metric will depend on that.	#	≥ 0	<i>appSlug</i> <i>metricType</i> <i>appType</i>
<code>pulse.feedbackstorage.available.publishers</code>	The total number of available publishers (that publish to the feedback queue on a REST/RMI request) at a given point in time.	#	> 0	<i>appSlug</i> <i>metricType</i> <i>appType</i>
Counters				
<code>pulse.feedbackstorage.deadletters.event-storage.counter</code>	The total number of events that are currently stored in a deadletter, for the event-storage inserts.	#	≥ 0	<i>appSlug</i> <i>metricType</i> <i>appType</i>
	The total number of events that are currently stored in a deadletter, for the feedback messaging publishes.	#	≥ 0	<i>appSlug</i> <i>metricType</i> <i>appType</i>

pulse.feedbackstorage.deadletters.messaging.counter				
Timers				
Timers contain the following metrics: • count: Number of events; • max: Max Latency; • mean: Mean Latency; • min: Minimum Latency; • p50: Latencies in the percentile 50; • p75: Latencies in the percentile 75; • p95: Latencies in the percentile 95; • p98: Latencies in the percentile 98; • p99: Latencies in the percentile 99; • p999: Latencies in the percentile 99.9; • stddev: Standard deviation of the latencies; • m15_rate: Rate in the last 15 minutes; • m1_rate: Rate in the last 1 minute; • m5_rate: Rate in the last 5 minutes; • mean_rate: Mean rate; • duration_units: Seconds; • rate_units: Calls/Second;				
pulse.feedbackstorage.feedbacks.consumption	The latency associated with consumption of feedback messages.	seconds	>= 0	appSlug metricType appType
pulse.feedbackstorage.feedbacks.publish	The latency associated with publishing of feedback messages.	seconds	>= 0	appSlug metricType appType
pulse.feedbackstorage.feedbacks.read	The latency associated with reads of feedback records from the event storage.	seconds	>= 0	appSlug metricType appType
pulse.feedbackstorage.feedbacks.write	The latency associated with writes of feedback records to the event storage.	seconds	>= 0	appSlug metricType appType

Response

Here you can see an excerpt of a response from a `/metrics` call focusing on the Feedback Storage Service monitoring metrics provided by Pulse:

[Click here to expand...](#)

```
{
  "version": "3.1.3",
  "gauges": {
    "pulse.feedbackstorage.available.publishers,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
      "value": 1
    },
    "pulse.feedbackstorage.event-storage-worker-app-pyRnWtaCjhc3bgXFjSpN_Q-wf-j0vmz74uFtlj0I7m_x5CYA.queue.size,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
```

```

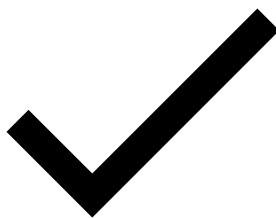
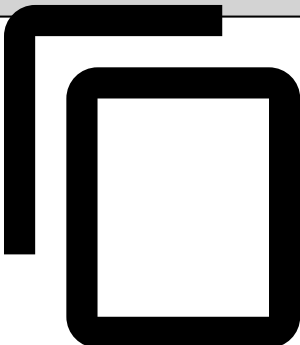
        "value": 0
    },
    "pulse.feedbackstorage.feedbacks.queue.size,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 0
    },
    "pulse.feedbackstorage.status,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
        "value": 1
    },
    },
    "counters": {
        "pulse.feedbackstorage.deadletters.event-storage.counter,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
            "count": 0
        },
        "pulse.feedbackstorage.deadletters.messaging.counter,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
            "count": 0
        }
    },
    "histograms": {},
    "meters": {},
    "timers": {
        "pulse.feedbackstorage.feedbacks.consumption,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
            "count": 30,
            "max": 0.060647473,
            "mean": 0.015117776753910768,
            "min": 0.006571861,
            "p50": 0.010779865000000001,
            "p75": 0.012081169000000001,
            "p95": 0.056311958,
            "p98": 0.060647473,
            "p99": 0.060647473,
            "p999": 0.060647473,
            "stddev": 0.0131817571588833,
            "m15_rate": 0.4164373137301911,
            "m1_rate": 0.6044556218797338,
            "m5_rate": 0.44797335122240606,
            "mean_rate": 0.9025331257980914,
            "duration_units": "seconds",
            "rate_units": "calls/second"
        },
        "pulse.feedbackstorage.feedbacks.publish,appSlug=FeedbackStorageApp,metricType=APPLICATION,appType=PULSE_APPLICATION": {
            "count": 30,
            "max": 0.060647473,
            "mean": 0.015117776753910768,
            "min": 0.006571861,
            "p50": 0.010779865000000001,
            "p75": 0.012081169000000001,
            "p95": 0.056311958,
            "p98": 0.060647473,
            "p99": 0.060647473,
            "p999": 0.060647473,
            "stddev": 0.0131817571588833,
            "m15_rate": 0.4164373137301911,
            "m1_rate": 0.6044556218797338,
            "m5_rate": 0.44797335122240606,
            "mean_rate": 0.9025331257980914,
            "duration_units": "seconds",
            "rate_units": "calls/second"
        }
    },
    },

```

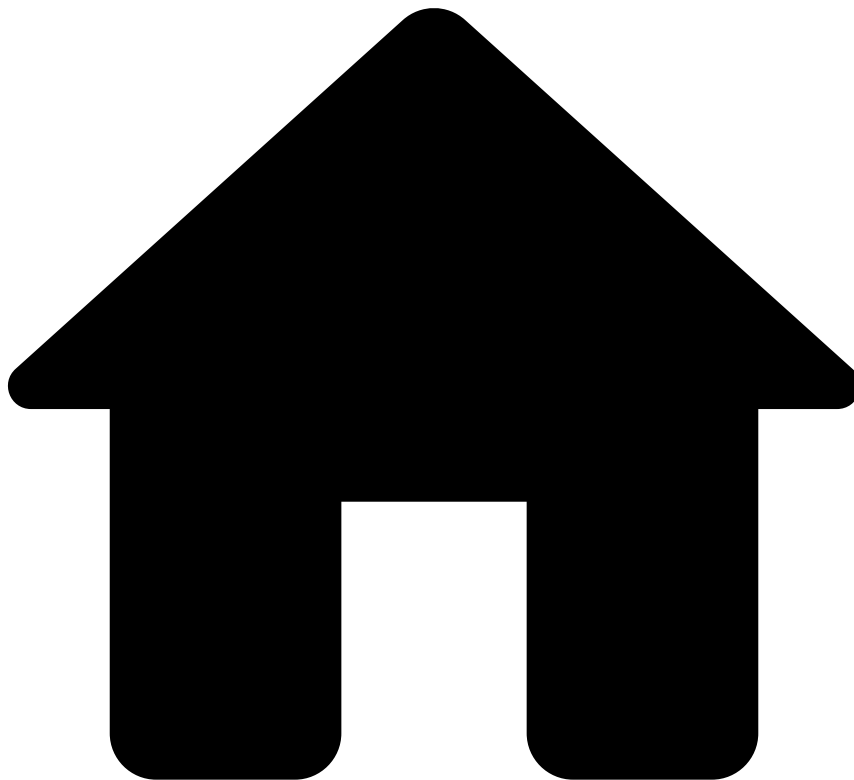
```

    "pulse.feedbackstorage.feedbacks.read,appSlug=FeedbackStorageApp,metricType=APPLICATION,
appType=PULSE_APPLICATION": {
      "count": 0,
      "max": 0,
      "mean": 0,
      "min": 0,
      "p50": 0,
      "p75": 0,
      "p95": 0,
      "p98": 0,
      "p99": 0,
      "p999": 0,
      "stddev": 0,
      "m15_rate": 0,
      "m1_rate": 0,
      "m5_rate": 0,
      "mean_rate": 0,
      "duration_units": "seconds",
      "rate_units": "calls/second"
    },
    "pulse.feedbackstorage.feedbacks.write,appSlug=FeedbackStorageApp,metricType=APPLICATION,
appType=PULSE_APPLICATION": {
      "count": 30,
      "max": 0.016729982,
      "mean": 0.009695945042182285,
      "min": 0.005439609000000001,
      "p50": 0.010094758,
      "p75": 0.010957537,
      "p95": 0.013827296000000001,
      "p98": 0.016729982,
      "p99": 0.016729982,
      "p999": 0.016729982,
      "stddev": 0.002369039049106484,
      "m15_rate": 0.4164373137301911,
      "m1_rate": 0.6044556218797338,
      "m5_rate": 0.44797335122240606,
      "mean_rate": 0.9025245514858581,
      "duration_units": "seconds",
      "rate_units": "calls/second"
    },
  },
}
}

```



•



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring Processing of Profiles in PKernel

Monitoring Processing of Profiles in PKernel

Was this page helpful? 

PKernel is used to calculate real-time metrics to enrich events.

Having in mind that the majority of projects create real-time metrics, it is essential to know if PKernel is operating within reasonable limits. By taking monitoring metrics inside PKernel, we let Pulse operators to have access to different information related to PKernel via the /metrics endpoint.

These metrics are associated with:

- a given PKernel, which is identifiable via a pkernelID tag;
- a plan workflow element, which is identifiable via the workflowElementID tag;
- a workflow, identifiable via the workflowID tag;
- an application, identifiable via an appSlug tag.

Besides using the abovementioned tags to identify the PKernel monitoring metrics in the scope of an application, in certain cases, it is also necessary to identify the metrics in relation to PKernel components where these are being collected. This is possible through the following tags:

- `queryName` - Used when metrics are being registered inside a PQL query. The value that is assigned to this tag is the name of the query on the PQL plan code.
- `stateType` - Used when the metrics are related to operations performed on a given type of state object. The values associated with this tag can be deque or map. Deques are windows that contain the events processed within a certain time-frame; maps are dictionaries that contain the aggregations (e.g., avg, max) grouped by the values present in certain fields of an event.

The `appType` and `metricType` tags, that are also associated with these monitoring metrics, are common to other metrics exposed in Pulse and App Engines. They inform where the metrics were collected (Pulse Server or App Engine) and what type of metric it is being reported (e.g., APPLICATION, BUSINESS), respectively.

Name	Description	Unit	Tags
Counters			
<code>pulse.pkernel.eventProcessing.exceptions.count</code>	The total number of exceptions raised during event processing.	#	<code>appSlug</code> <code>metricType</code> <code>appType</code> <code>pkernelID</code> <code>workflowID</code> <code>workflowElementID</code>
<code>pulse.pkernel.eventProcessing.count</code>	The total number of events processed.	#	<code>appSlug</code> <code>metricType</code> <code>pkernelID</code> <code>appType</code> <code>workflowID</code> <code>workflowElementID</code>
<code>pulse.pkernel.eventProcessing.query.count</code>	The total number of events processed on each PQL query, identifiable via the <code>queryName</code> tag.	#	<code>appSlug</code> <code>metricType</code> <code>appType</code> <code>pkernelID</code> <code>queryName</code> <code>workflowID</code> <code>workflowElementID</code>
<code>pulse.pkernel.persistedState.checkpoints.count</code>	The total number of checkpoints that were triggered.	#	<code>appSlug</code> <code>metricType</code> <code>appType</code> <code>pkernelID</code> <code>workflowID</code> <code>workflowElementID</code>
<code>pulse.pkernel.persistedState.reads.count</code>	The total number of read operations performed per state object type, identifiable via the <code>stateType</code> tag.	#	<code>appSlug</code> <code>metricType</code> <code>appType</code> <code>pkernelID</code> <code>stateType</code> <code>workflowID</code> <code>workflowElementID</code>
<code>pulse.pkernel.persistedState.writes.count</code>	The total number of write operations performed per state object type, identifiable via the <code>stateType</code> tag.	#	<code>appSlug</code> <code>metricType</code> <code>appType</code> <code>pkernelID</code> <code>stateType</code> <code>workflowID</code> <code>workflowElementID</code>
	The total number of delete operations performed per state object type, identifiable via the <code>stateType</code> tag.	#	<code>appSlug</code> <code>metricType</code> <code>appType</code>

pulse.pkernel.persisted State.deletes.count			pkernelID stateType workflowID workflowElementID
Gauges			
pulse.pkernel.persisted State.events	The total number of events that are currently being managed per state object type, identifiable via the stateType tag.	#	appSlug metricType appType pkernelID stateType workflowID workflowElementID
Timers			
pulse.pkernel.eventProc essing.latency	The latency associated with event processing.	seconds	appSlug metricType appType pkernelID workflowID workflowElementID
pulse.pkernel.eventProc essing.query.latency	The latency associated with event processing on each PQL query, identifiable via the queryName tag.	seconds	appSlug metricType appType pkernelID queryName workflowID workflowElementID
pulse.pkernel.persisted State.checkpoints.laten cy	The latency associated with real-time metrics checkpoints.	seconds	appSlug metricType appType pkernelID workflowID workflowElementID
pulse.pkernel.persisted State.reads.latency	The latency associated with read operations performed per state object type, identifiable via the stateType tag.	seconds	appSlug metricType appType pkernelID stateType workflowID workflowElementID
pulse.pkernel.persisted State.writes.latency	The latency associated with write operations performed per state object type, identifiable via the stateType tag.	seconds	appSlug metricType appType pkernelID stateType workflowID workflowElementID
pulse.pkernel.persisted State.deletes.latency	The latency associated with delete operations performed per state object type, identifiable via the stateType tag.	seconds	appSlug metricType appType pkernelID stateType workflowID workflowElementID

Here you can see an excerpt of a response from a `/metrics` call focusing on the PKernel monitoring metrics that can be enabled in Pulse:

[Click here to expand...](#)

```

{
  "version": "3.1.3",
  "gauges": {
    "pulse.pkernel.persistedState.events,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "value": 909
    },
    "pulse.pkernel.persistedState.events,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=map,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "value": 20
    },
  },
  "counters": {
    "pulse.pkernel.eventProcessing.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 909
    },
    "pulse.pkernel.eventProcessing.exceptions.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 0
    },
    "pulse.pkernel.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=clean_input_0,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 909
    },
    "pulse.pkernel.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=dict_win_0_name,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 909
    },
    "pulse.pkernel.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=input,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 909
    },
    "pulse.pkernel.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=input_light,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 0
    },
    "pulse.pkernel.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=outputstream,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 909
    },
    "pulse.pkernel.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=outputstream_light,workflowID=mxZUYgH6rymlJVWdQzLKyg": {
      "count": 909
    },
  },
}

```

```

    "pulse.pkernell.eventProcessing.query.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=win_0,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
      "count": 909
    },
    "pulse.pkernell.persistedState.checkpoints.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
      "count": 0
    },
    "pulse.pkernell.persistedState.deletes.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
      "count": 0
    }
  },
  "pulse.pkernell.persistedState.reads.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
    "count": 3632
  },
  "pulse.pkernell.persistedState.reads.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=map,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
    "count": 5449
  }
},
  "pulse.pkernell.persistedState.writes.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
    "count": 909
  },
  "pulse.pkernell.persistedState.writes.count,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=map,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
    "count": 1818
  }
},
  "histograms": {},
  "meters": {},
  "timers": {
    "pulse.pkernell.eventProcessing.latency,appSlug=myapp,metricType=APPLICATION,pkernellID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,workflowID=mxZUYgH6rymlJVWDqzLKyg": {
      "count": 909,
      "max": 0.000991575,
      "mean": 0.0004416305666666667,
      "min": 0.000131604,
      "p50": 0.000415535,
      "p75": 0.0005846940000000001,
      "p95": 0.0008414662499999997,
      "p98": 0.00098357932,
      "p99": 0.000991575,
      "p999": 0.000991575,
      "stddev": 0.00020898503632162623,
      "m15_rate": 1,
      "m1_rate": 1,
      "m5_rate": 1,
      "mean_rate": 1.0007287324651737,
      "duration_units": "seconds",
    }
  }
}

```

```

    "rate_units": "calls/second"
  },
  "pulse.pkernel.eventProcessing.query.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,queryName=clean_input_0,workflowID=mxZUYgH6rymlJVDqzLKyg": {
    "count": 909,
    "max": 0.000012565,
    "mean": 0.000006972966666666667,
    "min": 0.000002329,
    "p50": 0.00000722,
    "p75": 0.000009459250000000001,
    "p95": 0.0000107924,
    "p98": 0.000012287360000000001,
    "p99": 0.000012565,
    "p999": 0.000012565,
    "stddev": 0.0000027487680998343,
    "m15_rate": 1.220727664702864,
    "m1_rate": 1.0000001835413925,
    "m5_rate": 1.0298722410207188,
    "mean_rate": 1.0036337781096774,
    "duration_units": "seconds",
    "rate_units": "calls/second"
  },
  "pulse.pkernel.persistedState.checkpoints.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,workflowID=mxZUYgH6rymlJVDqzLKyg": {
    "count": 0,
    "max": 0,
    "mean": 0,
    "min": 0,
    "p50": 0,
    "p75": 0,
    "p95": 0,
    "p98": 0,
    "p99": 0,
    "p999": 0,
    "stddev": 0,
    "m15_rate": 0,
    "m1_rate": 0,
    "m5_rate": 0,
    "mean_rate": 0,
    "duration_units": "seconds",
    "rate_units": "calls/second"
  },
  "pulse.pkernel.persistedState.deletes.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbbe6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVDqzLKyg": {
    "count": 0,
    "max": 0,
    "mean": 0,
    "min": 0,
    "p50": 0,
    "p75": 0,
    "p95": 0,
    "p98": 0,
    "p99": 0,
    "p999": 0,
    "stddev": 0,
    "m15_rate": 0,
    "m1_rate": 0,
    "m5_rate": 0,
    "mean_rate": 0,

```

```

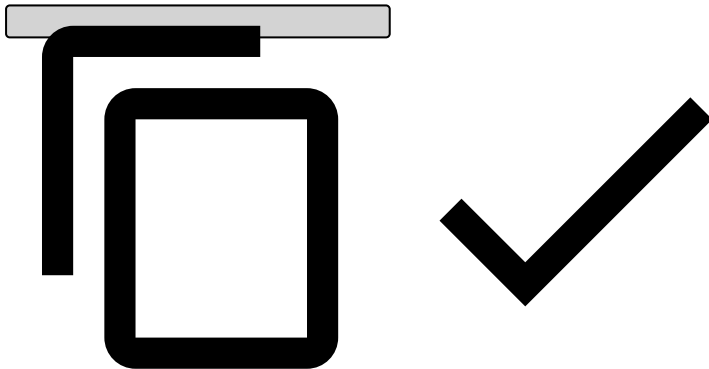
        "duration_units": "seconds",
        "rate_units": "calls/second"
    },
    "pulse.pkernel.persistedState.reads.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVVQzlkYg": {
        "count": 3632,
        "max": 0.000004223,
        "mean": 0.0000010414500000000002,
        "min": 6.3000000000000001e-8,
        "p50": 8.095e-7,
        "p75": 0.000001483,
        "p95": 0.0000027967,
        "p98": 0.00000341640000000000013,
        "p99": 0.0000039983300000000002,
        "p999": 0.000004223,
        "stddev": 8.668869311879352e-7,
        "m15_rate": 3.7056964470628473,
        "m1_rate": 3.999999755278144,
        "m5_rate": 3.960170345305707,
        "mean_rate": 3.998497591528649,
        "duration_units": "seconds",
        "rate_units": "calls/second"
    },
    "pulse.pkernel.persistedState.reads.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=map,workflowID=mxZUYgH6rymlJVVDqzlkYg": {
        "count": 5449,
        "max": 0.000565127,
        "mean": 0.00001713365,
        "min": 6.3000000000000001e-8,
        "p50": 0.0000013885000000000001,
        "p75": 0.00000393675,
        "p95": 0.0001296943,
        "p98": 0.0002498556999999992,
        "p99": 0.0002790459,
        "p999": 0.000565127,
        "stddev": 0.000058453089622799034,
        "m15_rate": 5.632120558828559,
        "m1_rate": 5.999999694097679,
        "m5_rate": 5.9502129316321355,
        "mean_rate": 5.998849917522664,
        "duration_units": "seconds",
        "rate_units": "calls/second"
    },
    "pulse.pkernel.persistedState.writes.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=62fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=deque,workflowID=mxZUYgH6rymlJVVQzlkYg": {
        "count": 909,
        "max": 0.0000201470000000000002,
        "mean": 0.0000032528000000000004,
        "min": 6.64e-7,
        "p50": 0.000002909,
        "p75": 0.000003883,
        "p95": 0.00000563765,
        "p98": 0.0000171497200000000018,
        "p99": 0.0000201470000000000002,
        "p999": 0.0000201470000000000002,
        "stddev": 0.0000025704318492279516,
        "m15_rate": 1,
        "m1_rate": 1,

```

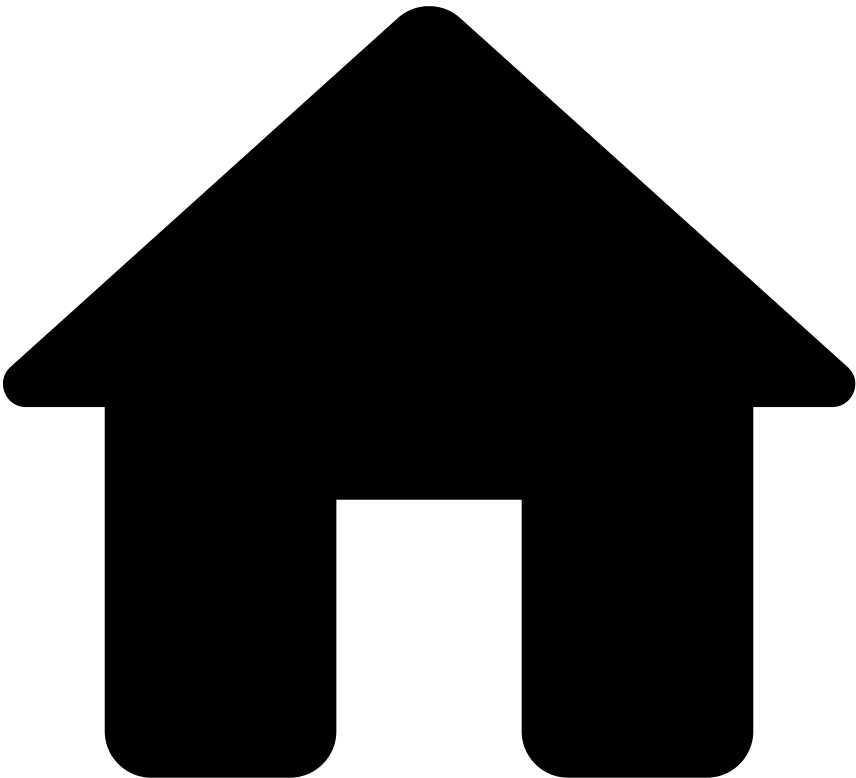
```

    "m5_rate": 1,
    "mean_rate": 1.000723693058066,
    "duration_units": "seconds",
    "rate_units": "calls/second"
  },
  "pulse.pkernel.persistedState.writes.latency,appSlug=myapp,metricType=APPLICATION,pkernelID=6
2fa7f857130338cbb6b9966319a8b1700c0a9f843523da99dca2ab6d421264,workflowElementID=f77043c7-
af5e-47e2-8c18-06d1e9f14131,appType=PULSE_APPLICATION,stateType=map,workflowID=mxZUYgH6rymIJVWDq
zLKyg": {
    "count": 1818,
    "max": 0.000284437,
    "mean": 0.00006079733333333334,
    "min": 6.64e-7,
    "p50": 0.000025976,
    "p75": 0.00012653175,
    "p95": 0.00018474519999999995,
    "p98": 0.00025243529999999996,
    "p99": 0.000279369069999999984,
    "p999": 0.000284437,
    "stddev": 0.00007068538603937175,
    "m15_rate": 2,
    "m1_rate": 2,
    "m5_rate": 2,
    "mean_rate": 2.001448399586337,
    "duration_units": "seconds",
    "rate_units": "calls/second"
  }
}
}

```



.



- Project Organization
- [Monitoring](#)
- [Monitoring Metrics](#)
- Monitoring Replication of Events in the Workflow

On this page

Monitoring Replication of Events in the Workflow

Was this page helpful?

When the event arrives to the end of a published workflow, it is replicated to other workflow replicas via a messaging broker.

The following table lists metrics available to monitor that execution:

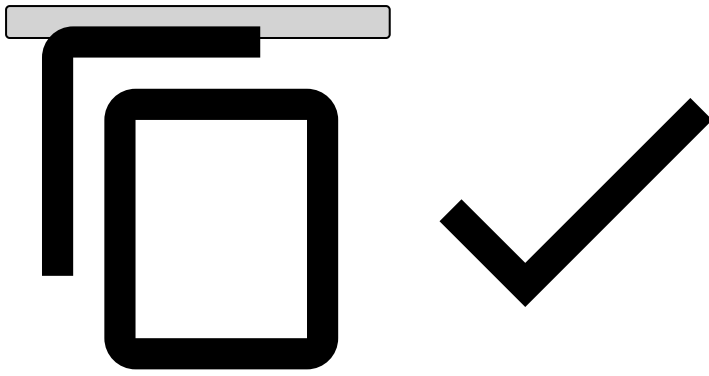
Name	Description	Unit	Tags
Gauge			

pulse.workflow.replication.replicationSendQueueSize	The events that are waiting to be sent to the messaging broker to be replicated after having been processed normally in this workflow.	#	workflowID appSlug metricType appType
pulse.workflow.replication.replicationReceiveQueueSize	The events that are waiting to be processed after having been received in a workflow replica from the messaging broker. They are delivered to this in-memory queue and are then posted to the workflow queue.	#	workflowID appSlug metricType appType
pulse.workflow.replication.routingReceiveQueueSize	The events that have been received from the messaging broker because they must be routed to this workflow as the workflows that received them are not responsible for their partitions. They are now held in an in-memory queue waiting to be posted to the workflow queue.		workflowID appSlug metricType appType
Counter			
pulse.workflow.replication.errorsSendReplication	The number of events that have failed to be replicated from the sender side to every replica.	#	workflowID appSlug metricType appType
pulse.workflow.replication.errorsReceiveReplication	The number of events that have failed to be received as replicas on the current replica.	#	workflowID appSlug metricType appType

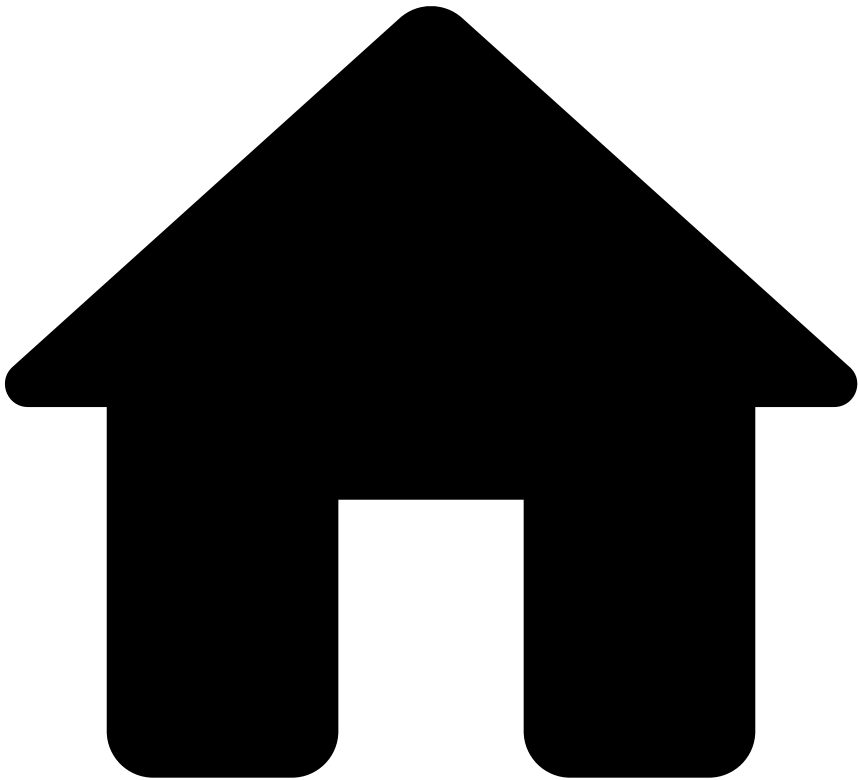
Example of live metrics [↗](#)

[Click here to expand...](#)

```
"pulse.workflow.replication.replicationReceiveQueueSize,appSlug=bank,metricType=APPLICATION,workflowDesc=payments,appType=PULSE_APPLICATION,workflowId=pBqsDBqu114nuQhbTKlPjw":{
  "value":11
},
"pulse.workflow.replication.replicationSendQueueSize,appSlug=bank,metricType=APPLICATION,workflowDesc=payments,appType=PULSE_APPLICATION,workflowId=pBqsDBqu114nuQhbTKlPjw":{
  "value":4
},
"pulse.workflow.replication.errorsSendReplication,appSlug=bank,metricType=APPLICATION,workflowDesc=payments,appType=PULSE_APPLICATION,workflowId=pBqsDBqu114nuQhbTKlPjw":{
  "value":5792
},
"pulse.workflow.replication.errorsReceiveReplication,appSlug=bank,metricType=APPLICATION,workflowDesc=payments,appType=PULSE_APPLICATION,workflowId=pBqsDBqu114nuQhbTKlPjw":{
  "value":38123
},
"pulse.workflow.replication.routingReceiveQueueSize,appSlug=bank,metricType=APPLICATION,workflowDesc=payments,appType=PULSE_APPLICATION,workflowId=pBqsDBqu114nuQhbTKlPjw":{
  "value":3
}
}
```



.



- Project Organization
- [Monitoring](#)
- [Pulse Probes](#)
- PKernel Probes

On this page

PKernel Probes

Was this page helpful?

PKernel probes, like the Pulse ones, provide a means to expose PKernel internal instrumentation data, allowing it to be stored in a central location for subsequent analysis.

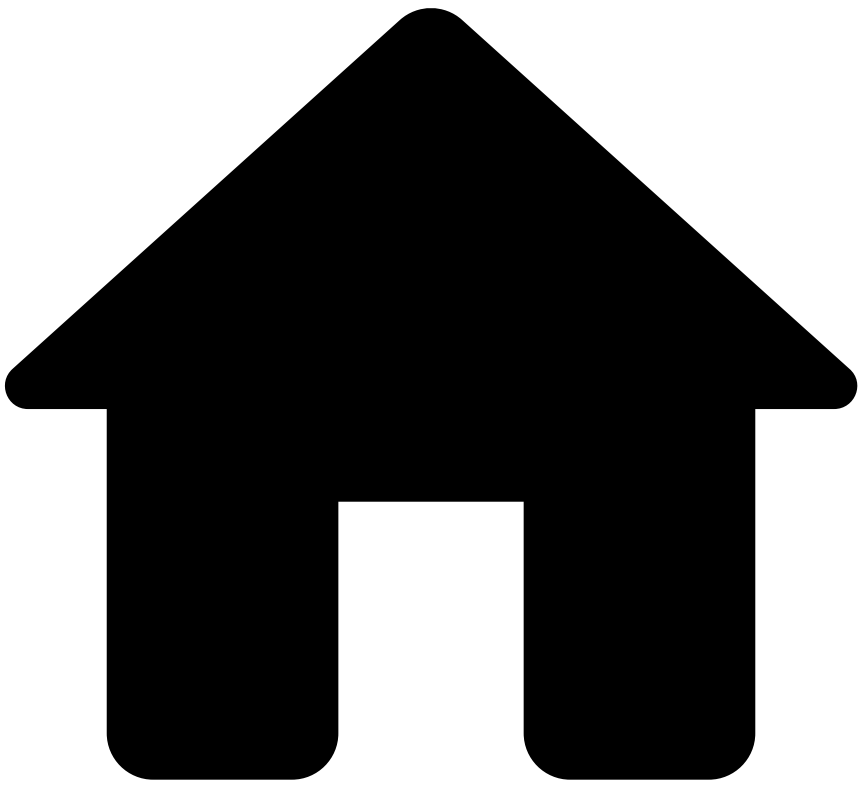
Existing Probes

The following probes are built-in in the PKernel 2.0 (with RocksDB):

Description	Probe Name	Schema	Notes
-------------	------------	--------	-------

The time that an Event took to be processed inside of PKernel.	pkernel_post_event	timestamp : Long	Current timestamp in milliseconds.
		latency_ns : Long	Latency measured in nanoseconds.
The time that an Event took to be processed inside of PKernel after acquiring the lock. This allows to understand the true time spent inside PKernel, ignoring possible queue effects of the lock. (Available from PKernel 2.1.0+)	pkernel_post_event_inside_lock	timestamp : Long	Current timestamp in milliseconds.
		latency_ns : Long	Latency measured in nanoseconds.
The time that an Array of Events took to be processed inside of PKernel.	pkernel_post_events	timestamp : Long	Current timestamp in milliseconds.
		latency_ns : Long	Latency measured in nanoseconds.

•



- [Core Concepts](#)
- [Data Sources](#)
- Data Source Analysis

On this page

Data Source Analysis

Was this page helpful? 

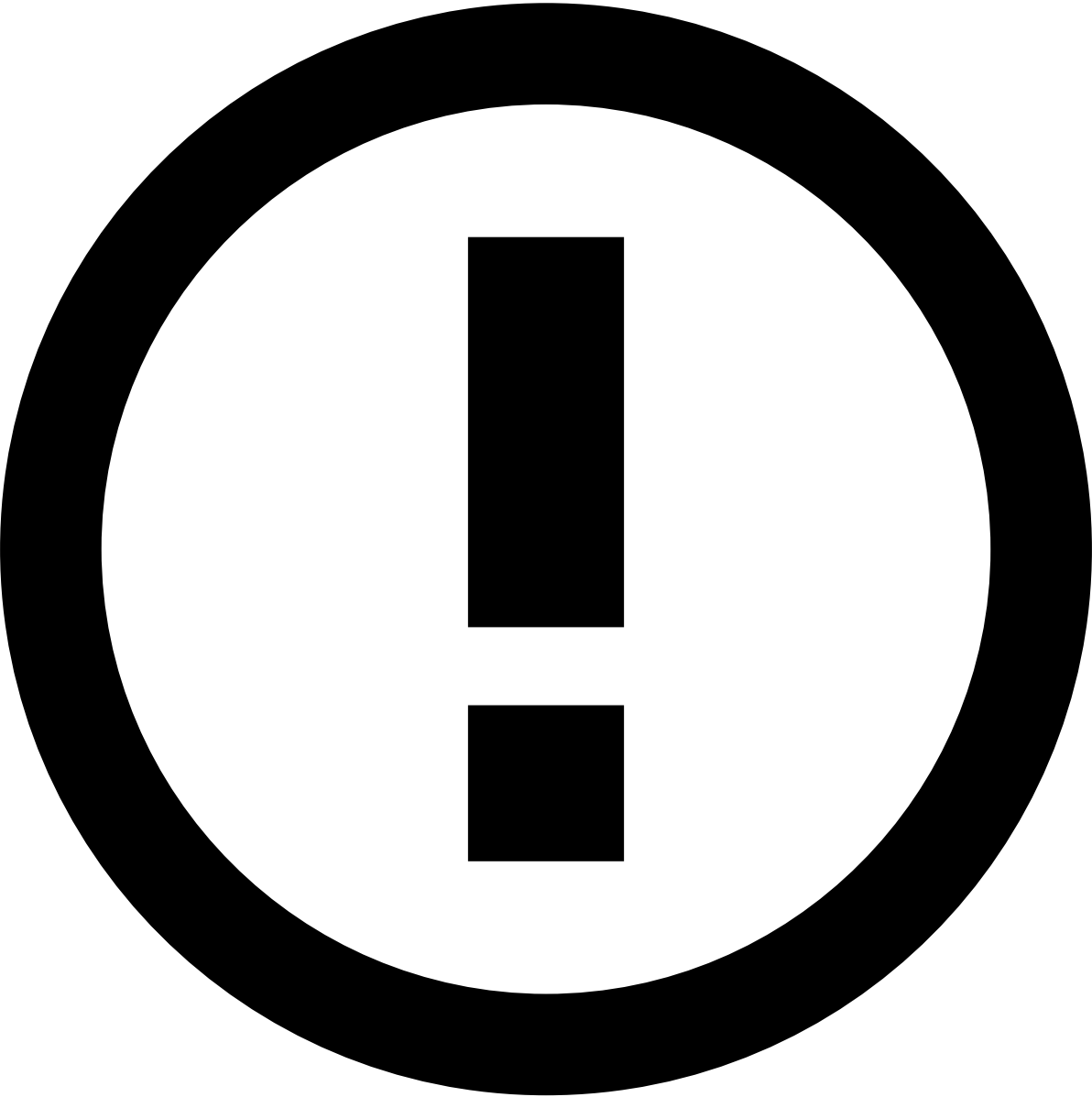
Pulse lets you extract simple metrics from your data so that you can easily perform high-level analyses and start uncovering some patterns in your data.

Depending on the data type, Pulse calculates different metrics for each data field. The following matrix shows which are the metrics available for each field type:

	Histogram	Mean	Std. Dev	Min	Max	Count	Distinct	Missing Values Count
Numeric	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Categorical	Yes	No	No	No	No	Yes	Yes	Yes

	Histogram	Mean	Std. Dev	Min	Max	Count	Distinct	Missing Values Count
String	No	No	No	No	No	Yes	No	Yes

This may help you uncover, for example, data quality issues where your dataset includes negative values in transaction amounts.



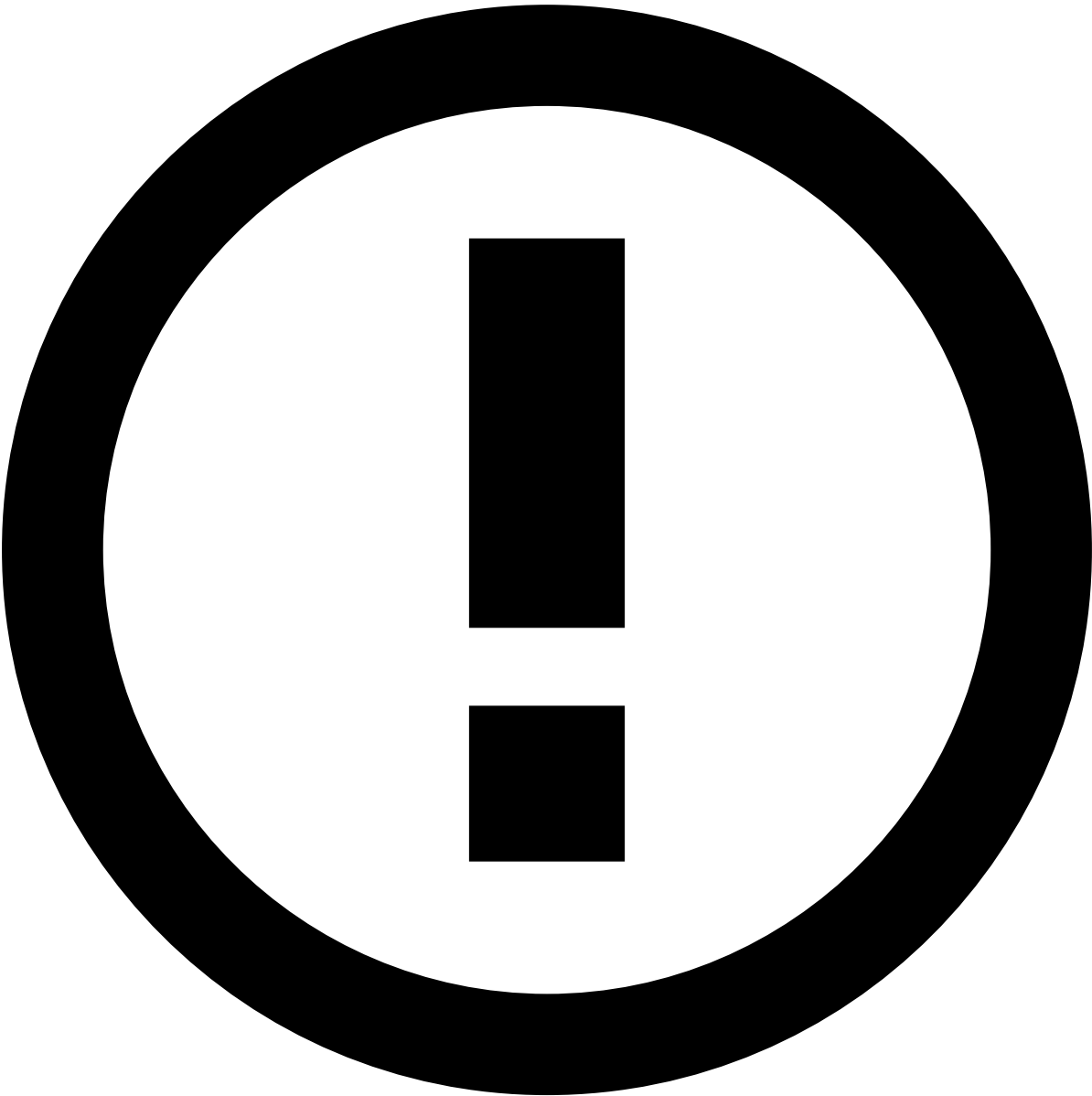
info

With Pulse, there is no need for you to load your data into a distributed database and manually run queries to analyze the data.

Statistics generation is distributed in Spark. This means that even for very large sets of data, with several TB in size, Pulse is very fast to compute metrics and to simplify your data validation and understanding stage.

Breakdowns

When you run an analysis, in addition to generating overall metrics, Pulse also calculates them over field combinations. This allows you to break them down by another categorical field and its values, usually the one that labels transactions, for example, as fraud or not fraud.



info

When you analyze a data source, you must always select for which breakdown fields and values you want Pulse to calculate statistics.

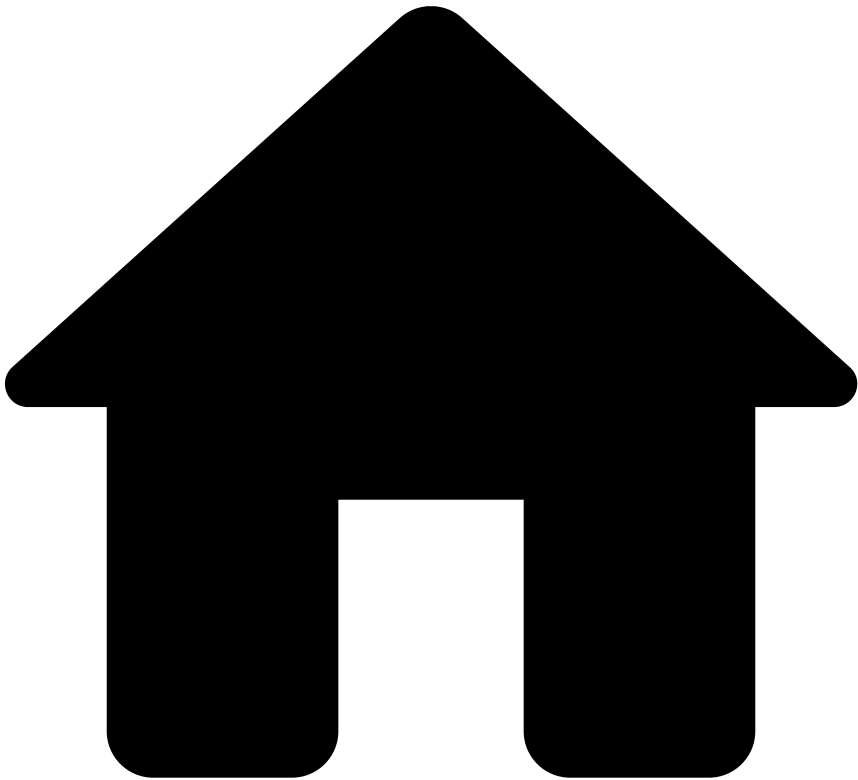
If Pulse allowed you to skip this action, and simply calculate broken-down statistics for all possible combination of fields and values, it would rapidly cause performance degradation due to lack of computational resources.

The following images display the analysis on the *TransactionAmount* field broken down by fraud and non-fraud transactions. A pattern emerges indicating that, in this dataset, fraud is more prevalent in lower value transactions.

Note that Pulse will not consider any invalid instance in your dataset for calculating these metrics. For example, rows with an incorrect date format or a value not specified for a categorical field are always skipped.

In histograms, the number of instances displayed for each bin is always an approximate value.

.



- [Core Concepts](#)
- [Data Sources](#)
- Data Types

On this page

Data Types

Was this page helpful? 

Each field of a [data source](#) is characterized by a type that indicates its purpose in a data science context. Data sources are specified by their schema, and by the mapping between field types for data science and an underlying native data type. Possible field types are the following:

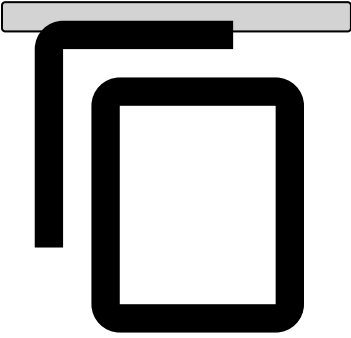
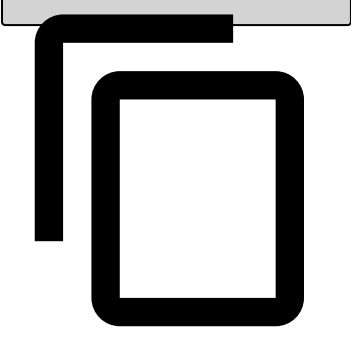
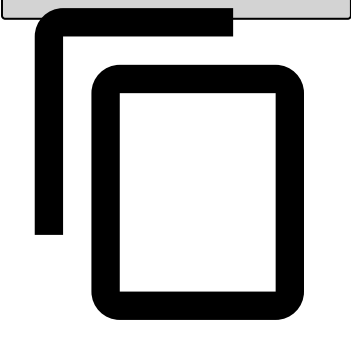
Simple fields

Type	Description	Configuration and native type	Example
Categorical			

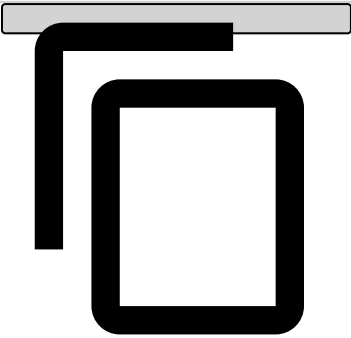
Type	Description	Configuration and native type	Example
	Fields that represent a finite amount of values of a discrete variable, that are useful in a Machine Learning setting. In Pulse you can use categorical fields to group specific business terms, or contract/domain specifications. As a heuristic, it is recommended that whenever the number of possible values exceeds ~10, you set the field type as String. In these cases, the number of possible values is too large for both a human and a machine learning algorithm to analyze and extract valuable insight. However, you can use Plans to transform (categorical or string) fields with too many categories into useful features. For example, using a UDF to categorize Merchant Category Code (MCC) values into Merchant Category Groups (MCG).	Each discrete value to be considered as valid for the field. All possible values of the field must have the same native type. Possible native types are: • String • Integer • Long • Boolean	CountryCode: 484, 124, 136, 840
Numeric	Fields that represent a continuous value. Pulse is able to generate arithmetic-based calculations and charts to analyze numeric fields such as distribution histograms, maximum, minimum, count, mean and standard deviation. You cannot perform Group By aggregations using Numeric fields as group by fields.	Possible native types are: • Integer • Float • Double	TransactionAmount
String	Text-based fields that in their raw format, usually, have no meaning in a data science context, when training a model. Although strings are not valuable in their raw format, they can be used for data science after some kind of processing or transformation with plans. For example, checking if an address starts with numbers or is in all caps. However, they may be useful in rules, for example to check if a value is in a list and may or may not be a categorical. Additionally, strings are also useful as support information for scored instances when visualizing model and rule evaluations, for example the client id, or even to analyse the performance of your models and rules broken down by these string fields.	Possible native types are: • String • Utf-8 String	CardholderAddress
Date Time	Fields that represent a date. Information like a timestamp is also numeric, however, looking at a timestamp and not considering its temporal properties would render it useless. Thus, Pulse processes data representations separately. Time-based information is useful for Pulse to validate data sequence and also very important to allow and facilitate time-aware operations. For example, helping the user selecting fields that represent time for time window-based aggregations.	Timestamp: A UNIX timestamp, in seconds or milliseconds. Date Time: A moment specified in a given format, for example dd/MM/yyyy.	Timestamp: 1350776834512 Date Time: 30/06/2016

Complex fields

Complex fields are used to describe structured data. When a complex field is defined it will represent a data structure like a list, map, tuple or set.

Type	Description	Configuration	Example
Map	Consider a transaction processing system with multiple POS. Each POS might have a lot of characteristics (e.g., battery usage, exterior temperature, etc), which might vary from one manufacturer to another. This information is difficult to maintain in a fixed data schema. Thus, you can define a map for POS properties (the information stored in the map must all have the same data type). Pulse only supports maps with primitive types as keys, and whose values are either primitive types or tuples.	Types of key-value mapping. For example, Map of Integer - String	<pre>test { 1: "Alaska", 2: "Minnesota" }</pre> 
List	Consider a transaction that has two processing codes. While the data schema might have two different fields for each of the processing codes, it might be more convenient to include a list or a set type field containing both.	Type of all items in the list. For example, List of Integers.	<pre>[1, 2, 44, 1]</pre> 
Set	Keep in mind that the difference between sets and lists is that Sets don't allow duplicate values. Pulse only supports lists and sets whose elements are either primitive types or tuples.	Type of all items in the set. For example, Set of Strings.	<pre>["Alaska", "Arizona"]</pre> 
Tuple	Aggregate several items which can be of any simple type. Use tuples for the fields that might contain different known attributes. Pulse only supports tuples whose elements are primitive types.	Types of each element of the tuple. For example, Tuple of 3 items.	<pre>{ amount: 2, country: "New York", timestamp: 123456 }</pre>



Type	Description	Configuration	Example
			

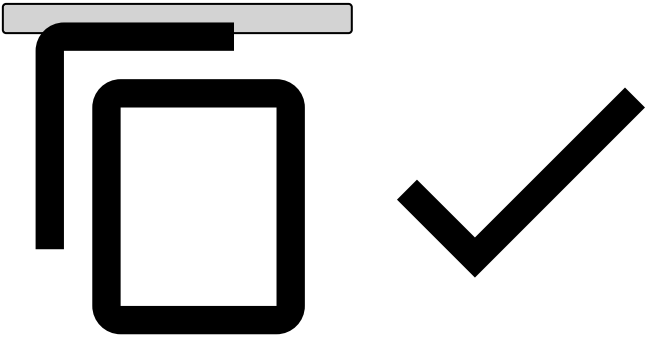
Importing complex fields in CSV files

Internally in the [Event Storage](#), Pulse uses system types directly. When importing data from a CSV file, fields with complex data must be represented as JSON strings.

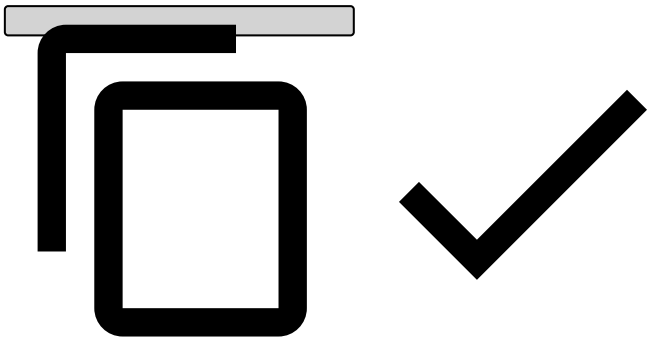
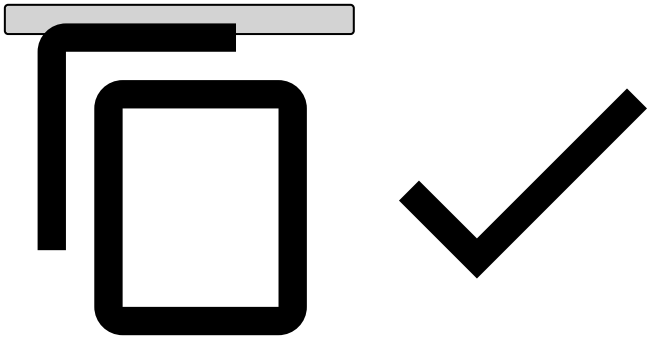
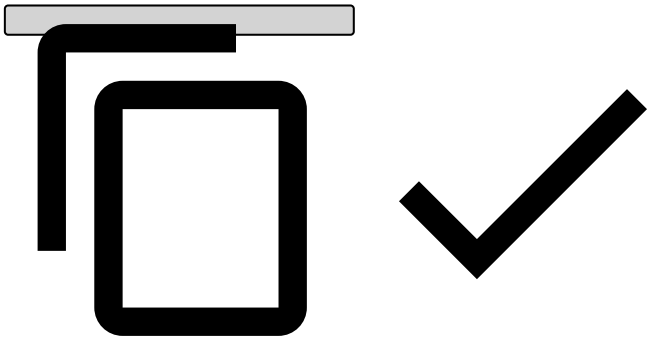
The syntax rules for complex data fields in CSV files are the following:

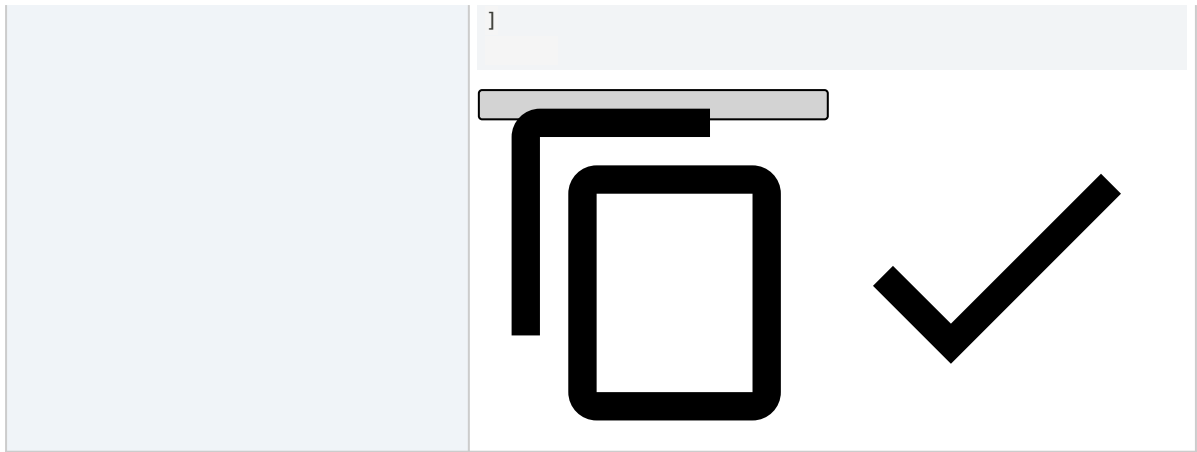
- Complex fields need to be represented as strings.
- Each complex field needs to be enclosed between quotation marks.
- If a complex field contains strings, the quotation marks around the strings must be escaped with a second quotation mark.
- Keys of complex fields must be strings. If in the schema they are defined as numeric type, they will be converted to the numeric type automatically, after reading the value from the string form in the CSV file.
- If you have stand-alone quotation marks within a string, it becomes `\`.

As an example, let's take the following data row (displayed as a column for better readability):

Timestamp	1350776834512 
Date	30/06/2016

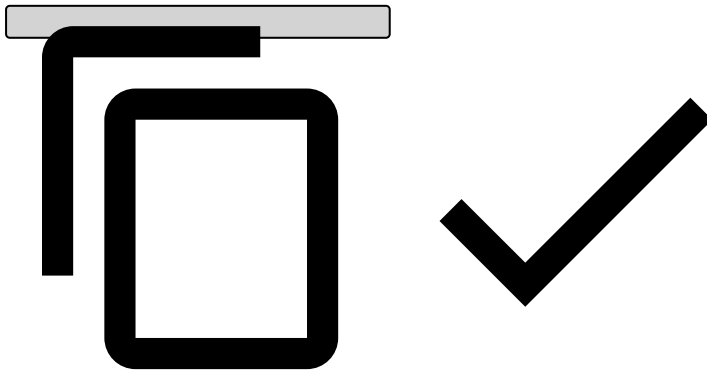
Country Code	124
Transaction Amount	124.99
Cardholder Address (string)	"221b Baker St, London NW1 6XE"
POSProperties (tuple)	

	<pre>{ 1: "Alaska", 2: "Minnesota" }</pre> 
ProcessingIDs (set of Integers)	<pre>[1, 2, 44, 1]</pre> 
ProcessingNames (list of strings)	<pre>["Alaska", "Arizona"]</pre> 
Items (list of tuples)	<pre>[{ AmountUSD: 2.00, Product: "Awesome Shirt" }, { AmountUSD: 242.00, Product: "LED television 42" smartTV" }]</pre>



To import this row, the CSV file must look the following way:

```
Timestamp,Date,CountryCode,TransactionAmount,CardholderAddress,POSProperties,ProcessingIDs,ProcessingNames,Items
1350776834512,30/06/2016,124,124.99,"221b Baker St, London NW1 6XE",{"1": "Alaska", "2": "Minnesota"},[1, 2, 44, 1],["Alaska", "Arizona"],[{"AmountUSD": 2.00, "Product": "Awesome Shirt"}, {"AmountUSD": 242.00, "Product": "LED television 42\" smartTV"}]
```

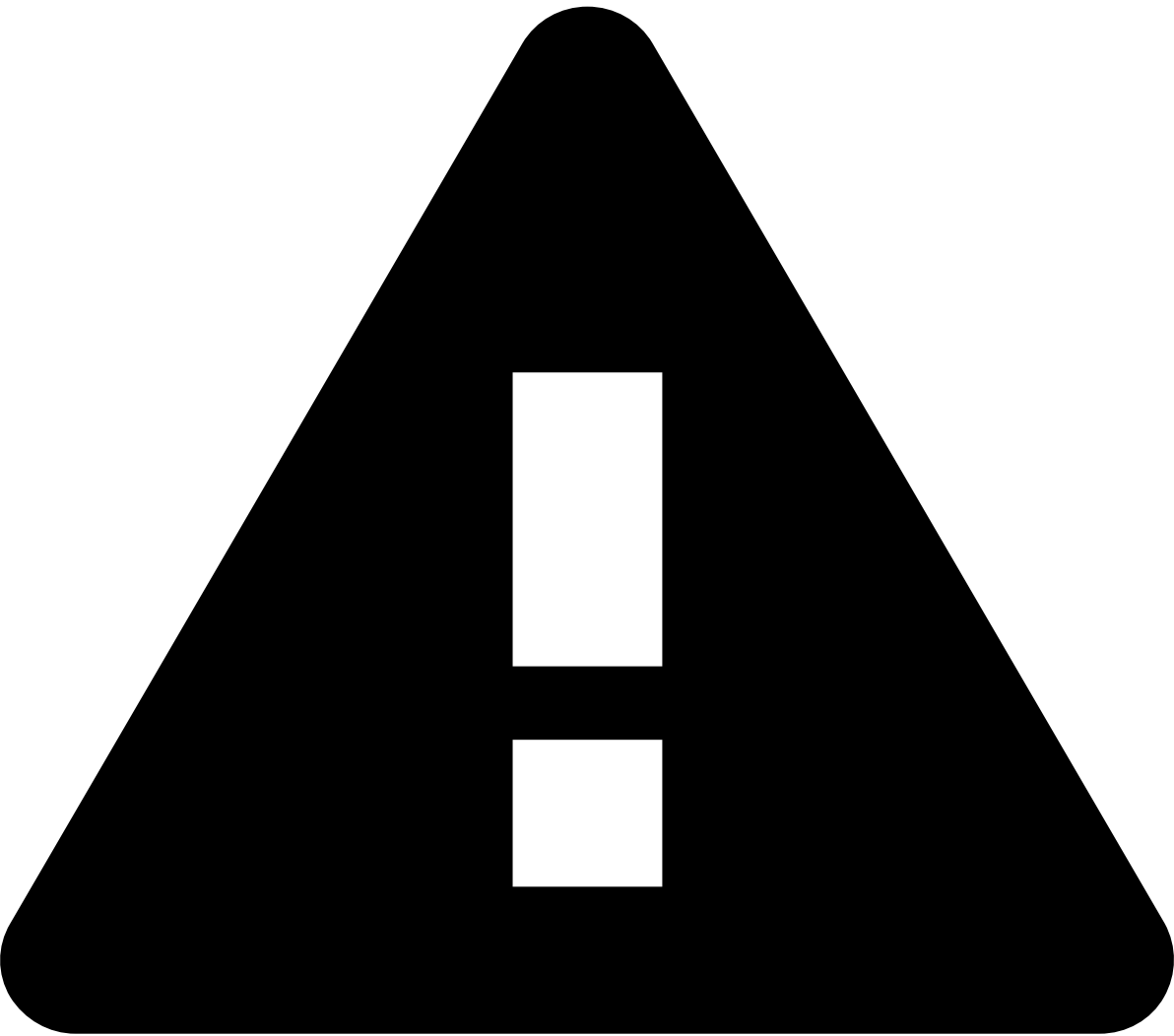


Mapping of data types from Parquet files

Parquet files encode the type of each field, in addition to the values. When Parquet files are [imported into Pulse](#), those types are automatically converted into the data types supported by Pulse, listed in the tables above. The mapping between Parquet data types and Pulse data types is as follows:

Parquet type	Pulse type	Configuration
INT32	Numeric	Integer
INT64	Numeric	Long
INT64 with TIMESTAMP logical type annotation	Date Time	Timestamp in milliseconds (Long)
INT96 (Deprecated in Parquet, but used in Spark for timestamps)	Date Time	Timestamp in milliseconds (Long)
FLOAT	Numeric	Float
DOUBLE	Numeric	Double
BINARY with STRING logical type annotation	String	String
BINARY with ENUM logical type annotation	String	String
BINARY with JSON logical type annotation	String	String
BINARY with NULL logical type annotation	String	String
BOOLEAN	Categorical	Boolean

Parquet type	Pulse type	Configuration
MAP	Complex Field	Map
MAP_KEY_VALUE	Complex Field	Map
LIST	Complex Field	List
Nested Object	Complex Field	Tuple



caution

Not all data types supported by the Parquet format can be imported into Pulse. Fields whose types are not mapped to a Pulse data type as described in the table above will be ignored.

Impact of data types on memory consumption📄

Adjusting the data types can significantly reduce the memory footprint of individual records. This can be extremely important in an application that handles thousands of events per second. The application will probably define multiple time windows that store events in memory.

You must be aware about the size occupied by your events, to be sure that there is enough memory for the events in the time windows you eventually define.

As an example, consider an application that processes 300 events per second and whose schema has 10 string fields that store 8 characters each. Defining a 24 hours window will result in the following memory consumption:

24 hours $\times$ 60 minutes $\times$ 60 seconds $\times$ 300 events $\times$ 10 fields $\times$ 88 bytes each = **21 GB**

If you replace all string types with integers you will end up with:

24 hours $\times$ 60 minutes $\times$ 60 seconds $\times$ 300 events $\times$ 10 fields $\times$ 16 bytes = **4 GB**

This alternative is much lighter than 21 GB, in fact it is about **5 times** smaller.

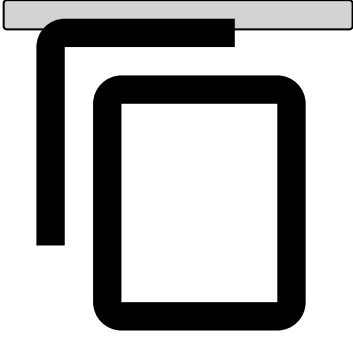
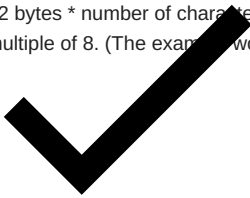
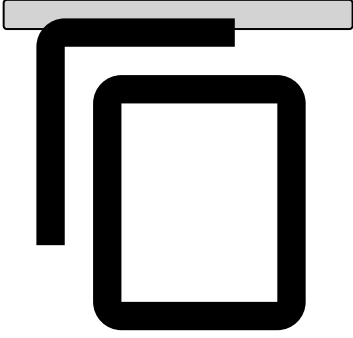
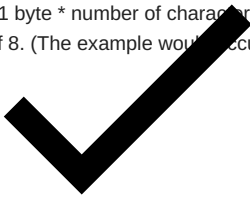
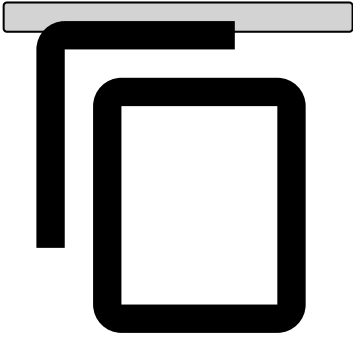
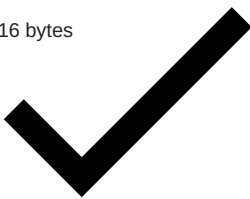
Try to choose the appropriate field types whenever possible to help reduce the memory used.

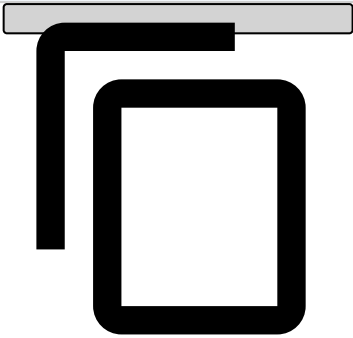
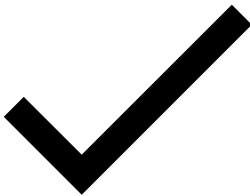
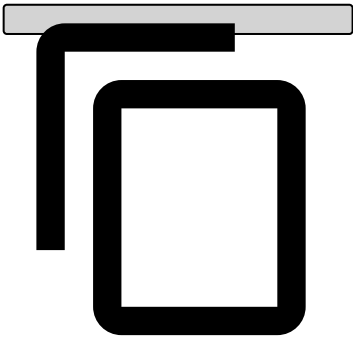
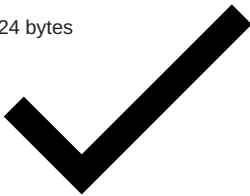
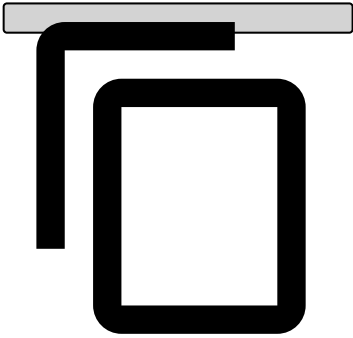
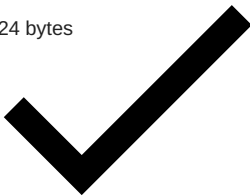
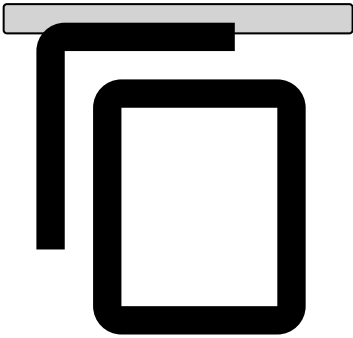
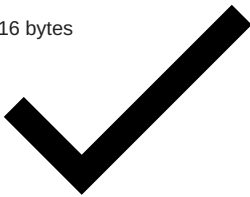


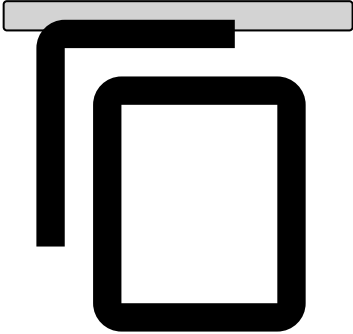
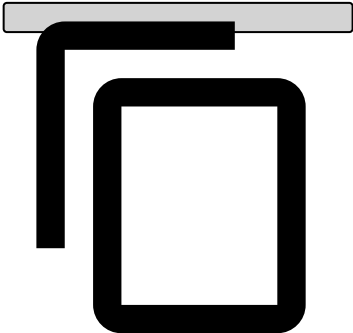
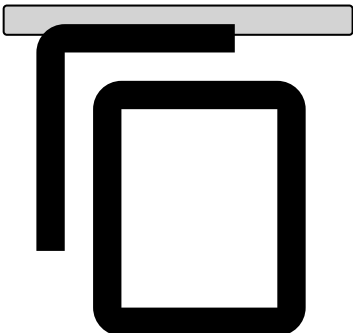
info

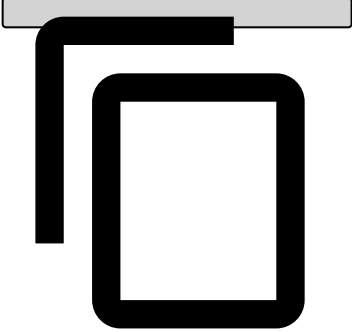
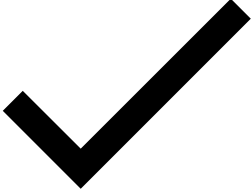
Since Strings use quite a lot of memory, only use them to save text that can't be categorized. Otherwise, there are other field types that suit data better.

The following table contains the size of each available data type:

Type	Example	Size (64bits JVM)
String	"New York" 	~2 bytes * number of characters + 68 bytes + padding needed to be a multiple of 8. (The example would occupy 88 bytes) 
UTF8 String	"New York" 	~1 byte * number of characters + 24 bytes + padding needed to be a multiple of 8. (The example would occupy 36 bytes) 
Integer	1 	~16 bytes 
Float	1.0	~16 bytes

Type	Example	Size (64bits JVM)
		
Double	1.0 	~24 bytes 
Long	1000000000 	~24 bytes 
Boolean	true 	~16 bytes 

Type	Example	Size (64bits JVM)
Tuple	<pre>{ amount: 2, country: "New York", timestamp: 123456 }</pre> 	Approximately the sum of all tuple fields' sizes. In the example's case: 16 bytes + 88 bytes + 16 bytes + Tuple object overhead, so about 120 bytes.
List	<pre>[1, 2, 44, 1]</pre> 	Approximately the sum of all list elements' sizes.
Set	<pre>["Alaska", "Arizona"]</pre> 	Approximately the sum of all set elements' sizes.
Map	<pre>[1: "Alaska", 2: "Minnesota"]</pre>	Approximately the sum of all map elements' sizes.

Type	Example	Size (64bits JVM)
		

•



- [Core Concepts](#)
- [Models](#)
- AutoML

On this page

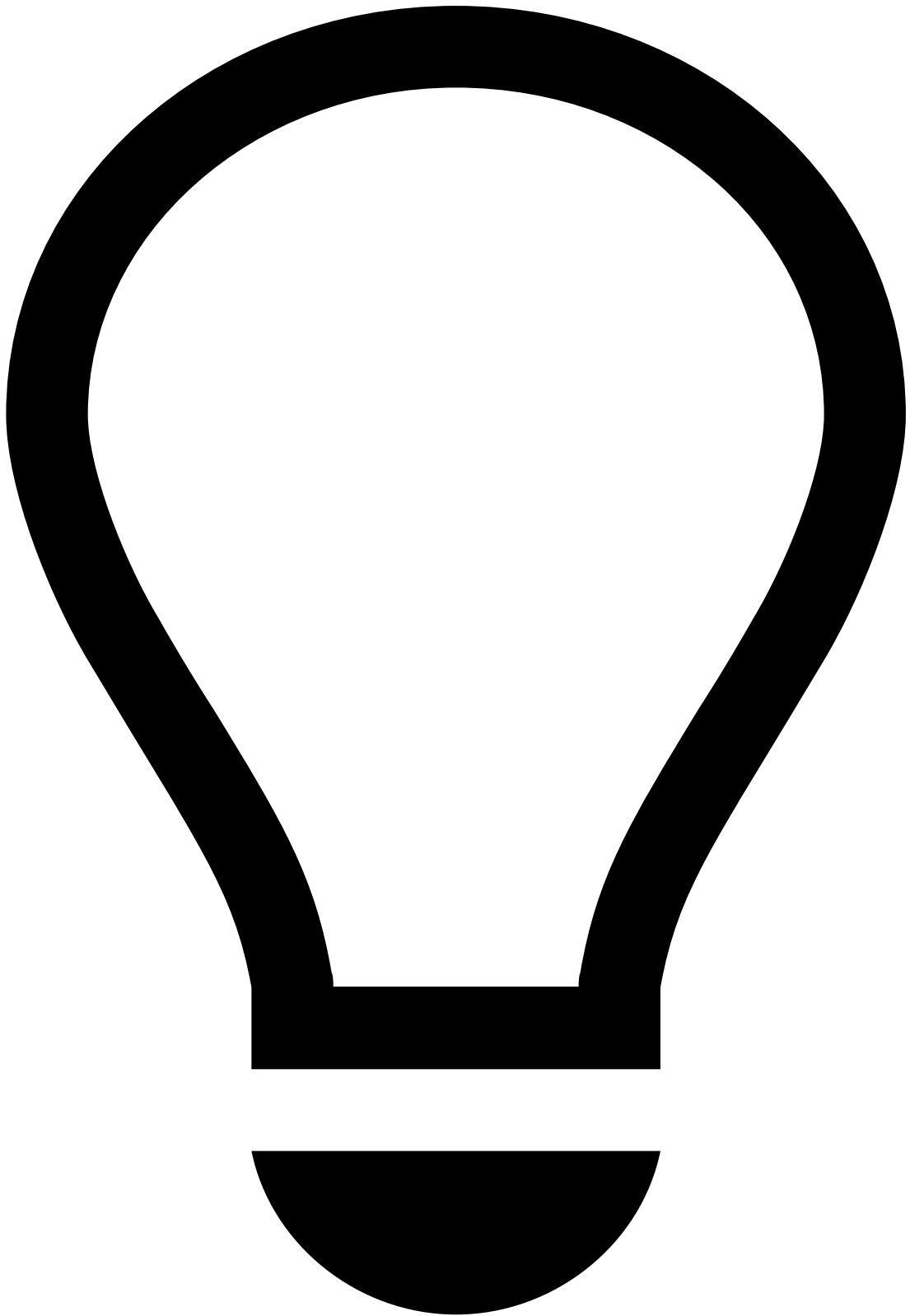
AutoML

Was this page helpful? 

AutoML automates feature engineering, model training and hyperparameter tuning. The following topic describes in more detail what AutoML does.

Why use AutoML?[^](#)[📄](#)

Finding the right features and models for a data set is a cumbersome task: creating features, training models, tuning their parameters and testing the changes are all repetitive tasks. AutoML automates these tasks, using the fraud detection experience of Feedzai. Data Scientists can kick-off new fraud fighting projects with AutoML, or can use it to find interesting features for existing models.



Feedzai IQ

Further time savings can be achieved with [Feedzai IQ](#), which provides ready-to-adopt risk assessment solutions that don't require extensive data science work, thanks to federated financial risk intelligence aggregated from the Feedzai global network.

Prerequisites📄📄

To be able to run AutoML against a data source, you need to prepare the data first.

- Date and datetime fields need to be converted to Unix timestamp format.
- The data cannot contain NULL values. Instead, you need to add a default value:
 - Strings coalesced to empty string (").
 - Categoricals should be coalesced to an existing value or to a new categorical value representing the NULL, typically a value called "other".
 - Numericals should be coalesced to a meaningful number, for example -999, -1 or 0, depending on their valid range.
 - Dates and datetimes should be coalesced to a default value, for example-2208988800, which corresponds to 01-01-1900 in Unix timestamp format.
- The input data source can't use empty string (") as NULL.

Feature engineering📄📄

Feature engineering is the process of applying domain knowledge to derive new characteristics from existing data. These new characteristics, the **features** might help to build better machine learning models for a specific purpose.

In Pulse, features are represented by [Maps and Metrics](#) in a [Plan](#). When executing a Plan, Pulse processes the input data and creates a new data source that contains new fields. The value of those fields is defined by the Map and Metric operators of the Plan.

AutoML has a list of data field transformations that can be useful according to Feedzai's experience in the fraud fighting domain. Those transformations are based on the semantics of the data fields, combining fields with specific meaning into features. For example, if a data source has two location data fields, it might be useful to calculate the distance between the two. So, AutoML has a transformation that computes the distance between two location fields.

To allow AutoML to apply those transformations, when [defining an AutoML run](#) you need to provide a [semantic file](#) that maps the input data fields to semantic tags. For example, the *Name* field can be tagged with the *name* tag. This way AutoML knows the meaning of this input field, and it can compute the number of words in the name, which can be useful to fight fraud.

In summary, AutoML follows these steps to create features:

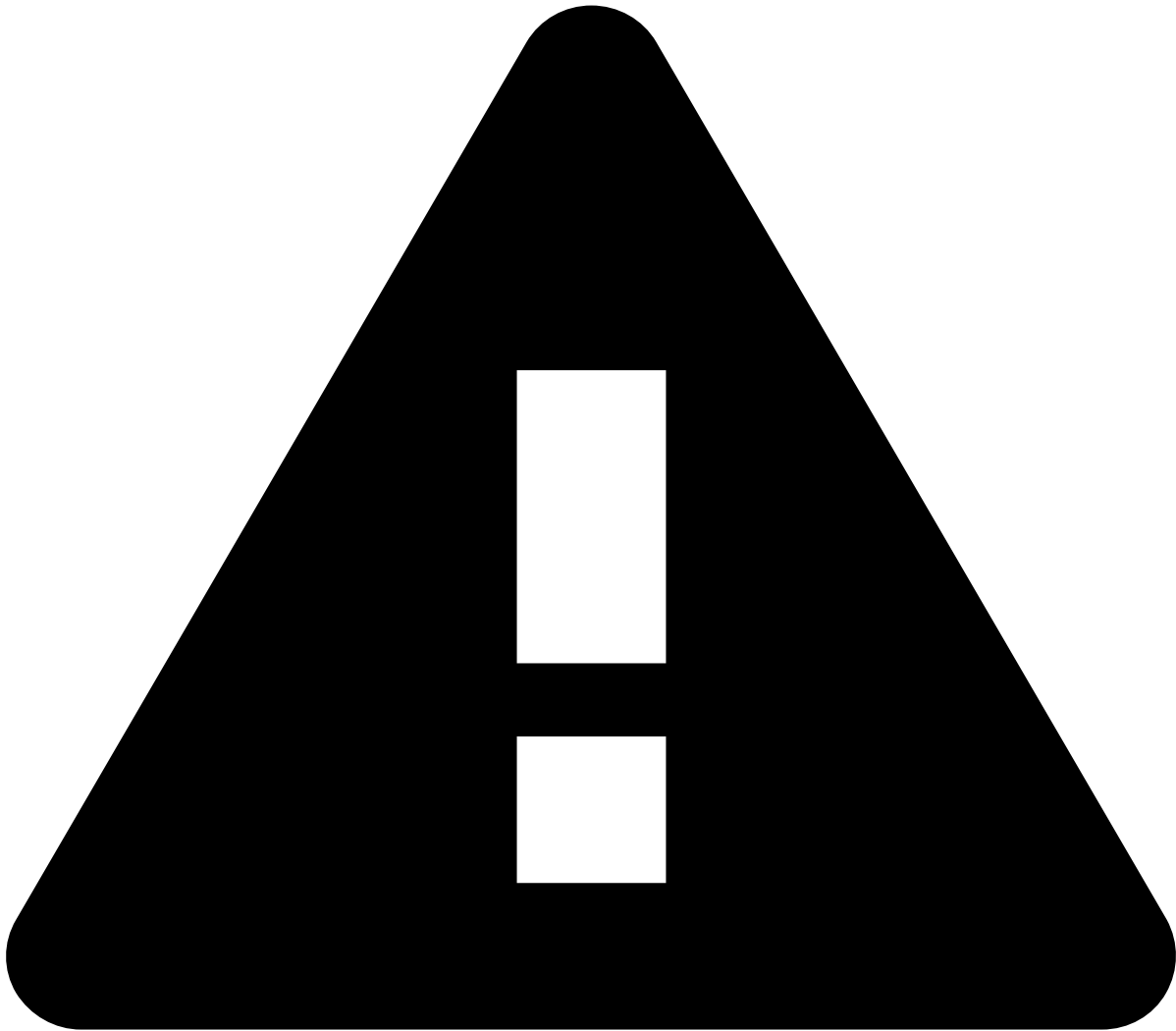
1. Looks at the semantic file and maps the data fields to their meaning.
2. Based on the data fields' meaning and on the list of transformations, AutoML creates a plan containing Map and Metric operators.
3. Executes the plan on the input data source, and saves the result in a **features data source** that contains the data enriched with the new data fields.

Dataset split📄📄

When you create the AutoML run, you define which parts of the input data source will be used for training and for validation. Optionally you can also choose to have a test data set to simulate a blind evaluation of the best model found.

You define these data sets by selecting the time range for each task: training, validation, and optionally test. That means for example that data records that have a timestamp falling into the training time range will be used for training the models.

AutoML creates the training, validation and optionally the test data source based on the defined time ranges.



Planning training, validation and test data sources

To avoid under- or overfitting, plan the time ranges carefully. Ensure that the resulting data sets contain enough data for their purpose. A typical ratio is 50% for the training, 30% for the validation, and 20% for the evaluation data set.

Note that if you don't define a test data set, AutoML uses the validation data set for the final evaluation of the best model found. Since the validation data set was used for finding the best model, it is not an optimal solution to use it to test the data. Hence it's recommended to use a test data set.

Filters

If the input data source contains different types of events, and you don't want AutoML to consider all of them when creating metrics or training models, you can apply filters.

AutoML allows you to define the definition of two kinds of filters:

- If you define a **profile filter**, AutoML will create metrics that consider events filtered by the defined expression. Example: you might want to filter for only transaction submissions and payment updates - `event_type = "transaction_submission" OR event_type = "transaction_payment_status_update"`
- If you define a **scorable filter**, AutoML will train the models based on events that satisfy the expression of this filter. Example: you might want to filter for only transaction submissions - `event_type = "transaction_submission"`

Filters in the first step of the AutoML run

The scorable filter will create a new filter operator in the plan that splits the data source.

Datasource Splitting plan with a Scorable Filter

Feature selection

In this step, AutoML first trains an H2OGradient Boosted Trees model on a sample of the training dataset. GBT models are quick to be trained, and provide a feature importance ranking out of the box.

Then Pulse reduces the features to the subset that has the highest feature importance according to the GBT model. This way, Pulse keeps only a subset of the features for the model training step of AutoML. The features that AutoML keeps contribute with 90% of the importance rating.

Note that Pulse has another [feature importance](#) calculation method as well, which you can use later on, after you evaluated a trained model. That feature importance calculation method is different from the process that AutoML uses.

Reduced features plan

The AutoML Pipeline combines the plan generated in the **Feature Engineering** step with the subset of features from the **Feature Selection** step, and reduces the plan. It does so by removing the least important features discovered in the Feature Selection step. This prevents data scientists from manually removing the operators that will produce useless features.

Model training and evaluation

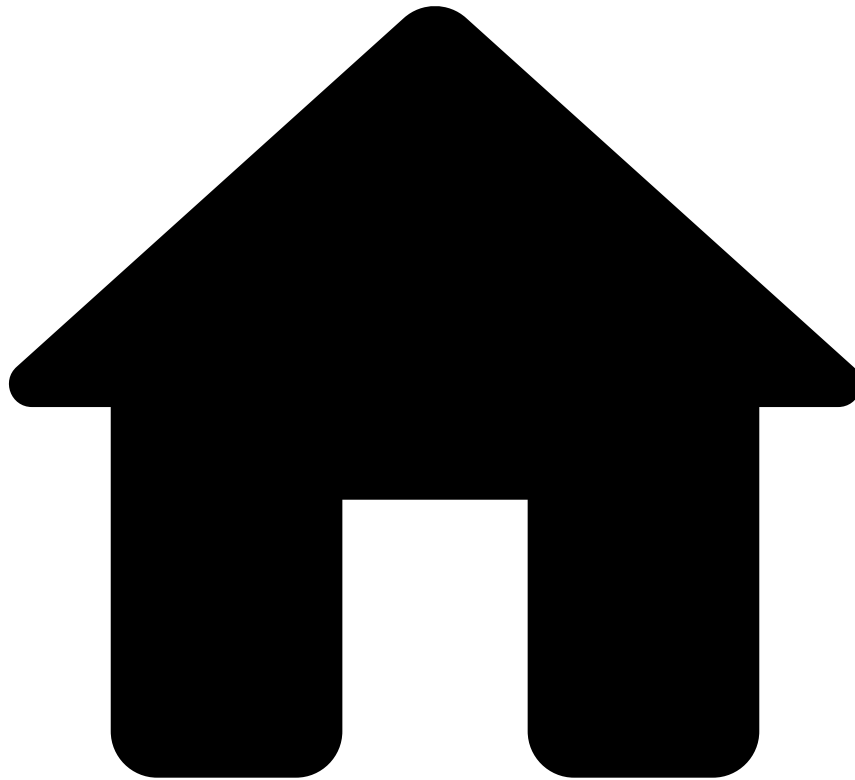
After selecting the most important features, AutoML creates a Hyperparameter Tuning job to train and evaluate models. Hyperparameter Tuning jobs distribute the model training and evaluation in the Spark cluster.

Finally, AutoML compares the models based on the metric that you selected to be optimized, and recommends the best model.

Learn more

- [Creating an AutoML run](#)
- [Plans](#)
- [Maps and Metrics](#)
- [Semantic file](#)
- [Hyperparameter Tuning](#)

•



- [Core Concepts](#)
- [Models](#)
- Feature Importance

On this page

Was this page helpful? 

Feature Importance

Machine learning models rely on many features to learn how to predict patterns. To develop a model with high performance in the field of fraud prevention, data scientists might need to select hundreds of features to train the model. The process of selecting features is complex, and there is a risk of including features that have none or even negative contribution to the model.

Feature importance is a functionality that can be used to determine the impact of each feature on a machine learning model.

Data scientists can use this functionality to identify the subset of variables that contribute the most and the least to the model predictions. With this information, data scientists have more visibility on how the model is using the features to score transactions. Additionally, data scientists can remove features that have low contribution to the model without

decreasing model performance (i.e. declining evaluation metrics). This allows reducing model complexity and computation time.

Ultimately, with the help of the feature importance, data scientists can develop models that fight fraud more efficiently and that are easy to interpret.

In Pulse, feature importance is calculated after training a model, during the evaluation stage. Check the [Evaluating a Model](#) page to learn how to use this functionality in Pulse.

How Pulse calculates feature importance

In Pulse, the importance of a feature is calculated by replacing the feature with noise and measuring the impact on model performance.

Pulse completes the following steps to compute feature importance:

1. Randomly samples the dataset using the *Sample Percentage* and *Random Seed* values defined by the user as input parameters.
2. Computes *Transaction Recall* (and *Cost Recall*, if an additional cost field is selected) at the closest False Positive Rate (FPR) value to the value defined by the user for the model with the original features.
3. For each feature used to train the model:
 1. Randomly shuffles values of the feature between the different instances.
 2. Computes *Transaction Recall* (and *Cost Recall* in case an additional cost field is selected) at the closest FPR value to the value defined by the user for the model with the feature shuffled.
 3. Subtracts the *Transaction Recall* (and *Cost Recall*, if an additional cost field is selected) computed in Step 3.b. to the value computed in Step 2. This yields the absolute importance of each feature:

$$AbsoluteImportance_i = Recall_baseline - Recall_shuffled_i$$

where *Recall_baseline* is the recall of the original evaluation, *Recall_shuffled_i* is the recall of the evaluation with feature_i shuffled. As an example, considering the calculation of absolute importance for the feature Transaction Amount, the expression becomes:

$$AbsoluteImportance_{TransactionAmount} = Recall_baseline - Recall_shuffled_{TransactionAmount}$$

4. For each feature, relative importance is calculated as follows:

where N is the total number of features. As an example, considering the calculation of relative importance for the feature Transaction Amount, using an evaluation that has 100 features including MCC Code, the expression becomes:

Using Recall to calculate absolute importance

As presented in the step-by-step description of the algorithm, FPR (i.e. percentage of genuine transactions incorrectly classified as fraudulent) and Recall (i.e. percentage of fraudulent transactions correctly classified as fraud) are both used to calculate absolute importance in Pulse.

This metric is calculated as the difference between Recall before and after shuffling the values of the feature between different instances at a selected FPR.

Recall can be plotted against FPR, which is the so-called **ROC Curve** (see figure below).

As shown in the figure, Recall after shuffling can be equal, higher or lower than the Recall before shuffling.

This yields to an absolute importance approximately equal to zero, negative or positive, respectively. Therefore, the outcomes are the following:

- **Absolute importance approximately equal to zero:** for features that have no significant contribution to the model predictions.
- **Negative absolute importance:** for features that have a negative contribution to the model predictions.

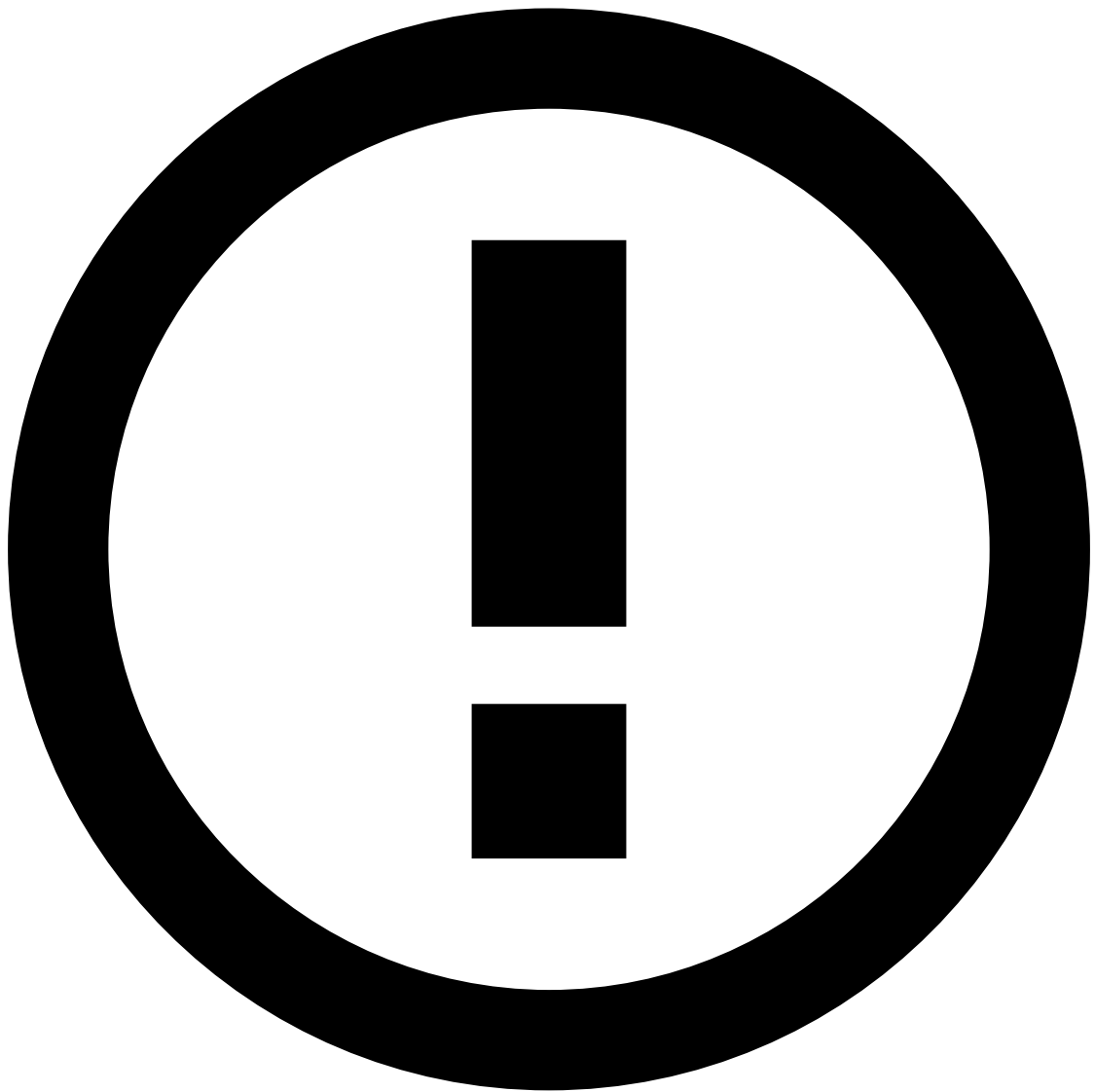
- **Positive absolute importance:** for features that are important to the model predictions.

There are different parameters that can be selected to compute feature importance which are described in the next subsection.

Parameters

Feature importance is computed using the following parameters selected by the user:

- **Sample Percentage:** percentage of randomly sampled instances in the data. When selecting this parameter the user should take into account the trade-off between having a representative set of data to compute feature importance and runtime. Additionally, the user should take into account that all data is loaded into memory to compute feature importance, and thus, high sample percentage values may lead Pulse to run out of memory in the case of extremely large datasets. The default value of the sample percentage is 30%.
- **FPR:** value at which the Recall is calculated. The default value of FPR is 1.00%.
- **Random Seed:** value that affects both of these operations: the order to which the instances of the features are shuffled and the instances that are sampled to compute feature importance. Note that the percentage is defined by sample percentage. This value can be used by data scientists to check the stability of the results using different seeds and keeping all the other conditions the same.



Check stability with random seeds

If the results are similar using different random seeds it means that the user is likely using enough data to compute feature importance.

The following parameter can be also optionally selected when computing feature importance:

- **Additional Cost Field:** a field that is used as a weight for the metrics of the evaluations (of both models and rules). In a typical fraud detection scenario, the selected feature for this field is the transaction amount. By adding an additional cost field, Recall is calculated based on the feature specified in this field (i.e. Cost Recall) in addition to be calculated based on the number of transactions (Transaction Recall).

Feature importance results📄📄

A typical report with results of feature importance is illustrated in the following image.

The features are ranked according to the level of importance. In this example, is easy to identify that the first four features impact the model whereas the remaining features have low impact on the outcome decisions of the model.

Extra functionalities that data scientists can use to analyze the results of feature importance, including comparing results of two model evaluations, are described in the [evaluating a model](#) page.

•



- [Core Concepts](#)
- [Models](#)
- Hyperparameter Tuning

On this page

Hyperparameter Tuning

Was this page helpful? 

Hyperparameter tuning is the process of fine-tuning parameters to obtain the optimal set of parameters for a specific use case. When fine-tuning parameters without automating this process, it is time-consuming to define a different set of parameters and run the model training in each iteration. Hyperparameter tuning helps the user to determine the optimum parameters for a given machine learning context in an automated way.

Introduction to hyperparameter tuning

Depending on the use case, a certain set of parameters will work better than others, so the goal of data scientists is to try as many sets as possible to know which one allows to achieve an optimized solution for that specific use case.

Hyperparameter tuning relies on experimental results, allowing data scientists to train and evaluate multiple models with a diverse set of parameters by algorithm simultaneously.

Pulse generates the performance statistics for all the selected algorithms with all the parameter combinations. This saves time and keeps the models organized. Data scientists can iterate faster to determine the algorithm and set of parameters that optimize model performance. They are, thus, able to spend more time searching for insights in the data, which could improve the performance even more. Given the massive quantity of model evaluations that a hyperparameter tuning can originate, the exploration mode functionality facilitates how the user can explore and compare those evaluations obtained. As such, it helps users to make a decision in a more informed manner.

Grid search (parameter sweep)

The **grid search** (aka. parameter sweep) is considered the standard way of performing a hyperparameter tuning. The grid search consists on an exhaustive search, or sweep, of an exponentially large grid of possible hyperparameter combinations for a learning algorithm. Some key points of this process to consider are:

- You manually specify a set of values for each hyperparameter.
- Pulse creates a model for every combination of values. Take a very simple example (as in the image below): if you have 3 hyperparameters and you specify 2 values for one and 3 values for another, after performing the sweep, the grid will return $(2 \times 3) = 6$ models.
- Once the sweeping is finished, you can make queries and sort the models by performance metrics (such as *FPR*) by using the exploration mode.

Hyperparameter tuning evaluation methods

Hyperparameter tuning will behave different according to the selected evaluation method.

When selecting either cross or split validation the several models will be all trained and evaluated within the same distributed job. This will cause the model evaluation to run on a single partition which will lead to evaluations time much larger than the single evaluation jobs. The evaluation results will only be available after all the models finish training.

For the validation data source method, the models will be trained within the same distributed job, but a single evaluation job will be triggered as soon as each model finishes its training. Once the models are evaluated, the results will be available at the hyperparameter tuning page, without being necessary to wait for the remaining trainings and evaluations to finish.

Exploration mode

Once the creation step of a hyperparameter tuning is finished, the exploration one follows. The **exploration mode** is a view where you explore and analyze results to learn which evaluations perform better for a certain business target. There are two exploration mode views available in Pulse: a global view in your model project, where you can combine single model evaluations with hyperparameter tuning evaluations, and another view specific to each hyperparameter tuning.

The key points of the exploration mode are:

- Possibility to add targets and metrics
- Search filter that supports a flexible listing:
 - Selecting model and hyperparameter tuning evaluations
 - Selecting by algorithm and data source
- Possibility to select evaluations and add them to the results comparison table
- Possibility to clear applied evaluations from a target

Single model evaluation vs. hyperparameter tuning

Hyperparameter tuning brings important advantages when compared to creating and evaluating models one by one. The following table sums up the most significant of them:

Single model	Hyperparameter tuning
• Trains and evaluates models one at a time.	• Tuning evaluation combinations: ◦ launches and combines the possible combinations of the selected parameters ◦ tries more than one algorithm and for each one of them specifies multiple parameters simultaneously
• The model is trained and evaluated in a distributed way. Typically performs better than hyperparameter tuning.	• Spark distributes the training so that several models are trained and evaluated simultaneously according to the selected validation method.
• Instances are partitioned across many tasks and collected in a final logical block resulting in an aggregation of scored instances.	• No output scored instances since the model binary is not saved.

Learn more

Check the following pages in the User Guide to learn how the hyperparameter tuning functionality fits in a model life cycle and how it works in Pulse:

- [Tuning Model Parameters](#)
- [Exploring Hyperparameter Tuning Results](#)

•



- [Core Concepts](#)
- [Models](#)
- Model Evaluation

On this page

Model Evaluation

Was this page helpful? 

Evaluating your model against a dataset means analyzing how it is performing, and how close you are to reaching the objectives of your data science project.

The performance of models can be analyzed in **model evaluations**. A model evaluation first runs the model to score the possible values of the target variable in a specific data source.

Then Pulse compares the scores with the actual labels of the target field. It calculates relevant model performance metrics, and displays them in an informative layout.

Model evaluations can be created in two ways:

- Automatically, in the Validation step of the [model training process](#).
- Manually, by creating a new model evaluation for any existing model.

The evaluation data source

The evaluation can be run against the data source of the model training, using cross-validation or split percentage strategies. Alternatively, when evaluating a model that has already been trained on a given data source, Pulse will execute the evaluation against a user-specified data source. Thus, no validation method is applicable as the entire dataset is used for testing. It's the user's responsibility to ensure that the data source provided for evaluation is not the same that was used during training.



Compatible data sources

You can evaluate the same model with any different data sources, as long as they contain all the fields that the model requires. This means that you must always evaluate a given model with [compatible data sources](#).

The evaluation target variable

The evaluation can use the same target variable as the model, if it has any. Alternatively you can select another field as target variable for the evaluation. The format (type and possible values) of the evaluation target variable must match the format of the target variable used in the model training, if there was any.

Also, the position of the specified target variable must correspond to the position of the target used for training. Learn more about [compatible data sources](#).

Model performance metrics

When the model evaluation is finished, Pulse displays **model performance metrics** such as ROC curve, confusion matrix, cost-based metrics, etc.

ROC curve

Most importantly, the **ROC curve** is a simple and visual way to assess a model's performance. ROC curve plots the Recall (Y axis) over the False Positive Rate (X axis).

The best possible prediction method would yield a point in the upper left corner ('A' in the picture) or coordinate (0,100) of the ROC space, representing 100% Detection with 0% False Positives. The (0,100) point is also called a perfect classification. A completely random guess would give a point along the diagonal line 'C' (the so-called line of no-discrimination).

Keep in mind that when the model's performance is sufficiently good, the difference between the curves might become unnoticeable. Analyze the computed metrics for more precise results.

Comparing models

Model evaluations are useful in your data science project to perform easy comparisons over earlier versions of the model, with a different parameter configuration, different features, etc.

Pulse allows you to compare the metrics of two different models, to evaluate their performance. For that, their target variables must have the same data type and possible values.

Breakdowns

When you generate a model evaluation, in addition to generating overall performance metrics Pulse also calculates them over field combinations. This allows you to break them down by another categorical field and its values, and evaluate your model performance for each of those possible fields and value combinations. For example, checking the performance of your model segmented by each suspicious country.

Score distribution

To analyze the model results, another useful information is the **score distribution**. The score distribution graph shows the distribution of scores predicted by the model for each value of the target variable.

It allows to visually inspect the separation power of the evaluated model. In a good model, the values of the target variable are separated into different areas of the graph. In the following example, setting the Threshold between 50 and 100 will separate *False* from *True*.

Learn more

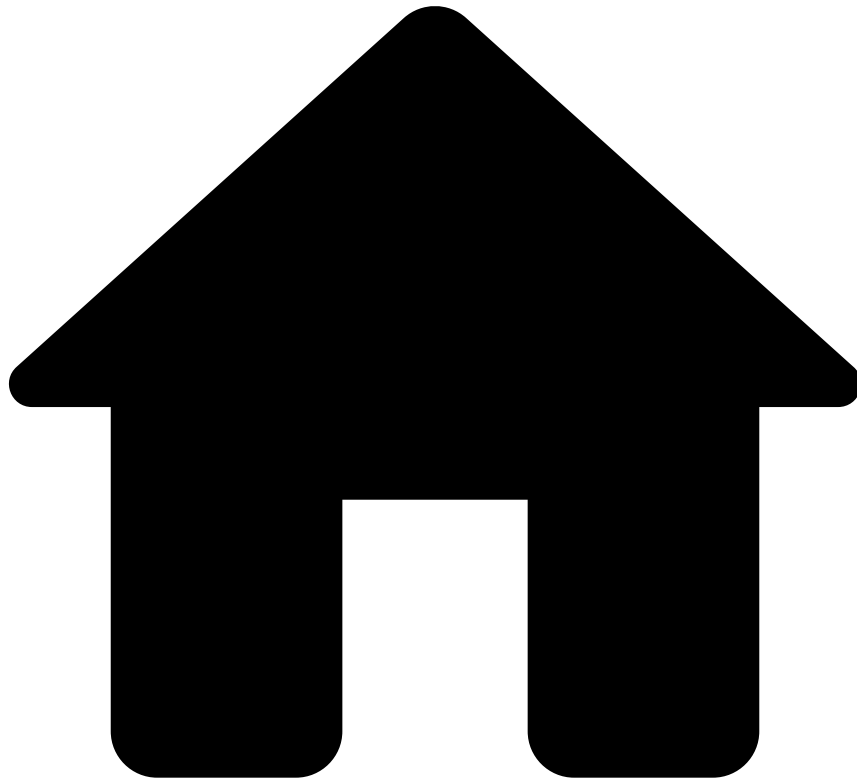
To fully understand model evaluations, make sure you also go through the following concepts:

- [Cost Fields](#)
- [Custom Evaluation Metrics](#)
- [Business Targets](#)

Check the following pages to learn how to put model evaluations into practice:

- [Evaluating a Model](#)
- [Tuning Model Parameters](#)
- [Comparing Models](#)

-



- [Core Concepts](#)
- [Models](#)
- Model Training

On this page

Model Training

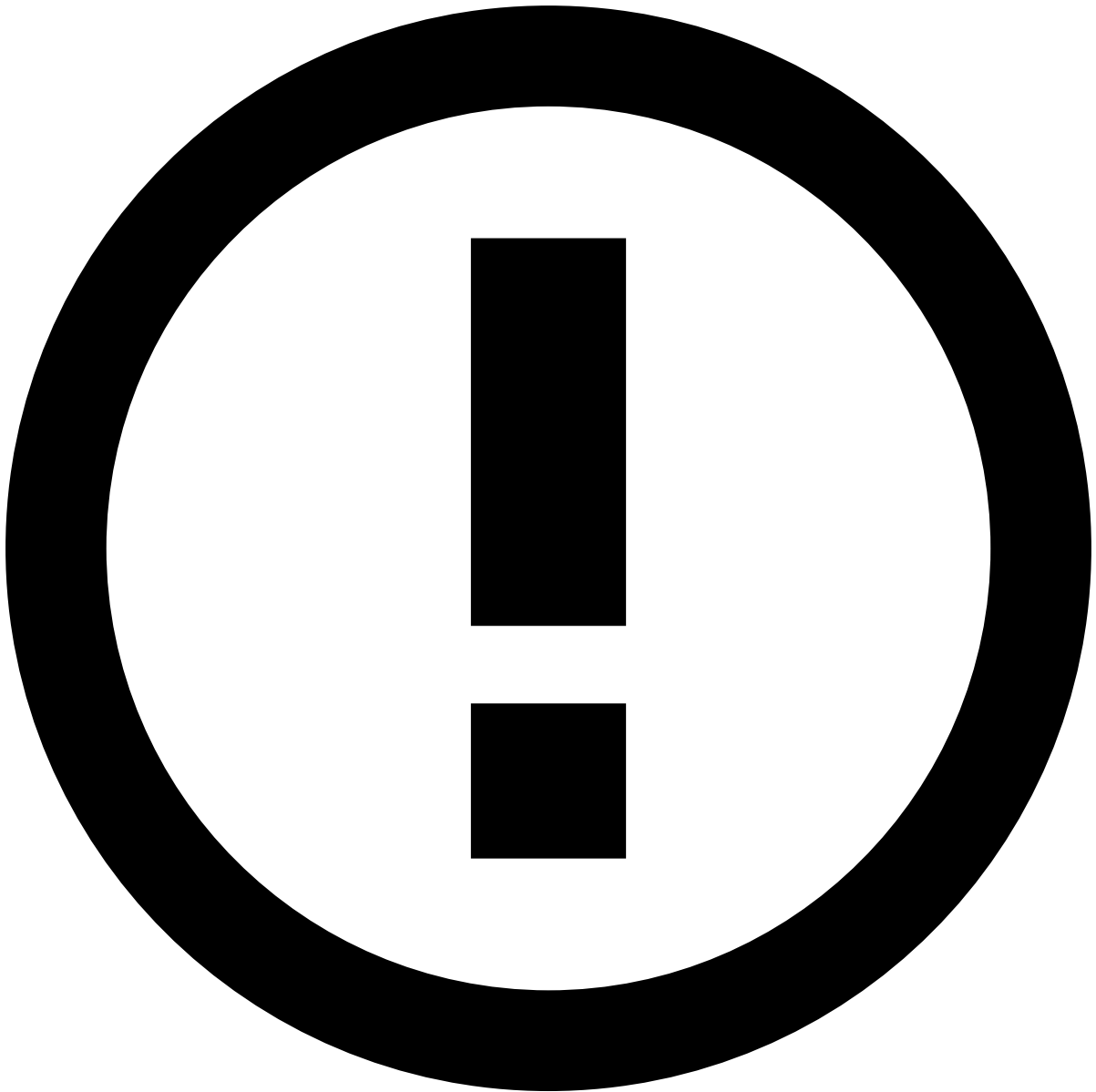
Was this page helpful? 

Training a model means running historical data through a machine learning algorithm to produce a model. You use the trained model in workflows to predict the value of a specified **target variable** in live data.

As part of the [data science loop](#), it is common to try different model training algorithms or keep making adjustments to model parameters and [feature plans](#), and repeat training until you reach a satisfactory performance.

The following sections give you more details on how models are trained in Pulse.

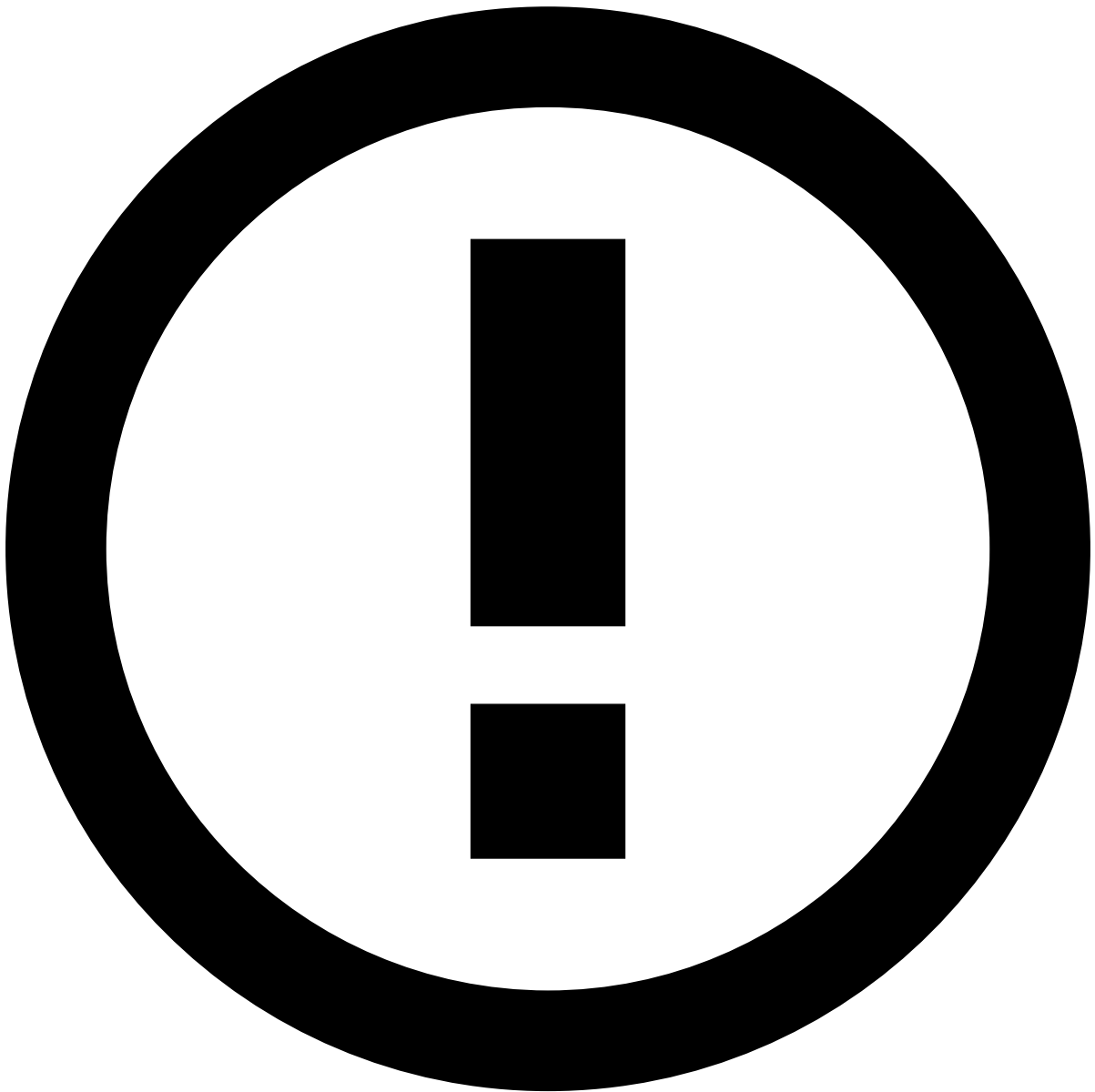
The event's journey during the model training



Naming conventions

When a transaction is flowing in the workflow, it is called an **event**. When an event is enriched with features, it is known as an **instance**. The instance does not traverse the workflow; it is computed using the event data and then used for training (offline data science) or scoring (runtime).

When Pulse loads a training event, it can be filtered before reaching a feature plan. If it reaches the feature plan, an instance with the computed real-time and scheduled metrics is generated. It also includes data transformations and enriched data, in addition to some of the raw features from the event. The instance can then be filtered again before generating the final data source, which Pulse will use to train the model.



Wasted computation?

It might seem odd to compute metrics for events that can later be dropped, but it actually has a simple use case. For instance, when training a model for fraud prevention in card-not-present transactions, it might be important to consider the number of card-present transactions. In this case, you could drop the card-present transactions using a [filter operator](#) before generating the plan output and after computing the count metric.

Omnichannel models

Note that in some cases you might want to train models using data from several data sources for [channels](#). For example, you might be scoring debit card transactions (main data source) and leveraging information about internet banking events (secondary data source). This can be achieved by executing an [omnichannel plan](#) on the input data sources. The output of that plan can then be used for model training.

Supervised and unsupervised machine learning

Pulse allows two different machine learning approaches:

- Training a model through **supervised machine learning**, where the purpose of the model is to predict the label of a target variable. To train a classification model, the data used for the training needs to be labeled.
- Training a model with **unsupervised machine learning**. In this case, a target variable is not used for training.

Target variable

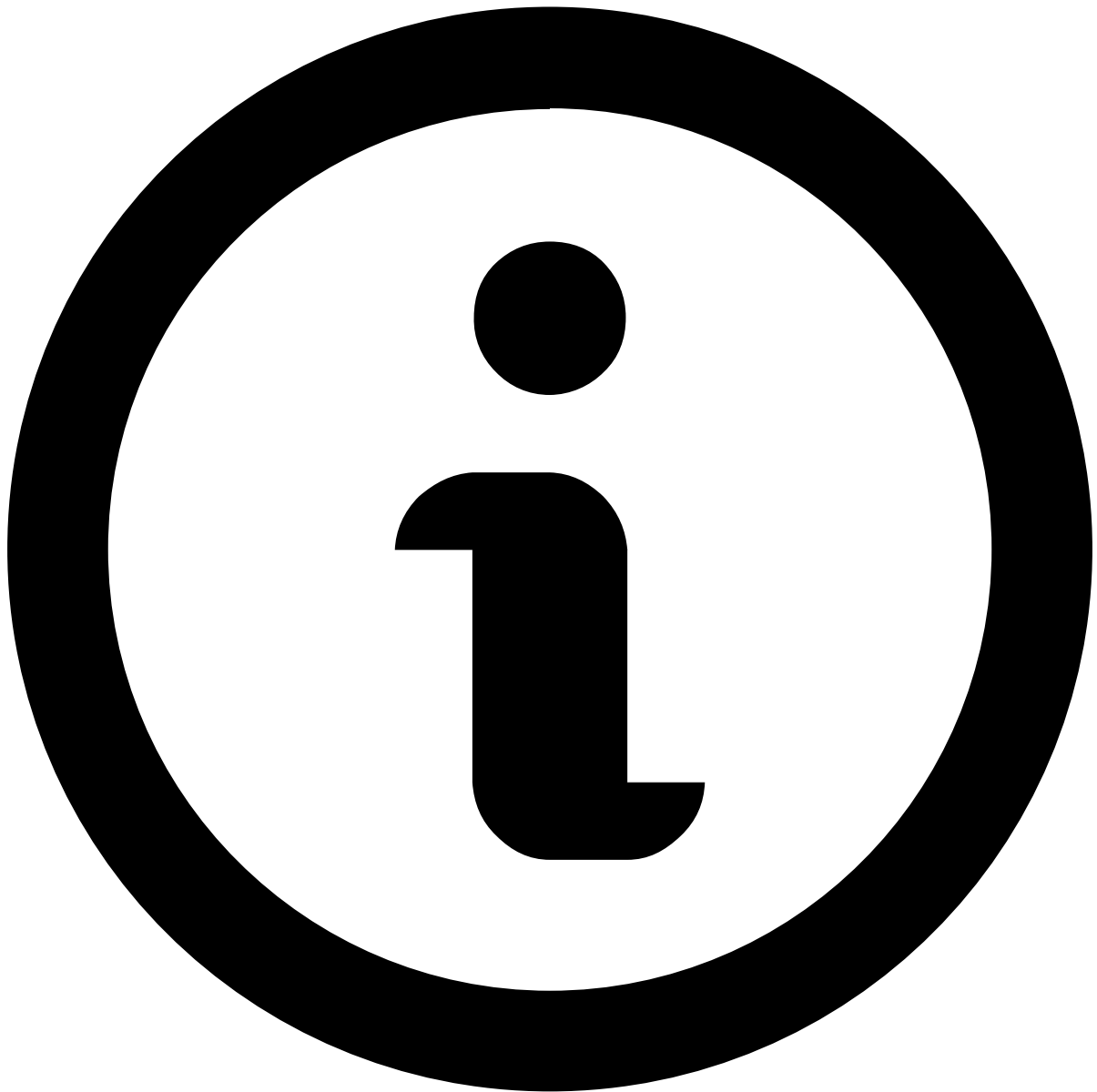
The purpose of a classification model is to predict the label of the target variable. Although Pulse's main use case is fraud detection, you can run your [data science loop](#) and train your models to predict the value for any [categorical](#) target variable, not just fraud/not-fraud. This way you can solve the following classifications problems, during both the data science stage and in Production:

- Binary classification, for example fraud/not fraud.
- Multiclass classification for target variables with more than two categories.

As mentioned before, unsupervised machine learning models don't require a target variable for training.

External models

Instead of creating and training your model in Pulse, you can [import a model](#) that you trained in an external system (using DataRobot, H2O, scikit-learn, the caret package in R, or custom code written in Python or R).



Known limitation - DataRobot

DataRobot allows training models based on string fields, but Pulse doesn't support that. To achieve the same in Pulse, convert the fields that have string type to categorical fields, before training a model based on that data.

Validation method

When training a classification model, it is important to validate the model performance with an independent set of data that was not used to train the model. Pulse automates the task of validation and allows you to select from the following options:

- Validation using the split percentage method for a given ratio
- Cross validation up to 20 folds
- Validation against a second data source
- No validation



Memory consumption of training models

Pulse executes model training jobs as a distributed job. It's recommended to monitor and optimize the memory

allocated to distributed jobs, to ensure that they have the necessary resources to run successfully. In case the distributed job encounters memory issues, consider the following:

- During model training, the training data set is fully in memory.
- For the split percentage and cross validation methods, the evaluation data set is also fully in memory.
- Validation against a second data source executes as a separate job. Therefore, it does not impact the memory of the training.

Split percentage 📄

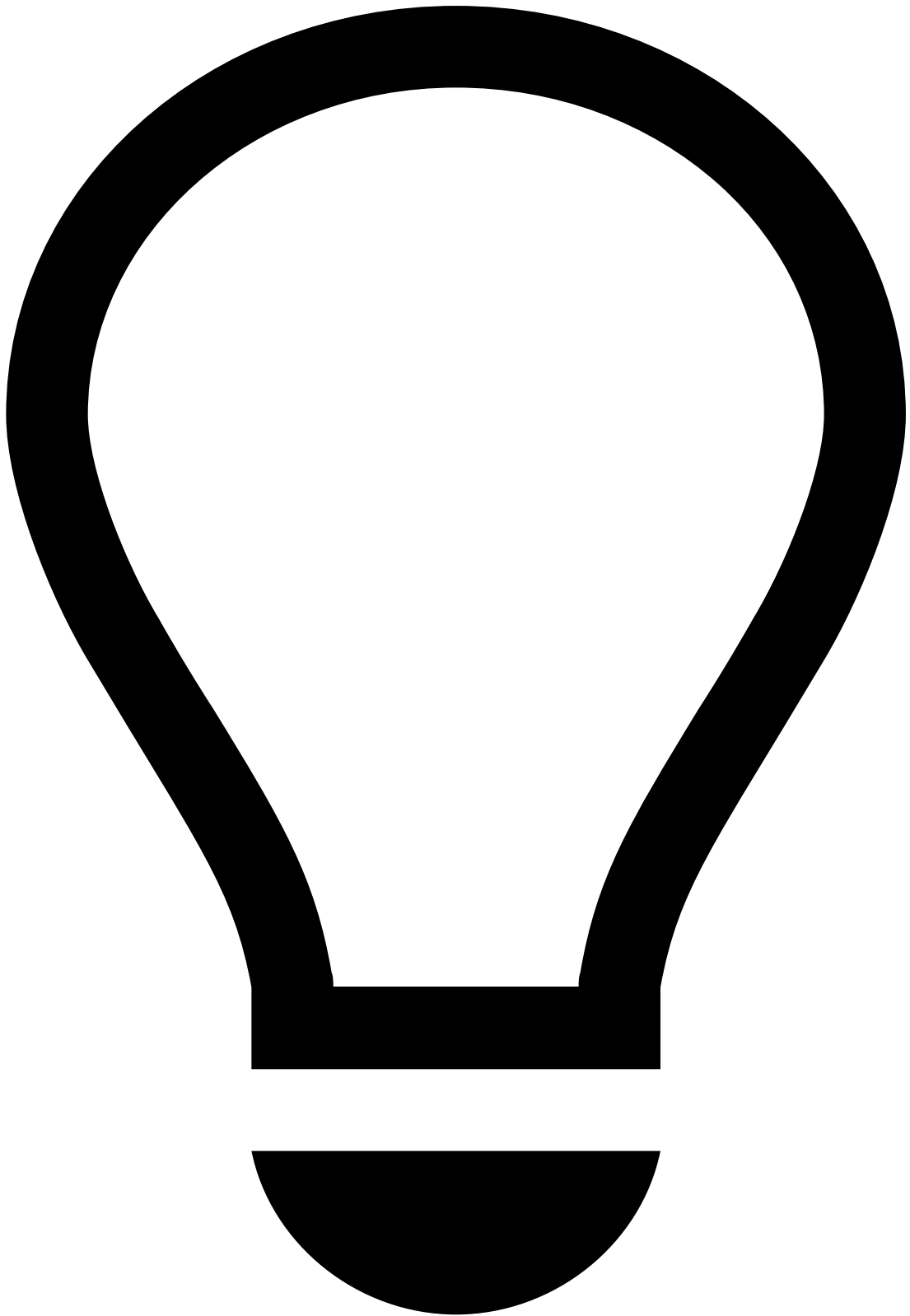
Split percentage of an $X\%$ ratio just uses that amount of data for training and the remaining $(100 - X)\%$ for validation.

The model that is stored, however, is trained with 100% of the data. This validation method is simple and fast.

Although the split is done deterministically, the obtained performance metrics is just a single data point, hence this validation method is not reliable for time-aware datasets. In addition, because we don't know the natural order of the instances of the dataset, if the specific problem being tackled is time-based (e.g. fraud detection), then we want to guarantee that the transactions are ordered in time before doing the split. Otherwise, we may be training the model with "future" patterns and testing with past ones.

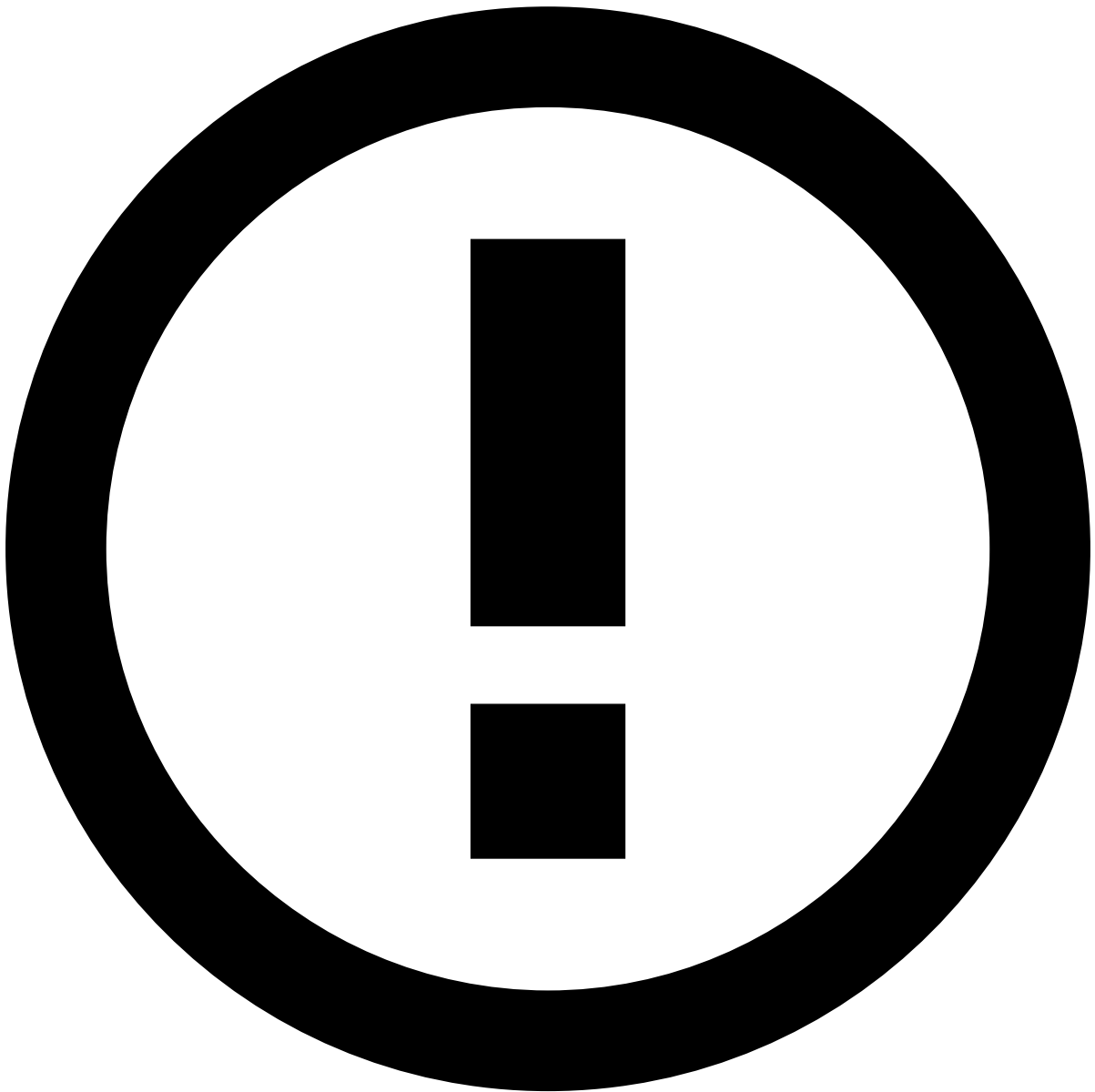
On the other hand, if all hard-to-classify instances are placed in the training dataset, the results will be inflated since it will only need to validate against easier instances. These are all important considerations to take in any type of evaluation in addition to the split percentage.

A default value of 70% ratio is a common practice.



When to use split percentage

You should opt for the split percentage method for quick iterations, especially if your dataset is very large, and for non-time-aware problems.



What makes an instance easy for a binary classifier?

Models attempt to extract patterns from the observed data. With more unique instances, the model is able to better understand what the target concept is, and how to define it (Is this transaction fraudulent? Yes, because ...).

Intuitively, this reasoning only has significance inside the domain of observed data. When scoring an instance that *looks like* fraudulent transactions that were previously observed by the classifier, it can easily tag the instance as fraud.

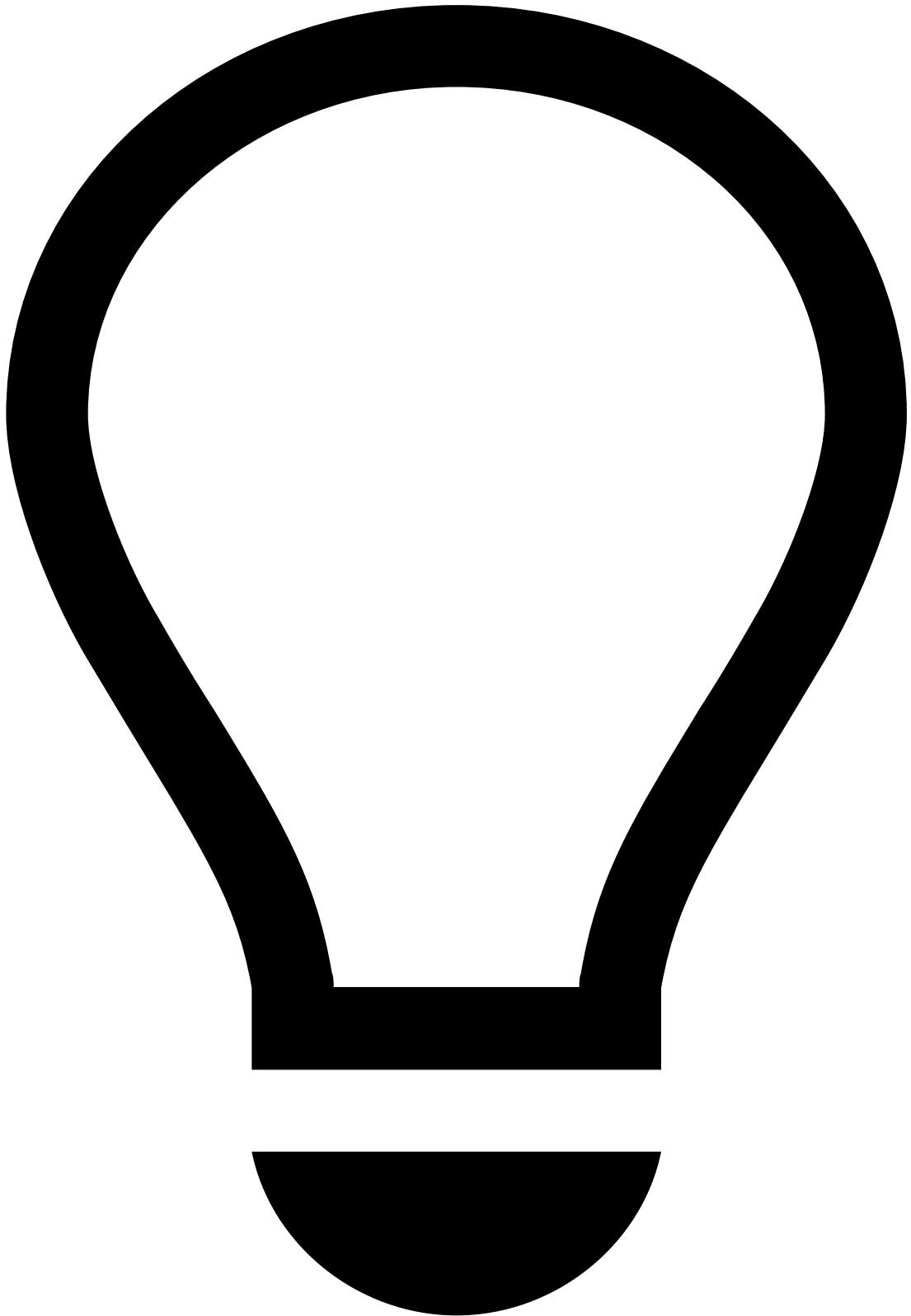
However, when there are no transactions that look like the one requested, or there are a few but they have mixed classes, this decision becomes harder.

Cross validation 📦

Cross validation is a method that attempts to minimize the variability observed with split percentage. It splits the data into N equally-sized blocks, and uses $N-1$ blocks for training and 1 for validation. The training/validation process is repeated by shifting the blocks until all N blocks have been used for validation. Choosing $N=10$ would yield the following sequence of training and validation runs:

The results are then equally averaged creating a final rundown. The averaging process inherently minimizes the variability of the result. However, the greatest advantage over split percentage is that each instance appears both in the training and in the test dataset (in different runs) thus ensuring that each instance influences the final result.

Nonetheless, cross-validation requires training $N-1$ models, which might be unwieldy for large datasets.



When to use cross validation

You should opt for this method in the final iterations of your [data science loop](#) when micro-optimizing the model definition, or if the training operation is fast enough that doesn't break your work cycle.

You shouldn't use it for time-aware problems.

Validation against a second data source

This validation method allows you to validate the model against a data source of Pulse, other than the training data source.

The data fields selected for the validation must match the fields of the training data set.

Sampling strategy

Sampling strategies are techniques that artificially modify the class distribution of the dataset. This can be used for biasing the model creator into believing that the class distribution is different from the one observed in the dataset. The class distribution is heavily biased in the real world (such is the case with fraud). Class imbalance is a widely known problem that affects many machine learning models. The problem arises when the number of instances from one class is so low that they become mostly irrelevant for training purposes. This generates classifiers that do not have enough sensitivity for the uncommon class (failing to detect fraudulent transactions and instead opting to score every transaction as valid; this has better accuracy compared to misclassifying a few instances of both classes).

When defining the sampling strategy, you can specify the $X\%$ that you would like the dominant class to have. The strategy will transform the dataset in a way that ensures this ratio.

Imagine that your training dataset consists of 99.5% non-fraud transactions and 0.5% fraud transactions, and you wish to train the model with 95% and 5% respectively.

You can achieve the desired ratio with either an undersampling or an oversampling filter both with 95% ratio (because non-fraud is the prevalent class).

- **Oversampling**

Produces more data of the minority class (by copying) to achieve the desired ratio. The instances of the dominant class are kept intact and the dataset grows in size.

- **Undersampling**

Deletes instances from the majority class to meet the ratio. Instances of the minority class are kept intact and the dataset shrinks in size.

Depending on your dataset, you should use a different sampling strategy. On the one hand, oversampling introduces more instances but the information contained in them is the same. This makes the dataset bigger (but not better) and therefore the training process is slower. On the other hand, undersampling loses information that was present in the deleted instances.

Improving models

A model is defined by a set of parameters: some are defined by the user, whereas others are selected based on the training data when applying the algorithm to the data.

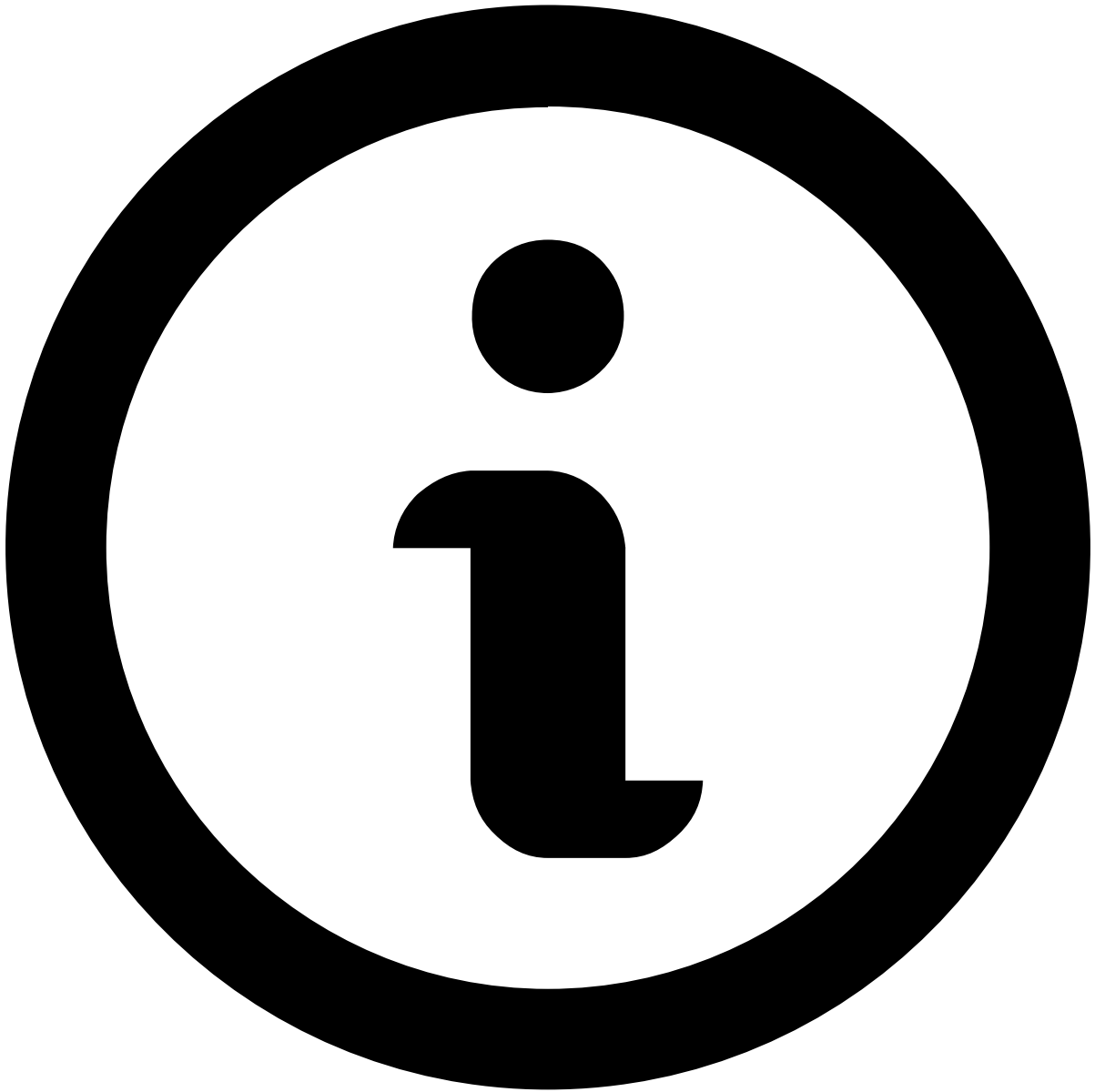
The performance of the model can be improved by:

- Optimising the hyperparameters (parameters that can be selected by the user). This process is named hyperparameter tuning.
- Comparing different machine learning algorithms. For example, XGBoost vs Random Forest vs Deep Learning.
- Changing the training dataset. For example, to a more recent dataset.

Adjust the feature set that is being used to train the model. Some patterns might not being captured by the current feature set. Bear in mind that for a fair and insightful comparison you should be assessing the performance in the same data set.

Configuring white-box explanations

During model training, users can configure the **white-box explanations**, which consists in the calculation of the features that led to a transaction's score. This calculation occurs in runtime after adding the **trained model** to the workflow.



note

Although white-box explanations are configured in the Pulse UI, their output is only visible in Case Manager.

For more information on the parameters that must be configured in this field, visit the [Training a Model](#) page.

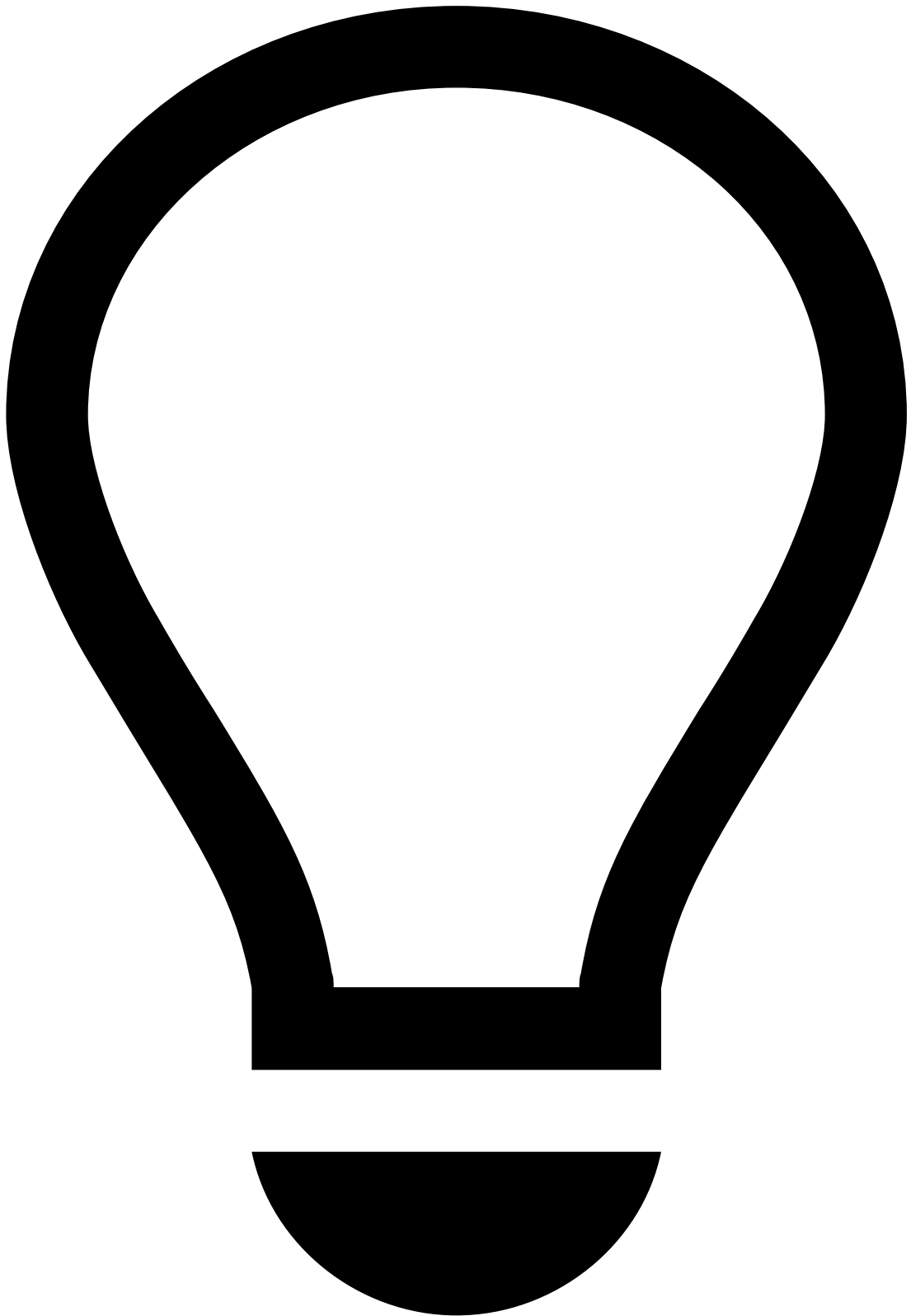
For more information on this subject, read the [White-Box Explanations](#) page.

Immutability

Since a model is immutable by definition, once you train it with a specific dataset and machine learning parameterization, it will always yield the same results for a given test dataset. If you change the parameters or the training dataset, you are effectively generating a different model.

In alignment with this, when you create a model in Pulse, the configuration you used to train it cannot be changed afterward. Once you configure algorithm parameters and the input data source when [training a model](#), these are permanently part of what distinguishes a model iteration from the previous one in your project.

So if you change any model dependency such as the data source, which is also immutable, you are creating a new iteration in your [data science loop](#). In addition, immutability allows Pulse to isolate the model during the data science phase from the same model when deployed to Production. This way you can safely correct and improve your models without affecting your Production workflow.



Deploying improved models

If you need to deploy corrections and improvements to your current model, you can simply run the training procedure and generate a new model. Then configure the scoring service in your workflow with the new model.

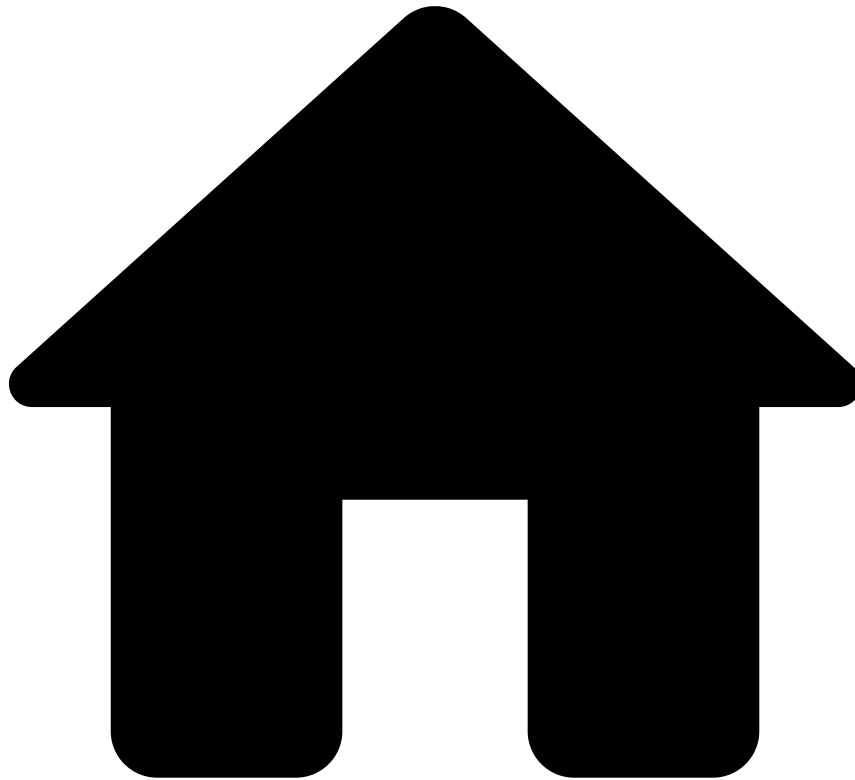
Nevertheless, Pulse still allows you to effectively manage your model project and the performance of multiple model iterations. [Model evaluations](#) are what exposes results and not the model itself. A model evaluation always produces the same results over a given model, reinforcing the need for different models for each iteration in your project.

Learn more

Check the following pages in the User Guide to learn how to put models into practice:

- [Training a Model](#)
- [Importing an External Model](#)
- [Model Evaluation](#)

•



- [Core Concepts](#)
- [Models](#)
- White-Box Explanations

On this page

White-Box Explanations

Was this page helpful? 

White-box explanations (WBE) is a tool that aims at providing explanations for predictions of machine learning models in Case Manager (CM). It reduces the amount of time data scientists spend setting up rules that will generate explanations in CM (i.e. less than 2 hours) and allows analysts in CM to understand why a transaction received a certain score, including a breakdown of the contributions to that score.

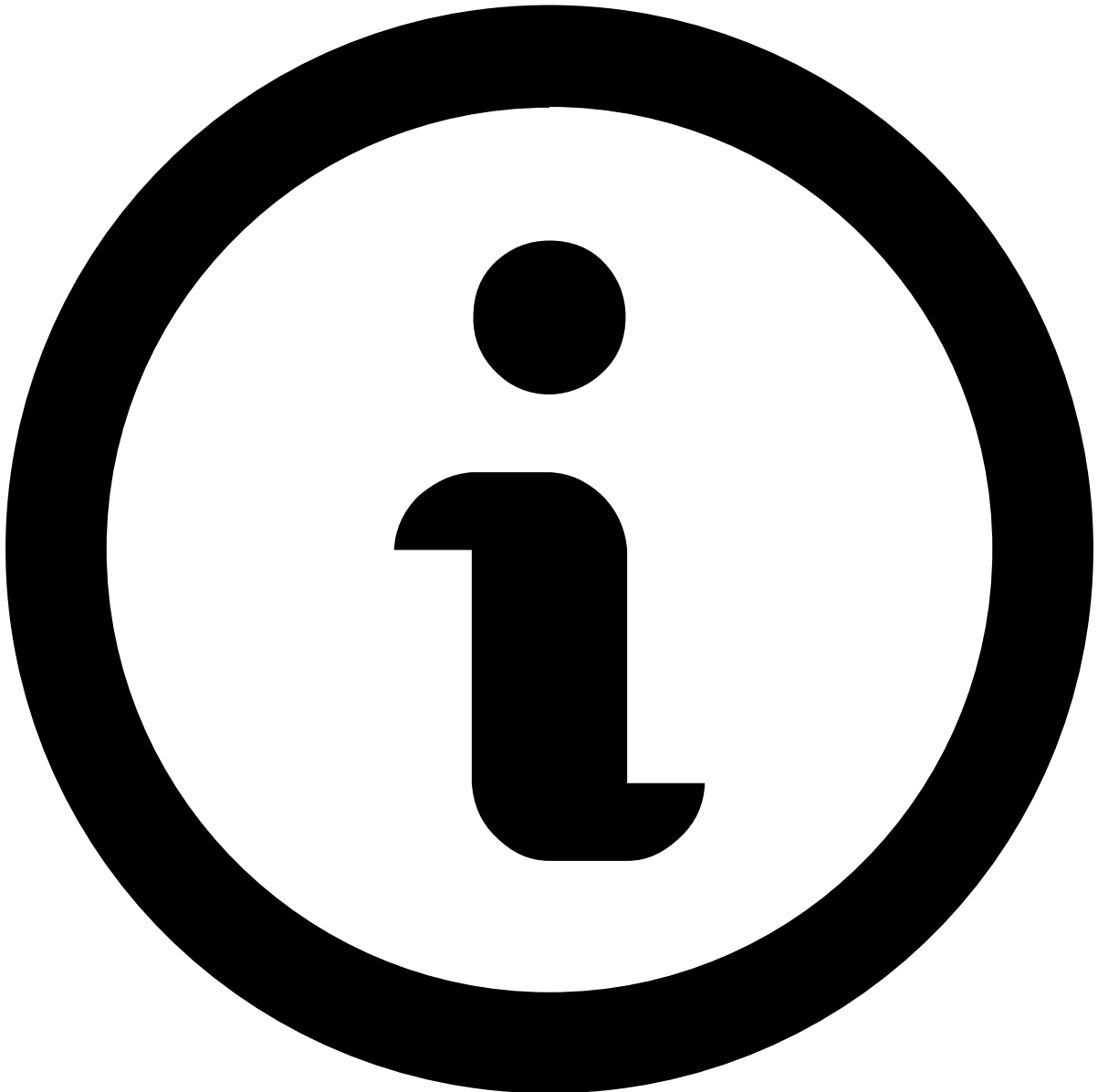
As a result of providing explanations for each transaction score in CM, WBE supports analysts in two main aspects:

- Guidance on how to proceed when they need to understand the reason behind a transaction alert.
- Insight on why the model is delivering high scores to certain transactions.

Introduction to WBE

WBE lists the most important features that influence a transaction's score in CM, allowing analysts to understand the reason behind that score.

The flow of WBE starts in the model, where you can configure explanations. During this configuration, data scientists can add explanation messages. After that, the model is put into Production (i.e. workflow), where it will score and create explanations for transactions. Finally, analysts using CM will be able to see the explanation messages previously generated by Pulse at the end of the workflow.



Explanations in model evaluation

Currently it is not possible to generate explanations in the model evaluation.

WBE algorithm⚠️

Depending on the algorithm you use to train the model, the following algorithms can be used for the explanations:

- **Feedzai Feature Contribution:** an algorithm based on the fraud ratio gain, for models trained with the [Feedzai Random Forest](#) algorithm.
- **TreeSHAP Feature Contribution:** an algorithm based on [Shapley values](#), for models trained with Microsoft LightGBM or Feedzai FairGBM.



Note

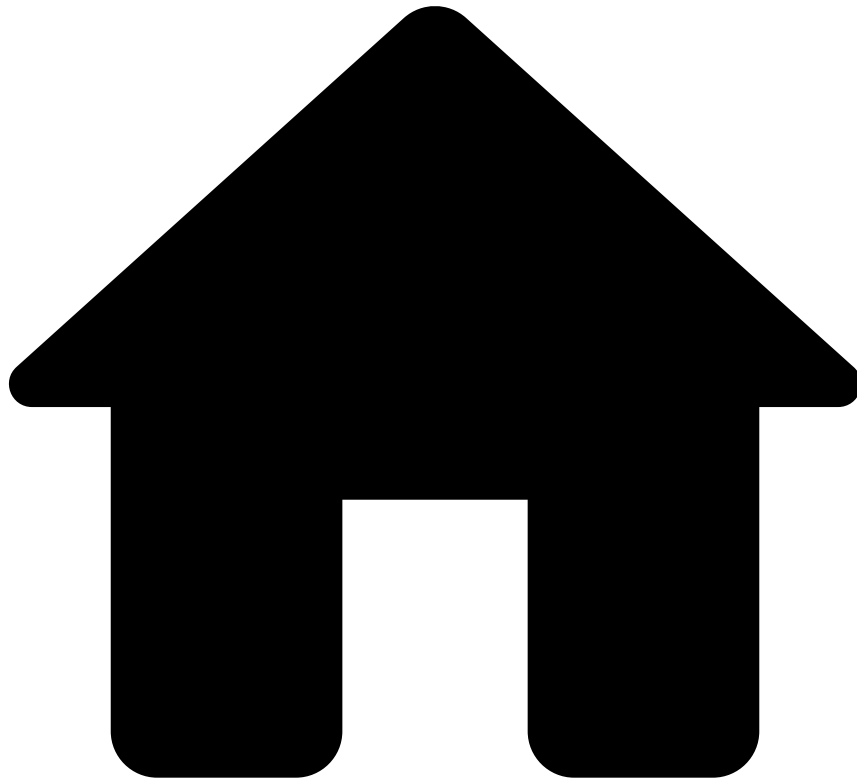
Explanations provided through Shapley values, which are our Feature Contribution (FC) weights, are computed based on the prevalence ratio of the dataset (e.g., the rate of fraudulent transactions) as the base value. Each FC weight demonstrates how much that feature contributed to the difference between the prediction score and that base value. Therefore, if the prediction score is higher than the base value, the sum of the normalized FC weights must be 100% of the prediction score. However, if the prediction score is lower than the base value, the sum of the normalized FC weights will be -100%. This is a rare event in financial crime use cases, given the low rate of financial crime when compared to the whole set of transactions.

Learn more

Check the following pages in the User Guide to learn how to put WBE into practice:

- To set up WBE, go to the [Training a Model](#) page and read the Training Parameters section.
- To check how WBE are produced in runtime, read the [Adding Models](#) page.

•



- [Core Concepts](#)
- [Plans](#)
- Data Partitioning for the Metrics Operator

On this page

Data Partitioning for the Metrics Operator

Was this page helpful? 

Data partitioning is a key step for processing distributed jobs in a Spark cluster spawned by [plan executions](#) to enable Pulse to handle large volumes of data efficiently.

When you add [metrics operators](#) to a [plan](#), Spark needs to re-organize the data before calculating the metrics defined in each metrics operator. During this data re-organization, all the rows (e.g., in fraud applications each row corresponds to a transaction) that share the same value in the fields that are configured as *Group By* in the metrics operator are placed within the same Spark partition.

Pulse allows you to configure data partitioning using flexible UI options to ensure that Spark performs data re-organization as homogenous and well distributed as possible.

How does it work?

To get the best out of the data partitioning options available in the UI, it is important that you understand the logic behind this concept.

When you complete the configurations of your plan with all the required operators, you can generate the output data source by running a plan execution. The plan execution spawns distributed Spark jobs that are responsible for handling the heavy work so that Pulse can process large volumes of data (i.e. often hundreds or even thousands of TBs).

Spark streams data through its graph of operators, which maps reasonably well to the graph of operators that you configure in the plan's workspace that is available in the Pulse UI. This means that, in general, every row of the input data (e.g. in fraud applications each row corresponds to a transaction) is fully processed from the source to the sink.

If you have a plan with metrics operators, **the following data partitioning logic is applied** when you run the plan execution:

1. Spark streams all the data and process upstream operators (i.e. processes all the data that flows through the operators that precede the metrics operator).
2. The data that arrives at the metric operator is re-organized according to each set of *Group By* fields (e.g. Set 1: *card_number + merchant*, Set 2: *city*) defined in this operator. This process also is known as shuffling in the Spark terminology. For each set of *Group By* fields, all the rows that share the same value in the fields as defined in the *Group By* are placed within the same Spark partition.

For example, consider that you define a set of *Group By* with the fields *card_number + merchant*.

All the transactions that share the same Card Number AND the same Merchant are placed within a single Spark partition. The example shown below uses only the *Merchant* as the *Group By* field to provide a simple visualization of this process. Note that one Spark partition can (and will likely) contain data from various sets of *Group By* fields.

3. Pulse takes each partition and feeds it in streaming to PKernel, which is the Feedzai's stream processor for profile calculation. In this step, PKernel is responsible for calculating the metric defined in the metrics operator using data organized by *Group By* as explained in the previous step.
4. Steps 2 and 3 are repeated for each set of *Group By* fields so that all metrics are covered with a shuffle. For instance, if you have two metrics, the step sequence 2-3 is repeated twice as shown in the example below:
 - Iteration #1 (e.g. Metric 1: *card_number + merchant*): Spark re-organizes all the data by *Card Number* AND *Merchants*, and then PKernel performs the profile calculations.
 - Iteration #2 (e.g. Metric 2: *city*): Spark re-organizes all the data by *City*, and then PKernel performs the profile calculations.

The challenge of unbalanced data

A typical data partitioning method is to hash the values of each row and then send the rows that share the same hashed values to the same partition. However, when dealing with unbalanced data this method might introduce some inefficiency.

For example, imagine that you configure a metrics operator with a metric that has the *Group By* field *Merchant*. Now assume that you have two large merchants (Retailer 1, Retailer 2) each of which represents 1/3 of the total data, and then the remaining 1/3 of the data is a long-tail of very small merchants (Merchant 1, Merchant 2, ... , Merchant N). If you use the hash approach to partitioning data, there is nothing from preventing the most popular values from falling in the same partition:

This unbalanced data distribution is oblivious to the population and would require extra memory in the Spark workers to compute the data that is shown in Partition 1.

Continuing with the example above, an efficient way of dividing the data into different partitions would be the following one:

The data-aware optimizer available in Pulse allows you to set a plan execution that spawns an efficient data partitioning to maximize the memory allocated to the Spark workers. This option is specially advantageous when you are dealing with unbalanced data because it takes into account the popularity of each set of values for *Group By* fields.

Data partitioning options

Pulse has the following data partitioning options available:

- hash-based partitioning
- manual partitioning
- data-aware optimizer

The data-aware optimizer is the default option for data partitioning, but you can configure a different partitioning method to be used by default in the main configuration file `$PULSE_HOME/conf/pulse.properties`, under the property `pulse.global.ui.plans.datapartitioning.method`.

The available values for this key are `data-aware`, `hash-based` and `manual`.

Hash-based

The **hash-based** option does not take into account the popularity of each set of *Group By* fields when performing the data re-distribution into different partitions.

Additionally, this method does not optimize the number of partitions used by the shuffle. In fact, the hash-based method sets the same number of partitions used in the plan input. This number is obtained by reading chunks of 128MB from the input and making each chunk a partition.

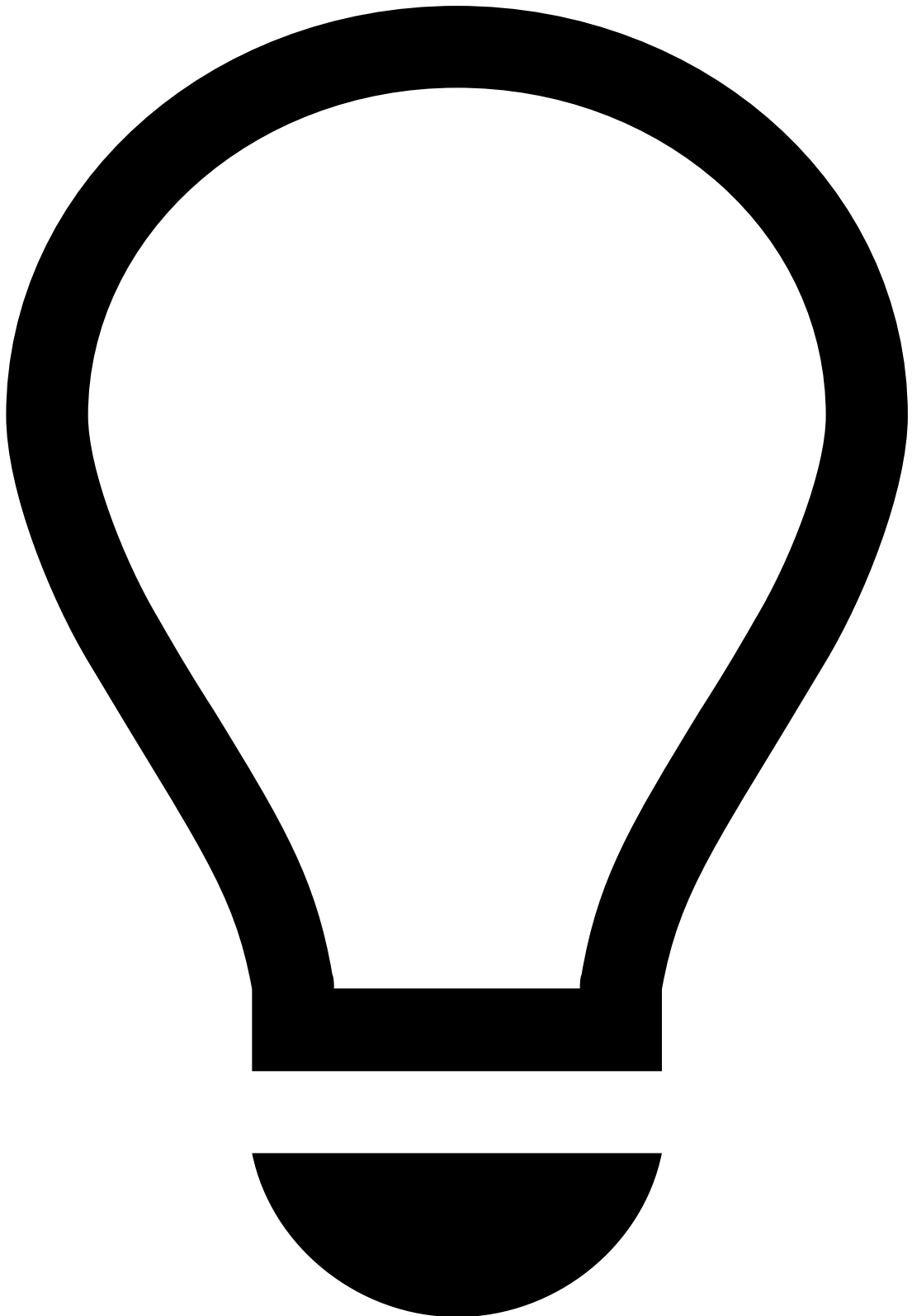
On the plus side, this method is less computationally expensive than the popularity aware method explained below. This option might be beneficial when you are computing metrics that have *Group By* fields that are expected to lead to a balanced data distribution.

Check the pages [Creating a stratified sample](#) and [Creating metrics](#) to learn how to choose the hash-based option.

Manual Partitioning

The **manual partitioning** option allows you to specify the number of partitions to which the values of your dataset are distributed. Manual partitioning uses the hash-based approach to distribute the rows with the same values to the same partition according to each set of *Group By* fields.

Check the pages [Creating a stratified sample](#) and [Creating metrics](#) to learn how to choose the manual partitioning option.



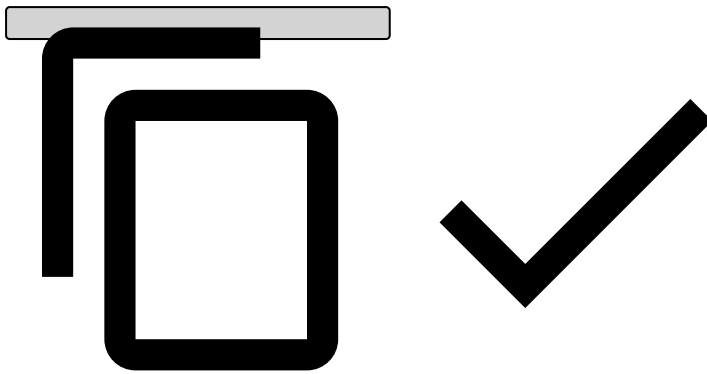
Configuration of manual partitioning

When selecting the `manual` partitioning option, you can manually set the number of partitions to a custom value in the Pulse UI. The default number of partitions is 100.

If you regularly need to specify a different number of partitions, you can set a custom number of partitions via configuration.

For example, to use manual partitioning with 20 000 partitions, you can set the following properties in the main configuration file `$PULSE_HOME/conf/pulse.properties` :

```
pulse.global.ui.plans.datapartitioning.method=manual  
pulse.global.ui.plans.datapartitioning.partitions=20000
```



Data-aware optimizer

The **data-aware optimizer** option takes into account the popularity of the values according to each set of *Group By* field.

The number of partitions used by the shuffle are dynamically computed, and they adapt to the actual data. The data-aware optimizer uses the minimum number of partitions that allows for a homogeneous distribution of the data that respects the set of *Group By* values.

The data-aware optimizer provides the following configurable options:

- **Most important K cut-off:** The number of the most popular entities (i.e. a set of equal values) that are tracked for collecting statistics and re-organizing data into different partitions. You can choose a large *K cut-off* value (e.g. 50 000).
 - Consider the example in which you configure a metrics operator with a metric that has the *Group By* field *Merchant*. Assume again that you have two large merchants (Retailer 1, Retailer 2) each of which represents 1/3 of the total data, and the remaining 1/3 of the data is a long-tail of very small merchants (Merchant 1, Merchant 2, ... , Merchant N). If you choose the *k cut-off* value equal to 2: the data associated with Retailer 1 and 2 will be re-organized using then data-aware optimizer option, whereas the remaining rows are re-organized with the hash-based approach.
- **Maximum rows per partition:** The number of maximum rows that fit on each partition. For example, if you know that each row in your dataset averages to 1KB, and that your Spark executors have 4 GB of heap for data (because they have 12 GB available for the executor, e.g.), then it is acceptable to configure 4 million rows per partition.

Check the pages [Creating a stratified sample](#) and [Creating metrics](#) to learn how to choose the data-aware optimizer option.

Computing statistics

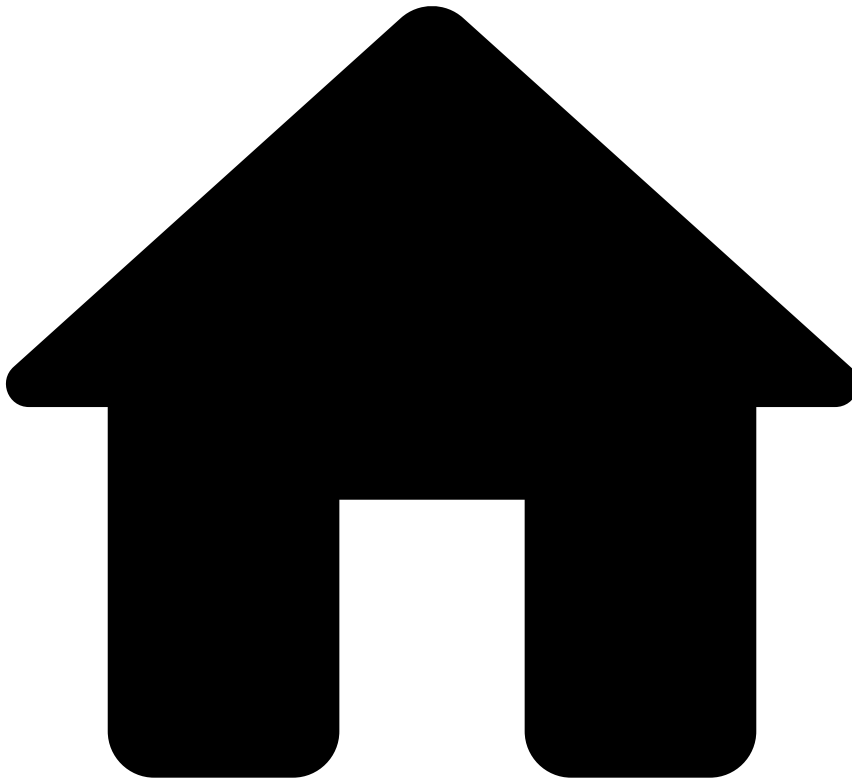
Pulse logs statistics into the logging files of the job when you run a plan execution with the data-aware optimizer option. This helps data scientists to know how the data is being partitioned and, in case of failing to compute a plan, it might also help them to understand the reason behind the failure.

For each set of *Group By* fields that is shuffled (e.g. set1: "card-number"+"merchant"; set2: "device-id"), the following data is computed:

- Median
- Average
- Top 10 most predominant set of *Group By* field values and the corresponding cardinality.

This statistics can be accessed in the log folder, using the logging level DEBUG.

•



- [Core Concepts](#)
- [Plans](#)
- Operator Reference

On this page

Operator Reference

Was this page helpful? 

When designing a [plan](#) to [process your data](#), the following operators are available:

Operators in deployable plans

Source

Defines an original data source the plan takes as input. Uses the schema of the data source to define the input format for the plan.

A [non-deployable plan](#) can take several sources, for example, coming from different files.

Mapâ

Performs major data transformations on your datasets. Lets you add new fields to the current data schema, remove existing ones, and update expressions that define how the value of a field is calculated. You can also reorder the fields on the "Add Fields" tab of a map operator in a manner that better suits your work.

Use the map to manipulate your data, [clean up your data](#), transform your data, perform calculations on existing fields to [derive features](#), etc.

You can configure field additions and updates with advanced expressions implemented in [UDFs](#).

Filterâ

Applies an expression to the data and narrows down its scope.

Use the filter to, for example, obtain only transactions from a specific merchant that require different processing.

You can apply filters with expressions implemented in [UDFs](#).

Listâ

Accesses the content of a list and stores it to a new field which gets added to the events that traverse the list operator.

Metricsâ

Generates measurable information, aggregates results and gives you valuable insight on your data, for the [time windows](#) you define.

Calculates all selected aggregations for all possible combinations of selected time windows and grouping fields.

Available aggregations are **avg**, **count**, **max**, **min**, **sum**, as well as advanced expressions implemented in [UDFs](#).

Use metrics, for example, when in a [features](#) plan you want to calculate the total amount (**sum**) spent by each card (**Group By**), in a single day (**Window**).

Sampleâ

Reduces the current data to a configurable percentage of the original size. Sampled values are randomly selected using the sampling percentage as the selection probability.

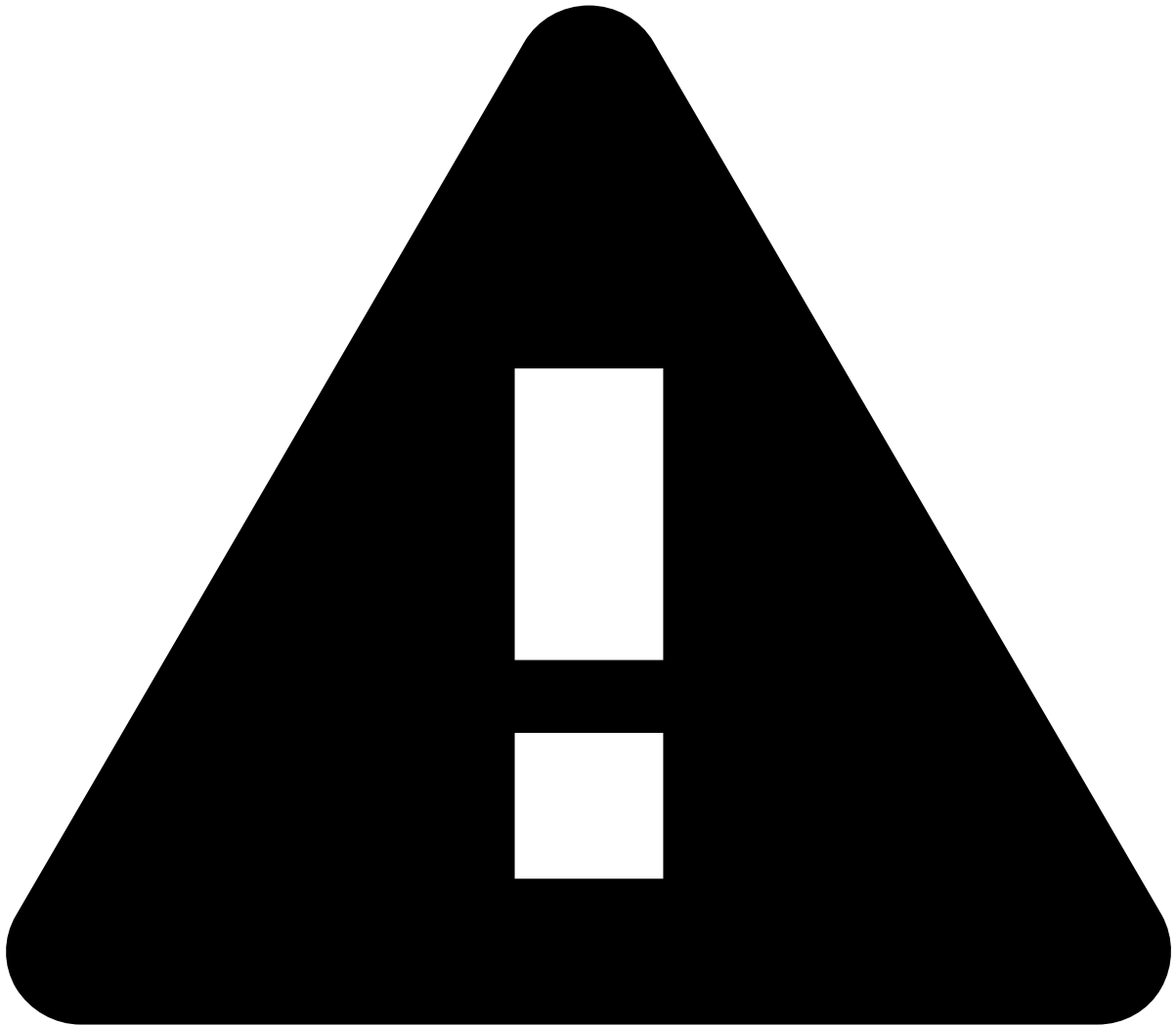
Sampling a dataset multiple times, with the same percentage, will always return the same resulting data.

Use the sample, for example, when implementing a [Stratified Sample](#) to reduce the number of non-fraud transactions in your dataset.

Sinkâ

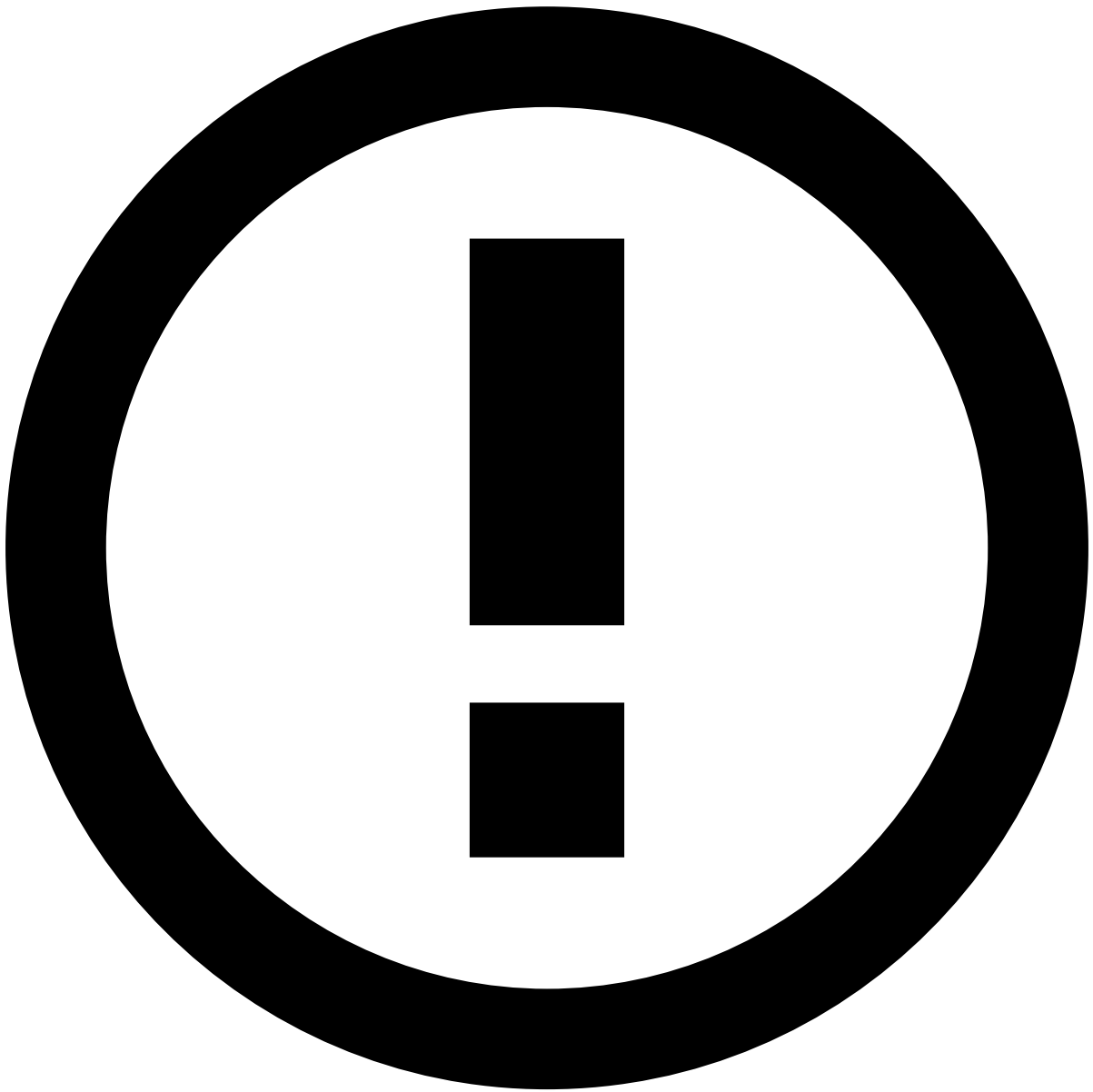
Terminates the logical flow of the plan.

When you run the plan, the sink outputs the result of the plan to a new data source and to a file in your filesystem.



Non-deterministic functions

When you apply UDFs in Map, Filter or Metrics operators, the usage of non-deterministic functions can result in non-deterministic results. For example, when you include any randomness factor into your expressions.



Source and sink operators

Pulse automatically adds a source and a sink operator to deployable plans. You need only to configure them when executing the plan.



Clipboard

You can use the clipboard in order to copy-paste operators in plans. This helps you save time by allowing you to create plans more efficiently without the need of specifying all their fields each time you create a new plan, letting you use the operators you have already configured.

Operators in non-deployable plans

Some operators only make sense at a design and exploration stage in your data science project. When processing a data stream in real-time to classify instances as `true` or `false` (let's take the fraud detection use case as an example), these activities are not repeatable nor applicable, for instance when joining data from separate sources.

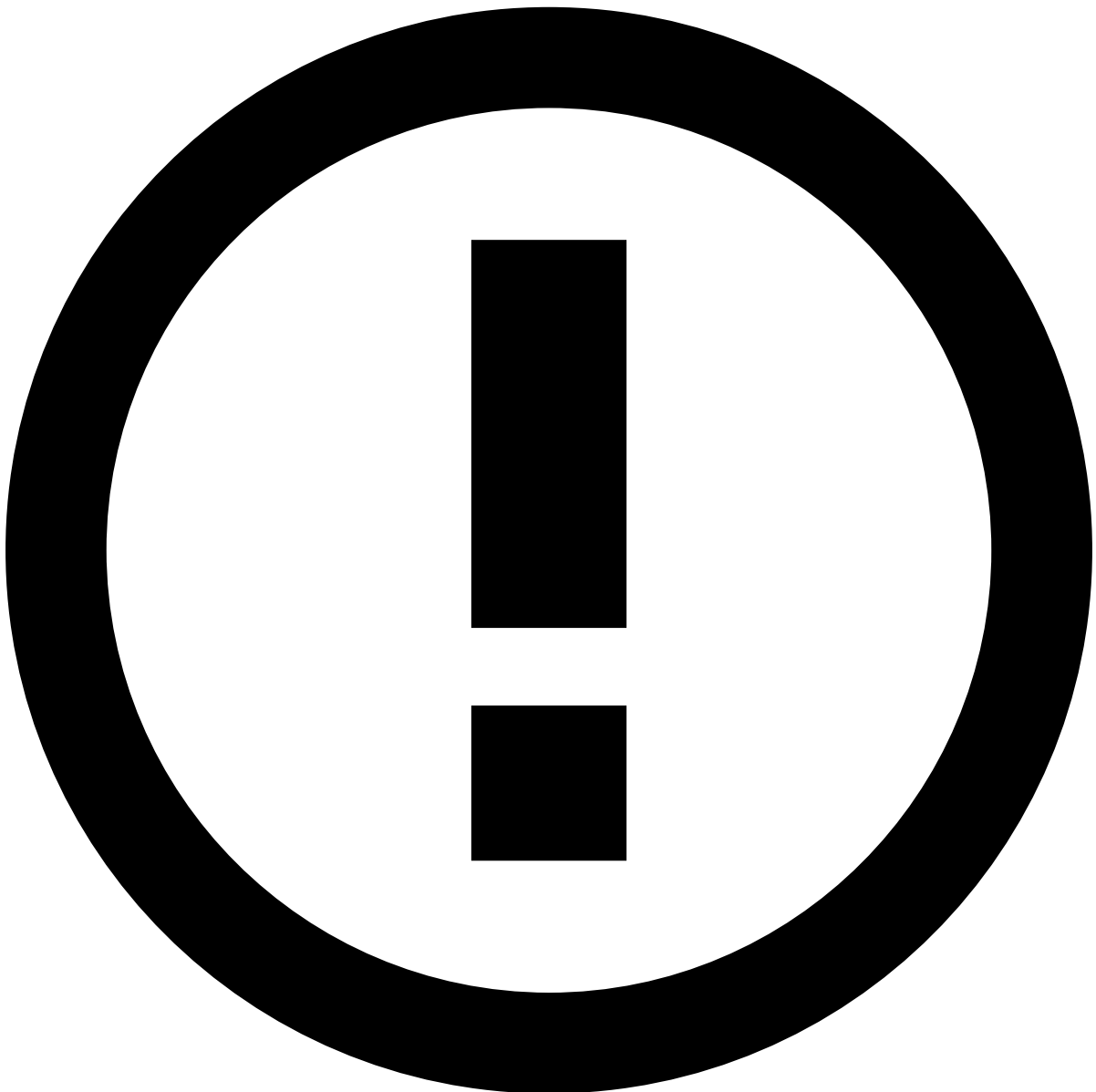
As such, the following operators are only available for [non-deployable plans](#), and perform typical data manipulation activities you might do in a database:

Group By

Groups data on a combination of fields, calculates aggregated metrics for each grouping, and outputs the result to a new field in the resulting dataset. Available aggregations are **avg**, **count**, **max**, **min**, **sum**.

For example, group by *card_number* and use the *max(amount)* aggregation over a one-month window to obtain the highest transaction amount for each card in the last month.

Lookup By



Availability of Lookup Metrics

The **Lookup By** operator is only available when using the [Railgun engine](#), and when explicitly enabled via configuration. To activate this functionality, reach out to the Feedzai Customer Success (CS) team.

Extends the capabilities of the **Group By** operator by providing a way to select which group to return the aggregated value of. (By default, **Group By** always returns the aggregation that matches the value of the grouping fields in the current row.)

The `Lookup By` operator allows specifying a retrieval field (or set of fields) that is different from the grouping field(s), thus overriding the default behavior of the `Group By` operator.

For example, group the data by *recipient_ID* and use the *sum(amount)* aggregation over a one-week window, and then use *sender_ID* as the lookup field to obtain the total amount received in the last week by the account that is sending money in a transaction.

Join

Combines rows from two workflow data paths that match on the specified fields.

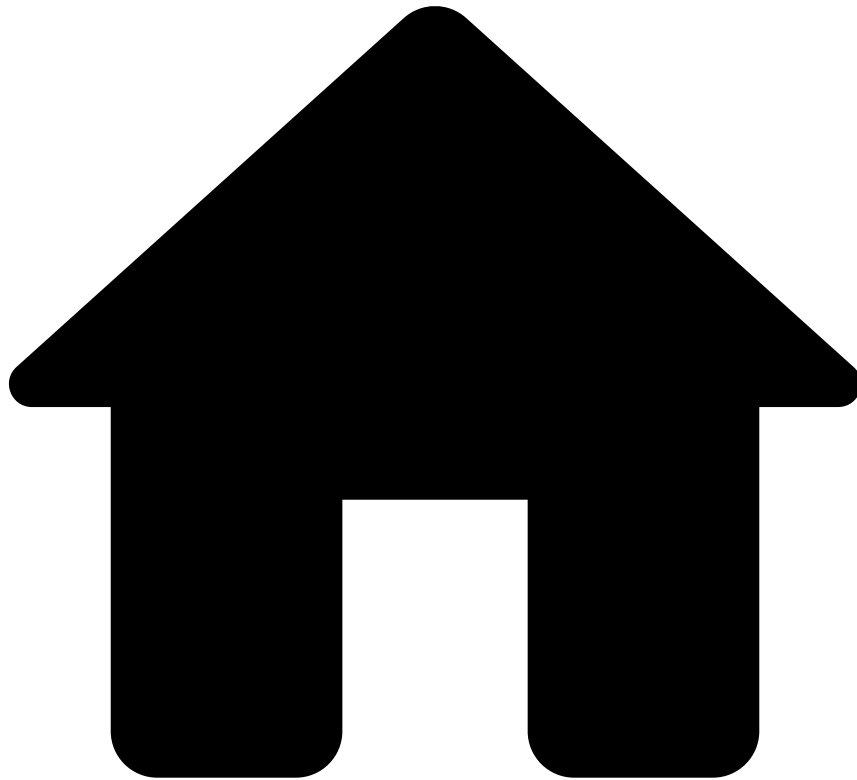
Allows for **Left**, **Inner**, **Right** and **Full** join.

Union

Combines the resulting data from two workflow data paths.

Both datasets must have the exact same fields with the same types, which can be specified in a different order. The output schema can be chosen when configuring the union.

-



- [Core Concepts](#)
- [Rules](#)
- Cost Fields in Rules

Was this page helpful? 

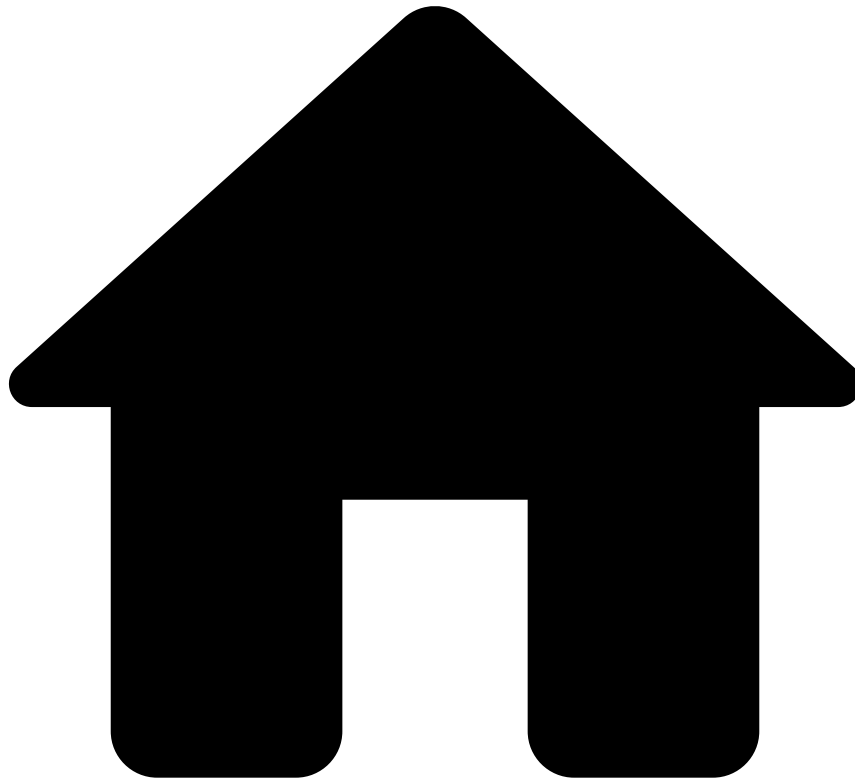
Cost Fields in Rules

A **cost field** is a field of the instances that is used as a weight for the metrics of the evaluations (of both models and rules), typically the transaction amount in a fraud detection scenario.

Using cost fields in model evaluations or rule evaluations allows you to transform any metric (which by default is count-based, e.g. Recall, Precision, etc.) into a cost-based metric (e.g. Money Recall, if the field chosen is a form of cost of the instance).

The following image displays cost fields included in a rule evaluation. This example depicts an evaluation of a block of rules where the plot shows the cost performance and the *Results* table displays the cost FPR and Recall.

•



- [Core Concepts](#)
- [Rules](#)
- Custom Evaluation Metrics in Rules

Custom Evaluation Metrics in Rules

Was this page helpful? 

An evaluation metric is a function that can be applied to the data obtained during a machine learning model/rule validation.

In a data science project, an evaluation metric allows you to inject business logic into an objective model/rule evaluation. This allows you to evaluate trained models/rules based on the business value they deliver.

You can add an evaluation metric to your project by defining a function that receives:

- The number of True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN), and outputs a number in an open domain.
- The cost-TP, -TN, -FP and -FN. The cost field variants are based on the weight a data field in the Instance, typically the transaction amount in a fraud detection scenario, rather than using a count-based approach.

The following image depicts where you can create a new evaluation metric. Check the User Guide to see how you can [define evaluation metrics](#), for example, in a model evaluation.

You can apply a custom evaluation metric, written by you and adjusted to your use case, to any existing model evaluation or rule evaluation.



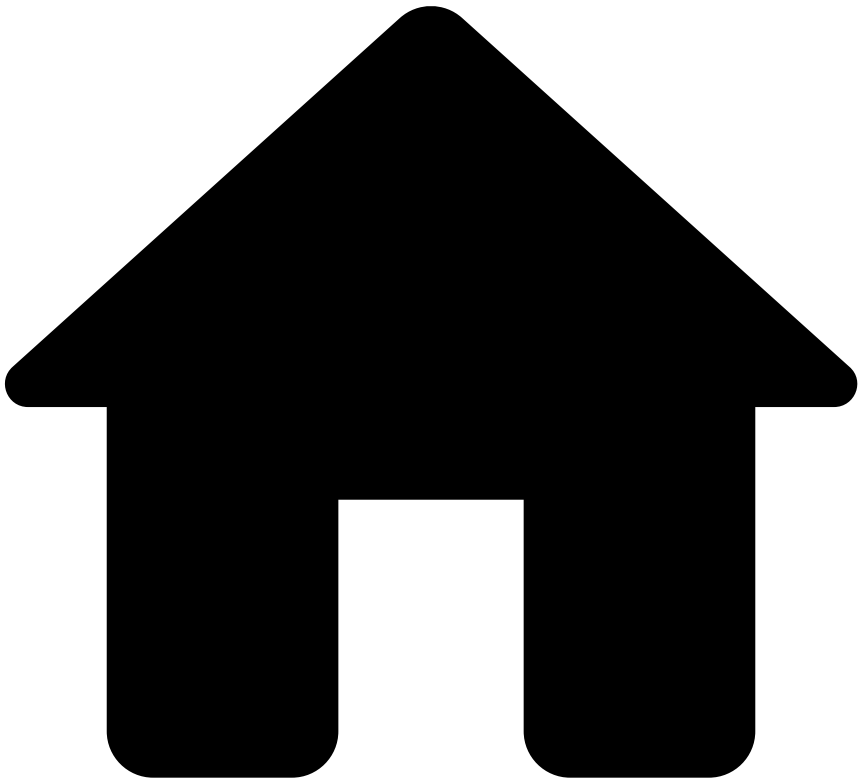
Availability

The values of cost-based evaluation metrics are available to model evaluations or rule evaluations only if you define a cost field for the evaluation.

If you do not define a cost field for an evaluation, then Pulse will not be able to compute your custom evaluation metrics and you will not have access to them.

The following image displays how you can add a custom evaluation metric to a model evaluation.

.



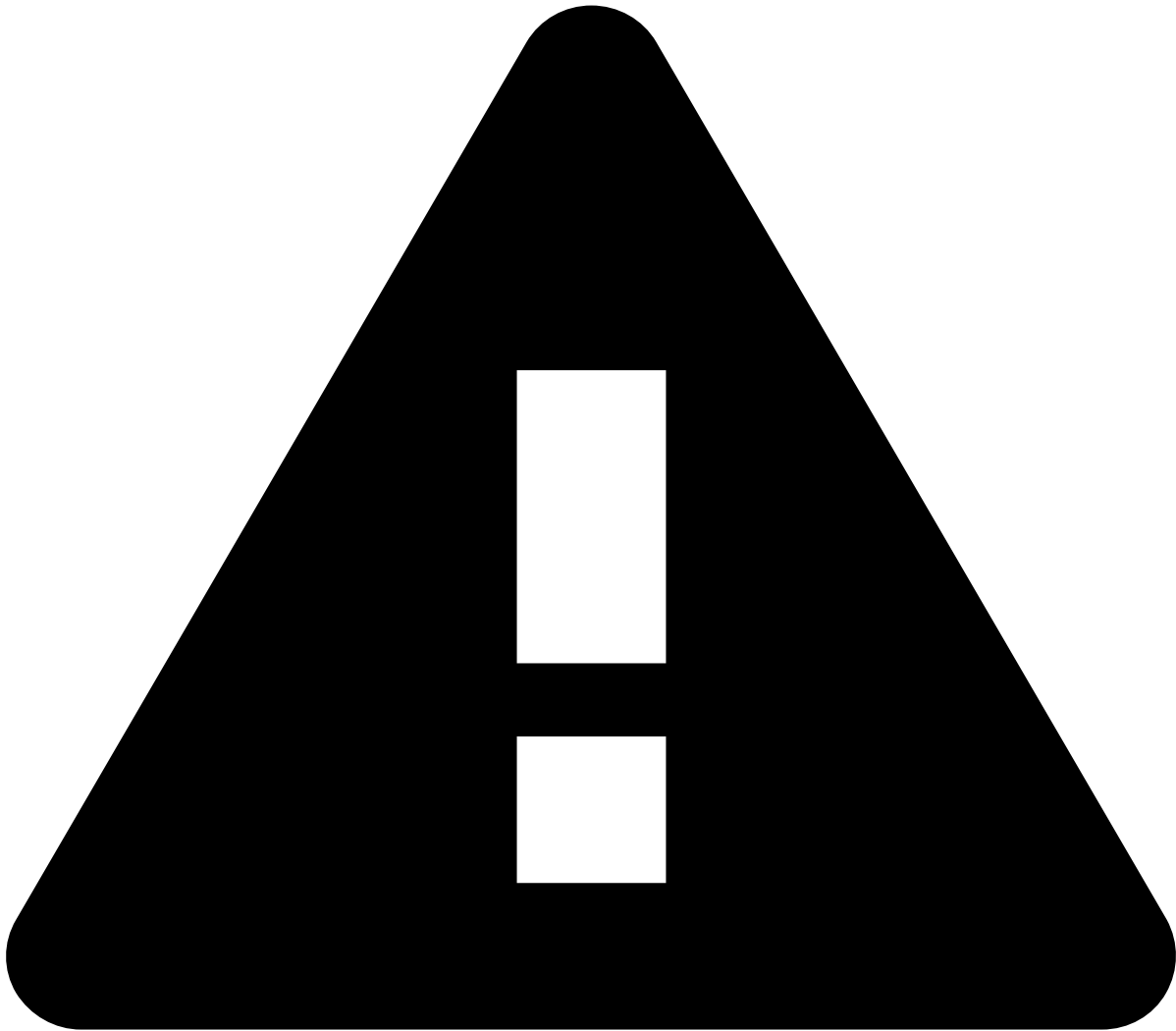
- [Core Concepts](#)
- [Rules](#)
- How Rules Work

On this page

Was this page helpful? 

How Rules Work

Rules can have lasting effects on both the workflow and the event itself.

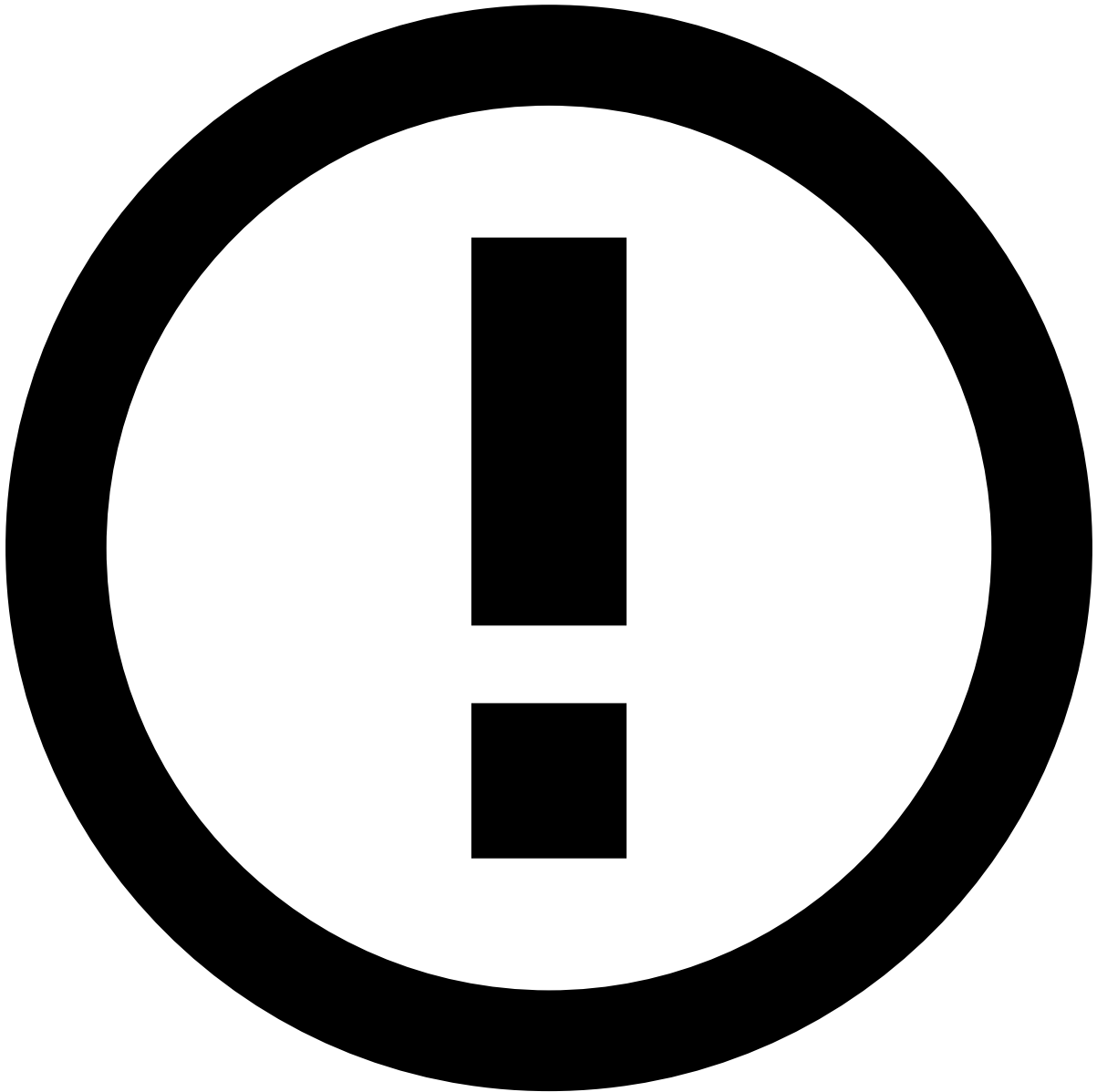


Workflow-halting rules

A single rule can stop an entire workflow. You must be careful when configuring the control flow of a rule.

If a rule is enabled and the [condition of a rule](#) is *true*, then Pulse executes the following actions:

1. An entry is added to score tracking.
2. One or more actions are triggered immediately.
3. The target variable of the event is set with a value as specified in the rule. For example, setting the *fraud* field as *true*.
4. The execution of the workflow proceeds as specified in the rule.



Changes to "event" are not visible to other rules

Note that any changes made to score results, target variable or outcome are not visible to other rules in the same rule service element. This enforces that rules evaluate independently of their order.

Execution order

There are several key aspects of how a rule is executed. For a rule to be executed or have any effect at all it needs to be enabled. This, however, doesn't guarantee that the rule is executed. You must consider both how the control flow works and the order of the rules, as defined in your project.

Each rule service has its own set of rules defined in a rules project. These rules are unique to the service instance and have a specific order by which they are executed. This is important because triggering one rule can cause others to be skipped.

Another key aspect of the rules is that they are executed independently from each other and in an ordered fashion.

If you need to reuse variables in several rules, check [this section about environments](#).

Triggering actionsâ

Rules can have a great impact on the whole workflow. One of the things they can do when fired is to trigger [actions](#), which will be caught and processed by action handlers.

Actions can have parameters whose value is defined in the rules. For example, a "sendEmail" action can have the parameters "to" and "subject" with values "" and "Rule Alert".

For details on how to define which parameters should be available in each action check the [Actions and Action Listeners](#) page.

Controlling the workflow executionâ

Each rule can define what should happen to the workflow when the rule is fired.

This is especially useful when you want to ignore the evaluation of further rules, or even abort the rest of the workflow, as soon as one rule is triggered. Available options are as follows:

- **Continue**

The execution continues to the next rule or, if there is none, to the next workflow element.

- **Stop Workflow**

The workflow execution stops. Actions resulting from the current rule will be triggered for the current event.

- **Next Workflow Element**

No more rules are executed in this rule service instance and the execution continues to the next workflow element.

-



- [Core Concepts](#)
- [Rules](#)
- Rule Components

On this page

Rule Components

Was this page helpful? 

The following sections give you details about how a rule is evaluated when processing an event, and about the components that you can use when building a rule.

Variables and conditions

Rules contain the following parts that will be evaluated to decide if the current event matches it or not:

- **Variables:** the section that contains the preparation code.
- **Condition:** a boolean expression that determines if a rule matches the event.

Pulse uses rules, variables and conditions in the following way:

1. Pulse calculates the value of the variables for the event that is currently processed.
2. Pulse evaluates the condition for the current event.
3. If the condition is true:
 - The actions defined by the rule are triggered.
 - An entry is added to score tracking.
 - The control flow defined in the rule dictates what to do next (check other rules, or stop processing, etc.).

If the condition is false, nothing happens and no entry is added to score tracking.

When you [create rules](#), you are building complex expressions in Rule Definition Language (RDL). **RDL** is a Pulse domain-specific language that you can use to describe complex rules. The following sections give you more details about how variables and conditions are created in RDL.

Variables🔗📄

You can add variables to rules to avoid having large condition statements, which can be error prone and difficult to maintain.

The following example creates a variable holding the last 24 hours of transactions grouped by card number:

```
events24Hours = event[24 hours by cardnumber];
```

The following variable holds the sum of the amounts for the last 24 hours of transactions:

```
sumAmount24Hours = events24Hours.sum(amount);
```

[Learn more about which RDL operators you can use in rules.](#)

Note that the scope of variables is local to the rule that they are defined in, so a variable of a rule cannot be reused when designing other rules.

To declare global variables that are accessible to a set of rules, you can use [environments](#) when configuring a rules project.

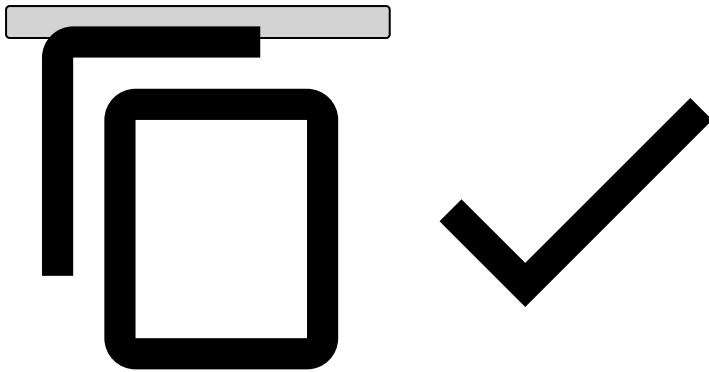
Condition🔗📄

A **condition** is an expression that must return a boolean value, and that dictates whether the event triggers a rule. The condition expression must evaluate to *true* in order to match the event. The following examples show condition expressions:

```
//Match transactions where the sum of the amount for the current card in
the last 24 hours exceeds $20000**
//Assumes the existence of a sumAmount24Hours variable
sumAmount24Hours > 20000

//Match transactions with processing code 10 and amount less than $2**
event.processing_code == 10 and event.amount < 2

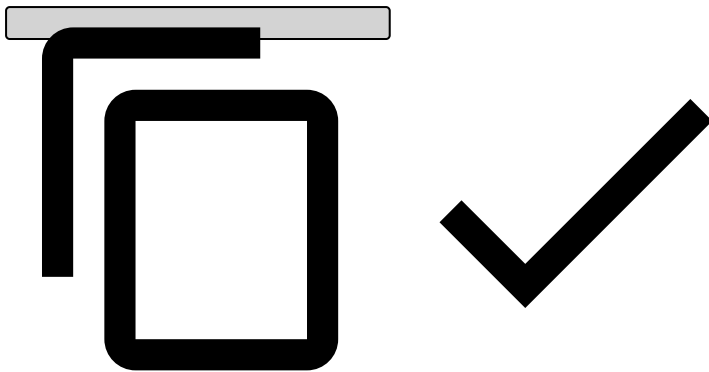
//Match transactions with processing code 10 and amount less than 2, or processing code 20 and
amount below $20
(event.processing_code == 10 and event.amount < 2) or (event.processing_code == 20 and event.amo
unt < 20)
```



The event variable

You can use the built-in *event* variable to access attributes of the current event. The following examples show how to do this:

```
//Accessing fields of the current event
cardnum = event.cardnum;
amount = event.amount;
scoreIsAbove400 = event.score > 400;
```



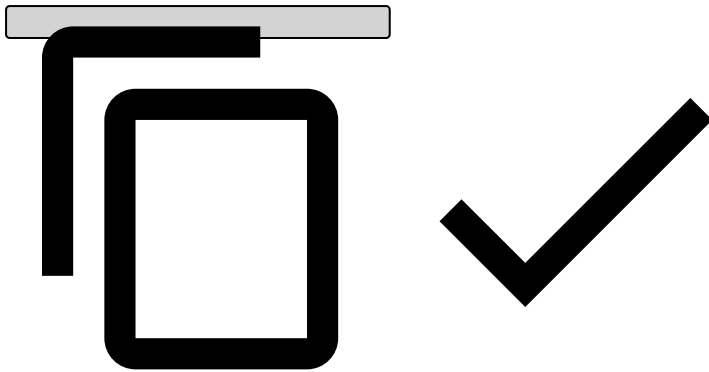
By using the *event* variable, you can also access earlier events with a technique called **window**:

```
//Getting the last five events by card number for the card number in the current event
lastEvents = event[5 tuples by cardnumber];

//by cardnumber creates a window with only the events that have card number
//equal to "event.cardnumber". Thus the last statement can be re-written as:
lastEvents2 = event[5 tuples by event.cardnumber];

//Last five events with terminal type X and Z
lastEventsWithTerminalTypeXZ = event[5 tuples where terminaltype == "X" or terminaltype ==
"Z" by cardnumber];

//Filter events by processing code
eventsWithProcessingCode5 = event.filter(e => e.processingcode == 5);
```

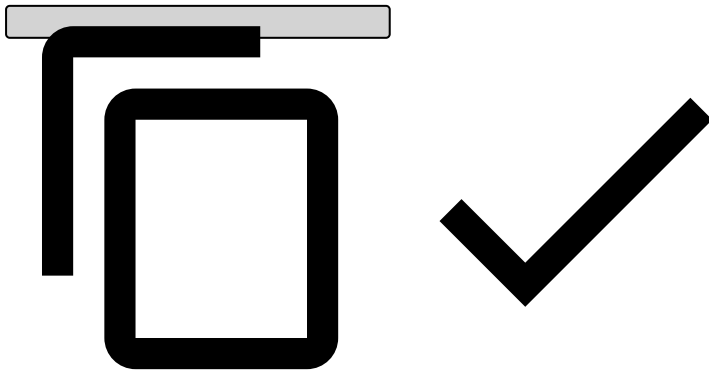


Learn more about the *event* variable and how you can use it to [access historical data](#).

Integration with lists

You can use [lists](#) in rules. The following RDL code checks if an event attribute exists in a list:

```
//blocklist is a previously defined list  
isInBlockList = blocklist.contains(event.cardnumber);
```



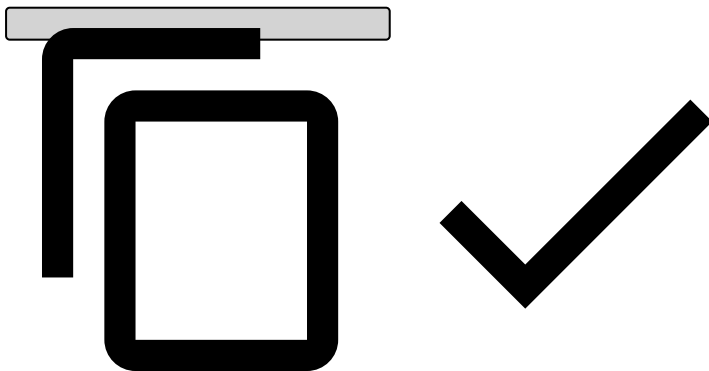
Learn more about [how lists connect to rules](#).

Integration with enrichment functions

Rules provide direct integration with data enrichment functions. This means that rule conditions can use the data enrichment service through [user-defined functions or UDFs](#). Since the data enrichment service provides a large set of valuable data, that's often useful when deciding which path a transaction should follow in a live workflow.

The following is an example on how to evaluate if a given IP address belongs to the TOR network or not.

```
Enrichment.isTorNode(event.ipaddress)
```



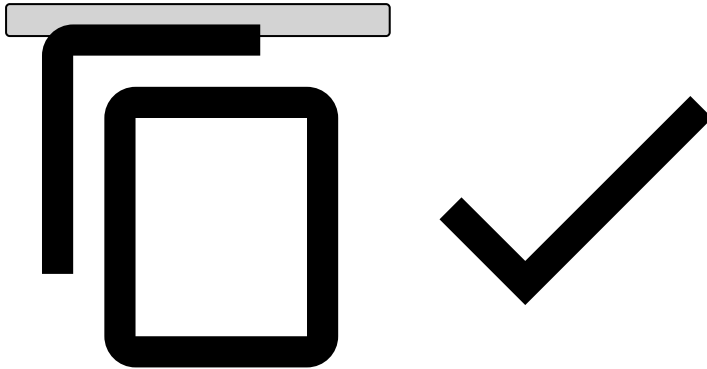
Integration with Feedzai TrustScore[®]

If Feedzai [TrustScore](#) is available in your environment, it can be used in rules to provide a risk score for the event being processed.

For example, the following RDL can be used to approve a transaction if the TrustScore-based fraud score for the current event is below the threshold:

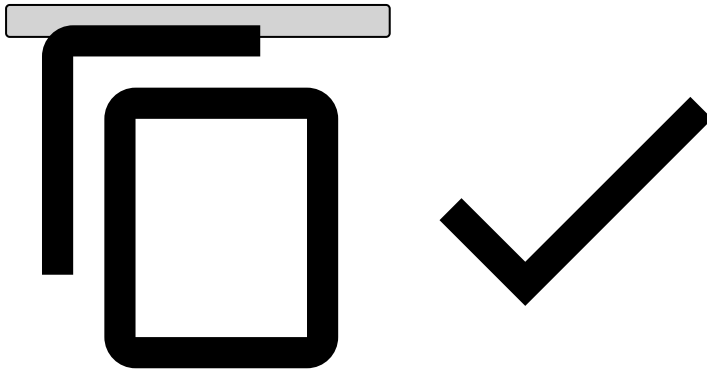
Variables:

```
LOW_SCORE_THRESHOLD = Rules.Thresholds["model_low_score"].threshold;  
score = fraud_score * 1000;
```



Condition:

```
(score ?? 0) < LOW_SCORE_THRESHOLD
```



Integration with environment metrics and variables[®]

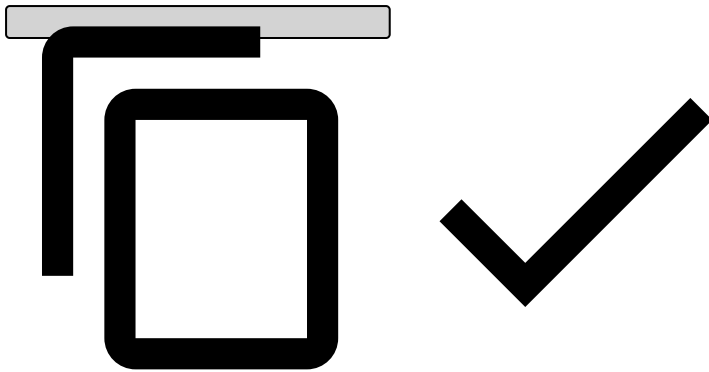
By default, a variable that you create in a rule can be used only in that specific rule. If you want to use the same variable in other rules as well, you must configure a so-called environment for that. An **environment** extends the scope of the variables present in it, it makes the variables available in every rule that belongs to that environment.

When creating a workflow, rules projects are represented by rule services. Each rule service has an isolated environment that you configure when adding the rule service to the workflow. The RDL variables (and also the PQL metrics) defined in that environment will be available in every rule of the environment.

Environment metrics[®]

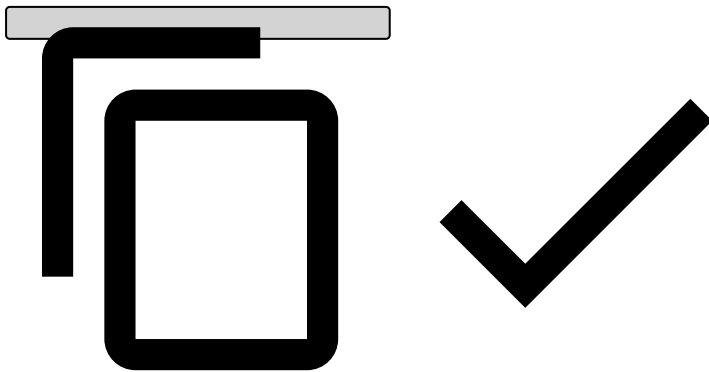
In the metrics code, an *input* stream is available. This stream represents the events of the respective workflow. The following example creates a metrics dictionary using PQL:

```
// Sums the amount for each card number in the last 24 hours  
sumAmount24Hours = from input[24 hours] group by cardnumber select amount:sum(amount);
```



The following example shows how to use this dictionary in RDL:

```
sumAmount24Hours[cardnumber].amount > 30000
```



RDL variables [↗](#)

Creating RDL variables to be accessible in environments is the same as for rules themselves. The same variable will be available in every rule of the same rules project.

Templates [↗](#)

Creating templates [↗](#)

Sometimes it is necessary to define the same rule with different parameters several times. For example, you might want to generate alerts if the total transaction amount for a card number in the last 4 hours is above a certain threshold.

Pulse provides **template** mechanism that enables code reuse and helps you avoid writing the same logic several times.

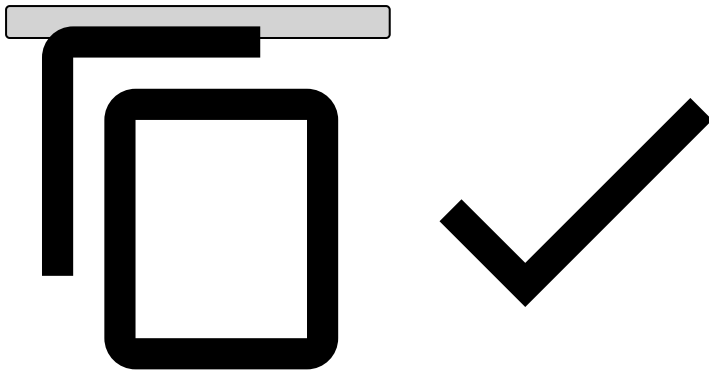
Templates allow a rule to be written without specifying all of its concrete parameters. Template parameters are used instead.

Each template parameter is characterized by:

- **Name** used to identify the parameters when applying the template to a rule
- **Type**
- **Default value**

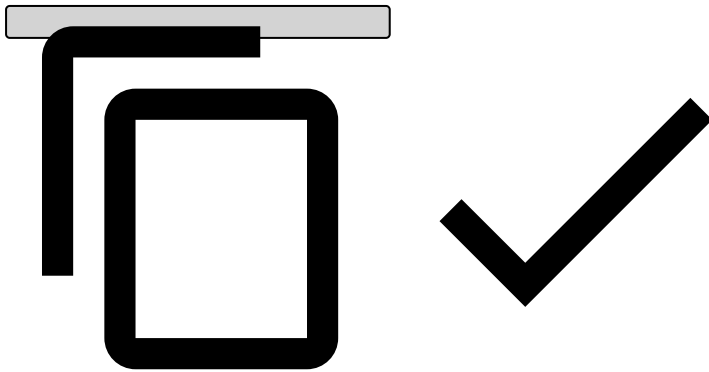
In terms of RDL code, templates work the same way as rules do, except that they allow defining template parameters. This is done using the `"${<TEMPLATE_NAME>}"` notation:

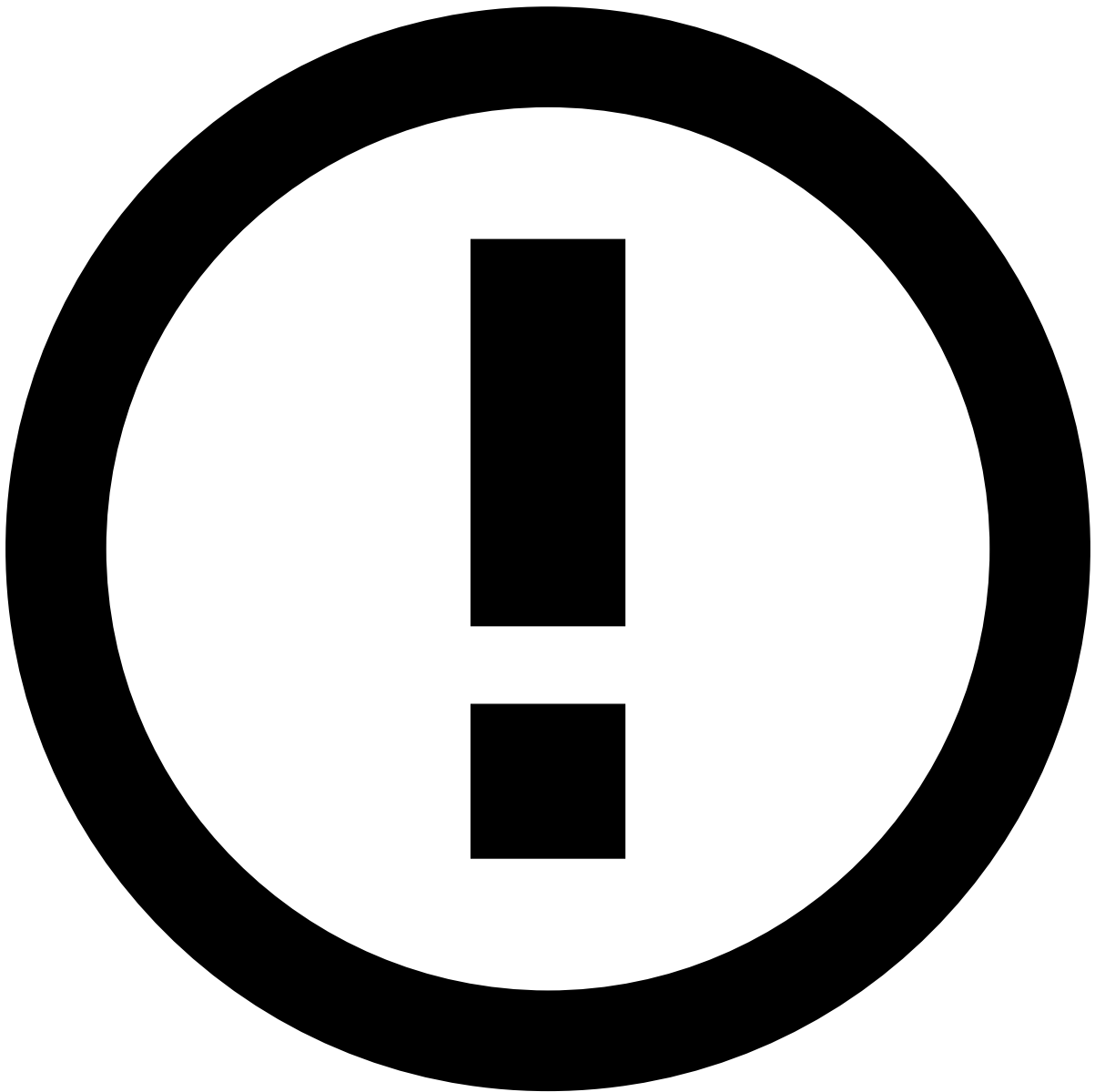
```
//Parameterize the size of the window using template parameter nMin  
lastEvents = event[${nMin minutes}];
```



You can define template parameters both in the **Variables** and **Condition** fields. You can also define several template parameters as follows:

```
//Parameterize window size with parameter nMin and condition with parameter cardnumber and terminaltype  
lastEvents = event[$nMin minutes where cardnumber == $cardnumber and terminaltype == $terminaltype];
```





info

Template parameters can only be used to replace literals.

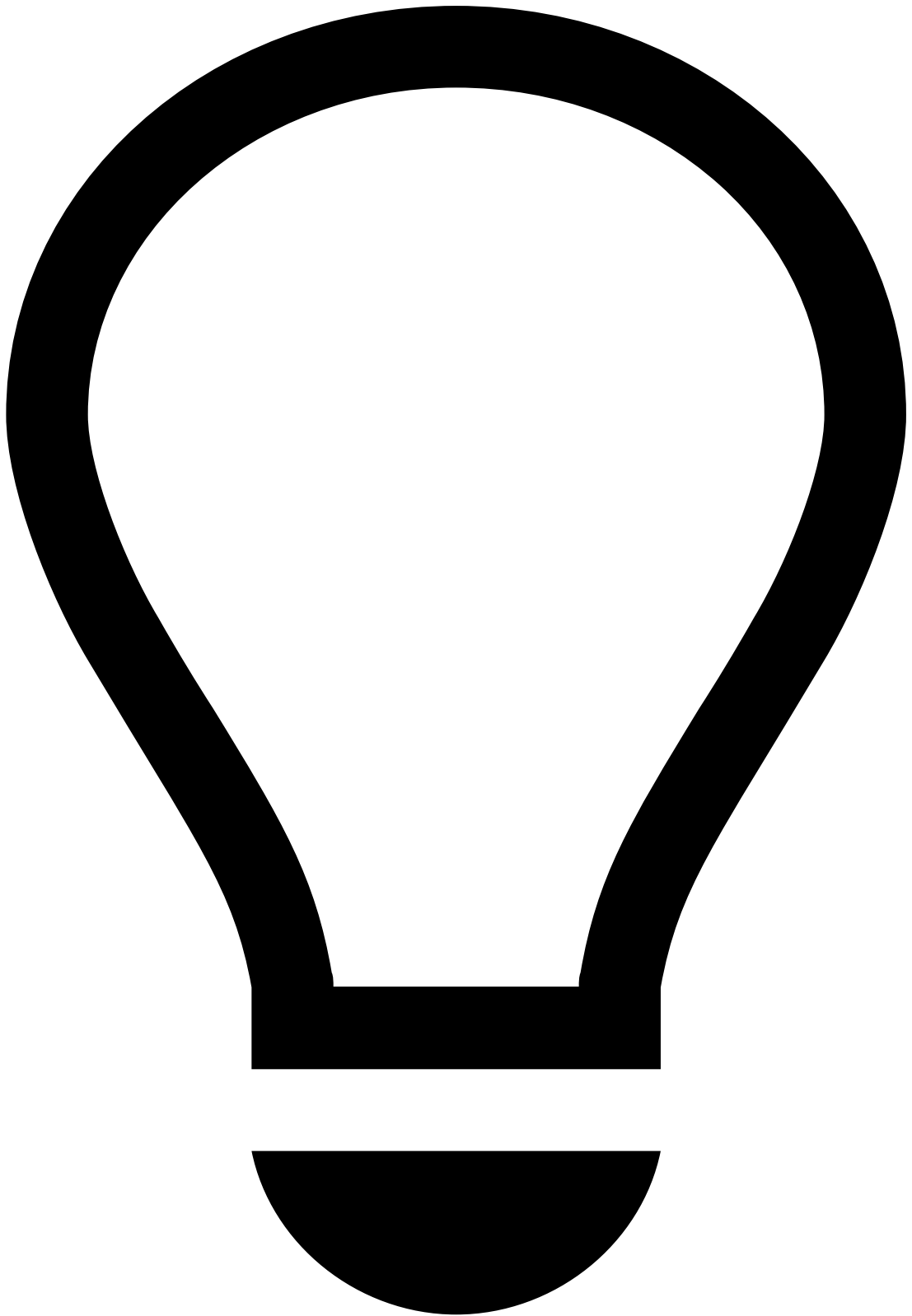
Rule explanation templates

Rules explanations allow you to learn more about why a rule triggered as an event goes through a workflow, in a human-readable way.

When you create a rule inside a rules project you can define a template for the explanation as to why a rule triggered. Templates can be parameterized with available fields from the input data, lists and variables, similarly to when you are creating rules.

If Pulse renders the explanation when a rule is triggered, it will replace the added fields, lists and/or variables with their actual values.

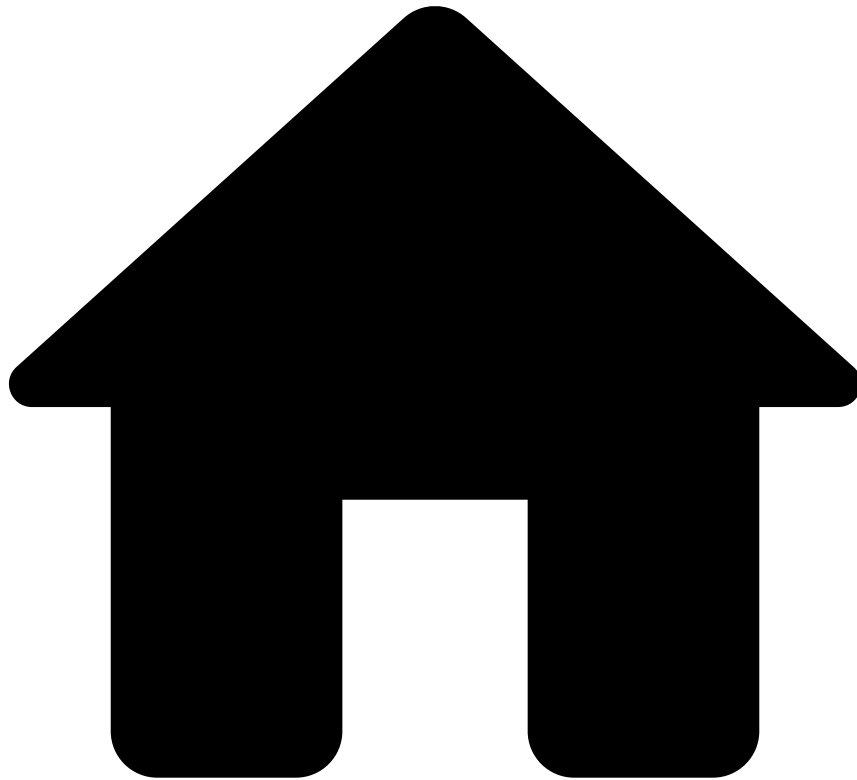
The following image illustrates an example of a rule explanation template for an account opening use case.



tip

To generate explanations in a runtime workflow, activate explanations for each specific rule service in the workflow.

-



- [Core Concepts](#)
- [Rules](#)
- Rule Snapshots

On this page

Rule Snapshots

Was this page helpful? 

A **rule snapshot** captures the configuration of a rules project at a given time, including which rules compose the block, and their execution order.

Snapshots allow the following:

- Use a specific version of a block of rules in a runtime workflow seamlessly.
- Evaluate the performance of a specific version of your rules before marking them production ready.

Snapshots and rule evaluations📄

A **rule evaluation** consists in the execution of a block of rules to classify the records of a data source, in a manner that lets you extract relevant performance metrics. Rule evaluations are useful in your data science project to perform easy comparisons over earlier versions of the rules, and assess whether they help you catching fraud (or predicting other target variables).



info

Sometimes you may want to just define and deploy a rule immediately, for example, when implementing a rule of business constraints. Therefore, **it is possible to generate a rule snapshot without generating an evaluation.**

Pulse runs the evaluation of rules in a distributed way, on a set of machines in a cluster. This means that even for very large test datasets with several TB in size, Pulse is very fast to compute metrics and generate statistics, thus contributing to better performance assessments and faster iterations on your rules.

The following image shows a rule evaluation taking into account the final result of running an instance through a block of sequential rules.

Evaluation methods

Pulse lets you analyze how rules perform over your data under different perspectives. Rule evaluation is available under the following points of view, known as **evaluation methods**:

- Block evaluation
- Block rule evaluation
- Individual rule evaluation

These evaluation methods are explained in the following sections.

Block evaluation

When you have a large set of rules, it may be very time consuming to analyze the performance of each rule individually.

The **block evaluation** shows the performance for all rules you selected to be part of the evaluation, as a single block. This means that Pulse uses a black-box approach. You obtain evaluation results regarding the final value of the target field after Pulse runs all rules.

Analyzing performance after Pulse runs an instance through them all as a single block, may immediately uncover issues or let you know that your set of rules, as a whole, performs at a high level.

Block rule evaluation

Most likely you will need to perform a detailed examination of how each rule performs within the block configured for the evaluation.

With the **block rule evaluation**, Pulse lets you drill down into each rule inside the block. You obtain the block performance broken down by each rule.

Analyzing each rule in isolation allows you to detect rules with very bad performance, for example, a low recall and a high FPR. In addition, if the block evaluation returns suspicious results, drilling down into each rule may let you understand which one is causing the bad performance.

Individual rule evaluation

The **individual rule evaluation** presents evaluation results as if each rule would run individually, without stopping the workflow. Pulse assumes that the control flow is always *Continue* for all rules.

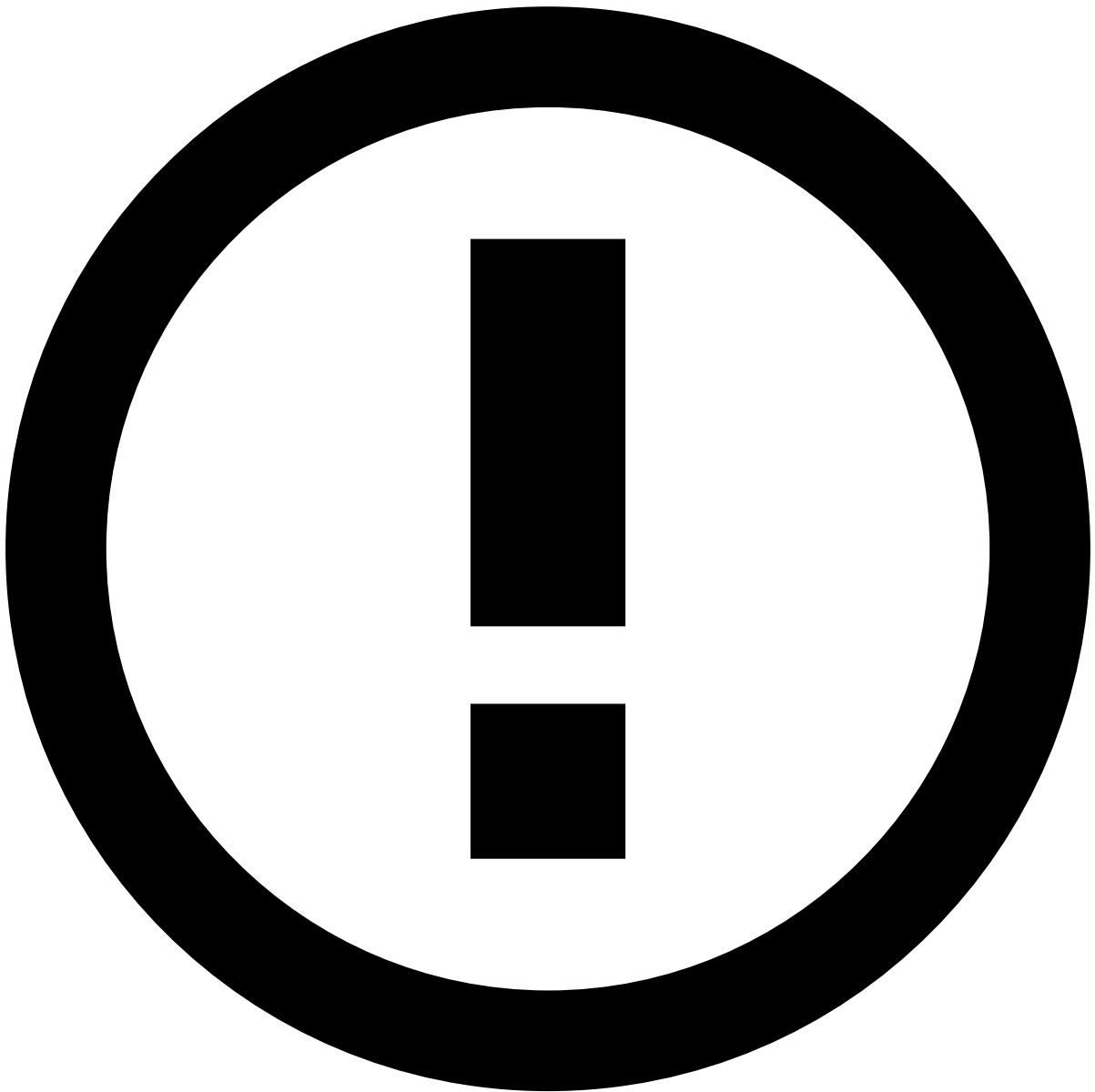
This method is useful in situations where there are rules that stop other rules from being evaluated. For example, if you define a blacklist rule to run at the beginning of the rule block and stop the workflow, it may prevent subsequent rules from being evaluated.

You may have a rule that catches 100% of fraud but since the block never reaches it, you would not have access to its performance and of the remaining rules.

If the control flow has been configured as *Continue* for all rules, then the block rule evaluation and the individual rule evaluation return identical results.

Breakdowns

When you generate a rule evaluation, in addition to generating overall performance metrics, Pulse also calculates them over field combinations. This allows you to break them down by another categorical field and its values, and evaluate your rule block performance for each of those possible field and value combinations, for each rule (depending on the evaluation method). For example, checking the performance of your rule block segmented by each suspicious country, as you identified when designing features, where fraud is more prevalent.



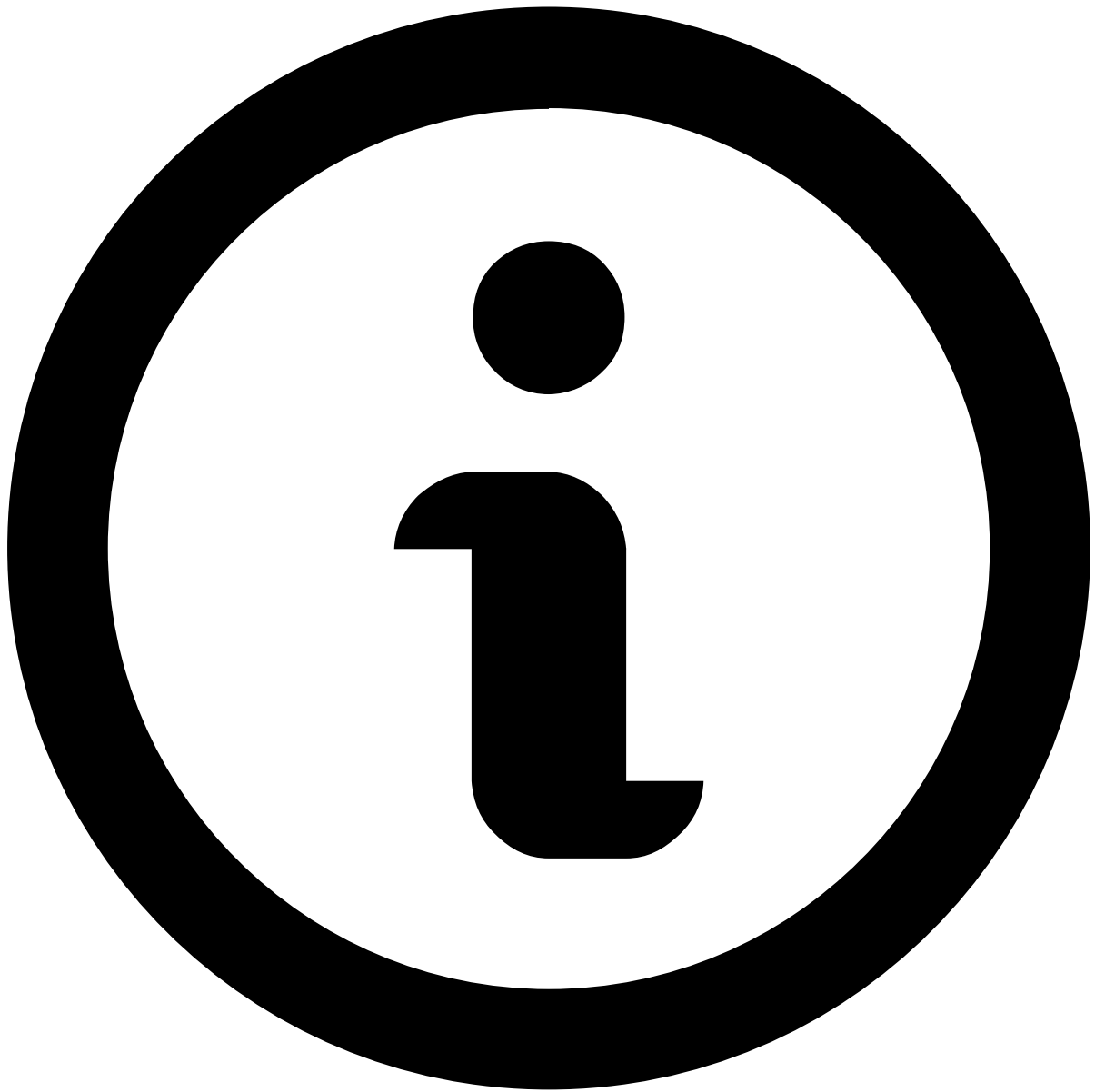
info

When you run a rule evaluation, you must always select for which breakdown fields and values you want Pulse to calculate statistics.

If Pulse allowed you to skip this action, and simply calculate broken-down statistics for all possible combination of fields and values, it would rapidly cause performance degradation due to lack of computational resources.

Snapshots and evaluations in a multi-tenant environment

Snapshots inherit the characteristics of the originating rules project, and thus, in the case of rules projects of the type *Global* and *Duplicated for Tenant*, the same rule applies for all the tenants. In the case of rules projects of the type *By Tenant*, the rules can be distinct by tenant. However, still in the latter case, only one snapshot is needed to capture the rules for each tenant (i.e. is not a snapshot for per tenant). A snapshot associated to a rules project of the type *By Tenant* is referred to as **overall snapshot**.



Evaluations are not available in multi-tenant environments

The evaluation feature is not available in rules projects that are set up for multi-tenancy (i.e. *Duplicated for Tenant* and *By Tenant*).

•



- [Core Concepts](#)
- [Workflows](#)
- Actions and Action Listeners

On this page

Actions and Action Listeners

Was this page helpful? 

Actions are generated at the **Workflow Element** level and are used to tag/mark the events flowing in the workflow.

Pulse offers two default actions – **Accept** and **Block** – but you can [define custom actions](#) according to your workflow requirements. In addition, it is possible to define the parameters for each action for the cases where it is necessary to pass extra information when an action is triggered.

When connecting Real-Time Analytics to Fraud Prevention Applications, these actions can be used to filter what kind of events are to be listened by the connector.

Moreover, when developing Custom Services,

special actions can be performed taking into account the type of action in place.

Custom services can be used to access or modify the event's state as events reach them, but they also can listen to specific actions triggered by [Rules](#).

Consider a service that is intended to modify the events (or add some important metadata to them), but only those that match a specific **Rule**. You can declare this service as an **Action Listener** choosing the **Action** to be listened and tell the specific **Rule** to trigger this action. When the **Action** is triggered, the service will receive the corresponding event and can process it as intended.

Trigger Actions🔗

Trigger Actions are used to notify action subscribers (**Action Listeners**) of some occurrence during the processing of a workflow element. The subscribers can be either another workflow element or any other module as long as it has access to the workflow.

Two mechanisms can be used to trigger actions:

1. By [creating new Rules](#) and [configuring them in the Rules Service](#).
2. By writing Custom Services that extend the `WorkflowService` class and calling `triggerAction(TriggeredAction, Message)` .

Action Parameters🔗

Each **Action** can have its own parameters. Parameters allow passing event-specific information when an action is triggered.

One example would be for a *sendEmail* action to have the parameters *to* and *title* with values [\[email protected\]](#) and *Rule Alert*.

Action Listeners🔗

Use **Action Listeners** in order to subscribe to triggers being sent from workflow elements. Custom Workflow Services can be notified of actions triggered by any rules configured in the workflow.

By default there are no **Action Listeners** configured in a workflow, so even if an element is configured to trigger an action, if there's no subscriber it will take no effect.



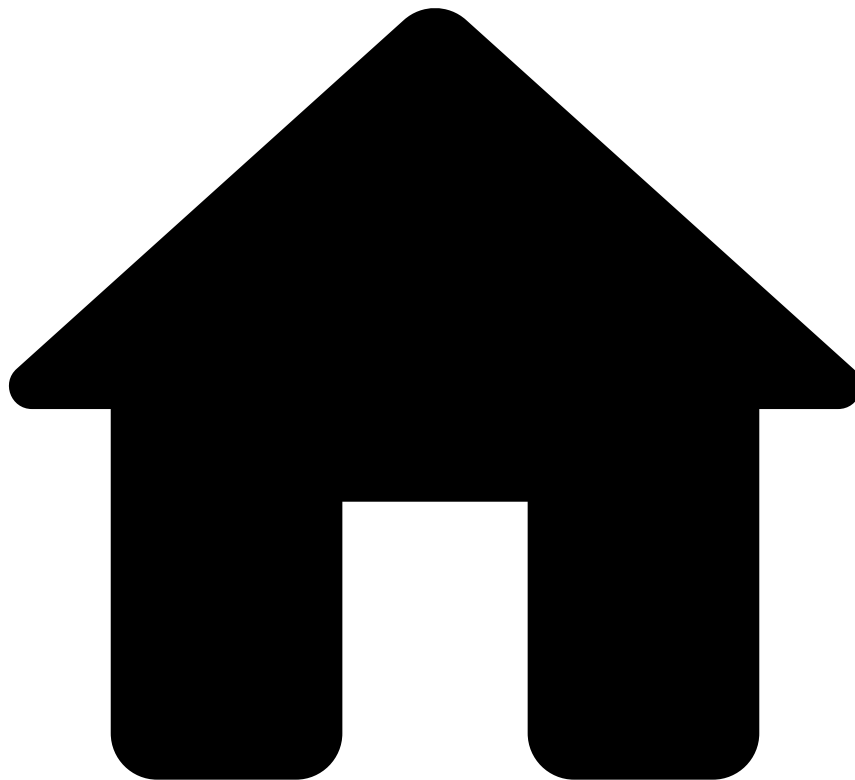
Actions handling

The triggering of **Actions** works synchronously, so be careful with the way you implement your listeners. It might be desirable to implement an asynchronous handling on the listener's side.

When configuring **Action Listeners**, the list of available listeners is populated from the existing services. For a service to be listed either:

- The service is a part of a workflow
- The service is part of the overall application environment and implements a `WorkflowActionHandler` interface.

•



- [Core Concepts](#)
- [Workflows](#)
- Automated Model Monitoring

On this page

Automated Model Monitoring

Was this page helpful? 

Automated Model Monitoring (AMM) is a feature that automatically generates alarms when there are significant changes in the model scores distribution after a model is deployed. Such anomalies can constitute threats, such as bot attacks, data corruption issues or sudden increase of a certain type of transactions.

A schematic overview of the algorithm is presented in the figure below:

In the **Data stream** graph (top), the normal events are represented with blue circles and the anomalous events with red circles. In the **Machine Learning Model** graph (middle), the algorithm computes a signal by using the output scores of a ML model. The anomalous periods are colored in red again. In the **Automated Model Monitoring** graph (bottom), the

signal is represented in a black solid line. If the signal, as to be further explained, is larger than a given automatically computed threshold (i.e. blue dashed line), an [alarm](#) is triggered. This is represented with red flags.

This system works without any input on the true label of the transactions, hence it is unsupervised.

Signal

The signal is a number - between 0 and 1 - that measures the dissimilarity between the model scores histogram in a **reference window** (various options described below) and the model scores histogram in a fixed-size **target window**. If the signal is small, it means that the reference and target windows are very similar. If it is close to 1, the reference and target windows are maximally different. The target window contains the last collected events (e.g., in the last 6 hours). The reference window contains events in a reference period, which is prior to the target period (e.g., in the 3 days before the target window).

This signal is designed to detect sudden changes in behavior, i.e. when unusual events suddenly enter the target window. The reference window defines the recent normal behavior for the signal's calculation.

Having this into consideration, there are two types of windows that are configurable and suitable for different use cases, as follows:

Contiguous windows

In this configuration, the reference window has a fixed-size (i.e. a fixed number of events) similarly to the target window. The size of the reference window is larger than the size of the target window. The diagram below represents this configuration, where the reference window is immediately before the target window. The configuration slides to the right whenever a new event in the data stream is processed.

Homologous windows

This configuration is suitable for use cases suffering from a strong seasonal behavior (e.g. banking datasets). The reference window is defined as a set of replica windows occurring in the same period of the day as the target window but on previous days (i.e. homologous periods). For example, if the target window occurs between 12:30 and 13:45 on a Monday, a homologous window configuration with three replicas (e.g. see figure below) would contain events from the three previous days (i.e. Friday, Saturday and Sunday, from 12:30 to 13:45).

Threshold

An adaptive threshold is used to trigger alarms whenever the signal becomes too large. The threshold is computed, whenever a new event arrives, together with the signal. The alarm is triggered when the signal becomes larger than the threshold. The AMM threshold uses a definition of outlier based on Tukey Fences, a method that focuses on the width of the central part of the distribution of past signal values. To estimate the percentiles of the distribution we employ an algorithm that estimates the percentiles giving higher weight to more recent signal values. The bare Tukey threshold is defined by calculating $\delta = \delta_{25} + \frac{1}{4}(\delta_{75} - \delta_{25})$, where δ_{25} and δ_{75} correspond, respectively, to the 25th and 75th percentile of the signal values distribution. $\frac{1}{4}$ is a free parameter to control how conservative the threshold is (larger K values correspond to a more conservative threshold). An additional delay is used to suppress the effect of the target window events. This delay and a smoothing are applied to obtain the final threshold used in the system.

Alarms

Alarms consist in automatic notifications that are triggered every time the signal rises above the threshold.

The various types of alarms (depending on the Scoring Service configuration) are described in the following sub-sections:

Email alarms

The email alarm is an optional feature. Pulse uses a Simple Mail Transport Protocol (SMTP) to automatically send emails. Configuring the SMTP is optional, which means that users can enable or disable the SMTP by setting up the properties of the host and recipients in the Scoring Service configuration.

If the Scoring Service's SMTP properties are not defined, the system will fallback to the global Pulse SMTP properties.

Typically, the email shows the following information:

- Model: <model id>
- Timestamp where the alarm was raised: <timestamp of the alarm>
- Signal value: <signal value>
- Threshold: <threshold value>
- Application Id: <application id>
- Hostname: <machine that sent the alert>

Logging alarms🔗📄

Logging alarms are also optional notifications. They are written to the logging file of the Pulse application. This type of notification displays only a subset of the information that is visible in the email. To receive logging alarms, users must enable the logging property by setting `com.feedzai.modelmonitoring.alert.LoggingModelAlertSink` to WARN level.

Alarm cooldown🔗📄

The alarm cooldown prevents alarm spamming. This means that users can configure the time interval in which all alarms are suppressed after an alarm is triggered.

-



- [Core Concepts](#)
- [Workflows](#)
- Event Storage

On this page

Event Storage

Was this page helpful? 

You can configure [workflows](#) to save events that they processed to the **Event Storage**. That way, when an event reaches the end of a workflow, it is saved asynchronously to the Event Storage. Pulse saves the event's original values, as they arrived to the workflow.

The events are stored in an event storage table dedicated for the workflow schema. This table consists of columns matching the [schema's fields](#). If the workflow contains plans with scheduled metrics, then the fields that were changed by the scheduled metrics are saved in the "event diffs" column. This way Pulse can rebuild the final state of the event.

The Event Storage saves any event that reaches the end of a workflow under one of the following conditions:

- Pulse generated an exception while processing the event, causing the workflow to end.

- The event was dropped from the workflow after being processed in a service. For example, the event triggered a [rule](#) configured to stop the workflow.
- The event went through all processing elements in the workflow and correctly reached the end.

The goals of the Event Storage are the following:

- **Recovery:** If the underlying Pulse application fails or crashes unexpectedly, the Event Storage enables replaying events that have already been processed and stored, to recover the previous stable state.
- **Auditing:** Keeps track of all context regarding events by storing also:
 - All the Actions the event triggered throughout the different services in the Workflow.
 - The results of processing by Scoring Services and Rules Services, including the score and weight for each service.
 - Human-readable [explanations](#) as to why a Scoring Service or a Rules Service classified the event in a certain way.
 - Auxiliary data, such as the metrics calculated for the event.
- **Reporting:** Extract knowledge from past events and generate reports on model accuracy and outcome distribution for a consistent evaluation of runtime performance.
- **Feedback Loop:** Retrain models using the Event Storage as a source of data after receiving human input. This is especially important to close the loop on human feedback, for example, when Pulse classifies a transaction as fraud but a human analyst overrides the decision based on real-world evidence.



Disabling Event Storage

If you don't enable the event storage, Pulse will never store events in this table. This means that services that need to retrieve information from the event storage in their processing phase will not work properly.

Back End implementation📄

The Event Storage can be implemented in different back ends, such as Cassandra or a relational database.

Additionally, the back-end can process data in different ways such as binary serialization and JSON representation. The following table summarizes the advantages and drawbacks of each alternative:

	Speed	Space efficiency	Human-readable	Data migration	Query complex fields
Binary Serialization	Faster	Higher	No	Harder	No
JSON	Slower	Lower	Yes	Easier	Yes ⁽¹⁾

⁽¹⁾ In databases that support JSON natively, e.g. PostgreSQL.

Event Storage API

Configuring and accessing the contents of the Event Storage can be performed programmatically via the API.

•



- [Core Concepts](#)
- [Workflows](#)
- Explanations

On this page

Explanations

Was this page helpful? 

Explanations simplify the end-user interpretation of the result of processed events. These explanations describe in a human-friendly way why events were classified in a certain way by [models](#); for example, why they represent fraudulent transactions, or why they activated [rules](#).

So, for each [workflow](#) element that can produce a score and contribute to an outcome, you can define locally how Pulse generates an explanation.

Each workflow stage will then be enabled to produce different explanations for a given event.

Explanations and Rules

When a transaction goes through the [workflow](#) and reaches a **Rules Service** workflow element, all the defined [rules](#) are tested. Whenever a rule is triggered, it can, if configured, generate an explanation. Explanations are generated from an [explanation template](#) defined locally at each rule and only contain one reason to explain why the rule was triggered.

The explanation template field is designed to contain a human-friendly description of why the transaction failed to satisfy the triggered rule. To improve these descriptions, the template field allows to perform parameter substitution and include, for example, transaction values or environment variable values.

Explanations and Models

When a transaction is scored in a **Scoring Service**, Pulse can, if configured, also generate an explanation. Explanations are generated to show how each model feature contributed to an [outcome](#) and outcome score.

Contrary to **Rules Services**, Pulse automatically builds explanations containing feature contributions to the outcome, so it is not necessary to configure any explanation template.

The result of explanations in a **Scoring Service** is a list of all data fields used as features for the model and the weight from each one in the decision.

Computing Explanations

Depending on how you configure each of your **Scoring Services** and **Rules Services**, Pulse generates explanations as follows:

- **Computed on critical path:** Pulse generates explanations immediately after the event is processed at a **Scoring** or **Rules Service**. This may be useful when you need to act upon an explanation as it occurs and triggers an external action in another system. For example, you can use a custom service to handle explanations and dispatch actions to external systems.



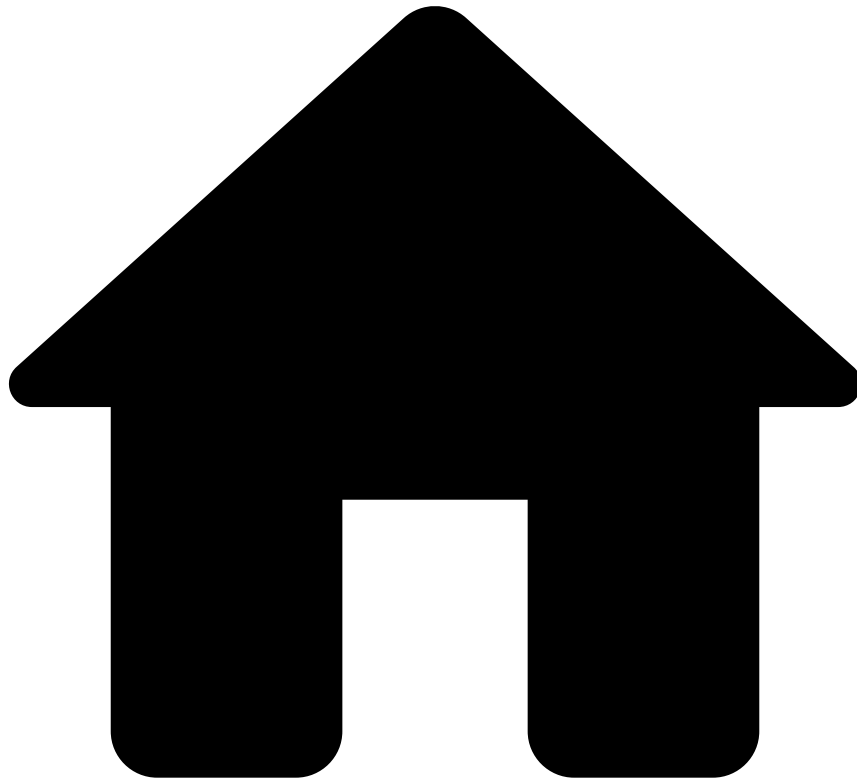
Explanations on critical path

Consider that computing explanations in the middle of a workflow will cause a performance degradation.

It is recommended that you use this option only when it is crucial that you analyze explanations right away.

- **Computed after workflow completion:** Pulse only generates explanations after the workflow has finished processing an event.

•



- [Core Concepts](#)
- [Workflows](#)
- Plans in workflows

On this page

Plans in Workflows

Was this page helpful? 

At the [runtime stage of your project](#), the main purpose of your [workflow](#) is to analyze each event in order to produce decisions on the event, for example detecting whether it represents a fraudulent transaction or not. Although the raw event does not contain enough information, you can apply [deployable plans](#) to derive features and transform the input into a very rich schema containing all information about a card, merchant, location, etc.

Still, it is not possible for a machine learning [model](#) to depict fraudulent behavior based only on transformations derived from the raw information contained in the transaction. Hence the need for more than single-moment information about transactions. You must build a card profile, based on daily, weekly, monthly transaction behaviors and collect all the information about location, merchants, and acquirers. Thus, understanding how to calculate average amounts, a sum of amounts, and maybe more complex patterns - aggregated for different periods of time - is also essential.

So, such a [plan](#) specifies how live data must be transformed, handled and prepared to be consumed by the machine learning model, for runtime scoring. When you execute a plan, Pulse translates it into [Pulse Query Language](#) that generates new augmented events ([instances](#)).

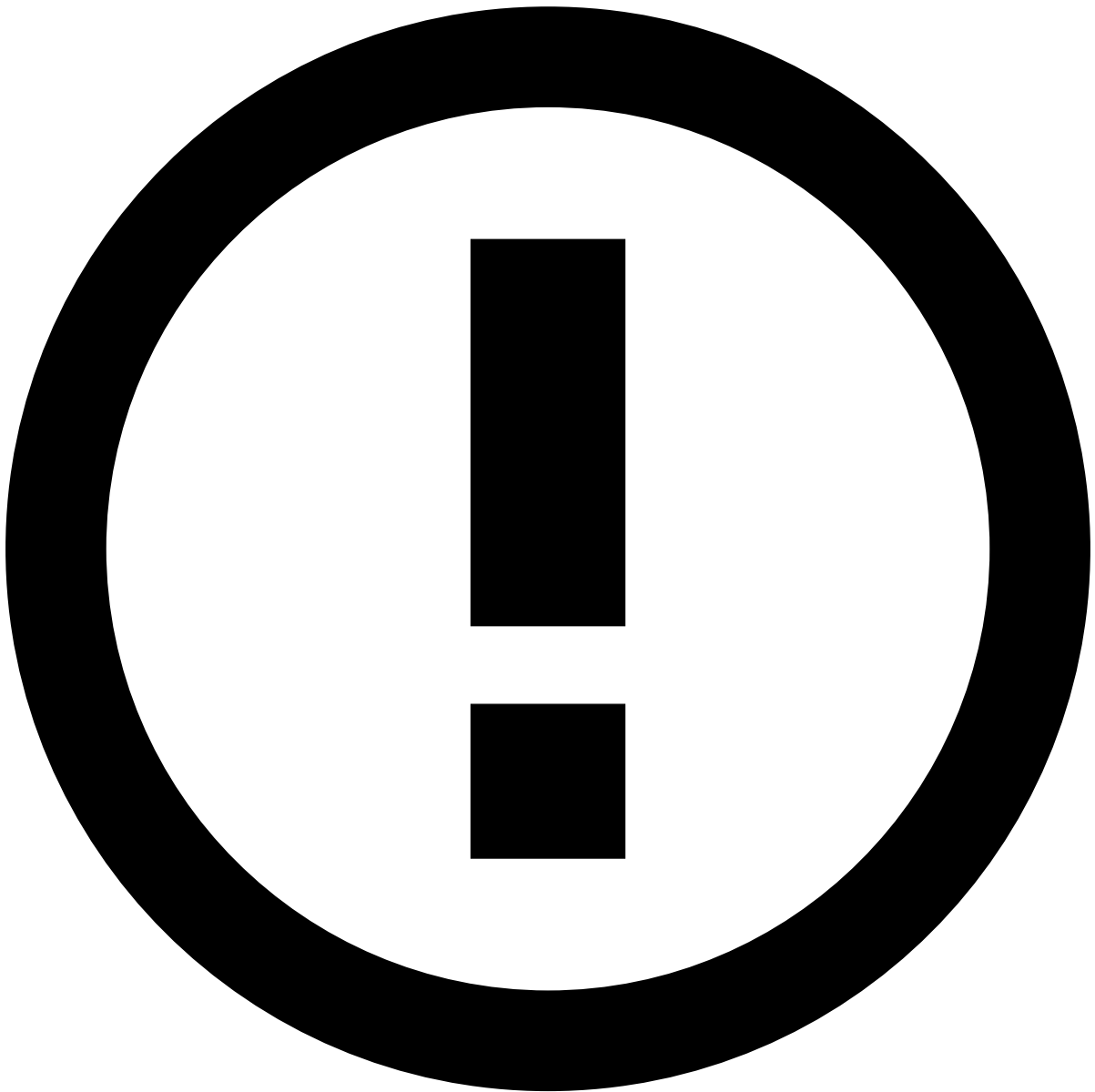
With such a Plan you can include the following types of information into runtime events and turn them into instances:

- **Metrics:** Time-based aggregations over data.
- **Data Enrichment Fields:** Fields consisting of data enrichment functions.
- **Fields:** [Fields](#) from the [schema](#) directly added into the model.
- **Features:** Combinations of metrics and/or schema fields that add value to the model.

Metrics

Metrics allow defining aggregations over multiple data rows, covering events over a given [time window](#) (e.g.: the *sum of the amount of transactions in the last month*).

Consider the scenario where the sum of some card's transacted amount per month is approximately \$10,000. It would be unusual if a single transaction exceeded this value; that could be considered as potential fraud. Other examples of metrics that could be used in this case are *minimum amount in last 24 hours*, or *number of transactions per week*. Metrics like these can help establishing a more complete picture of the card's behavior.



info

For details about metrics in the context of the Railgun engine, refer to the [Metrics in Railgun](#) page.

Components of a metric¹

Defining a metric implies the definition of the following components:

Time window¹

A **Window expression** can be defined to specify the size of the time window that a metric covers. The expression should be in the form `<number> <period>` ; for example, "4 weeks".

The following table lists the supported period units for window expressions, and the corresponding durations:

Term	Duration (ms)
ms / millis / millisecond / milliseconds ⁽¹⁾	1

Term	Duration (ms)
s / sec / secs / second / seconds	1000
min / mins / minute / minutes	60,000 (60 seconds)
hour / hours	3,600,000 (60 minutes)
day / days	84,400,000 (24 hours)
week / weeks	604,800,000 (7 days)

(1) Milliseconds are the base unit of window duration. All other window duration values are relative to one millisecond.

In addition to the duration, windows can have a [Filter](#) expression, which can be used to limit which events within the window are included in the metric. Filters can only use information about the current event.

Time windows can have different behaviors depending on their type. Refer to the [Glossary](#) for an overview of the various types of time windows available.

Aggregation function¹

The core of a metric is the **Aggregation** function, which determines how to aggregate the data within the window, i.e how to combine the multiple data points into a single value.

Supported aggregations are **avg**, **count**, **max**, **min**, **sum**, as well as advanced expressions implemented in [UDFs](#).

Check the [Aggregations](#) page for aggregation best practices.

Grouping fields¹

One or more grouping fields (or grouping entities) can be provided to cluster the data in buckets that share the same values in those fields. This means that the metric will calculate the aggregations for each cluster and return the results for the cluster that the current event belongs to.

Only strings or categorical variables can be used as grouping entities.

Timestamp¹

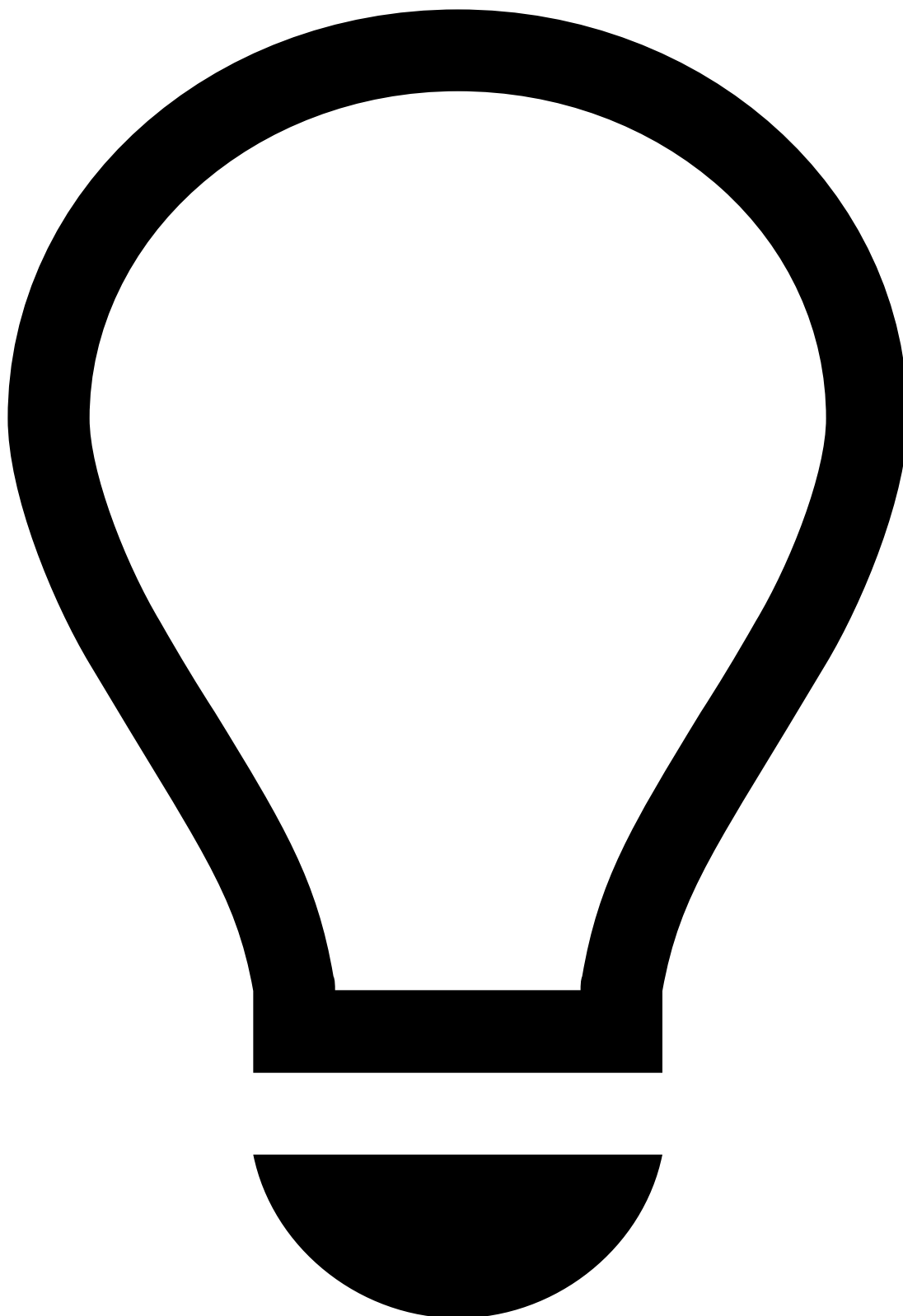
In a plan service of a [workflow](#), Pulse must use the event's timestamp to perform the required time-based aggregations for the calculation of metrics. Pulse ensures that the timestamp is used consistently whether you are performing data science tasks or the plan is deployed into a workflow.

When you design your [plan](#), if you need to calculate metrics, the input [data source](#) must contain a timestamp field. However, to ensure that the runtime behavior does not change after deployment, Pulse uses the [workflow clock](#) (that always keeps the value of the latest timestamp submitted to the workflow) to assign a timestamp value to the event.

This means that, in a workflow, the data schema of your runtime events does not require an explicit timestamp field.

When a plan service needs to calculate metrics, Pulse enforces that events are consistently ordered in time. So, the event's timestamp is internally interpreted as the workflowclock value (the latest timestamp available), in case that event's timestamp is lower. For more information, see [Time, schedules and timezones](#).

If the workflow pushes events with inconsistent timestamps to a plan service, for example, because of parallel processing of incoming events, the generated profiles may be ignoring events that entered the workflow first and will not reflect the reality.



Section not applicable to Pulse with Railgun

When Pulse is configured with the [Railgun engine](#), it is able to calculate all metrics in real time. Therefore, the concept of scheduled metrics is not relevant, and the limitations of real-time metrics below do not apply.

Metrics in Pulse can be of one of two different types: real-time metrics and scheduled metrics. While both allow storing aggregated data based on your transactions, there is a fundamental difference between them.



Restrictions

If you define more than one metric for your model you must choose at least one common *Group By* field between these metrics.

Real-time metrics in memory

Real-time metrics are updated for each transaction that traverses your [workflow](#). Each transaction will enter Pulse's high-performance event processing engine and within less than milliseconds, your metrics are updated and stored in memory.

These metrics are useful to track short-term behaviors such as a running sum of how much was spent in the past 24 hours. These metrics should be used judiciously for cases where immediate metric updates are desirable.



Precaution with Real-Time Metrics

There are performance and sizing issues to consider while creating real-time metrics. These metrics are being aggregated in real time, collecting events to fill up an in-memory window. All the information is being saved in memory.

Knowing your data rate and schema, you can calculate the amount of space needed to hold the necessary events for the defined window. Take a look at the [table of data types and sizes](#) and estimate the data size to make sure your system is capable of performing what you need.

You can also define real-time metrics which instead of being updated on every event that enters the workflow, are updated every X amount of time defined by the user.



Precaution with Real-Time Metrics

The time interval between every output is only started after an event enters the workflow, not when the workflow is published.

Note that this metric type is exclusive, meaning that it will not include an event that occurred at the same time the interval window expires.

For example, suppose you have a 5 day window which is updated every hour that will count the number of transactions done by a credit card number.

Let's assume for that an event occurs on Monday at midnight which marks the start of the time interval before outputting the metric.

The following events then occur:

Timestamp	Value	Metric value
00:05	(cardNumber: 1, transactionValue: Â£10)	Count(cardNumber: 1) = 0
00:27	(cardNumber: 2, transactionValue: Â£36)	Count(cardNumber: 1) = 0
00:42	(cardNumber: 1, transactionValue: Â£42)	Count(cardNumber: 1) = 0
01:00	(cardNumber: 1, transactionValue: Â£140)	Count(cardNumber: 1) = 0
01:01	(cardNumber: 2, transactionValue: Â£5)	Count(cardNumber: 1) = 2
02:00	(cardNumber: 3, transactionValue: Â£23)	Count(cardNumber: 1) = 3

Real-time metrics on disk

In a production environment, Pulse [can be configured](#) to store the state of real-time metrics on disk. This allows Pulse to calculate metrics over larger time windows. However, be aware of the limitations for some aggregations in the [Creating metrics](#) page.



Limitations

Note that this feature is in continuous development, and as of now it does not provide full feature parity to the previous, RAM-based version.

Scheduled metrics

Scheduled metrics are defined in the same way as their real-time counterparts but they are updated as a batch process from time to time in a user-defined schedule, hence the name.

In order to achieve this, each transaction that traverses your [workflow](#) is persisted to a database. Eventually, when the scheduling criteria are met (i.e. at the end of the month or end of the week), a batch process reads transactions from the database, computes the metrics using Pulse's high-performance event processing engine and stores them to a data store. Scheduled metrics eventually expire from the data store after the time window passes.

Each workflow service has its own implementation of scheduled jobs, depending on its type and its configuration, and may or may not have scheduled jobs to run.

In the specific case of [deployable plans](#), you should define scheduled metrics using the [Metrics operator](#).

When such plans are deployed to a [workflow](#) and configured to run in a plan service, Pulse generates one scheduled metrics job for each scheduled metric in the Plan.



Metrics performance

In order to maintain high performance when calculating metrics, it is recommended that each of your deployable plans contains only one Metrics operator.

To ensure the consistency of the output metrics, whenever processing reaches a Metrics operator Pulse must always keep the context of all calculations performed in previous Metrics operators.

To better understand how scheduled metrics relate to Pulse's job management model, check [how scheduled metrics jobs work](#).

Omnichannel metrics⚠️

To compute metrics based on information from multiple [channels](#), you can use [omnichannel plans](#):

- The **main channel** of the plan corresponds to the events that are processed and scored by the workflow.
- **Secondary channels** of the plan correspond to events of other workflows. You will need to specify which workflow to use for each secondary channel when adding an omnichannel plan to the workflow.

For real-time metrics that use information from multiple channels, events from other workflows will be automatically fed to the plan in the current workflow in real time.



caution

Real-time metrics are not yet supported in omnichannel plans.

However, with [Railgun](#) it is possible to use a subset of omnichannel metrics, called [cross-channel metrics](#).

For scheduled metrics, the batch job will process data from multiple workflows automatically â€” check [how scheduled metrics jobs work](#) for more information.

When an omnichannel plan in workflow A is processing an event from the secondary workflow B, an important aspect is: which fields does that event include? By default, the plan will receive the event as it entered workflow B, i.e. without any fields added by workflow B. This might not be ideal, if workflow B is modifying some of the fields to standardize the values, or is adding useful computed fields that workflow A needs to access. This problem can be solved with a configuration

property, `workflowReplicaSchemaMarker` . In the example above, you can set this property on the workflow B to mark a point in the workflow (an identifier of a workflow service) at which all the necessary fields have been added or modified. The event will then be shared with workflow A in the same state as when it leaves the marked service.

Data enrichment📖

Pulse's algorithms are integrated with federated data sources such as databases of TOR nodes, compromised IP addresses, malicious emails, etc, to understand if a transaction is related to any of these problematic resources, and thus becoming a suspicious transaction.

Adding these fields to your events raises the probability of obtaining better results, for example for fraud detection.



Feedzai IQ

[Feedzai IQ](#) provides an additional set of federated risk insights that can be used in your workflows, leveraging aggregated financial risk intelligence from the Feedzai global network.

Fields

Fields are the transaction attributes that are passed directly into the instance (i.e. the event that is actually sent to machine learning algorithms). These fields form a subset of the features/instances dataset schema.

For more details on the possible configurations for fields, check the Pulse [data types](#) reference.

Categorical fields

Categorical fields are fields whose values are represented by a set of different values that belong to the same category. For example, a fraud field will have two possible values: **true** or **false**. A country field will have a list of possible values describing country names.

This means that these categorical fields are marked as dimensions and their values are represented by dimension mappings. Make sure that categorical fields have all the necessary dimension mappings.



Caution with high cardinality fields

Be careful when using categorical fields with a high number of dimension mappings. If the cardinality is too high, it might be that some values are never observed during training.

A model that uses these fields will generally result in poor classifications when an instance with such a value is scored. To avoid this problem, create a custom feature and group the field values into buckets that are large enough (e.g. if your system supports hundreds of operation types you should consider grouping them into common categories such as *withdrawal*, *transfer*, *payment*, *query*, etc).

Missing valuesⓘ

In a long-term deployment, you must have resilience to errors. When receiving an event with an unknown value for a categorical field (maybe due to a new currency or operation type) or no value at all (perhaps caused by faulty equipment) that can lead to misleading scoring results.

During the Data Science stage, and whenever you execute a task that generates a [data source](#), for example, as the output of running a [plan](#), you can define how to represent missing values. Pulse sets the value as, for instance, NULL, so that in practice it is no longer missing.

Unlike Data Science, where you know the entirety of your datasets, in runtime, you cannot predict if and when data is incomplete or invalid. Additionally, if you have designed features and metrics that depend on a field value that Pulse cannot process, you would allow errors to propagate through your [workflow](#).

Depending on your scenario you must define a strategy to ensure that Pulse can score these instances, as part of a [data cleaning](#) pre-processing, and/or with custom logic defined at custom workflow services.



This usually involves setting as string the fields that you expect to have some volatility in terms of possible values, for example, *country*.

If you configure a field with the [string type](#), instead of a categorical, Pulse recognizes all values as valid. You will lose the ability to perform Data Science operations directly over the field. However, you can define a plan to remap string values back to categories.

Possible strategies are:

- **Defining a fallback value:** For instance, if the system operates mostly on US transactions you could assume that when an event arrives with an unknown country field, it should be treated as coming from the US.
- **Assigning an "undefined" or "unknown" value to the field:** This alternative explicitly creates an extra category to catch all instances that Pulse would not be able to score. However, you must be aware that you will be scoring instances that may not have all the necessary information for the [model](#) to produce an accurate result. You must ensure that features that depend on the field are not incorrectly generated.
- **Discarding the event:** If you don't handle the unknown values, then Pulse will discard these instances and not score them.

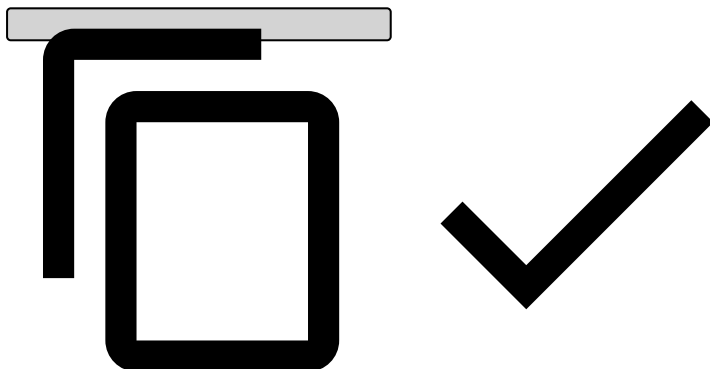
Features

Features allow combining different [metrics](#) and/or perform data transformations over the raw fields of an transaction.

Considering a [model](#) containing a field that indicates whether an operation was performed in a certain country (e.g. a domestic transaction in banks), you can create a feature derived from existing fields. In this case, it would be based on the expression that evaluates to **true** when the country code belongs to the **domestic** country.

For example:

```
(if (cardcountry == "USA") then 1 else 0)
```

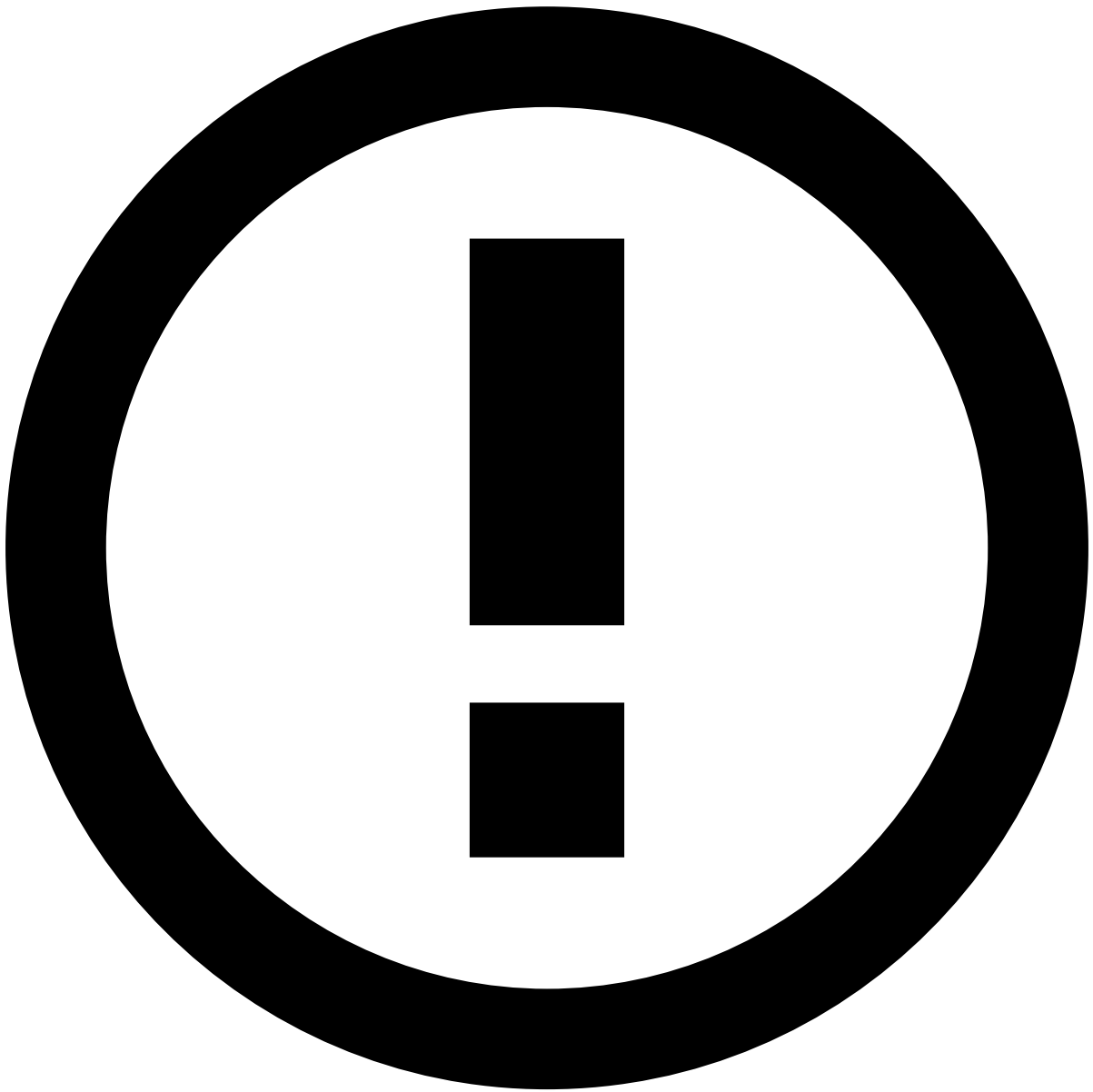


Consider also that you want to derive a feature from metrics: the average transaction amount by country, for the last 24 hours. You can create the following:

- Two metrics - the *total amount by country* and *total transactions by country* for last 24 hours.
- A feature based on the metrics where you divide the first metric by the second one.

The following pages provide practical guides explaining how to add features to your dataset:

- [Creating Derived Features](#)
- [Creating Metrics](#)



Missing values

When specifying the expression you should be aware that the fields and metrics can contain missing values and you need to handle them accordingly. If the user-provided expression does not handle these cases a missing value will be used in its place.

Learn More 

Check the [Workflow concept page](#) for all the details on workflows.

•



- [Core Concepts](#)
- [Workflows](#)
- Railgun Engine

On this page

Railgun Engine

Was this page helpful? 

Railgun is Pulse's real-time [profile engine](#) that calculates aggregate data values [metrics](#) over time windows of any length with high performance, accuracy and throughput.

Due to its optimized architecture, Railgun integrates Pulse's live data processing pipeline, enriching events with historical aggregations in real time as they flow through the stages of a [workflow](#).

Profiling

In the [Feature Engineering stage of the Data Science Loop](#), data scientists can define [metrics](#) that are calculated from the existing fields of a set of events from the data stream.

Such metrics usually represent aggregated statistics. Some typical examples include: average number of transactions in the past 24 hours, or total amount spent by a customer over 30 days, or number of different payment methods used over the last 6 months.

These highly targeted metrics allow *profiling* — i.e. determining patterns in user behavior or transaction activity — in order to detect outliers and suspicious behavior. For this reason, they are called **profiles**.

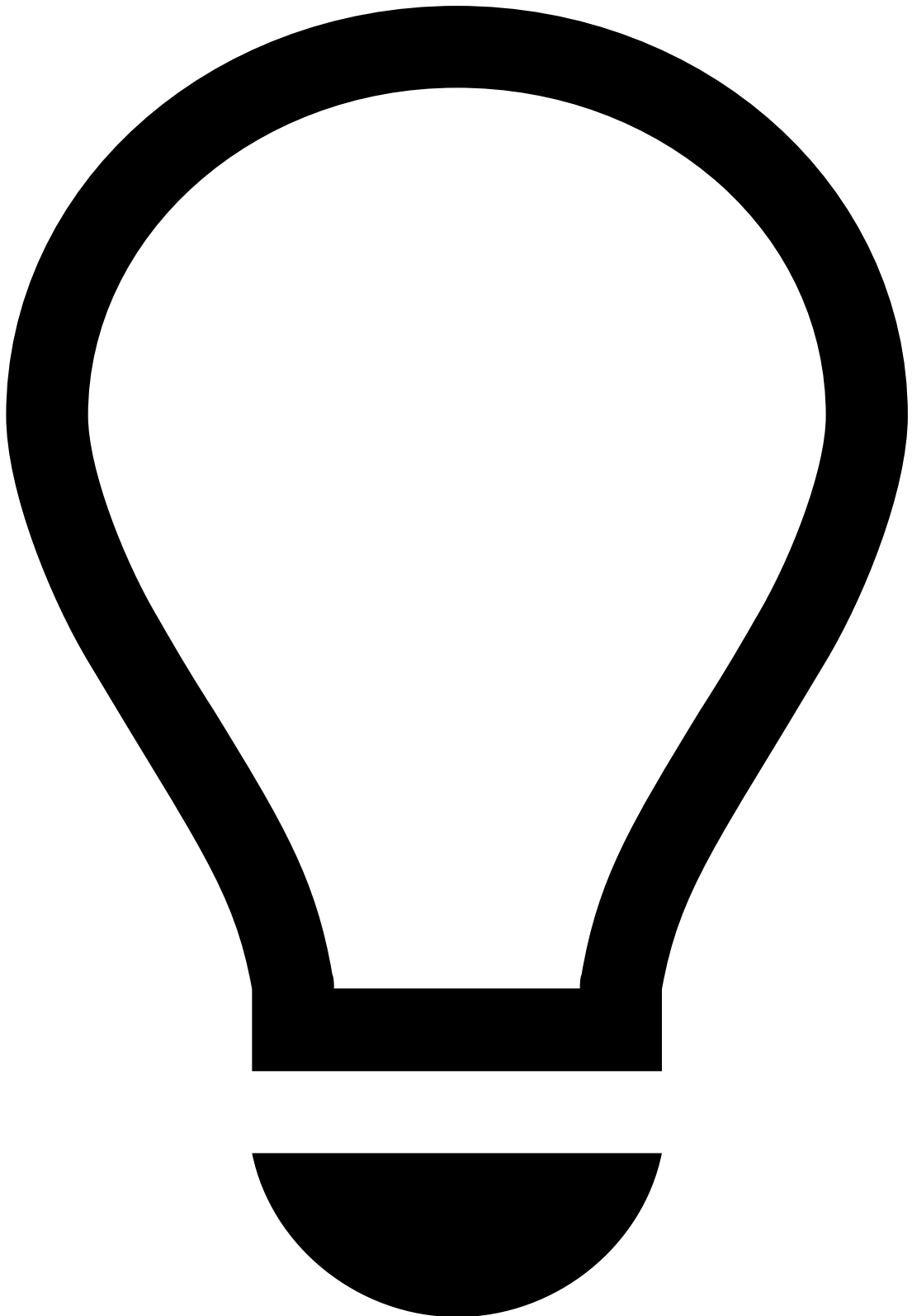
The Railgun engine has been developed specifically for calculating such profiles in an efficient manner, so they can be used when processing [events](#) in real time.

Profiles over long time windows [↗](#)

Calculating profiles for longer time windows (weeks, months or even years) is a common need in financial risk modeling, but it is typically an expensive operation since it requires collecting a potentially large amount of data if the events are frequent.

This means that real-time metrics have traditionally been limited to profiling short-term behaviors, and longer windows were handled by offline, batch jobs, i.e. [Scheduled Metrics Jobs](#) (SMJs).

Since Railgun can compute real-time metrics for long windows with low-memory consumption, it doesn't require the use of SMJs. Therefore, real-time profiles, with full responsiveness and accuracy, can be defined with arbitrarily long time windows, depending only on data retention policy agreements.



Backfills

One of the mechanisms used by Railgun to enable such efficient handling of long time windows is the strategy of making metrics immediately available with the most recent data, and then backfilling the historical data in the background. Learn more about this process in the [Backfills](#) page.

Non-windowed metrics🔖

Besides long time windows, Railgun also supports metrics that are not bound by a time window at all, being limited only by the available historical data at the time of the metric's creation or update (which is determined by the data retention policy).

Other than when creating the metric, or when updating its definition (which requires a backfill), the data retention policy does not affect the metric's temporal scope, since values calculated from data even beyond the data retention policy's window are retained in Railgun's persistent storage.

For more information on this type of metrics, refer to the [Non-windowed Metrics](#) section.

Real-time profiles

The ability to update profiles in real time, rather than as a periodic process, means that Pulse can detect changes in behavior immediately as they happen, rather than later at a scheduled profile calculation. This allows adopting a proactive risk strategy, preventing unwanted activity to take place instead of remediating after the fact, and avoiding costly reversals later. The added agility is also invaluable in thwarting rapid, automated attacks before they reach problematic scales.

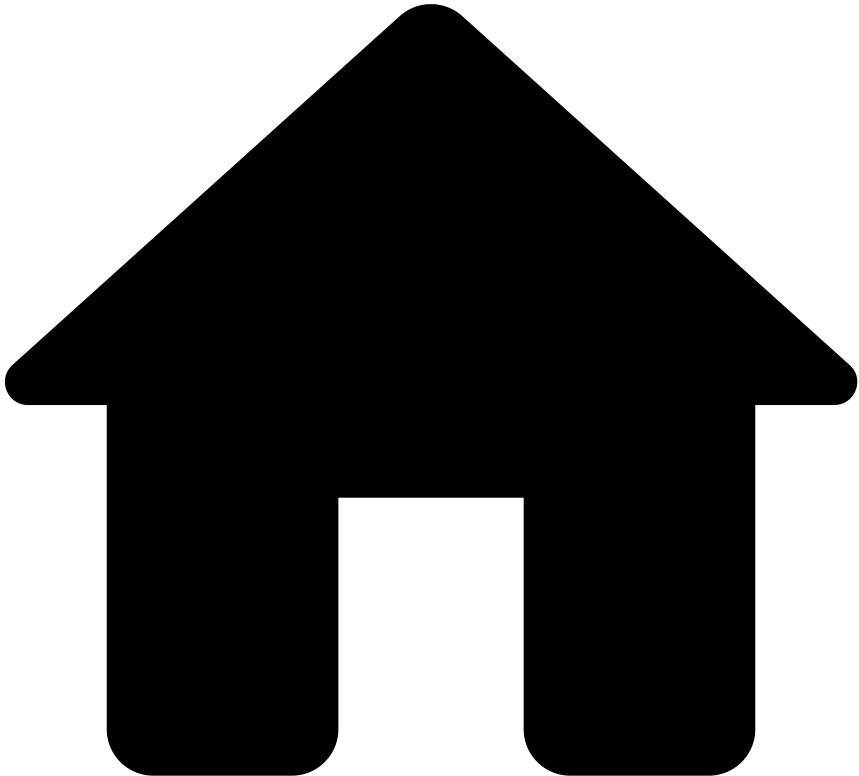
With Railgun, all profiles are calculated in real time, allowing fast and convenient risk strategy updates.

Performance and scalability

Railgun's performance is achieved by processing data as a stream, handling only small batches of data at a time, while updating the aggregated values in real time on persistent storage for use in the later stages of a workflow.

This approach allows maintaining very high throughput, and seamless scalability to larger time windows with moderate memory consumption, while keeping latency consistently low.

•

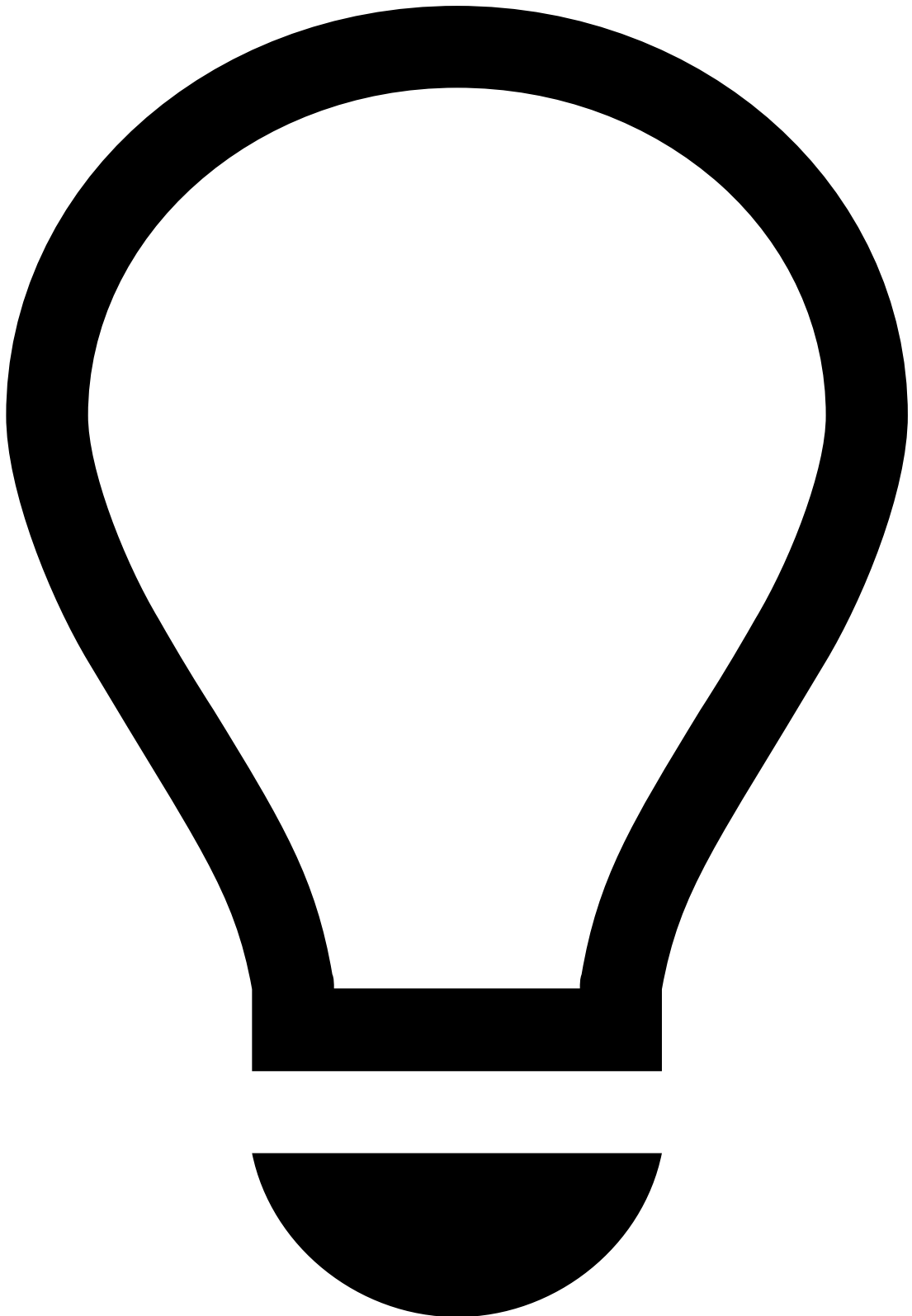


- [Core Concepts](#)
- [Workflows](#)
- Scheduled Metrics Jobs

On this page

Scheduled Metrics Jobs

Was this page helpful? 



Page content not applicable to Pulse with Railgun

When Pulse is configured with the [Railgun engine](#), it is able to calculate all metrics in real time. Therefore, the concept of scheduled metrics, as described in this page, is not applicable.

Scheduled Metrics Jobs (SMJs) are responsible for computing the non-real time metrics (i.e., [scheduled metrics](#)) of profiles at periodic times in batches and persisting them in the profile store, thus making the calculated results available to scoring.

SMJs are a particular type of job , and a specific type of workflow job, that gets automatically registered by Pulse, for example when a [Plan](#) that calculates scheduled metrics is deployed in a runtime [Workflow](#).

Any service in a workflow has the capability to execute jobs and implements them differently. A workflow job can be declared to run periodically, according to the [workflow clock](#), and/or on-demand. For example, if you write a custom service that updates the list of blacklisted emails in an external service, you can declare it to register a workflow job that runs every week.

In the case of plan Services in a workflow, it will register SMJs if there are scheduled metrics configured in the underlying Data Science [Plan](#).

Registered Jobs

An SMJ is registered for each unique metric schedule in a metrics [Plan](#). When the SMJ is executed, Pulse calculates all metrics corresponding to its schedule.

SMJs are launched automatically by the Workflow Job Manager scheduler as events are processed when the workflow clock reaches the SMJ schedule. As with the registered jobs, the SMJs that are launched by the scheduler are those for all metrics with the same schedule in each metrics plan.

Registered jobs are available in the Pulse UI (**Jobs** tab), where the user can launch them on-demand whenever he/she wishes (e.g., recalculate metrics in case of errors).

Regardless of being launched by the scheduler or on-demand, SMJs can run internally (within some AppEngine) or externally, in separate Java processes, making them isolated from each other and from the App Engine. The Job Manager ensures there are no more than a specified number of concurrent SMJs running. Should additional ones be scheduled for execution, they will wait until running jobs finish.

Job Execution

Each SMJ execution processes all metrics corresponding to its schedule, with a single pass in [Event Storage](#) for a period with a duration equal to the maximum window time span among all processed metrics.

In case of Omnichannel plans ([Plans- Deployable Omnichannel Plans](#)), the job will potentially access multiple Event Storages – the one for the main channel, as well as the ones for the secondary channels which are used in the scheduled metrics. Events across all Event Storages will be processed in chronological order (as opposed to processing the Event Storages one by one).

The period end time is determined by how the SMJ is launched:

- If launched by the scheduler, it is the scheduled time.
- If launched on demand, it is the current workflow clock, i.e., the timestamp of the last event processed.

Note that the scheduler fires SMJ executions based on the workflow clock, not on the system time. Thus, if no events are processed, no scheduled SMJs will be launched. See [Time, schedules and timezones](#) for more details on this.

Partitioned Scheduled Metrics Jobs

Processing all events in a single pass may require a significant amount of memory when the metric grouping columns are of high cardinality. You can reduce memory consumption by partitioning the data

and at the same time [configuring SMJs](#) to read the Event Storage in a partitioned way, processing only a part of it on each core.

All SMJ configurations are specified per **Plan Service** element of each workflow. Therefore, SMJs are partitioned only if partitioning is explicitly enabled in the workflow. If not, SMJs will run in normal mode.

Registered Jobs

A metrics [plan](#) is partitionable if and only if all its metrics are partitionable. A metric is partitionable if and only if all workflow partitions include all partition columns. For example:

A workflow is partitioned by:

- `account_id` and

- *merchant_id*

A plan service executes a plan with:

- A daily scheduled metric, grouped by *account_id*
- A daily scheduled metric, grouped by *merchant_id*
- A weekly scheduled metric, grouped by *account_id, merchant_id*
- A weekly scheduled metric grouped by *terminal_id, account_id, merchant_id*

The daily SMJ is not partitionable. In fact, neither of its scheduled metrics are partitionable because they do not contain all the workflow partition keys. The weekly SMJ, instead, is partitionable.

Non-partitionable plans are handled the same way as when partitioning is disabled, i.e., an SMJ is registered for each unique metric schedule. On partitionable plans there is also the same grouping by metric schedule but, rather than registering a single entry per schedule, the following entries are registered for each schedule:

- An entry for all partitions, that allows processing all partitions on demand in a single execution. In this case, all partitions are processed sequentially in a single job execution.
- An entry per partition, that allows processing a single partition on demand.

To improve the performance of non-partitionable SMJs, they execute in a simulated partitionable mode. An SMJ can still be split into several jobs, one per available and configured partition, even though the scheduled metrics are not all partitionable.

Although it is still partitioned, this mode causes each split/partitioned job to read all events from the [EventStorage](#), for the time range covered. This is caused due to the fact that the **Event Storage** does not have a corresponding partition for non-partitionable metrics. Natively partitionable SMJs end up being more efficient since they need only read the corresponding partition from the **Event Storage**.

Contrarily to regular SMJs, there is no one-to-one relation between registered jobs and scheduled job executions - see the scheduled job execution section below.

Scheduled Job Execution

With partitioned SMJs, multiple partitions are to be processed according to the metric schedule, with each partition being processed by a separate job execution. Pulse provides a flexible way to specify when these executions are to be launched, allowing two behaviors:

- All partitions on the scheduled time.
- Dividing the partitions into sets (partition buckets) and distributing bucket processing by the schedule period. This allows distributing processing over time rather than having all partitions processed at the same instant; for example, on a weekly metric schedule, create daily buckets so the processing is distributed on a daily basis.

Regardless of the way SMJs are scheduled, the level of parallelism on which partition SMJs are executed depends only on how the Job Manager is configured (`externalSlots` property), i.e., it can go from strictly sequential to all in parallel.

All Partitions at the Scheduled Time

This is the default behavior and doesn't require any configuration. A job execution is created for each partition at the scheduled time, and the job manager will execute each as soon as possible taking into consideration the `externalSlots`

property and running job executions.

Distributing SMJ Processing with Partition Buckets

In addition, to reduce memory consumption, SMJ partitioning can also be used to distribute processing over time. For example, if we have a weekly schedule and the job total execution time is too long (e.g., if it is supposed to take place during the night but finishes inside business hours), it is preferable to distribute the partitions by each day of the week and thus have a shorter SMJ execution times, although more frequent.

This can be achieved using partition buckets, that is based on the following:

- The metric schedule interval is divided into sub-intervals, with SMJ execution being triggered on each sub-interval. Each SMJ execution processes a subset of the partitions, named partition bucket.
- The number of buckets in each metric schedule interval is defined indirectly by assigning a bucket schedule to the metric schedule. On the previous example, a daily bucket schedule would be assigned to a weekly metric schedule; the metric schedule period (7 days) has 7 bucket schedule periods (1 day each), thus we have 7 buckets.
- Partitions are distributed in a uniform way by buckets. For example, if we had 8 partitions and 7 buckets we would have 6 buckets with one partition and one bucket with two partitions.
- SMJs are launched on the bucket schedule, and each launch processes the partitions of launch time bucket. The bucket correspondent to each time is calculated from the metric and bucket schedules, by calculating the number of bucket schedule intervals that took place between the previous metric schedule time and the current time.

Thus, in order to enable partition buckets, it is only necessary to configure a bucket schedule for each metric schedule where buckets are to be used. This way we define both the number of buckets inside the metric schedule interval and the time when each bucket is to be processed.

Learn More

Check the [Workflow concept page](#) for all the details on Workflows.

•



- [Core Concepts](#)
- [Workflows](#)
- Time, schedules and timezones

On this page

Time, schedules and timezones

Was this page helpful? 

Timestamps: moments in time

Every event that Pulse receives must carry a timestamp — a specification of *when* the event happened. This is necessary so that Pulse can compute metrics such as — number of events in the last 24 hours — (among other things), since this requires knowing which events happened when.

Timestamps can take several forms. The following are the two most common formats:

- UNIX timestamp, i.e. number of milliseconds (or seconds) since January 1, 1970 00:00 UTC. For example, 1620037816000.

- ISO 8601 timestamp, i.e. a string with date, time and timezone. For example, 2021-05-03T09:49:48+00:00.

Timezones

The same timestamp will correspond to different local time (and sometimes date) depending on the timezone:

Timestamp	Time in UTC (Iceland)	Time in UTC-12 (Baker Island)	Time in UTC+8:45 (Eucla)
1620037816000	Monday, 3 May 2021 10:30:16	Sunday, 2 May 2021 22:30:16	Monday, 3 May 2021 19:15:16
2021-05-03T10:30:16+03:00	Monday, 3 May 2021 07:30:16	Sunday, 2 May 2021 19:30:16	Monday, 3 May 2021 16:15:16

As you will see below, some Pulse operations are timezone-dependent. For example, to “run a job at midnight” Pulse needs to know: which midnight? Midnight in Iceland? Midnight in Eucla? Events that Pulse receives may happen in any timezone anywhere in the world. To schedule jobs and perform other operations consistently, Pulse needs to pick one reference timezone and stick with it. This timezone, which we will call the internal timezone, defaults to the local timezone of the machine where Pulse is running. It can be explicitly configured to a different one via the *pulse.global.timezone* configuration property.

This is similar to dealing with different units of measurement. For example, if you are comparing distances to a threshold of 100 meters, you will convert all measurements to meters – it does not matter if they were originally in miles or in yards. In fact, a timestamp does not necessarily convey *where* the event happened anyway. Just because a timestamp is in UTC does not mean the event happened in UTC – it might have been converted by some intermediate system. Similarly, if a distance is expressed in nautical miles, it does not necessarily mean it was measured at sea.

Changing the default value of the internal timezone can be advantageous. For example, consider a deployment for an Australian client where the system architect wants the jobs to run at midnight local time, when the system is under the lowest load. If the machines are not in Australia, the default timezone might be UTC – in this case the job execution time needs to be set to 10 PM. It is much more intuitive though to configure the internal timezone to UTC+10 and set the job execution time to 0 AM.

A word of caution: some timezones, such as “Europe/Lisbon” (as opposed to “WEST” or “UTC+1”), automatically incorporate daylight saving time. In these timezones, the time of 1:30 AM, for example, might be observed twice on the same day (in October) or not at all (in March). For this reason it is not recommended to use such timezones as the internal timezone. This is another reason to explicitly set the internal timezone instead of relying on the default.

Workflow clock

Pulse performs many time-based operations, for example:

- Compute a real-time metric (see [Plans in workflows - Metrics](#)) such as “number of events since the start of the day”
- Run a scheduled metrics job (see [Scheduled Metrics Jobs](#)) at midnight.
- Enable a list item (see [Lists](#)) on 2021-05-03 at 15:00.

From the section above, you already know that this time is relative to the reference timezone. But is it *the time* as we know it? Not quite.

Unlike in the outside world, **time in Pulse only moves when Pulse receives events**. For example, if Pulse needs to enable a list item at 15:00, it will not do so until it receives an event that happened after 15:00 (even if it has to wait for days). This allows Pulse to correctly process events if they are consistently late. For example, a bank might only send information about transfers made during the weekend on Monday morning. In this case, when processing each transfer, Pulse “pretends” that it is still the weekend.

It’s also important that (under normal circumstances) **time in Pulse only moves forward**. While Pulse is designed to handle consistently delayed events correctly (per above), it does not retroactively process late events if more recent events have arrived in the meantime.

Instead, Pulse processes these “out of order” events as if they were recent. This is a limitation of how Pulse computes real-time metrics, but it affects the logic for the entire workflow.

Here is how this works in practice. The current time in Pulse is stored in the **workflow clock**.

- When there are no events, the clock does not change.
- Each new event, if it's more recent than the workflow clock, will advance the clock. For example, if the clock is at 1620037816000 (10:30 UTC) and there is a new event with a timestamp of 09:40 UTC-1 (i.e. 10:40 UTC), the clock will advance to 1620038416000 (10:40 UTC).
- If a new event is older than the clock, Pulse will override its timestamp to equal the clock. The clock itself remains the same.

How workflow clock and internal timezone affect Pulse functionality

In this section you will see how Pulse clock and the internal timezone affect different functionality of Pulse.

Lists

Pulse [Lists](#) use the workflow clock to enable and disable items. For example, if an item needs to be enabled on 2021-05-03 at 10:30 UTC, it will happen when the workflow clock reaches that moment in time (i.e. when Pulse receives an event with a timestamp higher or equal to 2021-05-03 10:30 UTC).

Real-time metrics

Real-time metrics (see [Plans in workflows - Metrics](#)) use the workflow clock to compute aggregations over time periods. For example, in a metric like "total amount transacted by customer in the last 24 hours", 24 hours are counted backwards from the workflow clock, i.e. the latest event received by Pulse.

Real-time metrics may also rely on the internal timezone. For example, a metric with a window expression *reset (_: day)*, which means the metric is reset every day at midnight, will depend on Pulse's notion of midnight. If Pulse timezone is set to UTC, then the metric will reset at midnight UTC. (Remember, the timezone of where the events actually happened is irrelevant.)

Scheduled metrics jobs

[Scheduled Metrics Jobs](#) can be triggered in two ways:

- Automatically, based on their schedule (e.g. "run every midnight").
- Manually, using a button in the Pulse UI.

In both cases, Pulse uses the workflow clock, but the behavior is different. There are two main questions:

- When does the job start?
- What time period does the job consider?

Here is a quick summary, assuming the job computes metrics with windows of 2 days.

Job triggered...	Job starts...	Time period
Automatically on schedule (every midnight)	When workflow clock reaches "midnight" in internal timezone .	Events with timestamps within 2 days of "midnight" in internal timezone .
Manually (through the UI)	Immediately	Events with timestamps within 2 days of the current workflow clock .

Below are a few important considerations based on this behavior.

Automatically triggered jobs

Keep in mind when thinking about when the job starts:

- The job is triggered based on the workflow clock, not real-world clock. If it's 1 AM in the real world, but the last event received has a timestamp of 23:59, **the job will not trigger until a more recent event is received**.
- **If Pulse does not receive events on a given day, the job for that day will never trigger.** If Pulse only receives events on April 30 and then only on May 3, the first event of April 30 will trigger the job for that day, and the first event of May 3 will trigger the job for that day. No jobs will run for May 1 or May 2.

- In case of Omnichannel scheduled metrics (see [Plans in workflows - Omnichannel metrics](#)), there are multiple workflows involved: the main workflow and one or more secondary workflows. In this case Pulse uses the **clock of the main workflow**. The same considerations as above apply. For example, **if there are no events on the main channel, the job will not trigger**, no matter how many events are received on the secondary channels.

Keep in mind when thinking about the time period considered by job:

- The time period for the job is calculated as follows: time when the job is supposed to trigger + biggest metric window size. Note again that the time period is not counted from current value of the workflow clock, but from the time the job is scheduled to run (e.g. midnight).
- The time the job is scheduled to run (e.g. midnight) is interpreted according to the internal timezone.
- In case of Omnichannel scheduled metrics (see [Plans in workflows - Omnichannel metrics](#)), **the same time period is used consistently across all channels**. If the main workflow clock is on May 3, the job will correspond to midnight of May 3 and the time period will be counted from that moment in all channels. It does not matter if some of the secondary workflows, for example, have not received events since May 2 or, to the contrary, have already received events from May 4.

Example:

Workflow clock	Job schedule	Window size	Time period
May 3, 01:30 AM	midnight (0 AM)	2 days	May 1, 0 AM – May 3, 0 AM

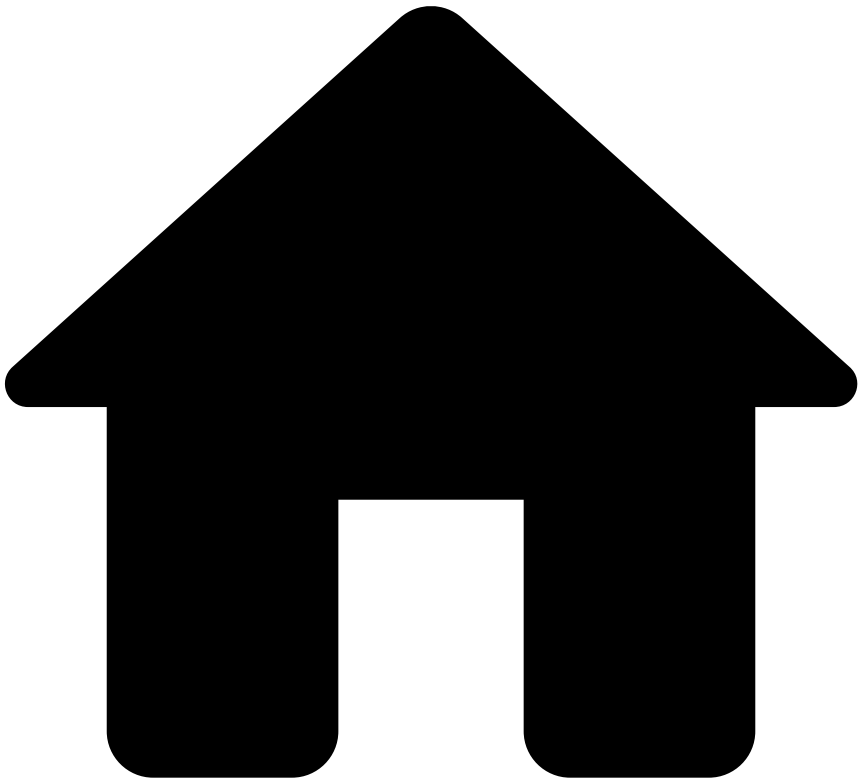
Manually triggered jobs

- The time period is until the timestamp of last event received. If you trigger a job on Monday but the last received event was on Friday, the time period will be until Friday, not Monday.
- In Omnichannel scheduled metrics, the time period is until the timestamp of the last event received on the main channel. Like with automatic jobs, the same time period is applied consistently across all channels.

Example:

Workflow clock	Job schedule	Window size	Time period
May 3, 10:30 AM	midnight (0 AM)	2 days	May 1, 10:30 AM – May 3, 10:30 AM

•



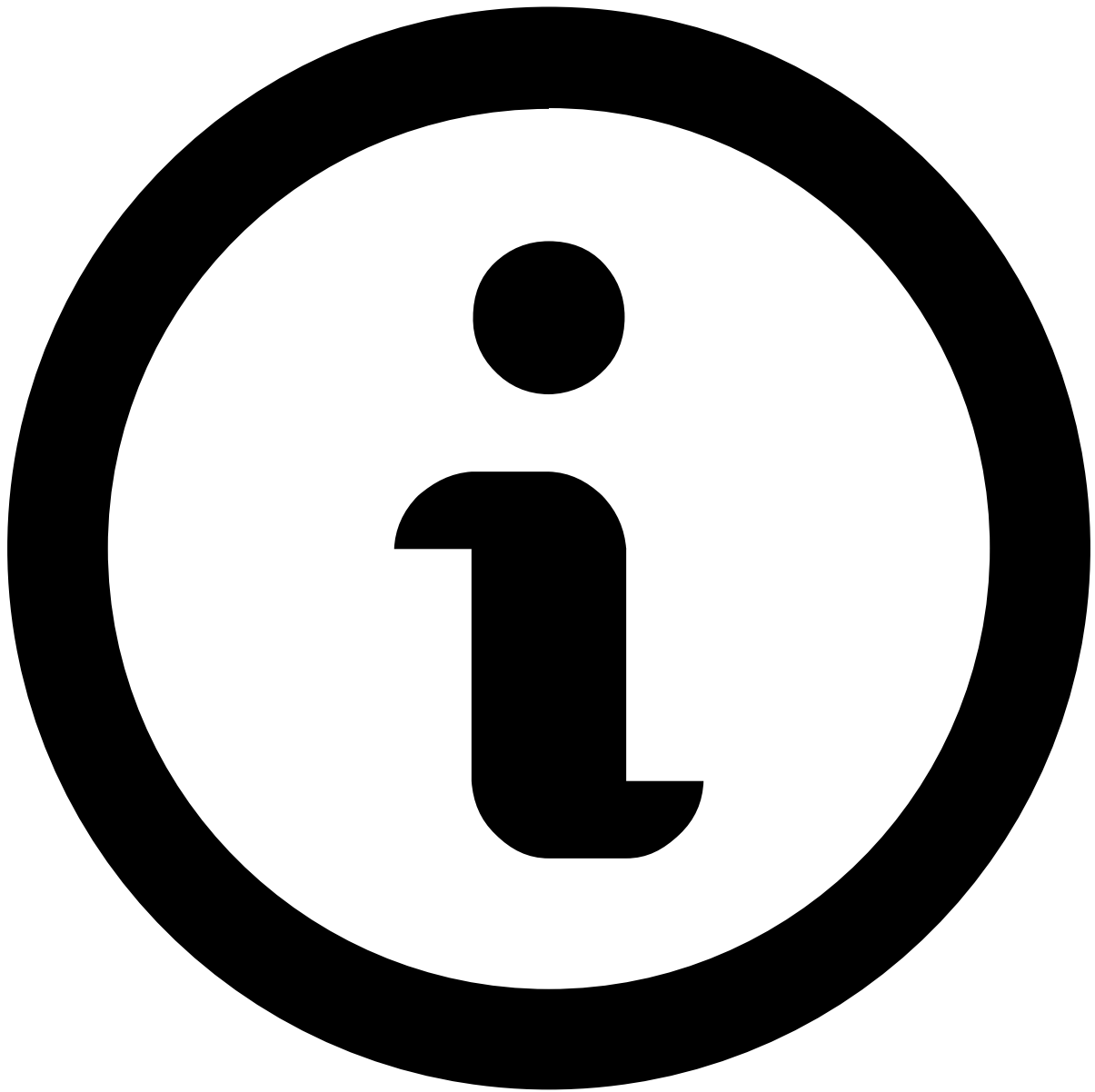
- Best Practices
- [AutoML](#)
- Feature Engineering Stage

On this page

Was this page helpful? 

Best Practices for AutoML's Feature Engineering Stage

At the start of a project, the schema is usually already defined. In addition, fraud or money laundering patterns should also be characterized and understood so that you know which fields to use in the transformations. The transformations already encode Feedzai's knowledge into what is the best strategy to build features to tackle financial crime. This is all that is needed for AutoML basics: you can now create a semantic file with such knowledge in your hand.



Schema definition

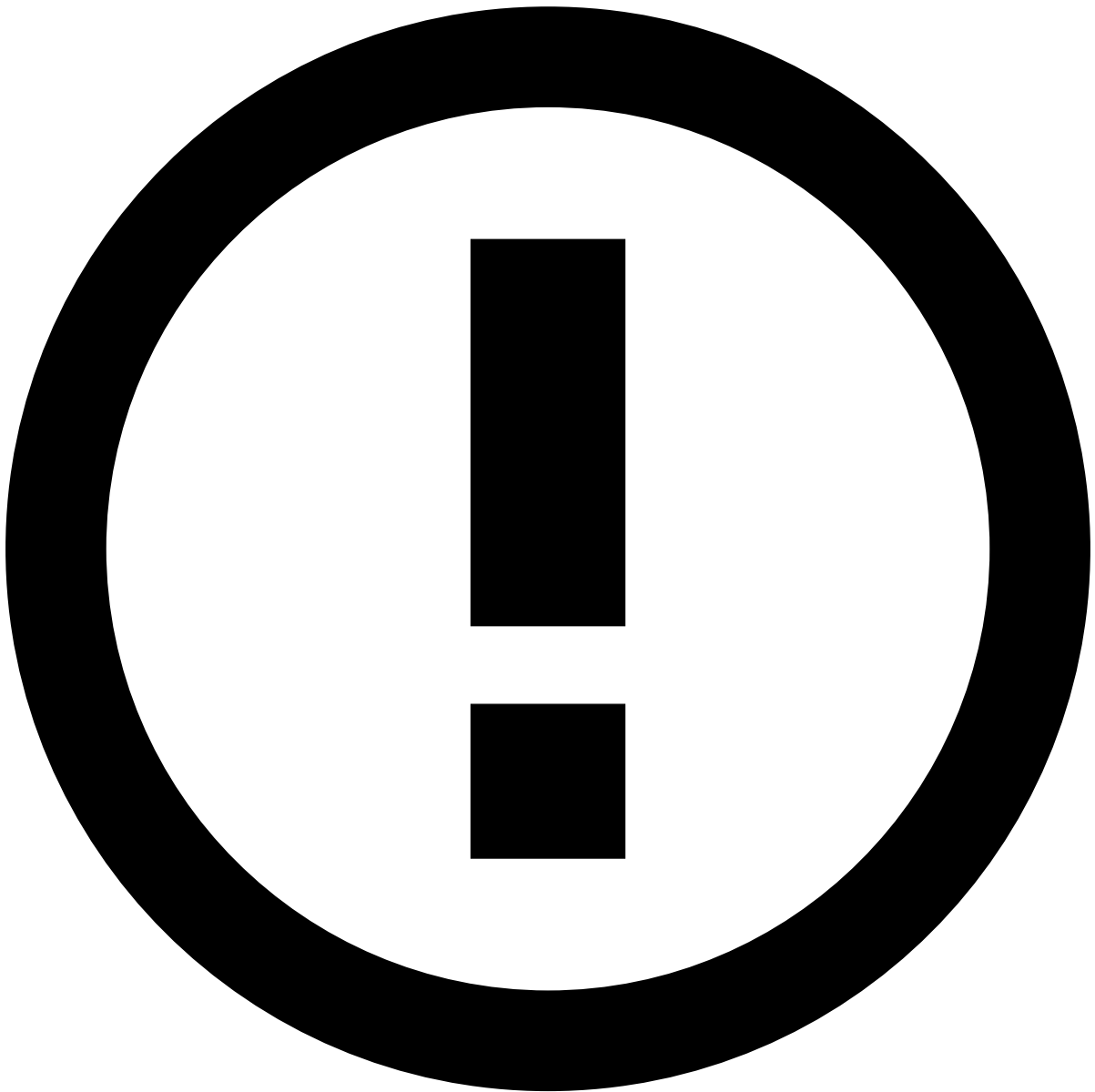
Sometimes it's hard to have schemas completely defined at the beginning of a project. The same applies to the definition of scorable events, and other important aspects of the underlying data generation process that we do not control.

Take this opportunity to reflect on what's the minimum reliable information you have about the data. With it, consider building the semantic file with the least assumptions possible. Start by selecting only one grouping entity and tagging the most important fields, e.g., event timestamp, and transaction amount.

Semantic files let you map schema fields to common entities that are useful for feature engineering. Doing this allows you to speed up the initial task of creating a feature engineering plan from nothing. So, in case you don't have any data yet, you can still use AutoML to create a plan with features that leverage domain knowledge, therefore expected to be useful.

[Learn which tags are supported by semantic files.](#)

When you want to have aggregations with more than one grouping entity, you need to create a *composite field*. This is useful for the case when you want to track behavior over an entity with low cardinality. For example, when processing money transfers, you will most likely want to track how many transactions a person does for a given country. **To do this, you need to define a composite field that combines the originator account and the destination country.**



entity vs grouping-entity tags

Entity tags will only be used for *countDistinct* aggregations. When defining a field as an entity, AutoML will not use it for counts, sums, or any other aggregation that is not a distinct aggregation.

Imagine this scenario for a payment processor's data set: define `customer_id` as grouping-entity and `merchant_country` as an entity. This combination will only create the following subset of profiles. For the sake of simplicity, we're omitting other aggregations based on timestamp (e.g. velocities), or numeric fields (e.g. avg amount).

- `group by customer_id count()`
- `group by customer_id countDistinct(merchant_country)`

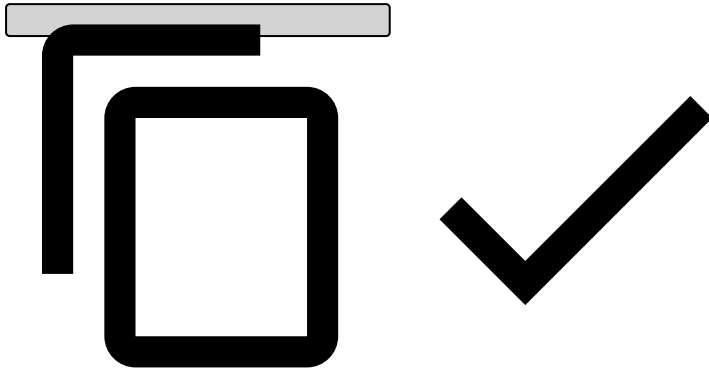
If you also want to track the counts per customer and merchant country, you need to define a composite grouping entity:

```
{
  "@type": "composite",
  "fields": [
    "customer_id",
    "merchant_country"
  ]
}
```

```

    ],
    "tags": [ "grouping-entity" ],
    "fieldDesc": "customer_id_merchant_country"
  }

```



Keep in mind that grouping entities with very low cardinality (<10) or highly skewed, can compromise your job execution. Check the [Don'ts](#) section below for more context.

Remember that grouping entities will not just create the aggregations/profiles. These metrics will also be used to compute ratios and other derived transformations.

The following is an example of a semantic file.

semantic-file.json

```

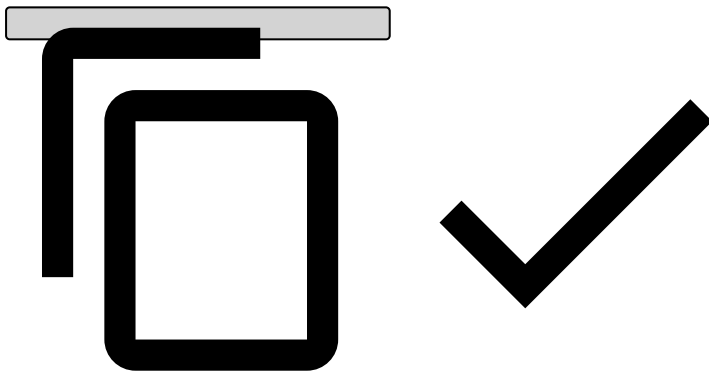
[
  {
    {
      "@type": "simple",
      "tags": [
        "amount"
      ],
      "fieldDesc": "amount"
    },
    {
      "@type": "simple",
      "tags": [
        "entity"
      ],
      "fieldDesc": "acceptor_id"
    },
    {
      "@type": "simple",
      "tags": [
        "entity",
        "country",
        "location"
      ],
      "fieldDesc": "acceptor_country_code"
    },
    {
      "@type": "simple",
      "tags": [
        "timestamp",
        "datetime"
      ],
      "fieldDesc": "event_occurred_at"
    },
    {
      "@type": "simple",
      "tags": [
        "grouping-entity"

```

```

    ],
    "fieldDesc": "card_pan"
  },
  {
    "@type": "simple",
    "tags": [
      "amount"
    ],
    "fieldDesc": "account_available_credit"
  },
  {
    "@type": "simple",
    "tags": [
      "datetime"
    ],
    "fieldDesc": "account_open_date"
  },
  {
    "@type": "simple",
    "tags": [
      "grouping-entity"
    ],
    "fieldDesc": "customer_id"
  }
]

```



Based on the semantic file, AutoML will create a **deployable plan** where it will expand as many features as it can.

Before submitting your AutoML run, be sure to create job profiles with appropriate Spark configurations considering your data volume and complexity. AutoML will use the default job profile of Pulse for each type of job it launches. The only exception is the job profile for feature selection, for which AutoML will use the profile configured in `pulse.properties`. If your AutoML run fails during the feature engineering phase, it's probably due to memory issues. Use job profiles to tune memory usage.

A crucial consideration that affects the feature engineering process is the existence of **partitioning in runtime**. Briefly, runtime partitioning is essential when processing very high throughputs, e.g. 1000 TPS at peak. Having your data partitioned across different machines means that machine 1 will not see **in real time** the data of machine 2. Because of this, you will need to decide what is your main grouping entity and make sure it is the same as the partitioning key. This is usually the customer/card/account field. If you don't match the partitioning keys in the grouping, then AutoML will generate real-time metrics that might not be possible to compute during runtime.

Dosâ

- Verify which timestamp field of your schema is the one that will be used to represent the time in which the event occurred. This field will need to be provided in the first page of the AutoML wizard. **This is the timestamp used to compute the metrics.**
- **Choose one main grouping entity only.** Even if your data is not partitioned in runtime now, consider the likelihood of having partitioning in the future. Remember that when your data is partitioned you won't be able to

leverage entities that can be seen across multiple runtime partitions. On the other hand, more grouping entities lead to a more thorough exploration of feature combinations.

Putting the runtime partitioning issue aside, also consider the pros/cons of having potential marginal gains on model performance versus strain on processing pipeline. Think about what behavior mostly describes your fraud or money laundering patterns. Typical grouping entities are: customer/account, or card (in a card channel).

- Create a job profile that is reasonable for your feature engineering plan.
- If you expect very large data volumes (1M events per day or more), tweak window sizes under the advanced options. As a general rule, avoid real-time windows greater than 1 day: real-time windows are kept in memory, and thus impact memory performance as well as recovery. Read more about this under the [Metrics best practices](#) pages.
- When starting from a Solution (e. g. Transaction Fraud for Retail Banking Cards or Transaction Fraud for Retail Banking Transfers and Instant Transfers), always check first its template semantic file.

Don'ts🚫🚫

- **Avoid grouping entities with very low cardinality** (i.e. low number of different values). This will depend on the project so be aware of this when reusing semantic files. For example, if you're looking at data mostly from 1 or 2 countries, using the country code as a grouping entity can be problematic. All your data will be split per these categories, and your pipeline will run in a very large chunk of data. This in turn will cause out-of-memory errors. However, if you're expecting your data to be well distributed across many countries **and you don't have partitioning in runtime**, this could be a reasonable **grouping** entity.
- Be mindful of your `Entity` tags. Entities are used for distinct counts (e.g. distinct beneficiaries per customer). Too many distinct aggregations on long windows in large data sets is known to have a high memory footprint in the Data Science environment. In addition, these aggregations are very computation-intensive in runtime. As a rule of thumb, try to keep the entities to the minimum needed for the project you are doing.

Data split stage🚫🚫

The data source splitting stage is a sub-stage in the feature engineering stage. At the data split stage, Pulse will split your data set into two or three subsets: training, validation, and optionally testing. Although this intermediate stage is rarely a point of failure, it still deserves some considerations.

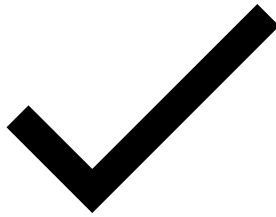
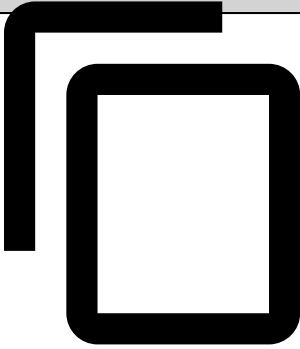
As a general Machine Learning guideline, these splits are 80/20/20, i.e., 80% of your data for training, 20% for validation, and 20% for testing (hold-out data set). Here, we enforce that the split be defined timewise, because our transformations are time sensitive.

Splitting is done through a filter, which can generate data with empty partitions (or blocks) for the validation and testing splits. This doesn't impact the model results. However, Data Scientists might want to use these validation and test sets for other operations. In this case, they will experience degraded computational performance compared to the same data with these empty blocks removed. In the long run, having empty data partitions will cause HDFS/S3 degradation related to having many empty files/blocks.

To solve this, manually partition your data through JupyterLab by coalescing the data source. As a rule of thumb, if the data source is 20% of the initial data, it should have 20% the number of partitions (see the snippet below).

```
>>> # Load the validation set through the ds-api
>>> df_valid = feedzai.pulse_to_pyspark(
...     spark_session, app_name, datasource_name
... )
>>> # The number of partitions will be the same as in the feature eng data source
>>> df_valid.rdd.getNumPartitions()
1000
>>> feedzai.pyspark_to_pulse(
...     spark_session,
...     # Save a new data source coalesced to have only 200 partitions
...     df_valid.coalesce(200),
...     app_name,
```

```
...     datasource_name + "_coalesced",  
...     "/path/for/this/new/datasource",  
... )
```



•



- Best Practices
- [AutoML](#)
- Feature Selection Stage

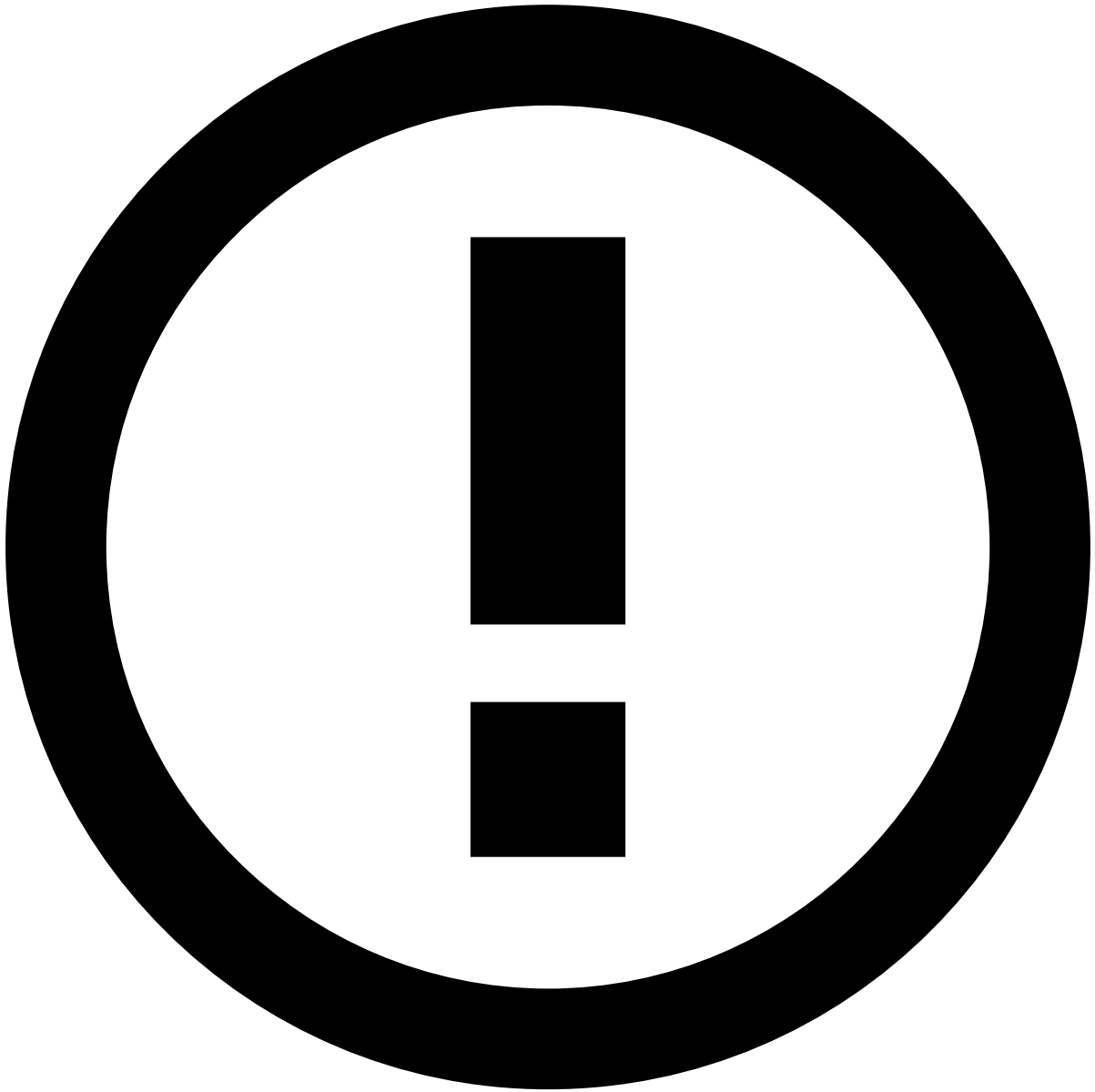
Was this page helpful? 

Best Practices for AutoML's Feature Selection Stage

The main concept behind AutoML's feature engineering approach is expansion-reduction. The goal is to explore as many possible features as it is computationally feasible. Then, only the most relevant features should be chosen.

In the reduction step, AutoML trains a Gradient Boosted Trees (GBT) model on a sample of your development data. The GBT model is used as a surrogate to get its model feature importance. By default this step selects the top-K features so that their relative importance is 90% of the total.

On the advanced options of the AutoML wizard you can select the sample size that will be used for the GBT model.



Feature importance

This methodology is different than the one used by Pulse for individual model importance. Speed is paramount for AutoML. Thus, it's preferable to use GBT on a sample of the training data. This enables faster iterations.

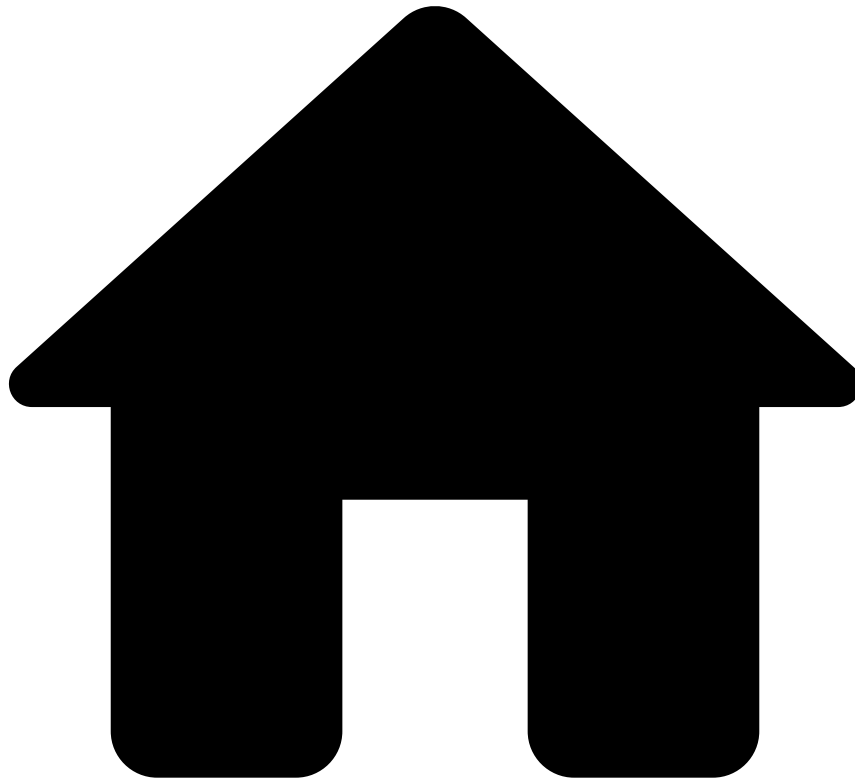
Pulse's feature importance is done by computing the difference in Recall after shuffling each feature individually, for a given FP-Rate. This makes this algorithm model agnostic. However, it also makes it computationally heavy when the number of features is large and/or you have a very unbalanced data set (and thus need a larger sample).

AutoML's feature importance for feature selection is a simpler algorithm that trains a GBT model and gets its feature importance. Only one model is trained, and thus it only goes through the randomly sampled data once.

If your target rate is very low on your features data (e. g. 1 basis point), bear in mind you should tune your feature importance job to use approximately 1M instances.

At the end of the feature selection stage, Pulse will create a new features plan with **only the selected features and their dependencies**.

•



- Best Practices
- [AutoML](#)
- Hyperparameter Tuning Stage

On this page

Was this page helpful? 

Best Practices for AutoML's Hyperparameter Tuning Stage

The last stage of AutoML is hyperparameter tuning (HPT). Here, AutoML will train a range of models through OpenML, tuning their hyperparameters in one go. There are some important considerations during this stage:

1. This will use the **model training** logic: 1 big Spark task per model.
2. Trained models are kept on the machine where they are trained, and then discarded. This happens because the goal of HPT is getting the best model configuration and not all the model binaries. At the end of the run the best model will be retrained and kept within Pulse.
3. Evaluations will **not be distributed**. They run with as much parallelism as the model training, i.e., 1 Spark task.

4. If 1 model fails to train, then the HPT stage also fails. This happens because HPT considered a single job as part of the AutoML pipeline. If you want to re-run HPT under a different job configuration, you can do so by using an empty semantic file, and selecting the appropriate data source as input, i.e., the one with features already computed.
5. It will use **Random Search**.

That being said, most best practices here rely on configuration. Check the [model training documentation](#) that explains how to create a job profile for model training. From there you will see that you should:

- Size the executor for the biggest machines in the cluster.
- Allocate all the cores of the machine for the single task with `spark.task.cpus`.
- Allocate all the cores and memory of the machine with `spark.executor.cores` and `spark.executor.memory`. Leave only a little overhead with `spark.executor.memoryOverhead` to account for serialization/deserialization overheads.
- No need for any meaningful YARN memory overhead since there are no shuffles.
- Reduce `spark.memory.fraction` significantly, e.g., to 0.05; since there is no shuffling or caching, it is all user memory for the training.

Remember that one of the possible sources of failure is that the data must fit in memory during training. Because of this you should tune your model training jobs to use the largest machines. The plural here is important. Be aware that if you tune your job to fit only a few machines (say, 1 or 2), you will only be able to run that same number of models in parallel. In addition, to speed up this job, try leveraging the CPU cores the machines have available. Don't forget that the job will not be distributed. It will run 1 task per model.

This approach contrasts with the typical Model evaluations. In this case your evaluation will be done streaming your validation data in one very long task. That being said, **if your data set is large, use a small time period for the evaluations in HPT.**

Hyperparameter tuning algorithm

The algorithm used in AutoML is different from the one used on a regular HPT job.

The manual HPT job uses grid search, i.e., the user specifies which parameters will be used. AutoML uses Random Search instead. For random search, Pulse will draw the possible parameters from the following probability functions:

Feedzai Random Forest

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
Depth	20	5	100	x
Trees	100	50	500	x
In-bag factor	0.65	0.2	1	x
N. Features	$0.3 \times k$	-4	-0.5	$k \times 2^x$, where k is the number of features.
Split method	one-vs-others	one-vs-others all-vs-all		Discrete.
Min. instances per split	2	0	1/3	n^x , where n is the number of samples.
Prior	Parent			No search.

Generalized Linear Model (GLM)

The lambda search is deferred to H2O to perform it internally.

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
alpha	empty	0	1	x
lambda_search	On			

XGBoost

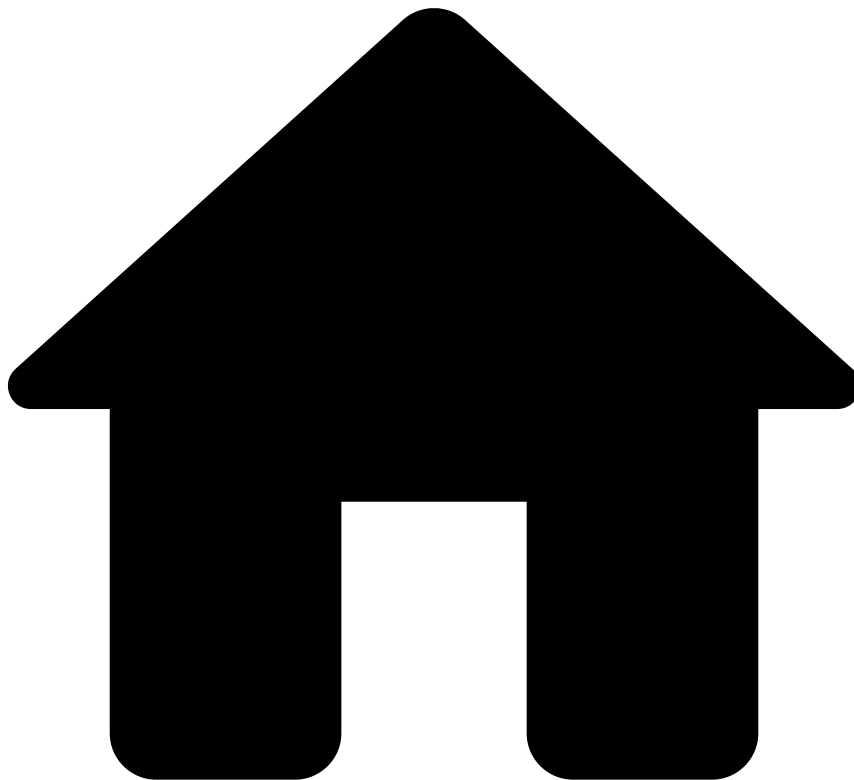
Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
ntrees	100	50	500	x
learn_rate	0.2	-6	-1	2^x
max_depth	6	2	15	x
min_child_weight	2	0	5	2^x
sample_rate	1	0	1	x
col_sample_rate	0.5	0	1	x
col_sample_rate_per_tree	1	0	1	x
reg_alpha	1	-5	3	2^x
reg_lambda	1	-5	3	2^x

Gradient Boosting Machine (GBM)

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
ntrees	100	50	500	x
learn_rate	0.1	-6	-1	2^x
max_depth	6	2	15	x
min_rows	2	0	5	2^x
sample_rate	1	0	1	x
col_sample_rate	0.5	0	1	x

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
col_sample_rate_per_tree	1	0	1	x

•



- Best Practices
- [Metrics](#)
- Aggregations

On this page

Was this page helpful? 

Aggregations

The core aspect of [metrics](#) is the ability to perform aggregations over time on a grouping entity (or a set of grouping entities). The general rule when creating the actual metrics is always to keep it as simple as possible. This applies also to the window definition and to the actual aggregation expression. Let's break down each one of these:

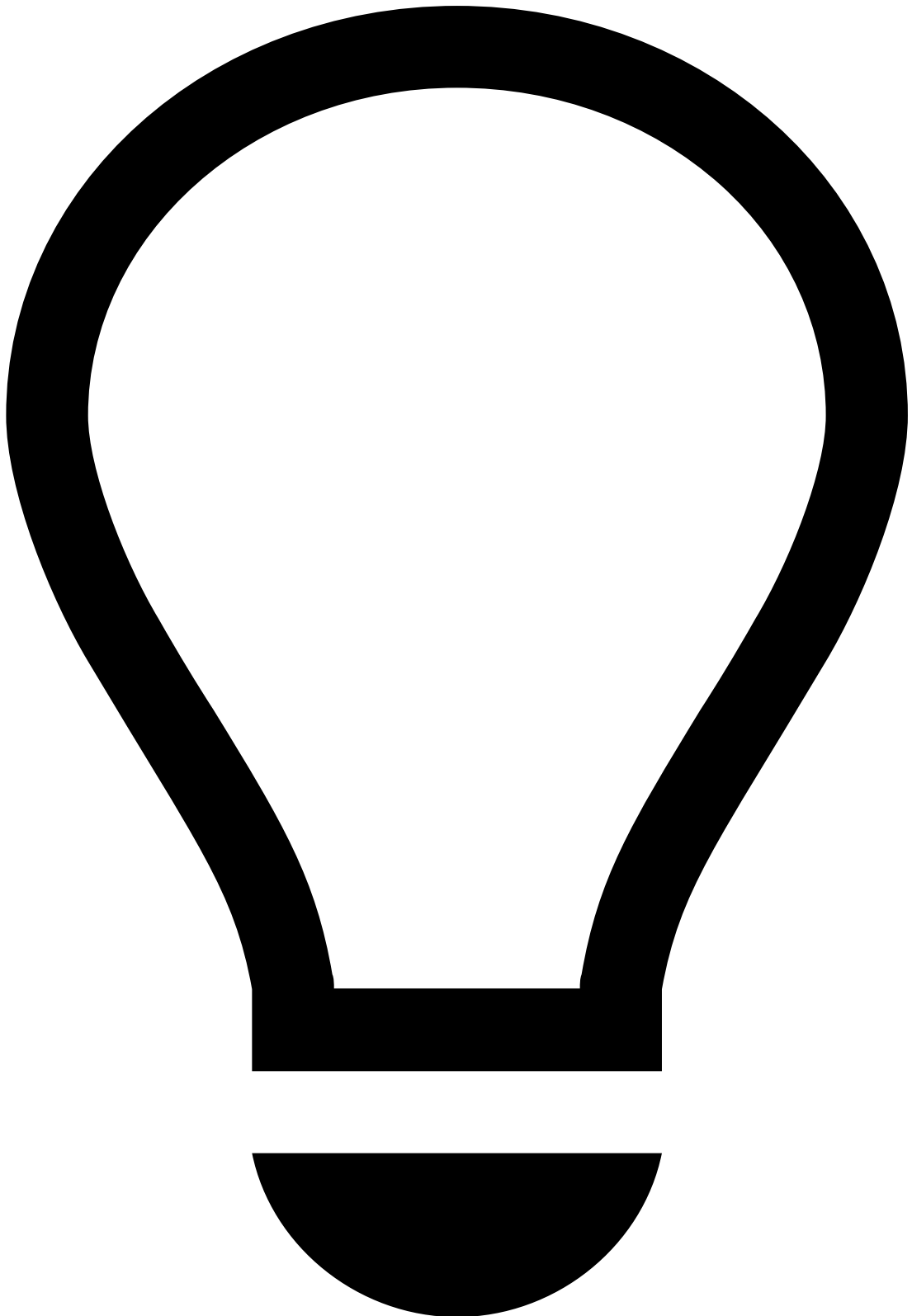
Windows

The rule of thumb is to have two main sets of windows: 1 for real time (e.g. 24 hours), and another for scheduled metrics (e.g. 1 month). However, note that with very large payloads in runtime, the size of the real time windows might need to be adjusted. Real time windows, albeit faster to compute, imply that data that goes inside the window (c.f. [filters](#)) must be

kept in memory. Also, during recovery, the data pertaining to the largest real-time window must be reinjected into the workflow before operations can resume.

Group-by

When using the [Group By operator](#) to group by multiple fields, always have the high-cardinality ones first since they will dictate [how the data is partitioned in the Spark shuffle](#) (if you want to group by `card_pan` and `merchant_id`, and there are 1M cards but just 100 merchants, you will want to select `card_pan` first to avoid having 100 very large tasks that might run out of memory).



tip

This manual optimization is not necessary when using [Railgun](#).

Aggregation📄📄

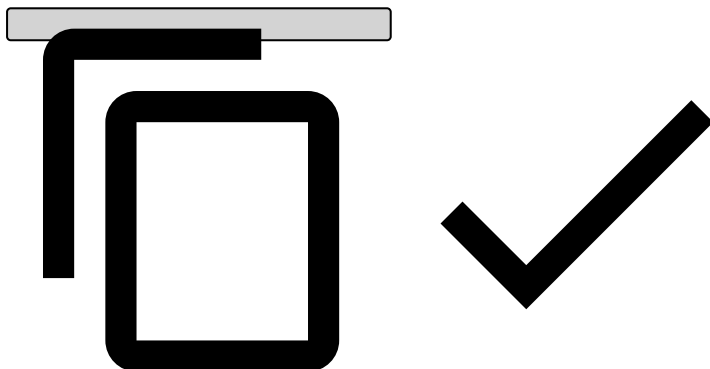
The "aggregation" input lets you define what type of aggregation is to be done. Note that the `countDistinct` aggregation is more performant in runtime than in the Data Science Platform. This means that having a lot of `countDistinct` s will make your Data Science job slower. In addition, these are memory intensive aggregations, as the whole state is needed to be able to expire elements as the clock moves forward.

Some advanced aggregation examples:

Velocity of transactions¹

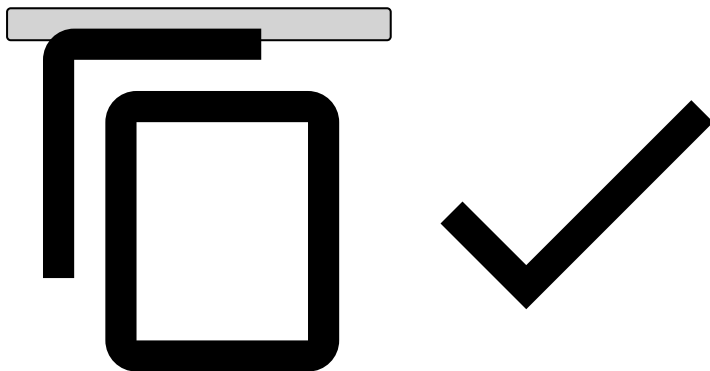
```
(
  let first_timestamp = first().event_occurred_at ?? 0 in
  let last_timestamp  = last().event_occurred_at  ?? 0 in
  let time_span       = (1 + last_timestamp - first_timestamp) / ( 60*60*1000.0 ) in
  let count           = count() in

  ( if ( count <= 1 ) then 0 else count / time_span )
)
```



The time since the previous event within this aggregation¹

```
// (in milliseconds)
(
  ( last().event_occurred_at ?? 1 ) - ( prev().event_occurred_at ?? 1 )
)
```



Do's¹

- Use "simple" aggregations (from the UI).
- Use Approximate count distincts if you expect the counts to be large.
- Create an "aggregation repository" with the expressions that are commonly used. This will help you avoid bugs.

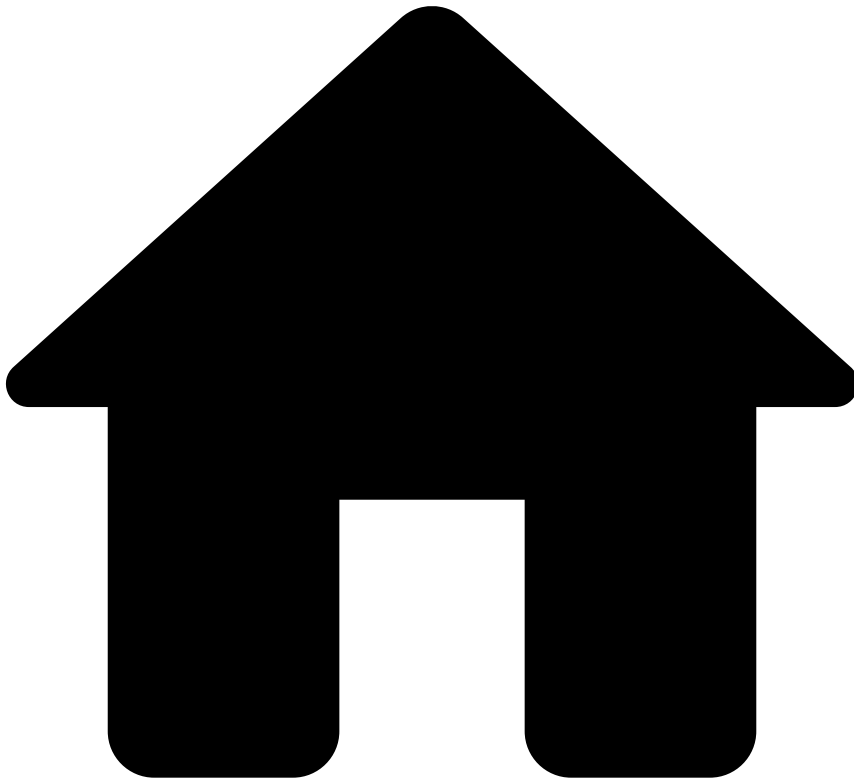
Don'ts¹

- Avoid writing too many advanced aggregations.
- Avoid count distincts with long windows. Distinct aggregations are the most costly, because all the events must be kept inside the window at all times. (c.f. Approximate aggregations). This cost mostly affects the Data Science Platform and not necessarily Runtime (see [Scheduled Metrics Jobs](#)).
- Don't use real-time metrics with long windows unless you have direct support from Feedzai's product team. The experimental nature of the feature means that it is not recommended for general usage.

Advanced optimization techniques

- Salting null values/large entities that can be ignored. SMJs already ignore null values.
- Metrics are very sensitive to data skew and suboptimal data partitioning. Make sure your data sources are well partitioned by checking that each uncompressed CSV part takes about 128 MB in HDFS or closer to 512 MB in S3. That being said, as a rule of thumb, as long as each part is about 100 MB in size, it should be good enough. If you have too many partitions, you can coalesce them manually. Check the Spark documentation for more information on coalescing.
- Fields that are not necessary in a feature engineering plan should be dropped at the beginning, or in a preprocessing plan or JupyterLab script. This is due to Spark that sends data across machines during execution. By removing columns that are unused, Spark will send less data between machines. This will allow you to use less memory per executor, thus providing more parallelism. **Rule of thumb:** start by looking at large complex fields and constant fields.
- Filters inside metrics do provide *some* improvements in memory usage. They are not as impactful as removing unnecessary fields. Generally speaking, if you are measuring something that can be tracked only in specific event types, try to have as specific a filter as possible. Even though this won't reduce the bulk memory consumption by Spark, it will reduce memory being consumed during the aggregation.

.



- Best Practices
- [Metrics](#)
- Constraints

On this page

Was this page helpful? 

Constraints

Although Pulse uses the same feature definition offline (Data Science Platform) and online (Runtime Environment), there are important distinctions between them.

Development vs. live scoring

This distinction means that as a best practice, runtime environment constraints should be known upfront before creating metrics manually.

In the following sections you can find the most common constraints when developing metrics.

Runtime Environment📄

When processing high throughputs in runtime (e.g. 1000 TPS at peak), it is advisable to have **runtime partitioning**. This means that events will be processed by different machines, according to a partitioning strategy. The most common strategies are to partition by card/account. This way it is guaranteed that all events from cards are processed in **real time** by the same machine.

So when designing **real time metrics** you must be aware if partitioning is active in your runtime environment. If it is, and the metric does not contain said key as the root grouping entity, there will be a discrepancy in the calculations between the Data Science and Runtime environments. Note that **this does not affect scheduled metric jobs (SMJs)**. Because SMJs are applied to events fetched from the Event Storage, they will always see all the events that are processed, independently of the partitioning key. For more details on SMJs best practices, go to the [Scheduled Metrics Jobs \(SMJs\)](#) page.

The following table summarizes the points to keep in mind when working with metrics:

	Context	Implication
Runtime Partitioning	When data is partitioned in runtime, each node will only see a subset (partition) in real time. Partitioning is needed to process large volumes (rule of thumb: >500 events per second).	Real-time metrics must use: 1. The same field as the partitioning key as the root grouping entity. 2. Or, an entity with its domain within the partitioning key.
Scheduled Metrics Jobs (SMJs)	Metrics with long windows run in batch jobs in runtime. These batches are scheduled to run at specific times to not overload the system.	Re-running the plan with the same data in the Data Science environment will lead to small errors (up to 1% mismatch in value) in a few samples of data (approx. 1% of samples, depending on job configuration and cluster capacity).
Optimizations in SMJs	Because SMJs are special jobs that fetch data from storage and write the results as a key-value pair, there are a few optimizations done that are not possible in the Data Science Platform.	Null values in grouping entities are excluded from the SMJ. This helps reduce failure risk due to skewness. Distinct aggregations are one-shot, meaning that the memory consumption is much less than in runtime.

Data Science Platform📄

Within the data science platform (offline) there are different considerations to take into account. The next table highlights scalability concerns that are relevant when executing plans. In these plans an input data source is processed, metrics are computed on top of it, and the resulting data source is persisted.

	Context	Implication
Spark Jobs & data volume	Metrics done in the Data Science Platform make use of Spark to distribute the workload across an Hadoop cluster. Distributing metrics through a Spark job implies data will be moved around in the cluster in large quantities. Due to the nature of profiles, each value of an entity (e.g. each card or account) will be processed in the same Spark machine (or executor).	Even though a metric might only need 2 or 3 fields to compute, Spark will still move the records with all their columns to the executors.
Spark Jobs & skewness	Very high skews in grouping entities (e.g. 10% of entities represent 90% of the total records), can make Spark Tuning more complex.	Avoid including highly skewed grouping entities in your metrics.

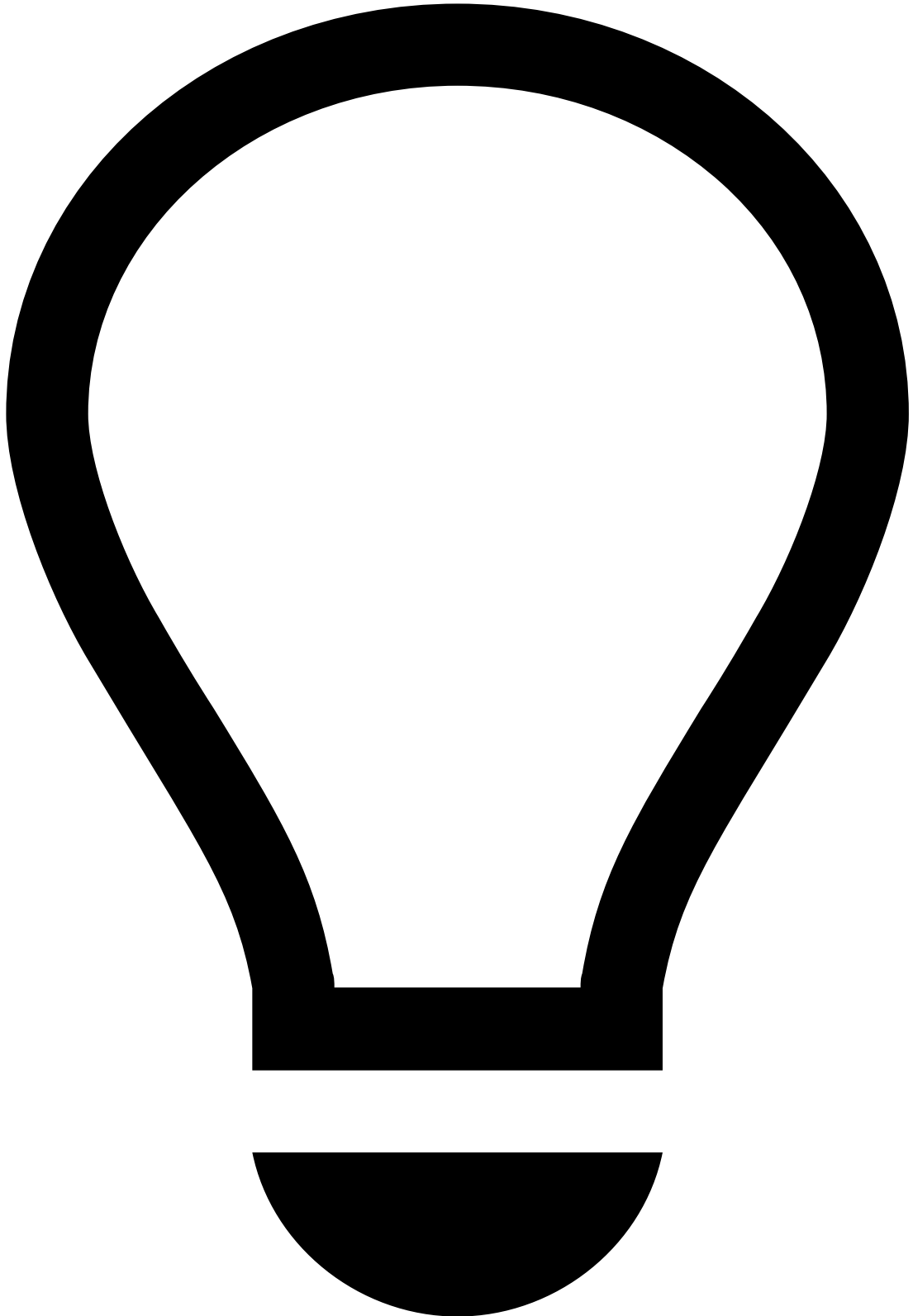
	Context	Implication
		If they cannot be avoided, there are two main avenues to solve it: • Randomize special entities (eg. govt, gmail, ...) • Salt the largest entities: added cost of then joining the features
Window sizes	In the Data Science Platform, all windows are processed through Spark. Even though the SMJs effect is simulated, a large window will still be computed "in-memory" inside the Spark executors.	Very large windows might be feasible in runtime, but not in Data Science Platform.

Learn more about the implications of distributed Spark jobs within the Data Science Platform on the [data partitioning](#) page.

Learn more about the implementation of [SMJs](#).

Dosa

- Use data-aware optimized partitioning.
- Only use **real-time** metrics with grouping entities that are guaranteed to be complete in runtime. (c.f. Runtime partitioning). Example: partitioning key = country, then if users are segregated by country, they can still be used as grouping entities. Partition key = CustomerID, then only customers or other entities that are not shared across customers can be used as grouping entities in real-time metrics.

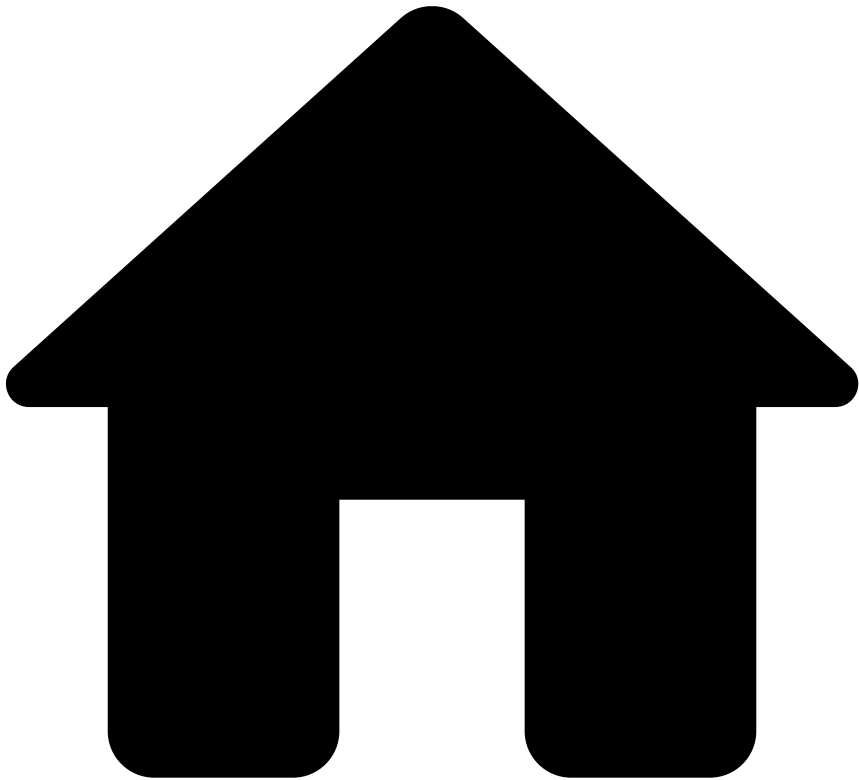


Railgun supports long time windows

These restrictions do not apply when using the [Railgun engine](#).

- Avoid creating metrics with very long time windows to mimic "write-once streaming" features (e.g. "Time since the first time this customer used this device").
- Don't use very large windows. For very large volumes, the rule of thumb is to use 1 month as the maximum window size.

.



- Best Practices
- [Metrics](#)
- Best Practices for Metrics in Railgun

On this page

Best Practices for Metrics in Railgun

Was this page helpful? 

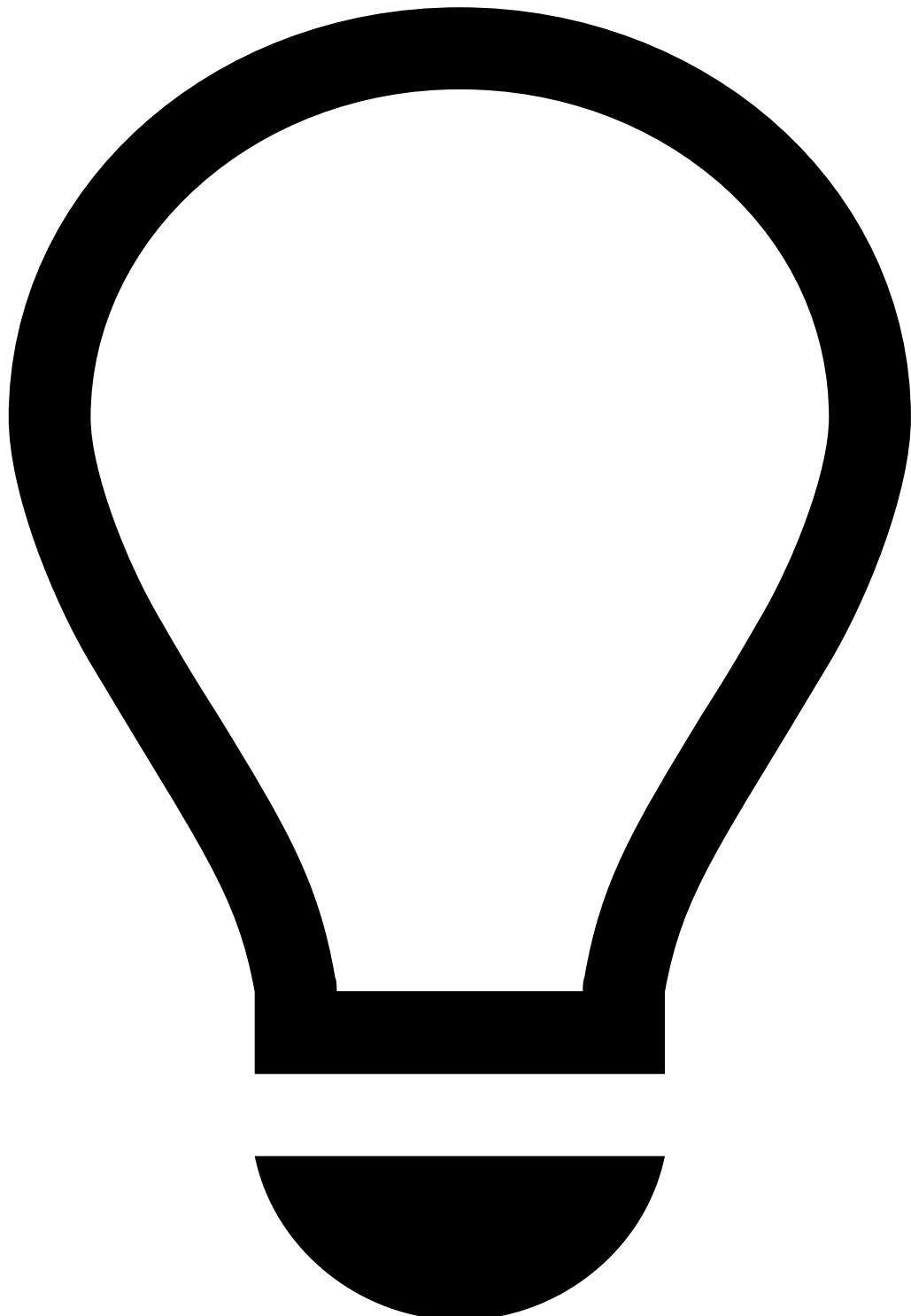
Usage of `min()` and `max()` aggregations

As explained in the [Metrics in Railgun](#) page, using `min()` and `max()` aggregations with field values that are **monotonically increasing or decreasing** (respectively) should be avoided, due to performance issues.

This constraint only applies to windowed metrics ([sliding](#) or [landmark windows](#)). [Non-windowed metrics](#) do not have this limitation, as they don't maintain a deque of expiring values.

If you need to compute `min` or `max` over monotonically changing values (such as timestamps), consider the following alternatives:

1. **Use [non-windowed metrics](#):** If you need to track the minimum or maximum value over the entire lifetime of an entity, non-windowed metrics are the recommended approach. However, since non-windowed metrics, by definition, include all events from the beginning of recorded data, this may not be suitable if you only need to compute aggregations over shorter periods (e.g., days rather than years).
2. **Reduce timestamp cardinality:** If you need to use windowed metrics with timestamps, reduce the cardinality of possible timestamp values by rounding them to a coarser granularity. For example, round timestamps to the nearest second, minute, 10 minutes, hour, or day. This approach has proven effective in practice, as it limits the number of distinct values that need to be tracked in the deque while still maintaining useful temporal precision.



Determining the appropriate cardinality

Depending on the use case, it may be possible to overshoot or undershoot the optimal granularity for timestamp values.

If the right temporal resolution is not immediately obvious based on your needs, reach out to the Feedzai Customer Service for guidance on how to determine the cardinality values for optimal performance.

Number of Metric operators

Similar to PKernel, Railgun's performance improves when the workflow includes fewer metric operators.

Backfill performance

Impact of cascading metrics in backfills

If a workflow has **cascading metrics** (i.e. metrics that are computed based on other metrics), then it is recommended that the upstream metrics (that are used downstream by other metrics) are all grouped in the **same metrics operator**. Additionally, this metrics operator should only contain those upstream metrics.

One important expectation is that the downstream metrics operators will only have a few upstream metrics (e.g. **5 upstream metrics**). That is because of how metrics backfill works and the optimizations that are applied to it:

1. A backfill is the process by which the state of newly created (or updated) metrics is "filled up" using past events that were originally processed before the metric was created.
2. To do so, events are replayed from the Event Storage, passed through a simplified version of the workflow, and sent to Railgun.
3. If there are no cascading metrics, there's an optimization that's applied on the Railgun side that consists in processing these events in **batches**. This means that, instead of updating the metrics being backfilled one event at a time, they are updated (for example) 500 events at a time. Because the processing is done in batch, it's not possible to output the updated value of the metrics for each event while the backfill is running. This is not a problem because, in this case, since there aren't any cascading metrics, the value of the metrics for each event is not needed during the backfill.
4. If cascading metrics are added (say, metrics M1 → M2, where M2 is calculated based on the values from M1), M1 cannot be backfilled with this batch optimization; the value of M1 needs to be updated for each event, in order to backfill M2 as well. In other words, there is a performance hit when cascading metrics are used: M1 now has to be updated one event at a time, even though M2 can still be updated in batches of multiple events. The performance of updating M1 one event at a time depends on how many other metrics are defined in the same Metrics Operator as M1. The more metrics there are in the operator, the lower the potential throughput; e.g., if there are **5 metrics** in the same operator, it may be possible to backfill M1 at around **5000 events/sec**, but if there are **200 metrics** the throughput may drop to around **200 events/sec**. Therefore, using fewer upstream metrics allows improved backfill performance.

Maintaining metric values during automatic backfills

Metrics backfills that are triggered manually will [continue to output values based on the original metric definition](#) (if that version was already available in Railgun when the backfill was initiated) until the backfill is complete.

However, when a backfill is automatically triggered by a metric definition change, the default behavior is for the metric to return the fallback value until the backfill is complete. Still, it is possible to employ a workaround to ensure that metrics in automatically triggered backfills continue outputting the values based on the original definition during the backfill process.

To do so, follow these steps:

1. **Clone the metric:**
Create a duplicate of the metric that requires changes.
2. **Apply changes to the clone:**
Implement the necessary modifications on the cloned metric.
3. **Publish the cloned metric:**
Publish the cloned metric with the applied changes.
4. **Wait for backfill completion:**
Allow the backfill process to complete for the cloned metric.

5. Delete the old metric:

Once the backfill is finished, delete the original metric. Do not yet publish this change; proceed directly to the next step.

6. Rename the new metric:

Rename the cloned metric to match the name of the original metric, and publish the app. This ensures continuity and avoids the need to update downstream dependencies that were using the original metric name.

These guidelines allow for a smoother transition during backfills, by maintaining the integrity of metrics during the backfill process.

Managing the queue of backfill jobs

Ordering backfill jobs by size

Because backfill jobs are added to an ordered queue as soon as changes to metrics are published, and get picked up from the queue according to their publish order (except where [automatic optimization](#) is possible), it is important to manage this queue to ensure that long-running backfills do not block more urgent ones.

Therefore, when there are changes to be done in multiple metrics using different window sizes, it is recommended to first change the small-window metrics, and publish the application, thus triggering the corresponding backfills. Only then the metrics with large windows should be changed and published. This ensures that the smaller metrics can be executed quickly, without having to wait for the longer backfills to finish first.

Another technique that can be used to manage the backfill queue is to [manually trigger backfill jobs](#), since such jobs are prioritized over automatically triggered ones.

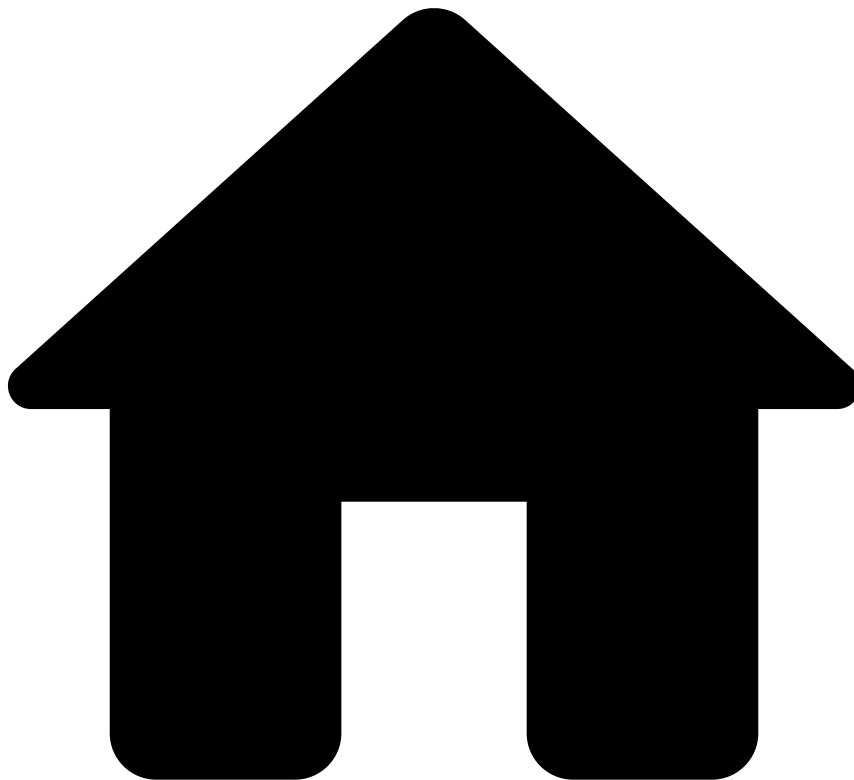
Handling duplicate metric backfills

[Pulse automatically optimizes backfill queues](#) by consolidating the backfill jobs in the queue whenever possible, and skipping backfills for metrics that have already been backfilled, to avoid redundant work that would impact the performance and responsiveness of the application.

However, these optimizations do not apply to metrics that are updated while a backfill for their older version is already running. Therefore, as additional steps to reduce the amount of duplicated work performed in backfills, it is recommended to:

- Avoid making rapid and frequent changes to metrics;
- Manually [cancel backfills](#) whose metrics have been updated in the meantime, and therefore have triggered a new backfill job.

•



- Best Practices
- [Metrics](#)
- Reproducibility Notes

Was this page helpful? 

Reproducibility Notes

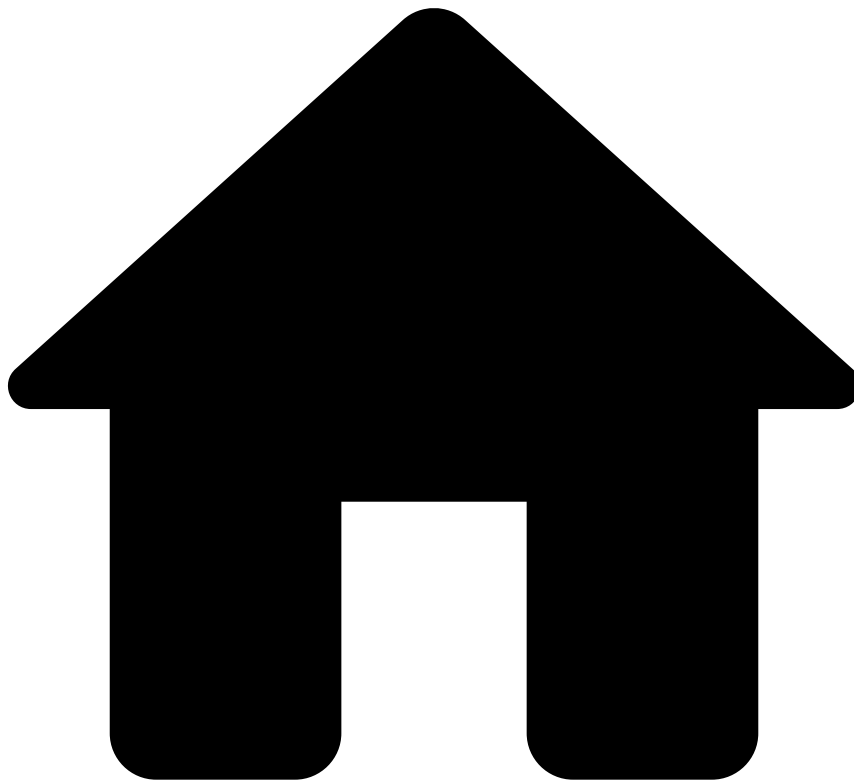
Even though the feature definition from Data Science is exported to Runtime as-is, there are a few reproducibility caveats. These are mostly due to the natural difference between processing historical data versus real time streaming.

When comparing feature values from both environments, be aware of:

- **Repeated timestamps.** If the timestamp precision is seconds, then with 100,000 events per day, you're guaranteed to have different events occurring at "the same time". With a millisecond precision, note that, while unlikely, it's also possible to have events with the same timestamp during peaks (think about >500 TPS). The consequence of this is that some of these simultaneous events might have their features computed in a different order in DS vs Runtime.
- **SMJs running time.** SMJs are "simulated" in Data Science by computing the metrics with the same schedule (CRON expression). However, at runtime the result of the SMJ will only be available **after the metrics are**

computed. In the meantime, some events were already processed through the workflow and were using the result from the previous SMJ. In Data Science, because we are batching these jobs, the metrics are available to all events happening after the schedule.

•



- Best Practices
- [Metrics](#)
- Scheduled Metrics Jobs

Was this page helpful? 

Scheduled Metrics Jobs

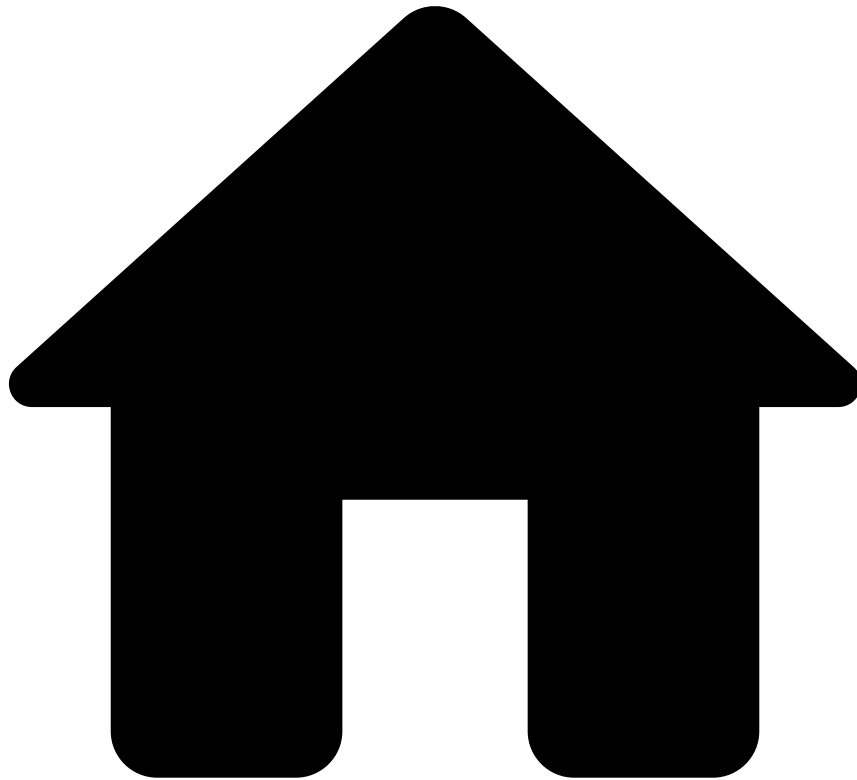
When writing metrics in the Data Science Platform, one must always be aware how these impact computation in runtime. As mentioned before, metrics with long windows should be defined as scheduled. This scheduling will make the aggregation update in a tumbling fashion (i.e., "every X time"), instead of sliding (i.e., "every event").

While in the Data Science Platform the implications of using SMJs are relatively easy to manage, there are important considerations to take into account for **runtime**:

- SMJs are optimized expressions that run as one-shot, i.e., only the minimum needed data is fetched from past events and the result is stored as a map between grouping entities and respective values. These data are stored in the so-called **profile storage**.
- There will be 1 Spark Shuffle per metrics operator inside a Plan. Remember this is the heaviest computation, as it reorganizes data per grouping entity.

- There are a few automatic plan optimizations specific to SMJs in the workflow:
 - operators removed if they don't impact SMJs;
 - operators merged if they have no inter-dependency (also in DS);
 - unused fields are not read from the event storage (remember SMJs in runtime read events pulled from Event Storage, i.e., events that already went through the Workflow).
- Null values in grouping entities are automatically filtered out. This leads to less data being used to compute the SMJ *and* it usually reduces skewness of grouping entities.
- There is an option to automatically select which grouping entities are used for the shuffle. Having this option enabled will reduce the number of shuffles, thus reducing the running time of SMJs. However, it does not take into account the grouping entity cardinality. If you are using low-cardinality entities in many aggregations (independently of being the first, second, or third grouping entity of the aggregation), disable this option.
- If you have SMJs that are dependent on one another (i.e. two subsequent Metrics blocks inside a Plan), **you must enable cascading scheduled metrics** in the workflow. If you don't, due to the one-shot nature of SMJs, the downstream SMJ won't be able to fetch the output of the upstream SMJ.

•



- [Creating a Data Science project](#)
- [Cleaning and loading your data](#)
- Cleaning up the data

On this page

Cleaning up the data

Was this page helpful? 

You'll often receive raw data with problems, limitations, inconsistencies and all-around quality issues. Once you have matched the raw data to your internal schema, you must make sure that you only work with relevant and high-quality data.

To clean up your data, and address data quality issues, you need to design and create a [Plan](#). This generates a new and clean [Data Source](#).

While this task is very similar in nature to the previous task in this guide â [Validating the schema](#) â its purpose is not the same. This task handles data quality from the perspective of the value and correctness of information contained in the dataset, while the previous one rather focuses on fixing the structure and formatting of your dataset.

How to

For example, to clean up your data and remove negative values from numerical fields, such as the *TransactionAmount*, execute the following actions:

1. In the **Data Science** tab , add a new Plan.
2. Set the Plan as [Non-Deployable](#).
3. Design your data cleansing Plan as displayed in the following image.

Configure the elements as follows:

1. Edit the Source element and specify the Data Source. For example, *Transactions*.
2. Edit the Map element to correct the negative values, as follows:

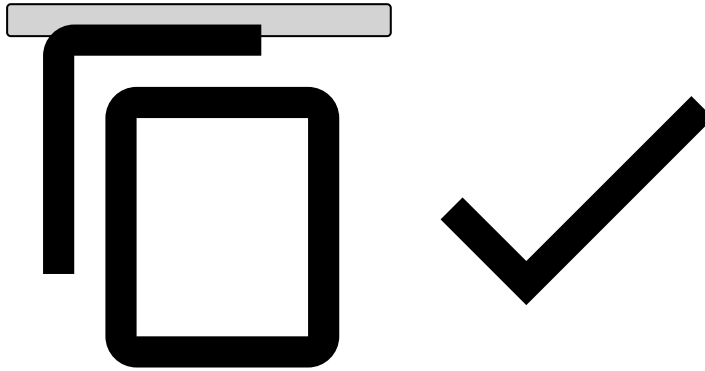


Clipboard

You can also copy Operators to the clipboard in order to paste them in other Plans. This helps you save time by allowing you to create plans more efficiently without the need of specifying all their fields each time you create a new plan, letting you use the operators you have already configured.

1. Select the **Fields to Update** tab and specify which is the field to be cleaned. For example, *TransactionAmount*.
2. Write the update expression for the field. For example:

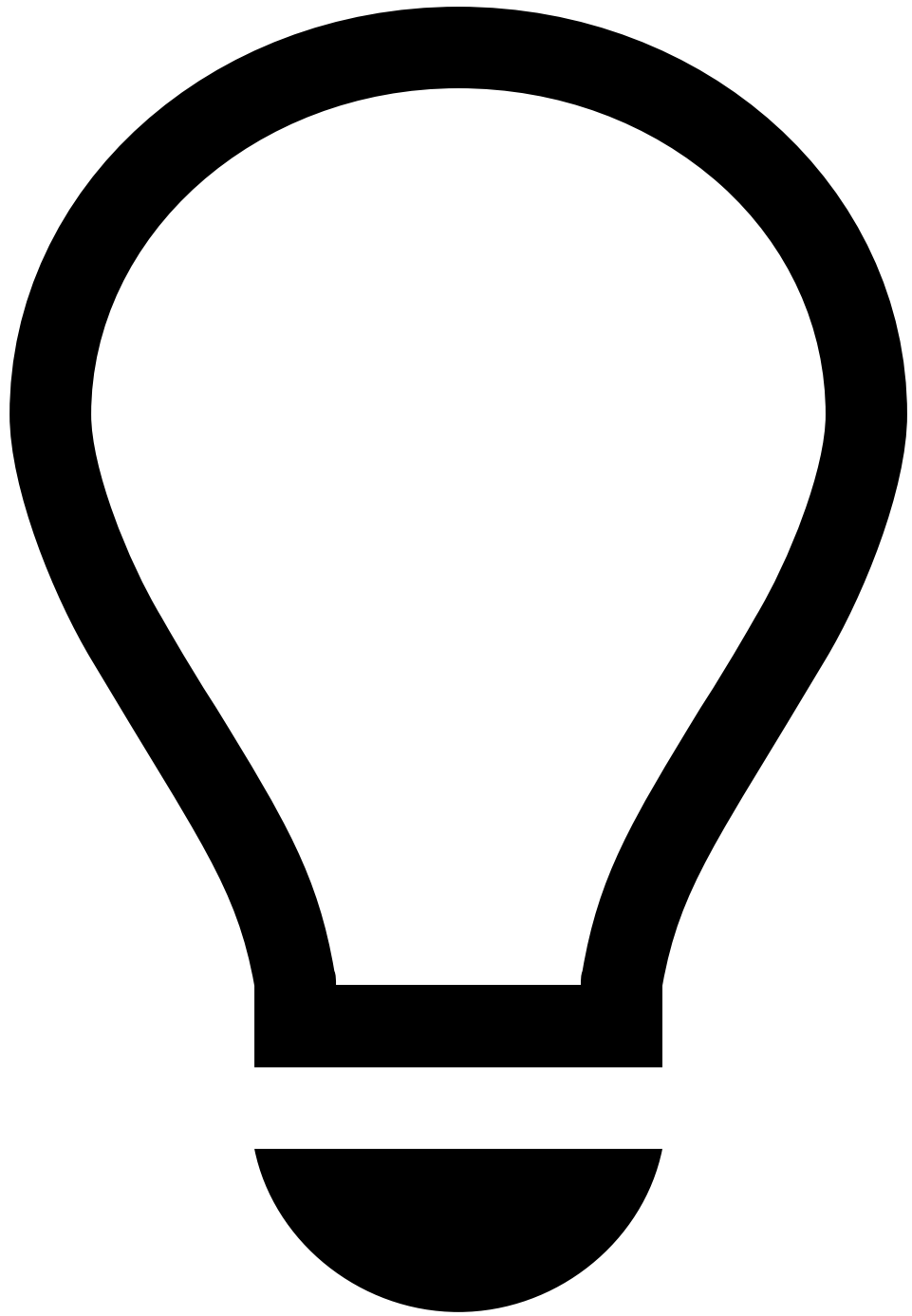
```
if (TransactionAmount < 0) then 0 else TransactionAmount
```



3. Add the field to the list of fields to be updated.

Finally:

1. Run the plan to execute all actions on the original Data Source and generate an improved one.
 1. Define the **Data Source with ID 1** as the original, lower quality, Data Source. For example, *Transactions*.
 2. Configure the **Data Sink with ID 3** as the new and clean Data Source. For example, *Cleaned Transactions*.



Job Profiles

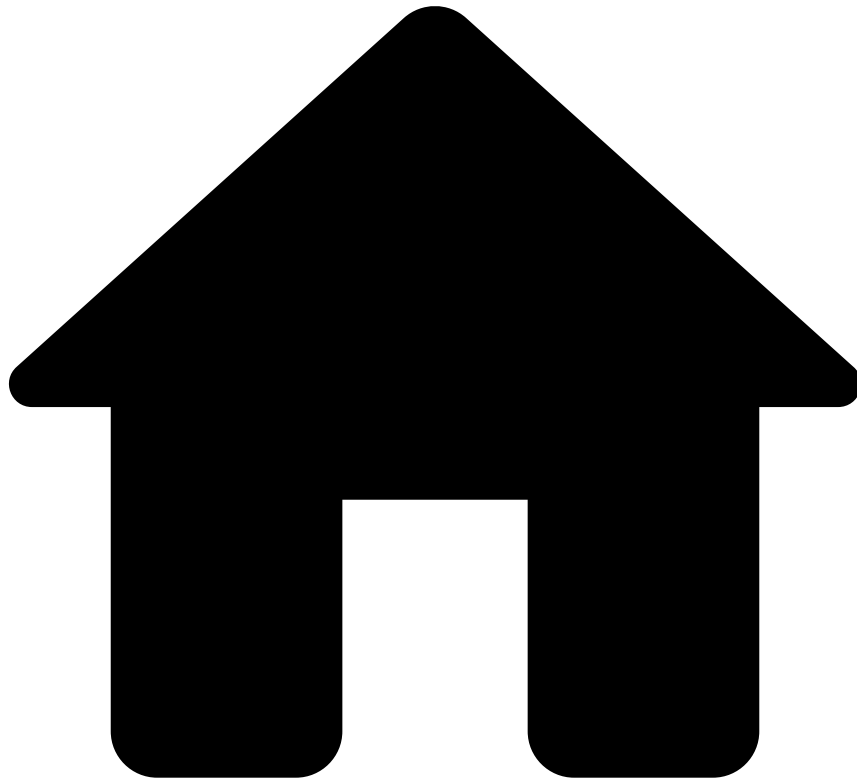
You can select a Job Profile by expanding the Advanced Options menu. This is helpful because Plans are executed by Spark Jobs, and by using this option you can select a profile with a set of Spark properties that best fulfil the needs of the task that you want to execute. Learn more about [Job Profiles](#) in Pulse, how to [create](#), [select](#), and [edit their properties on-demand](#).

3. Execute the Plan to generate a new Data Source, in Pulse, named *Cleaned Transactions*. Pulse also saves an output file with the transformed data to your filesystem.
2. [Analyze](#) the *Cleaned Transactions* Data Source to confirm that there are no negative values for the *TransactionAmount* field.

Next Steps

Once you have a clean and valid dataset, you are ready to [design advanced Plans](#), [analyze your data](#) and [generate features for your models](#).

•



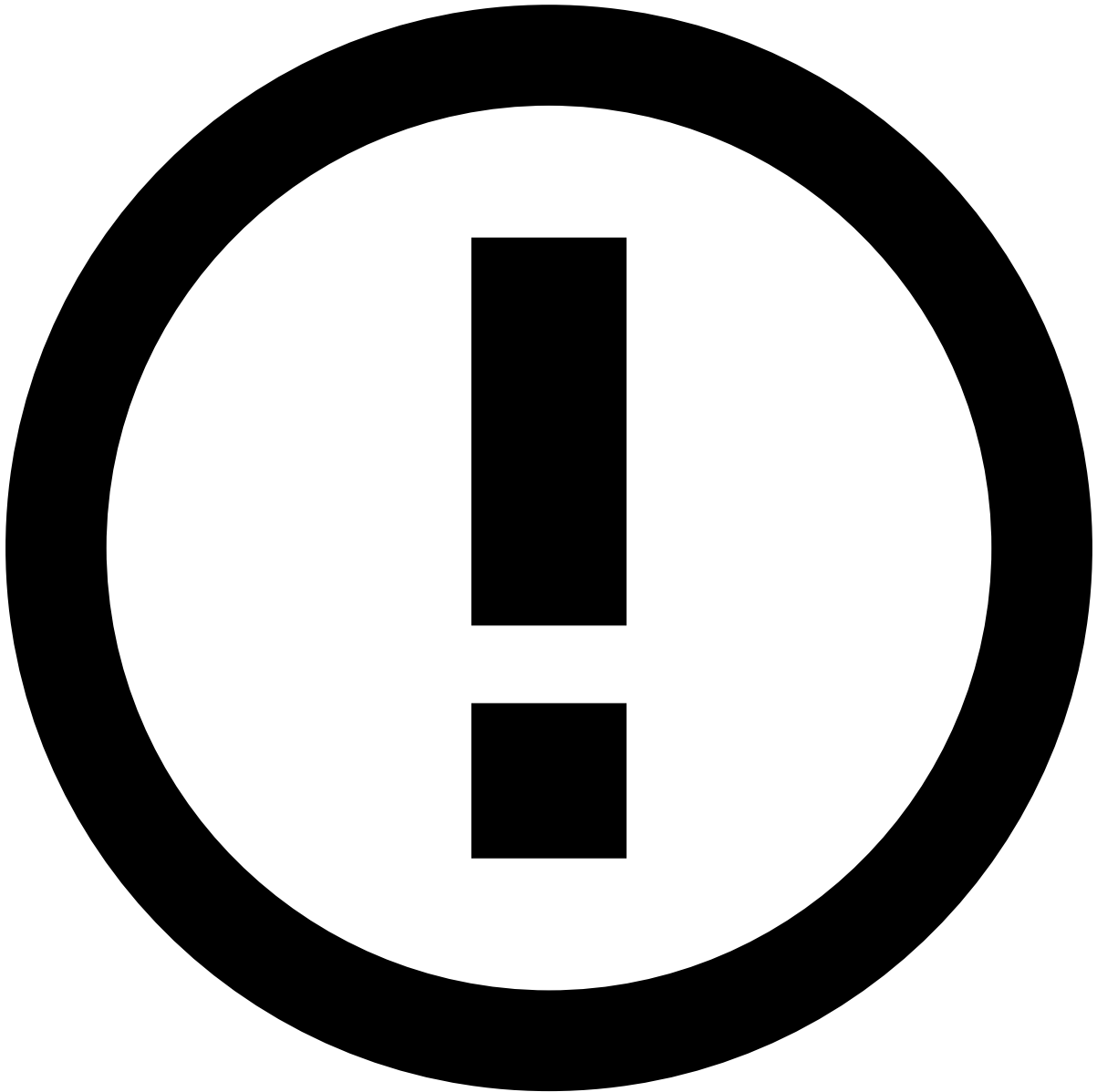
- [Creating a Data Science project](#)
- [Cleaning and loading your data](#)
- Importing your dataset into Pulse

On this page

Importing your dataset into Pulse

Was this page helpful? 

This article describes how to make data that is stored in a CSV or Parquet file available in Pulse as a [Data Source](#).



info

Alternatively, you can perform this action from the JupyterLab user interface. Standard notebooks are available within [JupyterLab](#) – in particular the Data Loading Standard Notebook, which demonstrates how to successfully load data into Pulse through Feedzai's Data Science API.

How to

To import a dataset into your Pulse application and create a [Data Source](#), execute the following steps:

1. Ensure that the data file contains valid values of the [Pulse data types](#).
2. In the Data Science tab, add a new Data Source.
3. Select the **URL Path** from where to import the data. The data file can either be stored locally, or in a Hadoop Distributed File System (HDFS). Examples for the URL Path: `/opt/datasets/transactions.csv` , or `file:///opt/datasets/transactions.parquet` .

4. In the **Advanced Options**, you can configure who has access to the Data Source.
5. Check the **Data Source Preview** to see how your data will be imported.

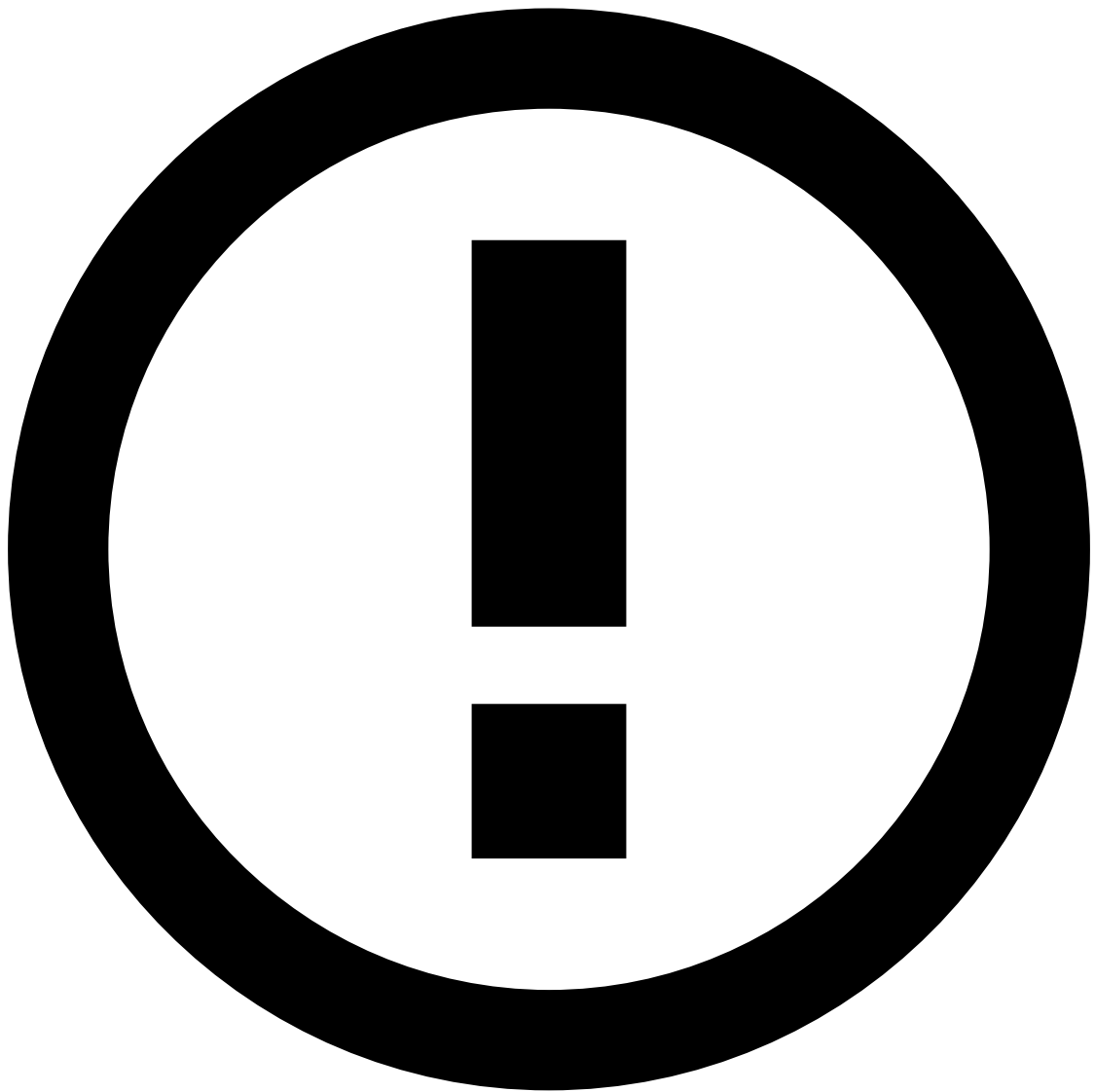


CSV-only

For CSV files, you can adjust the **Includes Header**, **Delimiter**, **Quotes**, **Line Separator** and [Missing Values](#) fields to define how Pulse should interpret and import the raw data from your file.

The Parquet format is more strictly defined, so these manual adjustments are not required.

6. Advance to the **Data Source Characterization** step, where you can review the **Type** and **Configuration** that Pulse assigned to the fields of the data source.



CSV-only

For CSV files, you can adjust the inferred characterization for the data fields. See the [Inferred types and values for CSV files](#) section for details.

Parquet files contain explicit type specifications, which are [mapped directly to Pulse types](#) and are not modifiable.

7. Select **Create Data Source** to make your dataset available in Pulse.

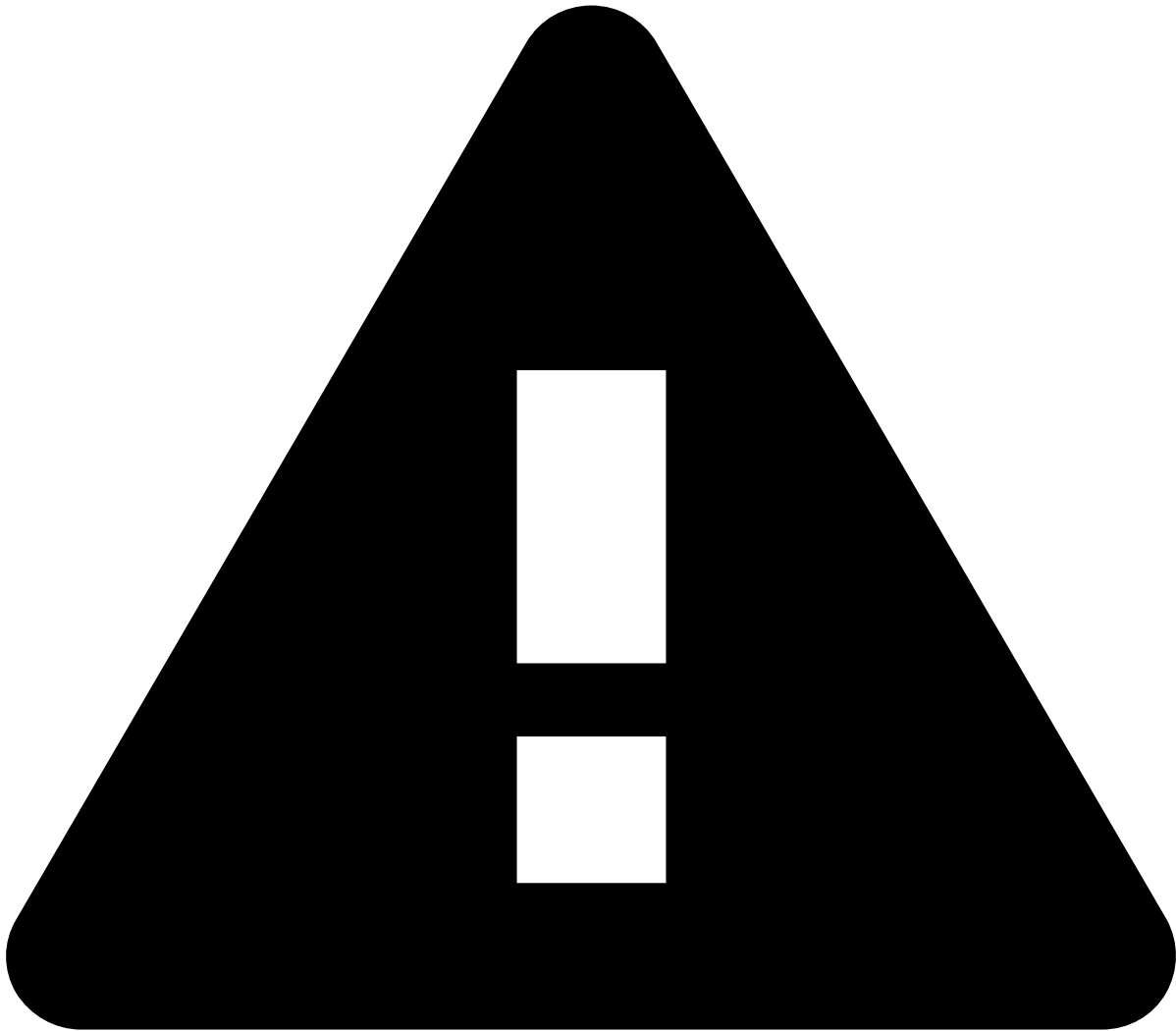
Inferred types and values for CSV files

CSV files don't support indicating the type of each field, which is required information for the Pulse data science loop.

To accelerate the Data Source creation process, Pulse automatically looks at the first few thousand rows of your dataset file and infers what is the data type and possible values for each field.

For example, Pulse can suggest that:

- A *Timestamp* field should be of type *Timestamp*, in milliseconds.
- A *code* field is *Categorical* and the possible categories are A, B, and C.



caution

Pulse infers the values for *Categorical* fields based on the observed occurrences in the first few thousand rows of your dataset. However, those rows might not contain all possible values. If there are other possible values, you must add them manually.

When importing a CSV file, you can verify the and change the data types for the fields, and configure their possible values:

Learn more about [data types](#).

Fixing invalid instances[ⓘ]

When importing a certain dataset for the first time, usually there are records that can not be imported. These failed records or data rows are also called [invalid instances](#). The Pulse UI shows the number of invalid instances only after you run a [Data Source analysis](#), using the Analyse button.

To get information about the invalid instances during this import, use the invalid instances button to display the list of fields that had values which couldn't be imported.

Hover over the information icon to get more information about what is wrong with the data field values.

To fix invalid instances, after investigating their cause in the UI, you can combine the following two approaches:

- Change the data: preprocess the dataset externally (e.g. using Python), to transform invalid values into valid ones. This approach helps when in the same data field, in different records there are different types or formats.
- Change the schema: [duplicate the data source](#), and change the data type, or the configuration of the data type. This can help if there is only a categorical value missing, or if you picked the wrong setting when defining a data type.

Duplicating a Data Source

Since data schemas in Data Sources are [immutable](#), if you need to make changes to a Data Source, you must create a new one. However, if you need to fix or adjust it without having to restart from scratch, you can duplicate an existing Data Source as follows:

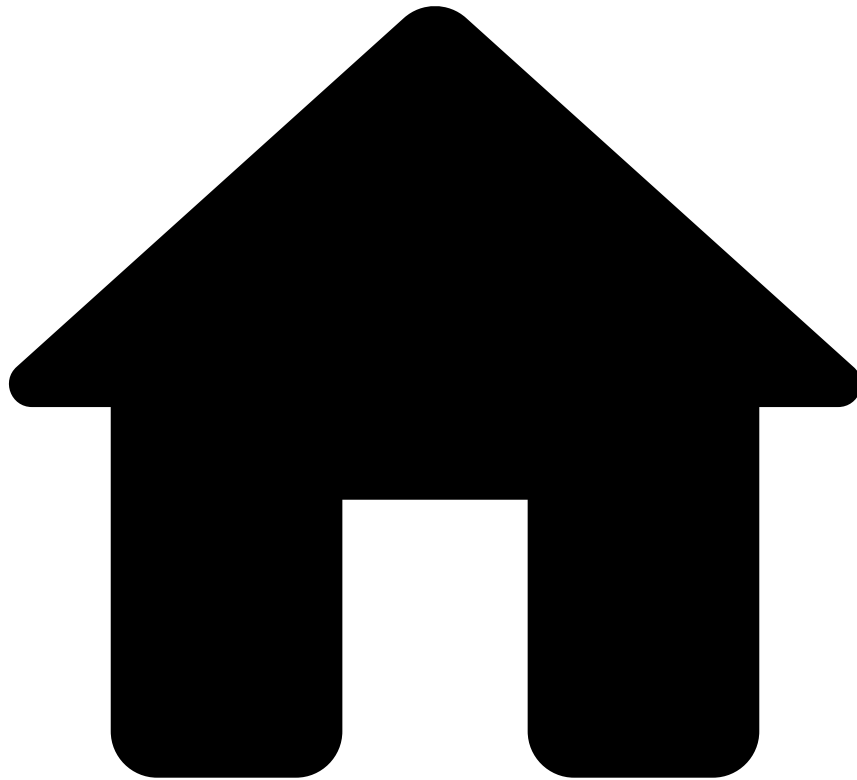
1. In the Data Science tab, Duplicate and Change an existing Data Source.
2. Specify the path from where to import the data. The data file can either be stored locally, for example, `/opt/datascience/transactions.csv` or in HDFS.
3. Preview how your data will be imported. The Delimiter, Quotes, Line Separator and Missing Values will be exactly the same as the original Data Source.
4. Change the Data Source Characterization, namely editing the [Type and Configuration](#) of any field.
5. Create a new Data Source.

Next steps

Once you have your raw data configured in Pulse, follow up by making sure that your original dataset is correctly [converted into a valid data schema](#).

Once you have a Data Source, you can then start analyzing and transforming your data with [Plans](#) and building [Models](#) and [Rules](#).

•



- [Creating a Data Science project](#)
- [Cleaning and loading your data](#)
- Validating the data schema

On this page

Validating the data schema

Was this page helpful? 

Your project's data schema often does not match the data schema you receive from the client. Pulse lets you convert each field in the raw data and define how it should be converted to match your project's data schema.

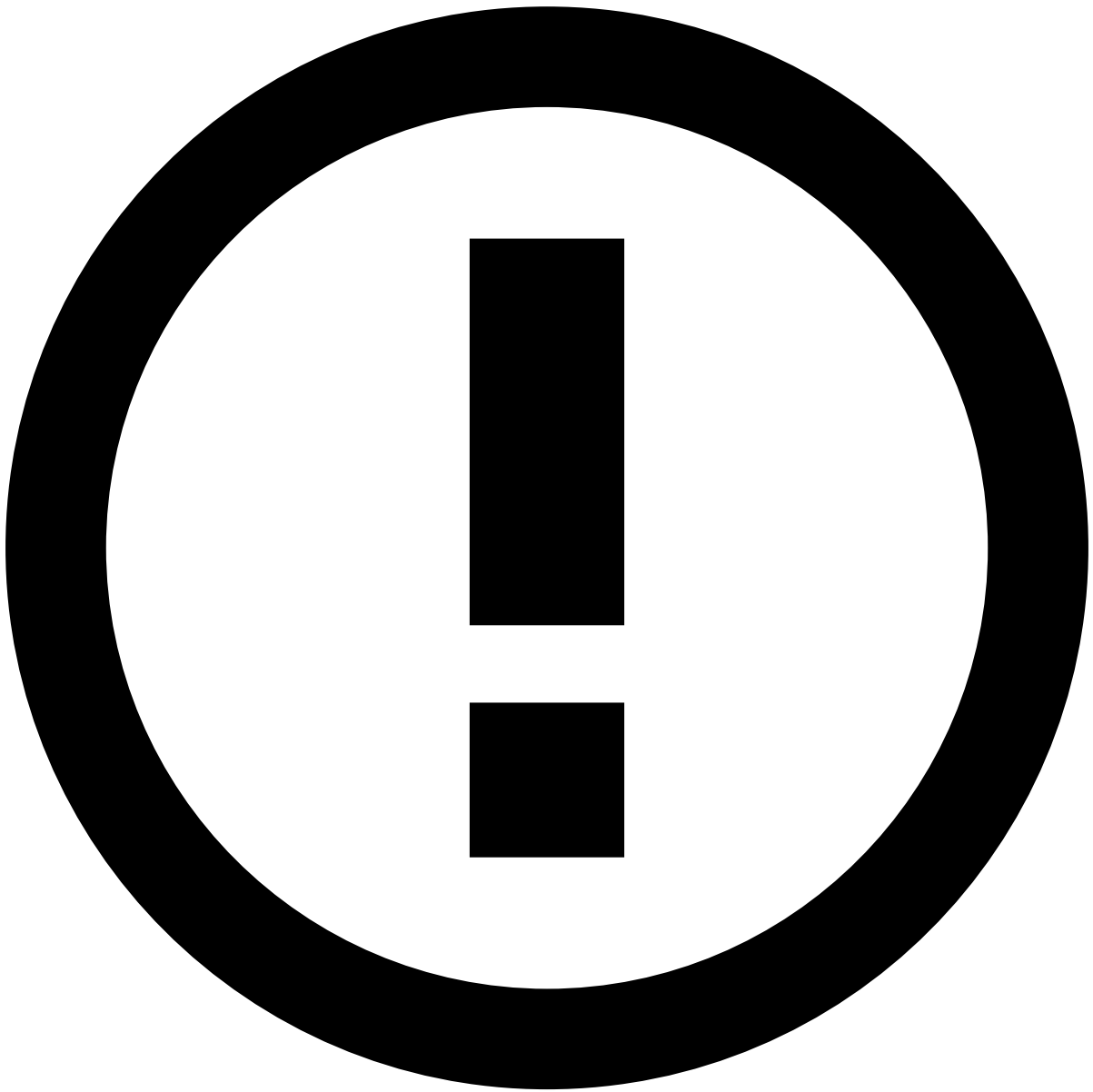
You must look into the data schema of the [Data Source](#) containing your raw data, and define how to convert each field to a Data Source that makes sense in your Data Science project.

This task represents solving a common data quality issue, focusing on fixing the structure and formatting of your dataset. Similarly, the next task in this guide [Cleaning up the data](#) handles data quality from the perspective of the value and correctness of information contained in the dataset.

How to

To convert the original dataset into the internal schema of your Data Science project, compose a [Plan](#) as follows. For example, to perform a direct conversion from a raw data field to an identical field with a different name:

1. In the **Data Science** tab , add a new Plan.
2. Set the Plan to [Non-Deployable](#).
3. Design your schema validation Plan as displayed in the following image:
4. Configure the elements as follows:
 1. Edit the **Source** element and specify the **Origin Schema**. For example, select the *Transactions* Data Source to copy and use its data schema in the Plan.
 2. Edit the **Map** element to:
 1. Add a field and set the **Output Field Name** as the target name for the field in your project, for example, *TransactionAmount*. Set the name and type from the original raw data field as the **Expression** (for example, *trAmt*) and **Type**. Confirm that you added the new field.
 2. Remove the original raw data field from the Data Source.
 3. Repeat step 4. b. to convert each data field, using the appropriate **Expression** and **Type** according to your schema validation requirements.
5. Run the plan to execute all actions on the original Data Source and generate an improved one.
 1. Define the **Data Source with ID 1** as the original Data Source. For example, *Transactions*.
 2. Configure the **Data Sink with ID 3** as the new and validated Data Source. For example, *Validated Schema*.
 3. Use the Run button to run the Plan and to generate a new Data Source in Pulse, named *Validated Schema*. Pulse also saves an output file with the transformed data to your filesystem.
6. [Analyze](#) the Validated Schema Data Source to confirm that the raw data was converted to the target schema for your Data Science project.



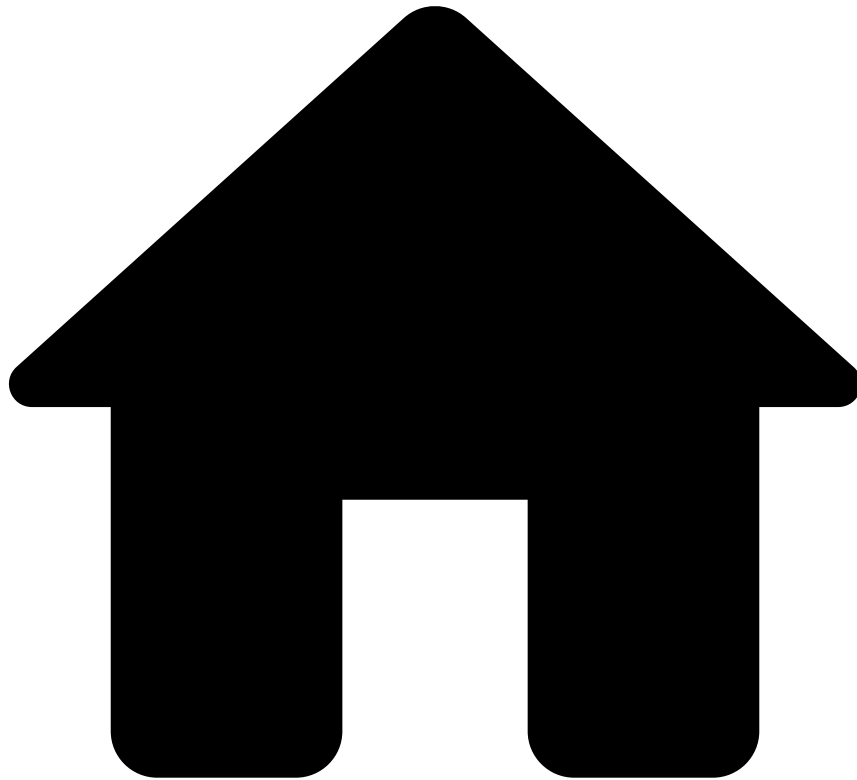
No mapping

When there is no mapping from the raw data to your target schema, you can use the Map operator in a Plan to add a new output field with an empty Expression. Run the Plan to fill in this field with the Missing Value you configured for the Sink.

Next steps

Once you have mapped the raw data to your project's schema, you may need to address quality issues and [clean up your data](#).

•



- [Creating a Data Science project](#)
- [Processing your data](#)
- Analyzing and sampling your data

Analyzing and sampling your data

Was this page helpful? 

Pulse provides a toolset to help you in uncovering interesting data insights, which you can explore as features in your models. It also helps you in balancing your data to make it easier to manipulate and understand.

With Pulse you can perform the following operations over your data in your Data Science project:

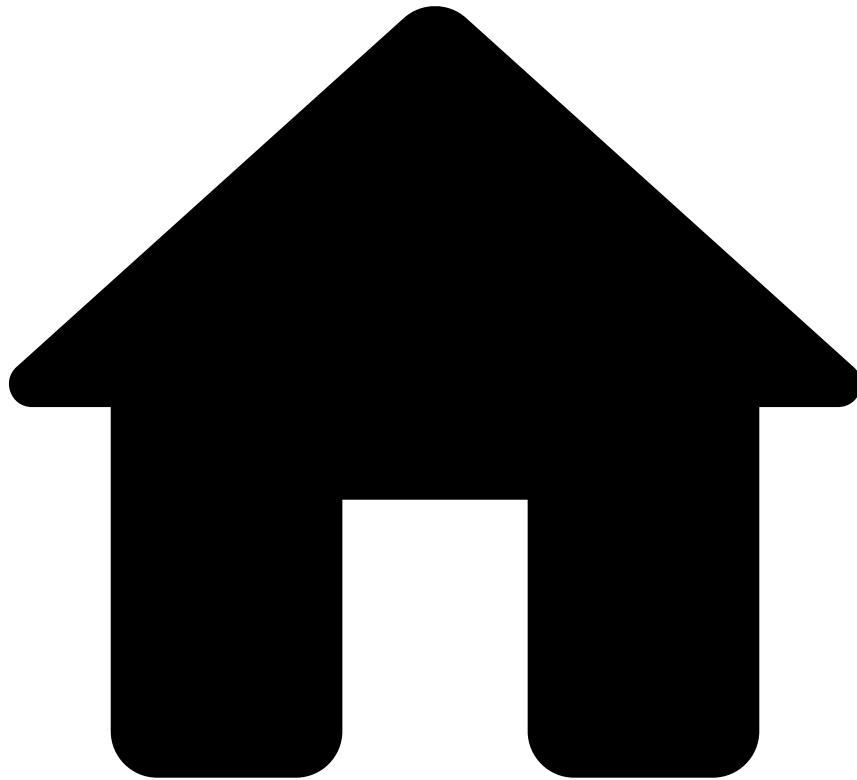
1. [Identifying patterns](#)

Run statistical analyses on your data to reveal patterns and gain some initial insight on both the contents and quality of your dataset, without writing any code. Pulse generates histograms, minimum, maximum, median, count and percentiles over each specific data field of the dataset and allows you to further explore these analytics by breaking them down by other fields of the dataset.

2. [Creating a stratified sample](#)

Learn how to design a [Plan](#) to generate a stratified sample out of the original data and start your [Feature Engineering](#) work from a stable and more balanced dataset.

•



- [Creating a Data Science project](#)
- [Testing your work in a Sandbox environment](#)
- Creating a Sandbox Evaluation

On this page

Creating a Sandbox Evaluation

Was this page helpful? 

Why

By creating a Sandbox Evaluation, you will be able to test your Rule and Model projects as in a production environment. After setting up the rules and models you have specified, you will be able to test their performance over your [Data Sources](#) as if they were in Production, before deploying them.

You can evaluate your rules and models, therefore being able to identify potential problems before deploying them and predict the performance of your system.

How to

To create a Sandbox Evaluation, execute the following actions:

1. In the **Data Science** tab,  add a Sandbox Evaluation.
2. Specify the **Data Source**, i.e. the data that will be tested in your Sandbox Evaluation. By choosing the **Partitioning Fields**, the Sandbox evaluates the performance of rules and models as it would do in workflows, once the solution is deployed. For example, select **Card** as a partition field: When the Sandbox Evaluation is created, all transactions from that card will have been processed together. Therefore, the node in charge of processing it will not have visibility of the transactions not associated to that card.
3. If the resources you are configuring require time-based metrics, specify the field of the **Data Source** that is to be used as the timestamp field.
4. Select the **Outcome Variable**, i.e. the ground truth: the value against which the evaluation will process its results.
5. Choose the **Resources** and [Outcome Combinator](#). You can select as resources the **Models** and **Rules projects** you have created, the **Resource Parent** being such project.



Bear in mind that you can reorder the resources so that your sandbox evaluations simulate a workflow as close as possible to the one that will be later deployed in production. By reordering the resources, the Sandbox Evaluation will simulate exactly the order in which events are processed.

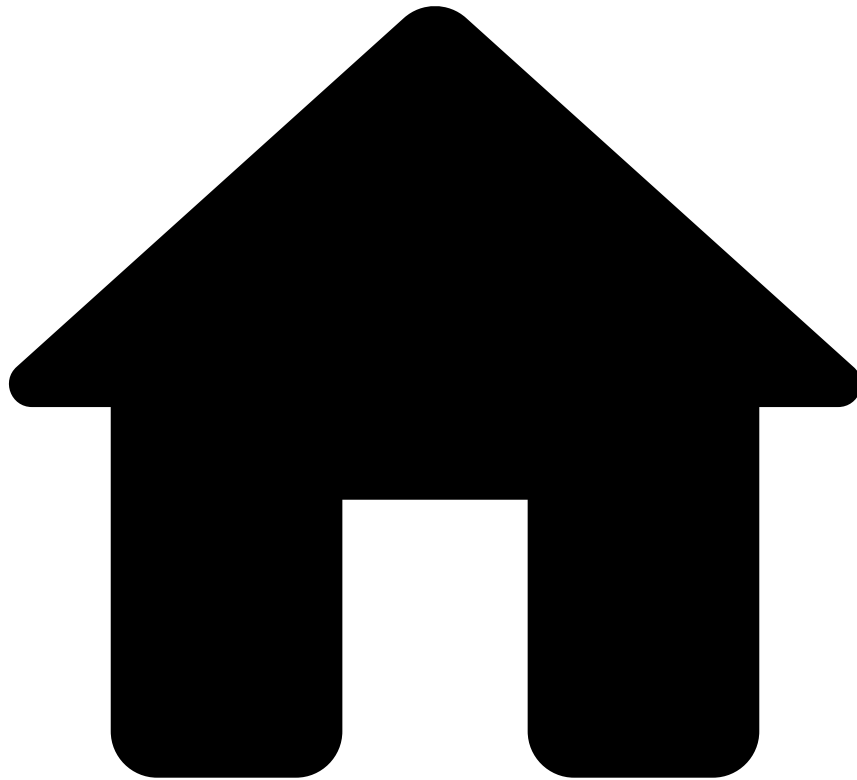
6. Choose your **Reporting** configuration and create the evaluation. In this step, you will choose how the results of the evaluation will be presented. By selecting no **Additional Cost** and no **Breakdowns**, you will see all metrics in general. If you add an additional cost field, the system will collect all those metrics that are also weighed by a data source field (e.g. making rows with higher values of the field impact the metrics more).
7. You can also choose **Breakdowns** to analyze metrics according to the possible values of a categorical field in the data source. Breakdowns allow users to understand the impact of Rules and Models as segmented by the particular values of the breakdown fields that are selected, similarly to rules and models evaluations.

Next Steps

After creating your Sandbox Evaluation, you will be able to [Evaluate your Sandbox Results](#). When the evaluation of the results is satisfactory, you will be able to move on to the [deployment of your work into Production](#).

Check also [how Pulse projects work](#) for some background to understand what is involved at this stage.

•



- [Creating a Data Science project](#)
- [Testing your work in a Sandbox environment](#)
- Evaluating your Sandbox Results

On this page

Evaluating your Sandbox Results

Was this page helpful? 

Why

Evaluating your Sandbox results against a test dataset is crucial in understanding how it is performing, and how close you are to reaching the objectives of your Data Science project.

Now that you have learned how to [create a Sandbox Evaluation](#), you will need to generate a Sandbox Evaluation over a trained set of [Rules](#) and [Models](#) and analyze the evaluation results.

How to

To analyze the **Sandbox Evaluation** results, execute the following actions:

1. In the **Data Science** tab, select a **Sandbox Evaluation** from the **Sandbox Evaluations** list.
2. Select an existing **Sandbox Evaluation**.

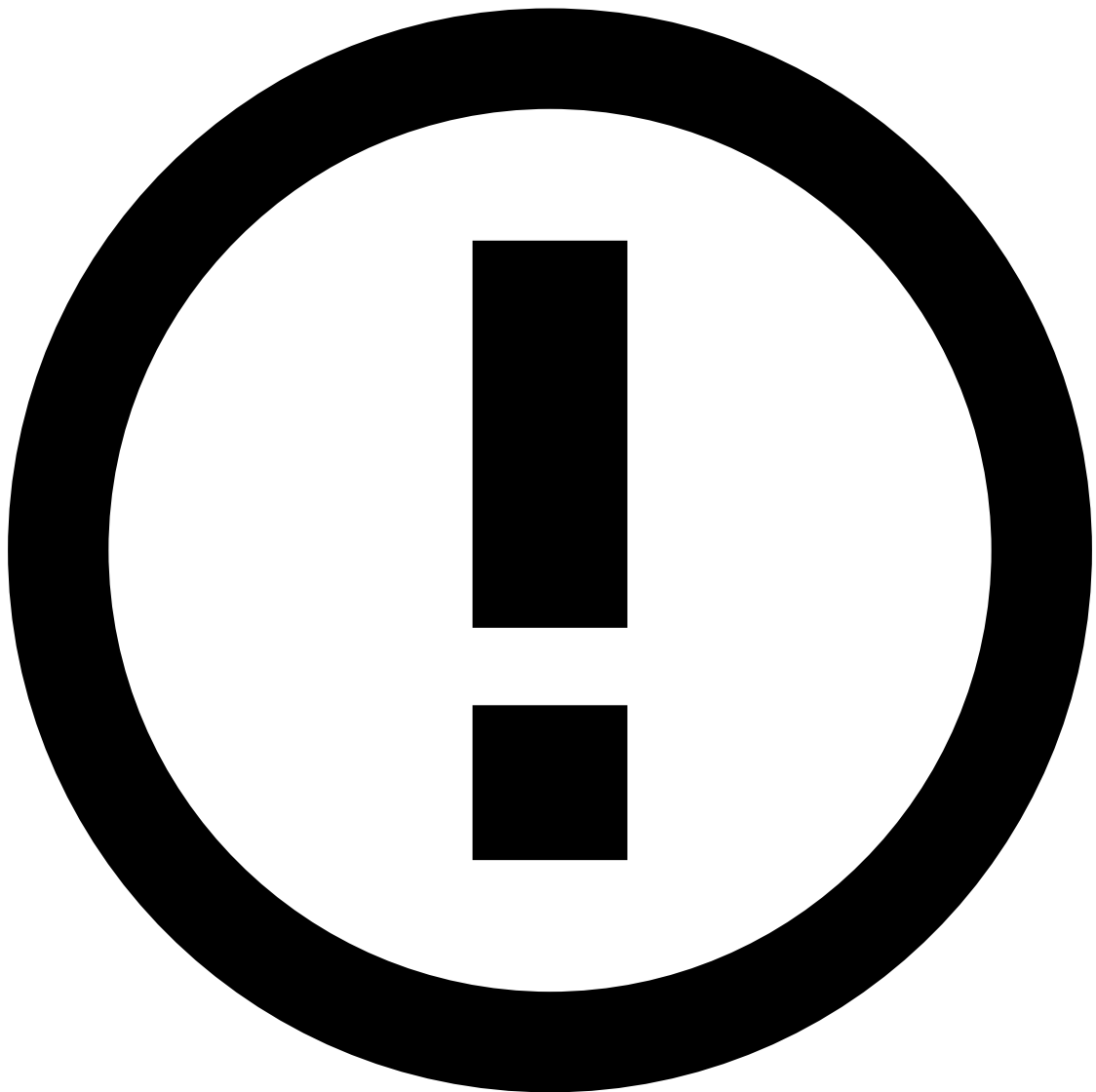
Results by Score

The **Results by Score** focus on the field of the **Outcome Score** and the real final label. Based on that particular field, this allows you to understand the performance of your **Sandbox Evaluation**.

Therefore, you can understand how the system behaves independently of the outcome decision that was specified on the outcome decision field. For example, it will help you decide whether to change the score threshold for the final decision.

In the **Results by Score** tab, you can perform the following actions:

1. Analyze the **ROC Curve** for the selected evaluation.



Hover the mouse pointer over the plot line to inspect the exact values for the **Recall** and **FPR** metrics and the respective **Threshold**.



Cost-based Metrics

If you define a **Cost Field**, then the **ROC Curve** will also plot the values for *Cost-Recall* vs. *Cost-FPR*.

2. Configure the **X/Y Plot** to display a curve for any **X-Axis** and **Y-Axis**.



Exact values

Hover the mouse pointer over the plot line to inspect the exact values for the **X-Axis** and **Y-Axis** metrics and the respective **Threshold**.

3. During the creation of a **Sandbox Evaluation**, you can select the breakdown for the metrics. Therefore, here you can see all the values for the metrics of the segmentation and its combinations. You can also change the threshold.
4. Use the **View ROC curve** option to see which of the breakdown metrics can be seen in the ROC curve.
5. You can also see the metrics per service for all the resources the **Sandbox Evaluation** has. This is useful for understanding the performance computation times of the resources analyzed in the sandbox. It helps you to grasp the performance gains or losses you will experience in your live Workflow and evaluate the tradeoffs. For instance, you can identify whether a Rules project takes too long (in the **Ind. Avg.**) whilst not adding much to the performance (by analyzing **Recall** and **FPR** metrics), therefore allowing you to consider whether to keep it.
6. Export the results to a CSV file, if desired.

Results by Outcome

By analyzing the **Results by Outcome**, you can understand how the system behaves regarding the outcome decision that was specified on the outcome decision field. It works similarly to the **Results by Score**, but it does not provide a movable threshold. You can learn more about the performance of your Sandbox evaluation by analyzing these results:

1. Select the outcome values to be shown either if `True` or `False`.
2. You can export the results to a CSV, both **Per Breakdown** and **Per Services**

Configuration

In the **Configuration tab**, you can see all the information regarding the **Sandbox Evaluation** you are seeing.

It contains all the information chosen in the creation of the Evaluation:

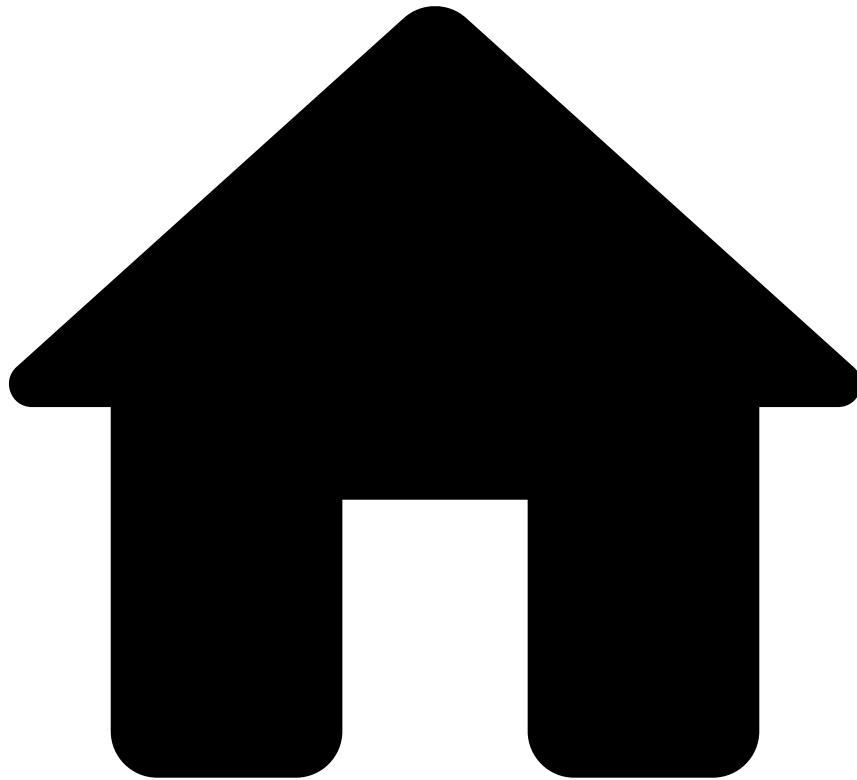
- Data Source
- Segments
- Resources to Analyze
- [Outcome Combinator](#)
- Additional Cost Field
- Output Result Name and Directory Path
- Time when the last update was made and who made it

Learn More

You can learn more about the different concepts related to Sandbox Evaluations in the following pages:

- [Outcome Combinators](#): Learn more about what they are and what purposes do they serve.
- [Sandbox Evaluations](#): Have a complete grasp on how Sandbox Evaluations are useful and all the details they comprise.
- [Evaluating a Model](#): Learn more about different ways of testing your system by evaluating models.

•



- [Creating a Data Science project](#)
- [Tuning Job executions with Job Profiles](#)
- Creating a Job Profile

On this page

Creating a Job Profile

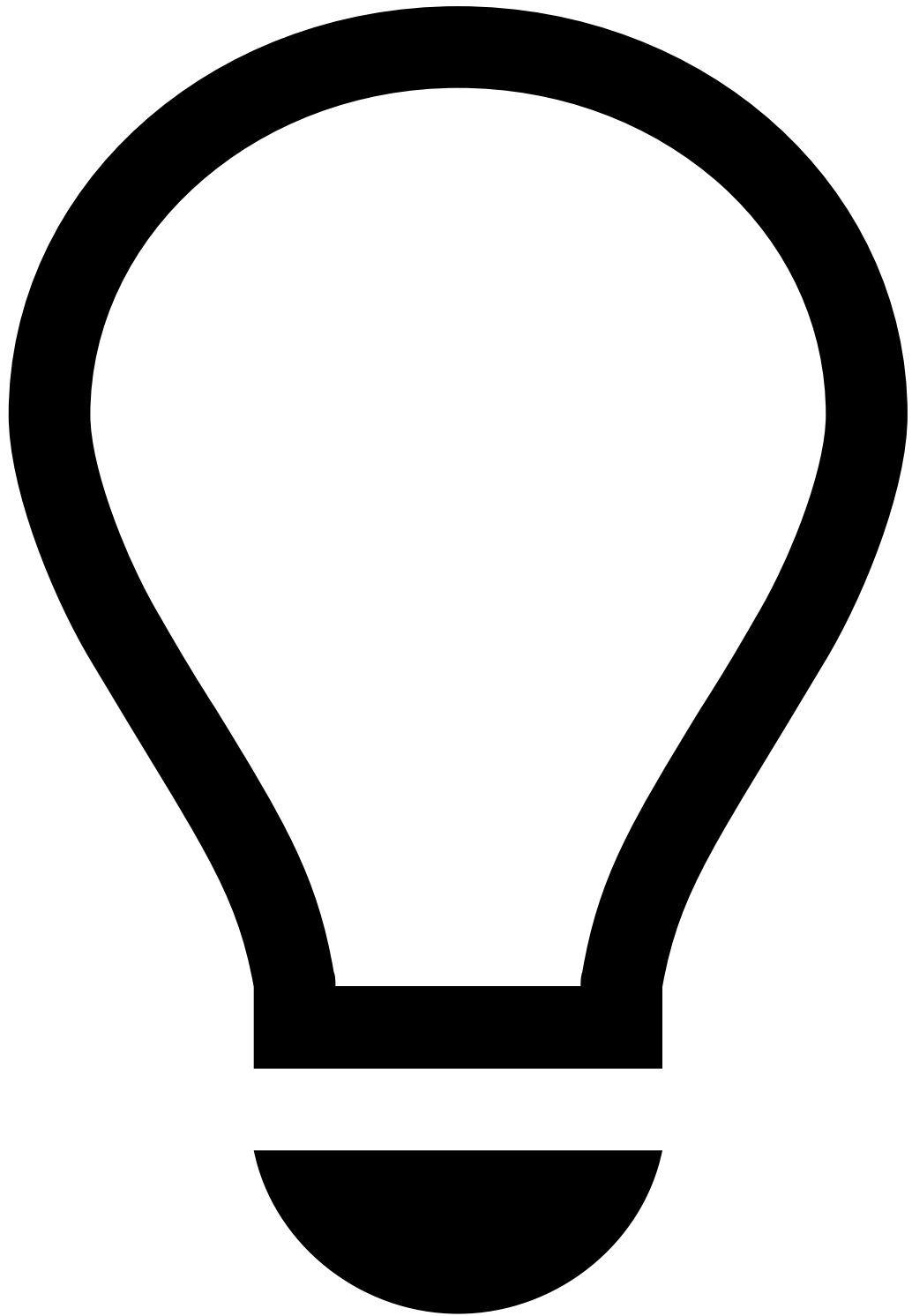
Was this page helpful? 

Pulse allows you to create [job profiles](#) that can be used to apply a set of property configurations when running jobs. This page describes how to create a job profile.

How to

To create a new job profile, access your application and execute the following actions:

1. Select the Jobs tab to have access to the **Job Configuration Profiles** list.



tip

By default, the **Jobs** Jobs tab is collapsed under the **Advanced Tabs**. Use the button to extend this section.

2. Select the Add button to configure a new job profile.



New Job Configuration Profile wizard

When you create a new job profile, the Spark properties that are displayed in the default profile (Pulse Properties Profile or PPP) are automatically copied to this new profile as a base configuration. You can edit these properties and add new ones. However, you cannot delete base properties using the Pulse UI. If you want to eliminate base properties, you must delete them directly from the pre-configured source (e.g. the *pulse.properties* file). By doing so, the changes are propagated to all the job profiles.

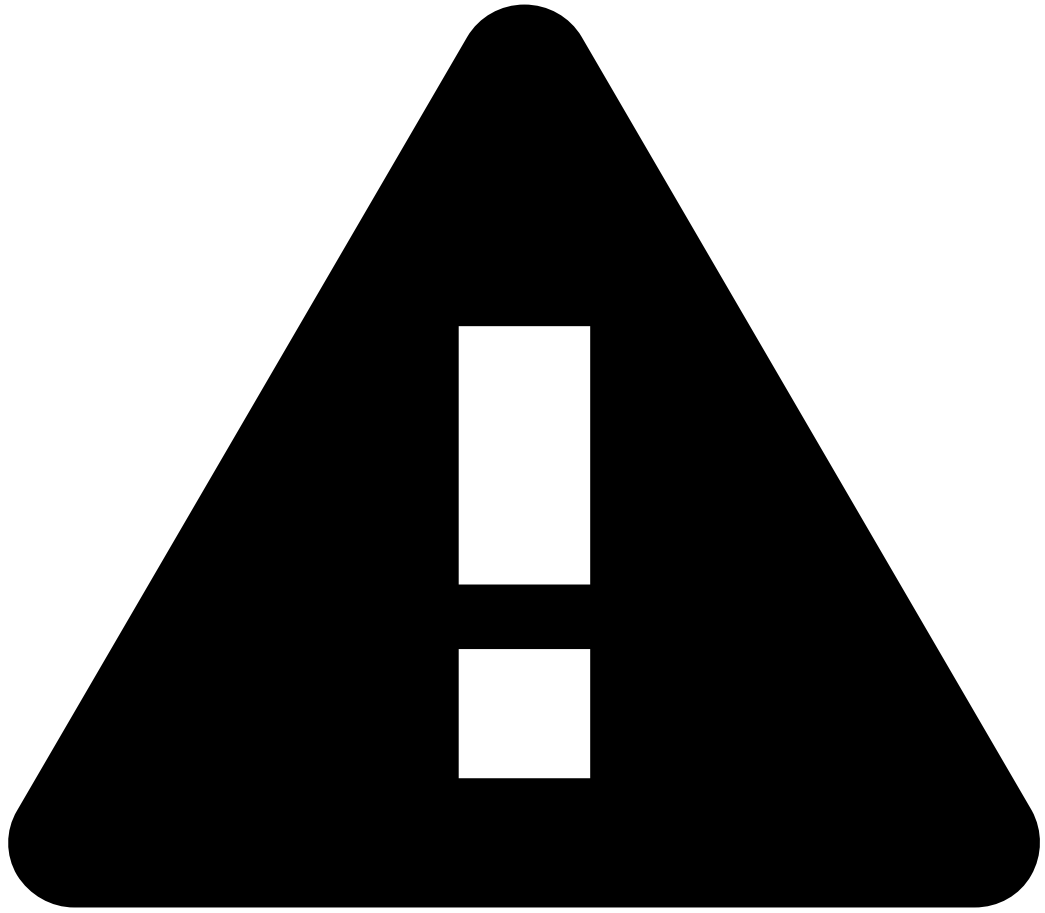
3. Add a **Name** for the new job profile.
4. To add new Spark properties:
 1. Insert the name of the Spark property in the **Key** column.
 2. Insert the **value** or encrypted value of the Spark property.
 3. Select the Add button.
5. To edit the value of a property:
 1. Locate the Spark property that you want to change.
 2. Press the current value of the property.

3. Add a new value or encrypted value and select the button to save the changes.



Editing spark properties

The values of the base properties are shown in light grey text. The value of new properties and the ones that were edited are shown in black text. You can revert the edited value of a base property to the one that is defined in the PPP. You can delete a non-base property from the list.



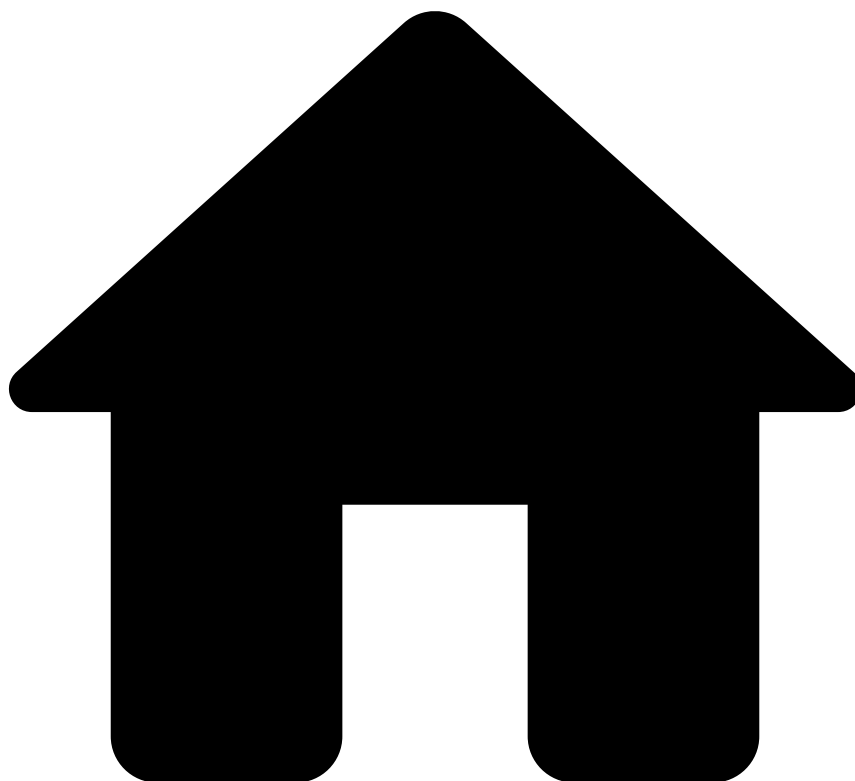
caution

The gray properties displayed in the job profile table are default properties used for data science jobs. Therefore do not use these properties as a reference for Scheduled Metric Jobs.

Next steps📖

Now that you know how to create job profiles, learn how to [select profiles](#) to be used in the data science tasks that are executed by Spark jobs.

•



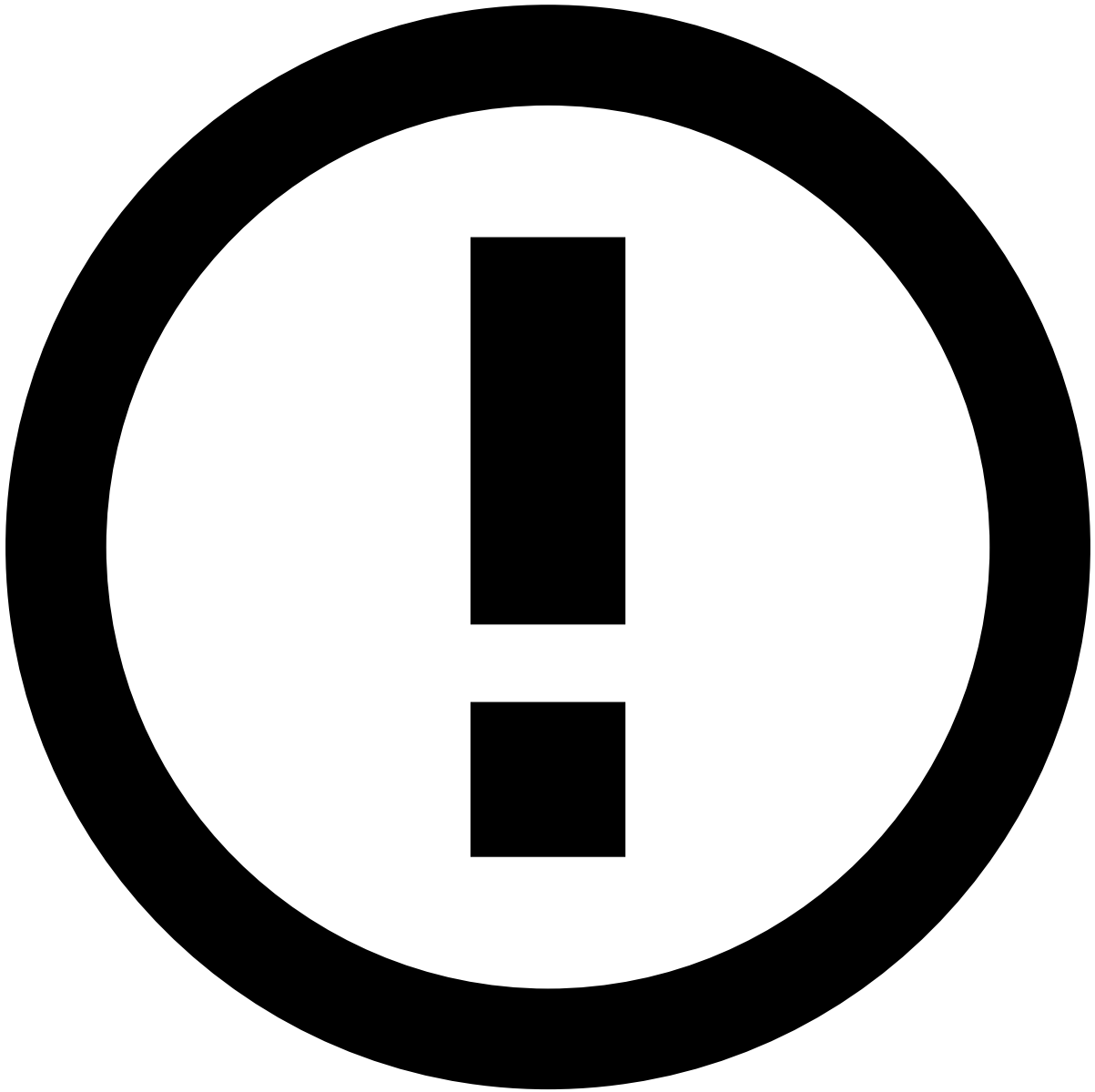
- [Creating a Data Science project](#)
- [Tuning Job executions with Job Profiles](#)
- Editing properties on demand

On this page

Editing properties on demand

Was this page helpful? 

Pulse allows you to configure Spark properties on demand, while setting up a Data Science resource (e.g. Model Execution, Hyperparameter Tuning) that is executed in a distributed way. This gives you full control over the configuration of the [Spark job](#) that performs the calculations for that specific resource.



Valid only for a single Job Execution

The properties that you define on demand are valid only for this single Job Execution. In other words, Pulse does not save the changed values to any Job Profile.

To configure properties on demand, perform the following actions:

1. While setting up a Data Science resource, in the **Advanced Options** select the **Job Profile** that you want to start with. Check [Selecting a profile on demand](#) to learn more about this action.
2. Use the button to load the properties of the selected profile for editing. As an example, the image below shows the editing menu for the task [Data Source Analysis](#):
3. To add new Spark properties:
 1. Insert the name of the Spark property in the column Key.
 2. Insert the value or encrypted value of the Spark property.
 3. Select the button .

4. To edit the value of a property:

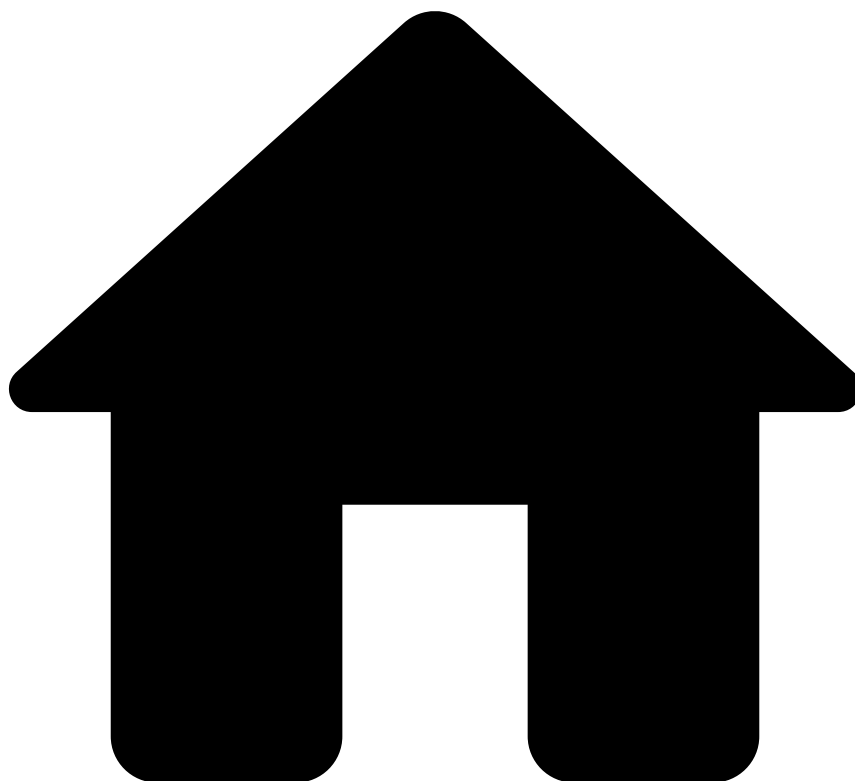
1. Locate the Spark property that you want to change.
2. Select the current value of the property.
3. Type a new value or encrypted value and **Save the Changes**.

5. Finish creating the resource that you are configuring. In the previous example, select the **Analyse** button to save the Data Source Analysis. It starts the Spark job that executes the necessary computational tasks using the settings you selected in the previous steps.

Next steps

Now that you have configured all aspects related to Job Profiles, you can start [processing your data](#) efficiently by taking advantage of the optimized profiles that you have created.

•



- [Creating a Data Science project](#)
- [Tuning Job executions with Job Profiles](#)
- Selecting profiles

On this page

Selecting profiles

Was this page helpful? 🗨️

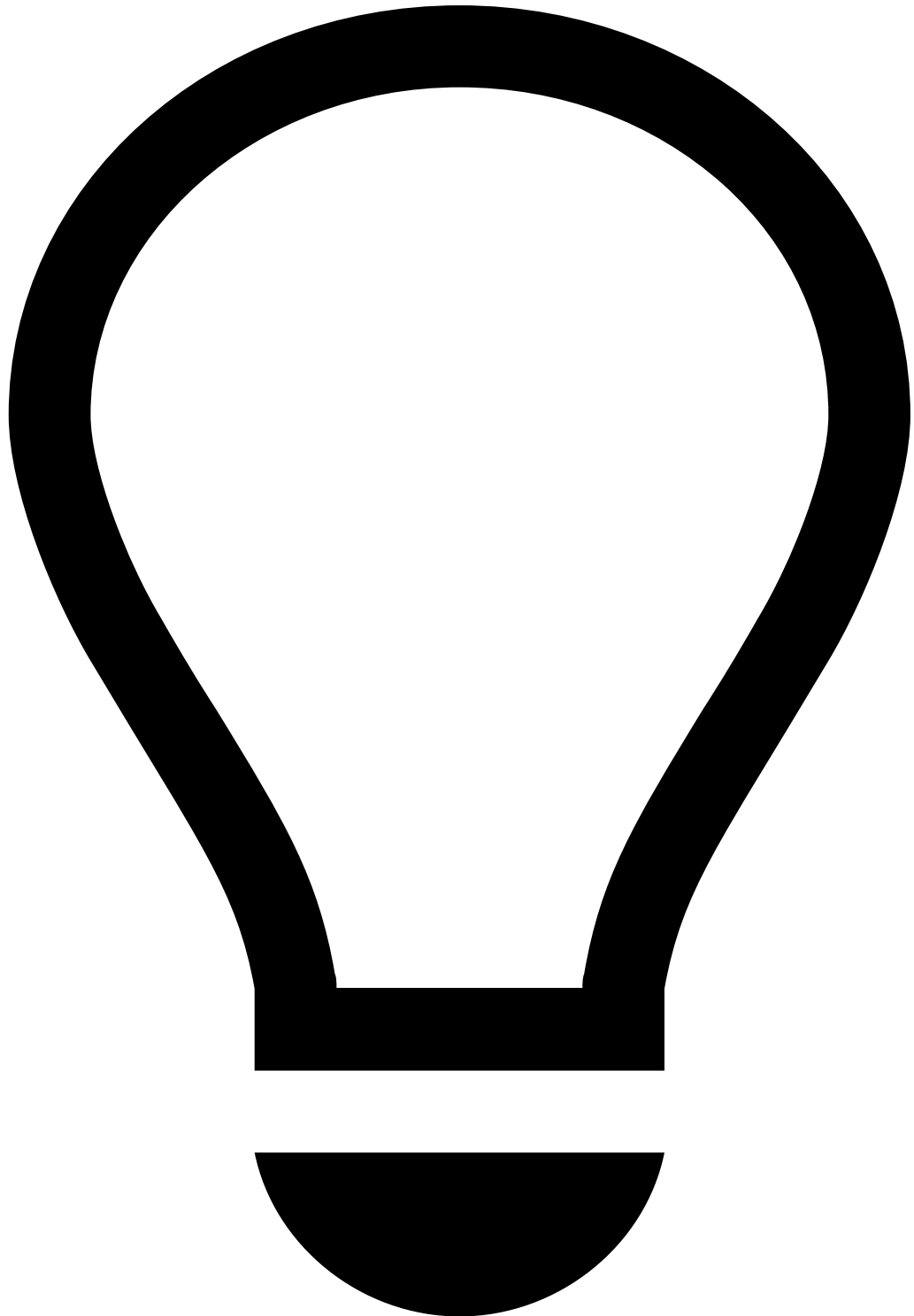
Pulse allows you to choose the profile that is more suitable for the Spark job that you want to run using the options below:

- **Setting the default profile for the application:** allows you to set a profile that is used by default every time that you create any Data Science resource. You can look at this option as a global setting for the whole application.
- **Setting a default profile for a Job type:** allows you to associate a profile to a Pulse resource (e.g. [Feature Importance](#)). This option prevails over the application default.
- **Selecting a profile/editing properties on demand, for a specific job:** allows you to choose a profile or edit properties directly when creating a specific Pulse resource. This option prevails over the other two, but only for the current resource that you are creating.

Setting a profile as default for the App

To set a profile as default for the whole application, perform the following steps:

1. Select the **Jobs** tab to have access to the **Job Configuration Profile** list:

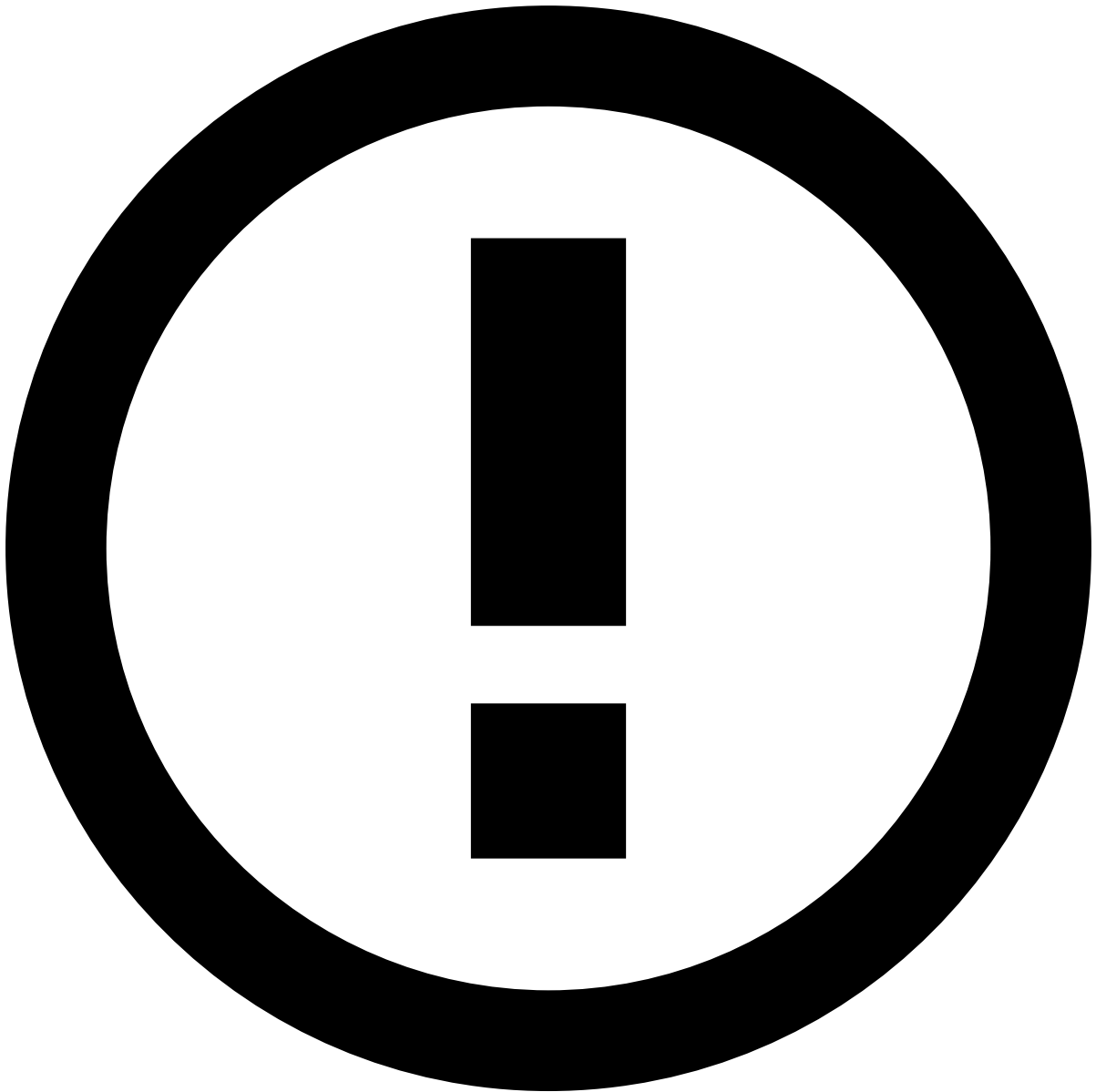


tip

By default, the **Jobs** tab is collapsed under the **Advanced Tabs**, and you need to use the button to reach it.

2. Locate the target profile that you want to choose as default, and select the **Star** button.

Setting a profile as default for a Job type



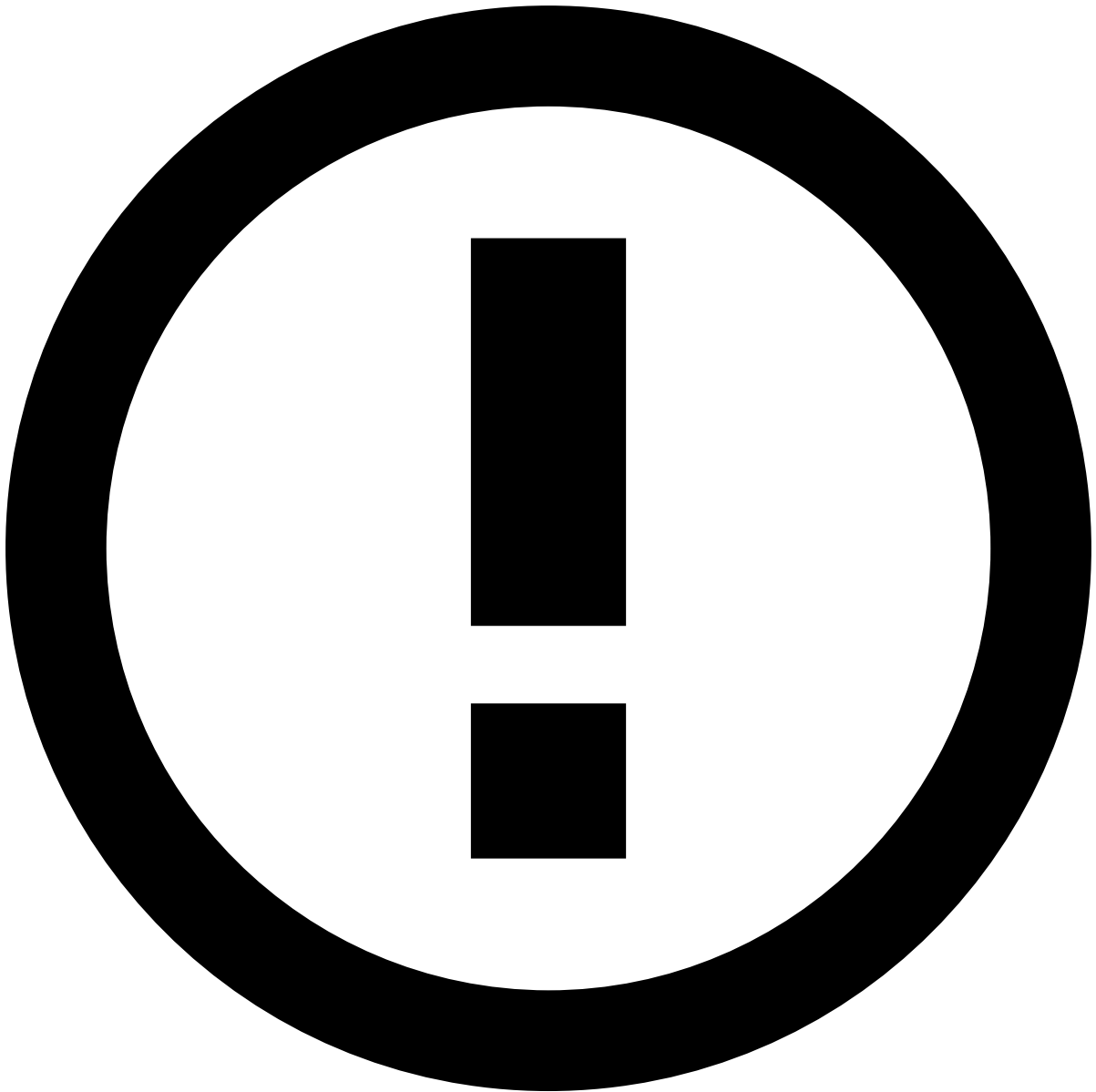
Preference over by default option

The option defining a profile by **Job type** prevails over the option [Defining a profile by default](#).

To associate a profile to a **Job type**, access your application and execute the following actions:

1. Select the **Jobs** tab to have access to the **Job Configuration Profile** table.
2. Select the button to have access to a list of **Resource Type** and **Job Profiles**.
3. For each **Resource Type** choose a target **Job Profile**.
4. Use the button to save your configurations.

Selecting or editing a profile on demand

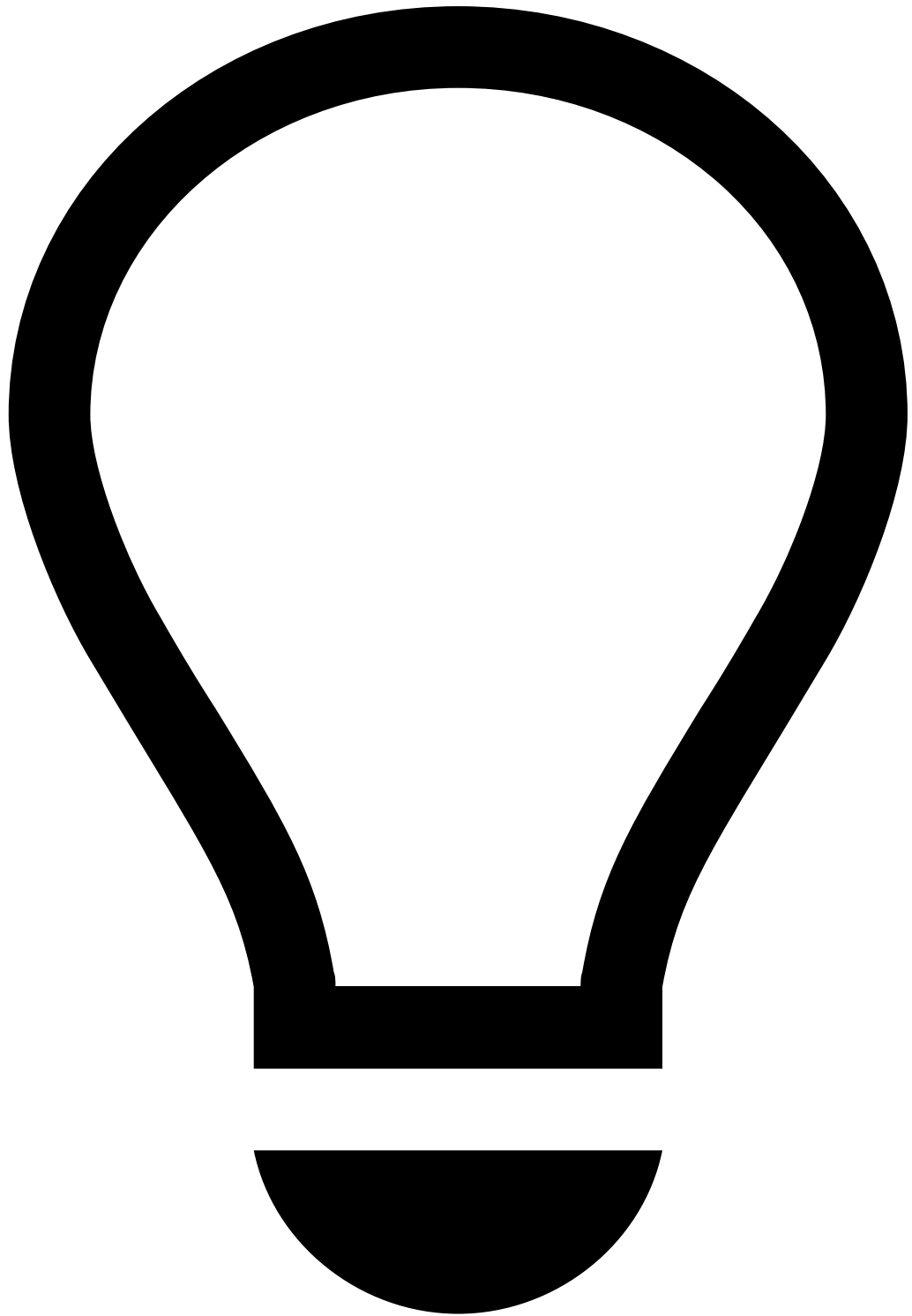


Editing properties on demand

In addition to the selection on demand you can **edit the properties** on a selected profile for this Job Execution. Check the page [Editing properties on demand](#) to learn more about this option.

To choose a profile, access your application and execute the following actions:

1. In the task that you are setting up expand the Advanced Options using the button. As an example, the following image shows the **Advanced Options** menu when you create a Data Source Analysis:
2. Select the Job Profile that you want to use to execute this task.



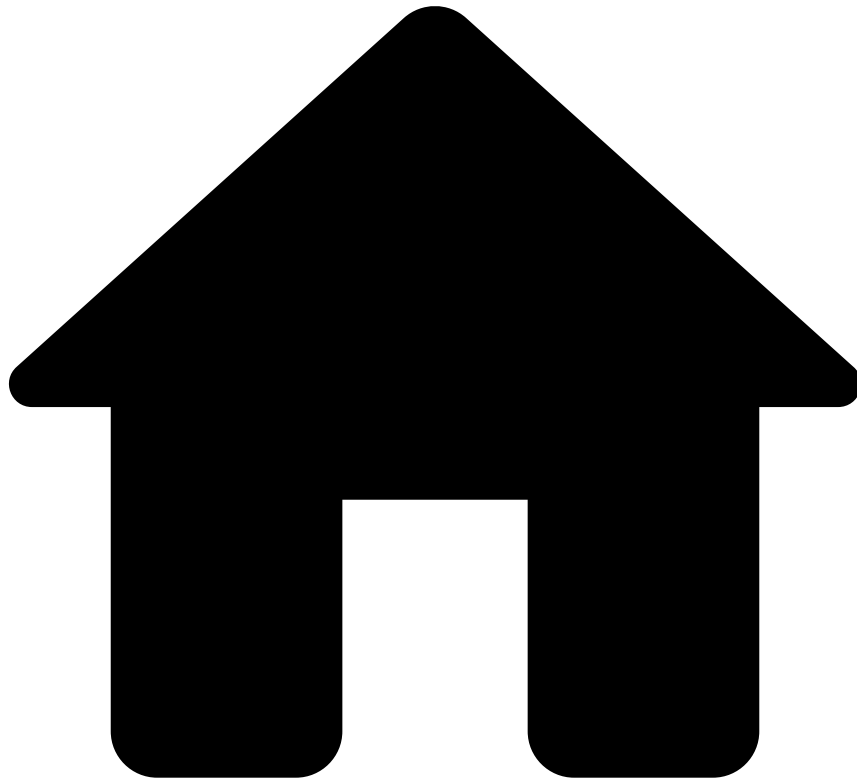
tip

You can open the profiles in separate windows to take a look at the properties by using the "Open Job Profile in new window" button. This option also allows you to edit the properties in the **Job Profiles**, except for the "Pulse Properties Profile" (PPP), which is profile read-only.

Next steps📄📄

Check the page [Editing properties on demand](#) to learn how to edit the properties in a **Job Profile** while setting up a Data Science task.

-



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding custom services

On this page

Adding custom services

Was this page helpful? 

Why

Custom Services implement custom logic which you can add to your runtime [Workflows](#). Other than Input Adapters you can use other **Custom Services** that react to messages as they are processed in the **Workflow**. For example, **Action Listeners**, merging decisions from different [Models](#), implementing caching, processing events to be displayed in dashboards, etc.

Depending on what the **Custom Service** is designed for, it may be connected in the data flow together with out-of-the-box services such as **Plan**, **Scoring**, and **Rules**, or just as a disconnected element that adds extra logic without affecting the flow of events.

How To

To add and configure a **Custom Workflow Element** to your **Workflow**, for example, to process events immediately before reaching the end of the **Workflow**:

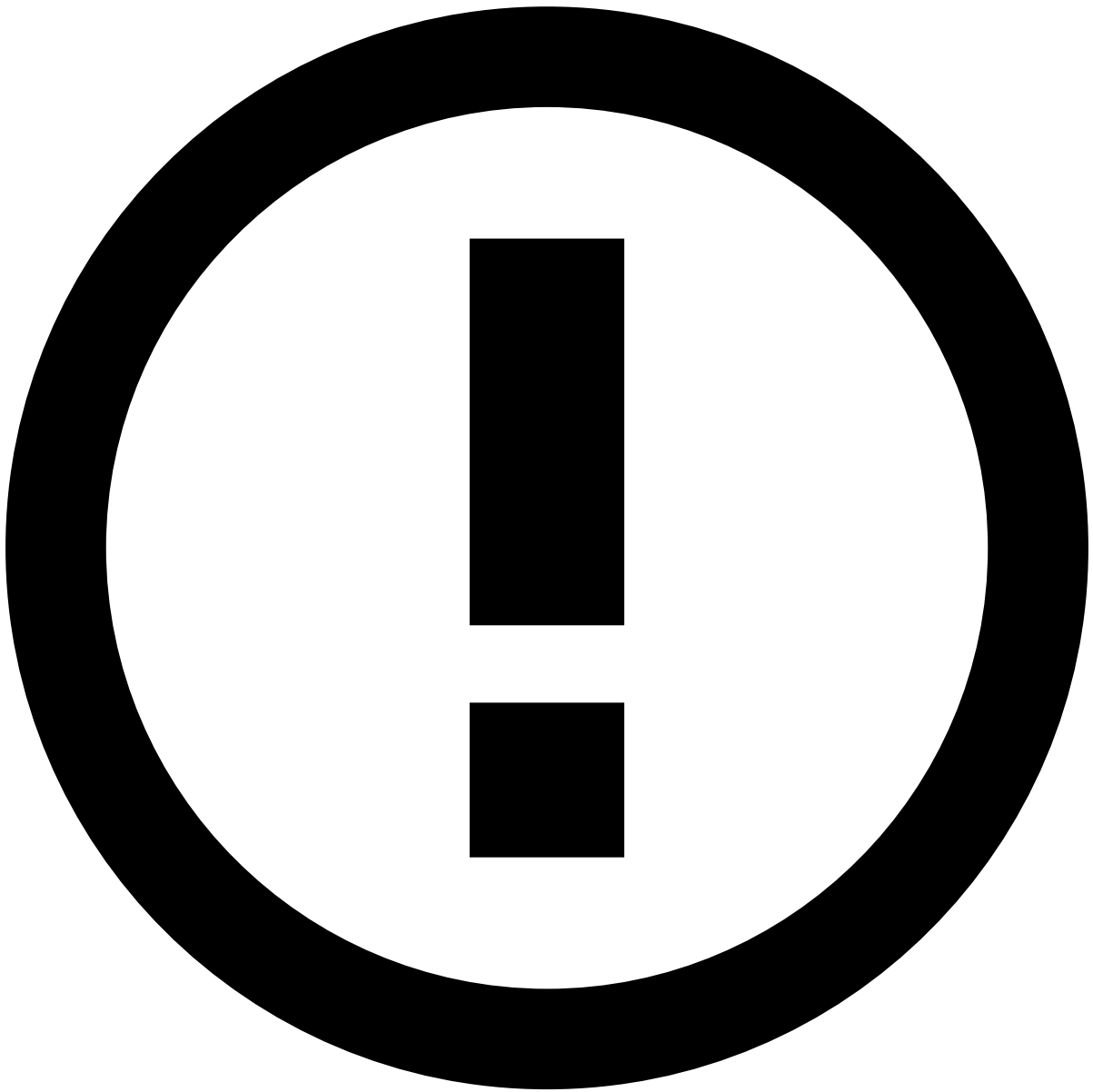
1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Custom Workflow Element** to the **Diagram** canvas.
3.  Edit the **Custom Workflow Element**.
4. In the **Main** tab:
 1. Select the **Custom Service** you want to apply in your **Workflow**.
 2. If the **Custom Service** has been developed to contribute to a [Workflow Outcome](#), select an **Outcome**.
After being processed at a **Custom Workflow Element**, the `<outcome_name>` and `_<outcome_name>score` fields in the event schema will reflect the processing.
5. In the **Schemas** tab, if the **Custom Service** operates over the data itself:
 1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Custom Service**.
 2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.
6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**.
Properties and default values depend on how the **Custom Service** is developed.
7.  **Connect** the previous service in the **Workflow** to the **Custom Workflow Element**, if it is part of the data flow.
For example, if you implement an **Action Listener** in a **Custom Service**, you just need to add it to the **Diagram** canvas, without having to **Connect** it.

Solving schema conflicts

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

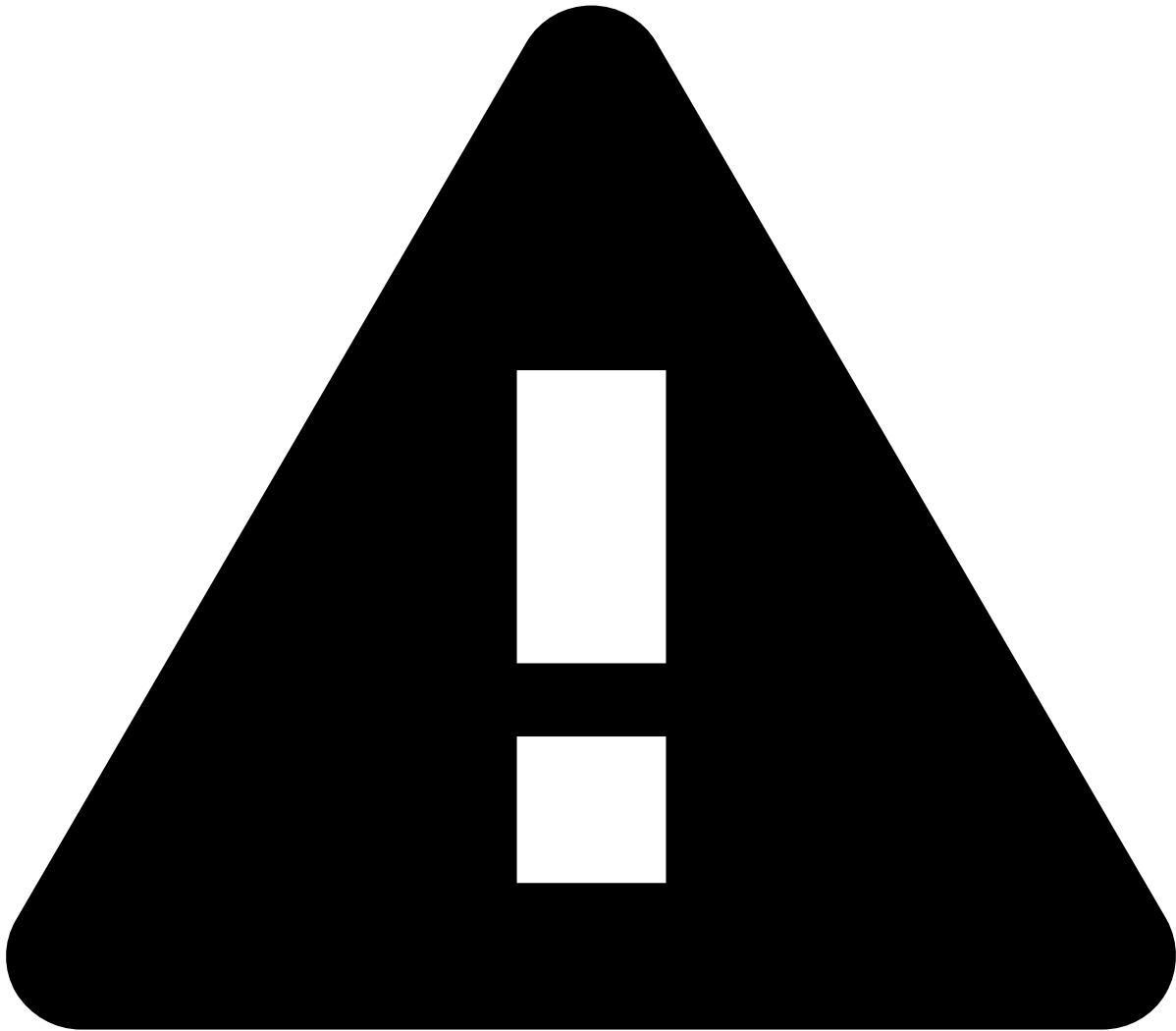
1.  Edit the **Service**.
2. Select the **Schemas** tab.
3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

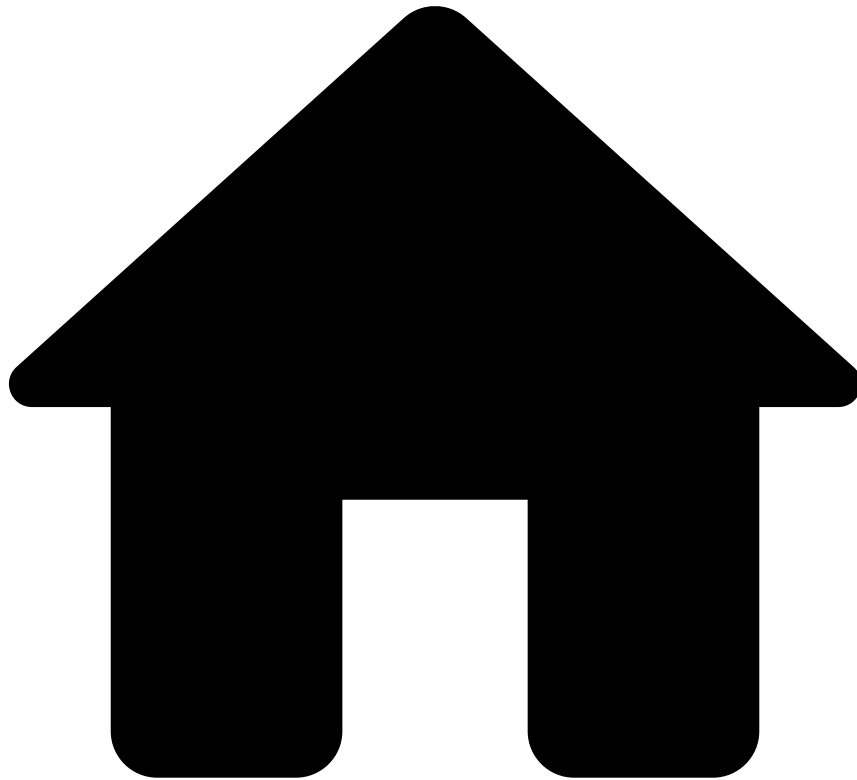
In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Next Steps📖

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding rules](#)

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding input adapters

On this page

Adding input adapters

Was this page helpful? 

Why

Input adapters are the [Workflow](#) element that allow your application to know how to consume data from an external source.

Your **Workflow** can consume data from different sources, such as CSV files, REST APIs, sockets reading ISO8583-formatted messages, etc. Thus, for each data input in your project, your **Workflow** needs a different **Input adapter**.

Adding **Input adapters** is the first step in designing your **Workflow**.

How to

To add and configure an Input adapter to your [Workflow](#), execute the following actions:

1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Input Adapter Service** to the **Diagram** canvas.
3.  **Edit** the **Input Adapter Service**.
4. In the **Main** tab, select which [Custom Service](#) implements the functionality for your **Input Adapter**. Pulse includes an embedded **Synthetic Generator** that acts as a test input Custom Service, and periodically injects dummy data into your **Workflow**, according to its **Schema**.
5. In the **Schemas** tab, confirm that the **Service Output** is consistent with the data you expect for your project.



Extra fields

Note that the **Service Output** contains more fields than the **Schema** you specified for the **Workflow**. Pulse automatically adds a *timestamp* and *UUID* fields to the event schema.

Additionally, for each [configured Outcome](#) Pulse adds the boolean `<outcome_name>` and `_<outcome_name>score` fields.

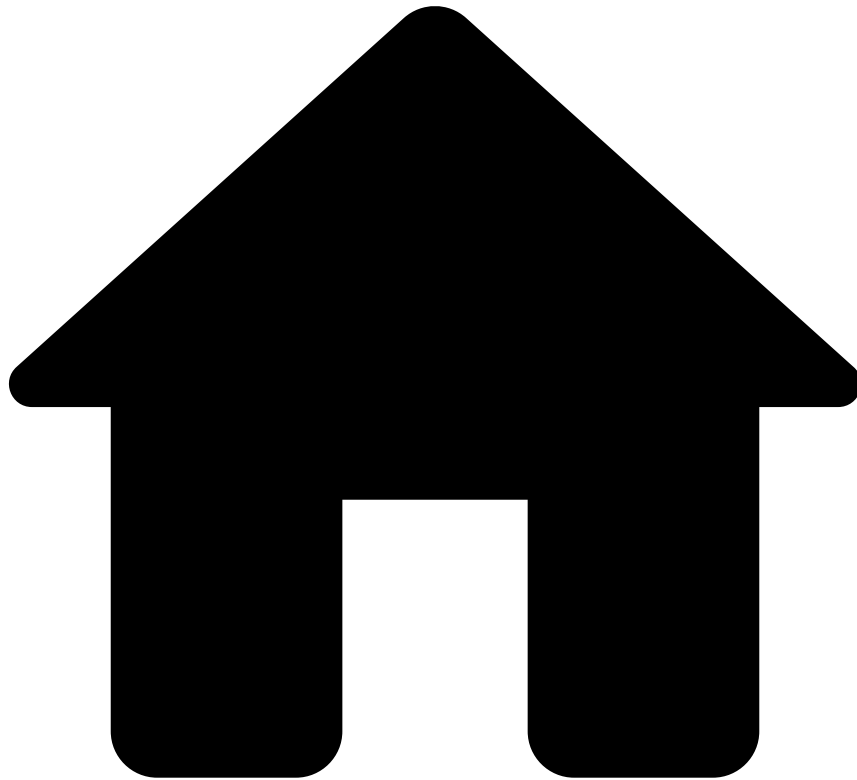
6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**. Properties and default values are defined at the **Custom Service** that implements your **Input Adapter**.

Next Steps

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding plans](#)
- [Adding models](#)
- [Adding rules](#)
- [Adding custom services](#)

-



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding models

On this page

Adding models

Was this page helpful? 

Why

Similarly to [Workflow Plans](#), Pulse lets you take the **Model** definition that yielded the best results during the Data Science stage directly into Production without significant effort.

Once you obtain a [Model Evaluation](#) that shows that your **Model** configuration has reached or surpassed business objectives, you can simply apply it in runtime.

The **Scoring Service** in a **Workflow** runs events through the Machine Learning algorithm, by default Feedzai Random Forest (FZRF), with the exact same parameter configuration as tested during the Data Science stage.

How to

To add and configure a [Model](#) to score events in your **Workflow**, execute the following actions:

1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Scoring Service** to the **Diagram** canvas.
3.  **Edit** the **Scoring Service**.
4. In the **Main** tab:
 1. Define to which of the [Workflow Outcomes](#) the **Model** will contribute. After being processed by a **Scoring Service**, the `<outcome_name>_and_<outcome_name>_score` fields in the event schema will reflect the **Model** decision.
 2. Select the **Model** you want to apply in your runtime processing.
 3. Optionally, define a **Filter** to narrow down events that are scored in your **Workflow**. Filter conditions are written in PulseQL.



Fields used in Filter

You can only use fields in the **Filter** which were present in the [data source](#) you used to train the **Model**.

4. Define how much **Weight** the **Model** results will have in the final decision, when the event arrives at the end of the **Workflow**. For example, you can have real-time events flowing through different **Scoring Services** and [Rules Services](#). These scores need to be combined to produce a final score for the event.
5. White-Box Explanations that were added with the new model in a first instance (see section **Training parameters** in [Training a Model](#)) are then visible in the **Scoring Service** and automatically used.



Migrating old White-Box Explanations

After upgrading your Pulse version to **Pulse 20.1**,

you may need to reconfigure the old WBE. This is in case you enabled the old **Feedzai Random Forest Feature Contribution**, which only generated contribution numbers and could not work with natural language sentences. To do so, **Edit** the **Scoring Service** and follow the instructions in the UI.

Furthermore, if you were using the old **Feature Contribution** for **Model Explanations**, then you were most likely using a Custom extended **Alert Storage Service**. In that case, please check **Case Manager 8.1** release notes for more information as you no longer need to do that.



Explanations on critical path

True â Explanations are calculated in the **Scoring element** and put in the event.

False â Explanations are only calculated in the **Dead Letter** (i.e. at the end of the workflow).

5. In the **Schemas** tab:

1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Model**.
2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.
6. â **Connect** the previous service in the **Workflow** to the Scoring Service.

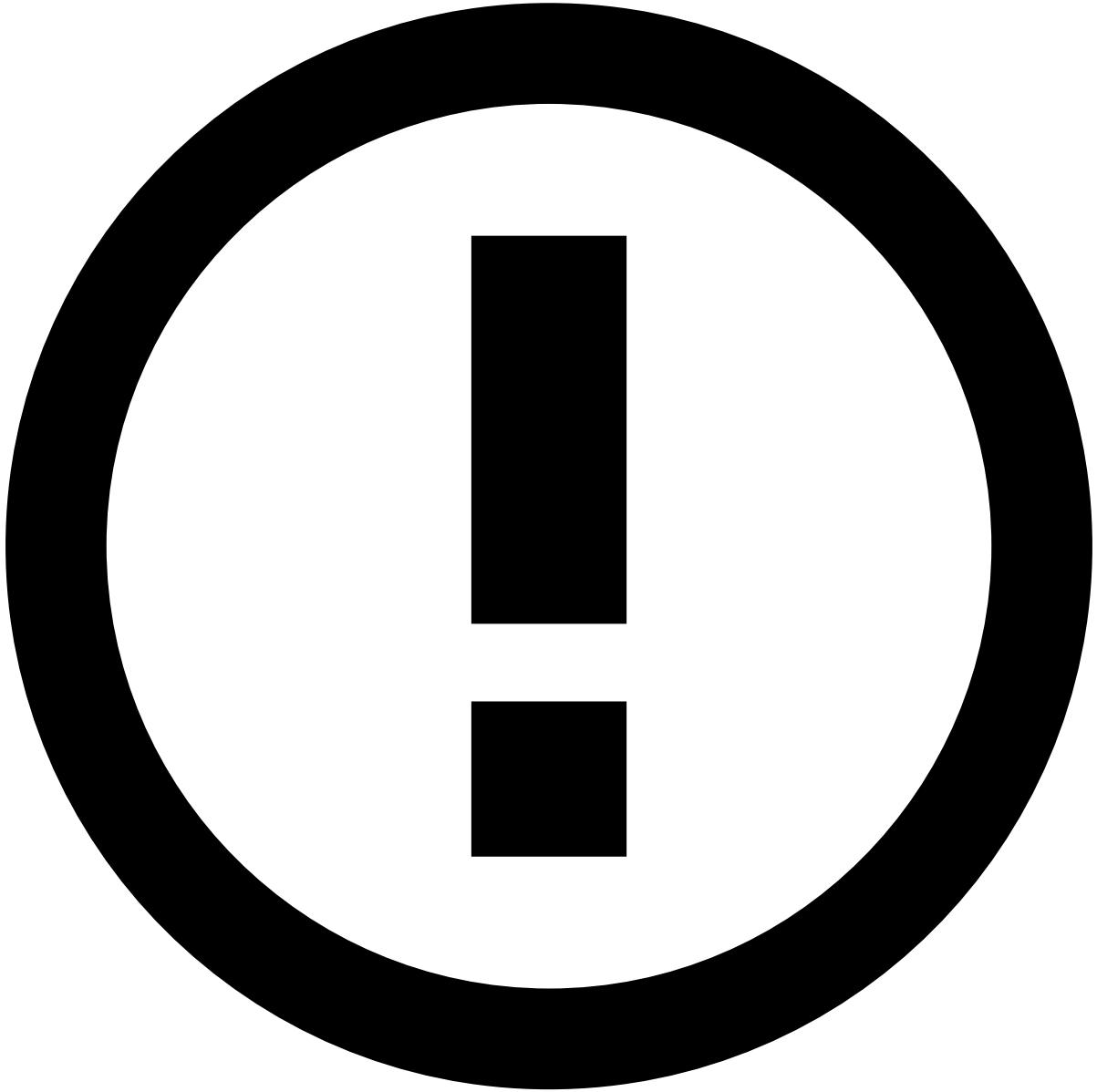
Solving schema conflictsâ

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

1. â Edit the **Service**.
2. Select the **Schemas** tab.
3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.

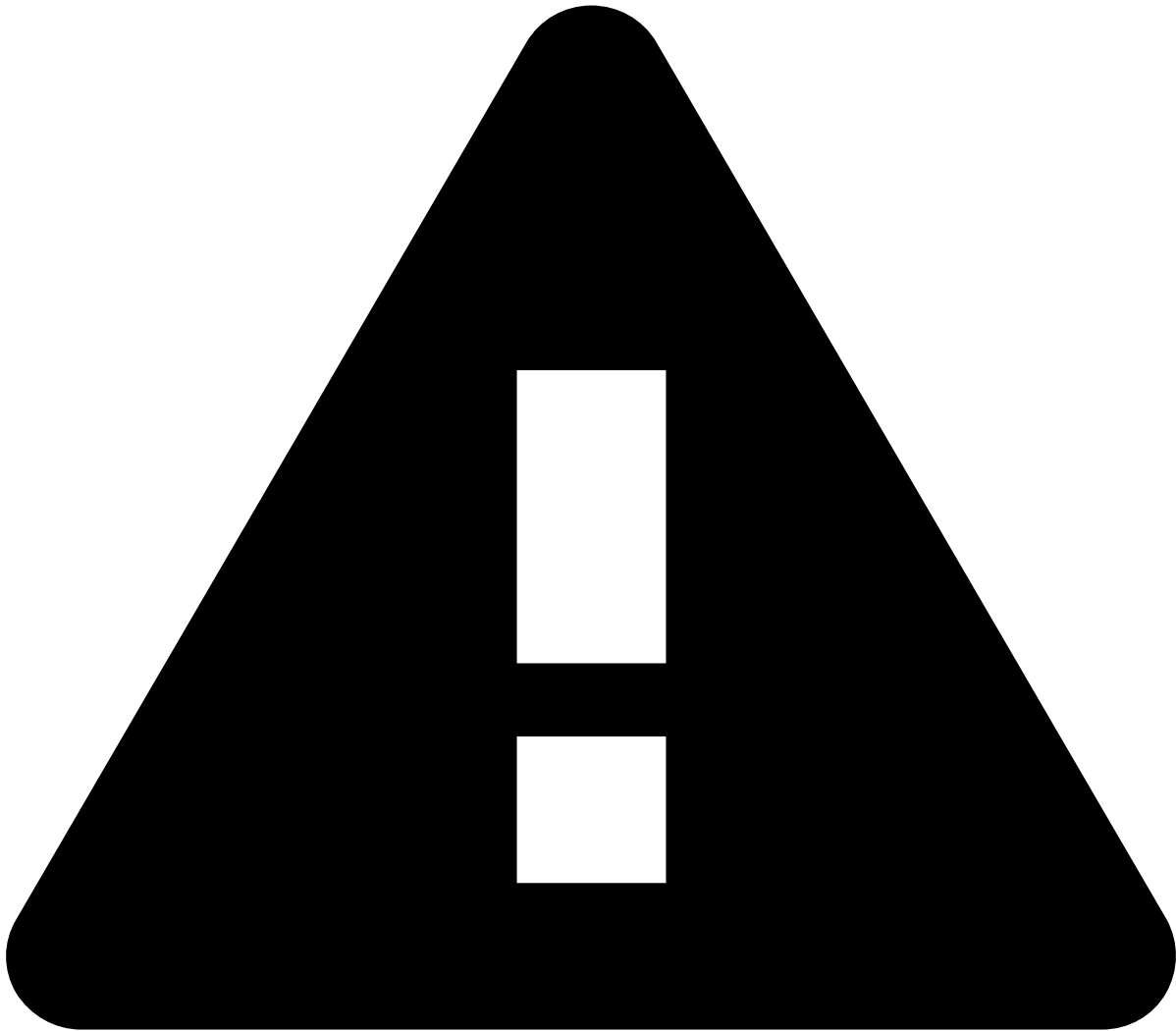
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Next Steps📖

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding rules](#)
- [Adding custom services](#)

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding plans

On this page

Adding plans

Was this page helpful? 

Why

One of the great advantages of Pulse is to make the deployment of Data Science work into runtime seamless and without any significant effort. **Workflow Plans** are the direct translation of deployable [Data Science Plans](#) as runtime services.

Workflow Plans thus allow you to process live data and generate features exactly the same way as during the Data Science stage of the project.

Once a deployable **Data Science Plan**, for example, a features plan, is tested and ready to deploy to Production, Pulse creates an execution. A **Plan Execution** represents a snapshot of the **Plan** and all of its configurations at the time it ran, making it available for runtime.

How to

To add and configure a **Plan** to your **Workflow**, for example, to calculate features on live data, execute the following actions:

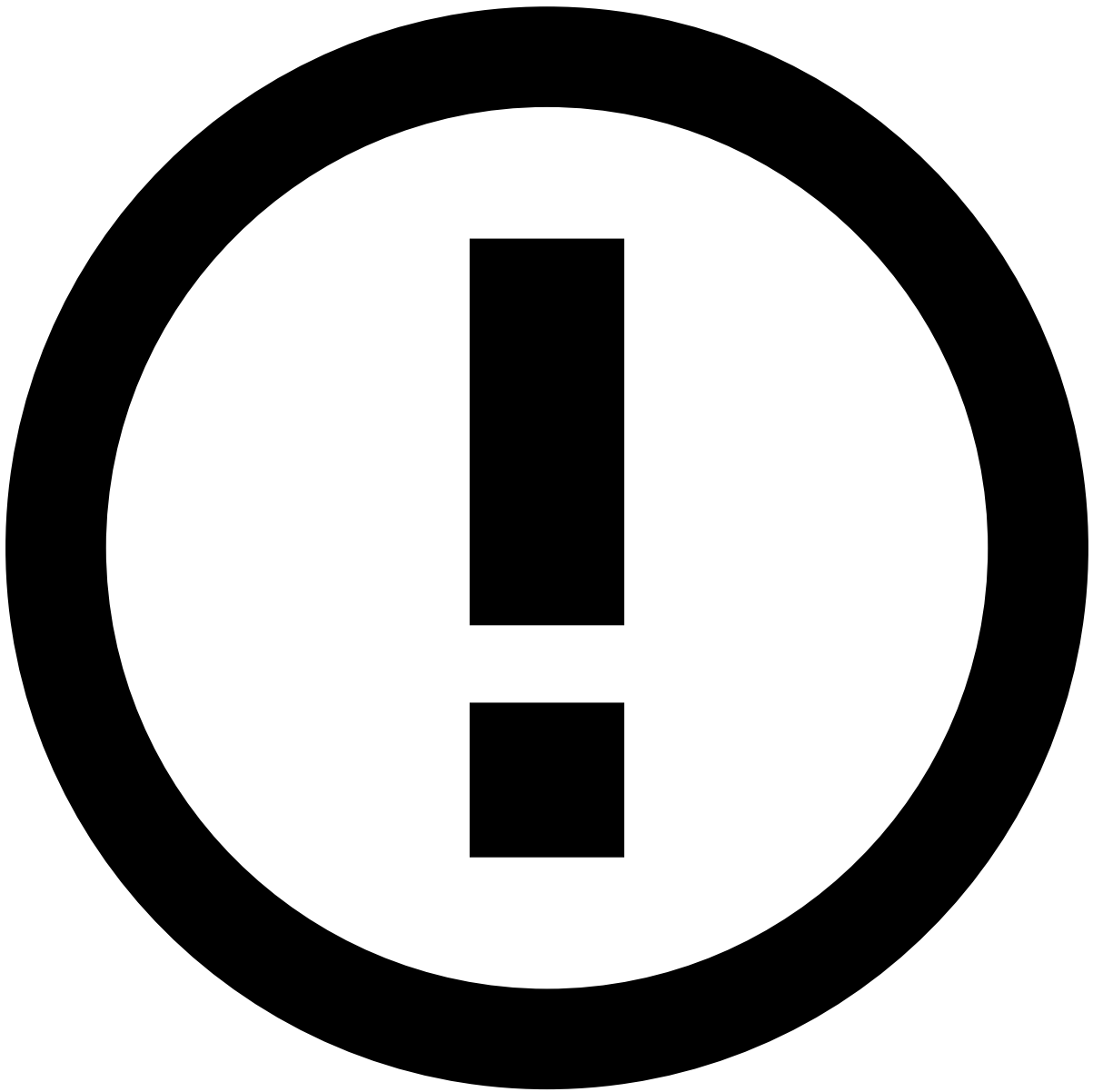
1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Plan Service** to the **Diagram** canvas.
3.  Edit the **Plan**.
4. In the **Main** tab:
 1. Select the [Data Science Plan](#) you want to apply in your runtime processing. Only **Deployable Plans** are available.
 2. Select the **Plan Execution** to carry the input and output schemas of the **Plan**, as defined during Data Science stage, into your workflow.
5. In the **Schemas** tab:
 1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Plan**.
 2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.
6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**. Properties and default values are defined by Pulse.
7.  **Connect** the previous service in the **Workflow** to the **Plan**.

Solving schema conflicts

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

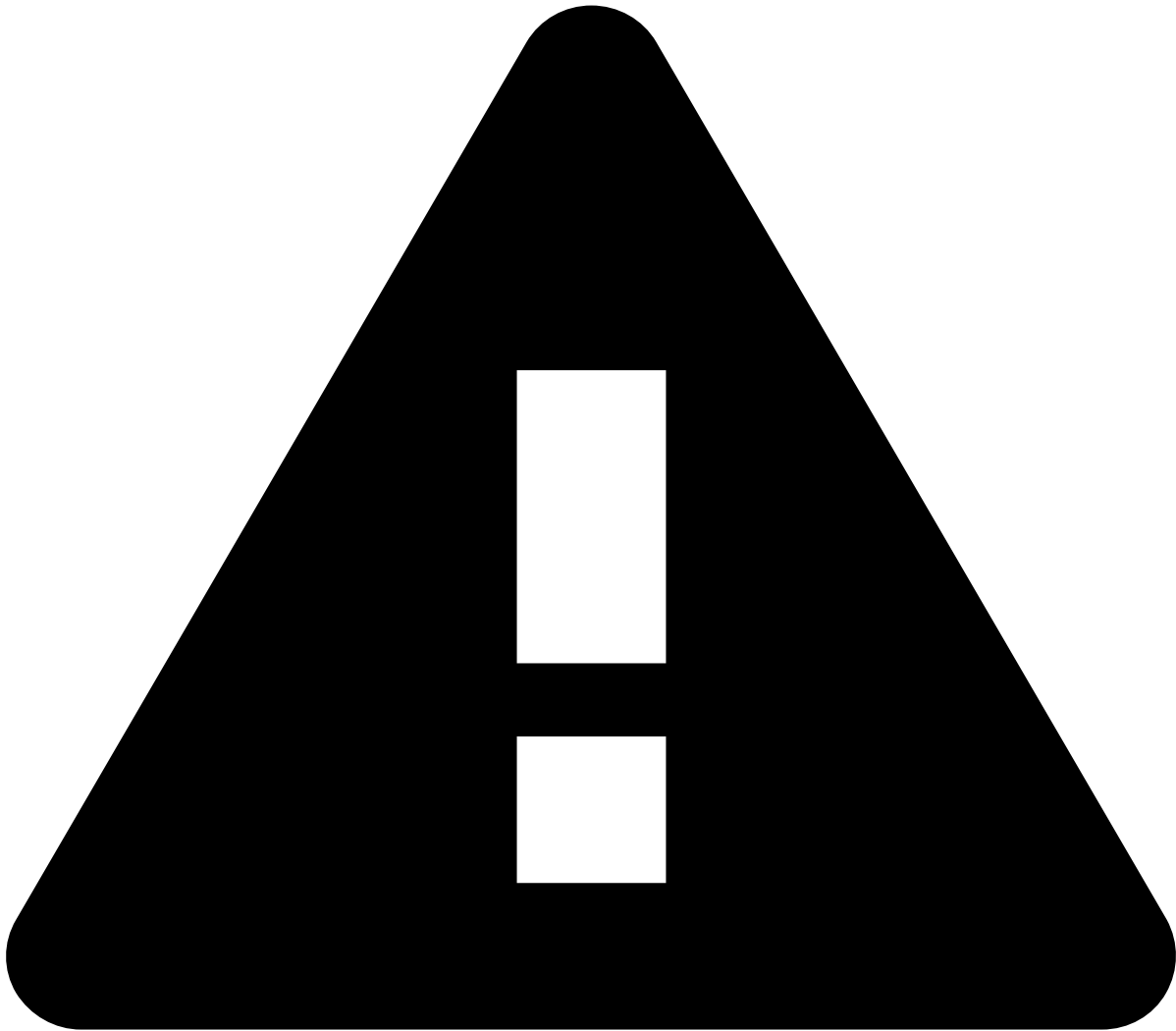
1.  Edit the **Service**.
2. Select the **Schemas** tab.
3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Addressing errors in plan definition🔗

When defining the components of a **Plan**, you may encounter errors that would result in failures during the [plan execution](#).

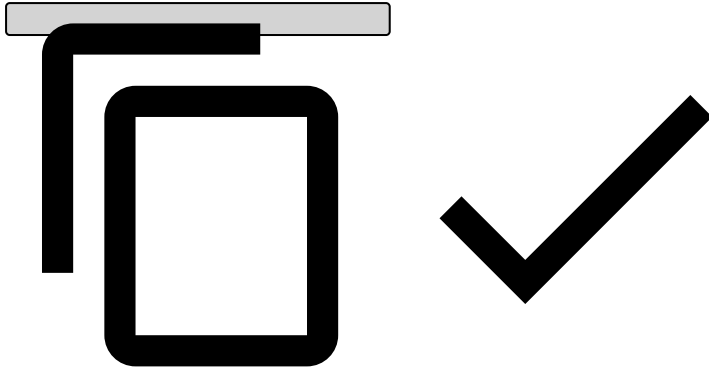
Some of those errors are detectable when defining the Plan, and Pulse will show warnings to allow them to be fixed before deploying the plan to production.

One such error is defining [fields](#) based on a [PQL expression](#) (e.g. within a [Metrics or Map operator](#)). Operators that require null coalescing (i.e. [default fallback values](#)) will produce a warning in the UI if the coalescing operator (`??`) and value are missing:

To fix this situation, provide a fallback value in the PQL expression input field, e.g. `?? 0` or `?? ""`.

A similar warning will be shown if built-in functions are used in collections (like `List` s, `Set` s or `Map` s). For example, an expression like

```
["test", "test1", "test2"].contains("testing")
```

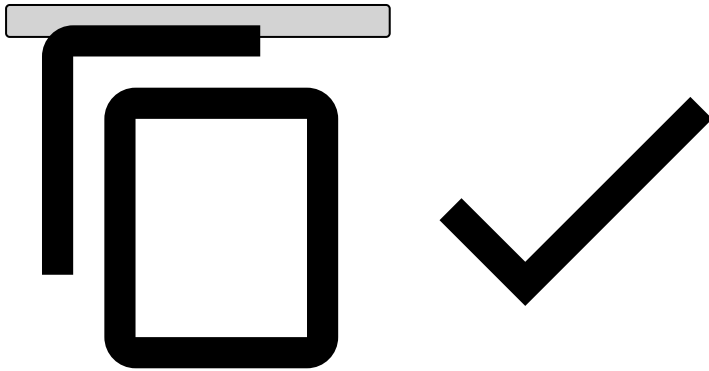


will fail because the `contains` function is not supported in collections used in the PQL fields of Map Operators or Metric Operators.

Attempting to use such functions in that context will result in a warning in the UI:

The alternative is to use [UDFs \(User Defined Functions\)](#), e.g.:

```
com.feedzai.pulse.pkernel.runtime.CollectionUDF.contains(["test", "test1", "test2"], "testing")
```



Besides these preventive warnings, Pulse will also allow recovering from execution errors that are not detectable at design time. Refer to [Automatic Application Repair](#) and [Workflow recovery](#) for more information on how to recover from execution errors.

Next Steps📄📄

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding models](#)
- [Adding rules](#)
- [Adding custom services](#)

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding rules

On this page

Adding rules

Was this page helpful? 

Why

Rules add an extra layer of scoring and decision-making to your project in runtime. Just like [Models](#), **Rules** contribute to the classification of events moving along the **Workflow** and to the final [Workflow outcomes](#).

Once you reach a [Rules Evaluation](#) that shows that your **Rules** block has reached or surpassed business objectives, you can simply configure the same block of Rules to run within a service in runtime.

You can deploy blocks of [Rules](#) in a runtime **Workflow** by adding **Rules Services**. Typically, you can deploy different **Rules Services** both for detection, for example, blacklisting users, and for a decision, for example weighing in on the final score.

How to

To add and configure a block of [Rules](#) to score events in your **Workflow**, execute the following actions:

1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Rules Service** to the **Diagram** canvas.



Rules Services

When selecting Rules Services in workflows, a snapshot is automatically generated and selected in the configuration dialog. You cannot select any other snapshot than the automatically-generated one.

3.  **Edit the Rules Service.**

4. In the **Main** tab:

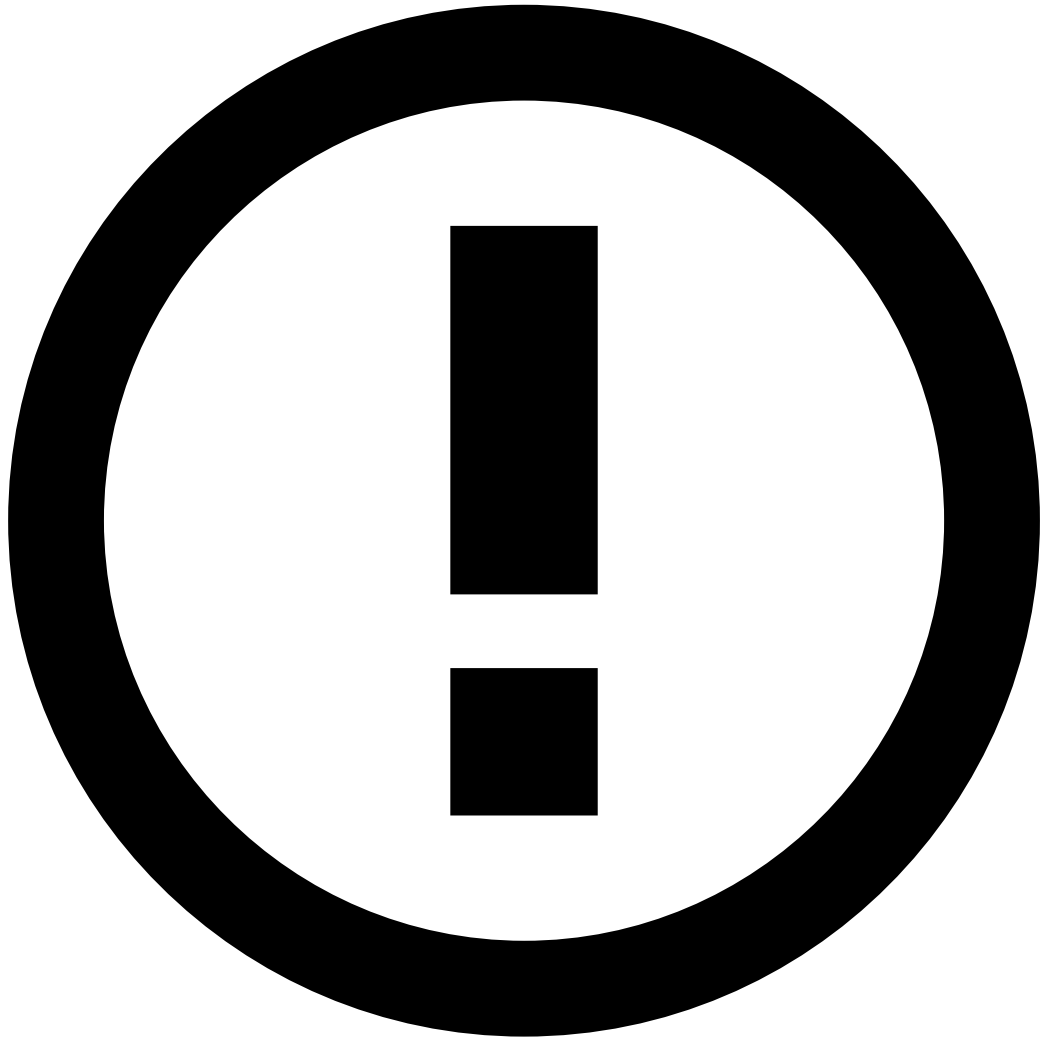
1. Define to which of the [Workflow Outcomes](#) the **Rules** will contribute. After being processed by a **Rules Service**, the `<outcome_name>` and `<outcome_name>_score` fields in the event schema will reflect the **Rules** decision.
2. Select the **Rules Project** that contains the block of **Rules** you want to apply in your runtime processing. The **Rules Project** also defines the input schema for the **Rules Service** as defined at the Data Science stage.



Self-Service Rules

All usage of self-services interface is performed through a Rules Workflow Element.

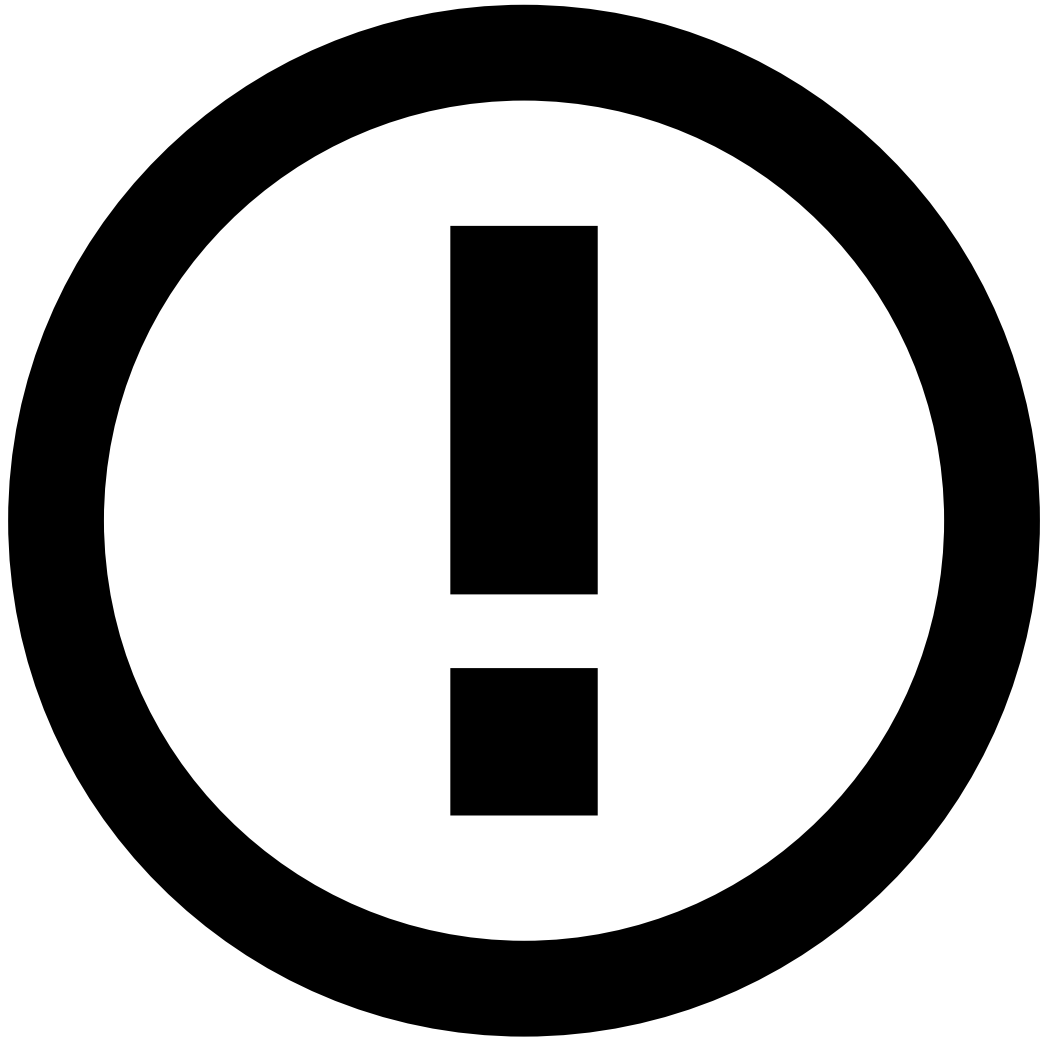
When adding a [Self-Service Rules project](#), a snapshot is automatically created and assigned. The latest snapshot is built adding changes on top of the one published before it (as reflected in the Publishing section). Older snapshots are kept and can be changed via API to allow a more sophisticated and greater control over the system. This allows you to easily look up the history of the published rules and roll back, if needed.



Self-Service Rules in a Multi-Tenancy scenario

In order to have a workflow ready for Self-Service Rules in a Multi-Tenanted scenario, you should add two rule elements:

- One pointing to a Global Simple rules projects
 - One pointing to a By-tenant Simple rules project.
3. Select the **Rules Snapshot** to specify which **Rules** to execute in the **Rules Service**. The **Rules Snapshot** carries the **Rules**, their order of execution and individual contribution to outcomes and scores, as defined and evaluated during the Data Science stage.



Multi-Tenancy Solution

If the Workflow is being configured with a [Rules Project](#) of the type Multi-Tenancy, the **Rules Snapshot** has information related to the **Rules** that are going to be applied to each Tenant.

4. Configure whether **Workflow** generates **Explanations** as to why the **Rules** triggered.



Explanations on critical path

If you choose to compute **Explanations** on the **Critical Path**, they will immediately be accessible to be processed in **Custom Service**, once they are generated.

5. In the **Schemas** tab:

1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Rules**.
2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.

6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**. Properties and default values are defined by Pulse.

7.  Connect the previous service in the Workflow to the Rules Service. You can also learn [to organize your work with blocks](#) in order to do this in a most efficient way.

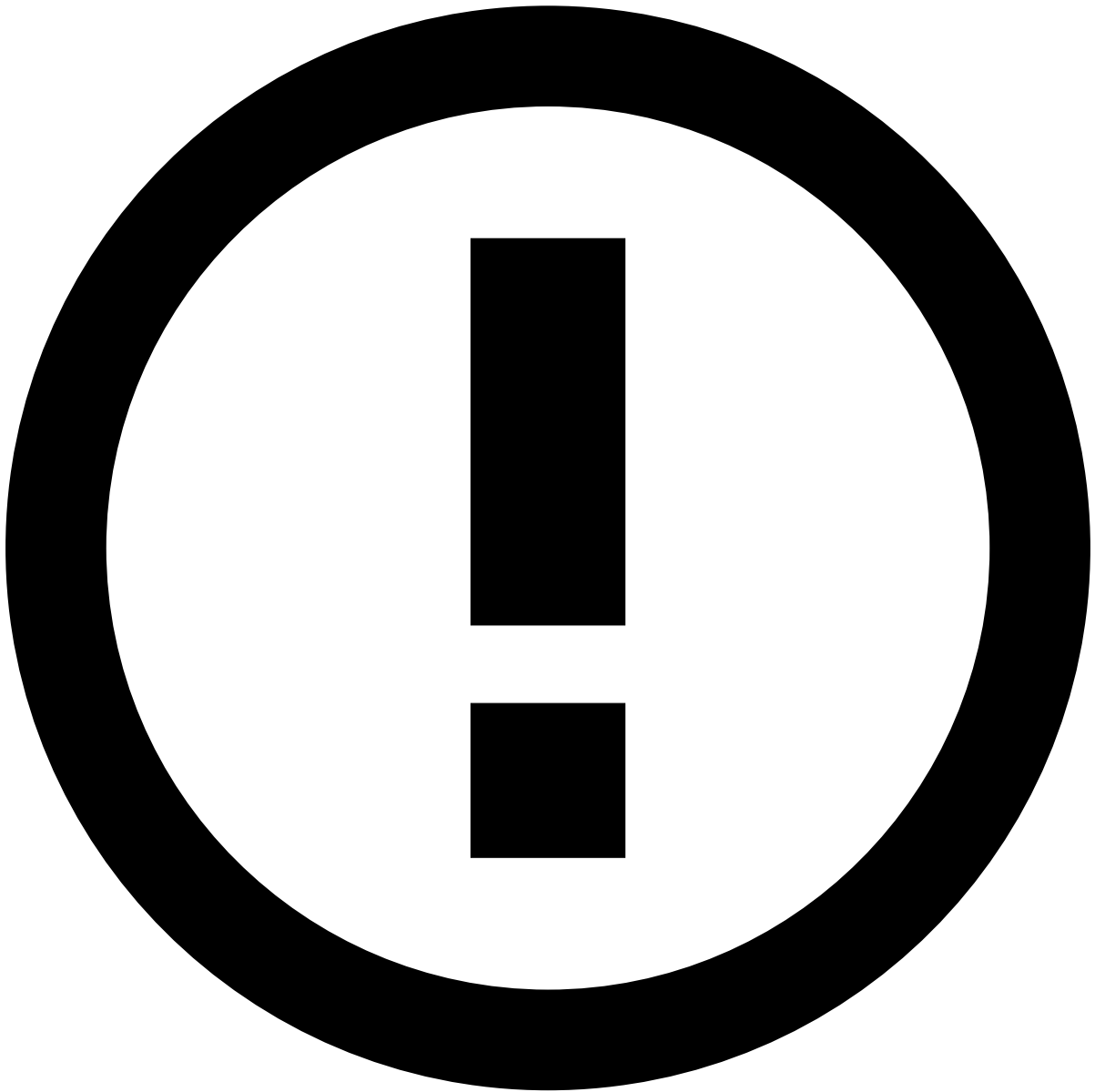
Solving schema conflicts

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

1.  Edit the **Service**.
2. Select the **Schemas** tab.

3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

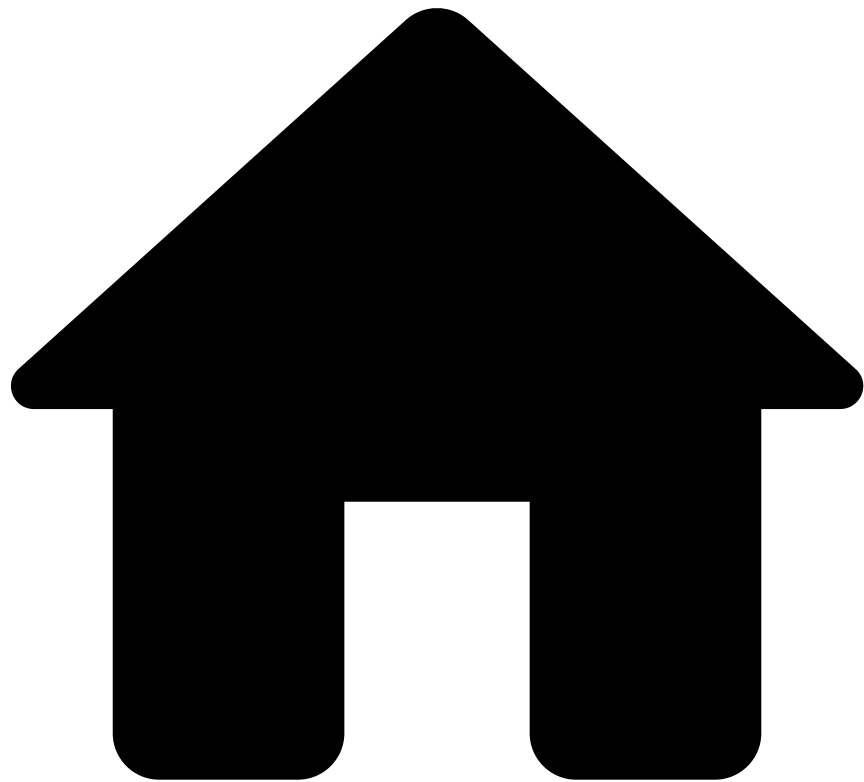
In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Next Steps

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding custom services](#)

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Advanced workflow configurations

On this page

Advanced workflow configurations

Was this page helpful? 

The following section explains the various configurations that you can set in the Advanced tab of a specific workflow (as seen in the following image):

Action parameters

Each action can have its own parameters. Parameters allow passing event specific information when an action is triggered.

One example would be for a "sendEmail" action to have the parameters "to" and "title" with values "" and "Rule Alert".

In the **Workflow Advanced** tab, it is possible to define which parameters each action has (by default has none) in the configure action dialog (accessible by clicking the config button on the action).

When defining parameters it is possible to choose their name, type and if its value is required or optional.

Parameters can have simple types like *String* or *Number* or more complex ones like:

- **Dimension**, restrict the set of possible values of the parameter to a Dimension's items.
- **List**, the value of the parameter will be a list (not a list item). This can be useful to reference, for example, card blacklists.
- **RDL Variable**, when configuring the value of the parameter it will be possible to choose from the list of RDL variables available within the rule. When the action is triggered the parameter will have the value of that variable.

Currently, actions can only be triggered directly by [rules](#) where it is possible to specify the value that each action parameter will have when the action is triggered. This can be done by clicking the config button available on the action that has more than one parameter.

Action listeners🔗🔗

When configuring action listeners, the list of available listeners is populated from the existing services. For a service to be listed either:

- The service is a part of a workflow
- The service is part of the overall application environment and implements a `com.feedzai.pulse.api.apps.model.rte.workflow.action.WorkflowActionHandler` interface from the `com.feedzai.pulse:pulse-api` module

Event Storage configuration🔗🔗

This option allows to define whether an Event Storage is created for the workflow. The recommendation is to enable this as it is a requirement for most workflows (it is necessary for Workflow recovery, Scheduled Metrics Jobs as well as feedback updates).

Workflow recovery configuration🔗🔗

This allows to define whether Workflow recovery is enabled. Most workflows should enable it and define the recovery period to be the same as the largest real-time metric (typically 24h or less).

When recovery is enabled, you must also provide the recovery time configuration in the *Interval* field, which will correspond to the time frame we would like to replay events from. Currently, two functions can be used to determine the interval of the data that is going to be recovered: **sinceLast** and **last**.

The **sinceLast** function will recover all events since the specified last point in time. The **last** function will recover all events from the specified past point in time to the last timestamp in the event storage.

The syntax for these functions is the following:

- `sinceLast-HHhMMm`
 - HH - The hours in 24-hour mode (0-23). This field is **optional**.
 - MM - The minutes (0-59). This field is **mandatory**.
- `last-HHhMMm`
 - HH - The hours. We are not restricted by the values 0-23. This field is **optional**.
 - MM - The minutes. We are not restricted by the values 0-59. This field is **mandatory**.

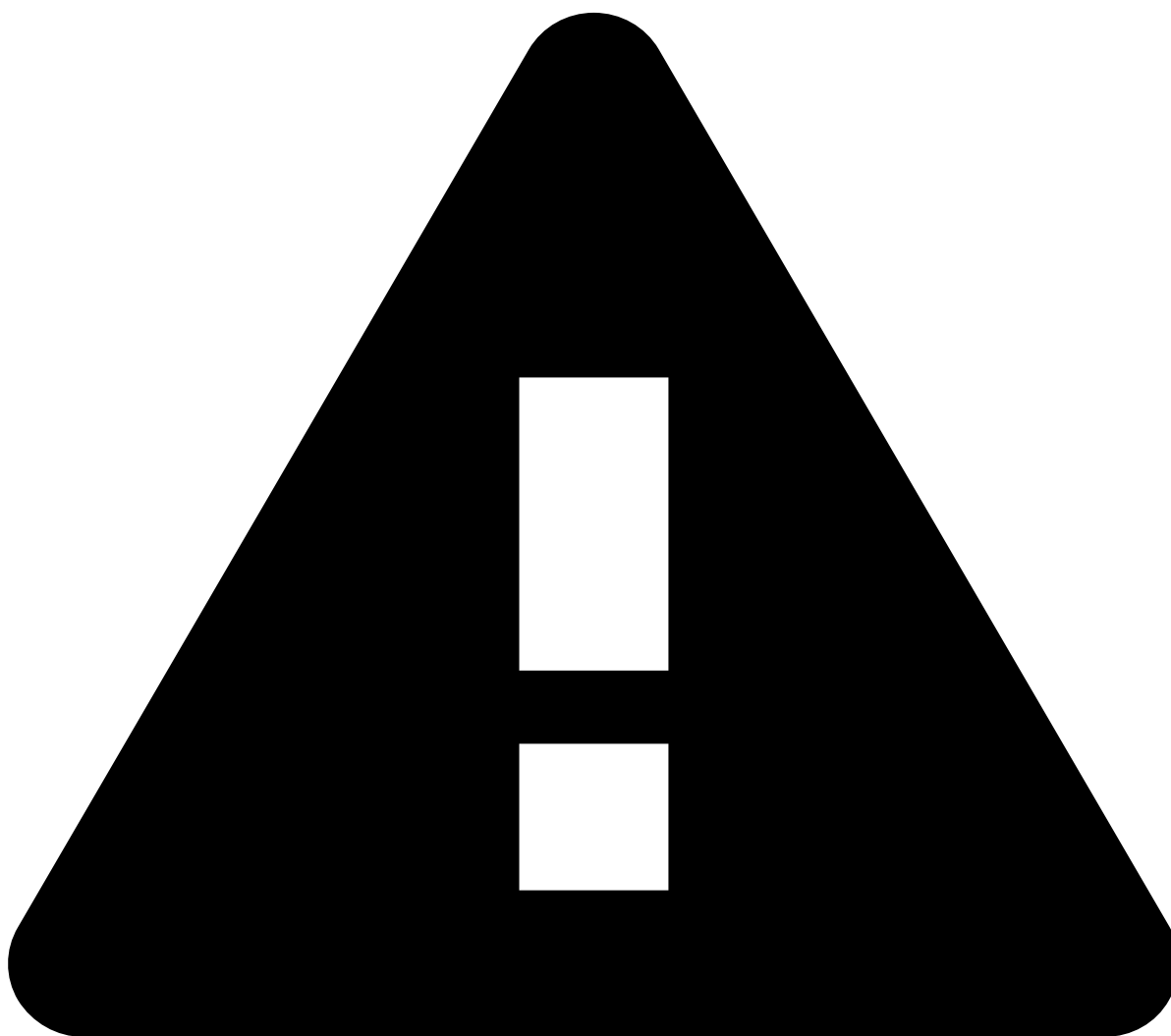
Below are some examples of recovery configurations:

- **last-30m** - With this configuration, if the last event in the event storage is at 16h30, it will recover events from 16h00. If the last event is at 00h10, it will recover events since 23h40 from the previous day. In other words, it always recovers the last 30 minutes counting back from the time of the last event in the event storage.
- **last-48h0m** - Recovers the last two days of events.
- **sinceLast-23h30m** - Using this configuration, if the last event is at 23h31, it will recover events from 23h30 in the present day (in this case 1 minute). If the last event is at 22h00, then it will recover events since 23h30 from the previous day, i.e. the last 22 hours and 30 minutes.

The default value is *last-30m*.

Advanced properties: Checkpoint-based workflow recovery [↗](#)

In a production environment, Pulse can be configured to store the necessary events for real-time metrics calculation on disk. In this case, Pulse creates **checkpoints**, i.e. regular snapshots about the events and the state of the metrics. After a server failure, Pulse can reload the last checkpoint and recover the metrics status starting from there. This saves time when the server needs to be restarted. However, it takes toll on the metrics calculation speed, as Pulse puts metrics calculation on hold while creating a checkpoint. Hence, this setting needs to be carefully tested.



Limitations

Note that this feature is in continuous development, and as of now it does not provide full feature parity to the previous, RAM-based version.

Additionally, if this feature is enabled, do not edit or remove existing metrics. Edited metrics are not recoverable after a server failure, meaning that those metrics will only take new events into consideration after any server restart.

Partition Keys

Allows to configure the partitioning keys for scaling horizontally the workflow processing across Pulse replicas.

Other properties

We also allow to add arbitrary key/value properties to the workflow advanced configuration. The possible properties that are considered here are the ones under the *advanced* portion in Workflow Manager Service Configuration.



info

We recommend using Pulse properties

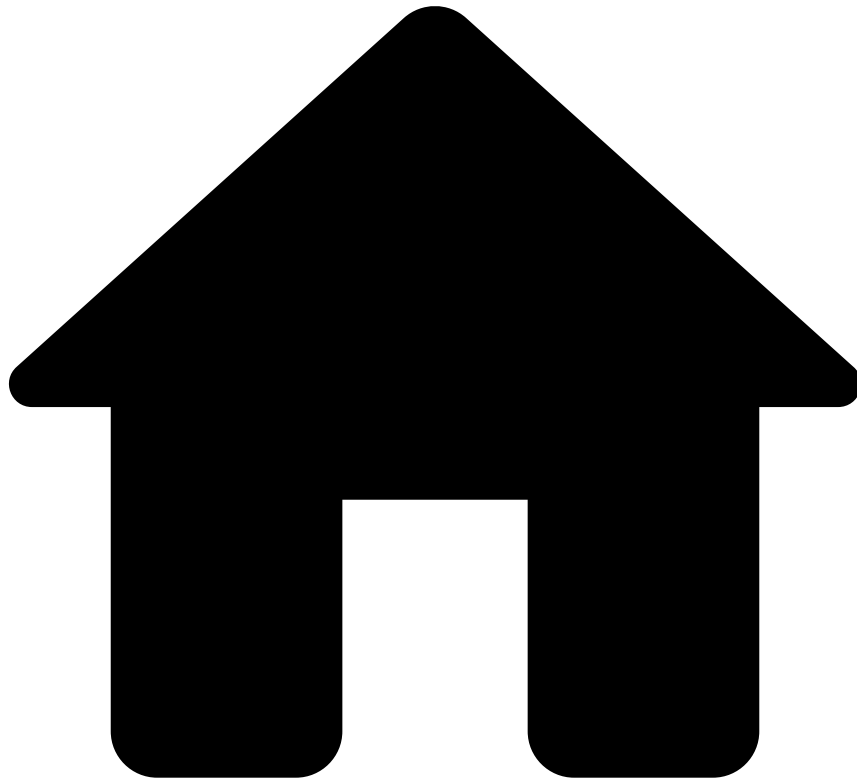
to manage those rather than defining them in the workflow, as doing so makes the properties bound to the application metadata when most of those properties are environment-specific (i.e., they can change from production to development).

Workflow slug

The only property that must be configured in the workflow advanced properties is the workflow slug. This allows to then define different Workflow Manager Service Configuration properties for different workflows (with various slugs) across the same application scoped prefix pulse property.

You can do so by setting the key to *workflowSlug* and the value to the intended slug. Then use that slug where mentioned in the Workflow Manager page.

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Configuring outcomes

On this page

Configuring outcomes

Was this page helpful? 

Why

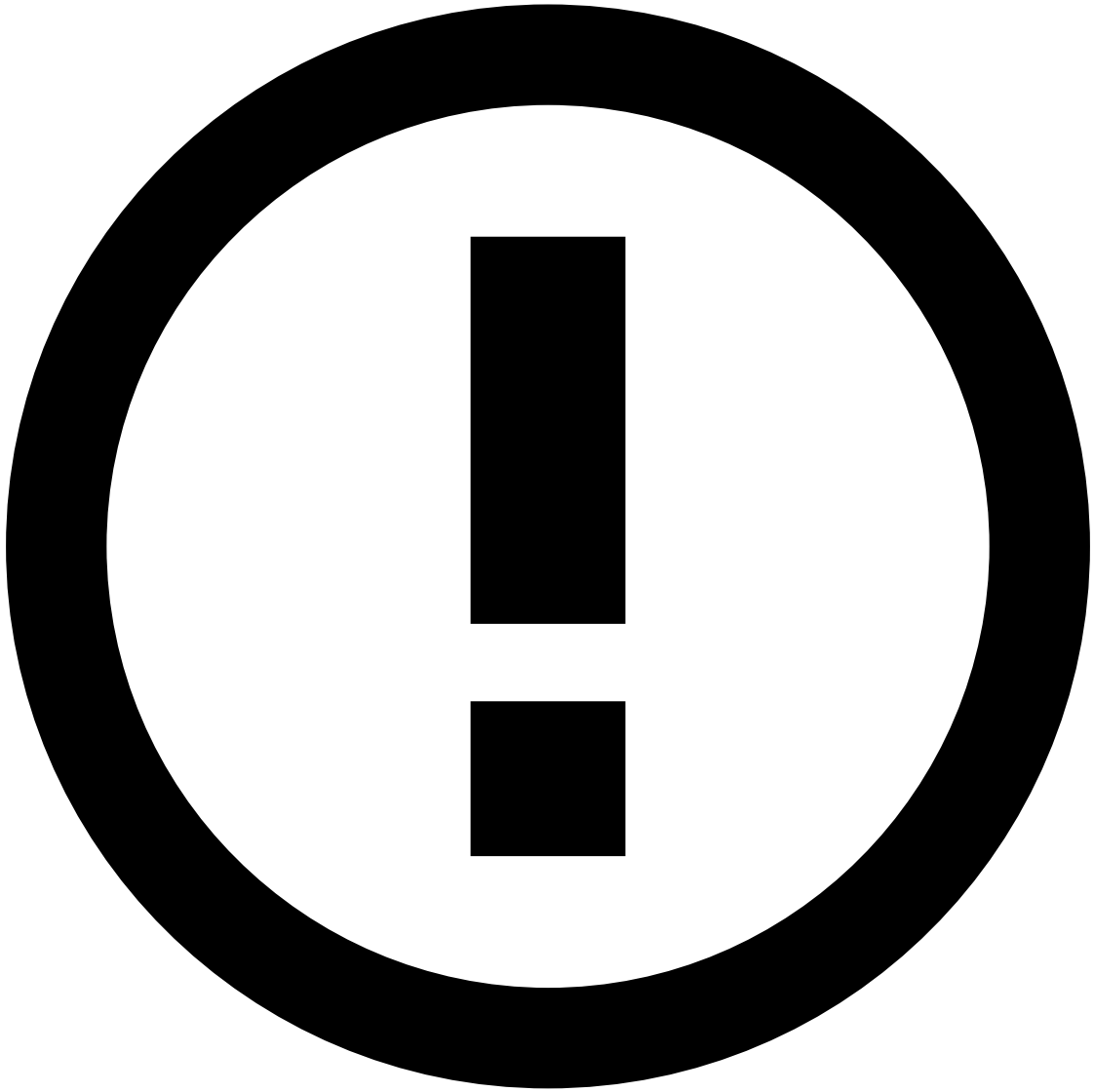
Before modeling your **Workflow** you must define what decisions it will produce as output. Outcomes represent those decisions, for example fraud/not fraud.

Pulse adds each outcome decision field (e.g. *fraud*) and outcome score field (e.g. *fraud_score*) to the schema of events moving along the workflow. As you add elements such as [Scoring Services](#) and [Rules Services](#), you must also configure how they affect these fields, i.e. how they contribute to each of the final decisions and scores of the **Workflow**.

How to

To define the outcomes of your workflow execute the following actions:

1. In the **Workflows** tab, select an existing **Workflow**.
2. the workflow's outcomes. Add a new outcome, and select if it is the outcome of a multi-classification problem or a binary classification.



Multiple outcomes

In the same **Workflow** you can have multiple outcomes to fine-grain the decision and perform classification in complex cases. Adding a second outcome, for example *fraud_type*, would allow you to harvest more information in case the transaction is fraudulent.

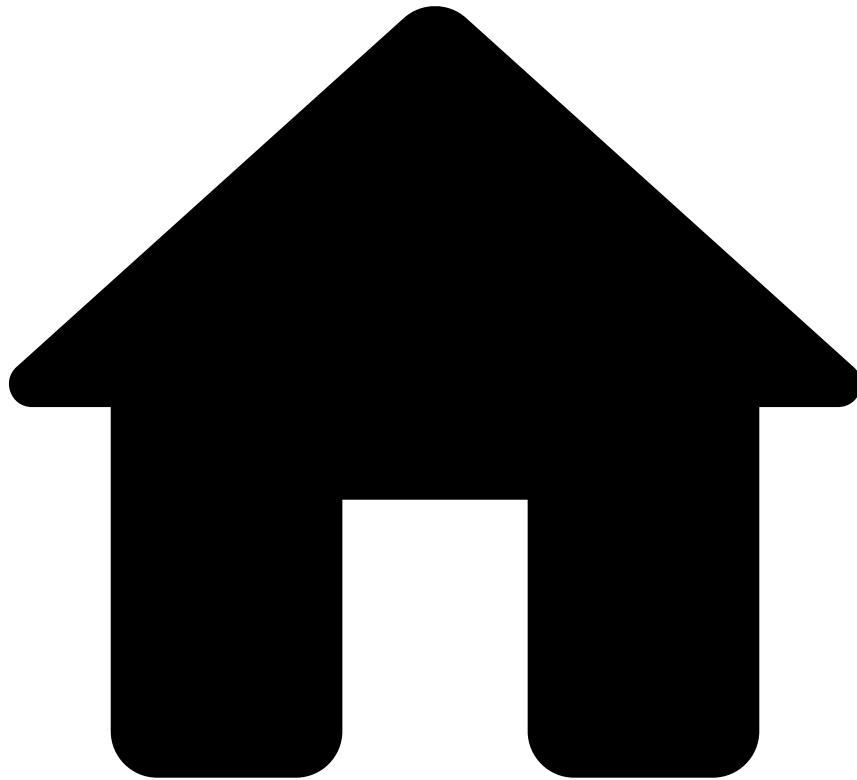
3. In case of a multi-classification problem, type the possible values separated by a comma.
4. Select the default value for the outcome.
5. Add another outcome, or **Save** the workflow outcome definition.

Next Steps

You have now finished performing the initial configurations required for your workflow. Proceed by adding elements to design the workflow. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding rules](#)
- [Adding custom services](#)

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Workflow services properties

On this page

Workflow services properties

Was this page helpful? 🗨️

It is possible to specify the configuration properties of any [Workflow Service](#) (**Input Adapter**, **Plan**, **Scoring**, **Rules** and **Custom**) in several ways. Any configuration property has the following components:

- **Identifier:** All **Workflow Services** have this property (which is `empty` by default). You can assign the same properties to a group of **Workflow Services** that share the same **Identifier** value.
- **Property Base:** All **Workflow Services** have this immutable attribute that defines which is the type of service. For example, a **Plan Workflow Service** has a property base of `services.rte.workflow` whereas **Input Adapters**

and **Custom Workflow Elements** have a different **Property Base** depending on how the underlying Custom Service was configured:

- **Synthetic Generator:** Has the property base `services.rte.workflow.syntheticgenerator`.
- **Custom Services:** Specified using the `@PropertyBase("services.modules.mycustomservice")` annotation in your service's `Configurator` class.

Using Pulse UI

Pulse allows you to manually configure property values and helps you by displaying the following indications:

- **Manually overridden values:**
- **Values inherited from Pulse configuration:**

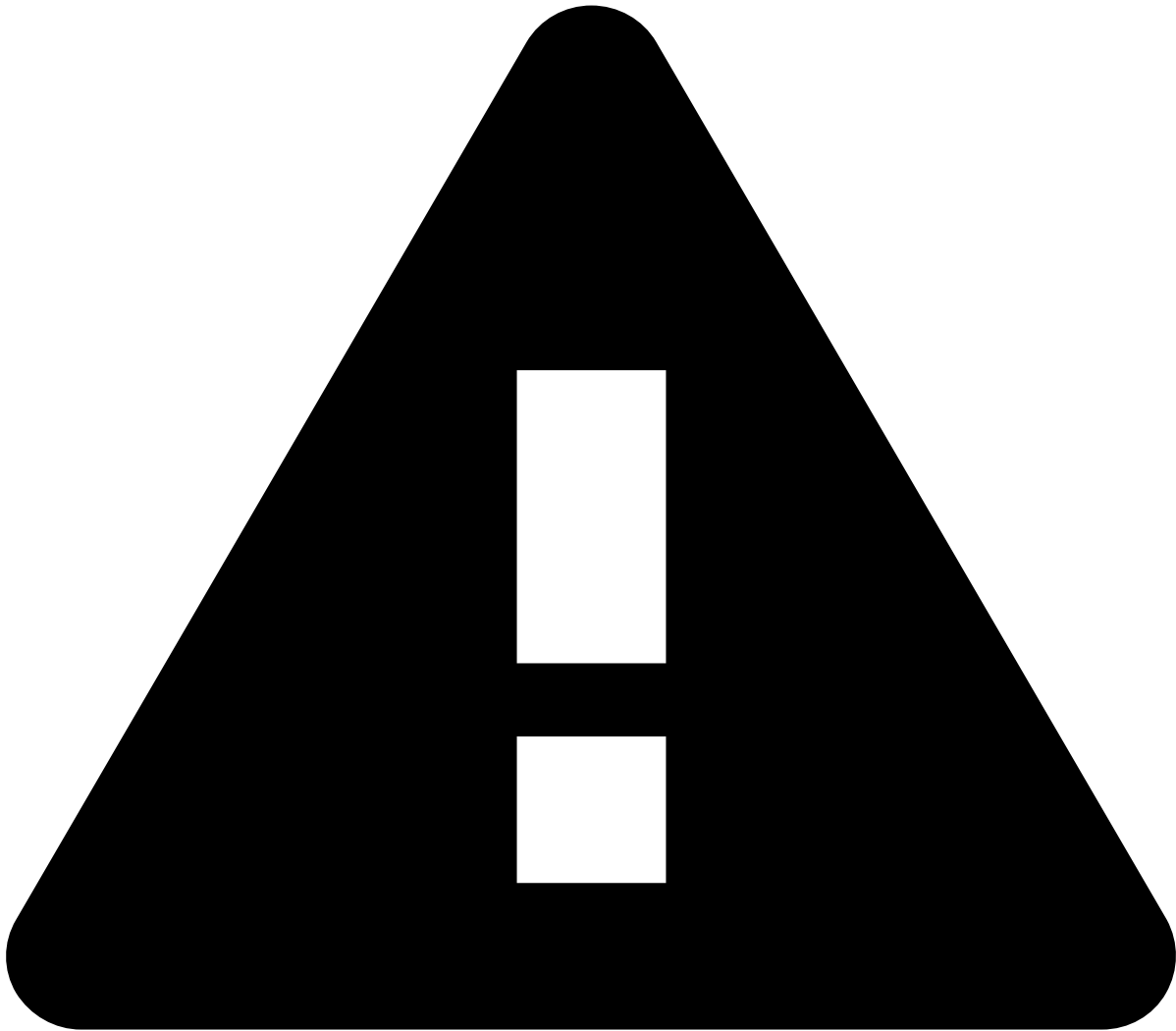
•



- [Feature Engineering](#)
- [Creating Metrics](#)
- Using Cross-channel Metrics

Using Cross-channel Metrics

Was this page helpful? 



caution

Cross-channel metrics are only supported when using the [Railgun Engine](#).

Pulse supports [Cross-channel metrics](#), when using [Railgun](#), and the following section explains how to create these type of metrics, in which only one Pulse application workflow calculates the metric and another workflow can fetch and use the metric value without contributing to it.

If your Risk Strategy needs to use some of these metrics, follow the guidelines below to know how to create them in Pulse.

Assuming that the **secondary workflow** already exists, then, to create some **Cross-channel** metrics via Pulse UI, it's necessary to do the following:

- Create a **schema** for the **Primary workflow**:

Main schema

- Since this is the main schema, it doesn't need to have any fields that the **Cross-channel** metric might use.

- Create an **Omnichannel plan**:

Omnichannel plan

- Since **Cross-channel** metrics is just a subset of **Omnichannel** metrics, the type of the plan should be set as **Omnichannel**.

Make sure to tick the **Omnichannel** as enabled, and configure both schemas properly.

- Create a **Cross-channel** metric:

Crosschannel metric

- Now, when adding a new metric to this **Omnichannel** plan, it will ask to configure both channels.

Note that the fields used by the metric will only be the ones in the secondary channel.

- Then, when saving the **Cross-channel** metric:

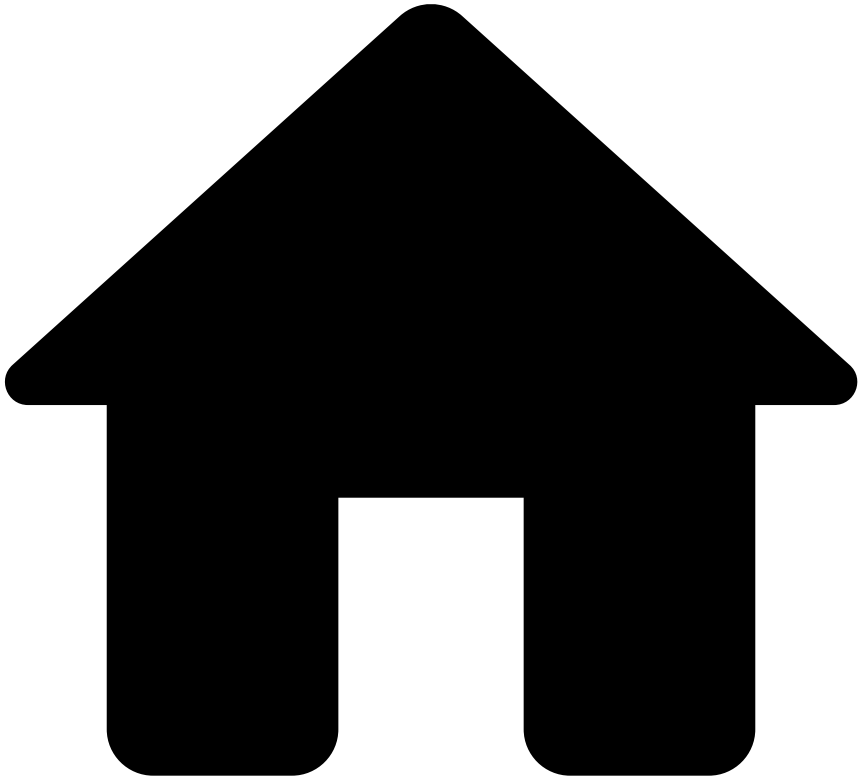
CrossChannel metrics saved

- The metric called `mainfieldd_avg_1h` is a **Cross-channel** metric, meaning that only events from the **secondary workflow** (that use the `secondary_schema` schema) can contribute to the metric, and the saved plan can also use it's value.

- Now, save the **plan** and create a **snapshot**.

- Finally, you can create/edit the **primary workflow** to use the **Omnichannel plan** with the **Cross-channel** metric.

•



- [Feature Engineering](#)
- [Creating Metrics](#)
- Using Lookup Metrics

On this page

Using Lookup Metrics

Was this page helpful? 



Availability of Lookup Metrics

The **Lookup By** operator is only available when using the [Railgun engine](#), and when explicitly enabled via configuration. To activate this functionality, reach out to the Feedzai Customer Success (CS) team.

[Lookup By metrics](#) provide a powerful extension to standard Group By metrics, by allowing users to choose which group's aggregation to retrieve for an event, based on fields other than the groupBy fields.

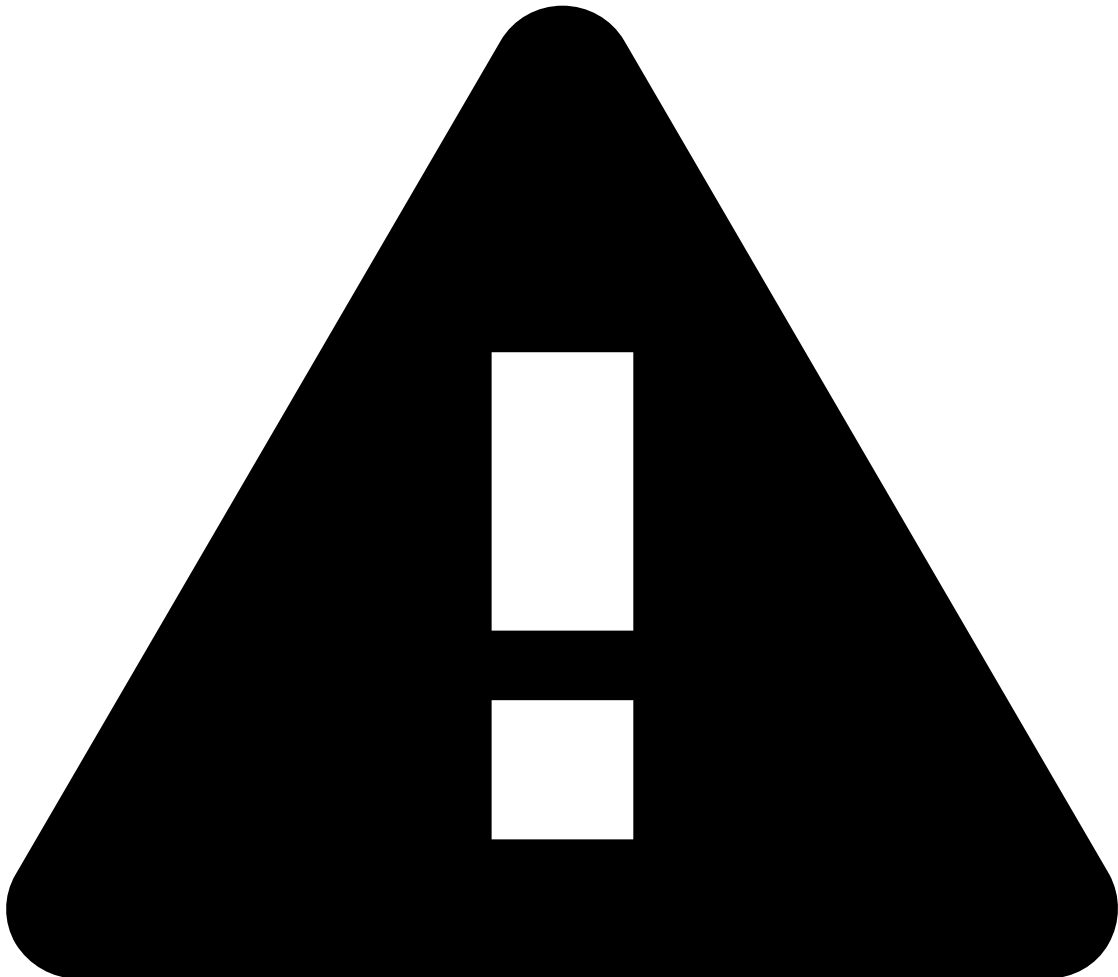
This page describes how to create a Lookup By metric in Pulse, and provides a practical example of how to combine it with standard Group By metrics to calculate new [derived fields](#).

Creating Lookup By Metrics📄

To create a Lookup By metric in Pulse, follow these steps:

1. In the **Data Science** tab, open the features plan.
2. Drag a Metrics operator to the plan if there is none yet.

3. Edit the Metrics operator:
4. Select the [Add new Metric] button to add a new metric to the Metrics operator.
5. Select the **Group By** fields over which to calculate aggregated metrics. For example, *receiver_ID*:
6. Select the [Add look-up by] button to activate the **Lookup By** input field.
7. Select the appropriate look-up fields from the available options. For example, *sender_ID*:



warning

The lookup fields must be in the same number and of the same data type as the group by fields, and arranged in the same order.

8. Define the remaining sections (Window, Aggregation, Description) as needed, according to the regular process for [Creating Metric](#).
9. Save the Metrics operator with look-up configuration.
10. Save the Plan with new metrics.

Combining Lookup By with Group By Metrics

Lookup By metrics are often useful when combined with standard Group By metrics. To do so, add both types of metrics to the same Metrics operator, and combine them with a [Map operator](#).

The following example demonstrates how to set up such a combination of metrics for the use case of calculating an account's recent running balance at each transaction. It assumes that the input data stream contains at least the fields `sender_ID`, `receiver_ID`, and `amount` (in addition to the obligatory `timestamp`).

1. Create (if necessary) a **Metrics** operator as described [above](#), and edit it.
2. Select the [Add new Metric] button and add the first metric to the Metrics operator. This will be a regular **Group By** metric, to get the total amount sent by the transaction's sender.
 1. In the **Group By** section, select the `sender_ID` field.
 2. In the **Window** section, set the window to *7 days*.
 3. In the **Aggregation** section, select the *sum* function over the `amount` field.
 4. Set the **Name** of the metric to `total_sent_1w`.
3. Select the [Add new Metric] button and add the second metric to the Metrics operator. This will be a **Lookup By** metric, to get the total amount received by the transaction's sender.
 1. In the **Group By** section, select the `receiver_ID` field; activate the Lookup By input area as described [above](#), and add the `sender_ID` field to it.
 2. In the **Window** section, set the window to *7 days*.
 3. In the **Aggregation** section, select the *sum* function over the `amount` field.
 4. Set the **Name** of the metric to `total_received_1w`.
4. Review the **Metrics** operator with the two metrics. Confirm that the grouping and lookup fields are correctly set for each metric by hovering over the  icon next to each field in the **Group By** column.

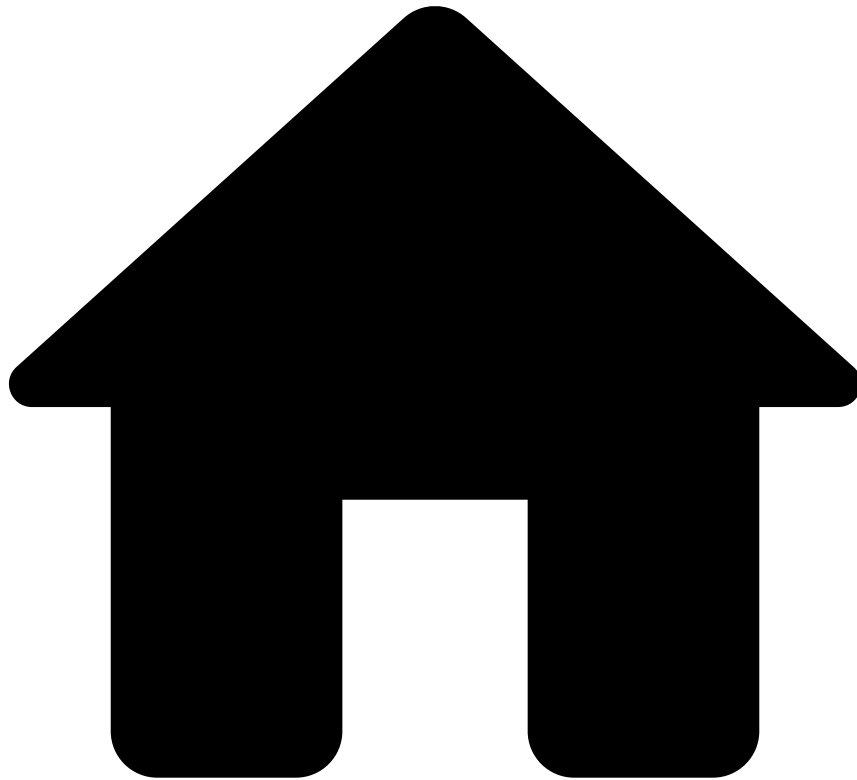
Then, [Save] the Metrics operator.

5. Add a **Map** operator to the plan by dragging the [Map] button to the plan's diagram, after the Metrics operator.
6. Edit the Map operator. In the **Fields To Add** tab, create a new field as follows:
 1. Set the **Output Field Name** to `balance_1w`,
 2. Set the **Expression** to `total_received_1w - total_sent_1w`.
 3. Leave the **Type** as the default *Numeric*.

Finally, [Save] the Map operator.

By completing these steps, the plan will now add the `balance_1w` field to each event, representing the account's running balance over the past week at the time of the transaction.

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Conditional Expressions in PQL

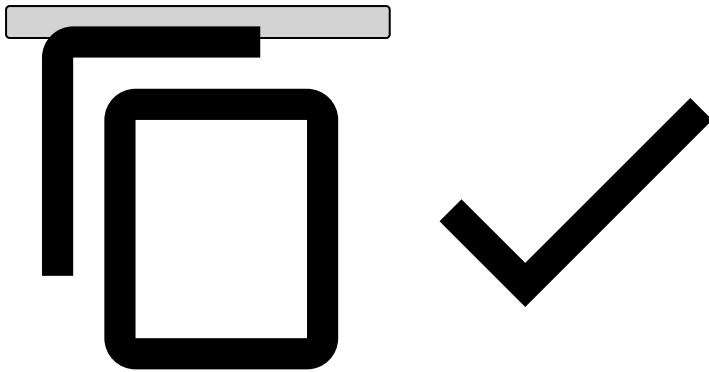
Conditional Expressions in PQL

Was this page helpful? 

Conditional expressions of PQL return a value based on a condition.

Syntax:

```
if <condition> then <then-case-expression> else <else-case-expression>
```



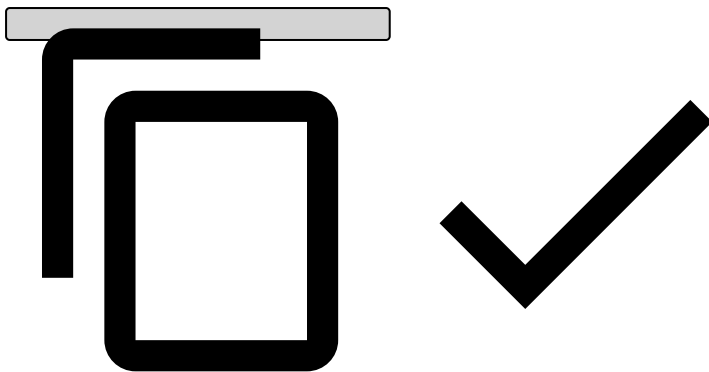
Where:

- `<condition>` is a boolean expression.
- `<then-case-expression>` and `<else-case-expression>` are obligatory values.

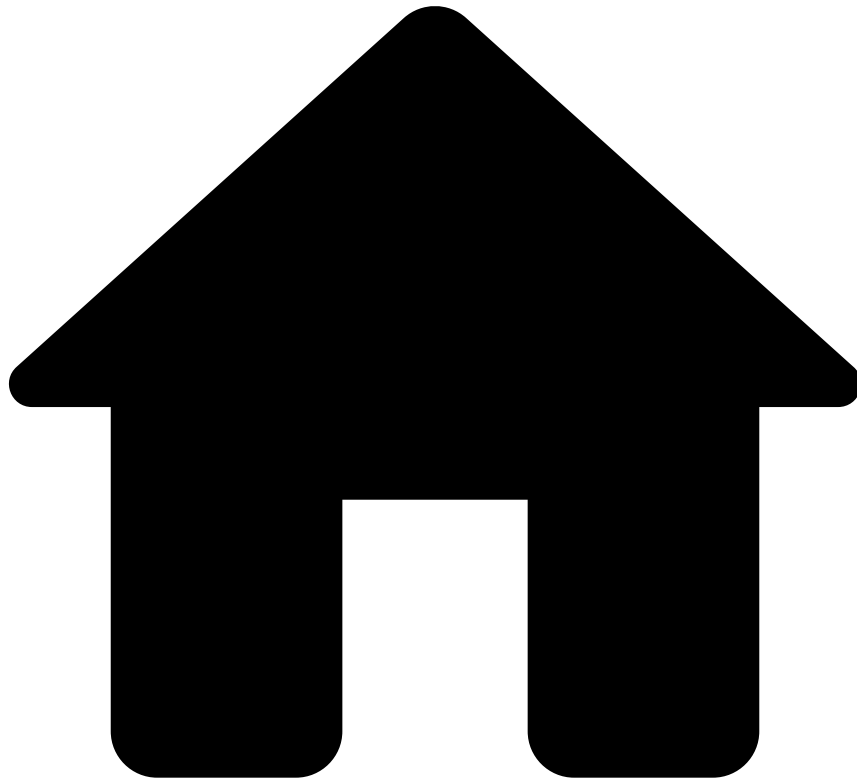
If the condition is true, the conditional expression returns the value of the then-case-expression, otherwise the value of the else-case-expression.

Example:

```
if( P0SEntryMode == 'Swiped' or P0SEntryMode == 'Keyed' or P0SEntryMode == 'Chip' )  
  then 'Card Present'  
  else P0SEntryMode
```



•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Function Calls in PQL

On this page

Function Calls in PQL

Was this page helpful? 

Function calls allow you to execute predefined functions on current values. There are two types of functions that can be called:

- [Built-in functions](#) of Pulse
- [User-Defined Functions \(UDFs\)](#) â a set of additional functions bundled with Pulse

This topic gives an overview of the possible function calls, and points you to the right topics to learn more.

Built-in functions

Checking NULLs

Return type	Method	Description
bool	isNull(int? a)	Checks if an integer value is null.
bool	isNull(long? a)	Checks if a long value is null.
bool	isNull(double? a)	Checks if a double value is null.
bool	isNull(float? a)	Checks if a float value is null.
bool	isNull(bool? a)	Checks if a boolean value is null.
bool	isNull(string? a)	Checks if a string value is null.
bool	isNull(tuple? a)	Checks if a tuple value is null.

Math functions

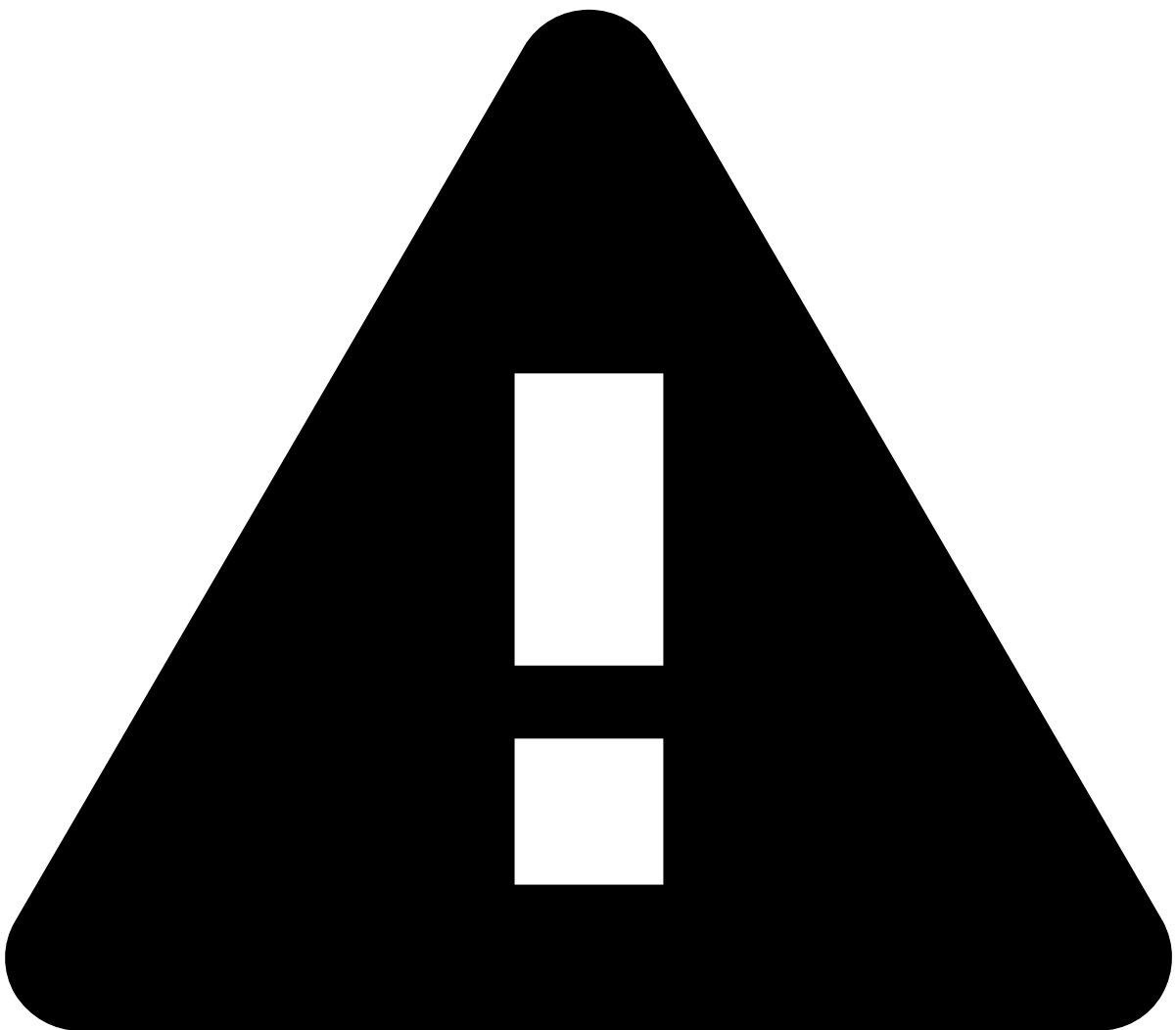
All methods of the [java.lang.Math](#) package are available in PQL. Besides those methods, you can use the following math functions:

Return type	Method	Description
int	max(list_of_int)	Returns the maximum value of a list of numbers.
long	max(list_of_long)	Returns the maximum value of a list of numbers.
double	max(list_of_double)	Returns the maximum value of a list of numbers.
float	max(list_of_float)	Returns the maximum value of a list of numbers.
int	min(list_of_int)	Returns the minimum value of a list of numbers.
long	min(list_of_long)	Returns the minimum value of a list of numbers.
double	min(list_of_double)	Returns the minimum value of a list of numbers.
float	min(list_of_float)	Returns the minimum value of a list of numbers.
boolean	between(int val, int left, int right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	between(long val, long left, long right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	between(double val, double left, double right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	between(float val, float left, float right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.

Strings

Return type	Method	Description
string	str(int a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(long a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(double a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(float a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(bool a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(string a)	Returns a new string that is the textual representation of <code>a</code> .

Return type	Method	Description
int	indexOf(string s, string c)	Returns the index within string <code>s</code> of the first occurrence of the string <code>c</code> .
string	substring(string s, int begin)	Returns a new string that is a substring of string <code>s</code> .
string	substring(string s, int begin, int end)	Returns a new string that is a substring of string <code>s</code> .
bool	contains(string s, string c)	Returns <code>true</code> if string <code>c</code> is a substring of string <code>s</code> , or <code>false</code> otherwise.
bool	startsWith(string s, string c)	Returns <code>true</code> if string <code>c</code> is a prefix of string <code>s</code> , or <code>false</code> otherwise.
bool	endsWith(string s, string c)	Returns <code>true</code> if string <code>c</code> is a suffix of string <code>s</code> , or <code>false</code> otherwise.



Container functions in `Map` and `Metrics` operators

Functions that operate on collections (like `List` s, `Set` s or `Map` s) are not supported in the PQL fields of [Map Operators](#) or [Metric Operators](#).

For example, using `contains` or `indexOf` on collections in PQL fields of such operators will result in an error.

The recommended alternative is to use [UDFs \(User Defined Functions\)](#) instead of these built-in functions.

UDF calls

Pulse comes with a set of [User-Defined Functions \(UDFs\)](#) that can be used in addition to the built-in functions.

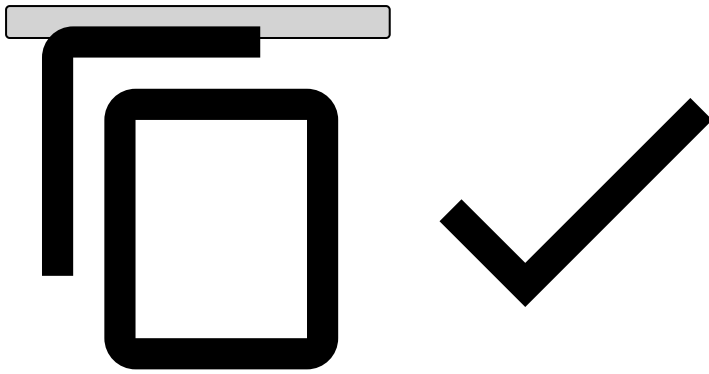


UDFs in Metrics

When using UDFs in metric filters and aggregations, UDFs receive fields from a single event and not from a collection of events.

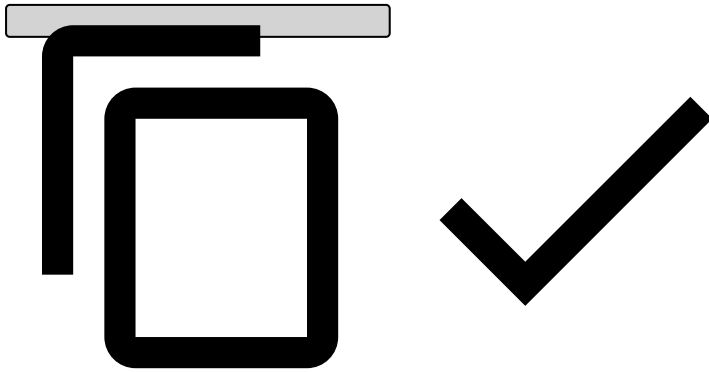
You can use the following syntax to call a UDF:

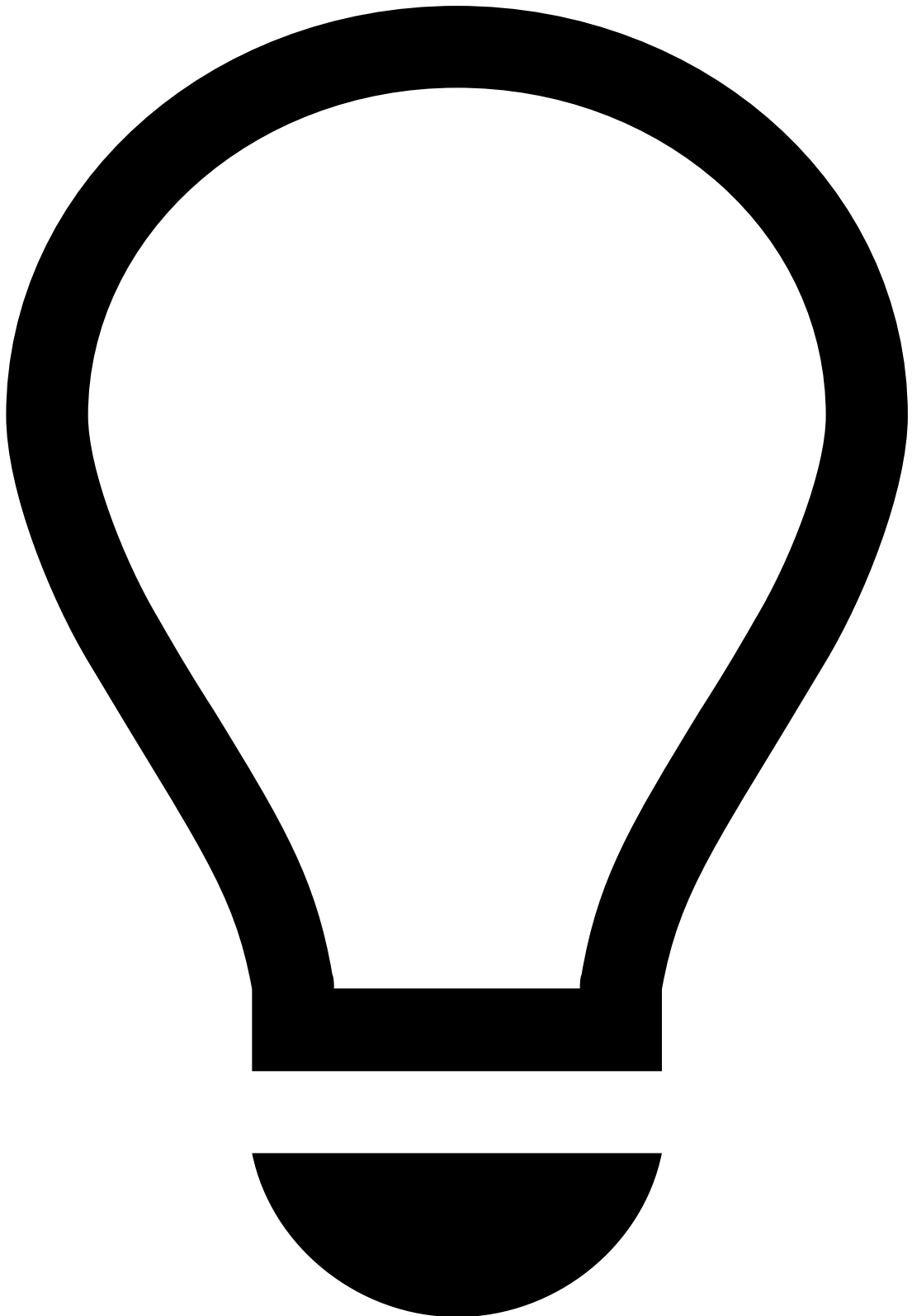
```
<class_name>.<function_name> ( <arguments> )
```



For example, to concatenate two strings:

```
Strings.strcat("One", "Two") // Returns "OneTwo"
```





java regex syntax

Pulse uses the Java regex flavor for UDFs, which means that all regular expressions must conform to the syntax and rules of Java's `java.util.regex.Pattern` class.

According to the usual regex syntax, when you need to match a literal special character that has meaning in regex (e.g. `*`, `+`, `?`, `.`), you must escape it using a backslash (`\`) within the regex pattern (e.g. `\*`, `\+`, and so on).

However, because the regex pattern itself is often passed as a Java String literal, the backslash (`\`) used for escaping the regex special character must itself be escaped according to Java String rules.

Therefore, to include an escaped character like `\*` (match literal asterisk) in a regular expression defined as a Java string, you must write it as `\\*`.

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Handling NULL Values with PQL

On this page

Handling NULL Values with PQL

Was this page helpful? 

NULL is a special value that represents the absence of a valid value. Any operation that acts on it as if it was a valid value fails with a `NullPointerException`. Untreated datasets might contain NULL values, and so they are potentially unsafe for any activity within Pulse. This topic describes how to make datasets more robust by handling NULL values using PQL expressions.

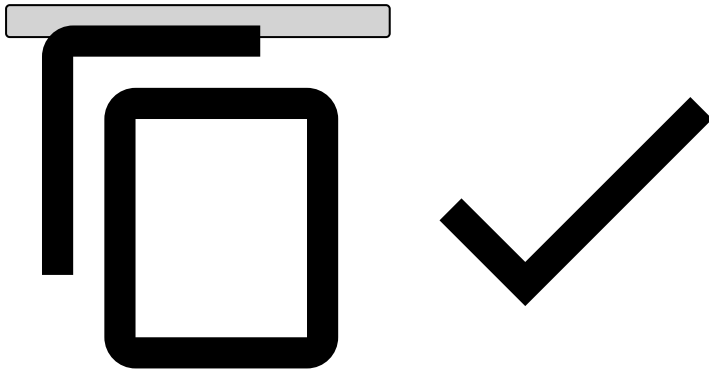
Replacing NULL values

One of the common ways to handle NULLs is to replace them with a value that fits into the data field's type. It's common to pick the substitute value so that it doesn't represent a meaningful value for that data field. For instance, a NULL in the

transaction amount field could be replaced by 0. To accomplish this replacement, you can use the **coalesce operator** (??) in a map operator or metric operator.

Usage of the ?? operator:

```
<good-case-value> ?? <null-case-value>
```



This expression returns `<good-case-value>` , unless `<good-case-value>` is NULL. If `<good-case-value>` is NULL, the expression returns `<null-case-value>` .

Example:

Coalescing aggregations to a value other than 0

Aggregations return 0 instead of a NULL when there are no values to count the aggregation on. For example, when the maximum amount is queried over a time window using a filter that filters out all the records.

In some cases you might want to return something different from 0.

The following example shows how to return -1 in case an aggregation of a metric is called on an empty subset of data:

Nullable types

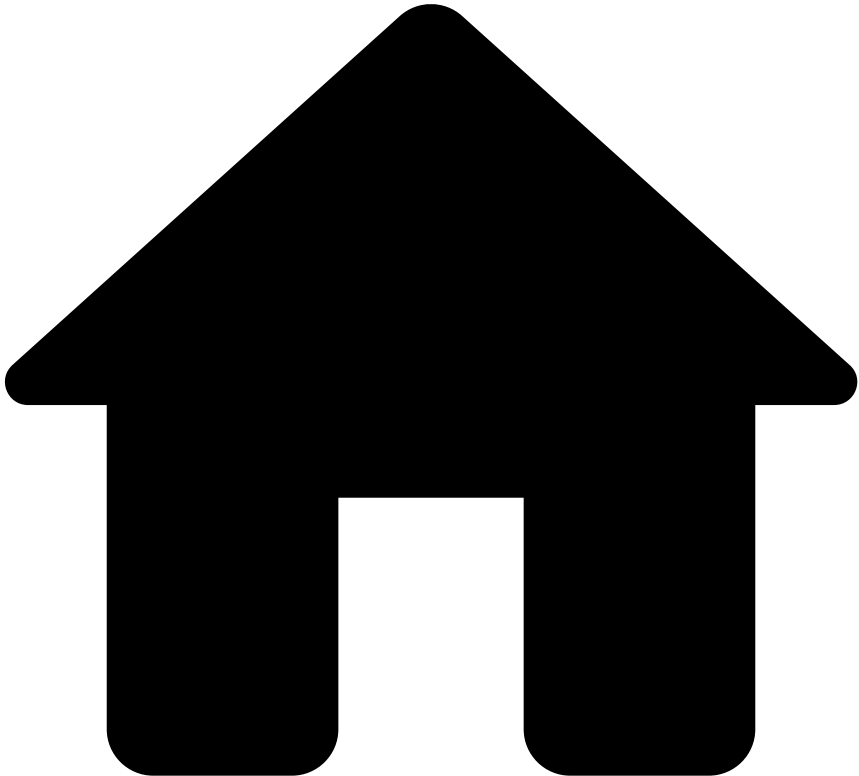
Untreated input fields can contain NULL values. Hence, PQL treats every input field as a nullable type, meaning a type that can contain NULL values in addition to an original data type.

This fact is marked with a ? in the data type name.

For example, the *Amount* field is not of *double* type, but of *double?*, meaning a type that contains all double values plus the NULL.

When using both nullable and non-nullable types in an [arithmetic operator](#) or in a [function](#) that returns a numeric value, the resulting data type is nullable.

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Operators in PQL

On this page

Operators in PQL

Was this page helpful? 

This topic gives you an overview of the arithmetic and boolean operators that you can use in PQL expressions, and the precedence of all operators.

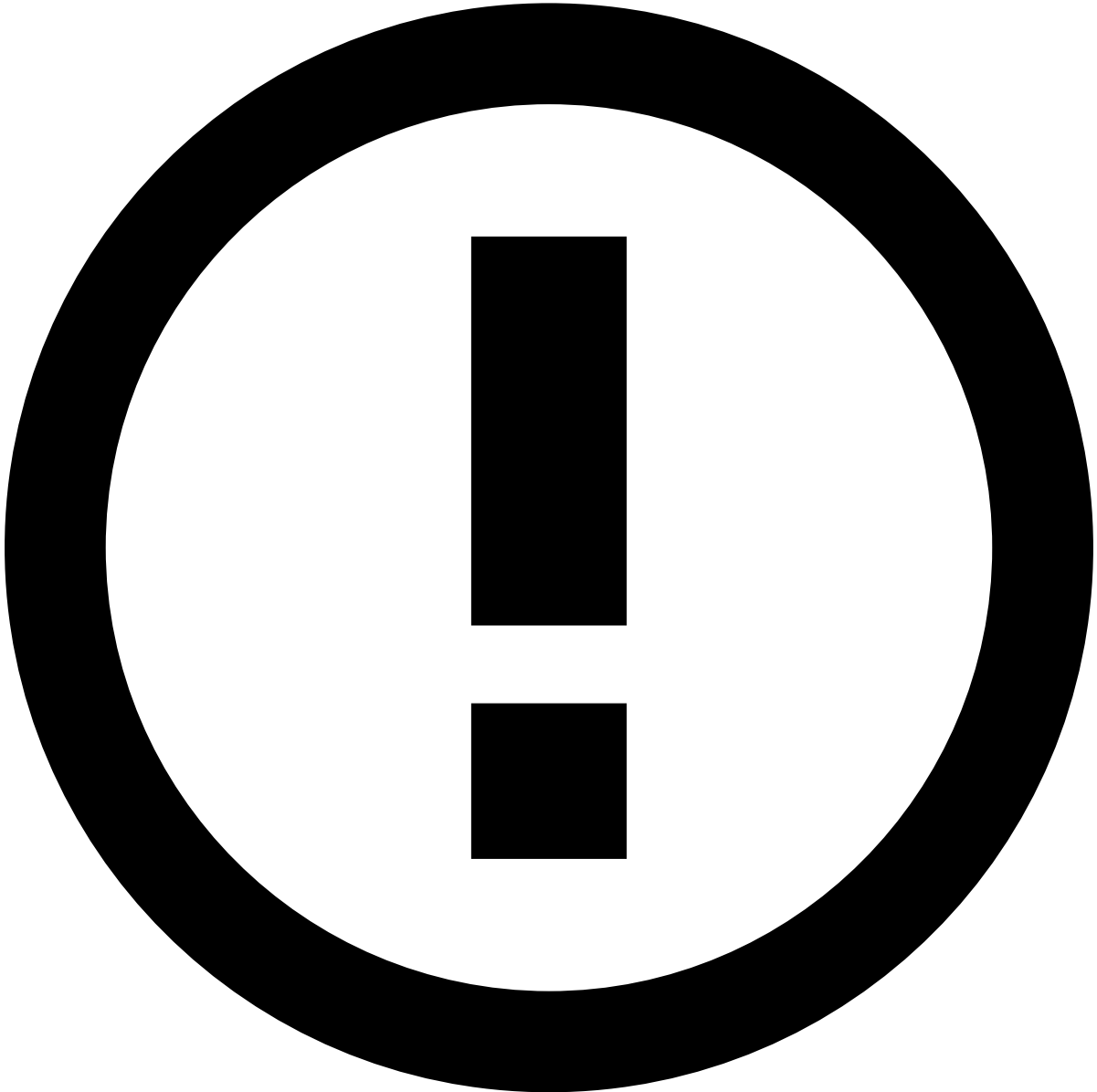
Arithmetic operators

PQL uses the following arithmetic operators:

Operator	Meaning
+	addition

Operator	Meaning
-	subtraction
*	multiplication
/	division
%	integer remainder
^	exponentiation

Example: `Balance - Amount`



Type promotion when using arithmetic operators

The arithmetic operators obey the following type promotion rules:

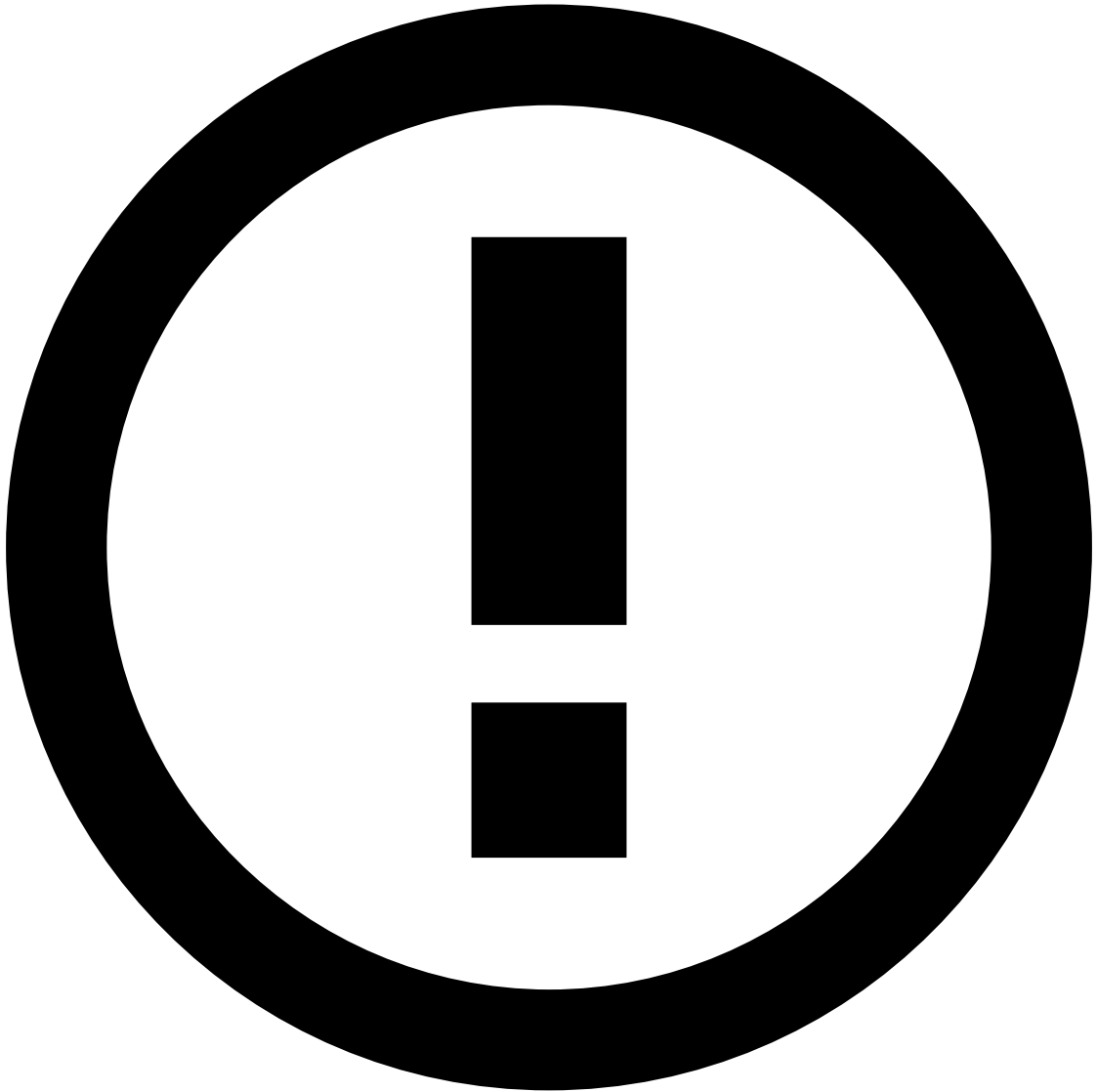
1. If at least one of the arguments is a double, the result is a double.
2. If at least one of the arguments is a float, the result is a float, unless the result is a double because of the first rule.

3. If at least one of the arguments is a long, the result is a long, unless the result is a double or float because of the first two rules.

Boolean operators📖

Boolean operators are operators that return a boolean value. They include:

- `<` , `<=` , `==` , `!=` , `>=` , `>`



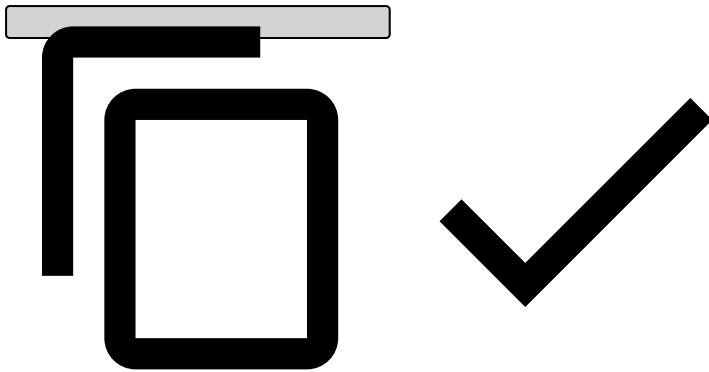
info

Only numeric values can be compared with the `<`, `<=`, `>=`, and `>` operators.

- `and` , `or` , `!` (not)

Example:

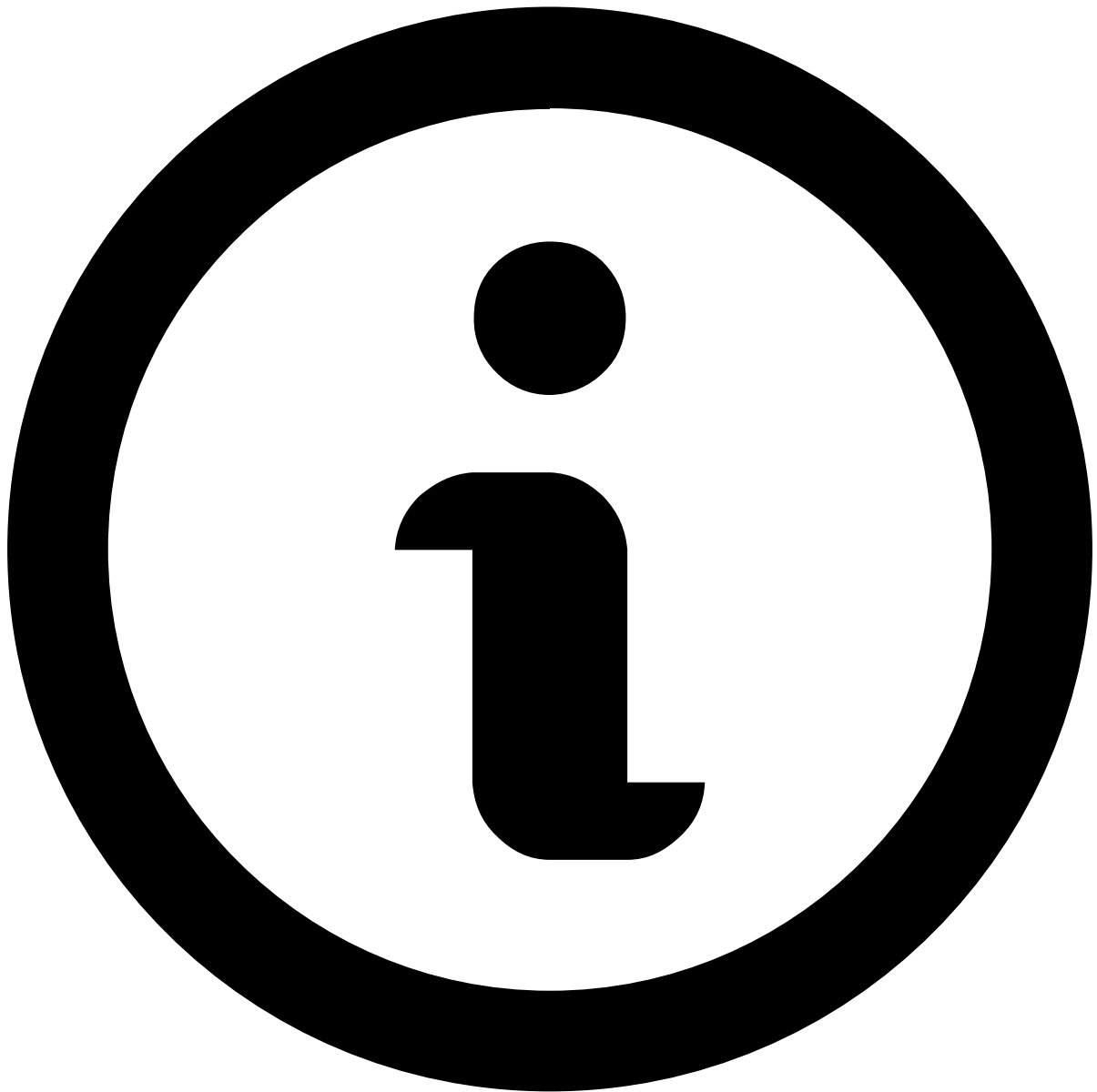
```
P0SEntryMode == 'Swiped' or P0SEntryMode == 'Keyed' or P0SEntryMode == 'Chip'
```



Operator precedence

The following table lists PQL operators according to their precedence order. Operators at the top of the table have higher precedence.

Operator type	Operator(s)
grouping	(expr)
unary	+expr , -expr , !
arithmetic - exponential	^
cast	exp:type
arithmetic multiplicative	*, / , %
arithmetic additive	+, -
coalesce	?? Note that this operator has a different precedence in RDL.
comparison	< , <= , == , >= , > , !=
logical AND	and
logical OR	or
assignment	=



Cast operator precedence

Notice that the *cast* operator precedence lies between arithmetic operators. If an expression needs to be cast before an exponentiation, it needs to be enclosed in parentheses (*grouping operator*).

The following rules apply:

- operators with higher precedence than other operators are evaluated before them;
- operators on the same line have the same precedence:
 - all binary operators except for the assignment operator are evaluated from left to right;
 - assignment operator is evaluated right to left.

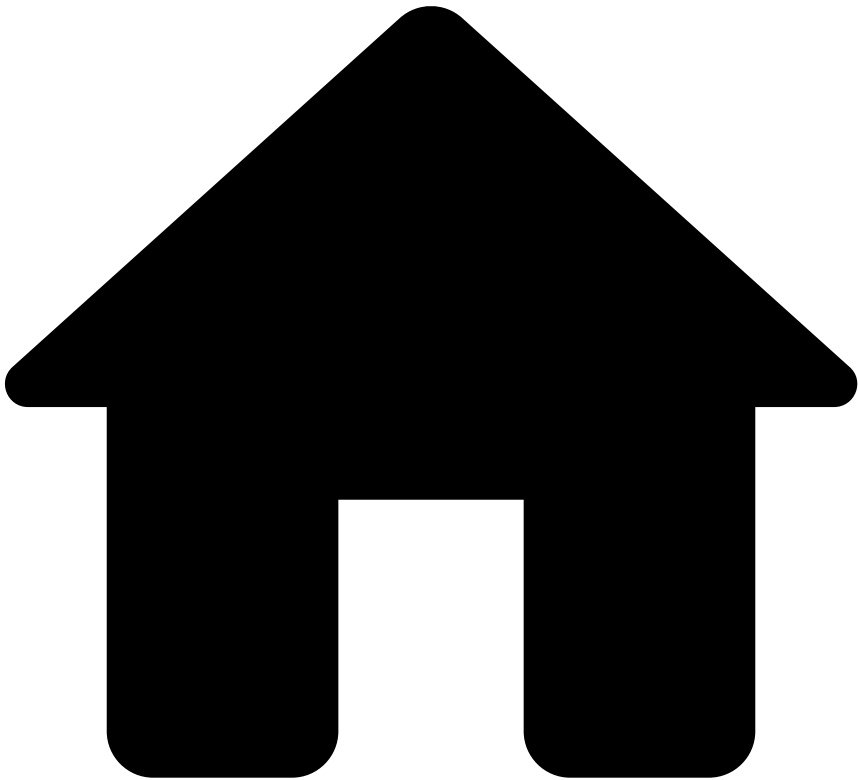
Examplesâ◻◻

Arithmetic and boolean operators of PQL follow the common rules of mathematics regarding functionality and precedence.

Arithmetic and boolean operators have higher precedence than the [?? \(coalesce\) operator](#), meaning that the [??](#) operator gets evaluated after arithmetic and boolean operators.

Example: $a + b \leq c + d$ is equivalent to $(a+b) \leq (c+d)$.

.



- [Feature Engineering](#)
- [Using PQL in Features](#)
- PQL Data Types And Conversions

On this page

PQL Data Types And Conversions

Was this page helpful? 

This topic gives you an overview of data types in PQL, and how to convert data from one data type to another.

PQL uses the following **primitive data types**.

Data type	Examples
int	-289 0

Data type	Examples
	42
long	123456789 1L -2L
float	0.99 -1.23e-100f 3.14F
double	1234567.89012 1.23456e300d 1.23456e-300d 1e1D
bool	true false
string	"John Doe" 'John Doe'

These data types are equivalent to the Java primitive data types. To see the exact definition or range for each, check the [Java documentation](#).

The **non-primitive data types** of PQL are:

- Tuples
- Collections:
 - Lists
 - Sets
 - Maps

Do not confuse these types with lists and map operators that you can create in the Pulse UI. They belong to two different levels of implementation. The non-primitive PQL data types are used only in PQL queries, when computing features. When the PQL computation is finished, Pulse stores the data of these non-primitive types as *Complex* fields in the output data source.

Data type mapping between Pulse and PQL

The following table shows to which PQL data type each [Pulse data type](#) is mapped when using input data fields:

Pulse field type (input)	PQL type
Categorical	string
Complex	the same non-primitive type of PQL

Pulse field type (input)	PQL type
Date Time	long (UNIX timestamp format)
Numeric	int, long, float, double
String	string

The following table shows which PQL data type can be passed as output to which type of output fields. The output field is the data field that the map or metric operator is creating/updating, and its type is one of the [Pulse data types](#).

PQL type	Pulse field type (output)
any data type	Categorical
int, long, float, double	Numeric
long, int (UNIX timestamp format)	Date Time
any non-primitive type	Complex
string	String

Converting data

In PQL, each data field, constant, variable, or aggregation result is of a primitive data type. To convert from one data type to another, you can use the following options:

- Cast operator, to cast a value of a numeric type into another numeric type. Learn more about this operator in the next section of this page.
- The `str()` [built-in function](#) to convert a numeric value to a string.
- [User-defined functions \(UDFs\)](#). There are [built-in UDFs](#) that can be used after creating an alias for them. Example: to convert the merchant category code to integer, you can use the `DataConversionTools.stringToInt()` function.

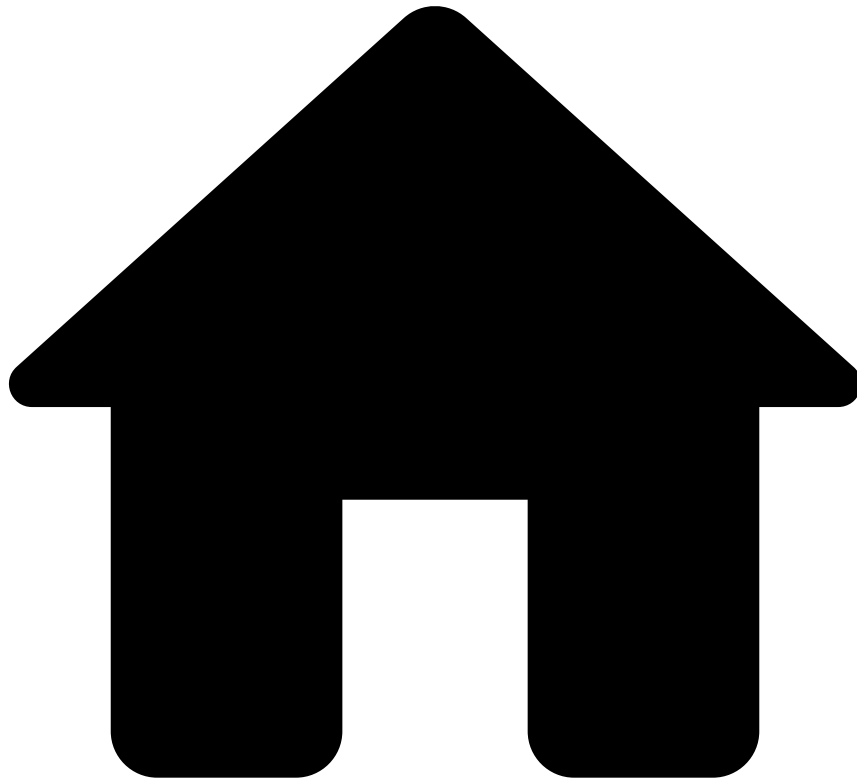
Cast operator

The cast operator allows you to cast values from a numeric type (int, long, float or double) to another numeric type. There is no built-in cast operator between numeric types and booleans or strings.

To cast a value, use the following syntax: `<value to convert>:<target type>`

Example: `Amount:int` is the value of the Amount field cast to integer.

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- PQL in Derived Features

On this page

PQL in Derived Features

Was this page helpful? 

Derived features are new data fields that were calculated based on the original data set.

To create derived features, there are two different cases:

- Computing data based on a single event (not on aggregated data), using a map operator.
- Calculating a value based on an aggregation of events, often combining map and metrics operators.

This topic describes how to use PQL in both cases.

Derived fields without aggregation

To calculate new values based on input fields, you can use **map operators** in plans. For that, add a map operator to the plan that processes the input data, and add the PQL expression in the *Expression* field.

You can use any of the [building blocks of PQL](#), except for the aggregations.

Example

The following example adds a field called *CountryMatch* that shows if the merchant country is the same as the country where the card was created.

Derived field based on aggregations

To aggregate data, use a **metrics operator** in the plan. The UI of the metrics operator allows you to use a shorter list of aggregations: Avg, Min, Max, Count, Count Distinct, Sum and Standard deviation. Other calculations need PQL statements.

PQL allows you to use the following **aggregations** in the *Expression* field of an aggregation when editing a metrics operator:

Aggregation	Description	Usage example
avg (field_name)	Returns the average value of the selected numeric field from the aggregated data set.	<code>avg(Amount)</code>
count ()	Returns the number of rows in the aggregated data set.	<code>count()</code>
max (field_name)	Returns the maximum value of the selected numeric field from the aggregated data set.	<code>max(Amount)</code>
maxBy (field_name)	Returns the event containing the maximum value in the given numeric field.	<code>maxBy(Amount).MerchantId</code>
min (field_name)	Returns the minimum value of the selected numeric field from the aggregated data set.	<code>min(Amount)</code>
minBy (field_name)	Returns the event containing the minimum value in the given numeric field.	<code>minBy(Amount).MerchantId</code>
stddev (field_name)	Returns the standard deviation of the selected numeric field's values within the aggregated data set.	<code>stddev(Amount)</code>
sum (field_name)	Returns the sum of the selected numeric field values in the aggregated data set.	<code>sum(Amount)</code>

Navigate on an aggregated data set

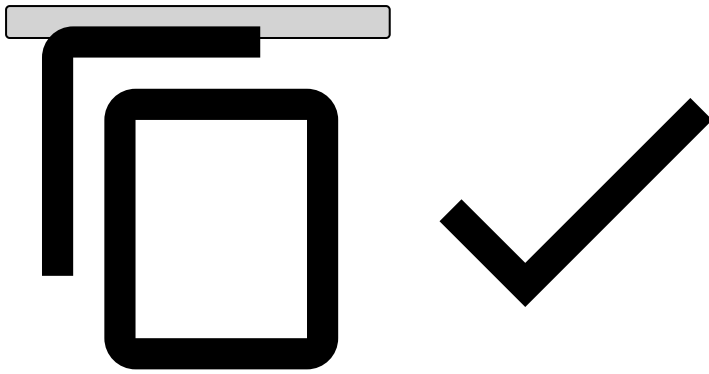
The `first()`, `last()`, and `prev()` methods allow to access the first, last, and previous row of the aggregated data set.

- The current record is the last row of the current dataset if it entered the window, depending on the filters.
- **first()** is the first transaction in the current window.
- **last()** is the last transaction in the current window.
- **prev()** is the transaction before `last()` that entered the window.

Examples

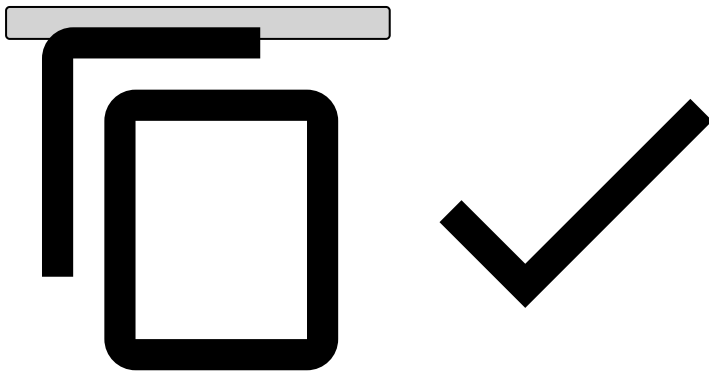
To calculate the geographical distance traveled between two transactions, you can create the following aggregation in a metrics operator:

```
DistanceUtils.haversine(latitude, longitude, prev().latitude, prev().longitude)
```



To calculate the time span between the first and last transaction within a given window per merchant, you can create the following aggregation in a metrics operator:

```
( let first_timestamp = first().Timestamp ?? 0 in
  let last_timestamp = last().Timestamp ?? 0 in
  ( last_timestamp - first_timestamp ) / 60*60*1000.0
)
```



•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Reserved Keywords in PQL

On this page

Reserved Keywords in PQL

Was this page helpful? 

Some keywords and symbols cannot be used outside of their original purpose. That means that within a PQL expression, they cannot be used as name of variables, fields or UDFs.

Besides the reserved keywords and symbols, there is a third list of words that cannot be used in PQL expressions due to limitations related to Java.

Reserved keywords

Keyword	Purpose
and	Comparison operator
as	Import aliasing
asc	Order criteria (ascending)
bool	Boolean type
by	Field descriptor (i.e. group by, order by)
desc	Order criteria (descending)
double	Double type
else	Conditional branching
float	Float type
from	Select descriptor (i.e. select from)
group	Grouping operator
if	Conditional branching
import	Import statement for UDFs
in	Variable binding (i.e. let a = 1 in)
int	Integer type
into	Select target (i.e. group by price into ev)
join	Join operator
let	Variable binding (i.e. let a = 1 in)
limit	Limit operator (i.e. limit 1, 10)
long	Long type
on	Join operator
or	Comparison operator
order	Order operator
select	Select operator
Stream	Stream declaration
string	String type
then	Conditional branching
unit	Unit type
where	Select discriminator

Reserved symbols

Symbol	Purpose
.	Field accessor
+	Addition
-	Subtraction
*	Multiplication
/	Division

Symbol	Purpose
%	Division remainder (mod)
^	Exponentiation
<	Less than
<=	Less than or Equal to
=	Assignment
==	Equal
!=	Different
>=	Greater than or Equal to
>	Greater than
{	Block separation
}	Block separation
:	Casting, selection, etc.
;	Statement separation
@	Annotation
??	Coalesce operator

Java keywords

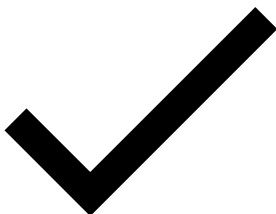
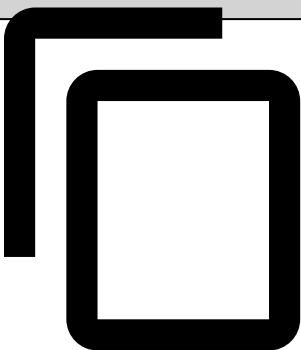
For internal compatibility reasons, the following Java keywords are not allowed:

```

abstract
assert
break
byte
case
catch
char
class
const
continue
default
do
enum
extends
final
finally
for
goto
implements
instanceof
interface
native
new
package
private
protected
public
return
static
strictfp
super
switch
synchronized

```

```
this  
throw  
throws  
transient  
try  
void  
volatile  
while
```



•



- [Model Guide](#)
- [Evaluating a Model](#)
- Handling Feature Importance errors

On this page

Handling Feature Importance errors

Was this page helpful? 

Why

When the computation of Feature Importance is not successfully executed, Pulse returns an error message. This can occur, for example, when the sample percentage is not low enough in a large dataset.

To compute Feature Importance all data is loaded in memory, and thus, high Sample Percentage values may lead Pulse to run out of memory in the case of extremely large datasets.

How to

When the computation of Feature Importance fails a label in the **Model Evaluation** list shows the status "FI:Error" as shown in the figure below:

To solve the error execute the following actions:

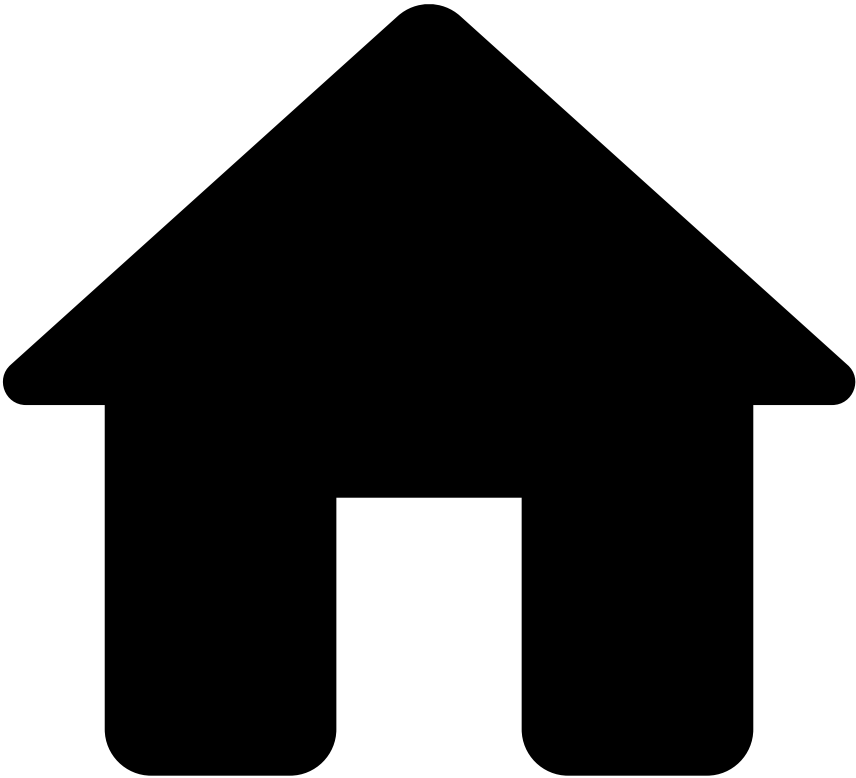
1. In the **Model Project** window, select the **Model Evaluation** where you want to solve the error.
2. Select to access the following view:
3. Use the button to compute Feature Importance. Alternatively, you can the computation of Feature Importance.
 1. **Try again:** Set the [input parameters](#) (i.e. False Positive Rate, Sample Percentage and Random Seed) to compute Feature Importance. By default these parameters are the same as chosen in the previous trial. One recommended approach is to choose a lower **Sample Percentage** to reduce the amount of data that is loaded in memory.
 2. **Dismiss:** Remove the error label "FI:Error" displayed in the **Model Evaluation** list.

Learn More

Once you learn how to handle Feature Importance errors, you can return to [evaluating a model](#).

Also you can also visit the page [Feature Importance](#) for more information about this functionality in Pulse.

•



- [Model Guide](#)
- [Training a Model](#)
- Importing an external model

On this page

Importing an external model

Was this page helpful? 

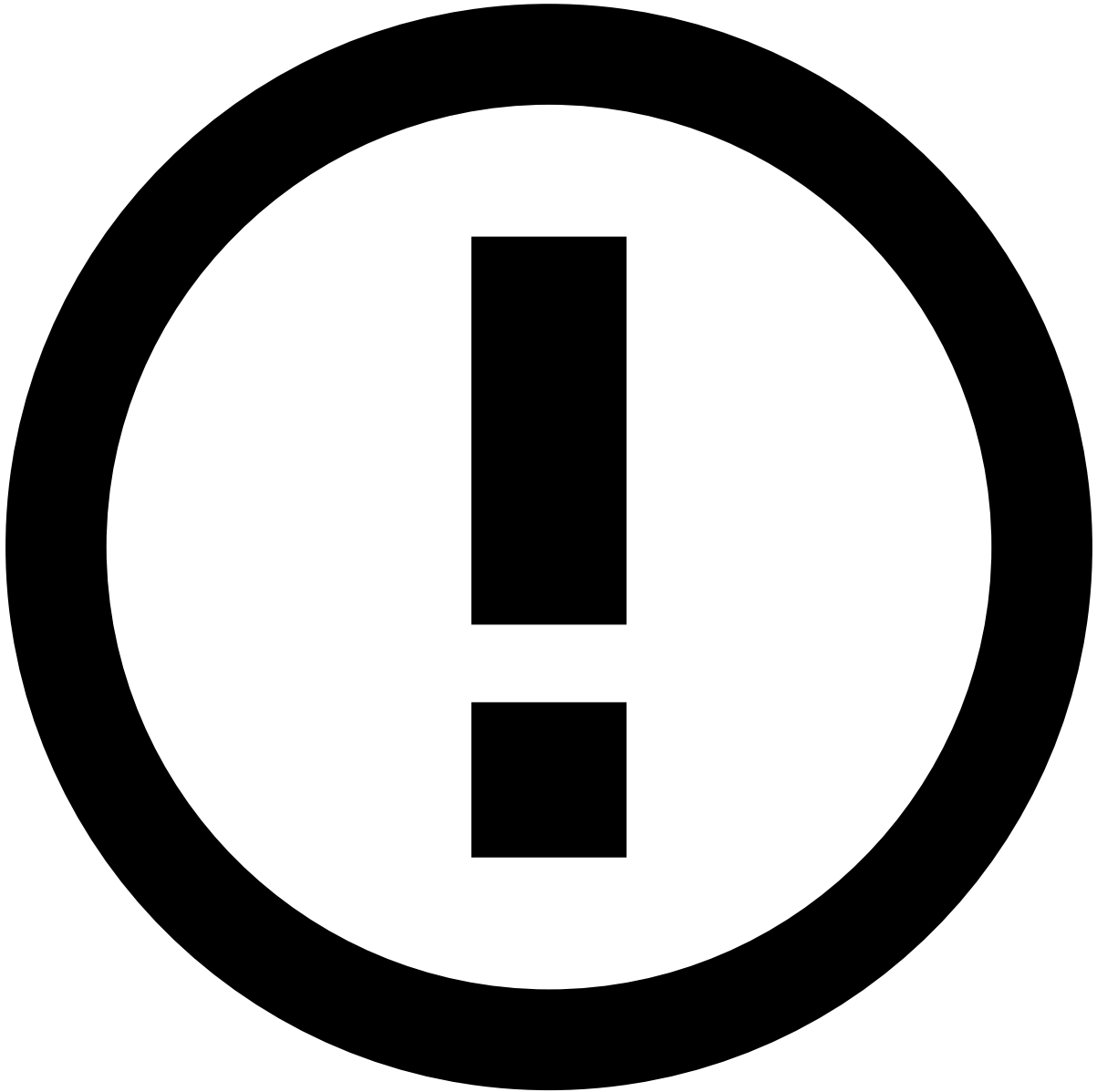
Why

Pulse can integrate with tools that you normally use and are familiar with. You can import models that you created and trained in third party systems. After importing a Model, you can evaluate it and use it in Workflows as any other Model in Pulse.

How to

To import an externally trained model, execute the following actions:

1. In the **Data Science** tab , select a Model Project from the Model Projects list.
2. Add a new model.



Duplicating an existing Model

When fine-tuning model parameters, instead of importing a Model again, you can duplicate an existing imported Model. For that, use the **Duplicate and Change** option on that model. This option loads a copy of it, with all the properties and values that were selected when the Model was originally imported.

1. Select **External Model**.
2. Select the **Algorithm** you used to train the model. Use the link button at the end of the line to learn more about each algorithm. If you don't find the algorithm you need in the list, contact your administrator to discuss the integration possibilities.

3. Type the **Model Path**, which is the path to the physical location of the model folder. For the requirements regarding the content of the model folder, see the table in the next section.
4. If the Model Path has a model.json file that defines the data schema, Pulse automatically loads the schema in the **Fields to Select** area. Otherwise, select **Use Pulse Schema**, and select a data source that is compatible with the data that was used for training the external model.



danger

If models are imported via App Collaboration to a clustered environment, the model binary is only propagated to all nodes if the `modelStorageDirectory` property is configured to use a distributed filesystem such as HDFS.

5. Select the **Target Variable** and the necessary fields for the data schema in the **Fields to Select** area.

In the case of an unsupervised machine learning algorithm (e. g. Isolation Forest), remove the field that you plan to use as target variable. This will avoid label leakage, since a realistic anomaly detection model does not have access to the label.

Requirements🔗📄

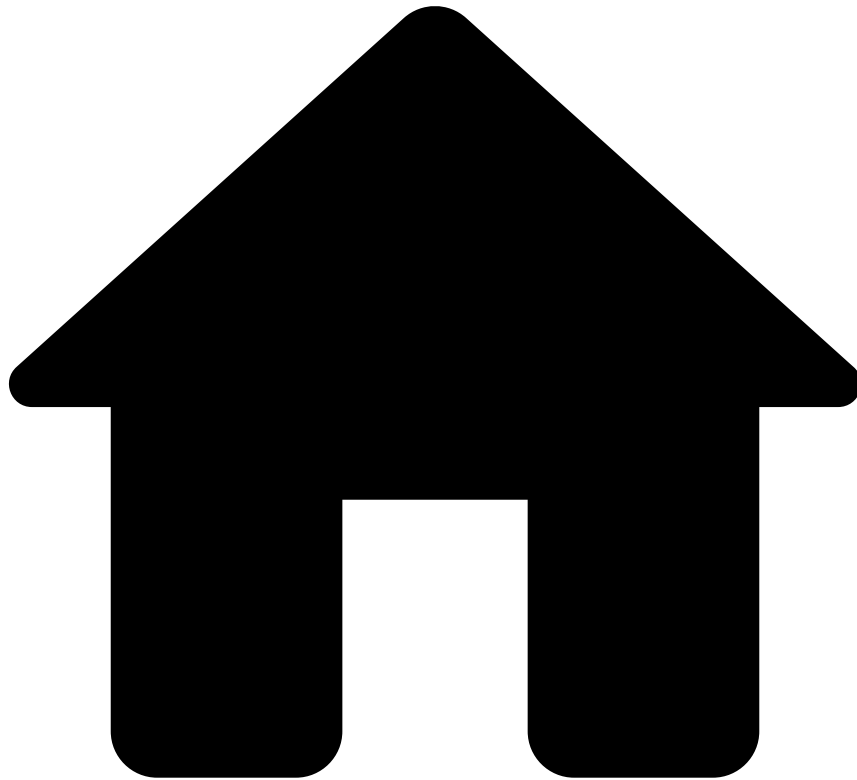
The Model import assumes the following about the imported model:

Provider	Requirements
Feedzai	*.model.gz (a serialized FZRForest object, compacted)
H2O	A folder called <i>model</i> , containing either a *.zip (MOJO) or *.jar (POJO)
DataRobot	A folder called <i>model</i> , containing a *.jar file. The model has to be trained in DataRobot version 4.4.1 or above.

Next Steps🔗📄

Now that your model is trained, you can [evaluate its performance](#), or [compare it with other models](#).

•



- [Model Guide](#)
- [Tuning Model Parameters](#)
- Exploring hyperparameter tuning results

On this page

Exploring hyperparameter tuning results

Was this page helpful? 

Why

After performing a hyperparameter tuning, you want to analyze and explore the evaluations that were generated to learn and choose which parameter combination you will want to use to train your model.

Since the dimension of all the variables to consider can be significantly long, the exploration mode view within hyperparameter tuning is available to help you perform this task. The exploration mode view displays the information in a way that makes it easier for you to compare the performance of the different evaluations and, thus, make a more informed decision on what set of parameters to choose.

How to

To start exploring the results of the hyperparameter tuning, go through the following steps:

1. Open the hyperparameter tuning from the corresponding list box in your **Model Project**.
2. Switch to the **Exploration Mode** tab.
3. Select and [Configure Targets for evaluation](#). As an example, consider you want to have a *Recall* at approximately 1%. Configure the target as follows:
 1. Name the metric that the target will reflect, in this case *Recall @ 1%*.
 2. Configure the new target by selecting the target metric, *#Recall*, and the preferred strategy to reach the target, Approximation 1 %.
 3. Choose which are the **Metrics** to Show. Besides *#Recall* you may want to add more metrics to be able to compare more information.



Adding targets

You can add more targets with other metrics to the exploration mode view.

4. [Analyze the results for each target](#) and compare them among all selected **Model Evaluations**.

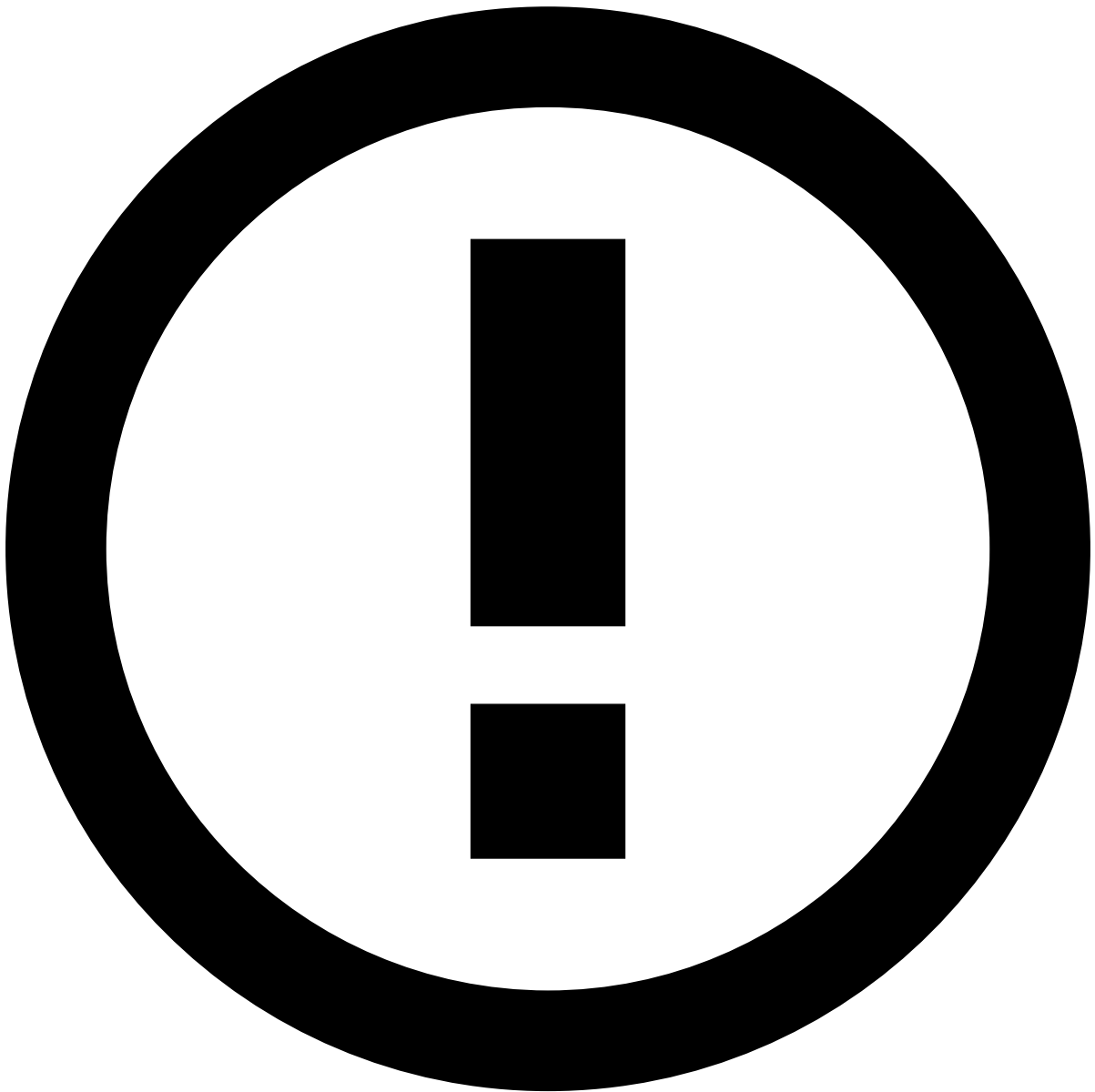


Changing values

By hovering over a result and selecting the available button, you are able to change that value back and forth. This enables you to experiment if another value changes that metric or any other metric, making it easier to compare of how values do or do not interfere with each other.

As you proceed with your analysis, you may want to mark an evaluation in the **Evaluations** list as reference , making it easier to make comparisons between that specific evaluation and any other.

5. Filter the results by algorithm and data source in **Evaluations**.
6. Open one specific evaluation to view it in more detail, namely the **ROC Curve**. You can add another evaluation in this area and, thus, have a detailed view of two evaluations at the same time.



Global Exploration Mode

Consult the [global Exploration Mode view](#) to learn about how it can also help you decide on the best evaluation for your model.

Exporting to a model📄

Once you have decided on the parameters set that has performed the best, you may want to export it to a model. For that proceed as follows:

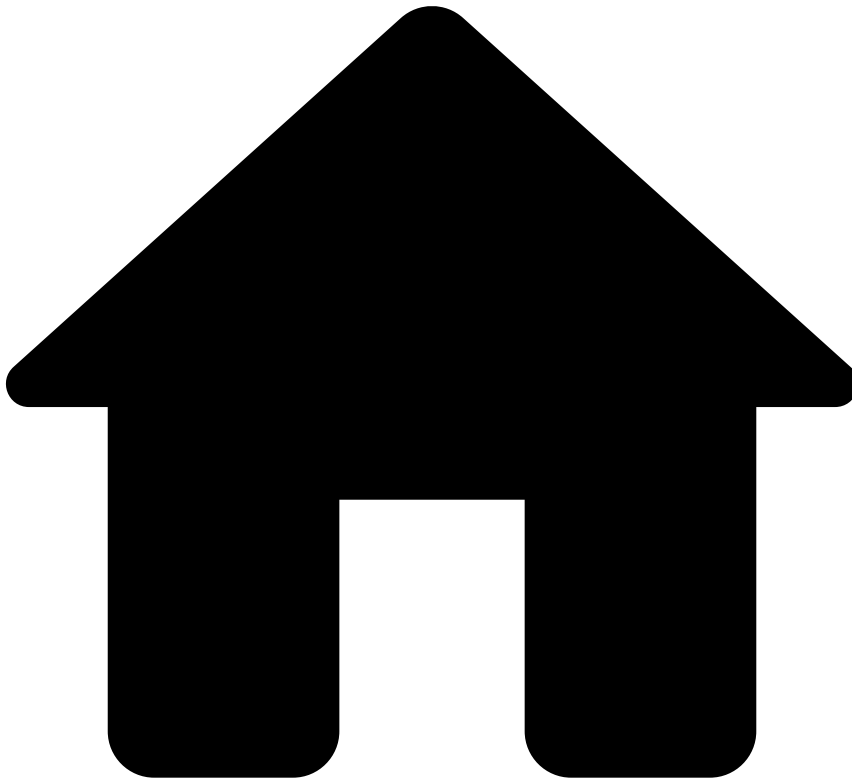
1. Open the **Hyperparameter Tuning** from the corresponding list box in your **Model Project**.
2. Open the **Evaluations** tab.
3. Select the [Export to model](#) button of the corresponding hyperparameter tuning.

Notice that the new model will display all fields already filled in with the parameters used in the evaluation you chose.

Next Steps

Once you have learned how to use the exploration mode, learn about the Export to Model process in the [Tuning model parameters](#) page and train your new model.

•



- [Rule Guide](#)
- [Creating Features](#)
- Best Practices for Plans

On this page

Best Practices for Plans

Was this page helpful? 

Within a map or metric operator, you cannot refer to a field that has been added inside that same operator. In case of such dependencies, you have to use more than one operator to calculate the final field. Also, performance considerations need to be taken into consideration when organizing your maps and metrics into plans.

The following sections describe best practices regarding maps, metrics and plans.

A simple plan

Due to performance reasons, it is recommended to organize map and metrics operators in a single plan, containing all feature engineering logic.

In a simplified example, the plan could have the following operators:

1. A map operator to calculate simple features.
2. A metrics operator to calculate time-based aggregations.
3. Typically another map operator to calculate features based on the metrics.

Large number of metrics

It can happen that the plan has to calculate hundreds of metrics.

Due to limitations of Java, when a metrics operator contains around 200 metrics, the plan might crash. This number depends on the length of the metrics' PQL code. The solution for this is to separate the metrics into two or more operators.

When separating the metrics into two or more different metrics operators, it's a good practice to keep metrics in the same operator if they have the same field in the first place in the *Group By* field. For example, if you have some metrics that group events by *Card*, and some other metrics that group events by *Card* and *ShippingAddress*, then you can keep these metrics in the same operator to speed up the computation.

This way, Spark has to do only one shuffle for those metrics, which is usually the most time-consuming part of Spark computations. However, this technique has the cost of higher memory usage.

Scheduled metrics

When you have scheduled metrics in a plan, Pulse has to recompute everything up to that point in the plan. If you have a lot of computations before the scheduled metrics operator, it might create a big overhead in computation time. This is usually a non-issue since scheduled metrics are normally computed during the night and are not subject to the real-time constraints. Nevertheless be aware of this in case you think your computations will be taking longer than a night in production.

As an exception to the one plan rule, having two plans in this situation can be a trade-off between storage space and computational times.

Addressing errors in plan definition

When defining the components of a **Plan**, you may encounter errors that would result in failures during the [plan execution](#).

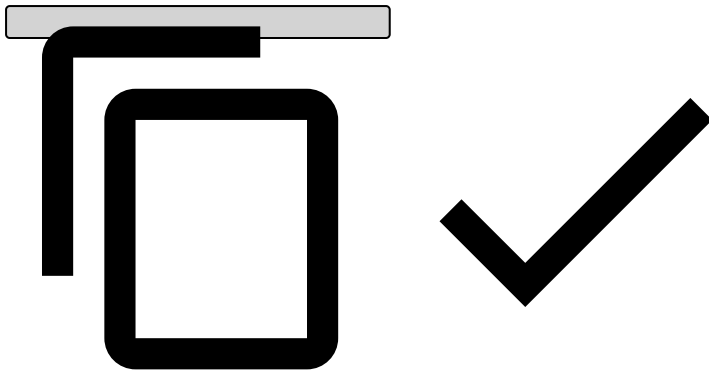
Some of those errors are detectable when defining the Plan, and Pulse will show warnings to allow them to be fixed before deploying the plan to production.

One such error is defining [fields](#) based on a [PQL expression](#) (e.g. within a [Metrics or Map operator](#)). Operators that require null coalescing (i.e. [default fallback values](#)) will produce a warning in the UI if the coalescing operator (`??`) and value are missing:

To fix this situation, provide a fallback value in the PQL expression input field, e.g. `?? 0` or `?? ""`.

A similar warning will be shown if built-in functions are used in collections (like `List` s, `Set` s or `Map` s). For example, an expression like

```
["test", "test1", "test2"].contains("testing")
```

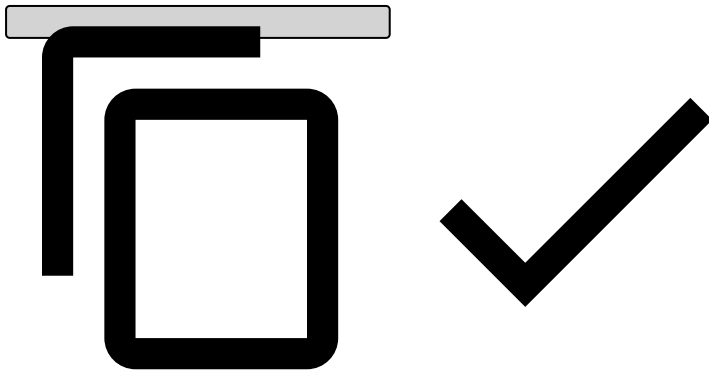


will fail because the `contains` function is not supported in collections used in the PQL fields of Map Operators or Metric Operators.

Attempting to use such functions in that context will result in a warning in the UI:

The alternative is to use [UDFs \(User Defined Functions\)](#), e.g.:

```
com.feedzai.pulse.pkernel.runtime.CollectionUDF.contains(["test", "test1", "test2"], "testing")
```



Besides these preventive warnings, Pulse will also allow recovering from execution errors that are not detectable at design time. Refer to [Automatic Application Repair](#) and [Workflow recovery](#) for more information on how to recover from execution errors.

-



- [Rule Guide](#)
- [Creating Features](#)
- Creating a Plan for Features

Creating a Plan for Features

Was this page helpful? 

In Pulse, a plan entity defines a transformation flow of one data source into another data source. In a plan it is possible to add, update and remove fields, or calculate metrics on the data.

To create a plan, execute the following steps.

1. In the **Data Science** tab, add a plan.
2. Add a **Name** for the plan.
3. Select the **Deployable** option. This option ensures that you can deploy your plan to another environment (e.g. QA or Production) later on.

4. In the **Schema** field, specify the data schema that the plan is built on. You can select an existing schema, or an existing data source.

For example, if you select the* Data with Features for Models* data source, then the plan will be based on a copy of the current schema of that data source.

5. Use the **Confirm** button to acknowledge that future changes in the origin schema won't change the schema of the plan.

6. In the **Timestamp Field**, select the schema field that represents the timestamp of the events. When Pulse aggregates the data over a time period, it will consider this field as the timestamp of the event.

Only fields of [timestamp](#) type can be selected.



info

If you don't select a timestamp field for the plan, you won't be able to create metrics in the plan later on.

7. If you want to use data from more than one channel, select the **Omnichannel** option, and configure the channels as follows:

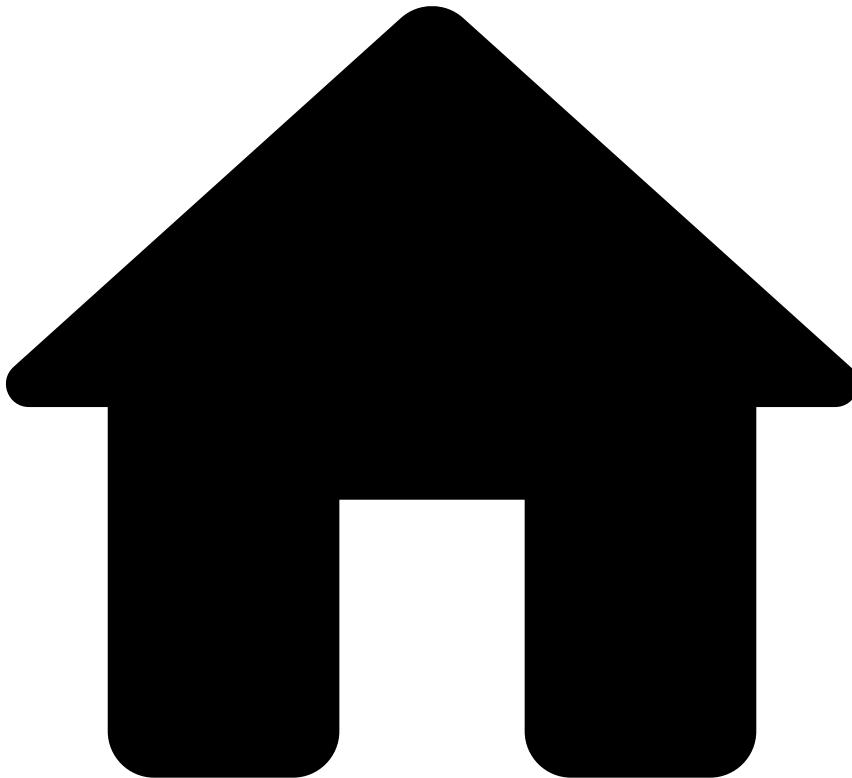
1. Add a **Name** for the **Main Channel**. This channel contains the events that you want to process, and uses the schema selected in step 4.
2. **Secondary channels** provide additional information that you can use to process the events of the main channel. You can use the **Add Channel** button to add secondary channels.

For each secondary channel, add a **Name**, a **Schema** and the **Timestamp Field** that that channel will use.

There should be at least some fields that can be cross-referenced between the channels. For example, both schemas might include a customer identifier, even though it might be actually called differently in each schema.

8. **Save** the plan settings.

•



- [Rule Guide](#)
- [Creating Features](#)
- Creating Derived Features

On this page

Creating Derived Features

Was this page helpful? 

This page describes how to edit and run an existing plan that creates a new data source with new features.

In the example shown, you can see how to add a new field that is calculated based on a single transaction. If you want to create a feature that is based on many aggregated transactions, you must use [metrics](#), i.e. historical profiles – refer to the page on [creating metrics](#).

Adding the features

To add new features, execute the following actions:

1. In the **Data Science** tab, open the features plan.
2. Drag a **Map** operator to the plan if there is none yet.

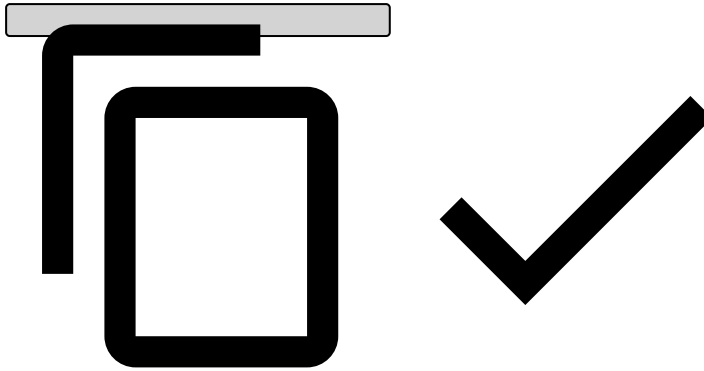


Clipboard

To save time, you can use the clipboard to copy-paste operators in plans. This allows you to add operators without the need of specifying all their fields each time you create a new plan.

3. Edit the map as follows:
 1. Select the **Fields to Add** tab.
 2. In the **Output Field Name** column, specify the name of the new feature.
 3. In the **Expression** column, write the [Pulse Query Language expression](#) that calculates the output value.
For example, the following expression calculates whether the credit card was present at the transaction:

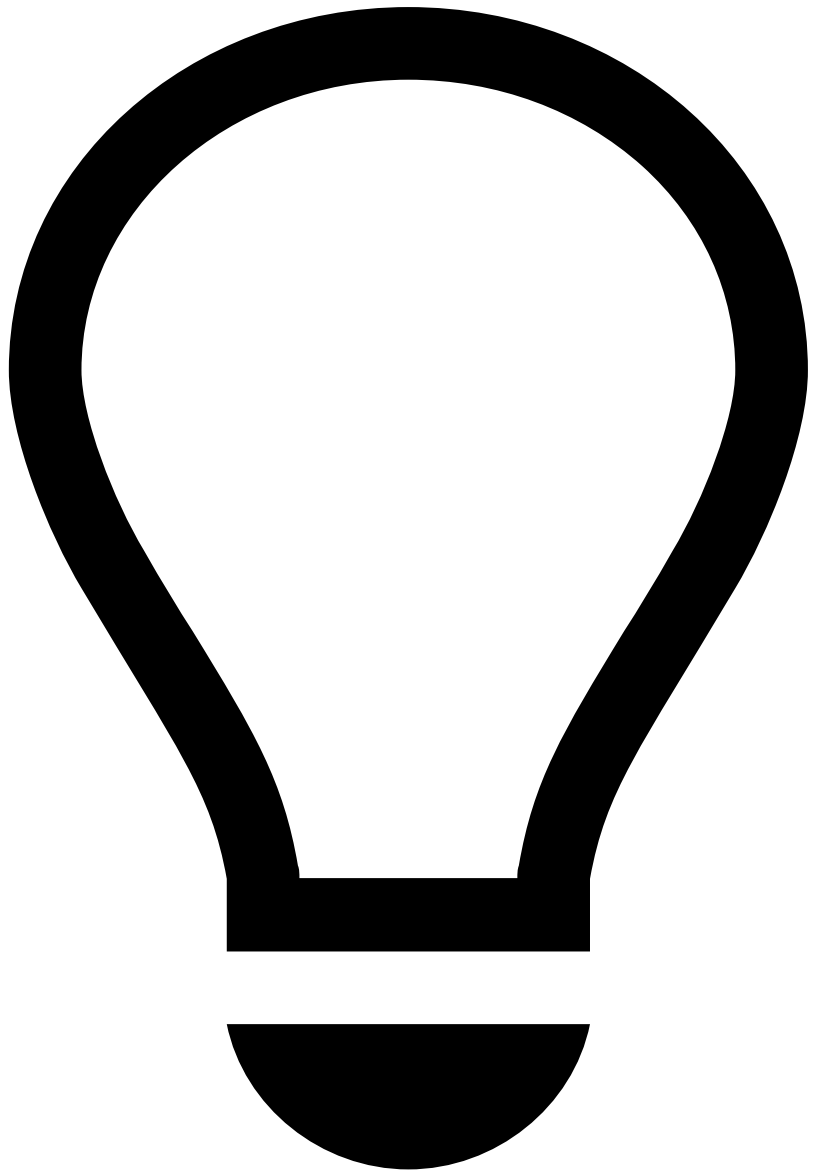
```
if (POSEntryMode == 'Swiped' or POSEntryMode == 'Keyed' or POSEntryMode == 'Chip')
then 'true' else 'false'
```



4. Define the **Type** of the output field. In the example of calculating whether the credit card was present at the transaction, the type of the output field is *Categorical*, with two possible values: *true* and *false*.
5. Confirm that you want to add the field to the output schema, using the add button.
6. Add all the features to the map operator by following the previous steps.
7. **Save** the map operator.

Creating the data source with the features

1. To create a new data source which includes the new field(s), execute the following actions:
 1. With the plan still open, use the button to execute the plan.
 1. In the **Data Source** tab, select the data source that will be the input for the plan. For example *Cleaned Transactions*, or *Model Features v1*. If the plan has more than one channel, select the input data source for each channel.
 2. In the **Data Sinks** tab, configure the output of the plan:
 - **Name** for the output data source. For example, *Rule Features v1*. **Type**: *Parquet* (default) or *CSV*, depending on what type of output file you want to generate.



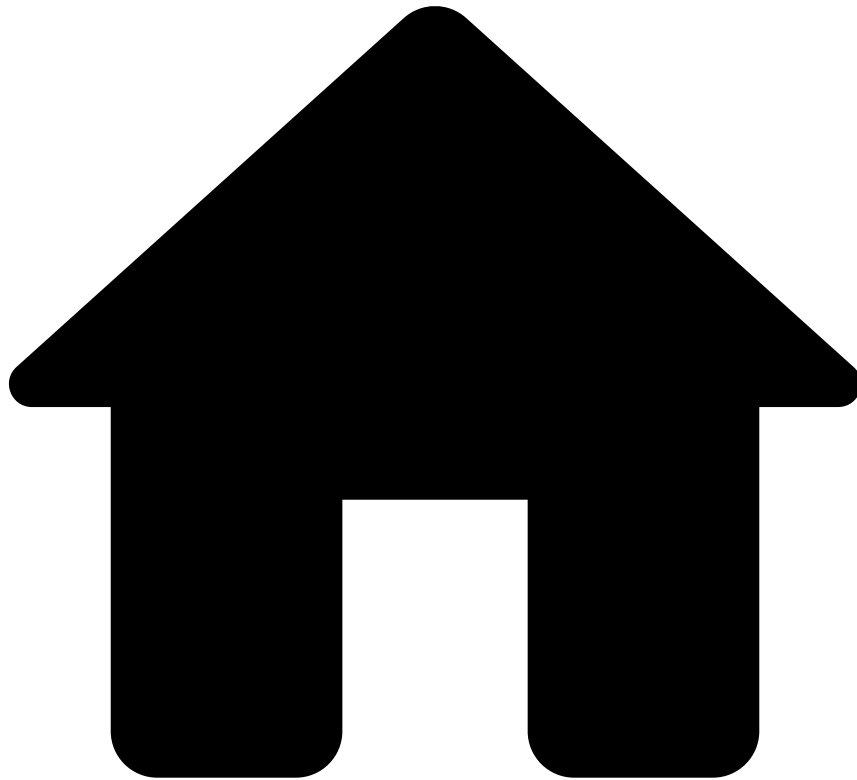
Parquet format is recommended

Using the Parquet file format for data files is highly recommended. Besides allowing a direct encoding of data types (which automates the mapping of field types when importing datasets), it also improves performance and disk usage: compared to CSV, Parquet files occupy less space and take less time to load.

- **Directory Path:** the path and file name where Pulse will store the output file, containing the data of the new data source. Example: `file:///datasets/rule-features-v1.parquet` . Note that the format varies according to the operating system.

3. Use the run button to execute the plan. It generates a new data source in Pulse, named *Rule Features v1*. Pulse also saves an output file with the transformed data to your filesystem.
2. Check the new data source to confirm that its schema includes new fields.

-



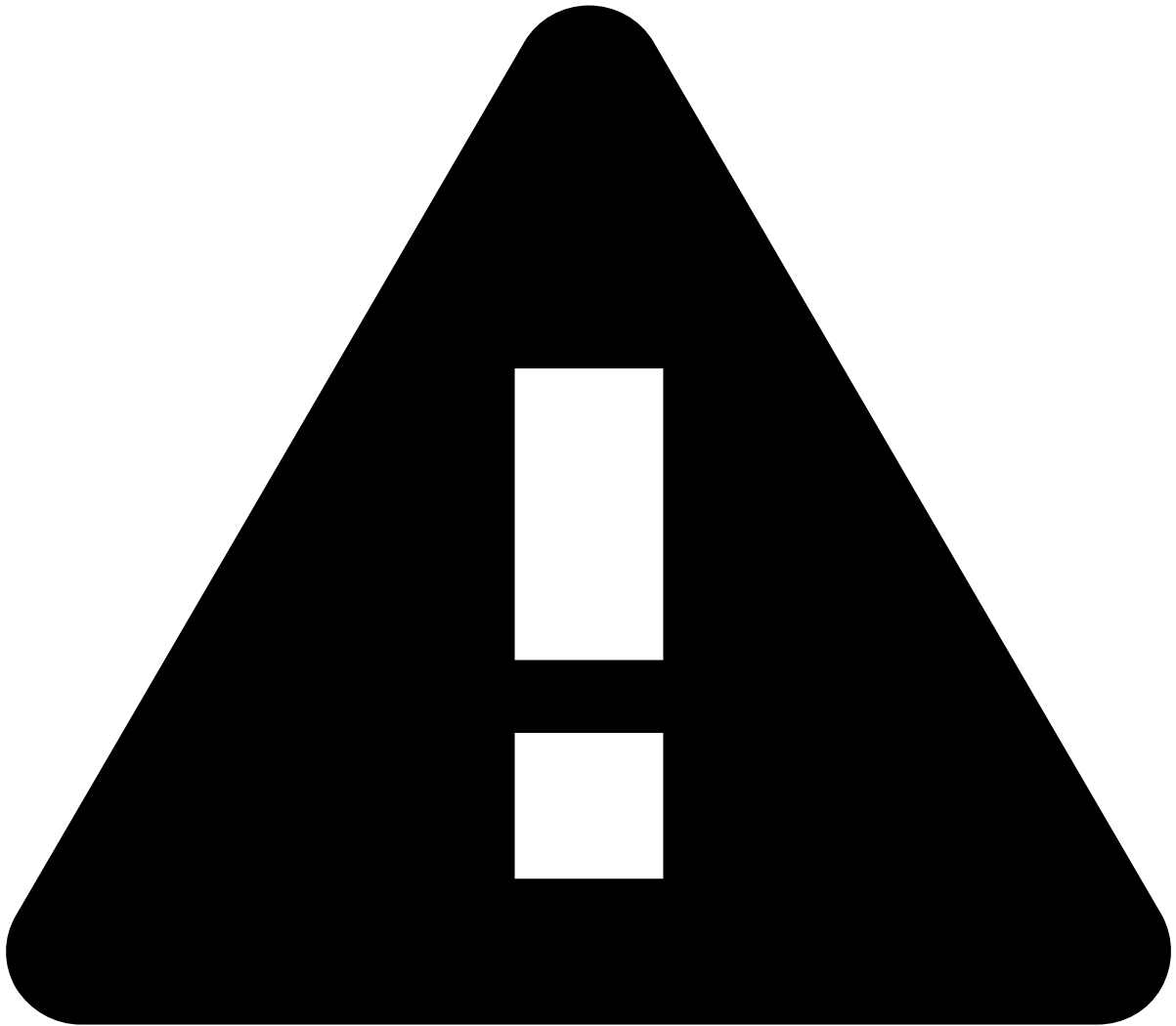
- [Rule Guide](#)
- [Creating Features](#)
- Creating Metrics

Creating Metrics

Was this page helpful? 

You can create new fields to augment a [data source](#) based on an aggregation of events over a period of time. These kind of features are called [metrics](#), or **historical profiles**.

Metrics describe typical behavior. For example: this customer typically uses two devices, and makes one transfer per month to pay the rent. Certain behavior patterns, or changes in the behavior, may help you detect financial crime.



Metrics operator limits

For performance reasons, there are limits to the number of Metrics operators that can be added to a Plan. For more information about this, see [Limits](#).

To add new metrics, execute the following actions:

1. In the **Data Science** tab, open the features plan.
2. Drag a **Metrics** operator to the plan if there is none yet.

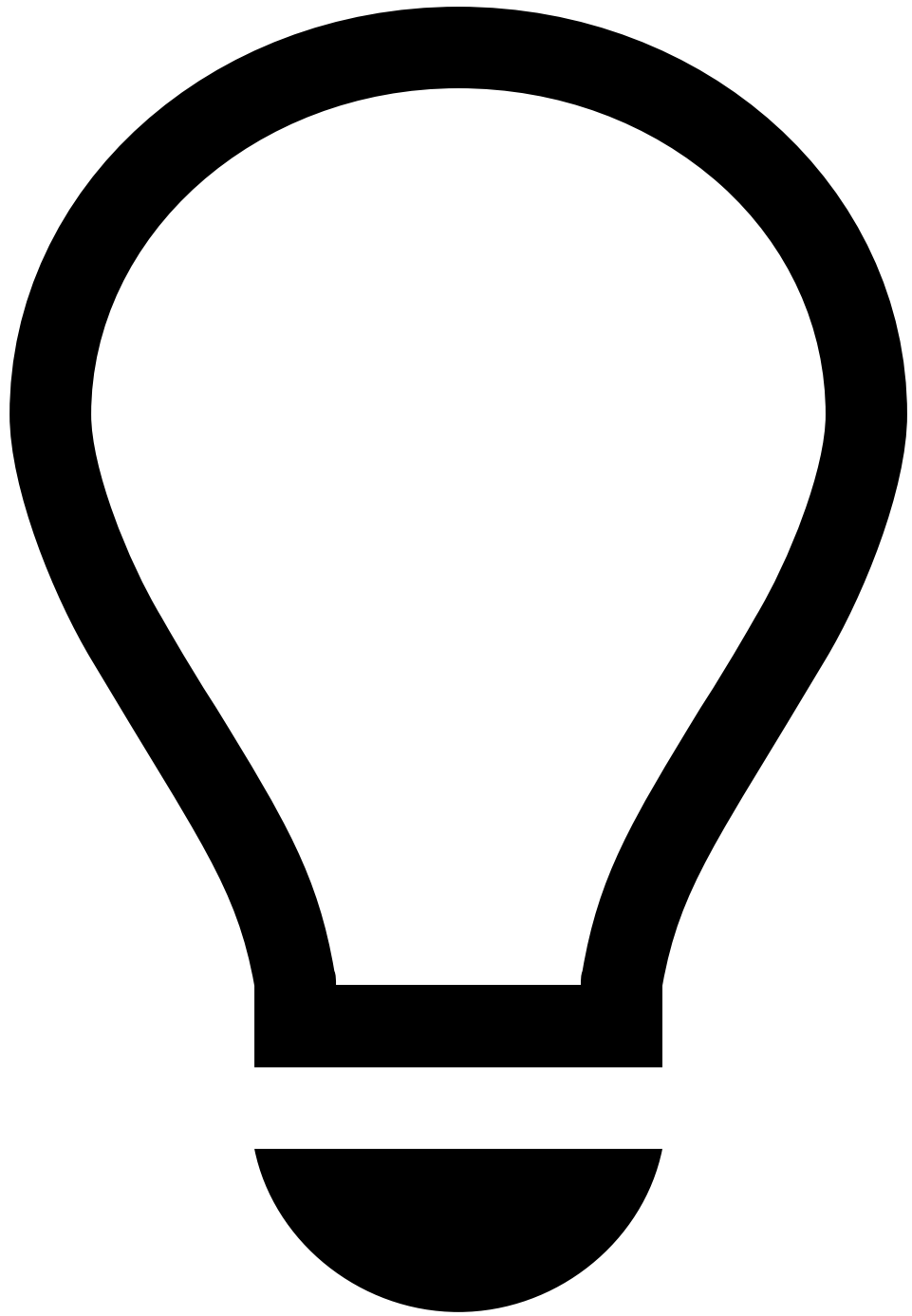


Adding metrics operators

To save time, you can use the clipboard to copy-paste operators in plans. This allows you to add operators without specifying all their fields each time you create a new plan.

3. Edit the metrics operator as follows:

1. Add an **Operator Name** for the metrics operator.
2. Add a new metric to the operator. Later, when you run the plan, Pulse will add this metric to the output data source as a new field.
3. Set the **Group By** fields over which to calculate aggregated metrics. For example:
 1. *AccountID*, to aggregate metrics for each card number.
 2. *MerchantID*, to aggregate metrics for each merchant.
 3. *AccountID*, *MerchantID*, to aggregate metrics for the profile of each card number in each merchant.
4. If relevant, set the **Lookup By** fields (matching the count and types of the **Group By** fields) to define which group to retrieve.



tip

For more detailed information on setting up Lookup By metrics, refer to the [Using Lookup Metrics](#) guide.

5. In the **Window** section, configure the time window over which to calculate the metric.

1. In the **Expression** field, add the time interval that you want to include. For example, *1 hour* or *7 days*.

This means that you want to calculate a metric based on the events of the last hour, or of the last seven days.

For details on the syntax supported in window expressions, please refer to the [Time window](#) section of the documentation on Plans in Workflows.

2. If you use the **Delay** field, recent events will not be immediately accounted for. For example, a 3-hour window with 1-hour delay means that out of the last $3+1=4$ hours of events, the last 1 hour

will not be considered. This is useful if you are computing a historical baseline profile and don't want it to be influenced by recent behavior.

3. **Is tumbling**: tumbling windows expire all events at once, once they reach their predetermined size. Afterwards, they start filling again and the cycle repeats itself.
4. You can define a **Filter** for the time window. Only events that fulfill the filter condition will be taken into account when calculating the metric. Use [Pulse Query Language \(PQL\) expressions](#) returning a boolean value to define the filter.
6. In the **Edit Update Period** section, choose if the metric should be updated in real time, or periodically on a schedule.

1. **Real Time**: ideal for metrics with a small window (e.g. less than one day), to ensure that rapid attacks are detected.

Under Real Time, select the *Update on every event* option. The *Update every* option is available for legacy reasons and should not be selected. This option is currently not accepted for omnichannel plans. It will be supported in future versions of Pulse.

2. **Scheduled**: ideal for metrics with a large window (e.g. 1 month). Since there is less fluctuation in the long-term behavior, the metric can be updated less frequently. To specify when the metric should be updated, you can either set *Start of* or *Scheduled*.

In the latter, you must type a [Cron expression](#), e.g. `0 0 12 * * ?` (i.e. *At noon every day*).

7. In the **Aggregation** section, select the **Function** you want to use to calculate the metric. If the function needs a parameter, select the **Field** that provides that parameter.



Note that storing the state of metric on disk is in continuous development, and it does not provide full feature parity to the previous, RAM-based version.

Examples of functions:

- To calculate the sum of the *TransactionAmount* field over each time window, for each *group by* field: the function is *Sum*, and the parameter field is *TransactionAmount*.
- To calculate the number of the transactions over each time window, for each *group by* field: the function is *Count*, and it has no parameters.

8. Optionally, in the **Advanced** tab of the Aggregation section, you can define a [Pulse Query Language \(PQL\) expression](#). This tab allows you to compute more complex values. When calculating metrics in a plan that is based on more than one channel, the Advanced tab is not available.

9. If your plan has more than one channel, do the following:

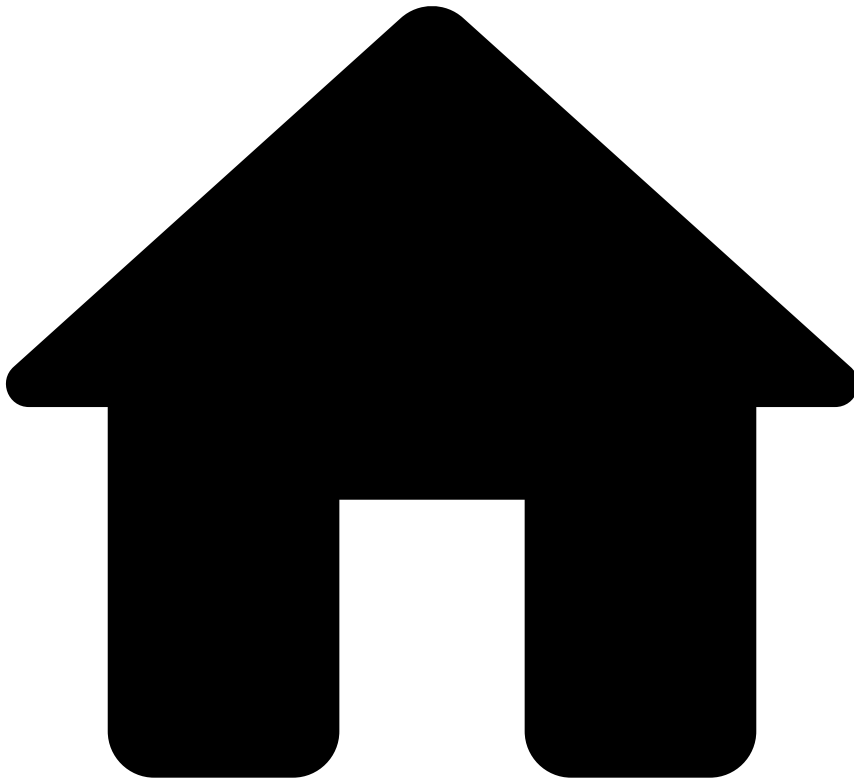
1. Toggle off all secondary channels that are not relevant for the metric using the toggle next to the channel's name in the Group By section.
2. You can decide not to include main channel data in the aggregation. To do so, select the **Do not include main channel data in the aggregation** option. Select this option when the main channel has no data relevant for the aggregation. For example, when processing debit card transactions, it might be useful to know how many devices the customer has used. This information can be aggregated from the online banking channel. But unlike online banking, the main channel (debit cards) will not contain any information about devices.
3. After selecting the *group by* field(s) for the main channel, select the matching field(s) for each active secondary channel. If there is more than one *group by* field, the order of the fields **must** match across the different channels.
4. If you selected an aggregation function that has parameters, select the parameter fields for each channel.
5. Note that only scheduled metrics can use data from secondary channels. If you create a real-time metric with data from secondary channels, you will receive an error message when saving the metric.

10. In the **Details** section, add a description for the metric.

4. Use the **Save** button to add the metric to the operator.

5. The metrics get a default name. You can hover over a metric and use the pencil button to rename it.

•



- [Rule Guide](#)
- [Creating Features](#)
- Creating the Data Source with the Features

On this page

Was this page helpful? 

Creating the Data Source with the Features

When you create a plan, you create a recipe of how to transform the input data source into the output data source. To actually create the output data source, you need to run the plan.



Testing in data science environment

If you create a plan execution in data science environment, the metrics are calculated immediately. However, in a real-time scoring environment, scheduled metrics (SMJs) may take considerably longer. This may lead to differences when you run a plan execution in a data science environment vs. when you use the same plan to score the same events in a real-time scoring environment.

To simulate the behavior of SMJs, you can use the `context.plan.delay.*` Pulse property. It delays the update of scheduled metrics in the data science environment. For each plan, set it to the time that it takes for the SMJs to finish. Get in touch with Feedzai if you want to use this setting.

How to

To create a new data source which includes the new field(s), execute the following actions:

1. In the **Data Science** tab, open the features plan.

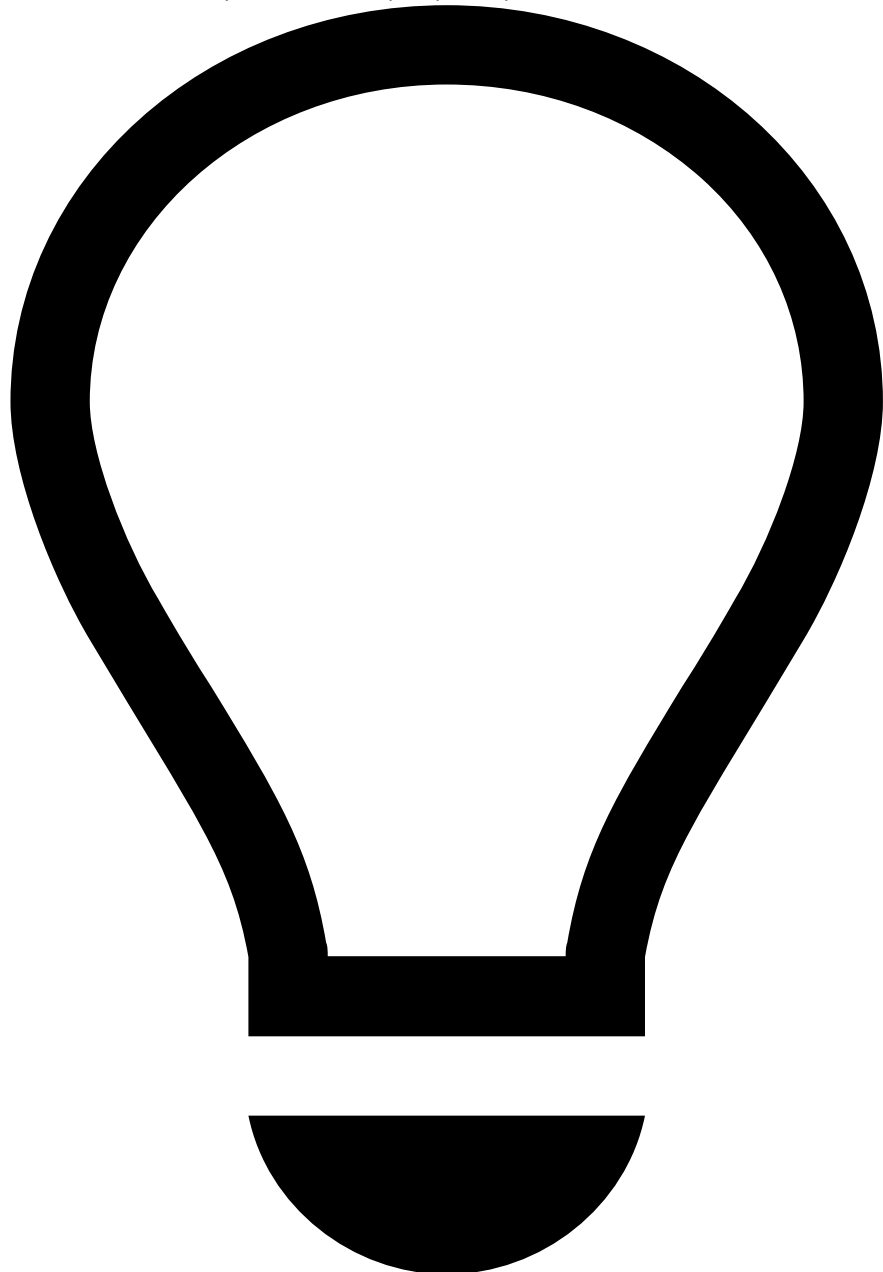
2. Use the run button to execute the plan.

1. In the Data Source tab, select the input data source for the plan execution. For example, *Cleaned Transactions*.

If the plan has more than one channel, select the input data source for each channel.

2. In the **Data Sinks** tab, configure the output of the plan:

- **Name** for the output data source. For example, *Rule Features v1*.
- **Type**: the file format of the output data – *Parquet* (default) or *CSV*.



Parquet format is recommended

Using the Parquet file format for data files is highly recommended. Besides allowing a direct encoding of data types (which automates the mapping of field types when importing datasets), it also improves performance and disk usage: compared to CSV, Parquet files occupy less space and take less time to load.

- **Directory Path**: The path and file name where Pulse will store the output file, containing the data of the new data source. Example: `<file:///datasets/rule-features-v1.parquet>`. Note that the format varies according to the operating system.

3. Use the advanced options button to have access to additional configurations:

1. **Job Profile:** Allows you to choose a profile to optimize CPU cores and memory assigned to the Spark job that will run the plan.
2. [Data Partitioning](#): If you have metrics operators in your plan, this functionality allows to choose the **Data Partitioning** option that enables Spark to reorganize the data into different partitions as homogenously and well distributed as possible. Pulse allows you to choose one of the following options: **Data-Aware Optimizer**, **Hash-Based**, and **Manual Partition**. These options are described in detail in the [Data Partitioning](#) page.

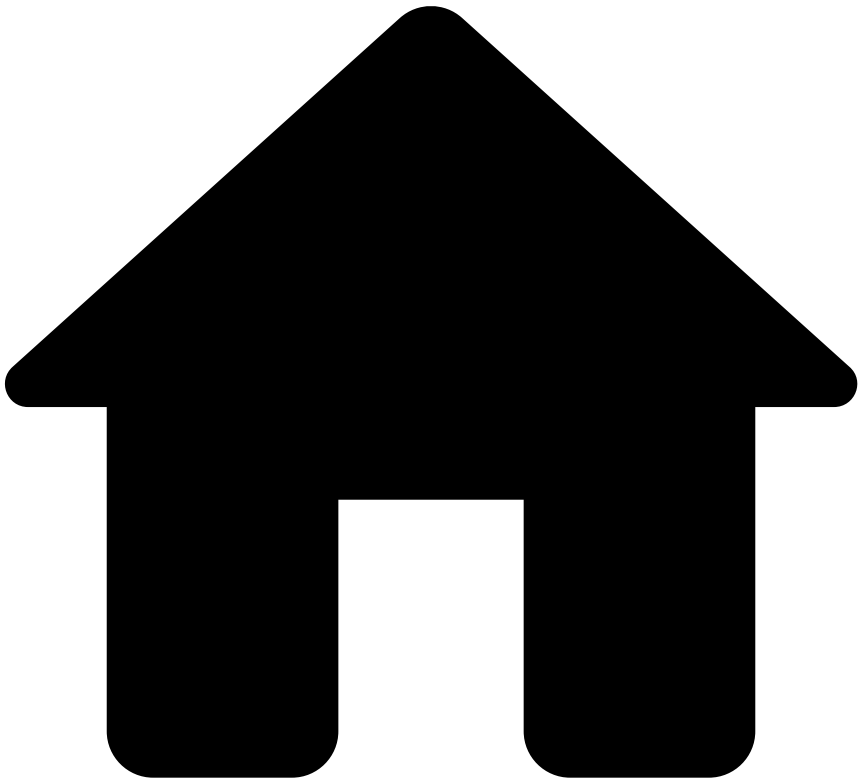


Metrics operators

The data partitioning option takes effect only if you have metrics operators in your plan. Otherwise, this option does not affect the performance of the plan execution.

4. Use the run button again to execute the plan. It generates a new data source in Pulse, with the name you selected for the data sink. Pulse also saves an output file with the transformed data to your file system.
3. When the plan execution is finished, check the new data source to confirm that its schema includes new fields.

-



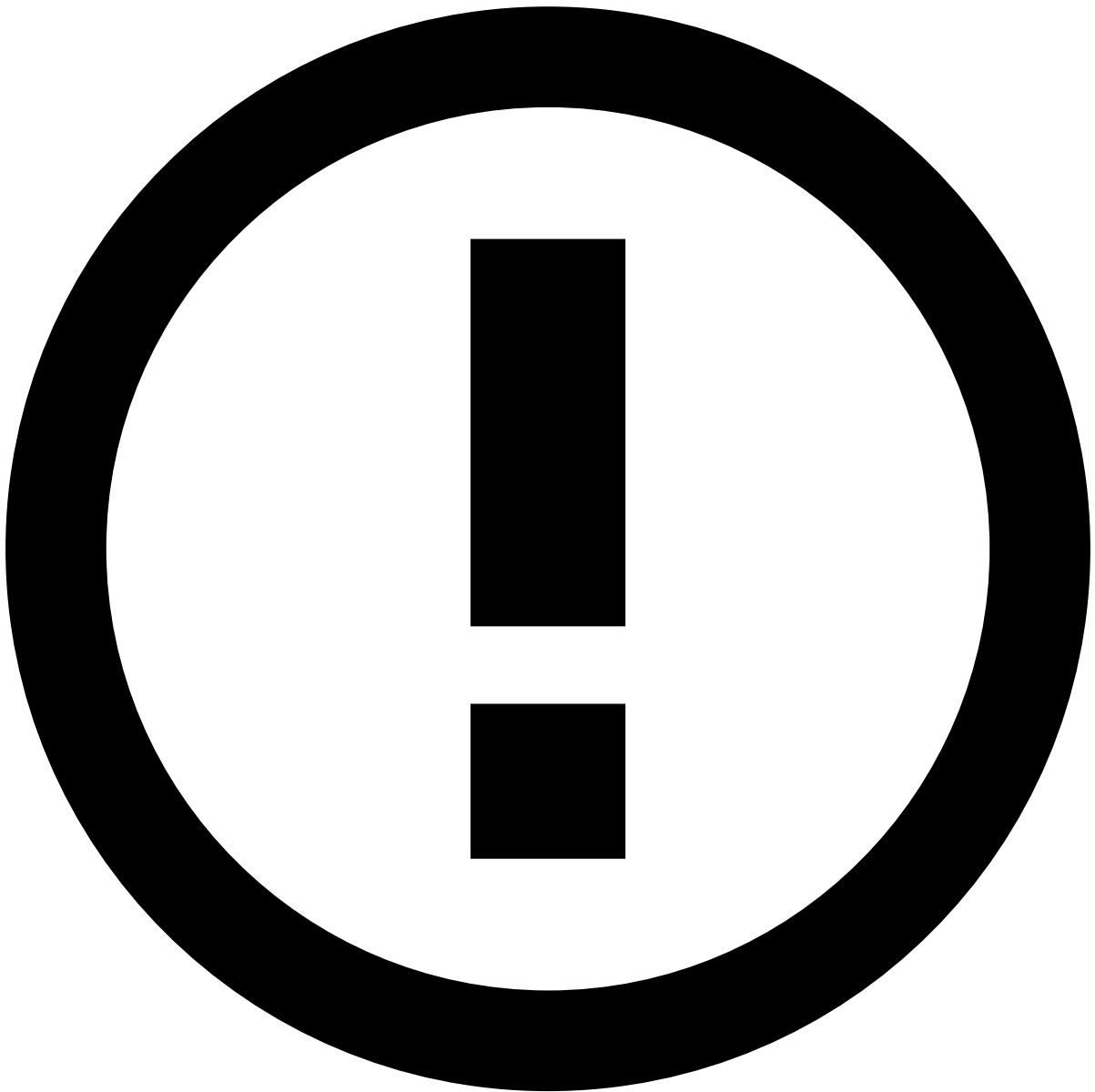
- [Rule Guide](#)
- [Creating Features](#)
- Using Lists in Features

On this page

Using Lists in Features

Was this page helpful? 

To access a field of a list item in a plan, you need to add the field to the plan. This can be useful when you have reference data stored in lists, like risk levels, or country-related information. This topic describes how to add fields of lists to a plan, and how to access them in a feature.



Features cannot use multi-tenant lists

Multi-tenant lists are not supported in plans. More specifically, the user cannot select multi-tenant lists in a list operator.

If in a plan you want to use list values to compute features, you need to add the list field to the plan. This is the objective of the list operator. For each event that traverses the list operator, this field will be populated with the corresponding list item value from the list.

List items can have an **expiration date**. This expiration date applies to the list operator as well, meaning that if the corresponding list item is not valid at the time of the timestamp of the event, the field in the event will have a null value. You can set the timestamp field in the following places:

- for a deployable plan under the Configure Plan Schema option.
- in a non-deployable plan: for each operator, you can select the timestamp field individually, in the Timestamp Field option.

If there is no timestamp field defined, the list items that have expiration date are ignored.

How to

To add a field of a list to a plan, perform the following steps:

1. In the **Data Science** tab, open the plan you want to edit.
2. Drag a **List** operator to the plan.
3. Type a **Name** for the list operator.
4. For each list field that you want to add to the plan, perform the following steps:
 1. In the **Output Field Name** column, type a name for the new field. This is the name by which you will be able to find the list value when using it in the plan.
 2. In the **List** column, select the list that you want to use in the plan.
 3. In the **Operation** column, select what you want to do with the list. The available operations are:
 1. **Get**: returns the value of the field specified in the List Field column, corresponding to the list item identified by the Item Value Expression. Applies only to complex lists.
 2. **Contains**: returns true if the list contains an item identified by the Item Value Expression.
 4. In the **Item Value Expression** column, type an expression that defines the key for which the list operator will retrieve the value. This expression is the connection between the event and the list. It is a [PQL expression](#), meaning that you can use anything you can normally use in a map operator, for example:
 - event fields
 - constants
 - boolean and arithmetic operators
 - UDFs
 - coalesce operator
 5. In the **List Field** column, select the field of the list that you want to use in the plan. This field is available only if you selected a complex list in the List column.
 6. Use the add button to add the field to the list operator.
5. **Save** the list operator.

As a result, in the features (map and metric operators) of the plan you can access the list value associated to the current event, by typing the name you gave in the *Output Field Name* column. The value that this field contains is the value that belongs to the key you defined in the *Item Value Expression*.

Example

You may have a positive/negative list about known merchants. A merchant may be risky, hence transactions with that certain merchant might need to be declined. Transactions with other merchants might need to be approved, or might need a review. A positive/negative list accumulates all this knowledge in one place.

In this case, the list is defined in the following way:

Example with a posneg list

The list items have the *Merchant Id* as key, and either 'decline', 'approve' or 'review' in the *Action* field. Example:

Merchant Id	Action
898776	'approve'
556949	'decline'
132214	'review'

Now, you may want to use the knowledge accumulated in this list in the plan. For instance, when processing a transaction, you might want to calculate the number of transactions initiated by the same card with risky merchants.

To allow that, add the *Action* field of the *Merchant PosNeg* list to the plan. The key that will connect the event with the list is the *MerchantId*. The new field of the event will be called *MerchantAction*.

Example list operator

Now you can use the *MerchantAction* field in the metric, as any other event field. It contains the action ('decline', 'approve' or 'review') according to the merchant of the event.

Example metric using the MerchantAction field

-



- [Rule Guide](#)
- [Creating Features](#)
- Using PQL in Features

On this page

Using PQL in Features

Was this page helpful? 

Pulse Query Language (PQL) is Pulse's query language that allows users to compute complex values based on input data fields.

This topic introduces the basic concepts of PQL.

To learn how to use PQL in some specific scenarios, read the following user guides:

- [Handling NULL values with PQL](#)
- [PQL in derived features](#)
- [Calling UDFs](#)

Usage

The following image shows an example of how PQL is used in a map operator to create a derived field in a plan.

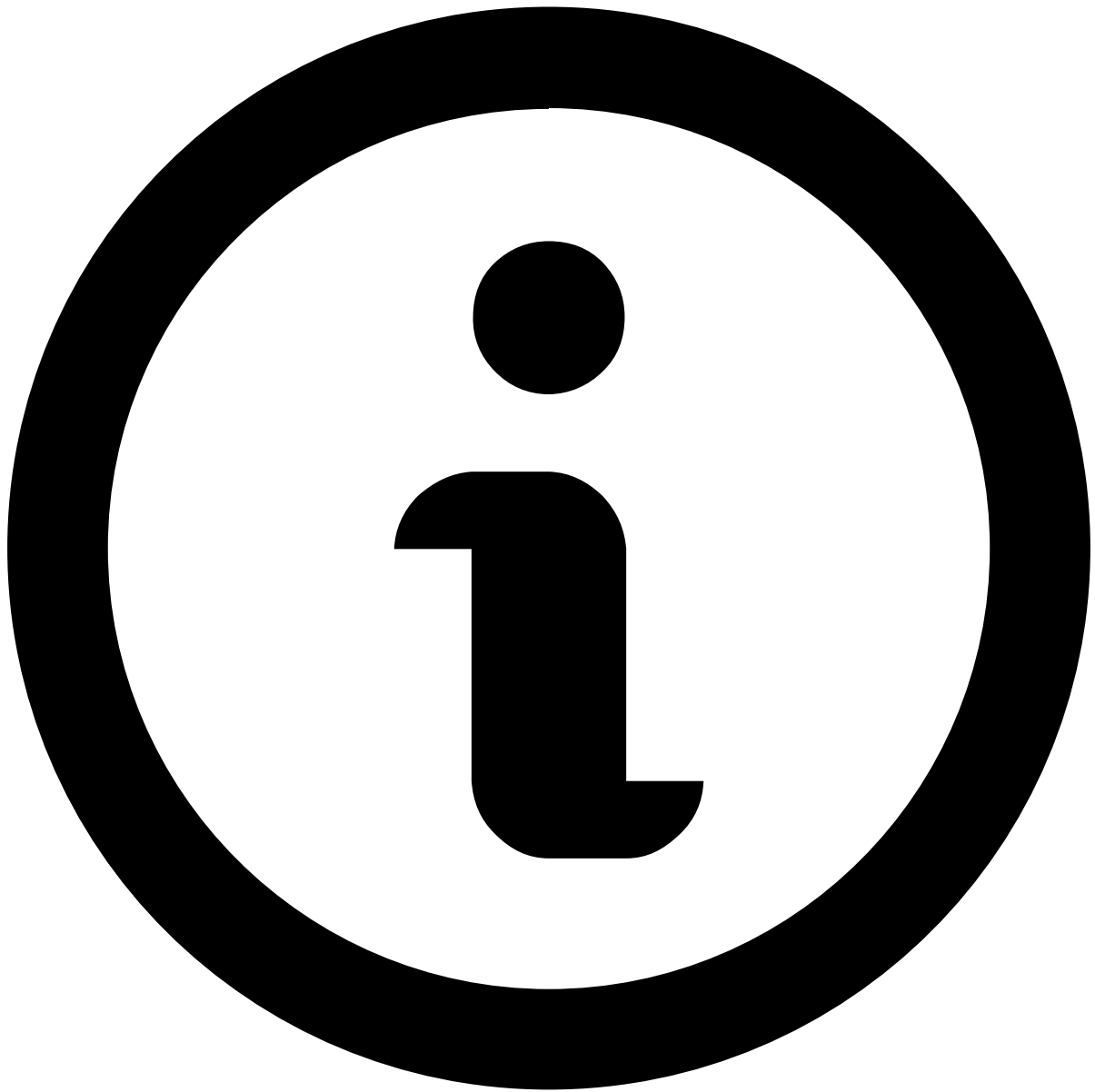
This map operator adds a categorical field called *Card Present Flag*, representing whether there was a bank card present in the transaction. The PQL expression is in the *Expression* column.

PQL expressions

In the Pulse UI, users can combine the following building blocks (values, operators, functions, subclauses) into PQL expressions:

Building block / subclause	Purpose, observation	Example(s)
Constants of primitive data types	Use a fixed value	1.999 'true' 'Swiped'
Input data fields	Use values of the input data. After typing the name of the data field, resolve the typed name to an existing data field name from the list that the UI offers.	Amount
: (i.e. the cast operator)	Convert data into another data type	Amount:int
Arithmetic operators	Calculate a new value from two values of int, long, float or double type	Balance - Amount
Boolean operators	Calculate a new true/false value	and or ! ==
Function calls	You can call two types of functions: built-in functions, and custom functions (aka. UDFs)	The following example calls the <i>between()</i> built-in function: between(Amount, 0, 200)
Conditional expressions	Return a value based on a condition	if Amount > 1000 then 'HighAmount' else 'LowAmount'
Variable declarations	Declare auxiliary variables that help you calculating other values	The following example shows how you define two variables (first_timestamp and last_timestamp) in order to calculate a third value (last_timestamp - first_timestamp) / 60*60*1000.0) (let first_timestamp = first().Timestamp ?? 0 in let last_timestamp = last().Timestamp ?? 0 in (last_timestamp - first_timestamp) / 60*60*1000.0)
Collections, tuples	Work on tuples, lists, sets, and maps of data instead of a single data field	

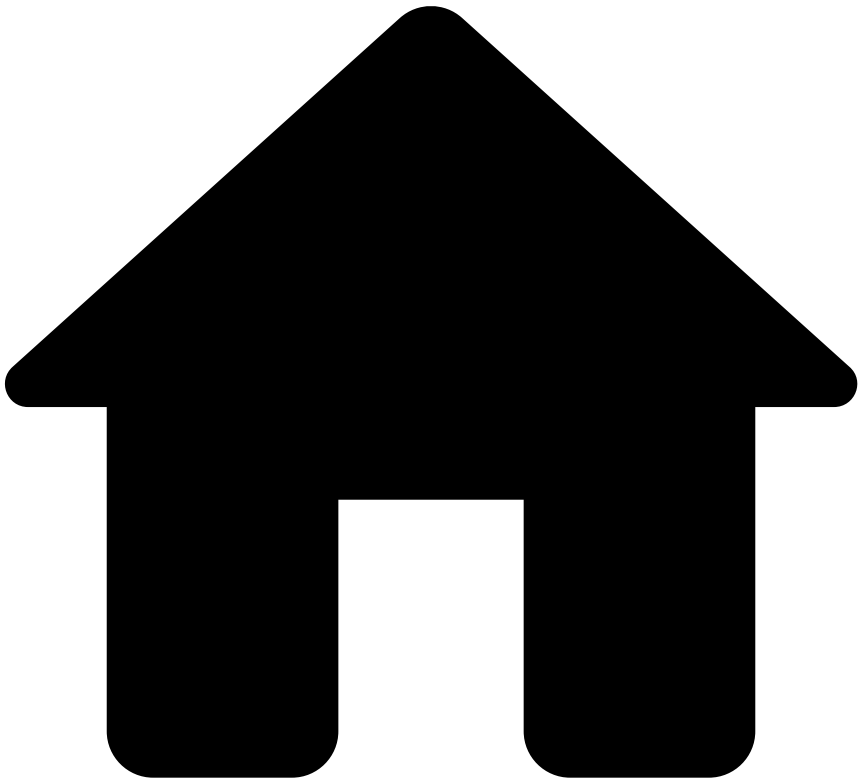
Building block / subclause	Purpose, observation	Example(s)
		In the following example, OrderItems is a list of orders. Each item of the list is a tuple with the following fields: ItemId, Quantity, Price <pre>OrderItems.get(0).Quantity</pre>
Aggregations, and functions over aggregated datasets	Aggregate data to calculate minimum, maximum, average, or alternatively to access specific records of the aggregated dataset	<pre>avg(Amount) - prev().Amount</pre>
?? (i.e. the coalesce operator)	Handle NULL values	<pre>Amount ?? 0</pre>



Reserved keywords

Some keywords and symbols cannot be used outside of their original purpose. See the list of [reserved keywords](#) for more information.

•



- [Rule Guide](#)
- [Creating Rules](#)
- Default UDFs

On this page

Default UDFs

Was this page helpful? 

Pulse comes with a set of User-Defined Functions (UDFs) that can be used in PQL expressions to implement richer logic than what's available from the basic built-in functions. This page provides an overview of the available UDFs.

To learn about how to use UDFs, see [Function Calls in PQL](#).

Working with strings

`DataConversionTools` class 

Package: `com.feedzai.common.udfs`

Modifier and Type	Function	Description
static Integer	stringToInt(String s)	Converts a String into an Integer. If the given s is null or the value exceeds Integer.MAX_VALUE or Integer.MIN_VALUE , then null is returned.
static Long	stringToLong(String s)	Converts a String into a Long. If the given s is null or the value exceeds Long.MAX_VALUE or Long.MIN_VALUE , the null is returned.
static Double	stringToDouble(String s)	Converts a String into a Double. If the given s is null or the value exceeds Double.MAX_VALUE or Double.MIN_VALUE , then null is returned.
static Float	stringToFloat(String s)	Converts a String into a Float. If the given s is null or the value exceeds Float.MAX_VALUE or Float.MIN_VALUE , then null is returned.
static String	integerToString(Integer i)	Converts an Integer into a String. If the given Integer is null, null is returned.
static String	longToString(Long l)	Converts a Long into a String. If the given Long is null, null is returned.
static String	doubleToString(Double d)	Converts a Double into a String. If the given Double is null, null is returned.
static String	floatToString(Float f)	Converts a Float into a String. If the given Float is null, null is returned.

Strings class (fraud)🔗📄

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Integer	levenshteinDistance(String str1, String str2)	Computes the Levenshtein distance (edit distance) of two strings.
static Boolean	matches(String input, String like)	Returns true if the input matches the 'like' pattern, using case-sensitive matching.
static Boolean	matchesIgnoreCase(String input, String like)	Returns true if the input matches the 'like' pattern, using case-insensitive matching.
static String	strcat(Object obj1, Object obj2)	Concatenates two strings.

Strings class (cloud)🔗📄

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Integer	tokenCount(str: String, delimiter: String)	Returns the number of tokens in a given string, separated by the defined delimiter. For example, <code>tokenCount("Number of substrings", " ") = 3</code> .
Double	twoGramSimilarity(str1: String, str2: String)	Computes the 2-gram similarity between two strings. The returned value is a double in a range from 0.0 to 1.0, where 0.0 represents the minimum similarity and 1.0 represents maximum similarity. The 2-gram similarity between strings is computed after white space removal. If the given strings are null they will be considered empty strings. For more details see the documentation of NGramDistance .
String	normalize(str: String)	Removes accents and converts the string to lower case. If the input string is null, an empty string is returned.
String	normalizeDelWhitespace(str: String)	Removes the accents, deletes white spaces and converts the string to lower case. If the input string is null, an empty string is returned.
String	normalizeToAlphanumeric(str: String)	Normalizes the string to contain only ascii alphanumeric symbols. Note that this will strip accents, so "ÃE" will become "a".

Modifier and Type	Function	Description
Boolean	isSubstring(str: String, sub: String)	Returns true if a string contains a given substring in a case insensitive manner.
Boolean	isAllCaps(str: String)	Returns true if the input string is all upper case.
Boolean	isAllNonWhitespaceCaps(str: String)	Returns true if the input string is all upper case. Ignores white spaces.
Integer	strDistance(str1: String, str2: String)	Returns the Levenshtein-distance between two strings.
String	extractDigits(str: String)	Returns the digits from a string.
String	dropDigits(str: String)	Returns the string after dropping the digits from it.
Boolean	isDigits(str: String)	Returns true if the string consists only of digits.
Integer	length(str: String)	Returns the length of the string, or 0 if the string is null.
Integer	countChars(str: String, chars: String)	Counts the number of characters of "chars" in "str", or 0 if any of the input strings is null.
Integer	longestSameLetterSubstringLength(str: String)	Returns the length of the longest substring that is a repetition of the same letter.
Integer	longestIncreasingSubstringLength(str: String)	The length of the longest substring that is strictly increasing by one unit, e.g. "defgh".
Boolean	containsCyrillic(str: String)	Returns true if a string contains Cyrillic characters. If the string is empty, returns false.
Boolean	regexFind(regex: String, str: String)	Returns true if a string matches a given regex pattern. If any of the parameters is empty, the function returns false.
Double	keyboardDistance(str: String, step: Int)	Returns the average keyboard distance between every character in a string and the character that precedes it by a given step.
Double	numRatio(str: String)	Returns the ratio of digits in the input string.
Double	vowelRatio(str: String)	Returns the ratio of vowels (a, e, i, o, u, Ð°, Ð¾, Ñ, Ñ, Ñ) in the input string.
Double	distinctCharsRatio(str: String)	Returns the ratio of distinct characters in the input string.

StringTools class

Package: `com.feedzai.commons.udfs`

Modifier and Type	Function	Description
static Integer	compareTo(String f, String s)	Lexicographically compares string f with string s.
static String	concat(String...s)	Joins the elements of the provided array s into a single string object.
static Boolean	contains(String f, String s)	Checks if string f contains string s.
static String	deleteWhitespace(String s)	Deletes all whitespaces from a string as defined by Character.isWhitespace(char).
static Double	getDiceIndex(String f, String s)	Returns the Š,ensenšDice index for a given pair of strings.
static Integer	getLevenshteinDistance(String s, String t)	Returns the Levenshtein distance between two strings.
static Double	getNormalizedLevenshteinDistance(String s, String t)	Returns the Levenshtein distance between two strings, divided by the length of the longest string.
static Integer	length(String text)	Returns the length of a string.
static String	lowerCase(String s)	Converts string s to lower case as per String.toLowerCase() .

Modifier and Type	Function	Description
static Boolean	regexMatches(String regex, String text)	Checks if a string matches a given regular expression.
static String	replaceAll(String regex, String text, String replacement)	In the text, this function replaces every subsequence that matches the regular expression with the defined replacement.
static String	replaceFirst(String regex, String text, String replacement)	In the text, this function replaces the first subsequence that matches the regular expression with the defined replacement.
static java.util.List<String>	regexSplit(String regex, String text)	Splits a given text according to a given regex.
static String	replace(String text, String find, String replace)	Replaces all occurrences of a string (find) within a text.
static java.util.List<String>	split(String s)	Splits the provided text into an array, using whitespace as the separator.
static java.util.List<String>	split(String s, String sep)	Splits the provided text into an array, using whitespace as the separator.
static String	stripAccents(String s)	Removes accents from a given string.
static String	substring(String text, Integer begin)	Gets a substring from the specified string, avoiding exceptions.
static String	substring(String text, Integer begin, Integer end)	Gets a substring from the specified string, avoiding exceptions.
static String	trim(String s)	Deletes all start and end whitespaces from a string as String.trim().
static String	trimOrNull(String s)	Deletes all start and end whitespaces from a string as String.trim().
static String	upperCase(String s)	Converts a string to upper case as per String.toUpperCase().

Parse class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Integer	asInt(str: String)	Convert the given string to an integer.
Long	asLong(str: String)	Convert the given string to a Long.
Double	asDouble(str: String)	Convert the given string to a Double.
Boolean	asBoolean(str: String)	Convert the given string to a Boolean.

MoreStrings class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Integer	countDigits(String s)	Counts the number of digits in a string.
static Integer	countLetters(String s)	Counts the number of letters in a string.
static Integer	countLowerCase(String s)	Counts the number of lower case letters in a string.
static Integer	countUpperCase(String s)	Counts the number of upper case letters in a string.
static String	replaceAll(String text, String regex, String replacement)	Replaces the regex occurrence in the text with the replacement.

Working with location and addresses

Address class

Package: com.feedzai.cloud.udf

Modifier and Type	Function	Description
String	joinAddressFields(name: String, line1: String, line2: String, zip: String, city: String, region: String, country: String)	Joins all the address fields into a single string, including the person's name. Null fields are treated as empty strings.
String	joinAddressFields(line1: String, line2: String, zip: String, city: String, region: String, country: String)	Joins all the address fields into a single string, excluding the person's name. Null fields are treated as empty strings.
String	extractNumericFields(str: String)	Returns the space separated numeric fields from the address, or an empty string if no numeric fields exist.
String	extractNumericFields(line1: String, line2: String, zip: String)	Returns the numeric fields from an address broken down into two lines and zip code. Returns an empty string if no numeric fields exist.
Boolean	isMismatch(fields1: String, fields2: String)	Returns true if the fields are not empty and different.
String	basicNormalization(address: String)	Normalizes the address to alphanumeric characters.
String	basicNormalization(name: String, line1: String, line2: String, zip: String, city: String, region: String, country: String)	Concatenates the parameters and normalizes the string to alphanumeric characters.
String	addressNormalizationReplacement(address: String, AddressNormalizationReplacements: Seq[(String, String)])	Applies replacement rules to a basic-normalized address, replacing each source token by its configured abbreviation/value.
String	advancedNormalization(address: String)	Replaces address suffixes with their abbreviation (e. g.: street to str), and normalizes the string to alphanumeric characters.
String	aggressiveNormalization(address: String)	Executes the advancedNormalization() function, then moves the numbers in the address to the front, and removes arithmetic progression substrings.
String	concatNormalizedFields(fields: String*)	Normalizes and concatenates the input strings into a single string. Removes all whitespaces and non-word characters, including applying Strings.normalizeToAlphanumeric().
String	concatNormalizedFieldsList(fields: JList[String])	Normalizes and concatenates the input strings into a single string. Removes all whitespaces and non-word characters, including applying Strings.normalizeToAlphanumeric().
String	normalizeUsaAddress(zip: String, line1: String)	Normalizes a US address to the most essential information of the address: the first digits of the zip code up to 4 digits, and the first characters of the normalized address line (10-14 characters, depending on the length of the zip), concatenated in <zip>_<normalizedaddress> format, and cut to 15 characters.
String	aggressiveNormalizations(input: String*)	Applies aggressive normalization after concatenating the nullable sequence of strings given as input.

AddressUtils class

Package: com.feedzai.ds.udf

Modifier and Type	Function	Description
static Integer	countAddressSuffixes(String input)	Counts the suffixes in an address. Examples for suffixes: <i>av</i> , <i>ave</i> , <i>avenue</i> , <i>st</i> , <i>str</i> , <i>street</i> .

Modifier and Type	Function	Description
static Boolean	isEqualNormalized(String a, String b)	Returns true if the two input strings are equal after making both lower case, removing the address suffixes, ordinals, special characters and spaces.
static String	normalizeZipUK(String input)	Normalizes a UK zip code by removing non-alphanumeric characters and, when possible, keeping only the outward-code portion.

Country class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
staticString	alpha2ToAlpha3(String country)	Converts an alpha2 country code to alpha3 .
staticInteger	alpha2ToIso(String country)	Converts an alpha2 country code to numeric.
staticString	alpha3ToAlpha2(String country)	Converts an alpha3 country code to alpha 2.
staticInteger	alpha3ToIso(String country)	Converts an alpha3 country code to numeric.
staticString	getContinentFromAlpha2(String country)	Gets the continent from a country by its alpha2 code.
staticString	getContinentFromAlpha3(String country)	Gets the continent from a country by its alpha3 code.
staticString	getContinentFromIso(Integer country)	Gets the continent from a country by its iso numeric code.
staticString	isoToAlpha2(Integer country)	Converts a numeric iso country code to alpha 2.
staticString	isoToAlpha3(Integer country)	Converts a numeric iso country code to alpha 3.

DistanceUtils class

Package: `com.feedzai.common.udfs.distance`

Modifier and Type	Function	Description
static Double	distance(String curZipCode, String prevZipCode, String curCityName, String prevCityName, String curStateCode, String prevStateCode)	Calculates the approximate distance between two transactions.
static Double	gaussianProbability(Double value, Double avg, Double std)	Calculates the gaussian probability (from avg, std) of the given value.
static Double	getMostAccurateLatitude(String zipCode, String cityName, String stateCode)	Calculates the most accurate latitude by zip code, city name and state code.
static Double	getMostAccurateLongitude(String zipCode, String cityName, String stateCode)	Calculates the most accurate longitude by zip code, city name and state code.
static Double	haversine(Double lat1, Double lon1, Double lat2, Double lon2)	Calculates the approximate distance between two locations.

Geo class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Double	easting(lat: Double, lon: Double, granularity: Double = 1.0)	Returns the X ordinate (easting) in kilometers from a latitude and longitude coordinate pair, apart from a granularity scaling factor. The granularity scale factor defaults to 1.0. Scales the result to 1:granularity km.
Double	northing(lat: Double, granularity: Double = 1.0)	Returns the Y ordinate (northing) in kilometers from a latitude coordinate pair, apart from a granularity scaling

Modifier and Type	Function	Description
		factor. The granularity scale factor defaults to 1.0. Scales the result to 1:granularity km.
String	extractContinentFromCountry(country: String)	Returns the continent code correspondent to the input country code.
Double	countryDistInKm(country1: String, country2: String)	Returns the distance of two countries in kilometers.
String	extractCountryCodeFromTopLevelDomain(topLevelDomain: String)	Returns an ISO 3166-1 alpha-2 country code from a top-level domain. Returns "AA" if there is no associated country, and null for invalid top-level domains. Example: <code>Geo.extractCountryCodeFromTopLevelDomain(".pt")</code> returns <code>PT</code> .

Location class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
staticDouble	calculateTravelVelocity(Double distance, Long timestampStart, Long timestampEnd)	Returns travel velocity in kilometers per hour.
staticDouble	distFrom(Double latitude1, Double longitude1, Double latitude2, Double longitude2)	Distance in meters between two points on Earth's surface.
staticDouble	distFromInKm(Double atitude1, Double longitude1, Double latitude2, Double longitude2)	Distance in kilometers between two points on Earth's surface.
static Double	getCartesianX(Double lat, Double lon)	Converts polar coordinates into absolute coordinates.
static Double	getCartesianY(Double lat, Double lon)	Converts polar coordinates into absolute coordinates.
static Double	getCartesianZ(Double lat)	Converts polar coordinates into absolute coordinates.
staticDouble	getHaversineDistance(Double latitude1, Double longitude1, Double latitude2, Double longitude2)	The Haversine distance (distance on a spherical surface) between two points on Earth's surface.
static Double	getLatitudeFromCartesian(Double x, Double y, Double z)	Converts absolute coordinates into polar coordinates.
static Double	getLongitudeFromCartesian(Double x, Double y, Double z)	Converts absolute coordinates into polar coordinates.

LocationBean class

Package: `com.feedzai.common.udfs.distance`

Functions using a JavaBean that stores the location information from the location CSV file in the ProfileUtils class. See the source of the location data in <http://download.geonames.org/export/zip/>.

Modifier and Type	Function	Description
String	getCityName()	Gets the city name of the location record.
Double	getLatitude()	Gets the latitude of the location record.
Double	getLongitude()	Gets the longitude of the location record.
String	getStateCode()	Gets the state code of the location record
String	getZipCode()	Gets the zip code of the location record.
void	setCityName(String cityName)	Sets the city name of the location record.

Modifier and Type	Function	Description
void	setLatitude(Double latitude)	Sets the latitude of the location record.
void	setLongitude(Double longitude)	Sets the longitude of the location record.
void	setStateCode(String stateCode)	Sets the state code of the location record.
void	setZipCode(String zipCode)	Sets the zip code of the location record.
String	toString()	Gets the string representation of the location record.

TaxHeaven class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Boolean	isTaxHaven(countryCode: String)	Returns true if the country is on the taxHavensISO2CountryCodes list of tax haven country codes. Based on Hines' list from 2010. The country code must be an ISO 3166-1 alpha-2 country code.

Zip class

Package: `com.feedzai.pulse.fraud.udf`

Functions for working with postal codes. Country strings must be in [ISO 3166-1 alpha-2](#) format, e.g. US, RU, PT.

Modifier and Type	Function	Description
static Double	distFrom(String country1, String zipCode1, String country2, String zipCode2)	Gets the distance in meters between two zip codes
static Double	getLatitude(String country, String zipCode)	Returns the latitude from a country and zip code pair
static Double	getLongitude(String country, String zipCode)	Returns the longitude from a country and zip code pair
static <code>com.feedzai.pulse.fraud.udf.Zip.ZipData</code>	getZipData(String country, String zipCode)	Returns the <code>{ZipData}</code> object with the specified country and zipCode

Working with business-related data

Card class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Boolean	isValidCardExp(cardExp: String)	Checks if a string is a valid card expiration date using <code>cardExp.matches(regex = "(?:0[1-9] 1[0-2])/[0-9]{2}")</code>
Boolean	isDebit(cardCategory: String)	Check whether the card category is "debit". Case insensitive.
Boolean	isCredit(cardCategory: String)	Check whether the card category is "credit". Case insensitive.
Boolean	isPremium(cardCategory: String)	Check whether the card category is "premium". Case insensitive.
Long	daysUntilExpiration(cardExp: String, timestamp: Long)	Returns the number of days until the card expires, counting from the value received in the timestamp parameter.
Integer	compareNameToCardname(uname: String, cname: String)	Normalizes the two received parameters, and returns the distance between the two strings using <code>Strings.strDistance()</code> . Case insensitive.

Business class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Integer	<code>extractMCGFromMCC(mcc: String)</code>	Returns the merchant category group (MCG) code of a given merchant category code (MCC).

Email class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Boolean	<code>isValid(email: String)</code>	Checks if an email is valid.
String	<code>normalizeEmail(email: String)</code>	Normalizes the given email string by converting to lower case and removing whitespace characters.
String	<code>extractEmailDomain(email: String)</code>	Extracts the domain of a nullable email address.
String	<code>extractEmailTopLevelDomain(email: String)</code>	Extracts the top-level domain of an email address (for example, <code>.pt</code>). Returns null if the email is null or invalid. Example: <code>Email.extractEmailTopLevelDomain("")</code> returns <code>.pt</code> .

EmailUtils class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static String	<code>normalizeEmailAddress(String str)</code>	Normalizes an email address: • Removes dots, whitespaces, plus signs, and any tag added after the plus sign. • Converts the email address to lower case. • Converts the <code>googlemail.com</code> domain to <code>gmail.com</code>.

Items class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Boolean	<code>keyContains(Object obj, String key, String value)</code>	Returns true if, for the given key, there is at least one value equal to the provided string value.
static Boolean	<code>keyContains(Object obj, String key, Boolean value)</code>	Returns true if, for the given key, there is at least one value equal to the provided boolean value.
static Boolean	<code>customFieldsContains(Object obj, String key, String value)</code>	Returns true if, for the given key, there is at least one value in <code>custom_fields</code> equal to the provided value.
static Boolean	<code>containsProduct(Object obj, String productName, String productStyle, String productColor)</code>	Returns true if there is at least one product matching the given name, style, and color combination.
static Integer	<code>getDistinctSizes(Object obj)</code>	Returns the number of distinct product sizes in the provided items set, or <code>-1</code> if the input is invalid.
static Integer	<code>getDistinctPrices(Object obj)</code>	Returns the number of distinct prices in the provided items set, or <code>-1</code> if the input is invalid.

Modifier and Type	Function	Description
static Integer	sumQuantity(Object obj)	Returns the sum of item <code>quantity</code> values, or <code>-1</code> if the input is invalid.
static Long	sumPrice(Object obj)	Returns the sum of item <code>price</code> values, or <code>-1</code> if the input is invalid.
static Long	sumPriceUSD(Object obj)	Returns the sum of item <code>price_usd</code> values, or <code>-1</code> if the input is invalid.

Phone class

Package: `com.feedzai.cloud.udf`

Country codes use ISO 3166-1 two-letter country-code format.

Modifier and Type	Function	Description
String	countryCode(number: String)	Returns the country code in ISO 3166-1 two-letter country-code format.
Boolean	isValidForCountry(number: String, country: String)	Returns true if the phone number is valid for a given country. If no country is provided, the function returns false.
String	numberType(number: String, country: String)	Gets the phone number type: Fixed-line, Mobile, Toll-free, Premium Rate, Shared Cost, VoIP and Personal Numbers.
Boolean	isMobile(number: String, country: String)	Returns true if the phone number type is mobile.
Boolean	isTollFree(number: String, country: String)	Returns true if the phone number type is toll-free.
Boolean	isVoIP(number: String, country: String)	Returns true if the phone number type is "VoIP".
Boolean	isPremiumRate(number: String, country: String)	Returns true if the phone number type is Premium Rate ("uan").
Boolean	isSharedCost(number: String, country: String)	Returns true if the phone number type is Shared Cost.
Boolean	isFixedLine(number: String, country: String)	Returns true if the phone number type is Fixed Line.
Boolean	isPersonalNumber(number: String, country: String)	Returns true if the phone number type is Personal Number.
String	normalizePhoneNumber(number: String)	Normalizes a phone number by removing spaces, replacing '+' by '00', and removing all other characters that are not numbers.

Working with timestamps

Time class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Long	getDays2Expire(Long timestamp, String expiredate)	Gets the time delta (in days) to expiration.
static Long	getDays2Expire(Long timestamp, String expiredate, String format)	Gets the time delta (in days) to expiration.
static Integer	getHourOfDay(Long timestamp, String timezone)	Gets the hour of the day for a specific timestamp.
	getLocalHourOfDay(Long timestamp)	

Modifier and Type	Function	Description
static Integer		Gets the hour of the day for a specific timestamp assuming the local timezone.
static Integer	getLocalMonth(Long timestamp)	Gets the month for a specific timestamp assuming the local timezone.
static Integer	getLocalMonthDay(Long timestamp)	Gets the day of the month for a specific timestamp assuming the local timezone.
static Integer	getLocalWeekDay(Long timestamp)	Gets the day of the week for a specific timestamp assuming the local timezone. 1 - Monday; 7 - Sunday.
static Integer	getLocalYear(Long timestamp)	Gets the year for a specific timestamp assuming the local timezone.
static Integer	getLocalYearDay(Long timestamp)	Gets the day of the year for a specific timestamp assuming the local timezone.
static Long	getMillis2Expire(Long timestamp, String expiredDate)	Gets the time delta (in milliseconds) to expiration.
static Long	getMillisAtStartOfDay(Integer year, Integer month, Integer dayOfMonth)	Obtains a UTC timestamp (in milliseconds) from the provided calendar year/month/dayOfMonth.
static Integer	getMonth(Long timestamp, String timezone)	Gets the month for a specific timestamp.
static Integer	getMonthDay(Long timestamp, String timezone)	Gets the day of the month for a specific timestamp.
static Integer	getUTCHourOfDay(Long timestamp)	Gets the hour of the day for a specific timestamp assuming the UTC timezone.
static Integer	getUTCMonth(Long timestamp)	Gets the month for a specific timestamp assuming the UTC timezone.
static Integer	getUTCMonthDay(Long timestamp)	Gets the day of the month for a specific timestamp assuming the UTC timezone.
static Integer	getUTCWeekDay(Long timestamp)	Gets the day of the week for a specific timestamp assuming the UTC timezone. 1 - Monday; 7 - Sunday.
static Integer	getUTCYear(Long timestamp)	Gets the year for a specific timestamp assuming the UTC timezone.
static Integer	getUTCYearDay(Long timestamp)	Gets the day of the year for a specific timestamp assuming the UTC timezone.
static Integer	getWeekDay(Long timestamp, String timezone)	Gets the day of the week for a specific timestamp. 1 - Monday; 7 - Sunday.
static Integer	getYear(Long timestamp, String timezone)	Gets the year for a specific timestamp.
static Integer	getYearDay(Long timestamp, String timezone)	Gets the day of the year for a specific timestamp.
static Long	parseDate(String strDate, String format)	Obtains a UTC timestamp (in milliseconds) from a string representation of a date.
static Long	parseExpirationDate(String strDate, String format)	Obtains a UTC timestamp (in milliseconds) from an expiration date string. If the format has no day component, returns the first timestamp of the subsequent month. Example: Time.parseExpirationDate("202003", "yyyyMM") returns the timestamp for 2020-04-01 00:00:00.000 (UTC).
static Long	parseDateOrNullOnError(String strDate, String format)	Checks if the given parameters can be parsed into a Date in milliseconds.

TimeTools class

Package: `com.feedzai.commons.udfs`

Modifier and Type	Function	Description
static Integer	dayOfMonth(Long timestamp)	Returns the "day of month" portion of the given timestamp at the default system timezone.
static Integer	dayOfMonth(Long timestamp, String timezone)	Returns the "day of month" portion of the given timestamp at the given timezone.
static Integer	dayOfWeek(Long timestamp)	Returns the "day of week" portion of the given timestamp at the default system timezone.
static Integer	dayOfWeek(Long timestamp, String timezone)	Returns the "day of week" portion of the given timestamp at the given timezone.
static Integer	dayOfYear(Long timestamp)	Returns the "day of year" portion of the given timestamp at the default system timezone.
static Integer	dayOfYear(Long timestamp, String timezone)	Returns the "day of year" portion of the given timestamp at the given timezone.
static Integer	hour(Long timestamp)	Returns the "hour" portion of the given timestamp at the default system timezone.
static Integer	hour(Long timestamp, String timezone)	Returns the "hour" portion of the given timestamp at the given timezone.
static Integer	minute(Long timestamp)	Returns the "minute" portion of the given timestamp at the default system timezone.
static Integer	minute(Long timestamp, String timezone)	Returns the "minute" portion of the given timestamp at the given timezone.
static Integer	month(Long timestamp)	Returns the "month" portion of the given timestamp at the default system
static Integer	month(Long timestamp, String timezone)	Returns the "month" portion of the given timestamp at the given timezone.
static Integer	second(Long timestamp)	Returns the "second" portion of the given timestamp at the default system timezone.
static Integer	second(Long timestamp, String timezone)	Returns the "second" portion of the given timestamp at the given timezone.
static Integer	weekOfYear(Long timestamp)	Returns the "weekOfYear" portion of the given timestamp at the default system timezone.
static Integer	weekOfYear(Long timestamp, String timezone)	Returns the "weekOfYear" portion of the given timestamp at the given timezone.
static Integer	year(Long timestamp)	Returns the "year" portion of the given timestamp at the default system timezone.
static Integer	year(Long timestamp, String timezone)	Returns the "year" portion of the given timestamp at the given timezone.

MoreTime class

Package: `com.feedzai.ds.udf`

Modifier and Type	Function	Description
static Integer	getMonthLength(Long timestamp, String timezone)	Calculates the number of days of the month for a given timestamp and timezone.

Modifier and Type	Function	Description
static Integer	getUTCMonthLength(Long timestamp)	Calculates the number of days of the month for a given timestamp in UTC timezone.

Working with JSON

Json [class](#)

Package: `com.feedzai.cloud.udf`

For big JSON documents, consider using the functions described in the `AbstractCachedJson` class.

Modifier and Type	Function	Description
String	getString(id: String, json: String, path: String, forceCoersion: Boolean)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, it can be coerced to one, if the corresponding parameter is set. Otherwise the function logs an error and returns null.
String	getString(id: String, json: String, path: String)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, the function logs an error and returns null.
String	getAsString(id: String, json: String, path: String)	Gets a field from a JSON document, as a String, according to a given JSON path expression. If the field is not a string, it will be coerced to one.
Boolean	getBoolean(id: String, json: String, path: String)	Gets a Boolean field from a JSON document, according to a given JSON path expression.
Integer	getInteger(id: String, json: String, path: String)	Gets an Integer field from a JSON document, according to a given JSON path expression.
Long	getLong(id: String, json: String, path: String)	Gets a Long field from a JSON document, according to a given JSON path expression.
Float	getFloat(id: String, json: String, path: String)	Gets a Float field from a JSON document, according to a given JSON path expression.
Double	getDouble(id: String, json: String, path: String)	Gets a Double field from a JSON document, according to a given JSON path expression.
String	getAndConcatStrings(id: String, json: String, paths: JList[String], delim: String)	Gets multiple fields, converted to String, from a JSON document as a concatenated string with their values, according to a given JSON path expression.
Boolean	contains(id: String, json: String, path: String)	Checks if a field exists in a JSON document, according to a given JSON path expression.

AbstractCachedJson [class](#)

Package: `com.feedzai.cloud.udf`

This class includes functions for handling big JSON strings. The functions in this class cache the result of the parsing to avoid reparsing on every call. For smaller files, consider using the functions described under the `Json` class.

Modifier and Type	Function	Description
String	getString(id: String, json: String, path: String, forceCoersion: Boolean)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, it can be coerced to one, if the corresponding parameter is set. Otherwise the function logs an error and returns null.
String		

Modifier and Type	Function	Description
	getString(id: String, json: String, path: String)	Gets a String field from a JSON document, according to a given JSON path expression. If the field is not a String, the function logs an error and returns null.
String	getAsString(id: String, json: String, path: String)	Gets a field from a JSON document, as a String, according to a given JSON path expression. If the field is not a string, it will be coerced to one.
Boolean	getBoolean(id: String, json: String, path: String)	Gets a Boolean field from a JSON document, according to a given JSON path expression.
Integer	getInteger(id: String, json: String, path: String)	Gets an Integer field from a JSON document, according to a given JSON path expression.
Long	getLong(id: String, json: String, path: String)	Gets a Long field from a JSON document, according to a given JSON path expression.
Float	getFloat(id: String, json: String, path: String)	Gets a Float field from a JSON document, according to a given JSON path expression.
Double	getDouble(id: String, json: String, path: String)	Gets a Double field from a JSON document, according to a given JSON path expression.
String	getAndConcatStrings(id: String, json: String, paths: JList[String], delim: String)	Gets multiple fields, converted to String, from a JSON document as a concatenated string with their values, according to a given JSON path expression.
Boolean	contains(id: String, json: String, path: String)	Checks if a field exists in a JSON document, according to a given JSON path expression.

Working with collections and complex fields

Collections class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static String	getFieldByKey(Object complexField, String key, String keyValue, String fieldToRetrieve)	Retrieves a field from a complex field by matching tuples with the provided key and key value. Returns an empty string on any validation or lookup error. Example: given <code>sender_account_limits</code> with <code>{"event_type": "ACC_LIMIT", "daily_amount": 500000}</code> , calling <code>com.feedzai.pulse.fraud.udf.Collections.getFieldByKey(sender_account_limits, "event_type", "ACC_LIMIT", "daily_amount")</code> returns <code>500000</code> .
static Boolean	contains(java.util.Collection<java.lang.Object> collection, java.lang.Object valueToFind)	Checks if a collection contains a specific value.
static Boolean	containsCaseInsensitive(java.util.Collection<java.lang.Object> collection, java.lang.String valueToFind)	Checks if a collection contains a specific value. Ignores casing when testing for equality.

Collections class

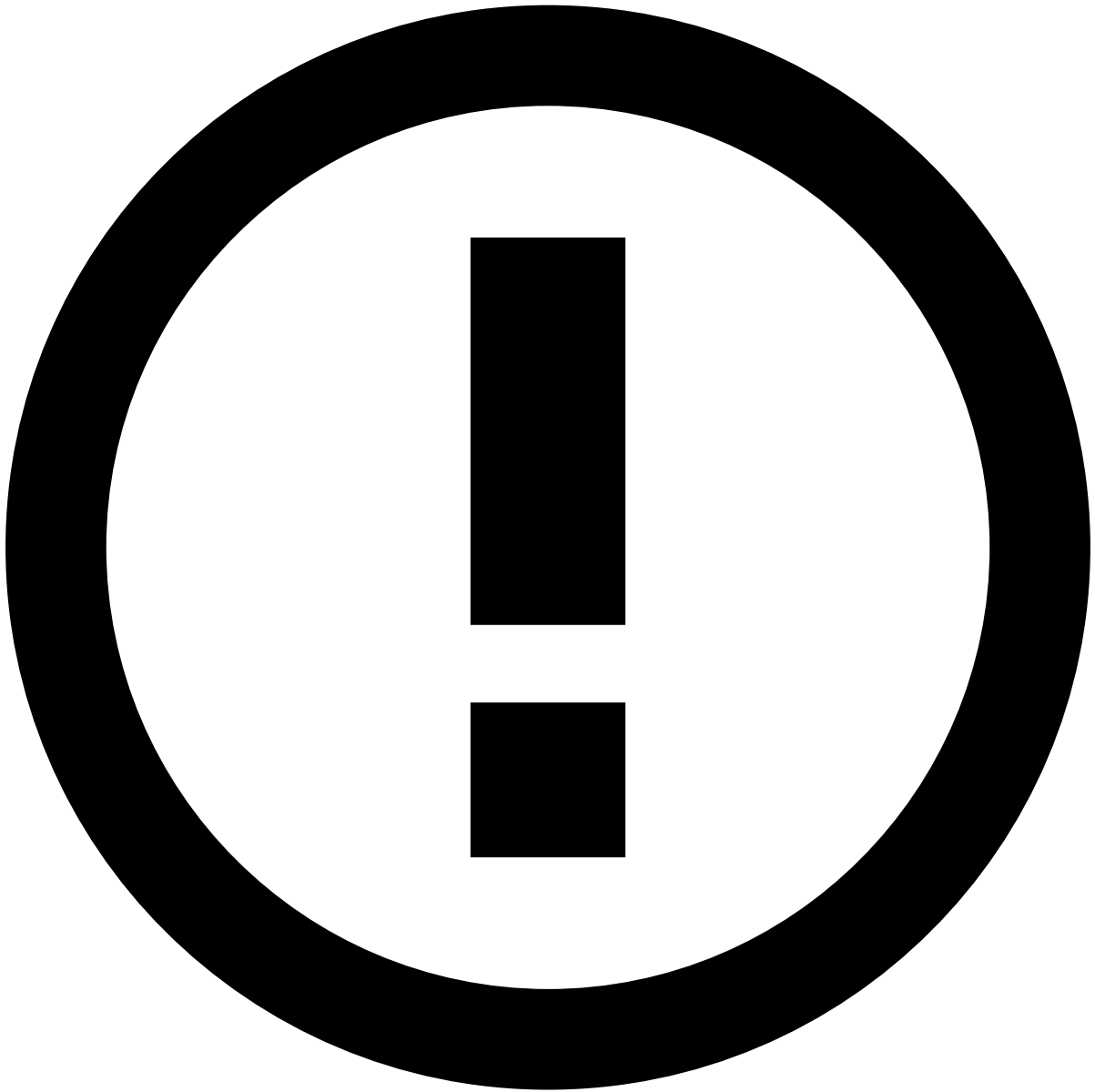
Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
Tuple	getSampleTuple(set: JSet[Tuple])	Retrieves the next tuple from a JSet of Tuples.

CollectionUDFs class

Package: `com.feedzai.pulse.pkernel.runtime`

As the CollectionUDFs class is included in the runtime package of Pulse, you don't need to define an alias to use the functions of this class.



Lists vs Maps

The collection called Map of Pulse Query Language (PQL) and Rule Definition Language (RDL) is the equivalent of a Pulse list created under **Lists** . The following table refers to them as **Maps**.

Modifier and Type	Function	Description
Integer	count(Object collection)	Returns the number of entries in a collection Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a Collection.

Modifier and Type	Function	Description
		<pre>// Returns 2 CollectionUDF.count(map("a": 1, "b": 2))</pre>
Boolean	contains(Object collection, Object element)	Checks if <i>element</i> is present in <i>collection</i>. Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a Collection. <pre>// Returns true CollectionUDF.contains(map("a": 1), 1)</pre>
Integer	getInt(Object collection, Object? key)	Returns the integer value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns 5 CollectionUDF.getInt(list(1, 3, 5), -1)</pre>
Long	getLong(Object collection, Object key)	Returns the long value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns 3 CollectionUDF.getLong(map("a": 3), "a")</pre>
Float	getFloat(Object collection, Object key)	Returns the float value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer.

Modifier and Type	Function	Description
		<pre>// Returns null CollectionUDF.getFloat(list(null), 0)</pre>
Double	getDouble(Object collection, Object key)	Returns the double value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a number. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns 2.5 CollectionUDF.getDouble(list(3.14, 2.5), 1)</pre>
string	getString(Object collection, Object key)	Returns the string value associated to <i>key</i>. If <i>collection</i> is a List, negative keys start at the end of the list (0 is the first and -1 is the last element). Fails with <i>UnsupportedOperationException</i> if <i>collection</i> is neither a Map nor a List or if the value matching key is not null and not a string. Fails with <i>IllegalArgumentException</i> if <i>collection</i> is a List but <i>key</i> is not a non-null Integer. <pre>// Returns "value" CollectionUDF.getString(map("key": "value"), "key")</pre>

Tuples class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
AnyRef	getField(tuple: Tuple, field: String)	Returns the field value, or null if the tuple doesn't contain the field or if the tuple is null.

ComplexFields class

Package: `com.feedzai.cloud.udf`

Modifier and Type	Function	Description
String	getString(map: JMap[String, String], key: String)	Get a string field from a string JMap, according to a given key. If the field is not a string or doesn't exist, the function returns null. This function can't be used in Plans, because PQL does not support calls to UDFs with non-primitive parameters. As an alternative use the getStringObj UDF instead.

Modifier and Type	Function	Description
		This function can't be used in Plans , because PQL does not support calls to UDFs with non-primitive parameters. As an alternative use the getStringObj UDF instead.
String	getStringObj(map: AnyRef, key: String)	Get a string field from a string JMap, according to a given key. If the field is not a string or doesn't exist, the function returns null. This function is analogous to getString() but receives the map as an AnyRef. This function is useful when using complex fields in PQL, because PQL does not support calls to UDFs with non-primitive parameters.

Mathematical calculations

Maths class

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Integer	binNumber(Double value, Double binSize)	Gets the bin number of a value
static Double	coalescedSafeDivision(Double numerator, Double denominator)	Divides numerator by denominator with safeguards for bad divisions
static Double	diffNormalizeAndCap(Double value, Double expected, Double cap)	Computes the difference between a value and the expected, normalized by the expected value, capped to a maximum
static Double	logOfNormalPDF(Double mean, Double std, Double x)	Natural logarithm of normalPDF(Double, Double, Double)
static Double	normalCDF(Double mean, Double std, Double x)	Cumulative distribution function (CDF) for a normal distribution with parameters <code>mean</code> and <code>std</code> at value <code>x</code> .
static Double	minimum(Double a, Double b, Double c)	Three-way minimum
static Integer	minimum(Integer a, Integer b, Integer c)	Three-way minimum
static Double	maximum(Double a, Double b, Double c)	Three-way maximum
static Integer	maximum(Integer a, Integer b, Integer c)	Three-way maximum
static Double	normalPDF(Double mean, Double std, Double x)	Density of the normal distribution with parameters <code>mean</code> and <code>std</code> at the value <code>x</code>
static Double	safeDivision(Double numerator, Double denominator)	Executes a safe division that returns the division value or zero if the operation is not a number or division by zero
static Double	safeDivision(Long numerator, Long denominator)	Executes a safe division that returns the division value or zero if the operation is not a number or division by zero
static Double	toDeg(Double angle)	Converts radians to degrees
static Double	toRad(Double angle)	Converts degrees to radians



PDF

The `normalPDF` function has a corner case defined for when the standard deviation (`std`) is zero. In that case, it will return either 1 if the value `x` equals the mean parameter or 0 otherwise. This might affect your feature distribution because the outcome of `normalPDF` is the likelihood, which is not bound.

Working with model scores

ScoreTransformationUdfs class

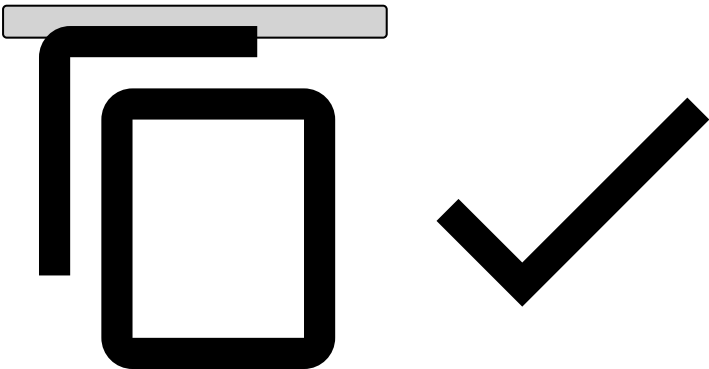
Package: com.feedzai.ds.udf

Modifier and Type	Function	Description
static Double	amountDependentScoreTransformation(Double rawScore, Double amount, Double intensity, Double cap)	Transforms the model score using the transaction amount parameterized by an intensity factor and an amount cap.
static Double	thresholdAndAmountDependentScoreTransformation(double rawScore, long amount, double anchor, Object amountBinAndThresholdObject, double defaultThreshold)	Transforms the model using amount-dependent thresholds and an anchor value.

Usage examples:

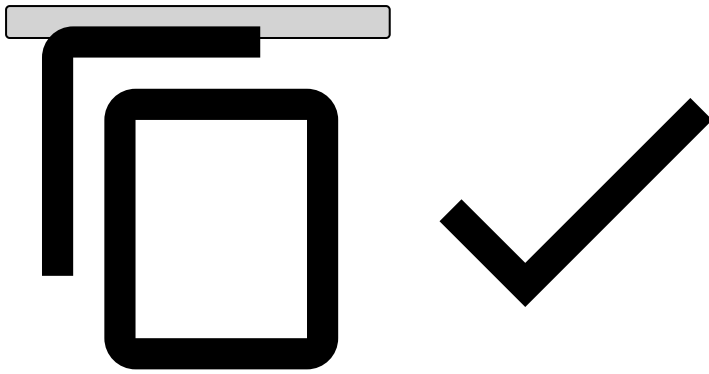
Transforming scores with the amount-dependent score transformation:

```
com.feedzai.ds.udf.ScoreTransformationUdfs.amountDependentScoreTransformation(  
    fraud_true_scored // original score to be transformed  
    amount           // amount field  
    0.3              // lower intensity means lower amounts are greatly affected  
  
    5000000 // when 'amount > cap', the score remains unchanged. In this case scores for amounts  
            over $50k are unchanged  
)
```



Transforming scores with piecewise linear transformations dependent on the amount:

```
com.feedzai.ds.udf.ScoreTransformationUdfs.thresholdAndAmountDependentScoreTransformation(  
    fraud_true_scored, // original score to be transformed  
    amount,           // amount field  
    0.850,             // lower anchor means the raw score is more affected by the transformations  
    {1000: 0.800, 6750: 0.750, 20420: 0.654}, // mapping amounts to thresholds {amount1:  
    threshold1, amount2: threshold2, ...}  
    0.500 // default threshold, ie for amounts greater than the max amount above  
)
```



Cryptography🔒📄

Crypto class🔒📄

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static String	cryptoHash(String algo, Object obj)	Hashes an object using a hashing algorithm.
static String	cryptoHash(String algo, Object obj, String salt)	Hashes an object using a hashing algorithm.
static String	md5(Object obj)	Computes the md5 hash of the object.
static String	sha512(Object obj)	Computes the sha-512 hash of the object.

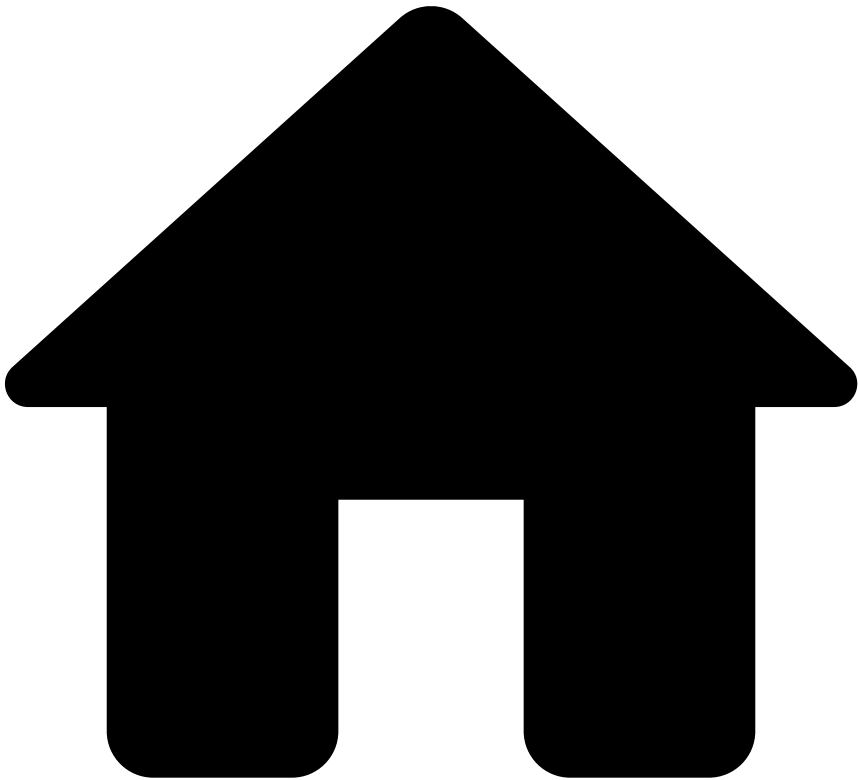
Debugging🔒📄

Debug class🔒📄

Package: `com.feedzai.pulse.fraud.udf`

Modifier and Type	Function	Description
static Double	debug(Double value, String comment)	Debug function to log a message and a value and return the original object.
static Float	debug(Float value, String comment)	Debug function to log a message and a value and return the original object.
static Integer	debug(Integer value, String comment)	Debug function to log a message and a value and return the original object.
static Long	debug(Long value, String comment)	Debug function to log a message and a value and return the original object.
static Short	debug(Short value, String comment)	Debug function to log a message and a value and return the original object.
static Integer	debugObject(Object value, String comment)	Debug function to log a message and a value.

.



- [Rule Guide](#)
- [Creating Rules](#)
- Data Types in Rules

On this page

Was this page helpful? 

Data Types in Rules

RDL uses its own data types. When using a field or a list, the values get automatically converted to these data types. This topic describes the data types that RDL uses.

Primitive data types

The following table shows the **primitive data types** of RDL. These data types are equivalent to the Java primitive data types. To see the exact definition or range for each, check the [Java documentation](#).

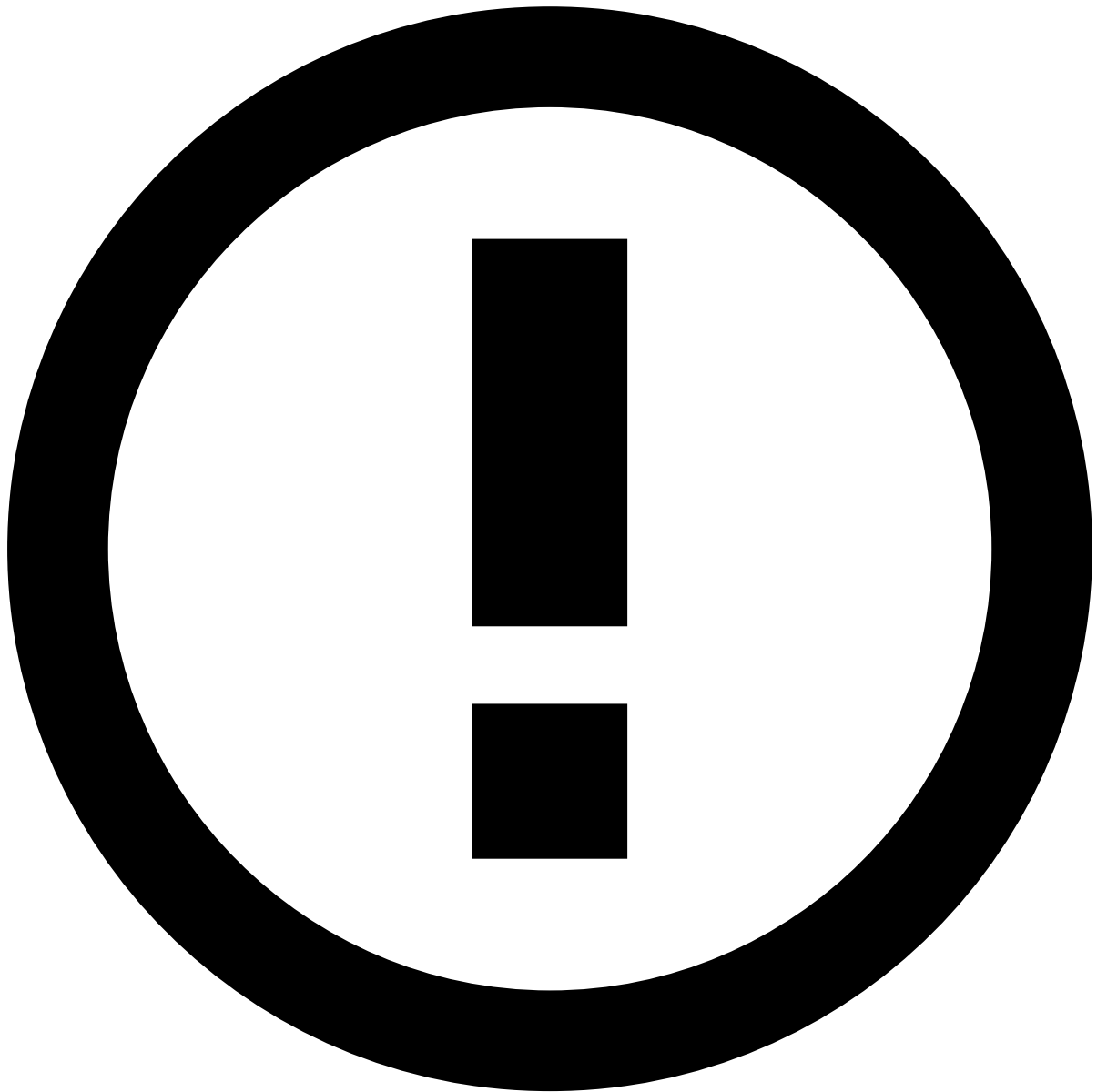
Data type	Examples
int	

Data type	Examples
	-289 0 42
long	123456789 1l -2L
float	0.99 -1.23e-100f 3.14F
double	1234567.89012 1.23456e300d 1.23456e-300d 1e1D
bool	true false
string	"John Doe" 'John Doe'

Collections

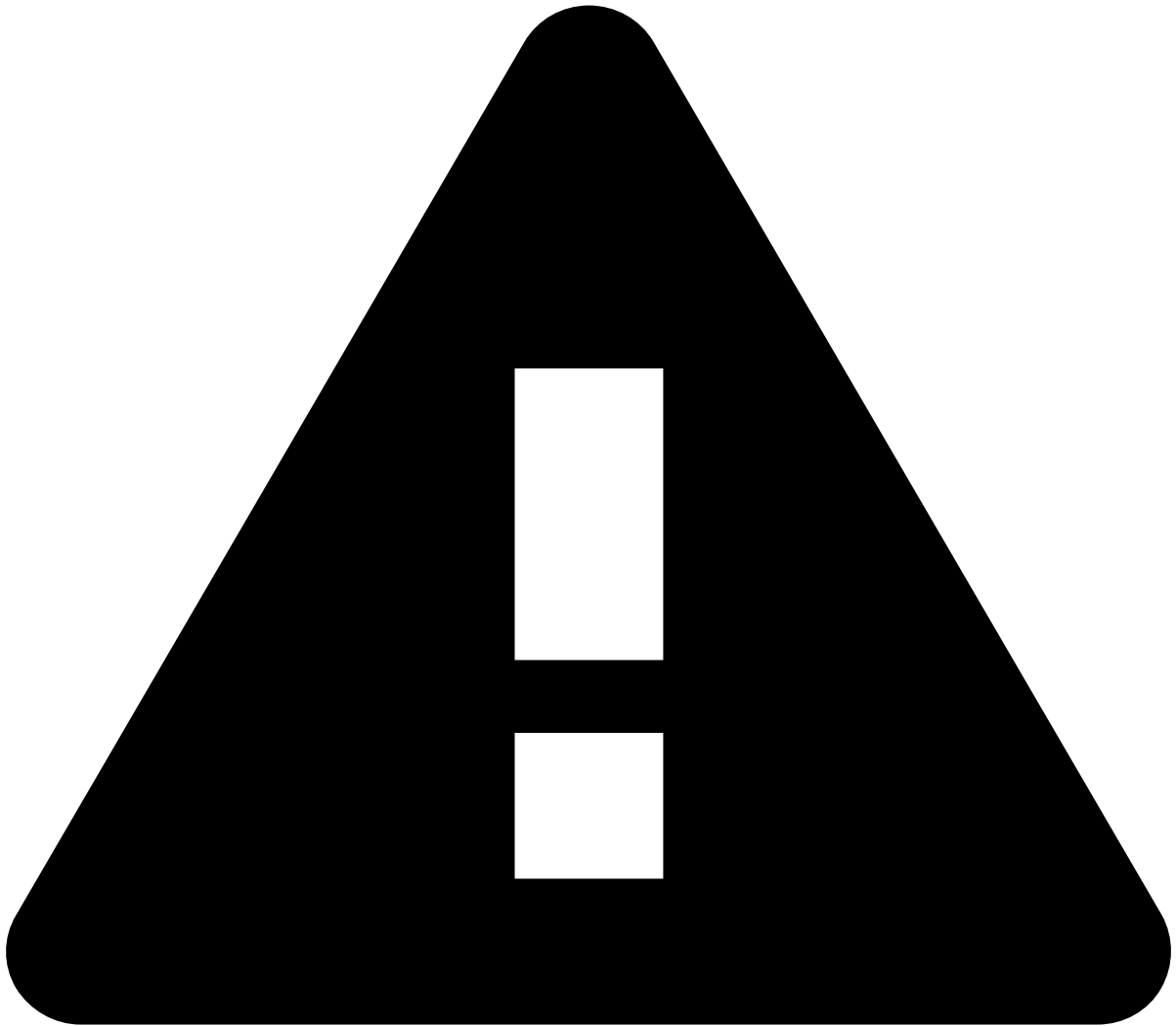
RDL uses the following collections: lists, sets, maps, and tuples. They are the equivalent of the [complex data types](#) that you can use in the schema of a data source.

In RDL, the collection called map is the equivalent of a [list](#) created under **Lists** . Hence all collection operators can be used on them.



Best practice

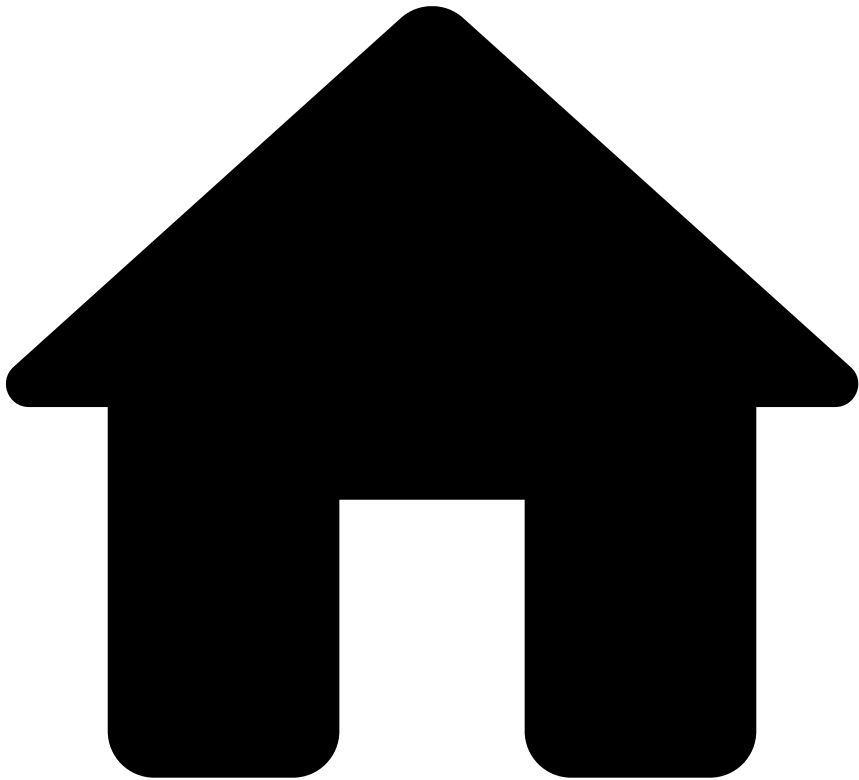
Although it is possible to define collections directly in the rule using RDL, **the recommended method is to create a [list](#) outside of the rule, under Lists** . This way you can maintain the rules more easily, since a change in the list items does not imply the change and deployment of the whole rule.



Explicitly iterating over lists is not recommended

It is not recommended to iterate explicitly over Pulse lists (e.g., using `filter`, `all`, etc), as this is highly inefficient. Pulse lists are optimized for `contains`/`lookup` operations.

•



- [Rule Guide](#)
- [Creating Rules](#)
- Operators in Rules

On this page

Operators in Rules

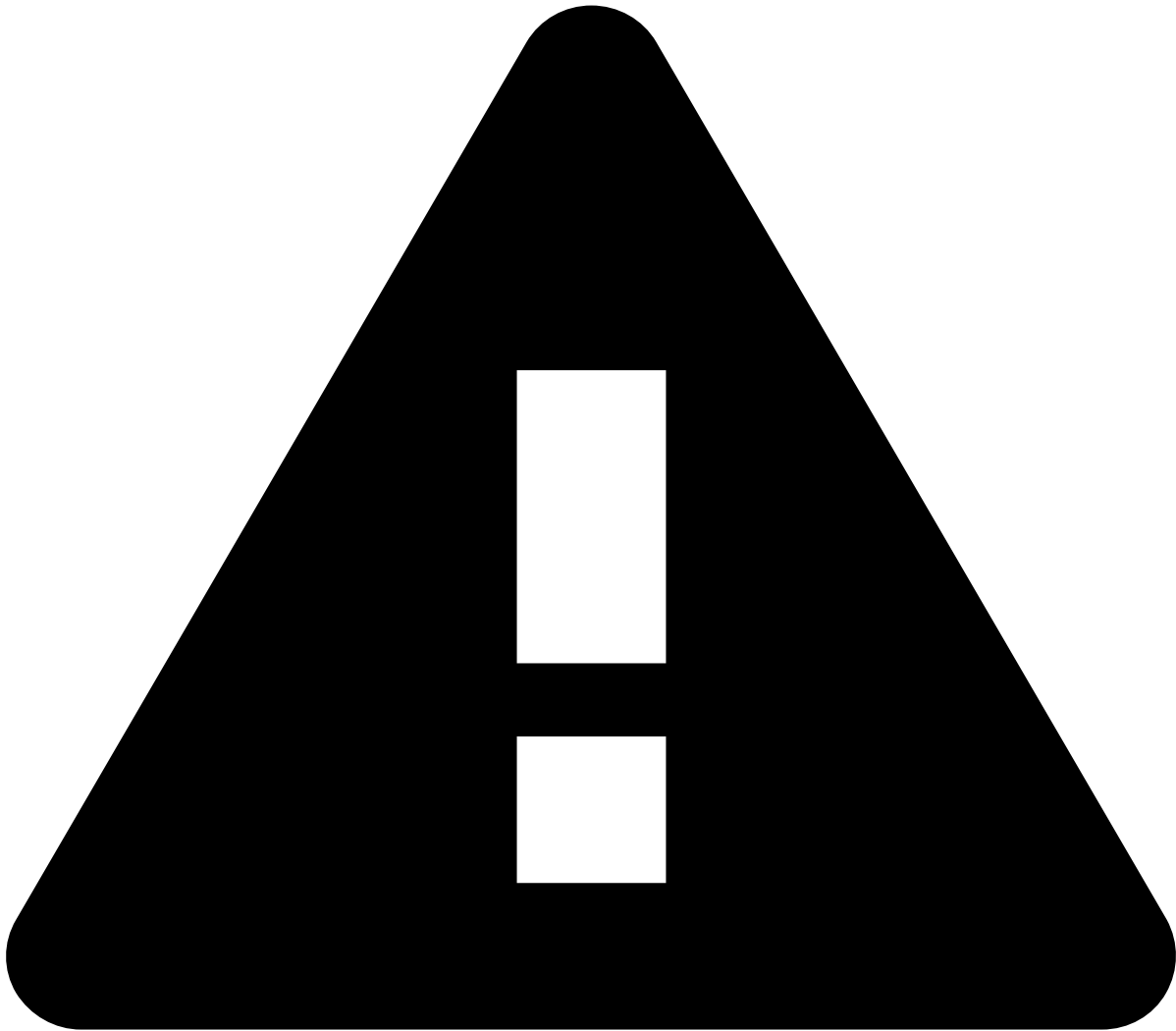
Was this page helpful? 

The Rule Definition Language (RDL) is designed for defining variables and conditions for Rules in Pulse. RDL queries event data, and transforms it using operators and function calls.

This topic gives you a list of operators that you can use when writing rules.

Operators

The following table explains the meaning of each supported operator.



Rules failing with null values

Rules fail to be evaluated if an RDL expression in the condition or in the variables returns a null error, i. e. if the last evaluated operator in the expression's tree returns a null error. In that case Pulse skips that rule and continues the workflow with the next rule or the next workflow element.

Check the **Input parameters** column in the following table to see whether the respective operator returns a null exception when receiving a null value in the input value.

In the case of operations over collections, check the **Description** column of the table to see which ones return a null exception when the collection is `null`.

Comparison operators

`==`, `!=`

Description	Input parameters	Examples
Checks if the two operands are equal or not.	Any primitive data type . The two operands must be of the same type.	<pre>card_present_flag == true</pre>

Description	Input parameters	Examples
		<pre>merchant_country == 'EU'</pre>

<, >, <=, >=

Description	Input parameters	Examples
Compares two values of an ordered domain.	Numeric data types: <code>int</code> , <code>long</code> , <code>float</code> , <code>double</code>	<pre>amount > average_amount_per_card_in_1_month * 1.5</pre>

Arithmetic operators

+, -, *, /, %

Description	Input parameters	Examples
The arithmetic operators obey the following type promotion rules: 1. If at least one of the arguments is a double, the result is a double. 2. If at least one of the arguments is a float, the result is a float, unless the result is a double because of the first rule. 3. If at least one of the arguments is a long, the result is a long, unless the result is a double or float because of the first two rules.	Numeric types: <code>int</code>, <code>long</code>, <code>float</code>, <code>double</code> The arithmetic operators throw a null exception if any of the input parameters is <code>null</code>.	<pre>5%2</pre> Returns the integer remainder of the division 5/2.

Boolean operators

and

Description	Input parameters	Examples
Returns <code>true</code> if both received arguments are true.	Boolean The and operator throws a null exception in the following cases: • <code>nullExpression</code> and <code>true</code> • <code>nullExpression</code> and <code>false</code> • <code>true</code> and <code>nullExpression</code>	<pre>card_present_flag == false and merchant_country == 'RU'</pre>

or

Description	Input parameters	Examples
Returns <code>true</code> if at least one of the received arguments is true.	Boolean	<pre>CardWatchList.contains(tokenized_card_PAN) or RiskyCountries.contains(card_country)</pre>

!

Description	Input parameters	Examples
Inverts the value of a boolean expression.	Boolean The <code>!</code> operator throws a null exception if the input parameter is <code>null</code> .	<pre>! CardWatchList.contains(tokeni zed_card_PAN)</pre>

Functions on RDL Maps, and Pulse Lists¹

Lists created in Pulse under **Lists** are represented in RDL as RDL Maps.

MyList [*my_key*]

Description	Input parameters	Examples
Returns the value stored in the List under the referred key.	<i>my_key</i> : the same type as the key of the list	<pre>transactionContinent = Countries[country_code].continent_code;</pre> Fetches the continent code for the current <i>country_code</i> from the <i>Countries</i> list.

MyList.contains(my_key);

Description	Input parameters	Examples
Checks if the List contains a value for a specific key.	<i>my_key</i> : the same type as the key of the list	<pre>isInBlackList = blackList.contains(tokenized_card_PAN) ;</pre>

RDL Lists and RDL Sets¹

Note that RDL Lists are not the same as Lists created in Pulse under **Lists** .

MyCollection [*condition*]

Description	Input parameters	Examples
Creates a window of the collection according to the condition. If the condition defines a single element of a collection, the [] can be used to access the data in that single element. Example: <pre>countryList[country_code].continent_c ode</pre>	condition: see examples on the right	- Create a new list with the countries where the current card (<i>card_number</i>) is whitelisted: <pre>myWhiteListCountries = WhiteList[card_number] .count ry_code;</pre> - Create a new list (aka. window) with the last 5 events that have the same card number as the current event: <pre>lastEvents = event [5 tuples by card_number];</pre> - Access the second event in the lastEvents window: <pre>secondEvent = lastEvents[2];</pre> - Create a new list with the last five events where the terminal type was X or Y:

Description	Input parameters	Examples
		<pre>lastEventsWithTerminalTypeXZ = event [5 tuples where terminal_type == "X" or terminal_type == "Y"];</pre> • Create a new list (aka. window) with the last 3 events of the last 24 hours that have the same card number as the current event: <pre>lastEvents24Hours = event [3 tuples by card_number for 24h];</pre>

MyCollection.contains(expression)

Description	Input parameters	Examples
Checks if the collection contains a specific element.	expression: must match the data type of the collection elements	<pre>cardWatchList.contains(tokeni zed_card_PAN)</pre>

MyCollection.filter(my_field => condition)

Description	Input parameters	Examples
Filters the elements of a collection based on a condition. The operator throws a null exception if the collection is <code>null</code>.	• <i>my_field</i>: a field of the collection • condition: boolean expression	<pre>lastEvents.filter(amount => amount > 200)</pre> Returns only the events where the amount is over 200.

Explicitly iterating over Lists is not recommended

It is not recommended to iterate explicitly over Pulse Lists (e.g., using filter, all, etc), as this is highly inefficient. Pulse Lists are optimized for contains/lookup operations.

MyCollection.any(my_field => condition)

Description	Input parameters	Examples
Returns <code>true</code> if at least one element of the collection fulfills the given condition. The operator throws a null exception if the collection is <code>null</code>. Use it carefully on large collections, as the rule will have to do a check on all incoming transactions, against all list elements.	• <i>my_field</i>: a field of the collection • condition: boolean expression	<pre>myWhiteListCountries.any(country => Countries[country].continent_code == transaction_continent)</pre> Returns <code>true</code> if the <i>myWhiteListCountries</i> list contains any country that is on the same continent as the current event.

Explicitly iterating over Lists is not recommended

It is not recommended to iterate explicitly over Pulse Lists (e.g., using filter, all, etc), as this is highly inefficient. Pulse Lists are optimized for contains/lookup operations.

MyCollection.all(my_field => condition)

Description	Input parameters	Examples
	• <i>my_field</i>: a field of the collection	

Description	Input parameters	Examples
Returns <code>true</code> if all elements of the collection fulfill the condition. Use it carefully on large collections, as the rule will have to do a check on all incoming transactions, against all collection elements. The operator throws a null exception if the collection is <code>null</code>.	condition: boolean expression	<pre>customerDevices.all(device -> device.score < 500)</pre>

Explicitly iterating over Lists is not recommended

It is not recommended to iterate explicitly over Pulse Lists (e.g., using filter, all, etc), as this is highly inefficient. Pulse Lists are optimized for contains/lookup operations.

MyCollection.sum(my_field)

Description	Input parameters	Examples
Sums all the numeric values in the collection. The operator throws a null exception if the collection is <code>null</code>.	<i>my_field</i>: a field of the collection	<pre>lastEvents.sum(amount)</pre>

MyCollection.count()

Description	Input parameters	Examples
Counts the elements in the collection. The operator throws a null exception if the collection is <code>null</code>.	N/A	<pre>lastEvents.count()</pre>

MyCollection.min(my_field)

Description	Input parameters	Examples
Returns the lowest element in the collection. The operator throws a null exception if the collection is <code>null</code>.	<i>my_field</i>: a field of the collection	<pre>lastEvents.min(amount)</pre>

MyCollection.max(my_field)

Description	Input parameters	Examples
Returns the highest element in the collection. The operator throws a null exception if the collection is <code>null</code>.	<i>my_field</i>: a field of the collection	<pre>lastEvents.max(amount)</pre>

MyCollection.avg(my_field)

Description	Input parameters	Examples
Returns the average of the elements in the collection. The operator throws a null exception if the collection is <code>null</code>.	<i>my_field</i>: a field of the collection	<pre>lastEvents.avg(amount)</pre>

RDL Lists

Note that RDL Lists are **not** the same as Lists created in Pulse under **Lists**. See the previous section.

MyRDLList.at(my_index)

Description	Input parameters	Examples
Applies only to RDL Lists. Returns the value stored in the RDL List under the referred index. The at operator throws a null exception if the input parameter is <code>null</code>.	<i>my_index</i>: integer	<pre>lastEvents.at(2)</pre> Returns the second event of the <i>lastEvents</i> list.

Selecting a value dynamically

outputVariable = **match** if condition then expression1; else expression2; end end

Description	Input parameters	Examples
Implements an if-then-else block. If the condition is true, the output variable receives the value of expression1. Otherwise the output variable receives the value of expression2.	<i>outputVariable</i>: any type. - condition: boolean expression. - expression1 and expression2: the same data type as the <i>outputVariable</i>.	<pre>events = event [3 tuples by card_number for 24h]; prevEvent = match if events.count() > 2 then return events.at(2); else if events. count() > 1 then return events.at(1); else return null; end end Returns the last event from the <i>events</i> list.</pre>

Others

expression1 ?? expression2

Description	Input parameters	Examples
The coalesce (??) operator returns expression1, unless expression1 is <code>null</code>, in which case it returns expression2. In RDL, the coalesce operator has the highest precedence when used in conjunction with arithmetic or boolean operators, meaning that it gets evaluated before the other operators.	Any	Return -1 if <i>MyList</i> is empty, otherwise return the amount average of the list: <pre>MyList.avg(amount) ?? -1</pre>

Description	Input parameters	Examples
		Example to illustrate the precedence in relation to other operators: <pre>1 + null ?? 3 + 4</pre> This expression is equivalent to <pre>1 + (null ?? 3) + 4</pre> ...and returns 8 as the result.

UDF (User-defined function) calls

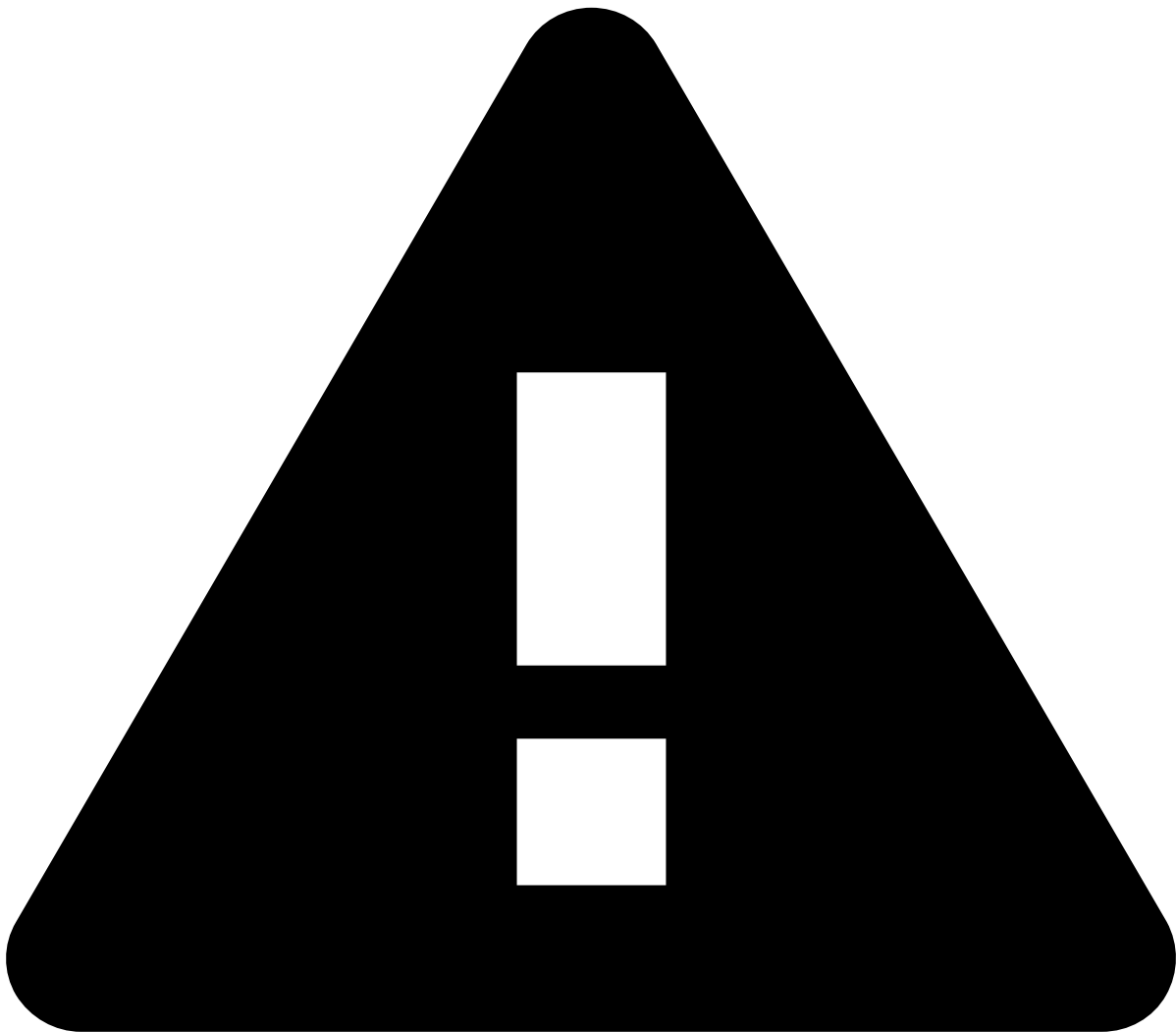
Description	Input parameters	Examples
You can call User-Defined Functions to perform specific computations.		- Find the location of the substring "775" in the <i>card_acceptor_id</i>: <pre>matchLocation = StringUtils.indexOf(card_acceptor_id , "775");</pre> - Fetch the first letter 3 positions after the match location found with the above query: <pre>letterAfterMatch = StringUtils.substring(card_acceptor_id);</pre> - Check if the email_domain field matches the defined Java pattern: <pre>isNumericDomain = StringTools.regexMatches("[0-9]+.com", email_domain);</pre> - From the complex field called <i>countries_map</i>, retrieves the string value associated to the "USA" key: <pre>CollectionUDFs.getString(countries_map, "USA")</pre> - From the complex field called <i>countries_map</i>, retrieves the integer value associated to the "USA" key: <pre>CollectionUDFs.getInt(call_code_map, "USA")</pre>

Operator precedence

The following table lists RDL operators according to their precedence order. Operators at the top of the table have higher precedence.

Operator	Precedence
grouping	<code>(expr)</code>
unary	<code>+expr</code> <code>-expr</code> <code>!</code>
cast	<code>(type) expr</code>
coalesce	

Operator	Precedence
	?? Note that this operator has a different precedence in PQL.
arithmetic – multiplicative	* / %
arithmetic – additive	+ -
comparison	> >= == <= < !=
logical – OR	or
logical – AND	and
assignment	=



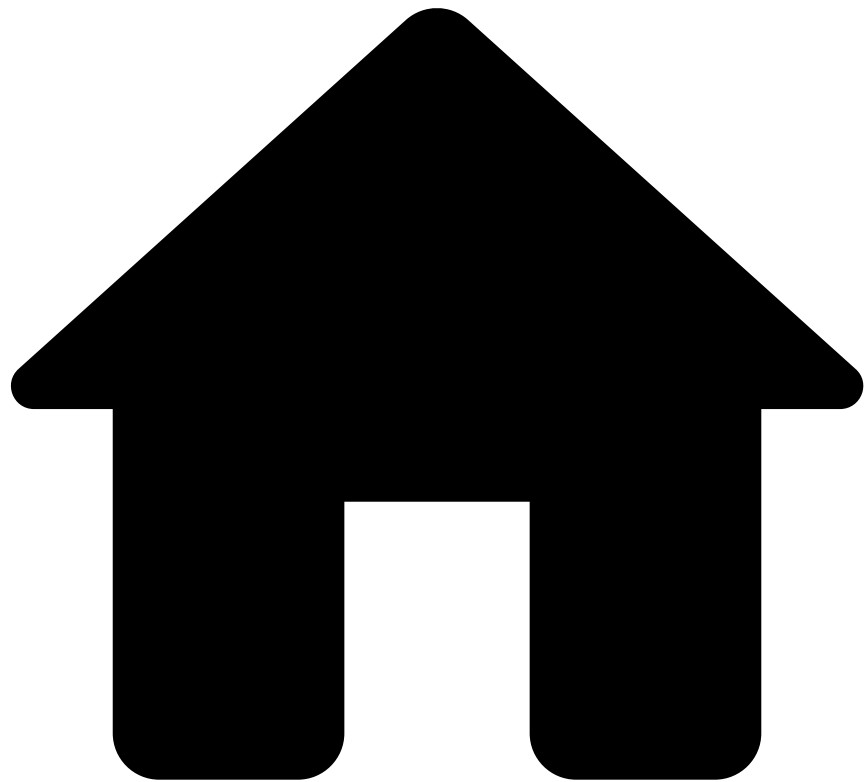
warning

Unlike other languages, in RDL the operator `or` takes precedence over the `and` operator.

The following rules apply:

- Operators with higher precedence are evaluated before the ones with lower precedence.
- Operators in the same line have the same precedence:
 - All binary operators except for the assignment operator are evaluated from left to right.
 - Assignment operator is evaluated right to left.

•



- [Signing in to Pulse](#)
- [Signing in with Single Sign-On](#)
- [Signing in as Pulse Administrator](#)

On this page

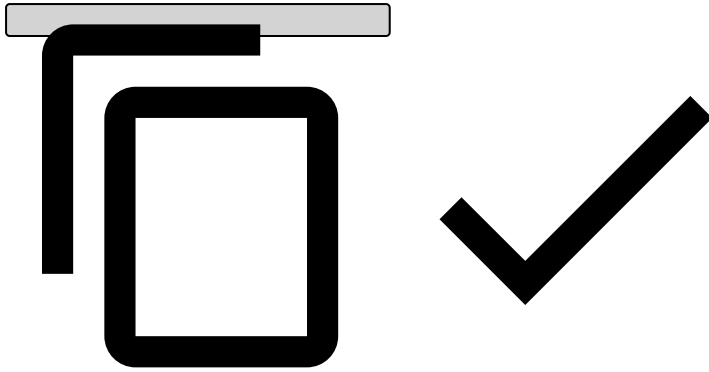
Signing in as Pulse Administrator

Was this page helpful? 

Why

With Single Sign on (SSO) enabled on Pulse, regular users can only sign in via SP-initiated. However, Pulse Administrators can still sign in using a static URL:

```
http://<BASE_PULSE_ADDRESS>/pulseviews/login/
```



The main driver for this option is to allow Pulse Administrators to create new users using this URL, which otherwise would only be possible with user provisioning. Note that this option should be only used in special situations since the suggested method to create a user is through user provisioning.

How To

To sign in as Pulse administrator and create a new user execute the following actions:

1. Access the static URL provided for Pulse Administrators:
`http://<BASE_PULSE_ADDRESS>/pulseviews/login/`
2. Enter your Administrator's credentials.
3. Select **Administration** in the toolbar.
4. Open the **Security** tab and add a new user.
5. Fill in the user information and select **Save**. On most SSO setups, user authorization is performed on the IDP. In this case the user creation form is the following:

Note that in the case of SSO, the fields are **read-only**. If SSO is configured with authorization in Pulse, you have to specify if the user is an SSO user or not, and set its authorization properties.

Next Steps

Learn more about sign in scenarios with SSO:

- [Signing in via SP-initiated](#)

On this page

Signing in via IdP-initiated

Was this page helpful? 

Why

IdP-initiated is a scenario of SSO that is supported by Pulse. In this case, the user signs in to the IdP domain and then has access to the applications configured in the IdP.

How To

For this example, two pulse instances have been [configured in the IdP as SPs](#):

- A Demo environment configured in the IdP as PULSE DEMO.
- A Local environment configured in the IdP as PULSE.



SP applications

Note that the two Pulse instances are used here as an example. In a production system the IdP could be configured with any application used by the client.

To use the example of sign in to Pulse via IdP-initiated execute the following actions (assuming that you are not already signed in to the IdP):

1. Access the IdP webpage configured for user authentication.
2. Choose the option *Sign in to this site* and click on Continue to Sign In. The following sign in form is provided:
3. Enter your credentials to Sign In. This user credentials should have been already [configured in the IdP](#).
4. Choose the Pulse instance (e.g. PULSE DEMO) that you would like to access and click on Go. Note that, if the user does not exist in the Pulse instance as a Pulse User, the user will be created automatically by user provisioning at the first sign in to the Pulse instance.

Note that you are redirected to an active session of the PULSE DEMO.

5. Navigate to the Administration tool.
6. Open the Security view and select that you have signed in. Note that the fields are read-only because users that have SSO enable can only be created with the concept of user provisioning. User information previously configured in the IdP is mapped to the Pulse User. This information may contain **User Description**, **Language**, **Groups**, **Roles**, and **Tenants**.

Next Steps📖

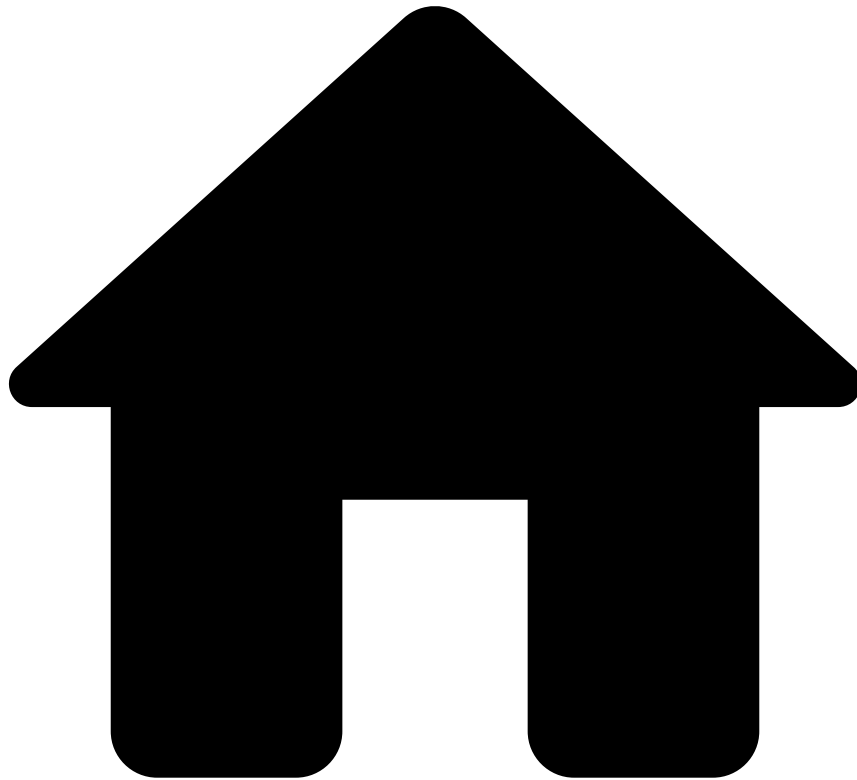
Learn more about the login scenario in which a user access a Pulse instance directly:

- [Signing in via SP-initiated](#)

Learn how to configure SPs and users in the IdP:

- [Add a Relying Party Trust](#)
- [Add new users](#)

•



- Signing in to Pulse
- [Signing in with Single Sign-On](#)
- Signing in via SP-initiated

On this page

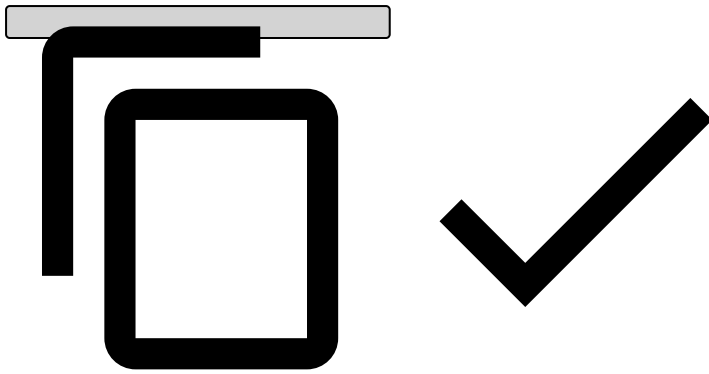
Signing in via SP-initiated

Was this page helpful? 

Why

SP-initiated is a use case of SSO that is supported by Pulse. A typical scenario of SP-initiated is when the user receives a URL by e-mail or the user wants to access a Feedzai product using a bookmark. E.g.:

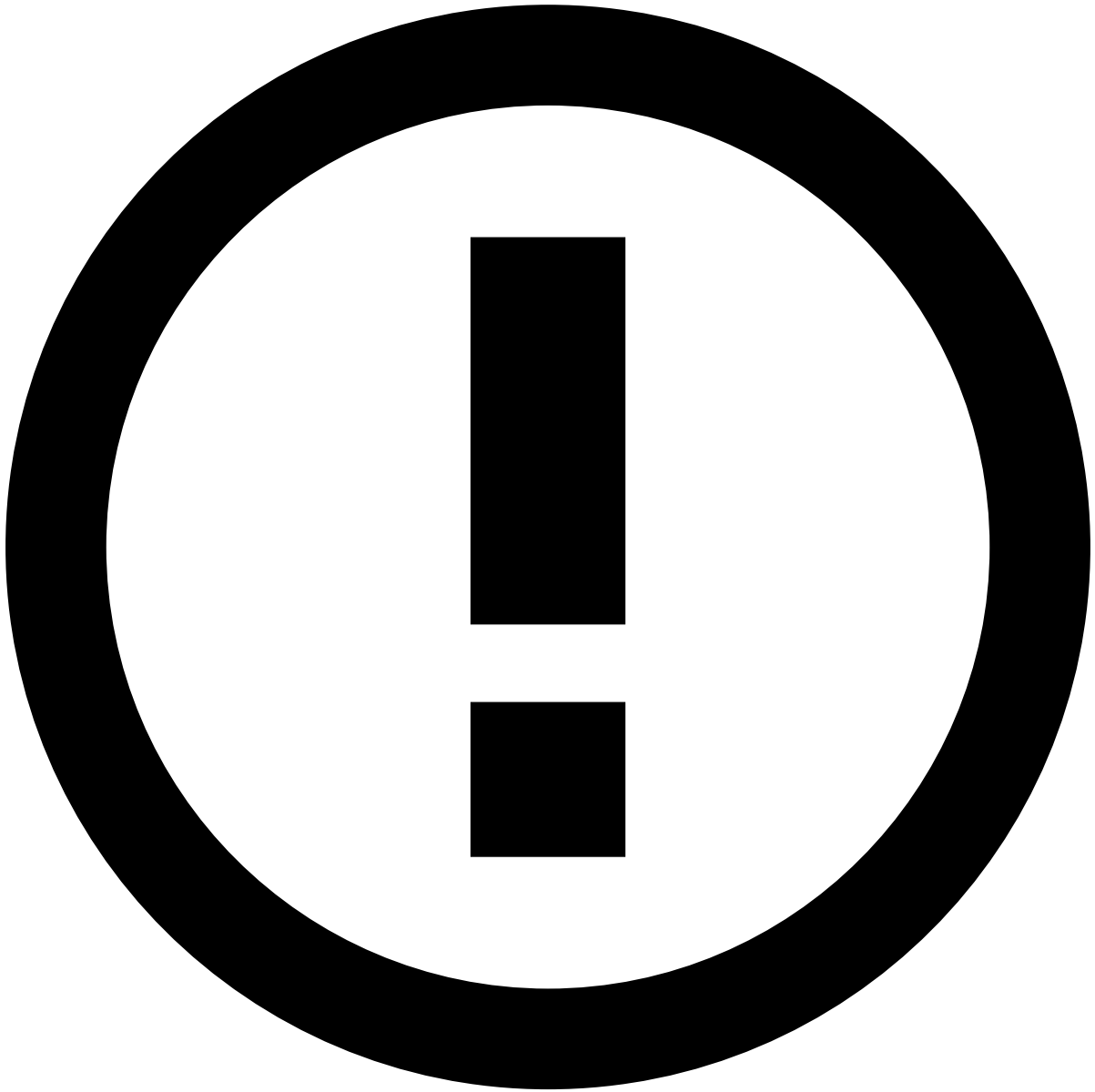
```
http://<BASE_PULSE_ADDRESS>/pulseviews/management/#apps/
```



How To

For this example, two pulse instances have been [configured in the IdP as SPs](#)

- A Demo environment configured in the IdP as PULSE DEMO.
- A Local environment configured in the IdP as PULSE.



SP applications

Note that the two Pulse instances are used here as an example. In a production system the IdP could be configured with any application used by the client.

To use the example of login to Pulse via SP-initiated, execute the following actions (assuming that you are not already signed in to the IdP):

1. Access one of the URLs that have been configured as SP in the IdP. Note that you are redirected to the IdP URL to enter your credentials.
2. Enter your SSO credentials to sign in. These user credentials should have been already [configured in the IdP](#). As a result, you are redirected to the Pulse URL entered initially, i.e. to the internal path corresponding to the URL that you were trying to access on Step 1. Note that, if the user does not exist in the Pulse instance as a **Pulse User**, the user will be created automatically by user provisioning at the first sign in to the Pulse instance.

If you check the IdP URL you can verify that you are signed in:

3. Access the other URL that has been configured as an SP. Note that, because you have already signed in to the IdP you have immediate access to this Pulse instance, as opposed to be redirected to the IdP webpage to enter again the credentials. However, the user authentication is still being performed between the SP and the IdP. Since this authentication occurs behind the scenes you might only notice in the title bar the URL flicking as a result of the internal authentication process (i.e. seamless login experience).
4. Navigate to the Administration tool.
5. Open the Security view and select that you have signed in. Note that the fields are read-only because users that have SSO enable can only be created with the concept of user provisioning. User information previously configured in the IdP is mapped to the **Pulse User**. This information may contain **User Description**, Language, Groups, Roles, and **Tenants**.

Next Steps

Learn how to configure SPs and users in the IdP:

- [Add a Relying Party Trust](#)
- [Add new users](#)

On this page

Signing out via IdP-initiated

Was this page helpful? 

Why

Sign out via IdP-initiated is a method to sign out from all the applications at once without having to sign out from each application individually.

How To

- For this example, two pulse instances have been [configured in the IdP as SPs](#):
 - A Demo environment configured in the IdP as PULSE DEMO.
 - A Local environment configured in the IdP as PULSE.



SP applications

Note that the two Pulse instances are used here as an example. In a production system the IdP could be configured with any application used by the client.

To understand the concept of signing out from all the applications via IdP-initiated execute the following actions:

1. Sign in to two Pulse instances using either [SP-initiated](#) or [IdP-initiated](#).
2. Access the main IdP URL and choose the first Sign Out option (Sign out from all the sites that you have accessed).



danger

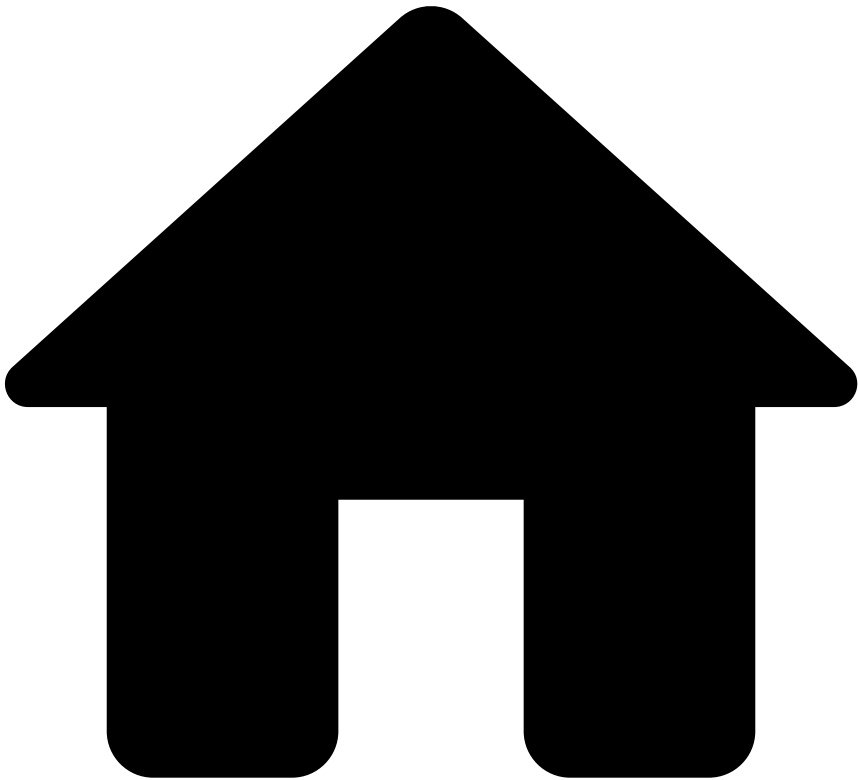
Note that the second Sign Out option (Sign out from this site) is only to sign out from the IdP website, and does not sign out the SP option that is selected under Select one of the following sites. Also note that, when selecting this second Sign Out option, the IdP does not communicate with any Pulse instance, and thus, the SP sessions that are already signed will remain active.

3. Verify that both Pulse instances initiated in Step 1 are signed out (you may have to reload the page or perform any operation within the Pulse instance). Note that you are redirected to an URL that allows you to sign in via SP-initiated.

Next Steps

- Learn more about sign in scenarios with SSO:
 - [Signing in via IdP-initiated](#)
 - [Signing in via SP-initiated](#)

•



- Signing in to Pulse
- Signing in with Two-Factor Authentication
- Configuring Two-Factor Authentication

On this page

Configuring Two-Factor Authentication

Was this page helpful? 

Why

To sign in with Two-Factor Authentication (**2FA**), first you need to configure **2FA** on your mobile device to be able to generate a code that can be then entered in the sign in form.

In Pulse, 2FA uses Time-Based One Time Password (TOTP) to generate a temporary code. Learn more about [TOTP in Pulse](#).

How to

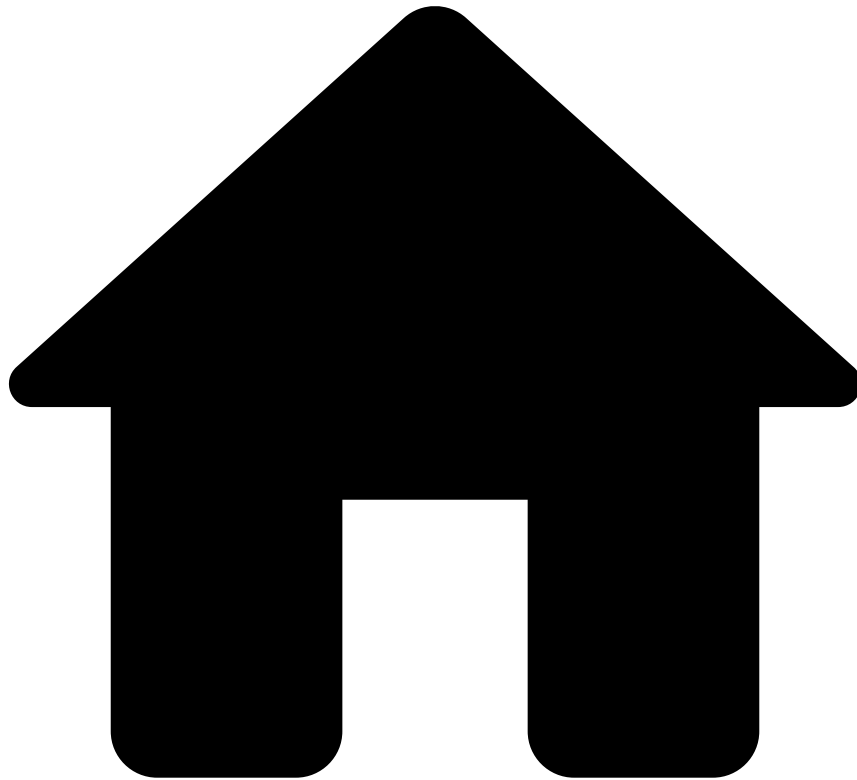
To configure **2FA**, execute the following steps:

1. Access a Pulse URL. If you are not signed in yet you are redirected to the main Pulse Sign in form.
2. In this form, insert your Username and Password and use the **Sign in** button to continue to the next step.
3. You are prompted with an interface showing an alert that **2FA** has not been configured yet. Use the **Configure** button to continue this process.
4. To configure **2FA** on your device you have two options:
 1. Automatically, by reading the QR code using the code generator application on your device.
 2. Manually, by selecting the option *You can view the code and add it manually* and typing on your mobile device the Account Key provided in the configuration interface.
5. Insert the PIN code generated by the mobile application that you are using and finish this configuration by selecting the **Register Application** button. If the PIN Code is valid, a new interface is displayed as shown in the figure below. The interface provides a reset code to reset the 2FA configuration. Learn more about [resetting 2FA](#).
6. Use the **Go to Application** button to access Pulse.

Learn More

Learn more about [Resetting Two-Factor Authentication](#).

•



- Signing in to Pulse
- Signing in with Two-Factor Authentication
- Resetting Two-Factor Authentication

On this page

Resetting Two-Factor Authentication

Was this page helpful? 

Why

If you want to add Two-Factor authentication (2FA) to one or more devices or if you lost your mobile device in which 2FA was configured, you can reset the 2FA configuration. To reset 2FA you have to follow one of these scenarios:

- Resetting 2FA of a signed in user: if you want to add more devices
- Resetting 2FA of a user signed out with the reset key: if you lost your mobile device but you still have the reset key
- Resetting 2FA of a user signed out without the reset key: if you lost your mobile device and you do not have the reset key.

The steps to reset 2FA for each of these scenarios are described below.

How to

To reset 2FA execute the following steps according to the scenario that suits you.

Resetting 2FA of a signed in user

1. Navigate to the Settings tool on Pulse UI to open the Settings menu
2. Use the **Reset Authentication** button to proceed to the reset the 2FA configuration.
3. You are redirected to the sign in form. Enter you PIN code and use the **Verify Code** button to finish the last step of reset your 2FA configuration.
4. Your are redirected to the sign in form that requires configuration of 2FA on your mobile device. To learn how to configure 2FA on a mobile device check the page [Configuring Two-Factor Authentication](#).

Resetting 2FA of a user signed out with the reset key

1. Access any Pulse URL to be redirected to the main Pulse sign in form.
2. In this form, insert your Username and Password use the **Sign in** button to continue to the next step.
3. You are prompted with the sign in form associated with the second validation step to access Pulse. Insert the reset code, instead of using the PIN Code generated by your mobile device, and Reset Two-Factor Authentication.

You are prompted with an interface to configure 2FA on your mobile. For a full description on how to configure 2FA on your device check the Section [Configuring Two-Factor Authentication](#).

Resetting 2FA of a user signed out without the reset key

In this scenario, you have to request your Administrator to execute the reset of your 2FA configuration. The Administrator has an endpoint that allows disabling 2FA to a specific user. The following steps show how an Administrator can disable your 2FA configuration so that you can re-configure it again.

1. The administrator navigates to the Administration tool.
2. The administrator open the Security view.
3. The administrator selects your user account to edit. The User form is opened with information regarding the **Status** of 2FA.
4. in the User form, the administrator selects the **Disable Authentication** button to reset your 2FA configurations on your mobile device(s).
5. The Administrator is prompted with a warning window with the option **Disable** to proceed with the reset.

Next time you sign in to Pulse you are requested to configure 2FA on your mobile device. To learn how to configure 2FA on a mobile device check the page [Configuring Two-Factor Authentication](#).

Learn More

Learn more about [Configuring Two-Factor Authentication](#).

-



- [Working with Applications](#)
- [Exporting resources](#)
- Bundle structure

On this page

Bundle structure

Was this page helpful? 

The bundle generated by the Export Resource option contains JSON files that are organized in separate folders according to resource type allowing easy management by different end-users. The content of the files is structured in a human readable format. This allows you to:

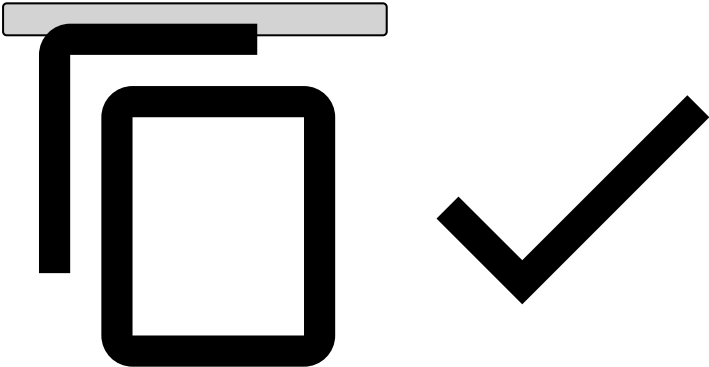
- Quickly inspect the content of each file.
- Add or delete any item of your resource list (e.g. add a **Rules Project** to the bundle).
- Peer-review your resources.

Folder structure

The **Export Resource** generates a resource bundle that is compressed in the zip format.

When you unzip the file you will notice that the content is organized in different folders with the following structure:

```
lists
  example_list_1.json
  example_list_2.json
models
  model_project_name_1
    example_model_1.json
    example_model_2.json
    example_model_1.model.gz # This is the binary of the example_model_1.json
  model_project_name_2
    example_model_3.json
    example_model_4.json
plans
  example_plan_1.json
  example_plan_2.json
rulesProjects
  example_rules_project_1.json
  example_rules_project_2.json
  example_rules_project_3.json
tenantGroups
  example_group_1.json
  example_group_2.json
tenant
  example_tenant_1.json
  example_tenant_2.json
  example_tenant_3.json
```



Each folder contains a JSON file for each element added to the resource bundle.



Export Lists

Lists are always exported by default, and thus, the folder will contain a JSON file for each List that you have on your Pulse Application. However, you can choose to include the **List** items of each **List**.

File structure📁

You can open JSON files using a standard editor to inspect the content. The table below provides the main key information that you can find inside in each file for each resource folder.

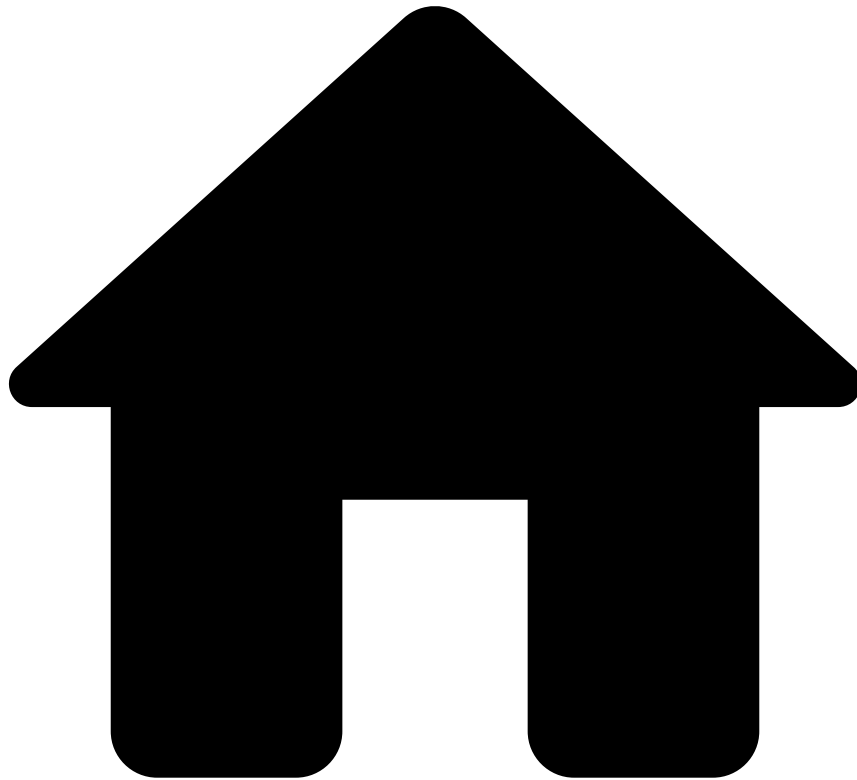
Folder	Description
lists	Lists are automatically exported. This folder contains a JSON file for each List that you have configured on your Pulse App .
models	

Folder	Description
	This folder contains a JSON file for each Model included in the resource bundle organized in separate folders for each Model Project. Note that you can include the binary of each Model when exporting the resources. In this case, the Model Project folder will contain the binary of each Model.
plans	This folder contains a JSON file for each Plan included in the resource bundle. Note that you can include the execution of each Plan when exporting the resources. The metadata of the execution is stored within the file JSON file of the Plan.
rulesProjects	This folder contains a JSON file for each Rules Project included in the resource bundle. Note that you can include a list of snapshots for each Rules Project when exporting the resources. The metadata of the snapshots is stored within the file JSON file of the Rules Project.
tenantGroups	This folder contains metadata of the Tenant Groups that you have in your Pulse app .
tenants	This folder contains metadata of the Tenant that you have in your Pulse app .

Learn More

Now that you know how the structure of the export bundle, check the page [Comparing resource versions](#) to learn how to analyze the content of different bundles.

•



- [Working with Applications](#)
- [Exporting resources](#)
- Comparing resource versions

On this page

Comparing resource versions

Was this page helpful? 

Why

In a typical project, as you are evolving a given Pulse resource, a common workflow is to export the resource and version the bundle using a distributed version control system to keep track of the changes. At some point, you might have the need to compare the JSON file of two resource versions to check the differences between the versions.

The JSON files are structured in a human readable format that allows you to quickly compare changes between two files.

How to

As a typical scenario, imagine that you need to compare a resource that is versioned in a storing system with a resource that is currently on your **Pulse app**. To do so, execute the following steps:

1. Select the Options button to open the application options menu:
2. Select the option  Export Resources to open the export wizard.
3. Select the tab corresponding to the resource that you want to export.
4. Add the resource that you want to compare to the export list.
5. Use the button  Export Bundle to save the resource bundle in your operating system.
6. Use any standard software that allows comparing two JSON files. See the next subsections that cover examples of different use cases.

Use cases

The aim of this guide is not to provide a solution to the possible use cases, but rather to give some examples to help you understand how to perform comparisons. You might have to adapt the method according to your needs.

Models

Imagine the following scenario:

- You have two **Models** with the same Data Source but using different fields.

When you compare the two **Models**, you can quickly inspect the features included in each **Model** by looking at "ignoredFields" as shown below:

Plans

Imagine the following scenario:

- You have a map operator to add the field Is Fraud. This operator converts the values of the feature *Fraud* from "True" or "False" to numerical values 1 or 0 and stores them in the new field.
- By mistake, this field was configured with the type Data Time, and once the bug was identified it was changed to the type Numeric. The following image shows the results between two JSON files:

Rules Projects

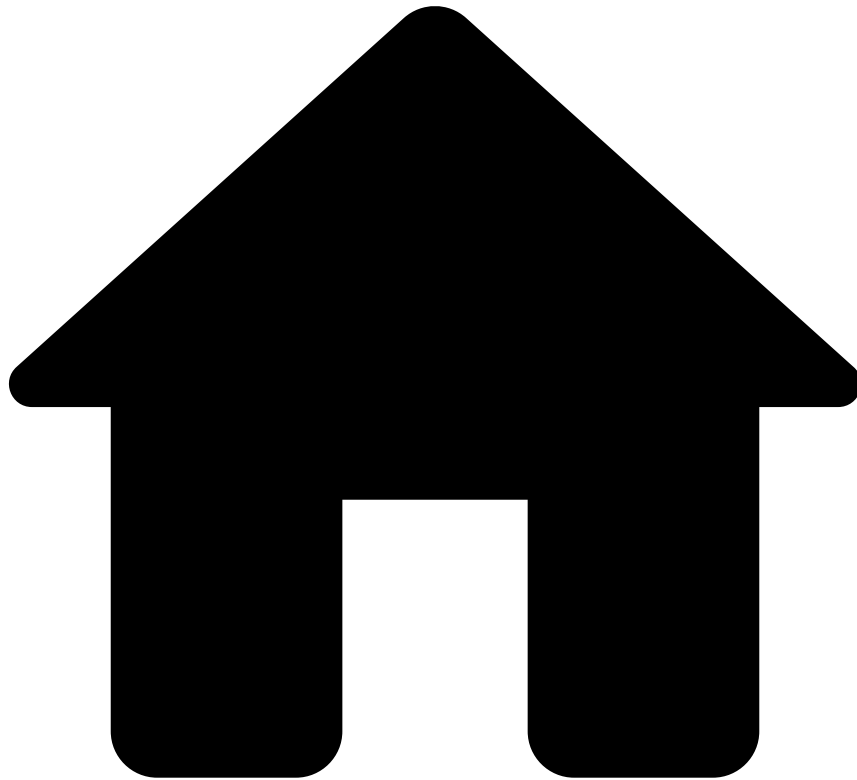
Imagine the following scenario:

- A new **Rule** has been added since the last time that the resource was exported.
- This **Rule** blocks credit card transactions whose number match the card number in a *Black List*. The following image shows the result between two JSON files, with and without the **Rule**:

Learn More

Now that you know how to export resources and compare different versions, check the page [Importing Resources](#) to learn how to import your resource bundle.

-



- [Component Encryption](#)
- Troubleshooting Tips

Was this page helpful? 

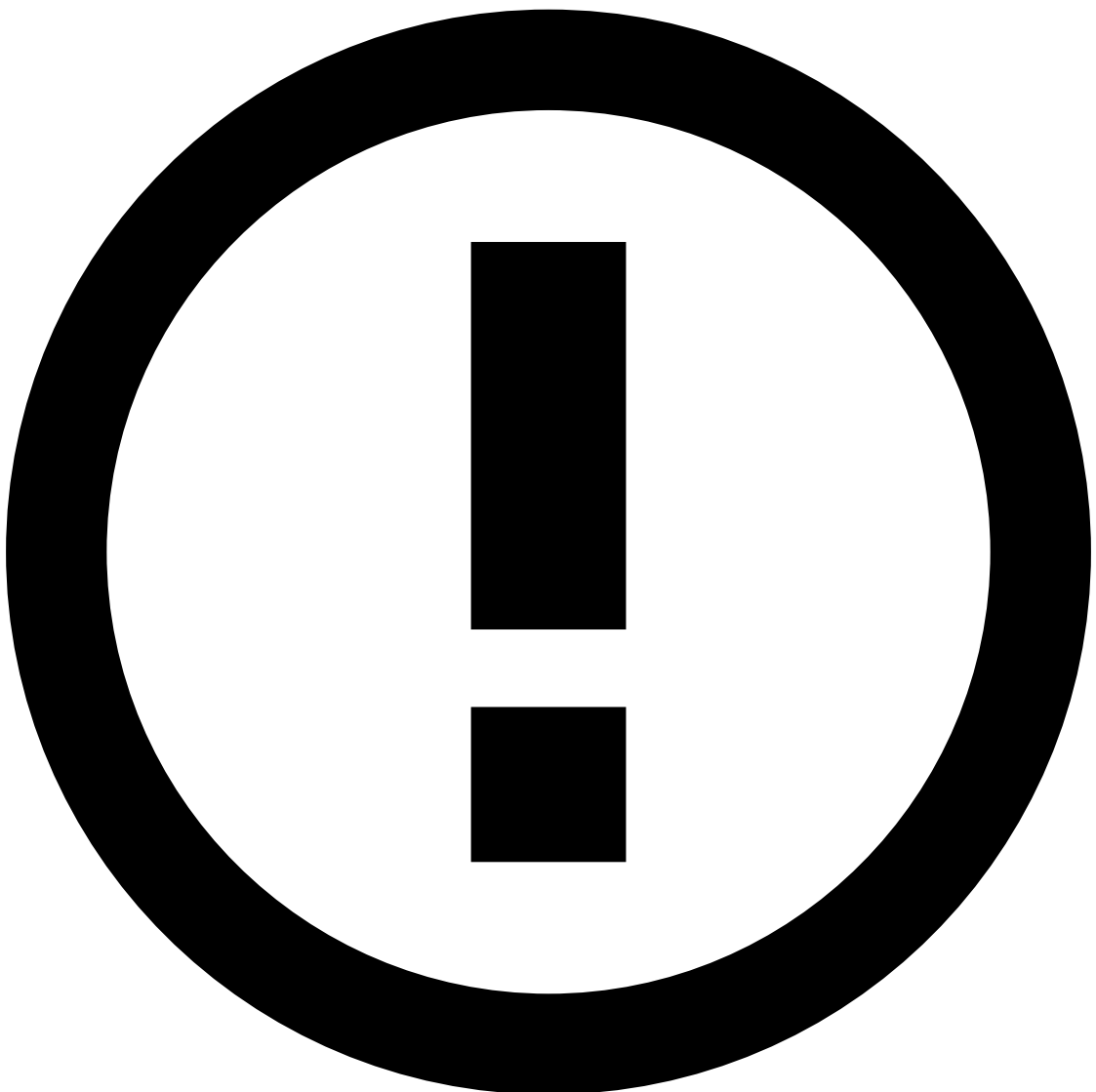
Troubleshooting Tips

Configuring custom properties can be challenging due to the number of different configurations that are required to fully set up [component encryption](#).

Here are some tips if you run into issues:

1. Ensure you're using the right software versions, as the most common source of errors is the usage of incorrect software versions. Ensure the following minimum versions are in use:
 - **Java:** 8u141
 - **Rabbit:** 3.6.14 / Erlang 20.1

2. Check for errors in the Pulse logs. The most common configuration errors which are easily identified in the logs are:
 - The client side is configured without encryption and the server side is configured with encryption (or vice-versa).
 - The public key of the certificate in the client truststore does not match the private key of the server keystore.
 - The truststore does not include the certificate of the CA that signs it.
3. Check if the server is configured with SSL by using `openssl s_client -connect <server:port>` and verify if the certificates are displayed in the command output.
4. Set the encryption lib log level to trace by adding `<logger name="com.feedzai.ssl" level="trace"/>` to `logger.xml`. If the logs are significantly longer than expected, it might be a sign that too many SSL connections are being established.



info

Establishing too many SSL connections might have an impact on performance.

5. Under extreme circumstances, run the following command: `set -Djavax.net.debug=all`. This should be used with caution because it produces a long log. Check the following page to understand the generated logs: <https://docs.oracle.com/javase/8/docs/technotes/guides/security/jsse/ReadDebug.html>.

•



- [Core Concepts](#)
- [Models](#)
- [AutoML](#)
- Semantic file

On this page

Semantic file

Was this page helpful? 

To give meaning to input data fields in an AutoML run, you need to provide a semantic file that tags the data fields to their meaning. The following sections describe how to create the semantic file, and give a list of tags you can use for input fields.

Why tagging?

When [defining an AutoML run](#), you provide a semantic file that maps the input data fields to semantic tags. The semantic tags translate the meaning of the fields, providing the only portion of human knowledge that is required to power AutoML.

If AutoML knows the meaning of the input fields, it can combine them into features using [map and metric operators](#). For example, the *Name* field can be tagged with the *name* tag. This way AutoML knows the meaning of this input field, and it can compute the number of words in the name, which can be useful to fight fraud.

Without tagging the input data fields, AutoML cannot generate features. The quality of the models generated by AutoML relies heavily on the quality of tagging.

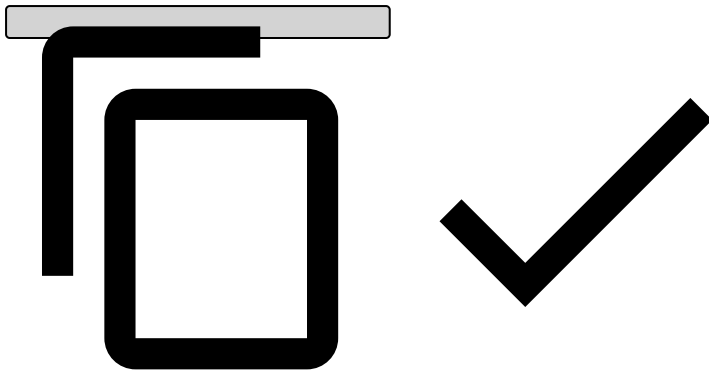
Creating the semantic file📄

The semantic file is a JSON file, in which each data field is an object with the following information:

- **@type**: A string containing the type of the tag, which can be *"simple"* or *"composite"*. Use the **composite tagging** to compose one or more tagged input fields into a single tag. By default, the *address* and *coordinate* tags are composite. See their description in the [list of tags](#).
- **tags**: A vector of strings with the tag(s). See their description in the [list of tags](#).
- **fieldDesc**: A string with the visible name of the field. In *composite* tags it is optional. If you provide a fieldDesc, it will be used as the name of the new field. Otherwise AutoML generates a field name by concatenating the tags you provided.

The following is a simple semantic file, for a data source with four data fields. The last tag creates a composite tag over two fields.

```
[
  {
    "@type": "simple",
    "tags": ["name"],
    "fieldDesc": "Name"
  },
  {
    "@type": "simple",
    "tags": ["amount"],
    "fieldDesc": "TransactionAmount"
  },
  {
    "@type": "simple",
    "tags": ["longitude"],
    "fieldDesc": "TransactionLongitude"
  },
  {
    "@type": "simple",
    "tags": ["latitude"],
    "fieldDesc": "TransactionLatitude"
  },
  {
    "@type": "composite",
    "fields": ["TransactionLatitude", "TransactionLongitude"],
    "tags": ["coordinate"],
    "fieldDesc": "TransactionCoordinates"
  }
]
```



To create the semantic file for your data source, you can download a template from Pulse. To do so, start [creating a new AutoML run](#), select the data source, and use the **Download template file** option in the Semantic File field. Then simply add the [tags](#) to the fields.

The following are the **rules of tagging**:

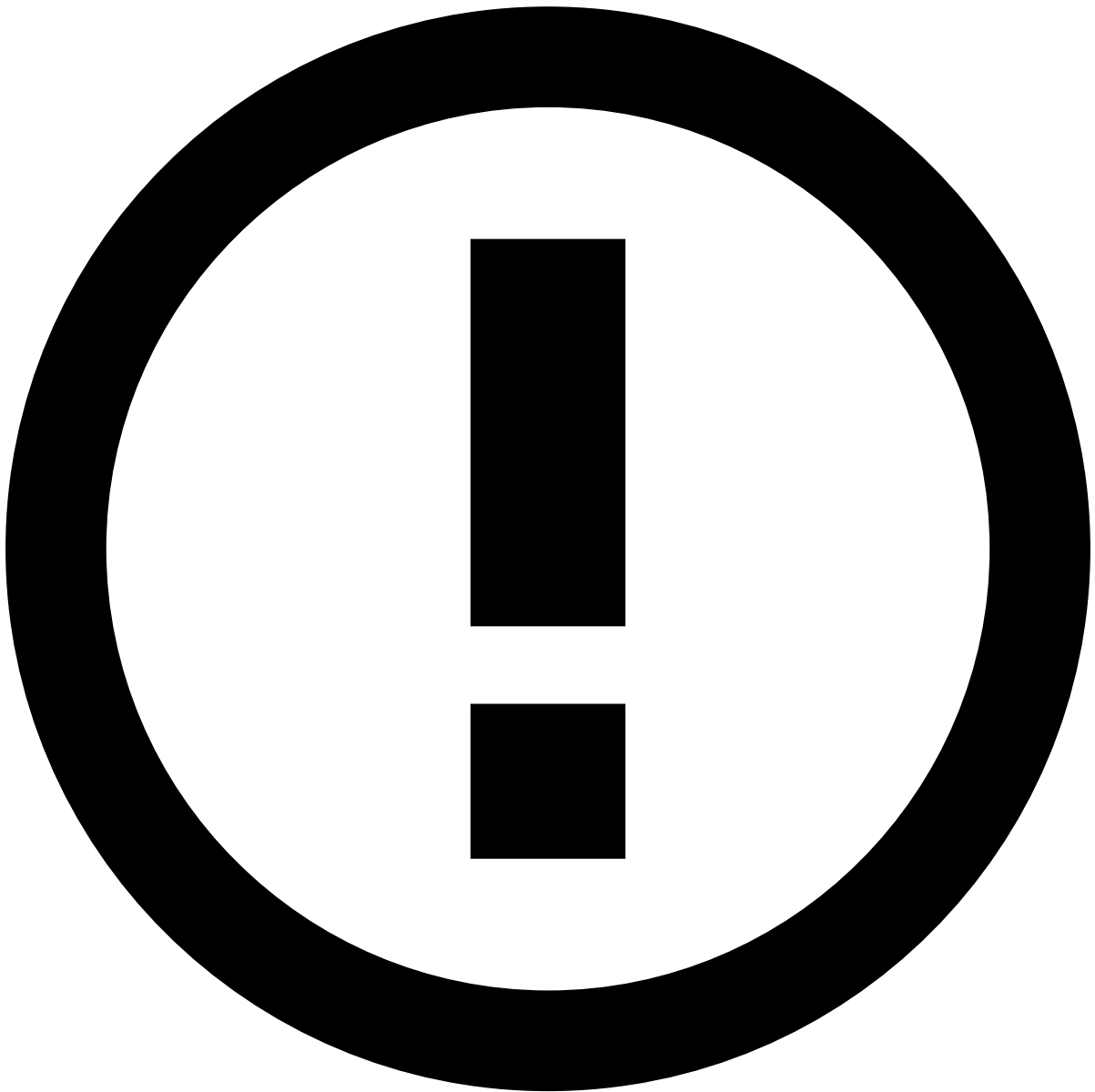
- At least one field must be tagged in the semantic file.
- Each field can appear only once.
- Each field must exist in the schema of the data source. However, [complex data fields](#) shouldn't be tagged, as they won't be used by AutoML.
- A composite tag must refer to fields that are tagged in the same file. For example, if you add the *coordinate* tag, in the same file there must be a field tagged with *longitude*, and one tagged with *latitude*, and the name of these fields must be listed in the *coordinate* tag, under *fields*.
- A contextual tag should be accompanied by the corresponding *grouping-entity* or *amount* tags.

List of input tags

The following table lists the built-in tags of Pulse that can be used by Data Scientists in the semantic file when creating a new AutoML run, to tag the fields of the input data source.

Note that Pulse doesn't set the tags on its own. If the semantic file doesn't provide a tag for a field, Pulse cannot create features based on it.

In addition, the below file can be extended by developers with custom tags used by custom transformations.



Contextual tags

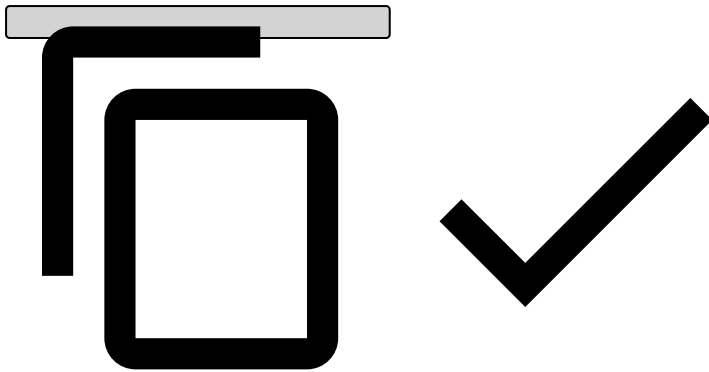
Contextual tags are tags that don't trigger any transformation *per se*. They are intended to be used by their companion tag.

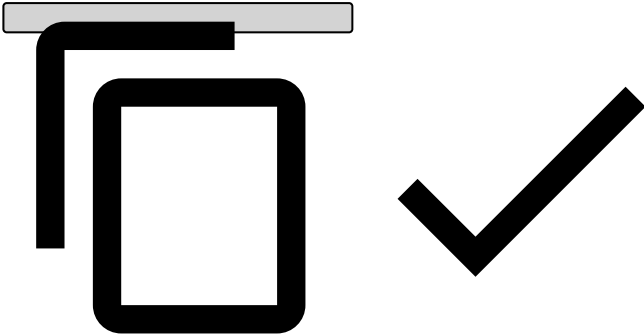
For example, the Data Scientist might want to tag the field "card_id" as a *grouping-entity*. Additionally adding the contextual tag *card* will make AutoML only trigger transformations that make sense for cards.

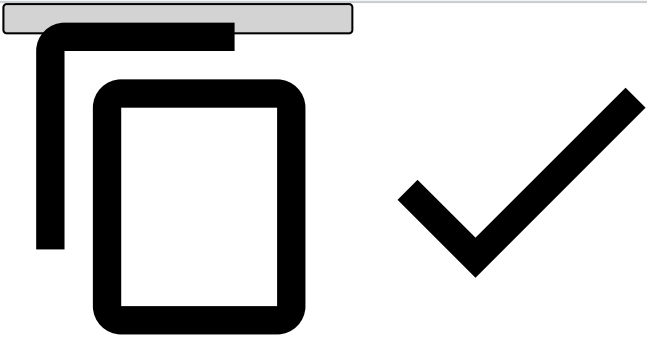
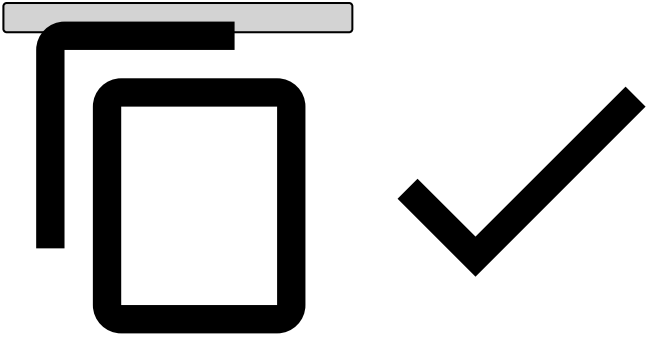
You can find existing contextual tags in the table below.

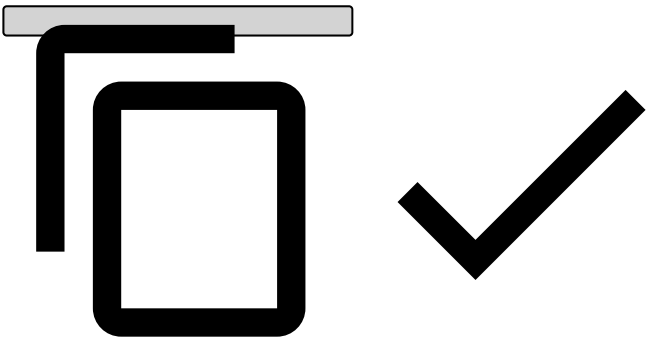
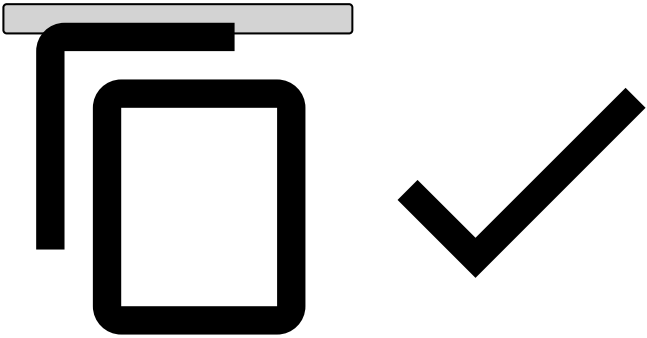
Usage example:

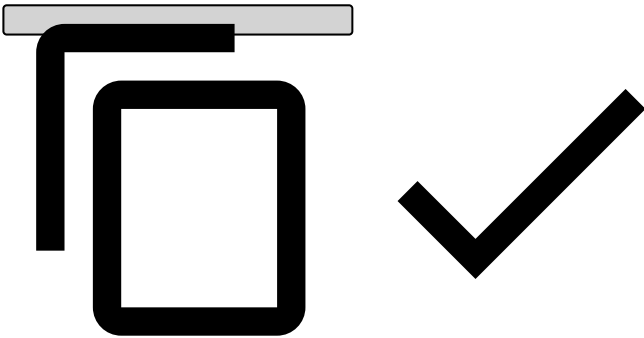
```
{
  "@type": "simple",
  "tags": [
    "grouping-entity"
    "card"
  ],
  "fieldDesc": "card_id"
},
```



Type	Observation
	A composite tag that combines fields with line1, line2, zip, city, region and country tags. Example for tagging: <pre>{ "@type": "composite", "fields": ["CustomerAddressLine1", "CustomerAddressLine2", "CustomerAddressZipCode", "CustomerAddressCity", "CustomerAddressRegion", "CustomerAddressCountry"], "tags": ["address"], "fieldDesc": "CustomerAddress" }</pre>
address	 To tag an address at least two components of the address must be present, so a minimal example would be: <pre>{ "@type": "composite", "fields": ["CustomerAddressLine1", "CustomerAddressZipCode",], "tags": ["address"], "fieldDesc": "CustomerAddress" }</pre>

Type	Observation
	
line1	The first line of an address
line2	The second line of an address
zip	
city	
region	
country	Note: This tag can also be used as a contextual tag for the grouping-entity tag.
coordinate	A composite tag that combines fields with longitude and latitude tags. Example for tagging: <pre> { "@type": "composite", "fields": ["TransactionLongitude", "TransactionLatitude"], "tags": ["coordinate"], "fieldDesc": "TransactionCoordinates" } </pre> 
longitude	A double, representing the longitude of a location
latitude	A double, representing the latitude of a location
location	This tag can be attributed to either countries or cities. Fields that have this tag will be compared. For example, if a data source has two fields that have the city and location tag, Pulse will generate a feature that checks if they match.
datetime	Tag for a field containing date and time in milliseconds, in Unix timestamp format
expiration-datetime	Tag for expiration dates, e. g. card expiration date.
yy-mm-datetime	Tag for fields of datetime with format yyMM.
name	Name of people, like the name on a card. It will be compared with the name in the email, for instance.

Type	Observation
email	
amount	
available-amount	Contextual tag for an amount field. Usage example: <pre>{ "@type": "simple", "tags": ["amount" "available-amount"], "fieldDesc": "available_credit" },</pre> 
entity	Tag for a field by which you want to order the data for aggregations to count distinct values.
grouping-entity	Tag for a field by which aggregations should be performed (i.e. we want to build profiles). Typical examples are cards, client ids or accounts. Typically a subset of the fields tagged with the entity tag. For example, imagine that we set the grouping-entity tag for card_id and client_id, and set the entity tag for card_id, client_id, user_phone and user_email. All the aggregations (averages, counts, sums, etc) will be performed by card, and also by client. Additionally, AutoML has features that count the number of distinct clients, emails and phones by card and distinct cards, emails and phones by client. Example for a composite field as grouping entity: <pre>{ "@type": "composite", "fields": ["customer_id", "mcc"], "tags": ["grouping-entity"], "fieldDesc": "customer_id_mcc" },</pre> 

Type	Observation
	A composite grouping entity can work independently. If we want a composite grouping-entity with A, B, no need for A and B need to be tagged independently, which would create profiles for A, B and A+B.
acquirer, card, country, customer, ip, merchant, account	Contextual tags that make grouping-entities only trigger transformations that make sense according to that semantic. Usage example: <pre>{ "@type": "simple", "tags": ["grouping-entity" "card"], "fieldDesc": "card_id" },</pre> 
timestamp	A field of date and time value based on which the data can be aggregated for the metrics. Expected format: Unix timestamp.

Learn more

- [AutoML](#)
- [Creating features and models automatically](#)

•



- [Core Concepts](#)
- [Models](#)
- [Model Evaluation](#)
- Business Targets

On this page

Was this page helpful? 

Business Targets

Pulse includes an exploration mode that lets you analyze your [model evaluations](#) and [hyperparameter tuning](#) results from a business perspective. This provides an extra layer of depth to data science model evaluations by letting you fine-tune them with extra business-related performance metrics.

In a fraud detection scenario, it is common that your [data science loop](#) is targeted at maximizing fraud detection without violating business goals. For example, if the business goal is to not exceed 1% FPR, the data science loop should just try to optimize your model performance at that fixed FPR. You'll probably reach a point where you have a large number of model evaluation with small differences in configuration and performance.

Business targets in Pulse are very useful to display comparative analyses of all these model evaluations at once, under different criteria, and help you navigate results. For example, determine, for each model evaluation, which one performs the best, i.e. captures the most fraud money, when you set the FPR as approximately 1%.

For each business target and model evaluation, Pulse displays the classification threshold and selected metrics for which the configured criteria reach their best performance in each evaluation.

The following image displays Pulse's exploration mode with different business targets configured.



Analyzing evaluations

Note that you can combine and add to your targets both single model evaluations and evaluations resulting from a hyperparameter tuning.

So, with business targets, you have an improved ability to:

- Check the performance of your model evaluation at different score thresholds.
- Assess how close model evaluations are to your success/acceptance criteria, without breaking SLAs.

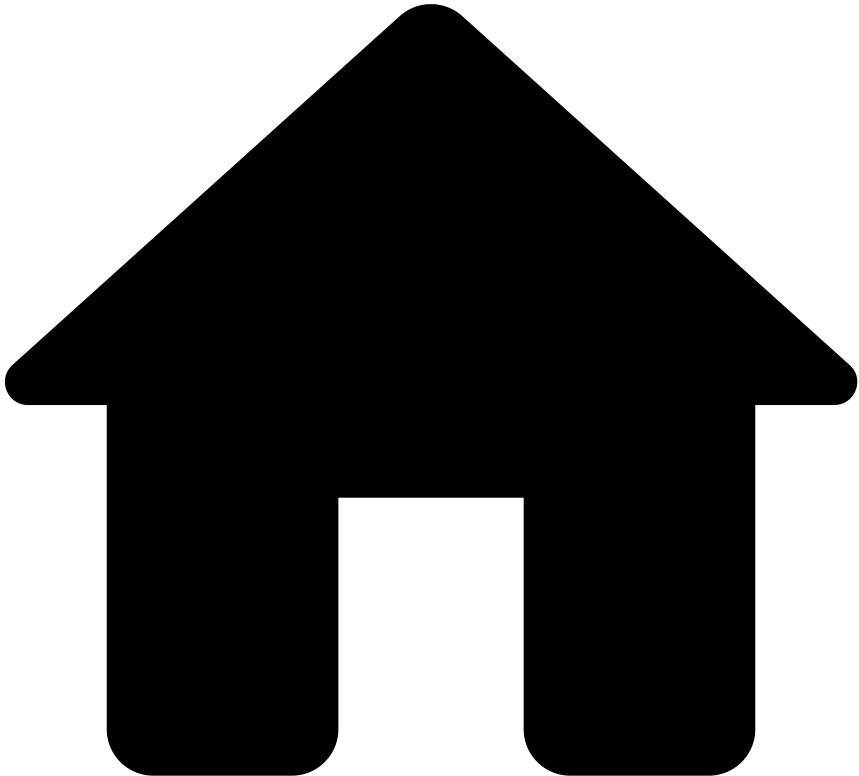
- Detect possible cost-efficiencies when you make very small adjustments. For example, understanding how much your model's Recall improves when you slightly decrease your classification threshold.

Learn more

Check the following pages to learn how to put model evaluations into practice:

- [Evaluating a Model](#)
- [Tuning Model Parameters](#)
- [Comparing Models](#)

•



- [Core Concepts](#)
- [Models](#)
- [Model Evaluation](#)
- Cost Fields

Was this page helpful? 

Cost Fields

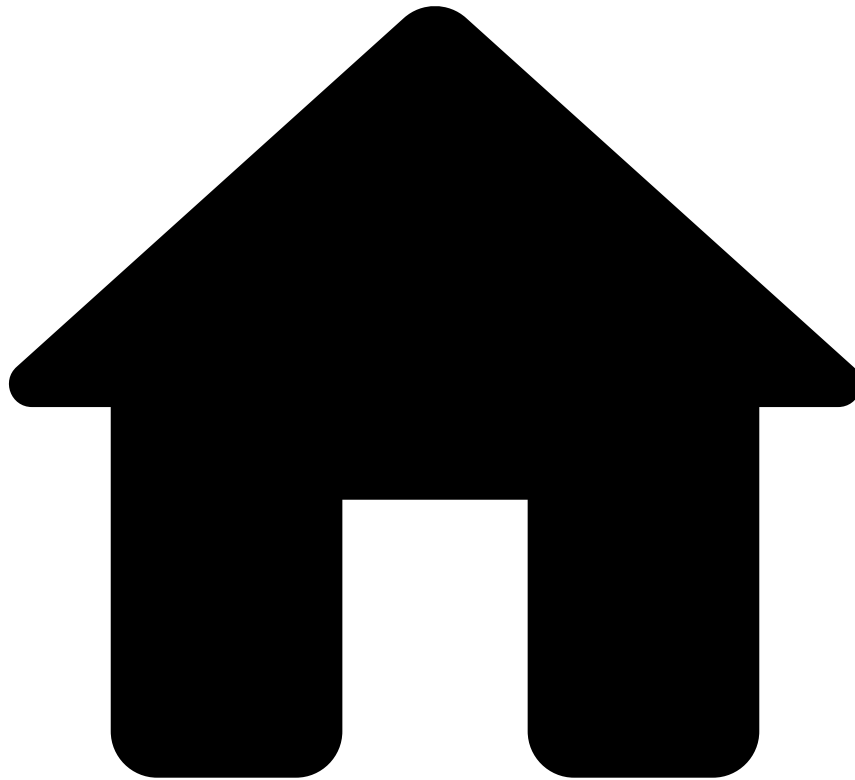
A **cost field** is a field of the instances that is used as a weight for the metrics of the evaluations (of both models and rules), typically the transaction amount in a fraud detection scenario.

Using cost fields in model evaluations or rule evaluations allows you to transform any metric (which by default is count based, e.g. Recall, Precision, etc.) into a cost-based metric (e.g. Money Recall, if the field chosen is a form of cost of the instance).

Non-cost metrics weigh each transaction equally, with cost 1. Using a cost field gives it a weight that is proportional to the value of the field. e.g., money value.

The following image displays cost fields included in a model evaluation. This example depicts a comparison between two evaluations both on count-based and cost metrics, and displays the cost FPR and Recall in the metrics table.

•



- [Core Concepts](#)
- [Models](#)
- [Model Evaluation](#)
- Custom Evaluation Metrics

Was this page helpful? 

Custom Evaluation Metrics

An **evaluation metric** is a function that can be applied to the data obtained during a model validation or rule validation.

In a data science project, an evaluation metric allows you to inject business logic into an objective model or rule evaluation. This allows you to evaluate trained models and rules based on the business value they deliver.

You can add an evaluation metric to your project by defining a function that receives:

- The number of True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN), and outputs a number in an open domain.
- The cost-TP, -TN, -FP and -FN. The cost fields variants are based on the weight a data field in the instance, typically the transaction amount in a fraud detection scenario, rather than using a count-based approach.

The following image depicts where you can create a new evaluation metric. Check the User Guide to see how you can [define evaluation metrics](#), for example, in a model evaluation.

You can apply a custom evaluation metric, written by you and adjusted to your use case, to any existing model or rule evaluation.



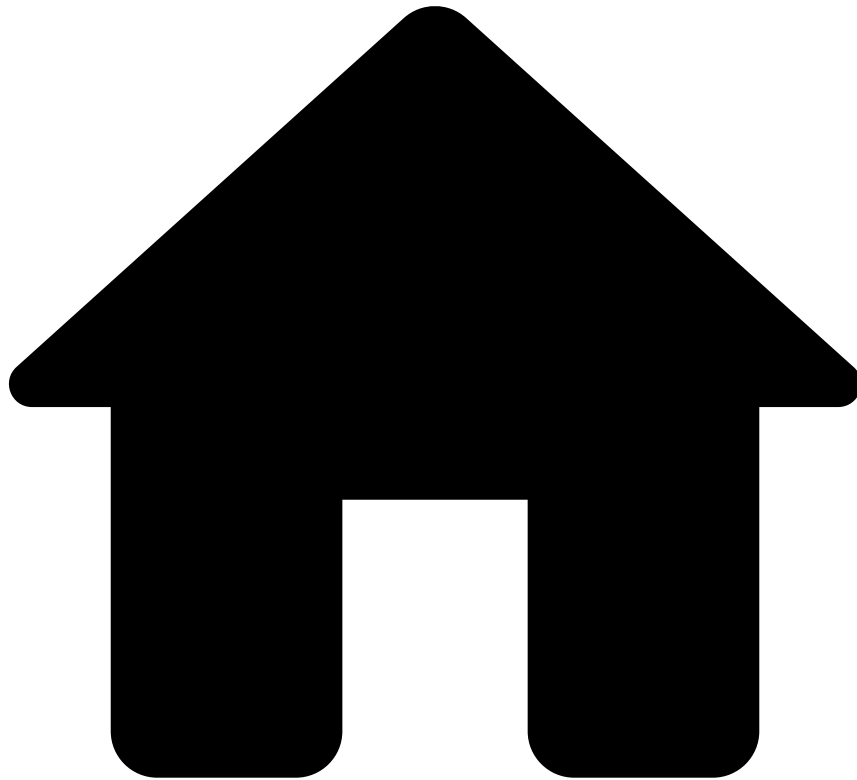
Availability

Since custom evaluation metrics can be built from both count and cost based way, they are available to model evaluations or rule evaluations only if you define a [cost field](#) for the evaluation.

If you do not define a cost field for an evaluation, then Pulse will not be able to compute your custom evaluation metrics and you will not have access to them.

The following image shows how you can add a custom evaluation metric to a model evaluation.

•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- FAQ on Gradient Boosting Frameworks

On this page

FAQ on Gradient Boosting Frameworks

Was this page helpful? 

This section contains answers to common questions when configuring a gradient boosting framework in Pulse.

What is string indexing?

String indexing goes by many other names as well, such as label encoding and integer encoding. For an overview of this technique, you can read [this article](#).

What is considered too many categories?🔗

According to Microsoft LightGBM's recommendations, you should not have too many categories in a categorical feature. This could mean hundreds or thousands, as it depends on the model. For performance, categorical values should not be limited to <1% of the training instances.

If there are too many categories, the recommendation is to index them and declare that feature as numerical. However, LightGBM / FairGBM will then assume such values **are** sorted. This means the mapping you choose matters. We recommend using frequency-based or risk-based indexing.

This choice implies that you must do the string indexing yourself, by adding a step to the features plan. In this step, take a mapping (through a list operator or user-defined function that loads the mapping from disk) and apply it to the input feature to generate the numerical feature to be used by the LightGBM / FairGBM model.

Remember that this indexing must work reliably and in the same manner in any environment, be it data science phase, or QA, or Production, to avoid compromising the model features and behavior.

There are cases with performance improvements where up to 10,000 categories were indexed through frequency for a categorical feature. Declaring that feature as numerical decreased performance around 1% recall.

As with all data science tasks, you should experiment and figure out how many categories give you the best results for your project.

Does Pulse's categorical features indexing algorithm impact my model?🔗

This particular way of categorical ordering by Pulse (lexicographic order) does not affect your model results because LightGBM / FairGBM assumes the categorical features have no order, and sorts the different categories prior to splitting by the objective function.

The essential part is that the indexing is the same during training and scoring (which Pulse does).

How are missing/null values handled?🔗

LightGBM and FairGBM handle nulls properly. In Pulse this is done by mapping any null / missing value to a NaN value in the backend, which is then interpreted as missing (you cannot disable this level of missing value handling in Pulse by design).

It is possible to consider 0 values as missing by enabling "Use zeroes as missing".

LightGBM and FairGBM handle splits and missing values in scoring using the following rules:

- For categorical features:
 - Categorical splits are performed in "one" vs "others" approach, with the split category of interest going to the left of the split, and the remaining categories go to the "others" side, on the right side of the split.
 - A missing value in a categorical feature split allots the instance to the right side of the split. This is the "other" categories side.
- For numerical features:
 - A numerical split places numbers smaller than the split point to the left, and larger ones to the right of it.
 - *If there were no missing values during model training*, the missing value in scoring is replaced by 0 and the splits are followed accordingly.
 - *If there were missing values during model training for that split*: missing values are allocated to the side of the split that reduces training loss the most (during prediction that side is chosen as well).

See [this thread](#) in the LightGBM forum for more details.



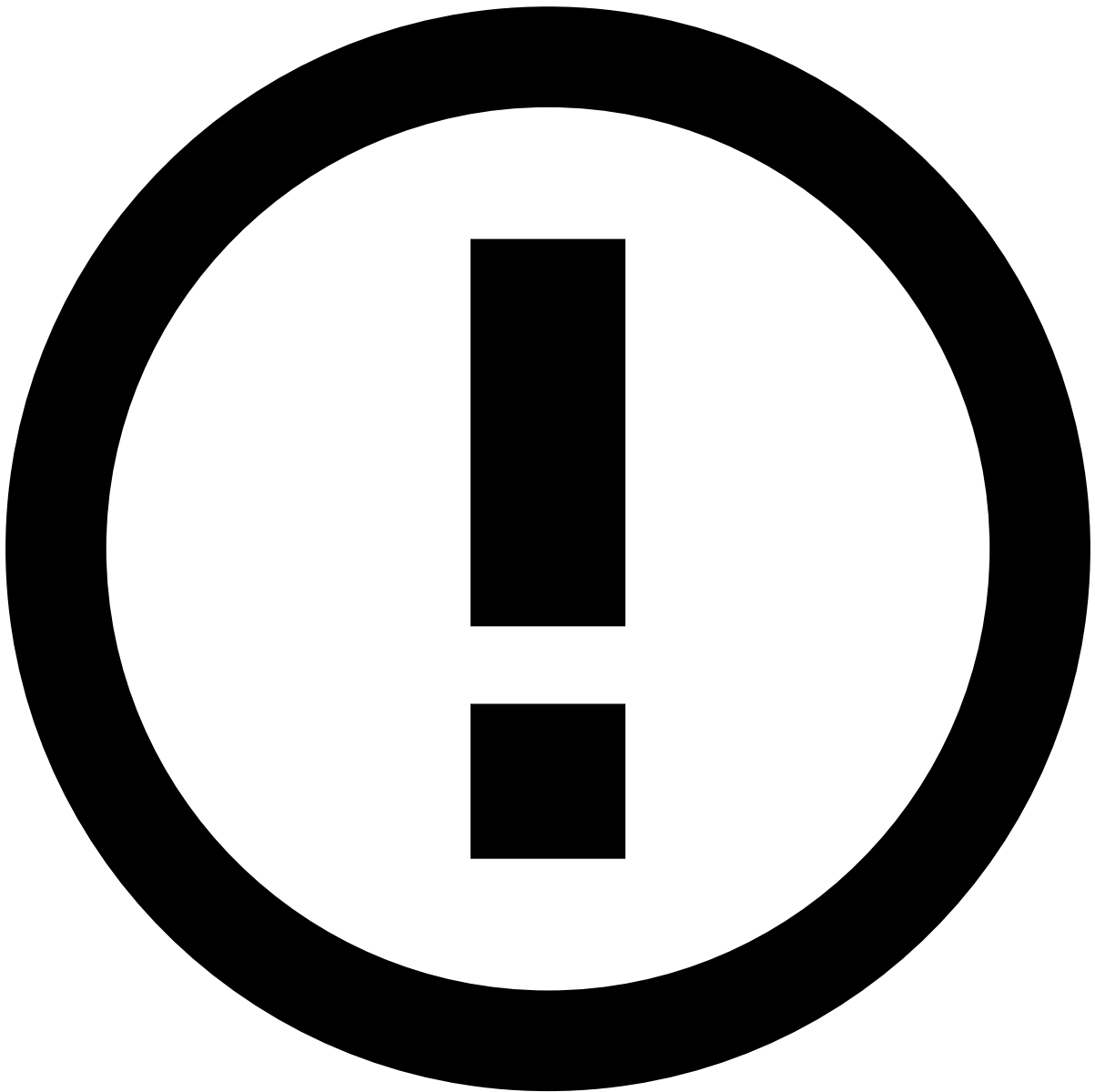
info

FairGBM does not handle missing sensitive attribute values.

What are the training speeds for the different boosting types?[🔗](#)

There aren't any benchmarks available online. However, by [modifying the benchmarks](#) and testing for the HIGGS dataset, it is possible to get an idea of the speed difference on a large dataset.

Feedzai has conducted tests for LightGBM. See the test results below.



info

Note that, for FairGBM, extra computations make it slower than LightGBM during train. For this reason, such results do not apply to FairGBM.

Training time for the Higgs dataset

The Higgs dataset is a dataset with 28 features and 11M rows, of which 10.5M rows are used for the model training. The tests were conducted on a Dell laptop with 4 cores (x2 because of hyper-threading) and 32G RAM.

`boosting_type=GBDT`

Measurement	Measurement value
Real	10m 12s
User	51m 57s
System	1m 8s

boosting_type=GOSS

Measurement	Measurement value
Real	8m 17s
User	42m 59s
System	1m 1s

boosting_type=DART

Measurement	Measurement value
Real	60m 9s
User	363m 31s
System	2m 8s

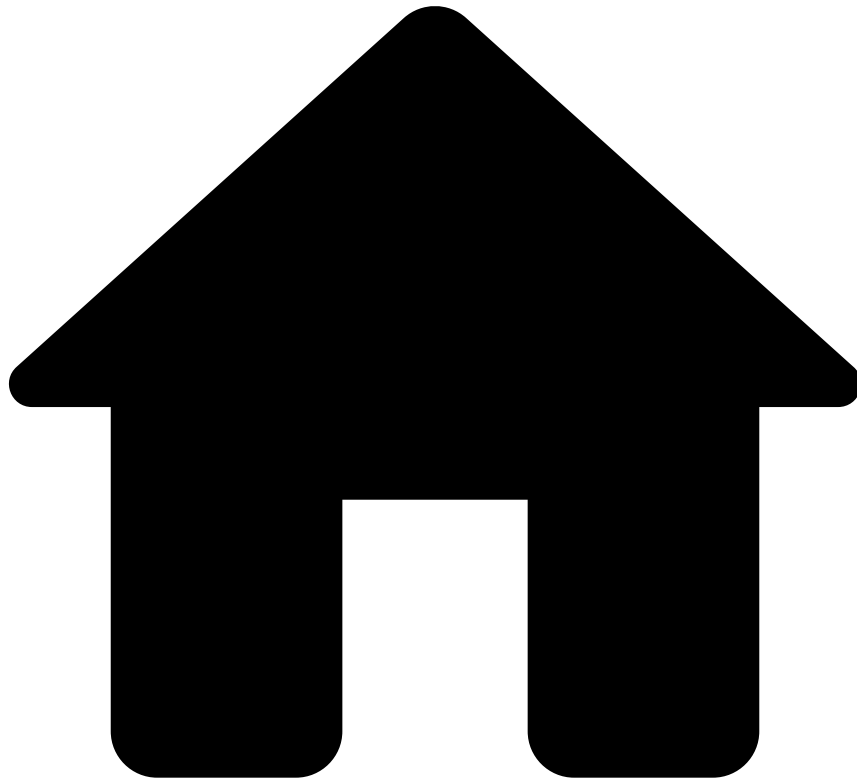
Conclusions

As it is a single dataset and machine under comparison, these results serve only to give an idea of the different boosting model training speeds.

This comparison in particular shows roughly a **20% reduction** in training time whilst using **GOSS** boosting type instead of **GBDT**.

The biggest difference comes from using `boosting_type=DART` , which is roughly **6x slower** than **GBDT** in this case.

•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- Feedzai FairGBM

On this page

Feedzai FairGBM

Was this page helpful? 

Fair Gradient Boosting Machine or FairGBM (FGBM) is a constrained optimization (CO) framework developed by Feedzai on top of Microsoft LightGBM.

FairGBM enables training LightGBM models with fairness constraints (e.g., equality among different ethnicities, genders, age-groups, or any other sensitive attribute). This requires defining a sensitive column that details the protected subgroup each training instance belongs to (note that this column must be in categorical format). FairGBM reduces FPR or FNR

disparities between the different subgroups of the population, according to your choice for the parameter `(Fairness)`
Constraint type :

- `FPR` reduces FPR disparities (useful for punitive contexts, such as fraud detection).
- `FNR` reduces FNR disparities (useful for assistive contexts, such as granting loans).
- `FPR, FNR` reduces both FPR and FNR disparities.

In this topic you can learn about the parameters and the usage in Pulse of the Feedzai FairGBM in Pulse.

Feedzai FairGBM

The biggest advantage of Feedzai FairGBM is that it improves group fairness (e.g., by age, ethnicity, gender), on top of LightGBM's existing advantages (low memory usage and good performance).

One of its limitations is that training is slightly slower than LightGBM in order to provide fairness.

You can choose from several boosting types:

- **Gradient Boosting Decision Trees (GBDT)**
- **Gradient Based One Side Sampling (GOSS)**: faster than the one above but may yield slightly lower predictive performance.
- **Dropouts meet Multiple Additive Regression Trees (DART)**: can be much slower but might yield higher predictive performance as dropout can yield less over-fitting.

FairGBM in Pulse

Pulse can train FairGBM models and even import external ones. Only numerical and categorical features are supported. String fields are not used in the FairGBM models.

Numerical features are passed along to the model without transformation, whilst categorical features go through an automatic string-indexing algorithm.

To train a model, FairGBM requires a sensitive column to be selected during train. That column must be categorical. This allows FairGBM to train a fairer model regarding the different population groups available in that column.

Categorical features handling in Pulse for FairGBM models

Since FairGBM can only take in numerical values, categorical features are fully supported by running a simple **string-indexing algorithm at both training and scoring time**, by transforming the category string value into the *n*th index in the lexicographically-sorted list of categories of a categorical variable.

See the example for the string indexing applied to a categorical feature value:

Property	Value
Input value	"adam"
Categorical feature categories	["John" , "42" , "Adam" , "zoe" , "adam" , "Jane"]
Categories in lexicographic order	["42" , "Adam" , "Jane" , "John" , "adam" , "zoe"]
Output value	4

Notice that indexes start from 0 and for ASCII character codes, numbers come before letters, and uppercase letters come before lowercase ones.

This arbitrary choice does not affect analytical performance as FairGBM does not consider any order for categorical features.

Training a model in Pulse

To train a model in Pulse, start a new model training just like any other model type. If you need more details, see the [Training a Model](#) page.

All categorical features in the training (and scoring) schema are automatically mapped using the algorithm explained in the section above before being sent to FairGBM. The same mapping is automatically used during scoring.

Numerical features are not transformed.

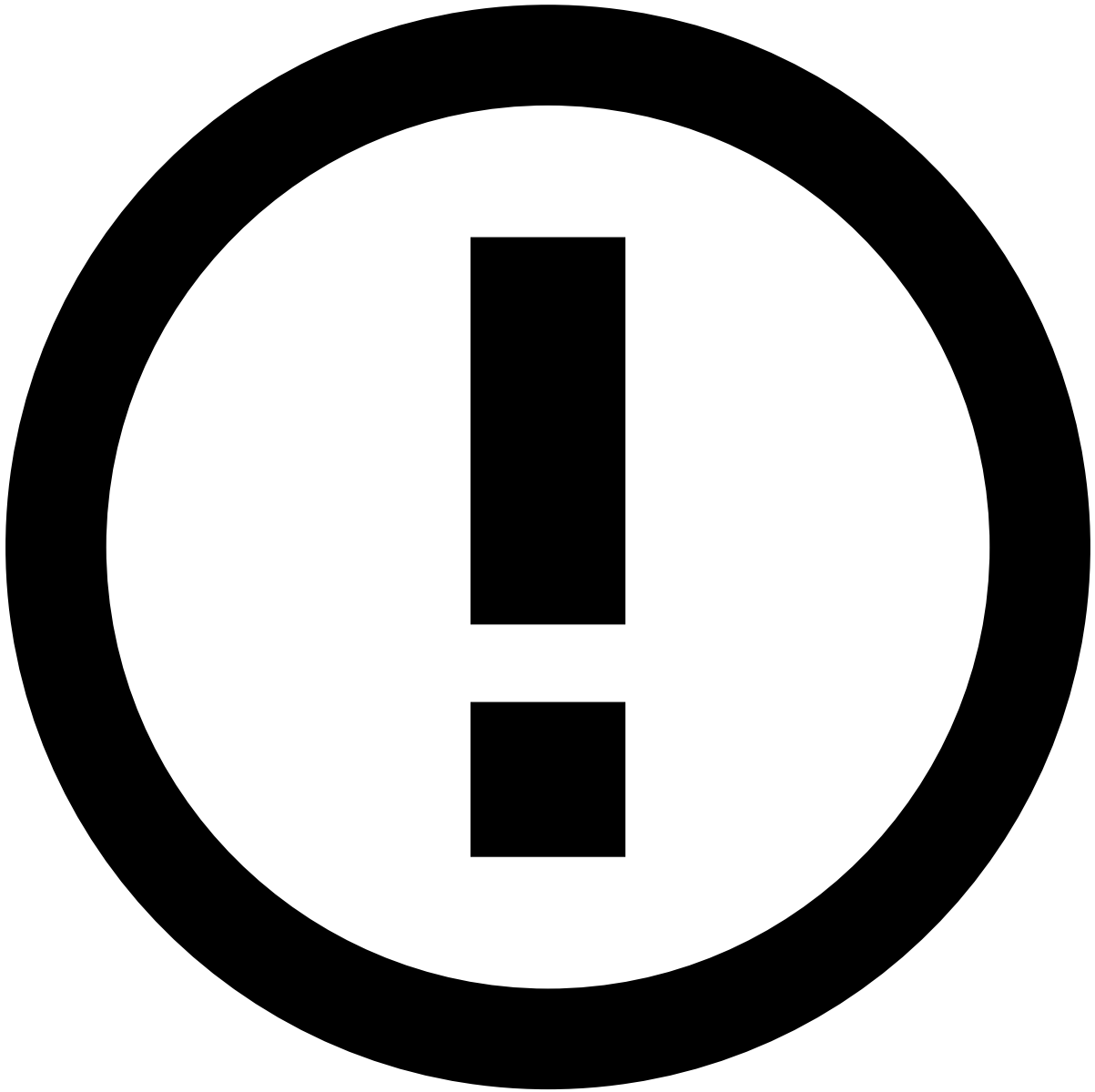
Importing a model to Pulse

You can import a FairGBM model trained outside Pulse as well.

Model import path

To import a model there are two options available:

1. Import the model from a model file path:
 1. The path needs to point to a valid FairGBM model file that FairGBM exports in .txt format.
 2. Since you are not providing the schema, you must define the schema inside the Pulse UI.
2. Import the FairGBM model from a folder path:
 1. The folder pointed to must contain at least the following files:
 1. `FairGBM_model.txt` (mandatory) - the model file as in 1.a.
 2. `model.json` (optional) - a JSON designating the model schema
 2. Since `model.json` is optional:
 1. If not present in the folder, you have to define the schema inside the Pulse UI, as in 1.b.
 2. If present, it will be used for the model schema, unless you override it as in 1.b.



info

Regarding section 2.a.ii, see this [example JSON file](#) for binary classification with both numerical and categorical features.

Importing models with categoricals

If an imported model had categoricals in training, you had to index them, and thus that same indexing must now be performed during scoring, in whichever environment is used.

If indexing during training and scoring does not match, the model scores will be affected.

You have several options and must ensure consistency of those transformations:

1. If you declare, in the Pulse model schema, a variable to be categorical, lexicographic order will automatically be used by Pulse for string indexing during scoring:
 - The only way to have correct scores is to train externally in the first place with the same indexing algorithm, otherwise the model's categorical features in training and scoring differ from each other.

2. If you declare, in the Pulse schema, a variable to be numerical (and at the input it is a string that you had to string-index), you're going to have to do the same exact string-indexing transformation for scoring yourself:
 - Make sure you use the same indexing during training and scoring, but that in scoring, such transformations are reliable across environments, with tolerable latencies and memory usage for your use case.

Parameters

Check the configuration parameters for FairGBM below:

Parameter	Description	Default value
(Fairness) Sensitive group column	Fairness constraints are enforced over this column.	(must be provided)
(Fairness) Constraint type	Enforces group-wise parity on the given target metric for the selected group column.	FPR
(Fairness) Global constraint type	Type of global constraint to enforce during training of fairness constraints.	FPR, FNR
(Fairness) Global target FPR	This is an inequality constraint: inactive when FPR is lower than the target.	0.05
(Fairness) Global target FNR	This is an inequality constraint: inactive when FNR is lower than the target.	0.5
(Fairness) Objective function	For FairGBM you must use a constrained optimization function.	constrained_cross_entropy
(Fairness) FPR tolerance for fairness	The tolerance when fulfilling fairness FPR constraints.	0.0
(Fairness) FNR tolerance for fairness	The tolerance when fulfilling fairness FNR constraints.	0.0
(Fairness) Multipliers' learning rate	The Lagrangian multipliers control how strict the constraint enforcement is. Note that you should use higher values for larger datasets.	10000
(Fairness) Initial multipliers	The Lagrangian multipliers control how strict the constraint enforcement is.	0
(Fairness) Stepwise proxy for fairness constraints	The type of proxy function to use for the fairness constraint.	cross_entropy
(Fairness) Stepwise proxy for global constraints	The proxy function to use for the objective function.	cross_entropy
Boosting type	The type of boosting model.	gbdt

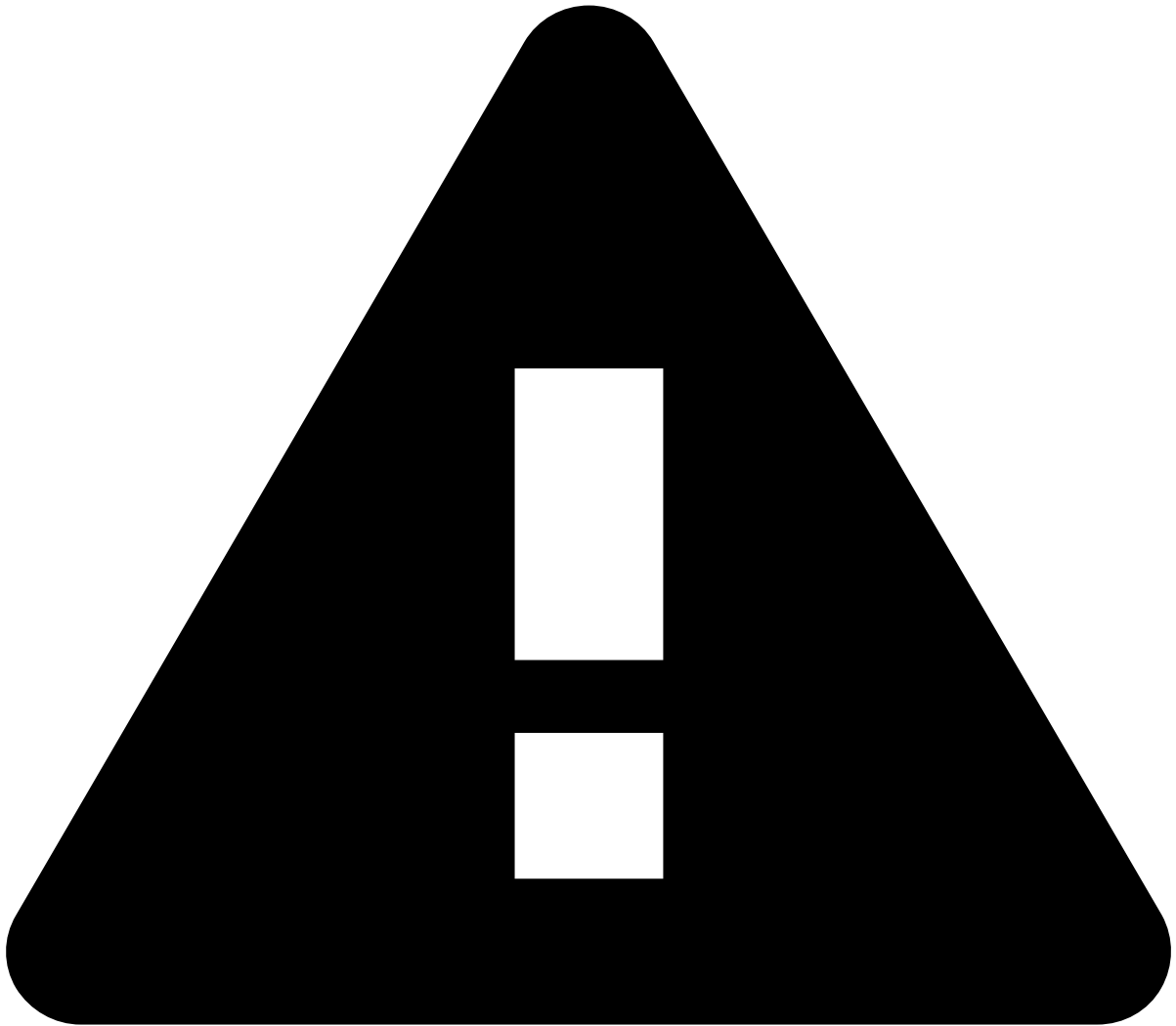
In Pulse, all parameters follow the above specifications, including default values. Handling missing values is always enabled.

Note that there are some parameters that only apply to GOSS and DART boosting modes. Such parameters always have respectively GOSS or DART in their names.



MULTIPLIER TUNING

The Lagrangian multipliers are essential to ensure fairness, and may not be trivial to tune. To tune the multipliers' learning rate, start with the default value (i.e. `10000`), and bear in mind that it roughly increases with the biggest group size. A crude rule of thumb is that the multipliers' learning rate should be between `0.01*n_rows` to `0.1*n_rows`, where `n_rows` is the number of training instances.



CAVEAT

Usually, you choose a decision threshold after training according to one or more of the following metrics: FPR, alert rate, recall, precision, sensitivity, money recall. However, when it comes to FairGBM, this target metric should be provided as a training hyperparameter via `(Fairness) Global target FPR` and/or `(Fairness) Global target FNR`, depending on the optimization goal(s) set in `(Fairness) Global constraint type`. For example, if you want to deploy a fair model at 5% FPR, you must set `"(Fairness) Global target FPR"=0.05` before training. This ensures that your target metric is fulfilled near a threshold of 0.5 (or 500). You can later fine-tune the final threshold after training, but the results should be around 0.5 (or 500).

Logging configuration🔗📄

If you want to know in more detail the progress of the process, you can check the [LightGBM Logging Configuration](#) page to configure logging to suit your needs.

Learn more🔗📄

- [Feedzai Fair GBM code on Github](#)
- [Feedzai FairGBM openml provider code on Github](#)
- [FAQ on Gradient Boosting Frameworks](#)

•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- Feedzai Random Forest

On this page

Feedzai Random Forest

Was this page helpful? 

Feedzai Random Forest (FZRF) is a machine learning algorithm developed by Feedzai. It is an enhanced variation of the [original Random Forest algorithm](#).

Random forests use labeled data to learn its patterns. A random forest consists of a typically large number of decision trees, where each decision tree is a model. Each decision tree model is created by feeding each tree a random subset of the data (i.e. bagging).

In this topic you can learn about the parameters and the usage of Feedzai Random Forest.

Parameters

You can learn more about each parameter of FZRF in the following table:

Parameters	Description	Default values	Possible values	Notes
In-bag factor	This parameter tells you which percentage of instances are used to train a particular tree, while the rest is used to calculate the out-of-bag error.	0.65	Double between 0 and 1.	See FAQ's question number 2 to learn more about how the bagging is performed.
Maximum depth of trees	The number of nodes in the longest path from the root of the tree to a leaf. The higher the number, the larger the model will be. Large models mean more splits, meaning they capture more information about the data. However, as mentioned, they result in large sized models that can reach GBs. Also, a high number of maximum depth of trees is likely to lead to overfitting.	20	Positive integer.	This can be a number from 1 to the number of instances in the training set. In practice, splits don't separate just one transaction from the others, so depending on the rate of the target class and features being used to train, you won't see a considerable difference in model performance (i.e., if you change the maximum depth from 100 to 200). However, the size of the model may change considerably so a small number is advisable.
Number of trees	The number of decision trees in the forest.	100	Positive integer.	There are diminishing returns, and training time increases linearly with the number of trees. Typically this parameter is set between 500 or 1000, but only if the dataset isn't particularly big.
Feature selection function	The metric that defines how to pick the feature for the next split.	Information gain	Information gain: The next split will be the one that maximizes the information gain, i.e., that splits the two classes better. This means these feature selection functions give you a concrete number on how well you separated your two classes (e.g. fraud vs non-fraud) after a split. Your goal is to be able to separate the two classes as best as possible.	
Minimum instances for split	For a split, the selected node has to have at least this number of instances in its subtree.	2	Positive integer.	The typical value is 10 or 20. If you select a number equal or bigger than the number of transactions, then you never do any split.
Number of candidate features	The number of features to be used in each node as candidates for the next split.	0	Positive integer, between 0 and the total number of features in the dataset.	The default value (i.e., 0, zero), means that the square-root of the total number of features will be used.
Prior function	This parameter is used only when estimating the distribution of a node that has no transactions.	Empty	Empty: Assigns an equal probability for each class. If it's a	The estimation happens when scoring an event that has null values in a feature that is used to split nodes in the trees.

Parameters	Description	Default values	Possible values	Notes
			fraud vs non-fraud scenario, 50% for each. Random: assigns a random probability for each class. Parent: uses the probability distribution of the parent node.	
Split method	The way the algorithm splits categorical nodes into new branches.	One vs Others	One vs others: the split is done into two new branches: one category branch vs an "others" branch. This one results in longer training time. All vs all: the split is done into as many branches as the number of categories. Typically, you will not obtain better results from this value, as it's prone to overfit if there are a lot of categories and it also requires larger disk space, which potentially means trouble.	

FAQ

What is the condition for the nodes to be split?

What is the condition for the nodes to be split, is it '<' goes left and '>=' goes right or '<=' goes left?

It shouldn't make any difference if you are trying every split.

Is bagging with or without repetition?

Without repetition for each tree. With repetition "across trees", as shown in the figure below.

This means that a tree will never look at the same instance twice, but multiple trees in the same "forest" will most likely see overlapping instances.

How do null values impact the computation of information gain, especially with categoricals?

If you are trying to split on a categorical and an instance has a null in that field, you must count the instance for the total. Yet you shouldn't count it for the specific subset when calculating the information gain.

A few more notes regarding FZRF and null values:

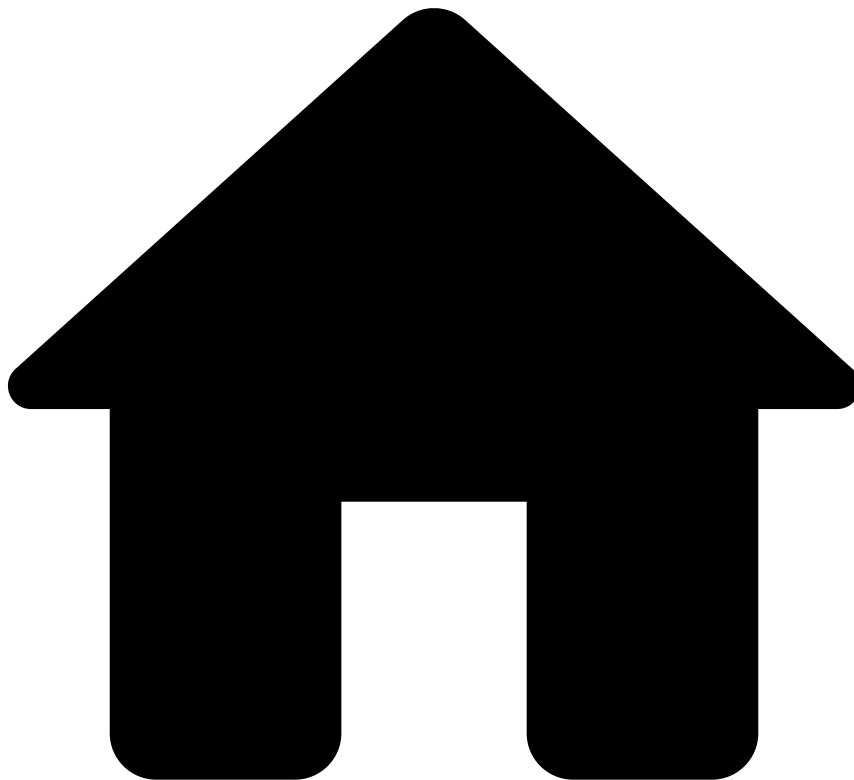
- FZRF will ignore any instance where the class label is missing. This means that the whole instance is dropped during training.
- FZRF will use instances where the features have missing values up to the first split using that feature (i.e. for each tree/branch).

The consequence of the last point is that when a feature with very high number of missing values is being tested for splitting, it may **have very high information gain**. This will lead the model to select the feature, hence dropping most of the instances in the training data.

Is the data replicated into memory for each tree due to bagging?

Yes, because trees are trained in parallel and each in-bag dataset is a partition of the initial data.

•



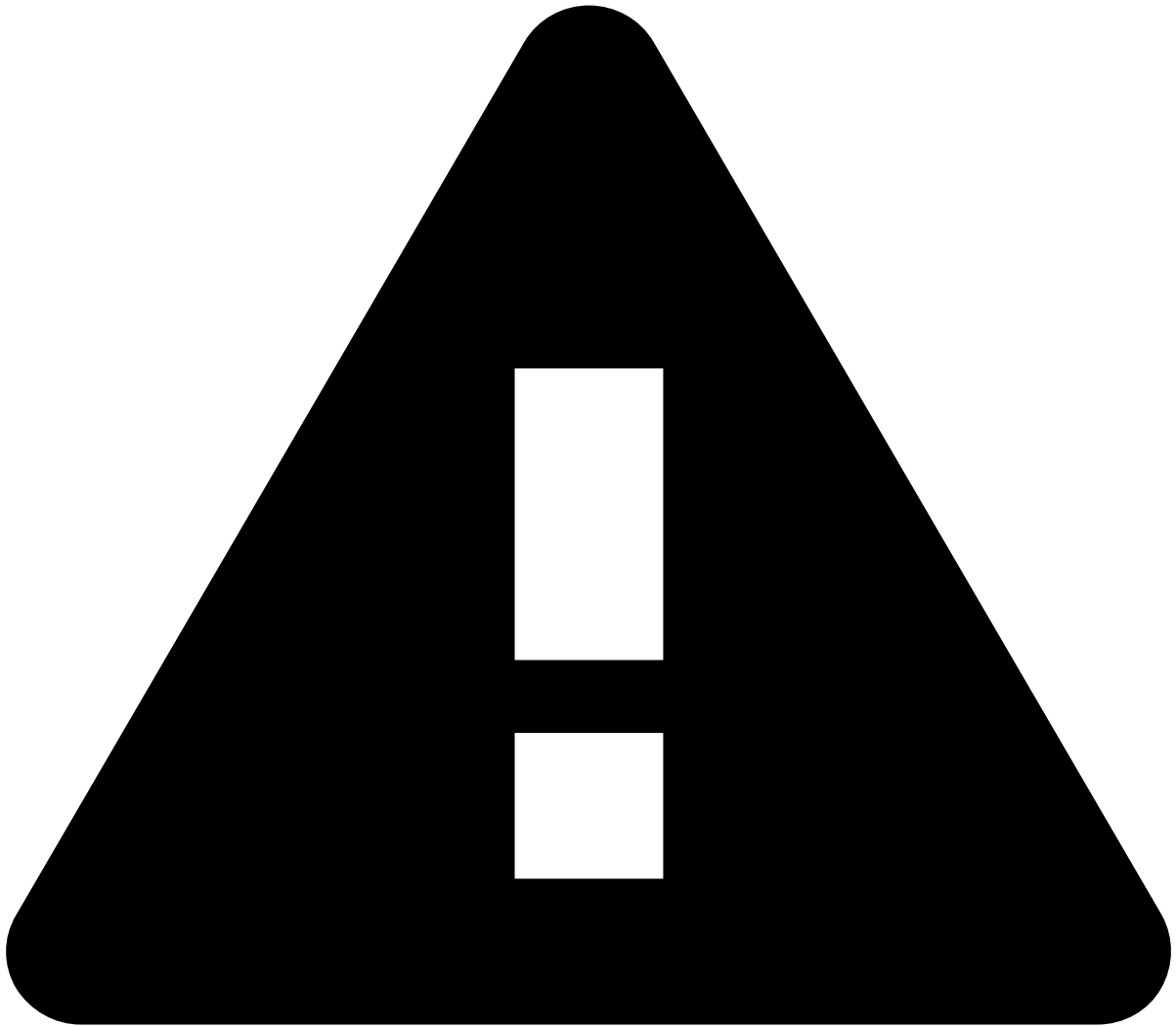
- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- LightGBM Locale Issues

On this page

LightGBM Locale Issues

Was this page helpful? 

When using versions older than LightGBM v3.2.0 or Pulse 22.0, there may be wrong scores during prediction due to locale mismatches between train and prediction environments.



caution

This caveat applies to versions **older** than Pulse 22.0, that includes a patched version of LightGBM, so that there are no locale issues

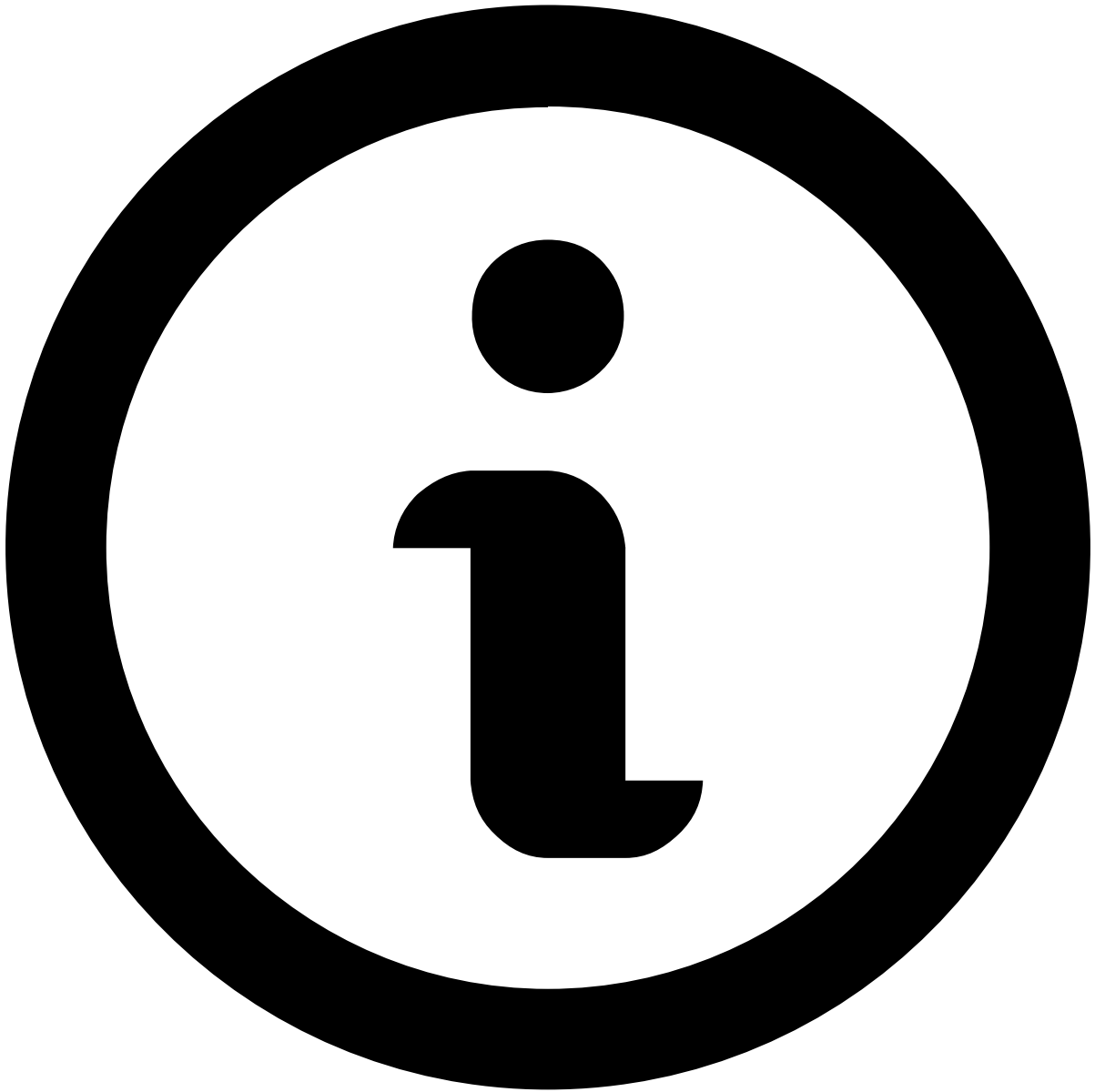
The fix is also present in all of the following Pulse hotfix versions:

- `>= 19.1.41`
- `>= 20.1.18`
- `>= 20.3.20`
- `>= 21.1.3`

When importing models that were trained with the help of Java in some way, follow this rule:

- When training LightGBM models outside Pulse using Java tools, enforce locale `LC_ALL=C` during model training to avoid broken scores.

This is true whether using a Java LightGBM provider such as MMLSpark, or any LightGBM provider such as Python's, together with Java tools like JEP.



note

This rule can only be ignored if you know your LightGBM provider uses the core LightGBM library release v3.2.0 or later.

Explanation[ⓘ]

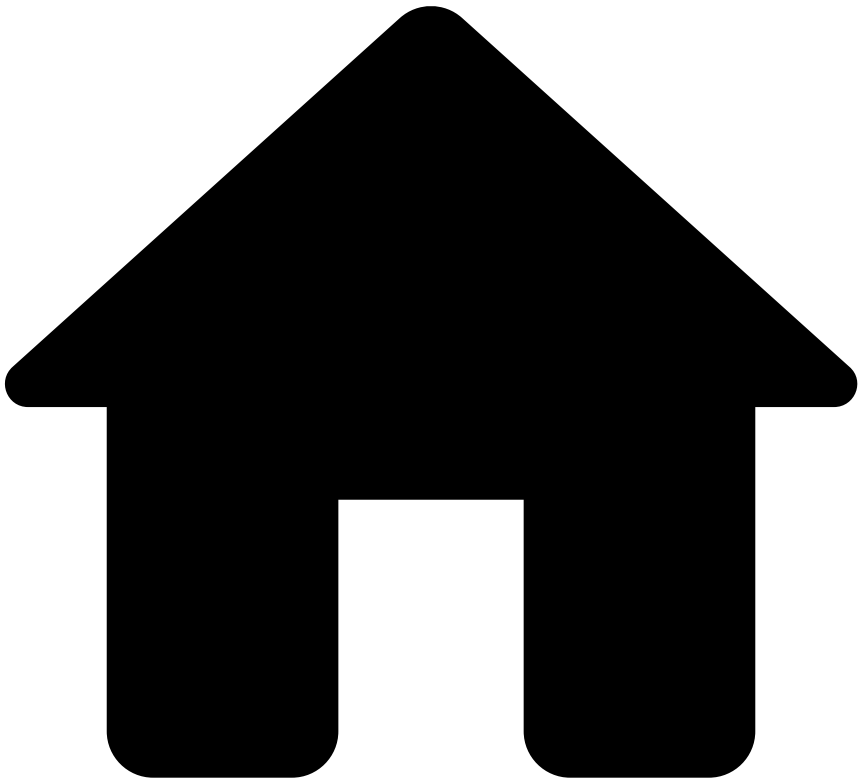
If Java is not using the default **C** locale, it will break LightGBM's, producing a LightGBM model file that will be read back incorrectly.

Although LightGBM in Pulse uses Java as well, it no longer suffers from this issue since we patched LightGBM and are using that patch, that will come out in the LightGBM v3.2.0 release.

How to control the locale?[ⓘ]

Depending on where the job is running, there are several ways to override a locale. If running a local process prepend `LC_ALL=C` to your command or do `export LC_ALL=C` before using your command. If you are using Spark, pass the Spark property `spark.executorEnv.LC_ALL=C`.

•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- LightGBM Logging Configuration

On this page

LightGBM Logging Configuration

Was this page helpful? 

The LightGBM provider of Pulse can be configured to log information. You can configure the logging levels, from the most basic INFO, up to the most extreme level TRACE on the several modules of the LightGBM provider.

This topic describes the logging levels, and how to set up logging.

Logging levels🔖

There are several modules available, each with different responsibilities. To fill out the logging configurations you must use the fully qualified name of the module (as in the examples on the bottom of the page), by adding the prefix "com.feedzai.openml.provider.lightgbm." to the module name. You can control the logging level per module.

Module name	Description of the module	Description of the logging levels
LightGBMBinaryClassificationModel	Implements the OpenML class responsible for loading a model and scoring instances. It is also called when saving the model to disk.	ERROR: Report that saving model to disk failed.
LightGBMBinaryClassificationModelTrainer	Responsible for training a LightGBM model. At the end of the train it saves an intermediate representation to disk that is then passed to LightGBMBinaryClassificationModel for the final model save to disk.	INFO: Information messages reporting the stage at which the training is ERROR: For errors during train DEBUG: Report #train-instances, training parameters and features, more details on the sub-stages of training, progress in number of train iterations
LightGBMModelCreator	Starts all load/fit operations and validations prior to starting the aforementioned model load/fit process.	INFO: Message with file path from which the model is being loaded ERROR: Errors when training the model or the validations for a model fit/load fail and where it is it trying to read from.
LightGBMSWIG	Low-level implementation for model scoring.	INFO: Saving model to disk calls. ERROR: Errors saving model to disk. DEBUG: Saving model to disk calls with filepaths. TRACE: Show prediction score per instance (might be too much logging!)
LightGBMUtils	Loads the LightGBM libraries.	INFO: When the libraries are loaded. DEBUG: Names of the libraries being loaded.
SWIGResources	Lowest-level implementation level for resources handling during model scoring.	DEBUG: In-memory instantiation of model. TRACE: Resource release calls.
SWIGTrainResources	Lowest-level implementation level for resources handling during model fitting.	DEBUG: Details for train memory allocation.

How to set up logging🔖

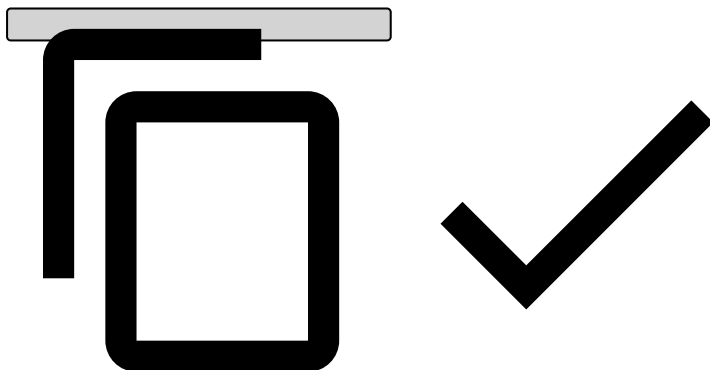
To configure such log levels, edit the file `conf/logger.xml`. Also, for Spark Standalone `spark-lib/logback-executors.xml` should also have such log levels.

Example configuration¹

The following snippet is an example of configuring the logging level for all modules of LightGBM. Debug is a good default level.

In files `conf/logger.xml` and `spark-lib/logback-executors.xml` :

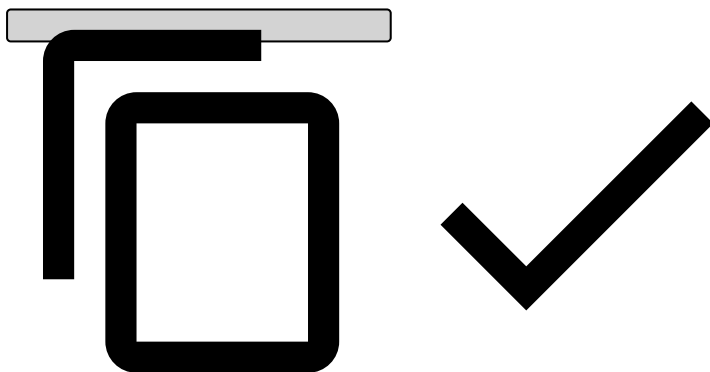
```
<logger name="com.feedzai.openml.provider.lightgbm" level="debug"/>
```



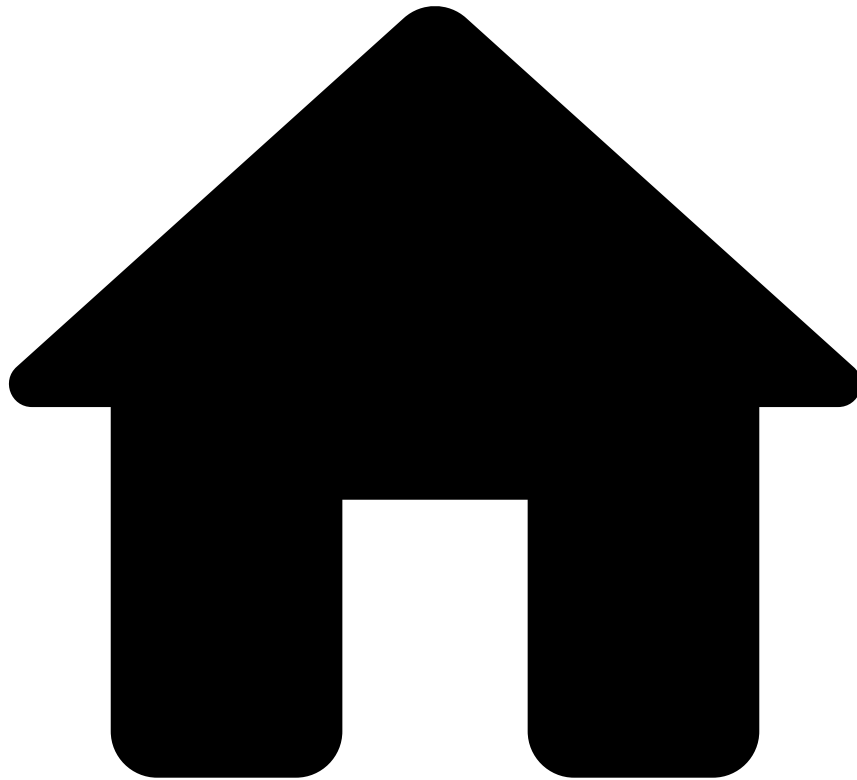
Alternatively, the debug level can be chosen/overridden individually for each component. The snippet below is equivalent to the snippet above, all components use the debug level:

In files `conf/logger.xml` and `spark-lib/logback-executors.xml` :

```
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMBinaryClassificationModel" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMBinaryClassificationModelTrainer"
level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMModelCreator" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMSWIG" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMUtils" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.SWIGResources" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.SWIGTrainResources" level="debug"/>
```



•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- Microsoft LightGBM Provider

On this page

Microsoft LightGBM Provider

Was this page helpful? 🗨️

Light Gradient Boosting Machine or LightGBM is a gradient boosting framework for several types of tree-based learning algorithms.

In this topic you can learn about the parameters and the usage of the Microsoft LightGBM Provider in Pulse.

Microsoft LightGBM

The biggest advantage of Microsoft LightGBM is that training can be very fast with low memory usage and good performance. Whilst Microsoft LightGBM models support ranking, classification and regression tasks in a distributed manner, Pulse's LightGBM machine learning provider only supports binary classification tasks.

One of its limitations is that LightGBM only receives numerical data (even for categorical features). This means that strings must undergo a process called *String Indexing*, where a mapping of 1:1 from a string to a numerical value is performed. Of course, this mapping must be maintained consistent at all times (training and scoring).

You can choose from several boosting types:

- **Gradient Boosting Decision Trees (GBDT)**
- **Gradient Based One Side Sampling (GOSS)**: faster than the one above but may yield slightly lower predictive performance.
- **Dropouts meet Multiple Additive Regression Trees (DART)**: can be much slower but might yield higher predictive performance as dropout can yield less over-fitting.
- **Random Forest (RF)**: not recommended as it is not a true Random Forest but uses boosting as well - use our FZRF instead.

LightGBM in Pulse

Pulse can train LightGBM models and even import external ones using this provider. Only numerical and categorical features are supported, string fields are not used in the LightGBM models.

Numerical features are passed along to the model without transformation, whilst categorical features go through an automatic string-indexing algorithm.

Categorical features handling in Pulse for LightGBM models

Since LightGBM can only take in numerical values, categorical features are fully supported by running a simple **string-indexing algorithm at both training and scoring time**, by transforming the category string value into the *n*th index in the lexicographically-sorted list of categories of a categorical variable.

See the example for the string indexing applied to a categorical feature value:

Property	Value
Input value	"adam"
Categorical feature categories	["John" , "42" , "Adam" , "zoe" , "adam" , "Jane"]
Categories in lexicographic order	["42" , "Adam" , "Jane" , "John" , "adam" , "zoe"]
Output value	4

Notice that indexes start from 0 and for ASCII character codes, numbers come before letters, and uppercase letters come before lowercase ones.

This arbitrary choice does not affect analytical performance as LightGBM does not consider any order for categorical features.

Training a model in Pulse

To train a model in Pulse, start a new model training just like any other model type. If you need more details, see the [Training a Model](#) page for a refresher.

All categorical features in the training (and scoring) schema are automatically mapped using the algorithm explained in the section above before being sent to LightGBM. The same mapping is automatically used during scoring.

Numerical features are not transformed.

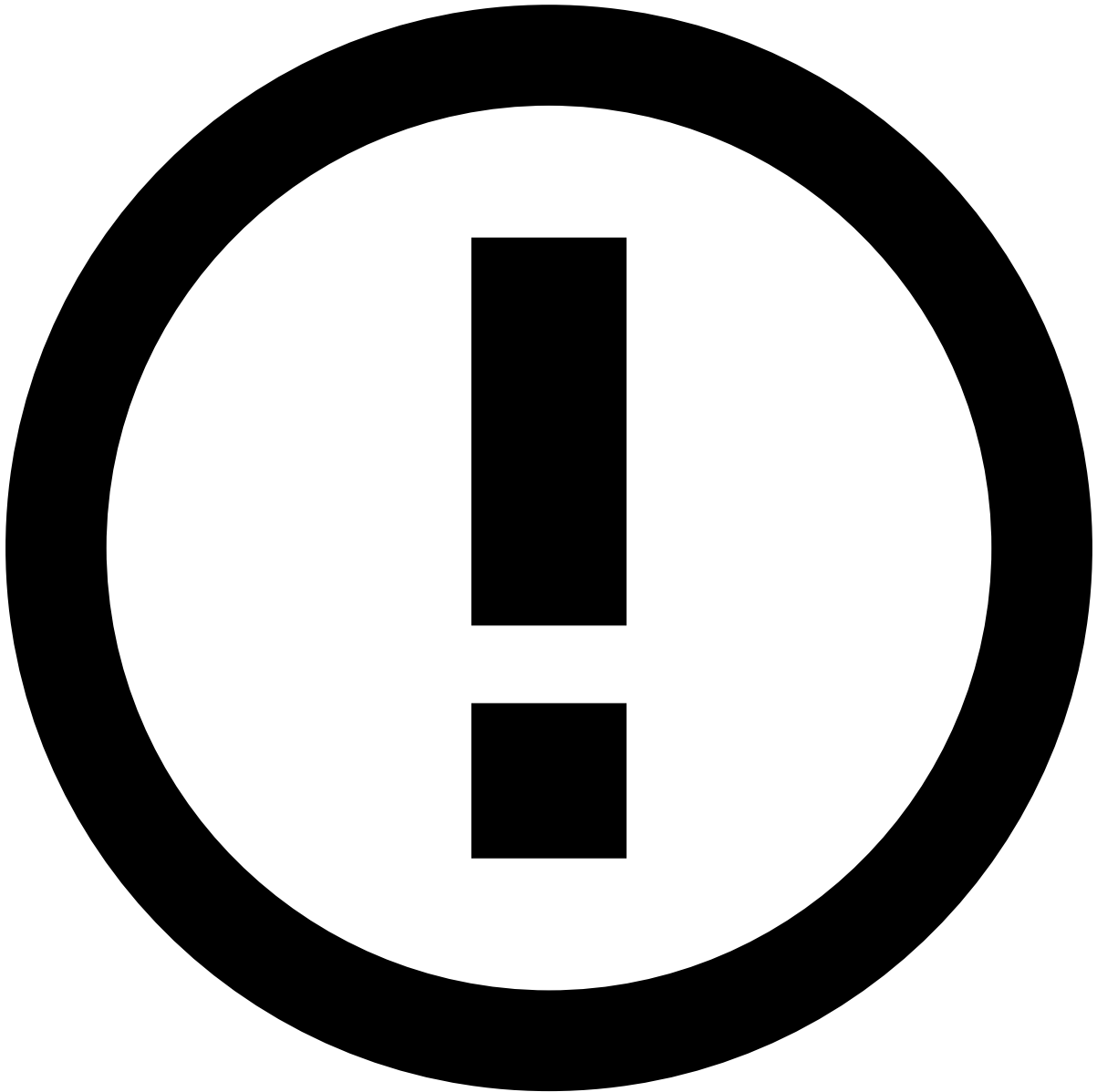
Importing a model to Pulse

You can import a LightGBM model trained outside Pulse as well.

Model import path

To import a model there are two options available:

1. Import the model from a model file path:
 1. The path needs to point to a valid LightGBM model file that LightGBM exports in .txt format.
 2. Since you are not providing the schema, you must define the schema inside the Pulse UI.
2. Import the LightGBM model from a folder path:
 1. The folder pointed to must contain at least the following files:
 1. `LightGBM_model.txt` (mandatory) - the model file as in 1.a.
 2. `model.json` (optional) - a JSON designating the model schema
 2. Since `model.json` is optional:
 1. If not present in the folder, you have to define the schema inside the Pulse UI, as in 1.b.
 2. If present, it will be used for the model schema, unless you override it as in 1.b.



info

Regarding section 2.a.ii, see this [example JSON file](#) for binary classification with both numerical and categorical features.

Importing models with categoricals

If an imported model had categoricals in training, you had to index them, and thus that same indexing must now be performed during scoring, in whichever environment is used.

If indexing during training and scoring does not match, the model scores will be affected.

You have several options and must ensure consistency of those transformations:

1. If you declare, in the Pulse model schema, a variable to be categorical, lexicographic order will automatically be used by Pulse for string indexing during scoring:
 - The only way to have correct scores is to train externally in the first place with the same indexing algorithm, otherwise the model's categorical features in training and scoring differ from each other.

2. If you declare, in the Pulse schema, a variable to be numerical (and at the input it is a string that you had to string-index), you're going to have to do the same exact string-indexing transformation for scoring yourself:
 - Make sure you use the same indexing during training and scoring, but that in scoring, such transformations are reliable across environments, with tolerable latencies and memory usage for your use case.

Parameters🔗

Check the configuration parameters in the [official documentation of LightGBM](#). In Pulse, all parameters follow those specifications, including default values. Handling missing values is always enabled.

Note that there are some parameters that only apply to GOSS and DART boosting modes. Such parameters always have respectively GOSS or DART in their names.

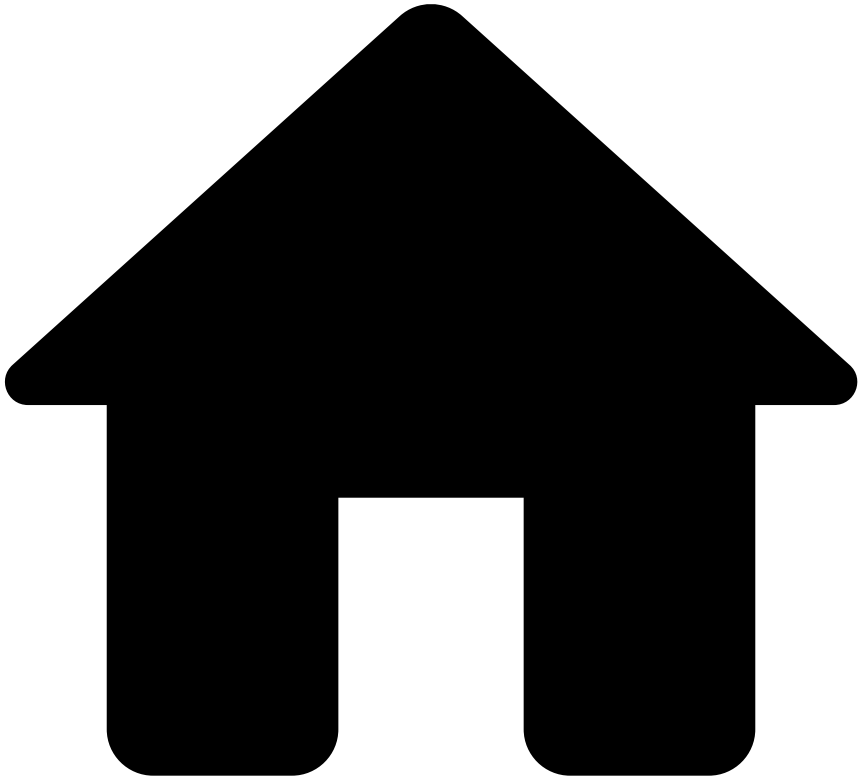
Logging configuration🔗

If you want to know in more detail the progress of the process, you can check the [LightGBM Logging Configuration](#) page to configure logging to suit your needs.

Learn more🔗

- [Microsoft LightGBM official documentation](#)
- [Microsoft LightGBM code on Github](#)
- [FAQ on Gradient Boosting Frameworks](#)

•



- [Core Concepts](#)
- [Workflows](#)
- [Railgun Engine](#)
- Backfills

On this page

Backfills

Was this page helpful? 

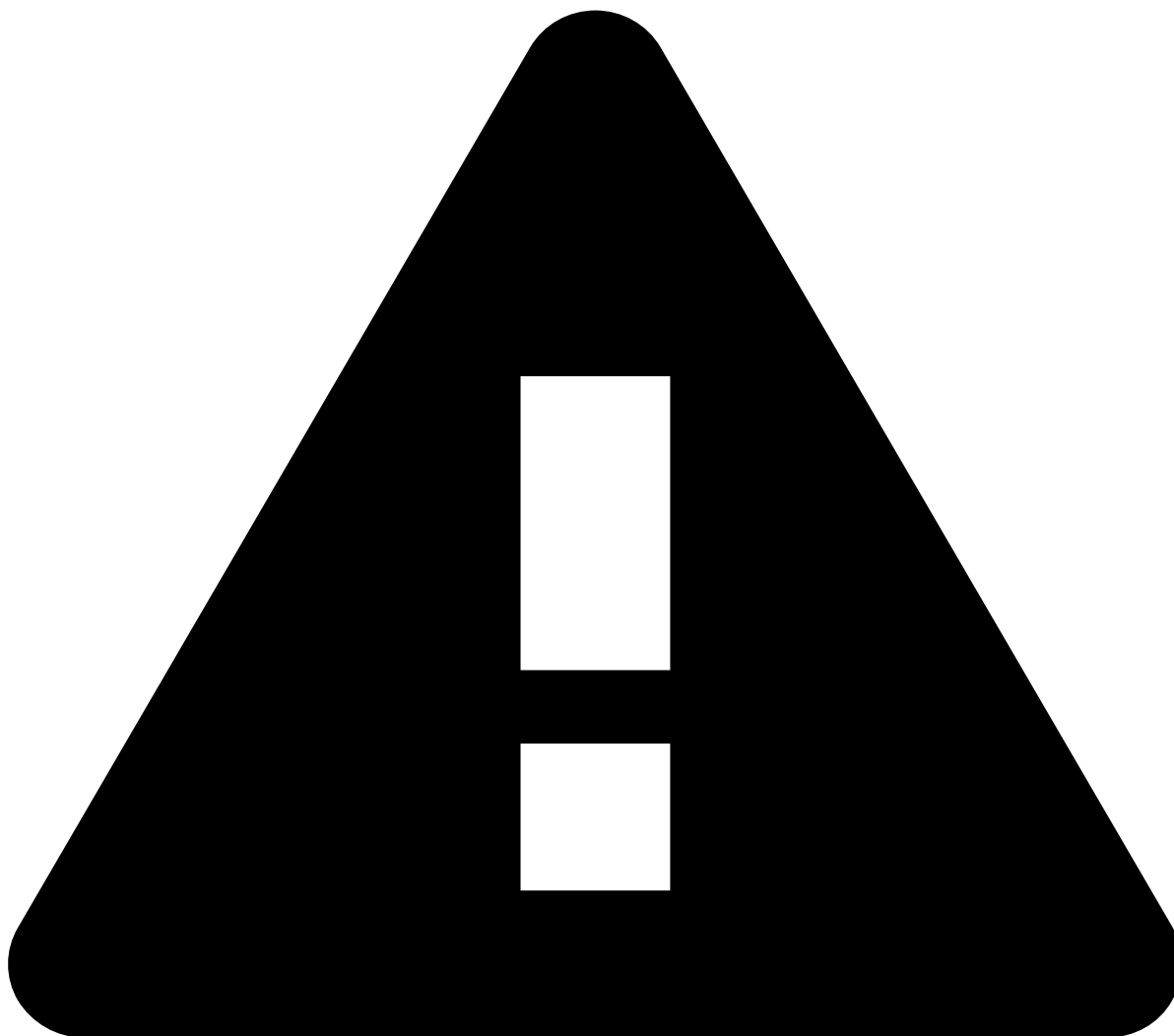
Due to Railgun's streaming nature, when a new metric is defined (or an existing one is updated), its value will initially only account for the events which are processed from that point forward. For accurate results, older data corresponding to the period covered by the metric's time window needs to be collected retroactively, in a process called a **backfill**. Backfills are triggered automatically for any new or updated metrics, forming an ordered queue.

Depending on the size of the profile's windows, and frequency of events, backfilling historical data can take some time due to the potentially high volume of data to be processed. Therefore, all backfills are executed asynchronously, so they won't interfere with application updates.

A UI is available to [manage the queue of backfill jobs](#), so that long-running backfills can be canceled to allow for quicker or more urgent ones to proceed, and restarted later.

How backfills are triggered¹

There are two main ways that backfills can be triggered: automatically due to the app publishes that create or modify metrics, or manually via the UI.



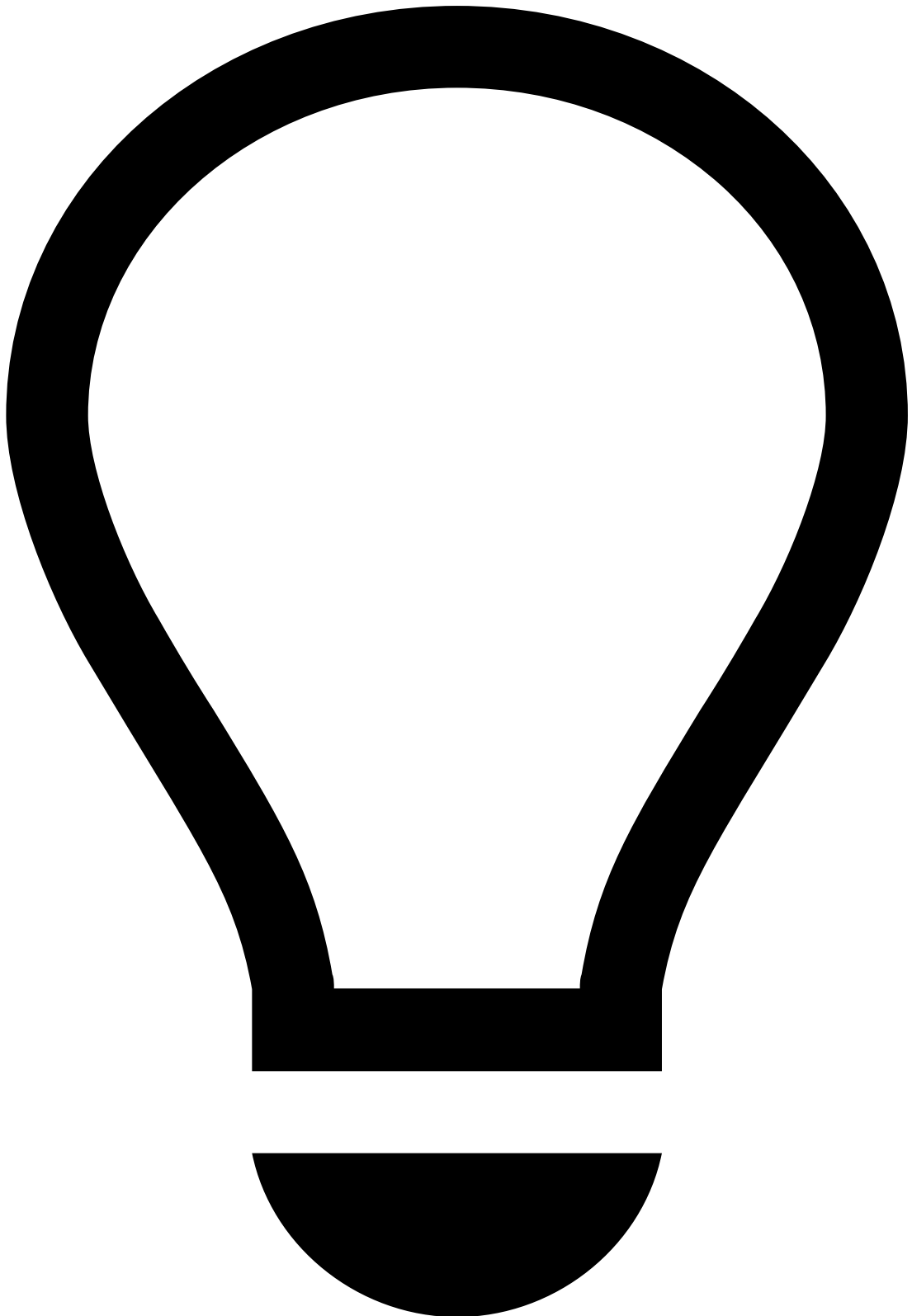
Backfills failing due to encrypted database password

If triggering a backfill results in a Pulse error stating that the Pulse property `pulse.app.database.password` could not be decrypted, then you need to update Pulse to a more recent version.

Backfills triggered via app publishes²

Whenever an app publish action includes the creation or modification of metrics, Pulse will automatically trigger a backfill for the affected metrics. The exception is when only the name or [fallback value](#) of a metric is changed.

When a backfill is initiated through a publish action, new metrics or existing metrics with modifications will return the fallback value until the backfill is complete. Metrics that have not been changed remain unaffected.



Preserving metrics values during backfills

Refer to the [Best practices for metrics in Railgun](#) page to learn how to preserve metrics' values when making changes that trigger automatic backfills.

See the [table below](#) for more details on the kinds of changes that trigger automatic backfills, and what is the impact of each change.

Backfills triggered manually via the UI

[Manual backfill initiation via the UI](#) is primarily intended for maintenance and support purposes.

When a backfill is manually triggered, the behavior of the affected metrics will depend on whether they had already been backfilled automatically or not:

- Metrics that had already been backfilled automatically and were outputting values computed from real-time data, will continue to output real-time computed values (based on the old definition) during the manual backfill process.
- Metrics that had not yet been automatically backfilled and therefore were not yet outputting values computed from real-time data (i.e. were returning their fallback values), will continue to output the fallback value during the manual backfill process.

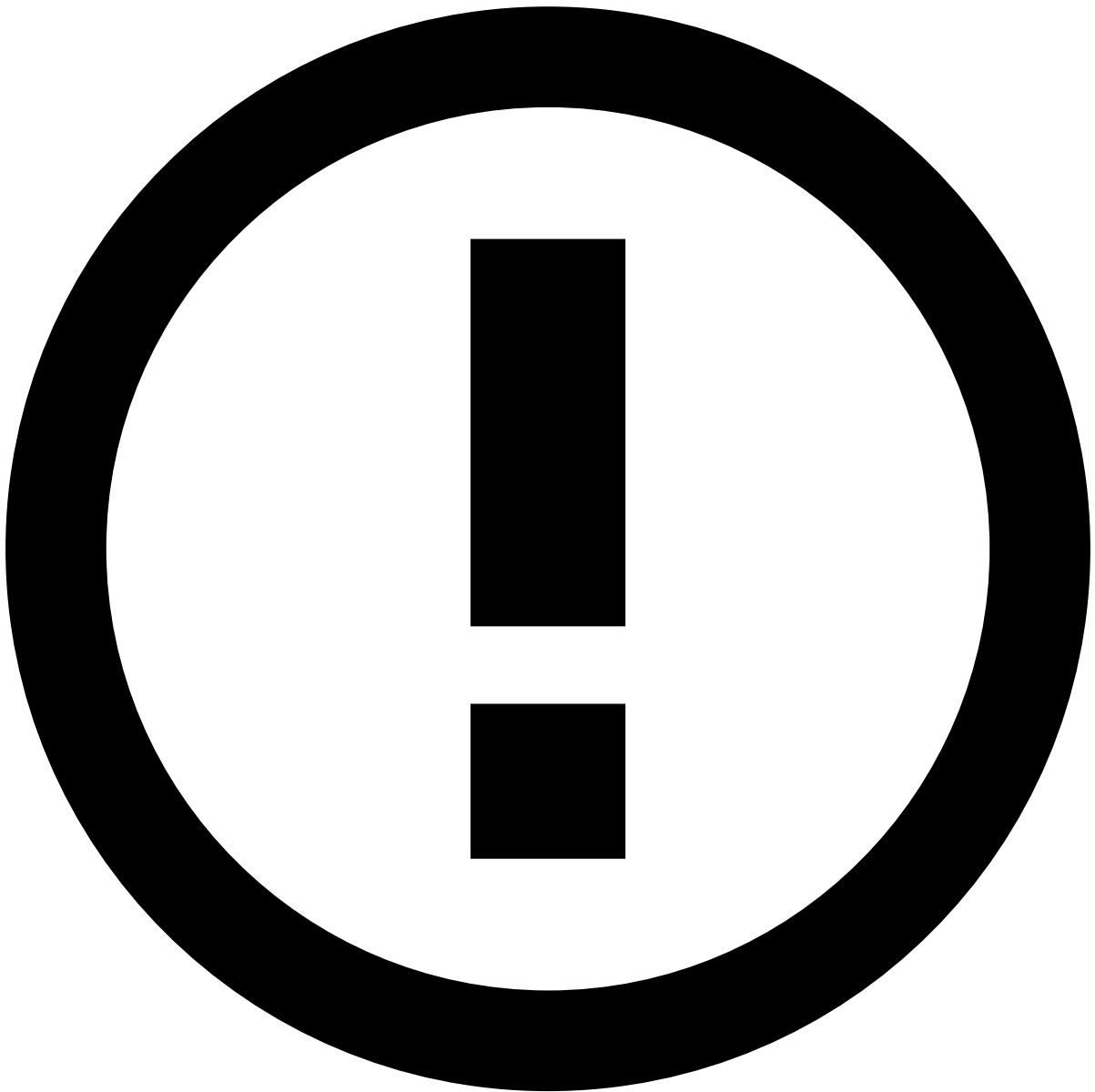
In both cases, like with automatically triggered backfills, the affected metrics will start outputting values based on the new definition once the manual backfill is complete.

Backfill optimization

Railgun backfill queues are automatically managed to avoid redundant work and ensure that each metric is only backfilled once.

For example, when an app is published with new or updated metrics, a new backfill job is added to the queue. This job backfills those new metrics, as well as any existing, unmodified ones that aren't yet fully backfilled. However, if a new backfill job is picked up and by then the previously existing metrics are already backfilled by earlier publish jobs, the system automatically skips those metrics in the new job.

Similarly, when multiple backfills are scheduled (not yet running) for the same app and workflow, the system will automatically merge them by skipping the original backfills and creating a new backfill job containing all metrics that need to be backfilled.



info

Backfills that are [manually triggered via the UI](#) are not affected by this optimization, and will always backfill their selected metrics, even if they have already been backfilled.

This behavior allows for maintenance operations where re-backfilling specific metrics is necessary.

This optimization does not apply to metrics that are updated while a backfill for their older version is still running, nor does it reorder backfill jobs in the queue to prioritize smaller or more urgent ones. For recommendations on how to handle these cases, refer to the [best practices for managing the queue of backfill jobs](#).

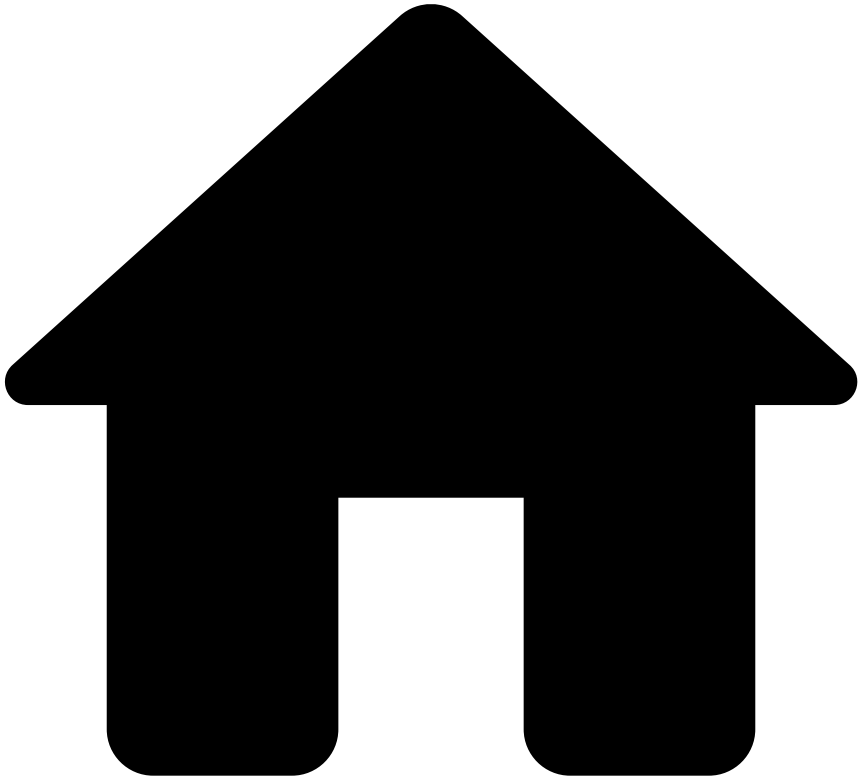
Backfill triggers and their effects📄

The following table details the different actions that can cause a metric backfill.

#	Location	Change	Impact
1		A new metric is created.	

#	Location	Change	Impact
	Metrics Operator		• The new metric will be backfilled. It will only start to output values once the backfill finishes. • No impact on the remaining metrics.
2	Metrics Operator	The definition of a metric is changed, e.g. window definition, group by fields, or aggregation with the following exceptions: • Changing the name of the metric. • Changing the fallback value.	• The modified metric will stop outputting values. It will only restart outputting values once the backfill finishes. • There's no impact on the remaining metrics.
3	Plans	Metrics are moved from one Metrics Operator to another (existing or new one).	Same impact as #1, since this will require new metrics to be created in one of the operators, and deleted from the other.
4	Plans	The type of a field is changed, either on the Plan Schema or in a field that is created inside the plan (e.g. by a map operator in the plan), without renaming the field, and the affected field is used in a metric. Note that if the field is renamed, then the metric definition will change and scenario #2 will apply.	Same impact as #2, since metric definition changes.
5	Workflows	A Plan Service with metrics is deleted and a new one with the same name is created, including plan and plan execution.	All metrics in the Plan Service will be backfilled (because the internal identifier that is generated for the service changes).
6	Backfills tab menu	A backfill is manually triggered .	• You can choose the Metrics within the Metric Operators of the Workflow Plan Services that should be backfilled. • Note that the metrics that are going to be backfilled will not have their values set to null / fallback value during the backfill process, which means that, until the backfill is complete, they will return the old values from before executing the backfill.

•



- [Core Concepts](#)
- [Workflows](#)
- [Railgun Engine](#)
- Metrics in Railgun

On this page

Metrics in Railgun

Was this page helpful? 

A [metric](#) is the smallest unit of work in Railgun. It is defined by a time window, a set of `group by` fields (optionally coupled with a matching set of `lookup by` fields), an aggregation function, and an optional filter.

As an example, the metric *"total amount spent by each card over the last month"* defines a window (*last month*), a `group by` field (*card*) and an aggregation (*total amount*).

Metric components

Windows

The time window that is defined for a metric determines the set of events that are used to compute the metric value. The events that are within the window can change over time as new events arrive to Railgun, some may be added to the window, while others, that were already in the window, may leave/expire from the window. Railgun ensures that the metric values are kept up to date as events enter and leave the window.

In stream processing, there are many types of windows that can be defined. Below, we list the window types that are supported by Railgun.

Sliding Windows

To define a sliding window we just need to specify its size (as a time interval). The window will hold all the events received within this time interval. For example, a sliding window with a 10 minutes size will contain only the events received during the past 10 minutes.

sliding window

The metric value is updated as soon as new events enter the time interval defined by the window, and/or old events are no longer within this time interval i.e. as the logical time in Railgun advances. Do note that, as Railgun processes events and the logical time in Railgun advances, it could happen that the window becomes empty (without events) if indeed there are no events for the time frame that is defined by the window.

In Railgun, the lower bound of the window (older events) is exclusive, while the upper bound (more recent events) is inclusive.

Landmark Windows

Unlike sliding windows, landmark windows expire all events in the window at once at well-defined moments in time. These instants are defined by a reset expression using the `reset(unit:_)` syntax.

New events are added to the window until the time reaches the trigger instant defined by the reset expression. At that moment, the window resets and starts to refill as new events arrive.

For instance, using `reset (day:_)` means the window resets every day at 00:00.

landmark window

Landmark windows have some limitations in Railgun that are important to point out:

1. The reset instant depends on both the expression and the system time zone. Railgun can be configured with a specific time zone for all windows.
2. The expression must define a **fixed interval** at which the window resets. Non-constant frequencies (such as specific weekdays or day-of-month ranges) are not supported, since they are also used by Railgun to configure time intervals for backfills and data retention, which require a predictable, constant repeating frequency.

The following table lists the supported expressions for landmark resets:

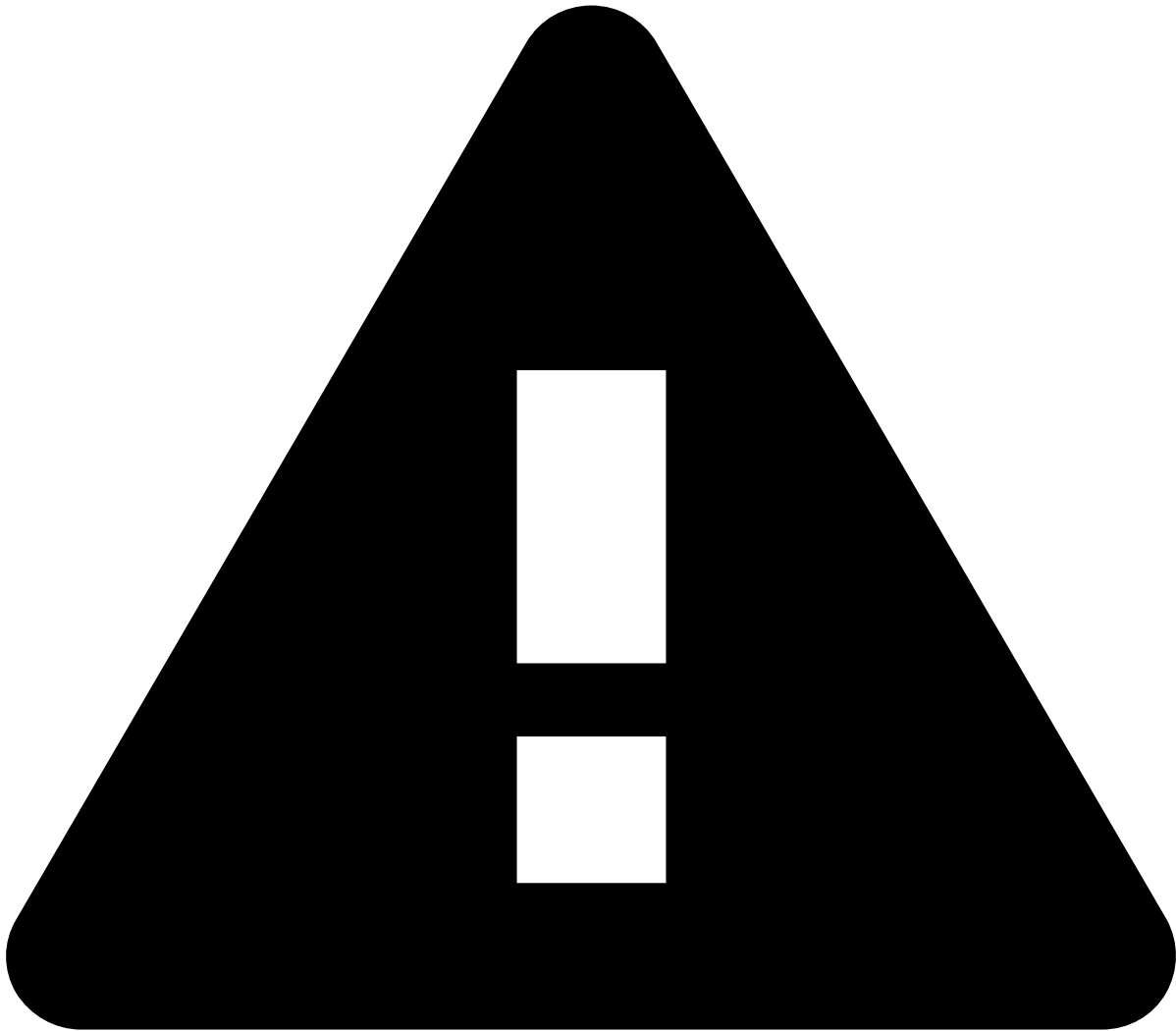
Expression	Reset frequency
<code>reset (day:_)</code>	Daily at 00:00
<code>reset (hour:_)</code>	Every hour at the start of the hour
<code>reset (min:_)</code>	Every minute at the <code>:00</code> second mark



Default values

When a unit is omitted from the expression, Railgun assumes the value is `0` for smaller units and `_` (every) for larger units. For example, `reset(day: _)` automatically assumes `hour: 0`, `min: 0`, and `sec: 0`.

3. The lower bound of the window (older events) is inclusive, while the upper bound (more recent events) is exclusive.
4. Currently, Railgun does not support using landmark windows together with:
 - [Count Distinct aggregations](#) being used in a metric inside those windows;
 - [Output Every](#);
 - [Snapshot Metrics](#);
 - [Delays](#) specified for those windows.



No support for tumbling windows

[Tumbling windows](#) are similar to [landmark windows](#). The difference is that tumbling windows are only defined by their size, and they tumble/reset every time they reach that size. Railgun only supports landmark windows because tumbling windows don't provide a practical advantage over them.

Additionally, tumbling windows may not yield the result that users are expecting. For instance, the tumbling of the windows starts counting at the origin of the Unix time. It is a common misunderstanding that a tumbling windows of 1 day would reset every day at 00:00, but this does not happen due to adjustments such as leap seconds.

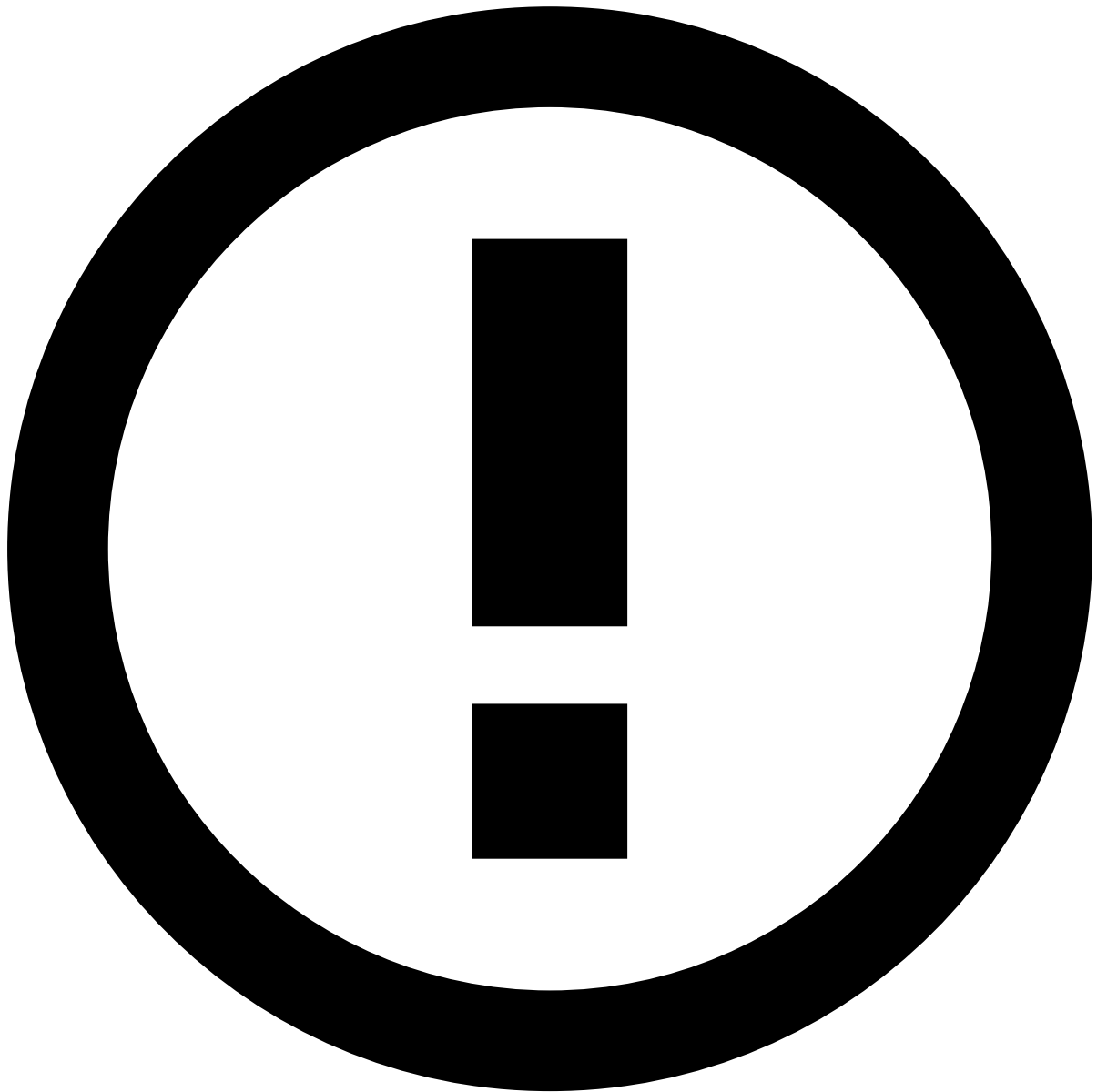
Adding Delays to Windows[↗](#)[📄](#)

In addition to the window types that are listed above, it is possible to add a delay to a window. For instance, Railgun allows to define a sliding window of one day with a delay of one day, as a way to compute the metric based on the events only from the previous day.

delayed windows

Non-windowed Metrics[↗](#)[📄](#)

Railgun makes it possible to define metrics that are not bound to a specific time window. These metrics include all events available in the system, and are updated as new events arrive. Such windowless metrics are useful for computing aggregations over the lifetime of the associated entities; for example, the total amount spent by a customer since the beginning of recorded data, or the first time a customer executed a certain type of transaction.

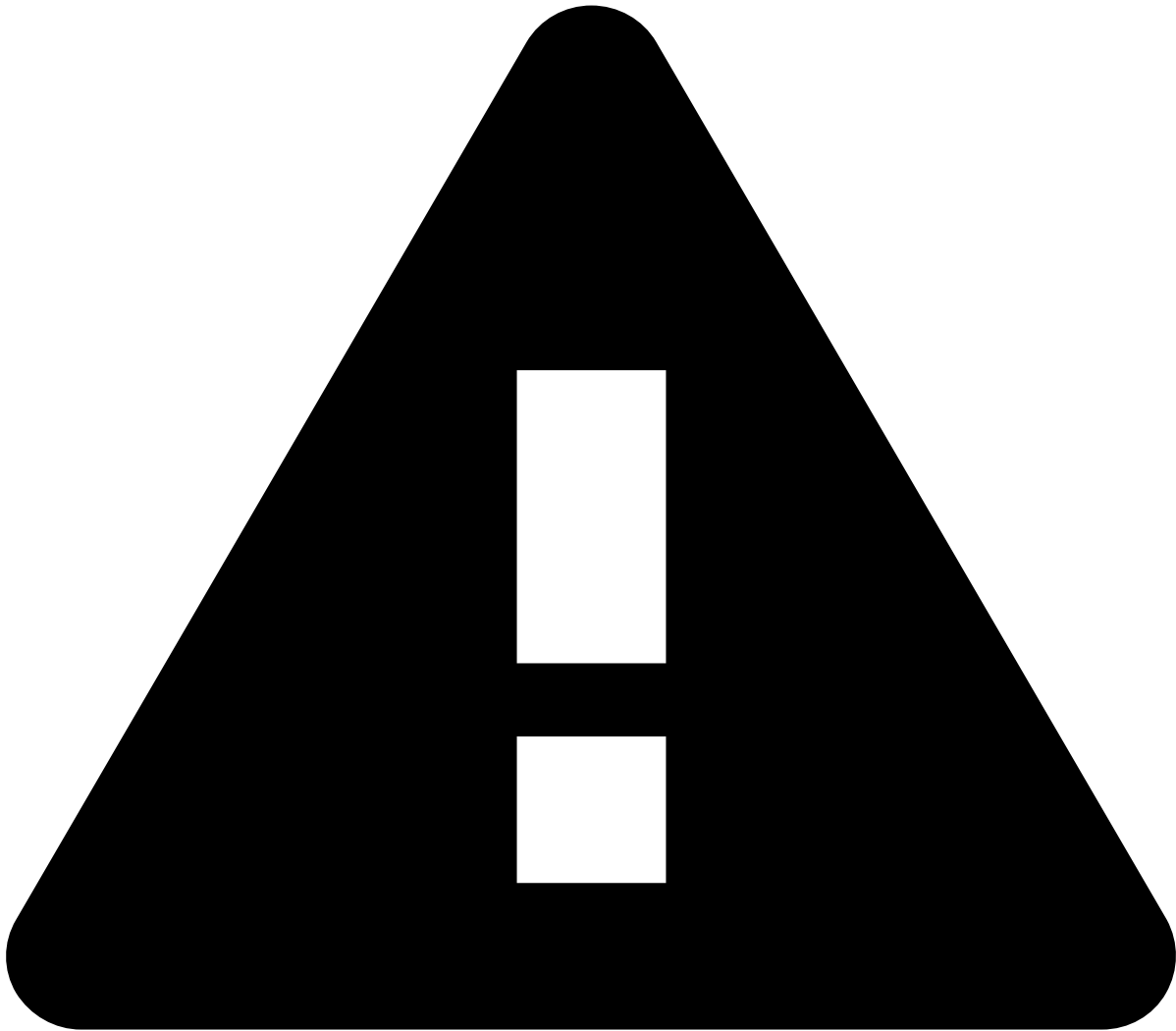


info

Non-windowed metrics can only use a limited set of aggregations. Whereas metrics with sliding time windows can use all aggregations, non-windowed metrics are limited to the `Max` and `Min` aggregations.

When first creating a non-windowed metric, the resulting [backfill](#) will process all events from the earliest available records until the current time. The value of the metric will then be continually updated as new events arrive, while keeping into account the values previously calculated, even when these values result from events that later expire from the maximum data lookup period due to the application of our data retention policies.

For example, a metric that computes the total amount transacted by a customer towards a specific beneficiary will seamlessly accumulate the value as new transactions are processed; older transactions that are no longer available in the system will not decrease the value of the metric, since Railgun preserves an internal state for metric values as part of its operation.



warning

If the definition of a non-windowed metric is updated, the metric will be backfilled again, but the new value will not take into account events that are no longer available due to the data retention policy.

Group By

Similar to SQL, we can define a (non-empty) set of fields in the events to partition windows into groups and compute aggregations individually over these groups. As an example, we can group by the country field to do a count of transactions per country.

group by

The metric then returns the aggregated value (count of transactions) of the group whose key matches the grouping field (country) of the current event.

Lookup By

Optionally, it is possible to specify a set of alternative lookup fields, compatible with the grouping fields, so that the metric will return the aggregation for the group matching the `Lookup By` fields of each event, instead of the group matching the `Group By` fields.

This overrides the default behavior of the `Group By` operator, and allows the retrieval of the aggregation for any of the partitions, as long as the schema of the data stream contains additional fields that can hold the same values as those of

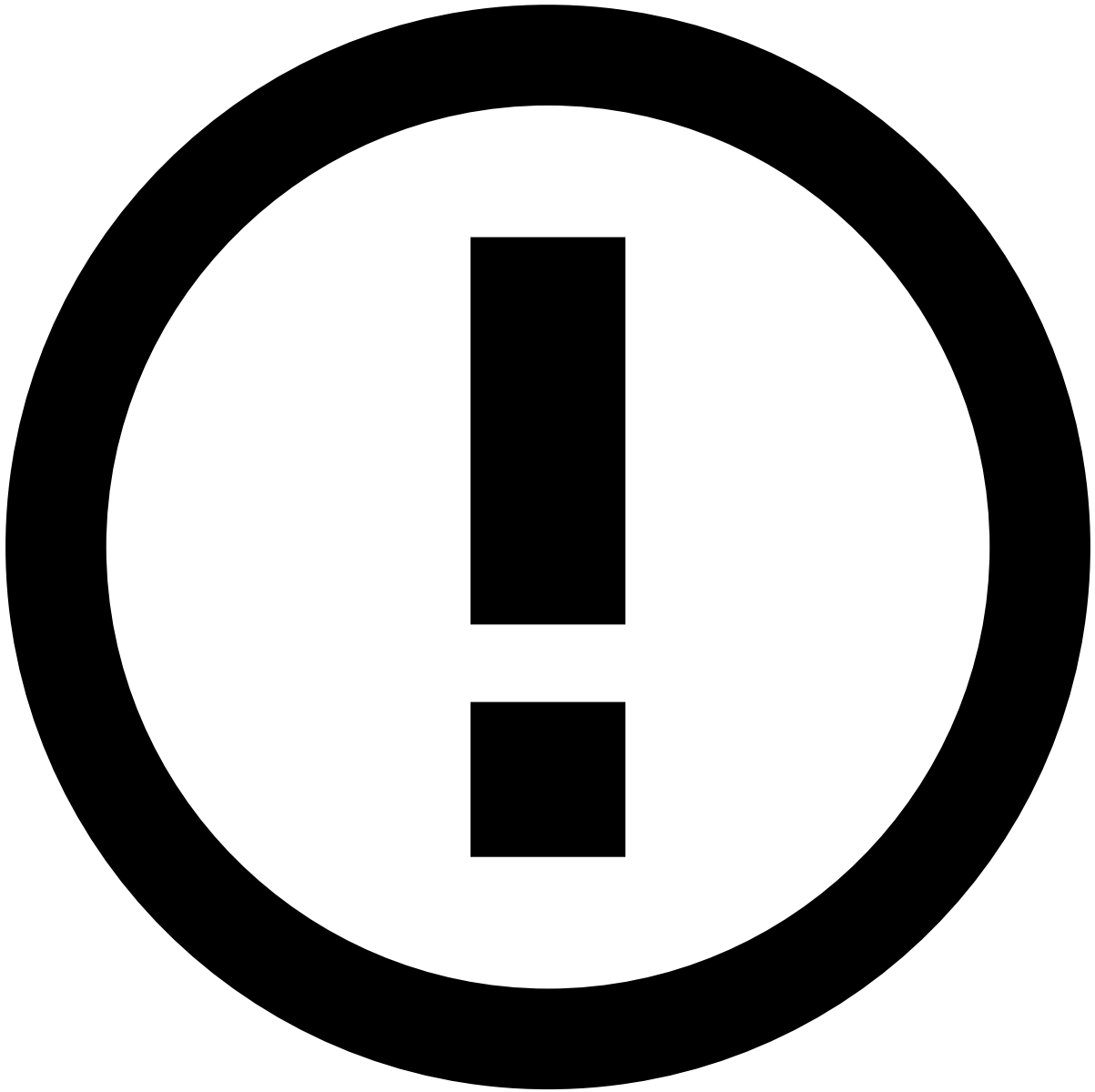
the grouping fields (e.g. a *sender_id* and a *recipient_id*, or a *origin_country* and a *destination_country*, etc.) and can therefore be specified in group-by/lookup-by pairs.

This expands the range of insights the metric can provide, such as determining the total amount received by the account that is sending money in a transaction.

A specific example where such lookup metrics might be useful is to calculate an account's recent running balance (net flow of money) within a rolling period of time, such as the past week prior to each transaction. To do so, one might set up the following structure:

- A regular group-by metric to get the total sent by the transaction's sender:
`{ groupBy: [$sender], aggregation: sum($amount), window: 7 days }`
- A lookup metric to get the total *received* by the transaction's sender:
`{ groupBy: [$receiver], lookupBy: [$sender], aggregation: sum($amount), window: 7 days }`
- A [map operator](#) that subtracts the former from the latter (`total_received_7d - total_sent_7d`), thus combining the two metrics into a third [derived field](#) providing the sender's current balance.

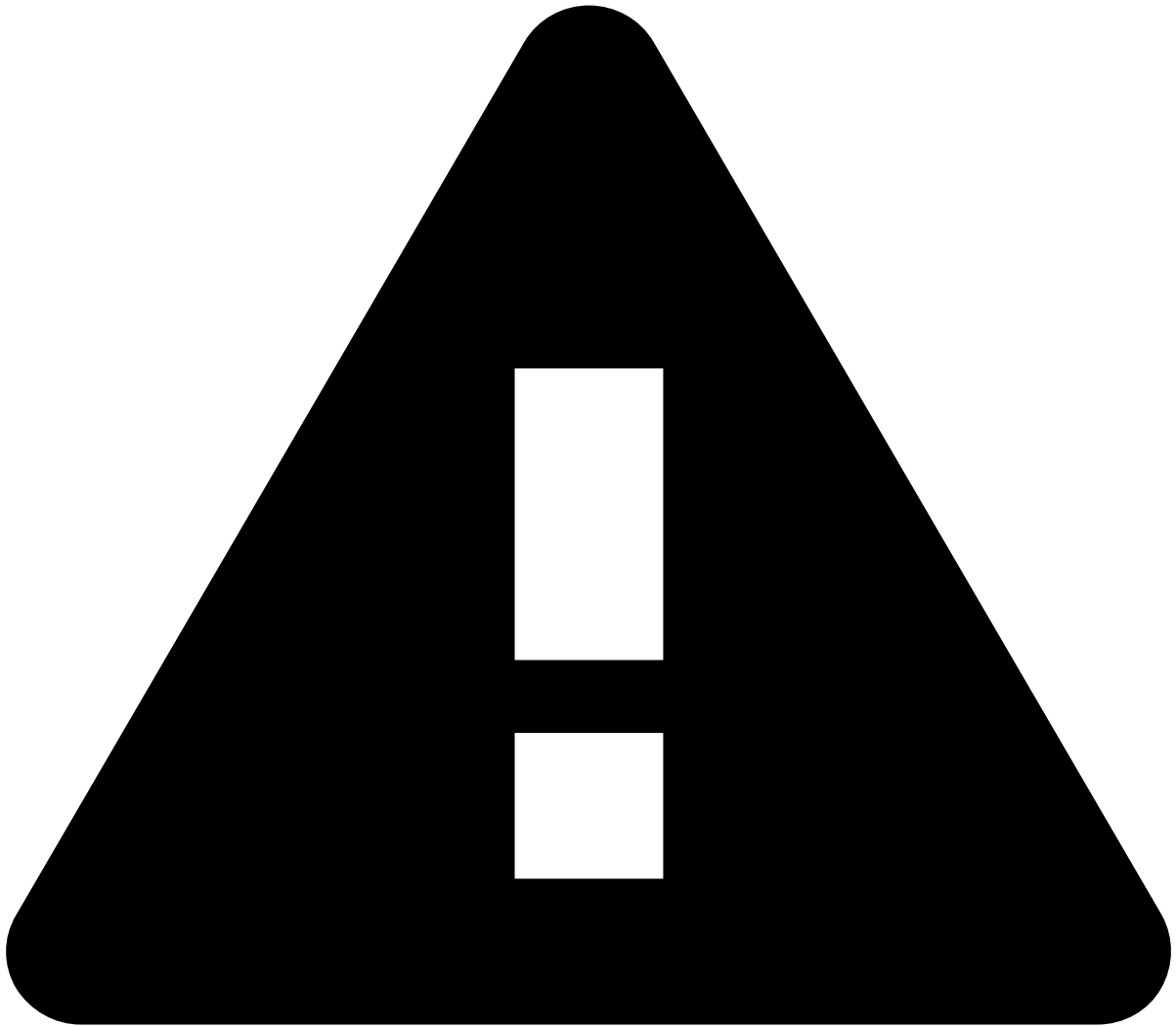
The [Using Lookup Metrics](#) page describes how to implement this example in detail.



Compatibility of the lookup and grouping fields

For the lookup to work, the lookup fields must be compatible both in number and [data type](#) with the grouping fields.

Refer to the [Using Lookup Metrics](#) page to learn how to create lookup metrics in Pulse.



warning

Lookup By fields are not supported in [cross-channel metrics](#).

Aggregations

Aggregations allow performing computations over the multiple events that are in the metric window at a given point in time.

Count

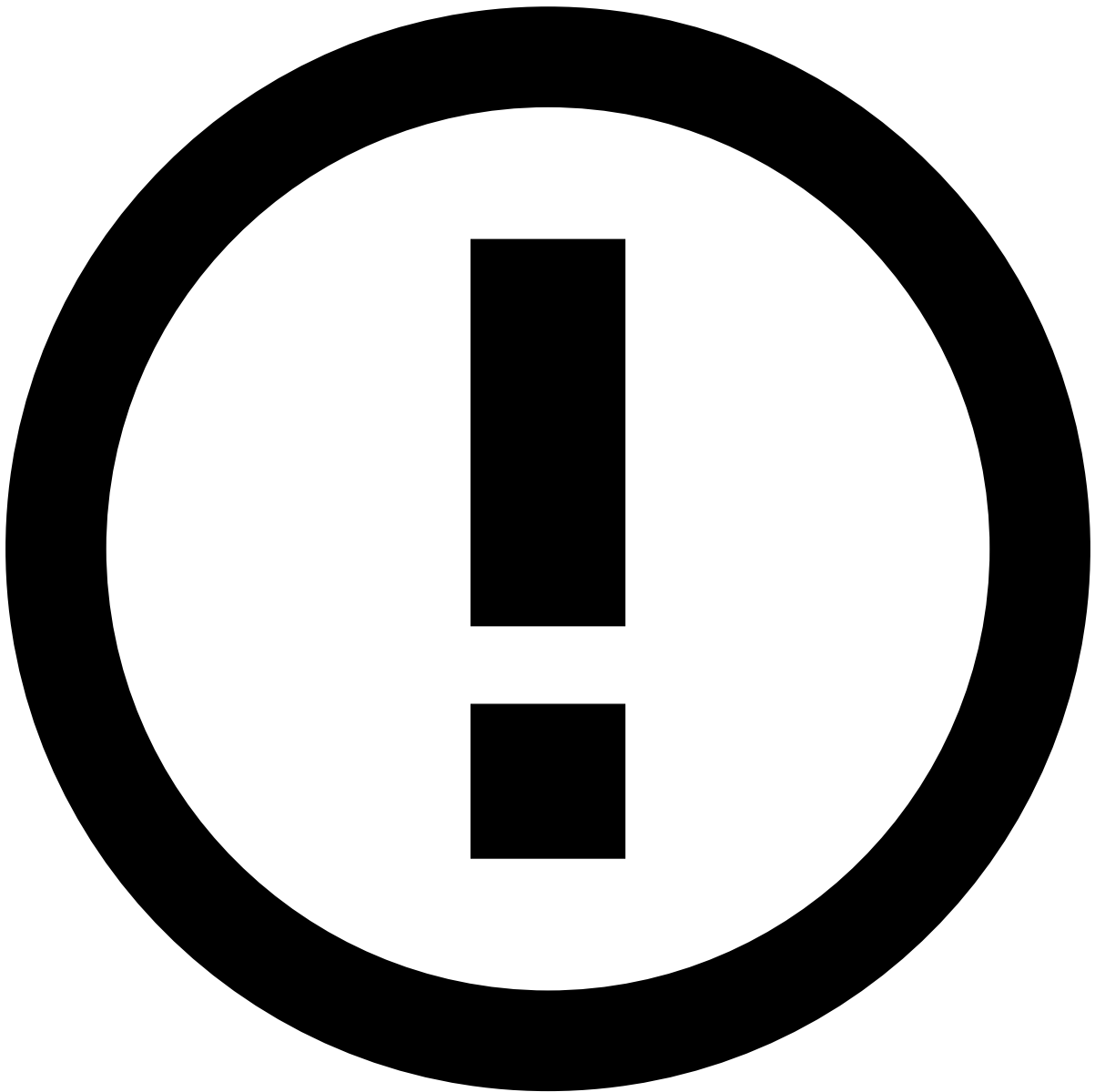
Returns the count of events.

- **Initial value:** 0
- **Return type:** long

Sum

Returns the sum of values of a given field in the events.

- **Supported operand types:** integer, long, float and double
- **Initial value:** null
- **Return type:** long (if operand is an integer or long) or double (if operand is a float or double)



info

The sum is computed as a plain Java sum. Railgun does not use algorithms such as [Kahan's summation](#) to reduce numerical errors when summing float or double values.

Average ⓘ ⓘ

Returns the average value of a given field in the events.

- **Supported operand types:** integer, long, float and double
- **Initial value:** 0
- **Return type:** double

Standard Deviation ⓘ ⓘ

Returns the standard deviation of the values of a given field in the events. The standard deviation is computed in an incremental way by using the [Knuth-Welford's online algorithm](#).

- **Supported operand types:** integer, long, float and double

- **Initial value:** 0
- **Return type:** double

Max / Min

Returns the maximum or minimum value that was observed for a given field in the events. For sliding windows, Railgun computes this aggregation by using the [sliding window minimum/maximum algorithm](#).

- **Supported operand types:** integer, long, float and double
- **Initial value:** null
- **Return type:** same type as the operand



Performance guideline(s)

The algorithm that is used is designed to be as computationally efficient as possible, taking into account that the state is persisted on disk. The algorithm keeps a sorted deque of values that will eventually become the current maximum/minimum values as events expire from the window.

However, if the values of the operand that is used in the aggregation are monotonically increasing (for `min`), such as a timestamp, or monotonically decreasing (for `max`), the underlying deque that is used will keep increasing in size because all values will eventually become the current maximum/minimum. As a consequence, latencies will increase as this deque grows (due to the cost of serializing/deserializing the deque) and may eventually hit a system limitation.

For recommendations on how to handle these kinds of metrics, refer to the [Usage of min\(\) and max\(\) aggregations](#) section in the "Best Practices for Metrics in Railgun" page.

Prevâ

Returns the previous value that was observed for a given field.

- **Supported operand types:** boolean, string, integer, long, float and double
- **Initial value:** null
- **Return type:** same type as the operand

Lastâ

Returns the last value that was observed for a given field. The use of this aggregation is **only useful when the metric uses a filter** where only a subset of events can update the aggregation (in this case it would return the value of the field for the last event that matched the filter). Otherwise, the value returned by this aggregation is the value of the field in the event that is currently being processed, which makes the metric useless.

- **Supported operand types:** boolean, string, integer, long, float and double
- **Initial value:** null
- **Return type:** same type as the operand

Count Distinctâ

Returns the **exact** count of distinct values of a set of fields (one or more fields) in the events.

As an example, if we observe the following values for the fields (country,amount) :

Event	Country	Amount
1	Portugal	100
2	Spain	200
3	Spain	300
4	Portugal	100

...then, a distinct count with the fields (country,amount) would return 3; a distinct count with field (country) would return 2; and a distinct count with field (amount) would return 3.

- **Supported operand types:** boolean, string, integer, long, float and double
- **Initial value:** 0
- **Return type:** long



Performance Guideline(s)

Even though there isn't a strict limit to the amount of operands that can be used in the aggregation, a maximum of 2 is recommended. The size/space of the values of the operand fields may also have an impact on the overall performance. Larger values imply larger keys in RocksDB, which reduces the amount of data that the system can cache in memory.

The cardinality of distinct values may also have a performance impact. More distinct values increases the likelihood of having to read values from disk (instead of reading them from memory), which has a performance impact.

Filters

A filter is a boolean expression against which events are evaluated to decide whether a given event should (if evaluates to true) or not contribute to the metric value. The filter expression **is optional** and is defined as a [PQL expression](#).

Functions available in filters

In the PQL expression that defines the filter, there are some utility functions that are available and can be used. As an example, the `StringUtils.toUpperCase` function can be used in a filter: `StringUtils.toUpperCase(country) == "PORTUGAL"` .

Railgun doesn't allow for clients to plug in their own functions. Instead, it supports the following commonly used functions:

Name	Fully Qualified Name
StringTools	com.feedzai.commons.udfs.StringTools
StringUtils	org.apache.commons.lang3.StringUtils
Collections	com.feedzai.pulse.fraud.udf.Collections
Country	com.feedzai.pulse.fraud.udf.Country
Crypto	com.feedzai.pulse.fraud.udf.Crypto
Location	com.feedzai.pulse.fraud.udf.Location
Maths	com.feedzai.pulse.fraud.udf.Maths
Strings	com.feedzai.pulse.fraud.udf.Strings
Time	com.feedzai.pulse.fraud.udf.Time
Zip	com.feedzai.pulse.fraud.udf.Zip
CollectionUDF	com.feedzai.pulse.pkernel.runtime.CollectionUDFs

More functions can be added to this list but must be first evaluated by the Railgun maintainers.



danger

This feature is maintained to facilitate Railgun adoption. It will eventually be deprecated, and as such it has limited support.



info

This limitation only applies to the usage of functions in metrics' filters. **It does not apply** to the usage of functions (UDFs) in Pulse map operators.

Fallback Value📄

When defining a metric in Pulse, users may choose to define a **fallback value** for the metric. When this fallback value is defined, Pulse will automatically coalesce `null` metric values to the value that the user defines.

fallback-value.png

There are two occasions where metrics calculated by Railgun can return a `null` value:

1. **When the metrics' aggregation can output a null value.** As an example, a "Last" aggregation can return `null` if there is no last value (i.e. no event is contributing to the metric), or if the last value exists but is null. But a "Count" will never output a `null` as it will return 0 if no event is within the metric window. See the [default values for each specific aggregation](#) that Railgun supports.

2. **While the metrics is being backfilled.** For example, when a metric was just created, or its definition was updated, it will output a `null` value until its backfill successfully completes.

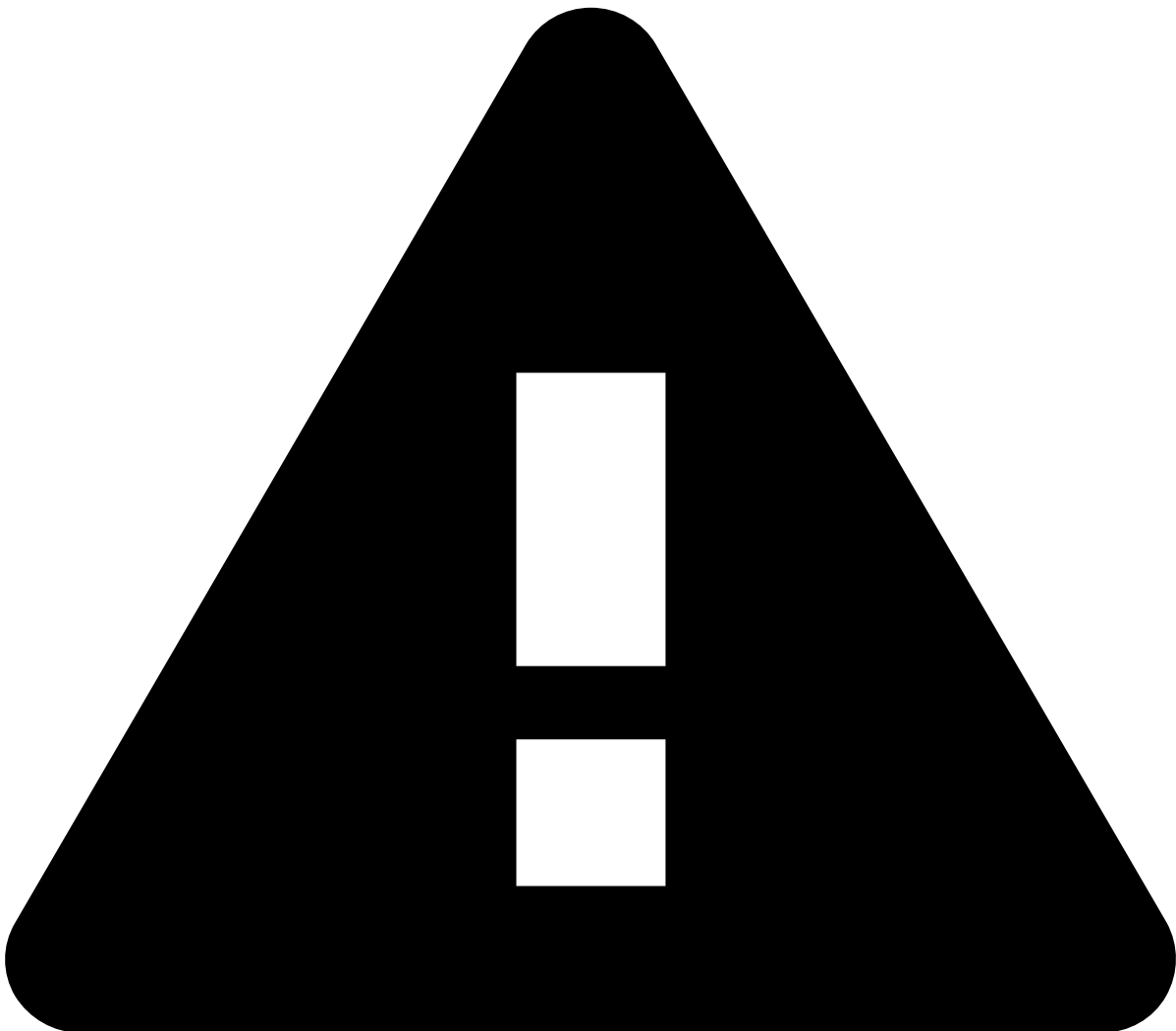
This means that any downstream logic in Pulse workflows must take into account that the metrics' values can be `null`. The possibility to define a fallback value for these cases makes the workflow's behavior more user-friendly and less error-prone.

By default, the fallback value is enabled with a predetermined value that is automatically derived from the metric output type:

- If the output type is a **String**, null values will be coalesced to an **empty string** (`""`).
- If the output type is an **Integer or floating point**, null values will be coalesced to **0**.
- If the output type is a **Boolean**, null values will not be coalesced, remaining as **null**.

The fallback value can also be **set to a custom value**, by entering the desired value in the input box next to the "Enable" checkbox:

custom-fallback-value.png



Custom fallback values

Custom fallback values must be appropriate for the metrics' data type; otherwise the validations will fail. If the input box is left empty, then a null or empty string (for String data types) will be used for the coalesced value.

It is also possible to **disable the fallback value** (by unmarking the "Enabled" checkbox). In this case, no coalescing will be applied. If the metric value is null, then it will be forwarded as null to the remainder of the workflow.

disabled-fallback-value.png

Output Every (Scheduled Metrics equivalent)

This is a feature that can be optionally enabled and that causes the metric value to only be updated/refreshed at a given frequency, defined by a reset expression. It is as if Railgun periodically takes a snapshot of the metric value at a given moment in time, and outputs that value until the next snapshot is taken.

The update frequency is defined by a reset expression (see [Landmark Windows](#) for supported units).

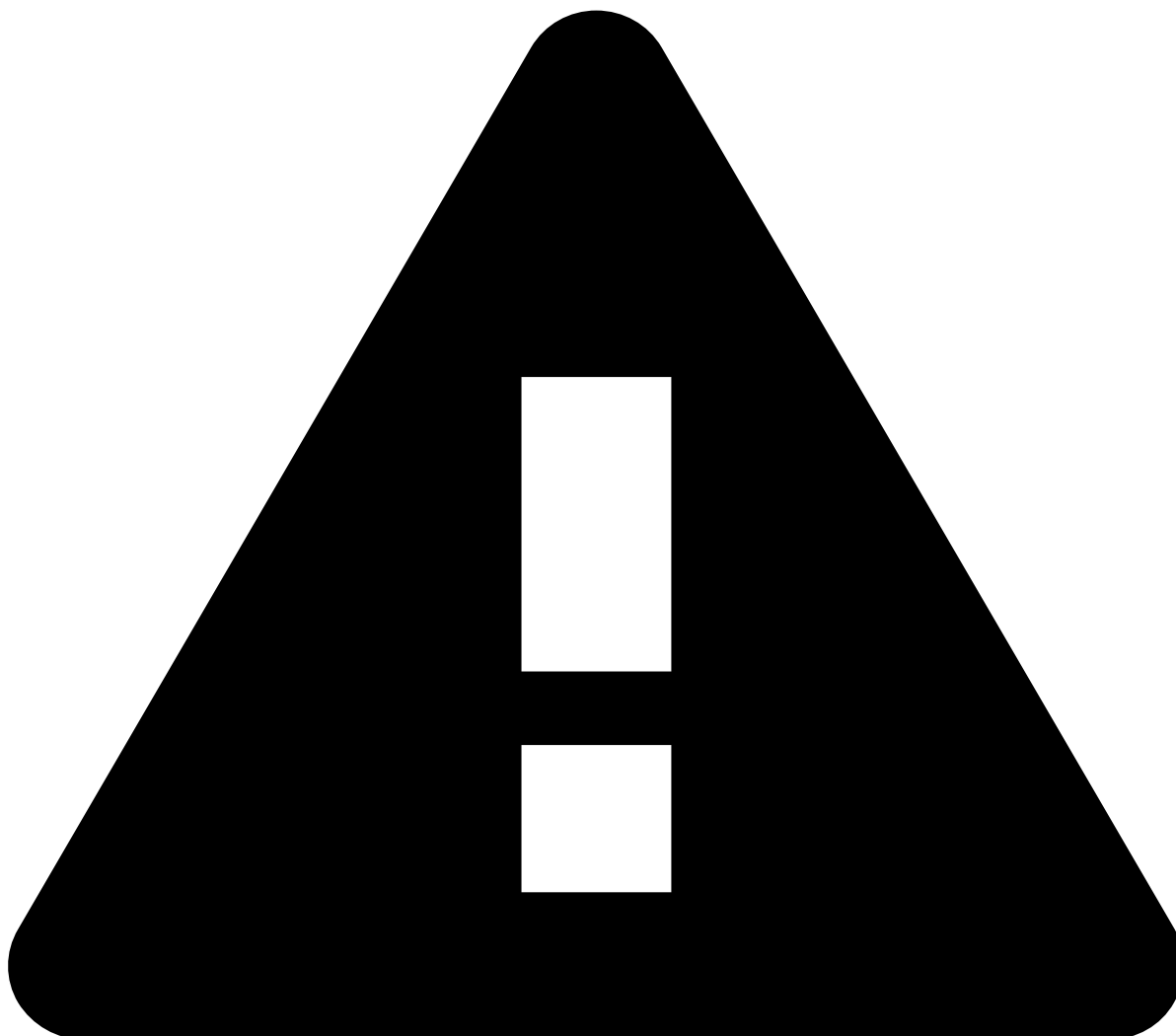
output every



Rationale for `output every`

This feature was added so that users can have metrics that semantically are equivalent to [scheduled metrics](#), i.e. metrics that only refresh/update with a given frequency, and that are computed by running Spark jobs against the Event Storage.

Performance-wise, the metric continues to be updated in real time â its internal state continues to be updated one event at a time.



No support for the `first` aggregation

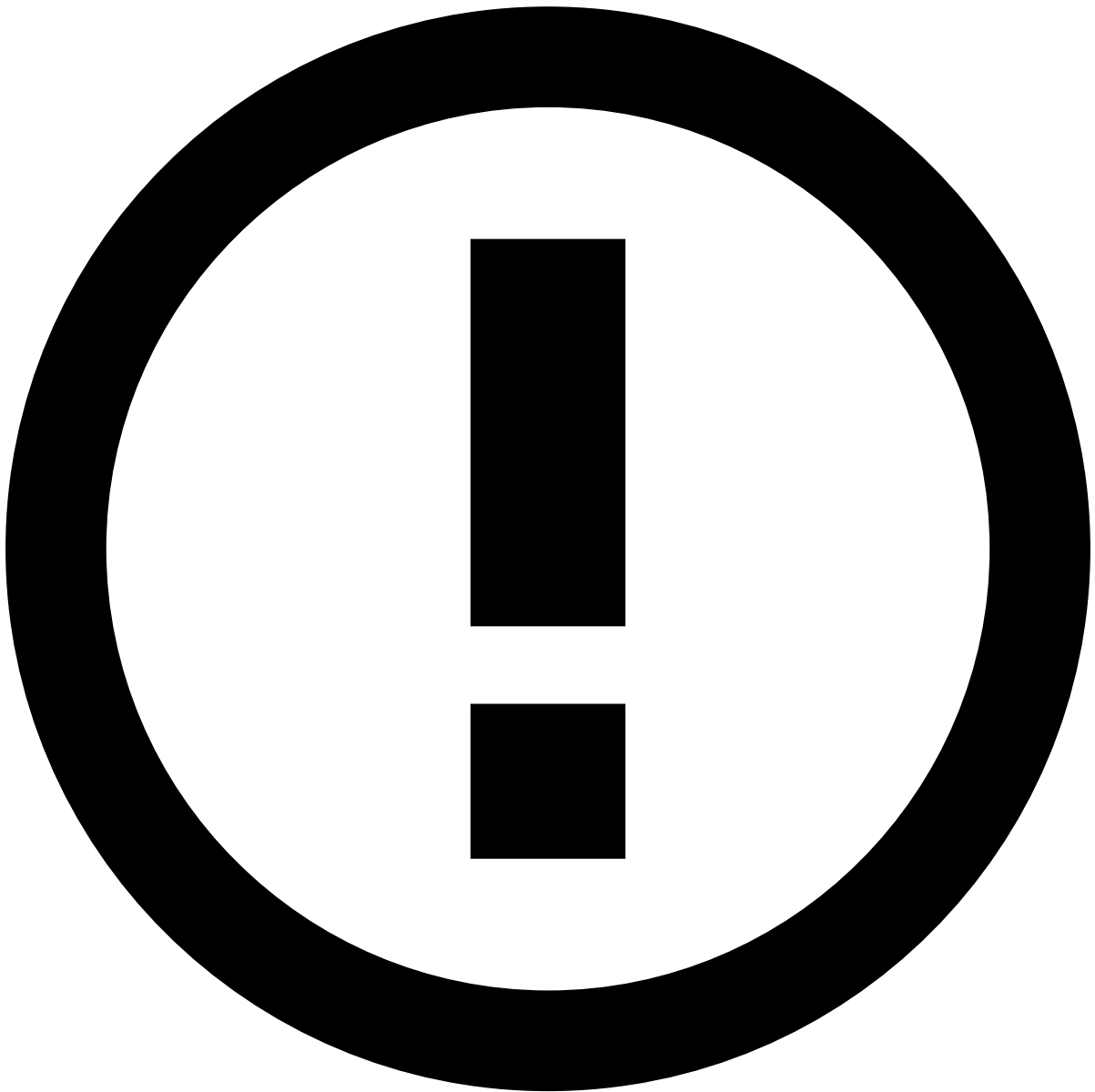
The `first` aggregation is not supported due to performance reasons. To compute the first aggregation Railgun would need to keep a queue of all event field values that can potentially become the output of the aggregation. This means that the queue would grow linearly with the number of events in the window.

For the same reason, it is not a good idea to use the minimum aggregation over a timestamp.

Snapshot Metrics

This feature is similar to [Output Every](#) metrics, with the caveat that instead of storing a single snapshot that is then replaced by a more recent one, Railgun allows a pre-defined number of snapshots to be stored and read.

As with Output Every, the frequency of the snapshots is defined by a reset expression; the difference is that Snapshot metrics also require the user to define a snapshot retention. For example, using `reset (day:_)` with a retention of 4 snapshots allows the user to read data from the past 4 days.



info

Snapshot metrics can be used as a replacement for the Output Every metrics by configuring a Snapshot metric with a retention value of 1.

However, keep in mind that Output Every metrics are more space conservative and will have better performance.

Snapshot metrics

Since we store multiple snapshots we must be able to query these snapshots. To achieve this, users can inject an event with a timestamp in the past and Railgun will look up the closest previous snapshot available.

The example below shows how injecting events in a certain day will return the snapshot stored in the previous day. If a snapshot is no longer available due to being expired, Railgun will return a 'null' value in response.

Snapshot metrics query

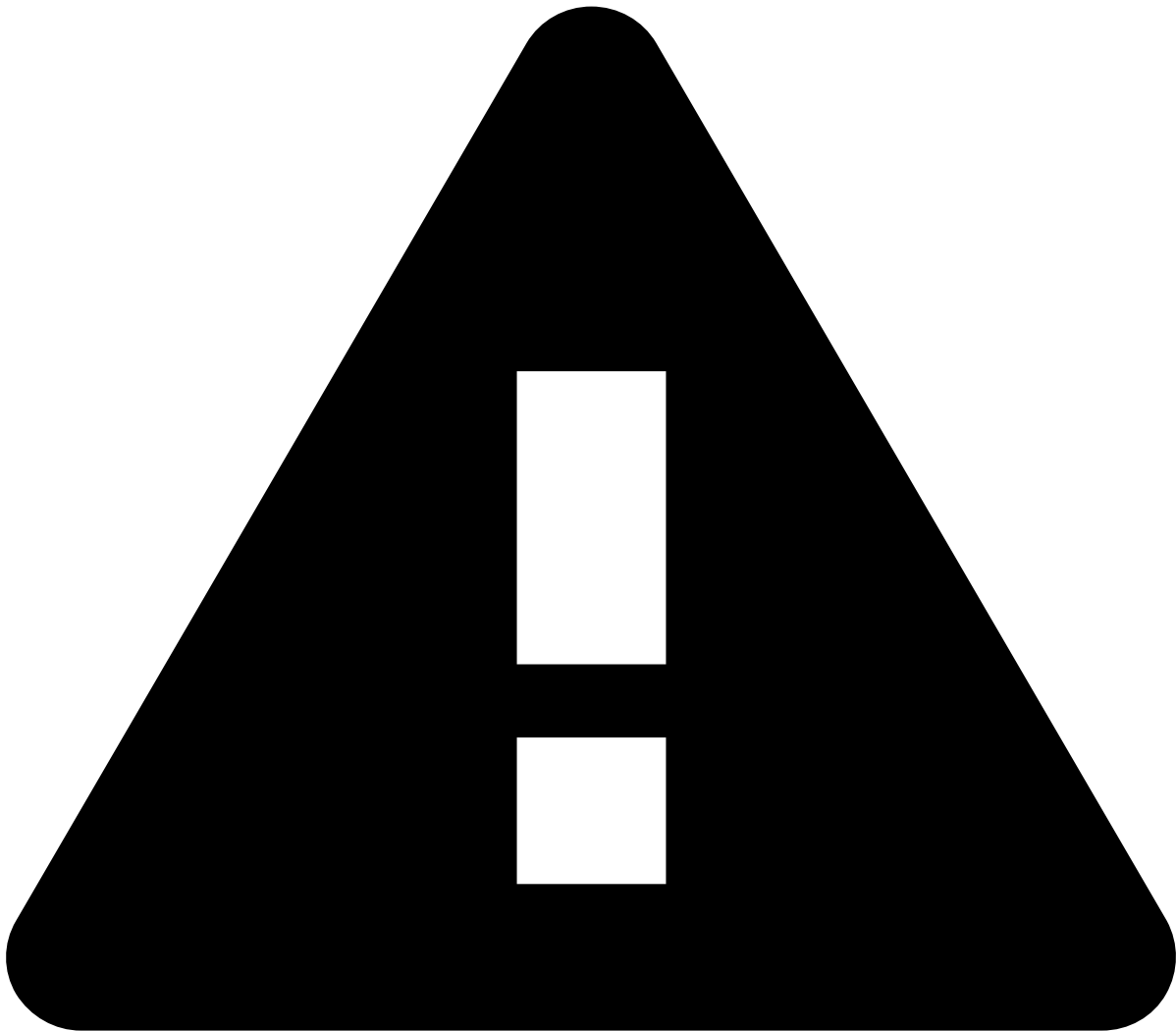
Cross-channel metrics⚠️

A **cross-channel** metric is a specific case of an [omnichannel metric](#) that only has **2 channels**, in which:

- Only one **Secondary** channel can contribute to the cross-channel metric.
- Only one **Main** channel will fetch the metric value, created and updated on the **Secondary** channel.
 - The **Main** channel **cannot contribute** to the cross-channel metric.

Note that each channel has its own detection strategy, and it's implemented in a Pulse application [workflow](#).

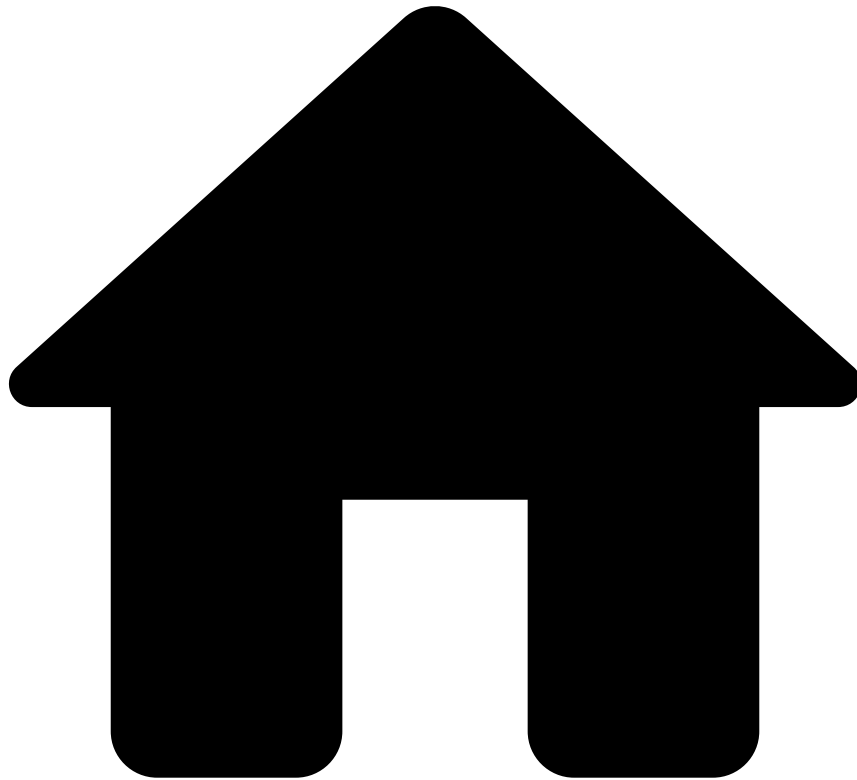
See the [Using cross-channel metrics](#) page to know how to create Cross-channel metrics in Pulse.



caution

Do not confuse these metrics with [omnichannel metrics](#), which are a superset of metrics that can be used in multiple channels, and can have **more than one** channel contribute to them. Also note that Railgun doesn't support omnichannel at the moment.

•



- [Creating a Data Science project](#)
- [Processing your data](#)
- [Analyzing and sampling your data](#)
- Creating a Stratified Sample

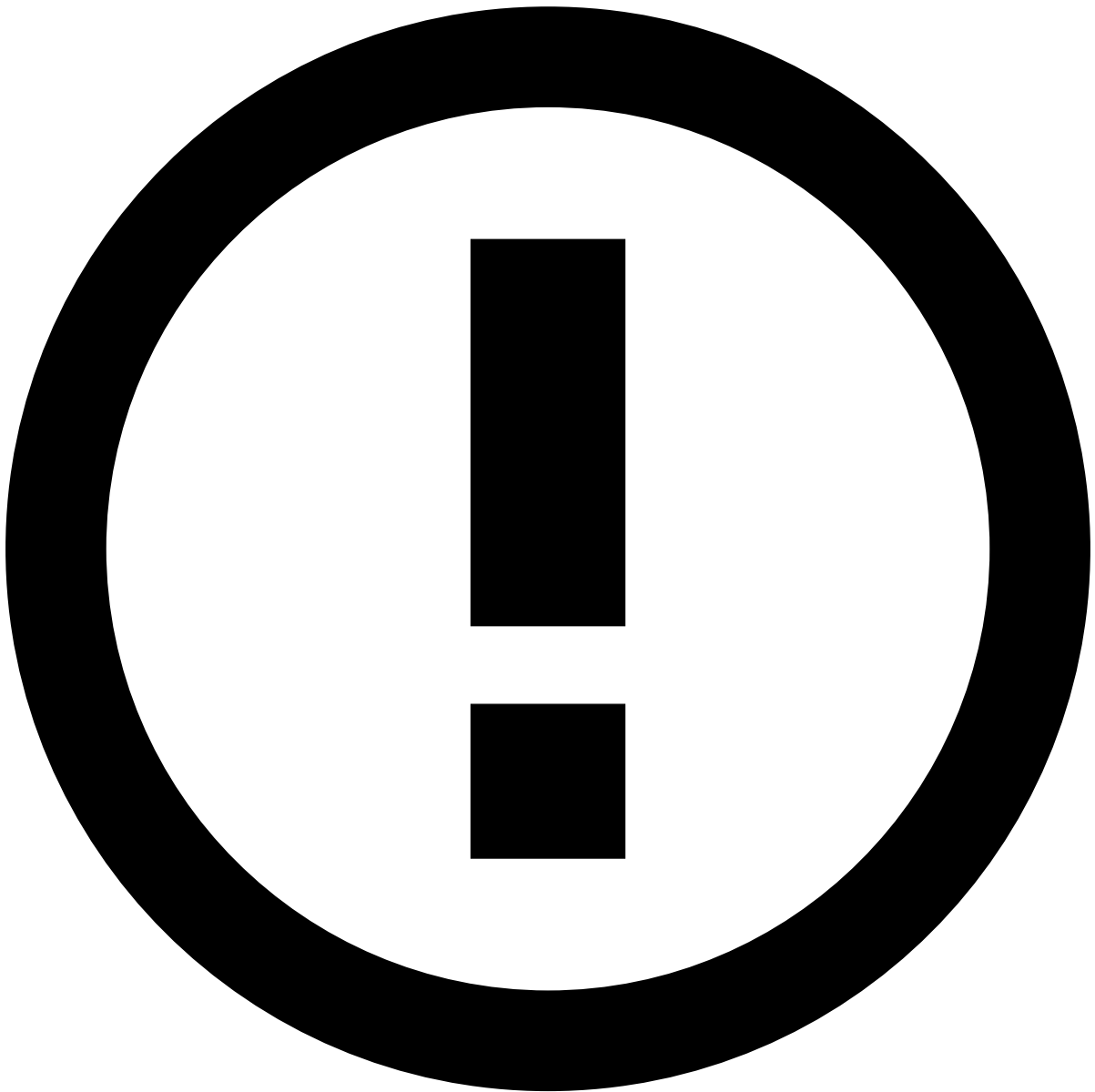
On this page

Creating a Stratified Sample

Was this page helpful? 🗨️

Generating a stratified sample is very common in fraud detection scenarios and represents the basis for [Feature Engineering](#).

Pulse enables you to design a [Plan](#) and generate a stratified sample from terabytes of historical data, with little need for manual actions, technical proficiency in external tools and access to cluster infrastructure.



info

Alternatively, you can perform these actions from the JupyterLab user interface. Standard notebooks are available within [JupyterLab](#) – in particular the Sampling Notebook, which demonstrates how to apply efficient sampling strategies over large datasets using Feedzai's sampling library.

How to

The following instructions exemplify a way to generate a stratified sample for a card fraud scenario. This plan extracts some cards that never had fraud, and all fraudulent cards, out of the original data, and obtains all transactions for those cards.

To design a plan and generate a stratified sample for the *Transactions* data source, execute the following actions:

1. In the **Data Science tab**, **Add** a new plan.
2. Set the plan as [Non-Deployable](#).

3. Design your Stratified Sample plan as displayed in the following image:

Configure the elements as follows:

1. Edit the **Source** element and specify the date source. For example, *Cleaned Transactions*.
2. Edit the **Map** element to add a new field to the date source, as follows:

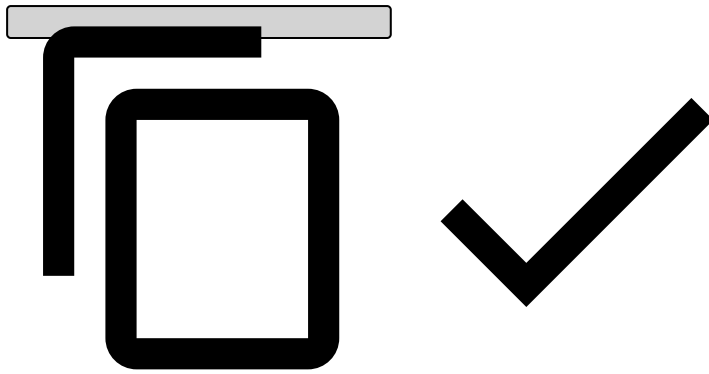


Clipboard

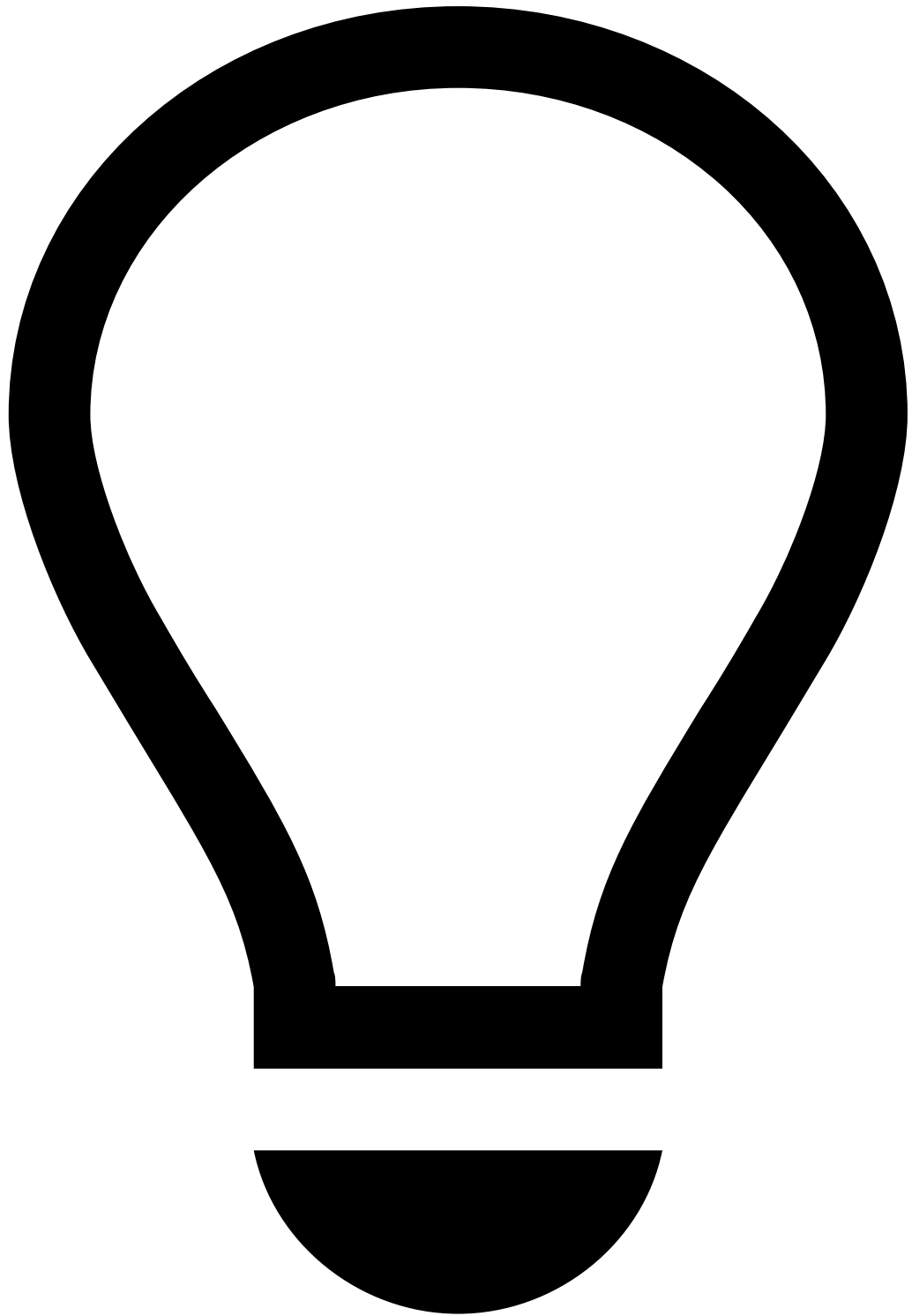
You can also copy Operators to the clipboard in order to paste them in other Plans.

1. Select the **Fields to Add** tab and name the new field as *isFraud*.
2. Write the following expression for the field value:

```
if (Fraud == 'True') then 1 else 0
```



3. Add the field to the list of fields to be added to the data source.
3. Edit Edit the **Group By** element to specify grouping by *AccountID* (represents the card number in this case) and calculate the *Sum* aggregation of the *isFraud* field.

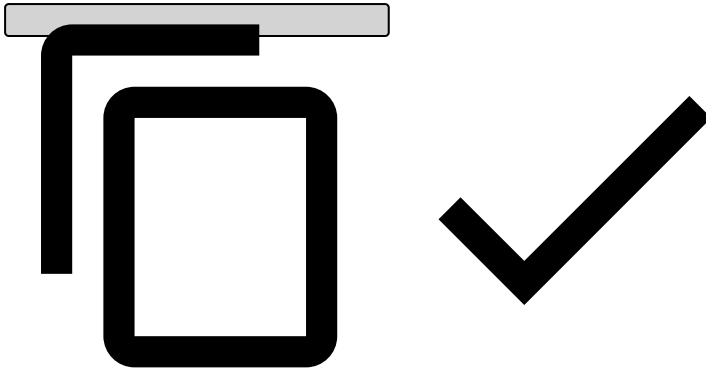


Checkpoint

At this stage you have the number of fraud occurrences for each card number, calculated into the *sumFraud* field. The plan workflow will branch to segment fraudulent cards from cards that never observed fraud.

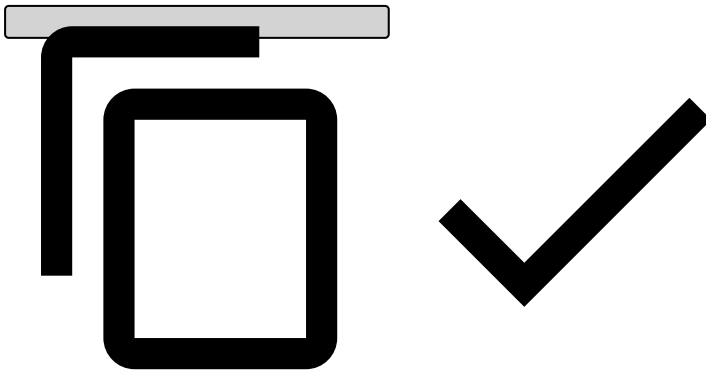
4. Edit Edit the **Filter** element with the following expression to obtain only the card numbers **with** fraudulent transactions.

```
sumFraud > 0
```

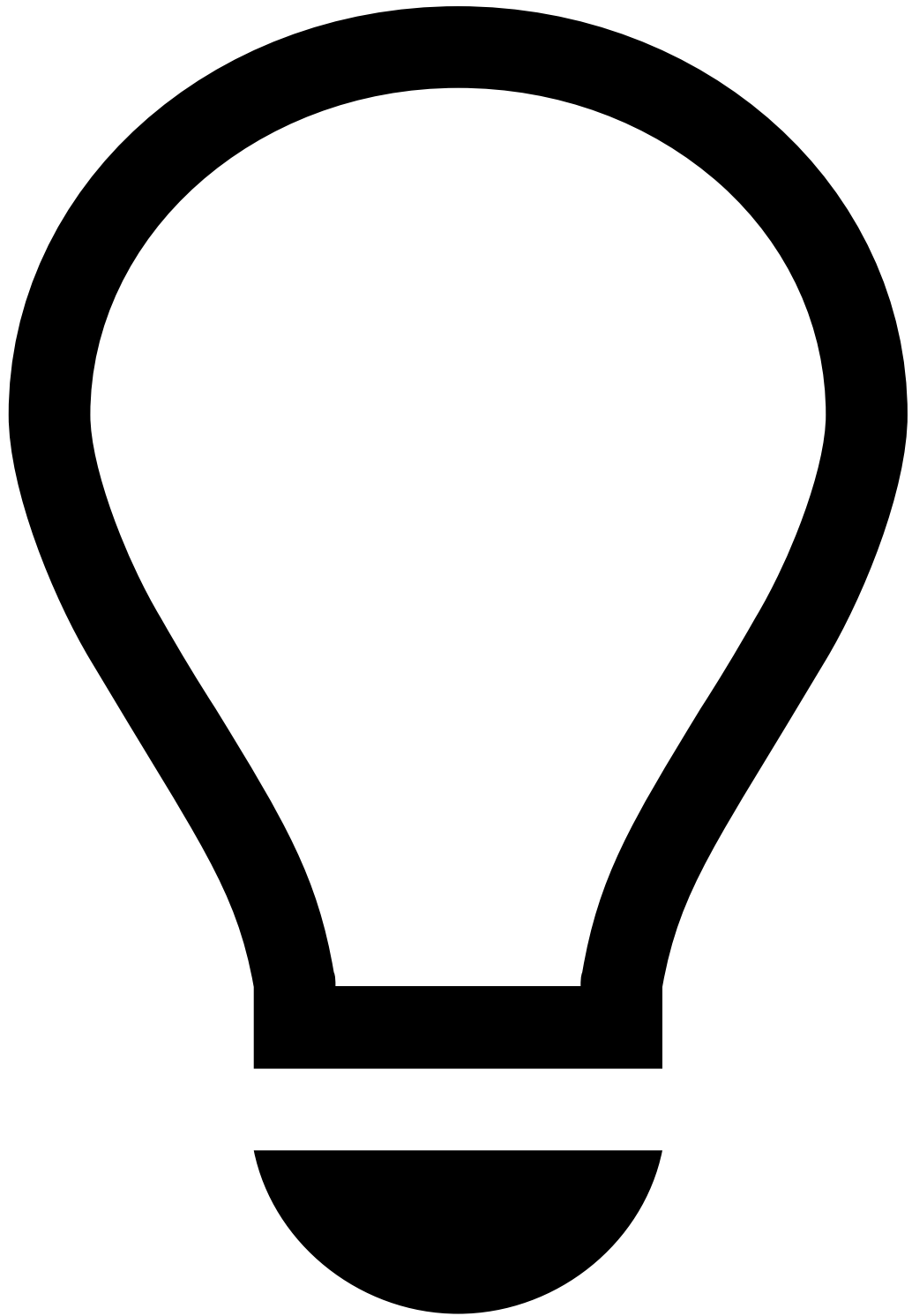


5. Edit Edit the **Filter** element with the following expression to obtain only the card numbers **without** fraudulent transactions.

```
sumFraud == 0
```



6. Edit Edit the **Sample** element to extract only 5% of all cards **without** fraudulent transactions.



Checkpoint

At this stage you have all fraudulent card numbers separate from a smaller subset of all cards that never had fraud. The plan workflow will recover the complete transactions for all cards in both branches and re-merge them together as a single dataset.

7. Edit Edit the **Join** element to perform a join operation, by *AccountID*, with the original data and recover the original transactions for those cards. You now have all transactions for cards where fraud **occurred**.
8. Edit Edit the **Join** element to perform a join operation, by *AccountID*, with the original data and recover the original transactions for those cards. You now have all transactions for cards where fraud **never occurred**.
9. Execute a **Union** of both **Join** elements.
10. Save the results of your plan to a data **Sink**. For example, a new data source named *Stratified Sample*.

Finally:

1. Run the plan to execute all actions on the original data source and generate an improved one.
 1. Define the **Data Source with ID source** as the original data source. For example, *Cleaned Transactions*.
 2. Expand the Advanced Options to have access to additional configurations:
 1. [Job Profile](#): Allows you to choose a profile containing properties that best fit the **Spark Job** that you want to run. Check the page [Selecting Profiles](#) to learn more about this functionality.
 2. [Data Partitioning](#): If you have [Metrics Operators](#) in your plan, this functionality allows to choose the **Data Partitioning** option that enables **Spark** to re-organize the data into different partitions as homogenous and well distributed as possible. Pulse allows you to choose the options: **Data-Aware Optimizer**, **Hash-Based**, and **Manual Partition**. These options are described in detail in the [Data Partitioning](#) page.



Metrics Operators

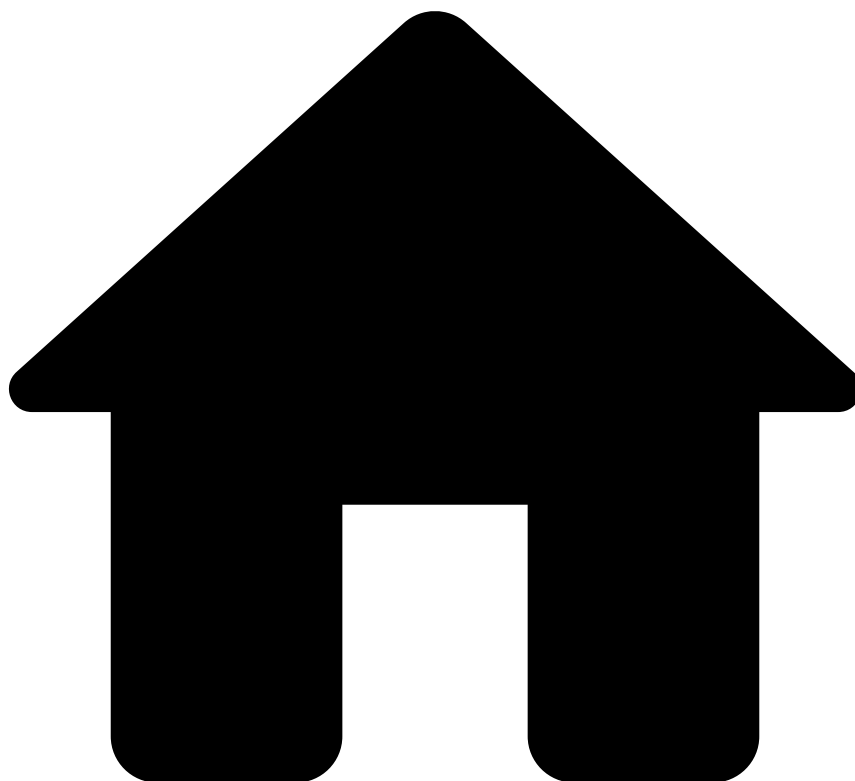
The **Data Partitioning** option takes effect only if you have metrics operators in your plan. Otherwise, this option does not affect the performance of the plan execution.

3. Configure the **Data Sink with ID sink** as the new and clean data source. For example, *Stratified Sample*.
4. **Run** the plan to generate a new data source, in Pulse, named *Stratified Sample*. Pulse also saves an output file with the transformed data to your filesystem.
2. [Analyze](#) the *Stratified Sample* data source to check the distribution of fraud and non-fraud transactions.

Next Steps

When you have sampled and understood your data, you are ready to move on to [designing features](#) for your model.

•



- [Creating a Data Science project](#)
- [Processing your data](#)
- [Analyzing and sampling your data](#)
- Identifying Patterns

On this page

Identifying Patterns

Was this page helpful? 🗨️

Before you generate metrics and design features you must first understand your data, and what are the clues that might indicate how each record should be classified in the end. For example, clues that point to fraudulent and non-fraudulent transactions.

To start gaining some initial insight into your dataset and uncover patterns, you must run a preliminary analysis on a [Data Source](#). Pulse lets you visualize statistics for each data field and break them down, for example, by fraud or not-fraud.



info

Alternatively, you can perform these actions from the JupyterLab user interface. Standard notebooks are available within [JupyterLab](#) – in particular the Data Exploration Standard Notebook, which demonstrates how to perform efficient data exploration using Feedzai's exploration library.

How to

To perform an analysis on your [Data Source](#) and, as an example, start identifying some fraud patterns, execute the following actions:

1. In the **Data Science** tab, select your **Data Source** from the **Data Sources list**.
2. Use the Analyse option to generate statistics for the **Data Source**. Select which are the breakdown fields and values to slice the results into. For example, **Fraud**.

3. Use the **View Statistics** option to go in depth into the generated insight for each field. When looking into the analysis details to understand the data, you can:

1. **Analyze statistics**

For example, look for negative values in *Numerical* fields such as *TransactionAmount*. This may suggest that there is work to do, namely [cleaning up your data](#).

2. **Breakdown the statistics**, for example by **Fraud**

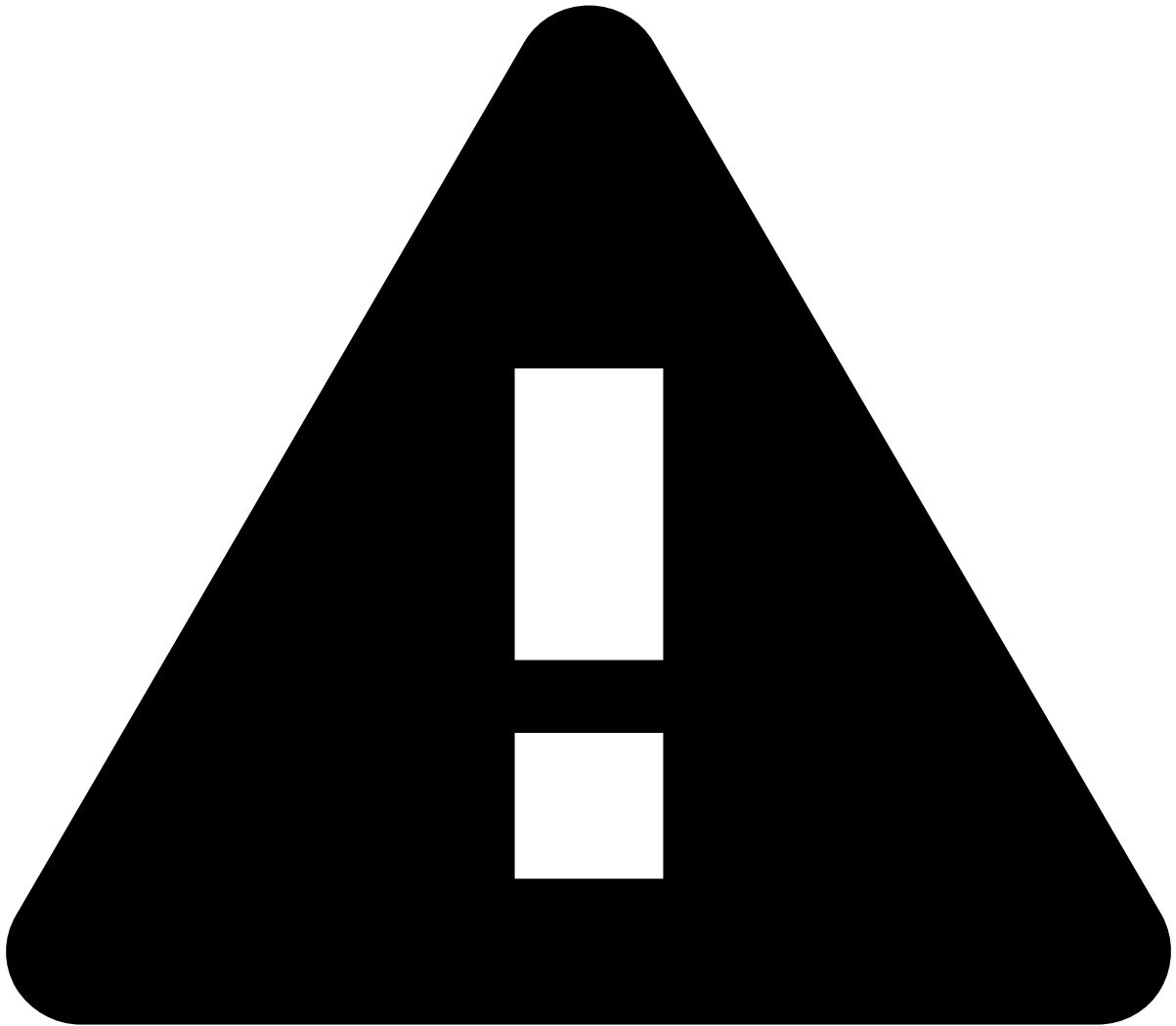
This lets you observe your data and start to identify patterns, such as different distributions when you look at the *CardholderPresent* field in fraudulent and non-fraudulent instances separately. This may hint at some correlation between the *TransactionAmount* and *Fraud* fields that you should further explore when [processing your data](#).



Generated statistics

Depending on the data types you defined for each field when [importing your data](#), Pulse [generates different metrics](#).

For *Numerical* fields, Pulse generates arithmetic metrics such as mean, minimum and maximum, while for *Categorical* data, Pulse only generates the total count and distinct values.



Breakdowns

When you [analyze a Data Source](#), you must always select for which breakdown fields and values you want Pulse to calculate statistics.

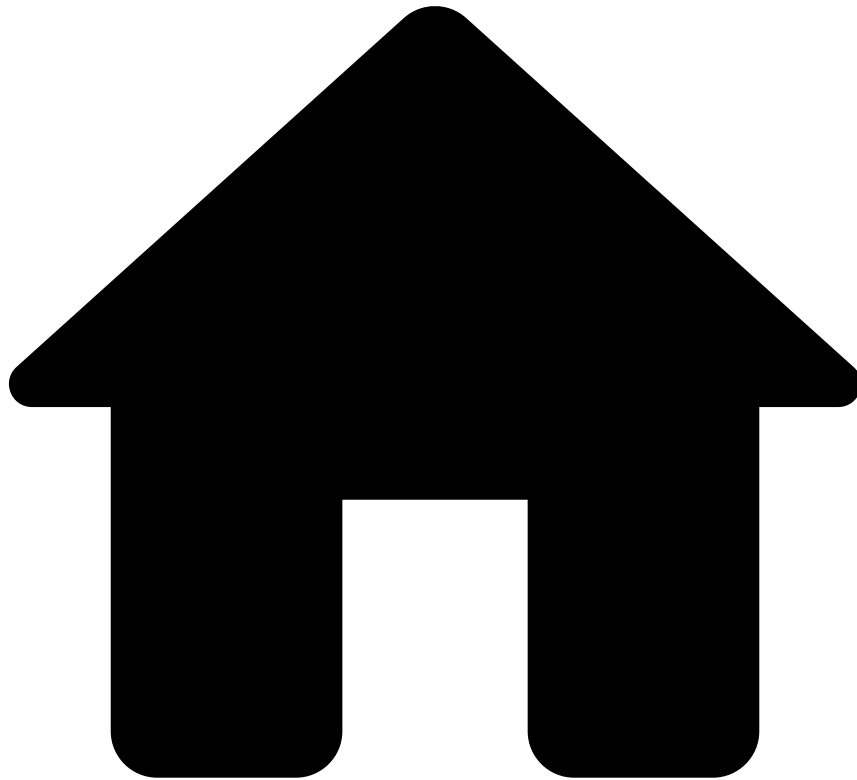
Skipping this action and simply calculating broken-down metrics for all possible combination of fields and values could rapidly cause performance degradation due to lack of computational resources.

Next Steps📖

You are now able to perform a high-level analysis on your historical data and have raised some insights, e.g. how the cardholder being present at the transaction or how the transaction country correlates with fraud.

You are ready to [sample your data](#) and start [designing features](#) for your model.

•



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- Conditional Expressions in PQL

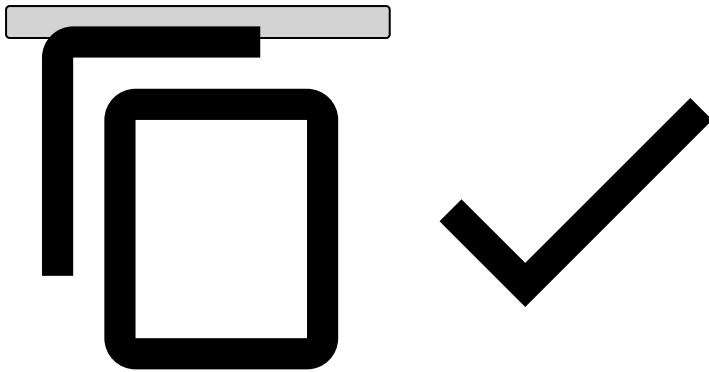
Conditional Expressions in PQL

Was this page helpful? 

Conditional expressions of PQL return a value based on a condition.

Syntax:

```
if <condition> then <then-case-expression> else <else-case-expression>
```



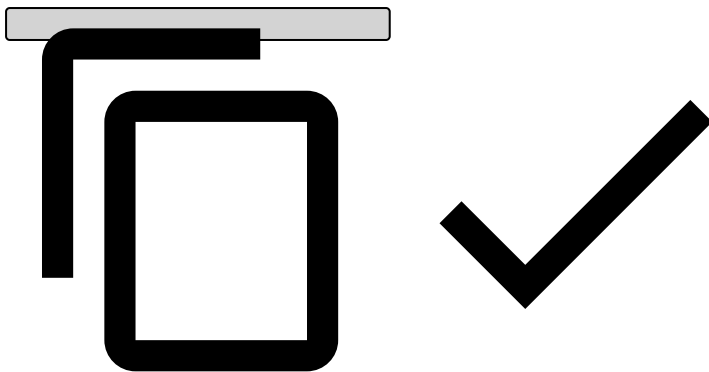
Where:

- `<condition>` is a boolean expression.
- `<then-case-expression>` and `<else-case-expression>` are obligatory values.

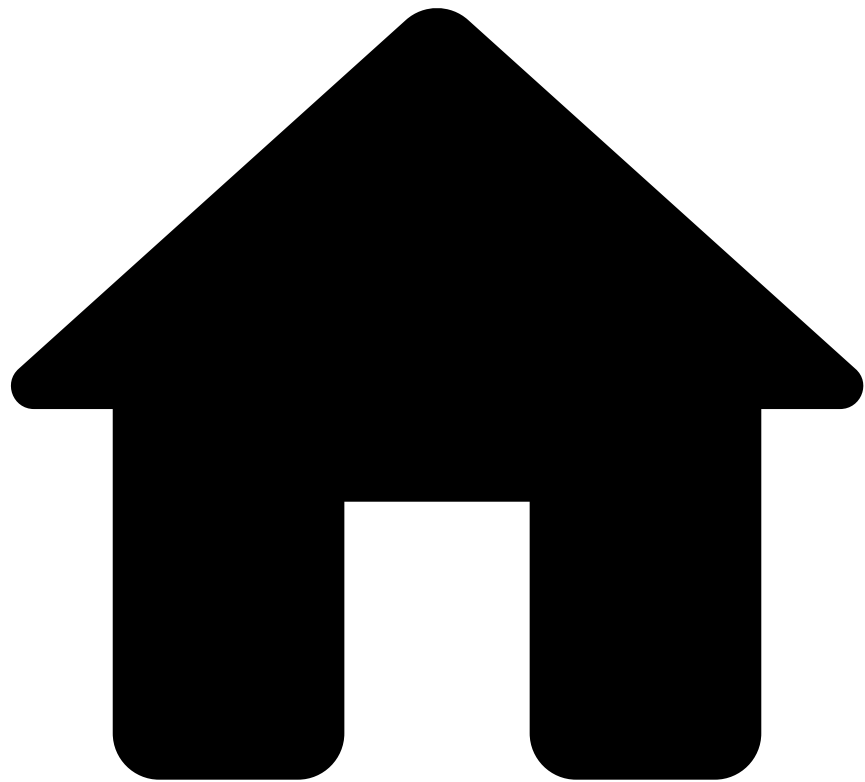
If the condition is true, the conditional expression returns the value of the then-case-expression, otherwise the value of the else-case-expression.

Example:

```
if( P0SEntryMode == 'Swiped' or P0SEntryMode == 'Keyed' or P0SEntryMode == 'Chip' )  
  then 'Card Present'  
  else P0SEntryMode
```



•



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- Function Calls in PQL

On this page

Function Calls in PQL

Was this page helpful? 

Function calls allow you to execute predefined functions on current values. There are two types of functions that can be called:

- [Built-in functions](#) of Pulse
- [User-Defined Functions \(UDFs\)](#) – a set of additional functions bundled with Pulse

This topic gives an overview of the possible function calls, and points you to the right topics to learn more.

Built-in functions

Checking NULLs

Return type	Method	Description
bool	isNull(int? a)	Checks if an integer value is null.
bool	isNull(long? a)	Checks if a long value is null.
bool	isNull(double? a)	Checks if a double value is null.
bool	isNull(float? a)	Checks if a float value is null.
bool	isNull(bool? a)	Checks if a boolean value is null.
bool	isNull(string? a)	Checks if a string value is null.
bool	isNull(tuple? a)	Checks if a tuple value is null.

Math functions

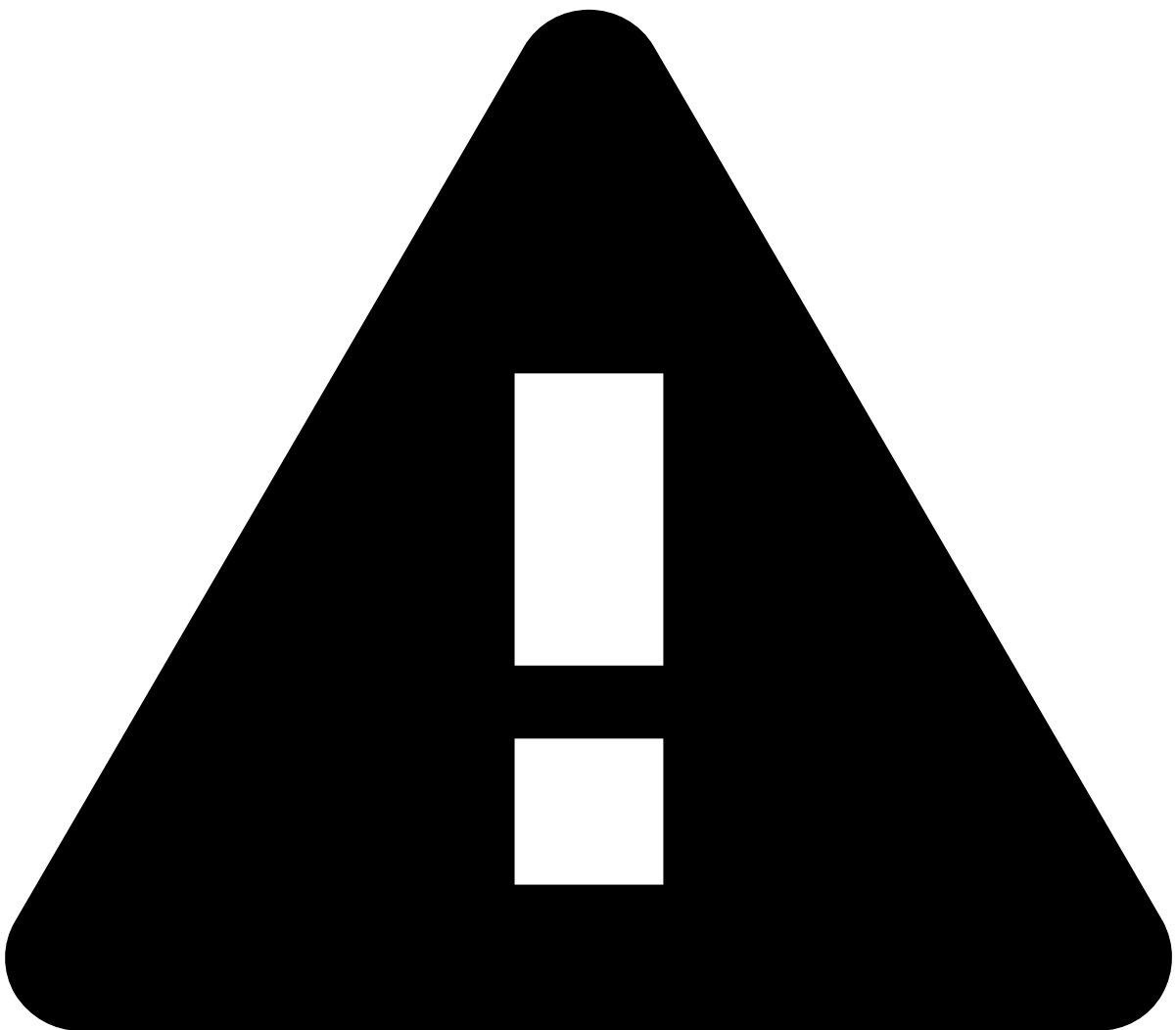
All methods of the [java.lang.Math](#) package are available in PQL. Besides those methods, you can use the following math functions:

Return type	Method	Description
int	max(list_of_int)	Returns the maximum value of a list of numbers.
long	max(list_of_long)	Returns the maximum value of a list of numbers.
double	max(list_of_double)	Returns the maximum value of a list of numbers.
float	max(list_of_float)	Returns the maximum value of a list of numbers.
int	min(list_of_int)	Returns the minimum value of a list of numbers.
long	min(list_of_long)	Returns the minimum value of a list of numbers.
double	min(list_of_double)	Returns the minimum value of a list of numbers.
float	min(list_of_float)	Returns the minimum value of a list of numbers.
boolean	between(int val, int left, int right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	between(long val, long left, long right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	between(double val, double left, double right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.
boolean	between(float val, float left, float right)	Returns true if the value is greater or equal to the left boundary and lower or equal to the right boundary.

Strings

Return type	Method	Description
string	str(int a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(long a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(double a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(float a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(bool a)	Returns a new string that is the textual representation of <code>a</code> .
string	str(string a)	Returns a new string that is the textual representation of <code>a</code> .

Return type	Method	Description
int	indexOf(string s, string c)	Returns the index within string <code>s</code> of the first occurrence of the string <code>c</code> .
string	substring(string s, int begin)	Returns a new string that is a substring of string <code>s</code> .
string	substring(string s, int begin, int end)	Returns a new string that is a substring of string <code>s</code> .
bool	contains(string s, string c)	Returns <code>true</code> if string <code>c</code> is a substring of string <code>s</code> , or <code>false</code> otherwise.
bool	startsWith(string s, string c)	Returns <code>true</code> if string <code>c</code> is a prefix of string <code>s</code> , or <code>false</code> otherwise.
bool	endsWith(string s, string c)	Returns <code>true</code> if string <code>c</code> is a suffix of string <code>s</code> , or <code>false</code> otherwise.



Container functions in `Map` and `Metrics` operators

Functions that operate on collections (like `List` s, `Set` s or `Map` s) are not supported in the PQL fields of [Map Operators](#) or [Metric Operators](#).

For example, using `contains` or `indexOf` on collections in PQL fields of such operators will result in an error.

The recommended alternative is to use [UDFs \(User Defined Functions\)](#) instead of these built-in functions.

UDF calls

Pulse comes with a set of [User-Defined Functions \(UDFs\)](#) that can be used in addition to the built-in functions.

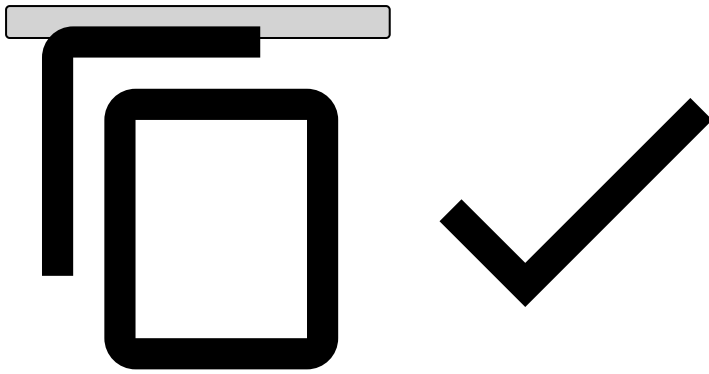


UDFs in Metrics

When using UDFs in metric filters and aggregations, UDFs receive fields from a single event and not from a collection of events.

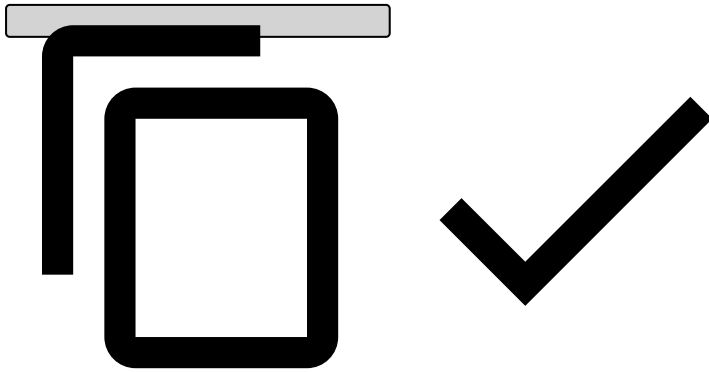
You can use the following syntax to call a UDF:

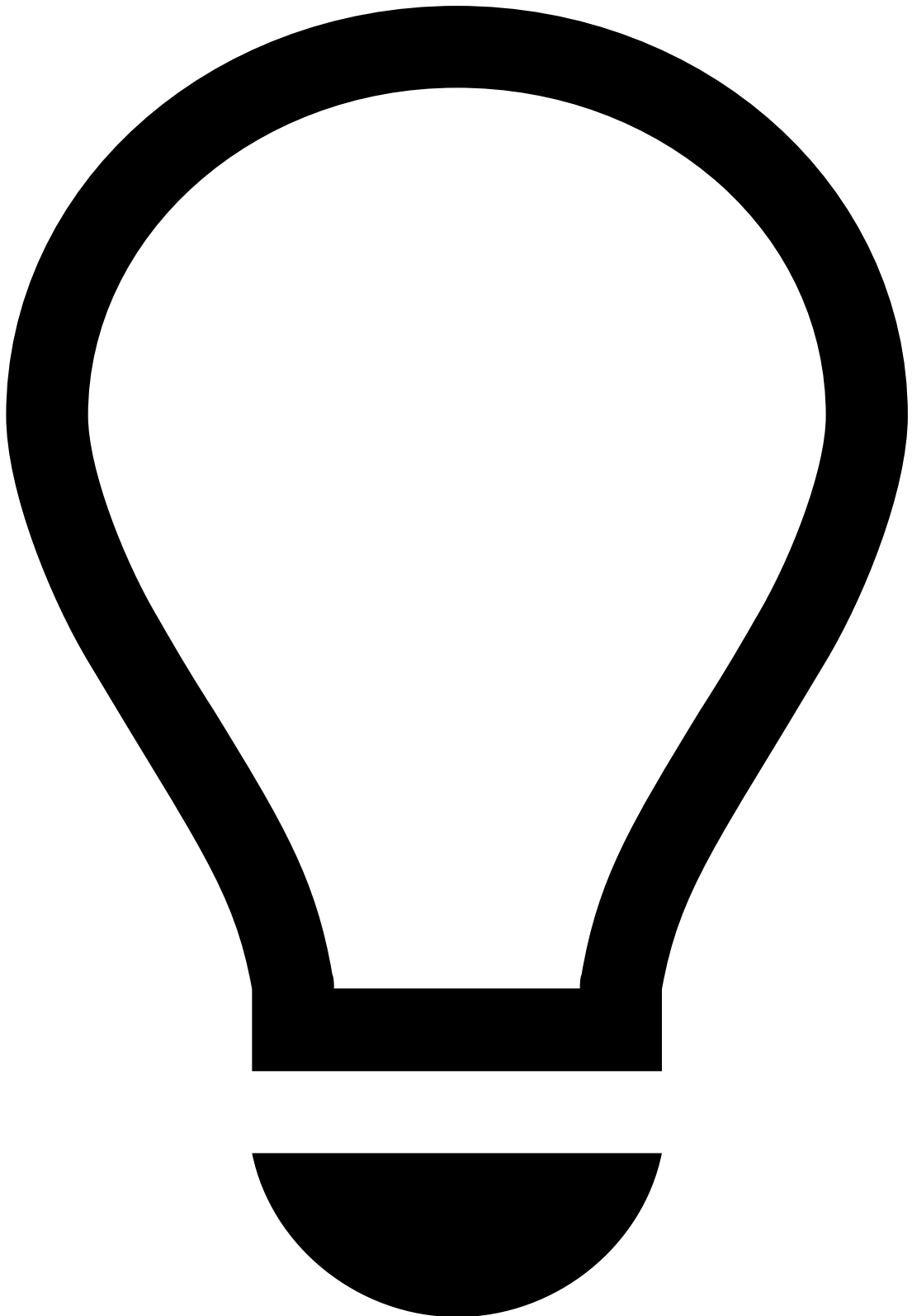
```
<class_name>.<function_name> ( <arguments> )
```



For example, to concatenate two strings:

```
Strings.strcat("One", "Two") // Returns "OneTwo"
```





java regex syntax

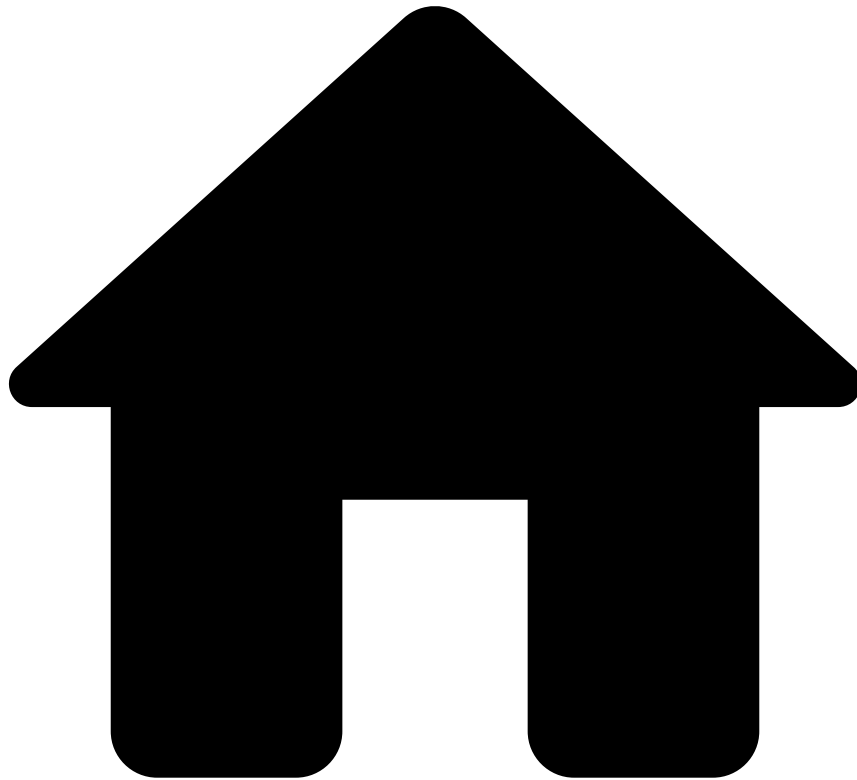
Pulse uses the Java regex flavor for UDFs, which means that all regular expressions must conform to the syntax and rules of Java's `java.util.regex.Pattern` class.

According to the usual regex syntax, when you need to match a literal special character that has meaning in regex (e.g. `*`, `+`, `?`, `.`), you must escape it using a backslash (`\`) within the regex pattern (e.g. `\*`, `\+`, and so on).

However, because the regex pattern itself is often passed as a Java String literal, the backslash (`\`) used for escaping the regex special character must itself be escaped according to Java String rules.

Therefore, to include an escaped character like `\*` (match literal asterisk) in a regular expression defined as a Java string, you must write it as `\\*`.

•



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- Handling NULL Values with PQL

On this page

Handling NULL Values with PQL

Was this page helpful? 

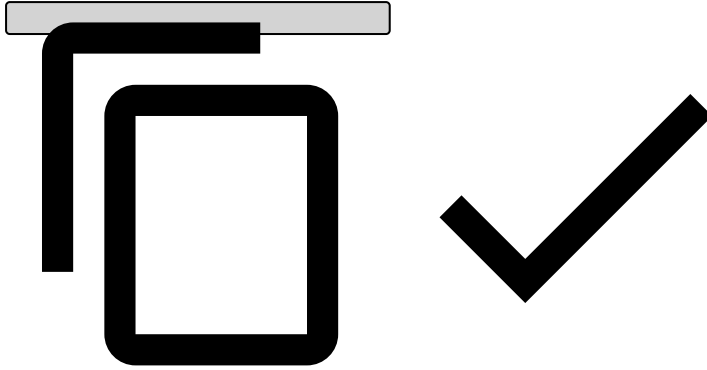
NULL is a special value that represents the absence of a valid value. Any operation that acts on it as if it was a valid value fails with a `NullPointerException`. Untreated datasets might contain NULL values, and so they are potentially unsafe for any activity within Pulse. This topic describes how to make datasets more robust by handling NULL values using PQL expressions.

Replacing NULL values

One of the common ways to handle NULLs is to replace them with a value that fits into the data field's type. It's common to pick the substitute value so that it doesn't represent a meaningful value for that data field. For instance, a NULL in the transaction amount field could be replaced by 0. To accomplish this replacement, you can use the **coalesce operator** (??) in a map operator or metric operator.

Usage of the ?? operator:

```
<good-case-value> ?? <null-case-value>
```



This expression returns `<good-case-value>` , unless `<good-case-value>` is NULL. If `<good-case-value>` is NULL, the expression returns `<null-case-value>` .

Example:

Coalescing aggregations to a value other than 0

Aggregations return 0 instead of a NULL when there are no values to count the aggregation on. For example, when the maximum amount is queried over a time window using a filter that filters out all the records.

In some cases you might want to return something different from 0.

The following example shows how to return -1 in case an aggregation of a metric is called on an empty subset of data:

Nullable types

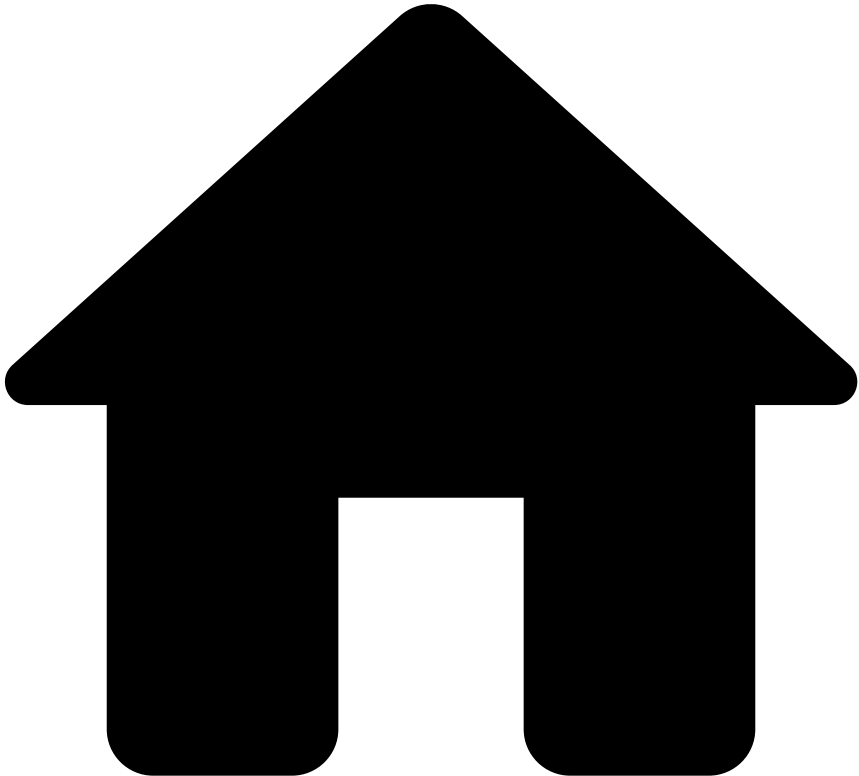
Untreated input fields can contain NULL values. Hence, PQL treats every input field as a nullable type, meaning a type that can contain NULL values in addition to an original data type.

This fact is marked with a ? in the data type name.

For example, the *Amount* field is not of *double* type, but of *double?*, meaning a type that contains all double values plus the NULL.

When using both nullable and non-nullable types in an [arithmetic operator](#) or in a [function](#) that returns a numeric value, the resulting data type is nullable.

•



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- Operators in PQL

On this page

Operators in PQL

Was this page helpful? 

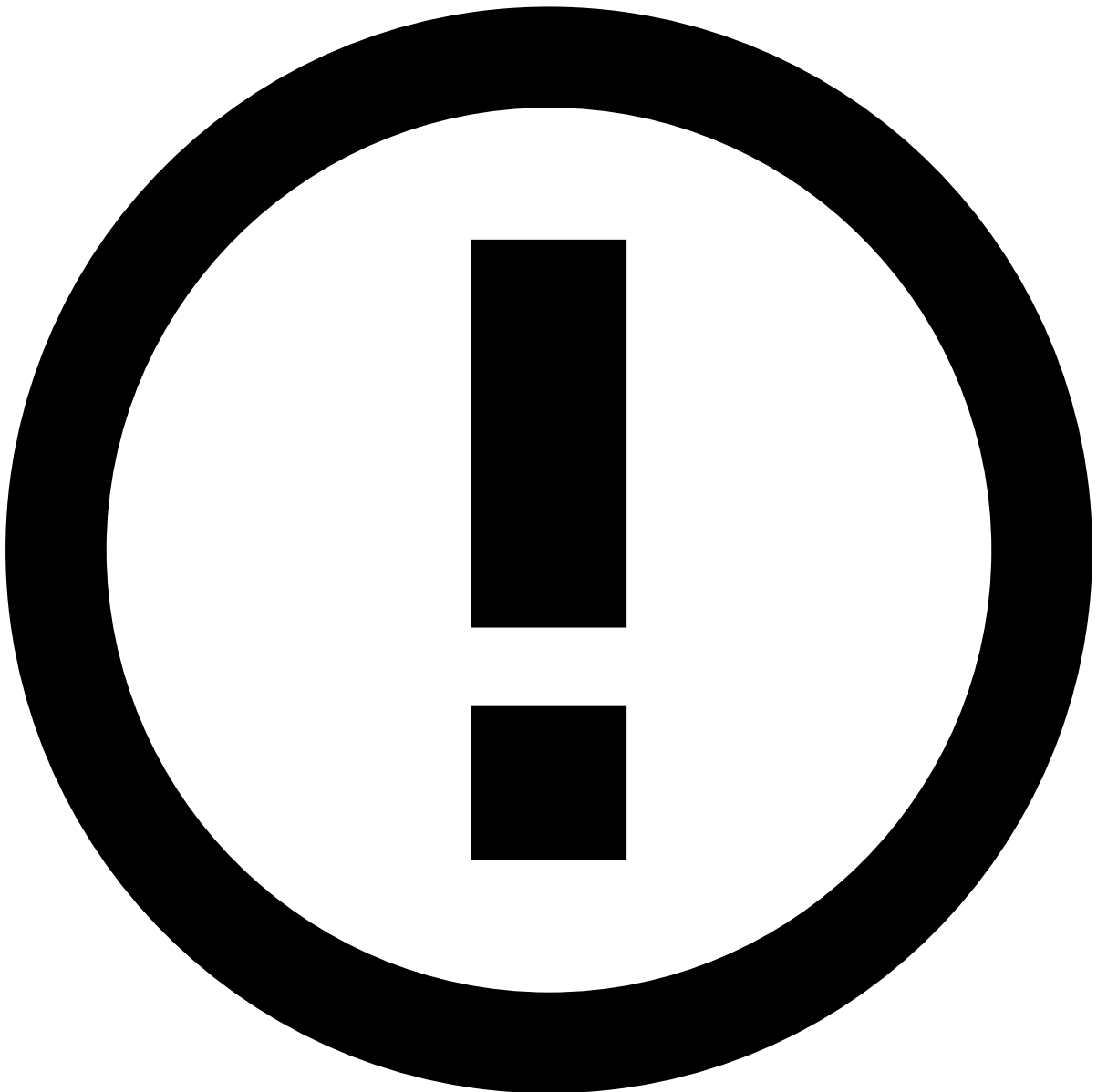
This topic gives you an overview of the arithmetic and boolean operators that you can use in PQL expressions, and the precedence of all operators.

Arithmetic operators

PQL uses the following arithmetic operators:

Operator	Meaning
+	addition
-	subtraction
*	multiplication
/	division
%	integer remainder
^	exponentiation

Example: `Balance - Amount`



Type promotion when using arithmetic operators

The arithmetic operators obey the following type promotion rules:

1. If at least one of the arguments is a double, the result is a double.

2. If at least one of the arguments is a float, the result is a float, unless the result is a double because of the first rule.
3. If at least one of the arguments is a long, the result is a long, unless the result is a double or float because of the first two rules.

Boolean operators

Boolean operators are operators that return a boolean value. They include:

- `<` , `<=` , `==` , `!=` , `>=` , `>`



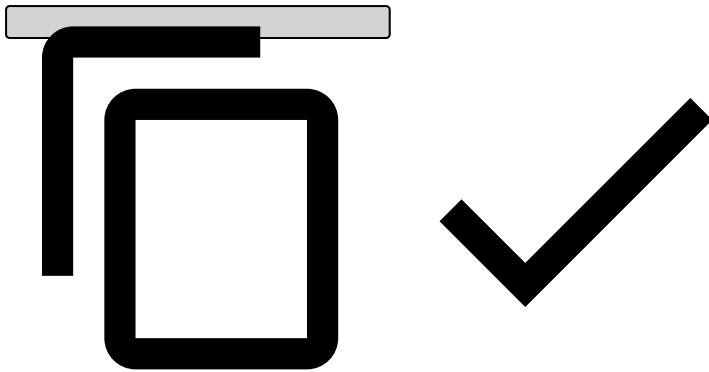
info

Only numeric values can be compared with the `<`, `<=`, `>=`, and `>` operators.

- `and` , `or` , `!` (not)

Example:

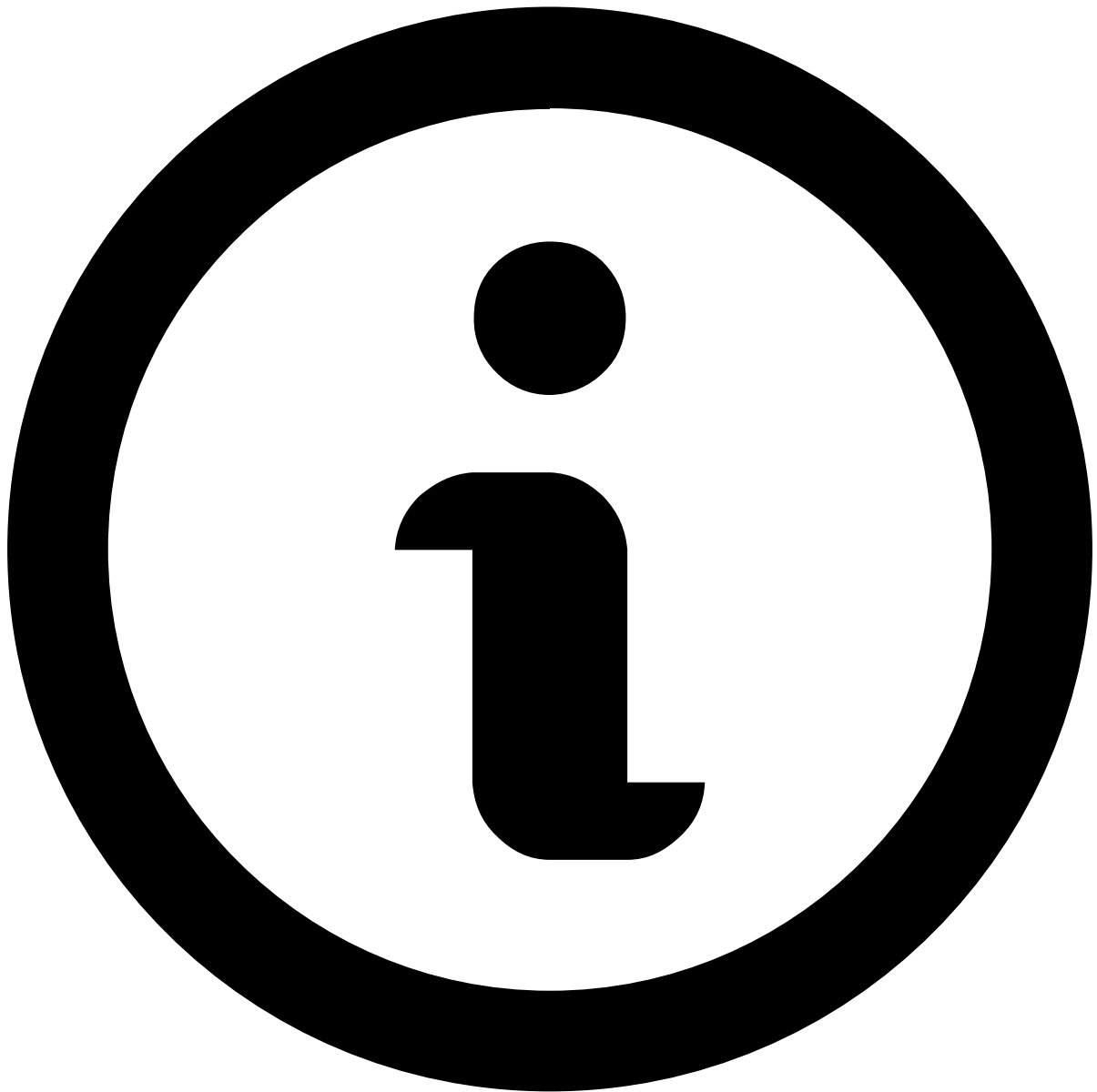
```
P0SEntryMode == 'Swiped' or P0SEntryMode == 'Keyed' or P0SEntryMode == 'Chip'
```



Operator precedence

The following table lists PQL operators according to their precedence order. Operators at the top of the table have higher precedence.

Operator type	Operator(s)
grouping	(expr)
unary	+expr , -expr , !
arithmetic - exponential	^
cast	exp:type
arithmetic multiplicative	*, / , %
arithmetic additive	+, -
coalesce	?? Note that this operator has a different precedence in RDL.
comparison	< , <= , == , >= , > , !=
logical AND	and
logical OR	or
assignment	=



Cast operator precedence

Notice that the *cast* operator precedence lies between arithmetic operators. If an expression needs to be cast before an exponentiation, it needs to be enclosed in parentheses (*grouping operator*).

The following rules apply:

- operators with higher precedence than other operators are evaluated before them;
- operators on the same line have the same precedence:
 - all binary operators except for the assignment operator are evaluated from left to right;
 - assignment operator is evaluated right to left.

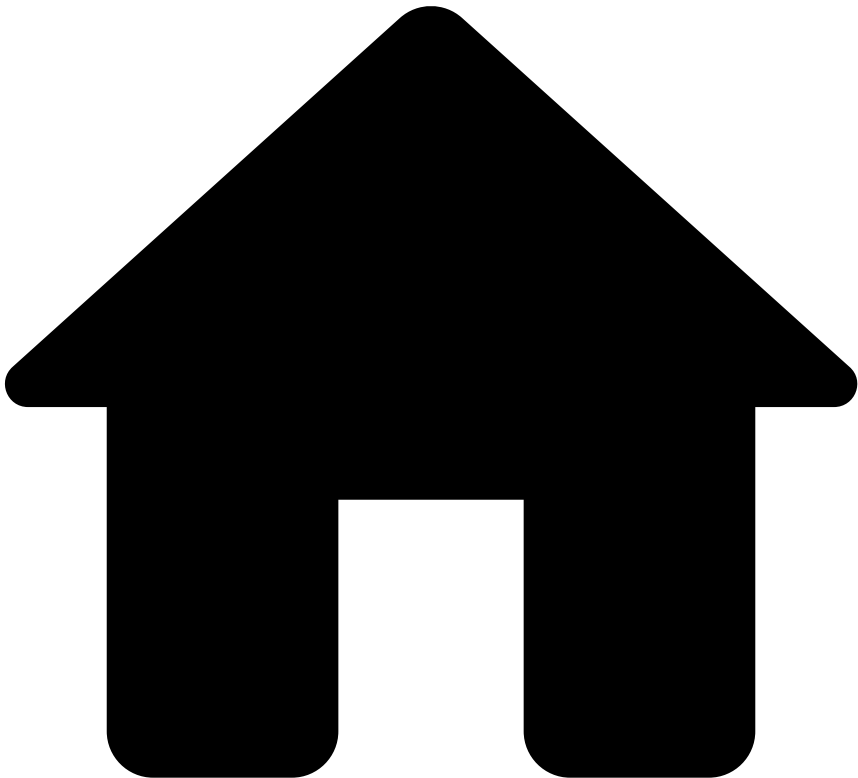
Examplesâ

Arithmetic and boolean operators of PQL follow the common rules of mathematics regarding functionality and precedence.

Arithmetic and boolean operators have higher precedence than the `??` ([coalesce](#)) operator, meaning that the `??` operator gets evaluated after arithmetic and boolean operators.

Example: $a + b \leq c + d$ is equivalent to $(a+b) \leq (c+d)$.

.



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- PQL Data Types and Type Conversions

On this page

PQL Data Types and Type Conversions

Was this page helpful? 

This topic gives you an overview of data types in PQL, and how to convert data from one data type to another.

PQL uses the following **primitive data types**.

Data type	Examples
int	-289

Data type	Examples
	0 42
long	123456789 1l -2L
float	0.99 -1.23e-100f 3.14F
double	1234567.89012 1.23456e300d 1.23456e-300d 1e1D
bool	true false
string	"John Doe" 'John Doe'

These data types are equivalent to the Java primitive data types. To see the exact definition or range for each, check the [Java documentation](#).

The **non-primitive data types** of PQL are:

- Tuples
- Collections:
 - Lists
 - Sets
 - Maps

Do not confuse these types with lists and map operators that you can create in the Pulse UI. They belong to two different levels of implementation. The non-primitive PQL data types are used only in PQL queries, when computing features. When the PQL computation is finished, Pulse stores the data of these non-primitive types as *Complex* fields in the output data source.

Data type mapping between Pulse and PQL

The following table shows to which PQL data type each [Pulse data type](#) is mapped when using input data fields:

Pulse field type (input)	PQL type
Categorical	string
Complex	the same non-primitive type of PQL

Pulse field type (input)	PQL type
Date Time	long (UNIX timestamp format)
Numeric	int, long, float, double
String	string

The following table shows which PQL data type can be passed as output to which type of output fields. The output field is the data field that the map or metric operator is creating/updating, and its type is one of the [Pulse data types](#).

PQL type	Pulse field type (output)
any data type	Categorical
int, long, float, double	Numeric
long, int (UNIX timestamp format)	Date Time
any non-primitive type	Complex
string	String

Converting data

In PQL, each data field, constant, variable, or aggregation result is of a primitive data type. To convert from one data type to another, you can use the following options:

- Cast operator, to cast a value of a numeric type into another numeric type. Learn more about this operator in the next section of this page.
- The `str()` [built-in function](#) to convert a numeric value to a string.
- [User-defined functions \(UDFs\)](#). There are [built-in UDFs](#) that can be used after creating an alias for them. Example: to convert the merchant category code to integer, you can use the `DataConversionTools.stringToInt()` function.

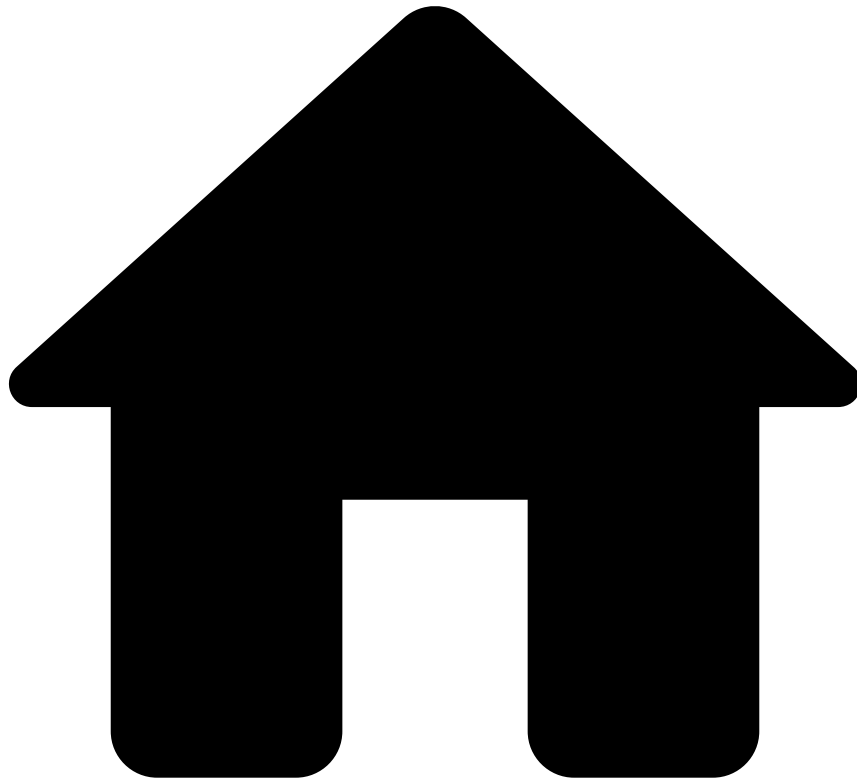
Cast operator

The cast operator allows you to cast values from a numeric type (int, long, float or double) to another numeric type. There is no built-in cast operator between numeric types and booleans or strings.

To cast a value, use the following syntax: `<value to convert>:<target type>`

Example: `Amount:int` is the value of the Amount field cast to integer.

•



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- PQL in Derived Features

On this page

PQL in Derived Features

Was this page helpful? 🗨️

Derived features are new data fields that were calculated based on the original data set.

To create derived features, there are two different cases:

- Computing data based on a single event (not on aggregated data), using a map operator.
- Calculating a value based on an aggregation of events, often combining map and metrics operators.

This topic describes how to use PQL in both cases.

Derived fields without aggregation

To calculate new values based on input fields, you can use **map operators** in plans. For that, add a map operator to the plan that processes the input data, and add the PQL expression in the *Expression* field.

You can use any of the [building blocks of PQL](#), except for the aggregations.

Example

The following example adds a field called *CountryMatch* that shows if the merchant country is the same as the country where the card was created.

Derived field based on aggregations

To aggregate data, use a **metrics operator** in the plan. The UI of the metrics operator allows you to use a shorter list of aggregations: Avg, Min, Max, Count, Count Distinct, Sum and Standard deviation. Other calculations need PQL statements.

PQL allows you to use the following **aggregations** in the *Expression* field of an aggregation when editing a metrics operator:

Aggregation	Description	Usage example
avg (field_name)	Returns the average value of the selected numeric field from the aggregated data set.	<code>avg(Amount)</code>
count ()	Returns the number of rows in the aggregated data set.	<code>count()</code>
max (field_name)	Returns the maximum value of the selected numeric field from the aggregated data set.	<code>max(Amount)</code>
maxBy (field_name)	Returns the event containing the maximum value in the given numeric field.	<code>maxBy(Amount).MerchantId</code>
min (field_name)	Returns the minimum value of the selected numeric field from the aggregated data set.	<code>min(Amount)</code>
minBy (field_name)	Returns the event containing the minimum value in the given numeric field.	<code>minBy(Amount).MerchantId</code>
stddev (field_name)	Returns the standard deviation of the selected numeric field's values within the aggregated data set.	<code>stddev(Amount)</code>
sum (field_name)	Returns the sum of the selected numeric field values in the aggregated data set.	<code>sum(Amount)</code>

Navigate on an aggregated data set

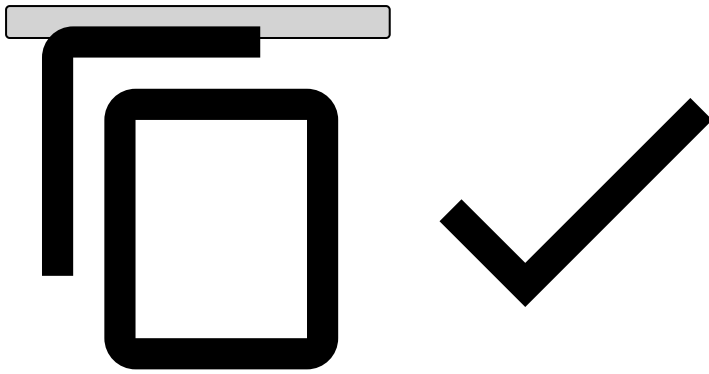
The `first()`, `last()`, and `prev()` methods allow to access the first, last, and previous row of the aggregated data set.

- The current record is the last row of the current dataset if it entered the window, depending on the filters.
- **first()** is the first transaction in the current window.
- **last()** is the last transaction in the current window.
- **prev()** is the transaction before `last()` that entered the window.

Examples

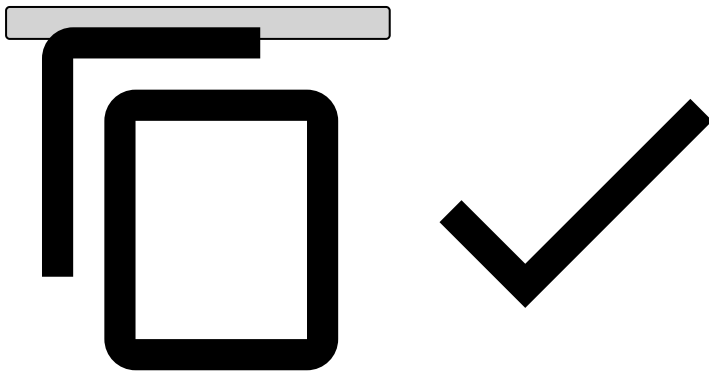
To calculate the geographical distance traveled between two transactions, you can create the following aggregation in a metrics operator:

```
DistanceUtils.haversine(latitude, longitude, prev().latitude, prev().longitude)
```

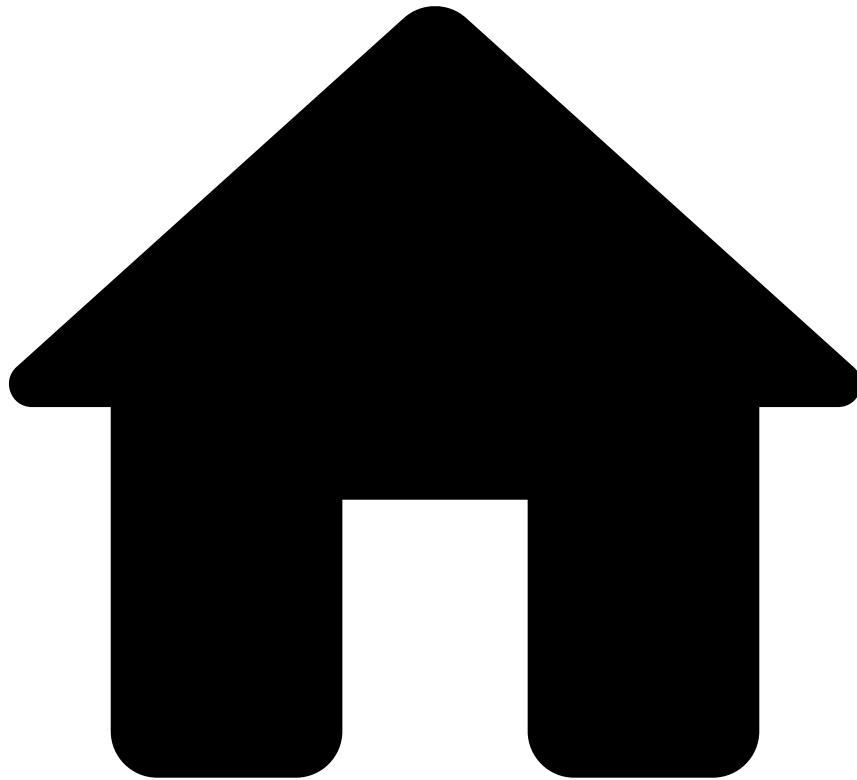


To calculate the time span between the first and last transaction within a given window per merchant, you can create the following aggregation in a metrics operator:

```
( let first_timestamp = first().Timestamp ?? 0 in
  let last_timestamp = last().Timestamp ?? 0 in
  ( last_timestamp - first_timestamp ) / 60*60*1000.0
)
```



•



- [Rule Guide](#)
- [Creating Features](#)
- [Using PQL in Features](#)
- Reserved Keywords in PQL

On this page

Reserved Keywords in PQL

Was this page helpful? 🗨️

Some keywords and symbols cannot be used outside of their original purpose. That means that within a PQL expression, they cannot be used as name of variables, fields or UDFs.

Besides the reserved keywords and symbols, there is a third list of words that cannot be used in PQL expressions due to limitations related to Java.

Reserved keywords

Keyword	Purpose
and	Comparison operator
as	Import aliasing
asc	Order criteria (ascending)
bool	Boolean type
by	Field descriptor (i.e. group by, order by)
desc	Order criteria (descending)
double	Double type
else	Conditional branching
float	Float type
from	Select descriptor (i.e. select from)
group	Grouping operator
if	Conditional branching
import	Import statement for UDFs
in	Variable binding (i.e. let a = 1 in)
int	Integer type
into	Select target (i.e. group by price into ev)
join	Join operator
let	Variable binding (i.e. let a = 1 in)
limit	Limit operator (i.e. limit 1, 10)
long	Long type
on	Join operator
or	Comparison operator
order	Order operator
select	Select operator
Stream	Stream declaration
string	String type
then	Conditional branching
unit	Unit type
where	Select discriminator

Reserved symbols

Symbol	Purpose
.	Field accessor
+	Addition
-	Subtraction
*	Multiplication
/	Division

Symbol	Purpose
%	Division remainder (mod)
^	Exponentiation
<	Less than
<=	Less than or Equal to
=	Assignment
==	Equal
!=	Different
>=	Greater than or Equal to
>	Greater than
{	Block separation
}	Block separation
:	Casting, selection, etc.
;	Statement separation
@	Annotation
??	Coalesce operator

Java keywords

For internal compatibility reasons, the following Java keywords are not allowed:

```

abstract
assert
break
byte
case
catch
char
class
const
continue
default
do
enum
extends
final
finally
for
goto
implements
instanceof
interface
native
new
package
private
protected
public
return
static
strictfp
super
switch
synchronized

```

```
this  
throw  
throws  
transient  
try  
void  
volatile  
while
```

